

Home Assistant Dokumentation

Vollständige Dokumentation

Quelle: <https://www.home-assistant.io/docs/>

Lizenz: CC BY-NC-SA 3.0

Inhaltsverzeichnis

1. Actions/Alarm Control Panel.Alarm Arm Away
2. Actions/Alarm Control Panel.Alarm Arm Custom Bypass
3. Actions/Alarm Control Panel.Alarm Arm Home
4. Actions/Alarm Control Panel.Alarm Arm Night
5. Actions/Alarm Control Panel.Alarm Arm Vacation
6. Actions/Alarm Control Panel.Alarm Disarm
7. Actions/Alarm Control Panel.Alarm Trigger
8. Actions/Button.Press
9. Actions/Fan.Decrease Speed
10. Actions/Fan.Increase Speed
11. Actions/Fan.Oscillate
12. Actions/Fan.Set Direction
13. Actions/Fan.Set Percentage
14. Actions/Fan.Set Preset Mode
15. Actions/Fan.Toggle
16. Actions/Fan.Turn Off
17. Actions/Fan.Turn On
18. Actions/Html5.Dismiss Message
19. Actions/Html5.Send Message
20. Actions/Light.Toggle
21. Actions/Light.Turn Off
22. Actions/Light.Turn On
23. Actions/Lock.Lock
24. Actions/Lock.Open
25. Actions/Lock.Unlock
26. Actions/Ntfy.Clear
27. Actions/Ntfy.Delete
28. Actions/Ntfy.Publish
29. Actions/Person.Reload
30. Actions/Vacuum.Clean Area
31. Actions/Vacuum.Clean Spot
32. Actions/Vacuum.Locate
33. Actions/Vacuum.Pause
34. Actions/Vacuum.Return To Base
35. Actions/Vacuum.Send Command
36. Actions/Vacuum.Set Fan Speed
37. Actions/Vacuum.Start

38. Actions/Vacuum.Start Pause
39. Actions/Vacuum.Stop
40. Actions/Vacuum.Toggle
41. Actions/Vacuum.Turn Off
42. Actions/Vacuum.Turn On
43. Conditions/Air Quality.Is Co2 Value
44. Conditions/Air Quality.Is Co Cleared
45. Conditions/Air Quality.Is Co Detected
46. Conditions/Air Quality.Is Co Value
47. Conditions/Air Quality.Is Gas Cleared
48. Conditions/Air Quality.Is Gas Detected
49. Conditions/Air Quality.Is N2O Value
50. Conditions/Air Quality.Is No2 Value
51. Conditions/Air Quality.Is No Value
52. Conditions/Air Quality.Is Ozone Value
53. Conditions/Air Quality.Is Pm10 Value
54. Conditions/Air Quality.Is Pm1 Value
55. Conditions/Air Quality.Is Pm25 Value
56. Conditions/Air Quality.Is Pm4 Value
57. Conditions/Air Quality.Is Smoke Cleared
58. Conditions/Air Quality.Is Smoke Detected
59. Conditions/Air Quality.Is So2 Value
60. Conditions/Air Quality.Is Voc Ratio Value
61. Conditions/Air Quality.Is Voc Value
62. Conditions/Alarm Control Panel.Is Armed
63. Conditions/Alarm Control Panel.Is Armed Away
64. Conditions/Alarm Control Panel.Is Armed Home
65. Conditions/Alarm Control Panel.Is Armed Night
66. Conditions/Alarm Control Panel.Is Armed Vacation
67. Conditions/Alarm Control Panel.Is Disarmed
68. Conditions/Alarm Control Panel.Is Triggered
69. Conditions/Calendar.Is Event Active
70. Conditions/Door.Is Closed
71. Conditions/Door.Is Open
72. Conditions/Fan.Is Off
73. Conditions/Fan.Is On
74. Conditions/Gate.Is Closed
75. Conditions/Gate.Is Open
76. Conditions/Humidifier.Is Drying
77. Conditions/Humidifier.Is Humidifying

- 78. Conditions/Humidifier.Is Mode
- 79. Conditions/Humidifier.Is Off
- 80. Conditions/Humidifier.Is On
- 81. Conditions/Humidifier.Is Target Humidity
- 82. Conditions/Humidity.Is Value
- 83. Conditions/Light.Is Brightness
- 84. Conditions/Light.Is Off
- 85. Conditions/Light.Is On
- 86. Conditions/Lock.Is Jammed
- 87. Conditions/Lock.Is Locked
- 88. Conditions/Lock.Is Open
- 89. Conditions/Lock.Is Unlocked
- 90. Conditions/Motion.Is Detected
- 91. Conditions/Motion.Is Not Detected
- 92. Conditions/Temperature.Is Value
- 93. Conditions/Vacuum.Is Cleaning
- 94. Conditions/Vacuum.Is Docked
- 95. Conditions/Vacuum.Is Encountering An Error
- 96. Conditions/Vacuum.Is Paused
- 97. Conditions/Vacuum.Is Returning
- 98. Conditions/Window.Is Closed
- 99. Conditions/Window.Is Open
- 100. Dashboards/Alarm Panel
- 101. Dashboards/Area
- 102. Dashboards/Button
- 103. Dashboards/Calendar
- 104. Dashboards/Clock
- 105. Dashboards/Conditional
- 106. Dashboards/Energy
- 107. Dashboards/Entities
- 108. Dashboards/Entity Filter
- 109. Dashboards/Entity
- 110. Dashboards/Gauge
- 111. Dashboards/Glance
- 112. Dashboards/Grid
- 113. Dashboards/Heading
- 114. Dashboards/History Graph
- 115. Dashboards/Horizontal Stack
- 116. Dashboards/Humidifier
- 117. Dashboards/Iframe

- 118. Dashboards/Light
- 119. Dashboards/Logbook
- 120. Dashboards/Map
- 121. Dashboards/Markdown
- 122. Dashboards/Masonry
- 123. Dashboards/Media Control
- 124. Dashboards/Panel
- 125. Dashboards/Picture Elements
- 126. Dashboards/Picture Entity
- 127. Dashboards/Picture Glance
- 128. Dashboards/Picture
- 129. Dashboards/Plant Status
- 130. Dashboards/Sections
- 131. Dashboards/Sensor
- 132. Dashboards/Shopping List
- 133. Dashboards/Shortcut
- 134. Dashboards/Sidebar
- 135. Dashboards/Statistic
- 136. Dashboards/Statistics Graph
- 137. Dashboards/Thermostat
- 138. Dashboards/Tile
- 139. Dashboards/ToDo List
- 140. Dashboards/Vertical Stack
- 141. Dashboards/Weather Forecast
- 142. Docs/Authentication/Multi Factor Auth
- 143. Docs/Authentication/Providers
- 144. Docs/Authentication
- 145. Docs/Automation/Action
- 146. Docs/Automation/Basics
- 147. Docs/Automation/Condition
- 148. Docs/Automation/Editor
- 149. Docs/Automation/Modes
- 150. Docs/Automation/Services
- 151. Docs/Automation/Templating
- 152. Docs/Automation/Trigger
- 153. Docs/Automation/Troubleshooting
- 154. Docs/Automation/Using Blueprints
- 155. Docs/Automation/Yaml
- 156. Docs/Automation
- 157. Docs/Blueprint/Schema

- 158. Docs/Blueprint/Selectors
- 159. Docs/Blueprint/Tutorial
- 160. Docs/Blueprint
- 161. Docs/Configuration/Basic
- 162. Docs/Configuration/Customizing Devices
- 163. Docs/Configuration/Entities Domains
- 164. Docs/Configuration/Events
- 165. Docs/Configuration/Packages
- 166. Docs/Configuration/Platform Options
- 167. Docs/Configuration/Remote
- 168. Docs/Configuration/Secrets
- 169. Docs/Configuration/Securing
- 170. Docs/Configuration/Splitting Configuration
- 171. Docs/Configuration/State Object
- 172. Docs/Configuration/Troubleshooting
- 173. Docs/Configuration/Yaml
- 174. Docs/Configuration
- 175. Docs/Energy/Battery
- 176. Docs/Energy/Electricity Grid
- 177. Docs/Energy/Faq
- 178. Docs/Energy/Gas
- 179. Docs/Energy/Individual Devices
- 180. Docs/Energy/Solar Panels
- 181. Docs/Energy/Water
- 182. Docs/Energy
- 183. Docs/Frontend/Icons
- 184. Docs/Glossary
- 185. Docs/Locked Out
- 186. Docs/Organizing/Areas
- 187. Docs/Organizing/Categories
- 188. Docs/Organizing/Floors
- 189. Docs/Organizing/Labels
- 190. Docs/Organizing/Tables
- 191. Docs/Organizing
- 192. Docs/Quality Scale
- 193. Docs/Scene/Editor
- 194. Docs/Scene
- 195. Docs/Scripts/Conditions
- 196. Docs/Scripts/Perform Actions
- 197. Docs/Scripts

198. Docs/Templating/Custom Templates
199. Docs/Templating/Dates And Times
200. Docs/Templating/Debugging
201. Docs/Templating/Errors
202. Docs/Templating/Introduction
203. Docs/Templating/Loops And Conditions
204. Docs/Templating/Patterns
205. Docs/Templating/Python Methods
206. Docs/Templating/States
207. Docs/Templating/Syntax
208. Docs/Templating/Tutorial Average Temperature
209. Docs/Templating/Tutorial Battery Alerts
210. Docs/Templating/Types
211. Docs/Templating/Where To Use
212. Docs/Templating/Yaml
213. Docs/Templating
214. Docs/Tools/Check Config
215. Docs/Tools/Dev Tools
216. Docs/Tools/Quick Search
217. Docs/Tools
218. Docs/Troubleshooting General
219. Docs/Z Wave/Controllers
220. Includes/Actions/More Examples
221. Includes/Actions/Related
222. Includes/Actions/Stuck
223. Includes/Actions/Targets
224. Includes/Actions/Try It
225. Includes/Actions/Ui Header
226. Includes/Actions/Yaml Header
227. Includes/Common Tasks/Backups
228. Includes/Common Tasks/Beta Version
229. Includes/Common Tasks/Commandline
230. Includes/Common Tasks/Configuration Check
231. Includes/Common Tasks/Data Disk
232. Includes/Common Tasks/Define Custom Polling
233. Includes/Common Tasks/Development Version
234. Includes/Common Tasks/Enable Entities
235. Includes/Common Tasks/Enable I2C
236. Includes/Common Tasks/File Access
237. Includes/Common Tasks/Filters

238. Includes/Common Tasks/Flashing M1S Otg
239. Includes/Common Tasks/Flashing N2 Otg
240. Includes/Common Tasks/Lost Password
241. Includes/Common Tasks/Network Storage
242. Includes/Common Tasks/Specific Version
243. Includes/Common Tasks/Third Party Addons
244. Includes/Common Tasks/Update
245. Includes/Conditions/Behavior
246. Includes/Conditions/More Examples
247. Includes/Conditions/Related
248. Includes/Conditions/Stuck
249. Includes/Conditions/Targets
250. Includes/Conditions/Try It
251. Includes/Conditions/Ui Header
252. Includes/Conditions/Yaml Header
253. Includes/Dashboard/Add Picture To Card
254. Includes/Dashboard/Edit Dashboard
255. Includes/Docs/Paste Yaml Tip
256. Includes/Energy/Ct Clamp
257. Includes/Energy/Rtl Sdr
258. Includes/Installation/Container/Alternative
259. Includes/Installation/Container/Cli
260. Includes/Installation/Container/Compose
261. Includes/Installation/Container
262. Includes/Installation/Operating System
263. Includes/Integrations/Actions
264. Includes/Integrations/Building Block Integration
265. Includes/Integrations/Conditions
266. Includes/Integrations/Config Flow
267. Includes/Integrations/Device Class Intro
268. Includes/Integrations/Google Api
269. Includes/Integrations/Google Client Secret
270. Includes/Integrations/Google Oauth
271. Includes/Integrations/Labs Entity Triggers Note
272. Includes/Integrations/Option Flow
273. Includes/Integrations/Remove Device Service
274. Includes/Integrations/Remove Device Service Steps
275. Includes/Integrations/Restart Ha After Config Inclusion
276. Includes/Integrations/Supported Brand
277. Includes/Integrations/Triggers

278. Includes/Integrations/Triggers Conditions Actions
279. Includes/Integrations/Using Templates
280. Includes/Integrations/Wwha
281. Includes/Organizing/Reorder Areas
282. Includes/Template Functions/More Examples
283. Includes/Template Functions/Related
284. Includes/Template Functions/Signatures
285. Includes/Template Functions/Stuck
286. Includes/Template Functions/Try It
287. Includes/Template Functions/Usage
288. Includes/Triggers/Behavior
289. Includes/Triggers/More Examples
290. Includes/Triggers/Related
291. Includes/Triggers/Stuck
292. Includes/Triggers/Targets
293. Includes/Triggers/Try It
294. Includes/Triggers/Ui Header
295. Includes/Triggers/Yaml Header
296. Template Functions/Abs
297. Template Functions/Acos
298. Template Functions/Add
299. Template Functions/Apply
300. Template Functions/Area Devices
301. Template Functions/Area Entities
302. Template Functions/Area Id
303. Template Functions/Area Name
304. Template Functions/Areas
305. Template Functions/As Datetime
306. Template Functions/As Function
307. Template Functions/As Local
308. Template Functions/As Timedelta
309. Template Functions/As Timestamp
310. Template Functions/Asin
311. Template Functions/Atan
312. Template Functions/Atan2
313. Template Functions/Attr
314. Template Functions/Average
315. Template Functions/Base64 Decode
316. Template Functions/Base64 Encode
317. Template Functions/Batch

318. Template Functions/Bitwise And
319. Template Functions/Bitwise Or
320. Template Functions/Bitwise Xor
321. Template Functions/Bool
322. Template Functions/Boolean Test
323. Template Functions/Callable
324. Template Functions/Capitalize
325. Template Functions/Center
326. Template Functions/Clamp
327. Template Functions/Closest
328. Template Functions/Combine
329. Template Functions/Config Entry Attr
330. Template Functions/Config Entry Id
331. Template Functions/Contains
332. Template Functions/Cos
333. Template Functions/Cycler
334. Template Functions/Default
335. Template Functions/Defined
336. Template Functions/Device Attr
337. Template Functions/Device Entities
338. Template Functions/Device Id
339. Template Functions/Device Name
340. Template Functions/Dict
341. Template Functions/Dictsort
342. Template Functions/Difference
343. Template Functions/Distance
344. Template Functions/Divisibleby
345. Template Functions/E
346. Template Functions/Entity Name
347. Template Functions/Eq
348. Template Functions/Escape
349. Template Functions/Even
350. Template Functions/Expand
351. Template Functions/False Test
352. Template Functions/Filesizeformat
353. Template Functions/First
354. Template Functions/Flatten
355. Template Functions/Float
356. Template Functions/Float Test
357. Template Functions/Floor Areas

358. Template Functions/Floor Entities
359. Template Functions/Floor Id
360. Template Functions/Floor Name
361. Template Functions/Floors
362. Template Functions/Forceescape
363. Template Functions/Format
364. Template Functions/From Hex
365. Template Functions/From Json
366. Template Functions/Ge
367. Template Functions/Groupby
368. Template Functions/Gt
369. Template Functions/Has Value
370. Template Functions/Lif
371. Template Functions/In Test
372. Template Functions/Indent
373. Template Functions/Int
374. Template Functions/Integer Test
375. Template Functions/Integration Entities
376. Template Functions/Intersect
377. Template Functions/Is Defined
378. Template Functions/Is Device Attr
379. Template Functions/Is Hidden Entity
380. Template Functions/Is Number
381. Template Functions/Is State
382. Template Functions/Is State Attr
383. Template Functions/Issue
384. Template Functions/Issues
385. Template Functions/Items
386. Template Functions/Iterable
387. Template Functions/Join
388. Template Functions/Joiner
389. Template Functions/Label Areas
390. Template Functions/Label Description
391. Template Functions/Label Devices
392. Template Functions/Label Entities
393. Template Functions/Label Id
394. Template Functions/Label Name
395. Template Functions/Labels
396. Template Functions/Last
397. Template Functions/Le

398. Template Functions/Length
399. Template Functions/Lipsum
400. Template Functions/List
401. Template Functions/Log
402. Template Functions/Lower
403. Template Functions/Lt
404. Template Functions/Map
405. Template Functions/Mapping
406. Template Functions/Match
407. Template Functions/Max
408. Template Functions/Md5
409. Template Functions/Median
410. Template Functions/Merge Response
411. Template Functions/Min
412. Template Functions/Multiply
413. Template Functions/Namespace
414. Template Functions/Ne
415. Template Functions/None
416. Template Functions/Now
417. Template Functions/Number Test
418. Template Functions/Odd
419. Template Functions/Ord
420. Template Functions/Ordinal
421. Template Functions/Pack
422. Template Functions/Pi
423. Template Functions/Pprint
424. Template Functions/Random
425. Template Functions/Range
426. Template Functions/Regex Findall
427. Template Functions/Regex Findall Index
428. Template Functions/Regex Match
429. Template Functions/Regex Replace
430. Template Functions/Regex Search
431. Template Functions/Reject
432. Template Functions/Rejectattr
433. Template Functions/Relative Time
434. Template Functions/Remap
435. Template Functions/Replace
436. Template Functions/Reverse
437. Template Functions/Round

438. Template Functions/Safe
439. Template Functions/Sameas
440. Template Functions/Search
441. Template Functions/Select
442. Template Functions/Selectattr
443. Template Functions/Sequence
444. Template Functions/Set
445. Template Functions/Sha1
446. Template Functions/Sha256
447. Template Functions/Sha512
448. Template Functions/Shuffle
449. Template Functions/Sin
450. Template Functions/Slice
451. Template Functions/Slugify
452. Template Functions/Sort
453. Template Functions/Sqrt
454. Template Functions/State Attr
455. Template Functions/State Attr Translated
456. Template Functions/State Translated
457. Template Functions/States
458. Template Functions/Statistical Mode
459. Template Functions/String
460. Template Functions/String Test
461. Template Functions/Striptags
462. Template Functions/Strptime
463. Template Functions/Sum
464. Template Functions/Symmetric Difference
465. Template Functions/Tan
466. Template Functions/Tau
467. Template Functions/Time Since
468. Template Functions/Time Until
469. Template Functions/Timedelta
470. Template Functions/Timestamp Custom
471. Template Functions/Timestamp Local
472. Template Functions/Timestamp Utc
473. Template Functions/Title
474. Template Functions/To Json
475. Template Functions/Today At
476. Template Functions/ToJson
477. Template Functions/Trim

478. Template Functions/True Test
479. Template Functions/Truncate
480. Template Functions/Tuple
481. Template Functions/Typeof
482. Template Functions/Undefined
483. Template Functions/Union
484. Template Functions/Unique
485. Template Functions/Unpack
486. Template Functions/Upper
487. Template Functions/Urlencode
488. Template Functions/Urlize
489. Template Functions/Utcnow
490. Template Functions/Version
491. Template Functions/Wordcount
492. Template Functions/Wordwrap
493. Template Functions/Wrap
494. Template Functions/Xmlattr
495. Template Functions/Zip
496. Triggers/Air Quality.Co2 Changed
497. Triggers/Air Quality.Co2 Crossed Threshold
498. Triggers/Air Quality.Co Changed
499. Triggers/Air Quality.Co Cleared
500. Triggers/Air Quality.Co Crossed Threshold
501. Triggers/Air Quality.Co Detected
502. Triggers/Air Quality.Gas Cleared
503. Triggers/Air Quality.Gas Detected
504. Triggers/Air Quality.N2O Changed
505. Triggers/Air Quality.N2O Crossed Threshold
506. Triggers/Air Quality.No2 Changed
507. Triggers/Air Quality.No2 Crossed Threshold
508. Triggers/Air Quality.No Changed
509. Triggers/Air Quality.No Crossed Threshold
510. Triggers/Air Quality.Ozone Changed
511. Triggers/Air Quality.Ozone Crossed Threshold
512. Triggers/Air Quality.Pm10 Changed
513. Triggers/Air Quality.Pm10 Crossed Threshold
514. Triggers/Air Quality.Pm1 Changed
515. Triggers/Air Quality.Pm1 Crossed Threshold
516. Triggers/Air Quality.Pm25 Changed
517. Triggers/Air Quality.Pm25 Crossed Threshold

518. Triggers/Air Quality.Pm4 Changed
519. Triggers/Air Quality.Pm4 Crossed Threshold
520. Triggers/Air Quality.Smoke Cleared
521. Triggers/Air Quality.Smoke Detected
522. Triggers/Air Quality.So2 Changed
523. Triggers/Air Quality.So2 Crossed Threshold
524. Triggers/Air Quality.Voc Changed
525. Triggers/Air Quality.Voc Crossed Threshold
526. Triggers/Air Quality.Voc Ratio Changed
527. Triggers/Air Quality.Voc Ratio Crossed Threshold
528. Triggers/Alarm Control Panel.Armed
529. Triggers/Alarm Control Panel.Armed Away
530. Triggers/Alarm Control Panel.Armed Home
531. Triggers/Alarm Control Panel.Armed Night
532. Triggers/Alarm Control Panel.Armed Vacation
533. Triggers/Alarm Control Panel.Disarmed
534. Triggers/Alarm Control Panel.Triggered
535. Triggers/Button.Pressed
536. Triggers/Calendar.Event Ended
537. Triggers/Calendar.Event Started
538. Triggers/Climate.Hvac Mode Changed
539. Triggers/Climate.Turned Off
540. Triggers/Climate.Turned On
541. Triggers/Door.Closed
542. Triggers/Door.Opened
543. Triggers/Fan.Turned Off
544. Triggers/Fan.Turned On
545. Triggers/Gate.Closed
546. Triggers/Gate.Opened
547. Triggers/Humidifier.Mode Changed
548. Triggers/Humidifier.Started Drying
549. Triggers/Humidifier.Started Humidifying
550. Triggers/Humidifier.Turned Off
551. Triggers/Humidifier.Turned On
552. Triggers/Humidity.Changed
553. Triggers/Humidity.Crossed Threshold
554. Triggers/Light.Brightness Changed
555. Triggers/Light.Brightness Crossed Threshold
556. Triggers/Light.Turned Off
557. Triggers/Light.Turned On

558. Triggers/Lock.Jammed
559. Triggers/Lock.Locked
560. Triggers/Lock.Opened
561. Triggers/Lock.Unlocked
562. Triggers/Motion.Cleared
563. Triggers/Motion.Detected
564. Triggers/Temperature.Changed
565. Triggers/Temperature.Crossed Threshold
566. Triggers/Vacuum.Docked
567. Triggers/Vacuum.Errorred
568. Triggers/Vacuum.Paused Cleaning
569. Triggers/Vacuum.Started Cleaning
570. Triggers/Vacuum.Started Returning
571. Triggers/Window.Closed
572. Triggers/Window.Opened
573. Android/Index
574. Cloud/Alexa
575. Cloud/Google Assistant
576. Cloud/Index
577. Cloud/Troubleshooting
578. Common Tasks/Container
579. Common Tasks/General
580. Common Tasks/Os
581. Dashboards/Actions
582. Dashboards/Badges
583. Dashboards/Cards
584. Dashboards/Dashboards
585. Dashboards/Features
586. Dashboards/Header Footer
587. Dashboards/Index
588. Dashboards/Naming
589. Dashboards/Views
590. Developers/Cla
591. Developers/Credits
592. Developers/License
593. Faq/Index
594. Getting Started/Automation
595. Getting Started/Concepts Terminology
596. Getting Started/Configuration
597. Getting Started/Index

598. Getting Started/Integration

599. Getting Started/Join The Community

600. Getting Started/Onboarding

601. Getting Started/Onboarding Dashboard

602. Getting Started/Presence Detection

603. Help/Index

604. Help/Reporting Issues

605. Help/Talking Points

606. Help/Trivia

607. Installation/Alternative

608. Installation/Generic X86 64

609. Installation/Linux

610. Installation/Macos

611. Installation/Odroid

612. Installation/Raspberrypi Other

613. Installation/Raspberrypi

614. Installation/Troubleshooting

615. Installation/Windows

616. Ios/Beta Auth

617. Ios/Dev Auth

618. Ios/Index

619. Ios/Nfc

620. More Info/Backup Emergency Kit

621. More Info/Dockerhub Rate Limit

622. More Info/Free Space

623. More Info/Local Media/Add Media

624. More Info/Local Media/Setup Media

625. More Info/Nest Auth Deprecation

626. More Info/No Url Available

627. More Info/Pwned Passwords

628. More Info/Removed Integration

629. More Info/Statistics

630. More Info/System Information

631. More Info/Unhealthy/Docker

632. More Info/Unhealthy/Docker Gateway Unprotected

633. More Info/Unhealthy/Duplicate Os Installation

634. More Info/Unhealthy/Oserror Bad Message

635. More Info/Unhealthy/Privileged

636. More Info/Unhealthy/Setup

637. More Info/Unhealthy/Supervisor

638. More Info/Unhealthy/Untrusted
639. More Info/Unsupported/Apparmor
640. More Info/Unsupported/Cgroup Version
641. More Info/Unsupported/Connectivity Check
642. More Info/Unsupported/Dbus
643. More Info/Unsupported/Dns Server
644. More Info/Unsupported/Docker Configuration
645. More Info/Unsupported/Docker Version
646. More Info/Unsupported/Home Assistant Core Version
647. More Info/Unsupported/Job Conditions
648. More Info/Unsupported/Lxc
649. More Info/Unsupported/Network Manager
650. More Info/Unsupported/Os
651. More Info/Unsupported/Os Agent
652. More Info/Unsupported/Os Version
653. More Info/Unsupported/Restart Policy
654. More Info/Unsupported/Software
655. More Info/Unsupported/Supervisor Version
656. More Info/Unsupported/System Architecture
657. More Info/Unsupported/Systemd
658. More Info/Unsupported/Systemd Journal
659. More Info/Unsupported/Systemd Resolved
660. More Info/Unsupported/Virtualization Image
661. Voice Control/About Wake Word
662. Voice Control/Aliases
663. Voice Control/Android
664. Voice Control/Apple
665. Voice Control/Assign Areas Floors
666. Voice Control/Assist Create Open Ai Personality
667. Voice Control/Assist Daily Summary
668. Voice Control/Best Practices
669. Voice Control/Builtin Sentences
670. Voice Control/Contribute Voice
671. Voice Control/Create Wake Word
672. Voice Control/Custom Sentences
673. Voice Control/Custom Sentences Yaml
674. Voice Control/Expanding Assist
675. Voice Control/Index
676. Voice Control/Install Wake Word Add On
677. Voice Control/Make Your Own

- 678. Voice Control/S3 Box Customize
 - 679. Voice Control/S3 Box Voice Assistant
 - 680. Voice Control/Start Assist From Dashboard
 - 681. Voice Control/Thirteen Usd Voice Remote
 - 682. Voice Control/Troubleshooting
 - 683. Voice Control/Troubleshooting The S3 Box
 - 684. Voice Control/Using Tts In Automation
 - 685. Voice Control/Voice Remote Cloud Assistant
 - 686. Voice Control/Voice Remote Expose Devices
 - 687. Voice Control/Voice Remote Local Assistant
 - 688. Voice Control/Worlds Most Private Voice Assistant
-
-

Actions/Alarm Control Panel.Alarm Arm Away

The **Arm alarm away** action arms your alarm control panel in away mode. Heading out to work? Your alarm locks everything down automatically as you leave, protecting all zones while nobody is home.

This action works with any alarm control panel entity in Home Assistant that supports away mode. If the alarm is already armed in away mode, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To arm an alarm in away mode from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Arm alarm away**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to arm the alarm. Not every alarm panel requires a code for arming. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_arm_away`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_arm_away
  target:
    entity_id: alarm_control_panel.home_alarm
```

This arms `alarm_control_panel.home_alarm` in away mode without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_arm_away
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to arm the alarm. Not every alarm panel requires a code for arming. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Arm alarm away** action works on any alarm control panel entity in Home Assistant that supports away mode.
- Away mode typically activates all sensors and zones, including interior motion sensors. Use this when nobody is home.
- Whether a code is required depends on your alarm panel and its configuration.
- To disarm the alarm when you return, use [Disarm alarm](#). To arm while staying inside, use [Arm alarm home](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: arm the alarm in away mode with a code

Arm the home alarm in away mode using a PIN code. Useful as a quick action on a dashboard tile before you head out.

- **Action:** Alarm control panel: Arm alarm away
- **Target:** Home alarm
- **Code:** 1234

YAML example for arming away with a code

****action****

```
action: |
  action: alarm_control_panel.alarm_arm_away
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Automation: arm the alarm when everyone leaves

When the last person walks out the door, arm the alarm in away mode. A simple way to make sure you never forget to set the alarm.

- **Trigger:** Zone: Everyone leaves home
- **Action:** Alarm control panel: Arm alarm away
- **Target:** Home alarm

YAML example for arming when everyone leaves

****automation****

```

automation: |
  alias: "Arm alarm when everyone leaves"
  triggers:
    - trigger: state
      entity_id: person.paulus
      to: not_home
  conditions:
    - condition: state
      entity_id: group.family
      state: not_home
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Automation: arm away at a set time each workday

If you leave for work at the same time every day, arm the alarm automatically at 8:30 in the morning on weekdays.

- **Trigger:** Time: 08:30
- **Condition:** Day of the week is Monday to Friday
- **Condition:** Nobody is home
- **Action:** Alarm control panel: Arm alarm away
- **Target:** Home alarm

YAML example for a scheduled workday arm

```

**automation**

```

```
automation: |
  alias: "Arm alarm on workday mornings"
  triggers:
    - trigger: time
      at: "08:30:00"
  conditions:
    - condition: time
      weekday:
        - mon
        - tue
        - wed
        - thu
        - fri
    - condition: state
      entity_id: group.family
      state: not_home
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Alarm Control Panel.Alarm Arm Custom Bypass

The **Arm alarm with custom bypass** action arms your alarm control panel while allowing you to bypass specific zones. Think of it as arming the house but leaving the dog door zone active so your pet can come and go, or skipping a sensor on a window you want to keep open for fresh air.

This action works with any alarm control panel entity in Home Assistant that supports custom bypass. If the alarm is already armed with custom bypass, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To arm an alarm with custom bypass from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Arm alarm with custom bypass**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to arm the alarm. Not every alarm panel requires a code for arming. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_arm_custom_bypass`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_arm_custom_bypass
  target:
    entity_id: alarm_control_panel.home_alarm
```

This arms `alarm_control_panel.home_alarm` with custom bypass without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_arm_custom_bypass
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to arm the alarm. Not every alarm panel requires a code for arming. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Arm alarm with custom bypass** action works on any alarm control panel entity in Home Assistant that supports custom bypass.
- Which zones are bypassed depends on your alarm panel's configuration. The bypass rules are typically set up in the alarm panel itself, not in Home Assistant.
- Whether a code is required depends on your alarm panel and its configuration.
- For full protection without any bypasses, use [Arm alarm away](#) or [Arm alarm home](#). To disarm the alarm, use [Disarm alarm](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: arm with bypass from a dashboard

Set up a quick action on your dashboard to arm the alarm while keeping certain zones open. Useful when you know a window is cracked or the dog door is in use.

- **Action:** Alarm control panel: Arm alarm with custom bypass
- **Target:** Home alarm
- **Code:** 1234

YAML example for arming with custom bypass

****action****

```
action: |
  action: alarm_control_panel.alarm_arm_custom_bypass
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Automation: arm with bypass when a window is left open

If you leave the house and a window is still open, arm the alarm with custom bypass instead of leaving the alarm off entirely. This way, the rest of the house stays protected.

- **Trigger:** Person: Paulus changes to not_home
- **Condition:** Window sensor is open
- **Action:** Alarm control panel: Arm alarm with custom bypass
- **Target:** Home alarm

YAML example for arming with bypass on open window

****automation****

```
automation: |
  alias: "Arm with bypass when window open"
  triggers:
    - trigger: state
      entity_id: person.paulus
      to: not_home
  conditions:
    - condition: state
      entity_id: binary_sensor.living_room_window
      state: "on"
  actions:
    - action: alarm_control_panel.alarm_arm_custom_bypass
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Alarm Control Panel.Alarm Arm Home

The **Arm alarm home** action arms your alarm control panel in home mode. This is perfect for when you're relaxing inside but still want the perimeter secured. Doors and windows stay monitored while interior motion sensors stay quiet, so you can move around freely.

This action works with any alarm control panel entity in Home Assistant that supports home mode. If the alarm is already armed in home mode, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To arm an alarm in home mode from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Arm alarm home**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to arm the alarm. Not every alarm panel requires a code for arming. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_arm_home`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_arm_home
  target:
    entity_id: alarm_control_panel.home_alarm
```

This arms `alarm_control_panel.home_alarm` in home mode without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_arm_home
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to arm the alarm. Not every alarm panel requires a code for arming. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Arm alarm home** action works on any alarm control panel entity in Home Assistant that supports home mode.
- Home mode typically monitors perimeter sensors (doors and windows) while keeping interior motion sensors inactive. This lets you move around inside without triggering the alarm.
- Whether a code is required depends on your alarm panel and its configuration.
- To arm all zones when leaving the house, use [Arm alarm away](#). To disarm the alarm entirely, use [Disarm alarm](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: arm in home mode from a dashboard

Set up a quick action on your dashboard so you can arm the perimeter with a single tap while you're lounging on the couch.

- **Action:** Alarm control panel: Arm alarm home
- **Target:** Home alarm
- **Code:** 1234

YAML example for arming home with a code

****action****

```
action: |
  action: alarm_control_panel.alarm_arm_home
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Automation: arm the perimeter when you arrive home at night

When you get home after dark, automatically arm the alarm in home mode. Your doors and windows stay protected while you settle in for the evening.

- **Trigger:** Person: Paulus changes to home
- **Condition:** Sun is below horizon
- **Action:** Alarm control panel: Arm alarm home
- **Target:** Home alarm

YAML example for arming home on nighttime arrival

****automation****

```

automation: |
  alias: "Arm home mode on nighttime arrival"
  triggers:
    - trigger: state
      entity_id: person.paulus
      to: home
  conditions:
    - condition: sun
      after: sunset
  actions:
    - action: alarm_control_panel.alarm_arm_home
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Automation: arm the perimeter every evening

Each evening at 10 o'clock, arm the alarm in home mode so the perimeter is secured while you sleep. Pair this with a night mode automation for even tighter protection later on.

- **Trigger:** Time: 22:00
- **Action:** Alarm control panel: Arm alarm home
- **Target:** Home alarm

YAML example for an evening perimeter arm

****automation****

```

automation: |
  alias: "Arm perimeter every evening"
  triggers:
    - trigger: time
      at: "22:00:00"
  actions:
    - action: alarm_control_panel.alarm_arm_home
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Alarm Control Panel.Alarm Arm Night

The **Arm alarm night** action arms your alarm control panel in night mode. Bedtime security without lifting a finger. Night mode typically protects the perimeter and selected interior zones while you sleep, so you stay safe without triggering the alarm on a late-night trip to the kitchen.

This action works with any alarm control panel entity in Home Assistant that supports night mode. If the alarm is already armed in night mode, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To arm an alarm in night mode from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Arm alarm night**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to arm the alarm. Not every alarm panel requires a code for arming. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_arm_night`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_arm_night
  target:
    entity_id: alarm_control_panel.home_alarm
```

This arms `alarm_control_panel.home_alarm` in night mode without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_arm_night
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to arm the alarm. Not every alarm panel requires a code for arming. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Arm alarm night** action works on any alarm control panel entity in Home Assistant that supports night mode.
- Night mode is similar to home mode but is designed for sleeping hours. Which zones it activates depends on your alarm panel's configuration.
- Whether a code is required depends on your alarm panel and its configuration.
- For broader protection while still at home, consider [Arm alarm home](#). To disarm the alarm in the morning, use [Disarm alarm](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: switch to night mode from a dashboard

Add a dashboard tile that arms the alarm in night mode with a single tap. A quick way to lock down the house before you head upstairs.

- **Action:** Alarm control panel: Arm alarm night
- **Target:** Home alarm
- **Code:** 1234

YAML example for arming night with a code

****action****

```
action: |
  action: alarm_control_panel.alarm_arm_night
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Automation: arm night mode at bedtime

At 11 in the evening, switch the alarm to night mode automatically. You go to sleep knowing the house is protected without having to walk to the keypad.

- **Trigger:** Time: 23:00
- **Action:** Alarm control panel: Arm alarm night
- **Target:** Home alarm

YAML example for a bedtime night mode automation

****automation****

```

automation: |
  alias: "Arm night mode at bedtime"
  triggers:
    - trigger: time
      at: "23:00:00"
  actions:
    - action: alarm_control_panel.alarm_arm_night
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Automation: arm night mode when the bedroom lights go off

When the last light in the bedroom turns off, arm the alarm in night mode. This ties your security to your actual routine instead of a fixed time.

- **Trigger:** Light: Bedroom light turns off
- **Action:** Alarm control panel: Arm alarm night
- **Target:** Home alarm

YAML example for arming night when bedroom lights go off

****automation****

```

automation: |
  alias: "Arm night mode when bedroom lights off"
  triggers:
    - trigger: state
      entity_id: light.bedroom
      to: "off"
  actions:
    - action: alarm_control_panel.alarm_arm_night
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Alarm Control Panel.Alarm Arm Vacation

The **Arm alarm vacation** action arms your alarm control panel in vacation mode. Heading out for an extended trip? Vacation mode gives your home the highest level of protection while you're away for days or weeks. Some alarm panels use this mode to enable additional monitoring or longer alert windows.

This action works with any alarm control panel entity in Home Assistant that supports vacation mode. If the alarm is already armed in vacation mode, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To arm an alarm in vacation mode from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Arm alarm vacation**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to arm the alarm. Not every alarm panel requires a code for arming. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_arm_vacation`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_arm_vacation
  target:
    entity_id: alarm_control_panel.home_alarm
```

This arms `alarm_control_panel.home_alarm` in vacation mode without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_arm_vacation
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to arm the alarm. Not every alarm panel requires a code for arming. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Arm alarm vacation** action works on any alarm control panel entity in Home Assistant that supports vacation mode.
- Vacation mode is designed for extended absences. Depending on your alarm panel, it activates all sensors and zones with heightened sensitivity or longer siren durations.
- Whether a code is required depends on your alarm panel and its configuration.
- For shorter trips, [Arm alarm away](#) is typically sufficient. To disarm when you return, use [Disarm alarm](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: switch to vacation mode from a script

Create a "leaving for vacation" script that arms the alarm in vacation mode. You can add other departure tasks to the same script separately.

- **Action:** Alarm control panel: Arm alarm vacation
- **Target:** Home alarm
- **Code:** 1234

YAML example for arming vacation with a code

****action****

```
action: |
  action: alarm_control_panel.alarm_arm_vacation
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Automation: arm vacation mode when away for 24 hours

If everyone has been away from home for a full day, upgrade the alarm from away mode to vacation mode automatically. This gives you extended protection without having to remember to switch modes before a trip.

- **Trigger:** Person: Paulus has been away for 24 hours
- **Action:** Alarm control panel: Arm alarm vacation
- **Target:** Home alarm

YAML example for upgrading to vacation mode after 24 hours

****automation****

```
automation: |
  alias: "Arm vacation after 24 hours away"
  triggers:
    - trigger: state
      entity_id: group.family
      to: not_home
      for: "24:00:00"
  actions:
    - action: alarm_control_panel.alarm_arm_vacation
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Alarm Control Panel.Alarm Disarm

The **Disarm alarm** action disarms your alarm control panel. Picture this: you walk through the front door after a long day, and your alarm automatically disarms for you. No rushing to the keypad, no frantic code entry.

This action works with any alarm control panel entity in Home Assistant. If the alarm is already disarmed, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To disarm an alarm from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Disarm alarm**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to disarm the alarm. Not every alarm panel requires a code. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_disarm`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_disarm
  target:
    entity_id: alarm_control_panel.home_alarm
```

This disarms `alarm_control_panel.home_alarm` without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_disarm
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to disarm the alarm. Not every alarm panel requires a code. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Disarm alarm** action works on any alarm control panel entity in Home Assistant.
- If the alarm is already disarmed, calling this action does nothing.
- Whether a code is required depends on your alarm panel and its configuration. Some panels always require a code, others never do.
- To arm the alarm again after disarming, use [Arm alarm away](#) or [Arm alarm home](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: disarm the alarm with a code

Disarm the home alarm using a PIN code. Useful as a quick action on a dashboard or from a script.

- **Action:** Alarm control panel: Disarm alarm
- **Target:** Home alarm
- **Code:** 1234

YAML example for disarming with a code

****action****

```
action: |
  action: alarm_control_panel.alarm_disarm
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Automation: disarm when you arrive home

When you pull into the driveway and your phone reports you're home, disarm the alarm automatically. No more rushing to the keypad before the siren goes off.

- **Trigger:** Person: Paulus changes to home
- **Action:** Alarm control panel: Disarm alarm
- **Target:** Home alarm

YAML example for disarming on arrival

****automation****

```

automation: |
  alias: "Disarm alarm on arrival"
  triggers:
    - trigger: state
      entity_id: person.paulus
      to: home
  actions:
    - action: alarm_control_panel.alarm_disarm
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Automation: disarm every morning on weekdays

Start the day without the alarm blaring when you open the bedroom door. Disarm it automatically at 7 in the morning on weekdays.

- **Trigger:** Time: 07:00
- **Condition:** Day of the week is Monday to Friday
- **Action:** Alarm control panel: Disarm alarm
- **Target:** Home alarm

YAML example for a weekday morning disarm

****automation****

```

automation: |
  alias: "Disarm alarm on weekday mornings"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: time
      weekday:
        - mon
        - tue
        - wed
        - thu
        - fri
  actions:
    - action: alarm_control_panel.alarm_disarm
      target:
        entity_id: alarm_control_panel.home_alarm
      data:
        code: "1234"

```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Alarm Control Panel.Alarm Trigger

The **Trigger alarm** action manually triggers your alarm control panel. This is useful for testing your alarm setup to make sure the siren, notifications, and automations all respond correctly. You might also use it in an emergency scenario where you want to sound the alarm on demand.

This action works with any alarm control panel entity in Home Assistant that supports triggering.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To trigger an alarm from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Trigger alarm**.
7. *Optional:* enter the **Code** if your alarm panel requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to trigger the alarm. Not every alarm panel requires a code for triggering. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `alarm_control_panel.alarm_trigger`. A basic example looks like this:

action

```
action: |
  action: alarm_control_panel.alarm_trigger
  target:
    entity_id: alarm_control_panel.home_alarm
```

This triggers `alarm_control_panel.home_alarm` without a code.

If your alarm panel requires a code, include it in the `data` section:

action

```
action: |
  action: alarm_control_panel.alarm_trigger
  target:
    entity_id: alarm_control_panel.home_alarm
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to trigger the alarm. Not every alarm panel requires a code for triggering. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Trigger alarm** action works on any alarm control panel entity in Home Assistant that supports triggering.
- Triggering the alarm sets it to the triggered state, which typically sounds the siren and sends notifications. Use with care in a production setup.
- Whether a code is required depends on your alarm panel and its configuration.
- To silence the alarm after triggering, use [Disarm alarm](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: trigger the alarm for testing

Manually trigger the alarm to verify your siren and notification automations are working. Remember to disarm it again once you've confirmed everything works.

- **Action:** Alarm control panel: Trigger alarm
- **Target:** Home alarm

YAML example for manually triggering the alarm

****action****

```
action: |
  action: alarm_control_panel.alarm_trigger
  target:
    entity_id: alarm_control_panel.home_alarm
```

Automation: trigger the alarm on a panic button press

Wire a physical button to trigger the alarm instantly. Keep the button in a discreet location so you have a quick way to sound the alarm in an emergency.

- **Trigger:** Device: Panic button pressed
- **Action:** Alarm control panel: Trigger alarm
- **Target:** Home alarm

YAML example for a panic button alarm trigger

****automation****

```

automation: |
  alias: "Panic button triggers alarm"
  triggers:
    - trigger: state
      entity_id: binary_sensor.panic_button
      to: "on"
  actions:
    - action: alarm_control_panel.alarm_trigger
      target:
        entity_id: alarm_control_panel.home_alarm

```

Automation: trigger the alarm on smoke detection

If a smoke sensor goes off while the alarm is not disarmed, trigger the alarm to sound the siren and alert you immediately.

- **Trigger:** Smoke sensor detects smoke
- **Condition:** Alarm is not disarmed
- **Action:** Alarm control panel: Trigger alarm
- **Target:** Home alarm

YAML example for triggering the alarm on smoke detection

****automation****

```

automation: |
  alias: "Trigger alarm on smoke detection"
  triggers:
    - trigger: state
      entity_id: binary_sensor.smoke_detector
      to: "on"
  conditions:
    - condition: not
      conditions:
        - condition: state
          entity_id: alarm_control_panel.home_alarm
          state: disarmed
  actions:
    - action: alarm_control_panel.alarm_trigger
      target:
        entity_id: alarm_control_panel.home_alarm

```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Button.Press

Use this action when an integration exposes a button entity and you want an automation to press it for you. This is useful for tasks like restarting a device, starting an update, or running another one-time device action.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To press a button from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), select the area, floor, device, label, or entity you want to control.
6. From the actions shown for that target, select **Press button**.
7. Select **Save**.

Options in the UI

This action has no additional options beyond the target.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `button.press`. A basic example looks like this:

action

```
action: |
  action: button.press
  target:
    entity_id: button.router_restart
```

This presses `button.router_restart`.

Options in YAML

This action has no additional YAML options beyond the target.

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works with button entities.
- If the button entity is `unavailable`, the action cannot run until the entity is available again.
- Button actions do not take extra fields. You only need to select the target.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press **Ctrl+V** (or **Cmd+V** on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: restart a router when the internet is down

Use this automation to press a restart button after the internet connection has been down for a while.

- **Trigger:** Internet connection turns off for 10 minutes
- **Action:** Press button
- **Target:** Router restart button

Show example YAML

****automation****

```
automation: |
  - alias: "Restart the router when the internet has been down"
    triggers:
      - trigger: state
        entity_id: binary_sensor.internet_connection
        to: "off"
        for: "00:10:00"
    actions:
      - action: button.press
        target:
          entity_id: button.router_restart
```

Automation: run a firmware update overnight

Use this automation when a device exposes an update button and you want to run it at a quiet time.

- **Trigger:** Time: 03:00
- **Condition:** Garden controller firmware update is available
- **Action:** Press button
- **Target:** Garden controller update button

Show example YAML

****automation****

```
automation: |
  - alias: "Run the garden controller firmware update overnight"
    triggers:
      - trigger: time
        at: "03:00:00"
    conditions:
      - condition: state
        entity_id: update.garden_controller_firmware
        state: "on"
    actions:
      - action: button.press
        target:
          entity_id: button.garden_controller_update
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Decrease Speed

The **Decrease fan speed** action is useful when you want a gentler airflow without picking an exact final value. Use it to lower the fan by one step or by a percentage you choose.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Decrease fan speed**.
7. Optional: under **Decrement**, set how much the speed should decrease.
8. Select **Save**.

Options in the UI

{% options_ui %} Decrement: description: How much the speed should decrease, in percent.
required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.decrease_speed`. A basic example looks like this:

action

```
action: |
  action: fan.decrease_speed
  target:
    entity_id: fan.bedroom
  data:
    percentage_step: 15
```

This decreases `fan.bedroom` by 15 percent.

Options in YAML

{% options_yaml %} percentage_step: description: How much the speed should decrease, in percent. required: false type: integer

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support speed control.
- If you leave **Decrement** empty, the integration decides the step size.
- To set an exact speed instead, use [Set fan speed](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press **Ctrl+V** (or **Cmd+V** on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: slow the bedroom fan before you fall asleep

Reduce the airflow a little once the evening has settled down.

- **Trigger:** Time: 23:00
- **Action:** Decrease fan speed
- **Target:** Bedroom fan
- **Decrement:** 10

YAML example for a quieter bedtime fan

****automation****

```
automation: |
  alias: "Decrease bedroom fan speed at night"
  triggers:
    - trigger: time
      at: "23:00:00"
  actions:
    - action: fan.decrease_speed
      target:
        entity_id: fan.bedroom
      data:
        percentage_step: 10
```

Automation: lower the living room fan after sunset

Once the day cools down, you may only need a softer airflow.

- **Trigger:** Sun: Below horizon
- **Action:** Decrease fan speed
- **Target:** Living room fan
- **Decrement:** 20

YAML example for a sunset fan slowdown

****automation****

```
automation: |
  alias: "Decrease living room fan after sunset"
  triggers:
    - trigger: sun
      event: sunset
  actions:
    - action: fan.decrease_speed
      target:
        entity_id: fan.living_room
      data:
        percentage_step: 20
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Increase Speed

The **Increase fan speed** action is useful when you want more airflow without choosing an exact final value. Use it to nudge the fan up by one step or by a percentage you choose.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Increase fan speed**.
7. Optional: under **Increment**, set how much the speed should increase.
8. Select **Save**.

Options in the UI

{% options_ui %} Increment: description: How much the speed should increase, in percent.
required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.increase_speed`. A basic example looks like this:

action

```
action: |
  action: fan.increase_speed
  target:
    entity_id: fan.office
  data:
    percentage_step: 10
```

This increases `fan.office` by 10 percent.

Options in YAML

{% options_yaml %} percentage_step: description: How much the speed should increase, in percent. required: false type: integer

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support speed control.
- If you leave **Increment** empty, the integration decides the step size.
- To set an exact speed instead, use [Set fan speed](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: increase the office fan speed at midday

If the room gets warmer as the day goes on, you can raise the fan speed a little without switching straight to full power.

- **Trigger:** Time: 12:00
- **Action:** Increase fan speed
- **Target:** Office fan
- **Increment:** 10

YAML example for a midday speed boost

****automation****

```
automation: |
  alias: "Increase office fan at midday"
  triggers:
    - trigger: time
      at: "12:00:00"
  actions:
    - action: fan.increase_speed
      target:
        entity_id: fan.office
      data:
        percentage_step: 10
```

Automation: give the kitchen fan a bigger boost after cooking starts

When cooking begins, you may want to push the fan up more quickly.

- **Trigger:** State: Stove hood light changes to on
- **Action:** Increase fan speed
- **Target:** Kitchen fan
- **Increment:** 20

YAML example for a kitchen airflow boost

****automation****

```
automation: |
  alias: "Increase kitchen fan when cooking starts"
  triggers:
    - trigger: state
      entity_id: light.stove_hood
      to: "on"
  actions:
    - action: fan.increase_speed
      target:
        entity_id: fan.kitchen
      data:
        percentage_step: 20
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Oscillate

The **Oscillate fan** action is useful when you want to spread airflow across a wider area. Use it to turn oscillation on when more people are in the room, or turn it off when you want air aimed in one direction.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Oscillate fan**.
7. Under **Oscillating**, choose whether oscillation should be on or off.
8. Select **Save**.

Options in the UI

{% options_ui %} Oscillating: description: Turns oscillation on or off. required: true

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.oscillate`. A basic example looks like this:

action

```
action: |
  action: fan.oscillate
  target:
    entity_id: fan.living_room
  data:
    oscillating: true
```

This turns oscillation on for `fan.living_room`.

Options in YAML

{% options_yaml %} oscillating: description: Turns oscillation on or off. Accepts `true` or `false`.

required: true type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support oscillation.
- It changes only the oscillation setting. It does not turn the fan on or off by itself.
- To start the fan and set other options, use [Turn on fan](#).

Try it yourself

Ready to test this? Open **Developer tools** > **Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn oscillation on when dinner starts

If several people are in the dining room, wider airflow can make the room feel more comfortable.

- **Trigger:** Time: 18:30
- **Action:** Oscillate fan
- **Target:** Dining room fan
- **Oscillating:** On

YAML example for wider airflow at dinner time

****automation****

```
automation: |
  alias: "Dining room fan oscillation on"
  triggers:
    - trigger: time
      at: "18:30:00"
  actions:
    - action: fan.oscillate
      target:
        entity_id: fan.dining_room
      data:
        oscillating: true
```

Automation: turn oscillation off during a video call

If the fan points papers around your desk, you can stop oscillation before a scheduled meeting.

- **Trigger:** Time: 09:00
- **Action:** Oscillate fan
- **Target:** Office fan
- **Oscillating:** Off

YAML example for steady airflow during a call

****automation****

```
automation: |
  alias: "Office fan oscillation off for calls"
  triggers:
    - trigger: time
      at: "09:00:00"
  actions:
    - action: fan.oscillate
      target:
        entity_id: fan.office
      data:
        oscillating: false
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Set Direction

The **Set fan direction** action is useful for fans that can reverse their rotation. Use it when you want one direction for cooling and the opposite direction for a gentler airflow pattern.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Set fan direction**.
7. Under **Direction**, choose **Forward** or **Reverse**.
8. Select **Save**.

Options in the UI

{% options_ui %} Direction: description: The direction of the fan rotation. required: true

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.set_direction`. A basic example looks like this:

action

```
action: |
  action: fan.set_direction
  target:
    entity_id: fan.ceiling_fan
  data:
    direction: forward
```

This sets `fan.ceiling_fan` to the `forward` direction.

Options in YAML

{% options_yaml %} direction: description: The direction of the fan rotation. Accepts `forward` or `reverse`. required: true type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support changing direction.
- The valid YAML values are `forward` and `reverse`.
- Changing direction does not turn the fan on by itself.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: set the ceiling fan to forward in the afternoon

If you prefer a stronger downward breeze during the hottest part of the day, switch the fan to forward automatically.

- **Trigger:** Time: 13:00
- **Action:** Set fan direction
- **Target:** Ceiling fan
- **Direction:** Forward

YAML example for afternoon forward airflow

****automation****

```
automation: |
  alias: "Ceiling fan forward in the afternoon"
  triggers:
    - trigger: time
      at: "13:00:00"
  actions:
    - action: fan.set_direction
      target:
        entity_id: fan.ceiling_fan
      data:
        direction: forward
```

Automation: set the bedroom fan to reverse at night

If your fan supports it, reverse rotation can give you a gentler feel while you sleep.

- **Trigger:** Time: 22:00
- **Action:** Set fan direction
- **Target:** Bedroom fan
- **Direction:** Reverse

YAML example for nighttime reverse airflow

****automation****

```
automation: |
  alias: "Bedroom fan reverse at night"
  triggers:
    - trigger: time
      at: "22:00:00"
  actions:
    - action: fan.set_direction
      target:
        entity_id: fan.bedroom
      data:
        direction: reverse
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Set Percentage

The **Set fan speed** action is useful when you know the exact speed you want. Use it to move a fan to a specific level, like 25% for quiet airflow or 100% for fast cooling.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Set fan speed**.
7. Under **Percentage**, set the speed you want.
8. Select **Save**.

Options in the UI

{% options_ui %} Percentage: description: The speed to set, from 0 to 100 percent. required: true

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.set_percentage`. A basic example looks like this:

action

```
action: |
  action: fan.set_percentage
  target:
    entity_id: fan.office
  data:
    percentage: 40
```

This sets `fan.office` to 40% speed.

Options in YAML

{% options_yaml %} percentage: description: The speed to set, from 0 to 100 percent. required: true type: integer

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support speed control.
-  is allowed by the selector, but some devices may treat that as off or ignore it.
- To move the speed up or down by a step instead, use [Increase fan speed](#) or [Decrease fan speed](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: set the office fan to a quiet speed in the morning

Start the day with a light airflow that is less distracting during calls.

- **Trigger:** Time: 08:00
- **Action:** Set fan speed
- **Target:** Office fan
- **Percentage:** 35

YAML example for a quiet office fan

****automation****

```
automation: |
  alias: "Office fan to 35 percent"
  triggers:
    - trigger: time
      at: "08:00:00"
  actions:
    - action: fan.set_percentage
      target:
        entity_id: fan.office
      data:
        percentage: 35
```

Automation: raise the living room fan to full speed when the sun comes in

When the afternoon sun starts heating the room, you can push the fan to full speed for faster cooling.

- **Trigger:** Time: 15:00
- **Action:** Set fan speed
- **Target:** Living room fan
- **Percentage:** 100

YAML example for strong afternoon airflow

****automation****

```
automation: |
  alias: "Living room fan to full speed"
  triggers:
    - trigger: time
      at: "15:00:00"
  actions:
    - action: fan.set_percentage
      target:
        entity_id: fan.living_room
      data:
        percentage: 100
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Set Preset Mode

The **Set fan preset mode** action is useful when your fan offers named modes like low, medium, high, sleep, or auto. Use it when you want to switch between those built-in modes instead of setting a raw speed percentage.

Available preset modes come from the integration that provides the fan entity. For example, the ESPHome [Speed Fan](#) component provides **Low**, **Medium**, and **High** presets by default.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Set fan preset mode**.
7. Under **Preset mode**, select the mode you want to use.
8. Select **Save**.

Options in the UI

{% options_ui %} Preset mode: description: The preset mode to apply to the selected fan.
required: true

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.set_preset_mode`. A basic example looks like this:

action

```
action: |
  action: fan.set_preset_mode
  target:
    entity_id: fan.bedroom
  data:
    preset_mode: "sleep"
```

This sets `fan.bedroom` to the `sleep` preset mode.

Options in YAML

{% options_yaml %} preset_mode: description: The preset mode to apply to the selected fan.
required: true type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support preset modes.
- Available preset modes come from the fan integration, so names vary by device.
- To set a percentage instead of a named mode, use [Set fan speed](#).

Try it yourself

Ready to test this? Open **Developer tools** > **Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: set the bedroom fan to low at night

If your bedroom fan uses ESPHome Speed Fan presets, you can switch it to **Low** automatically at bedtime for quieter airflow.

- **Trigger:** Time: 22:30
- **Action:** Set fan preset mode
- **Target:** Bedroom fan
- **Preset mode:** low

YAML example for a bedtime low preset

****automation****

```
automation: |
  alias: "Bedroom fan low preset"
  triggers:
    - trigger: time
      at: "22:30:00"
  actions:
    - action: fan.set_preset_mode
      target:
        entity_id: fan.bedroom
      data:
        preset_mode: "low"
```

Automation: set the living room fan to high when you get home

When you arrive home on a warm day, you can switch an ESPHome Speed Fan to **High** for stronger airflow right away.

- **Trigger:** Zone: Person enters home zone
- **Action:** Set fan preset mode
- **Target:** Living room fan
- **Preset mode:** high

YAML example for an arrival high preset

****automation****

```
automation: |
  alias: "Living room fan high preset on arrival"
  triggers:
    - trigger: zone
      entity_id: person.alex
      zone: zone.home
      event: enter
  actions:
    - action: fan.set_preset_mode
      target:
        entity_id: fan.living_room
      data:
        preset_mode: "high"
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Toggle

The **Toggle fan** action is useful when you want one control to flip a fan between on and off. Use it when the current state can vary and you want a single action that handles both directions.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Toggle fan**.
7. Select **Save**.

Options in the UI

This action has no additional options beyond the target.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.toggle`. A basic example looks like this:

action

```
action: |
  action: fan.toggle
  target:
    entity_id: fan.guest_room
```

This flips `fan.guest_room` from off to on, or from on to off.

Options in YAML

This action has no additional YAML options beyond the target.

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- `fan.toggle` works with fan entities even if other fan features are not supported.
- Because it flips the current state, it is best when you do not need to know in advance whether the fan is on or off.
- If you need a specific result, use [Turn on fan](#) or [Turn off fan](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: toggle the guest room fan at bedtime

If the fan is usually off at night but sometimes already running, a toggle lets one automation handle both cases.

- **Trigger:** Time: 22:15
- **Action:** Toggle fan
- **Target:** Guest room fan

YAML example for a bedtime toggle

****automation****

```
automation: |
  alias: "Toggle guest room fan at bedtime"
  triggers:
    - trigger: time
      at: "22:15:00"
  actions:
    - action: fan.toggle
      target:
        entity_id: fan.guest_room
```

Automation: toggle the patio fan when the deck light turns on

If you use the deck in the evening, the patio fan can follow the deck light with a single action.

- **Trigger:** State: Deck light changes to on
- **Action:** Toggle fan
- **Target:** Patio fan

YAML example for a patio fan toggle

****automation****

```
automation: |
  alias: "Toggle patio fan with deck light"
  triggers:
    - trigger: state
      entity_id: light.deck
      to: "on"
  actions:
    - action: fan.toggle
      target:
        entity_id: fan.patio
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Turn Off

The **Turn off fan** action is useful when you want to stop airflow at a specific time or after another event happens. Use it to save energy, reduce noise, or end a cooling routine.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Turn off fan**.
7. Select **Save**.

Options in the UI

This action has no additional options beyond the target.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.turn_off`. A basic example looks like this:

action

```
action: |
  action: fan.turn_off
  target:
    entity_id: fan.kitchen
```

This turns off `fan.kitchen`.

Options in YAML

This action has no additional YAML options beyond the target.

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support turning off.
- If the fan is already off, the action does nothing.
- To switch between on and off with one action, use [Toggle fan](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn off the bedroom fan in the morning

If you use a fan overnight, this automation turns it off when the day starts.

- **Trigger:** Time: 07:00
- **Action:** Turn off fan
- **Target:** Bedroom fan

YAML example for a morning fan stop

****automation****

```
automation: |
  alias: "Turn off bedroom fan in the morning"
  triggers:
    - trigger: time
      at: "07:00:00"
  actions:
    - action: fan.turn_off
      target:
        entity_id: fan.bedroom
```

Automation: turn off the living room fan when everyone leaves

This avoids running the fan when no one is home.

- **Trigger:** Zone: Person leaves home zone
- **Condition:** Group family is away
- **Action:** Turn off fan
- **Target:** Living room fan

YAML example for turning the fan off when the home is empty

****automation****

```
automation: |
  alias: "Turn off living room fan when home is empty"
  triggers:
    - trigger: zone
      entity_id: person.alex
      zone: zone.home
      event: leave
  conditions:
    - condition: state
      entity_id: group.family
      state: not_home
  actions:
    - action: fan.turn_off
      target:
        entity_id: fan.living_room
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Fan.Turn On

The **Turn on fan** action is useful when you want to start airflow right away. You can simply turn the fan on, or turn it on with a specific speed or preset mode in the same step.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're creating an automation, add a trigger in the **When** section.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the fan you want to control. You can also select an area, a floor, a device, or a label.
6. From the actions shown for that target, select **Turn on fan**.
7. Optional: under **Percentage**, set the speed you want.
8. Optional: under **Preset mode**, select the mode you want.
9. Select **Save**.

Options in the UI

{% options_ui %} Percentage: description: The speed to set when the fan turns on. required: false
Preset mode: description: The preset mode to apply when the fan turns on. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `fan.turn_on`. A basic example looks like this:

action

```
action: |
  action: fan.turn_on
  target:
    entity_id: fan.bedroom
```

This turns on `fan.bedroom`.

Options in YAML

{% options_yaml %} percentage: description: The speed to set when the fan turns on. required: false type: integer preset_mode: description: The preset mode to apply when the fan turns on. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for fans that support turning on.
- The `percentage` field is available only for fans that support speed control.
- The `preset_mode` field is available only for fans that support preset modes.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the bedroom fan at bedtime

Start the fan automatically when you usually go to bed.

- **Trigger:** Time: 22:00
- **Action:** Turn on fan
- **Target:** Bedroom fan

YAML example for a bedtime fan

****automation****

```
automation: |
  alias: "Turn on bedroom fan at bedtime"
  triggers:
    - trigger: time
      at: "22:00:00"
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom
```

Automation: turn on the office fan at 60 percent in the afternoon

If your office warms up later in the day, you can start the fan at a moderate speed automatically.

- **Trigger:** Time: 14:00
- **Action:** Turn on fan
- **Target:** Office fan
- **Percentage:** 60

YAML example for an afternoon office fan

****automation****

```
automation: |
  alias: "Turn on office fan in the afternoon"
  triggers:
    - trigger: time
      at: "14:00:00"
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.office
      data:
        percentage: 60
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Html5.Dismiss Message

The **Dismiss message** action removes messages previously delivered via HTML5 Push Notifications.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To send a notification from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **HTML5 Push Notifications: Dismiss message**.
6. Under **Targets**, select the device to dismiss the message from (see [Targets](#)).
7. *Optional:* under **Tag**, enter the tag of the message you want to dismiss.
8. Select **Save**.

Options in the UI

{% options_ui %} Tag: description: The tag of the notifications to dismiss. If not specified, all notifications to the selected devices will be dismissed. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `html5.dismiss_message`. A basic example looks like this:

action

```
action: |
  action: html5.dismiss_message
  target:
    entity_id: notify.my_desktop
  data:
    tag: "message-group-1"
```

Options in YAML

{% options_yaml %} tag: description: > The tag of the notifications to dismiss. If not specified, all notifications to the selected devices will be dismissed. required: false type: string

[Include not found: actions/targets.md domain="notify"]

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: dismiss notification when motion is cleared

When motion in the backyard is cleared, a previously sent motion-detected notification is automatically dismissed.

- **Trigger:** Motion cleared
- **Target:** Backyard area
- **Action:** HTML5 Push Notifications: Dismiss message
- **Target:** My Desktop `notify.my_desktop`

automation

```
automation: |
  triggers:
    - trigger: motion.cleared
      target:
        area_id: backyard
  actions:
    - action: html5.dismiss_message
      data:
        tag: "motion-detected"
      target:
        entity_id: notify.my_desktop
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Html5.Send Message

The **Send message** action sends a notification to a browser or installed web app (PWA) registered with HTML5 Push Notifications.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To send a notification from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts do not need a trigger.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **HTML5 Push Notifications: Send message**.
6. Under **Target**, select the notification device to send the message to (see [Targets](#)).
7. Add a title for the message.
8. *Optional*: add a message, icon, action buttons, or other settings.
9. Select **Save**.

 **Note:** Keep in mind that support for the features described below can vary depending on the browser and platform you are using. Refer to the [MDN Notifications API documentation](#) for a detailed overview of compatibility across environments.

Options in the UI

{% options_ui %} Title: description: Title for your notification. required: true Message: description: The message body of the notification. required: false Icon: description: URL or relative path of an image to display as the main notification icon. Maximum size is 320px by 320px. required: false Badge: description: URL or relative path of a small image to replace the browser icon on mobile platforms. Maximum size is 96px by 96px. required: false Image: description: URL or relative path of a larger image to display in the main body of the notification. Experimental support; may not be displayed on all platforms. required: false Tag: description: The identifier of the notification. Sending a new notification with the same tag replaces the existing one. If not specified, a unique tag is generated for each notification. required: false Actions: description: Adds action buttons to the notification. When the user clicks a button, an event is sent back to Home Assistant. The number of actions supported may vary between platforms. (See [action button options](#)). required: false Text direction: description: The direction of the notification's text. Adopts the browser's language setting behavior by default. required: false Renotify: description: If enabled, the user is alerted again (sound/vibration) when a notification with the same tag replaces a previous one. required: false Silent: description: If enabled, the notification does not play sounds or trigger vibration, regardless of the device's settings. required: false Require interaction: description: If enabled, the notification remains active until the user clicks or dismisses it, rather than automatically closing after a few seconds. This provides the same behavior on desktop as on

mobile platforms. required: false Vibration pattern: description: A vibration pattern to run with the notification. An array of integers representing alternating periods of vibration and silence in milliseconds. For example, `[200, 100, 200]` vibrates for 200ms, pauses for 100ms, then vibrates for another 200ms. required: false Language: description: The language of the notification's content. required: false Timestamp: description: The timestamp of the notification. By default, uses the time when the notification is sent. required: false Time to live: description: Specifies how long the push service retains the message if the user's browser or device is offline. After this period, the notification expires. A value of `0` means the notification is discarded immediately if the target is not connected. Defaults to 1 day. required: false Urgency: description: Whether the push service tries to deliver the notification immediately or defers it in accordance with the user's power-saving preferences. required: false Extra data: description: Additional custom key-value pairs to include in the payload of the push message. This can be used to include extra information that can be accessed in the notification click event. required: false

Action button options

Action buttons are configured in the **Actions** field. Each item supports:

{% options_ui %} Action identifier: description: The identifier of the action. This is sent back to Home Assistant when the user clicks the button. required: true Title: description: The label of the button displayed to the user. required: true Icon: description: URL or relative path of an image displayed as the icon for this button. Maximum size is 128px by 128px. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `html5.send_message`. A basic example looks like this:

action

```
action: |
  action: html5.send_message
  data:
    title: "Reminder"
    message: "Have you considered frogs?"
    icon: /static/icons/favicon-192x192.png
    badge: /static/images/notification-badge.png
    tag: message-group-1
    actions:
      - action: test-action
        title:  Click here!
    require_interaction: true
    vibrate:
      - 125
      - 75
      - 125
      - 275
      - 200
      - 275
      - 125
      - 75
      - 125
      - 275
      - 200
      - 600
      - 200
      - 600
  target:
    entity_id: notify.my_desktop
```

 **Note:** When using a relative path for an image or icon URL, the path is resolved relative to the base URL of your Home Assistant instance.

Options in YAML

{% options_yaml %} title: description: > Title for your notification message. required: true type: string message: description: > The message body of the notification. required: false type: string icon: description: > URL or relative path of an image to display as the main icon in the notification. Maximum size is 320px by 320px. required: false type: string badge: description: > URL or relative path of a small image to replace the browser icon on mobile platforms. Maximum size is 96px by 96px. required: false type: string image: description: > URL or relative path of a larger image to

display in the main body of the notification. Experimental support; may not be displayed on all platforms. required: false type: string tag: description: > The identifier of the notification. Sending a new notification with the same tag replaces the existing one. If not specified, a unique tag is generated for each notification. required: false type: string actions: description: > Adds action buttons to the notification. When the user clicks a button, an event is sent back to Home Assistant. The number of actions supported may vary between platforms. (See [action button options](#)). required: false type: list dir: description: > The direction of the notification's text. Adopts the browser's language setting behavior by default. required: false type: string renotify: description: > If enabled, the user is alerted again (sound/vibration) when a notification with the same tag replaces a previous one. required: false type: boolean default: false silent: description: > If enabled, the notification does not play sounds or trigger vibration, regardless of the device's settings. required: false type: boolean default: false require_interaction: description: > If enabled, the notification remains active until the user clicks or dismisses it, rather than automatically closing after a few seconds. This provides the same behavior on desktop as on mobile platforms. required: false type: boolean default: false vibrate: description: > A vibration pattern to run with the notification. An array of integers representing alternating periods of vibration and silence in milliseconds. For example, `[200, 100, 200]` vibrates for 200ms, pauses for 100ms, then vibrates for another 200ms. required: false type: list lang: description: > The language of the notification's content. required: false type: string timestamp: description: > The timestamp of the notification. By default, uses the time when the notification is sent. required: false type: string ttl: description: > Specifies how long the push service retains the message if the user's browser or device is offline. After this period, the notification expires. A value of `0` means the notification is discarded immediately if the target is not connected. Defaults to 1 day. required: false type: map urgency: description: > Whether the push service tries to deliver the notification immediately or defers it in accordance with the user's power-saving preferences. required: false type: string data: description: > Additional custom key-value pairs to include in the payload of the push message. This can be used to include extra information that can be accessed in the notification click event. required: false type: map

Action button options in YAML

Action buttons are configured in the `actions` field. Each item supports:

```
{% options_yaml %} action: description: > The identifier of the action. This is sent back to Home Assistant when the user clicks the button. required: true type: string title: description: > The label shown on the button. required: true type: string icon: description: > Optional URL or relative path to an image for the button. Maximum size is 128px by 128px. required: false type: string
```

```
[Include not found: actions/targets.md domain="notify"]
```

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: send a doorbell-rang notification with an open-door action button

If the doorbell rings, receive an actionable notification with a camera snapshot. Opens the door when you tap the "Open door" action button.

- **Trigger:** Doorbell rang (`event.doorbell`)
- **Action:** Send message
- **Target:** My desktop (`notify.my_desktop`)

automation

```
automation: |
  alias: "Send doorbell-rang notification to my desktop"
  triggers:
    - trigger: doorbell.rang
      target:
        entity_id: event.doorbell
  actions:
    - action: html5.send_message
      target:
        entity_id: notify.my_desktop
      data:
        title: 🚪 The doorbell rang.
        message: Someone is at the entrance door.
        icon: https://homeassistant.example.com/www/entrancecamera_snapshot.jpg
        actions:
          - action: open-door
            title: Open door
        require_interaction: true
      ttl:
        minutes: 5
      urgency: high
```

- **Trigger:** Event
- **Condition:** Template
- **Action:** Open lock
- **Target:** Entrance door

automation

```
automation: |
  alias: "Open entrance door when open-door action button is tapped"
  triggers:
    - trigger: event.received
      target:
        entity_id: event.pc
      options:
        event_type:
          - clicked
  conditions:
    - condition: template
      value_template: "{{ trigger.to_state.attributes.action == 'open-door' }}"
  actions:
    - action: lock.open
      target:
        entity_id: lock.entrance_door
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Light.Toggle

The **Toggle light** action flips a light to the opposite state. If the light is off, it turns on. If it's on, it turns off. This is handy for a single button or a motion sensor that should cycle a light without you having to know what state it's in.

When **Toggle light** turns a light on, you can also set the brightness, color, color temperature, or a transition at the same time, just like you would with **Turn on light**. Those options are ignored when **Toggle light** turns the light off.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To toggle a light from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **Light: Toggle light**.
6. Under **Targets**, choose what you want to toggle:
 - To toggle a specific light, select the entity.
 - To toggle every light in a room, select an area.
 - To toggle every light on a floor, select a floor.
 - To toggle lights sharing a tag, select a label.
7. *Optional:* under **Advanced options**, set the brightness, color, color temperature, or transition that should apply when the light turns on.
8. Select **Save**.

Options in the UI

{% options_ui %} Transition: description: How long, in seconds, the light takes to reach the new state. Use this for a smooth fade instead of switching instantly. required: false Brightness: description: How bright the light should be when the toggle turns it on, on a scale from 0 (off) to 255 (brightest). required: false Brightness percentage: description: How bright the light should be when the toggle turns it on, from 0% (off) to 100% (brightest). required: false Color: description: A color for the light when the toggle turns it on. You can pick a named color, a color from the color wheel, or a specific value in RGB, hue/sat, or XY format. required: false Color temperature: description: Warm or cool white, measured in Kelvin, for when the toggle turns the light on. Lower is warmer (more yellow), higher is cooler (more blue). required: false Effect: description: A light effect to play when the toggle turns the light on, for example a color loop or a candle flicker. Available effects depend on the light. required: false Flash: description: Ask the light to flash briefly, either short or long. Useful as a visual notification. required: false Profile: description: The name of a light profile to apply when the toggle turns the light on. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `light.toggle`. A basic example looks like this:

action

```
action: |
  action: light.toggle
  target:
    entity_id: light.hallway
```

This flips `light.hallway` to the opposite state.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} transition: description: > Duration, in seconds, it takes to reach the next state. Use this to fade smoothly instead of switching instantly. required: false type: integer brightness: description: > Number indicating brightness when the light turns on, where 0 is off, 1 is the minimum, and 255 is the maximum. required: false type: integer brightness_pct: description: > Percentage of full brightness when the light turns on, where 0 is off, 1 is the minimum, and 100 is the maximum. required: false type: integer color_name: description: > A human-readable color name to apply when the light turns on, for example `warm_white`, `tomato`, or `cornflowerblue`. required: false type: string color_temp_kelvin: description: > Color temperature in Kelvin to apply when the light turns on. Lower values are warmer (more yellow), higher values are cooler (more blue). required: false type: integer rgb_color: description: > The color in RGB format to apply when the light turns on. A list of three integers between 0 and 255 representing red, green, and blue. required: false type: list hs_color: description: > Color in hue/sat format to apply when the light turns on. A list of two integers, where hue is 0 to 360 and saturation is 0 to 100. required: false type: list xy_color: description: > Color in XY format to apply when the light turns on. A list of two decimal numbers between 0 and 1. required: false type: list effect: description: > Light effect to apply when the light turns on. Available effects depend on the specific light. required: false type: string flash: description: > Tell the light to flash briefly. Accepts either `short` or `long`. required: false type: string profile: description: > Name of a light profile to apply when the light turns on. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Toggle light** action works on any light entity, such as bulbs, groups, fixtures, or strips.
- When **Toggle light** turns a light on, any brightness, color, or transition options you set are applied. When it turns a light off, those options are ignored.
- If you use **Toggle light** on a group of lights, each light in the group flips its own state. Some lights may turn on while others turn off. To treat a group as one unit, create a dedicated [light group](#) first.
- If you already know the state you want, use [Turn on a light](#) or [Turn off a light](#) instead. They make the intent clearer in the automation's name.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: flip the hallway light with a single button press

Wire a physical button or a dashboard tile to a single toggle action so it acts like a light switch.

- **Action:** Light: Toggle light
- **Target:** Hallway light

YAML example for a one-button hallway toggle

****action****

```
action: |
  action: light.toggle
  target:
    entity_id: light.hallway
```

Action: toggle the kitchen light to a warm white tone

When the toggle turns the kitchen light on, have it come up dim and warm instead of at full blast.

- **Action:** Light: Toggle light
- **Target:** Kitchen light
- **Brightness percentage:** 40
- **Color:** warm_white

YAML example for a warm toggle

****action****

```

action: |
  action: light.toggle
  target:
    entity_id: light.kitchen
  data:
    brightness_pct: 40
    color_name: warm_white

```

Automation: toggle the hallway light with a physical button

Keep a smart button on the wall and flip the hallway light whenever you press it. A great way to add a light switch where there isn't one.

- **Trigger:** Device: Button pressed
- **Action:** Light: Toggle light
- **Target:** Hallway light

YAML example for a button-driven hallway toggle

****automation****

```

automation: |
  alias: "Hallway button toggle"
  triggers:
    - trigger: state
      entity_id: event.hallway_button
    - trigger: event
      event_type: zha_event
      event_data:
        device_ieee: "00:11:22:33:44:55:66:77"
        command: "single"
  actions:
    - action: light.toggle
      target:
        entity_id: light.hallway

```

Automation: toggle the bathroom light on motion

When the bathroom motion sensor fires, flip the light. The next motion event flips it back, so you can also use the same automation to cut the light when you leave.

- **Trigger:** Motion sensor detects motion
- **Action:** Light: Toggle light
- **Target:** Bathroom light

YAML example for toggling the bathroom light on motion

****automation****

```

automation: |
  alias: "Toggle bathroom light on motion"
  triggers:
    - trigger: state
      entity_id: binary_sensor.bathroom_motion
      to: "on"
  actions:
    - action: light.toggle
      target:
        entity_id: light.bathroom

```

Automation: toggle the pantry light when the door opens

When the pantry door opens, flip the light. Close the door and the same trigger flips it back off the next time around.

- **Trigger:** Door sensor opens
- **Action:** Light: Toggle light
- **Target:** Pantry light

YAML example for a pantry door toggle

****automation****

```

automation: |
  alias: "Toggle pantry light on door"
  triggers:
    - trigger: state
      entity_id: binary_sensor.pantry_door
      to: "on"
  actions:
    - action: light.toggle
      target:
        entity_id: light.pantry

```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Light.Turn Off

The **Turn off light** action turns a light off. You can switch it off instantly, add a transition so it fades out smoothly, or ask it to flash briefly before going dark.

This action works with any light entity in Home Assistant, whether it's a single bulb, a group of lights, or a smart fixture. If the light is already off, calling the action does nothing.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To turn a light off from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **Light: Turn off light**.
6. Under **Targets**, choose what you want to turn off:
 - To turn off a specific light, select the entity.
 - To turn off every light in a room, select an area.
 - To turn off every light on a floor, select a floor.
 - To turn off lights sharing a tag, select a label.
7. *Optional:* under **Advanced options**, set a transition or a flash effect.
8. Select **Save**.

Options in the UI

{% options_ui %} Transition: description: How long, in seconds, it takes to fade the light out. Use this for a smooth fade instead of switching off instantly. required: false Flash: description: Ask the light to flash briefly before it turns off, either short or long. Useful as a visual notification. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `light.turn_off`. A basic example looks like this:

action

```
action: |
  action: light.turn_off
  target:
    entity_id: light.kitchen
```

This turns off `light.kitchen`.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} transition: description: > Duration, in seconds, it takes to fade the light out. Use this to dim down smoothly instead of switching off instantly. type: integer flash: description: > Tell the light to flash briefly before turning off. Accepts either `short` or `long`. type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Turn off light** action works on any light entity, such as bulbs, groups, fixtures, or strips.
- If the light is already off, calling this action does nothing.
- Not every light supports a transition or a flash. Home Assistant quietly skips options the device can't handle.
- To reverse this action, use [Turn on a light](#). To flip a light between on and off with a single call, use [Toggle a light](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: fade out gently at the end of movie night

Fade the bedroom light out over five seconds, which is a much nicer way to end a movie than an instant off.

- **Action:** Light: Turn off light
- **Target:** Bedroom light
- **Transition:** 5 seconds

YAML example for a gentle fade-out

****action****

```
action: |
  action: light.turn_off
  target:
    entity_id: light.bedroom
  data:
    transition: 5
```

Action: turn off every light on a floor

Target a floor instead of a specific entity and Home Assistant resolves it to every light on that floor.

- **Action:** Light: Turn off light
- **Target:** Ground floor

YAML example for turning off every light on a floor

****action****

```
action: |
  action: light.turn_off
  target:
    floor_id: ground_floor
```

Automation: porch light off at sunrise

Turn the porch light off automatically as the sun comes up. No more wasted electricity after you've already gone to bed or left for work.

- **Trigger:** Sun: Above horizon
- **Action:** Light: Turn off light
- **Target:** Porch light

YAML example for a sunrise porch light off

****automation****

```
automation: |
  alias: "Porch light off at sunrise"
  triggers:
    - trigger: sun
      event: sunrise
  actions:
    - action: light.turn_off
      target:
        entity_id: light.porch
```

Automation: turn every light off when the house is empty

When the last person leaves home, turn off every light in the house. A simple way to save energy without having to think about it.

- **Trigger:** Zone: Everyone leaves home
- **Action:** Light: Turn off light
- **Target:** All lights (by label)

YAML example for turning off all lights when nobody is home

****automation****

```
automation: |
  alias: "Lights off when everyone leaves"
  triggers:
    - trigger: zone
      entity_id: person.paulus
      zone: zone.home
      event: leave
  conditions:
    - condition: state
      entity_id: group.family
      state: not_home
  actions:
    - action: light.turn_off
      target:
        label_id: all_lights
```

Automation: bedtime fade-out

At 11 in the evening, fade every light in the living room out over ten seconds. A calmer way to end the day than flipping a switch.

- **Trigger:** Time: 23:00
- **Action:** Light: Turn off light
- **Target:** Living room
- **Transition:** 10 seconds

YAML example for a bedtime fade-out

****automation****

```
automation: |
  alias: "Bedtime fade-out"
  triggers:
    - trigger: time
      at: "23:00:00"
  actions:
    - action: light.turn_off
      target:
        area_id: living_room
      data:
        transition: 10
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Light.Turn On

The **Turn on light** action turns a light on. You can simply switch it on, or go further and set the brightness, color, color temperature, or an effect at the same time.

This action works with any light entity in Home Assistant, whether it's a single bulb, a group of lights, or a smart fixture. If the light is already on, calling the action updates its attributes (such as brightness or color) without flashing off first.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To turn a light on from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **Light: Turn on light**.
6. Under **Targets**, choose what you want to turn on:
 - To turn on a specific light, select the entity.
 - To turn on every light in a room, select an area.
 - To turn on every light on a floor, select a floor.
 - To turn on lights sharing a tag, select a label.
7. *Optional:* under **Advanced options**, set the brightness, color, color temperature, or transition.
8. Select **Save**.

Options in the UI

{% options_ui %} Transition: description: How long, in seconds, it takes to get to the next state. Use this for a smooth fade instead of switching instantly. required: false Brightness: description: How bright the light should be, on a scale from 0 (off) to 255 (brightest). required: false Brightness percentage: description: How bright the light should be, from 0% (off) to 100% (brightest). required: false Color: description: A color for the light. You can pick a named color, a color from the color wheel, or a specific value in RGB, hue/sat, or XY format. required: false Color temperature: description: Warm or cool white, measured in Kelvin. Lower is warmer (more yellow), higher is cooler (more blue). required: false Effect: description: A light effect to play, for example a color loop or a candle flicker. Available effects depend on the light. required: false Flash: description: Ask the light to flash briefly, either short or long. Useful as a visual notification. required: false Profile: description: The name of a light profile to apply. Profiles bundle a brightness and color together under a single name. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `light.turn_on`. A basic example looks like this:

action

```
action: |
  action: light.turn_on
  target:
    entity_id: light.kitchen
```

This turns on `light.kitchen` at its previous brightness and color.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} transition: description: > Duration, in seconds, it takes to get to the next state. Use this to fade smoothly instead of switching instantly. required: false type: integer brightness: description: > Number indicating brightness, where 0 turns the light off, 1 is the minimum brightness, and 255 is the maximum brightness. required: false type: integer brightness_pct: description: > Number indicating the percentage of full brightness, where 0 turns the light off, 1 is the minimum brightness, and 100 is the maximum brightness. required: false type: integer color_name: description: > A human-readable color name, for example `warm_white`, `tomato`, or `cornflowerblue`. required: false type: string color_temp_kelvin: description: > Color temperature in Kelvin. Lower values are warmer (more yellow), higher values are cooler (more blue). required: false type: integer rgb_color: description: > The color in RGB format. A list of three integers between 0 and 255 representing the values of red, green, and blue. required: false type: list hs_color: description: > Color in hue/sat format. A list of two integers, where hue is 0 to 360 and saturation is 0 to 100. required: false type: list xy_color: description: > Color in XY format. A list of two decimal numbers between 0 and 1. required: false type: list effect: description: > Light effect to apply. Available effects depend on the specific light. required: false type: string flash: description: > Tell the light to flash briefly. Accepts either `short` or `long`. required: false type: string profile: description: > Name of a light profile to apply. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The **Turn on light** action works on any light entity, such as bulbs, groups, fixtures, or strips.
- If the light is already on, the call updates its attributes (brightness, color) without flashing off first.
- Not every light supports every field. A bulb that only dims ignores color fields, and a color-only light ignores color temperature. Home Assistant quietly skips fields the device can't handle.
- To reverse this action, use [Turn off a light](#). To flip a light between on and off with a single call, use [Toggle a light](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: set a cozy warm white tone

When you start winding down in the evening, dim the kitchen light to a warm white tone.

- **Action:** Light: Turn on light
- **Target:** Kitchen light
- **Brightness percentage:** 80
- **Color:** warm_white

YAML example for a cozy warm white scene

****action****

```
action: |
  action: light.turn_on
  target:
    entity_id: light.kitchen
  data:
    brightness_pct: 80
    color_name: warm_white
```

Action: fade the bedroom light up over ten seconds

A long transition is a gentle way to wake up. Instead of snapping the light on, fade it up slowly.

- **Action:** Light: Turn on light
- **Target:** Bedroom light
- **Brightness percentage:** 100
- **Transition:** 10 seconds

YAML example for a sunrise-style fade-in

****action****


```

action: |
  action: light.turn_on
  target:
    entity_id: light.bedroom
  data:
    brightness_pct: 100
    transition: 10

```

Action: turn on every light in a room

Target an area instead of a specific entity and Home Assistant resolves it to every light inside the room.

- **Action:** Light: Turn on light
- **Target:** Living room
- **Brightness percentage:** 60

YAML example for turning on every light in a room

****action****

```

action: |
  action: light.turn_on
  target:
    area_id: living_room
  data:
    brightness_pct: 60

```

Automation: porch light at sunset

Greet the evening by turning the porch light on at a warm white tone as the sun drops below the horizon. Nice and welcoming without running the light at full power.

- **Trigger:** Sun: Below horizon
- **Action:** Light: Turn on light
- **Target:** Porch light
- **Brightness percentage:** 60
- **Color:** warm_white

YAML example for a sunset porch light automation

****automation****

```

automation: |
  alias: "Porch light at sunset"
  triggers:
    - trigger: sun
      event: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id: light.porch
      data:
        brightness_pct: 60
        color_name: warm_white

```

Automation: gentle weekday wake-up

Fade the bedroom light up over ten seconds at 7 in the morning on weekdays. A kinder way to start the day than a sudden flick.

- **Trigger:** Time: 07:00
- **Condition:** Day of the week is Monday to Friday
- **Action:** Light: Turn on light
- **Target:** Bedroom light
- **Brightness percentage:** 100
- **Transition:** 10 seconds

YAML example for a weekday sunrise alarm

****automation****

```

automation: |
  alias: "Gentle weekday wake-up"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: time
      weekday:
        - mon
        - tue
        - wed
        - thu
        - fri
  actions:
    - action: light.turn_on
      target:
        entity_id: light.bedroom
      data:
        brightness_pct: 100
        transition: 10

```

Automation: welcome home lighting

When you arrive home after dark, turn on every light in the living room at a comfortable level so the house greets you instead of leaving you fumbling for switches.

- **Trigger:** Person: Paulus changes to home
- **Condition:** Sun is below horizon
- **Action:** Light: Turn on light
- **Target:** Living room
- **Brightness percentage:** 60

YAML example for welcome home lighting

****automation****

```
automation: |
  alias: "Welcome home lighting"
  triggers:
    - trigger: state
      entity_id: person.paulus
      to: home
  conditions:
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_on
      target:
        area_id: living_room
      data:
        brightness_pct: 60
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Lock.Lock

The **Lock lock** action lets you secure a door from an automation or script. Use it when you want Home Assistant to lock a door after a routine, like when everyone leaves, or after a door has been left unlocked for too long.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Lock lock**.
7. *Optional:* Enter **Code** if your lock requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to use when locking the door, if your lock requires one. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `lock.lock`. A basic example looks like this:

action

```
action: |
  action: lock.lock
  target:
    entity_id: lock.front_door
```

This locks `lock.front_door`.

If your lock requires a code, include it in the `data` section:

action

```
action: |
  action: lock.lock
  target:
    entity_id: lock.front_door
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to use when locking the door, if your lock requires one. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Some locks require a code, and others do not.
- A lock may already have a default code configured by its integration.
- To do the opposite action, use [Unlock](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: lock the front door every night

If you want a simple nightly routine, lock the front door at the same time each evening. This automation locks the door at 11 PM.

- **Trigger:** Time: 23:00
- **Action:** Lock lock
- **Target:** Front door lock

YAML example for locking the front door every night

****automation****

```
automation: |
  alias: "Lock the front door every night"
  triggers:
    - trigger: time
      at: "23:00:00"
  actions:
    - action: lock.lock
      target:
        entity_id: lock.front_door
```

Automation: lock the back door after it stays unlocked for 10 minutes

If a back door is often left unlocked, you can have Home Assistant secure it for you. This automation locks the back door after it has stayed unlocked for 10 minutes.

- **Trigger:** Lock unlocked
- **Target:** Back door lock
- **Trigger when:** Each
- **For at least:** 00:10:00
- **Action:** Lock lock

YAML example for locking a door after it stays unlocked

****automation****

```
automation: |
  alias: "Lock the back door after 10 minutes"
  triggers:
    - trigger: lock.unlocked
      target:
        entity_id: lock.back_door
      options:
        behavior: any
        for: "00:10:00"
  actions:
    - action: lock.lock
      target:
        entity_id: lock.back_door
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Lock.Open

The **Open lock** action lets you unlatch a supported lock from an automation or script. Use it when you want Home Assistant to open a door for a short, specific moment, like letting someone in after you verify who is there.

The difference between **Open lock** and [Unlock lock](#) is that **Open** unlatches the door on locks that support that feature, while **Unlock** only changes the lock to the unlocked state. If you want the door ready to push open right away, use **Open**. If you only want to unlock the door, use [Unlock](#). The difference between **Open lock** and [Unlock lock](#) is that **Open lock** unlatches the door on locks that support that feature, while **Unlock lock** only changes the lock to the unlocked state. If you want the door ready to push open right away, use **Open lock**. If you only want to unlock the door, use [Unlock lock](#).

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Open lock**.
7. *Optional:* Enter **Code** if your lock requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to use when opening the lock, if your lock requires one. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `lock.open`. A basic example looks like this:

action

```
action: |
  action: lock.open
  target:
    entity_id: lock.front_door
```

This unlatches `lock.front_door`.

If your lock requires a code, include it in the `data` section:

action

```
action: |
  action: lock.open
  target:
    entity_id: lock.front_door
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to use when opening the lock, if your lock requires one. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action is available only for locks that support opening or unlatching.
- Some locks require a code, and others do not.
- A lock may already have a default code configured by its integration.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: open the front gate when you arrive home

If you have a lockable gate that supports opening, Home Assistant can unlatch it when you arrive. This automation opens the front gate when your person entity changes to home.

- **Trigger:** Person changes to home
- **Action:** Open lock
- **Target:** Front gate lock

YAML example for opening the front gate on arrival

****automation****

```
automation: |
  alias: "Open the front gate on arrival"
  triggers:
    - trigger: state
      entity_id: person.alex
      to: home
  actions:
    - action: lock.open
      target:
        entity_id: lock.front_gate
```

Automation: open the building door after a button press

If you use a user-created helper to expose a dashboard button, you can use that helper to open a supported door. This automation opens the building door when the helper is turned on. Create the helper separately before using this example.

- **Trigger:** User-created helper turns on
- **Action:** Open lock
- **Target:** Building door lock

YAML example for opening a door from a helper

****automation****

```
automation: |
  alias: "Open the building door from a helper"
  triggers:
    - trigger: state
      entity_id: input_boolean.open_building_door
      to: "on"
  actions:
    - action: lock.open
      target:
        entity_id: lock.building_door
    - action: input_boolean.turn_off
      target:
        entity_id: input_boolean.open_building_door
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Lock.Unlock

The **Unlock lock** action lets you release a lock from an automation or script. Use it when you want Home Assistant to let someone in, prepare for an arrival, or remove one step from an emergency exit path.

The difference between **Unlock lock** and [Open lock](#) is that **Unlock lock** only changes the lock to the unlocked state, while **Open lock** unlatches the door on locks that support that feature. If you want to unlock the door but not unlatch it, use **Unlock lock**. If you want the door ready to push open right away, use [Open lock](#).

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. Select what you want to control. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
6. From the actions shown for that target, select **Unlock lock**.
7. *Optional:* Enter **Code** if your lock requires one.
8. Select **Save**.

Options in the UI

{% options_ui %} Code: description: The code to use when unlocking the door, if your lock requires one. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `lock.unlock`. A basic example looks like this:

action

```
action: |
  action: lock.unlock
  target:
    entity_id: lock.front_door
```

This unlocks `lock.front_door`.

If your lock requires a code, include it in the `data` section:

action

```
action: |
  action: lock.unlock
  target:
    entity_id: lock.front_door
  data:
    code: "1234"
```

Options in YAML

{% options_yaml %} code: description: > The code to use when unlocking the door, if your lock requires one. required: false type: string

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Some locks require a code, and others do not.
- A lock may already have a default code configured by its integration.
- To release a latch on a supported lock instead of only unlocking it, use [Open](#).

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: unlock the front door when you arrive home

If you want a smoother arrival, Home Assistant can unlock the front door when you get home. This automation unlocks the door when your person entity changes to home.

- **Trigger:** Person changes to home
- **Action:** Unlock lock
- **Target:** Front door lock

YAML example for unlocking the front door on arrival

****automation****

```
automation: |
  alias: "Unlock the front door on arrival"
  triggers:
    - trigger: state
      entity_id: person.alex
      to: home
  actions:
    - action: lock.unlock
      target:
        entity_id: lock.front_door
```

Automation: unlock the front door during a smoke alarm

If you want to remove one step during an emergency, Home Assistant can unlock the front door when smoke is detected. This automation unlocks the door as soon as a smoke sensor reports smoke.

- **Trigger:** Smoke detected
- **Action:** Unlock lock
- **Target:** Front door lock

YAML example for unlocking a door during a smoke alarm

****automation****


```
automation: |
  alias: "Unlock the front door when smoke is detected"
  triggers:
    - trigger: state
      entity_id: binary_sensor.hall_smoke
      to: "on"
  actions:
    - action: lock.unlock
      target:
        entity_id: lock.front_door
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Ntfy.Clear

The **Dismiss notification** action marks a previously sent message in a **ntfy** topic as read without deleting it. This is useful when the notification no longer requires attention but should remain available for later review or reference.

To dismiss a notification, you must provide its message ID or sequence ID. The sequence ID can be specified when sending a notification.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To send a notification from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **ntfy: Dismiss notification**.
6. Under **Targets**, select the topic of the notification you want to dismiss (see [Targets](#)).
7. Under **Sequence ID**, enter the sequence ID or message ID of the notification you want to dismiss.
8. Select **Save**.

Options in the UI

{% options_ui %} Sequence ID: description: The sequence ID or message ID of the notification to dismiss. required: true

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `ntfy.clear`. A basic example looks like this:

action

```
action: |
  action: ntfy.clear
  target:
    entity_id: notify.mytopic
  data:
    sequence_id: "motion-detected"
```

Options in YAML

{% options_yaml %} sequence_id: description: > The sequence ID or message ID of the notification to dismiss. required: true type: string

[Include not found: actions/targets.md domain="notify"]

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: dismiss notification when motion is cleared

When motion in the backyard is cleared, a previously sent motion-detected notification is automatically dismissed. The notification remains available for later review.

- **Trigger:** Motion cleared
- **Target:** Backyard area
- **Action:** ntfy: Dismiss notification
- **Target:** ntfy topic `mytopic`

YAML example for dismissing a notification when motion is cleared

****automation****

```
automation: |
  triggers:
    - trigger: motion.cleared
      target:
        area_id: backyard
  actions:
    - action: ntfy.clear
      data:
        sequence_id: "motion-detected"
      target:
        entity_id: notify.mytopic
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Ntfy.Delete

The **Delete notification** action deletes a notification from a ntfy topic.

To delete a notification, you must provide its message ID or sequence ID. The sequence ID can be specified when sending a notification.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **ntfy: Delete notification**.
6. Under **Targets**, select the topic of the notification you want to delete (see [Targets](#)).
7. Under **Sequence ID**, enter the sequence ID or message ID of the notification you want to delete.
8. Select **Save**.

Options in the UI

{% options_ui %} Sequence ID: description: The sequence ID or message ID of the notification to delete. required: true

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `ntfy.delete`. A basic example looks like this:

action

```
action: |
  action: ntfy.delete
  target:
    entity_id: notify.mytopic
  data:
    sequence_id: "motion-detected"
```

Options in YAML

{% options_yaml %} sequence_id: description: > The sequence ID or message ID of the notification to delete. required: true type: string

[Include not found: actions/targets.md domain="notify"]

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: delete notification when motion is cleared

When motion in the backyard is cleared a previously sent motion-detected notification is deleted.

- **Trigger:** Motion cleared
- **Target:** Backyard area
- **Action:** ntfy: Delete notification
- **Target:** ntfy topic `mytopic`

YAML example for deleting a notification when motion is cleared

****automation****

```
automation: |
  triggers:
    - trigger: motion.cleared
      target:
        area_id: backyard
  actions:
    - action: ntfy.delete
      data:
        sequence_id: "motion-detected"
      target:
        entity_id: notify.mytopic
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Ntfy.Publish

The **Publish notification** action publishes a notification message to a **ntfy** topic.

With the **Publish notification** action you can take full advantage of the **ntfy** service's capabilities. You can customize the message content and appearance with priority, links, attachments, tags, emojis, and more.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To send a notification from an automation or a script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create automation > Create new automation**.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger. They run when something else calls them.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **ntfy: Publish notification**.
6. Under **Targets**, select the topics you want to notify (see [Targets](#)).
7. *Optional*: Customize message priority, incorporate emojis, or add interactive elements like URLs, attachments, and action buttons.
8. Select **Save**.

Options in the UI

{% options_ui %} Title: description: Title for your notification message. required: false Message: description: Your notification message. Defaults to the string `triggered` if no value is provided. required: false Format as Markdown: description: Enable Markdown formatting for the message body. See the [Markdown guide](#) for syntax details. required: false Tags/Emojis: description: Add tags or emojis to the notification. Emojis (using shortcodes like `smile`) will appear in the notification title or message. Other tags will be displayed below the notification content. required: false Message priority: description: All messages have a priority that defines how urgently your phone notifies you, depending on the configured vibration patterns, notification sounds, and visibility in the notification drawer or pop-over. required: false Click URL: description: URL that is opened when the notification is clicked. required: false Delay delivery: description: Set a delay for message delivery. Minimum delay is 10 seconds, maximum is 3 days. required: false Attachment URL: description: Attach images or other files by URL. required: false Attach local file: description: Attach images or other files by uploading from a local file, camera, or image media source. When selecting a camera entity, a snapshot of the current view will be captured and attached to the notification. required: false Attachment filename: description: Specify a custom filename for the attachment, including the file extension (for example, `snapshot.jpg`). If not provided, the filename defaults to `attachment` (for example, `attachment.jpg`). required: false Forward to email: description: Specify the address to forward the notification to, for example `mail@example.com`. required: false Phone call: description: Phone number to call and read the message out loud using text-to-speech. Requires ntfy Pro and prior phone number verification. required: false Icon URL: description: Include an icon that will appear next to the text of the notification. Only JPEG and PNG images are supported. required: false Action buttons: description: Up to three actions (**Open website/app**, **Send Android broadcast**, **Send HTTP request**, or **Copy to clipboard**) can be

added as buttons below the notification. Actions are executed when the corresponding button is tapped or clicked. required: false Sequence ID: description: Enter a message or sequence ID to update an existing notification, or specify a sequence ID to reference later when updating, clearing (mark as read and dismiss), or deleting a notification. required: false

 **Note:** All parameters are optional. If **message** is left empty, the notification will use the default text: `triggered`. If **priority** is not specified, the default priority (3) will be used.

 **Tip:** Check out the [emoji reference](#) for a full list of supported emoji shortcodes.

Action button options

Open a website or app

{% options_ui %} Action type: description: Select **Open website/app** to open a website or app when the button is clicked or tapped. required: true Label: description: Label of the action button in the notification. required: true URL: description: URL to open when action is tapped. required: true Clear: description: Clear notification after action button is tapped. required: false

Send HTTP request

{% options_ui %} Action type: description: Select **Send HTTP request** to send an HTTP request when the button is clicked or tapped. required: true Label: description: Label of the action button in the notification. required: true URL: description: URL to which the HTTP request will be sent. required: true HTTP method: description: HTTP method to use for request, default is `POST`. required: false HTTP headers: description: HTTP headers to pass in request (key-value pairs). required: false HTTP body: description: Payload to send in the HTTP body. required: false Clear: description: Clear notification after action button is tapped. required: false

Send Android broadcast

{% options_ui %} Action type: description: Select **Send Android broadcast** to send an Android broadcast intent when the button is clicked or tapped. required: true Label: description: Label of the action button in the notification. required: true Intent: description: Android intent name. Defaults to `io.heckel.ntfy.USER_ACTION`. required: false Intent extras: description: Android intent extras (key-value pairs). required: false Clear: description: Clear notification after action button is tapped. required: false

Copy to clipboard

{% options_ui %} Action type: description: Select **Copy to clipboard** to copy a given value to the clipboard when the button is clicked or tapped. required: true Label: description: Label of the action button in the notification. required: true Value: description: Value to copy to the clipboard. required: true Clear: description: Clear notification after action button is tapped. required: false

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `ntfy.publish`. A basic example looks like this:

action

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
```

This sends a notification to the topic `mytopic` with the message content `triggered`.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} title: description: > Title for your notification message. required: false type: string message: description: > Your notification message. Defaults to the string `triggered` if no value is provided. required: false type: string markdown: description: > Set to true to enable Markdown formatting for the message body. required: false type: boolean default: false tags: description: > Add tags or emojis to the notification. A list of strings representing tags or emoji codes. required: false type: list priority: description: > The priority of the notification (1 = minimum, 2 = low, 3 = default, 4 = high, 5 = maximum). required: false type: integer click: description: > URL that is opened when the notification is clicked. required: false type: string delay: description: > Set a delay for message delivery. The minimum delay is 10 seconds, and the maximum delay is 3 days. required: false type: map attach: description: > Attach images or other files by URL. required: false type: string attach_file: description: > Attach images or other files by uploading from a local file or camera media source. required: false type: map filename: description: > Custom filename for the attachment, including the file extension. required: false type: string email: description: > The email address to forward the notification to. required: false type: string call: description: > Phone number to call and read the message out loud using text-to-speech. required: false type: string icon: description: > URL of an icon that will appear next to the text of the notification. required: false type: string action: description: > Up to three actions (`view`, `broadcast`, `http`, or `copy`) can be added as buttons below the notification. Actions are executed when the corresponding button is tapped or clicked. required: false type: list sequence_id: description: > A message or sequence ID to update an existing notification, or to reference later when updating, clearing, or deleting a notification. required: false type: string

Action button options in YAML

Action `view`

{% options_yaml %} type: description: > Enter `view` to open a website or app when the button is clicked or tapped. required: true type: string label: description: > Label of the action button in the notification. required: true type: string url: description: > Label of the action button in the notification. required: true type: string clear: description: > Clear notification after action button is tapped. required: false type: boolean default: false

Action `http`

{% options_yaml %} type: description: > Enter `http` to send an HTTP request when the button is clicked or tapped. required: true type: string label: description: > Label of the action button in the notification. required: true type: string url: description: > URL to which the HTTP request will be sent. required: true type: string method: description: > HTTP method to use for request, default is `POST`. required: false type: string headers: description: > Additional HTTP headers as key-value pairs to send with the HTTP request. required: false type: map body: description: > The body of the HTTP. required: false type: string clear: description: > Clear notification after action button is tapped. required: false type: boolean default: false

Action `broadcast`

{% options_yaml %} type: description: > Enter `broadcast` to send an Android broadcast intent when the button is clicked or tapped. required: true type: string label: description: > Label of the action button in the notification. required: true type: string intent: description: > Android intent to send when the `broadcast` action is triggered. Defaults to `io.heckel.ntfy.USER_ACTION`. required: false type: string extras: description: > Extras to include in the intent as key-value pairs. required: false type: map clear: description: > Clear notification after action button is tapped. required: false type: boolean default: false

Action `copy`

{% options_yaml %} type: description: > Enter `copy` to copy a given value to the clipboard when the button is clicked or tapped. required: true type: string label: description: > Label of the action button in the notification. required: true type: string value: description: > Value to copy to the clipboard. required: true type: string clear: description: > Clear notification after action button is tapped. required: false type: boolean default: false

[Include not found: actions/targets.md domain="notify"]

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Action: send a notification with a camera snapshot

You can send a notification with a camera snapshot, for example when someone rings the doorbell.

- **Action:** ntfy: Publish notification
- **Target:** ntfy topic

YAML example for a notification with a camera snapshot

****action****

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
  data:
    title: Someone is at the door
    attach_file:
      media_content_id: media-source://camera/camera.demo_camera
      media_content_type: application/vnd.apple.mpegurl
    filename: camera-snapshot.jpg
    tags:
      - bellhop_bell
```

Action: send a dead man's switch notification

This action sends a notification that will only be delivered after a specified delay, acting as a so-called dead man's switch.

To reset the timer (for example, after a successful daily check-in), you must send the notification again. This cancels the previously scheduled notification and starts a new 24-hour countdown.

- **Action:** ntfy: Publish notification
- **Target:** ntfy topic

YAML example for a dead man's switch

****action****

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
  data:
    title: "Dead Man's Switch Activated"
    message: "I haven't checked in for 24 hours. Please check on me."
    priority: 5
    delay:
      hours: 24
    sequence_id: "dead-mans-switch-check-in"
    tags:
      - warning
      - skull
```

Action: send a notification to open a URL

This action sends a notification with an action button that, when tapped, opens a specific URL. For example, you can send an alert that includes a direct link to a relevant dashboard or camera feed.

- **Action:** ntfy: Publish notification
- **Target:** ntfy topic

YAML example for a notification with a button to open a URL

****action****

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
  data:
    message: "The garage door has been open for 10 minutes."
    actions:
      - type: view
        label: "View Garage Camera"
        url: "http://homeassistant.local/lovelace/garage"
        clear: true
```

Action: send a notification to trigger a webhook

This action sends a notification that can trigger a webhook or another automation via an HTTP request. For example, you can create a "Party Mode" button that, when tapped, tells Home Assistant to run a script.

- **Action:** ntfy: Publish notification
- **Target:** ntfy topic

YAML example for a notification with a button to trigger a webhook

****action****

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
  data:
    message: "The party is starting!"
    actions:
      - type: http
        label: "Start Party Mode"
        url: "http://homeassistant.local/api/webhook/party-mode-webhook"
        method: "POST"
```

Action: send a notification to copy text to the clipboard

This action sends a notification that allows you to copy a value to the clipboard. This is useful for sharing temporary information like guest Wi-Fi passwords or access codes.

- **Action:** ntfy: Publish notification
- **Target:** ntfy topic

YAML example for a notification with a copy to clipboard button

****action****

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
  data:
    title: "Guest Wi-Fi Password"
    message: "Here is the guest Wi-Fi password for today."
    actions:
      - type: copy
        label: "Copy Password"
        value: "GuestPass1234!"
```

Action: send a notification to start sleep tracking

This action sends a notification that can trigger an Android broadcast intent to start sleep tracking in Sleep as Android.

- **Action:** ntfy: Publish notification
- **Target:** ntfy topic

YAML example for a notification with a button to trigger an Android broadcast

****action****

```
action: |
  action: ntfy.publish
  target:
    entity_id: notify.mytopic
  data:
    message: "Time for bed?"
    actions:
      - type: broadcast
        label: "Start sleep tracking"
        intent: "com.urbandroid.sleep.START_SLEEP_TRACK"
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Person.Reload

The **Reload persons** action reloads person definitions from YAML. Use it after you change YAML-defined persons and want Home Assistant to apply those changes without a restart.

This action only reloads persons that are defined in YAML. It does not change people you manage from the UI.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or script, or select **Create** to start a new one.
3. If you're setting up a new automation, add a trigger in the **When** section. Scripts don't need a trigger.
4. In the **Then do** section, select **Add action**.
5. From the search box, search for and select **Reload persons**.
6. Select **Save**.

This action does not support targets. In the UI, you are not prompted to choose an area, device, entity, or label.

Options in the UI

This action has no additional options in the UI.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this action as `person.reload`:

action

```
action: |  
  action: person.reload
```

This reloads persons that are defined in YAML.

Options in YAML

This action has no additional options in YAML.

Good to know

- This action reloads YAML-defined persons only.
- People you add or edit from the UI are not changed by this action.
- If you have not changed your YAML, running this action does not make any visible changes.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: reload YAML-defined persons from a helper

If you want a simple way to apply person changes from a dashboard, you can create a helper separately and let it trigger this action.

- **Trigger:** A user-created helper turns on
- **Action:** Reload persons

YAML example for reloading persons from a helper

****automation****

```
automation: |
  alias: "Reload persons from a helper"
  triggers:
    - trigger: state
      entity_id: input_boolean.reload_persons
      to: "on"
  actions:
    - action: person.reload
    - action: input_boolean.turn_off
      target:
        entity_id: input_boolean.reload_persons
```

Automation: reload YAML-defined persons after a scheduled sync

If another process updates your person YAML before a set time, you can schedule this action so Home Assistant picks up those changes automatically.

- **Trigger:** A scheduled time
- **Action:** Reload persons

YAML example for reloading persons after a scheduled sync

****automation****

```
automation: |
  alias: "Reload persons after nightly YAML sync"
  triggers:
    - trigger: time
      at: "03:05:00"
  actions:
    - action: person.reload
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Clean Area

The **Clean area with vacuum cleaner** action sends your vacuum to clean one or more mapped Home Assistant areas.

Use it when only part of the home needs attention, like the kitchen after dinner or the hallway after muddy shoes, without sending the robot through every room.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Clean area with vacuum cleaner**.
4. Select your vacuum entity.
5. In **Areas**, choose one or more mapped Home Assistant areas.
6. Save the automation.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.clean_area
  target:
    entity_id: vacuum.cleaner
  cleaning_area_id:
    - living_room
    - kitchen
```

This sends `vacuum.cleaner` to clean the `living_room` and `kitchen` areas.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum to send to specific areas. required: true type: map cleaning_area_id: description: The areas to clean. Use Home Assistant area IDs. required: true type: list

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- You must first map vacuum segments to Home Assistant areas in the entity settings.
- If mapping or area selection does not appear, your vacuum does not support this feature.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: clean the kitchen and dining room after dinner

After dinner, this automation checks whether anyone is still in the kitchen. If the room is empty, it sends the vacuum only to the areas that usually need attention.

- **Trigger:** Time: 20:30
- **Condition:** Kitchen presence sensor is off
- **Action:** Clean area
- **Target:** Main vacuum
- **Area:** Kitchen, dining room

YAML example for a targeted after-dinner cleanup

****automation****

```
automation: |
  alias: "After dinner vacuum cleanup"
  triggers:
    - trigger: time
      at: "20:30:00"
  conditions:
    - condition: state
      entity_id: binary_sensor.kitchen_occupancy
      state: "off"
  actions:
    - action: vacuum.clean_area
      target:
        entity_id: vacuum.main_floor
      cleaning_area_id:
        - kitchen
        - dining_room
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Clean Spot

The **Clean spot with vacuum cleaner** action asks the vacuum to perform a concentrated cleaning cycle at its current position.

Use it when a small area needs extra attention, like around a chair, near pet bowls, or where crumbs have just landed.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Clean spot with vacuum cleaner**.
4. Select the target vacuum, area, or group.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.clean_spot
  target:
    entity_id: vacuum.office
```

This starts a spot clean at `vacuum.office`'s current location.

Omitting `entity_id` targets all supported vacuums.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or group to spot clean. required: false
type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Not all vacuum models support spot cleaning.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: run a quick spot clean from a dashboard button

If you notice a small mess, you can trigger a dedicated spot clean without starting a full-house run. This example starts a spot clean when a helper button is pressed.

- **Trigger:** Helper button pressed
- **Action:** Clean spot
- **Target:** Office vacuum

YAML example for starting a quick spot clean

****automation****

```
automation: |
  alias: "Quick spot clean"
  triggers:
    - trigger: state
      entity_id: input_button.quick_spot_clean
  actions:
    - action: vacuum.clean_spot
      target:
        entity_id: vacuum.office
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Locate

The **Locate vacuum cleaner** action causes the vacuum to play a sound or flash lights, making it easier to find.

Use it when the robot has ended up under a bed, behind furniture, or somewhere else that is hard to spot at a glance.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Locate vacuum cleaner**.
4. Select one or more vacuums, area, or group.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.locate
  target:
    entity_id: vacuum.upstairs
```

This makes `vacuum.upstairs` play its locate signal.

Omitting `entity_id` will target all supported vacuums in your system.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to locate. required: false
type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- The locate function's effects (sound, lights) depend on your vacuum model.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: locate the vacuum when the dust bin is full

Some vacuum integrations expose a binary sensor when the dust bin is full (for example, [iRobot Roomba and Braava](#)). If your vacuum reports that its bin needs attention, this automation makes it play its locate signal so you can find it quickly.

- **Trigger:** Roomba bin full sensor turns on
- **Action:** Locate vacuum
- **Target:** Upstairs vacuum

YAML example for locating a vacuum that needs emptying

****automation****

```
automation: |
  alias: "Locate vacuum when bin is full"
  triggers:
    - trigger: state
      entity_id: binary_sensor.roomba_bin_full
      to: "on"
  actions:
    - action: vacuum.locate
      target:
        entity_id: vacuum.upstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Pause

The **Pause vacuum cleaner** action instructs your vacuum to pause its current operation.

Use it when you need the robot to stop temporarily without ending the run, like during a phone call, while someone is sleeping, or when the doorbell rings.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. In **Add action**, search for **Vacuum: Pause vacuum cleaner**.
4. Choose the vacuum, area, or device to pause.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.pause
  target:
    entity_id: vacuum.downstairs
```

This pauses `vacuum.downstairs`.

The `entity_id` target is optional. If omitted, all connected vacuums will pause.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to pause. required: false
type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Some vacuums may not support pausing if they are not currently cleaning.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: pause cleaning when the doorbell rings

If the vacuum is running near the front door, a visitor can be hard to hear. This automation pauses the robot when the doorbell is pressed.

- **Trigger:** Doorbell pressed
- **Action:** Pause cleaning
- **Target:** Hallway vacuum

YAML example for pausing a vacuum on doorbell press

****automation****

```
automation: |
  alias: "Pause vacuum for doorbell"
  triggers:
    - trigger: state
      entity_id: binary_sensor.doorbell
      to: "on"
  actions:
    - action: vacuum.pause
      target:
        entity_id: vacuum.hallway
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Return To Base

The **Return vacuum cleaner to dock** action instructs the vacuum to stop its current task and return to its charging dock.

Use it when you want the robot to head home in an orderly way, like before bedtime, before guests arrive, or when you want the floor clear without leaving the vacuum stranded in the middle of a room.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Return vacuum cleaner to dock**.
4. Select a vacuum, area, or group.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.return_to_base
  target:
    entity_id: vacuum.living_room
```

This sends `vacuum.living_room` back to its dock.

If you omit `entity_id`, the action will target all vacuums.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to send to base. required: false type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Not all vacuums support returning to base from every state.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send the vacuum back before bedtime

At the end of the evening, this automation sends the vacuum back to its dock so the floor is clear and the robot is charging overnight.

- **Trigger:** Time: 22:30
- **Action:** Return to base
- **Target:** Downstairs vacuum

YAML example for sending a vacuum home at night

****automation****

```
automation: |
  alias: "Return vacuum to dock at bedtime"
  triggers:
    - trigger: time
      at: "22:30:00"
  actions:
    - action: vacuum.return_to_base
      target:
        entity_id: vacuum.downstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Send Command

The **Send command to vacuum cleaner** action passes a custom command (and optional parameters) directly to your vacuum for advanced or platform-specific control.

Use it for features that your vacuum integration exposes but that do not have a dedicated Home Assistant action, like toggling a do-not-disturb mode or changing a vendor-specific setting.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Send command to vacuum cleaner**.
4. Enter the desired command.
5. Optionally add parameters.
6. Select the vacuum target and save.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.send_command
  target:
    entity_id: vacuum.upstairs
  command: set_do_not_disturb
  params:
    enabled: true
```

This sends the `set_do_not_disturb` command to `vacuum.upstairs`.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum to send the command to. required: false type: map command: description: The command name the vacuum platform expects (string). required: true type: string params: description: (Optional) Parameters for the command (YAML or JSON mapping). required: false type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Platform-specific commands may not be documented. Consult your integration's documentation for command names and parameters.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: enable Do Not Disturb at night

Some vacuum platforms support a Do Not Disturb mode through a custom command. This automation sends that command each night so the robot stays quiet during sleeping hours.

- **Trigger:** Time: 23:00
- **Action:** Send command
- **Target:** Upstairs vacuum
- **Command:** `set_do_not_disturb`

YAML example for sending a custom vacuum command

****automation****

```
automation: |
  alias: "Vacuum Do Not Disturb at night"
  triggers:
    - trigger: time
      at: "23:00:00"
  actions:
    - action: vacuum.send_command
      target:
        entity_id: vacuum.upstairs
      command: set_do_not_disturb
      params:
        enabled: true
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Set Fan Speed

The **Set vacuum cleaner fan speed** action changes the fan or suction power level of the vacuum while running or before cleaning starts.

Use it when you want stronger suction for dirtier rooms, a quieter mode during the evening, or different cleaning intensity for different schedules.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Set vacuum cleaner fan speed**.
4. Choose the target vacuum, then select or enter the desired fan speed/power.
5. Save the automation.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.set_fan_speed
  target:
    entity_id: vacuum.cleaner
  fan_speed: turbo
```

This sets `vacuum.cleaner` to `turbo`.

The `fan_speed` value (label or number) is platform-dependent. Allowed values are typically found in your vacuum's manual or entity attributes.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

```
{% options_yaml %} target: description: Vacuum entity to control. required: false type: map
fan_speed: description: Fan speed as a label (like 'eco', 'turbo') or percentage (0-100). required:
true type: string
```

```
{%- assign target_domain = include.domain | default: page.domain -%}
```

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- Some platforms use named speeds; others use numeric values.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: use turbo mode for weekday cleaning

Before the weekday cleaning run starts, this automation sets the vacuum to a stronger fan speed so it can do a deeper clean while the house is empty.

- **Trigger:** Time: 09:00
- **Action:** Set fan speed
- **Target:** Main vacuum
- **Fan speed:** turbo

YAML example for increasing vacuum suction before cleaning

****automation****

```
automation: |
  alias: "Weekday vacuum turbo mode"
  triggers:
    - trigger: time
      at: "09:00:00"
  actions:
    - action: vacuum.set_fan_speed
      target:
        entity_id: vacuum.main_floor
      fan_speed: turbo
    - action: vacuum.start
      target:
        entity_id: vacuum.main_floor
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Start

The **Start vacuum cleaner** action sends a start or resume command to a vacuum, beginning or continuing a cleaning job.

Use it when you want the robot to begin on a schedule, resume after a pause, or start automatically once the house is empty.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an automation, or select **Create automation**.
3. In the **Add action** section, search for and select **Vacuum: Start vacuum cleaner**.
4. Choose one or more vacuum entities or an area.
5. Configure as needed and select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.start
  target:
    entity_id:
      - vacuum.downstairs
      - vacuum.upstairs
```

This starts `vacuum.downstairs` and `vacuum.upstairs`.

The `entity_id` is optional; omit it to target all vacuums.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to start cleaning. required: false type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works if your vacuum supports the start function.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start cleaning after everyone leaves

When the last person leaves home, this automation starts the downstairs vacuum so it can clean while the house is empty.

- **Trigger:** Everyone leaves home
- **Action:** Start cleaning
- **Target:** Downstairs vacuum

YAML example for starting the vacuum when nobody is home

****automation****

```
automation: |
  alias: "Start vacuum when house is empty"
  triggers:
    - trigger: state
      entity_id: group.family
      to: not_home
  actions:
    - action: vacuum.start
      target:
        entity_id: vacuum.downstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Start Pause

The **Start/pause vacuum cleaner** action starts, pauses, or resumes a supported vacuum cleaner's cleaning task.

Use it when you want a single action to handle the current cleaning state without first checking whether the vacuum is idle, cleaning, or paused.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Start/pause vacuum cleaner**.
4. Choose the vacuum, area, or device to control.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.start_pause
  target:
    entity_id: vacuum.downstairs
```

This starts, pauses, or resumes `vacuum.downstairs`, depending on its current state.

The `entity_id` target is optional. If omitted, all targeted supported vacuums receive the command.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to control. required: false type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works for vacuums that support start/pause control.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start or pause cleaning with one helper button

If you want one control for both actions, this automation uses a helper button to start cleaning when the vacuum is idle and pause it when it is already running.

- **Trigger:** Helper button pressed
- **Action:** Start or pause cleaning
- **Target:** Downstairs vacuum

YAML example for starting or pausing a vacuum

****automation****

```
automation: |
  alias: "Start or pause vacuum"
  triggers:
    - trigger: state
      entity_id: input_button.vacuum_start_pause
  actions:
    - action: vacuum.start_pause
      target:
        entity_id: vacuum.downstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Stop

The **Stop vacuum cleaner** action immediately stops the vacuum's current activity (cleaning, returning to dock, spot clean, etc.).

Use it when you want the robot to stop right away instead of pausing or returning to the dock, like during an unexpected spill, a pet accident, or another situation where you need it out of the area immediately.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Create or edit an automation.
3. Add an action and search for **Vacuum: Stop vacuum cleaner**.
4. Select the target vacuum, area, or group.
5. Save your automation.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.stop
  target:
    entity_id: vacuum.upstairs
```

This stops `vacuum.upstairs` .

`entity_id` is optional. Omitting it stops all connected vacuums.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or entity to stop. required: false type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works for vacuums that are currently active or returning.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: stop the vacuum if a leak is detected

If a leak sensor activates in the kitchen, this automation stops the vacuum immediately so it does not drive through water.

- **Trigger:** Leak detected
- **Action:** Stop vacuum
- **Target:** Downstairs vacuum

YAML example for stopping a vacuum on leak detection

****automation****

```
automation: |
  alias: "Stop vacuum on leak"
  triggers:
    - trigger: state
      entity_id: binary_sensor.kitchen_leak
      to: "on"
  actions:
    - action: vacuum.stop
      target:
        entity_id: vacuum.downstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Toggle

The **Toggle vacuum cleaner** action switches a supported vacuum cleaner between on and off.

Use it when you want one automation to flip the vacuum power state without first checking whether it is already on or off.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Toggle vacuum cleaner**.
4. Choose the vacuum, area, or device to control.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.toggle
  target:
    entity_id: vacuum.downstairs
```

This toggles the power state of `vacuum.downstairs`.

The `entity_id` target is optional. If omitted, all targeted supported vacuums toggle.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to toggle. required: false
type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works for vacuums that support both turning on and turning off.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: toggle the vacuum power from a helper button

If you want a quick manual control, this automation toggles the vacuum power whenever a helper button is pressed.

- **Trigger:** Helper button pressed
- **Action:** Toggle vacuum power
- **Target:** Downstairs vacuum

YAML example for toggling vacuum power

****automation****

```
automation: |
  alias: "Toggle vacuum power"
  triggers:
    - trigger: state
      entity_id: input_button.toggle_vacuum_power
  actions:
    - action: vacuum.toggle
      target:
        entity_id: vacuum.downstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Turn Off

The **Turn off vacuum cleaner** action turns off a supported vacuum cleaner.

Use it when your vacuum supports a separate power state and you want to shut it down after cleaning or before maintenance.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Turn off vacuum cleaner**.
4. Choose the vacuum, area, or device to turn off.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.turn_off
  target:
    entity_id: vacuum.downstairs
```

This turns off `vacuum.downstairs`.

The `entity_id` target is optional. If omitted, all targeted supported vacuums turn off.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to turn off. required: false
type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works for vacuums that support turning off.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn off the vacuum after it returns to the dock

If your vacuum supports a separate power state, this automation turns it off after it has finished cleaning and returned to the dock.

- **Trigger:** Vacuum returned to dock
- **Action:** Turn off vacuum
- **Target:** Downstairs vacuum

YAML example for turning off a vacuum after docking

****automation****

```
automation: |
  alias: "Turn off vacuum after docking"
  triggers:
    - trigger: vacuum.docked
      target:
        entity_id: vacuum.downstairs
  actions:
    - action: vacuum.turn_off
      target:
        entity_id: vacuum.downstairs
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Actions/Vacuum.Turn On

The **Turn on vacuum cleaner** action turns on a supported vacuum cleaner.

Use it when your vacuum supports a separate power state and you want to make sure it is powered on before you run another action.

Using this action from the user interface

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

To use this action from an automation or script:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. Add an action and search for **Vacuum: Turn on vacuum cleaner**.
4. Choose the vacuum, area, or device to turn on.
5. Select **Save**.

Using this action in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

action

```
action: |
  action: vacuum.turn_on
  target:
    entity_id: vacuum.downstairs
```

This turns on `vacuum.downstairs`.

The `entity_id` target is optional. If omitted, all targeted supported vacuums turn on.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} target: description: The vacuum, area, or device to turn on. required: false
type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the action

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

Good to know

- This action only works for vacuums that support turning on.

Try it yourself

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

More examples

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the vacuum before a scheduled cleaning run

If your vacuum uses a separate power state, this automation turns it on a few minutes before a scheduled cleaning run starts.

- **Trigger:** Time: 08:55
- **Action:** Turn on vacuum
- **Target:** Downstairs vacuum

YAML example for turning on a vacuum before cleaning

****automation****

```
automation: |
  alias: "Turn on vacuum before cleaning"
  triggers:
    - trigger: time
      at: "08:55:00"
  actions:
    - action: vacuum.turn_on
      target:
        entity_id: vacuum.downstairs
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

Related actions

These actions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Co2 Value

The **Carbon dioxide value** condition passes when a carbon dioxide (CO2) sensor's reading meets a specific level. A stuffy meeting room, a crowded living room on movie night, or a bedroom with the door closed overnight all push CO2 levels higher than you would expect. This condition lets your automation act only when CO2 is genuinely elevated, so the ventilation fan starts when it is truly needed and stays off when the air is fine.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Carbon dioxide value**.
6. Under **Threshold type**, set the CO2 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The carbon dioxide level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_co2_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_co2_value
  target:
    entity_id: sensor.office_co2
  options:
    threshold: 1000
    behavior: any
```

This passes when the office CO2 sensor reads at or above 1000 ppm.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The carbon dioxide level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Outdoor CO2 levels are typically around 420 ppm. Indoor levels above 1000 ppm often suggest the room needs better ventilation.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on ventilation at bedtime only if CO2 is elevated

After you spend an evening in the living room with the doors closed, CO2 levels are sometimes higher than you would expect by the time you head to bed. This automation triggers at your usual bedtime and checks the bedroom CO2 reading. If the level is at or above 1000 ppm, the ventilation fan turns on so you sleep with fresh air. On evenings when the room already has good airflow, the fan stays off.

- **Trigger:** Time: 22:30
- **Condition:** Air Quality: Carbon dioxide value
- **Target:** Bedroom CO2 sensor
- **Threshold type:** 1000
- **Condition passes if:** Any
- **Action:** Fan: Turn on

YAML example for bedtime ventilation on high CO2

****automation****

```
automation: |
  alias: "Bedtime ventilation if CO2 is high"
  triggers:
    - trigger: time
      at: "22:30:00"
  conditions:
    - condition: air_quality.is_co2_value
      target:
        entity_id: sensor.bedroom_co2
      options:
        threshold: 1000
        behavior: any
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom_ventilation
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Co Cleared

The **Carbon monoxide cleared** condition passes when one or more carbon monoxide sensors are no longer detecting carbon monoxide (CO). After a CO event, you want to be absolutely sure the air is safe before letting your automation silence the alarm or tell the household everything is fine.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your CO sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Carbon monoxide cleared**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple sensors are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor is cleared, or **All** to pass only when every targeted sensor is cleared.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_co_cleared`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_co_cleared
  target:
    entity_id: binary_sensor.hallway_co
```

This passes when the hallway carbon monoxide sensor is no longer detecting CO.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as cleared. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted sensor is unavailable.
- To check whether carbon monoxide is currently detected, use [Carbon monoxide detected](#).
- To check the actual CO concentration, use [Carbon monoxide value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only silence the alarm once every CO sensor has cleared

After a CO event, you want the alarm to keep sounding until every room is safe. This automation triggers when you press the silence button, but the condition requires *every* CO sensor in the house to read clear before the siren actually turns off. If any sensor still detects carbon monoxide, the alarm keeps going.

- **Trigger:** State: Silence alarm button pressed
- **Condition:** Air Quality: Carbon monoxide cleared
- **Target:** All CO sensors (hallway, basement)
- **Condition passes if:** All
- **Action:** Siren: Turn off, then notify the household

YAML example for silencing the alarm only after full CO all-clear

```
**automation**
```

```
automation: |
  alias: "Silence alarm only after full CO all-clear"
  triggers:
    - trigger: state
      entity_id: input_button.silence_alarm
  conditions:
    - condition: air_quality.is_co_cleared
      target:
        entity_id:
          - binary_sensor.hallway_co
          - binary_sensor.basement_co
      options:
        behavior: all
  actions:
    - action: siren.turn_off
      target:
        entity_id: siren.house_alarm
    - action: notify.mobile_app_phone
      data:
        title: "CO all-clear"
        message: >
          Every CO sensor reads clear.
          The alarm has been silenced.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Co Detected

The **Carbon monoxide detected** condition passes when one or more carbon monoxide sensors are actively detecting carbon monoxide (CO). Because CO is colorless and odorless, your sensors are the only way to know it is there. Adding this condition to your automation ensures that safety actions, like sounding an alarm, turning on ventilation, or sending an urgent notification, only happen while the danger is confirmed. It prevents false alarms from a sensor that briefly flickered and keeps your response focused on real threats.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your CO sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Carbon monoxide detected**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple sensors are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor detects carbon monoxide, or **All** to pass only when every targeted sensor detects carbon monoxide.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_co_detected`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_co_detected
  target:
    entity_id: binary_sensor.hallway_co
```

This passes when the hallway carbon monoxide sensor is currently detecting CO.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as detecting. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted sensor is unavailable.
- To check whether carbon monoxide is no longer detected, use [Carbon monoxide cleared](#).
- To check the actual CO concentration rather than just a binary detection, use [Carbon monoxide value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: warn the family when someone arrives home during a CO event

If carbon monoxide built up while the family was out, the first person home needs a warning before walking inside. This automation triggers when someone enters the home zone and checks whether the hallway CO sensor is still detecting carbon monoxide. If it is, an urgent notification tells them to stay outside and call emergency services.

- **Trigger:** Zone: Person enters home zone
- **Condition:** Air Quality: Carbon monoxide detected
- **Target:** Hallway CO sensor
- **Condition passes if:** Any
- **Action:** Notify: Send urgent notification

YAML example for a CO warning on arrival home

```
**automation**
```

```
automation: |
  alias: "CO warning on arrival home"
  triggers:
    - trigger: zone
      entity_id: person.frenck
      zone: zone.home
      event: enter
  conditions:
    - condition: air_quality.is_co_detected
      target:
        entity_id: binary_sensor.hallway_co
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Carbon monoxide detected at home"
        message: >
          The hallway CO sensor is detecting carbon
          monoxide. Stay outside and call emergency services.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Co Value

The **Carbon monoxide value** condition passes when a carbon monoxide (CO) sensor's reading meets a specific level. Carbon monoxide is a colorless, odorless gas produced by incomplete combustion, and even moderate levels deserve attention. This condition gives you finer control than simpler detected or cleared checks, letting you start ventilation at a lower reading and sound the full alarm only when concentrations climb higher.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Carbon monoxide value**.
6. Under **Threshold type**, set the CO level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The carbon monoxide level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_co_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_co_value
  target:
    entity_id: sensor.hallway_co
  options:
    threshold: 35
    behavior: any
```

This passes when the hallway CO sensor reads at or above 35 ppm.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The carbon monoxide level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- For simple binary detection without a specific threshold, use `Carbon monoxide detected` or `Carbon monoxide cleared`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: check CO levels when you arrive home

When you pull into the driveway, you want to know if the air inside is safe before settling in. This automation triggers when you enter the home zone and checks the hallway CO reading. If the level is at or above 20 ppm, you get a notification right away so you know to open the windows or stay outside until the air clears.

- **Trigger:** Zone: Person enters home zone
- **Condition:** Air Quality: Carbon monoxide value
- **Target:** Hallway CO sensor
- **Threshold type:** 20
- **Condition passes if:** Any
- **Action:** Notify: Send notification

YAML example for a CO check on arrival home

```
**automation**
```

```
automation: |
  alias: "CO check on arrival home"
  triggers:
    - trigger: zone
      entity_id: person.frenck
      zone: zone.home
      event: enter
  conditions:
    - condition: air_quality.is_co_value
      target:
        entity_id: sensor.hallway_co
      options:
        threshold: 20
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "CO level elevated at home"
        message: >
          The hallway CO reading is above 20 ppm.
          Open the windows before settling in.
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Gas Cleared

The **Gas cleared** condition passes when one or more gas sensors are no longer detecting gas. After a gas event, you want to be sure the air is truly safe before your automation reopens a shut-off valve or sends an all-clear notification. This condition acts as that safeguard, preventing your automation from acting too early while a reading persists in another room.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your gas sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Gas cleared**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple sensors are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor is cleared, or **All** to pass only when every targeted sensor is cleared.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_gas_cleared`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_gas_cleared
  target:
    entity_id: binary_sensor.kitchen_gas
```

This passes when the kitchen gas sensor is no longer detecting gas.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as cleared. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted sensor is unavailable.
- To check whether gas is currently detected, use [Gas detected](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only silence the alarm once every sensor has cleared

After a gas event, pressing a button to silence the alarm too early is risky if gas lingers in another room. This automation triggers when you press the silence button, but the condition requires every gas sensor in the house to read clear before the siren actually turns off. If any sensor still detects gas, the alarm keeps sounding.

- **Trigger:** State: Silence alarm button pressed
- **Condition:** Air Quality: Gas cleared
- **Target:** All gas sensors (kitchen, basement)
- **Condition passes if:** All
- **Action:** Siren: Turn off, then notify the household

YAML example for silencing the alarm only after full all-clear

```
**automation**
```

```
automation: |
  alias: "Silence alarm only after full gas all-clear"
  triggers:
    - trigger: state
      entity_id: input_button.silence_alarm
  conditions:
    - condition: air_quality.is_gas_cleared
      target:
        entity_id:
          - binary_sensor.kitchen_gas
          - binary_sensor.basement_gas
      options:
        behavior: all
  actions:
    - action: siren.turn_off
      target:
        entity_id: siren.house_alarm
    - action: notify.mobile_app_phone
      data:
        title: "Gas all-clear"
        message: >
          Every gas sensor reads clear.
          The alarm has been silenced.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Gas Detected

The **Gas detected** condition passes when one or more gas sensors are actively detecting gas. Gas sensors watch for combustible or toxic gases in the air, helping protect your home from leaks and hazardous buildups. Add this condition to your automation so it only takes action while a gas hazard is still present, for example keeping the kitchen exhaust fan running for as long as the sensor reports gas, or making sure an emergency notification goes out only when the threat is real and ongoing.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#). Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your gas sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Gas detected**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple sensors are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor detects gas, or **All** to pass only when every targeted sensor detects gas.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_gas_detected`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_gas_detected
  target:
    entity_id: binary_sensor.kitchen_gas
```

This passes when the kitchen gas sensor is currently detecting gas.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as detecting. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted sensor is unavailable.
- To check whether gas is no longer detected, use `Gas cleared`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press **Ctrl+V** (or **Cmd+V** on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: alert when you arrive home and gas is detected

If a gas leak started while you were away, you want to know the moment you pull into the driveway. This automation triggers when you arrive home and checks whether the kitchen gas sensor is still detecting gas. If it is, you get an urgent notification before you even open the front door so you know to stay outside and call for help.

- **Trigger:** Zone: Person enters home zone
- **Condition:** Air Quality: Gas detected
- **Target:** Kitchen gas sensor
- **Condition passes if:** Any
- **Action:** Notify: Send urgent notification

YAML example for a gas alert on arrival home

****automation****

```
automation: |
  alias: "Gas alert on arrival home"
  triggers:
    - trigger: zone
      entity_id: person.frenck
      zone: zone.home
      event: enter
  conditions:
    - condition: air_quality.is_gas_detected
      target:
        entity_id: binary_sensor.kitchen_gas
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Gas detected at home"
        message: >
          The kitchen gas sensor is detecting gas.
          Do not enter the house.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is N2O Value

The **Nitrous oxide value** condition passes when a nitrous oxide (N2O) sensor's reading meets a specific level. N2O is a greenhouse gas that shows up in agricultural settings, greenhouses, and some industrial spaces. If you monitor a greenhouse or workshop, this condition lets your automation turn on ventilation only when N2O levels genuinely need attention, keeping fans off during normal readings so you save energy and reduce noise.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Nitrous oxide value**.
6. Under **Threshold type**, set the N2O level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The nitrous oxide level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_n2o_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_n2o_value
  target:
    entity_id: sensor.greenhouse_n2o
  options:
    threshold: 500
    behavior: any
```

This passes when the greenhouse N2O sensor reads at or above 500 ppb.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The nitrous oxide level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- For related nitrogen-based pollutants, see [Nitrogen monoxide value](#) and [Nitrogen dioxide value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: ventilate the greenhouse each morning only if N2O is elevated

Fertilizer off-gassing raises N2O levels overnight in an enclosed greenhouse, especially on warm nights. This automation runs each morning at sunrise and checks the current reading. If N2O is at or above 500 ppb, the ventilation fan turns on so the air is fresh before you start working. On mornings when levels stayed low, the fan stays off, keeping the greenhouse warm and saving energy.

- **Trigger:** Sun: At sunrise
- **Condition:** Air Quality: Nitrous oxide value
- **Target:** Greenhouse N2O sensor
- **Threshold type:** 500
- **Condition passes if:** Any
- **Action:** Fan: Turn on

YAML example for morning greenhouse ventilation on high N2O

****automation****

```
automation: |
  alias: "Morning greenhouse ventilation if N2O is high"
  triggers:
    - trigger: sun
      event: sunrise
  conditions:
    - condition: air_quality.is_n2o_value
      target:
        entity_id: sensor.greenhouse_n2o
      options:
        threshold: 500
        behavior: any
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.greenhouse_ventilation
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is No2 Value

The **Nitrogen dioxide value** condition passes when a nitrogen dioxide (NO₂) sensor's reading meets a specific level. NO₂ is a reddish-brown gas that comes from traffic and combustion, and elevated levels irritate the airways. This condition helps your automation make informed decisions about outdoor air, for example holding off on opening the windows during rush hour or sending you a notification recommending indoor exercise when NO₂ is too high.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Nitrogen dioxide value**.
6. Under **Threshold type**, set the NO2 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The nitrogen dioxide level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_no2_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_no2_value
  target:
    entity_id: sensor.outdoor_no2
  options:
    threshold: 40
    behavior: any
```

This passes when the outdoor NO2 sensor reads at or above 40 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The nitrogen dioxide level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- For related pollutants, see [Nitrogen monoxide value](#), [Nitrous oxide value](#), and [Ozone value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: suggest indoor exercise before your morning run

If you have a daily running routine, you want to know whether the outdoor air is safe before heading out. This automation triggers at 6:30 AM and checks whether outdoor NO₂ is at or above 40 µg/m³. If it is, you get a friendly suggestion to exercise indoors instead. On clean-air mornings, you never hear a thing and head out as usual.

- **Trigger:** Time: 06:30
- **Condition:** Air Quality: Nitrogen dioxide value
- **Target:** Outdoor NO₂ sensor
- **Threshold type:** 40
- **Condition passes if:** Any
- **Action:** Notify: Send notification

YAML example for an NO₂ exercise suggestion before your run

****automation****

```
automation: |
  alias: "Suggest indoor exercise on high NO2"
  triggers:
    - trigger: time
      at: "06:30:00"
  conditions:
    - condition: air_quality.is_no2_value
      target:
        entity_id: sensor.outdoor_no2
      options:
        threshold: 40
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "High NO2 outside"
        message: >
          Outdoor NO2 is above 40. Consider
          exercising indoors today.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is No Value

The **Nitrogen monoxide value** condition passes when a nitrogen monoxide (NO) sensor's reading meets a specific level. NO is a reactive gas that shows up mainly around vehicle exhaust and industrial activity. If you live near a busy road, rush-hour traffic raises NO levels noticeably. This condition lets your automation respond to that pattern, closing the garage ventilation when NO spikes during the morning commute and reopening it once readings settle down.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Nitrogen monoxide value**.
6. Under **Threshold type**, set the NO level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The nitrogen monoxide level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_no_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_no_value
  target:
    entity_id: sensor.outdoor_no
  options:
    threshold: 100
    behavior: any
```

This passes when the outdoor NO sensor reads at or above 100 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The nitrogen monoxide level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Nitrogen monoxide is closely related to nitrogen dioxide (NO₂). For NO₂ readings, see [Nitrogen dioxide value](#). For nitrous oxide, see [Nitrous oxide value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: keep the garage sealed during the morning commute

If you live near a busy road, rush-hour exhaust raises outdoor NO levels fast. This automation triggers at 7:30 AM and checks the outdoor NO reading. If the level is at or above 100 µg/m³, the garage ventilation closes so fumes stay outside. On mornings with light traffic, the ventilation stays open as usual.

- **Trigger:** Time: 07:30
- **Condition:** Air Quality: Nitrogen monoxide value
- **Target:** Outdoor NO sensor
- **Threshold type:** 100
- **Condition passes if:** Any
- **Action:** Cover: Close cover

YAML example for closing the garage during the morning commute

****automation****

```
automation: |
  alias: "Seal garage during morning commute"
  triggers:
    - trigger: time
      at: "07:30:00"
  conditions:
    - condition: air_quality.is_no_value
      target:
        entity_id: sensor.outdoor_no
      options:
        threshold: 100
        behavior: any
  actions:
    - action: cover.close_cover
      target:
        entity_id: cover.garage_ventilation
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Ozone Value

The **Ozone value** condition passes when an ozone (O3) sensor's reading meets a specific level. Ground-level ozone is a key component of smog and tends to spike on hot, sunny afternoons. This condition helps your automation make smarter ventilation decisions, keeping the windows shut during a smog alert but allowing fresh air in after the ozone reading decreases to a comfortable level.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Ozone value**.
6. Under **Threshold type**, set the ozone level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The ozone level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_ozone_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_ozone_value
  target:
    entity_id: sensor.outdoor_ozone
  options:
    threshold: 100
    behavior: any
```

This passes when the outdoor ozone sensor reads at or above 100 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The ozone level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Ozone levels tend to peak on hot, sunny afternoons. Pair this condition with time-based triggers to limit outdoor ventilation during those hours.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only open windows in the afternoon if ozone is low enough

On hot summer days, ozone builds up through the afternoon and peaks around 3 PM. This automation triggers at that time and checks whether outdoor ozone is at or above 100 µg/m³. If the reading is still high, the windows stay shut, and you get a notification explaining why. On breezy days with lower ozone, the automation does nothing, and you enjoy fresh air as usual.

- **Trigger:** Time: 15:00
- **Condition:** Air Quality: Ozone value
- **Target:** Outdoor ozone sensor
- **Threshold type:** 100
- **Condition passes if:** Any
- **Action:** Cover: Close cover, then notify

YAML example for keeping windows shut on high ozone afternoons

```
**automation**
```

```
automation: |
  alias: "Keep windows shut on high ozone afternoons"
  triggers:
    - trigger: time
      at: "15:00:00"
  conditions:
    - condition: air_quality.is_ozone_value
      target:
        entity_id: sensor.outdoor_ozone
      options:
        threshold: 100
        behavior: any
  actions:
    - action: cover.close_cover
      target:
        area_id: living_room
    - action: notify.mobile_app_phone
      data:
        title: "Ozone is high outside"
        message: >
          Outdoor ozone is above 100. Windows
          are staying closed this afternoon.
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Pm10 Value

The **PM10 value** condition passes when a PM10 sensor's reading meets a specific level. PM10 covers coarse particulate matter smaller than 10 micrometers in diameter, which includes dust, pollen, and mold spores. If someone in your household has allergies, this condition is especially useful for closing the windows automatically during high-pollen hours and keeping them open when the reading is low, so you enjoy fresh air without the sneezing.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **PM10 value**.
6. Under **Threshold type**, set the PM10 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The PM10 level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_pm10_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_pm10_value
  target:
    entity_id: sensor.outdoor_pm10
  options:
    threshold: 50
    behavior: any
```

This passes when the outdoor PM10 sensor reads at or above 50 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The PM10 level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- PM10 includes larger particles like dust and pollen. For finer readings, see:
 - PM1 value
 - PM2.5 value
 - PM4 value

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: check outdoor air before opening windows in the morning

On spring mornings, pollen and dust push PM10 readings up before you even notice. This automation triggers when you open the bedroom window cover and checks the outdoor PM10 reading first. If the level is at or above 50 $\mu\text{g}/\text{m}^3$, the cover closes right back and you get a notification explaining why. On clean-air mornings, nothing happens and you enjoy the fresh breeze.

- **Trigger:** State: Bedroom window cover opened
- **Condition:** Air Quality: PM10 value
- **Target:** Outdoor PM10 sensor
- **Threshold type:** 50
- **Condition passes if:** Any
- **Action:** Cover: Close cover, then notify

YAML example for closing windows back on high PM10

```
**automation**
```

```
automation: |
  alias: "Close windows if outdoor PM10 is high"
  triggers:
    - trigger: state
      entity_id: cover.bedroom_window
      to: open
  conditions:
    - condition: air_quality.is_pm10_value
      target:
        entity_id: sensor.outdoor_pm10
      options:
        threshold: 50
        behavior: any
  actions:
    - action: cover.close_cover
      target:
        entity_id: cover.bedroom_window
    - action: notify.mobile_app_phone
      data:
        title: "PM10 is high outside"
        message: >
          Outdoor PM10 is above 50. The window
          has been closed to keep allergens out.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Pm1 Value

The **PM1 value** condition passes when a PM1 sensor's reading meets a specific level. PM1 refers to ultra-fine particulate matter smaller than 1 micrometer in diameter, particles so tiny they travel deep into the lungs. Cooking on a gas stove or burning a candle sends PM1 readings up quickly. This condition lets you start an air purifier only when ultra-fine particles are genuinely elevated, saving energy on quiet days and protecting your family when it matters.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **PM1 value**.
6. Under **Threshold type**, set the PM1 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The PM1 level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_pml_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_pml_value
  target:
    entity_id: sensor.bedroom_pml
  options:
    threshold: 25
    behavior: any
```

This passes when the bedroom PM1 sensor reads at or above 25 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The PM1 level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- For coarser particulate readings, see
 - [PM2.5 value](#)
 - [PM4 value](#)
 - [PM10 value](#)

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start the bedroom purifier at bedtime only if PM1 is elevated

Ultra-fine particles from cooking, candles, or the heating system linger in a closed bedroom. This automation triggers at bedtime and checks the current PM1 reading. If the level is at or above 25 $\mu\text{g}/\text{m}^3$, the air purifier turns on so you breathe clean air while you sleep. On evenings when the air is already fresh, the purifier stays off and the bedroom stays quiet.

- **Trigger:** Time: 22:00
- **Condition:** Air Quality: PM1 value
- **Target:** Bedroom PM1 sensor
- **Threshold type:** 25
- **Condition passes if:** Any
- **Action:** Fan: Turn on (air purifier)

YAML example for starting the purifier at bedtime on high PM1

****automation****

```
automation: |
  alias: "Bedroom purifier at bedtime if PM1 is high"
  triggers:
    - trigger: time
      at: "22:00:00"
  conditions:
    - condition: air_quality.is_pm1_value
      target:
        entity_id: sensor.bedroom_pm1
      options:
        threshold: 25
        behavior: any
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom_purifier
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Pm25 Value

The **PM2.5 value** condition passes when a PM2.5 sensor's reading meets a specific level. PM2.5 is the most widely used indicator of air quality, and for good reason: these fine particles (smaller than 2.5 micrometers) come from traffic, wildfires, and everyday cooking, and they affect respiratory health at surprisingly low concentrations. This condition gives your automation precision, closing the windows only when outdoor PM2.5 is above a safe limit or holding off on opening them until the reading drops back down.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **PM2.5 value**.
6. Under **Threshold type**, set the PM2.5 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The PM2.5 level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_pm25_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_pm25_value
  target:
    entity_id: sensor.living_room_pm25
  options:
    threshold: 35
    behavior: any
```

This passes when the living room PM2.5 sensor reads at or above 35 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The PM2.5 level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- The World Health Organization recommends keeping 24-hour average PM2.5 exposure below 15 µg/m3.
- For other particle sizes, see
 - [PM1 value](#)
 - [PM4 value](#)
 - [PM10 value](#)

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: check outdoor PM2.5 before opening the windows each morning

During wildfire season, outdoor PM2.5 readings spike overnight while you sleep. This automation triggers when you open the living room window cover and checks the outdoor PM2.5 reading first. If the level is at or above 35 $\mu\text{g}/\text{m}^3$, the cover closes right back and you get a notification letting you know the air outside is not safe for ventilation. On clear mornings, nothing happens and you enjoy the fresh air.

- **Trigger:** State: Living room window cover opened
- **Condition:** Air Quality: PM2.5 value
- **Target:** Outdoor PM2.5 sensor
- **Threshold type:** 35
- **Condition passes if:** Any
- **Action:** Cover: Close cover, then notify

YAML example for closing windows back on high outdoor PM2.5

****automation****

```
automation: |
  alias: "Close windows if outdoor PM2.5 is high"
  triggers:
    - trigger: state
      entity_id: cover.living_room_window
      to: open
  conditions:
    - condition: air_quality.is_pm25_value
      target:
        entity_id: sensor.outdoor_pm25
      options:
        threshold: 35
        behavior: any
  actions:
    - action: cover.close_cover
      target:
        entity_id: cover.living_room_window
    - action: notify.mobile_app_phone
      data:
        title: "PM2.5 is high outside"
        message: >
          Outdoor PM2.5 is above 35. The window
          has been closed to keep the air clean.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Pm4 Value

The **PM4 value** condition passes when a PM4 sensor's reading meets a specific level. PM4 covers particulate matter smaller than 4 micrometers in diameter, a size range that bridges fine and coarse particles. Some sensors report PM4 alongside PM2.5 and PM10, giving you a more complete picture of what is floating in the air. This condition lets your automation react when PM4 readings are elevated, for example sending a notification that your air filter is due for a check.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **PM4 value**.
6. Under **Threshold type**, set the PM4 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The PM4 level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_pm4_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_pm4_value
  target:
    entity_id: sensor.living_room_pm4
  options:
    threshold: 50
    behavior: any
```

This passes when the living room PM4 sensor reads at or above 50 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The PM4 level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- For other particle sizes, see
 - [PM1 value](#)
 - [PM2.5 value](#)[/conditions/air_quality.is_pm25_value/]
 - [PM10 value](#)

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send a weekly filter reminder only if PM4 is elevated

If your indoor PM4 readings are consistently high, the air filters are likely overdue for a swap. This automation runs every Sunday morning and checks the living room PM4 reading. If the level is at or above 50 µg/m3, you get a reminder to check the filters. On weeks when the air is clean, no notification is sent.

- **Trigger:** Time: Every Sunday at 09:00
- **Condition:** Air Quality: PM4 value
- **Target:** Living room PM4 sensor
- **Threshold type:** 50
- **Condition passes if:** Any
- **Action:** Notify: Send notification

YAML example for a weekly PM4 filter reminder

```
**automation**
```

```
automation: |
  alias: "Weekly filter reminder if PM4 is high"
  triggers:
    - trigger: time
      at: "09:00:00"
  conditions:
    - condition: time
      weekday:
        - sun
    - condition: air_quality.is_pm4_value
      target:
        entity_id: sensor.living_room_pm4
      options:
        threshold: 50
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Time to check the air filters"
        message: >
          The living room PM4 reading is above 50.
          Your air filters might need replacing.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Smoke Cleared

The **Smoke cleared** condition passes when one or more smoke sensors are no longer detecting smoke. After a smoke event, the last thing you want is to restore normal lighting or send an all-clear while one room still has hazy air. This condition acts as your safety gate, letting your automation continue only after every sensor in the house confirms the air is clear again.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your smoke sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Smoke cleared**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple sensors are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor is cleared, or **All** to pass only when every targeted sensor is cleared.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_smoke_cleared`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_smoke_cleared
  target:
    entity_id: binary_sensor.kitchen_smoke
```

This passes when the kitchen smoke sensor is no longer detecting smoke.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as cleared. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted sensor is unavailable.
- To check whether smoke is currently detected, use [Smoke detected](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only restore normal lighting once every smoke sensor has cleared

After a smoke event, you want to keep the emergency lights on until every room is safe. This automation triggers when you press the reset button, but the condition requires every smoke sensor to read clear before the normal lighting scene restores. If any sensor still detects smoke, the emergency lights stay on.

- **Trigger:** State: Reset lighting button pressed
- **Condition:** Air Quality: Smoke cleared
- **Target:** All smoke sensors (kitchen, hallway)
- **Condition passes if:** All
- **Action:** Scene: Turn on (normal lighting), then notify

YAML example for restoring lighting only after full smoke all-clear

****automation****


```
automation: |
  alias: "Restore lights only after full smoke all-clear"
  triggers:
    - trigger: state
      entity_id: input_button.reset_lighting
  conditions:
    - condition: air_quality.is_smoke_cleared
      target:
        entity_id:
          - binary_sensor.kitchen_smoke
          - binary_sensor.hallway_smoke
      options:
        behavior: all
  actions:
    - action: scene.turn_on
      target:
        entity_id: scene.normal_lighting
    - action: notify.mobile_app_phone
      data:
        title: "Smoke all-clear"
        message: >
          Every smoke sensor reads clear.
          Normal lighting has been restored.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Smoke Detected

The **Smoke detected** condition passes when one or more smoke sensors are actively detecting smoke. When seconds count, your automation needs to act on confirmed smoke, not on a brief sensor glitch. This condition makes sure that emergency lighting, alarm sirens, or urgent phone notifications only fire while smoke is truly present.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your smoke sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Smoke detected**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple sensors are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor detects smoke, or **All** to pass only when every targeted sensor detects smoke.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_smoke_detected`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_smoke_detected
  target:
    entity_id: binary_sensor.kitchen_smoke
```

This passes when the kitchen smoke sensor is currently detecting smoke.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as detecting. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted sensor is unavailable.
- To check whether smoke is no longer detected, use [Smoke cleared](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send a reminder every 5 minutes while smoke is detected

During a smoke event, a single notification is easy to miss. This automation fires every 5 minutes and checks whether the kitchen smoke sensor is still detecting smoke. As long as smoke is present, you keep getting reminders so you stay up to date.

- **Trigger:** Time pattern: Every 5 minutes
- **Condition:** Air Quality: Smoke detected
- **Target:** Kitchen smoke sensor
- **Condition passes if:** Any
- **Action:** Notify: Send reminder notification

YAML example for repeating smoke reminders

****automation****

```
automation: |
  alias: "Repeating smoke reminder"
  triggers:
    - trigger: time_pattern
      minutes: "/5"
  conditions:
    - condition: air_quality.is_smoke_detected
      target:
        entity_id: binary_sensor.kitchen_smoke
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Smoke still detected"
        message: >
          The kitchen smoke sensor is still
          detecting smoke. Check the house.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is So2 Value

The **Sulphur dioxide value** condition passes when a sulphur dioxide (SO₂) sensor's reading meets a specific level. SO₂ is a sharp-smelling gas released by burning fossil fuels and volcanic activity. Elevated levels irritate the respiratory system and make outdoor air uncomfortable. This condition lets your automation respond to real readings, closing the windows and notifying you to stay indoors when SO₂ is too high, and letting fresh air back in once the reading drops.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Sulphur dioxide value**.
6. Under **Threshold type**, set the SO2 level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The sulphur dioxide level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_so2_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_so2_value
  target:
    entity_id: sensor.outdoor_so2
  options:
    threshold: 40
    behavior: any
```

This passes when the outdoor SO2 sensor reads at or above 40 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The sulphur dioxide level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- For related pollutant conditions, see [Nitrogen dioxide value](#) and [Ozone value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: warn you before heading outside in the morning

If you live near an industrial area, SO₂ readings sometimes spike overnight. This automation triggers at 7:00 AM and checks the outdoor SO₂ level. If the reading is at or above 40 µg/m³, you get a notification recommending you stay indoors. On mornings with clean air, you can leave without interruption.

- **Trigger:** Time: 07:00
- **Condition:** Air Quality: Sulphur dioxide value
- **Target:** Outdoor SO₂ sensor
- **Threshold type:** 40
- **Condition passes if:** Any
- **Action:** Notify: Send notification

YAML example for a morning SO₂ warning

****automation****

```
automation: |
  alias: "Morning SO2 warning"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: air_quality.is_so2_value
      target:
        entity_id: sensor.outdoor_so2
      options:
        threshold: 40
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "SO2 is high outside"
        message: >
          Outdoor SO2 is above 40. Consider
          staying indoors this morning.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Voc Ratio Value

The **Volatile organic compounds ratio value** condition passes when a VOC ratio sensor's reading meets a specific level. Some sensors express VOC levels as a ratio or index rather than an absolute concentration, which makes it easier to compare readings across different environments. This condition lets your automation act on that relative reading, for example sending a reminder to open a window in the bedroom when the VOC ratio climbs after a room has been closed up all day.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Volatile organic compounds ratio value**.
6. Under **Threshold type**, set the VOC ratio the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The VOC ratio the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_voc_ratio_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_voc_ratio_value
  target:
    entity_id: sensor.bedroom_voc_ratio
  options:
    threshold: 150
    behavior: any
```

This passes when the bedroom VOC ratio sensor reads at or above 150.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The VOC ratio the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- If your sensor reports VOC as an absolute concentration instead of a ratio, use [Volatile organic compounds value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: remind you to open a window when your morning alarm goes off

After a full night with the door closed, the bedroom VOC ratio creeps up from off-gassing furniture and bedding. This automation triggers when your morning alarm goes off and checks the current VOC ratio. If the reading is at or above 150, you get a reminder to crack a window. On nights when you already slept with the window open, the ratio stays low and no notification is sent.

- **Trigger:** Time: 07:00
- **Condition:** Air Quality: Volatile organic compounds ratio value
- **Target:** Bedroom VOC ratio sensor
- **Threshold type:** 150
- **Condition passes if:** Any
- **Action:** Notify: Send notification

YAML example for a morning VOC ratio reminder

```
**automation**
```

```
automation: |
  alias: "Morning VOC ratio reminder"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: air_quality.is_voc_ratio_value
      target:
        entity_id: sensor.bedroom_voc_ratio
      options:
        threshold: 150
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Time to air out the bedroom"
        message: >
          The bedroom VOC ratio is above 150.
          Opening a window for a few minutes helps.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Air Quality.Is Voc Value

The **Volatile organic compounds value** condition passes when a VOC sensor's reading meets a specific level. VOCs are invisible gases released by paints, cleaning supplies, new furniture, and even scented candles. You often notice them as that "new car" or "fresh paint" smell, and at higher concentrations they affect comfort and health. This condition lets your automation respond proportionally, turning on the ventilation fan only when VOC readings are genuinely elevated and leaving it off during a normal day so you save energy.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Volatile organic compounds value**.
6. Under **Threshold type**, set the VOC level the condition checks against.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The VOC level the sensor has to meet or exceed for the condition to pass. Condition passes if: description: When multiple sensors are targeted, controls how results combine. Pick **Any** to pass if at least one sensor meets the threshold, or **All** to pass only when every targeted sensor does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `air_quality.is_voc_value`. A basic example looks like this:

condition

```
condition: |
  condition: air_quality.is_voc_value
  target:
    entity_id: sensor.living_room_voc
  options:
    threshold: 300
    behavior: any
```

This passes when the living room VOC sensor reads at or above 300 µg/m³.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The VOC level the sensor has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Sensors that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Some sensors report VOCs as a ratio instead of an absolute concentration. For those sensors, use [Volatile organic compounds ratio value](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start the air purifier when you wake up only if VOC levels are high

Overnight, off-gassing from furniture, carpets, and cleaning products pushes VOC levels up in a closed room. This automation triggers at wake-up time and checks the living room VOC reading. If the level is at or above 300 $\mu\text{g}/\text{m}^3$, the air purifier turns on to clear the air before you start your day. On mornings when the air is already fresh, the purifier stays off and you save energy.

- **Trigger:** Time: 07:00
- **Condition:** Air Quality: Volatile organic compounds value
- **Target:** Living room VOC sensor
- **Threshold type:** 300
- **Condition passes if:** Any
- **Action:** Fan: Turn on (air purifier)

YAML example for starting the purifier on high VOCs at wake-up

****automation****

```
automation: |
  alias: "Morning purifier if VOCs are high"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: air_quality.is_voc_value
      target:
        entity_id: sensor.living_room_voc
      options:
        threshold: 300
        behavior: any
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.living_room_purifier
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Armed

The **Alarm is armed** condition passes when one or more alarm control panel entities are currently armed, regardless of the arming mode. Use it to ensure automations only run when the alarm is actually set, so your motion-triggered lights stay quiet when nobody is home to see them.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is armed**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been armed before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is armed, or **All** to pass only when every targeted alarm is armed. For at least: description: How long the alarm must have been armed before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_armed`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_armed
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently armed in any mode.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been armed before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- This condition matches any arming mode, including away, home, night, vacation, and custom bypass. If you need to check a specific mode, use the dedicated condition for that mode, such as [Alarm is armed away](#) or [Alarm is armed home](#).
- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as armed. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- To check the opposite state, use [Alarm is disarmed](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn off idle appliances at midnight if the alarm is armed

At midnight, power off standby appliances like the TV and coffee maker, but only when the alarm is armed. If nobody bothered to arm the alarm, someone is probably still awake and using them.

- **Trigger:** Time: 00:00
- **Condition:** Alarm is armed
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Switch: Turn off (TV, coffee maker)

YAML example for turning off appliances when armed at midnight

****automation****

```
automation: |
  alias: "Turn off appliances at midnight when armed"
  triggers:
    - trigger: time
      at: "00:00:00"
  conditions:
    - condition: alarm_control_panel.is_armed
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: switch.turn_off
      target:
        entity_id:
          - switch.tv_plug
          - switch.coffee_maker
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Armed Away

The **Alarm is armed away** condition passes when one or more alarm control panel entities are armed in away mode. Use it to stop automations that make no sense in an empty house, like skipping thermostat schedules, holding back welcome-home lighting, or pausing the robot vacuum. On the flip side, use it to *enable* away-only automations, like dropping the heating to a setback temperature to save energy while nobody is home.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is armed away**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been in this state before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is armed away, or **All** to pass only when every targeted alarm is armed away. For at least: description: How long the alarm must have been armed in away mode before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_armed_away`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_armed_away
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently armed in away mode.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been armed in away mode before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as armed away. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- If you want to check whether the alarm is armed in any mode (not just away), use `Alarm is armed`.
- To check the opposite state, use `Alarm is disarmed`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: lower the thermostat when a water leak is detected and nobody is home

When a water leak sensor detects a leak, turn the thermostat down to prevent further damage, but only if the alarm is armed in away mode. If someone is home, they should handle it themselves.

- **Trigger:** State: Water leak sensor detects a leak
- **Condition:** Alarm is armed away
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Climate: Set temperature

YAML example for lowering the thermostat on a leak when away

****automation****

```
automation: |
  alias: "Lower thermostat on leak when away"
  triggers:
    - trigger: state
      entity_id: binary_sensor.basement_water_leak
      to: "on"
  conditions:
    - condition: alarm_control_panel.is_armed_away
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: climate.set_temperature
      target:
        entity_id: climate.thermostat
      data:
        temperature: 15
```


Automation: skip the thermostat schedule while nobody is home

Your thermostat normally follows a comfort schedule throughout the day. When the alarm is armed away, there is no reason to heat or cool an empty house. This automation checks the away state before applying the scheduled temperature, saving energy until someone comes home.

- **Trigger:** Time: 07:00 (morning schedule)
- **Condition:** Alarm is *not* armed away
- **Action:** Set the thermostat to 21 degrees

YAML example for skipping the thermostat schedule when away

****automation****

```
automation: |
  alias: "Skip heating schedule when away"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: not
      conditions:
        - condition: alarm_control_panel.is_armed_away
          target:
            entity_id: alarm_control_panel.home_alarm
          options:
            behavior: any
  actions:
    - action: climate.set_temperature
      target:
        entity_id: climate.living_room
      data:
        temperature: 21
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Armed Home

The **Alarm is armed home** condition passes when one or more alarm control panel entities are armed in home mode. Use it to gate automations to your nighttime or stay-at-home routine, for example, keeping exterior motion lights active while interior motion sensors stay quiet.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is armed home**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been in this state before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is armed home, or **All** to pass only when every targeted alarm is armed home. For at least: description: How long the alarm must have been armed in home mode before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_armed_home`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_armed_home
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently armed in home mode.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been armed in home mode before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as armed home. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- If you want to check whether the alarm is armed in any mode (not just home), use `Alarm is armed`.
- To check the opposite state, use `Alarm is disarmed`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: activate exterior motion lights only when armed home

When an exterior motion sensor detects movement, turn on the porch and driveway lights, but only while the alarm is armed in home mode. During the day when the alarm is disarmed, you probably don't need those lights.

- **Trigger:** State: Exterior motion sensor detects motion
- **Condition:** Alarm is armed home
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Light: Turn on (porch, driveway)

YAML example for exterior motion lights when armed home

****automation****

```
automation: |
  alias: "Exterior motion lights when armed home"
  triggers:
    - trigger: state
      entity_id: binary_sensor.driveway_motion
      to: "on"
  conditions:
    - condition: alarm_control_panel.is_armed_home
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: light.turn_on
      target:
        entity_id:
          - light.porch
          - light.driveway
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Armed Night

The **Alarm is armed night** condition passes when one or more alarm control panel entities are armed in night mode. Use it to adjust how your home behaves while everyone is asleep. Hallway lights dim to a gentle glow instead of full brightness, the doorbell stops chiming through the speakers, and automations that would disturb sleep simply don't run.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is armed night**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been in this state before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is armed night, or **All** to pass only when every targeted alarm is armed night. For at least: description: How long the alarm must have been armed in night mode before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_armed_night`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_armed_night
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently armed in night mode.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been armed in night mode before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as armed night. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- If you want to check whether the alarm is armed in any mode (not just night), use `Alarm is armed`.
- To check the opposite state, use `Alarm is disarmed`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: dim the hallway nightlight when motion is detected at night

When the hallway motion sensor detects movement, turn on the hallway light at 10% brightness, but only while the alarm is armed in night mode. During the day, you want full brightness instead.

- **Trigger:** State: Hallway motion sensor detects motion
- **Condition:** Alarm is armed night
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Light: Turn on at 10% brightness

YAML example for a dim hallway nightlight when armed night

****automation****

```
automation: |
  alias: "Dim hallway light on motion at night"
  triggers:
    - trigger: state
      entity_id: binary_sensor.hallway_motion
      to: "on"
  conditions:
    - condition: alarm_control_panel.is_armed_night
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: light.turn_on
      target:
        entity_id: light.hallway
      data:
        brightness_pct: 10
```

Automation: silence the doorbell while the house is asleep

The doorbell normally announces visitors through every speaker, but nobody wants that at 2 AM. Add a night-mode check so the speakers stay silent overnight and you get a quiet phone notification instead.

- **Trigger:** State: Doorbell pressed
- **Condition:** Alarm is armed night
- **Action:** Send a quiet phone notification (skip the speaker announcement)

YAML example for silencing the doorbell at night

****automation****

```
automation: |
  alias: "Quiet doorbell at night"
  triggers:
    - trigger: state
      entity_id: binary_sensor.doorbell
      to: "on"
  conditions:
    - condition: alarm_control_panel.is_armed_night
      target:
        entity_id: alarm_control_panel.home_alarm
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Doorbell"
        message: "Someone is at the door."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Armed Vacation

The **Alarm is armed vacation** condition passes when one or more alarm control panel entities are armed in vacation mode. Use it to skip daily routines that make no sense while you are away, like stopping the morning wake-up automation from turning on lights and heating in an empty house. You could also use it the other way around: only run vacation-specific automations (like randomly toggling lights to simulate occupancy) while the alarm is in vacation mode.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is armed vacation**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been in this state before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is armed vacation, or **All** to pass only when every targeted alarm is armed vacation. For at least: description: How long the alarm must have been armed in vacation mode before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_armed_vacation`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_armed_vacation
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently armed in vacation mode.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been armed in vacation mode before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as armed vacation. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- If you want to check whether the alarm is armed in any mode (not just vacation), use `Alarm is armed`.
- To check the opposite state, use `Alarm is disarmed`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: toggle a lamp in the evening while on vacation

Every evening, toggle the living room lamp on or off to make the house look lived-in, but only while the alarm is armed in vacation mode. When you're home, the lamp follows your normal routine instead.

Don't forget to pair it with an automation to turn the lamp off, or you'll come back from vacation to a high energy bill!

- **Trigger:** Time: 20:30
- **Condition:** Alarm is armed vacation
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Light: Toggle

YAML example for an evening lamp toggle on vacation

****automation****

```
automation: |
  alias: "Simulate occupancy when on vacation"
  triggers:
    - trigger: time
      at: "20:30:00"
  conditions:
    - condition: alarm_control_panel.is_armed_vacation
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: light.toggle
      target:
        entity_id: light.living_room_lamp
```

Automation: skip the morning routine while on vacation

Your morning routine turns on the lights, starts the coffee maker, and bumps the heating. None of that makes sense in an empty house. Add a "not armed vacation" check so the routine only runs when you are actually home.

- **Trigger:** Time: 06:30 on weekdays
- **Condition:** Alarm is *not* armed vacation
- **Action:** Run the morning routine script

YAML example for skipping the morning routine on vacation

****automation****

```
automation: |
  alias: "Skip morning routine on vacation"
  triggers:
    - trigger: time
      at: "06:30:00"
  conditions:
    - condition: not
      conditions:
        - condition: alarm_control_panel.is_armed_vacation
          target:
            entity_id: alarm_control_panel.home_alarm
          options:
            behavior: any
  actions:
    - action: script.morning_routine
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Disarmed

The **Alarm is disarmed** condition passes when one or more alarm control panel entities are currently disarmed. Use this to make sure the welcome-home routine only runs when the alarm has already been turned off, so lights and music stay quiet while the alarm is still active.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is disarmed**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been disarmed before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is disarmed, or **All** to pass only when every targeted alarm is disarmed. For at least: description: How long the alarm must have been disarmed before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_disarmed`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_disarmed
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently disarmed.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been disarmed before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as disarmed. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- To check whether the alarm is armed in any mode, use [Alarm is armed](#).
- To check whether the alarm is actively triggered, use [Alarm is triggered](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: run the welcome-home routine when someone arrives

When a person arrives home, turn on the entryway lights and start playing music, but only if the alarm is already disarmed. If the alarm is still armed, the person probably hasn't entered yet.

- **Trigger:** Person: Someone arrives home
- **Condition:** Alarm is disarmed
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Light: Turn on, Media player: Play media

YAML example for a welcome-home routine gated on disarmed alarm

****automation****

```
automation: |
  alias: "Welcome home when alarm disarmed"
  triggers:
    - trigger: state
      entity_id: person.jane
      to: home
  conditions:
    - condition: alarm_control_panel.is_disarmed
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: light.turn_on
      target:
        entity_id: light.entryway
    - action: media_player.play_media
      target:
        entity_id: media_player.living_room
      data:
        media_content_id: "media-source://radio/favorite"
        media_content_type: music
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Alarm Control Panel.Is Triggered

The **Alarm is triggered** condition passes when one or more alarm control panel entities are in a triggered state. Use it to gate your emergency response so sirens and notifications only fire while the alarm is genuinely going off, preventing false follow-up actions after the situation has been resolved.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Alarm is triggered**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must have been in the triggered state before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple alarm panels are targeted, controls how results combine. Pick **Any** to pass if at least one targeted alarm is triggered, or **All** to pass only when every targeted alarm is triggered. For at least: description: How long the alarm must have been in the triggered state before the condition passes. Set to zero to pass immediately.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `alarm_control_panel.is_triggered`. A basic example looks like this:

condition

```
condition: |
  condition: alarm_control_panel.is_triggered
  target:
    entity_id: alarm_control_panel.hallway
```

This passes when the hallway alarm panel is currently in a triggered state.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple alarm panels are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > Duration the alarm must have been in the triggered state before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Alarm panels that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as triggered. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted alarm is unavailable.
- To check whether the alarm is armed, use `Alarm is armed`.
- To check the opposite state, use `Alarm is disarmed`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send an emergency notification when a panic button is pressed

When a panic button is pressed, send an urgent push notification with the alarm status, but only if the alarm is genuinely triggered. This prevents false alerts from accidental button presses when the alarm is in a normal state.

- **Trigger:** State: Panic button pressed
- **Condition:** Alarm is triggered
- **Target:** Hallway alarm panel
- **Condition passes if:** Any
- **Action:** Notify: Send a mobile notification

YAML example for an emergency notification gated on triggered alarm

****automation****

```
automation: |
  alias: "Emergency notification on panic button"
  triggers:
    - trigger: state
      entity_id: input_button.panic
  conditions:
    - condition: alarm_control_panel.is_triggered
      target:
        entity_id: alarm_control_panel.hallway
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Alarm triggered"
        message: >
          The alarm has been triggered.
          Check the cameras immediately.
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Calendar.Is Event Active

The **Calendar event is active** condition passes when a calendar entity has an active event. Use it to gate an automation so it only runs when a specific calendar event has started and not yet ended.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Calendar event is active**.
5. Under **Targets**, select a calendar entity, an area, a floor, or a label.
6. If you selected more than one target, under **Condition passes if**, pick **Any** or **All**.
7. Under **For at least**, set how long the event must be active before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple calendars are targeted, controls how results combine. Pick **Any** to pass if at least one targeted calendar has an active event, or **All** to pass only when every calendar has an active event. For at least: description: How long the event must be active before the condition passes. The default is hours, minutes and seconds.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `calendar.is_event_active`. A basic example looks like this:

condition

```
condition: |
  condition: calendar.is_event_active
  target:
    entity_id: calendar.my_calendar
```

This passes when an event of `calendar.my_calendar` is active.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple calendars are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the event must be active before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send reminder for a sunset run if no calendar event is active

Half an hour before sunset, if there is not an active event in the calendar, this automation sends a notification to the mobile phone with a reminder to go for a sunset run.

- **Trigger:** Sun
- **Condition:** Calendar event is active
- **Blocks:** Not
- **Action:** Send notification via mobile_app_phone

YAML example for sending reminder for sunset run if no calendar event is active

****automation****

```
automation: |
  alias: "Send reminder for a sunset run if no calendar event is active"
  triggers:
    - trigger: sun
      event: sunset
      offset: "-00:30:00"
  conditions:
    - condition: not
      conditions:
        - condition: calendar.is_event_active
          target:
            entity_id: calendar.my_calendar
  actions:
    - action: notify.mobile_app_phone
      data:
        message: Let's go for a sunset run!
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Door.Is Closed

The **Door is closed** condition passes when one or more targeted doors are currently closed. Use it when an automation should continue only after a door is shut.

This condition is useful for safety checks and routines that depend on a closed door, like starting a robot vacuum only after the patio door is shut or arming the house only after every entry point is closed.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Door is closed**.
5. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your door is in, like your entryway or garage. You can also select a floor, a device, a specific entity, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the door must have stayed closed before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple doors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted door is closed, or **All** to pass only when every targeted door is closed. For at least: description: How long the door must have stayed closed before the condition passes.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `door.is_closed`. A basic example looks like this:

condition

```
condition: |
  condition: door.is_closed
  target:
    entity_id: binary_sensor.front_door
```

This passes when `binary_sensor.front_door` is currently closed.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple doors are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the door must have stayed closed before the condition passes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- This condition works with door contact sensors and door covers, like garage doors, as long as they use the `door` device class.
- Entities in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- With **Any**, the condition passes if at least one available targeted door is closed.
- With **All**, the condition passes only if every available targeted door is closed. If every targeted door is `unavailable` or `unknown`, **All** passes and **Any** fails.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start robot vacuum only after patio door has been closed for 10 minutes

If pets go in and out through the patio door, you may not want the robot vacuum to start while that door is still being used. This automation waits for a scheduled time, then checks that the patio door has been closed for at least 10 minutes before starting the vacuum.

- **Trigger:** Time
- **Condition:** Door is closed
- **Target:** Patio door
- **For at least:** 00:10:00
- **Action:** Vacuum: Start

YAML example for delaying vacuum cleaning until the patio door is shut

****automation****

```
automation: |
  alias: "Start vacuum after patio door has been closed"
  triggers:
    - trigger: time
      at: "10:00:00"
  conditions:
    - condition: door.is_closed
      target:
        entity_id: binary_sensor.patio_door
      options:
        behavior: any
        for: "00:10:00"
  actions:
    - action: vacuum.start
      target:
        entity_id: vacuum.downstairs
```

Automation: arm the house at night only when every exterior door is closed

At bedtime, this automation arms the house only if the front door, back door, and garage door are all closed. That keeps you from arming the house while an entry point is still open.

- **Trigger:** Time
- **Condition:** Door is closed
- **Target:** Front door, back door, and garage door
- **Condition passes if:** All
- **Action:** Alarm control panel: Arm away

YAML example for arming only after all doors are closed

****automation****

```
automation: |
  alias: "Arm house only when all doors are closed"
  triggers:
    - trigger: time
      at: "23:00:00"
  conditions:
    - condition: door.is_closed
      target:
        entity_id:
          - binary_sensor.front_door
          - binary_sensor.back_door
          - cover.garage_door
      options:
        behavior: all
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home_alarm
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Door.Is Open

The **Door is open** condition passes when one or more targeted doors are currently open. Use it when an automation should continue only if a door is still open at the moment the automation runs.

This condition is useful for reminders, security checks, and routines that should stop or warn you when a front door, patio door, or garage door has not been closed yet.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Door is open**.
5. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your door is in, like your entryway or garage. You can also select a floor, a device, a specific entity, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the door must have stayed open before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple doors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted door is open, or **All** to pass only when every targeted door is open. For at least: description: How long the door must have stayed open before the condition passes.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `door.is_open`. A basic example looks like this:

condition

```
condition: |
  condition: door.is_open
  target:
    entity_id: binary_sensor.patio_door
```

This passes when `binary_sensor.patio_door` is currently open.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple doors are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the door must have stayed open before the condition passes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- This condition works with door contact sensors and door covers, like garage doors, as long as they use the `door` device class.
- Entities in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- With **Any**, the condition passes if at least one available targeted door is open.
- With **All**, the condition passes only if every available targeted door is open. If every targeted door is `unavailable` or `unknown`, **All** passes and **Any** fails.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: warn you if any exterior door is still open at bedtime

If you have created a helper to mark bedtime, this automation can use it to check whether any exterior door is still open. If one is, Home Assistant sends a notification instead of letting you discover it later.

- **Trigger:** Bedtime helper turns on
- **Condition:** Door is open
- **Target:** Front door, patio door, and garage door
- **Condition passes if:** Any
- **Action:** Notifications: Send a notification via `mobile_app_<name>`

YAML example for a bedtime door check

****automation****

```
automation: |
  alias: "Warn if a door is open at bedtime"
  triggers:
    - trigger: state
      entity_id: input_boolean.bedtime_mode
      to: "on"
  conditions:
    - condition: door.is_open
      target:
        entity_id:
          - binary_sensor.front_door
          - binary_sensor.patio_door
          - cover.garage_door
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "A door is still open"
        message: "Check the front door, patio door, or garage door before bed"
```

Automation: alert you if the garage door is still open when you leave home

When the last person leaves home, this automation checks whether the garage door is still open. If it is, Home Assistant sends an alert so you can close it before getting too far away.

- **Trigger:** Person leaves home zone
- **Condition:** Door is open
- **Target:** Garage door
- **Condition passes if:** Any
- **Action:** Notifications: Send a notification via `mobile_app_<name>`

YAML example for checking the garage door when leaving

****automation****

```
automation: |
  alias: "Warn if garage door is open when leaving home"
  triggers:
    - trigger: zone
      entity_id: person.frenck
      zone: zone.home
      event: leave
  conditions:
    - condition: door.is_open
      target:
        entity_id: cover.garage_door
      options:
        behavior: any
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Garage door is open"
        message: "The garage door is still open, and you just left home."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Fan.Is Off

The **Fan is off** condition is useful when an automation should continue only if a fan is not running. Use it to avoid repeated stop commands, prevent unnecessary noise at night, or start a fan only when it is currently off.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the fan you want to check.
You can also select an area, a floor, a device, or a label.
5. From the conditions shown for that target, select **Fan is off**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, set how long the fan must have been off.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple fans are targeted, controls whether **Any** targeted fan must be off or **All** targeted fans must be off. required: false For at least: description: How long the fan must have been off for the condition to pass. required: false

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `fan.is_off`. A basic example looks like this:

condition

```
condition: |
  condition: fan.is_off
  target:
    entity_id: fan.living_room
  options:
    behavior: any
    for: "00:30:00"
```

This passes when `fan.living_room` has been off for 30 minutes.

Options in YAML

{% options_yaml %} behavior: description: When multiple fans are targeted, controls whether `any` or `all` targeted fans must be off. required: false type: string default: any for: description: How long the fan must have been off for the condition to pass. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- A fan in the `unknown` or `unavailable` state does not count as off.
- With **All**, every targeted fan must match. With **Any**, one matching fan is enough.
- To check for the opposite state, use `Fan is on`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start the bedroom fan only if it is currently off

This avoids sending the same command again when a bedtime automation runs.

- **Trigger:** Time: 22:00
- **Condition:** Fan is off
- **Target:** Bedroom fan
- **Condition passes if:** Any
- **For at least:** 00:00:00
- **Action:** Turn on fan

YAML example for a bedtime fan start

****automation****

```
automation: |
  alias: "Start bedroom fan at bedtime if needed"
  triggers:
    - trigger: time
      at: "22:00:00"
  conditions:
    - condition: fan.is_off
      target:
        entity_id: fan.bedroom
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom
```

Automation: close the patio door only after the fan has been off

If you use a whole-room fan in the evening, you may want to close the patio door only after the fan has been off for a while.

- **Trigger:** Time: 23:00
- **Condition:** Fan is off
- **Target:** Living room fan
- **Condition passes if:** Any
- **For at least:** 00:15:00
- **Action:** Close cover

YAML example for closing a patio door cover

****automation****

```
automation: |
  alias: "Close patio cover after fan stops"
  triggers:
    - trigger: time
      at: "23:00:00"
  conditions:
    - condition: fan.is_off
      target:
        entity_id: fan.living_room
      options:
        behavior: any
        for: "00:15:00"
  actions:
    - action: cover.close_cover
      target:
        entity_id: cover.patio_door
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Fan.Is On

The **Fan is on** condition is useful when an automation should continue only if a fan is already running. Use it to avoid duplicate actions, wait for ventilation before doing something else, or branch your automation based on whether a room already has airflow.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the fan you want to check.
You can also select an area, a floor, a device, or a label.
5. From the conditions shown for that target, select **Fan is on**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, set how long the fan must have been on.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple fans are targeted, controls whether **Any** targeted fan must be on or **All** targeted fans must be on. required: false For at least: description: How long the fan must have been on for the condition to pass. required: false

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `fan.is_on`. A basic example looks like this:

condition

```
condition: |
  condition: fan.is_on
  target:
    entity_id: fan.office
  options:
    behavior: any
    for: "00:15:00"
```

This passes when `fan.office` has been on for 15 minutes.

Options in YAML

{% options_yaml %} behavior: description: When multiple fans are targeted, controls whether `any` or `all` targeted fans must be on. required: false type: string default: any for: description: How long the fan must have been on for the condition to pass. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- A fan in the `unknown` or `unavailable` state does not count as on.
- With **All**, every targeted fan must match. With **Any**, one matching fan is enough.
- To check for the opposite state, use `Fan is off`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send a reminder if a window opens while the fan is running

If a window opens while the bedroom fan is running, you may want a reminder so you can decide whether to keep using the fan.

- **Trigger:** State: Window changes to open
- **Condition:** Fan is on
- **Target:** Bedroom fan
- **Condition passes if:** Any
- **For at least:** 00:00:00
- **Action:** Send a notification via mobile_app_phone

YAML example for a window and fan reminder

****automation****

```
automation: |
  alias: "Bedroom window opened while fan runs"
  triggers:
    - trigger: state
      entity_id: binary_sensor.bedroom_window
      to: "on"
  conditions:
    - condition: fan.is_on
      target:
        entity_id: fan.bedroom
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: notify.mobile_app_phone
      data:
        message: "The bedroom window is open and the fan is still running."
```

Automation: lower the blinds only after the office fan has been running

If the office is already warm enough for the fan to be running, you can also lower the blinds when the sun gets strong.

- **Trigger:** Sun: Above horizon
- **Condition:** Fan is on
- **Target:** Office fan
- **Condition passes if:** Any
- **For at least:** 00:10:00
- **Action:** Close cover

YAML example for an office shade routine

****automation****

```
automation: |
  alias: "Lower office blinds when fan is already running"
  triggers:
    - trigger: sun
      event: sunrise
  conditions:
    - condition: fan.is_on
      target:
        entity_id: fan.office
      options:
        behavior: any
        for: "00:10:00"
  actions:
    - action: cover.close_cover
      target:
        entity_id: cover.office_blinds
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Gate.Is Closed

The **Gate is closed** condition passes when one or more targeted gates are currently closed. Use it when an automation should continue only after a gate is shut.

This condition is useful for safety checks and routines that should run only when access is secured, like arming your home or starting irrigation.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Gate is closed**.
5. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your gate is in, like your driveway or courtyard. You can also select a floor, a device, a specific entity, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the gate must have stayed closed before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple gates are targeted, controls how results combine. Pick **Any** to pass if at least one targeted gate is closed, or **All** to pass only when every targeted gate is closed. For at least: description: How long the gate must have stayed closed before the condition passes.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `gate.is_closed`. A basic example looks like this:

condition

```
condition: |
  condition: gate.is_closed
  target:
    entity_id: cover.driveway_gate
```

This passes when `cover.driveway_gate` is currently closed.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple gates are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the gate must have stayed closed before the condition passes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- This condition works only with `cover` entities that use the `gate` device class.
- Entities in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- With **Any**, the condition passes if at least one available targeted gate is closed.
- With **All**, the condition passes only if every available targeted gate is closed. If every targeted gate is `unavailable` or `unknown`, **All** passes and **Any** fails.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: arm the house at night only when every gate is closed

At bedtime, this automation arms your home only if the driveway gate and courtyard gate are both closed. That keeps you from arming the house while access is still open.

- **Trigger:** Time
- **Condition:** Gate is closed
- **Target:** Driveway gate and courtyard gate
- **Condition passes if:** All
- **Action:** Alarm control panel: Arm away

YAML example for arming only after all gates are closed

****automation****

```
automation: |
  alias: "Arm the house only when all gates are closed"
  triggers:
    - trigger: time
      at: "23:00:00"
  conditions:
    - condition: gate.is_closed
      target:
        entity_id:
          - cover.driveway_gate
          - cover.courtyard_gate
      options:
        behavior: all
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home_alarm
```

Automation: start irrigation only after the side gate has been closed for 5 minutes

If people often walk through the side gate in the morning, this automation waits until the gate has stayed closed for 5 minutes before it turns on irrigation. That helps avoid spraying someone who is still using the path.

- **Trigger:** Time
- **Condition:** Gate is closed
- **Target:** Side gate
- **For at least:** 00:05:00
- **Action:** Switch: Turn on

YAML example for waiting to start irrigation

****automation****

```
automation: |
  alias: "Start irrigation only after the side gate is closed"
  triggers:
    - trigger: time
      at: "06:00:00"
  conditions:
    - condition: gate.is_closed
      target:
        entity_id: cover.side_gate
      options:
        behavior: any
        for: "00:05:00"
  actions:
    - action: switch.turn_on
      target:
        entity_id: switch.garden_irrigation
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Gate.Is Open

The **Gate is open** condition passes when one or more targeted gates are currently open. Use it when an automation should continue only if a gate is still open at the moment the automation runs.

This condition is useful for reminders, security checks, and routines that should warn you before you leave the property or settle in for the night.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Gate is open**.
5. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your gate is in, like your driveway or courtyard. You can also select a floor, a device, a specific entity, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the gate must have stayed open before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple gates are targeted, controls how results combine. Pick **Any** to pass if at least one targeted gate is open, or **All** to pass only when every targeted gate is open. For at least: description: How long the gate must have stayed open before the condition passes.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `gate.is_open`. A basic example looks like this:

condition

```
condition: |
  condition: gate.is_open
  target:
    entity_id: cover.driveway_gate
```

This passes when `cover.driveway_gate` is currently open.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple gates are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the gate must have stayed open before the condition passes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- This condition works only with `cover` entities that use the `gate` device class.
- Entities in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- With **Any**, the condition passes if at least one available targeted gate is open.
- With **All**, the condition passes only if every available targeted gate is open. If every targeted gate is `unavailable` or `unknown`, **All** passes and **Any** fails.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: remind you at sunset if the driveway gate is still open

At sunset, this automation checks whether the driveway gate is still open. If it is, Home Assistant sends a reminder so you can secure the property before it gets dark.

- **Trigger:** Sun: Sunset
- **Condition:** Gate is open
- **Target:** Driveway gate
- **Condition passes if:** Any
- **Action:** Send a notification message
- **Target:** Mobile app

YAML example for an open-gate reminder at sunset

****automation****

```
automation: |
  alias: "Remind me if the driveway gate is open at sunset"
  triggers:
    - trigger: sun
      event: sunset
  conditions:
    - condition: gate.is_open
      target:
        entity_id: cover.driveway_gate
      options:
        behavior: any
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_mobile_app
      data:
        title: "Driveway gate is open"
        message: "The driveway gate is still open after sunset."
```

Automation: warn you if any gate is open when you leave home

When you leave home, this automation checks whether any targeted gate is still open. If one is, you get a warning before you get too far away to do anything about it.

- **Trigger:** Person leaves home zone
- **Condition:** Gate is open
- **Target:** Driveway gate and courtyard gate
- **Condition passes if:** Any
- **Action:** Send a notification message
- **Target:** Mobile app

YAML example for checking gates when you leave home

****automation****

```
automation: |
  alias: "Warn me if a gate is open when I leave home"
  triggers:
    - trigger: zone
      entity_id: person.frenck
      zone: zone.home
      event: leave
  conditions:
    - condition: gate.is_open
      target:
        entity_id:
          - cover.driveway_gate
          - cover.courtyard_gate
      options:
        behavior: any
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_mobile_app
      data:
        title: "A gate is still open"
        message: "The driveway gate or courtyard gate is still open."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidifier.Is Drying

The **Humidifier is drying** condition passes when a humidifier entity is actively removing moisture from the air. This typically applies to dehumidifiers and devices set to a dehumidification mode. Like a humidifier that idles once it reaches its target, a dehumidifier pauses once the air is dry enough and resumes when humidity rises again. Use **Humidifier is drying** to confirm the device is in an active drying cycle, not just powered on.

When you target more than one humidifier, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted device to be actively drying, or demand that all of them are.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier is drying** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your dehumidifier is in (like your basement or bathroom). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Humidifier is drying**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple devices are targeted.
7. Under **For at least**, set how long the humidifier must have been actively drying before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple humidifiers are targeted, controls how results combine. Pick **Any** to pass if at least one targeted device is actively drying, or **All** to pass only when every targeted device is actively drying. Default is **Any**. For at least: description: How long the humidifier must have been continuously drying before the condition passes. Default is (passes immediately).

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier is drying** is referred to as `humidifier.is_drying`. A basic example looks like this:

condition

```
condition: |
  condition: humidifier.is_drying
  target:
    entity_id: humidifier.basement
```

This passes when the basement dehumidifier is actively removing moisture from the air.

Options in YAML

{% options_yaml %} behavior: description: > When multiple humidifiers are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the humidifier must have been continuously drying before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- A dehumidifier can be on but not drying if it has already reached its target humidity and is idling. Use [Humidifier is on](#) if you only care about whether the device is powered.
- Humidifiers that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as actively drying. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted device is unavailable.
- This condition applies to devices in a drying cycle. For humidifiers that are actively adding moisture, use [Humidifier is humidifying](#) instead.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: open a window vent only while the dehumidifier is running

When the basement dehumidifier is actively removing moisture, open the basement vent to help air circulation. This keeps the vent open during active drying cycles and avoids unnecessary operation when the dehumidifier is idling.

- **Trigger:** Time pattern: Every 5 minutes
- **Condition:** Humidifier is drying
- **Target:** Basement dehumidifier
- **Condition passes if:** Any
- **Action:** Cover: Open cover

YAML example for opening a vent while the dehumidifier is active

****automation****

```
automation: |
  alias: "Open vent while dehumidifier is active"
  triggers:
    - trigger: time_pattern
      minutes: "/5"
  conditions:
    - condition: humidifier.is_drying
      target:
        entity_id: humidifier.basement
      options:
        behavior: any
  actions:
    - action: cover.open_cover
      target:
        entity_id: cover.basement_vent
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidifier.Is Humidifying

The **Humidifier is humidifying** condition passes when a humidifier entity is actively adding moisture to the air. A humidifier that is switched on does not necessarily run continuously. It pauses once it reaches its target humidity and resumes only when the air dries out again. Use **Humidifier is humidifying** to confirm the device is in an active cycle, rather than just powered on and idle.

When you target more than one humidifier, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted humidifier to be actively humidifying, or demand that all of them are.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier is humidifying** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Humidifier is humidifying**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple humidifiers are targeted.
7. Under **For at least**, set how long the humidifier must have been actively humidifying before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple humidifiers are targeted, controls how results combine. Pick **Any** to pass if at least one targeted humidifier is actively humidifying, or **All** to pass only when every targeted humidifier is actively humidifying. Default is **Any**. For at least: description: How long the humidifier must have been continuously humidifying before the condition passes. Default is (passes immediately).

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier is humidifying** is referred to as `humidifier.is_humidifying`. A basic example looks like this:

condition

```
condition: |
  condition: humidifier.is_humidifying
  target:
    entity_id: humidifier.bedroom
```

This passes when the bedroom humidifier is actively adding moisture to the air.

Options in YAML

{% options_yaml %} behavior: description: > When multiple humidifiers are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the humidifier must have been continuously humidifying before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- A humidifier can be on but not humidifying if it has already reached its target humidity and is idling. Use [Humidifier is on](#) if you only care about whether the device is powered.
- Humidifiers that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as actively humidifying. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted humidifier is unavailable.
- This condition only applies to devices in a humidification cycle. For dehumidifiers that are actively removing moisture, use [Humidifier is drying](#) instead.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: run an air purifier only while the humidifier is active

When the bedroom humidifier is confirmed to be in an active humidification cycle, turn on the air purifier alongside it. This keeps both devices running together during active cycles without running the purifier while the humidifier idles.

- **Trigger:** Time pattern: Every 5 minutes
- **Condition:** Humidifier is humidifying
- **Target:** Bedroom humidifier
- **Condition passes if:** Any
- **Action:** Fan: Turn on

YAML example for running a purifier alongside the humidifier

****automation****

```
automation: |
  alias: "Run purifier while humidifier is active"
  triggers:
    - trigger: time_pattern
      minutes: "/5"
  conditions:
    - condition: humidifier.is_humidifying
      target:
        entity_id: humidifier.bedroom
      options:
        behavior: any
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom_purifier
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidifier.Is Mode

The **Humidifier is in mode** condition passes when a humidifier entity is set to a specific operating mode. Modes are device-specific and typically include options like **Normal**, **Eco**, **Sleep**, **Auto**, or **Baby**, though the exact modes available depend on your device. Use **Humidifier is in mode** to have an automation run only when the humidifier is set to a specific mode. For example, to skip a scene change if the humidifier is already in sleep mode.

When you target more than one humidifier, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted humidifier to be in the selected mode, or demand that all of them are.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier is in mode** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Humidifier is in mode**.
6. Under **Mode**, select one or more modes to check for. Only modes available on the targeted device are shown.
7. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple humidifiers are targeted.
8. Under **For at least**, set how long the humidifier must have been in the selected mode before the condition passes. Leave it at zero to pass immediately.
9. Select **Save**.

Options in the UI

{% options_ui %} Mode: description: The mode or modes to check for. Only the modes available on the targeted device are shown. Typical modes include **Normal**, **Eco**, **Not home**, **Boost**, **Comfort**, **Home**, **Sleep**, **Auto**, and **Baby**, though the exact modes depend on your device. Condition passes if: description: When multiple humidifiers are targeted, controls how results combine. Pick **Any** to pass if at least one targeted humidifier is in the selected mode, or **All** to pass only when every targeted humidifier is in the selected mode. Default is **Any**. For at least: description: How long the humidifier must have been continuously in the selected mode before the condition passes. Default is (passes immediately).

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier is in mode** is referred to as `humidifier.is_mode`. A basic example looks like this:

condition

```
condition: |
  condition: humidifier.is_mode
  target:
    entity_id: humidifier.bedroom
  options:
    mode: "sleep"
```

This passes when the bedroom humidifier is currently set to sleep mode.

To check for any one of several modes:

condition

```
condition: |
  condition: humidifier.is_mode
  target:
    entity_id: humidifier.bedroom
  options:
    mode:
      - "sleep"
      - "eco"
```

Options in YAML

{% options_yaml %} mode: description: > The mode or modes to check for. Accepts a single mode string or a list of modes. Typical modes include `normal`, `eco`, `away`, `boost`, `comfort`, `home`, `sleep`, `auto`, and `baby`, though the exact modes available depend on your device. required: false type: [string, list] behavior: description: > When multiple humidifiers are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the humidifier must have been continuously in the selected mode before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- The available modes depend entirely on the device. Check your humidifier's documentation or the entity's attributes in Home Assistant to see which modes are supported.
- This condition checks the mode the humidifier is *currently set to*, not whether it is actively running in that mode.
- Humidifiers that do not support modes will never pass this condition. To check general on/off state instead, use [Humidifier is on](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: skip the night scene if sleep mode is already active

When you press the bedtime button, activate the night scene. But if the bedroom humidifier is already in sleep mode, skip the scene because the room is clearly already set up for rest.

- **Trigger:** State: Bedtime button pressed
- **Condition:** Humidifier is in mode (negated: not in sleep mode)
- **Target:** Bedroom humidifier
- **Condition passes if:** Any
- **Action:** Scene: Activate night scene

YAML example for skipping the night scene in sleep mode

****automation****

```
automation: |
  alias: "Activate night scene if not in sleep mode"
  triggers:
    - trigger: state
      entity_id: input_button.bedtime
  conditions:
    - condition: not
      conditions:
        - condition: humidifier.is_mode
          target:
            entity_id: humidifier.bedroom
          options:
            mode: "sleep"
            behavior: any
  actions:
    - action: scene.turn_on
      target:
        entity_id: scene.bedroom_night
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidifier.Is Off

The **Humidifier is off** condition passes when a humidifier entity is currently switched off. Use it to prevent automations from running actions that only make sense on a powered-on device, or to send a reminder when a humidifier that should be running has been left off.

When you target more than one humidifier, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted humidifier to be off, or demand that all of them are.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier is off** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Humidifier is off**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple humidifiers are targeted.
7. Under **For at least**, set how long the humidifier must have been off before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple humidifiers are targeted, controls how results combine. Pick **Any** to pass if at least one targeted humidifier is off, or **All** to pass only when every targeted humidifier is off. Default is **Any**. For at least: description: How long the humidifier must have been continuously off before the condition passes. Default is (passes immediately).

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier is off** is referred to as `humidifier.is_off`. A basic example looks like this:

condition

```
condition: |
  condition: humidifier.is_off
  target:
    entity_id: humidifier.bedroom
```

This passes when the bedroom humidifier is currently off.

Options in YAML

{% options_yaml %} behavior: description: > When multiple humidifiers are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the humidifier must have been continuously off before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Humidifiers that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as off. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted humidifier is unavailable.
- To gate an automation on the humidifier being on instead, use [Humidifier is on](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send a bedtime reminder if the humidifier is still off

At 22:30 on weeknights, check whether the bedroom humidifier has been left off. If so, send a reminder so you can switch it on before you sleep.

- **Trigger:** Time: 22:30
- **Condition:** Day of the week is Monday to Friday
- **Condition:** Humidifier is off
- **Target:** Bedroom humidifier
- **Condition passes if:** Any
- **Action:** Send a notification message
- **Target:** My device (`notify.my_device`)

YAML example for a bedtime humidifier reminder

```
**automation**
```



```
automation: |
  alias: "Bedtime reminder if humidifier is off"
  triggers:
    - trigger: time
      at: "22:30:00"
  conditions:
    - condition: time
      weekday:
        - mon
        - tue
        - wed
        - thu
        - fri
    - condition: humidifier.is_off
      target:
        entity_id: humidifier.bedroom
      options:
        behavior: any
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: >
          The bedroom humidifier is off.
          Switch it on before you sleep.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidifier.Is On

The **Humidifier is on** condition passes when a humidifier entity is currently switched on. For example, you can adjust the target humidity only when the humidifier is ready to act on the change.

When you target more than one humidifier, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted humidifier to be on, or demand that all of them are.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier is on** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Humidifier is on**.
6. If you targeted more than one humidifier, an extra option appears: under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check combines results.
7. Under **For at least**, set how long the humidifier must have been on before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: Only shown when multiple humidifiers are targeted. Controls how results combine. Pick **Any** to pass if at least one targeted humidifier is on, or **All** to pass only when every targeted humidifier is on. Default is **Any**. For at least: description: How long the humidifier must have been continuously on before the condition passes. Useful to confirm the device has been running for a meaningful amount of time before taking action. Default is (passes immediately).

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier is on** is referred to as `humidifier.is_on`. A basic example looks like this:

condition

```
condition: |
  condition: humidifier.is_on
  target:
    entity_id: humidifier.bedroom
```

This passes when the bedroom humidifier is currently on.

Options in YAML

{% options_yaml %} behavior: description: > Only relevant when multiple humidifiers are targeted. Controls how results combine. Use `any` to pass if at least one targeted humidifier is on, or `all` to pass only when every targeted humidifier is on. required: false type: string default: any for: description: > How long the humidifier must have been continuously on before the condition passes. Useful to confirm the device has been running for a meaningful amount of time before taking action. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Being "on" does not mean the humidifier is actively running. A device that is on may idle once it reaches its target humidity and only resume when the air dries out. To check for an active humidification cycle, use [Humidifier is humidifying](#).
- Humidifiers that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as on. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted humidifier is unavailable.
- To gate an automation on the humidifier being off instead, use [Humidifier is off](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: adjust target humidity only when the humidifier is on

At 22:00, lower the bedroom humidity target for sleeping, but only if the humidifier is already on. Skip the action entirely if the device has been switched off.

- **Trigger:** Time: 22:00
- **Condition:** Humidifier is on
- **Target:** Bedroom humidifier
- **Condition passes if:** Any
- **Action:** Humidifier: Set humidity

YAML example for a gated humidity adjustment

****automation****

```
automation: |
  alias: "Lower humidity at night if humidifier is on"
  triggers:
    - trigger: time
      at: "22:00:00"
  conditions:
    - condition: humidifier.is_on
      target:
        entity_id: humidifier.bedroom
      options:
        behavior: any
  actions:
    - action: humidifier.set_humidity
      target:
        entity_id: humidifier.bedroom
      data:
        humidity: 45
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidifier.Is Target Humidity

The **Humidifier target humidity** condition passes when a humidifier entity's target humidity setting meets a threshold you define. The target humidity is the setpoint you configure on the device, not the actual current humidity reading from its sensor. For example, you can use it to turn on a ventilation fan when the humidifier is set to 70% or above.

When you target more than one humidifier, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted humidifier to meet the threshold, or demand that all of them do.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier target humidity** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Humidifier target humidity**.
6. Under **Threshold type**, set the comparison direction (**Above**, **Below**, **In range**, or **Outside range**) and the threshold value.
7. Choose **Number** to enter a fixed humidity percentage between 0 and 100, or **Entity** to use a humidity sensor or input number as the threshold.
8. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple humidifiers are targeted.
9. Under **For at least**, set how long the humidifier must have been at the threshold before the condition passes. Leave it at zero to pass immediately.
10. Select **Save**.

Options in the UI

{% options_ui %} **Threshold type:** description: Controls how the target humidity is compared and where the threshold value comes from. Use **Above**, **Below**, **In range**, or **Outside range** to set the comparison direction. Then choose **Number** to enter a fixed percentage between 0 and 100, or **Entity** to use a humidity sensor or input number as the threshold value. **Condition passes if:** description: When multiple humidifiers are targeted, controls how results combine. Pick **Any** to pass if at least one targeted humidifier meets the threshold, or **All** to pass only when every targeted humidifier does. Default is **Any**. **For at least:** description: How long the humidifier must have continuously met the threshold before the condition passes. Default is (passes immediately).

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier target humidity** is referred to as `humidifier.is_target_humidity`. A basic example looks like this:

condition

```
condition: |
  condition: humidifier.is_target_humidity
  target:
    entity_id: humidifier.bedroom
  options:
    threshold:
      above: 50
```

This passes when the bedroom humidifier's target humidity is set above 50%.

Options in YAML

{% options_yaml %} threshold: description: > The threshold to check the target humidity against. Accepts a mapping with the comparison direction as the key and the humidity percentage (0–100) as the value. Use `above`, `below`, or both (`above` and `below` together for a range) as keys. Instead of a fixed number, you can reference a `sensor`, `input_number`, or `number` entity as the value. required: false type: map behavior: description: > When multiple humidifiers are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the humidifier must have continuously met the threshold before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- This condition checks the humidifier's *target humidity* setpoint, not the actual measured humidity in the room. To react to the measured humidity, use a sensor-based numeric condition instead.
- Humidifiers that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Target humidity is expressed as a percentage. The valid range depends on the device, but is typically between 20% and 90%.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the ventilation fan when the humidifier is set high

When the bedroom humidifier's target humidity is set to 70% or above, turn on the ventilation fan to reduce the risk of condensation. This keeps air circulating whenever the humidifier is running at a high setting.

- **Trigger:** State change of the bedroom humidifier's `humidity` attribute
- **Condition:** Humidifier target humidity (70% or above)
- **Target:** Bedroom humidifier
- **Condition passes if:** Any
- **Action:** Switch: Turn on ventilation fan

YAML example for turning on the ventilation fan when the target is high

****automation****

```
automation: |
  alias: "Ventilation fan on when humidifier target is high"
  triggers:
    - trigger: state
      entity_id: humidifier.bedroom
      attribute: humidity
  conditions:
    - condition: humidifier.is_target_humidity
      target:
        entity_id: humidifier.bedroom
      options:
        threshold:
          above: 70
        behavior: any
  actions:
    - action: switch.turn_on
      target:
        entity_id: switch.ventilation_fan
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Humidity.Is Value

The **Relative humidity** condition passes when a humidity reading meets a threshold you define. You can check that humidity is above, below, or within a specific range. The condition works with humidity sensors, climate devices, humidifiers, and weather entities. Use it to run an automation only when the bedroom feels too damp, or only when the air is dry enough to need attention.

When you target more than one entity, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted entity to meet the threshold, or demand that all of them do.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Relative humidity** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your humidity sensor is in (like your bedroom or bathroom). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Relative humidity**.
6. Under **Threshold type**, set the humidity level the condition checks against:
7. Pick whether the reading must be **Above**, **Below**, **In range**, or **Outside range** of the threshold.
8. Select **Number** or **Entity**:
 - **Number**: Enter a fixed percentage directly, for example for 65%. For **In range** or **Outside range**, enter both a lower and upper bound.
 - **Entity**: Use a sensor entity or a [number helper](#) entity as the threshold:
 - **Number helper**: You can adjust the threshold value without editing the automation. The sensor reading is compared against the number helper's current value.
 - **Sensor**: Its current reading becomes the threshold and updates automatically as the sensor changes. This is useful for comparing two humidity readings, for example to check whether indoor humidity is higher than outdoor humidity.
 - For **In range** or **Outside range**, you need two entities: one for the lower bound and one for the upper bound (for example, two separate number helpers).
 - If you don't have a number helper, you can create one by selecting **Create a new number helper**.
9. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
10. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: | The humidity level the entity has to meet for the condition to pass. Options are **Above**, **Below**, **In range**, or **Outside range**. **Number** provides a fixed percentage (0-100%) or both a lower and upper bound for ranges. **Entity** uses a sensor or number helper as a dynamic threshold. Condition passes if: description: | When multiple entities are targeted, controls how results combine:

- ****Any****: The condition passes if at least one targeted entity meets the thresh
- ****All****: The condition passes only when every targeted entity meets the thresh

{% endoptions_ui %}

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `humidity.is_value`. A basic example looks like this:

condition

```
condition: |
  condition: humidity.is_value
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: above
      value:
        number: 60
    behavior: any
```

This passes when the bedroom humidity sensor reads above 60%.

To check that humidity stays below a certain level:

condition

```
condition: |
  condition: humidity.is_value
  target:
    area_id: basement
  options:
    threshold:
      type: below
      value:
        number: 40
    behavior: all
```

This passes when all humidity sensors in the basement area read below 40%.

To check that humidity stays within a comfortable range:

condition

```

condition: |
  condition: humidity.is_value
  target:
    entity_id:
      - sensor.bedroom_humidity
      - sensor.bathroom_humidity
  options:
    threshold:
      type: between
      value_min:
        number: 40
      value_max:
        number: 60
    behavior: any

```

This passes when at least one of the humidity sensors reads between 40% and 60%.

To check that humidity stays outside a range:

condition

```

condition: |
  condition: humidity.is_value
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: outside
      value_min:
        number: 40
      value_max:
        number: 60

```

This passes when the bedroom humidity sensor reads below 40% or above 60%.

To use a number helper as a dynamic threshold that you can adjust without editing the automation:

condition

```

condition: |
  condition: humidity.is_value
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: above
      value:
        entity: input_number.humidity_alert_threshold
    behavior: any

```

This passes when the bedroom humidity sensor reads above the number helper's value.

Options in YAML

{% options_yaml %} threshold: description: | The humidity level the entity has to meet for the condition to pass:

- ``above``: Sets a minimum
- ``below``: Sets a maximum
- ``between``: Defines a range
- ``outside``: Defines an outside-range

For ``above`` and ``below``, use ``value`` with either ``number`` (0 to 100) or ``entity``

- A fixed percentage (0-100%).
- A reference to an ``input_number``, ``number``, or ``sensor`` entity.
 - ``input_number``: Lets you change the threshold without editing the automation
 - ``number``: Uses the current value of a number entity as the threshold.
 - ``sensor``: Uses the current reading as the threshold when the condition is ev

required: false type: map behavior: description: | When multiple entities are targeted, controls how results combine:

- ``any``: The condition passes if at least one targeted entity meets the threshold
- ``all``: The condition passes only when every targeted entity meets the threshold

required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- The condition works with humidity sensors, climate entities (using the current humidity reading), humidifier entities (using the current humidity reading), and weather entities.
- Entities that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Humidity is expressed as a percentage. Indoor comfort is generally between 40% and 60%. Below 30% often feels dry and can irritate airways. Above 65% can encourage mold and dust mites.
- This condition checks the entity's *current* humidity reading, not its target setpoint. To check a humidifier's target setpoint instead, use the [Humidifier target humidity](#) condition.
- When you use a sensor as a dynamic threshold, its value is read at the moment the condition runs. The threshold is not continuously tracked; it is re-evaluated each time the automation fires.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: run a dehumidifier when the room gets too damp

When the bedroom humidity sensor reads above 65%, turn on the dehumidifier to bring levels back to a comfortable range. The condition prevents the dehumidifier from running when the air is already dry enough.

- **Trigger:** Time pattern: Every 15 minutes
- **Condition:** Relative humidity (above 65%)
- **Target:** Bedroom humidity sensor
- **Condition passes if:** Any
- **Action:** Turn on switch
- **Target:** switch.bedroom_dehumidifier

YAML example for running a dehumidifier when humidity is high

```
**automation**
```

```

automation: |
  alias: "Run dehumidifier when bedroom is too damp"
  triggers:
    - trigger: time_pattern
      minutes: "/15"
  conditions:
    - condition: humidity.is_value
      target:
        entity_id:
          - sensor.bedroom_humidity
          - sensor.closet_humidity
      options:
        threshold:
          type: above
          value:
            number: 65
        behavior: any
  actions:
    - action: switch.turn_on
      target:
        entity_id: switch.bedroom_dehumidifier

```

Automation: send an alert when the air gets too dry

At midnight, check the living room humidity. If it has dropped below 30%, send a notification so you can switch on a humidifier before you go to sleep.

- **Trigger:** Time: 00:00
- **Condition:** Relative humidity (below 30%)
- **Target:** Living room humidity sensor
- **Condition passes if:** Any
- **Action:** Send a notification
- **Target:** notify.mobile_app_phone

YAML example for a low humidity alert

```

**automation**

```



```

automation: |
  alias: "Alert when living room air is too dry"
  triggers:
    - trigger: time
      at: "00:00:00"
  conditions:
    - condition: humidity.is_value
      target:
        area_id: living_room
      options:
        threshold:
          type: below
          value:
            number: 30
        behavior: all
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.mobile_app_phone
      data:
        message: >
          The living room humidity is below 30%.
          Consider switching on the humidifier.

```

Automation: adjust humidifier based on comfort range

Check every 15 minutes whether the bedroom humidity is outside your personal comfort range. Use number helpers to set the range, so you can easily adjust it through the UI without editing the automation.

- **Trigger:** Time pattern: Every 15 minutes
- **Condition:** Relative humidity (outside range, using number helpers)
- **Target:** Bedroom humidity sensor
- **Condition passes if:** Any
- **Action:** Turn on switch
- **Target:** switch.bedroom_humidifier

YAML example for using number helpers as threshold

```

**automation**

```

```

automation: |
  alias: "Turn on humidifier when outside comfort range"
  triggers:
    - trigger: time_pattern
      minutes: "/15"
  conditions:
    - condition: humidity.is_value
      target:
        entity_id: sensor.bedroom_humidity
      options:
        threshold:
          type: outside
          value_min:
            entity: input_number.comfort_humidity_min
          value_max:
            entity: input_number.comfort_humidity_max
  actions:
    - action: switch.turn_on
      target:
        entity_id: switch.bedroom_humidifier

```

Automation: run the ventilation fan when indoor humidity exceeds outdoor humidity

Every 15 minutes, check whether the living room is more humid than the outside air. If so, open the ventilation to let drier air in. The outdoor humidity sensor acts as a live threshold, so the condition always compares the two current readings.

- **Trigger:** Time pattern: Every 15 minutes
- **Condition:** Relative humidity (above, entity: outdoor humidity sensor)
- **Target:** Living room humidity sensor
- **Condition passes if:** Any
- **Action:** Turn on switch
- **Target:** switch.ventilation_fan

YAML example for comparing indoor to outdoor humidity

```

**automation**

```

```
automation: |
  alias: "Ventilate when indoor humidity exceeds outdoor"
  triggers:
    - trigger: time_pattern
      minutes: "/15"
  conditions:
    - condition: humidity.is_value
      target:
        entity_id: sensor.living_room_humidity
      options:
        threshold:
          type: above
          value:
            entity: sensor.outdoor_humidity
  actions:
    - action: switch.turn_on
      target:
        entity_id: switch.ventilation_fan
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Light.Is Brightness

The **Light brightness** condition passes when a light entity's brightness meets a threshold you set. Use it to gate an automation based on how bright the light currently is, not just whether it's on or off.

When you target more than one light, the condition's **behavior** option controls how the check combines. You can require any targeted light's brightness to match, or demand that all of them do.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Light: Light brightness**.
5. Under **Targets**, select the light entity, an area, a floor, or a label.
6. Under **Threshold type**, set the brightness percentage the condition checks against.
7. Under **Condition passes if**, pick **Any** or **All**.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The brightness level the light has to meet or exceed. Expressed as a percentage of full brightness. required: true Condition passes if: description: When multiple lights are targeted, controls how results combine. Pick **Any** to pass if at least one light meets the threshold, or **All** to pass only when every targeted light does.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `light.is_brightness`. A basic example looks like this:

condition

```
condition: |
  condition: light.is_brightness
  target:
    entity_id: light.living_room
  options:
    threshold: 50
    behavior: any
```

This passes when the living room light's brightness is at or above 50%.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The brightness percentage the light has to meet or exceed for the condition to pass. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: false type: any behavior: description: > When multiple lights are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Good to know

- A light that is off has brightness zero, so it never meets a positive threshold. Combine with [Light is on](#) if you want to check both.
- Lights that are unavailable (`unavailable`) or have an unknown state (`unknown`) are skipped for **Any** and fail for **All**.
- Pair with [Light brightness crossed threshold](#) as a matching trigger when you need the automation to run the moment the brightness crosses that line.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only start the movie scene if the room is dim enough

When the movie-night button is pressed, only start the movie scene if the living room light is dimmer than 40%. If the room is still bright, prompt for manual dimming first.

- **Trigger:** State: Movie night button pressed
- **Condition:** Light brightness
- **Target:** Living room light
- **Threshold type:** 40
- **Condition passes if:** Any
- **Action:** Scene: Movie night

YAML example for a dim-gated movie scene

****automation****

```
automation: |
  alias: "Movie scene only if dim"
  triggers:
    - trigger: state
      entity_id: input_button.movie_night
  conditions:
    - condition: light.is_brightness
      target:
        entity_id: light.living_room
      options:
        threshold: 40
        behavior: any
  actions:
    - action: scene.turn_on
      target:
        entity_id: scene.movie_night
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Light.Is Off

The **Light is off** condition passes when a light entity is currently off. Use it to gate an automation so it only runs when a specific light (or every targeted light) is already dark.

When you target more than one light, the condition's **behavior** option controls how the check combines results. You can require any targeted light to be off, or demand that all of them are.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Light: Light is off**.
5. Under **Targets**, select the light entity, an area, a floor, or a label.
6. Under **Condition passes if**, pick **Any** or **All**.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple lights are targeted, controls how results combine. Pick **Any** to pass if at least one targeted light is off, or **All** to pass only when every targeted light is off.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `light.is_off`. A basic example looks like this:

condition

```
condition: |
  condition: light.is_off
  target:
    entity_id: light.bedroom
```

This passes when the bedroom light is currently off.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple lights are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Good to know

- Lights that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as off. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted light is unavailable.
- To gate an automation on a light being on instead, use `Light is on`.
- Pair with the `Light turned off` trigger to react only when a transition to off happens.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only run the morning wake-up if the bedroom is still dark

At 07:00 on weekdays, start the morning wake-up routine, but only if the bedroom light is still off. Skip the routine on days you're already awake.

- **Trigger:** Time: 07:00
- **Condition:** Day of the week is Monday to Friday
- **Condition:** Light is off
- **Target:** Bedroom light
- **Condition passes if:** All
- **Action:** Script: Morning wake-up

YAML example for a gated morning wake-up

****automation****

```
automation: |
  alias: "Morning wake-up only if bedroom dark"
  triggers:
    - trigger: time
      at: "07:00:00"
  conditions:
    - condition: time
      weekday:
        - mon
        - tue
        - wed
        - thu
        - fri
    - condition: light.is_off
      target:
        entity_id: light.bedroom
      options:
        behavior: all
  actions:
    - action: script.morning_wake_up
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Light.Is On

The **Light is on** condition passes when a light entity is currently on. Use it to gate an automation so it only runs when a specific light (or every targeted light) is already lit.

When you target more than one light, the condition's **behavior** option controls how the check combines results. You can require any targeted light to be on, or demand that all of them are.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Light: Light is on**.
5. Under **Targets**, select the light entity, an area, a floor, or a label.
6. Under **Condition passes if**, pick **Any** or **All** to control how the check behaves when multiple lights are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple lights are targeted, controls how results combine. Pick **Any** to pass if at least one targeted light is on, or **All** to pass only when every targeted light is on.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `light.is_on`. A basic example looks like this:

condition

```
condition: |
  condition: light.is_on
  target:
    entity_id: light.living_room
```

This passes when the living room light is currently on.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple lights are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Good to know

- Lights that are unavailable (`unavailable`) or have an unknown state (`unknown`) do not count as on. With **Any** behavior, they are skipped. With **All** behavior, the condition fails if every targeted light is unavailable.
- To gate an automation on a light being off instead, use `Light is off`.
- Pair with `Light is brightness` when you also want to check whether the light's brightness meets a threshold.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: only announce the doorbell if the living room is lit

When the doorbell rings, only announce it through the living room speaker if the living room light is already on. Keeps the house quiet when the room is empty.

- **Trigger:** State: Doorbell button pressed
- **Condition:** Light is on
- **Target:** Living room light
- **Condition passes if:** Any
- **Action:** Media player: Play media

YAML example for a doorbell announcement gated on lights

****automation****

```
automation: |
  alias: "Doorbell announce when living room lit"
  triggers:
    - trigger: state
      entity_id: binary_sensor.doorbell
      to: "on"
  conditions:
    - condition: light.is_on
      target:
        entity_id: light.living_room
      options:
        behavior: any
  actions:
    - action: media_player.play_media
      target:
        entity_id: media_player.living_room
      data:
        media_content_id: "media-source://tts/cloud?message=Someone+is+at+the-
        media_content_type: music
        announce: true
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Lock.Is Jammed

The **Lock is jammed** condition helps you check whether a lock is currently stuck. Use it when you want an automation to react only while the problem is still present, like sending repeated reminders or turning on more light at the door.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Lock is jammed**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay jammed before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple locks are targeted, controls how results combine. Pick **Any** to pass if at least one targeted lock is jammed, or **All** to pass only when every targeted lock is jammed. required: false For at least: description: How long the lock must stay jammed before the condition passes. required: false

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `lock.is_jammed`. A basic example looks like this:

condition

```
condition: |
  condition: lock.is_jammed
  target:
    entity_id: lock.front_door
```

This passes when `lock.front_door` is currently jammed.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the lock must stay jammed before the condition passes. Accepts a duration like `00:01:00` for one minute. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Locks in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- Use **For at least** if you want to wait for a lasting problem instead of a brief status report.
- To check for the normal secure state instead, use `Lock is locked`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send a reminder if the front door lock is still jammed

If a first alert was missed, a follow-up reminder can help you fix the problem before leaving the house unsecured. This automation checks every 10 minutes and sends a reminder while the front door lock is still jammed.

- **Trigger:** Time pattern: Every 10 minutes
- **Condition:** Lock is jammed
- **Target:** Front door lock
- **Condition passes if:** Any
- **For at least:** 00:01:00
- **Action:** Send a notification via mobile_app_phone

YAML example for repeated jammed lock reminders

****automation****

```
automation: |
  alias: "Remind me that the front door lock is jammed"
  triggers:
    - trigger: time_pattern
      minutes: "/10"
  conditions:
    - condition: lock.is_jammed
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:01:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.mobile_app_phone
      data:
        title: "Front door lock still jammed"
        message: "Check the front door lock and clear the obstruction."
```

Automation: turn on the porch light while any outside door lock is jammed

Extra light can make it easier to see why a lock is stuck. This automation runs after sunset and turns on the porch light if any targeted outside lock has been jammed for at least 15 seconds.

- **Trigger:** Time pattern: Every 1 minute
- **Condition:** Lock is jammed
- **Target:** Outside door locks (by label)
- **Condition passes if:** Any
- **For at least:** 00:00:15
- **Condition:** Sun is below the horizon
- **Action:** Turn on

YAML example for lighting an area while a lock is jammed

****automation****

```
automation: |
  alias: "Turn on the porch light while a lock is jammed"
  triggers:
    - trigger: time_pattern
      minutes: "/1"
  conditions:
    - condition: lock.is_jammed
      target:
        label_id: outside_locks
      options:
        behavior: any
        for: "00:00:15"
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id: light.porch
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Lock.Is Locked

The **Lock is locked** condition helps you check whether a lock is currently secure. Use it when an automation should continue only after a door has been locked, like arming an alarm or turning off devices near an entry.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Lock is locked**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay locked before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple locks are targeted, controls how results combine. Pick **Any** to pass if at least one targeted lock is locked, or **All** to pass only when every targeted lock is locked. required: false For at least: description: How long the lock must stay locked before the condition passes. required: false

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `lock.is_locked`. A basic example looks like this:

condition

```
condition: |
  condition: lock.is_locked
  target:
    entity_id: lock.front_door
```

This passes when `lock.front_door` is currently locked.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the lock must stay locked before the condition passes. Accepts a duration like `00:05:00` for five minutes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Locks in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- Use **For at least** when you want to avoid acting on a short state change.
- To check whether a door is not secured, use `Lock is unlocked`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: arm the alarm only if all the outside door locks are locked

Before arming the house for the night, you may want one final check that every outside door is secure. This automation runs at bedtime and arms the alarm only if all targeted locks are locked.

- **Trigger:** Time: 23:00
- **Condition:** Lock is locked
- **Target:** Outside door locks (by label)
- **Condition passes if:** All
- **For at least:** 00:00:00
- **Action:** Arm alarm away

YAML example for checking all outside locks

****automation****

```
automation: |
  alias: "Arm the alarm only when all locks are locked"
  triggers:
    - trigger: time
      at: "23:00:00"
  conditions:
    - condition: lock.is_locked
      target:
        label_id: outside_locks
      options:
        behavior: all
        for: "00:00:00"
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home_alarm
```

Automation: turn off the porch light if the front door has been locked for 5 minutes

Once the front door has stayed locked for a while, you may not need the porch light anymore. This automation checks every evening and turns the light off after the door has remained locked for 5 minutes.

- **Trigger:** Time pattern: Every 5 minutes
- **Condition:** Lock is locked
- **Target:** Front door lock
- **Condition passes if:** Any
- **For at least:** 00:05:00
- **Condition:** Sun is below the horizon
- **Action:** Turn off

YAML example for turning off the porch light

****automation****

```
automation: |
  alias: "Turn off the porch light after the door stays locked"
  triggers:
    - trigger: time_pattern
      minutes: "/5"
  conditions:
    - condition: lock.is_locked
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:05:00"
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_off
      target:
        entity_id: light.porch
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Lock.Is Open

The **Lock is open** condition helps you check whether a lock is currently open. Use it when an automation should continue only while a door is still open, like leaving a light on or delaying another action until the door is closed again.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Lock is open**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay open before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple locks are targeted, controls how results combine. Pick **Any** to pass if at least one targeted lock is open, or **All** to pass only when every targeted lock is open. required: false For at least: description: How long the lock must stay open before the condition passes. required: false

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `lock.is_open`. A basic example looks like this:

condition

```
condition: |
  condition: lock.is_open
  target:
    entity_id: lock.front_door
```

This passes when `lock.front_door` is currently open.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the lock must stay open before the condition passes. Accepts a duration like `00:01:00` for one minute. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Locks in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- Not every lock reports an open state. Use this condition only with locks that support open-state reporting.
- To check for the secure state instead, use [Lock is locked](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: keep the hallway light on while the front door is open

If the front door stays open while you carry things inside, it helps to keep the hallway lit. This automation checks every minute and turns the light on while the door remains open.

- **Trigger:** Time pattern: Every 1 minute
- **Condition:** Lock is open
- **Target:** Front door lock
- **Condition passes if:** Any
- **For at least:** 00:00:10
- **Action:** Turn on

YAML example for keeping a light on while the door is open

****automation****

```
automation: |
  alias: "Keep the hallway light on while the door is open"
  triggers:
    - trigger: time_pattern
      minutes: "/1"
  conditions:
    - condition: lock.is_open
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:00:10"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.hallway
```

Automation: send a reminder if any patio door lock stays open

If you want a simple reminder before bed, check whether any patio door is still open. This automation runs at night and sends a message if any targeted patio lock has stayed open for 2 minutes.

- **Trigger:** Time: 22:30
- **Condition:** Lock is open
- **Target:** Patio door locks (by label)
- **Condition passes if:** Any
- **For at least:** 00:02:00
- **Action:** Send a notification via mobile_app_phone

YAML example for a patio door reminder

****automation****

```
automation: |
  alias: "Remind me if a patio door is still open"
  triggers:
    - trigger: time
      at: "22:30:00"
  conditions:
    - condition: lock.is_open
      target:
        label_id: patio_locks
      options:
        behavior: any
        for: "00:02:00"
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Patio door still open"
        message: "A patio door has remained open for at least 2 minutes."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Lock.Is Unlocked

The **Lock is unlocked** condition helps you check whether a lock is currently unlocked. Use it when an automation should continue only while a door is not secured, like reminding you to lock up before bed.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Lock is unlocked**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay unlocked before the condition passes.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple locks are targeted, controls how results combine. Pick **Any** to pass if at least one targeted lock is unlocked, or **All** to pass only when every targeted lock is unlocked. required: false For at least: description: How long the lock must stay unlocked before the condition passes. required: false

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `lock.is_unlocked`. A basic example looks like this:

condition

```
condition: |
  condition: lock.is_unlocked
  target:
    entity_id: lock.front_door
```

This passes when `lock.front_door` is currently unlocked.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long the lock must stay unlocked before the condition passes. Accepts a duration like `00:10:00` for 10 minutes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

Good to know

- Locks in the `unavailable` or `unknown` state are ignored when Home Assistant evaluates the condition.
- Use **For at least** if you want to ignore a short unlock before the door is secured again.
- To check for the secure state instead, use `Lock is locked`.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send a bedtime reminder if the front door is still unlocked

If you sometimes forget to lock the door before bed, a gentle reminder can help. This automation runs at night and sends a phone notification if the front door has stayed unlocked for 10 minutes.

- **Trigger:** Time: 22:00
- **Condition:** Lock is unlocked
- **Target:** Front door lock
- **Condition passes if:** Any
- **For at least:** 00:10:00
- **Action:** Send a notification via mobile_app_phone

YAML example for a bedtime lock reminder

****automation****

```
automation: |
  alias: "Remind me if the front door is unlocked at night"
  triggers:
    - trigger: time
      at: "22:00:00"
  conditions:
    - condition: lock.is_unlocked
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:10:00"
  actions:
    - action: notify.mobile_app_phone
      data:
        title: "Front door still unlocked"
        message: "The front door has stayed unlocked for 10 minutes."
```

Automation: turn on a lamp if any outside door lock stays unlocked after sunset

If an outside door stays unlocked after dark, a visible reminder can help. This automation checks every 5 minutes and turns on a lamp when any targeted outside lock remains unlocked.

- **Trigger:** Time pattern: Every 5 minutes
- **Condition:** Lock is unlocked
- **Target:** Outside door locks (by label)
- **Condition passes if:** Any
- **For at least:** 00:05:00
- **Condition:** Sun is below the horizon
- **Action:** Turn on

YAML example for a visual unlocked reminder

****automation****

```
automation: |
  alias: "Turn on a lamp when an outside lock stays unlocked"
  triggers:
    - trigger: time_pattern
      minutes: "/5"
  conditions:
    - condition: lock.is_unlocked
      target:
        label_id: outside_locks
      options:
        behavior: any
        for: "00:05:00"
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id: light.living_room_lamp
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Motion.Is Detected

The **Motion is detected** condition passes when one or more motion sensors are detecting motion. Use it in an automation for turning devices on or off, running security checks or sending alerts if motion is detected.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Motion is detected**.
5. Under **Targets** (see [Targets](#)), select one or more motion entities, motion devices, an area, a floor, or a label.
6. If you selected more than one target, under **Condition passes if**, pick **Any** or **All**.
7. Under **For at least**, you can set for how long one or more sensors must be detecting motion before the condition passes. Leave it at zero for the condition to pass as soon as the sensors start detecting motion.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple motion sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor is detecting motion, or **All** to pass only when every sensor is detecting motion. For at least: description: How long one or more sensors must be continuously detecting motion before the condition passes. The default is hours, minutes and seconds.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `motion.is_detected`. A basic example looks like this:

condition

```
condition: |
  condition: motion.is_detected
  target:
    entity_id: motion.entrance_motion_sensor
  options:
    for: "00:05:00"
```

This passes when the entity `motion.entrance_motion_sensor` has been continuously detecting motion for 5 minutes.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple motion sensors are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long one or more motion sensors must be continuously detecting motion before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- When using the **For at least** option in your automation, consider that the automation will not run if there is a stop in motion detection during that time period.
- In large rooms, for example, motion sensors might not detect people that are quietly sitting there. If you want to make sure presence is detected, use multiple motion sensors in one area or combine sensors with presence detection.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: set the office fan to medium when you are at the desk

When it gets warm, if there is someone working at the office desk, this automation sets the office fan to medium preset mode.

- **Trigger:** Temperature crossed threshold (25 °C)
- **Target:** Office (by area)
- **Condition:** Motion is detected
- **Target:** Office motion sensor
- **Action:** Set fan preset mode (medium)
- **Target:** Office fan

YAML example for setting office fan to medium when warm and if desk is occupied

****automation****

```
automation: |
  alias: "Set the office fan to medium on warm days if motion is detected"
  triggers:
    - trigger: temperature.crossed_threshold
      target:
        area_id: office
      threshold:
        type: above
        value:
          active_choice: number
          number: 25
          unit_of_measurement: °C
  conditions:
    - condition: motion.is_detected
      target:
        entity_id: binary_sensor.movement_office
  actions:
    - action: fan.set_preset_mode
      target:
        entity_id: fan.office_fan
      data:
        preset_mode: medium
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Motion.Is Not Detected

The **Motion is not detected** condition passes when one or more motion sensors are not detecting motion. Use it in an automation for turning devices on or off, running security checks or sending alerts if motion is not detected. You can set up an automation to run only if motion is not being detected in a particular area of the house, for example.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Motion is not detected**.
5. Under **Targets**, select one or more motion entities, devices, an area, a floor, or a label.
6. If you selected more than one target, under **Condition passes if**, pick **Any** or **All**.
7. Under **For at least**, you can set for how long one or more motion sensors must be without detecting motion before the condition passes. Leave it at zero for the condition to pass as soon as the sensors stop detecting motion.
8. Select **Save**.

Options in the UI

{% options_ui %} Condition passes if: description: When multiple motion sensors are targeted, controls how results combine. Pick **Any** to pass if at least one targeted sensor is not detecting motion, or **All** to pass only when every sensor is not detecting motion. For at least: description: How long one or more sensors must remain without detecting motion before the condition passes. The default is hours, minutes and seconds.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `motion.is_not_detected`. A basic example looks like this:

condition

```
condition: |
  condition: motion.is_not_detected
  target:
    entity_id: motion.sensor_backyard
  options:
    for: "01:10:05"
```

This passes when the sensor `motion.sensor_backyard` has not detected any motion for 1 hour, 10 minutes and 5 seconds.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple motion sensors are targeted, controls how results combine. Accepts `all` or `any`. required: false type: string default: any for: description: > How long one or more motion sensors must remain without detecting motion before the condition passes. Accepts a duration string in `HH:MM:SS` format. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the condition

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- When using the **For at least** option in your automation, if the sensors detect motion during that time period the condition will not pass and actions will not run. This is useful to make sure no one is at home or in an area before turning devices off, for example.
- In large rooms, motion sensors might not detect people that are quietly sitting in a place. If you want to make sure presence is detected, use multiple motion sensors in one area or combine sensors with presence detection.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: close bedroom cover if no one is in the main floor

At the time of day when the sun is directly hitting the bedroom window, if no motion has been detected for 15 minutes in the main floor, close the bedroom cover to assure a refreshing night.

- **Trigger:** Time (at 17.30)
- **Condition:** Motion is not detected
- **Target:** Sensors main floor
- **For at least:** 00:15:00
- **Action:** Close cover
- **Target:** Bedroom

YAML example for closing bedroom cover for a fresh night

****automation****

```
automation: |
  alias: "Close the bedroom cover in the evening if no movement is detected"
  triggers:
    - trigger: time
      at: "17:30:00"
  conditions:
    - condition: motion.is_not_detected
      target:
        label_id: sensors_main_floor
      options:
        for: "00:15:00"
  actions:
    - action: cover.close_cover
      target:
        entity_id: cover.bedroom_window
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

Related conditions

These conditions work well alongside this one:

{% endif %}

Conditions/Temperature.Is Value

The **Temperature value** condition passes when a temperature reading meets a threshold you define. You can check that the temperature is above, below, or within a specific range. The condition works with temperature sensors, climate devices, water heaters, and weather entities. Use it to run an automation only when the bedroom is too warm, or only when the temperature is low enough to need heating.

When you target more than one entity, the condition's **Condition passes if** option controls how the check combines results. You can require any targeted entity to meet the threshold, or demand that all of them do.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this condition from the user interface

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use **Temperature** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area your temperature sensor is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Temperature value**.
6. Under **Threshold type**, set the temperature level the condition checks against:
7. Pick whether the reading must be **Above**, **Below**, **In range**, or **Outside range** of the threshold.
8. Select **Number** or **Entity**:
 - **Number**: Enter a fixed temperature directly, for example for 20°C. For **In range** or **Outside range**, enter both a lower and upper bound.
 - **Entity**: Use a sensor entity or a [number helper](#) entity as the threshold:
 - Number helper: You can adjust the threshold value without editing the automation. The sensor reading is compared against the number helper's current value.
 - Sensor: Its current reading becomes the threshold and updates automatically as the sensor changes. This is useful for comparing two temperature readings, for example to check whether indoor temperature is higher than outdoor temperature.
 - For **In range** or **Outside range**, you need two entities: one for the lower bound and one for the upper bound (for example, two separate number helpers).
 - If you don't have a number helper, you can create one by selecting **Create a new number helper**.
9. Under **Unit**, select the temperature unit (°C or °F) to use for the threshold comparison.
10. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
11. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: | The temperature level the entity has to meet for the condition to pass. Options are **Above**, **Below**, **In range**, or **Outside range**. **Number** provides a fixed temperature value (or both a lower and upper bound for ranges). **Entity** uses a sensor or number helper as a dynamic threshold. Unit: description: The temperature unit to use for threshold comparison. Accepts or . Required when using numerical thresholds (not required when using entity references). default: °C Condition passes if: description: When multiple entities are

targeted, controls how results combine. Pick **Any** to pass if at least one targeted entity meets the threshold, or **All** to pass only when every targeted entity does. Default is **Any**.

Using this condition in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `temperature.is_value`. A basic example looks like this:

condition

```
condition: |
  condition: temperature.is_value
  target:
    entity_id: sensor.living_room_temperature
  options:
    threshold:
      type: above
      value:
        number: 20
        unit_of_measurement: "°C"
    behavior: any
```

This passes when the living room temperature sensor reads above 20°C.

To check that temperature stays below a certain level:

condition

```
condition: |
  condition: temperature.is_value
  target:
    entity_id: sensor.living_room_temperature
  options:
    threshold:
      type: below
      value:
        number: 24
        unit_of_measurement: "°C"
    behavior: any
```

This passes when the living room temperature sensor reads below 24°C.

To check that temperature stays within a comfortable range:

condition

```

condition: |
  condition: temperature.is_value
  target:
    entity_id: sensor.living_room_temperature
  options:
    threshold:
      type: between
      value_min:
        number: 20
        unit_of_measurement: "°C"
      value_max:
        number: 22
        unit_of_measurement: "°C"
    behavior: any

```

This passes when the living room temperature sensor reads between 20 and 22°C.

Options in YAML

{% options_yaml %} threshold: description: | The temperature level the entity has to meet for the condition to pass:

- `above`: Sets a minimum
- `below`: Sets a maximum
- `between`: Defines a range
- `outside`: Defines an outside-range

For `above` and `below`, use `value` with either `number` and `unit\_of\_measurement`

```

```yaml
threshold:
 type: between
 value_min:
 entity: input_number.comfort_temperature_min
 value_max:
 number: 22
 unit_of_measurement: °C
...

```

When using an `entity`, its current reading is used as the threshold at the moment

required: true type: map behavior: description: > Controls how results combine when multiple entities are targeted. Accepts `all` or `any`. required: false type: string default: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.



## Good to know

---

- The condition works with temperature sensors, [climate](#) entities (using the current temperature reading), [water heater](#) entities (using the current temperature reading), and [weather](#) entities.
- Climate, water heater, and weather entities that don't report a current temperature attribute are automatically excluded from evaluation. Only entities with a valid temperature value are considered.
- Entities that have an `unavailable` or `unknown` state are skipped for **Any** and fail for **All**.
- This condition checks the entity's current temperature reading, not its target setpoint. To check a climate device's target setpoint instead, use the climate target temperature condition.
- When you use a sensor as a dynamic threshold, its value is read at the moment the condition runs. The threshold is not continuously tracked; it is re-evaluated each time the automation fires.
- All temperature values are automatically converted to the unit you specify. For example, if your sensor reports in Fahrenheit but you configure the condition in Celsius, the conversion happens automatically.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press **Ctrl+V** (or **Cmd+V** on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: cool only when temperature is high

This automation runs a fan only when the bedroom temperature is above 24°C, helping you save energy by avoiding unnecessary cooling.

- **Trigger:** State: Fan is off
- **Condition:** Temperature (above 24°C)
- **Target:** Bedroom temperature sensor
- **Condition passes if:** Any
- **Action:** Fan: Turn on

#### YAML example for cooling when warm

**\*\*automation\*\***

```
automation: |
 alias: "Run fan when bedroom is warm"
 triggers:
 - trigger: state
 entity_id: fan.bedroom_fan
 to: "off"
 conditions:
 - condition: temperature.is_value
 target:
 entity_id: sensor.bedroom_temperature
 options:
 threshold:
 type: above
 value:
 number: 24
 unit_of_measurement: "°C"
 behavior: any
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.bedroom_fan
```

## Automation: alert when temperature is outside comfort range

This automation sends a notification only when the living room temperature is outside the comfort range of 20 to 22°C, helping you maintain consistent conditions.

- **Trigger:** Time pattern (every hour)
- **Condition:** Temperature value (outside 20-22°C range)
- **Target:** Living room temperature sensor
- **Condition passes if:** Any
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

### YAML example for temperature out of range alert

**\*\*automation\*\***

```
automation: |
 alias: "Alert when temperature is uncomfortable"
 triggers:
 - trigger: time_pattern
 hours: "/1"
 conditions:
 - condition: temperature.is_value
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: outside
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
 behavior: any
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: >
 Living room temperature is {{ states('sensor.living_room_temperature') }}
 Comfortable range is 20-22°C.
```

## Automation: turn off climate when temperature is comfortable

When the bedroom temperature is already within your comfort range, this automation turns off the climate system to save energy. Use number helpers to define your preferred temperature range so you can easily adjust it without editing the automation.

- **Trigger:** Time pattern (every 30 minutes)
- **Condition:** Temperature (in range, using number helpers)

- **Target:** Bedroom temperature sensor
- **Condition passes if:** Any
- **Action:** Set thermostat HVAC mode (state: off)

#### YAML example for turning off climate when comfortable

**\*\*automation\*\***

```
automation: |
 alias: "Turn off climate when bedroom is comfortable"
 triggers:
 - trigger: time_pattern
 minutes: "/30"
 conditions:
 - condition: temperature.is_value
 target:
 entity_id: sensor.bedroom_temperature
 options:
 threshold:
 type: between
 value_min:
 entity: input_number.comfort_temperature_min
 value_max:
 entity: input_number.comfort_temperature_max
 behavior: any
 actions:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.bedroom
 data:
 hvac_mode: "off"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Conditions/Vacuum.Is Cleaning

---

The **Vacuum cleaner is cleaning** condition passes when one or more targeted vacuums are actively cleaning.

Use this when you want an automation to continue only if the robot is actively cleaning, like pausing it for a quiet activity, avoiding another floor-cleaning routine, or sending a status update only while the run is still underway.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.



## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Vacuum: Vacuum cleaner is cleaning**.
5. Under **Targets**, select the vacuum entity, an area, a floor, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the vacuum must keep cleaning before the condition passes.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple vacuums are targeted, controls how results combine. Pick **Any** to pass if at least one targeted vacuum is cleaning, or **All** to pass only when every targeted vacuum is cleaning. required: true For at least: description: The time the vacuum must keep cleaning before the condition passes. required: false

## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `vacuum.is_cleaning`. A basic example looks like this:

### condition

```
condition: |
 condition: vacuum.is_cleaning
 target:
 entity_id: vacuum.living_room
 options:
 behavior: any
```

This passes when `vacuum.living_room` is actively cleaning.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: The time the vacuum must keep cleaning before the condition passes. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

## Good to know

---

- Entities with state `unavailable` or `unknown` are ignored when Home Assistant evaluates the condition.
- With **Any** (default), the condition passes if at least one targeted vacuum is cleaning.
- With **All**, the condition passes only if every targeted vacuum that Home Assistant can evaluate is cleaning.
- If every targeted vacuum is `unavailable` or `unknown`, **Any** fails and **All** passes.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: pause the vacuum for movie time

When movie mode starts, this automation first checks whether the living room vacuum is actively cleaning. If it is, Home Assistant pauses it so the room is quiet.

- **Trigger:** Movie mode turns on
- **Condition:** Vacuum is cleaning
- **Target:** Living room vacuum
- **Action:** Pause cleaning

#### YAML example for pausing a vacuum during movie mode

**\*\*automation\*\***

```
automation: |
 alias: "Pause vacuum for movie mode"
 triggers:
 - trigger: state
 entity_id: input_boolean.movie_mode
 to: "on"
 conditions:
 - condition: vacuum.is_cleaning
 target:
 entity_id: vacuum.living_room
 options:
 behavior: any
 actions:
 - action: vacuum.pause
 target:
 entity_id: vacuum.living_room
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---



## Conditions/Vacuum.Is Docked

---

The **Vacuum cleaner is docked** condition passes when one or more targeted vacuums are on their dock or charging base.

Use this when you want to continue only if the robot is safely parked, like before turning off a light near the charger, starting maintenance, or sending a reminder that the cleaning cycle is complete.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Vacuum: Vacuum cleaner is docked**.
5. Under **Targets**, select the vacuum entity, an area, a floor, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the vacuum must stay docked before the condition passes.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple vacuums are targeted, controls how results combine. Pick **Any** to pass if at least one targeted vacuum is docked, or **All** to pass only when every targeted vacuum is docked. required: true For at least: description: The time the vacuum must stay docked before the condition passes. required: false

## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `vacuum.is_docked`. A basic example looks like this:

### condition

```
condition: |
 condition: vacuum.is_docked
 target:
 entity_id: vacuum.laundry_room
 options:
 behavior: any
```

This passes when `vacuum.laundry_room` is docked.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: The time the vacuum must stay docked before the condition passes. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

## Good to know

---

- Entities with state `unavailable` or `unknown` are ignored when Home Assistant evaluates the condition.
- With **Any** (default), the condition passes if at least one targeted vacuum is docked.
- With **All**, the condition passes only if every targeted vacuum that Home Assistant can evaluate is docked.
- If every targeted vacuum is `unavailable` or `unknown`, **Any** fails and **All** passes.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn off the laundry room light after docking

At bedtime, this automation checks whether the vacuum is already docked. If it is, the light near the charging station turns off.

- **Trigger:** Time: 23:00
- **Condition:** Vacuum is docked
- **Target:** Laundry room vacuum
- **Action:** Turn off light

#### YAML example for turning off a light once the vacuum is docked

**\*\*automation\*\***

```
automation: |
 alias: "Docked vacuum light off"
 triggers:
 - trigger: time
 at: "23:00:00"
 conditions:
 - condition: vacuum.is_docked
 target:
 entity_id: vacuum.laundry_room
 options:
 behavior: any
 actions:
 - action: light.turn_off
 target:
 entity_id: light.laundry_room
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.



## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Conditions/Vacuum.Is Encountering An Error

---

The **Vacuum cleaner is encountering an error** condition passes when one or more targeted vacuums are in an error state.

Use this when you want an automation to act only if the robot still needs attention, like sending a reminder later in the day, turning on a helper light, or skipping a follow-up routine until the issue is fixed.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Vacuum: Vacuum cleaner is encountering an error**.
5. Under **Targets**, select the vacuum entity, an area, a floor, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the vacuum must remain in the error state before the condition passes.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple vacuums are targeted, controls how results combine. Pick **Any** to pass if at least one targeted vacuum is in an error state, or **All** to pass only when every targeted vacuum is in an error state. required: true For at least: description: The time the vacuum must remain in the error state before the condition passes. required: false

## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `vacuum.is_encountering_an_error`. A basic example looks like this:

### condition

```
condition: |
 condition: vacuum.is_encountering_an_error
 target:
 entity_id: vacuum.upstairs
 options:
 behavior: any
```

This passes when `vacuum.upstairs` is reporting an error.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: The time the vacuum must remain in the error state before the condition passes. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

## Good to know

---

- Entities with state `unavailable` or `unknown` are ignored when Home Assistant evaluates the condition.
- With **Any** (default), the condition passes if at least one targeted vacuum is in an error state.
- With **All**, the condition passes only if every targeted vacuum that Home Assistant can evaluate is in an error state.
- If every targeted vacuum is `unavailable` or `unknown`, **Any** fails and **All** passes.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: remind you about an unresolved vacuum error

This automation checks every evening whether the upstairs vacuum is still in an error state. If it is, Home Assistant sends a reminder so the problem does not go unnoticed until the next cleaning run.

- **Trigger:** Time: 18:00
- **Condition:** Vacuum is encountering an error
- **Target:** Upstairs vacuum
- **Action:** Send a notification via mobile\_app\_phone

#### YAML example for an unresolved vacuum error reminder

**\*\*automation\*\***

```
automation: |
 alias: "Reminder for vacuum error"
 triggers:
 - trigger: time
 at: "18:00:00"
 conditions:
 - condition: vacuum.is_encountering_an_error
 target:
 entity_id: vacuum.upstairs
 options:
 behavior: any
 actions:
 - action: notify.mobile_app_phone
 data:
 title: "Vacuum still needs help"
 message: "The upstairs vacuum is still reporting an error."
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Conditions/Vacuum.Is Paused

---

The **Vacuum cleaner is paused** condition passes when one or more targeted vacuums are paused in the middle of a cleaning run.

Use this when you want an automation to continue only if the robot is stopped mid-run, like sending a reminder, turning on a nearby light, or resuming later as part of a scheduled routine.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Vacuum: Vacuum cleaner is paused**.
5. Under **Targets**, select the vacuum entity, an area, a floor, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the vacuum must stay paused before the condition passes.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple vacuums are targeted, controls how results combine. Pick **Any** to pass if at least one targeted vacuum is paused, or **All** to pass only when every targeted vacuum is paused. required: true For at least: description: The time the vacuum must stay paused before the condition passes. required: false

## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `vacuum.is_paused`. A basic example looks like this:

### condition

```
condition: |
 condition: vacuum.is_paused
 target:
 entity_id: vacuum.hallway
 options:
 behavior: any
```

This passes when `vacuum.hallway` is paused.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: The time the vacuum must stay paused before the condition passes. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

## Good to know

---

- Entities with state `unavailable` or `unknown` are ignored when Home Assistant evaluates the condition.
- With **Any** (default), the condition passes if at least one targeted vacuum is paused.
- With **All**, the condition passes only if every targeted vacuum that Home Assistant can evaluate is paused.
- If every targeted vacuum is `unavailable` or `unknown`, **Any** fails and **All** passes.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.



## More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: remind you if the vacuum is still paused

This automation checks every 15 minutes whether the hallway vacuum is paused. If it is, Home Assistant sends a reminder so you can decide whether to resume it or clear an obstacle.

- **Trigger:** Every 15 minutes
- **Condition:** Vacuum is paused
- **Target:** Hallway vacuum
- **Action:** Send notification via mobile\_app\_phone

#### YAML example for a paused vacuum reminder

**\*\*automation\*\***

```
automation: |
 alias: "Reminder for paused vacuum"
 triggers:
 - trigger: time_pattern
 minutes: "/15"
 conditions:
 - condition: vacuum.is_paused
 target:
 entity_id: vacuum.hallway
 options:
 behavior: any
 actions:
 - action: notify.mobile_app_phone
 data:
 title: "Vacuum is paused"
 message: "The hallway vacuum is still paused."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Conditions/Vacuum.Is Returning

---

The **Vacuum cleaner is returning** condition passes when one or more targeted vacuums are returning to their dock or base.

Use this when you only want an automation to run while the robot is on its way home, like turning on a light near the dock, delaying another routine, or waiting to start cleanup until the path is clear again.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. From the search box, search for and select **Vacuum: Vacuum cleaner is returning**.
5. Under **Targets**, select the vacuum entity, an area, a floor, or a label.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All**.
7. Under **For at least**, enter how long the vacuum must keep returning before the condition passes.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple vacuums are targeted, controls how results combine. Pick **Any** to pass if at least one targeted vacuum is returning, or **All** to pass only when every targeted vacuum is returning. required: true For at least: description: The time the vacuum must keep returning before the condition passes. required: false

## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `vacuum.is_returning`. A basic example looks like this:

### condition

```
condition: |
 condition: vacuum.is_returning
 target:
 entity_id: vacuum.downstairs
 options:
 behavior: any
```

This passes when `vacuum.downstairs` is returning to its dock.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: The time the vacuum must keep returning before the condition passes. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

## Good to know

---

- Entities with state `unavailable` or `unknown` are ignored when Home Assistant evaluates the condition.
- With **Any** (default), the condition passes if at least one targeted vacuum is returning.
- With **All**, the condition passes only if every targeted vacuum that Home Assistant can evaluate is returning.
- If every targeted vacuum is `unavailable` or `unknown`, **Any** fails and **All** passes.



## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

---

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn on the dock light if the vacuum is returning

At night, this automation checks whether the vacuum is currently returning to the dock. If it is, the nearby light turns on to help light the final part of its path.

- **Trigger:** Time: 22:00
- **Condition:** Vacuum is returning
- **Target:** Downstairs vacuum
- **Action:** Turn on light

#### YAML example for lighting the dock area

**\*\*automation\*\***

```
automation: |
 alias: "Light dock area for returning vacuum"
 triggers:
 - trigger: time
 at: "22:00:00"
 conditions:
 - condition: vacuum.is_returning
 target:
 entity_id: vacuum.downstairs
 options:
 behavior: any
 actions:
 - action: light.turn_on
 target:
 entity_id: light.dock_area
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Conditions/Window.Is Closed

---

The **Window is closed** condition passes when a targeted window is currently closed. Use it when you want to make sure the house is shut before you lock a door, arm an alarm, turn heating back on, or continue only if a window has stayed closed for a while.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area the window is in, like your bedroom or kitchen. You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Window is closed**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple windows are targeted.
7. Under **For at least**, set how long the window must have been closed before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple windows are targeted, controls how results combine. Pick **Any** to pass if at least one targeted window is closed, or **All** to pass only when every targeted window is closed. required: true For at least: description: How long the window must have been closed before the condition passes. Set to zero to pass immediately. required: true

## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `window.is_closed`. A basic example looks like this:

### condition

```
condition: |
 condition: window.is_closed
 target:
 entity_id: binary_sensor.bedroom_window
```

This passes when `binary_sensor.bedroom_window` is currently closed.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple windows are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: > Duration the window must have been closed before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.



## Good to know

---

- Windows that are unavailable ( `unavailable` ) or have an unknown state ( `unknown` ) do not count as closed. Home Assistant skips them and evaluates the condition using the remaining targeted windows.
- This condition works with binary sensors and covers that use the `window` device class.
- To check the opposite state, use [Window is open](#).

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: lock front door at night only if all downstairs windows are closed

Before locking up for the night, you can check that every downstairs window is already shut.

- **Trigger:** Time: 23:00
- **Condition:** Window is closed
- **Target:** Downstairs windows label
- **Condition passes if:** All
- **Action:** Lock: Lock

#### YAML example for locking up after all windows are closed

**\*\*automation\*\***

```
automation: |
 alias: "Lock the front door when all downstairs windows are closed"
 triggers:
 - trigger: time
 at: "23:00:00"
 conditions:
 - condition: window.is_closed
 target:
 label_id: downstairs_windows
 options:
 behavior: all
 for: "00:00:00"
 actions:
 - action: lock.lock
 target:
 entity_id: lock.front_door
```

## Automation: restart the bedroom air conditioner only if the window has been closed for 15 minutes

If you use a motorized bedroom window to cool the room naturally, you might only want to turn the bedroom air conditioner back on after the window has been closed for a while.

- **Trigger:** Time pattern: Every 15 minutes
- **Condition:** Window is closed
- **Target:** Bedroom roof window cover
- **Condition passes if:** Any
- **For at least:** 00:15:00
- **Action:** Climate: Turn on the bedroom air conditioner

### YAML example for restarting the bedroom air conditioner after the window stays closed

**\*\*automation\*\***

```
automation: |
 alias: "Restart bedroom air conditioner after the window stays closed"
 triggers:
 - trigger: time_pattern
 minutes: "/15"
 conditions:
 - condition: window.is_closed
 target:
 entity_id: cover.bedroom_roof_window
 options:
 behavior: any
 for: "00:15:00"
 actions:
 - action: climate.turn_on
 target:
 entity_id: climate.bedroom
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Conditions/Window.Is Open

---

The **Window is open** condition passes when a targeted window is currently open. Use it when you want an automation to continue only if fresh air is coming in, or when you want to warn someone before leaving the house with a window still open.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

To use this condition in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **And if** section, select **Add condition**.
4. Select what you want to check. Under **By target** (see [Targets](#)), pick the area the window is in, like your bedroom or kitchen. You can also select a floor, a device, a specific entity, or a label.
5. From the conditions shown for that target, select **Window is open**.
6. Under **Condition passes if** (see [Behavior](#)), pick **Any** or **All** to control how the check behaves when multiple windows are targeted.
7. Under **For at least**, set how long the window must have been open before the condition passes. Leave it at zero to pass immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Condition passes if: description: When multiple windows are targeted, controls how results combine. Pick **Any** to pass if at least one targeted window is open, or **All** to pass only when every targeted window is open. required: true For at least: description: How long the window must have been open before the condition passes. Set to zero to pass immediately. required: true



## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this condition as `window.is_open`. A basic example looks like this:

### condition

```
condition: |
 condition: window.is_open
 target:
 entity_id: binary_sensor.kitchen_window
```

This passes when `binary_sensor.kitchen_window` is currently open.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple windows are targeted, controls how results combine. Accepts `all` or `any`. required: true type: string default: any for: description: > Duration the window must have been open before the condition passes. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
- **All:** the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.

## Good to know

---

- Windows that are unavailable ( `unavailable` ) or have an unknown state ( `unknown` ) do not count as open. Home Assistant skips them and evaluates the condition using the remaining targeted windows.
- This condition works with binary sensors and covers that use the `window` device class.
- To check the opposite state, use [Window is closed](#).

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

## More examples

---

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: get reminder if any downstairs window is still open at bedtime

At bedtime, a reminder can help you notice an open window before you lock up for the night.

- **Trigger:** Time: 22:30
- **Condition:** Window is open
- **Target:** Downstairs windows label
- **Condition passes if:** Any
- **Action:** Send a mobile notification

#### YAML example for a bedtime open-window reminder

**\*\*automation\*\***

```
automation: |
 alias: "Remind me if a downstairs window is open at bedtime"
 triggers:
 - trigger: time
 at: "22:30:00"
 conditions:
 - condition: window.is_open
 target:
 label_id: downstairs_windows
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: notify.mobile_app_phone
 data:
 title: "Window still open"
 message: "At least one downstairs window is still open."
```

## Automation: turn off heater if roof window has been open for 10 minutes

If a motorized roof window has been open for a while, turning off the heater can help avoid wasting energy.

- **Trigger:** Time pattern: Every 10 minutes
- **Condition:** Window is open
- **Target:** Bedroom roof window cover
- **Condition passes if:** Any
- **For at least:** 00:10:00
- **Action:** Climate: Turn off

### YAML example for turning off heating when a roof window stays open

**\*\*automation\*\***

```
automation: |
 alias: "Turn off heating if the roof window stays open"
 triggers:
 - trigger: time_pattern
 minutes: "/10"
 conditions:
 - condition: window.is_open
 target:
 entity_id: cover.bedroom_roof_window
 options:
 behavior: any
 for: "00:10:00"
 actions:
 - action: climate.turn_off
 target:
 entity_id: climate.bedroom
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---



## Dashboards/Alarm Panel

---

The alarm panel card allows you to arm and disarm your [alarm control panel](#) integrations.

Screenshot of the alarm panel card Screenshot of the alarm panel card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>alarm-panel</code>
<code>entity</code>	string	Yes		Entity ID of <code>alarm_control_panel</code> domain.
<code>name</code>	[string, map, list]	No	Current state of the alarm entity.	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>states</code>	list	No	<code>" arm_home, arm_away "</code>	Arm Custom Bypass
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .

## Examples

Title example:

```
type: alarm-panel
name: House Alarm
entity: alarm_control_panel.alarm
```

Screenshot of the alarm panel card Screenshot of the alarm panel card.

Define the state list:

```
type: alarm-panel
name: House Alarm
entity: alarm_control_panel.alarm
states:
 - arm_home
 - arm_away
 - arm_night
 - armed_custom_bypass
```

## Dashboards/Area

---

The area card lets you control and monitor an individual area.

Screenshot of the area cards Screenshot of the area cards.

All options for this card can be configured via the user interface.

As shown in the screenshot of the area cards, they can display values and buttons of entities and devices that you have assigned the area to, such as:

- Buttons for entities such as fan, light, and switch that are in the area of the card.
- The measured value of a sensor, if the sensor is in the area of the card or if the sensor is assigned to the area in **Settings > Areas, labels & zones**.
- The median of the values measured by temperature sensors, if more than one temperature sensor is in the area of the card.
- The median of the values measured by humidity sensors, if more than one humidity sensor is in the area of the card.
- A motion sensor in the top left of the card, if a motion sensor is in the area of the card.
- The camera feed instead of the area picture, if a camera is added to the area of the card.



**Note:** The device is in an area if you have previously [assigned the area to the device](#).

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## Adding buttons to the area card for controlling devices

---

You can add buttons to the area card that will allow you to control different devices in that area.

1. Depending on your goal, do one of the following:
2. Assign the area of the card to the device by following the steps in [Assigning an area to a device](#).
3. Assign the area of the card to a group of devices by following the steps in [Assigning an area to multiple items](#).
4. Go to your dashboard and, in the top-right corner, select the `mdi:pencil` button.
5. In the area card that you have previously created, select **Edit**.
6. Expand the **Features** section and select **Add feature > Area controls**.
7. You can also:
8. Define the **Features position** by selecting **Bottom** or **Inline**.
9. Customize controls to add a button for each device or entity, for example.
  1. Select the `mdi:pencil` button next to **Area controls**.
  2. Turn on **Customize controls**.
  3. Select **Controls** and then select the entity from the list.
  4. Select **Save**.

If you want to control only certain devices that are assigned to an area altogether, you can still use an area card. [Create a new area](#) and then follow the previous steps using the new area.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>area</code>
<code>area</code>	string	Yes		ID of the <code>area</code> .
<code>color</code>	string	No		Set the color for the icon and the hover/focus state. It accepts <a href="#">color token</a> or hex color code.
<code>display_type</code>	string	No	"picture"	Defines the card's display style. Options include <code>compact</code> (a minimal layout), <code>icon</code> (shows an area icon), <code>picture</code> (displays an image of the area), or <code>camera</code> (shows the live camera feed).
<code>camera_view</code>	string	No	auto	If showing a camera, <code>live</code> will show the live view if <code>stream</code> is enabled.
<code>aspect_ratio</code>	string	No	"16:9"	Forces the height of the image to be a ratio of the width. Valid formats: Height percentage value ( <code>23%</code> ) or ratio expressed with colon or "x" separator ( <code>16:9</code> or <code>16x9</code> ). For a ratio, the second element can be omitted and will default to "1" ( <code>1.78</code> equals <code>1.78:1</code> ).
<code>tap_action</code>	map	No	none	Action taken on card tap. See <a href="#">action documentation</a> .
<code>image_tap_action</code>	map	No	" <code>more-info</code> for camera display type, <code>none</code> otherwise"	Action taken on image tap (only available when <code>display_type</code> is <code>icon</code> , <code>picture</code> or <code>camera</code> ). When not configured, image taps use the card's <code>tap_action</code> . See <a href="#">action documentation</a> .
<code>alert_classes</code>	list	No	"moisture, motion"	A list of binary sensor device classes which will populate alert icons in the card when the state is on. If the display type is set to <code>compact</code> , only the first alert icon will be displayed.
<code>sensor_classes</code>	list	No	"temperature, humidity"	A list of sensor device classes to display for the area. Most classes (such as temperature, humidity, or pressure) show the median value when multiple sensors are



Option	Type	Required	Default	Description
				present. Sensors representing cumulative measurements (such as power, energy, gas, or water) show the sum instead.
<code>features</code>	list	No		Additional widgets to control entities in the area. See <a href="#">available features</a> .
<code>features_position</code>	string	No	bottom	Position of the features on the area card. Can be <code>bottom</code> or <code>inline</code> . Only the first feature will be displayed when the option is set to <code>inline</code> .
<code>exclude_entities</code>	list	No		A list of entities that will be excluded from the card. It will affect <code>sensor_classes</code> , <code>alert_classes</code> , and <code>features</code> .

## Example

Basic example:

```
type: area
area: bedroom
```

Complex example

```
type: area
area: bedroom
display_type: picture
tap_action:
 action: navigate
 navigation_path: /lovelace/my_bedroom
sensor_classes:
 - temperature
 - humidity
alert_classes:
 - moisture
 - motion
features:
 - type: area-controls
```

## Available colors

---

The following colors are available to colorize the area card: `primary`, `accent`, `disabled`, `red`, `pink`, `purple`, `deep-purple`, `indigo`, `blue`, `light-blue`, `cyan`, `teal`, `green`, `light-green`, `lime`, `yellow`, `amber`, `orange`, `deep-orange`, `brown`, `grey`, `blue-grey`, `black`, `white`, or any hex color code (for example, `#93c47d`).

---

## Dashboards/Button

---

The button card allows you to add buttons to perform tasks.

Screenshot of three button cards Screenshot of three button cards.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## Card settings

Option	Description
Entity	The entity ID the card interacts with, for example, <code>light.living_room</code> .
Name	The button name that is displayed on the card. If this field is left blank and the card interacts with an entity, the button name defaults to the entity name. Otherwise, no name is displayed.
Icon	The icon that is displayed on the card. If this field is left blank and the card interacts with an entity, the icon defaults to the entity domain icon. Otherwise, no icon is displayed.
Icon Height	The height of the icon, in pixels.
Color	The color of the icon.
Theme	Name of any loaded theme to be used for this card. For more information about themes, see the <a href="#">frontend documentation</a> .
Show Name	A toggle to show or hide the button name.
Show State	A toggle to show or hide the state of the entity.
Show Icon	A toggle to show or hide the icon.
Tap Action	The action taken on card tap. For more information, see the <a href="#">action documentation</a> .
Hold Action	The action taken on card tap and hold. For more information, see the <a href="#">action documentation</a> .

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>button</code>
<code>entity</code>	string	No		The entity ID the card interacts with, for example, <code>light.living_room</code> .
<code>name</code>	[string, map, list]	No	Entity name	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> . It defaults to the entity name only if the card interacts with an entity. Otherwise, if not configured, no name is displayed.
<code>icon</code>	string	No	Entity domain icon	The icon that is displayed on the card. It defaults to the entity domain icon only if the card interacts with an entity. Otherwise, if not configured, no icon is displayed.
<code>show_name</code>	boolean	No	"true"	If false, the button name is not shown on the card.
<code>show_icon</code>	boolean	No	"true"	If false, the icon is not shown on the card.
<code>show_state</code>	boolean	No	"false"	Show state.
<code>icon_height</code>	string	No	auto	The height of the icon. Any CSS value may be used.
<code>color</code>	string	No	state	Set the color for the icon. By default, the color is based on <code>state</code> , <code>domain</code> , and <code>device_class</code> of your entity. It accepts <a href="#">color token</a> or hex color code.
<code>tap_action</code>	map	No		The action taken on card tap. For more information, see the <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		The action taken on card tap and hold. For more information, see the <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		The action taken on card double-tap. For more information, see the <a href="#">action documentation</a> .
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>action_name</code>	string	No	Run	Override the default action name for a button row.

## Examples

Basic example:

```
type: button
entity: light.living_room
```

Button card with a button name and a [script](#) that runs when card is tapped:

Screenshot of the Button card with script action Screenshot of the button card with script action.

```
type: button
name: Turn Off Lights
show_state: false
tap_action:
 action: perform-action
 perform_action: script.turn_on
 data:
 entity_id: script.turn_off_lights
```

Example of 4 buttons on a vertical stack card:

Screenshot of a vertical stack card with 4 buttons and an entity selector Screenshot of a vertical stack card with 4 buttons and an entity selector.

The image shows a vertical stack card with 4 buttons arranged in a horizontal stack card and an entity selector. The buttons use the toggle action to run a script, for example, the Netflix script, which starts up the TV and opens Netflix. To learn how to create scripts, refer to [scripts](#).



```

type: vertical-stack
cards:
 - entities:
 - entity: input_select.living_room_scene
 name: Scene
 show_header_toggle: false
 type: entities
 - type: horizontal-stack
 cards:
 - name: Watch Netflix
 entity: script.netflix
 type: button
 tap_action:
 action: toggle
 hold_action:
 action: more-info
 show_name: true
 show_icon: true
 - name: Watch YouTube
 entity: script.youtube
 type: button
 tap_action:
 action: toggle
 hold_action:
 action: more-info
 show_name: true
 show_icon: true
 - name: Wake PC
 entity: script.wake_on_lan
 type: button
 tap_action:
 action: toggle
 icon: mdi:desktop-tower
 show_name: true
 show_icon: true
 show_state: false
 - name: Go to sleep
 entity: script.sleep
 type: button
 tap_action:
 action: toggle
 icon: mdi:sleep
 hold_action:
 action: more-info
 show_name: true
 show_icon: true

```

## Available colors

---

The following colors are available to colorize the button card: `primary`, `accent`, `disabled`, `red`, `pink`, `purple`, `deep-purple`, `indigo`, `blue`, `light-blue`, `cyan`, `teal`, `green`, `light-green`, `lime`, `yellow`, `amber`, `orange`, `deep-orange`, `brown`, `grey`, `blue-grey`, `black`, `white`, or any hex color code (for example, `#93c47d`).

---

## Dashboards/Calendar

---

The calendar card displays your calendar entities in a month, day, and list view (7 days).

Screenshot of the calendar card Screenshot of the calendar card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>calendar</code>
<code>title</code>	string	No		The title of the card.
<code>initial_view</code>	string	No		The view that will show first when the card is loaded onto the page. Options are <code>dayGridMonth</code> , <code>dayGridDay</code> , and <code>listWeek</code> . Note that <code>listWeek</code> does show the next 7 days, not a calendar week.
<code>entities</code>	list	Yes		A list of calendar entities that will be displayed in the card.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .

## Examples

```
type: calendar
entities:
 - calendar.calendar_1
 - calendar.calendar_2
```

## Dashboards/Clock

---

The Clock card shows the current time in a variety of formats, sizes and time zones.

Screenshot of the clock card Screenshot of the clock card

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## Card settings

Option	Type	Required	Default	Description
<code>title</code>	string	No		Adds a title to the top of the card
<code>clock_style</code>	list	No	digital	Analog clock style.
<code>clock_size</code>	list	No	small	Large clock size.
<code>show_seconds</code>	boolean	No	false	Shows seconds alongside the clock, providing the time format is in a 12-hour format.
<code>no_background</code>	boolean	No	false	Removes the background of the clock card.
<code>time_format</code>	string	No		Allows the time format to be changed on a per-card level. Defaults to the user profile setting.
<code>time_zone</code>	string	No		Change the timezone used for the time on a per-card level. Defaults to the user profile setting.
<code>analog_options</code>	list	No	hour	Minute ticks are shown.

## Examples

Basic example:

```
type: clock
```

Screenshot of the basic clock card Screenshot of the basic clock card

Example of a larger clock card for tablet dashboards:

```
type: clock
clock_size: large
time_format: "12"
show_seconds: true
```

Screenshot of a large sized, 12 hour clock card showing am/pm and seconds Screenshot of a large sized, 12 hour clock card showing am/pm and seconds

A medium-sized clock card better suited for desktop dashboards:

```
type: clock
clock_size: medium
time_format: "12"
show_seconds: false
```



Screenshot of a medium sized, 12 hour clock showing am/pm Screenshot of a medium sized, 12 hour clock showing am/pm

A medium-sized, 24 hour clock using the London timezone with a title

```
type: clock
clock_size: medium
time_zone: Europe/London
title: London 🇬🇧
```

Screenshot of a medium sized, 24 hour clock showing seconds based in London along with a title Screenshot of a medium sized, 24 hour clock showing seconds based in London along with a title

A medium-sized, 12 hour clock using the New York timezone with a title

```
type: clock
clock_size: medium
time_format: "12"
time_zone: America/New_York
title: New York 🇺🇸
```

Screenshot of a medium sized, 12 hour clock showing am/pm and seconds based in New York along with a title Screenshot of a medium sized, 12 hour clock showing am/pm and seconds based in New York along with a title

Analog clock with no border and hour ticks:

```
type: clock
clock_style: analog
clock_size: medium
analog_options:
 border: false
 ticks: hour
```

Screenshot of a medium sized, analog clock and hour ticks Screenshot of a medium sized, analog clock and hour ticks

Analog clock with border and minute ticks showing seconds:

```
type: clock
clock_style: analog
clock_size: medium
show_seconds: true
analog_options:
 border: true
 ticks: minute
```

Screenshot of a medium sized, analog clock and minute ticks showing seconds Screenshot of a medium sized, analog clock and minute ticks showing seconds

Analog clock with a title, no ticks and border with seconds:

```
type: clock
clock_style: analog
clock_size: medium
show_seconds: true
analog_options:
 border: true
 ticks: none
title: Mountain Time
```

Screenshot of a medium sized, analog clock with a title, no ticks and border showing seconds  
Screenshot of a medium sized, analog clock with a title, no ticks and border showing seconds

---

## Dashboards/Conditional

---

The conditional card displays another card based on conditions.

Screenshot of the conditional card

Most options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. [Add a card and customize actions and features](#) to your dashboard.

Note that while editing the dashboard, the card will always be shown, so be sure to exit editing mode to test the conditions.

The conditional card can still be used. However, it is now possible to define a setting to show or hide a card conditionally directly on each card type, on its [Visibility](#) tab.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		conditional
<code>conditions</code>	list	Yes		List of conditions to check. See <a href="#">available conditions</a> .
<code>card</code>	map	Yes		Card to display if all conditions match.

## Examples

---

Only show when all the conditions are met:

```
type: conditional
conditions:
 - condition: state
 entity: light.bed_light
 state: "on"
 - condition: state
 entity: light.bed_light
 state_not: "off"
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
card:
 type: entities
 entities:
 - device_tracker.demo_paulus
 - cover.kitchen_window
 - group.kitchen
 - lock.kitchen_door
 - light.bed_light
```

Example condition where only one of the conditions needs to be met:

```
type: conditional
conditions:
 - condition: or
 conditions:
 - condition: state
 entity: binary_sensor.co_alert
 state: 'on'
 - condition: state
 entity: binary_sensor.rookmelder
 state: 'on'
card:
 type: entities
 entities:
 - binary_sensor.co_alert
 - binary_sensor.rookmelder
```

## Conditions options

### State

Tests if an entity has a specified state.

```
condition: state
entity: climate.thermostat
state: heat
```

```
condition: state
entity: climate.thermostat
state_not: "off"
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>state</code>
<code>entity</code>	string	Yes		Entity ID.
<code>state</code>	[list, string]	No		Entity state or ID to be equal to this value. Can contain an array of states.*
<code>state_not</code>	[list, string]	No		Entity state or ID to not be equal to this value. Can contain an array of states.*

\*one is required ( `state` or `state_not` )

### Numeric state

Tests if an entity state matches the thresholds.

```
condition: numeric_state
entity: sensor.outside_temperature
above: 10
below: 20
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>numeric_state</code>
<code>entity</code>	string	Yes		Entity ID.
<code>above</code>	string	No		Entity state or ID to be above this value.*
<code>below</code>	string	No		Entity state or ID to be below this value.*

\*at least one is required ( `above` or `below` ), both are also possible for values between.

## Screen

Specify the visibility of the card per screen size. Some screen size presets are available in the UI but you can use any CSS media query you want in YAML.

```
condition: screen
media_query: "(min-width: 1280px)"
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>screen</code>
<code>media_query</code>	string	Yes		Media query to check which screen size are allowed to display the card.

## User

Specify the visibility of the card per user.

```
condition: user
users:
 - 581fca7fdc014b8b894519cc531f9a04
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>user</code>
<code>users</code>	list	Yes		User ID that can see the card (unique hex value found on the Users configuration page).

## Location

Specify the visibility of the card based on the current user's current location. The location is based on the state of the `person` entity associated with the current user. If the current user does not have a `person` entity, this condition will always resolve to false.

```
condition: location
locations:
 - home
 - Home Neighborhood
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>location</code>
<code>locations</code>	list	Yes		A list of zones, which if any match the current state of the <code>person</code> , will cause this condition to be true.



## Time

Specify the visibility of the card based on the current time and day of the week.

```
condition: time
after: "08:00"
before: "17:00"
weekdays:
 - mon
 - tue
 - wed
 - thu
 - fri
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>time</code>
<code>after</code>	string	No		Time in 24-hour format (HH:MM) after which the card should be visible.*
<code>before</code>	string	No		Time in 24-hour format (HH:MM) before which the card should be visible.*
<code>weekdays</code>	list	No		List of weekdays on which the card should be visible. Valid values are <code>mon</code> , <code>tue</code> , <code>wed</code> , <code>thu</code> , <code>fri</code> , <code>sat</code> , <code>sun</code> .

At least one of `after` or `before` must be used for this condition to be valid. Both can be used together to define a time range as in the example above.

## And

Specify that all conditions must be met.

```
condition: and
conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>and</code>
<code>conditions</code>	list	No		List of conditions to check. See <a href="#">available conditions</a> .

## Or

Specify that at least one of the conditions must be met.

```
condition: or
conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>or</code>
<code>conditions</code>	list	No		List of conditions to check. See <a href="#">available conditions</a> .

## Not

Specify that at least one of the conditions must not be met.

```
condition: not
conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

Option	Type	Required	Default	Description
<code>condition</code>	string	Yes		<code>not</code>
<code>conditions</code>	list	No		List of conditions to check. See <a href="#">available conditions</a> .

---

## Dashboards/Energy

---

This is a list of all the cards and badges used in the energy dashboard. You can also place them anywhere you want in your dashboard.

You can add these cards using the visual card editor or by editing the YAML directly. In the visual editor, these are located in the **Energy cards** section of the card picker dialog.

You can configure them on the energy configuration page.

## Energy date picker

Screenshot of the energy date selection card Screenshot of the Energy date selection card.

This card allows you to pick what data to show. Changing it in this card will influence the data in all other cards. Specific dates and ranges can be selected by opening the date range picker. The current period can be compared to the previous one using the compare data option within the menu.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-date-selection</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>vertical_opening_direction</code>	string	No	auto	<code>up</code> , <code>down</code> or <code>auto</code> . Determines the vertical direction to open the date range picker. <code>auto</code> changes it based on the screen size.
<code>opening_direction</code>	string	No	auto	<code>left</code> , <code>right</code> , <code>center</code> or <code>auto</code> . Determines the horizontal direction to open the date range picker. <code>auto</code> changes it based on the screen size.
<code>disable_compare</code>	boolean	No	false	When true, will disable the option to compare data periods.

### Example

```
type: energy-date-selection
```

# Energy compare alert

Screenshot of the energy compare alert card Screenshot of the energy compare alert card.

This card shows which date ranges are being compared, and provides the option to switch between different compare modes.

The card is only visible when comparing data periods, otherwise it will be hidden.

## YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-compare-card</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.

## Example

```
type: energy-compare-card
```

## Energy usage graph

---

Screenshot of the energy usage graph card Screenshot of the Energy usage graph card.

The energy usage graph card shows the amount of energy your house has consumed, and from what source this energy came. It will also show the amount of energy your have returned to the grid.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-usage-graph</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>title</code>	string	No		When defined, shows a card header with the title string and total energy consumed chip.

### Example

```
type: energy-usage-graph
```

## Solar production graph

---

Screenshot of the solar graph card Screenshot of the Solar production graph card.

The solar production graph card shows the amount of energy your solar panels have produced per source, and if setup and available the forecast of the solar production.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-solar-graph</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>title</code>	string	No		When defined, shows a card header with the title string and total solar produced chip.

### Example

```
type: energy-solar-graph
```

## Gas consumption graph

---

Screenshot of the gas consumption graph card Screenshot of the gas consumption graph card.

The gas consumption graph card shows the amount of gas consumed per source.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-gas-graph</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>title</code>	string	No		When defined, shows a card header with the title string and total gas consumed chip.

### Example

```
type: energy-gas-graph
```



# Water consumption graph

Screenshot of the water consumption graph card Screenshot of the water consumption graph card.

The water consumption graph card shows the amount of water consumed per source.

## YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
type	string	Yes		energy-water-graph
collection_key	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
title	string	No		When defined, shows a card header with the title string and total water consumed chip.

## Example

```
type: energy-water-graph
```

## Water Sankey graph

Screenshot of the water Sankey graph card Screenshot of the water sankey graph card.

The water Sankey graph shows the flow of water consumption in your home. It visualizes how water flows from sources to the various consumers. Devices are grouped into floors and areas if these are configured.

This card displays historical water data based on the selected date range from the energy date picker.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>water-sankey</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>title</code>	string	No		The title of the card.
<code>layout</code>	string	No	auto	<code>vertical</code> , <code>horizontal</code> or <code>auto</code> . Determines the orientation (flow direction) of the card. <code>auto</code> changes it based on the screen size.
<code>group_by_area</code>	boolean	No	true	Whether to group the devices by area
<code>group_by_floor</code>	boolean	No	true	Whether to group the devices by floor

### Examples

```
type: water-sankey
```

The following example orients the flow from left to right:

```
type: water-sankey
layout: horizontal
```

## Energy distribution

---

Screenshot of the energy distribution card Screenshot of the Energy distribution card.

The energy distribution card shows how the energy flowed, from the grid to your house, from your solar panels to your house and/or back to the grid.

If setup, it will also tell you how many kWh of the energy you got from the grid was produced without using fossil fuels.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-distribution</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>title</code>	string	No		The title of the card.
<code>link_dashboard</code>	boolean	No	false	Whether to include a link to the energy dashboard.

### Example

```
type: energy-distribution
link_dashboard: true
```

## Energy sources table

Screenshot of the energy sources table card Screenshot of the Energy sources table card.

The energy sources table card shows all your energy sources, and the corresponding amount of energy. If setup, it will also show the costs and compensation per source and the total.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-sources-table</code>
<code>types</code>	list	No		If defined, table displays listed types of consumption only. Valid values are: <code>grid</code> , <code>solar</code> , <code>battery</code> , <code>gas</code> , and <code>water</code> .
<code>show_only_totals</code>	boolean	No	false	Display table as a summarized version with only the resource totals listed.

### Example

```
type: energy-sources-table
types:
 - gas
 - water
```

## Grid neutrality gauge

---

Screenshot of the grid neutrality gauge card Screenshot of the Grid neutrality gauge card.

The grid neutrality gauge card represents your energy dependency. If the needle is in the purple, you returned more energy to the grid than you consumed from it. If it's in the blue, you consumed more energy from the grid than you returned.

### Example

```
type: energy-grid-neutrality-gauge
```

## Grid energy balance

---

Screenshot of the grid energy balance card Screenshot of the Grid energy balance card.

The grid energy balance card shows your net grid energy as an equation: imported energy minus exported energy. A positive value means you imported more energy from the grid than you exported. A negative value means you exported more energy to the grid than you imported. It includes a visual bar gauge that represents the ratio between imported and exported energy, with the bar filling from a center line toward the dominant direction.

### Example

```
type: energy-grid-balance
```

## Solar consumed gauge

---

Screenshot of the solar consumed gauge card Screenshot of the Solar consumed gauge card.

The solar consumed gauge represents how much of the solar energy was used by your home and was not returned to the grid. If you frequently return a lot, try to conserve this energy by installing a battery or buying an electric car to charge.

### Example

```
type: energy-solar-consumed-gauge
```

## Carbon consumed gauge

---

Screenshot of the carbon consumed gauge card Screenshot of the Carbon consumed gauge card.

The carbon consumed gauge card represents how much of the energy consumed by your home was generated using non-fossil fuels like solar, wind and nuclear. It includes the solar energy you generated your self.

### Example

```
type: energy-carbon-consumed-gauge
```



## Self-sufficiency gauge

---

Screenshot of the self-sufficiency gauge card Screenshot of the self-sufficiency gauge card.

The self-sufficiency gauge represents how self-sufficient your home is. If you rely on grid imports, this value decreases. You can increase this value by adding more solar capacity or battery storage.

### Example

```
type: energy-self-sufficiency-gauge
```

## Devices energy graph

Screenshot of the devices energy graph card Screenshot of the devices energy graph card.

The devices energy graph shows the energy usage per device. It is sorted by usage.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-devices-graph</code>
<code>title</code>	string	No		The title of the card.
<code>max_devices</code>	integer	No		By default, this card will show all your devices. Optionally, the number of devices can be limited by adding the <code>max_devices</code> option and specifying the maximum number of devices to show. If there are more devices available than shown, the devices with the highest energy usage are shown.
<code>hide_compound_stats</code>	boolean	No	false	Hide upstream energy devices like breakers. These are devices that are set as <code>included_in_stat</code> of another device.

### Examples

```
type: energy-devices-graph
```

The following example limits the number of shown devices to 5:

```
type: energy-devices-graph
max_devices: 5
```

## Detail devices energy graph

---

Screenshot of the devices energy graph card Screenshot of the detail devices energy graph card.

The **Detail devices energy graph** card is similar to the **Devices energy graph** card, but shows the individual usage on a time scale.

By default, this card will show all your devices. Optionally, the number of devices can be limited by adding the `max_devices` option and specifying the maximum number of devices to show. If there are more devices available than shown, the devices with the highest energy usage are shown.

### Examples

```
type: energy-devices-detail-graph
```

The following example limits the number of shown devices to 5:

```
type: energy-devices-detail-graph
max_devices: 5
```

## Sankey energy graph

Screenshot of the sankey energy graph card Screenshot of the sankey energy graph card.

The sankey energy graph shows the flow of energy in your home. It starts with sources and flows into the various consumers. Devices are grouped into floors and areas if these are configured.

### YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>energy-sankey</code>
<code>title</code>	string	No		The title of the card.
<code>layout</code>	string	No	auto	<code>vertical</code> , <code>horizontal</code> or <code>auto</code> . Determines the orientation (flow direction) of the card. <code>auto</code> changes it based on the screen size.
<code>group_by_area</code>	boolean	No	true	Whether to group the devices by area
<code>group_by_floor</code>	boolean	No	true	Whether to group the devices by floor

### Examples

```
type: energy-sankey
```

The following example orients the flow from top to bottom:

```
type: energy-sankey
layout: vertical
```

# Power flow Sankey graph

Screenshot of the power Sankey graph card Screenshot of the power Sankey graph card.

The power Sankey graph shows the real-time flow of power in your home. Unlike the energy Sankey card, which shows historical energy data based on the selected date range, this card displays current power values and is not affected by the date picker selection.

It visualizes the instantaneous power flow from sources (like the grid, solar panels, and battery) to consumers in your home. Devices are grouped into floors and areas if these are configured.

## YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>power-sankey</code>
<code>collection_key</code>	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
<code>title</code>	string	No		The title of the card.
<code>layout</code>	string	No	auto	<code>vertical</code> , <code>horizontal</code> or <code>auto</code> . Determines the orientation (flow direction) of the card. <code>auto</code> changes it based on the screen size.
<code>group_by_area</code>	boolean	No	true	Whether to group the devices by area
<code>group_by_floor</code>	boolean	No	true	Whether to group the devices by floor

## Examples

```
type: power-sankey
```

The following example orients the flow from left to right:

```
type: power-sankey
layout: horizontal
```

# Power sources graph

Screenshot of the Sankey sources graph card Screenshot of the power Sankey graph card.

The power sources graph shows historical power data.

## YAML configuration

The following YAML options are available:

Option	Type	Required	Default	Description
type	string	Yes		power-sources-graph
collection_key	string	No		Collection key to use for the card. This links the card to a specific energy dashboard collection. If not provided, defaults to the current dashboard page URL.
title	string	No		The title of the card.
show_legend	boolean	No	true	Show or hide the legend

## Examples

```
type: power-sources-graph
```

## Power consumption badge

---

Screenshot of the power consumption badge Screenshot of the power consumption badge.

The power consumption badge displays the current total power consumption of your home. It calculates the total power by combining grid import, solar, and battery sources.

### YAML configuration

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>power-total</code>
<code>collection_key</code>	string	No	energy_dashboard	Collection key to use for the badge. This links the badge to a specific energy dashboard collection. Defaults to <code>energy_dashboard</code> .

### Example

```
type: power-total
```

## Gas flow rate badge

---

Screenshot of the gas flow rate badge Screenshot of the gas flow rate badge.

The gas flow rate badge displays the current total gas flow rate from all configured gas sources.

### YAML configuration

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>gas-total</code>
<code>collection_key</code>	string	No	energy_dashboard	Collection key to use for the badge. This links the badge to a specific energy dashboard collection. Defaults to <code>energy_dashboard</code> .

### Example

```
type: gas-total
```



## Water flow rate badge

---

Screenshot of the water flow rate badge Screenshot of the water flow rate badge.

The water flow rate badge displays the current total water flow rate from all configured water sources.

### YAML configuration

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>water-total</code>
<code>collection_key</code>	string	No	energy_dashboard	Collection key to use for the badge. This links the badge to a specific energy dashboard collection. Defaults to <code>energy_dashboard</code> .

### Example

```
type: water-total
```

## Using multiple collections

---

By default, all energy cards on the current dashboard are linked together. Any `energy-date-selection` cards on this dashboard control what data is shown. If there are none, a default date of today is used. When you add multiple date selection cards, they always show the same date. Any `energy-date-selection` card on a different dashboard does not affect energy cards on the current dashboard.

To enable multiple different date selections on the same dashboard, they must be linked to different collections. This is done using the `collection_key` parameter, either in the visual editor or in the card YAML, with a value of any custom string that begins with `energy_` (strings that do not start with `energy_` will generate an error).

All energy cards support use of `collection_key` option.

## Examples

Example view with multiple collections:

```
type: masonry
path: example
cards:
 - type: energy-date-selection
 - type: energy-date-selection
 collection_key: energy_2

 # This card is linked to the first (default) date selection
 - type: energy-usage-graph

 # This card is linked to the second date selection
 - type: energy-usage-graph
 collection_key: energy_2
```

## Dashboards/Entities

---

The entities card is the most common type of card. It groups items together into lists. It can be used to display an entity's state or attribute, but also contain buttons, web links, and more.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>entities</code>
<code>entities</code>	list	Yes		A list of entity IDs or <code>entity</code> objects or special row objects (see below).
<code>title</code>	string	No		Card title.
<code>icon</code>	string	No		An icon to display to the left of the title.
<code>show_header_toggle</code>	boolean	No	true	Button to turn on/off all entities.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>state_color</code>	boolean	No	false	Set to <code>true</code> to have icons colored when entity is active.
<code>header</code>	map	No		Header widget to render. See <a href="#">header documentation</a> .
<code>footer</code>	map	No		Footer widget to render. See <a href="#">footer documentation</a> .

## Options for entities

---

If you define entities as objects instead of strings (by adding `entity:` before entity ID), you can add more customization and configuration.

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>type</code>	string	No		Sets a custom card type: <code>custom:my-custom-card</code> . It also can be used to force entities with a default special row format to render as a simple state. You can do this by setting the type: <code>simple-entity</code> . This can be used, for example, to replace a helper with an editable control with a read-only value.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No		Overwrites icon or entity picture.
<code>image</code>	string	No		Overwrites entity picture.
<code>secondary_info</code>	string	No		Show additional info. Values: <code>entity-id</code> , <code>last-changed</code> , <code>last-updated</code> , <code>area</code> , <code>last-triggered</code> (only for automations and scripts), <code>position</code> or <code>tilt-position</code> (only for supported covers), <code>brightness</code> (only for lights).
<code>format</code>	string	No		How the state should be formatted. Currently only used for timestamp sensors. Valid values are: <code>relative</code> , <code>total</code> , <code>date</code> , <code>time</code> and <code>datetime</code> .
<code>action_name</code>	string	No		Button label (only applies to <code>script</code> and <code>scene</code> rows).
<code>state_color</code>	boolean	No	false	Set to <code>true</code> to have icons colored when entity is active.
<code>tap_action</code>	map	No		Action taken on row tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on row tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on row double tap. See <a href="#">action documentation</a> .
<code>confirmation</code>	map	No		For entities that display a button element in the row (for example, button, lock, script), this option adds a confirmation dialog to the press of the

Option	Type	Required	Default	Description
				button. See <a href="#">options for confirmation</a> for configuration options.



## Special row elements

Rather than only displaying an entity's state as a text output, the entities card supports multiple special rows for buttons, attributes, web links, dividers, sections, and more.

### Attribute

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>attribute</code>
<code>entity</code>	string	Yes		Entity ID.
<code>attribute</code>	string	Yes		Attribute to display from the entity.
<code>prefix</code>	string	No		Text before entity state.
<code>suffix</code>	string	No		Text after entity state.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No		Icon to use. Defaults to icon of entity.
<code>format</code>	string	No		How the attribute value should be formatted. Currently only supported for timestamp attributes. Valid values are: <code>relative</code> , <code>total</code> , <code>date</code> , <code>time</code> and <code>datetime</code> .

### Button

Row with an (optional) icon, label and a single text button at the end of the row that can trigger a defined action.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>button</code>
<code>entity</code>	string	No		Entity ID. Either <code>entity</code> or <code>name</code> (or both) needs to be provided.
<code>name</code>	string	No	"Friendly name of <code>entity</code> if specified."	Row label. Either <code>entity</code> or <code>name</code> (or both) needs to be provided.
<code>icon</code>	string	No		An icon to display to the left of the main label.
<code>action_name</code>	string	No	" <code>Run</code> "	Button label.
<code>tap_action</code>	map	Yes		Action taken on button tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on button tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on button double tap. See <a href="#">action documentation</a> .

## Buttons

Multiple buttons displayed in a single row next to each other. See examples further below.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>buttons</code>
<code>entities</code>	map	No	"true"	Action taken on button double tap. See <a href="#">action documentation</a> .

## Cast

Special row to start Home Assistant Cast.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>cast</code>
<code>dashboard</code>	string	No		Path to the dashboard of the view that needs to be shown.
<code>view</code>	string	Yes		Path to the view that needs to be shown.
<code>name</code>	string	No	Home Assistant Cast	Name to show in the row.
<code>icon</code>	string	No	" <code>hass:television</code> "	Icon to use.
<code>hide_if_unavailable</code>	boolean	No	false	Hide this row if casting is not available in the browser.

## Conditional

Special row that displays based on entity states.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>conditional</code>
<code>conditions</code>	list	Yes		List of conditions to check. See <a href="#">available conditions</a> .
<code>row</code>	map	Yes		Row to display if all conditions match. Can be any of the various supported rows described on this page.

## Divider

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>divider</code>
<code>style</code>	map	No	"height: 1px, background-color: var(--divider-color)"	Style the element using CSS.

## Section

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>section</code>
<code>label</code>	string	No		Section label.

## Weblink

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>weblink</code>
<code>url</code>	string	Yes		Website URL, or internal URL such as <code>/hassio/dashboard</code> or <code>/panel_custom_name</code> .
<code>name</code>	string	No	URL path	Link label.
<code>icon</code>	string	No	" <code>mdi:link</code> "	Icon to display, for example <code>mdi:home</code> .
<code>new_tab</code>	boolean	No	false	Open link in new tab. If link is external URL or a download link, this will automatically be true. Use if internal URL should be opened in new tab.
<code>download</code>	boolean	No	false	Is link a download?

## Examples

### Entity rows

```
type: entities
title: Entities card sample
show_header_toggle: true
header:
 image: "https://www.home-assistant.io/images/dashboards/header-footer/balloons"
 type: picture
entities:
 - entity: alarm_control_panel.alarm
 name: Alarm Panel
 - device_tracker.demo_paulus
 - switch.decorative_lights
 - group.all_lights
 - group.all_locks
```

### Buttons row

Above the divider are regular entity rows, below one of type `buttons`. Note that regular entity rows automatically show the entity name, whereas for buttons you have to explicitly specify a label / name.

Screenshot of buttons row Screenshot of buttons row.

```
type: entities
entities:
 - entity: light.office_ceiling
 - entity: light.dining_ceiling
 - type: divider
 - type: buttons
 entities:
 - entity: light.office_ceiling
 name: Office Ceiling
 - entity: light.dining_ceiling
 name: Dining Ceiling
```

### Other special rows

Screenshot of other special rows Screenshot of other special rows.

```
type: entities
title: Entities card sample
entities:
 - type: button
 icon: mdi:power
 name: Bed light transition
 action_name: Toggle light
 tap_action:
 action: perform-action
 perform_action: light.toggle
 data:
 entity_id: light.bed_light
 transition: 10
 - type: divider
 - type: attribute
 entity: sun.sun
 attribute: elevation
 name: Sun elevation
 prefix: "~"
 suffix: Units
 - type: conditional
 conditions:
 - entity: sun.sun
 state: above_horizon
 row:
 entity: sun.sun
 type: attribute
 attribute: azimuth
 icon: mdi:angle-acute
 name: Sun azimuth
 - type: section
 label: Section example
 - type: weblink
 name: Home Assistant
 url: https://www.home-assistant.io/
 icon: mdi:home-assistant
 - type: button
 name: Power cycle LibreELEC
 icon: mdi:power-cycle
 tap_action:
 action: perform-action
 confirmation:
 text: Are you sure you want to restart?
 perform_action: script.libreelec_power_cycle
```

---

## Dashboards/Entity Filter

---

The entity filter card allows you to define a list of entities that you want to track only when in a certain state. Very useful for showing lights that you forgot to turn off or show a list of people only when they're at home.

Screenshot of the entity filter card Screenshot of the entity filter card.

This type of card can also be used together with other cards that allow multiple entities, allowing you to use [glance](#) or [picture-glance](#). By default, it uses the [entities](#) card model.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.



## YAML configuration

This card can only be configured in YAML.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>entity-filter</code>
<code>entities</code>	list	Yes		A list of entity IDs or <code>entity</code> objects, see below.
<code>conditions</code>	list	No		List of conditions to check. See <a href="#">available conditions</a> .*
<code>state_filter</code>	list	No		(legacy) List of strings representing states or filters to check. See <a href="#">available legacy filters</a> .*
<code>card</code>	map	No	entities card	Extra options to pass down to the card rendering the result.
<code>show_empty</code>	boolean	No	true	Allows hiding of card when no entities returned by filter.

\*one is required ( `conditions` or `state_filter` )

### Options for entities

If you define entities as objects instead of strings (by adding `entity:` before entity ID), you can add more customization and configurations:

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>type</code>	string	No		Sets a custom card type: <code>custom:my-custom-card</code> .
<code>name</code>	string	No		Overwrites friendly name.
<code>icon</code>	string	No	Entity domain icon	Overwrites icon or entity picture. You can use any icon from <a href="#">Material Design Icons</a> . Prefix the icon name with <code>mdi:</code> , ie <code>mdi:home</code> .
<code>secondary_info</code>	string	No		Show additional info. Values: <code>entity-id</code> , <code>last-changed</code> .
<code>format</code>	string	No		How the state should be formatted. Currently only used for timestamp sensors. Valid values are: <code>relative</code> , <code>total</code> , <code>date</code> , <code>time</code> and <code>datetime</code> .
<code>conditions</code>	list	No		List of conditions to check. See <a href="#">available conditions</a> .*
<code>state_filter</code>	list	No		(legacy) List of strings representing states or filters to check. See <a href="#">available legacy filters</a> .*

\*only one filter will be applied: `conditions` or `state_filter` if `conditions` is not present

## Conditions options

---

You can specify multiple `conditions`, in which case the entity will be displayed if it matches every condition.

### State

Tests if an entity has a specified state.

```
type: entity-filter
entities:
 - climate.thermostat_living_room
 - climate.thermostat_bed_room
conditions:
 - condition: state
 state: heat
```

```
type: entity-filter
entities:
 - climate.thermostat_living_room
 - climate.thermostat_bed_room
conditions:
 - condition: state
 state_not: "off"
```

```
type: entity-filter
entities:
 - sensor.gas_station_1
 - sensor.gas_station_2
 - sensor.gas_station_3
conditions:
 - condition: state
 state: sensor.gas_station_lowest_price
```

{% configuration condition\_state %} condition: required: true description: "`state`" type: string  
state: required: false description: Entity state or ID to be equal to this value. Can contain an array of states. type: [list, string] state\_not: required: false description: Entity state or ID to not be equal to this value. Can contain an array of states. type: [list, string] entity: required: false description: An optional entity ID to be used for testing the state condition. If not provided, the state of the entity being displayed is tested. type: string

\*one is required ( `state` or `state_not` )

### Numeric state

Tests if an entity state matches the thresholds.

```

type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: numeric_state
 above: 10
 below: 20

```

{% configuration condition\_numeric\_state %} condition: required: true description: " **numeric\_state** " type: string above: required: false description: Entity state or ID to be above this value. *type: string below: required: false description: Entity state or ID to be below this value.* type: string entity: required: false description: An optional entity ID to be used for testing the numeric state condition. If not provided, the numeric state of the entity being displayed is tested. type: string

\*at least one is required ( **above** or **below** ), both are also possible for values between.

## Screen

Specify the visibility of the entity per screen size. Some screen size presets are available in the UI but you can use any CSS media query you want in YAML.

```

type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: screen
 media_query: "(min-width: 1280px)"

```

{% configuration condition\_screen %} condition: required: true description: " **screen** " type: string media\_query: required: true description: Media query to check which screen size are allowed to display the entity. type: string

## User

Specify the visibility of the entity per user.

```

type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04

```

{% configuration condition\_user %} condition: required: true description: " **user** " type: string  
users: required: true description: User ID that can see the entity (unique hex value found on the Users configuration page). type: list

## And

Specify that both conditions must be met.

```
type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: and
 conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

{% configuration condition\_and %} condition: required: true description: " **and** " type: string  
conditions: required: false description: List of conditions to check. See [available conditions](#). type: list

## Or

Specify that at least one of the conditions must be met.

```
type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: or
 conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

{% configuration condition\_or %} condition: required: true description: " **or** " type: string  
conditions: required: false description: List of conditions to check. See [available conditions](#). type: list

## Legacy state filters

---

### String filter

Show only active switches or lights in the house.

```
type: entity-filter
entities:
 - entity: light.bed_light
 name: Bed
 - light.kitchen_lights
 - light.ceiling_lights
state_filter:
 - "on"
```

Show only people that are at home using [glance](#):

```
type: entity-filter
entities:
 - device_tracker.demo_paulus
 - device_tracker.demo_anne_therese
 - device_tracker.demo_home_boy
state_filter:
 - home
card:
 type: glance
 title: People at home
```

Entity filter combined with glance card Entity filter combined with glance card.

You can also specify multiple `state_filter` conditions, in which case the entity will be displayed if it matches any condition.

If you define `state_filter` as objects instead of strings, you can add more customization to your filter, as described below.

### Operator filter

Tests if an entity state correspond to the applied `operator`.

{% configuration condition\_operator %} value: required: true description: String representing the state. type: string operator: required: true description: Operator to use in the comparison. Can be `==`, `<=`, `<`, `>=`, `>`, `!=`, `in`, `not in`, or `regex`. type: string attribute: required: false description: Attribute of the entity to use instead of the state. type: string

### Examples

Displays everyone who is at home or at work.

```

type: entity-filter
entities:
 - device_tracker.demo_paulus
 - device_tracker.demo_anne_therese
 - device_tracker.demo_home_boy
state_filter:
 - operator: "=="
 value: home
 - operator: "=="
 value: work
card:
 type: glance
 title: Who's at work or home

```

Specify filter for a single entity.

```

type: entity-filter
state_filter:
 - "on"
 - operator: ">"
 value: 90
entities:
 - sensor.water_leak
 - sensor.outside_temp
 - entity: sensor.humidity_and_temp
 state_filter:
 - operator: ">"
 value: 50
 attribute: humidity

```

Use a regex filter against entity attributes. This regex filter below looks for expressions that are 1 digit in length and where the number is between 0-7 (so show holidays today or in the next 7 days) and displays those holidays as entities in the Entity Filter card.

```

type: entity-filter
card:
 title: "Upcoming Holidays In Next 7 Days"
 show_header_toggle: false
state_filter:
 - operator: regex
 value: "^[0-7]{1})$"
 attribute: eta
entities:
 - entity: sensor.upcoming_ical_holidays_0
 - entity: sensor.upcoming_ical_holidays_1
 - entity: sensor.upcoming_ical_holidays_2
 - entity: sensor.upcoming_ical_holidays_3
 - entity: sensor.upcoming_ical_holidays_4
show_empty: false

```

## Dashboards/Entity

---

The entity card gives you a quick overview of your entity's state.

Screenshot of the entity card Screenshot of the Entity card.

{% endcapture %} {% endcapture %}



## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
  2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
    - By editing the dashboard, you are taking over control of this dashboard.
    - This means that it is no longer automatically updated when new dashboard elements become available.
    - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
    - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
  3. [Add a card and customize actions and features](#) to your dashboard.
- All options for this card can be configured via the user interface.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>entity</code>
<code>entity</code>	string	Yes		Entity ID.
<code>name</code>	[string, map, list]	No	Entity name.	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No		Overwrites icon.
<code>state_color</code>	boolean	No	false	Set to <code>true</code> to have icon colored when entity is active.
<code>attribute</code>	string	No		An attribute associated with the <code>entity</code> .
<code>unit</code>	string	No	Unit of measurement given by entity.	Unit of measurement given to data.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .
<code>footer</code>	map	No		Footer widget to render. See <a href="#">footer documentation</a> .

## Example

```
- type: entity
 entity: cover.kitchen_window
- type: entity
 entity: light.bedroom
 attribute: brightness
 unit: "%"
- type: entity
 entity: vacuum.downstairs
 name: Vacuum
 icon: "mdi:battery"
 attribute: battery_level
 unit: "%"
```

---

## Dashboards/Gauge

---

The gauge card is a basic card that allows visually seeing sensor data.

Screenshot of the Gauge card Screenshot of the gauge card.

Screenshot of the Gauge card in needle mode Screenshot of the gauge card in needle mode.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>gauge</code>
<code>entity</code>	string	Yes		Entity ID to show.
<code>attribute</code>	string	No		Attribute from the selected entity to display
<code>name</code>	[string, map, list]	No	Entity name	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>unit</code>	string	No	Unit of measurement given by entity	Unit of measurement given to data.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>min</code>	integer	No	0	Minimum value for graph.
<code>max</code>	integer	No	100	Maximum value for graph.
<code>needle</code>	boolean	No	false	Show the gauge as a needle gauge. Required to be set to true, if using segments.
<code>severity</code>	integer	Yes		Value from which to start red color.
<code>segments</code>	string	No		Label of the segment. This will be shown instead of the value.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## Examples

Title and unit of measurement:

```
type: gauge
name: CPU Usage
unit: '%'
entity: sensor.cpu_usage
```

Screenshot of the gauge card with custom title and unit of measurement Screenshot of the gauge card with custom title and unit of measurement.

Define the severity map:

```
type: gauge
name: With Severity
unit: '%'
entity: sensor.cpu_usage
severity:
 green: 0
 yellow: 45
 red: 85
```

Multiple segments:

Screenshot of the gauge card with multiple colored segments. Screenshot of the gauge card with multiple colored segments.

```
type: gauge
entity: sensor.kitchen_humidity
needle: true
min: 20
max: 80
segments:
 - from: 0
 color: '#db4437'
 - from: 35
 color: '#ffa600'
 - from: 40
 color: '#43a047'
 - from: 60
 color: '#ffa600'
 - from: 65
 color: '#db4437'
```

CSS variables can be used (instead of CSS '#rrggbb') for default gauge segment colors:

- `var(--success-color)` for green color
- `var(--warning-color)` for yellow color
- `var(--error-color)` for red color
- `var(--info-color)` for blue color

Therefore, the previous example can be defined also as:

```
type: gauge
entity: sensor.kitchen_humidity
needle: true
min: 20
max: 80
segments:
 - from: 0
 color: var(--error-color)
 - from: 35
 color: var(--warning-color)
 - from: 40
 color: var(--success-color)
 - from: 60
 color: var(--warning-color)
 - from: 65
 color: var(--error-color)
```

Display attribute of an entity instead of its state:

```
type: gauge
entity: sensor.back_door_info
attribute: battery_level
unit: '%'
max: 100
```

In this example, the card displays the `battery_level` attribute of the `sensor.back_door_info` entity.

---



## Dashboards/Glance

---

The glance card is useful to group multiple sensors in a compact overview. Keep in mind that this can be used together with [entity-filter](#) cards to create dynamic cards.

Screenshot of the glance card Screenshot of the glance card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. [Add a card and customize actions and features](#) to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>glance</code>
<code>entities</code>	list	Yes		A list of entity IDs or <code>entity</code> objects, see below.
<code>title</code>	string	No		Card title.
<code>show_name</code>	boolean	No	"true"	Show entity name.
<code>show_icon</code>	boolean	No	"true"	Show entity icon.
<code>show_state</code>	boolean	No	"true"	Show entity state text.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>columns</code>	integer	No		Number of columns to show. If not specified the number will be set automatically.
<code>state_color</code>	boolean	No	true	Set to <code>true</code> to have icons colored when entity is active.

### Options for entities

If you define entities as objects instead of strings, you can add more customization and configuration:

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No		Overwrites icon.
<code>image</code>	string	No		Overwrites entity picture.
<code>show_last_changed</code>	boolean	No	false	Overwrites the state display with the relative time since last changed.
<code>show_state</code>	boolean	No	true	Show entity state text.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## Examples

Basic example:

```
type: glance
title: Glance card sample
entities:
 - binary_sensor.movement_backyard
 - light.bed_light
 - binary_sensor.basement_floor_wet
 - sensor.outside_temperature
 - light.ceiling_lights
 - switch.ac
 - lock.kitchen_door
```

Screenshot of the glance card with custom title Screenshot of the glance card with custom title.

Define entities as objects and apply a custom name:

```
type: glance
title: Better names
entities:
 - entity: binary_sensor.movement_backyard
 name: Movement?
 - light.bed_light
 - binary_sensor.basement_floor_wet
 - sensor.outside_temperature
 - light.ceiling_lights
 - switch.ac
 - lock.kitchen_door
 - entity: switch.wall_plug_switch
tap_action:
 action: toggle
```

---

## Dashboards/Grid

---

The grid card allows you to show multiple cards in a grid. It will first fill the columns, automatically adding new rows as needed.

Screenshot of the grid card Screenshot of the grid card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>grid</code>
<code>title</code>	string	No		Title of grid.
<code>square</code>	boolean	No	true	Should the cards be shown square.
<code>columns</code>	integer	No	3	Number of columns in the grid.
<code>cards</code>	list	Yes		List of cards.



## Examples

---

Basic example:

```
type: grid
cards:
 - type: picture-entity
 entity: camera.demo_camera
 show_info: false
 - type: entities
 entities:
 - binary_sensor.movement_backyard
```

Defining columns and disabling the square option:

```
type: grid
title: Backyard
columns: 2
square: false
cards:
 - type: picture-entity
 entity: group.all_lights
 image: /local/house.png
 - type: horizontal-stack
 cards:
 - type: picture-entity
 entity: light.ceiling_lights
 image: /local/bed_1.png
 - type: picture-entity
 entity: light.bed_light
 image: /local/bed_2.png
```

---

## Dashboards/Heading

---

The Heading card structures your dashboard by providing title, icon and navigation. This card supports [actions](#).

Screenshot of heading cards Screenshot of heading cards with entity badges and a button badge.

```
type: heading
heading: Kitchen
icon: mdi:fridge
badges:
 - type: entity
 entity: sensor.kitchen_temperature
 - type: entity
 entity: sensor.kitchen_humidity
```

{% configuration entity %} type: required: true description: "[heading](#)" type: string heading: required: false description: Heading text type: string heading\_style: required: false description: Style of the heading. Can be either [title](#) or [subtitle](#). type: string default: title icon: required: false description: Icon displayed before the heading text. type: string tap\_action: required: false description: Action taken on card tap. See [action documentation](#). By default, it will do nothing. If an action is configured, a chevron will appear next to the heading text. type: map badges: required: false description: Additional small badges to display entity information. See [heading badges](#). type: list

## Heading badges

---

In addition to the heading text, each heading card can show small badges. They are smaller than regular [badges](#) and don't have a background. The heading badges can display sensor information or action buttons in a compact and minimal style. Heading badges also support [actions](#).

The following badge types are available for heading cards:

### Entity badge

The Entity badge allows you to display the state of an entity in the heading card.

```
type: entity
entity: light.living_room
```

{% configuration entity %} type: required: true description: "[entity](#)" type: string entity: required: true description: Entity ID. type: string name: required: false description: Overwrites the entity name. The name will be only displayed if [state\\_content](#) includes [name](#) token. type: string icon: required: false description: Overwrites the entity icon. type: string color: required: false description: Set the color when the entity is active. By default, it will not be colored. It can be set to the [state](#) special token to dynamically color the icon based on [state](#), [domain](#), and [device\\_class](#) of your entity. It also accepts [color token](#) or hex color code. type: string default: none show\_icon: required: false description: Show the icon type: boolean default: "true" show\_state: required: false description: Show the state. type: boolean default: "false" state\_content: required: false description: > Content to display for the state. Can be [state](#), [name](#), [last\\_changed](#), [last\\_updated](#), or any attribute of the entity. Can be either a string with a single item, or a list of string items. Default depends on the entity domain. type: [string, list] tap\_action: required: false description: Action taken on card tap. See [action documentation](#). By default, it will do nothing. type: map hold\_action: required: false description: Action taken on card hold. See [action documentation](#). By default, it will do nothing. type: map double\_tap\_action: required: false description: Action taken on card double tap. See [action documentation](#). By default, it will do nothing. type: map

### Button badge

The Button badge allows you to display a customizable button with an icon and/or text. This is useful for quick actions like turning off lights, triggering automations, or opening dashboards.

```
type: button
icon: mdi:lightbulb-off
text: Turn off lights
color: yellow
```

You can also create icon-only buttons or text-only buttons:

```
badges:
 - type: button
 icon: mdi:play
 color: green
 - type: button
 text: Run automation
 color: blue
```

{% configuration entity %} type: required: true description: " **button** " type: string icon: required: false description: Icon to display. type: string text: required: false description: Text label to display on the button. type: string color: required: false description: Color of the badge. It accepts [color token](#) or hex color code. type: string tap\_action: required: false description: Action taken on card tap. See [action documentation](#). By default, it will do nothing. type: map hold\_action: required: false description: Action taken on card hold. See [action documentation](#). By default, it will do nothing. type: map double\_tap\_action: required: false description: Action taken on card double tap. See [action documentation](#). By default, it will do nothing. type: map

---

## Dashboards/History Graph

---

The history graph card allows you to display a graph for each of up to eight entities.

Screenshot of the history graph card for entities without a `unit_of_measurement` Screenshot of the history graph card, when the sensor has no `unit_of_measurement` defined.

Screenshot of the history graph card for entities with a `unit_of_measurement` Screenshot of the history graph card, when the sensor has a `unit_of_measurement` defined.

Only the y-axis and logarithmic scale settings can be configured via the user interface. To configure the other options for this card, you need to edit the YAML configuration.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		history-graph
<code>entities</code>	list	Yes		A list of entity IDs or <code>entity</code> objects, see below.
<code>hours_to_show</code>	integer	No	24	Hours to show in graph. Minimum is 1 hour. Big values can result in delayed rendering, especially if the selected entities have a lot of state changes.
<code>title</code>	string	No		The card title.
<code>show_names</code>	boolean	No	true	If false, no entity names are shown in the card.
<code>logarithmic_scale</code>	boolean	No	false	If true, numerical values on the Y-axis will be displayed with a logarithmic scale.
<code>min_y_axis</code>	float	No		Lower bound for the Y-axis range.
<code>max_y_axis</code>	float	No		Upper bound for the Y-axis range.
<code>fit_y_data</code>	boolean	No	false	If true, configured Y-axis bounds would automatically extend (but not shrink) to fit the data.
<code>expand_legend</code>	boolean	No	false	If true, the legend will show all items initially

### Options for entities

If you define entities as objects instead of strings, you can add more customization and configuration:

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .

## Long term statistics

Home Assistant saves long-term statistics for a sensor if the entity has a `state_class` of `measurement`, `total`, or `total_increasing`. For long-term statistics, an hourly aggregate is stored from the sensor history. Long-term statistics are never purged.

In the history graph card, if the `hours_to_show` variable is set to a figure higher than the recorder retention period, long-term statistics will backfill the older parts of the history graph, with more recent actual sensor values from the recorder shown in bold.

## Examples

```
type: history-graph
title: 'My Graph'
entities:
 - sensor.outside_temperature
 - entity: media_player.lounge_room
 name: Main player
```

Or with longer time frame, and multiple entities (as long as they share the same `unit_of_measurement`) in one graph:

```
type: history-graph
title: "Temperatures in the last 48 hours"
hours_to_show: 48
entities:
 - sensor.outside_temperature
 - entity: sensor.lounge_temperature
 name: "Lounge"
 - entity: sensor.attic_temperature
 name: "Attic"
```



## Dashboards/Horizontal Stack

---

The horizontal stack card allows you to stack together multiple cards, so they always sit next to each other in the space of one column.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>horizontal-stack</code>
<code>title</code>	string	No		Title of stack.
<code>cards</code>	list	Yes		List of cards.

### Example

```
type: horizontal-stack
title: Lights
cards:
 - type: picture-entity
 image: /local/bed_1.png
 entity: light.ceiling_lights
 - type: picture-entity
 image: /local/bed_2.png
 entity: light.bed_light
```

Two picture cards in a horizontal stack card Two picture cards in a horizontal stack card.

---

## Dashboards/Humidifier

---

The humidifier card lets you control and monitor humidifiers, dehumidifiers, and hygostat devices.

Screenshot of the humidifier card Screenshot of the humidifier card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>humidifier</code>
<code>entity</code>	string	Yes		Entity ID of <code>humidifier</code> domain.
<code>name</code>	[string, map, list]	No	Entity name	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>show_current_as_primary</code>	boolean	No	false	Show current humidity as primary information instead of target humidity. The target humidity will be displayed as secondary information.
<code>features</code>	list	No		Additional widgets to control your entity. See <a href="#">available features</a> . Only humidifier related features are supported.

### Example

```
type: humidifier
entity: humidifier.bedroom
name: Bedroom Humidifier
```

## Dashboards/Iframe

---

The webpage card allows you to embed your favorite webpage right into Home Assistant. You can also embed files stored in your `<config-directory>/www` folder and reference them using `/local/<file>`.

The webpage card is used on the [Webpage dashboard](#).

Windy weather radar as Webpage Windy weather radar as webpage.

All options for this card can be configured via the user interface.

Note that not every webpage can be embedded due to security restrictions that some sites have in place. These restrictions are enforced by your browser and prevent embedding them into a Home Assistant dashboard.

 **Important:** You can't embed sites using HTTP if you are using HTTPS for your Home Assistant.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.



## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>iframe</code>
<code>url</code>	string	Yes		Website URL.
<code>aspect_ratio</code>	string	No	"50%"	Forces the height of the image to be a ratio of the width. Valid formats: Height percentage value ( <code>23%</code> ) or ratio expressed with colon or "x" separator ( <code>16:9</code> or <code>16x9</code> ). For a ratio, the second element can be omitted and will default to "1" ( <code>1.78</code> equals <code>1.78:1</code> ).
<code>allow_open_top_navigation</code>	boolean	No	false	Allow the user to open iframe content links by opening the default browser in the Home Assistant mobile app. It is false by default because it adds allow-top-navigation-by-user-activation on the iframe sandbox attribute which is less secure. So set it to true if you need it and are confident with the iframe content.
<code>hide_background</code>	boolean	No	false	Hide the card background, making it transparent. This removes the background color, box-shadow, and border. Useful for pages which allow transparent backgrounds so the iframe can blend into the dashboard view.
<code>title</code>	string	No		The card title.
<code>allow</code>	string	No	"fullscreen"	The <a href="#">Permissions Policy</a> of the iframe, that is, the value of the <code>allow</code> attribute.
<code>disable_sandbox</code>	boolean	No	false	Disables the <a href="#">sandbox</a> attribute of the iframe, for example when

Option	Type	Required	Default	Description
				viewing PDFs in Chrome. This is less secure and should only be used if you trust the content of the iframe.

## Examples

```
type: iframe
url: https://www.home-assistant.io
aspect_ratio: 75%
```

## Dashboards/Light

---

The light card allows you to change the brightness of a light.

Screenshot of the Light card Screenshot of the light card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>light</code>
<code>entity</code>	string	Yes		Entity ID of <code>light</code> domain.
<code>name</code>	[string, map, list]	No	Name of entity	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No	Entity domain icon	Overwrites icon.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## Examples

Basic example:

```
type: light
entity: light.bedroom
```

Overwriting names example:

```
type: light
entity: light.bedroom
name: Kids Bedroom
```

```
type: light
entity: light.office
name: My Office
```

Screenshot of the Light card Screenshot of the Light card names.

## Dashboards/Logbook

---

The activity card displays entries from the activity for specific entities, devices, areas, and/or labels.

Screenshot of the activity card Screenshot of the activity card.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.



## Card settings

---

Option	Description
<code>Target</code>	The entities, devices, areas and labels whose activity entries will show in the card. See <a href="#">target selector</a> for more information.
<code>Title</code>	The title that shows on the top of the card.
<code>Hours to show</code>	The number of hours in the past that will be tracked in the card.
<code>Theme</code>	Name of any loaded theme to be used for this card. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>State filter</code>	Limit the displayed logbook entries to only the specified states.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI. Activity used to be called "logbook" in the past, and is still called logbook in YAML.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>logbook</code>
<code>target</code>	map	Yes		The target to use for the card.
<code>title</code>	string	No		Title of the card.
<code>hours_to_show</code>	integer	No	24	Number of hours in the past to track. Minimum is 1 hour. Big values can result in delayed rendering, especially if the selected entities have a lot of state changes.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>state_filter</code>	list	No		Limit the displayed logbook entries to only the selected states. For example a list of <code>['on']</code> will display entries when targeted entities turn on, but not when they turn off.

## Examples

```
type: logbook
target:
 entity_id:
 - fan.ceiling_fan
 - fan.living_room_fan
 - light.ceiling_lights
hours_to_show: 24
```

```
type: logbook
target:
 area_id: living_room
 device_id:
 - ff22a1889a6149c5ab6327a8236ae704
 - 52c050ca1a744e238ad94d170651f96b
 entity_id:
 - light.hallway
 - light.landing
 label_id:
 - lights
```

## Dashboards/Map

---

The map card allows you to display your home zone, entities, and other predefined zones on a map. This card is used on the [Map dashboard](#), which is one of the default dashboards.

Screenshot of the map card Screenshot of the map card.

## Adding the map card to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't set this dashboard to update automatically anymore. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add the map card to your dashboard.
4. By default, you see the house `mdi:house` icon on your map. It represents your [home zone](#).
5. To change the location of your home, you need to [edit your home's location in the general settings](#).

Edit map card settings 4. To learn how to show additional zones on your map, follow the steps on [adding a new zone](#). 5. To show other elements on the map, either add them under **Entities**, or use the **Geolocation sources**. - For a description of the options, refer to the [YAML configuration](#) section. It also applies to the options shown in the UI. - `mdi:info` **Info:** The list of entities shows the device trackers available for your home, such as a mobile phone with the companion app. - If you want to see a trace of the past locations of your entities, you need to define a time frame under **Hours to show**. - For more information about presence detection, refer to the [getting started tutorial on presence detection](#).

## Configuration options

---

All options for this card can be configured via the user interface. For a detailed description of the options, refer to the [YAML configuration](#) section. It also applies to the options shown in the UI.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>map</code>
<code>entities</code>	list	No		List of entity IDs or <code>entity</code> objects (see <a href="#">below</a> ). Either this, <code>show_all</code> , or the <code>geo_location_sources</code> configuration option is required.
<code>geo_location_sources</code>	list	No		List of geolocation sources or <code>source</code> objects (see <a href="#">below</a> ). All current entities with that source will be displayed on the map. See <a href="#">Geolocation</a> platform for valid sources. Set to <code>all</code> to use all available sources. Either this, <code>show_all</code> , or the <code>entities</code> configuration option is required.
<code>show_all</code>	boolean	No	false	Automatically add all entities with coordinates to the map card. (Default behavior of Map panel)
<code>auto_fit</code>	boolean	No	false	The map will follow moving <code>entities</code> by adjusting the viewport of the map each time an entity is updated.
<code>fit_zones</code>	boolean	No	false	Whether the map should consider the zones in the list of specified entities when fitting its viewport.
<code>title</code>	string	No		The card title.
<code>aspect_ratio</code>	string	No		Forces the height of the image to be a ratio of the width. Valid formats: Height percentage value ( <code>23%</code> ) or ratio expressed with colon or "x" separator ( <code>16:9</code> or <code>16x9</code> ). For a ratio, the second element can be omitted and will default to "1" ( <code>1.78</code> equals <code>1.78:1</code> ).
<code>default_zoom</code>	integer	No	14 (or whatever zoom level is required to fit all	The default zoom level of the map. Use a lower number to zoom out and a higher number to zoom in.

Option	Type	Required	Default	Description
			visible markers)	
<code>theme_mode</code>	string	No	'auto'	Override the theme to force the map to display in either a light mode ( <code>theme_mode: light</code> ) or a dark mode ( <code>theme_mode: dark</code> ). Default ( <code>theme_mode: auto</code> ) will follow the theme settings.
<code>hours_to_show</code>	integer	No	0	Shows a path of previous locations. Hours to show as path on the map.
<code>cluster</code>	boolean	No	true	When set to <code>false</code> , the map will not cluster the markers. This is useful when you want to see all markers at once, but it may cause performance issues with a large number of markers.
<code>conditions</code>	list	No		List of conditions to check for entity visibility. See <a href="#">description</a> .

**⚠ Important:** Only entities that have latitude and longitude attributes will be displayed on the map.

**📝 Note:** The `default_zoom` value will be ignored if it is set higher than the current zoom level after fitting all visible entity markers in the map window. In other words, this can only be used to zoom the map *out* by default.



## Conditions options

---

You can specify one or more `conditions`, in which case every selected entity will be tested against each condition and shown if it passes every condition. See [available conditions](#). For conditions which accept an `entity` id, this will be automatically set to the entity being tested.

### Examples

Map all locatable entities, except hiding those that have a state of `home`.

```
type: map
auto_fit: true
show_all: true
conditions:
 - condition: state
 state_not: home
```

## Options for entities

If you define entities as objects instead of strings (by adding `entity:` before entity ID), you can add more customization and configuration.

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>name</code>	string	No		Replace the default label for the marker.
<code>label_mode</code>	string	No	name	When set to <code>icon</code> , renders the entity's icon in the marker instead of text. When set to <code>state</code> or <code>attribute</code> , renders the entity's state or attribute as the label for the map marker instead of the entity's name. This option doesn't apply to <a href="#">zone</a> entities because they don't use a label but an icon.
<code>attribute</code>	string	No		An entity's attribute when <code>label_mode</code> set to <code>attribute</code> .
<code>unit</code>	string	No		A unit for a value of an attribute when <code>label_mode</code> set to <code>attribute</code> .
<code>focus</code>	boolean	No	true	When set to <code>false</code> , this entity will not be considered for determining the default zoom or fit of the map.

## Options for geolocation sources

If you define geolocation sources as objects instead of strings (by adding `source:` before the ID), you can add more customization and configuration.

Option	Type	Required	Default	Description
<code>source</code>	string	Yes		Name of a geolocation source, or <code>all</code> .
<code>label_mode</code>	string	No	name	When set to <code>icon</code> , renders the entity's icon in the marker instead of text. When set to <code>state</code> or <code>attribute</code> , renders the entity's state or attribute as the label for the map marker instead of the entity's name.
<code>attribute</code>	string	No		An entity's attribute when <code>label_mode</code> set to <code>attribute</code> .
<code>unit</code>	string	No		A unit for a value of an attribute when <code>label_mode</code> set to <code>attribute</code> .
<code>focus</code>	boolean	No	true	When set to <code>false</code> , the entities of this source will not be considered for determining the default zoom or fit of the map.

## Examples

---

```
type: map
aspect_ratio: 16:9
default_zoom: 8
auto_fit: true
entities:
 - device_tracker.demo_paulus
 - zone.home
```

```
type: map
geo_location_sources:
 - nsw_rural_fire_service_feed
 - source: gdacs
 focus: false
entities:
 - zone.home
```

```
type: map
entities:
 - device_tracker.demo_paulus
 - entity: sensor.gas_station_gas_price
 label_mode: state
 focus: false
hours_to_show: 48
```

---

## Dashboards/Markdown

---

The Markdown card is used to render [Markdown](#).

Screenshot of the markdown card Screenshot of the Markdown card.

The renderer uses [Marked.js](#), which supports [several specifications of Markdown](#), including CommonMark, GitHub Flavored Markdown (GFM) and `markdown.pl`. JavaScript in HTML blocks is not supported.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. [Add a card and customize actions and features](#) to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>markdown</code>
<code>content</code>	string	Yes		Content to render as <a href="#">Markdown</a> . May contain <a href="#">templates</a> .
<code>title</code>	string	No	none	The card title.
<code>card_size</code>	integer	No	none	The algorithm for placing cards aesthetically may have problems with the Markdown card if it contains templates. You can use this value to help it estimate the height of the card in units of 50 pixels (approximately 3 lines of text in default size), for example <code>4</code> .
<code>entity_id</code>	[string, list]	No	none	A list of entity IDs so a template in <code>content:</code> only reacts to the state changes of these entities. This can be used if the automatic analysis fails to find all relevant entities.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>show_empty</code>	boolean	No	true	By default, an empty card will still be shown (resulting in a small empty box). Setting this to <code>false</code> hides that empty card instead.
<code>text_only</code>	boolean	No	false	Display the card without border, background, padding and title.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## Example

```
type: markdown
content: >
 ## Dashboards

 Starting with Home Assistant 0.72, we're experimenting with a new way of defin
```

## Template variables

A special template variable - `config` is set up for the `content` of the card. It contains the configuration of the card.

For example:

```
type: entity-filter
entities:
 - light.bed_light
 - light.ceiling_lights
 - light.kitchen_lights
state_filter:
 - 'on'
card:
 type: markdown
 content: |
 The lights that are on are:
 {% for l in config.entities %}
 - {{ l.entity }}
 {%- endfor %}

 And the door is {% if is_state('binary_sensor.door', 'on') %} open {% else %}
```

A special template variable - `user` is set up for the `content` of the card. It contains the currently logged in user.

For example:

```
type: markdown
content: |
 Hello, {{user}}
```

## Icons

You can use [Material Design Icons](#) icons in the `content` of the card.

For example:

```
type: markdown
content: |
 <ha-icon icon="mdi:home-assistant"></ha-icon>
```



## ha-alert

---

You can also use our `ha-alert` component in the Markdown card.

Example:

Screenshot of the ha-alert elements in a markdown card Screenshot of the ha-alert elements in a markdown card.

```
type: markdown
content: |
 <ha-alert alert-type="error">This is an error alert – check it out!</ha-alert>
 <ha-alert alert-type="warning">This is a warning alert – check it out!</ha-alert>
 <ha-alert alert-type="info">This is an info alert – check it out!</ha-alert>
 <ha-alert alert-type="success">This is a success alert – check it out!</ha-alert>
 <ha-alert title="Test alert">This is an alert with a title</ha-alert>
```

## ha-qr-code

---

You can also create QR-Codes in the Markdown card.

Screenshot of the markdown card with QR codes Screenshot of the Markdown card with QR codes.

Available parameters:

- data: The actual data to encode in the QR-Code
- scale: A scale factor for the QR code, default is 4
- width: Width of the QR code in pixels
- margin: A margin around the QR code
- error-correction-level: low; medium; quartile; high
- center-image: An image to place on top of the qr code (might need a higher error-correction-level)

```
type: markdown
content: >-
 <ha-qr-code data='hallo' width="180"></ha-qr-code>

 <ha-qr-code data='hallo' scale="6" margin="0"
 center-image="/static/icons/favicon-192x192.png"></ha-qr-code>

 <ha-qr-code data='hallo' error-correction-level="quartile" scale="6"
 center-image="https://brands.home-assistant.io/_/tuya/icon@2x.png"></ha-qr-cod
```

# Presentation Tables

---

HTML tables with `role="presentation"` receive special styling optimized for layout purposes rather than data display. These tables are useful for creating structured layouts with icons, status information, and formatted content.

## Default Styling

Tables marked with `role="presentation"` have:

- No borders by default
- No padding by default
- Middle vertical alignment for cells

Example: Status Card Here's an example creating a status notification with an icon and multi-line text:

```
<table role="presentation">
 <tr>
 <td rowspan="3" width="70">

 </td>
 <td>System Status Alert</td>
 </tr>
 <tr>
 <td>Priority: High - Requires attention</td>
 </tr>
 <tr>
 <td>Active since: 2024-01-22 14:30</td>
 </tr>
</table>
```

# Dashboards/Masonry

---

The masonry view sorts cards in columns based on their card size.

Screenshot of the masonry view Screenshot of the masonry view.

Masonry sorts cards in columns based on size and places the next card below the smallest card on the dashboard.

Image showing how masonry arranges cards based on size. Masonry arranges cards based on size.

To group cards, you have to use [horizontal stack](#), [vertical stack](#), or [grid](#) cards.

Option	Type	Required	Default	Description
<code>type</code>	string	No		<code>masonry</code>

---

## Dashboards/Media Control

---

The media control card is used to display [media player](#) entities on an interface with easy to use controls.

Screenshot of the media player control card Screenshot of the media control card.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>media-control</code>
<code>entity</code>	string	Yes		Entity ID of <code>media_player</code> domain.
<code>name</code>	[string, map, list]	No	Name of entity	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .

### Example

Basic example:

```
type: media-control
entity: media_player.lounge_room
```

# Dashboards/Panel

---

The panel view must have exactly one card. This card is rendered full-width.

Screenshot of the panel view Screenshot of the panel view.

This view doesn't have support for badges.

This view is good when using cards like [map](#), [horizontal stack](#), [vertical stack](#), [picture elements](#), or [picture glance](#).

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>panel</code>



## Dashboards/Picture Elements

---

The picture elements card is one of the most versatile types of cards.

A functional floorplan powered by picture elements A functional floorplan powered by picture elements.

The cards allow you to position icons or text and even buttons on an image based on coordinates. Imagine floor plan, imagine [picture-glance](#) with no restrictions!

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>picture-elements</code>
<code>image</code>	string	Yes		The URL of an image. To use a locally hosted image, see <a href="#">Hosting</a> , or use a <code>media-source://</code> URL for Media content.
<code>image_entity</code>	string	No		Image or person entity to display.
<code>camera_image</code>	string	No		A camera entity.
<code>camera_view</code>	string	No	auto	"live" will show the live view if <code>stream</code> is enabled.
<code>elements</code>	list	Yes		List of elements.
<code>title</code>	string	No		Card title.
<code>state_filter</code>	map	No		<a href="#">State-based CSS filters</a>
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>dark_mode_image</code>	string	No		This image is used when the dark mode is activated and no state image is set. To use a locally hosted image, see <a href="#">Hosting</a> , or use a <code>media-source://</code> URL for Media content.
<code>dark_mode_filter</code>	string	No		This CSS filter is used when the dark mode is activated.

## Elements

---

Elements are the active components (icons, badges, buttons, text, and more) that overlay the image.

There are several different element types that can be added to a Picture Elements card:

- State badge
- State Icon
- State Label
- Perform action button
- Icon
- Image
- Conditional
- Custom

### State badge

This element creates a badge representing the state of an entity.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>state-badge</code>
<code>entity</code>	string	Yes		Entity ID.
<code>style</code>	map	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element</a> using CSS.
<code>name</code>	string	No		An optional alternative name displayed below the state badge. Defaults to the entity name if not provided. Set to null to hide.
<code>title</code>	string	No		State badge tooltip. Defaults to the entity name if not provided. Set to null to hide.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## State icon

This element represents an entity state using an icon.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>state-icon</code>
<code>entity</code>	string	Yes		The entity ID to use.
<code>icon</code>	string	No		Overwrites icon.
<code>title</code>	string	No		Icon tooltip. Set to null to hide.
<code>state_color</code>	boolean	No	true	Set to <code>true</code> to have icons colored when entity is active.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .
<code>style</code>	string	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element</a> using CSS.

## State label

This element represents an entity's state via text.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>state-label</code>
<code>entity</code>	string	Yes		Entity ID.
<code>attribute</code>	string	No		If present, the corresponding attribute will be shown, instead of the entity's state.
<code>prefix</code>	string	No		Text before entity state.
<code>suffix</code>	string	No		Text after entity state.
<code>title</code>	string	No		Label tooltip. Set to null to hide.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .
<code>style</code>	string	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element</a> using CSS.

## Perform action button

This entity creates a button (with arbitrary text) that can be used to perform an action.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>action-button</code>
<code>title</code>	string	Yes		Button label.
<code>action</code>	string	Yes		<code>light.turn_on</code>
<code>target</code>	map	No		The target to use for the action.
<code>data</code>	map	No		The data to use for the action.
<code>style</code>	string	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element</a> using CSS.

## Icon element

This element creates a static icon that is not linked to the state of an entity.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>icon</code>
<code>icon</code>	string	Yes		Icon to display, for example <code>mdi:home</code> .
<code>title</code>	string	No		Icon tooltip. Set to null to hide.
<code>entity</code>	string	No		Entity to use for more-info/toggle.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .
<code>style</code>	string	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element</a> using CSS.

## Image element

This creates an image element that overlays the background image.



Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>image</code>
<code>entity</code>	string	No		Entity to use for <code>state_image</code> and <code>state_filter</code> and also target for actions.
<code>title</code>	string	No		Image tooltip. Set to null to hide.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .
<code>image</code>	string	No		The image to display. To use a locally hosted image, see <a href="#">Hosting</a> , or use a <code>media-source://</code> URL for Media content.
<code>camera_image</code>	string	No		A camera entity.
<code>camera_view</code>	string	No	auto	"live" will show the live view if <code>stream</code> is enabled.
<code>state_image</code>	map	No		<a href="#">State-based images</a>
<code>filter</code>	string	No		Default: <code>grayscale(100%)</code> when entity state is <code>off</code> . Set to <code>none</code> to remove this.
<code>state_filter</code>	map	No		<a href="#">State-based CSS filters</a>
<code>aspect_ratio</code>	string	No	"50%"	Forces the height of the image to be a ratio of the width. Valid formats: Height percentage value ( <code>23%</code> ) or ratio expressed with colon or "x" separator ( <code>16:9</code> or <code>16x9</code> ). For a ratio, the second element can be omitted and will default to "1" ( <code>1.78</code> equals <code>1.78:1</code> ).
<code>style</code>	string	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element using CSS</a> .

## Conditional element

Much like the Conditional card, this element will let you show its sub-elements based on entity states.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>conditional</code>
<code>conditions</code>	string	No		Entity state is unequal to this value.*
<code>elements</code>	list	Yes		One or more elements of any of the <a href="#">listed types</a> to show when conditions are met. See below for an example.

## Custom elements

The process for creating and referencing custom elements is the same as for custom cards. Please see the [developer documentation](#) for more information.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		Card name with <code>custom:</code> prefix, for example <code>custom:my-custom-card</code> .
<code>style</code>	string	Yes	"position: absolute, transform: translate(-50%, -50%)"	<a href="#">Position and style the element</a> using CSS.

## Notes on element attributes

---

### How to use the style object

Position and style your elements using [CSS](#). More/other keys are also possible. Note, the default style for most elements includes [translate\(-50%, -50%\)](#), which means that the coordinates you provide will set the position of the center of the element. Use `transform: none` to disable this behavior.

```
style:
 # Positioning of the element
 left: 50%
 top: 50%
```

### How to use state\_image

Specify a different image to display based on the state of the entity (supports local, web, or `media-source://` URLs):

```
state_image:
 "on": /local/bed_light_on.png
 "off": https://demo.home-assistant.io/stub_config/bedroom.png
 "unavailable": media-source://image_upload/123456789
```

### How to use state\_filter

Specify different [CSS filters](#)

```
state_filter:
 "on": brightness(110%) saturate(1.2)
 "off": brightness(50%) hue-rotate(45deg)
```

### How to use click-and-hold

If the option `hold_action` is specified, that action will be performed when the entity is clicked and held for half a second or more.

```
tap_action:
 action: toggle
hold_action:
 action: perform-action
 perform_action: light.turn_on
 data:
 entity_id: light.bed_light
 brightness_pct: 100
```

## Examples

---

### Example of icons, labels and buttons

```
type: picture-elements
image: /local/floorplan.png
elements:
 - type: state-icon
 tap_action:
 action: toggle
 entity: light.ceiling_lights
 style:
 top: 47%
 left: 42%
 - type: state-icon
 tap_action:
 action: toggle
 entity: light.kitchen_lights
 style:
 top: 30%
 left: 15%
 - type: state-label
 entity: sensor.outside_temperature
 style:
 top: 82%
 left: 79%
 - type: state-label
 entity: climate.kitchen
 attribute: current_temperature
 suffix: "°C"
 style:
 top: 33%
 left: 15%
 - type: action-button
 title: Turn lights off
 style:
 top: 95%
 left: 60%
 action: homeassistant.turn_off
 target:
 entity_id: group.all_lights
 - type: icon
 icon: mdi:home
 tap_action:
 action: navigate
 navigation_path: /lovelace/0
 style:
 top: 10%
 left: 10%
```

## Images example

```
type: picture-elements
image: /local/floorplan.png
elements:
 # state_image & state_filter - toggle on click
 - type: image
 entity: light.living_room
 tap_action:
 action: toggle
 image: /local/living_room.png
 state_image:
 "off": /local/living_room_off.png
 filter: saturate(.8)
 state_filter:
 "on": brightness(120%) saturate(1.2)
 style:
 top: 25%
 left: 75%
 width: 15%
 # Camera, red border, rounded-rectangle - show more-info on click
 - type: image
 entity: camera.driveway_camera
 camera_image: camera.driveway_camera
 style:
 top: 5%
 left: 10%
 width: 10%
 border: 2px solid red
 border-radius: 10%
 # Single image, state_filter - perform action on click
 - type: image
 entity: media_player.living_room
 tap_action:
 action: perform-action
 perform_action: media_player.media_play_pause
 target:
 entity_id: media_player.living_room
 image: /local/television.jpg
 filter: brightness(5%)
 state_filter:
 playing: brightness(100%)
 style:
 top: 40%
 left: 75%
 width: 5%
```

## Conditional example

```
type: picture-elements
image: /local/House.png
elements:
 # conditionally show TV off button shortcut when dad's away and daughter is home
 - type: conditional
 conditions:
 - entity: sensor.presence_daughter
 state: "home"
 - entity: sensor.presence_dad
 state: "not_home"
 elements:
 - type: state-icon
 entity: switch.tv
 tap_action:
 action: toggle
 style:
 top: 47%
 left: 42%
```

---

## Dashboards/Picture Entity

---

The picture entity card displays an entity in the form of an image. Instead of images from URL, it can also show the picture of `camera` entities.

Picture entity card Background changes according to the entity state.

{% endcapture %} {% endcapture %}

## Adding a to your dashboard

---

1. To add a card, follow steps 1-4 on [adding a card from a view](#).
2. In step 2, on the **By card** tab, select .
3. Add a picture:
4. **Upload picture** lets you pick an image from the system used to show your Home Assistant UI.
5. **Local path** lets you pick an image stored on Home Assistant. For example:  
`/homeassistant/images/lights_view_background_image.jpg` .
  - To store an image on Home Assistant, you need to [configure access to files](#), for example via [Samba](#) or the [Studio Code Server](#) app (formerly known as add-on).
6. **web URL** let you use an image from the web. For example `https://www.home-assistant.io/images/frontpage/assist_wake_word.png` .
7. Define the parameters that are specific to the .
8. For a description of the specific settings, refer to the description under YAML configuration.
9. They also apply to the UI.
10. Save your changes.



## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>picture-entity</code>
<code>entity</code>	string	Yes		A camera, image, or person <code>entity_id</code> used for the picture.
<code>camera_image</code>	string	No		Camera <code>entity_id</code> to use. (not required if <code>entity</code> is already a camera-entity).
<code>camera_view</code>	string	No	auto	"live" will show the live view if <code>stream</code> is enabled.
<code>image</code>	string	No		URL of an image. To use a locally hosted image, see <a href="#">Hosting</a> , or use a <code>media-source://</code> URL for Media content.
<code>state_image</code>	map	No		Map entity states to images ( <code>state: image URL</code> , check the example below).
<code>state_filter</code>	map	No		<a href="#">State-based CSS filters</a>
<code>aspect_ratio</code>	string	No		Forces the height of the image to be a ratio of the width. Valid formats: Height percentage value ( <code>23%</code> ) or ratio expressed with colon or "x" separator ( <code>16:9</code> or <code>16x9</code> ). For a ratio, the second element can be omitted and will default to "1" ( <code>1.78</code> equals <code>1.78:1</code> ).
<code>fit_mode</code>	string	No	cover	Defines the manner in which the image is stretched/clipped to fit the card area. <code>cover</code> : The image keeps its aspect ratio and fills the given dimension. The image will be clipped to fit. <code>contain</code> : The image keeps its aspect ratio, but is resized to fit within the given dimension. <code>fill</code> : The image is resized to fill the given dimension. If necessary, the image will be stretched or squished to fit.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>show_name</code>	boolean	No	true	Shows name in footer.
<code>show_state</code>	boolean	No	true	Shows state in footer.
<code>theme</code>	string	No		

Option	Type	Required	Default	Description
				Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## How to use state\_filter

Specify different [CSS filters](#)

```
state_filter:
 "on": brightness(110%) saturate(1.2)
 "off": brightness(50%) hue-rotate(45deg)
```

## Examples

Basic example:

```
type: picture-entity
entity: light.bed_light
image: /local/bed_light.png
```

Different images for each state (supports local, web, or `media-source://` URLs):

```
type: picture-entity
entity: light.bed_light
state_image:
 "on": /local/bed_light_on.png
 "off": https://demo.home-assistant.io/stub_config/bedroom.png
 "unavailable": media-source://image_upload/123456789
```

Displaying a live feed from an FFmpeg camera:

```
type: picture-entity
entity: camera.backdoor
camera_view: live
tap_action:
 action: perform-action
 perform_action: camera.snapshot
 data:
 entity_id: camera.backdoor
 filename: '/shared/backdoor-{{ now().strftime("%Y-%m-%d-%H%M%S") }}.jpg'
```

The filename needs to be a path that is writable by Home Assistant in your system. You may need to configure `allowlist_external_dirs` ([documentation](#)).

---

## Dashboards/Picture Glance

---

The picture glance card shows an image and lets you place small icons of entity states on top of that card to control those entities from there. In the image below: the entities on the right allow toggle actions, the others show the more information dialog.

Picture glance card for a living room Picture glance card for a living room.

{% endcapture %} {% endcapture %}

## Adding a to your dashboard

---

1. To add a card, follow steps 1-4 on [adding a card from a view](#).
2. In step 2, on the **By card** tab, select .
3. Add a picture:
4. **Upload picture** lets you pick an image from the system used to show your Home Assistant UI.
5. **Local path** lets you pick an image stored on Home Assistant. For example:  
`/homeassistant/images/lights_view_background_image.jpg` .
  - To store an image on Home Assistant, you need to [configure access to files](#), for example via [Samba](#) or the [Studio Code Server](#) app (formerly known as add-on).
6. **web URL** let you use an image from the web. For example `https://www.home-assistant.io/images/frontpage/assist_wake_word.png` .
7. Define the parameters that are specific to the .
8. For a description of the specific settings, refer to the description under YAML configuration.
9. They also apply to the UI.
10. Save your changes.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>picture-glance</code>
<code>entities</code>	list	Yes		List of entities or entity objects.
<code>title</code>	string	No		The card title.
<code>image</code>	string	No		Background image URL (local, web, or <code>media-source://</code> )
<code>image_entity</code>	string	No		Image or person entity to display.
<code>camera_image</code>	string	No		Camera entity as Background image.
<code>camera_view</code>	string	No	auto	"live" will show the live view if <code>stream</code> is enabled.
<code>state_image</code>	string	No		<code>state: image-url</code> , check the example below.
<code>state_filter</code>	map	No		<a href="#">State-based CSS filters</a>
<code>aspect_ratio</code>	string	No		Forces the height of the image to be a ratio of the width. Valid formats: Height percentage value ( <code>23%</code> ) or ratio expressed with colon or "x" separator ( <code>16:9</code> or <code>16x9</code> ). For a ratio, the second element can be omitted and will default to "1" ( <code>1.78</code> equals <code>1.78:1</code> ).
<code>fit_mode</code>	string	No	cover	Defines the manner in which the image is stretched/clipped to fit the card area. <code>cover</code> : The image keeps its aspect ratio and fills the given dimension. The image will be clipped to fit. <code>contain</code> : The image keeps its aspect ratio, but is resized to fit within the given dimension. <code>fill</code> : The image is resized to fill the given dimension. If necessary, the image will be stretched or squished to fit.
<code>entity</code>	string	No		Entity to use for <code>state_image</code> and <code>state_filter</code> .
<code>show_state</code>	boolean	No	false	Show entity state text.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .



Option	Type	Required	Default	Description
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## Options for entities

If you define entities as objects instead of strings, you can add more customization and configuration:

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>attribute</code>	string	No		Attribute of the entity to display instead of the state.
<code>prefix</code>	string	No		Prefix to display before the attribute's value.
<code>suffix</code>	string	No		Suffix to display after the attribute's value.
<code>icon</code>	string	No		Overwrites default icon.
<code>show_state</code>	boolean	No	true	Show entity state text.
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## How to use `state_filter`

Specify different [CSS filters](#)

```
state_filter:
 "on": brightness(110%) saturate(1.2)
 "off": brightness(50%) hue-rotate(45deg)
entity: switch.decorative_lights
```

## Examples

This section lists a few examples of how the picture glance card can be used.

## Creating a card to control the camera

If your camera supports PTZ (can be moved in different directions), you can use the picture glance card to control the camera.

Picture glance card to control the camera Picture glance card to control the camera.

1. Select your camera entity.
  - **Image path** and **Image entity** are not required. Select camera entity
2. If you want something to happen when you tap the card itself, define a tap action.
3. Here, we toggle a light. Select camera entity
4. Select the entities to move the camera left, right, up, or down. Select camera entity
5. Select **Show code editor**.
6. For each of the entities, specify an icon, as indicated in the YAML example.
7. For the buttons to react on press (instead of bringing up the dialog):
8. For each of the entities, under `tap_action`, use a `button.press` action.

```
yaml camera_view: live type: picture-glance title: Desk entities: - entity:
button.cameral_ptz_left icon: mdi:pan-left tap_action: action: perform-action
perform_action: button.press data: entity_id: button.cameral_ptz_left -
entity: button.cameral_ptz_right icon: mdi:pan-right tap_action: action:
perform-action perform_action: button.press data: entity_id:
button.cameral_ptz_right - entity: button.cameral_ptz_up icon: mdi:pan-up
tap_action: action: perform-action perform_action: button.press data:
entity_id: button.cameral_ptz_up - entity: button.cameral_ptz_down icon:
mdi:pan-down tap_action: action: perform-action perform_action: button.press
data: entity_id: button.cameral_ptz_down camera_image: camera.cameral_sub
tap_action: action: perform-action perform_action: light.toggle target:
entity_id: light.philips_929003052501_01_huelight
```

7. That's it. You can now control your camera from the picture glance card on your dashboard.

## More examples

```
type: picture-glance
title: Living room
entities:
 - switch.decorative_lights
 - light.ceiling_lights
 - lock.front_door
 - binary_sensor.movement_backyard
 - binary_sensor.basement_floor_wet
image: /local/living_room.png
```

Display a camera image as background:

```
type: picture-glance
title: Living room
entities:
 - switch.decorative_lights
 - light.ceiling_lights
camera_image: camera.demo_camera
```

Display a camera image without additional entities:

```
type: picture-glance
title: Front garden
entities: []
camera_image: camera.front_garden_camera
```

Use different images based on entity state (supports local, web, or `media-source://` URLs):

```
type: picture-glance
title: Living room
entities:
 - switch.decorative_lights
 - light.ceiling_lights
state_image:
 "on": /local/living_room_on.png
 "off": https://demo.home-assistant.io/stub_config/living_room.png
 "unavailable": media-source://image_upload/123456789
entity: group.living.room
```

---

## Dashboards/Picture

---

The picture card allows you to set an image to use for navigation to various paths in your interface or to perform an action.

Screenshot of the picture card Screenshot of the picture card.

{% endcapture %} {% endcapture %}

## Adding a to your dashboard

---

1. To add a card, follow steps 1-4 on [adding a card from a view](#).
2. In step 2, on the **By card** tab, select .
3. Add a picture:
4. **Upload picture** lets you pick an image from the system used to show your Home Assistant UI.
5. **Local path** lets you pick an image stored on Home Assistant. For example:  
`/homeassistant/images/lights_view_background_image.jpg` .
  - To store an image on Home Assistant, you need to [configure access to files](#), for example via [Samba](#) or the [Studio Code Server](#) app (formerly known as add-on).
6. **web URL** let you use an image from the web. For example `https://www.home-assistant.io/images/frontpage/assist_wake_word.png` .
7. Define the parameters that are specific to the .
8. For a description of the specific settings, refer to the description under YAML configuration.
9. They also apply to the UI.
10. Save your changes.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>picture</code>
<code>image</code>	string	Yes		The URL of an image. When you want to store images in your Home Assistant installation use the <a href="#">hosting files documentation</a> . After storing your files, use the <code>/local</code> path, for example, <code>/local/filename.jpg</code> . To use an image from an existing <a href="#">media</a> directory, provide the full media-source identifier (see examples).
<code>image_entity</code>	string	No		Image or person entity to display.
<code>alt_text</code>	string	No		Alternative text for the image. This is necessary for users of assistive technology. The <a href="#">W3C images tutorial</a> provides simple guidance for writing alternative text.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		Action taken on card tap and hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on card double tap. See <a href="#">action documentation</a> .

## Examples

Go to another view:

```
type: picture
image: /local/home.jpg
tap_action:
 action: navigate
 navigation_path: /lovelace/home
```

Check the [views](#) setup on how to setup custom IDs.

Toggle entity using an action:

```
type: picture
image: /local/light.png
tap_action:
 action: perform-action
 perform_action: light.toggle
 data:
 entity_id: light.ceiling_lights
```

Show an image from a [media](#) directory:

```
type: picture
image: media-source://media_source/local/test.jpg
```

---

## Dashboards/Plant Status

---

The plant status card is for all the lovely botanists out there.

Screenshot of the plant status card Screenshot of the plant status card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}



## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. [Add a card and customize actions and features](#) to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>plant-status</code>
<code>entity</code>	string	Yes		Entity ID of <code>plant</code> domain. For more information, see the <code>plant</code> integration.
<code>name</code>	[string, map, list]	No	Entity name	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .

### Example

Basic example:

```
type: plant-status
entity: plant.bonsai
```

## Dashboards/Sections

---

The sections view lets you organize your cards in sections on a grid. You can group cards without using horizontal or vertical stack cards.

A fully populated dashboard in Sections view layout A fully populated dashboard in Sections view layout

## Creating a sections view

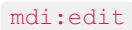
---

1. If you have multiple dashboards, in the left sidebar, select the dashboard to which you want to add the sections view.
2. Follow the steps on [adding a new view](#).
3. Under **View type**, select **Sections**.
4. Under **Max number of sections wide**, select the maximum number of columns you want to see in the new sections view.
5. Under **Dense section placement**, select if you want to allow the cards to be arranged automatically in order to fill gaps between cards.
6. This will remove some gaps, but it also means you have less control over the order of the cards.
7. Note that this only applies to horizontal gaps if you used sections more than one column wide.
8. When you are done, select **Save**.
9. You are now presented with a new, empty view.
10. If you chose a background image, the page is filled with that image.
11. Once you have created a sections view, you can start curating it:
12. [Add sections and cards](#).
13. [Rearrange](#) and [show or hide sections conditionally](#).
14. [Add a dashboard header with a title and badges](#).

## Editing the header

---

Editing the header Editing the header

1. To add a title, select the **Add title** button. The title supports [Markdown](#) and [templating](#).
2. To add badges, select the **Add badge** button. Follow [steps on adding badges](#) to see the different possible options.
3. To change the title and badges disposition, select the edit  button to access header settings.

Edit view heading section button

## Adding sections and cards to a sections view

---

The view comes with one section to which you can directly add a card.

1. To add a card, select the **Add card** button.
2. Follow the [steps on adding cards](#).

Add Section button

1. To add a new section, select the **Create section** button.
2. A [heading card](#) will be automatically added to the top of the section.
3. To edit it, select the card.
4. If you don't want a heading title at the top of the section, delete this card.
5. The title can be added again later, like any other card.
6. If you want this section to be visible only to specific users or under a certain condition, you can define those conditions:
7. On the **Visibility** tab, select **Add condition**.
8. Select the type of condition, and enter the parameters.
9. If you define multiple conditions, the section is only shown when all conditions are met.
10. If you did not define any conditions, the section is always shown, to all users.

## Adding a section background

---

You can add a colored background to individual sections. This is a great way to visually group related cards or highlight important sections on your dashboard.

1. To edit your dashboard, in the top right corner, select the edit `mdi:edit` button.
2. Select the edit `mdi:edit` button on the section you want to customize.
3. Enable the **Background** toggle.
4. To change the background color and opacity, expand **Background options**.
5. Pick a color from the predefined list, or enter a custom hex color code.
6. Use the **Opacity** slider to adjust the transparency of the background.

## Deleting a section

---

1. To delete a section, go to the dashboard and in the top right corner, select the edit `mdi:edit` button.
2. Open the view with the section you want to delete.
3. Select the delete `mdi:trash` button.



## Rearranging sections and cards

---

In the sections view, you can rearrange sections and cards by dragging them to a new location. This is not yet possible in other views.

1. To edit your dashboard, in the top right corner, select the edit `mdi:edit` button.
2. To rearrange sections, hold the move `mdi:cursor-move` button and move the card.

Rearranging sections by dragging Rearranging sections by dragging

3. To rearrange cards, tap and hold the card and move it to your desired location.

Rearranging cards by dragging Rearranging cards by dragging

## Show or hide section conditionally

---

You can choose to show or hide certain sections based on different conditions. The [available conditions](#) are the same as that for the conditional card.

To edit the section visibility conditions, select the edit `mdi:edit` button and then select the **Visibility** tab.

## Editing the footer

---

The footer lets you choose one card to show at the bottom of the view. This card stays on top of other cards while you scroll and only moves out of the way when you reach the bottom of the view.

1. To add a footer, select the **Add footer** button.
2. Select a card type to be used as the footer.
3. To change the maximum width of the footer, select the edit `mdi:edit` button to access footer settings.

## Check out the demo

---

Check out the demo from the March live stream on dashboards.

## About the sections view layout

---

To learn all about the design decisions and the grid layout used for the sections view, refer to the [Dashboard chapter 1 blog post](#).

# YAML configuration

Option	Type	Required	Default	Description
<code>type</code>	string	No		<code>sections</code>

## Header YAML configuration

Option	Type	Required	Default	Description
<code>layout</code>	string	No	center	Layout of the different elements. Can be <code>start</code> , <code>center</code> , or <code>responsive</code> . <code>responsive</code> is the same as <code>start</code> on mobile devices. It places badges and title side by side on desktop.
<code>badges_position</code>	string	No	bottom	Badges position. Can be <code>bottom</code> or <code>top</code> .
<code>card</code>	map	Yes		Card to be used as title. If you are configuring the view using the visual editor, the configuration of the <a href="#">Markdown card</a> is used.

## Section YAML configuration

---

Option	Type	Required	Default	Description
<code>background</code>	integer	No	50	The opacity of the background, from fully transparent to fully opaque.

### Examples

```
Section with default background
background: true
```

```
Section with custom background color and opacity
background:
 color: "red"
 opacity: 80
```



## Footer YAML configuration

---

Option	Type	Required	Default	Description
<code>max_width</code>	integer	No	600	Maximum width of the footer.

---

## Dashboards/Sensor

---

The sensor card gives you a quick overview of a sensor's state with an optional graph to visualize change over time.

Screenshot of the sensor card Screenshot of the sensor card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>sensor</code>
<code>entity</code>	string	Yes		Entity ID of <code>sensor</code> domain.
<code>icon</code>	string	No		The card icon.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>graph</code>	string	No		Type of graph ( <code>none</code> or <code>line</code> ).
<code>unit</code>	string	No		The unit of measurement.
<code>detail</code>	integer	No	1	Detail of the graph <code>1</code> or <code>2</code> ( <code>1</code> = one point/hour, <code>2</code> = six points/hour).
<code>hours_to_show</code>	integer	No	24	Hours to show in graph. Minimum is 1 hour. Big values can result in delayed rendering, especially if the selected entities have a lot of state changes.
<code>limits</code>	float	No	The maximum sample among the displayed values.	Maximum value of the graph Y-axis.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .

 **Note:** The `hours_to_show` option controls the time range of historical data shown in the graph. The amount of history available depends on the Recorder's `purge_keep_days` setting. By default, the Recorder purges data older than 10 days. See the [Recorder integration documentation](#) for more information.

## Examples

Basic sensor card:

```
type: sensor
entity: sensor.illumination
name: Illumination
```

Sensor card with historical data graph:

```
type: sensor
entity: sensor.my_temperature
graph: line
hours_to_show: 720 # shows 30 days of history only if history exists for this se
```

---

## Dashboards/Shopping List

---

Note: the shopping list card is no longer available as a card to add from the user interface. Use the [to-do list card](#) instead.

The shopping list card allows you to add, edit, check-off, and clear items from your shopping list.

Screenshot of the shopping list card Screenshot of the shopping list card.

Setup of the [shopping list integration](#) is required.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>shopping-list</code>
<code>title</code>	string	No		Title of shopping list.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .

## Examples

Title example:

```
type: shopping-list
title: shopping list
```



## Dashboards/Shortcut

---

The shortcut card gives you a quick way to trigger an action from your dashboard. You can use it to navigate to another page, open a URL, launch the voice assistant, or perform an action. It renders as a tile, so it fits nicely alongside other tile cards on your dashboard.

The label, icon, and color are automatically resolved from the action you configure. For example, if you navigate to a dashboard view, the shortcut picks up the view's title and icon. You can override any of these values if you want something different.

```
{% endcapture %} {% endcapture %}
```

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## Card settings

---

Most options can be configured in the UI.

Option	Description
<code>Label</code>	The text displayed on the card. If left empty, the label is resolved automatically from the action. For example, the view title for a navigation action, or <b>Assist</b> for an assist action.
<code>Description</code>	An optional secondary line displayed under the label.
<code>Icon</code>	The icon displayed on the card. If left empty, the icon is resolved automatically from the action.
<code>Color</code>	The color of the icon and background accent. Accepts a <a href="#">color token</a> or a hex color code. Defaults to the primary color of your theme.
<code>Vertical</code>	Displays the icon above the label instead of next to it.
<code>Tap action</code>	The action taken when the card is tapped. For more information, see the <a href="#">action documentation</a> .
<code>Hold action</code>	The action taken when the card is tapped and held. For more information, see the <a href="#">action documentation</a> .
<code>Double tap action</code>	The action taken when the card is double-tapped. For more information, see the <a href="#">action documentation</a> .

## YAML configuration

The following YAML options are available when you use YAML mode or prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>shortcut</code>
<code>label</code>	string	No		The text displayed on the card. If not set, the label is resolved automatically from the configured <code>tap_action</code> .
<code>description</code>	string	No		An optional secondary line displayed under the label.
<code>icon</code>	string	No		The icon displayed on the card. If not set, the icon is resolved automatically from the configured <code>tap_action</code> .
<code>color</code>	string	No	primary	The color of the icon and background accent. Accepts a <code>color token</code> or hex color code.
<code>vertical</code>	boolean	No	false	Displays the icon above the label instead of next to it.
<code>tap_action</code>	map	Yes		The action taken on card tap. For more information, see the <a href="#">action documentation</a> .
<code>hold_action</code>	map	No		The action taken on card tap and hold. For more information, see the <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		The action taken on card double tap. For more information, see the <a href="#">action documentation</a> .

## Examples

---

Navigate to a dashboard view. The label and icon are taken from the view:

```
type: shortcut
tap_action:
 action: navigate
 navigation_path: /lovelace/kitchen
```

Open an external URL with a custom label and icon:

```
type: shortcut
label: "Home Assistant docs"
icon: mdi:book-open-variant
tap_action:
 action: url
 url_path: https://www.home-assistant.io
```

Launch the voice assistant:

```
type: shortcut
tap_action:
 action: assist
```

Perform an action, with a custom color and a description:

```
type: shortcut
label: "Good night"
description: "Turn off all lights"
color: indigo
tap_action:
 action: perform-action
 perform_action: script.good_night
```

Vertical layout:

```
type: shortcut
label: "Kitchen"
vertical: true
tap_action:
 action: navigate
 navigation_path: /lovelace/kitchen
```

## Available colors

---

The following colors are available to colorize the shortcut card: `primary`, `accent`, `disabled`, `red`, `pink`, `purple`, `deep-purple`, `indigo`, `blue`, `light-blue`, `cyan`, `teal`, `green`, `light-green`, `lime`, `yellow`, `amber`, `orange`, `deep-orange`, `brown`, `grey`, `blue-grey`, `black`, `white`, or any hex color code (for example, `#93c47d`).

---

## Dashboards/Sidebar

---

The sidebar view has 2 columns, a wide one and a smaller one on the right.

Screenshot of the sidebar view Screenshot of the sidebar view used for the energy dashboard.

To move the card from the main column into the sidebar (right) (and vice versa), select the arrow `mdi:arrow-left-bold` `mdi:arrow-right-bold` button on the card.

Screenshot showing how to move a card Screenshot showing the arrow buttons to move a card.

On mobile, all cards are rendered in 1 column and kept in the order indicated in the YAML configuration.

1. To view the YAML configuration, on the view tab, select the `mdi:pencil` icon to open the edit view.
2. In the configuration dialog, select the three dots `mdi:dots-vertical` menu, and **Edit in YAML**.

Screenshot showing where to edit the view configuration Screenshot showing where to edit the view configuration.

## View configuration

---

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>sidebar</code>



## Cards configuration

---

Option	Type	Required	Default	Description
<code>view_layout.position</code>	string	No		The position of the card, <code>main</code> or <code>sidebar</code>

### Example

The position of the card is configured using YAML with the `view_layout` option:

```
type: sidebar
cards:
 - type: entities
 entities:
 - media_player.lounge_room
 view_layout:
 position: sidebar
```

## Dashboards/Statistic

---

The statistic card allows you to display a statistical value for an entity.

Statistics are gathered every 5 minutes for sensors that support it. It will either keep the `min`, `max` and `mean` of a sensors value for a specific period, or the `sum` for a metered entity.

If your sensor doesn't work with statistics, check [this](#).

Screenshot of the statistic card for a temperature sensor Screenshot of the statistic card for a temperature sensor.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. [Add a card and customize actions and features](#) to your dashboard.

All options for this card can be configured via the user interface, but if you want more options for the period, you will have to define them in `yaml`.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		statistic
<code>entity</code>	string	Yes		An entity ID of a sensor with statistics, or an external statistic id
<code>stat_type</code>	string	Yes		The statistics types to render. <code>min</code> , <code>max</code> , <code>mean</code> , <code>change</code>
<code>name</code>	[string, map, list]	No	Entity name.	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No		Overwrites icon.
<code>unit</code>	string	No	Unit of measurement given by entity.	Unit of measurement given to data.
<code>period</code>	map	Yes		The period to use for the calculation. <a href="#">See below</a> .
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>footer</code>	map	No		Footer widget to render. See <a href="#">footer documentation</a> .
<code>collection_key</code>	string	No		If using <code>period:</code> <code>energy_date_selection</code> , you can set a custom key to match the optional key of an <code>energy-date-selection</code> card. This is not typically required, but can be useful if multiple date selection cards are used on the same view. See <a href="#">energy documentation</a> .

## Example

---

Alternatively, the card can be configured using YAML:

```
type: statistic
entity: sensor.energy_consumption
period:
 calendar:
 period: month
stat_type: change
```

## Options for period

Periods can be configured in 4 different ways:

### Calendar

Use a fixed period with an offset from the current period.

Option	Type	Required	Default	Description
<code>period</code>	string	Yes		The period to use. <code>day</code> , <code>week</code> , <code>month</code> , <code>year</code>
<code>offset</code>	integer	No		The offset of the current period, so 0 means the current period, -1 is the previous period.

Example, the change of the energy consumption during last month:

```
type: statistic
entity: sensor.energy_consumption
period:
 calendar:
 period: month
 offset: -1
stat_type: change
```

### Fixed period

Specify a fixed period, the start and end are optional.

Option	Type	Required	Default	Description
<code>start</code>	string	No		The start of the period
<code>end</code>	string	No		The end of the period

Example, the change in 2022:

```
type: statistic
entity: sensor.energy_consumption
period:
 fixed_period:
 start: 2022-01-01
 end: 2022-12-31
stat_type: change
```

Example, all time change, without a start or end:

```

type: statistic
entity: sensor.energy_consumption
period:
 fixed_period:
stat_type: change

```

## Rolling window

Option	Type	Required	Default	Description
<code>duration</code>	map	Yes		The duration of the period
<code>offset</code>	map	No		The offset of the current time, 0 means the current period, -1 is the previous period.

Example, a period of 1 hour, 10 minutes and 5 seconds ending 2 hours, 20 minutes and 10 seconds before now:

```

type: statistic
entity: sensor.energy_consumption
period:
 rolling_window:
 duration:
 hours: 1
 minutes: 10
 seconds: 5
 offset:
 hours: -2
 minutes: -20
 seconds: -10
stat_type: change

```

## Dynamic date selection

When placed on a view with an Energy date selection card, the statistic card can be linked to show data from the period selected on the date selection card.

Example of a period from the date selector:

```

type: statistic
entity: sensor.energy_consumption
period: energy_date_selection
stat_type: change

```

## Dashboards/Statistics Graph

---

The statistics graph card allows you to display a graph of statistics data for each of the entities listed.

Screenshot of the statistics graph card for power entities Screenshot of the statistics graph card with no metered entities and ``chart_type` `line``.

Screenshot of the statistics graph card for energy entities Screenshot of the statistics graph card with a metered entity and ``chart_type` `bar``.

Screenshot of the statistics graph card as a bar chart with stacked values Screenshot of the statistics graph card with values stacked using ``chart_type` `bar-stack``.

Statistics are gathered every 5 minutes and also hourly for sensors with a `state_class` of `measurement`, `total` or `total_increasing`. The 5-minute statistics will be retained for the duration set in the [recorder configuration](#), and hourly statistics will be retained indefinitely. It will either keep the `min`, `max`, and `mean` of a sensor's value for a specific hour or the `sum` for a metered entity.

If your sensor doesn't work with statistics, check [this](#).

All options for this card can be configured via the user interface.

```
{% endcapture %} {% endcapture %}
```



## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		statistics-graph
<code>entities</code>	list	Yes		A list of entity IDs or <code>entity</code> objects (see below), or an external statistic id
<code>days_to_show</code>	integer	No	30	Days to show in graph. Minimum is 1 day.
<code>chart_type</code>	string	No		If the graph should be rendered as a <code>bar</code> or a <code>line</code> chart. Alternatively, using <code>bar-stack</code> or <code>line-stack</code> select stacked versions of the chart.
<code>stat_types</code>	list	No		The statistics types to render. <code>min</code> , <code>max</code> , <code>mean</code> , <code>sum</code> , <code>state</code> , <code>change</code> . When using stacked charts, it is recommended to select only one option.
<code>title</code>	string	No		The card title.
<code>period</code>	string	No		The period of the rendered graph. <code>5minute</code> , <code>hour</code> , <code>day</code> , <code>week</code> , <code>month</code> or <code>year</code> . If <code>energy_date_selection</code> is true, and <code>period</code> is not defined or set to <code>auto</code> , the chart period will auto-select between month/day/hour based on the selected date range.
<code>hide_legend</code>	boolean	No	false	If true, the legend will be hidden.
<code>logarithmic_scale</code>	boolean	No	false	If true, numerical values on the Y-axis will be displayed with a logarithmic scale.
<code>min_y_axis</code>	float	No		Lower bound for the Y-axis range.
<code>max_y_axis</code>	float	No		Upper bound for the Y-axis range.
<code>fit_y_data</code>	boolean	No	false	If true, configured Y-axis bounds would automatically extend (but not shrink) to fit the data.
<code>expand_legend</code>	boolean	No	false	If true, the legend will show all items initially
<code>energy_date_selection</code>	boolean	No	false	If true, chart date range will follow the date selected on an <code>energy-date-selection</code> card on the

Option	Type	Required	Default	Description
				same view, similar to energy cards.
<code>collection_key</code>	string	No		If using <code>energy_date_selection</code> , you can set a custom key to match the optional key of an <code>energy-date-selection</code> card. This is not typically required, but can be useful if multiple date selection cards are used on the same view.

## Options for entities

If you define entities as objects instead of strings, you can add more customization and configuration:

Option	Type	Required	Default	Description
<code>entity</code>	string	Yes		Entity ID.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .

## Example

```
type: statistics-graph
title: 'My Graph'
entities:
 - sensor.outside_temperature
 - entity: sensor.inside_temperature
 name: Inside
```

## Dashboards/Thermostat

---

The thermostat card gives control of your [climate](#) entity or [water heater](#) entity, allowing you to change the temperature and mode of the entity.

Screenshot of the thermostat card Screenshot of the thermostat card.

All options for this card can be configured via the user interface.

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

## YAML configuration

---

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>thermostat</code>
<code>entity</code>	string	Yes		Entity ID of <code>climate</code> or <code>water_heater</code> domain.
<code>name</code>	[string, map, list]	No	Name of entity.	Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>show_current_as_primary</code>	boolean	No	false	Show current temperature as primary information instead of target temperature. The target temperature will be displayed as secondary information.
<code>features</code>	list	No		Additional widgets to control your entity. See <a href="#">available features</a> . Only climate or water heater related features are supported.

### Example

```
type: thermostat
entity: climate.nest
```

## Dashboards/Tile

---

The tile card gives you a quick overview of an entity. The card allows you to add tap actions, and features to control the entity. You can also select the entity to open the **More info** dialog. A badge is shown for some entities like the [climate](#) or [person](#) entities.

Screenshot of tile cards The circular background behind an icon indicates that there is a tap action. The "Downstairs" and "Upstairs" climate entities have a badge and a feature that is bottom-aligned.

{% endcapture %} {% endcapture %}



## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
  - By editing the dashboard, you are taking over control of this dashboard.
  - This means that it is no longer automatically updated when new dashboard elements become available.
  - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
  - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>tile</code>
<code>entity</code>	string	Yes		Entity ID.
<code>name</code>	[string, map, list]	No		Overwrites friendly name. Can be a string, or a name configuration object. See <a href="#">naming documentation</a> .
<code>icon</code>	string	No		Overwrites the entity icon.
<code>color</code>	string	No	state	Set the color when the entity is active. By default, the color is based on <code>state</code> , <code>domain</code> , and <code>device_class</code> of your entity. It accepts <code>color token</code> or hex color code.
<code>show_entity_picture</code>	boolean	No	false	If your entity has a picture, it will replace the icon.
<code>vertical</code>	boolean	No	false	Displays the icon above the name and state.
<code>hide_state</code>	boolean	No	false	Hide the entity state.
<code>state_content</code>	[string, list]	No		>
<code>tap_action</code>	map	No		Action taken on card tap. See <a href="#">action documentation</a> . By default, it will show the "more-info" dialog.
<code>hold_action</code>	map	No		Action taken on tap-and-hold. See <a href="#">action documentation</a> .
<code>double_tap_action</code>	map	No		Action taken on double tap. See <a href="#">action documentation</a> .
<code>icon_tap_action</code>	map	No		Action taken on icon card tap. See <a href="#">action documentation</a> . By default, it will <code>toggle</code> the entity (if possible), otherwise, show the "more-info" dialog.
<code>icon_hold_action</code>	map	No		Action taken on icon tap-and-hold. See <a href="#">action documentation</a> .
<code>icon_double_tap_action</code>	map	No		Action taken on icon double tap. See <a href="#">action documentation</a> .
<code>features</code>	list	No		Additional widgets to control your entity. See <a href="#">available features</a> .

Option	Type	Required	Default	Description
<code>features_position</code>	string	No	bottom	Position of the features on the tile card. Can be <code>bottom</code> or <code>inline</code> . Only the first feature will be displayed when the option is set to <code>inline</code> . <code>inline</code> is not compatible with the <code>vertical</code> option.

## Examples

---

Alternatively, the card can be configured using YAML:

```
type: tile
entity: cover.kitchen_window
```

```
type: tile
entity: light.bedroom
icon: mdi:lamp
color: yellow
```

```
type: tile
entity: person.anne_therese
show_entity_picture: true
```

```
type: tile
entity: person.anne_therese
vertical: true
hide_state: true
```

```
type: tile
entity: light.living_room
state_content:
 - state
 - brightness
 - last-changed
```

```
type: tile
entity: vacuum.ground_floor
features:
 - type: vacuum-commands
 commands:
 - start_pause
 - return_home
```

## Available colors

---

The following colors are available to colorize the tile card: `primary`, `accent`, `disabled`, `red`, `pink`, `purple`, `deep-purple`, `indigo`, `blue`, `light-blue`, `cyan`, `teal`, `green`, `light-green`, `lime`, `yellow`, `amber`, `orange`, `deep-orange`, `brown`, `grey`, `blue-grey`, `black`, `white`, or any hex color code (for example, `#93c47d`).

---

## Dashboards/ToDo List

---

The to-do list card allows you to add, edit, check-off, and clear items from your to-do list.

Screenshot of the to-do list card Screenshot of the to-do list card.

## Adding the to-do list card to a dashboard

---

1. Add the card using the **Add card** button.
2. In the **By card** dialog, select the **To-do list** card.
3. In the **Entity** dropdown menu, select your list type.
4. If it is your first time working with to-do lists, there is only **Shopping list** in the menu.
5. This comes from the [shopping list integration](#), which is installed by default.
6. This is the same **Shopping list** as the one on the **To-do list** dashboard (accessible via sidebar). To-do card, list entities.
7. The to-do list card can display lists from different [to-do list](#) integrations, such as **Bring!** or **Todoist**.
8. If you don't see your desired to-do list entity, you need to add its integration first.
9. Once you've added a to-do list integration, the lists are also available on the to-do list dashboard.

## YAML configuration

All options for this card can be configured via the user interface.

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>todo-list</code>
<code>entity</code>	string	Yes		The to-do entity to show
<code>title</code>	string	No		Title of to-do list.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the <a href="#">frontend documentation</a> .
<code>hide_completed</code>	boolean	No	"false"	Hide the completed items section in the card.
<code>hide_create</code>	boolean	No	"false"	Hide the textbox for creating new tasks at the top of the card.
<code>hide_section_headers</code>	boolean	No	"false"	Hide the 'Active' and 'Completed' sections with the overflow menus.
<code>display_order</code>	string	No	"none"	Optionally sorts the items in the to-do list for display. Options are: <code>none</code> : Show the list in its original order. <code>alpha_asc</code> : Sort the list in alphabetical order. <code>alpha_desc</code> : Sort the list in reverse alphabetical order. <code>duedate_asc</code> : Sort the list by due date (soonest first). <code>duedate_desc</code> : Sort the list by reverse due date (soonest last).
<code>item_tap_action</code>	string	No	"edit"	Defines the behavior when an item's body is clicked. Options are: <code>edit</code> (opens the edit dialog), <code>toggle</code> (marks or unmarks the item as completed, hiding the edit dialog).
<code>due_date_period</code>	map	No		Filters tasks based on their due date. <a href="#">See below</a> .



## Options for due date period

---

Due date filtering can be configured with a calendar object:

### Calendar

Use a fixed period with an offset from the current period.

Option	Type	Required	Default	Description
<code>period</code>	string	Yes		The period to use. <code>day</code> , <code>week</code> , <code>month</code> , <code>year</code>
<code>offset</code>	integer	No		The offset of the current period, so 0 means the current period, 1 is the next period.

Example, show all tasks due before the end of the current month:

```
type: todo-list
entity: todo.todo_list
due_date_period:
 calendar:
 period: month
```

Example, show all tasks due before the end of next week:

```
type: todo-list
entity: todo.todo_list
due_date_period:
 calendar:
 period: week
 offset: 1
```

Example, show all tasks due in the next 7 days:

```
type: todo-list
entity: todo.todo_list
due_date_period:
 calendar:
 period: day
 offset: 6

Examples

Title example:

```yaml
type: todo-list
entity: todo.todo_list
title: Todo List
```

Dashboards/Vertical Stack

The vertical stack card allows you to group multiple cards so they always sit in the same column.

Screencast showing how to edit a dashboard customize a vertical stack card Screencast showing how to edit a dashboard and customize a vertical stack card.

{% endcapture %} {% endcapture %}

Adding the to a dashboard

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
 - By editing the dashboard, you are taking over control of this dashboard.
 - This means that it is no longer automatically updated when new dashboard elements become available.
 - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
 - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>vertical-stack</code>
<code>title</code>	string	No		Title of stack.
<code>cards</code>	list	Yes		List of cards.

Examples

Basic example:

```
type: vertical-stack
title: Backyard
cards:
  - type: picture-entity
    entity: camera.demo_camera
    show_info: false
  - type: entities
    entities:
      - binary_sensor.movement_backyard
```

Picture- and entities-card in a stack Picture and entities-card in a stack.

Combination of vertical and horizontal stack card:

```
type: vertical-stack
cards:
  - type: picture-entity
    entity: group.all_lights
    image: /local/house.png
  - type: horizontal-stack
    cards:
      - type: picture-entity
        entity: light.ceiling_lights
        image: /local/bed_1.png
      - type: picture-entity
        entity: light.bed_light
        image: /local/bed_2.png
```

Create a grid layout using vertical and horizontal stack Create a grid layout using vertical and horizontal stack.

Dashboards/Weather Forecast

The weather forecast card displays the weather. This card is particularly useful on wall-mounted displays.

Screenshot of the weather card Screenshot of the weather card.

{% endcapture %} {% endcapture %}

Adding the to a dashboard

1. In the top right of the screen, select the edit `mdi:edit` button.
2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
 - By editing the dashboard, you are taking over control of this dashboard.
 - This means that it is no longer automatically updated when new dashboard elements become available.
 - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
 - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
3. Add a card and customize actions and features to your dashboard.

Card settings

Option	Description
Entity	The entity of the <code>weather</code> platform to use.
Name	The name of the location where the weather platform is located. If not set, the name will be the name set on the weather entity
Show Forecast	Check this if you would like to show the upcoming forecast under the current weather.
Forecast type	Select the forecast to display between Daily , Hourly , and Twice daily .
Secondary Info Attribute	Here you can specify a secondary attribute to show under the current temperature. Ex. Extrema, Precipitation, Humidity. If not set, it will default to Extrema (High/Low) if available, if not available then Precipitation and if precipitation isn't available then Humidity.
Round temperature	Check this if you would like to round all temperature values in the card to its nearest integer.
Theme	Name of any loaded theme to be used for this card. For more information about themes, see the frontend documentation .

 **Important:** This card works only with platforms that define a `weather` entity. E.g., it works with [OpenWeatherMap](#) but not [OpenWeatherMap Sensor](#)

YAML configuration

The following YAML options are available when you use YAML mode or just prefer to use YAML in the code editor in the UI.

Option	Type	Required	Default	Description
<code>type</code>	string	Yes		<code>weather-forecast</code>
<code>entity</code>	string	Yes		Entity ID of <code>weather</code> domain.
<code>name</code>	[string, map, list]	No	Entity name	Overwrites friendly name. Can be a string, or a name configuration object. See naming documentation .
<code>show_current</code>	boolean	No	true	Show the current weather conditions above the forecast.
<code>show_forecast</code>	boolean	No	true	Show next hours/days forecast.
<code>forecast_type</code>	string	Yes		Type of forecast to display, one of <code>daily</code> , <code>hourly</code> or <code>twice_daily</code> .
<code>secondary_info_attribute</code>	string	No	Defaults to <code>extrema</code> if available, if not available then <code>precipitation</code> and if precipitation isn't available then <code>humidity</code> .	Which attribute to display under the temperature.
<code>round_temperature</code>	boolean	No	false	Round temperature values to the closest integer.
<code>theme</code>	string	No		Override the used theme for this card with any loaded theme. For more information about themes, see the frontend documentation .

Option	Type	Required	Default	Description
<code>tap_action</code>	map	No		The action taken on card tap. For more information, see the action documentation .
<code>hold_action</code>	map	No		The action taken on card tap and hold. For more information, see the action documentation .
<code>double_tap_action</code>	map	No		The action taken on card double-tap. For more information, see the action documentation .

Example

```
show_current: true
show_forecast: true
type: weather-forecast
entity: weather.openweathermap
forecast_type: daily
```

Advanced

Themeable icons

The default weather icons are themeable via a [theme](#). Theme variables include:

```
--weather-icon-cloud-front-color
--weather-icon-cloud-back-color
--weather-icon-sun-color
--weather-icon-rain-color
--weather-icon-moon-color
```

Example theme configuration:

```
--weather-icon-cloud-front-color: white
--weather-icon-cloud-back-color: blue
--weather-icon-sun-color: orange
--weather-icon-rain-color: purple
```

Personal icons

Weather icons can be overwritten with your own personal images via a [theme](#). Theme variables include:

```
--weather-icon-clear-night
--weather-icon-cloudy
--weather-icon-fog
--weather-icon-lightning
--weather-icon-lightning-rainy
--weather-icon-partlycloudy
--weather-icon-pouring
--weather-icon-rainy
--weather-icon-hail
--weather-icon-snowy
--weather-icon-snowy-rainy
--weather-icon-sunny
--weather-icon-windy
--weather-icon-windy-variant
--weather-icon-exceptional

// If your state is not above, use this format
--weather-icon-<state>
```

Example theme configuration:

```
--weather-icon-sunny: url("/local/sunny.png")
```

Docs/Authentication/Multi Factor Auth

The Multi-factor Authentication (MFA) modules require you to solve a second challenge after you provide your password.

A password can be compromised in a number of ways, for example, it can be guessed if it is a simple password. MFA provides a second level of defense by requiring:

- something you know, like your username and password, and
- something you have, like a one-time password sent to your phone.

You can use MFA with any of the other authentication providers. If more than one MFA module is enabled, you can choose one when you log in.

You can turn MFA on and off in the [profile page](#) for your user account.

Available MFA modules

Time-based One-Time Password MFA module

Time-based One-Time Password (TOTP) is widely adopted in modern authentication systems.

Home Assistant generates a secret key which is synchronized with an app on your phone. Every thirty seconds or so the phone app generates a random six digit number. Because Home Assistant knows the secret key, it knows which number will be generated. If you enter the correct digits, then you're in.

Setting up TOTP

Enable TOTP in your `configuration.yaml` like this:

```
homeassistant:
  auth_mfa_modules:
    - type: totp
```

If no `auth_mfa_modules` configuration section is defined in `configuration.yaml` a TOTP module named **Authenticator app** will be autoloaded.

You will need an authenticator app on your phone. We recommend either [Google Authenticator](#) or [Authy](#). Both are available for iOS or Android.

1. Restart Home Assistant.
2. Go to your **User profile** and select the **Security** tab.
3. In the **Multi-factor authentication modules** section, select **Enable** and a new secret key will be generated.
4. Go to your phone app and enter the key, either by scanning the QR code or typing in the key below the QR code manually.

Screenshot of setting up multi-factor authentication

◆ **Caution:** Please treat the secret key like a password - never expose it to others.

1. Your phone app will now start generating a different six-digit code every thirty seconds or so. Enter one of these into Home Assistant under the QR code where it asks for a **Code**.
2. Result: Home Assistant and your phone app are now in sync and you can now use the code displayed in the app to log in.

Using TOTP

Once TOTP is enabled, Home Assistant requires the latest code from your phone app before you can log in.

 **Note:** TOTP is *time based* so it relies on your Home Assistant clock being accurate. If the verification keeps failing, make sure the clock on Home Assistant is correct.

Notify multi-factor authentication module

The Notify MFA module uses the [notify integration](#) to send you an [HMAC-based One-Time Password](#). It is typically sent to your phone, but can be sent to any destination supported by a `notify` action. You use this password to log in.

Setting up MFA notify

Add Notify MFA to your `configuration.yaml` file like this:

```
homeassistant:
  auth_mfa_modules:
    - type: notify
      include:
        - notify_entity
```

Option	Type	Required	Default	Description
<code>exclude</code>	list	No		The list of notifying entities you want to exclude.
<code>include</code>	list	No		The list of notifying entities you want to include.
<code>message</code>	template	No		The message template.

```
# Example configuration, with a message template.
homeassistant:
  auth_mfa_modules:
    - type: totp
      name: "Authenticator app"
    - type: notify
      message: "I almost forget, to get into my clubhouse, you need to say {}"
```

After restarting Home Assistant, go to **User profile** and select the **Security** tab. In the **Multi-factor authentication modules** section. Under **Notify one-time password**, select **Enable**.

Try logging out, then logging in again. You will be asked for the six-digit one-time password that was sent to your notify entity. Enter the password to log in.

If the validation failed, a new one-time password will be sent again.

 **Note:** The Notify MFA module can't tell if the one-time password was delivered successfully. If you don't get the notification, you won't be able to log in.

You can disable the Notify MFA module by editing or removing the file `[your_config_dir]/.storage/auth_module.notify`.

Docs/Authentication/Providers

◆ **Caution:** This is an advanced feature.

When you log in, an *auth provider* checks your credentials to make sure you are an authorized user.

Configuring auth providers

 **Warning:** Home Assistant automatically configures the standard auth providers so you don't need to specify `auth_providers` in your `configuration.yaml` file unless you are configuring more than one. Specifying `auth_providers` will disable all auth providers that are not listed, so you could reduce your security or create difficulties logging in if it is not configured correctly.

If you decide to use `trusted_networks` as your `auth_provider` there won't be a way to authenticate for a device outside of your listed trusted network. To overcome this ensure you add the default `auth_provider` with `type: homeassistant` back in manually. This will then present you with the default auth login screen when trusted network authentication fails as expected from outside your LAN.

Authentication providers are configured in your `configuration.yaml` file under the `homeassistant:` block. If you are moving configuration to packages, this particular configuration must stay within 'configuration.yaml'. See Issue 16441 in the warning block at the bottom of this page.

You can supply more than one, for example:

```
homeassistant:
  auth_providers:
    - type: homeassistant
    - type: trusted_networks
  trusted_networks:
    - 192.168.0.0/24
```


Available auth providers

Home Assistant auth provider

This is the default auth provider. The first user created is designated as the *owner* and can create other users.

User details are stored in the `[your config]/.storage` directory. All passwords are stored hashed and with a salt, making it almost impossible for an attacker to figure out the password even if they have access to the file.

Users can be managed in Home Assistant by the owner. Select **Settings > People** and open the **Users** tab.

This is the entry in `configuration.yaml` for Home Assistant auth:

```
homeassistant:
  auth_providers:
    - type: homeassistant
```

If you don't specify any `auth_providers` section in the `configuration.yaml` file then this provider will be set up automatically.

Trusted networks

The trusted networks auth provider defines a range of IP addresses for which no authentication will be required (also known as "allowlisting"). For example, you can allowlist your local network so you won't be prompted for a password if you access Home Assistant from inside your home.

When you log in from one of these networks, you will be asked which user account to use and won't need to enter a password.

 **Note:** The [multi-factor authentication module](#) will not participate in the login process if you are using this auth provider.

 **Important:** You cannot trust a network that you are using in any [trusted_proxies](#). The `trusted_networks` authentication will fail with the message: Your computer is not allowed

Here is an example in `configuration.yaml` to set up Trusted Networks:

```
homeassistant:
  auth_providers:
    - type: trusted_networks
      trusted_networks:
        - 192.168.0.0/24
        - fd00::/8
```

Option	Type	Required	Default	Description
<code>trusted_networks</code>	list	Yes		A list of IP addresses or an IP network you want allowlisted. It accepts both IPv4 and IPv6 IP address or network
<code>trusted_users</code>	[list, string]	No		List of user ids available to select on this IP address or network.
<code>allow_bypass_login</code>	boolean	No	false	You can bypass login page if you have only one user available for selection.

Trusted users examples

```
homeassistant:
  auth_providers:
    - type: trusted_networks
      trusted_networks:
        - 192.168.0.0/24
        - 192.168.10.0/24
        - fd00::/8
      trusted_users:
        192.168.0.1: user1_id
        192.168.0.0/24:
          - user1_id
          - user2_id
        "fd00::/8":
          - user1_id
          - group: system-users
```

First note, for `trusted_users` configuration you need to use `user id` .

1. To find the user ID, in your browser, make sure the URL of your Home Assistant ends in `config/users/` .
2. For example: `homeassistant:8123/config/users` .
3. Select the user from the list, and copy the ID.
4. For example: `acbbff56461748718f3650fb914b88c9` .
5. The `trusted_users` configuration will not validate the existence of the user, so please make sure you have put in the correct user id.
6. A trusted user with an IPv6 address must put the IPv6 address in quotes as shown.

In the above example, if user try to access Home Assistant from 192.168.0.1, they will have only one user available to choose. They will have two users available if access from 192.168.0.38 (from 192.168.0.0/24 network). If they access from 192.168.10.0/24 network, they can choose from all available users (non-system and active users).

Specially, you can use `group: GROUP_ID` to assign all users in certain `user group` to be available to choose. Group and users can be mix and match.

Skip login page examples

This is a feature to allow you to bring back some of the experience before the user system was implemented. You can directly jump to the main page if you are accessing from trusted networks, the `allow_bypass_login` is on, and you have ONLY ONE available user to choose from in the login form.

If you allow bypass login then your cookie will not be stored and every time you refresh the page in Home Assistant a new login will be created. This is because bypassing the login does not give you the option to save the login.

```
# assuming you have only one non-system user
homeassistant:
  auth_providers:
    - type: trusted_networks
      trusted_networks:
        - 192.168.0.0/24
        - 127.0.0.1
        - ::1
      allow_bypass_login: true
    - type: homeassistant
```

Assuming you have only the owner created through onboarding process, no other users ever created. The above example configuration will allow you directly access Home Assistant main page if you access from your internal network (192.168.0.0/24) or from localhost (127.0.0.1). If you get a login abort error, then you can change to use Home Assistant Authentication Provider to login, if you access your Home Assistant instance from outside network.

 **Note:** The order of `auth_providers` is critical as providers are evaluated top to bottom. To enable skip login as intended, the `trusted_networks` provider must be listed before the `homeassistant` provider. If `type: homeassistant` is configured first, Home Assistant will immediately present the login page and the skip login logic will never be reached, even if the client is on a trusted network.

Command line

The command line auth provider executes a configurable shell command to perform user authentication. Two environment variables, `username` and `password`, are passed to the command. Access is granted when the command exits successfully (with exit code 0).

This provider can be used to integrate Home Assistant with arbitrary external authentication services, from plaintext databases over LDAP to RADIUS.

Here is a configuration example:

```
homeassistant:
  auth_providers:
    - type: command_line
      command: /absolute/path/to/command
      # Optionally, define a list of arguments to pass to the command.
      #args: ["--first", "--second"]
      # Uncomment to enable parsing of meta variables (see below).
      #meta: true
```

When `meta: true` is set in the auth provider's configuration, your command can write some variables to standard output to populate the user account created in Home Assistant with additional data. These variables have to be printed in the form:

```
name = John Doe
group = system-users
local_only = true
```

Leading and trailing whitespace, as well as lines starting with `#` are ignored. The following variables are supported. More may be added in the future.

- `name` : The real name of the user to be displayed in their profile.
- `group` : The user group uses the value `system-admin` for administrator (this is the default) or `system-users` for regular users.
- `local_only` : The user can only log in from the local network if you set the value to `true`. If you do not define this variable, the user can log in from anywhere.

Stderr is not read at all and just passed through to that of the Home Assistant process, hence you can use it for status messages or suchlike.

 **Note:** Any leading and trailing whitespace is stripped from usernames before they're passed to the configured command. For instance, " hello " will be rewritten to just "hello".

 **Note:** For now, meta variables are only respected the first time a particular user is authenticated. Upon subsequent authentications of the same user, the previously created user object with the old values is reused.

Docs/Authentication

The authentication system secures access to Home Assistant.

Login screen

You are greeted with a log in screen, asking you for username and password.

Screenshot of the login screen, when logging in from within the local network

User accounts

When you start Home Assistant for the first time, the *owner* user account is created. This account has some special privileges and can:

- Create and manage other user accounts.
- Configure integrations and other settings (coming soon).

 **Warning:** For the moment, other user accounts will have the same access as the owner account. In the future, non-owner accounts will be able to have restrictions applied.

 **Note:** If you want to manage users and you're an owner but you do not see "Users" in your main configuration menu, make sure that **Advanced Mode** is enabled for your user in your profile.

Your account profile

Once you're logged in, you can see the details of your account on the **User profile** page by selecting on the circular at the very bottom of the sidebar.

Screenshot of the profile page

You can:

- Change your password.
- Enable or disable [multi-factor authentication](#).
- Delete **Refresh tokens**. These are created when you log in from a device. Delete them if you want to force the device to log out.
- Create [Long-lived access tokens](#) so scripts can securely interact with Home Assistant.
- Define language and other locale settings.
- Log out of Home Assistant.

 **Note:** Unused refresh tokens will be automatically removed. A refresh token is considered unused if it has not been used for a login within 90 days. If you need a permanent token, then we recommend using [Long-lived access tokens](#).

Securing your login

Make sure to choose a secure password! At some time in the future, you will probably want to access Home Assistant from outside your local network. This means you are also exposed to random black-hats trying to do the same. Treat the password like the key to your house.

As an extra level of security, you can turn on [multi-factor authentication](#).

Adding a person to Home Assistant

If you have administrator rights, you can [add a person to Home Assistant](#) and create them a user account.

Changing display or username

To learn how to change a display or username, refer to [setting up basic information](#).

Other authentication techniques

Home Assistant provides several ways to authenticate. See the [Auth providers](#) section.

Troubleshooting

Authentication failures from 127.0.0.1

If you're seeing authentication failures from 127.0.0.1 and you're using the nmap device tracker, you should [exclude the Home Assistant IP](#) from being scanned.

Bearer token warnings

Under the new authentication system you'll see the following warning logged when the [legacy API password](#) is supplied, but not configured in Home Assistant:

```
WARNING (MainThread) [homeassistant.components.http.auth] You need to use a bearer
```

If you see this, you need to add an `api_password` to your `http:` configuration.

Bearer token informational messages

If you see the following, then this is a message for integration developers, to tell them they need to update how they authenticate to Home Assistant. As an end user you don't need to do anything:

```
INFO (MainThread) [homeassistant.components.http.auth] You need to use a bearer
```

Lost owner password

If you lose the password associated with the owner account, you need to [start a new onboarding process](#).

Error: invalid client id or redirect URL

Screenshot of Error: invalid client id or redirect url

You have to use a domain name, not IP address, to remote access Home Assistant otherwise you will get `Error: invalid client id or redirect url` error on the login form. However, you can use the IP address to access Home Assistant in your home network.

This is because we only allow an IP address as a client ID when your IP address is an internal network address (such as 192.168.0.1) or loopback address (such as 127.0.0.1).

If you don't have a valid domain name for your Home Assistant instance, you can modify the `hosts` file on your computer to fake one. On Linux edit the `/etc/hosts` file, and add following entry:

```
12.34.56.78 homeassistant.home
```

Replace 12.34.56.78 with your Home Assistant's public IP address.

This will allow you to open Home Assistant at `http://homeassistant.home:8123/`

Stuck on loading data

Some ad blocking software, such as Wipr, also blocks WebSockets. If you're stuck on the Loading data screen, try disabling your ad blocker.

Docs/Automation/Action

The action of an automation is what is being executed when an automation fires. The action part follows the [script syntax](#) which can be used to interact with anything via other actions or events.

For actions, you can specify the `entity_id` that it should apply to and optional parameters (to specify for example the brightness).

You can also perform the action to activate a [scene](#) which will allow you to define how you want your devices to be and have Home Assistant perform the right action.

```
automation:
  # Change the light in the kitchen and living room to 150 brightness and color
  triggers:
    - trigger: sun
      event: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id:
          - light.kitchen
          - light.living_room
      data:
        brightness: 150
        rgb_color: [255, 0, 0]

automation 2:
  # Notify me on my mobile phone of an event
  triggers:
    - trigger: sun
      event: sunset
      offset: -00:30
  variables:
    notification_action: notify.paulus_iphone
  actions:
    # Actions are scripts so can also be a list of actions
    - action: "{{ notification_action }}"
      data:
        message: "Beautiful sunset!"
    - delay: 0:35
    - action: notify.notify
      data:
        message: "Oh wow you really missed something great."
```

Conditions can also be part of an action. You can combine multiple actions and conditions in a single action, and they will be processed in the order you put them in. If the result of a condition is false, the action will stop there so any action after that condition will not be executed.

```
automation:
- alias: "Office at evening"
  triggers:
    - trigger: state
      entity_id: sensor.office_occupancy
      to: "on"
  actions:
    - action: notify.notify
      data:
        message: "Testing conditional actions"
    - condition: or
      conditions:
        - condition: numeric_state
          entity_id: sun.sun
          attribute: elevation
          below: 4
        - condition: state
          entity_id: sensor.office_illuminance
          below: 10
    - action: scene.turn_on
      target:
        entity_id: scene.office_at_evening
    - action: light.turn_on
      target: "{{ {'entity_id': ['light.office', 'light.office_2']} }}"
    - action: switch.turn_on
      target:
        label_id: "{{ ['office_evening', 'office_after_15'] }}"
```

Docs/Automation/Basics

All automations are made up of a trigger and an action. Optionally combined with a condition. Take for example the automation:

When Paulus arrives home and it is after sunset: Turn the lights on in the living room.

We can break up this automation into the following three parts:

```
(trigger)    When Paulus arrives home
(condition)  and it is after sunset:
(action)     Turn the lights on in the living room
```

The first part is the **trigger** of the automation. Triggers describe events that should trigger the automation. In this case, it is a person arriving home, which can be observed in Home Assistant using devices/sensors by observing the state of Paulus changing from `not_home` to `home`.

The second part is the **condition**. Conditions are optional tests that can limit an automation to only work in your specific use cases. A condition will test against the current state of the system. This includes the current time, devices, people and other things like the sun. In this case, we only want to act when the sun has set.

The third part is the **action**, which will be performed when an automation is triggered and all conditions are met. For example, it can turn a light on, set the temperature on your thermostat or activate a scene.

 **Note:** The difference between a trigger and a condition can be confusing as they are very similar.

Triggers require an event to happen for the conditions to be evaluated using current state information.

Event: Arrive home \ Condition: After Sunset? \ Action: Turn lights on

Exploring the internal state

Automations interact directly with the internal state of Home Assistant, so you'll need to familiarize yourself with it. Home Assistant exposes its current state via the developer tools. These are available at the bottom of the sidebar in the frontend. **Settings > Developer tools > States** will show all currently available states. An entity can be anything. A light, a switch, a person and even the sun. A state consists of the following parts:

Name	Description	Example
Entity ID	Unique identifier for the entity.	<code>light.living_room</code>
State	The current state of the device.	<code>off</code>
Attributes	Extra data related to the device and/or current state.	<code>brightness</code>

State changes can be used as the source of triggers and the current state can be used in conditions.

To explore the available *actions* open the **Settings > Developer tools > Actions**. *Actions* allow changing anything. For example, turn on a light, run a script, or enable a scene. Each *action* has a domain and a name. For example, the *action* `developer_call_service` is capable of turning on any light in your system. Parameters can be passed to an *action* to indicate, for example, which device to activate or which color to use.

Creating automations

Now that you've got a sneak peek of what is possible, it's time to get your feet wet and create your first automation.

Using the automation editor »

Docs/Automation/Condition

Conditions are an optional part of an automation rule. They can be used to prevent the automation's actions from being run. After a trigger occurred, all conditions will be checked. The automation will be executed if all conditions return `true`. If any of the conditions returns `false`, the automation won't start.

Conditions look very similar to triggers, but they are very different — a trigger can observe events that may have happened and start an automation. A condition will only see the current state after the automation is started from the trigger. Take the example of a switch being turned on and then off in quick succession. That switch turned on event will start an automation regardless the fact is now off again. By the time the automation checks the conditions from the switch on event, it may already be off again as its current state. This scenario is also known as a race condition.

The available conditions for an automation are the same as for the script syntax so see that page for a [full list of available conditions](#).

Example of using condition:

```
automation:
- alias: "Turn on office lights"
  triggers:
    - trigger: state
      entity_id: sensor.office_motion_sensor
      to: "on"
  conditions:
    - or:
      - condition: numeric_state
        entity_id: sun.sun
        attribute: elevation
        below: 4
      - condition: numeric_state
        entity_id: sensor.office_lux_sensor
        below: 10
  actions:
    - action: scene.turn_on
      target:
        entity_id: scene.office_lights
```

The `condition` option of an automation, also accepts a single condition template directly. For example:

```
automation:
- alias: "Turn on office lights"
  triggers:
    - trigger: state
      entity_id: sensor.office_motion_sensor
      to: "on"
  conditions: "{{ state_attr('sun.sun', 'elevation') < 4 }}"
  actions:
    - action: scene.turn_on
      target:
        entity_id: scene.office_lights
```

Docs/Automation/Editor

The automation editor lets you create and edit automations directly from the Home Assistant user interface, without writing any YAML. The editor walks you through choosing a trigger, optional conditions, and the actions to run.

This tutorial uses the [Random sensor](#) because it generates data (by default, values between 0 and 20). This enables us to walk through the example, even if you do not have any actual sensors connected yet. You could use any other sensor that outputs a numeric value.

1. Go to **Settings > Automations & scenes** and in the lower right corner, select the **Create automation** button.

2. Select **Create new automation**.

Create automation dialogue box

3. Select **Add trigger**, and in the **Search trigger** field, type "num".

4. Select **Numeric state**.

Add trigger

5. Enter the trigger conditions:

6. Define the sensor: Under **Entity**, enter "sensor.random_sensor".

7. If the sensor value is above 10, we want the automation to trigger.

- In the **Above** field, enter "10".

Automation trigger

8. Define the action that should happen:

9. In the **Then do** section, select **Add action**.

Add action

10. We want to create a [persistent notification](#).

11. Enter "No" and select **Notifications: send a persistent notification**.

Automation action

12. As the message, we want a simple text that is shown as part of the notification.

```
yaml message: Sensor value greater than 10
```

13. Select **Save**, give your automation a meaningful name, and **Save** again.

New automation editor

- Result: Automations created or edited via the user interface are activated immediately after saving the automation.
- To learn more about automations, read the documentation for [Automating Home Assistant](#).

Troubleshooting missing automations

When you're creating automations using the GUI and they don't appear in the UI, make sure that you add back `automation: !include automations.yaml` from the default configuration to your `configuration.yaml` .

Docs/Automation/Modes

Sometimes an automation gets triggered again before it has finished running from the previous time. The mode setting tells Home Assistant what to do in that situation: ignore the new trigger, start over, queue it up, or run another copy in parallel.

Most of the time, the default `single` mode is exactly what you want. The other modes are there for special cases, like a notification automation that should run a fresh copy for every event, or a long-running sequence that should restart from the top whenever something changes.

Mode	Description
<code>single</code>	(Default) Do not start a new run. Issue a warning.
<code>restart</code>	Start a new run after first stopping the previous run. The automation only restarts if the conditions are met.
<code>queued</code>	Start a new run after all previous runs complete. Runs are guaranteed to execute in the order they were queued. Note that subsequent queued automations will only join the queue if any conditions it may have are met at the time it is triggered.
<code>parallel</code>	Start a new, independent run in parallel with previous runs.

For both `queued` and `parallel` modes, configuration option `max` controls the maximum number of runs that can be executing and/or queued up at a time. The default is 10.

When `max` is exceeded (which is effectively 1 for `single` mode) a log message will be emitted to indicate this has happened. Configuration option `max_exceeded` controls the severity level of that log message. Set it to `silent` to ignore warnings or set it to a [log level](#). The default is `warning`.

Example throttled automation

Some automations you only want to run every 5 minutes. This can be achieved using the `single` mode and silencing the warnings when the automation is triggered while it's running.

```
automation:
  - mode: single
    max_exceeded: silent
    triggers:
      - ...
    actions:
      - ...
      - delay: 300 # seconds (=5 minutes)
```

Example queued

Sometimes an automation is doing an action on a device that does not support multiple simultaneous actions. In such cases, a queue can be used. In that case, the automation will be executed once it's current invocation and queue are done.

```
automation:
  - mode: queued
    max: 25
    triggers:
      - ...
    actions:
      - ...
```


Docs/Automation/Services

The automation integration has actions to control automations, like turning automations on and off. This can be useful if you want to disable an automation from another automation.

Action developer_call_service

This action enables the automation's triggers.

Data attribute	Optional	Description
<code>entity_id</code>	no	Entity ID of automation to turn on. Can be a list. <code>none</code> or <code>all</code> are also accepted.

Action developer_call_service

This action disables the automation's triggers, and optionally stops any currently active actions.

Data attribute	Optional	Description
<code>entity_id</code>	no	Entity ID of automation to turn off. Can be a list. <code>none</code> or <code>all</code> are also accepted.
<code>stop_actions</code>	yes	Stop any currently active actions (defaults to true).

Action developer_call_service

This action enables the automation's triggers if they were disabled, or disables the automation's triggers, and stops any currently active actions, if the triggers were enabled.

Data attribute	Optional	Description
<code>entity_id</code>	no	Entity ID of automation to turn on. Can be a list. <code>none</code> or <code>all</code> are also accepted.

Action developer_call_service

This action will trigger the action of an automation. By default it bypasses any conditions, though that can be changed via the `skip_condition` attribute.

Data attribute	Optional	Description
<code>entity_id</code>	no	Entity ID of automation to trigger. Can be a list. <code>none</code> or <code>all</code> are also accepted.
<code>skip_condition</code>	yes	Whether or not the condition will be skipped (defaults to true).

Action developer_call_service

This action is only required if you create/edit automations in YAML. Automations via the UI do this automatically.

This action reloads all automations, stopping all currently active automation actions.

Docs/Automation/Templating

Automations support the advanced features of [templating](#) in the same way as scripts do. In addition to the [Home Assistant template extensions](#) available to scripts, the `trigger` and `this` template variables are available for automations.

Example of variables used in templates:

- `%name%` is the name of the automation executing from this trigger
- `%trigger%` is the type of trigger object, like `calendar`

Available state data

The template variable `this` is an object that contains the `state` of the automation at the moment of triggering the actions and can be used to evaluate `trigger_variables` declared in the configuration of the active trigger. State objects also contain context data which can be used to identify the user that caused a script or automation to execute. Note that `this` will not change while executing the actions.

Available trigger data

The template variable `trigger` is an object that contains details about which platform triggered the automation. The `platform` property contains the name of the platform whose event triggered the automation.

Templates can use the data to modify the actions performed by the automation or displayed in a message. For example, you could create an automation that multiple sensors can trigger and then use the sensor's location to specify a light to activate; or you could send a notification containing the friendly name of the sensor that triggered it.

Each [trigger](#) platform includes additional data specific to that platform.

All

Triggers from all platforms will include the following properties.

Template variable	Data
<code>trigger.platform</code>	Trigger object type.
<code>trigger.alias</code>	Alias of the trigger.
<code>trigger.id</code>	The <code>id</code> of the trigger.
<code>trigger.idx</code>	Index of the trigger. (The first trigger idx is <code>0</code> .)

Calendar

These are the properties available for a [Calendar trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>calendar</code>
<code>trigger.event</code>	The trigger event type, either <code>start</code> or <code>end</code> .
<code>trigger.calendar_event</code>	The calendar event object matched.
<code>trigger.calendar_event.summary</code>	The title or summary of the calendar event.
<code>trigger.calendar_event.start</code>	String representation of the start date or date time of the calendar event, for example <code>2022-04-10</code> , or <code>2022-04-10 11:30:00-07:00</code>
<code>trigger.calendar_event.end</code>	String representation of the end time of date time the calendar event in UTC, for example <code>2022-04-11</code> , or <code>2022-04-10 11:45:00-07:00</code>
<code>trigger.calendar_event.all_day</code>	Indicates the event spans the entire day.
<code>trigger.calendar_event.description</code>	A detailed description of the calendar event, if available.
<code>trigger.calendar_event.location</code>	Location information for the calendar event, if available.
<code>trigger.offset</code>	Timedelta object with offset to the event, if any.

Device

These are the properties available for a [Device trigger](#).

Inherits template variables from [event](#) or [state](#) template based on the type of trigger selected for the device.

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>device</code>

Event

An [Event](#) trigger is fired each time an entity state changes or an event matching the configured `event_type` occurs.

These are the properties available for an [Event trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>event</code>
<code>trigger.event</code>	Event object that matched.
<code>trigger.event.event_type</code>	Event type.
<code>trigger.event.data</code>	Optional event data.

Geolocation

These are the properties available for a [Geolocation trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>geo_location</code>
<code>trigger.event</code>	The trigger event type, either <code>enter</code> or <code>leave</code> .
<code>trigger.source</code>	The Geolocation platform creating the trigger event.
<code>trigger.zone</code>	State object of the zone.

Home Assistant

The Home Assistant trigger is recommended for automations instead of `homeassistant_start` or `homeassistant_stop` events.

These are the properties available for a [Home Assistant trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>homeassistant</code>
<code>trigger.event</code>	The trigger event type, either <code>start</code> or <code>shutdown</code> .

MQTT

These are the properties available for an [MQTT trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>mqtt</code>
<code>trigger.topic</code>	Topic that received payload.
<code>trigger.payload</code>	Payload.
<code>trigger.payload_json</code>	Dictionary of the JSON parsed payload.
<code>trigger.qos</code>	QOS of payload.

Numeric state

These are the properties available for a [numeric state trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>numeric_state</code>
<code>trigger.entity_id</code>	Entity ID that we observe.
<code>trigger.below</code>	The below threshold, if any.
<code>trigger.above</code>	The above threshold, if any.
<code>trigger.from_state</code>	The previous state object of the entity.
<code>trigger.to_state</code>	The new state object that triggered trigger.
<code>trigger.for</code>	Timedelta object how long state has met above/below criteria, if any.

Sentence

These are the properties available for a [Sentence trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>conversation</code>
<code>trigger.sentence</code>	Text of the sentence that was matched.
<code>trigger.slots</code>	Object with matched slot values.
<code>trigger.details</code>	Object with matched slot details by name, such as wildcards . Each detail contains: <code>name</code> - name of the slot <code>text</code> - matched text <code>value</code> - output value (see lists) .
<code>trigger.device_id</code>	The device ID that captured the command, if any.
<code>trigger.satellite_id</code>	The entity ID of the satellite that captured the command, if any.

State

These are the properties available for a [State trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>state</code>
<code>trigger.entity_id</code>	Entity ID that we observe.
<code>trigger.from_state</code>	The previous <code>state</code> object of the entity.
<code>trigger.to_state</code>	The new <code>state</code> object that triggered trigger.
<code>trigger.for</code>	Timedelta object how long state has been to state, if any.

Sun

These are the properties available for a [Sun trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>sun</code>
<code>trigger.event</code>	The event that just happened: <code>sunset</code> or <code>sunrise</code> .
<code>trigger.offset</code>	Timedelta object with offset to the event, if any.

Tag

These are the properties available for a [Tag trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>tag</code>
<code>trigger.tag_id</code>	The tag ID captured.
<code>trigger.event.data.device_id</code>	Optional device ID that captured the tag.

Template

These are the properties available for a [Template trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>template</code>
<code>trigger.entity_id</code>	Entity ID that caused change.
<code>trigger.from_state</code>	Previous <code>state</code> object of entity that caused change.
<code>trigger.to_state</code>	New <code>state</code> object of entity that caused template to change.
<code>trigger.for</code>	Timedelta object how long state has been to state, if any.

Time

These are the properties available for a [Time trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>time</code>
<code>trigger.now</code>	DateTime object that triggered the time trigger.

Time pattern

These are the properties available for a [time pattern trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>time_pattern</code>
<code>trigger.now</code>	DateTime object that triggered the time_pattern trigger.

Persistent notification

These properties are available for a [persistent notification trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>persistent_notification</code>
<code>trigger.update_type</code>	Type of persistent notification update <code>added</code> , <code>removed</code> , <code>current</code> , or <code>updated</code> .
<code>trigger.notification</code>	Notification object that triggered the persistent notification trigger.
<code>trigger.notification.notification_id</code>	The notification ID.
<code>trigger.notification.title</code>	Title of the notification.
<code>trigger.notification.message</code>	Message of the notification.
<code>trigger.notification.created_at</code>	DateTime object indicating when the notification was created.

Webhook

These are the properties available for a [Webhook trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>webhook</code>
<code>trigger.webhook_id</code>	The webhook ID that was triggered.
<code>trigger.json</code>	The JSON data of the request (if it had a JSON content type) as a mapping.
<code>trigger.data</code>	The form data of the request (if it had a form data content type).
<code>trigger.query</code>	The URL query parameters of the request (if provided).

Zone

These are the properties available for a [Zone trigger](#).

Template variable	Data
<code>trigger.platform</code>	Hardcoded: <code>zone</code>
<code>trigger.entity_id</code>	Entity ID that we are observing.
<code>trigger.from_state</code>	Previous state object of the entity.
<code>trigger.to_state</code>	New state object of the entity.
<code>trigger.zone</code>	State object of the zone.
<code>trigger.event</code>	Event that trigger observed: <code>enter</code> or <code>leave</code> .

Examples

```

# Example configuration.yaml entries
automation:
  triggers:
    - trigger: state
      entity_id: device_tracker.paulus
      id: paulus_device
  actions:
    - action: notify.notify
      data:
        message: >
          Paulus just changed from {{ trigger.from_state.state }}
          to {{ trigger.to_state.state }}

          This was triggered by {{ trigger.id }}

automation 2:
  triggers:
    - trigger: mqtt
      topic: "/notify/+"
  actions:
    - action: >
        notify.{{ trigger.topic.split('/')[1] }}
      data:
        message: "{{ trigger.payload }}"

automation 3:
  triggers:
    # Multiple entities for which you want to perform the same action.
    - trigger: state
      entity_id:
        - light.bedroom_closet
        - light.kiddos_closet
        - light.linen_closet
      to: "on"
      # Trigger when someone leaves one of those lights on for 10 minutes.
      for: "00:10:00"
  actions:
    - action: light.turn_off
      target:
        # Turn off whichever entity triggered the automation.
        entity_id: "{{ trigger.entity_id }}"

automation 4:
  triggers:
    # When an NFC tag is scanned by Home Assistant...
    - trigger: event
      event_type: tag_scanned
      # ...By certain people
      context:
        user_id:
          - 06cbf6deafc54cf0b2ffa49552a396ba
          - 2df8a2a6e0be4d5d962aad2d39ed4c9c
  conditions:
    # Check NFC tag (ID) is the one by the front door
    - condition: template
      value_template: "{{ trigger.event.data.tag_id == '8b6d6755-b4d5-4c23-818b-
  actions:
    # Turn off various lights
    - action: light.turn_off
      target:

```

```
entity_id:
  - light.kitchen
  - light.bedroom
  - light.living_room
```

Docs/Automation/Trigger

A trigger is what wakes an automation up. Until something triggers it, an automation just sits there quietly, waiting. The moment a trigger fires, Home Assistant checks any [conditions](#) you set, and if they pass, it runs the [actions](#).

Triggers can be almost anything that happens in your home or in Home Assistant itself. A motion sensor detecting movement. The sun going down. A specific time of day. A person arriving home. A button on a remote being pressed. Even a voice command spoken to Assist. You can give a single automation more than one trigger, and the automation will start as soon as *any* of them fires.

- [Trigger ID](#)
- [Trigger variables](#)
- [Event trigger](#)
- [Home Assistant trigger](#)
- [MQTT trigger](#)
- [Numeric state trigger](#)
- [State trigger](#)
- [Sun trigger](#)
- [Tag trigger](#)
- [Template trigger](#)
- [Time trigger](#)
- [Time pattern trigger](#)
- [Persistent notification trigger](#)
- [Webhook trigger](#)
- [Zone trigger](#)
- [Geolocation trigger](#)
- [Device triggers](#)
- [Calendar trigger](#)
- [Sentence trigger](#)
- [Multiple triggers](#)
- [Multiple Entity IDs for the same Trigger](#)
- [Disabling a trigger](#)
- [Merging lists of triggers](#)

Trigger ID

All triggers can be assigned an optional `id`. If the ID is omitted, it will instead be set to the index of the trigger. The `id` can be referenced from [trigger conditions](#) and [actions](#). The `id` does not have to be unique for each trigger, and it can be used to group similar triggers for use later in the automation (such as several triggers of different types that should all turn some entity on).

Video tutorial

This video tutorial explains how trigger IDs work.

```
automation:
  triggers:
    - trigger: event
      event_type: "MY_CUSTOM_EVENT"
      id: "custom_event"
    - trigger: mqtt
      topic: "living_room/switch/ac"
      id: "ac_on"
    - trigger: state # This trigger will be assigned id="2"
      entity_id:
        - device_tracker.paulus
        - device_tracker.anne_therese
      to: "home"
```

Trigger variables

There are two different types of variables available for triggers. Both work like [script level variables](#).

The first variant allows you to define variables that will be set when the trigger fires. The variables will be able to use templates and have access to the `trigger` variable.

The second variant is setting variables that are available when attaching a trigger when the trigger can contain templated values. These are defined using the `trigger_variables` key at an automation level. These variables can only contain [limited templates](#). The triggers will not re-apply if the value of the template changes. Trigger variables are a feature meant to support using blueprint inputs in triggers.

```
automation:
  trigger_variables:
    my_event: example_event
  triggers:
    - trigger: event
      # Able to use `trigger_variables`
      event_type: "{{ my_event }}"
      # These variables are evaluated and set when this trigger is triggered
      variables:
        name: "{{ trigger.event.data.name }}"
```

Event trigger

An event trigger fires when an [event](#) is being received. Events are the raw building blocks of Home Assistant. You can match events on just the event name or also require specific event data or context to be present.

Events can be fired by integrations or via the API. There is no limitation to the types. A list of built-in events can be found [here](#).

```
automation:
  triggers:
    - trigger: event
      event_type: "MY_CUSTOM_EVENT"
      # optional
      event_data:
        mood: happy
      context:
        user_id:
          # any of these will match
          - "MY_USER_ID"
          - "ANOTHER_USER_ID"
```

It is also possible to listen for multiple events at once. This is useful for event that contain no, or similar, data and contexts.

```
automation:
  triggers:
    - trigger: event
      event_type:
        - automation_reloaded
        - scene_reloaded
```

It's also possible to use [limited templates](#) in the `event_type`, `event_data` and `context` options.

⚠ Important: The `event_type`, `event_data` and `context` templates are only evaluated when setting up the trigger, they will not be reevaluated for every event.

```
automation:
  trigger_variables:
    sub_event: ABC
    node: ac
    value: on
  triggers:
    - trigger: event
      event_type: "{{ 'MY_CUSTOM_EVENT_' ~ sub_event }}"
```

Home Assistant trigger

Fires when Home Assistant starts up or shuts down.

```
automation:
  triggers:
    - trigger: homeassistant
      # Event can also be 'shutdown'
      event: start
```

 **Note:** Automations triggered by the `shutdown` event have 20 seconds to run, after which they are stopped to continue with the shutdown.

MQTT trigger

Fires when a specific message is received on given MQTT topic. Optionally can match on the payload being sent over the topic. The default payload encoding is 'utf-8'. For images and other byte payloads use `encoding: ''` to disable payload decoding completely.

```
automation:
  triggers:
    - trigger: mqtt
      topic: "living_room/switch/ac"
      # Optional
      payload: "on"
      encoding: "utf-8"
```

The `payload` option can be combined with a `value_template` to process the message received on the given MQTT topic before matching it with the payload. The trigger in the example below will trigger only when the message received on `living_room/switch/ac` is valid JSON, with a key `state` which has the value `"on"`.

```
automation:
  triggers:
    - trigger: mqtt
      topic: "living_room/switch/ac"
      payload: "on"
      value_template: "{{ value_json.state }}"
```

It's also possible to use [limited templates](#) in the `topic` and `payload` options.

 **Note:** The `topic` and `payload` templates are only evaluated when setting up the trigger, they will not be re-evaluated for every incoming MQTT message.

```
automation:
  trigger_variables:
    room: "living_room"
    node: "ac"
    value: "on"
  triggers:
    - trigger: mqtt
      topic: "{{ room ~ '/switch/' ~ node }}"
      # Optional
      payload: "{{ 'state:' ~ value }}"
      encoding: "utf-8"
```


Numeric state trigger

Fires when the numeric value of an entity's state (or attribute's value if using the `attribute` property, or the calculated value if using the `value_template` property) **crosses** a given threshold (equal excluded). On state change of a specified entity, attempts to parse the state as a number and fires if the value is changing from above to below or from below to above the given threshold (equal excluded).

 **Note:** Crossing the threshold means that the trigger only fires if the state wasn't previously within the threshold. If the current state of your entity is `50` and you set the threshold to `below: 75`, the trigger would not fire if the state changed to `49` or `72` because the threshold was never crossed. The state would first have to change to `76` and then to `74` for the trigger to fire.

```
automation:
  triggers:
    - trigger: numeric_state
      entity_id: sensor.temperature
      # If given, will trigger when the value of the given attribute for the given
      attribute: attribute_name
      # ..or alternatively, will trigger when the value given by this evaluated
      value_template: "{{ state.attributes.value - 5 }}"
      # At least one of the following required
      above: 17
      below: 25
      # If given, will trigger when the condition has been true for X time; you
      for:
        hours: 1
        minutes: 10
        seconds: 5
```

 **Note:** Listing above and below together means the numeric_state has to be between the two values. In the example above, the trigger would fire a single time if a numeric_state goes into the 17.1-24.9 range (above 17 and below 25). It will only fire again, once it has left the defined range and enters it again.

When the `attribute` option is specified the trigger is compared to the given `attribute` instead of the state of the entity.

```
automation:
  triggers:
    - trigger: numeric_state
      entity_id: climate.kitchen
      attribute: current_temperature
      above: 23
```

More dynamic and complex calculations can be done with `value_template`. The variable 'state' is the `state object` of the entity specified by `entity_id`.

The state of the entity can be referenced like this:

```
automation:
  triggers:
    - trigger: numeric_state
      entity_id: sensor.temperature
      value_template: "{{ state.state | float * 9 / 5 + 32 }}"
      above: 70
```

Attributes of the entity can be referenced like this:

```
automation:
  triggers:
    - trigger: numeric_state
      entity_id: climate.kitchen
      value_template: "{{ state.attributes.current_temperature - state.attribute
      above: 3
```

Number helpers (`input_number` entities), `number`, `sensor`, and `zone` entities that contain a numeric value, can be used in the `above` and `below` thresholds. However, the comparison will only be made when the entity specified in the trigger is updated. This would look like:

```
automation:
  triggers:
    - trigger: numeric_state
      entity_id: sensor.outside_temperature
      # Other entity ids can be specified for above and/or below thresholds
      above: sensor.inside_temperature
```

The `for:` can also be specified as `HH:MM:SS` like this:

```
automation:
  triggers:
    - trigger: numeric_state
      entity_id: sensor.temperature
      # At least one of the following required
      above: 17
      below: 25

      # If given, will trigger when condition has been for X time.
      for: "01:10:05"
```

You can also use templates in the `for` option.

```

automation:
  triggers:
    - trigger: numeric_state
      entity_id:
        - sensor.temperature_1
        - sensor.temperature_2
      above: 80
      for:
        minutes: "{{ states('input_number.high_temp_min')|int }}"
        seconds: "{{ states('input_number.high_temp_sec')|int }}"
  actions:
    - action: persistent_notification.create
      data:
        message: >
          {{ trigger.to_state.name }} too high for {{ trigger.for }}!

```

The `for` template(s) will be evaluated when an entity changes as specified.

⚠ Important: Use of the `for` option will not survive Home Assistant restart or the reload of automations. During restart or reload, automations that were awaiting `for` the trigger to pass, are reset.

If for your use case this is undesired, you could consider using the automation to set an `input_datetime` to the desired time and then use that `input_datetime` as an automation trigger to perform the desired actions at the set time.

State trigger

In general, the state trigger fires when the state of any of given entities **changes**. The behavior is as follows:

- If only the `entity_id` is given, the trigger fires for **all** state changes, even if only a state attribute changed.
- If at least one of `from`, `to`, `not_from`, or `not_to` are given, the trigger fires on any matching state change, but not if only an attribute changed.
- To trigger on all state changes, but not on changed attributes, set at least one of `from`, `to`, `not_from`, or `not_to` to `null`.
- Use of the `for` option doesn't survive a Home Assistant restart or the reload of automations.
- During restart or reload, automations that were awaiting `for`, the trigger to pass, are reset.
- If for your use case this is undesired, you could consider using the automation to set an `input_datetime` to the desired time and then use that `input_datetime` as an automation trigger to perform the desired actions at the set time.

 **Tip:** The values you see in your overview will often not be the same as the actual state of the entity. For instance, the overview may show `Connected` when the underlying entity is actually `on`. You should check the state of the entity by checking the states in the developer tool, under **Settings > Developer tools > States**.

Examples

This automation triggers if either Paulus or Anne-Therese are home for one minute.

```
automation:
  triggers:
    - trigger: state
      entity_id:
        - device_tracker.paulus
        - device_tracker.anne_therese
      # Optional
      from: "not_home"
      # Optional
      to: "home"
      # If given, will trigger when the condition has been true for X time; you
      for:
        hours: 0
        minutes: 1
        seconds: 0
```

It's possible to give a list of `from` states or `to` states:

```
automation:
  triggers:
    - trigger: state
      entity_id: vacuum.test
      from:
        - "cleaning"
        - "returning"
      to: "error"
```

If you want to trigger on all state changes, but not on attribute changes, you can `to` to `null` (this would also work by setting `from`, `not_from`, or `not_to` to `null`):

```
automation:
  triggers:
    - trigger: state
      entity_id: vacuum.test
      to:
```

If you want to trigger on all state changes *except* specific ones, use `not_from` or `not_to`. The `not_from` and `not_to` options are the counterparts of `from` and `to`. They can be used to trigger on state changes that are **not** the specified state.

```
automation:
  triggers:
    - trigger: state
      entity_id: vacuum.test
      not_from:
        - "unknown"
        - "unavailable"
      to: "on"
```

You cannot use `from` and `not_from` at the same time. The same applies to `to` and `not_to`.

Triggering on attribute changes

When the `attribute` option is specified, the trigger only fires when the specified attribute **changes**. Changes to other attributes or state changes are ignored.

For example, this trigger only fires when the boiler has been heating for 10 minutes:

```
automation:
  triggers:
    - trigger: state
      entity_id: climate.living_room
      attribute: hvac_action
      to: "heating"
      for: "00:10:00"
```

This trigger fires whenever the boiler's `hvac_action` attribute changes:

```
automation:
  triggers:
    - trigger: state
      entity_id: climate.living_room
      attribute: hvac_action
```

Holding a state or attribute

You can use `for` to have the state trigger only fire if the state holds for some time.

This example fires, when the entity state changed to `"on"` and holds that state for 30 seconds:

```
automation:
  triggers:
    - trigger: state
      entity_id: light.office
      # Must stay "on" for 30 seconds
      to: "on"
      for: "00:00:30"
```

When holding a state, changes to attributes are ignored. Changes to attributes don't cancel the hold time.

You can also fire the trigger when the state value changed from a specific state, but hasn't returned to that state value for the specified time.

This can be useful for checking if a media player hasn't turned "off" for the time specified, but doesn't care about "playing" or "paused".

```
automation:
  triggers:
    - trigger: state
      entity_id: media_player.kitchen
      # Not "off" for 30 minutes
      from: "off"
      for: "00:30:00"
```

Please note, that when using `from`, `to` and `for`, only the value of the `to` option is considered for the time specified.

In this example, the trigger fires if the state value of the entity remains the same for `for` the time specified, regardless of the current state value.

```
automation:
  triggers:
    - trigger: state
      entity_id: media_player.kitchen
      # The media player remained in its current state for 1 hour
      for: "01:00:00"
```

You can also use templates in the `for` option.

```
automation:
  triggers:
    - trigger: state
      entity_id:
        - device_tracker.paulus
        - device_tracker.anne_thereise
      to: "home"
      for:
        minutes: "{{ states('input_number.lock_min')|int }}"
        seconds: "{{ states('input_number.lock_sec')|int }}"
  actions:
    - action: lock.lock
      target:
        entity_id: lock.my_place
```

The `for` template(s) will be evaluated when an entity changes as specified.

 **Tip:** Use quotes around your values for `from` and `to` to avoid the YAML parser from interpreting values as booleans.

Sun trigger

Sunset / Sunrise trigger

Fires when the sun is setting or rising—that is, when the sun elevation reaches 0°.

An optional time offset can be given to have it fire a set time before or after the sun event (for example, 45 minutes before sunset). A negative value makes it fire before sunrise or sunset, a positive value afterwards. The offset needs to be specified in number of seconds, or in a hh:mm:ss format.

 **Tip:** Since the duration of twilight is different throughout the year, it is recommended to use [sun elevation triggers](#) instead of `sunset` or `sunrise` with a time offset to trigger automations during dusk or dawn.

```
automation:
  triggers:
    - trigger: sun
      # Possible values: sunset, sunrise
      event: sunset
      # Optional time offset. This example will trigger 45 minutes before sunset
      offset: "-00:45:00"
```

Sun elevation trigger

Sometimes you may want more granular control over an automation than simply sunset or sunrise and specify an exact elevation of the sun. This can be used to layer automations to occur as the sun lowers on the horizon or even after it is below the horizon. This is also useful when the "sunset" event is not dark enough outside and you would like the automation to run later at a precise solar angle instead of the time offset such as turning on exterior lighting. For most automations intended to run during dusk or dawn, a number between 0° and -6° is suitable; -4° is used in this example:

```
automation:
  - alias: "Exterior Lighting on when dark outside"
    triggers:
      - trigger: numeric_state
        entity_id: sun.sun
        attribute: elevation
        # Can be a positive or negative number
        below: -4.0
    actions:
      - action: switch.turn_on
        target:
          entity_id: switch.exterior_lighting
```


If you want to get more precise, you can use this [solar calculator](#), which will help you estimate what the solar elevation will be at any specific time. Then from this, you can select from the defined twilight numbers.

Although the actual amount of light depends on weather, topography and land cover, they are defined as:

- Civil twilight: $0^\circ > \text{Solar angle} > -6^\circ$

This is what is meant by twilight for the average person: Under clear weather conditions, civil twilight approximates the limit at which solar illumination suffices for the human eye to clearly distinguish terrestrial objects. Enough illumination renders artificial sources unnecessary for most outdoor activities.

- Nautical twilight: $-6^\circ > \text{Solar angle} > -12^\circ$
- Astronomical twilight: $-12^\circ > \text{Solar angle} > -18^\circ$

A very thorough explanation of this is available in the Wikipedia article about the [Twilight](#).

Tag trigger

Fires when a [tag](#) is scanned. For example, an NFC tag is scanned using the Home Assistant Companion mobile application.

```
automation:
  triggers:
    - trigger: tag
      tag_id: A7-6B-90-5F
```

Additionally, you can also only trigger if a card is scanned by a specific device/scanner by setting the `device_id`:

```
automation:
  triggers:
    - trigger: tag
      tag_id: A7-6B-90-5F
      device_id: 0e19cd3cf2b311ea88f469a7512c307d
```

Or trigger on multiple possible devices for multiple tags:

```
automation:
  triggers:
    - trigger: tag
      tag_id:
        - "A7-6B-90-5F"
        - "A7-6B-15-AC"
      device_id:
        - 0e19cd3cf2b311ea88f469a7512c307d
        - d0609cb25f4a13922bb27d8f86e4c821
```

Template trigger

Template triggers work by evaluating a `template` when any of the recognized entities change state. The trigger will fire if the state change caused the template to render 'true' (a non-zero number or any of the strings `true`, `yes`, `on`, `enable`) when it was previously 'false' (anything else).

This is achieved by having the template result in a true boolean expression (for example `` ) or by having the template render true`` (example below).

With template triggers you can also evaluate attribute changes by using `is_state_attr` (like ``

```
automation:
  triggers:
    - trigger: template
      value_template: "{% if is_state('device_tracker.paulus', 'home') %}true{%

      # If given, will trigger when template remains true for X time.
      for: "00:01:00"
```

You can also use templates in the `for` option.

```
automation:
  triggers:
    - trigger: template
      value_template: "{{ is_state('device_tracker.paulus', 'home') }}"
      for:
        minutes: "{{ states('input_number.minutes')|int(0) }}"
```

The `for` template(s) will be evaluated when the `value_template` becomes 'true'.

Templates that do not contain an entity will be rendered once per minute.

⚠ Important: Use of the `for` option will not survive Home Assistant restart or the reload of automations. During restart or reload, automations that were awaiting `for` the trigger to pass, are reset.

If for your use case this is undesired, you could consider using the automation to set an `input_datetime` to the desired time and then use that `input_datetime` as an automation trigger to perform the desired actions at the set time.

Time trigger

The time trigger is configured to fire once a day at a specific time, or at a specific time on a specific date. There are three allowed formats:

Time string

A string that represents a time to fire on each day. Can be specified as `HH:MM` or `HH:MM:SS`. If the seconds are not specified, `:00` will be used.

```
automation:
- triggers:
- trigger: time
  # 24-hour time format. This trigger will fire at 3:32 PM
  at: "15:32:00"
```

Input datetime

The entity ID of an `input_datetime`.

has_date	has_time	Description
true	true	Will fire at specified date & time.
true	false	Will fire at midnight on specified date.
false	true	Will fire once a day at specified time.

```
automation:
- triggers:
- trigger: state
  entity_id: binary_sensor.motion
  to: "on"
actions:
- action: climate.turn_on
  target:
    entity_id: climate.office
- action: input_datetime.set_datetime
  target:
    entity_id: input_datetime.turn_off_ac
  data:
    datetime: >
      {{ (now().timestamp() + 2*60*60)
        | timestamp_custom('%Y-%m-%d %H:%M:%S') }}
- triggers:
- trigger: time
  at: input_datetime.turn_off_ac
actions:
- action: climate.turn_off
  target:
    entity_id: climate.office
```

Sensors of datetime device class

The Entity ID of a [sensor](#) with the "timestamp" device class.

```
automation:
- triggers:
  - trigger: time
    at: sensor.phone_next_alarm
actions:
- action: light.turn_on
  target:
    entity_id: light.bedroom
```

Sensors of datetime device class with offsets

When the time is provided using a sensor of the timestamp device class, an offset can be provided. This offset will be added to (or subtracted from when negative) the sensor value.

For example, this trigger fires 5 minutes before the phone alarm goes off.

```
automation:
- triggers:
  - trigger: time
    at:
      entity_id: sensor.phone_next_alarm
      offset: -00:05:00
actions:
- action: light.turn_on
  target:
    entity_id: light.bedroom
```

⚠ Important: When using a positive offset the trigger might never fire. This is due to the sensor changing before the offset is reached. For example, when using a phone alarm as a trigger, the sensor value will change to the new alarm time when the alarm goes off, which means this trigger will change to the new time as well.

Multiple times

Multiple times can be provided in a list. All formats can be intermixed.

```
automation:
triggers:
- trigger: time
  at:
    - input_datetime.leave_for_work
    - "18:30:00"
    - entity_id: sensor.bus_arrival
      offset: "-00:10:00"
```

Limited templates

It's also possible to use [limited templates](#) for times.

```
blueprint:
  input:
    alarm:
      name: Alarm
      selector:
        text:
    hour:
      name: Hour
      selector:
        number:
          min: 0
          max: 24

  trigger_variables:
    my_alarm: !input alarm
    my_hour: !input hour
  trigger:
    - platform: time
      at:
        - "sensor.{{ my_alarm | slugify }}_time"
        - "{{ my_hour }}:30:00"
```

Weekday filtering

Time triggers can be filtered to fire only on specific days of the week using the `weekday` option. This allows you to create automations that only run on certain days, such as weekdays or weekends.

The `weekday` option accepts: - A single weekday as a string: `"mon"`, `"tue"`, `"wed"`, `"thu"`, `"fri"`, `"sat"`, `"sun"` - A list of weekdays using the expanded format

Single weekday

This example will turn on the lights only on Mondays at 8:00 AM:

```
automation:
  - triggers:
    - trigger: time
      at: "08:00:00"
      weekday: "mon"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.bedroom
```

Multiple weekdays

This example will run a morning routine only on weekdays (Monday through Friday) at 6:30 AM:

```

automation:
  - triggers:
    - trigger: time
      at: "06:30:00"
      weekday:
        - "mon"
        - "tue"
        - "wed"
        - "thu"
        - "fri"
    actions:
      - action: script.morning_routine

```

Weekend example

This example demonstrates a different wake-up time for weekends:

```

automation:
  - alias: "Weekday alarm"
    triggers:
      - trigger: time
        at: "06:30:00"
        weekday:
          - "mon"
          - "tue"
          - "wed"
          - "thu"
          - "fri"
    actions:
      - action: script.weekday_morning

  - alias: "Weekend alarm"
    triggers:
      - trigger: time
        at: "08:00:00"
        weekday:
          - "sat"
          - "sun"
    actions:
      - action: script.weekend_morning

```

Combined with input datetime

The `weekday` option works with all time formats, including input datetime entities:

```
automation:
  - triggers:
    - trigger: time
      at: input_datetime.work_start_time
      weekday:
        - "mon"
        - "tue"
        - "wed"
        - "thu"
        - "fri"
  actions:
    - action: notify.mobile_app
      data:
        title: "Work Day!"
        message: "Time to start working"
```


Time pattern trigger

With the time pattern trigger, you can match if the hour, minute or second of the current time matches a specific value. You can prefix the value with a `/` to match whenever the value is divisible by that number. You can specify `*` to match any value.

```
automation:
  triggers:
    - trigger: time_pattern
      # Matches every hour at 5 minutes past whole
      minutes: 5

automation 2:
  triggers:
    - trigger: time_pattern
      # Trigger once per minute during the hour of 3
      hours: "3"
      minutes: "*"

automation 3:
  triggers:
    - trigger: time_pattern
      # You can also match on interval. This will match every 5 minutes
      minutes: "/5"
```

 **Note:** Do not prefix numbers with a zero - using `'01'` instead of `'1'` for example will result in errors.

Persistent notification trigger

Persistent notification triggers are fired when a `persistent_notification` is `added` or `removed` that matches the configuration options.

```
automation:
  triggers:
    - trigger: persistent_notification
      update_type:
        - added
        - removed
      notification_id: invalid_config
```

See the [Persistent Notification](#) integration for more details on event triggers and the additional event data available for use by an automation.

Webhook trigger

Webhook trigger fires when a web request is made to the webhook endpoint: `/api/webhook/<webhook_id>`. The webhook endpoint is created automatically when you set it as the `webhook_id` in an automation trigger. The `webhook_id` can either be a static value or computed using [limited templates](#).

 **Note:** The `webhook_id` template is only evaluated when setting up the trigger, they will not be re-evaluated for incoming webhook triggers.

```
automation:
  trigger_variables:
    webhook_id_variable: "template_webhook_id"
  triggers:
    - trigger: webhook
      webhook_id: "some_hook_id"
      allowed_methods:
        - POST
        - PUT
      local_only: true
    - trigger: webhook
      webhook_id: "{{ webhook_id_variable }}"
      allowed_methods:
        - POST
```

You can run this automation by sending an HTTP POST request to `http://your-home-assistant:8123/api/webhook/some_hook_id`. Here is an example using the `curl` command line program, with an example form data payload:

```
curl -X POST -d 'key=value&key2=value2' https://your-home-assistant:8123/api/web
```

Webhooks support HTTP POST, PUT, HEAD, and GET requests; PUT requests are recommended. HTTP GET and HEAD requests are not enabled by default but can be enabled by adding them to the `allowed_methods` option. The request methods can also be configured in the UI by selecting the settings gear menu button beside the Webhook ID.

By default, webhook triggers can only be accessed from devices on the same network as Home Assistant or via [Nabu Casa Cloud webhooks](#). The `local_only` option should be set to `false` to allow webhooks to be triggered directly via the internet. This option can also be configured in the UI by selecting the settings gear menu button beside the Webhook ID.

Remember to use an HTTPS URL if you've secured your Home Assistant installation with SSL/TLS.

Note that a given webhook can only be used in one automation at a time. That is, only one automation trigger can use a specific webhook ID.

Webhook data

Payloads may either be encoded as form data or JSON. Depending on that, its data will be available in an automation template as either `trigger.data` or `trigger.json`. URL query parameters are also available in the template as `trigger.query`.

Note that to use JSON encoded payloads, the `Content-Type` header must be set to `application/json`, for example:

```
curl -X POST -H "Content-Type: application/json" -d '{ "key": "value" }' https://
```

Webhook security

Webhook endpoints don't require authentication, other than knowing a valid webhook ID. Security best practices for webhooks include:

- Do not use webhooks to trigger automations that are destructive, or that can create safety issues. For example, do not use a webhook to unlock a lock, or open a garage door.
- Treat a webhook ID like a password: use a unique, non-guessable value, and keep it secret.
- Do not copy-and-paste webhook IDs from public sources, including blueprints. Always create your own.
- Keep the `local_only` option enabled for webhooks if access from the internet is not required.

Zone trigger

Zone trigger fires when an entity is entering or leaving the zone. The entity can be either a person, or a device_tracker. For zone automation to work, you need to have setup a device tracker platform that supports reporting GPS coordinates. This includes [GPS Logger](#), the [OwnTracks platform](#) and the [iCloud platform](#).

```
automation:
  triggers:
    - trigger: zone
      entity_id: person.paulus
      zone: zone.home
      # Event is either enter or leave
      event: enter # or "leave"
```

Geolocation trigger

Geolocation trigger fires when an entity is appearing in or disappearing from a zone. Entities that are created by a [Geolocation](#) platform support reporting GPS coordinates. Because entities are generated and removed by these platforms automatically, the entity ID normally cannot be predicted. Instead, this trigger requires the definition of a `source`, which is directly linked to one of the Geolocation platforms.

 **Tip:** This isn't for use with `device_tracker` entities. For those look above at the `zone` trigger.

```
automation:
  triggers:
    - trigger: geo_location
      source: nsw_rural_fire_service_feed
      zone: zone.bushfire_alert_zone
      # Event is either enter or leave
      event: enter # or "leave"
```

Device triggers

Device triggers encompass a set of events that are defined by an integration. This includes, for example, state changes of sensors as well as button events from remotes. [MQTT device triggers](#) are set up through autodiscovery.

In contrast to state triggers, device triggers are tied to a device and not necessarily an entity. To use a device trigger, set up an automation through the browser frontend. If you would like to use a device trigger for an automation that is not managed through the browser frontend, you can copy the YAML from the trigger widget in the frontend and paste it into your automation's trigger list.

Calendar trigger

Calendar trigger fires when a [Calendar](#) event starts or ends, allowing for much more flexible automations than using the Calendar entity state which only supports a single event start at a time.

An optional time offset can be given to have it fire a set time before or after the calendar event (such as 5 minutes before event start).

```
automation:
  triggers:
    - trigger: calendar
      # Possible values: start, end
      event: start
      # The calendar entity_id
      entity_id: calendar.light_schedule
      # Optional time offset
      offset: "-00:05:00"
```

See the [Calendar](#) integration for more details on event triggers and the additional event data available for use by an automation.

Sentence trigger

A sentence trigger fires when [Assist](#) matches a sentence from a voice assistant using the default [conversation agent](#). Sentence triggers work with Home Assistant Assist. They will not work with external conversation agents such as OpenAI or Google Generative AI unless "Prefer handling commands locally" is enabled in the conversation agent settings.

Sentences are allowed to use some basic [template syntax](#) like optional and alternative words. For example, `[it's ]party time` will match both "party time" and "it's party time".

```
automation:
  triggers:
    - trigger: conversation
      command:
        - "[it's ]party time"
        - "happy (new year|birthday)"
```

The sentences matched by this trigger will be:

- party time
- it's party time
- happy new year
- happy birthday

Punctuation and casing are ignored, so "It's PARTY TIME!!!" will also match.

Related topic

- [Adding a custom sentence to trigger an automation](#)

Sentence wildcards

Adding one or more `{lists}` to your trigger sentences will capture any text at that point in the sentence. A `slots` object will be [available in the trigger data](#). This allows you to match sentences with variable parts, such as album/artist names or a description of a picture.

For example, the sentence `play {album} by {artist}` will match "play the white album by the beatles" and have the following variables available in the action templates:

- `` - "the white album"
- `` - "the beatles"

Wildcards will match as much text as possible, which may lead to surprises: "play day by day by taken by trees" will match `album` as "day" and `artist` as "day by taken by trees". Including extra words in your template can help: `play {album} by artist {artist}` can now correctly match "play day by day by artist taken by trees".

Multiple triggers

It is possible to specify multiple triggers for the same rule. To do so just prefix the first line of each trigger with a dash (-) and indent the next lines accordingly. Whenever one of the triggers fires, processing of your automation rule begins.

```
automation:
  triggers:
    # first trigger
    - trigger: time_pattern
      minutes: 5
    # our second trigger is the sunset
    - trigger: sun
      event: sunset
```

Multiple entity IDs for the same trigger

It is possible to specify multiple entities for the same trigger. To do so add multiple entities using a nested list. The trigger will fire and start, processing your automation each time the trigger is true for any entity listed.

```
automation:
  triggers:
    - trigger: state
      entity_id:
        - sensor.one
        - sensor.two
        - sensor.three
```

Disabling a trigger

Every individual trigger in an automation can be disabled, without removing it. To do so, add `enabled: false` to the trigger. For example:

```
# Example script with a disabled trigger
automation:
  triggers:
    # This trigger will not trigger, as it is disabled.
    # This automation does not run when the sun is set.
    - enabled: false
      trigger: sun
      event: sunset

    # This trigger will fire, as it is not disabled.
    - trigger: time
      at: "15:32:00"
```

Triggers can also be disabled based on limited templates or blueprint inputs. These are only evaluated once when the automation is loaded.

```
blueprint:
  input:
    input_boolean:
      name: Boolean
      selector:
        boolean:
    input_number:
      name: Number
      selector:
        number:
          min: 0
          max: 100

  trigger_variables:
    _enable_number: !input input_number

  triggers:
    - trigger: sun
      event_type: sunrise
      enabled: !input input_boolean
    - trigger: sun
      event_type: sunset
      enabled: "{{ _enable_number < 50 }}"
```

Merging lists of triggers

◆ **Caution:** This feature requires Home Assistant version 2024.10 or later. If using this in a blueprint, set the `min_version` for the blueprint to at least this version. See the [blueprint schema documentation](#) for more details.

In some advanced cases (like for blueprints with trigger selectors), it may be necessary to insert a second list of triggers into the main trigger list. This can be done by adding a dictionary in the main trigger list with the sole key `triggers`, and the value for that key contains a second list of triggers. These will then be flattened into a single list of triggers. For example:

```
blueprint:
  name: Nested Trigger Blueprint
  domain: automation
  input:
    usertrigger:
      selector:
        trigger:

  triggers:
    - trigger: event
      event_type: manual_event
    - triggers: !input usertrigger
```

This blueprint automation can then be triggered either by the fixed `manual_event` trigger, or additionally by any triggers selected in the trigger selector. This is also applicable for `wait_for_trigger` action.

Docs/Automation/Troubleshooting

Sometimes an automation does not do what you expect. Maybe it does not run at all, maybe it runs at the wrong moment, or maybe one of the actions in the middle quietly fails. Home Assistant has built-in tools to help you find out exactly what happened, without having to dig through log files.

The most useful tool is the **trace**. Every time an automation runs, Home Assistant records a step-by-step timeline of what was triggered, which conditions were checked, and what each action did. You can also test parts of an automation directly from the editor, without waiting for a real trigger.

Testing your automation

Many automations can be tested directly in the automation editor UI.

Running the entire automation

In the three dots menu in the automation list or automation editor UI, select the **Run actions** button. This will execute all the actions, while skipping all triggers and conditions. This lets you test the full sequence of actions, as if the automation was triggered and all conditions were true. Note that any **trigger ID** used in your triggers will not be active when you test this way. The Trigger ID or any data passed by in the `trigger` data in conditions or actions can't be tested directly this way.

You can also trigger an automation manually. This can test the conditions as if the automation was triggered by an event. Go to **Settings > Developer tools > Actions**. In the **Action** drop-down, select **Automation: Trigger**, then **Choose entity** to select the automation you are testing. Toggle whether to skip the conditions, then **Perform action**. If needed, additional `trigger` or other data can be added in the YAML view for testing. The **trigger** page has more information about data within the trigger.

Testing with complex triggers, conditions, and variables can be difficult. Note that using the **Run actions** button will skip all triggers and conditions, while **Developer tools** can be used with or without checking conditions.

Running individual actions or conditions

In the automation editor UI, each condition and action can be tested individually. Select the three dots `mdi:dots-vertical` menu, then the **Test** button.

- Testing a condition will highlight it to show whether the condition passed at the moment it was tested. If all conditions pass, then the automation will run when triggered. Testing building blocks like an **and** condition will report whether the whole block registers as true or false, or you can test individual conditions within the building block.
- Testing an action block will run that block immediately.

Note that complex automations that depend on previous blocks, such as trigger IDs, variables in templates, or action calls that return data to use in subsequent blocks, cannot be tested this way.

If you are writing automations in YAML, it is also useful to go to **Developer tools > YAML** and in the Configuration validation section, select the **Check configuration** button. This is to make sure there are no syntax errors before restarting Home Assistant. In order for **Check configuration** to be visible, you must enable **Advanced Mode** on your user profile.

Traces

When an automation is run, all steps are recorded and a trace is made. To open the automation editor, go to **Settings > Automations & scenes**.

From the automation editor UI, or in the automations list in the three dots menu, select **Traces**. Alternatively, select an automation entry shown under **Activity**.

Automation tracing example

The above screenshot shows a previous run of an automation. The automation is displayed using an interactive graph, highlighting which path the automation took. Each node in the graph can be selected to view the details on what happened with the automation during that specific step. It traces the complete run of an automation.

The right side of the trace screen has tabs with more information:

- **Step Details** shows data and results of the step that is currently highlighted.
- **Automation Config** shows the full YAML configuration at the time the automation was run.
- **Trace Timeline**, shown in the screenshot above, lists the steps that were executed and their timing.
- **Related activity**, shows the activity for all the entries related to the specific trace.
- **Blueprint Config** will only be shown if the automation was created from a blueprint.

The top bar shows the date and time the automation was triggered. Use the left and right arrows to view previous runs of the automation.

Automations created in YAML must have an `id` assigned in order for debugging traces to be stored.

Trace configuration

The last 5 traces are recorded for all automations. It is possible to change this by adding the following code to your automation.

```
trace:
  stored_traces: 20
```


Testing templates

If your automation uses [templates](#) in any part, you can do the following to make sure it works as expected:

1. Go to **Settings > Developer tools > Template** tab.
 2. Create all variables (sources) required for your template as described at the end of [this](#) paragraph.
 3. Copy your template code and paste it in Template editor straight after your variables.
 4. If necessary, change your sources' value and check if the template works as you want and does not generate any errors.
-

Docs/Automation/Using Blueprints

Blueprints are the easiest way to add an automation to your Home Assistant. They are ready-made automations shared by the Home Assistant community: someone else has already done the thinking and the writing, and you just plug in your own devices.

You can find blueprints for almost any common use case, from "turn the lights on when motion is detected" to "announce the weather every morning at 7" to "notify me when the laundry is done". You install a blueprint once and can use it as many times as you like, with different devices and settings each time.

Quick links:

- [Blueprints in the Home Assistant forums](#)

Blueprint automations

Automations based on a blueprint need to be configured. What needs to be configured differs by blueprint.

1. To create your first automation based on a blueprint, go to **Settings > Automations & scenes > Blueprints**.
2. Find the blueprint that you want to use and select **Create automation**.
3. This opens the automation editor with the blueprint selected.
4. Give it a name and configure the blueprint.
5. Select the blue **Save automation** button in the bottom right corner.

Done! If you want to revisit the configuration values, go to **Settings > Automations & scenes > Blueprints**.

Importing blueprints

Home Assistant can import blueprints from the Home Assistant forums, GitHub, and GitHub gists.

1. To import a blueprint, first [find a blueprint you want to import](#).
2. If you just want to practice importing, you can use this URL:

```
text https://github.com/home-assistant/core/blob/dev/homeassistant/components/automation/blueprints/motion_light.yaml
```

3. Go to **Settings > Automations & scenes > Blueprints**.
4. Select the blue **blueprint_import** button in the bottom right.
5. A new dialog will pop-up asking you for the URL.
6. Enter the URL and select **Preview**.
7. This will load the blueprint and show a preview in the import dialog.
8. You can change the name and finish the import.

The blueprint can now be used for creating automations.

Editing an imported blueprint

You can tweak an imported blueprint by "taking control" of this blueprint. Home Assistant then converts the blueprint automation into a regular automation, allowing you to make any tweak without having to fully re-invent the wheel.

To edit an imported blueprint, follow these steps:

1. Go to **Settings > Automations & scenes > Blueprints**.
2. Select the blueprint from the list.
3. Select the `mdi:dots-vertical` and select **Take control**.
4. A preview of the automation is shown.
5. **Info:** By taking control, the blueprint is converted into an automation. You won't be able to convert this back into a blueprint.
6. To convert it into an automation and take control, select **Yes**.
7. If you change your mind and want to keep the blueprint, select **No**.

Screencast showing how to take control of a blueprint

Re-importing a blueprint

Blueprints created by the community may go through multiple revisions. Sometimes a user creates a blueprint, the community provides feedback, new functionality is added.

The quickest way to get these changes, is by re-importing the blueprint. This will overwrite the blueprint you currently have.

◆ **Caution: Before you do this:** If the re-imported blueprint is not compatible, it can break your automations.

- In this case, you will need to manually adjust your automations.

To re-import a blueprint

1. Go to **Settings > Automations & scenes > Blueprints**.
2. On the blueprint that you want to re-import, select the three dots `mdi:dots-vertical` menu, and select **Re-import blueprint**.

Updating an imported blueprint in YAML

Blueprints created by the community may go through multiple revisions. Sometimes a user creates a blueprint, the community provides feedback, new functionality is added.

If you do not want to [re-import the blueprint](#) for some reason, you can manually edit its YAML content to keep it up to date:

1. Go to the blueprints directory (`blueprints/automation/`). The location of this directory depends on the installation type. It's similar to how you find `configuration.yaml` .
2. Next, you must find the blueprint to update. The path name of a blueprint consists of:
3. The username of the user that created it. The name depends on the source of the blueprint: the forum, or GitHub.
4. The name of the YAML file. For the forum it's the title of the topic in the URL, for GitHub it's the name of the YAML file.
5. Open the YAML file with your editor and update its contents.
6. Reload the automations for the changes to take effect.

The new changes will appear to your existing automations as well.

Finding new blueprints

The Home Assistant Community forums have a specific tag for blueprints. This tag is used to collect all blueprints.

[Visit the Home Assistant forums](#)

Creating new blueprints

Using blueprints is nice and easy, but what if you could create that one missing blueprint that our community definitely needs?

Learn more about blueprints by [reading our tutorial on creating a blueprint](#).

Troubleshooting missing automations

When you're creating automations using blueprints and they don't appear in the UI, make sure that you add back `automation: !include automations.yaml` from the default configuration to your `configuration.yaml` .

Docs/Automation/Yaml

Automations are created in Home Assistant via the UI, but are stored in a YAML format. If you want to edit the YAML of an automation, select the automation, select the menu button in the top right then on **Edit in YAML**.

The UI will write your automations to `automations.yaml`. This file is managed by the UI and should not be edited manually.

It is also possible to write your automations directly inside `configuration.yaml` or other YAML files. You can do this by adding a labeled `automation` block to your `configuration.yaml`:

```
# The configuration required for the UI to work
automation: !include automations.yaml

# Labeled automation block
automation kitchen:
  - triggers:
    - trigger: ...
```

You can add as many labeled `automation` blocks as you want.

Option	Type	Required	Default	Description
<code>alias</code>	string	No		Friendly name for the automation.
<code>id</code>	string	No		A unique id for your automation, will allow you to make changes to the name and <code>entity_id</code> in the UI, and will enable debug traces.
<code>description</code>	string	No	"	A description of the automation.
<code>initial_state</code>	boolean	No	Restored from last run	Used to define the state of your automation at startup. When not set, the state will be restored from the last run. See Automation initial state .
<code>trace</code>	integer	No	5	The number of traces which will be stored. See Number of debug traces stored .
<code>variables</code>	any	No	{}	The value of the variable. Any YAML is valid. Templates can also be used to pass a value to the variable.
<code>trigger_variables</code>	any	No	{}	The value of the variable. Any YAML is valid. Only limited templates can be used.
<code>mode</code>	string	No	single	Controls what happens when the automation is invoked while it is still running from one or more previous invocations. See Automation modes .
<code>max</code>	integer	No	10	Controls maximum number of runs executing and/or queued up to run at a time. Only valid with modes <code>queued</code> and <code>parallel</code> .
<code>max_exceeded</code>	string	No	warning	When <code>max</code> is exceeded (which is effectively 1 for <code>single</code> mode) a log message will be emitted to indicate this has happened. This option controls the severity level of that log message. See Log Levels for a list of valid options. Or <code>silent</code> may be specified to suppress the message from being emitted.
<code>triggers</code>	any	No	{}	The value of the variable. Any YAML is valid. Templates can also be used to pass a value to the variable.
<code>conditions</code>	list	No		Conditions that have to be <code>true</code> to start the automation. By default all conditions listed have to be <code>true</code> ,

Option	Type	Required	Default	Description
				you can use logical conditions to change this default behavior.
<code>actions</code>	list	Yes		The sequence of actions to be performed in the script.

Automation modes

Mode	Description
<code>single</code>	Do not start a new run. Issue a warning.
<code>restart</code>	Start a new run after first stopping previous run.
<code>queued</code>	Start a new run after all previous runs complete. Runs are guaranteed to execute in the order they were queued.
<code>parallel</code>	Start a new, independent run in parallel with previous runs.

YAML example

Example of a YAML based automation that you can add to `configuration.yaml`.

```

# Example of entry in configuration.yaml
automation my_lights:
  # Turns on lights 1 hour before sunset if people are home
  # and if people get home between 16:00-23:00
  - alias: "Rule 1 Light on in the evening"
    triggers:
      # Prefix the first line of each trigger configuration
      # with a '-' to enter multiple
      - trigger: sun
        event: sunset
        offset: "-01:00:00"
      - trigger: state
        entity_id: all
        to: "home"
    conditions:
      # Prefix the first line of each condition configuration
      # with a '-' to enter multiple
      - condition: state
        entity_id: all
        state: "home"
      - condition: time
        after: "16:00:00"
        before: "23:00:00"
    actions:
      # With a single action entry, we don't need a '-' before action - though y
      - action: homeassistant.turn_on
        target:
          entity_id: group.living_room

# Turn off lights when everybody leaves the house
- alias: "Rule 2 - Away Mode"
  triggers:
    - trigger: state
      entity_id: all
      to: "not_home"
  actions:
    - action: light.turn_off
      target:
        entity_id: all

# Notify when Paulus leaves the house in the evening
- alias: "Leave Home notification"
  triggers:
    - trigger: zone
      event: leave
      zone: zone.home
      entity_id: device_tracker.paulus
  conditions:
    - condition: time
      after: "20:00"
  actions:
    - action: notify.notify
      data:
        message: "Paulus left the house"

# Send a notification via Pushover with the event of a Xiaomi cube. Custom eve
- alias: "Xiaomi Cube Action"
  initial_state: false
  triggers:
    - trigger: event

```

```
    event_type: cube_action
    event_data:
      entity_id: binary_sensor.cube_158d000103a3de
actions:
- action: notify.pushover
  data:
    title: "Cube event detected"
    message: "Cube has triggered this event: {{ trigger.event }}"
```


Extra options

When writing automations directly in YAML, you will have access to advanced options that are not available in the user interface.

Automation initial state

At startup, automations by default restore their last state of when Home Assistant ran. This can be controlled with the `initial_state` option. Set it to `false` or `true` to force initial state to be off or on.

```
automation:
  - alias: "Automation Name"
    initial_state: false
    triggers:
      - trigger: ...
```

Number of debug traces stored

When using YAML you can configure the number of debugging traces stored for an automation. This is controlled with the `stored_traces` option under `trace`. Set `stored_traces` to the number of traces you wish to store for the particular automation. If not specified the default value of 5 will be used.

```
automation:
  - alias: "Automation Name"
    trace:
      stored_traces: 10
    triggers:
      - trigger: ...
```

Migrating your YAML automations to `automations.yaml`

If you want to migrate your manual automations to use the editor, you'll have to copy them to `automations.yaml`. Make sure that `automations.yaml` remains a list! For each automation that you copy over, you'll have to add an `id`. This can be any string as long as it's unique.

```
# Example automations.yaml entry. Note, automations.yaml is always a list!
- id: my_unique_id # <-- Required for editor to work, for automations created w
  alias: "Hello world"
  triggers:
    - trigger: state
      entity_id: sun.sun
      from: below_horizon
      to: above_horizon
  conditions:
    - condition: numeric_state
      entity_id: sensor.temperature
      above: 17
      below: 25
      value_template: "{{ float(state.state) + 2 }}"
  actions:
    - action: light.turn_on
```

Deleting automations

When automations remain visible in the Home Assistant dashboard, even after having deleted in the YAML file, you have to delete them in the UI.

To delete them completely, go to UI **Settings** > **Devices & services** > **Entities** and find the automation in the search field or by scrolling down.

Check the square box aside of the automation you wish to delete and from the top-right of your screen, select 'REMOVE SELECTED'.

Docs/Automation

Automations are how you make your home work for you. They let Home Assistant automatically respond to things that happen, such as turning the lights on at sunset or pausing the music when you receive a call.

You build automations in Home Assistant with the visual automation editor, so no coding is required. Home Assistant already knows about all your devices and services, so you can pick from them directly when you decide what should trigger an automation and what should happen as a result.

If you are just starting out, we recommend that you start with blueprint automations. These are ready-made automations from the community that you only need to configure.

[Learn about automation blueprints »](#)

If you have got the hang of blueprints and would like to explore more, it's time for the next step. But before you start creating automations, you will need to learn about the automation basics.

[Learn about automation basics »](#)

The blueprint schema

Blueprint schemas currently supports three types of schema depending on its domain: `automation`; `script`; and `template`.

The configuration schema of a blueprint consists of 2 parts:

1. The blueprint's high-level metadata: name, domain and, optionally, any input required from the user.
2. The schema for the blueprint domain it describes.

The first part is referred to as the *blueprint schema*. It contains the blueprint's metadata.

Minimum required metadata for a blueprint is its name and domain. In its most basic form, a blueprint looks like:

```
blueprint:
  name: Example blueprint
  domain: automation
```

Although this is a valid blueprint, it is not very useful.

The second part depends on its domain, the type of blueprint. For example, when creating a blueprint for an automation, the full schema for an [automation](#) applies.

This is the full blueprint schema:

Option	Type	Required	Default	Description
<code>name</code>	string	Yes		The name of the blueprint. Keep this short and descriptive.
<code>description</code>	string	No		>
<code>domain</code>	string	Yes		>
<code>author</code>	string	No		The name of the blueprint author.
<code>homeassistant</code>	string	No		>
<code>input</code>	map	No		>

Blueprint inputs

A blueprint can accept one or multiple inputs from the user, but does not require any input.

These inputs can be of any type (string, boolean, list, map). They can have a default value and provide a [selector](#) that ensures a matching input field in the user interface.

A blueprint input has the following configuration:

Option	Type	Required	Default	Description
<code>name</code>	any	No		>

Each input field can be referred to, outside of the blueprint metadata, using the `!input` custom YAML tag before its name.

The following example shows a minimal *blueprint schema* with a single input:

```
blueprint:
  name: Example blueprint
  description: Example showing an input
  domain: automation
  input:
    my_input:
      name: Example input
```

In the above example, `my_input` is the identifier of the input. It can be referenced by using the `!input my_input` custom tag.

In this example, no `selector` was provided. In the user interface, a text input field would be shown to the user. It is then up to the user to find out what to enter there. Blueprints that come with [selectors](#) are easier to use.

A blueprint can have as many inputs as you like.

Blueprint input sections

One or more input sections can be added under the main `input` key. Each section visually groups the inputs in that section, allows an optional description, and optionally allows for collapsing those inputs. Note that the section only impacts how inputs are displayed to the user when they fill in the blueprint. Inputs must have unique names and be referenced directly by their name; not by section and name.

A section is differentiated from an input by the presence of an additional `input` key within that section.

 **Caution:** Input sections are a new feature in version 2024.6.0. Set the `min_version` for the blueprint to at least this version if using input sections. Otherwise, the blueprint will generate errors on older versions. See [this section](#) for more details.

The full configuration for an input section is below:

Option	Type	Required	Default	Description
<code>name</code>	string	No		A name for the section. If omitted the key of the section is used.
<code>icon</code>	string	No		An icon to display next to the name of the section.
<code>description</code>	string	No		>
<code>collapsed</code>	boolean	No	false	If <code>true</code> , the section will be collapsed by default. Useful for optional or less important inputs. All collapsed inputs must also have a defined <code>default</code> before they can be hidden.
<code>input</code>	map	Yes		>

The following example shows a *blueprint schema* with some inputs in a section:

```

blueprint:
  name: Example sections blueprint
  description: Example showing a section
  input:
    base_input:
      name: An input not in the section
    my_section:
      name: My Section
      icon: mdi:cog
      description: These options control a specific feature of this blueprint
      input:
        my_input:
          name: Example input
        my_input_2:
          name: 2nd example input

```

Blueprint inputs in templates

The inputs are available as custom YAML tags, but not as template variables. To use a blueprint input in a template, it first needs to be exposed as either a [script level variable](#) or in a [variable script step](#).

```

variables:
  # Make input my_input available as a script level variable
  my_input: !input my_input

```

Example blueprints

The [built-in blueprints](#) are great examples to get a bit of a feeling of how blueprints work.

Here is the built-in motion light automation blueprint. Note the *blueprint schema* under the blueprint key is followed by its domain schema. In this example, an automation schema.

```

blueprint:
  name: Motion-activated Light
  description: Turn on a light when motion is detected.
  domain: automation
  input:
    motion_entity:
      name: Motion Sensor
      selector:
        entity:
          filter:
            device_class: motion
            domain: binary_sensor
    light_target:
      name: Light
      selector:
        target:
          entity:
            domain: light
    no_motion_wait:
      name: Wait time
      description: Time to leave the light on after last motion is detected.
      default: 120
      selector:
        number:
          min: 0
          max: 3600
          unit_of_measurement: seconds

# If motion is detected within the delay,
# we restart the script.
mode: restart
max_exceeded: silent

triggers:
  - trigger: state
    entity_id: !input motion_entity
    from: "off"
    to: "on"

actions:
  - action: light.turn_on
    target: !input light_target
  - wait_for_trigger:
    - trigger: state
      entity_id: !input motion_entity
      from: "on"
      to: "off"
  - delay: !input no_motion_wait
  - action: light.turn_off
    target: !input light_target

```


Docs/Blueprint/Selectors

Selectors can be used to specify what values are accepted for a blueprint input. The selector also defines how the input is shown in the user interface.

Some selectors can, for example, show a toggle button to turn something on or off, while another select can filter a list of devices to show only devices that have motion-sensing capabilities.

Having good selectors set on your blueprint automation inputs makes a blueprint easier to use from the UI.

The following selectors are currently available:

- [Action selector](#)
- [App selector](#)
- [Area selector](#)
- [Attribute selector](#)
- [Assist pipeline selector](#)
- [Backup location selector](#)
- [Boolean selector](#)
- [Choose selector](#)
- [Color temperature selector](#)
- [Condition selector](#)
- [Config entry selector](#)
- [Constant selector](#)
- [Conversation agent selector](#)
- [Country selector](#)
- [Date selector](#)
- [Date \& time selector](#)
- [Device selector](#)
- [Duration selector](#)
- [Entity selector](#)
- [Floor selector](#)
- [Icon selector](#)
- [Label selector](#)
- [Language selector](#)
- [Location selector](#)
- [Media selector](#)
- [Number selector](#)
- [Object selector](#)
- [QR code selector](#)
- [RGB color selector](#)
- [Select selector](#)

- [State selector](#)
- [Statistic selector](#)
- [Target selector](#)
- [Template selector](#)
- [Text selector](#)
- [Theme selector](#)
- [Time selector](#)
- [Trigger selector](#)

Interactive demos of each of these selectors can be found on the [Home Assistant Design portal](#).

If no selector is defined, a text input for a single line will be shown.

Action selector

The action selector allows you to input one or more sequences of actions. On the user interface, the action part of the automation editor will be shown. The value of the input will contain a list of actions to perform.

Screenshot of an action selector

This selector does not have any other options; therefore, it only has its key.

```
action:
```

The output of this selector is a list of actions. For example:

```
# Example action selector output result
- action: scene.turn_on
  target:
    entity_id: scene.watching_movies
  metadata: {}
```

App selector

This can only be used on a Home Assistant Operating System installation. For Home Assistant Container installations, an error will be displayed.

The app selector (formerly known as add-on selector) allows you to input an app slug. On the user interface, it will list all installed Home Assistant apps and use the slug of the selected app.

Screenshot of an app selector

This selector does not have any other options; therefore, it only has its key.

```
# Example app selector
app:
```

The output of this selector is the slug of the selected app. For example: `core_ssh`.

Area selector

The area selector shows an area finder that can pick a single or multiple areas based on the selector configuration. The value of the input will be the area ID, or a list of area IDs, based on if `multiple` is set to `true`.

An area selector can filter the list of areas, based on properties of the devices and entities that are assigned to those areas. For example, the areas list could be limited to areas with entities provided by the [ZHA](#) integration.

In its most basic form, this selector doesn't require any options, which will show all areas.

Screenshot of an area selector

```
area:
```

{% configuration area %} device: description: > When device options are provided, the list of areas is filtered by areas that at least provide one device that matches the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of areas that provide devices by the set integration domain, for example, `zha`. type: string required: false manufacturer: description: > When set, it limits the list of areas that provide devices by the set manufacturer name. type: string required: false model: description: > When set, it limits the list of areas that provide devices that have the set model. type: string required: false model_id: description: > When set, the list of areas is limited to areas with devices that have the set model ID. type: string required: false entity: description: > When entity options are provided, the list of areas is filtered by areas that at least provide one entity that matches the given conditions. Can be either a object or a list of object. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of areas that provide entities by the set integration domain, for example, `zha`. type: string required: false domain: description: > Limits the list of areas that provide entities of a certain domain(s), for example, `light` or `binary_sensor`. Can be either a string with a single domain, or a list of string domains to limit the selection to. type: [string, list] required: false device_class: description: > Limits the list of areas to areas that have entities with a certain device class(es), for example, `motion` or `window`. Can be either a string with a single device_class, or a list of string device_class to limit the selection to. type: [device_class, list] required: false supported_features: description: > Limits the list of areas to areas that have entities with a certain supported feature, for example, `light.LightEntityFeature.TRANSITION` or `climate.ClimateEntityFeature.TARGET_TEMPERATURE`. Should be a list of features. For a list of supported features for each entity type, refer to the [entity documentation](#). type: list required: false multiple: description: > Allows selecting multiple areas. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false required: false reorder: description: > Allows reordering of areas (only applies if `multiple` is set to `true`). type: boolean default: false required: false

The output of this selector is the area ID, or (in case `multiple` is set to `true`) a list of area IDs.

```
# Example area selector output result, when multiple is set to false
living_room

# Example area selector output result, when multiple is set to true
- living_room
- kitchen
```

Example area selectors

An example area selector only shows areas that provide one or more lights or switches provided by the [ZHA](#) integration.

```
area:
  entity:
    integration: zha
    domain:
      - light
      - switch
```

Another example uses the area selector, which only shows areas that provide one or more remote controls provided by the [deCONZ](#) integration. Multiple areas can be selected.

```
area:
  multiple: true
  device:
    - integration: deconz
      manufacturer: IKEA of Sweden
      model: TRADFRI remote control
```

Attribute selector

The attributes selector shows a list of state attributes from a provided entity of which one can be selected.

This allows for selecting, for example, the "Effect" attribute from a light entity, or the "Next dawn" attribute from the `sun` entity.

Screenshot of an attribute selector

{% configuration attribute %} entity_id: description: The entity ID of which a state attribute can be selected from. type: string required: true

The output of this selector is the selected attribute key (not the translated or prettified name shown in the frontend). For example: `next_dawn`.

Assist pipeline selector

The assist pipeline selector shows all available assist pipelines (assistants) of which one can be selected.

Screenshot of an assist pipeline selector

This selector does not have any other options; therefore, it only has its key.

```
assist_pipeline:
```


Backup location selector

This can only be used on an installation with a Home Assistant Operating System. For Home Assistant Container installations, an error

will be displayed.

The backup location selector shows a list of places a backup could go, depending on what you have configured in [storage](#).

Screenshot of an assist pipeline selector

The output of this selector is the name of the selected network storage. It may also be the value `/backup`, if the user chooses to use the local data disk option instead of one of the configured network storage locations.

```
backup_location:
```

Boolean selector

The boolean selector shows a toggle that allows you to turn on or off the selected option.

Screenshot of a boolean selector

The boolean selector is suitable for adding feature switches to, for example, blueprints.

This selector does not have any other options; therefore, it only has its key.

```
boolean:
```

The output of this selector is `true` when the toggle is on, `false` otherwise.

Choose selector

The choose selector allows you to present multiple selectors to the user for a single field, and the user can pick a desired selector and enter a value using that selection.

Screenshot of a choose selector

```
choose:
```

{% configuration choose %} choices: description: > A dictionary of choices for the choose option. Each key in the dictionary represents one selector choice, and the string value of the key will be displayed in the selector picker. Each entry is itself another dictionary, with one mandatory key of "selector", and the value of that key is any other valid selector definition. type: map required: true

The output of a choose selector is an object with one key called 'active_choice' representing which selector was chosen, and one key per selector choice representing the value entered for that choice.

Example choose selector

An example choose selector that allows user to either select an icon from a dropdown, or enter an arbitrary template.

```
choose:
  choices:
    Icon:
      selector:
        icon: {}
    Template:
      selector:
        template: {}
```

Following this example, if the user entered a value in both selectors, but submitted with 'Icon' option selected, the output might be:

```
active_choice: Icon
Icon: mdi:light
Template: "{{ something else }}"
```

Color temperature selector

The color temperature selector allows you to select a color temperature from a gradient using a slider.

Screenshot of the Color temperature selector

```
color_temp:
```

{% configuration color_temp %} unit: description: The chosen unit for the color temperature. This can be either `kelvin` or `mired`. `mired` is the default for historical reasons. type: string required: false default: mired min: description: The minimum color temperature in the chosen unit. type: integer default: 2700 for kelvin 153 for mired required: false max: description: The maximum color temperature in the chosen unit. type: integer default: 6500 for kelvin 500 for mired required: false

The output of this selector is the number representing the chosen color temperature for the unit used.

Condition selector

The condition selector allows you to input one or more conditions. On the user interface, the condition part of the automation editor will be shown. The value of the input will contain a list of conditions.

Screenshot of an condition selector

This selector does not have any other options; therefore, it only has its key.

```
condition:
```

The output of this selector is a list of conditions. For example:

```
# Example condition selector output result
- condition: numeric_state
  entity_id: "sensor.outside_temperature"
  below: 20
```

Config entry selector

The config entry selector allows you to select an integration configuration entry. The selector returns the entry ID of the selected integration configuration entry.

Screenshot of the Configuration entry selector

```
config_entry:
```

{% configuration config_entry %} integration: description: Limits the list of selectable configuration entries to a single integration domain. type: string required: false

The output of this selector is the entry ID of the config entry, for example, `6b68b250388cbe0d620c92dd3acc93ec`.

Constant selector

The constant selector, when used in an optional setting, shows a toggle that allows you to enable the selected option. This is similar to the [boolean selector](#), the difference is that the constant selector has no value when it's not enabled.

 **Note:** A constant selector is only useful in a context where selectors may be designated as optional, such as in an action schema or script field. It is not recommended for blueprints because they do not have optional inputs, so the selector cannot be toggled.

Screenshot of a constant selector

The selector's value must be configured, and optionally, a label.

```
constant:
  value: true
  label: Enabled
```

The output of this selector is the configured value when the toggle is on, it has no output otherwise.

Conversation agent selector

The conversation agent selector allows picking a conversation agent.

Screenshot of a conversation agent selector

The selector has 1 option, `language`. This filters the conversation agents shown, depending on the language.

```
conversation_agent:
  language: en
```

{% configuration conversation_agent %} language: description: Limits the list of conversation agents to those supporting the specified language. type: string required: false

The output of this selector is the ID of the conversation agent.

Country selector

The country selector allows a user to pick a country from a list of countries.

Screenshot of a country selector

```
country:
```

{% configuration entity %} countries: description: A list of countries to pick from, this should be ISO 3166 country codes. type: list default: The available countries in the Home Assistant frontend required: false no_sort: description: > Should the options be sorted by name, if set to true, the order of the provided countries is kept. type: boolean default: false required: false

The output of this selector is an ISO 3166 country code.

Date selector

The date selector shows a date input that allows you to specify a date.

Screenshot of the Date selector

This selector does not have any other options; therefore, it only has its key.

```
date:
```

The output of this selector will contain the date in Year-Month-Day (`YYYY-MM-DD`) format, for example, `2022-02-22` .

Date & time selector

The date selector shows a date and time input that allows you to specify a date with a specific time.

Screenshot of the Date & time selector

This selector does not have any other options; therefore, it only has its key.

```
datetime:
```

The output of this selector will contain the date in Year-Month-Day (`YYYY-MM-DD`) format and the time in 24-hour format, for example: `2022-02-22 13:30:00` .

Device selector

The device selector shows a device finder that can pick a single or multiple devices based on the selector configuration. The value of the input will contain the device ID or a list of device IDs, based on if `multiple` is set to `true`.

A device selector can filter the list of devices, based on things like the manufacturer, model, or model ID of the device, the entities the device provides or based on the domain that provided the device.

Screenshot of a device selector

In its most basic form, this selector doesn't require any options, which will show all devices.

```
device:
```

{% configuration device %} entity: description: > When entity options are provided, the list of devices is filtered by devices that at least provide one entity that matches the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of devices that provide entities by the set integration domain, for example, `zha`. type: string required: false domain: description: > Limits the list of devices that provide entities of a certain `domain(s)`, for example, `light` or `binary_sensor`. Can be either a string with a single domain, or a list of string domains to limit the selection to. type: string required: false device_class: description: > Limits the list of devices to devices that have entities with a certain device class(es), for example, `motion` or `window`. Can be either a string with a single device_class, or a list of string device_class to limit the selection to. type: [device_class, list] required: false supported_features: description: > Limits the list of devices to devices that have entities with a certain supported feature, for example, `light.LightEntityFeature.TRANSITION` or `climate.ClimateEntityFeature.TARGET_TEMPERATURE`. Should be a list of features. For a list of supported features for each entity type, refer to the [entity documentation](#). type: list required: false filter: description: > When filter options are provided, the list of devices is filtered by devices that at least provide one entity that matches the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of devices to devices provided by the set integration domain. type: string required: false manufacturer: description: > When set, it limits the list of devices to devices provided by the set manufacturer name. type: string required: false model: description: > When set, it limits the list of devices to devices that have the set model. type: string required: false model_id: description: > When set, the list of devices is limited to devices that have the set model ID. type: string required: false multiple: description: > Allows selecting multiple devices. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false required: false

The output of this selector is the device ID, or (in case `multiple` is set to `true`) a list of devices IDs.

```
# Example device selector output result, when multiple is set to false
faadde5365842003e8ca55267fe9d1f4

# Example device selector output result, when multiple is set to true
- faadde5365842003e8ca55267fe9d1f4
- 3da77cb054352848b9544d40e19de562
```

Example device selector

An example entity selector that, will only show devices that are:

- Provided by the [deCONZ](#) integration.
- Are a Philips Hue Remote of Model RWL021.
- Provide a battery [sensor](#).

And this is what it looks like in YAML:

```
device:
  filter:
    - integration: deconz
      manufacturer: Philips
      model: RWL021
  entity:
    - domain: sensor
      device_class: battery
```

Duration selector

The duration selector lets you select a time duration. This can be helpful, for example, for delays or offsets.

Screenshot of the Duration selector

```
duration:
```

{% configuration attribute %} enable_day: description: When `true`, the duration selector will allow selecting days. type: boolean default: false required: false enable_second: description: When `true`, the duration selector will allow selecting seconds. type: boolean default: true required: false enable_millisecond: description: When `true`, the duration selector will allow selecting milliseconds. type: boolean default: false required: false allow_negative: description: When `true`, the duration selector will allow for selecting positive or negative values. type: boolean default: false required: false

The output of this selector is a mapping of the time values the user selected. For example:

```
days: 1 # Only when enable_day is set to true
hours: 12
minutes: 30
seconds: 15 # Only when enable_second is set to true (default)
milliseconds: 500 # Only when enable_millisecond was set to true
```

Entity selector

The entity selector shows an entity finder that can pick a single entity or a list of entities based on the selector configuration. The value of the input will contain the entity ID, or list of entity IDs, based on if `multiple` is set to `true`.

An entity selector can filter the list of entities, based on things like the class of the device, the domain of the entity or the domain that provided the entity.

Screenshot of an entity selector

In its most basic form, this selector doesn't require any options, which will show all entities.

```
entity:
```

{% configuration entity %} exclude_entities: description: List of entity IDs to exclude from the selectable list. type: list required: false include_entities: description: List of entity IDs to limit the selectable list to. type: list required: false filter: description: > When filter options are provided, the entities are limited by entities that at least match the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of entities to entities provided by the set integration domain, for example, `zha`. type: string required: false domain: description: > Limits the list of entities to entities of a certain domain(s), for example, `light` or `binary_sensor`. Can be either a string with a single domain, or a list of string domains to limit the selection to. type: [string, list] required: false device_class: description: > Limits the list of entities to entities that have a certain device class(es), for example, `motion` or `window`. Can be either a string with a single device_class, or a list of string device_class to limit the selection to. type: [device_class, list] required: false supported_features: description: > Limits the list of entities to entities that have a certain supported feature, for example, `light.LightEntityFeature.TRANSITION` or `climate.ClimateEntityFeature.TARGET_TEMPERATURE`. Should be a list of features. type: list required: false multiple: description: > Allows selecting multiple entities. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false required: false reorder: description: > Allows reordering of entities (only applies if `multiple` is set to `true`). type: boolean default: false required: false

The output of this selector is the entity ID, or (in case `multiple` is set to `true`) a list of entity IDs.

```
# Example entity selector output result, when multiple is set to false
light.living_room

# Example entity selector output result, when multiple is set to true
- light.living_room
- light.kitchen
```

Example entity selector

An example entity selector that, will only show entities that are:

- Provided by the [ZHA](#) integration.
- From the [Binary sensor](#) domain.
- Have presented themselves as devices of a motion device class.
- Allows selecting one or more entities.

And this is what it looks like in YAML:

```
entity:
  multiple: true
  filter:
    - integration: zha
      domain: binary_sensor
      device_class: motion
```


Floor selector

The floor selector shows a floor finder that can pick floors based on the selector configuration. The value of the input will be the floor ID. If `multiple` is set to `true`, the value is a list of floor IDs.

A floor selector can filter the list of floors based on the properties of the devices and entities assigned to the areas on those floors. For example, the floor list could be limited to floors with entities provided by the [ZHA](#) integration, based on the areas they are in.

In its most basic form, this selector doesn't require any options. It will show all floors.

Screenshot of a floor selector

```
floor:
```

{% configuration floor %} device: description: > When device options are provided, the list of floors is filtered by floors that have at least one device matching the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of floors that have devices by this integration domain. For example, `zha`. type: string required: false manufacturer: description: > When set, the list only includes floors that have devices by the set manufacturer name. type: string required: false model: description: > When set, the list only includes floors that have devices which have the set model. type: string required: false model_id: description: > When set, the list only includes floors with devices that have the set model ID. type: string required: false entity: description: > When entity options are provided, the list only includes floors that at least have one entity that matches the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits the list of floors that have entities by the set integration domain. For example, `zha`. type: string required: false domain: description: > When set, the list only includes floors that have entities of certain [domains](#), for example, `light` or `binary_sensor`. Can be either a string with a single domain, or a list of string domains to limit the selection to. type: [string, list] required: false device_class: description: > When set, the list only includes floors that have entities with a certain device class, for example, `motion` or `window`. Can be either a string with a single device_class, or a list of string device_class to limit the selection. type: [device_class, list] required: false supported_features: description: > When set, the list only includes floors that have entities with a certain supported feature, for example, `light.LightEntityFeature.TRANSITION` or `climate.ClimateEntityFeature.TARGET_TEMPERATURE`. Should be a list of features. type: list required: false multiple: description: > Allows selecting multiple floors. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false required: false

The output of this selector is the floor ID, or (in case `multiple` is set to `true`) a list of floor IDs.

```
# Example floor selector output result, when multiple is set to false
first_floor

# Example floor selector output result, when multiple is set to true
- first_floor
- second_floor
```

Example floor selectors

An example floor selector only shows floors that have one or more lights or switches provided by the [ZHA](#) integration.

```
floor:
  entity:
    integration: zha
  domain:
    - light
    - switch
```

Another example using the floor selector, which only shows floors that have one or more remote controls provided by the [deCONZ](#) integration. Multiple floors can be selected.

```
floor:
  multiple: true
  device:
    - integration: deconz
      manufacturer: IKEA of Sweden
      model: TRADFRI remote control
```

Icon selector

The icon selector shows an icon picker that allows you to select an icon.

```
icon:
```

{% configuration entity %} placeholder: description: Placeholder icon to show, when no icon is selected. type: string required: false

The output of this selector is a string containing the selected icon, for example: `mdi:bell`.

Label selector

The label selector shows a label finder that can pick labels. The value of the input is the label ID. If `multiple` is set to `true`, the value is a list of label IDs.

Screenshot of the label selector

In its most basic form, this selector doesn't require any options. It will show all labels.

```
label:
```

{% configuration text %} multiple: description: > Allows selecting multiple labels. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false required: false

The output of this selector is the label ID, or (in case `multiple` is set to `true`) a list of label IDs.

```
# Example label selector output result, when multiple is set to false
energy_saving

# Example label selector output result, when multiple is set to true
- energy_saving
- christmas_decorations
```

Language selector

The language selector allows a user to pick a language from a list of languages.

Screenshot of an language selector

```
language:
```

{% configuration entity %} languages: description: A list of languages to pick from, this should be RFC 5646 languages codes. type: list default: The available languages in the Home Assistant frontend required: false native_name: description: > Should the name of the languages be shown in the language of the user, or in the language itself. type: boolean default: false required: false no_sort: description: > Should the options be sorted by name, if set to true, the order of the provided languages is kept. type: boolean default: false required: false

The output of this selector is an RFC 5646 language code.

Location selector

The location selector allow a user to pick a location from a map and returns the matching longitude and latitude coordinators. Optionally it supports selecting the radius of the location.

Screenshot of the Location selector

```
location:
```

{% configuration entity %} icon: description: An optional icon to show on the map. type: string required: false radius: description: > Allow selecting the radius of the location. If enabled, the radius will be returned in meters. type: boolean default: false required: false

The output of this selector is a mapping containing the latitude and longitude of the selected location, and, if enabled, the radius. For example:

```
latitude: 50.935
longitude: 6.95
radius: 500 # Only provided when radius was set to true.
```

Media selector

The media selector is a powerful selector that allows a user to easily select media to play on a media device. Media can be a lot of things, for example, cameras, local media, text-to-speech, Home Assistant Dashboards, and many more.

You are prompted to select the device used to play media. Once the device is selected, the media selector only shows media that is suitable for this device.

Screenshot of the Media selector

To ask the user to select a media device and suitable media, you can use the media selector without any options:

```
media:
```

You can also use the media selector with an optional `accept` filter to limit the media types that can be selected. The user will not be asked to pick a device.

```
media:
  accept:
    - image/*
```

{% configuration media %} `accept`: description: > List of media types the user is allowed to select. `type`: list required: false multiple: description: > Allows selecting multiple media items. If set to `true`, the resulting value of this selector will be a list instead of a single object. `type`: boolean default: false required: false

The output of the media selector is a mapping with information about the selected media device and the selected media to play. There is also metadata, which is used by the frontend and should not be used in the backend.

Example output:

```
entity_id: media_player.living_room
media_content_id: media-source://tts/cloud?message=TTS+Message&language=en-US&ge
media_content_type: provider
metadata:
  title: TTS Message
  thumbnail: https://brands.home-assistant.io/_/cloud/logo.png
  media_class: app
  children_media_class: null
  navigateIds:
    - {}
    - media_content_type: app
      media_content_id: media-source://tts
    - media_content_type: provider
      media_content_id: >-
        media-source://tts/cloud?message=TTS+Message&language=en-US&gender=femal
```

Example output if accept filter is used. Note that the `entity_id` is not present:

```

media_content_id: media-source://tts/cloud?message=TTS+Message&language=en-US&ge
media_content_type: provider
metadata:
  title: TTS Message
  thumbnail: https://brands.home-assistant.io/_/cloud/logo.png
  media_class: app
  children_media_class: null
  navigateIds:
    - {}
    - media_content_type: app
      media_content_id: media-source://tts
    - media_content_type: provider
      media_content_id: >-
        media-source://tts/cloud?message=TTS+Message&language=en-US&gender=femal

```

Example output when `multiple` is set to `true` (a list of media objects):

```

- media_content_id: media-source://media_source/local/image1.jpg
  media_content_type: image/jpeg
  metadata:
    title: image1.jpg
- media_content_id: media-source://media_source/local/image2.jpg
  media_content_type: image/jpeg
  metadata:
    title: image2.jpg

```


Number selector

The number selector shows either a number input or a slider input, that allows the user to specify a numeric value. The value of the input will contain the select value.

Screenshot of a number selector

On the user interface, the input can either be in a slider or number mode. Both modes limit the user input by a minimum and maximum value, and can have a unit of measurement to go with it.

In its most basic form, this selector requires a minimum and maximum value:

```
number:
  min: 0
  max: 100
```

{% configuration number %} min: description: The minimum user-settable number value. type: [integer, float] required: false max: description: The maximum user-settable number value. type: [integer, float] required: false step: description: The step size of the number value. Set to `"any"` to allow any number. type: [integer, float, "any"] required: false default: 1 unit_of_measurement: description: Unit of measurement in which the number value is expressed in. type: string required: false mode: description: This can be either `box` or `slider` mode. type: string required: false default: slider if min and max are set, otherwise box translation_key: description: > Allows translations provided by an integration where `translation_key` is the translation key that is providing the unit_of_measurement string translation. See the documentation on [Backend Localization](#) for more information. type: string required: false

The output of this selector is a number, for example: `42`

Example number selectors

An example number selector that allows a user a percentage, directly in a regular number input box.

```
number:
  min: 0
  max: 100
  unit_of_measurement: "%"
```

A more visual variant of this example could be achieved using a slider. This can be helpful for things like allowing the user to select a brightness level of lights. Additionally, this example changes the brightness in incremental steps of 10%.

```
number:
  min: 0
  max: 100
  step: 10
  unit_of_measurement: "%"
  mode: slider
```

Object selector

The object selector can be used to input arbitrary data in YAML form. This is useful for lists and dictionaries containing data for actions, for example. The value of the input will contain the provided data.

When used without options, the selector will accept any valid YAML content, such as objects, arrays, strings, or other YAML types. The input box is displayed as an editor with syntax highlighting.

Screenshot of an object selector

```
object:
```

When used with `fields` specified, the selector will force the object to be in this format by displaying a form.

Screenshot of an object selector

```
object:
  label_field: name
  description_field: percentage
  multiple: true
  fields:
    name:
      label: Name
      selector:
        text:
    percentage:
      label: Percentage
      selector:
        number:
          unit_of_measurement: "%"
```

The output of this selector is a YAML object.

{% configuration object %} fields: description: > List of fields of the object. type: map required: false keys: label: description: The label of the field required: false type: string required: description: If set to true, the field must be present. required: false default: false type: boolean selector: description: The selector to use for this field. It can be any available selector. required: true type: string label_field: description: > The field to use as a label. By default, it will be the first field defined. This option is only used if `fields` option set. type: string required: false description_field: description: > The field to use as a description. This option is only used if `fields` option set. type: string required: false translation_key: description: > Allows translations provided by an integration where `translation_key` is the translation key that is providing the selector option strings translation. See the documentation on [Backend Localization](#) for more information. type: string required: false multiple: description: > Allows adding multiple objects. If set to `true`, the resulting value of this selector will be a list instead of a single YAML object. This option is only used if `fields` option set. type: boolean required: false default: false

QR code selector

The QR code selector shows a QR code. It has no return value.

Screenshot of a QR code selector

The QR code's data must be configured, and optionally, the scale, and error correction level can be set. The scale makes the QR code bigger or smaller.

{% configuration qr_code %} data: description: The data that should be represented in the QR code. type: any required: true scale: description: The scale factor to use, this will make the QR code bigger or smaller. type: integer required: false default: 4 error_correction_level: description: The error correction level of the QR code, with a higher error correction level the QR code can be scanned even when some pieces are missing. Can be "low", "medium", "quartile" or "high". type: string required: false default: medium

```
qr_code:
  data: "https://home-assistant.io"
  scale: 5
  error_correction_level: quartile
```

RGB color selector

The RGB color selector allows you to select a color from a color picker from the user interface, and returns the RGB color value.

Screenshot of the RGB Color selector

```
color_rgb:
```

This selector does not have any other options; therefore, it only has its key.

The output of this selector is a list with the three (RGB) color value, for example: `[255, 0, 0]`.

Select selector

The select selector shows a list of available options from which the user can choose. The value of the input contains the value of the selected option. Only a single option can be selected at a time.

Screenshot of a select selector

The selector requires a list of options that the user can choose from.

```
select:
  options:
    - Red
    - Green
    - Blue
```

{% configuration select %} options: description: > List of options that the user can choose from. Small lists (5 items or less), are displayed as radio buttons. When more items are added, a dropdown list is used. type: list required: true multiple: description: > Allows selecting multiple options. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean required: false default: false custom_value: description: > Allows the user to enter and select a custom value (or multiple custom values in addition to the listed options if `multiple` is set to `true`). type: boolean required: false default: false mode: description: > This can be either `list` (radio buttons) or `dropdown` (combobox) mode. When not specified, small lists (5 items or less), are displayed as radio buttons. When more items are added, a dropdown list is used. If `custom_value` is `true`, this setting will be ignored and the frontend will use a `dropdown` input. type: string required: false translation_key: description: > Allows translations provided by an integration where `translation_key` is the translation key that is providing the selector option strings translation. See the documentation on [Backend Localization](#) for more information. type: string required: false sort: description: > Display options in alphabetical order. type: boolean required: false default: false

Alternatively, a mapping can be used for the options. When you want to return a different value compared to how it is displayed to the user.

```
select:
  options:
    - label: Red
      value: r
    - label: Green
      value: g
    - label: Blue
      value: b
```

{% configuration select_map %} options: description: > List of options that the user can choose from. Small lists (5 items or less), are displayed as radio buttons. When more items are added, a dropdown list is used. type: map required: true keys: label: description: The description to show in the UI for this item. required: true type: string value: description: The value to return when this label is selected. required: true type: string

When `multiple` is `false`, the output of this selector is the string of the selected option value. When selecting `Green` in the last example, it returns: `g`, in the first example it would return `Green`.

When `multiple` is `true`, the output of this selector is the list of selected option values. In this case, if `Green` was selected, in the first example it would return `["Green"]` and in the last example it returns `["g"]`.

State selector

The state selector shows a list of states for a provided entity of which one or more can be selected.

Screenshot of a state selector

{% configuration state %} entity_id: description: The entity ID of which an state can be selected from. type: string required: false hide_states: description: The states to exclude from the list of options type: list required: false multiple: description: > Allows selecting multiple states. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false

The output of this selector is the select state (not the translated or prettified name shown in the frontend), or a list of states if `multiple` is true.

For example: `heat_cool`.

Statistic selector

The statistic selector selects the statistic ID of an entity that records Long-term statistics. It may resemble an entity ID (like `sensor.temperature`), or an external statistic ID (like `external:temperature`).

Screenshot of a statistic selector

{% configuration statistic %} multiple: description: > If set to true, the selector returns a list of statistic IDs. type: boolean default: false required: false

The output of this selector is a string representing a statistic ID, or a list of statistic IDs if `multiple` is set to `true`.

Target selector

The target selector is a rather special selector, allowing the user to select targeted entities, devices, or areas for actions. The value of the input will contain a special target format, that is accepted by actions.

The selectable targets can be filtered, based on entity or device properties. Areas are only selectable as a target, if some entities or devices match those properties in those areas.

Screenshot of a target selector

In its most basic form, this selector does not require any options, which will allow the user to target any entity, device or area available in the system.

```
target:
```

{% configuration target %} entity: description: > When entity options are provided, the targets are limited by entities that at least match the given conditions. Can be either an object or a list of objects. type: list required: false keys: integration: description: > Can be set to an integration domain. Limits targets to entities provided by the set integration domain, for example, `zha`. type: string required: false domain: description: > Limits the targets to entities of a certain [domain\(s\)](#), for example, `light` or `binary_sensor`. Can be either a with a single domain, or a list of string domains to limit the selection to. type: [string, list] required: false device_class: description: > Limits the targets to entities with a certain device class(es), for example, `motion` or `window`. Can be either a string with a single device_class, or a list of string device_class to limit the selection to. type: [device_class, list] required: false supported_features: description: > Limits the targets to entities with a certain supported feature, for example, `light.LightEntityFeature.TRANSITION` or `climate.ClimateEntityFeature.TARGET_TEMPERATURE`. Should be a list of features. For a list of supported features for each entity type, refer to the [entity documentation](#). type: list required: false

⚠ Important: Targets are meant to be used with the `target` property of an action in a script sequence. For example:

```
```yaml actions: - action: light.turn_on target: !input lights
```

```
Example target selectors <!-- omit from toc -->
```

An example target selector that only shows targets that at least provide one or more lights, provided by the [ZHA](/integrations/zha) integration.

```
```yaml
```

```
target:
  entity:
    - integration: zha
      domain: light
```

Template selector

The template selector can be used to input a Jinja2 template. This is useful for allowing more advanced user-input that use Jinja2 templates.

Screenshot of an template selector

This selector does not have any other options; therefore, it only has its key.

```
template:
```

The output of this selector is a template string.

Text selector

The text selector can be used to enter a text string. It can also be used to enter a list of text strings; if `multiple` is set to `true`. The value of the input will contain the selected text. This can be used in shopping lists, for example.

Screenshot of text selectors

Unless `multiline` is set to `true`, this selector behaves exactly like if no selector at all was specified, and will display a single line text input box on the user interface.

```
text:
```

{% configuration text %} multiline: description: Set to true to display the input as a multi-line text box on the user interface. type: boolean default: false required: false prefix: description: An optional prefix to show before the text input box. type: string required: false suffix: description: An optional suffix to show after the text input box. type: string required: false type: description: > The type of input. This supplies the HTML `type` attribute, which controls how the browser displays and validates the field. A subset of types available to the attribute are supported, since some are handled by other selectors. Possible types are: `color`, `date`, `datetime-local`, `email`, `month`, `number`, `password`, `search`, `tel`, `text`, `time`, `url`, `week`. type: string default: text required: false autocomplete: description: > Guides the browser on the type of information which should automatically fill the field. This supplies the HTML `autocomplete` attribute. Any value supported by the HTML attribute is valid. type: string required: false multiple: description: > Allows adding list of text strings. If set to `true`, the resulting value of this selector will be a list instead of a single string value. type: boolean default: false required: false

The output of this selector is a single string value.

Theme selector

The theme selector allows for selecting a theme from the available themes installed in Home Assistant.

Screenshot of the Theme selector

```
theme:
```

{% configuration theme %} include_default: description: Includes Home Assistant default theme in the list. type: boolean default: false required: false

The output of this selector will contain the selected theme, for example: `waves_dark`.

Time selector

The time selector shows a time input that allows you to specify a time of the day.

Screenshot of a time selector

This selector does not have any other options; therefore, it only has its key.

```
time:
```

The output of this selector will contain the time in 24-hour format, for example, `23:59:59` .

Trigger selector

The triggers selector allows you to input one or more triggers. On the user interface, the trigger part of the automation editor is shown. The value of the input contains a list of triggers.

Screenshot of an trigger selector

This selector does not have any other options; therefore, it only has its key.

```
trigger:
```

The output of this selector is a list of triggers. For example:

```
# Example trigger selector output result
- trigger: numeric_state
  entity_id: "sensor.outside_temperature"
  below: 20
```

Example - Merging with existing triggers

If the trigger(s) should exist within a blueprint that already has some default triggers defined, and an additional customizable trigger should be merged, you need to use the `- triggers` syntax in the blueprint.

```
# Example trigger selector
input:
  my_trigger_input:
    selector:
      trigger:
triggers:
- triggers: !input my_trigger_input
- platform: numeric_state
[...]
```

Docs/Blueprint/Tutorial

💡 **Tip:** While the tutorial only shows how to create an automation blueprint, scripts also support blueprints in the same way.

Creating an automation blueprint

In this tutorial, we're going to create an automation blueprint that controls a light based on a motion sensor. We will do this by taking an existing automation and converting it to a blueprint.

Prerequisites

- This tutorial assumes knowledge in the following topics:
- [Editing the configuration file](#)
- [YAML](#), and specifically, [YAML used in automations](#)
- [Scripts](#)

Creating an automation

To create a blueprint, we first need to have a working automation. For this tutorial, we use a simple automation. The process for converting a complex automation is no different.

The automation we're going to use in this tutorial controls a light based on a motion sensor:

```
triggers:
- trigger: state
  entity_id: binary_sensor.motion_kitchen

actions:
- action: >
  {% if trigger.to_state.state == "on" %}
    light.turn_on
  {% else %}
    light.turn_off
  {% endif %}
  target:
    entity_id: light.kitchen
```

The options that can be used with the `trigger` object are listed under [automation trigger variables](#). In this example, a [state trigger](#) is used. `turn_on` and `turn_off` are [homeassistant actions](#). They are not tied to a specific domain. You can use them on lights, switches, and other domains.

Creating the blueprint file

Automation blueprints are YAML files (with the `.yaml` extension) and live in the `<config>/blueprints/automation/` folder. You can create as many subdirectories in this folder as you want.

To get started with our blueprint, we're going to copy the above automation YAML and save it in that directory with the name `motion_light_tutorial.yaml`.

Add basic blueprint metadata

Home Assistant needs to know about the blueprint. This is achieved by adding a `blueprint:` section. It should contain the `domain` of the integration it is for (`automation`) and `name` , the name of your blueprint. Optionally, you can also include a `description` for your blueprint.

Add this to the top of the file:

```
blueprint:
  name: Motion Light Tutorial
  description: Turn a light on based on detected motion
  domain: automation
```

Define the configurable parts as inputs

Now we have to decide what steps we want to make configurable. We want to make it as reusable as possible, without losing its original intent of turning on a light-based on a motion sensor.

Configurable parts in blueprints are called `inputs`. To make the motion sensor entity configurable, we're replacing the entity ID with a custom YAML tag `!input` . This YAML tag has to be combined with the name of the input:

```
triggers:
- trigger: state
  entity_id: !input motion_sensor
```

For the light, we can offer some more flexibility. We want to allow the user to be able to define any device or area as the target. The `target` property in the action can contain references to areas, devices, and/or entities, so that's what we will use.

Inputs are not limited to strings. They can contain complex objects too. So in this case, we're going to mark the whole `target` as input:

```
actions:
- action: >
  {% if trigger.to_state.state == "on" %}
    light.turn_on
  {% else %}
    light.turn_off
  {% endif %}
  target: !input target_light
```

Add the inputs to the metadata

All parts that are marked as inputs need to be added to the metadata. The minimum is that we add their names as used in the automation:

```
blueprint:
  name: Motion Light Tutorial
  description: Turn a light on based on detected motion
  domain: automation
  input:
    motion_sensor:
    target_light:
```

For more information on blueprint inputs, refer to the documentation of the [blueprint schema](#)

Using your blueprint via `configuration.yaml`

With the bare minimum metadata added, your blueprint is ready to use.

Open your `configuration.yaml` and add the following:

```
automation tutorial:
  use_blueprint:
    path: motion_light_tutorial.yaml
  input:
    motion_sensor: binary_sensor.kitchen
    target_light:
      entity_id: light.kitchen
```

Reload automations and your new automation should pop up. Because we configured the exact values as the original automation, they should work exactly the same.

Improving the inputs

Blueprints are easier to use if it's easy to see what each field is used for.

Add a user friendly names to the inputs

We can improve this experience by adding names and descriptions to our inputs:

```
blueprint:
  name: Motion Light Tutorial
  description: Turn a light on based on detected motion
  domain: automation
  input:
    motion_sensor:
      name: Motion Sensor
      description: This sensor will be synchronized with the light.
    target_light:
      name: Lights
      description: The lights to keep in sync.
```

Describe the inputs

Our blueprint doesn't currently describe what the inputs should contain. Without this information, Home Assistant will offer the user an empty text box.

To instead allow Home Assistant to offer more assistance, we will use [selectors](#). Selectors describe a type and can be used to help the user pick a matching value.

The selector for the motion sensor entity should describe that we want entities from the binary sensor domain that have the device class `motion`.

The selector for the target light should describe that we want to target light entities.

```
blueprint:
  name: Motion Light Tutorial
  domain: automation
  input:
    motion_sensor:
      name: Motion Sensor
      description: This sensor will be synchronized with the light.
      selector:
        entity:
          filter:
            - domain: binary_sensor
              device_class: motion
    target_light:
      name: Lights
      description: The lights to keep in sync.
      selector:
        target:
          entity:
            - domain: light
```

By limiting our blueprint to working with lights and motion sensors, we unlock a couple of benefits: the UI will be able to limit suggested values to lights and motion sensors instead of all devices. It will also allow the user to pick an area to control the lights in.

The final blueprint

After we have added all the steps, our blueprint will look like this:

```
blueprint:
  name: Motion Light Tutorial
  description: Turn a light on based on detected motion
  domain: automation
  input:
    motion_sensor:
      name: Motion Sensor
      description: This sensor will be synchronized with the light.
      selector:
        entity:
          filter:
            - domain: binary_sensor
              device_class: motion
    target_light:
      name: Lights
      description: The lights to keep in sync.
      selector:
        target:
          entity:
            - domain: light

  triggers:
    - trigger: state
      entity_id: !input motion_sensor

  actions:
    - action: >
      {% if trigger.to_state.state == "on" %}
        light.turn_on
      {% else %}
        light.turn_off
      {% endif %}
    target: !input target_light
```

Using the blueprint via the UI

1. To configure your blueprint via the UI, go to **Settings > Automations & scenes > Blueprints**.
2. Find the **Motion Light Tutorial** blueprint and select **Create Automation**.

⚠ Important: Don't forget to reload automations after you make changes to your blueprint to have the UI and the automation integration pick up the latest blueprint changes.

Screenshot of the blueprint UI

Video tutorial

This video tutorial explains how to create a blueprint that toggles a light on motion when the lux value is below a certain threshold.

Share the love

The final step is to share this blueprint with others. For this tutorial we're going to share it on GitHub Gists.

Share informally

For this tutorial, we're going to share it on GitHub Gists. This is a good option if you don't want to publish your blueprint to a larger audience.

1. Go to [GitHub Gists](#)
2. **Gist description:** blueprint tutorial
3. **Filename including extension:** `motion_light_tutorial.yaml`
4. **Content** is the content of the blueprint file.
5. Select **Create Gist**.
6. To share your blueprint with other people, copy the URL of your new Gist. They can import it by going to **Settings > Automations & scenes > Blueprints** and select **Import blueprint**.
7. Celebrate! Cheers to you. You created your first blueprint and helped someone in the community.

Share on the Blueprint Exchange

If you follow the [Rules and format for posting](#), you can share your blueprint on the Home Assistant Blueprint Exchange forum. This option is accessible to the general Home Assistant community but recommended only for your original blueprints. Please don't post this tutorial to the Blueprint Exchange, but instead, remember this as an option for releasing your real blueprints.

Docs/Blueprint

Blueprints are the easiest way to add automations, scripts, or template entities to your Home Assistant. Someone in the community has already done the work of writing the configuration, and you fill in the bits that are specific to your home, like which sensor to watch and which light to control.

You can find a blueprint for almost any common use case in the [community blueprint forum](#): motion-activated lights, low-battery notifications, holiday lighting, presence-based heating, and many more.

This page is a high-level introduction. If you want to create your own blueprint to share, see [About the blueprint schema](#).

What is a blueprint?

A blueprint is a script, automation, or [template entity](#) configuration where some parts have been left blank, ready for you to fill in. That way, the same blueprint can be reused over and over with different devices and settings.

Imagine you want to turn on a light when motion is detected. A blueprint provides the generic automation, while letting you select *which* motion sensor and *which* light. You can use that same blueprint twice, once for the hallway and once for the bathroom, and end up with two completely independent automations that each behave the way you configured them.

Automations inherit from the blueprint they were built on, so if the blueprint is updated, all automations using it pick up the change the next time Home Assistant reloads them. To reload manually, go to **Settings** > **Developer tools** > **YAML** and reload the automations.

Blueprints are shared by the community in the [blueprint community forum](#).

Docs/Configuration/Basic

As part of the default onboarding process, Home Assistant can detect your location from IP address geolocation. Home Assistant will automatically select a unit system and time zone based on this location. If you didn't adjust this directly during onboarding, you can do it later.

Screenshot showing General settings page Screenshot showing the General settings page.

The general settings described here are managed by the [Home Assistant Core integration](#). If you are interested in the actions offered by this integration, check out the integration documentation.

Editing the general settings

To change the general settings that were defined during onboarding, follow these steps:

1. Go to **Settings > System > General**.
2. Make your changes.
3. To change location or radius, under **Edit location**, select edit.
4. Then, adjust location and radius. Screenshot showing how to zoom and pan to change location and radius on the Edit home page
5. To add a new zone, select **Add zone**.
6. To save your changes, select **Update**.
7. To change network-related configuration, such as the network name, go to **Settings > System > Network**.
8. If some of the settings are not visible, you may need to enable **Advanced mode**.
9. In the bottom left, select your username to go to your **User profile**, and enable **Advanced mode**.
10. Troubleshooting: If any of the settings are grayed out and can't be edited, this is because they are defined in the `configuration.yaml` file.
11. If you prefer editing the settings in the UI, you have to delete these entries from the `configuration.yaml` file.
12. For more information about the general settings in YAML, refer to the [Home Assistant Core integration documentation](#).

Setting fields are grayed out because the configuration settings stored in configuration.yaml file
13. To apply the changes, follow the steps on [reloading the configuration](#).

Changing a person's display name

The display name is the name that is shown in Home Assistant. It can differ from the username, which is the name used to log in.

Prerequisites

- You need administrator rights to change a display name.

To change a display name

1. To edit the display name of a person using Home Assistant, go to **Settings** > **People** and select the person for which you want to change the display name.
2. Change the display name and select **Update** to save the change.

Changing a username

The username is the name that is used to log in. It can differ from the display name.

Prerequisites

- You need owner rights to change a username.

To change a username

1. To edit the username of a person using Home Assistant, go to **Settings > People** and select the person for which you want to change the display name.
2. Change the username and select **Update** to save the change.
3. It must be lowercase and contain no spaces.
4. The log in is case-sensitive.

Changing authentication settings

To learn how to edit authentication settings such as password or multi-factor authentication, refer to the following topics:

- [Authentication](#)
 - [multi-factor authentication](#)
 - [Help, I'm locked out](#)
-

Docs/Configuration/Customizing Devices

After adding a new device, you might find the automatically assigned entity ID too technical and the entity lacking a friendly name. You can personalize these elements to better fit your naming conventions or modify other attributes like the icon.

To change entity attributes, follow these steps:

1. Go to **Settings > Devices & services > Entities** and select the entity from the list.
2. In the top-right corner, select the `mdi:cog` cog icon.

Entity dialog box with cog icon.

1. Enter or edit the attributes:
2. For example, the entity ID here could be shortened to `binary_sensor.lumi_sensor_aq2_opening`.
 - You can use lowercase letters, numbers, and underscores.
 - The ID must not start or end with an underscore.
 - To undo the change and revert the ID to the default, select the `mdi:restore` icon.
 - **Note:** You can only reset the ID to the default ID for entities with a unique ID.
 - IDs of entities that are disabled or for which the integration is not set up cannot be reverted.
 - To revert all the entity IDs for a device, on the device page, select the three dots `mdi:dots-vertical` menu, then select **Recreate entity IDs**.
 - Result: This resets the entity ID and applies the current default naming convention.
 - The terms used to generate the entity ID depends on a few factors. Prioritization is as follows:
 1. If you changed the friendly name of the entity, the friendly name will be used.
 2. The entity ID suggested by the integration (just a few integrations do this).
 3. The default name in the user language, if using Latin script.
 - If the something other than Latin script is used, the entity ID is based on the English default name.
 - This is because entity IDs must use lowercase alphanumeric characters in the range of [a-z,1-9].
3. Enter or edit the friendly name.
 - In this example, this would change "Opening".
4. If needed, from the **Shown as** menu, you can select a different [device class](#).
5. If you like, add a [label](#).

Settings for entity.

1. To apply the changes, select **Update**.
2. If you have used this entity in automations and scripts, you need to rename the entity ID there, too.

3. Go to **Settings** > **Automations & scenes** open the respective tab and find your automation or script.

Customizing an entity in YAML

If your entity is not supported, or you could not customize what you need via the user interface, you need to edit the settings in your `configuration.yaml` file. For a detailed description of the entity configuration variables and `device class` information, refer to the [Home Assistant Core integration documentation](#).

Docs/Configuration/Entities Domains

Your devices are represented in Home Assistant as entities. Entities are the basic building blocks to hold data in Home Assistant. An entity represents a sensor, actor, or function in Home Assistant. Entities are used to monitor physical properties or to control other entities. An entity is usually part of a device or a service. Entities have [states](#) and [state attributes](#).

All your entities are listed in the entities table, under **Settings > Devices & services > Entities**.

Screenshot showing the Entities tableScreenshot of the Entities table. Each line represents an entity.

Domains

Each integration in Home Assistant has a unique identifier: a domain. All entities and actions available in Home Assistant are provided by integrations and thus belong to such a domain. The first part of the entity or action, before the `.` shows the domain they belong to. For example, `light.bed_light` is an entity in the light domain. `bed_light` is the ID of the entity.

The domain provides entities, services, and other functionality that other integrations can use. For example, IKEA and Philips Hue both use functionalities provided by the light integration. This is why the look and feel and behavior is similar in Home Assistant.

There are different types of domains: integration domains and entity domains:

- Integration domains provide functionality primarily for itself: examples are Hue, Matter, or Zigbee.
- Entity domains don't use their own functionality as such. But they provide it for other integrations to use.

The integrations listed below are used as entity domains. They are also referred to as *building block integrations* or *entity integrations*:

```
{%- for integration in site.integrations %} {%- if integration.ha_integration_type == "entity" %}
{%- assign name = integration.title -%} {%- assign target = integration.ha_domain | prepend:
"/integrations/" -%}
* {%- endif -%} {%- endfor %}
```

Docs/Configuration/Events

The core of Home Assistant is the event bus. The event bus allows any integration to fire or listen for events.

Events and state changes

All entities produce state change events. Every time a state changes, a state change event is produced. State change events are just one type of event on the event bus, but there are other kinds of events, such as the [built-in events](#) that are used to coordinate between various integrations.

State change events versus event entity

State change events are not to be confused with the [event entity](#). The event entity is a specific type of entity that itself produces event state changes, just like all other entities.

Any state change will be announced on the event bus as a `state_changed` event, containing the previous and the new state of an entity.

Common fields

All events share these basic fields.

Field	Description
<code>event_type</code>	Type of the event. Example: <code>call_service</code> .
<code>origin</code>	Origin of the event. <code>REMOTE</code> (coming in from the API, such as a webhook) or <code>LOCAL</code> (everything else).
<code>time_fired</code>	When the event was fired. Example: <code>2022-01-28T12:19:53.736380+00:00</code> .
<code>context</code>	Dictionary with the <code>context</code> . Example: <code>{ 'id': '123', "parent_id": null, 'user_id': 'abc' }</code> .

In addition, all events contain a `data` dictionary with event-specific information. These are described below.

Built-in Events (core)

`call_service`

This event is fired when a service action is performed

Field	Description
<code>domain</code>	Domain of the action. Example: <code>light</code> .
<code>service</code>	The service action that is performed. Example: <code>turn_on</code>
<code>service_data</code>	Dictionary with the call parameters. Example: <code>{ 'brightness': 120 }</code> .
<code>service_call_id</code>	String with a unique call id. Example: <code>23123-4</code> .

`component_loaded`

This event is fired when a new integration has been loaded and initialized.

Please note that while this event is fired for each loaded integration during Home Assistant startup, the automation engine of Home Assistant is started last. Thus, this event can not be used to run automations during startup as it would have missed these events.

Field	Description
<code>component</code>	Domain of the integration that has just been initialized. Example: <code>light</code> .

`core_config_updated`

This event is fired when the core configuration is updated, for example when the location has been changed.

It contains no additional data.

`data_entry_flow progressed`

This event is fired when a data entry flow has changed and is used by the frontend to reload the flow state.

Field	Description
<code>handler</code>	The flow handler.
<code>flow_id</code>	Identification of the flow.

`homeassistant_start`, `homeassistant_started`

These events are fired during the startup of Home Assistant, in the following order:

1. `homeassistant_start`
2. `homeassistant_started`

These events contain no additional data.

If you want to trigger automation on a Home Assistant start event, we recommend using the special [Home Assistant trigger](#) instead of listening to these events.

`homeassistant_stop`, `homeassistant_final_write`, `homeassistant_close`

These events are fired during the shutdown of Home Assistant, in the following order:

1. `homeassistant_stop`
2. `homeassistant_final_write`
3. `homeassistant_close`

These events contain no additional data.

Please note that `homeassistant_final_write` and `homeassistant_close`, cannot be used with automations, as the automation engine would already have been stopped when those are fired.

If you want to trigger automation on a Home Assistant stop event, we recommend using the special [Home Assistant trigger](#) instead of listening to these events.

`logbook_entry`

Field	Description
<code>name</code>	Name of the entity. Example: <code>Kitchen light</code> .
<code>message</code>	Message. Example: <code>was turned on</code>
<code>domain</code>	Optional, domain of the entry. Example: <code>light</code>
<code>entity_id</code>	Optional, identifier of the entity that was logged.

`service_registered`

This event is fired when a new service action has been registered within Home Assistant.

Field	Description
<code>domain</code>	The domain of the integration that offers this action. Example: <code>light</code> .
<code>service</code>	The name of the service action. Example: <code>turn_on</code>

`service_removed`

This event is fired when a service action has been removed from Home Assistant.

Field	Description
<code>domain</code>	The domain of the integration that offers this action. Example: <code>light</code> .
<code>service</code>	The name of the service action. Example: <code>turn_on</code> .

`state_changed`

This event is fired when a state has changed. It contains the entity identifier and both the `new_state` and `old_state` of the entity as [state objects](#).

Field	Description
<code>entity_id</code>	Identifier of the entity that has changed. Example: <code>light.kitchen</code> .
<code>old_state</code>	The previous state of the entity before it changed. Omitted if the state is set for the first time.
<code>new_state</code>	The new state of the entity. Omitted if the state has been removed.

`themes_updated`

This event is fired after a theme has been set or reloaded. It contains no additional data.

`user_added`

This event is fired when a user has been added.

Field	Description
<code>user_id</code>	Identification of the new user.

`user_removed`

This event is fired when a user has been removed.

Field	Description
<code>user_id</code>	Identification of the removed user.

Built-in events (default integrations)

`automation_reloaded`

Integration: `automation`

This event is fired when automations have been reloaded and thus might have changed.

This event contains no additional data.

`automation_triggered`

Integration: `automation`

This event is fired when an automation is triggered.

Field	Description
<code>name</code>	The name of the automation.
<code>entity_id</code>	The identifier of the automation.

`scene_reloaded`

Integration: `homeassistant`

This event is fired when scenes have been reloaded and thus might have changed.

This event contains no additional data.

`script_started`

Integration: `script`

This event is fired when a script is run. A script can be invoked by a user or triggered by an automation. The resulting changes can be tracked because all related events will share the same context as this event.

Field	Description
<code>name</code>	Name of the script that was run.
<code>entity_id</code>	Identifier of the script that was run.

Docs/Configuration/Packages

Packages in Home Assistant provide a way to bundle configurations from multiple integrations. You can use packages to include multiple integrations, or parts of integrations, using any of the `!include` directives introduced in [splitting the configuration](#).

Packages are configured under the core `homeassistant/packages` in the configuration and take the format of a package name (no spaces, all lowercase) followed by a dictionary with the package configuration. For example, package `pack_1` would be created as:

```
homeassistant:
  ...
  packages:
    pack_1:
      ...package configuration here...
```

The package configuration can include: `switch`, `light`, `automation`, `groups`, or most other Home Assistant integrations including hardware platforms.

It can be specified inline or in a separate YAML file using `!include`.

Inline example, main `configuration.yaml`:

```
homeassistant:
  ...
  packages:
    pack_1:
      switch:
        - platform: rest
        ...
      light:
        - platform: rpi
        ...
```

Include example, main `configuration.yaml`:

```
homeassistant:
  ...
  packages:
    pack_1: !include my_package.yaml
```

The file `my_package.yaml` contains the "top-level" configuration:

```
switch:
  - platform: rest
  ...
light:
  - platform: rpi
  ...
```

There are some rules for packages that will be merged:

1. Platform based integrations (`light` , `switch` , etc) can always be merged.
2. Integrations where entities are identified by a key that will represent the `entity_id` (`{key: config}`) need to have unique 'keys' between packages and the main configuration file.

For example, if we have the following in the main configuration. You are not allowed to re-use "my_input" again for `input_boolean` in a package:

```
yaml input_boolean: my_input:
```

3. Any integration that is not a platform [1], or dictionaries with Entity ID keys [2] can only be merged if its keys, except those for lists, are solely defined once.

 **Tip:** Integrations inside packages can only specify platform entries using configuration style 1, where all the platforms are grouped under the integration name.

Create a packages folder

One way to organize packages is to create a folder named "packages" in your Home Assistant configuration directory. In this packages folder, you can store any number of packages in YAML files, and organize those packages into YAML files and subfolders as you see fit. With `!include_dir_named`, the filename is used as the package name. This means that file names must be globally unique, even across subfolders. This entry in your `configuration.yaml` will load all YAML-files in this *packages* folder and its sub folders:

```
homeassistant:
  packages: !include_dir_named packages
```

The benefit of this approach is to pull all configurations required to integrate a system into one file, rather than keeping them spread across several files.

You can also use `!include_dir_merge_named` for packages. The two directives differ in how they handle the file contents. With `!include_dir_named`, each file's content is placed directly under the package name (which is the filename), so the content uses the same indentation as it would inside the `packages:` key in `configuration.yaml`. This means you can copy and paste elements from the main config file. With `!include_dir_merge_named`, the package name has to be the top-level key inside the file, so the content needs an additional level of indentation and you cannot directly copy and paste from the configuration file.

```
homeassistant:
  packages: !include_dir_merge_named packages
```

and in `packages/subsystem1/functionality1.yaml`:

```
subsystem1_functionality1:
  input_boolean:
  ...
  binary_sensor:
  ...
  automation:
```


Customizing entities with packages

It is possible to [customize entities](#) within packages. Just create your customization entries under:

```
homeassistant:  
  customize:
```

⚠ Important: If you are moving configuration to packages, `auth_providers` must stay within your `configuration.yaml` file. See the general documentation for [Authentication Providers](#).

This is because Home Assistant processes the `auth_providers` configuration early during startup, before packages are processed.

Docs/Configuration/Platform Options

 **Important:** These options are being phased out and are only available for single platform integrations.

Some integrations or platforms (those that are based on the [entity](#) class) allow various extra options to be set.

Entity namespace

By setting an entity namespace, all entities will be prefixed with that namespace. That way, `light.bathroom` can become `light.holiday_house_bathroom`.

```
# Example configuration.yaml entry
light:
  - platform: your_lights
    entity_namespace: holiday_house
```

Scan interval

Platforms that require polling will be polled in an interval specified by the main integration. For example, a light will check every 30 seconds for a changed state. It is possible to overwrite this scanning interval for any platform that is being polled by specifying a `scan_interval` configuration key. In the example below, we set up the `your_lights` platform but tell Home Assistant to poll the devices every 10 seconds instead of the default 30 seconds.

```
# Example configuration.yaml entry to poll your_lights every 10 seconds.
light:
  - platform: your_lights
    scan_interval: 10
```

Docs/Configuration/Remote

By default, your Home Assistant only listens on your local network, which keeps things private and secure. If you want to reach it from outside your home, for example to control your devices while you are at work or on holiday, you have a few options.

The easiest and safest option for most people is [Home Assistant Cloud](#). Other options are listed further down for those who prefer to set things up themselves.

 **Tip:** Before exposing Home Assistant to the internet, follow the [securing checklist](#).

Home Assistant Cloud

[Home Assistant Cloud](#) gives you remote access to your Home Assistant from anywhere, without opening any ports on your router and without exposing your home network to the internet. Setup takes a single toggle in the user interface.

A unique remote URL is generated for you, and all traffic between your device and your home is encrypted automatically. Your Home Assistant Cloud subscription also helps fund the development of Home Assistant itself.

VPN

A secure way to remotely access your Home Assistant is to use a Virtual Private Network (VPN) service such as [Tailscale](#) or [ZeroTier One](#).

A VPN connection needs to be established before you can connect to your Home Assistant from outside your local network. The VPN makes this connection secure. When using the Home Assistant Companion app (such as on a mobile device), without this connection, your sensors will not update in Home Assistant.

Port forwarding

Set up port forwarding (for any port) from your router to port 8123 on the computer that is hosting Home Assistant. General instructions on how to do this can be found by searching `<router model> port forwarding instructions`. You can use any free port on your router and forward that to port 8123.

A problem with making a port accessible is that some Internet Service Providers only offer dynamic IPs. This can cause you to lose access to Home Assistant while away. You can solve this by using a free Dynamic DNS service like [DuckDNS](#).

If you cannot access your Home Assistant installation remotely, remember to check if your ISP provides you with a dedicated IP, instead of one shared with other users via a [CG-NAT](#). This is becoming fairly common nowadays due to the shortage of IPv4 addresses. Some, if not most ISPs will require you to pay an extra fee to be assigned a dedicated IPv4 address.

◆ **Caution:** Just putting a port up is not secure. You should definitely consider encrypting your traffic if you are accessing your Home Assistant installation remotely. For details, please check the [set up encryption using Let's Encrypt](#) blog post or this [detailed guide](#) to using Let's Encrypt with Home Assistant.

Adding a remote URL to Home Assistant

To set the URL under which your Home Assistant can be accessed from outside your local network, follow these steps:

1. In the bottom left, select your username to go to your **User profile**, and make sure **Advanced mode** is enabled.
 2. Go to **Settings > System > Network**.
 3. Under **Home Assistant URL**, enter the external URL that you previously set up for your instance.
-

Docs/Configuration/Secrets

The `configuration.yaml` file is a plain-text file, thus it is readable by anyone who has access to the file. The file contains passwords and API tokens which need to be redacted if you want to share your configuration.

By using `!secret` you can remove any private information from your configuration files. This separation can also help you to keep easier track of your passwords and API keys, as they are all stored at one place and no longer spread across the `configuration.yaml` file or even multiple YAML files if you [split up your configuration](#).

Using `secrets.yaml`

The workflow for moving private information to `secrets.yaml` is very similar to the [splitting of the configuration](#). Create a `secrets.yaml` file in your Home Assistant [configuration directory](#).

The entries for password and API keys in the `configuration.yaml` file usually looks like the example below.

```
rest:
  - authentication: basic
    username: "admin"
    password: "YOUR_PASSWORD"
    ...
```

Those entries need to be replaced with `!secret` and an identifier.

```
rest:
  - authentication: basic
    username: "admin"
    password: !secret rest_password
    ...
```

The `secrets.yaml` file contains the corresponding password assigned to the identifier.

```
rest_password: "YOUR_PASSWORD"
```

Debugging secrets

When you start splitting your configuration into multiple files, you might end up with configuration in sub folders. Secrets will be resolved in this order:

- A `secrets.yaml` located in the same folder as the YAML file referencing the secret,
- next, parent folders will be searched for a `secrets.yaml` file with the secret, stopping at the folder with the main `configuration.yaml`.

To see where secrets are being loaded from, you can add an option to your `secrets.yaml` file.

Print where secrets are retrieved from to the Home Assistant log by adding the following to `secrets.yaml`:

```
logger: debug
```

This will not print the actual secret's value to the log.

Docs/Configuration/Securing

Home Assistant runs on your own hardware and does not depend on any cloud service to work, which already removes a large category of risks that come with internet-connected smart home platforms. Even so, there are a few simple steps you should take to keep your Home Assistant secure, especially if you plan to access it from outside your home network.

Checklist

The most important things to do to keep your Home Assistant secure:

- Centralize sensitive data in `secrets` (and remember to back them up).
- **Note:** Storing secrets in `secrets.yaml` does not encrypt them.
- Keep your system up to date with each monthly release.

Remote access

If you want secure remote access, the easiest option is to use [Home Assistant Cloud](#) by which you also support the [Open Home Foundation](#), which develops Home Assistant, ESPHome and much more.

Another option is to use TLS/SSL via the app [Duck DNS](#) integrating Let's Encrypt.

To expose your instance to the internet, use a [VPN](#), or an [SSH tunnel](#). Make sure to expose the used port in your router.

Extras for manual installations

Besides the above, we advise that you consider the following to improve security:

- For systems that use SSH, set `PermitRootLogin no` in your sshd configuration (usually `/etc/ssh/sshd_config`) and use SSH keys for authentication instead of passwords. This is particularly important if you enable remote access to your SSH services.
 - Lock down the host following good practice guidance, for example:
 - [Securing Debian Manual](#) (this also applies to Raspberry Pi OS)
 - [Red Hat Enterprise Linux 7 Security Guide](#), [CIS Red Hat Enterprise Linux 7 Benchmark](#)
-

Docs/Configuration/Splitting Configuration

So you've been using Home Assistant for a while now and your `configuration.yaml` file brings people to tears because it has become so large. Or, you simply want to start off with the distributed approach. Here's how to split the `configuration.yaml` into more manageable (read: human-readable) pieces.

Example configuration files for inspiration

First off, several community members have sanitized (read: without API keys/passwords) versions of their configurations available for viewing. You can see a [list of example configuration on GitHub](#).

As commenting code doesn't always happen, please read on to learn in detail how configuration files can be structured.

Analyzing the configuration files

In this section, we are going to use some example configuration files and look at their structure and format in more detail.

Now you might think that the `configuration.yaml` will be replaced during the splitting process. However, it will in fact remain, albeit in a much less cluttered form.

The core configuration file

In this lighter version, we will still need what could be called the core snippet:

```
homeassistant:
  # Name of the location where Home Assistant is running
  name: "My Home Assistant Instance"
  # Location required to calculate the time the sun rises and sets
  latitude: 37
  longitude: -121
  # 'metric' for Metric, 'us_customary' for US Customary
  unit_system: us_customary
  # Pick yours from here: https://en.wikipedia.org/wiki/List_of_tz_database_time_zones
  time_zone: "America/Los_Angeles"
  customize: !include customize.yaml
```

Indentation, includes, comments, and modularization

Note that each line after `homeassistant:` is indented two (2) spaces. Since the configuration files in Home Assistant are based on the YAML language, indentation and spacing are important. Also note that seemingly strange entry under `customize:`.

`!include customize.yaml` is the statement that tells Home Assistant to insert the parsed contents of `customize.yaml` at that point. The contents of the included file must be YAML data that is valid at the location it is included. This is how we are going to break a monolithic and hard-to-read file (when it gets big) into more manageable chunks.

For example, `customize.yaml` could contain the following:

```
light.living_room:
  friendly_name: "Living Room"
  icon: mdi:ceiling-light

switch.patio:
  friendly_name: "Patio Switch"
```

Notice that `customize:` is not written again inside `customize.yaml`. The key in `configuration.yaml` already defines the context — the included file only contains the entries that belong under it.

Now before we start splitting out the different components, let's look at the other integrations (in our example) that will stay in the base file:

```

history:
frontend:
logbook:
http:
  api_password: "ImNotTelling!"

ifttt:
  key: ["nope"]

mqtt:
  sensor:
    - name: "test sensor 1"
      state_topic: "test/some_topic1"
    - name: "test sensor 2"
      state_topic: "test/some_topic2"

```

As with the core snippet, indentation makes a difference:

- The integration headers (`mqtt:`) should be fully left aligned (aka no indent).
- The key (`sensor:`) should be indented two (2) spaces.
- The list `-` under the key `sensor` should be indented another two (2) spaces followed by a single space.
- The `mqtt` sensor list contains two (2) configurations, with two (2) keys each.

Comments

The `#` symbol (hash/pound) represents a "comment" as far as the commands are interpreted. Put another way, any line prefixed with a `#` will be ignored by the software. It is for humans only. Comments allow breaking up files for readability, as well as turning off features while leaving the entry intact.

Modularization and granularity

While some of these integrations could technically be moved to a separate file, they are so small or "one off's" where splitting them off is superfluous.

Now, lets assume that a blank file has been created in the Home Assistant configuration directory for each of the following:

```

automation.yaml
zone.yaml
sensor.yaml
switch.yaml
device_tracker.yaml
customize.yaml

```

`automation.yaml` will hold all the automation integration details. `zone.yaml` will hold the zone integration details and so forth. These files can be called anything but giving them names that match their function will make things easier to keep track of.

Inside the base configuration file, add the following entries:

```
automation: !include automation.yaml
zone: !include zone.yaml
sensor: !include sensor.yaml
switch: !include switch.yaml
device_tracker: !include device_tracker.yaml
```

Include statements and packages to split files

Nesting `!include` statements (having an `!include` within a file that is itself `!include`d) will also work.

Some integrations support multiple top-level `!include` statements. This includes integrations defining an IoT domain. For example, `light`, `switch`, or `sensor`; as well as the `automation`, `script`, and `template` integrations, if you give a different label to each one.

Configuration for other integrations can instead be split up by using packages. To learn more about packages, see the [Packages](#) page.

Top level keys

Example of multiple top-level keys for the `light` platform.

```
light:
- platform: group
  name: "Bedside Lights"
  entities:
    - light.left_bedside_light
    - light.right_bedside_light

# define more light groups in a separate file
light groups: !include light-groups.yaml

# define some light switch mappings in a different file
light switches: !include light-switches.yaml
```

where `light-groups.yaml` might look like:

```
- platform: group
  name: "Outside Lights"
  entities:
    - light.porch_lights
    - light.patio_lights
...

with `light-switches.yaml` containing:

```yaml
- platform: switch
 name: "Patio Lights"
 entity_id: switch.patio_lights

- platform: switch
 name: "Floor Lamp"
 entity_id: switch.floor_lamp_plug
```

Alright, so we've got the single integrations and the include statements in the base file, what goes in those extra files?

Let's look at the `device_tracker.yaml` file from our example:

```
- platform: owntracks
- platform: nmap_tracker
 home_interval: 3
 hosts: 192.168.2.0/24

 track_new_devices: true
 interval_seconds: 40
 consider_home: 120
```

This small example illustrates how the "split" files work. In this case, we start with two (2) device tracker entries ( `owntracks` and `nmap` ). These files follow "style 1" that is to say a fully left aligned leading entry ( `- platform: owntracks` ) followed by the parameter entries indented two (2) spaces.

This (large) sensor configuration gives us another example:

```
sensor.yaml
METEOBRIDGE
- platform: tcp
 name: "Outdoor Temp (Meteobridge)"
 host: 192.168.2.82
 timeout: 6
 payload: "Content-type: text/xml; charset=UTF-8\n\n"
 value_template: "{{value.split (' ')[2]}}"
 unit: C
- platform: tcp
 name: "Outdoor Humidity (Meteobridge)"
 host: 192.168.2.82
 port: 5556
 timeout: 6
 payload: "Content-type: text/xml; charset=UTF-8\n\n"
 value_template: "{{value.split (' ')[3]}}"
 unit: Percent

STEAM FRIENDS
- platform: steam_online
 api_key: ["not telling"]
 accounts:
 - 76561198012067051

TIME/DATE
- platform: time_date
 display_options:
 - "time"
 - "date"
- platform: worldclock
 time_zone: Etc/UTC
 name: "UTC"
- platform: worldclock
 time_zone: America/New_York
 name: "Ann Arbor"
```

You'll notice that this example includes a secondary parameter section (under the steam section) as well as a better example of the way comments can be used to break down files into sections.

All of the above can be applied when splitting up files using packages. To learn more about packages, see the [Packages](#) page.

That about wraps it up.

If you have issues, check the file indentations and check [the Home Assistant logs](#). If all else fails, head over to our [Discord chat server](#) and ask away.

## Debugging configuration files

---

If you have many configuration files, Home Assistant provides a CLI that allows you to see how it interprets them. Each installation type has its own section in the common-tasks about this:

- [Operating System](#)
- [Container](#)

## Advanced usage

We offer four advanced options to include whole directories at once. Please note that your files must have the `.yaml` file extension; `.yml` is not supported.

This will allow you to `!include` files with `.yaml` extensions from within the `.yaml` files; without those `.yaml` files being imported by the following commands themselves.

- `!include_dir_list` will return the content of a directory as a list with each file content being an entry in the list. The list entries are ordered based on the alphanumeric ordering of the names of the files.
- `!include_dir_named` will return the content of a directory as a dictionary which maps filename => content of file.
- `!include_dir_merge_list` will return the content of a directory as a list by merging all files (which should contain a list) into 1 big list.
- `!include_dir_merge_named` will return the content of a directory as a dictionary by loading each file and merging it into 1 big dictionary.

These work recursively. As an example using `!include_dir_list automation`, will include all 6 files shown below:

```
.
├── .homeassistant
│ ├── automation
│ │ ├── lights
│ │ │ ├── turn_light_off_bedroom.yaml
│ │ │ ├── turn_light_off_lounge.yaml
│ │ │ ├── turn_light_on_bedroom.yaml
│ │ │ └── turn_light_on_lounge.yaml
│ │ ├── say_hello.yaml
│ │ └── sensors
│ │ └── react.yaml
│ └── configuration.yaml (not included)
```

**Example:** `!include_dir_list`

```
configuration.yaml
```



```

automation:
 - alias: "Automation 1"
 triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 to: "home"
 actions:
 - action: light.turn_on
 target:
 entity_id: light.entryway
 - alias: "Automation 2"
 triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 from: "home"
 actions:
 - action: light.turn_off
 target:
 entity_id: light.entryway

```

can be turned into:

`configuration.yaml`

```
automation: !include_dir_list automation/presence/
```

`automation/presence/automation1.yaml`

```

alias: "Automation 1"
triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 to: "home"
actions:
 - action: light.turn_on
 target:
 entity_id: light.entryway

```

`automation/presence/automation2.yaml`

```

alias: "Automation 2"
triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 from: "home"
actions:
 - action: light.turn_off
 target:
 entity_id: light.entryway

```

It is important to note that each file must contain only **one** entry when using `!include_dir_list`.

## Example: `!include_dir_named`

`configuration.yaml`

```
alexa:
 intents:
 LocateIntent:
 actions:
 action: notify.pushover
 data:
 message: "Your location has been queried via Alexa."
 speech:
 type: plaintext
 text: >
 {%- for state in states.device_tracker -%}
 {%- if state.name.lower() == User.lower() -%}
 {{ state.name }} is at {{ state.state }}
 {%- endif -%}
 {%- else -%}
 I am sorry. Pootie! I do not know where {{User}} is.
 {%- endfor -%}
 WhereAreWeIntent:
 speech:
 type: plaintext
 text: >
 {%- if is_state('device_tracker.iphone', 'home') -%}
 iPhone is home.
 {%- else -%}
 iPhone is not home.
 {% endif %}
```

can be turned into:

`configuration.yaml`

```
alexa:
 intents: !include_dir_named alexa/
```

`alexa/LocateIntent.yaml`

```
actions:
 action: notify.pushover
 data:
 message: "Your location has been queried via Alexa."
speech:
 type: plaintext
 text: >
 {%- for state in states.device_tracker -%}
 {%- if state.name.lower() == User.lower() -%}
 {{ state.name }} is at {{ state.state }}
 {%- endif -%}
 {%- else -%}
 I am sorry. Pootie! I do not know where {{User}} is.
 {%- endfor -%}
```

`alexa/WhereAreWeIntent.yaml`

```
speech:
 type: plaintext
 text: >
 {%- if is_state('device_tracker.iphone', 'home') -%}
 iPhone is home.
 {%- else -%}
 iPhone is not home.
 {% endif %}
```

### Example: `!include_dir_merge_list`

`configuration.yaml`

```
automation:
- alias: "Automation 1"
 triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 to: "home"
 actions:
 - action: light.turn_on
 target:
 entity_id: light.entryway
- alias: "Automation 2"
 triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 from: "home"
 actions:
 - action: light.turn_off
 target:
 entity_id: light.entryway
```

can be turned into:

`configuration.yaml`

```
automation: !include_dir_merge_list automation/
```

`automation/presence.yaml`

```

- alias: "Automation 1"
 triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 to: "home"
 actions:
 - action: light.turn_on
 target:
 entity_id: light.entryway
- alias: "Automation 2"
 triggers:
 - trigger: state
 entity_id: device_tracker.iphone
 from: "home"
 actions:
 - action: light.turn_off
 target:
 entity_id: light.entryway

```

It is important to note that when using `!include_dir_merge_list`, you must include a list in each file (each list item is denoted with a hyphen [-]). Each file may contain one or more entries.

### Example: `!include_dir_merge_named`

`configuration.yaml`

```

group:
 bedroom:
 name: "Bedroom"
 entities:
 - light.bedroom_lamp
 - light.bedroom_overhead
 hallway:
 name: "Hallway"
 entities:
 - light.hallway
 - thermostat.home
 front_yard:
 name: "Front Yard"
 entities:
 - light.front_porch
 - light.security
 - light.pathway
 - sensor.mailbox
 - camera.front_porch

```

can be turned into:

`configuration.yaml`

```
group: !include_dir_merge_named group/
```

`group/interior.yaml`

```

bedroom:
 name: "Bedroom"
 entities:
 - light.bedroom_lamp
 - light.bedroom_overhead
hallway:
 name: Hallway
 entities:
 - light.hallway
 - thermostat.home

```

group/exterior.yaml

```

front_yard:
 name: "Front Yard"
 entities:
 - light.front_porch
 - light.security
 - light.pathway
 - sensor.mailbox
 - camera.front_porch

```

### Example: Combine `!include_dir_merge_list` with `automations.yaml`

You want to go the advanced route and split your automations, but still want to be able to create automations in the UI? In a chapter above we write about nesting `!includes`. Here is how we can do that for automations.

Using labels like `manual` or `ui` allows for using multiple keys in the config:

configuration.yaml

```

My own handmade automations
automation manual: !include_dir_merge_list automations/

Automations I create in the UI
automation ui: !include automations.yaml

```

## Docs/Configuration/State Object

---

Devices are represented in Home Assistant as entities. The state of an entity (for example, if a light is on, at 50% brightness in orange) can be shown on the dashboard or be used in automations. This page looks at the concepts *state*, *state object*, and *entity state attribute*.

## State versus state object

In Home Assistant, the state object is the current representation of the entity with all its attributes at a given moment in time. This state is recorded as a *state object*. Entities constantly keep track of their state and write it into a state object, so that other entities/templates/frontend can access it. In the example—the light is on, at 50% brightness in orange—*on* is the actual state of the light. 50% brightness and the color are entity state attributes.

### About the state object

The state object represents the state of an entity with its attributes at a specific point in time. All state objects will always have an entity id, a state, and timestamps when last updated, last changed, and last reported. The `state` prefix indicates that this information is part of the state object (which is related to the entity). For example, `state.state` is the state of the entity at a given time.

Field	Description
<code>state.state</code>	String representation of the current state of the entity. Example <code>off</code> .
<code>state.entity_id</code>	Entity ID. Format: <code>&lt;domain&gt;.&lt;object_id&gt;</code> . Example: <code>light.kitchen</code> .
<code>state.domain</code>	Domain of the entity. Example: <code>light</code> .
<code>state.object_id</code>	Object ID of entity. Example: <code>kitchen</code> .
<code>state.name</code>	Name of the entity. Based on <code>friendly_name</code> attribute with fall back to object ID. Example: <code>Kitchen ceiling</code> .
<code>state.last_changed</code>	Time the state changed in the state machine in UTC time. This is not updated if only state attributes change. Example: <code>2013-09-17 07:32:51.715874+00:00</code> .
<code>state.last_reported</code>	Time the state was written to the state machine in UTC time. This timestamp is updated regardless of any changes to the state or state attributes. Example: <code>2013-09-17 07:32:51.715874+00:00</code> .
<code>state.last_updated</code>	Time the state or state attributes changed in the state machine in UTC time. This is not updated if neither state nor state attributes changed. Example: <code>2013-09-17 07:32:51.715874+00:00</code> .
<code>state.attributes</code>	A dictionary with extra attributes related to the current state.
<code>state.context</code>	A dictionary with extra attributes related to the context of the state.

### About the state

The screenshot of the Developer tools States page shows three lights in different states (the `state.state`): `on`, `off`, and `unavailable`. Each light comes with its own entity state

attributes such as `supported_color_modes` , `supported_features` . These attributes have their own state: the state of the `supported_color_modes` attribute is `color_temp` and `hs` , the state of the `supported_features` attribute is `4` .

Screenshot showing three lights with different states: `on`, `off`, or `unavailable` Three lights with different states: `on`, `off`, or `unavailable`.

The `state.state` is the heart of the `state object`. State holds the information of interest of an entity. For example, if a light is on or off, the current temperature, or the amount of energy used. The state object stores 3 timestamps related to the state: `last_updated` , `last_changed` , and `last_reported` . Each entity has exactly one state, and the state only holds one value at a time.

## About entity state attributes

The state only holds one value at a time. However, entities can store related entity state attributes in the state object. For example, the state of a light is *on*, and the related attributes could be its current brightness and color values. `State change events` can be used as triggers. The current state can be used in `conditions`. The example below shows three lights with different entity state attributes.

Screenshot showing three lights with different states and attributes Example showing three lights with different entity state attributes.

Entities have some attributes that are not related to its state, such as `friendly_name` . A few attributes are available on all entities, such as `friendly_name` or `icon` . In addition to those, each integration has its own attributes to represent extra state data about the entity. For example, the light integration has attributes for the current brightness and color of the light. When an attribute is not available, Home Assistant will not write it to the state. Entity attributes are optional.

When using templates, attributes will be available by their name. For example `state.attributes.assumed_state` .

The table lists common state attributes that may be present, depending on the entity domain.



Attribute	Description
<code>friendly_name</code>	Name of the entity. Example: <code>Kitchen Ceiling</code> .
<code>icon</code>	Icon to use for the entity in the frontend. Example: <code>mdi:home</code> .
<code>entity_picture</code>	URL to a picture that should be used instead of showing the domain icon. Example: <code>http://example.com/picture.jpg</code> .
<code>assumed_state</code>	Boolean if the current state is an assumption. <a href="#">More info</a> Example: <code>True</code> .
<code>unit_of_measurement</code>	The unit of measurement the state is expressed in. Used for grouping graphs or understanding the entity. Example: <code>°C</code> .
<code>attribution</code>	The provider of the data. For example, "Data provided by rejseplanen.dk", "Data provided by openSenseMap"
<code>device_class</code>	The type of device that an entity represents. Used to display device specific information in the UI.
<code>supported_features</code>	The features an entity supports. For covers, for example, it might list <code>opening</code> , <code>closing</code> , <code>stopping</code> , <code>setting position</code> . For media players, it might list <code>play</code> , <code>pause</code> , <code>stop</code> , and <code>volume control</code> .

When an attribute contains spaces, you can retrieve it with the `state_attr` function:  
`state_attr('sensor.livingroom', 'Battery numeric')` .

## Context

---

Context is a property used in state objects and events. It ties events and states together in Home Assistant. Whenever an automation or user interaction causes a state to change, a new context is assigned in the state object. This context will be attached to all events and states that happen as a result of the change.

Field	Description
<code>id</code>	Unique identifier for the context.
<code>user_id</code>	Unique identifier of the user that started the change. Will be <code>None</code> if the action was not started by a user (for example, started by an automation).
<code>parent_id</code>	Unique identifier of the parent context that started the change, if available. For example, if an automation is triggered, the context of the trigger will be set as parent.

## Examples

---

- Evaluate the `state.last_changed` of a switch entity:

```
jinja {{ states.switch.my_switch.last_changed }}
```

result type: `string` representing a datetime object, for example

```
2025-11-11 12:56:10.244125+00:00
```

---

- Evaluate the `state.context.id` of this switch:

```
jinja {{ states.switch.my_switch.context.id }}
```

result type: `string` representing an id code, for example

```
01K9SF2R36KRV5N4PTC38S6KJ2F
```

---

- Evaluate the `state.context.user_id` of this switch:

```
``jinja
```

```
{{ states.switch.my_switch.context.user_id }} ``
```

result type: `string` representing a user id code, for example `01K9SF2R36KRV5N4PTC38SKS4LW6`

---

## Docs/Configuration/Troubleshooting

---

It can happen that you run into trouble while configuring Home Assistant. Perhaps an integration is not showing up or is acting strangely. This page will discuss a few of the most common problems.

Before we dive into common issues, make sure you know where your configuration directory is. Home Assistant will print out the configuration directory it is using when starting up.

Whenever an integration or configuration option results in a warning, it will be stored in [the logs](#).

## My integration does not show up

---

When an integration does not show up, many different things can be the case. Before you try any of these steps, make sure to look at the [the logs](#) and see if there are any errors related to your integration you are trying to set up.

If you have incorrect entries in your configuration files you can use the configuration check command (below) to assist in identifying them.

### Problems with the configuration

One of the most common problems with Home Assistant is an invalid `configuration.yaml` or other configuration file.

- Home Assistant provides a CLI that allows you to see how it interprets them, each installation type has its own section in the common-tasks about this:
- [Operating System](#)
- [Container](#)
- The configuration files, including `configuration.yaml` must be UTF-8 encoded. If you see error like `'utf-8' codec can't decode byte`, edit the offending configuration and re-save it as UTF-8.
- You can verify your configuration's YAML structure using [this online YAML parser](#) or [YAML Validator](#).
- To learn more about the quirks of YAML, read [YAML IDIOSYNCRASIES](#) by SaltStack (the examples there are specific to SaltStack, but do explain YAML issues well).

`configuration.yaml` does not allow multiple sections to have the same name. If you want to load multiple platforms for one integration, you can append a number or string to the name or nest them:

```
sensor:
 - platform: forecast
 ...
 - platform: bitcoin
 ...
```

Another common problem is that a required configuration setting is missing. If this is the case, the integration will report this in [the logs](#). You can have a look at [the various integration pages](#) for instructions on how to setup the integrations.

See the [logger](#) integration for instructions on how to define the level of logging you require for specific modules.

If you find any errors or want to expand the documentation, please [let us know](#).

## Problems with dependencies

Almost all integrations have external dependencies to communicate with your devices and services. Sometimes Home Assistant is unable to install the necessary dependencies. If this is the case, it should show up in [the logs](#).

The first step is trying to restart Home Assistant and see if the problem persists. If it does, look at the log to see what the error is. If you can't figure it out, please [report it](#) so we can investigate what is going on.

## Problems with integrations

It can happen that some integrations either do not work right away or stop working after Home Assistant has been running for a while. If this happens to you, please [report it](#) so that we can have a look.

## Multiple files

If you are using multiple files for your setup, make sure that the pointers are correct and the format of the files is valid. It's important to understand the different types of `!include` and how the contents of each file should be structured - more information on the various methods of splitting your configuration into multiple files can be found [here](#).

```
light: !include devices/lights.yaml
sensor: !include devices/sensors.yaml
```

Contents of `lights.yaml` (notice it does not contain `light:`):

```
- platform: hyperion
 host: 192.168.1.98
 ...
```

Contents of `sensors.yaml`:

```
- platform: mqtt
 name: "Room Humidity"
 state_topic: "room/humidity"
- platform: mqtt
 name: "Door Motion"
 state_topic: "door/motion"
 ...
```



**Note:** Whenever you report an issue, be aware that we are volunteers who do not have access to every single device in the world nor unlimited time to fix every problem out there.

## Entity names

The only characters valid in entity names are:

- Lowercase letters
- Numbers
- Underscores

The entity name must not start or end with an underscore. If you create an entity with other characters from the UI, Home Assistant validates the name. If you change the name directly in the YAML file, then Home Assistant may not generate an error for that entity. However, attempts to use that entity will generate errors (or possibly fail silently).

For instructions on how to change an entity name, refer to the section on [customizing entities](#).

## Debug logs and diagnostics

---

The first thing you will need before reporting an issue online is debug logs and diagnostics (if available) for the integration giving you trouble. Getting those ahead of time will ensure someone can help resolve your issue in the fastest possible manner.

### Enabling debug logging

To enable debug logging for a specific integration, follow these steps:

1. Go to **Settings > Devices & services**.
2. Select the integration for which you want to enable debug logging.
3. In the top right of the page, open the three dots `mdi:dots-vertical` menu, and select **Enable debug logging**.

Screenshot showing the Enable debug logging button on an integration detail page  
Screenshot showing the **Enable debug logging** menu item.

4. To see the error in the logs, you need to reproduce the error. - Run your automation, change up your device or whatever was giving you an error.
5. Continue with the steps on [disabling debug logging and download logs](#).

### Disable debug logging and download logs

1. Go to **Settings > Devices & services**.
2. Select the integration for which you want to disable debug logging.
3. In the top right of the page, open the three dots `mdi:dots-vertical` menu, and select **Disable debug logging**.
4. After you disable it, you will be prompted to download your log file.
5. Save this to a safe location to upload later.

### Download diagnostics

After you download logs, you will also want to download the diagnostics for the integration giving you trouble. Not all integrations provide diagnostics, but if they do, you can download them by following these steps:

1. Go to **Settings > Devices & services**.
2. Select the integration.
3. In the top right of the integration card, open the three dots `mdi:dots-vertical` menu, and select **Download diagnostics**.

Example of Download Diagnostics Example showing the **Download diagnostics** button on the Zigbee integration card.



## Handling unexpected restarts or crashes

Suppose you find that Home Assistant unexpectedly restarts or crashes; it's likely that you have a misbehaving integration impacting system stability. Home Assistant has a built-in debug option that can help find implementation errors. It can also block many unsafe thread operations from crashing the system. Enabling debug has a slight performance impact on the system and is not recommended for long-term use. To enable debug, add the following to your

`configuration.yaml`:

```
homeassistant:
 debug: true
```

Once debug is enabled, periodically check [Home Assistant System Logs](#) for new messages.

---

## Docs/Configuration/Yaml

---

Most things in Home Assistant can be set up directly from the user interface, and you never need to touch a YAML file. A small number of advanced features and a few integrations still require entries in `configuration.yaml`, and this page explains the bit of YAML syntax you need for those cases.

## YAML style guide

---

This page gives a high-level introduction to the YAML syntax used in Home Assistant. For a more detailed description and more examples, refer to the [YAML style guide for Home Assistant developers](#).

## A first example

---

The following YAML example entry assumes that you would like to set up the [notify integration](#) with the [pushbullet platform](#).

```
notify:
 platform: pushbullet
 api_key: "o.1234abcd"
 name: pushbullet
```

- An **integration** provides the core logic for some functionality (like `notify` provides sending notifications).
- A **platform** makes the connection to a specific software or hardware platform (like `pushbullet` works with the service from pushbullet.com).

The basics of YAML syntax are block collections and mappings containing key-value pairs. Each item in a collection starts with a `-` while mappings have the format `key: value`. This is somewhat similar to a Hash table or more specifically a dictionary in Python. These can be nested as well. **Beware that if you specify duplicate keys, the last value for a key is used.**

## Indentation in YAML

---

In YAML, indentation is important for specifying relationships. Indented lines are nested inside lines that are one level higher. In the above example, `platform: pushbullet` is a property of (nested inside) the `notify` integration.

Getting the right indentation can be tricky if you're not using an editor with a fixed-width font. Tabs are not allowed to be used for indentation. The convention is to use 2 spaces for each level of indentation.

## Comments

---

Strings of text following a `#` are comments. They are ignored by the system. Comments explain in plain language what a particular code block is supposed to do. For future-you or someone else looking at the file.

### Example with comment and nesting

The next example shows an `input_select` integration that uses a block collection for the values of options. The other properties (like `name:` ) are specified using mappings. Note that the second line just has `threat:` with no value on the same line. Here, `threat` is the name of the `input_select`. The values for it are everything nested below it.

```
input_select:
 threat:
 name: "Threat level"
 # A collection is used for options
 options:
 - 0
 - 1
 - 2
 - 3
 initial: 0
```

### Example of nested mapping

The following example shows nesting a collection of mappings in a mapping. In Home Assistant, this would create two sensors that each use the MQTT platform but have different values for their `state_topic` (one of the properties used for MQTT sensors).

```
sensor:
 - platform: mqtt
 state_topic: "sensor/topic"
 - platform: mqtt
 state_topic: "sensor2/topic"
```

## Including values

---

### Environment variables

On Home Assistant Core installations, you can include values from your system's environment variables with `!env_var`. Note that this will only work for Home Assistant Core installations, in a scenario where it is possible to specify these. Regular Home Assistant users are recommended to use `!include` statements instead.

```
example:
 password: !env_var PASSWORD
```

### Default value

If an environment variable is not set, you can fall back to a default value.

```
example:
 password: !env_var PASSWORD default_password
```

### Including entire files

To improve readability, you can source out certain domains from your main configuration file with the `!include`-syntax.

```
light: !include lights.yaml
```

More information about this feature can also be found at [splitting configuration](#).

## Common issues

---

### found character '\t'

If you see the following message:

```
found character '\t' that cannot start any token
```

This means that you've mistakenly entered a tab character, instead of spaces.

### Upper and lower case

Home Assistant is case-sensitive, a state of `'on'` is not the same as `'On'` or `'ON'`. Similarly an entity of `group.Doors` is not the same as `group.doors`.

If you're having trouble, check the case that Home Assistant is reporting in the dev-state menu, under *Developer tools*.

### Booleans

YAML treats `Y`, `true`, `Yes`, `ON` all as `true` and `n`, `FALSE`, `No`, `off` as `false`. This means that if you want to set the state of an entity to `on` you *must* quote it as `'on'` otherwise it will be translated as setting the state to true. The same applies to `off`.

Not quoting the value may generate an error such as:

```
not a valid value for dictionary value @ data
```



## Validating YAML syntax

---

With all these indents and rules, it is easy to make a mistake. The best way to check if your YAML syntax is correct (validate) depends on the editor you use. We can't list them all here.

- If you edit the files directly in Home Assistant, refer to the section: [Validating the configuration](#)
-

## Docs/Configuration

---

While you can configure most of Home Assistant from the user interface, a small number of integrations and power-user features still need a few lines in the `configuration.yaml` file. This page explains how that file works, so you can use it when you need to.

Screenshot of an example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation. Example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation.

## Editing `configuration.yaml`

---

How you edit your `configuration.yaml` file depends on your editor preferences and the installation type you used to set up Home Assistant. Follow these steps:

1. Set up file access and prepare an editor.
2. Find the configuration directory.
3. Edit the `configuration.yaml` file.
4. Save your changes and reload the configuration to apply the changes.

### To set up file access and prepare an editor

Before you can edit a file, you need to know how to access files in Home Assistant and setup an editor. File access depends on your installation type. If you use Home Assistant Operating System, you can use editor apps, for example. If you use Home Assistant Container, apps are not available.

To set up file access on the Home Assistant Operating System, follow these steps:

- If you are unsure which option to choose, install the [file editor app](#).
- Alternatively, use the [Studio Code Server app](#). This editor offers live syntax checking and auto-fill of various Home Assistant entities. But it looks more complex than the file editor.
- If you prefer to use a file editor on your computer, use the [Samba app](#).

### To find the configuration directory

1. To look up the path to your configuration directory, go to **Settings > System > Repairs**.
2. Select the three dots menu and select **System information**.

Show system information option

3. Find out the location of the **Configuration directory**.

Screenshot showing the top of the system information panel - Unless you changed the file structure, the default is as follows: - Home Assistant Operating System: the `configuration.yaml` is in the `/config` folder of the installation. - Home Assistant Container: the `configuration.yaml` is in the config folder that you mounted in your container.

### To edit the configuration file

Once you have located the config folder, you can edit your `configuration.yaml` file. How you edit the file depends on the editor you set up in step 1:

- **If you are using the File editor app:** Open the app, navigate to the `/config` folder in the file browser on the left, and select the `configuration.yaml` file to open it in the editor.
- **If you are using the Studio Code Server app:** Open the app, use the file explorer on the left to navigate to the `configuration.yaml` file, and select it to open in the editor.

- **If you are using Samba to access files:** Navigate to the shared folder on your computer, locate the `configuration.yaml` file, and open it with your favorite text editor like [Notepad++](#) or [Visual Studio Code](#).

 **Note:** If you have watched any videos about setting up Home Assistant using `configuration.yaml` (particularly ones that are old), you might notice your default configuration file is much smaller than what the videos show. Don't be concerned, you haven't done anything wrong. Many items in the default configuration files shown in those old videos are now included in the `default_config:` line that you see in your configuration file. Refer to the [default config integration](#) for more information on what's included in that line.

## Validating the configuration

---

After changing configuration or automation files, you can check if the configuration is valid. A configuration check is also applied automatically when you reload the configuration or when you restart Home Assistant.

The method for running a configuration check depends on your [installation type](#). Check the common tasks for your installation type:

- [Configuration check on Operating System](#)
- [Configuration check on Container](#)

## Reloading the configuration to apply changes

---

For configuration changes to become effective, the configuration must be reloaded. Most integrations in Home Assistant (that do not interact with devices or services) can reload changes made to their configuration in `configuration.yaml` without needing to restart Home Assistant.

1. Under **Settings**, select the three dots menu (top right) `mdi:dots-vertical`, select **Restart Home Assistant > Quick reload**.

Settings, three dot menu, restart Home Assistant

1. If you find that your changes were not applied, you need to restart.
2. Select **Restart Home Assistant**.
3. Note: This interrupts automations and scripts.

Reload and restart buttons

## Troubleshooting the configuration

---

If you run into trouble while configuring Home Assistant, refer to the [configuration troubleshooting page](#).

---

## Docs/Energy/Battery

---

A home battery allows homes to store energy when you are either producing more solar power than you're using, or store energy from the grid if the current price is low.

Home Assistant allows you to track how much energy flows from/to your battery.



## Hardware

---

Home Assistant will need to know the amount of energy flowing from/to your batteries. This data can be tracked in various ways.

### Provided by the battery

Some battery vendors have an API to integrate the data into your Home Assistant instance. An example is [Tesla Powerwall](#).

### Using a CT clamp sensor

Current transformer (CT) clamp sensors measure your energy usage by looking at the current passing through an electrical wire. This makes it possible to calculate the energy usage. In Home Assistant we have support for off-the-shelf CT clamp sensors or you can build your own.

- The off-the-shelf solution that we advise is the [Shelly EM](#). The device has a local API, updates are pushed to Home Assistant and it has a high quality [integration](#).
- You can build your own using ESPHome's [CT Clamp Current sensor](#) or energy meter sensors like the [ATM90E32](#). For the DIY route, check out [this video by digiblur](#) to get started.
- Using a Raspberry Pi, you can use a CT clamp HAT from LeChacal called [RPICT hats](#). They can be stacked to expand the number of lines to monitor. They also provide Active, Apparent, and Reactive power and power factor for single-phase and three-phase installations. They integrate with Home Assistant using MQTT.

*Attention! Installing CT clamp sensor devices requires opening your electrical cabinet. This work should be done by someone familiar with electrical wiring and may require a licensed professional in some regions. Your qualified installer will know how to do this.*

*Disclaimer: Some links in this section are affiliate links.*

---

## Docs/Energy/Electricity Grid

---

Energy management is all about knowing how much energy you're consuming, where it's coming from and where it's going.

Almost all houses are connected to the electricity grid which provides the energy your home will need. The energy usage is being tracked by your energy meter and is billed to you by your energy provider. Energy prices can differ based on a schedule or change according to market price.

Graphic showing energy flowing from the grid to Home Assistant.

## Tariffs

---

It has become popular for energy utilities to split the price of energy based on time of the day; this is done in order to incentivise consumers to shift their power needs towards times where the grid has lower loads. These periods of time are commonly referred to as Peak and Off-Peak. They match periods when everyone is consuming energy (Peak) and periods when energy is abundant, but no one is using it (Off-Peak). Therefore, Peak energy is more expensive than Off Peak energy.

If you want to split energy usage into multiple tariffs, [read this](#).

## Hardware

---

Home Assistant will need to know the amount of energy flowing through your meter. This data can be tracked in various ways.

### Connect to your meter

The best way to get this data is directly from your electricity meter that sits between your house and the grid. In certain countries these meters contain standardized ways of reading out the information locally.

#### Connect using a P1 port

The P1 port is a standardized port in the Netherlands, Belgium and Luxembourg. A P1 reader can connect to this port and receive real-time information.

We have worked with creator [Marcel Zuidwijk](#) to develop [SlimmeLezer+](#). It's an affordable P1 reader powered by [ESPHome](#) that will seamlessly integrate this information in Home Assistant. It is being sold on [his website](#) and the firmware is open source [on GitHub](#).

Photo of SlimmeLezer attached to a smart electricity meter

#### Connect via Zigbee Energy Profile

The Zigbee Energy Profile is a wireless energy standard to provide real-time information about electricity usage. This standard is available in some meters in the US, UK, Canada, and Australia. This is not "normal" Zigbee as implemented by Home Assistant but requires special certified hardware and often requires that the Zigbee connection be provisioned by your utility. As such, your utility, assuming they support this at all, will have a list of currently supported hardware.

The [Rainforest Automation Eagle](#) is one such device that implements this which supports a local API and is compatible with Home Assistant.

#### Reading the meter via a pulse counter

Many meters, including older ones, have an LED that will flash whenever energy passes through it. For example, each flash is a 1/1000th kWh. By monitoring the time between flashes it's possible to determine the energy consumption.

We have developed [Home Assistant Glow](#), an open source solution powered by ESPHome's [pulse meter sensor](#). You put it on top of the activity LED of your electricity meter and it will bring your consumption into Home Assistant.

Photo of Home Assistant Glow attached to an electricity meter

## Reading the meter via IEC62056-21

The IEC62056-21 is a common protocol not only for electric meters. It uses an infrared port to read data. [Aquaticus](#) has created an [ESPHome component](#) for reading this data. [PiggyMeter](#) is a



complete project that allows easy installation.

## Using (Smart Message Language) interface

In countries like Germany, SML (Smart Message Language) is used typically. ESPHome's [SML \(Smart Message Language\)](#) is one way to integrate it. If you prefer to integrate it via MQTT, [sml2mqtt](#) is another open source option.

## Read the meter using an AI-on-the-edge-device

[AI-on-the-edge-device](#) is a project running on an ESP32-CAM and can be fully integrated into Home Assistant using the Home Assistant discovery functionality of MQTT. It digitalizes your gas/water/electricity meter display and provides its data in various ways.

Photo of the AI-on-the-edge-device Workflow

## Reading the meter wirelessly via RTL-SDR

In the United States and Canada, many electricity, gas, and water meters use [AMR \(Automatic Meter Reading\)](#) or [ERT \(Encoder Receiver Transmitter\)](#) protocols to wirelessly broadcast their readings. You can receive these broadcasts using an inexpensive [RTL-SDR](#) USB dongle and decode them with [rtlamr](#), an open-source receiver for these protocols.

The community project [rtlamr2mqtt](#) packages this into a Home Assistant add-on that automatically publishes your meter readings via MQTT discovery, making them available in Home Assistant without any physical connection to the meter.

This approach can work with many Itron, Badger, and other AMR-compatible meters commonly deployed by US and Canadian utilities, but compatibility varies by meter model and region, and some meters use encryption. Check the [rtlamr wiki](#) to confirm that your specific meter type is supported.

## Using a CT clamp sensor

Current transformer (CT) clamp sensors measure your energy usage by looking at the current passing through an electrical wire. This makes it possible to calculate the energy usage. In Home Assistant we have support for off-the-shelf CT clamp sensors or you can build your own.

- The off-the-shelf solution that we advise is the [Shelly EM](#). The device has a local API, updates are pushed to Home Assistant and it has a high quality [integration](#).
- You can build your own using ESPHome's [CT Clamp Current sensor](#) or energy meter sensors like the [ATM90E32](#). For the DIY route, check out [this video by digiblur](#) to get started.
- Using a Raspberry Pi, you can use a CT clamp HAT from LeChacal called [RPiCT hats](#). They can be stacked to expand the number of lines to monitor. They also provide Active, Apparent, and Reactive power and power factor for single-phase and three-phase installations. They integrate with Home Assistant using MQTT.

*Attention! Installing CT clamp sensor devices requires opening your electrical cabinet. This work should be done by someone familiar with electrical wiring and may require a licensed professional in some regions. Your qualified installer will know how to do this.*

*Disclaimer: Some links in this section are affiliate links.*

## Data provided by your energy provider

Some energy providers will provide you real-time information about your usage and have this data integrated into Home Assistant.

## Manual integration

If you manually integrate your sensors, for example, using the [MQTT](#) or [Template](#) integrations: Make sure you set and provide the `device_class`, `state_class`, and `unit_of_measurement` for those sensors.

## Troubleshooting

If you are unable to select your energy or power sensor in the grid consumption drop-down, make sure that its value is being recorded in the Recorder settings.

### [Energy integrations](#)

*Disclaimer: Some links on this page are affiliate links helping support the Home Assistant project.*

---



## Energy vs power

---

People often confuse **power** with **energy**; they are different physical quantities. Power is the rate at which energy is transferred or converted (how fast), while energy is the amount that has been transferred or converted (how much).

Power is measured in watts (W) and energy is commonly measured in kilowatt-hours (kWh). Think of this as analogous to speed and distance: power is like the speed at which you are travelling, and energy is like the distance driven.

Mathematically, energy is the integral of power over time. When working with sampled power values, the energy over an interval is the time integral of power (or a numerical approximation computed from the samples).

This distinction is important because you need to use the correct entities in the Energy dashboard.



## Creating an energy sensor out of a power sensor

---

Since Home Assistant works with discrete samples of power rather than continuous power functions, you can't obtain an exact energy value by integrating from a single, sparsely sampled stream. Instead, you must approximate the integral from the available samples.

If you can sample power values frequently enough (for example, every few seconds), you can reliably estimate transferred energy using numerical approximations such as [Riemann sums](#).

## Split consumption by tariffs

---

If you are using a third-party device (for example, not reading directly from your utility meter or from the utility provider's cloud service) you need Home Assistant to split your energy measurements into two or more tariffs in accordance with your utility provider contract.

To accomplish this, you can use the [utility\\_meter integration](#). With this integration you define as many tariffs as required by your utility provider.

## The Energy dashboard is not visible

---

If you do not see the Energy dashboard in the sidebar, make sure you have not removed `default_config:` from your `configuration.yaml`. If you have, you will need to enable the integrations and UI elements required for the dashboard to appear.

## Troubleshooting missing entities

---

### Condition

You are trying to add a sensor to the Energy dashboard, but it does not appear in the selection list.

### Resolution

To find out why the sensor is not showing, check the following points:

- The sensor must have the appropriate attributes. Check your entity attributes in **Settings > Developer tools > States** to confirm the following:
  - `device_class` must be `energy` or `power` for electricity grid, solar, or battery categories. It must be `gas` for gas, or `water` for water.
  - `state_class` must be `measurement` for power sensors and `total` or `total_increasing` for all others.
- The sensor must have an appropriate `unit_of_measurement`. See the help text for each category to see which units are accepted. Units containing an exponent must match superscript characters exactly.

If any of the attributes are not correct, please open an issue against the integration that provides your sensor, or if you are developing custom template sensors, make sure the templates have the correct attributes.

- The entity must be a `sensor`. If you are trying to add something from another domain (for example an `input_number`), then you must first create a template sensor from it.
- The entity must not have any statistics errors. Go to **Settings > Developer tools > Statistics** to check your specific entity. If your unit has a listed issue here, address that first.

## Docs/Energy/Gas

---

Some homes are connected to gas. Gas is being used to heat water, cook and heat up the home.

Home Assistant allows you to track your gas usage and easily compare it against your energy usage for the same period of time.

## Hardware

---

Home Assistant will need to know the amount of gas that is being consumed.

### Connect to your meter

The best way to get this data is directly from your gas meter that sits between your house and the grid. In certain countries these meters contain standardized ways of reading out the information locally or provide this information via the electricity meter.

#### Connect using a P1 port

The P1 port is a standardized port on electricity meters in the Netherlands, Belgium and Luxembourg which also provides gas consumption information. A P1 reader can connect to this port and receive real-time information.

We have worked with creator [Marcel Zuidwijk](#) to develop [SlimmeLezer+](#). It's an affordable P1 reader powered by [ESPHome](#) that will seamlessly integrate this information in Home Assistant. It is being sold on [his website](#) and the firmware is open source [on GitHub](#).

Photo of SlimmeLezer attached to a smart electricity meter

#### Read the gas meter using an AI-on-the-edge-device

[AI-on-the-edge-device](#) is a project running on an ESP32-CAM and can be fully integrated into Home Assistant using the Home Assistant Discovery Functionality of MQTT. It digitalizes your gas/water/electricity meter display and provides its data in various ways.

Photo of the AI-on-the-edge-device Workflow

#### Read the gas meter using a magnetometer

[Diaphragm/bellows gas meters](#) are the most common type of gas meter, seen in almost all residential installations, and their movement can frequently be observed with a magnetometer. The [QMC5883L](#) and [HMC5883L](#) are common and inexpensive options that ESPHome supports. A project that makes it easy to use these magnetometers and calibrate them is [this water-gas-meter project on GitHub](#).

#### Reading the meter wirelessly via RTL-SDR

In the United States and Canada, many electricity, gas, and water meters use [AMR \(Automatic Meter Reading\)](#) or [ERT \(Encoder Receiver Transmitter\)](#) protocols to wirelessly broadcast their readings. You can receive these broadcasts using an inexpensive [RTL-SDR](#) USB dongle and decode them with [rtlamr](#), an open-source receiver for these protocols.

The community project [rtlamr2mqtt](#) packages this into a Home Assistant add-on that automatically publishes your meter readings via MQTT discovery, making them available in Home Assistant without any physical connection to the meter.

This approach can work with many Itron, Badger, and other AMR-compatible meters commonly deployed by US and Canadian utilities, but compatibility varies by meter model and region, and some meters use encryption. Check the [rtlamr wiki](#) to confirm that your specific meter type is supported.

---

## Docs/Energy/Individual Devices

---

Home Assistant can integrate the energy usage of individual devices into Home Assistant. That way you can see the impact of individual devices on your total energy consumption. In addition to energy usage, Home Assistant also supports tracking individual water device usage, allowing you to monitor water consumption of specific devices in your home.



## Hardware for energy monitoring

---

### Smart plugs

Smart plugs sit between the device and the outlet and measure the energy flowing through the device.

Depending on what protocols you use at home, you can use Zigbee, Z-Wave or Wi-Fi based plugs.

### Smart relays

Smart relays sit behind your "normal" switches and make them smart. It allows you to control the devices via Home Assistant and via the connected buttons/switches.

## Hardware for water monitoring

---

For tracking individual water devices, you can use:

- Smart water meters with device-level monitoring capabilities.
- Inline flow meters that measure water flowing to specific appliances.
- Smart appliances (such as washing machines and dishwashers) that report their own water consumption.

For more information on water metering hardware and integrations, see the [water usage documentation](#).

## Devices with power (W) sensors

---

Some smart devices, such as air conditioning, boilers, and others, may provide a power sensor, measured in Watts. You can use the [Integration \(Riemann sum integral\) integration](#) to calculate the energy your device is using. You can then use the energy sensor in the Energy Dashboard, as individual devices. You can add the power sensor directly if it has the appropriate attributes. For information on setting up an entity for use in the **Energy** dashboard, refer to the [energy FAQ](#).

Graphic showing energy flowing from the home to individual devices.

## Upstream devices and hierarchies

---

You can create a hierarchy of devices by setting one device as an "upstream device" of another. This means you can now establish parent-child relationships between devices within your energy configuration. This works for both energy and water devices.

For example, imagine having a breaker monitoring the total energy consumption of a circuit, but also separately tracking individual devices connected to that circuit.

For water usage, you might have a main water line meter and individual meters for appliances like washing machines or dishwashers. Without setting the device hierarchies, Home Assistant might double-count this usage. By setting the hierarchy, it understands these relationships and accurately shows the individual device usage without duplication. To set up an upstream device relationship:

1. Add an energy or water consumption entity as an individual device.
2. Then, when configuring other individual devices, you can select the previously added individual entity as their upstream device.

This hierarchical view helps you understand which devices are consuming energy or water from which sources and prevents usage from being counted multiple times.

 **Important:** To set up a hierarchy, you must first add all related entities as individual devices in the energy dashboard. Only devices that are already listed under individual devices can be selected as "upstream device" for other devices.

## Docs/Energy/Solar Panels

---

Gain insight into your energy production by integrating your solar panels into Home Assistant.

If you also set up [the Solar Forecast integration](#), you will be able to see expected solar production and automate based on planned production.

Graphic showing energy flowing from the solar panels to Home Assistant and back to the grid.

## Hardware

---

Home Assistant will need to know the amount of energy that is being produced. This can be done in various ways.

### Using a CT clamp sensor

Current transformer (CT) clamp sensors measure your energy usage by looking at the current passing through an electrical wire. This makes it possible to calculate the energy usage. In Home Assistant we have support for off-the-shelf CT clamp sensors or you can build your own.

- The off-the-shelf solution that we advise is the [Shelly EM](#). The device has a local API, updates are pushed to Home Assistant and it has a high quality [integration](#).
- You can build your own using ESPHome's [CT Clamp Current sensor](#) or energy meter sensors like the [ATM90E32](#). For the DIY route, check out [this video by digiblur](#) to get started.
- Using a Raspberry Pi, you can use a CT clamp HAT from LeChacal called [RPICT hats](#). They can be stacked to expand the number of lines to monitor. They also provide Active, Apparent, and Reactive power and power factor for single-phase and three-phase installations. They integrate with Home Assistant using MQTT.

*Attention! Installing CT clamp sensor devices requires opening your electrical cabinet. This work should be done by someone familiar with electrical wiring and may require a licensed professional in some regions. Your qualified installer will know how to do this.*

*Disclaimer: Some links in this section are affiliate links.*

### Connecting to your inverter

Some solar inverters have APIs that can be read by Home Assistant.

[Energy integrations](#)

---

## Docs/Energy/Water

---

Home Assistant allows you to track your water usage in the home energy management too.

Although water usage is not strictly "energy", it is still a valuable resource to track and monitor as it is often tightly coupled with energy usage (like gas). Additionally, it can help you reduce your ecological footprint by using less water.

### Home water meters

There are several ways to measure water usage in your home. Multiple methods exist for reading your water usage. Older water meters typically feature a common arrow or only display total consumption. For these meters, you may require an [AI-on-the-edge-device](#) with an ESP32 camera. While effective, this solution can be tedious to set up as it leans towards a DIY approach.

Newer water meters are equipped with a rotary disk that can be read using two methods. The first method utilizes light sensors, while the second method employs proximity sensors. The proximity sensor detects changes in the magnetic field, with each rotation of the disk representing one liter of water used. Meanwhile, the light sensor method operates on an autocorrelation technique, providing accuracy down to 100 milliliters instead of the traditional one-liter step.

For most water meters, the rotary encoder disk suffices the light sensor version. However, some older or specialized meters may necessitate the use of a proximity meter instead.

Home Assistant also has integrations build into the platform that connect with existing products

## Home Assistant integrations

---

Home Assistant will need to know the amount of water that is being consumed to be able to track usage. Several [water metering \(fluid flow rate sensor device\)](#) hardware options are available to do this. Depending on your setup, the required hardware is provided by your public water utility company, or you may need to buy your own.

Some hardware with water meters may also provide additional practical functions or sensors, such as [valve](#), for example, for controlling water shutoff, or temperature and pressure (to enable freeze alarms).

We have the following integrations available for existing products that can provide information about water usage:

- [Droplet](#)
- [Flo](#)
- [Flume](#)
- [HomeWizard Energy](#)
- [StreamLabs](#)
- [Suez Water](#)
- [Watergate](#)

There are also products for water usage monitoring that are based on existing common IoT protocol standards:

- [Z-Wave](#)
- [Zigbee](#)
- [Matter](#)



## Individual water devices

---

Similar to tracking individual energy devices, Home Assistant supports tracking water usage of individual devices. This feature allows you to monitor water consumption from specific appliances or fixtures in your home, such as washing machines, dishwashers, or individual faucets.

You can create hierarchies of water devices by setting one device as an "upstream device" of another. This prevents double-counting when you have, for example, a main water meter and individual device meters. For more details on setting up device hierarchies and preventing double-counting, see the [individual devices documentation](#).

## Community-made sensors

---

If your water meter lacks a rotary disk, magnetic disk, or coil. There are alternative solutions available to seamlessly integrate water monitoring into your smart home setup:

- [AI-on-the-edge-device](#) is a project running on an ESP32-CAM and can be fully integrated into Home Assistant using the Home Assistant Discovery Functionality of MQTT. It digitalizes your gas/water/electricity meter display and provides its data in various ways. Photo of the AI-on-the-edge-device Workflow

If you have a Culligan Water Softener, you may be able to interface with the inbuilt `DEBUG PORT` and receive water usage stats including `Gallons` (gal), `Gallons Per Minute` (gal/min), and `Gallons to Recharge` (gal):

- [cullAssistant](#) (ESPHome)

Alternatively, the following shops sell ESPHome-based devices that use a 3-phase light sensor to detect a rotating disk in your water meter and convert this to the amount of water used in milliliters (ml): - [Muino water meter reader](#) (ESPHome)

Alternatively, the following shops sell ESPHome-based devices, that use a proximity sensor to detect a rotating magnet in your water meter and use that pulse to count each liter of water used: - [S0tool](#) ("Made for ESPHome" approved) - [Waterlezer dongle](#) (Dutch) - [Slimme Watermeter Gateway](#) (Dutch) - [watermeterkit.nl](#) (Dutch)

## DIY

---

Maybe you like to build one yourself? - Pieter Brinkman has quite a [nice blog article on how to create your own water sensor](#) using ESPHome, or [build a water meter](#) that works with the [P1 Monitor](#) integration. - [AI-on-the-edge-device](#) is a project running on an ESP32-CAM and can be fully integrated into Home Assistant using the Home Assistant Discovery Functionality of MQTT. It digitalizes your gas/water/electricity meter display and provides its data in various ways. Photo of the AI-on-the-edge-device Workflow - [watermeter](#) running classic OCR and statistical pattern recognition on any system supporting Docker - [Muino water meter reader 3-phase](#) Using the 3-phase sensor technique, a battery-powered version can be possible with this sensor. - [Read water meter with magnetometer](#) using [QMC5883L](#) or [HMC5883L](#), common and inexpensive magnetometers. This should be compatible with all the water meters the Flume water sensor is compatible with, which is [compatible](#) with about 95% of water meters in the United States. - Some watermeters use [Wireless M-Bus](#) for remote metering. [wmbusmeters project](#) can automatically capture, decode, decrypt and convert M-Bus packets to MQTT. It supports several M-Bus receivers, including RTL-SDR using [rtl-wmbus library](#). You can also build a WMBus [ESPHome-based receiver](#). An [app](#) for Home Assistant exists for easy installation and configuration. See the [community page](#) for more. - Read water (or gas) usage data from the Itron EverBlu Cyble Enhanced RF meters using the RADIANT protocol over 433 MHz [everblu-meters-esp8266/esp32](#), via an ESP32/ESP8266 and a CC1101 transceiver. Used across the UK and Europe. Fully integrates with Home Assistant using MQTT AutoDiscovery. According to available documentation, this method may also work with AnyQuest Cyble Enhanced, EverBlu Cyble, and AnyQuest Cyble Basic, but these remain untested.

If you manually integrate your sensors, for example, using the [MQTT](#) or [RESTful](#) integrations: Make sure you set and provide the `device_class`, `state_class`, and `unit_of_measurement` for those sensors.

For any of the above-listed options, make sure it actually works with the type of water meter you have before getting one.

## Reading the meter wirelessly via RTL-SDR

In the United States and Canada, many electricity, gas, and water meters use [AMR \(Automatic Meter Reading\)](#) or [ERT \(Encoder Receiver Transmitter\)](#) protocols to wirelessly broadcast their readings. You can receive these broadcasts using an inexpensive [RTL-SDR](#) USB dongle and decode them with [rtlamr](#), an open-source receiver for these protocols.

The community project [rtlamr2mqtt](#) packages this into a Home Assistant add-on that automatically publishes your meter readings via MQTT discovery, making them available in Home Assistant without any physical connection to the meter.

This approach can work with many Itron, Badger, and other AMR-compatible meters commonly deployed by US and Canadian utilities, but compatibility varies by meter model and region, and some meters use encryption. Check the [rtlamr wiki](#) to confirm that your specific meter type is supported.

## Docs/Energy

---

Home Assistant turns your home into a clear, easy-to-read picture of how energy flows in and out of it. You can see how much electricity you draw from the grid and when, how much your solar panels produced today, how full your home battery is, and which appliances are quietly costing you the most. With that information, you can plan when to run the dishwasher, charge the car when power is cheap or your panels are at their peak, and set automations that quietly save you money in the background.

energy config\_energy

The energy dashboard works with electricity, gas, and water. For each one, your usage is grouped into three simple types: what you consume, what you produce, and what you store. You can start with a single source, even just your electricity meter, and add more as you go. Every source you add makes the picture more complete.

Home Assistant is open and works with hardware from many different brands, so you are not locked into one ecosystem. Any energy monitor, smart plug, solar inverter, or utility meter that integrates with Home Assistant can feed data into the energy dashboard.

- [Integrate your energy use from the electricity grid](#)
- [Integrate your solar panels](#)
- [Integrate your home batteries](#)
- [Integrate your gas consumption](#)
- [Integrate your water consumption](#)
- [Integrate individual devices](#)

If you have a sensor that returns instantaneous power readings (W or kW), then to add a sensor that returns energy usage or generation (kWh), refer to the [Riemann sum integral integration](#).

You can also configure power sensors alongside energy sensors in the Energy dashboard. Power inputs accept sensors with `state_class: measurement` and appropriate units (for example `W` or `kW`).

Visual representation of how all different energy forms relate.

---

## Docs/Frontend/Icons

---

### Material Design Icons

Home Assistant utilizes the community-driven [Material Design Icons](#) (MDI) project for icons in the frontend. The icon library is a superset of the base icon library provided by Google and contains thousands of community-made icons for very specific applications, industries, and use-cases.

## Default icons

---

Every entity in Home Assistant has a default icon assigned to it. There are way too many to list out here, but you'll see them in your dashboard. You can [customize any of your entities](#) to change the icons displayed to you.

## Finding icons

---

### Icon picker

The most common way you can find icons is by using the icon picker built right into Home Assistant. Select the **Icon** field when customizing an entity and start typing. The list will filter to icons that match your search criteria. You can also scroll through all available icons when the field is empty.

Icon Picker in Home Assistant

 **Tip:** The icon picker will filter by icon name and by aliases applied to the icon by the MDI project. For example, typing "user" will show you most "account"-named icons.

For more detailed steps on customizing entities, including their icon, refer to [customizing entities](#).

### Material design icons picker browser extension

The easiest way to browse and find icons outside of Home Assistant is with the official [Material Design Icons Picker](#) browser extension. The extension is available for Chrome, Firefox, and Edge and is maintained by the MDI team.

Material Design Icons Picker

 **Note:** Not all icons that appear in the MDI Picker Browser Extension may be available in Home Assistant (yet!). While the browser extension is updated as MDI releases new packages, Home Assistant may lag behind until its next release.

### Material design icons on the Pictogrammers website

The last way to browse through available icons is by viewing the library on the Pictogrammers website, <https://pictogrammers.com/library/mdi/>. Select an icon you'd like to use, then select "Home Assistant" to see an example of its usage.

 **Note:** The Pictogrammers website will always show the latest release of the material design icons library. However, you may find icons that may not yet be available in Home Assistant (yet!). Watch the Home Assistant release notes for announcements on upgrades of the Material Design Icons library.

## Suggesting or contributing new icons

---

Being open-source like Home Assistant, the material design icons library is always accepting suggestions and contributions to expand the library.

 **Note:** Before suggesting or creating a new icon, it is very important that you [search the current library](#) and [search all issues](#), open and closed, on their GitHub. Try searching with different terms that might mean the same thing. For example: "user", "person", or "account".

### Suggesting a new icon

If you have an idea for an icon that isn't currently in the library, but are not interested in creating it yourself, [open a new icon suggestion](#).

### Contributing a new icon

If you want to contribute a new icon to the library, familiarize yourself with the [System icons guidelines](#) in the Material Design system. Then create your icon and [submit it to the Pictogrammers team for review](#).

#### Tips for creating new icons

- Really pay attention to [Material Design guidelines](#).
- Keep in mind that icons are meant to be contextual, not literal.
- When it comes to little details, less is more.
- If you're unsure, open an issue on their GitHub. They're more than happy to help you!
- Not all icons make it into the library and that is okay!

### Suggesting an icon alias

Sometimes an icon exists, but you aren't able to find it with the terms you were searching for. If this has ever happened to you, please [open an issue with the Pictogrammers team to suggest new aliases](#) that can be added to existing icons.

---



## Docs/Glossary

---

Home Assistant has a vocabulary of its own. Most of it comes up in everyday use: integrations, entities, devices, automations, dashboards. This page is a quick reference for what every term means and how the pieces fit together. Whenever you see a term you do not recognize in the documentation or in the user interface, you can come back here.

---

## Docs/Locked Out

---

The sections below deal with recovering from a situation where you are not able to sign in, or need to recover your data.

## Forgot username

---

### Symptom: I'm the owner and I forgot my username

You are the **owner** of the Home Assistant server and you cannot login because you forgot your username.

#### Remedy

1. Check if the following conditions are met:
2. you are using the Home Assistant Operating System
3. you have access to the Home Assistant server.
4. Open a terminal connection to Home Assistant:
5. If you are using a Home Assistant Green, follow these steps [to access the console](#).
6. If you are using a Home Assistant Yellow, follow these steps:
  - [to access the console from Windows](#)
  - [to access the console from Linux or macOS](#).
7. If you are using another system, connect keyboard and monitor. The procedure might be similar the one used for Green.
8. If you are using a Home Assistant OVA (virtualization image):
  - Access the system console by opening the terminal through your virtualization platform's interface (for example, Proxmox, VMware, VirtualBox).
  - Follow the platform-specific steps to interact with the virtual machine's console.
9. In the terminal, enter the `auth list` command.
10. This command lists all users that are registered on your Home Assistant.

# Forgot password

---

## Symptom: I'm the owner and I forgot my password

You are the owner or administrator of Home Assistant and forgot your password.

## Remedy: resetting an owner's password

If you are the owner or have administrator, there are different methods to reset a password, depending on your situation:

- [Reset a password while still logged in](#)
- [Reset an owner's password when logged out](#)
- [Reset a user's password, via the container command line](#)

### To reset a password while still logged in

The method used to reset a password depends on your user rights:

- If you are a regular user without administrator rights, ask the owner to [give you a new password](#).
- If you are the owner, choose one of the procedures below to reset your password.
- You cannot recover an owner password from within Home Assistant.
- There is only one owner per system. You cannot add a new owner.
- If you are an administrator, add a new user as an administrator and give the new user a password you can remember.
- Then log out, and log in with this new user.
- Reset your password via this new administrator account (and then [delete this new account](#)).
  - Your configuration will remain, and you don't have to do a new onboarding process.

### To reset an owner's password, via console

Use this procedure only if the following conditions are met:

- You can access the Home Assistant console **on the device itself** (not via the SSH terminal from the apps).
- If you are using a Home Assistant Yellow or Green, refer to their documentation.
- If you are using a Home Assistant Yellow, refer to the following procedure:
  - [Resetting the owner password on Home Assistant Yellow](#)
- If you are using a Home Assistant Green, refer to the following procedure:
  - [Resetting the owner password on Home Assistant Green](#)
- If you are not using a Yellow or Green: Connect to the console of the Home Assistant server:
- If you are using a virtual machine, connect to your virtual machine console.

- If you are using another board, connect a keyboard and monitor to your device and access the terminal. The procedure is likely very similar to the one described for the Home Assistant Green.
- Once you have opened the Home Assistant command line, enter the following command:
- **Command:** `auth reset --interactive`
- This will display a list of users. Select your user and enter a new password when prompted.
- Troubleshooting: If you see the message `zsh: command not found: auth`, you likely did not enter the command in the serial console connected to the device itself, but in the terminal within Home Assistant.
- You can now log in to Home Assistant using this new password.

### To reset a user's password, via the container command line

If you are running Home Assistant in a container, you can use the command line in the container with the `hass` command to change your password. The steps below refer to a Home Assistant container in Docker named `homeassistant`. Note that while working in the container, commands will take a few moments to execute.

1. `docker exec -it homeassistant bash` to open to the container command line
2. `hass` to create a default user, if this is your first time using the tool
3. `hass --script auth --config /config change_password existing_user new_password` to change the password
4. `exit` to exit the container command line
5. `docker restart homeassistant` to restart the container.

### To reset a user's password, as an owner via the web interface

Only the owner can change other user's passwords.

1. In the bottom left, select your user to go to the **Profile** page and make sure **Advanced Mode** is activated.
2. Go to **Settings > People** and select the person for which you want to change the password.
3. At the bottom of the dialog box, select **Change password**.
4. Note: this is available as the owner, not administrator.
5. Enter the new password, and select **OK**.
6. Confirm the new password by entering it again, and select **OK** again.
7. A confirmation box will be displayed with the text **Password was changed successfully**.

## Preparing the system to start a new onboarding process

---

If you lose the password associated with the owner account and the steps above do not work to reset the password, the only way to resolve this is to start a new onboarding process.

- If you have an external backup with an administrator account of which you still know the login credentials, you can restore that backup.
- If you do not have a backup, resetting the device will erase all data.
- If you have a Home Assistant Green, [reset the Green](#).
- If you have a Home Assistant Yellow, [reset the Yellow](#).

## Recovering data for Home Assistant

---

Unless your SD card/data is corrupted, you can still get to your files or troubleshoot further. There are a few routes:

- Connect a USB keyboard and HDMI monitor directly to the Raspberry Pi.
- Remove the SD and access the files from another machine (preferably one running Linux).

## Connect directly

---

If you're using a Raspberry Pi, you're likely going to have to pull the power in order to get your monitor recognized at boot. Pulling power has a risk of corrupting the SD, but you may not have another option. Most standard USB keyboards should be recognized easily.

Once you're connected, you'll see a running dmesg log. Hit the enter key to interrupt the log. Sign in as "root". There is no password.

You will then be at the Home Assistant CLI, where you can run the custom commands. These are the same as you would run using the SSH app but without using `ha` in front of it. For example:

- `core logs` for Home Assistant Core log
- `supervisor logs` for supervisor logs
- `host reboot` to reboot the host
- `dns logs` for checking DNS
- etc (typing `help` will show more)



## Accessing files from the SD/HDD

---

### Remove the SD and access the files from another computer

The files are on an EXT4 partition ( `hassos-data` ) and the path is `/mnt/data/supervisor` . These are easily accessed using another Linux machine with EXT support.

For Windows or macOS you will need third party software. Below are some options.

- Windows: <https://www.diskinternals.com/linux-reader/> (read-only access to the SD)
- macOS: <https://osxfuse.github.io/>

## Deleting a user

---

You need to be an owner or have administrator rights to delete a user.

1. Go to **Settings** > **People** and select the person which you want to delete.
  2. Note: you cannot delete the owner.
  3. At the bottom of the dialog box, select **Delete**.
  4. A confirmation dialog box will be displayed.
  5. To confirm, select **OK**.
-

## Docs/Organizing/Areas

---

An area in Home Assistant is a logical grouping of devices and entities that are meant to match areas (or rooms) in the physical world of your home. Areas allow you to target an entire group of devices with an action.

For example, the "Living room" area groups devices and entities in your living room. This allows you to turn off all the lights in the living room with a single action. Areas can be assigned to floors. Areas can also be used to automatically generate cards, such as the [Area card](#).

## Creating an area

---

Follow these steps to create a new area from the **Areas** view.

1. Go to **Settings > Areas, labels & zones** and select **Create area**.
2. In the dialog, enter the area details:
3. Give the area a **Name** (required).
4. Add an icon (use [Material Design Icons \(MDI\)](#)).
5. Assign the area to a floor.
  - If you have not created floors yet, you can [create a new floor](#).
  - The number can be negative. For example, for underground floors.
  - This number can later be used for sorting.
6. Add an image representing that area.
7. Add an **Alias**.
  - Aliases are alternative names used in [voice assistants](#) to refer to an area, entity, or floor.

Create area dialog 3. Select **Create**.

Result: A new area is created.

## Assigning areas to floors and adding labels

---

If an area has not yet been assigned to a floor, it is shown in the **Unassigned areas** section. Follow these steps to assign an area to a floor.

1. Go to **Settings > Areas, labels & zones**.
2. Drag and drop the area card to the target floor.
3. Alternatively, on the area card, select the edit  button and assign the floor and labels.

## Assigning an area to multiple items

---

You can assign an area to multiple items at once in the automation, scene, script, and device pages.

1. Depending on what you want to assign, go to one of the following pages:
2. For automations, scripts, or scenes **Settings > Automations & scenes** and open the respective tab.
3. For devices, go to **Settings > Devices & services > Devices**.
4. In the list, [select all the items](#) you want to assign to an area.

Screenshot showing how to assign multiple devices to an area

5. In the top right corner, select **Move to area** and select the target area from the list.

## Assigning an area to a device from the devices dashboard

---

The **Overview** (built-in) dashboard has a dedicated **Devices** section that shows all devices that are not assigned to an area.

1. Go to **Settings** > **Dashboards** and select the **Overview** (built-in) dashboard.
2. Scroll down to the **Other areas** section and select the **Devices** card.
3. For your device, select **Assign area** and choose the target area from the list.

Screenshot of the Devices card on the Overview dashboard showing the Assign area action for an unassigned device

## Editing an area

---

Follow these steps to edit an area.

1. Go to **Settings** > **Areas, labels & zones** and on the area card, select the edit `mdi:edit` button.
2. In the dialog, adjust the area details you want to change:
3. Edit the area **Name**.
4. Add an icon (We use [Material icons](#)).
5. Assign the area to a floor.
  - If you have not created floors yet, you can [create a new one](#).
  - The number can be negative. For example for underground floors.
  - This number can later be used for sorting.
6. Add an image representing that area.
7. Add an **Alias**.
  - Aliases are alternative names used in [voice assistants](#) to refer to an area, entity, or floor.



## Editing an area from the area dashboard

---

You can also edit the area details when you're on the area dashboard.

1. Go to **Settings** > **Dashboards** and select the **Overview** (built-in) dashboard.
2. Select the area.
3. In the top-right corner, select the edit `mdi:edit` button.
4. In the **Update area** dialog, [edit the area details](#).

## Reordering areas on built-in dashboards

---

Follow these steps to rearrange floors and areas on the built-in dashboards (such as **Overview**, **Lights**, and **Security**).

1. Go to **Settings > Areas, labels & zones**.
2. There are 2 options to rearrange items:
3. **Option 1:** Use drag-and-drop.
4. **Option 2:** In the top-right corner, select the three dots `mdi:dots-vertical` menu and select **Reorder floors and areas**.
  - In the dialog, move the floors or areas you want to rearrange:

Reorder floors and areas

## Sending a vacuum to a dedicated area

---

If you have a vacuum that supports area cleaning, you can [set up an automation](#) to send the vacuum to a specific area.

## Deleting an area

---

Follow these steps to delete an area. It will be removed from all the floors it was assigned to. All the devices that were assigned to this area will become unassigned. If you used this area in automations or script as targets, or with voice assistant, these will no longer work.

1. Go to **Settings > Areas, labels & zones** and select the area card.
2. In the top right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Delete**.

Delete area

3. If you used this area in automations or script as targets, or with voice assistant, they will no longer work.
  4. You can adjust or delete the related scripts or automations.
  5. If you still had devices in that area, they are no longer assigned to any room.
  6. If you have moved the devices, you can now reassign them to a new area.
-

## Docs/Organizing/Categories

---

Categories let you group and filter items in a table. Like labels, categories allow grouping irrespective of the items physical location.

For example, on the automations page, you can create the categories “Notifications” or “NFC tags” to view your automations grouped or filtered. These categories group automations on the automation page, but have no effect anywhere else. Categories are unique for each table. The automations page can have different categories than the scene, scripts, or helpers settings page.

## Creating a category

---

Follow these steps to create a new category.

1. Go to **Settings > Automations & scenes** and open the respective tab.
2. In the top left, select the **Filters** button.

Select the filter button 3. Select **Category**, then **Add category**. 4. Enter a name, select an icon and select **Add**.

Result: A new category is created.

## Assigning a category

---

1. Go to **Settings > Automations & scenes** and open the respective tab.
2. To assign a category to a single item:
3. Find the item in the list and select the three dots `mdi:dots-vertical` menu.
4. Select **Assign category** and select the category from the list.
5. If the category is not in the list, select **Add new category** and make a new one.
6. To assign a category to multiple items:
7. Select the multi-select `mdi:order-checkbox-ascending` button.
8. From the list, select all the items to which you want to apply a category.
9. In the top right corner, select **Move to category**.
10. Then, select the category from the list.
11. Once categories are applied, the table items are grouped by those categories.
12. The example shows 2 categories: Coffee and housekeeping.

Group table items by category

## Editing or deleting a category

---

To rename or delete a category, follow these steps:

1. Go to **Settings > Automations & scenes** and open the respective tab.
2. In the top left, select the **Filters** button.

Select the filter button 3. In the list, find the category you want to edit and select the three dots `mdi:dots-vertical` menu next to it. 4. Select **Edit category** or **Delete category**.

Screenshot showing the edit and delete buttons for categories

---



## Docs/Organizing/Floors

---

A floor in Home Assistant is a logical grouping of areas meant to match your home's physical floors.

Devices and entities cannot be assigned to floors directly but to areas. Floors can be used in automations and scripts as a target for actions. For example, to turn off all the lights on the downstairs floor when you go to bed.

## Creating a floor

---

Follow these steps to create a new floor.

1. Go to **Settings** > **Areas, labels & zones** and select **Create floor**.
2. In the dialog, enter the floor details:
3. Give the floor a **Name** (required).
4. Add a floor **Level**.
  - The number can be negative. For example for underground floors.
  - This number can later be used for sorting.
5. Add an icon (We use [Material icons](#)).
6. Add an **Alias**.
  - Aliases are alternative names used in [voice assistants](#) to refer to an entity, area, or floor.

Create floor dialog 3. Select **Add**.

Result: A new floor is created.

![Create floor dialog] (/images/organizing/create\_floor\_02.png)

1. You can now [assign areas to that floor](#).

## Reordering floors on built-in dashboards

---

Follow these steps to rearrange floors and areas on the built-in dashboards (such as **Overview**, **Lights**, and **Security**).

1. Go to **Settings > Areas, labels & zones**.
2. There are 2 options to rearrange items:
3. **Option 1:** Use drag-and-drop.
4. **Option 2:** In the top-right corner, select the three dots `mdi:dots-vertical` menu and select **Reorder floors and areas**.
  - In the dialog, move the floors or areas you want to rearrange:

Reorder floors and areas

## Deleting a floor

---

Follow these steps to delete a floor. Areas that are assigned to a floor will become unassigned. Automations and scripts or voice assistants that used a floor as a target will no longer work as they no longer have a target.

1. Go to **Settings > Areas, labels & zones**.
2. Next to the floor, select the three dots `mdi:dots-vertical` menu and select **Delete floor**.

Screenshot showing the dialog to delete a floor

3. If you have automations, scripts, or voice assistants that used floors as a target, you will need to update these.
-

## Docs/Organizing/Labels

---

Labels in Home Assistant allow grouping elements irrespective of their physical location or type. Labels can be assigned to areas, devices, entities, automations, scenes, scripts, and helpers. Labels can be used in automations and scripts as a target for actions. Labels can also be used to filter data.

For example, you can filter the list of devices to show only devices with the label `heavy energy usage` or turn these devices off when there is not a lot of solar energy available.

## Creating a label

---

Follow these steps to create a new label from the **Labels** view.

1. Go to **Settings > Areas, labels & zones** and on top, select the **Labels** tab.
2. Select the **Create label** button.
3. In the dialog, enter the label details:
4. Give the label a **Name** (required).
5. Add an icon (We use [Material icons](#)).
6. Add a **Color**.

Create label dialog 4. Select **Create**.

Result: A new label is created.

## Applying labels

---

Follow these steps to apply a label

1. To apply a label to an area:
2. Go to **Settings > Areas, labels & zones**.
3. On the area card, select the edit `mdi:edit` button.
4. Select one or more labels or select **Add new label** to create a new one.
5. To apply a label to a device, entity, or helper:
6. Go to **Settings > Devices & services** and open the respective tab.
7. Select the `mdi:order-checkbox-ascending` button.
8. From the list, select all the list entries to which you want to apply a label.
9. In the top right corner, select **Add label**. Then, select the labels from the list.

Apply label 3. To apply a label to an automation, scene, or script: - Go to **Settings > Automations & scenes** and open the respective tab. - Select the `mdi:order-checkbox-ascending` button. - From the list, select all the list entries to which you want to apply a label. - In the top right corner, select the three dots `mdi:dots-vertical` menu, then select **Add label**. Then, select the labels from the list.

## Deleting a label

---

Follow these steps to delete a label. It will be removed from all the list entries it was applied to. If you used this label in automations or script as targets, you need to adjust those.

1. Go to **Settings > Areas, labels & zones** and on top, select the **Labels** tab.
2. In the list of labels, find the label you want to delete and select the three dots `mdi:dots-vertical` menu.
3. Select **Delete**.
4. If you used this label in automations or script as targets, you need to adjust those.



## Removing labels

---

1. Go to the data table that contains the element from which you want to remove the label:
  2. Go to **Settings** > **Devices & services** and open the respective tab.
  3. Or, go to **Settings** > **Automations & scenes** and open the respective tab.
  4. Select the `mdi:order-checkbox-ascending` button.
  5. From the list, select all the items from which you want to remove a label.
  6. In the top right corner, select the three dots `mdi:dots-vertical` menu, then select **Add label**.
  7. Then, deselect the checkbox for the label you want to remove.
-

## Docs/Organizing/Tables

---

When working with tables, you can select multiple items to apply an action. If you have [grouped](#) items by assigning them to floors, areas, labels, or directories, you can also filter your data accordingly.

## Selecting multiple items in a table

---

1. In your table, select the `mdi:order-checkbox-ascending` button.

Screenshots point out the enable selection mode button in the toolbar of the tables in Home Assistant

1. In the list, select the items of interest.

Selecting multiple elements in a list

1. You can now apply changes to all selected elements, such as [applying labels](#) or [enabling or disabling entities and automations](#).

## Filtering items in a table

---

You can filter a table so that only items matching certain criteria are shown.

To filter items in a table, follow these steps:

1. In the top left corner above the table, select the **Filters** button.

Select the filter button

2. In the filters panel, select your filter criteria.
3. You can filter for [floors](#), [areas](#), [labels](#), and [categories](#) if you have previously defined them.
4. The list of available criteria depends on the type of table.

Screenshots showing the filter panel that tables can have, allowing you to easily find what you are looking for

## Grouping and sorting items in a table

---

You can group items in a table according to certain criteria. The number of shown items stays the same. No items will be hidden.

To group items in a table, follow these steps:

1. In the top right above the table, select the **Group by** button.
2. The items will be grouped according to the criteria you chose.
3. The list of available criteria depends on the type of table.
  - The example shows a list of devices, grouped by manufacturer.
  - In contrast, the entities table does not allow grouping by manufacturer, but by entity domains.

Select the Group by button

4. To sort the items, select the **Sort by** button.
5. To get a better overview, you can collapse groups in the list.

Collapse groups

## Customizing columns

---

You can show or hide columns and change the order. Your customized columns are stored in your browser, so you only have to set it up once, and it will be remembered for the next time you visit the page.

To customize columns, follow these steps:

1. In the top right corner of the table, select the cog wheel.
2. To hide a column, deselect it.
3. To rearrange the order, grab the column and move it to its new position.
4. To sort, select the column header of interest.

Screencast showing how to show, hide, and rearrange columns

---

## Docs/Organizing

---

Once you have more devices, you may want to target entire groups of devices in automations. It also becomes more challenging to find items in lists.

There are a few tools to group your assets: [Areas](#), [floors](#), [labels](#), [categories](#) and [group integration](#).

Taxonomy	Automation target	Entity can have multiple
Area	<code>openmoji:check-mark</code>	<code>openmoji:cross-mark</code>
Floor	<code>openmoji:check-mark</code>	<code>openmoji:cross-mark</code>
Label	<code>openmoji:check-mark</code>	<code>openmoji:check-mark</code>
Category	<code>openmoji:cross-mark</code>	<code>openmoji:cross-mark</code>
Group integration	<code>openmoji:check-mark</code>	<code>openmoji:check-mark</code>

For example, a smart plug with energy monitoring is a single device, but it has multiple entities of different types: a switch entity and a sensor entity. Each of those entities can only belong to one area and one floor, but they can each have multiple labels and be a member of multiple groups.

When you assign a device to an area, all its entities inherit that area. You can override this for individual entities. For example, you could keep the sensor entities of the smart plug in the living room, but assign the switch entity to the kitchen.

## Area

---

- Groups devices and entities.
- Can be assigned to one floor.
- Reflects a physical area (or room) in your home.
- Can be used in automations: Allows targeting an entire group of devices with an action. For example, turning off all the lights in the living room.
- Areas can also be used to automatically generate cards, such as the [Area card](#).



## Floor

---

- Groups areas.
- Devices and entities cannot be assigned to floors, but to areas only.
- Can have multiple areas.
- Can be used in automations and scripts as a target for actions. For example, to turn off all the lights on the downstairs floor when you go to bed.

Screenshots showing areas settings page, which now also shows the areas grouped by floor.

## Labels

---

- Can be assigned to areas, devices, entities, automations, scenes, scripts, and helpers.
- Can be used in automations and scripts as a target for actions.
- Labels can also be used to filter data in tables. For example, you can filter the list of devices to show only devices with the label `heavy energy usage` or turn these devices off when there is not a lot of solar energy available.

Screenshots showing the new labels assigned to automations.

## Category

---

- Groups items in a table.
- Categories are unique for each table. The automations page can have different categories than the scene, scripts, or helpers settings page.

Screenshots the new categories. Automations are grouped into their categories, making it easier to get an overview or to filter them.

## Group Integration

---

- Designed to combine multiple entities into one entity representing the group.
  - The combined entity can also have an area and labels.
  - An entity can be a member of multiple groups.
  - Can be used in automations and scripts as a target for actions.
  - Does not assist with organizing entities in the UI like the other methods above. For example, you cannot use group integration to sort or filter other entities.
-

## Docs/Quality Scale

---

The integration quality scale is a framework for Home Assistant to grade integrations based on user experience, features, code quality, and developer experience. To grade this, the project has come up with a set of tiers, each with its own set of criteria.

## Scaled tiers

---

There are 4 scaled tiers, bronze, silver, gold, and platinum. To reach a tier, the integration must fulfill all rules of that tier and the tiers below.

These tiers are defined as follows.

### **Bronze**

The bronze tier is the baseline standard and requirement for all new integrations. It meets the minimum requirements in code quality, functionality, and user experience. It complies with the fundamental expectations and provides a reliable foundation for users to interact with their devices and services.

The documentation provides guidelines for setting up the integration directly from the Home Assistant user interface.

From a technical perspective, this integration has been reviewed to comply with all baseline standards, which we require for all new integrations, including automated tests for setting up the integration.

The bronze tier has the following characteristics:

- Can be easily set up through the UI.
- The source code adheres to basic coding standards and development guidelines.
- Automated tests that guard this integration can be configured correctly.
- Offers basic end-user documentation that is enough to get users started step-by-step easily.

### **Silver**

The silver tier builds upon the *Bronze* level by improving the reliability and robustness of integrations, ensuring a solid runtime experience. It ensures an integration handles errors properly, such as when authentication to a device or service fails, handles offline devices, and other errors.

The documentation for these integrations provides information on what is available in Home Assistant when this integration is used, as well as troubleshooting information when issues occur.

This integration has one or more active code owners who help maintain it to ensure the experience on this level lasts now and in the future.

The silver tier has the following characteristics:

- Provides everything the *Bronze* tier has.
- Provides a stable user experience under various conditions.
- Has one or more active code owners who help maintain the integration.
- Correctly and automatically recover from connection errors or offline devices, without filling log files and without unnecessary messages.
- Automatically triggers re-authentication if authentication with the device or service fails.

- Offers detailed documentation of what the integration provides and instructions for troubleshooting issues.

## Gold

The gold standard in integration user experience, providing extensive and comprehensive support for the integrated devices & services. A gold-tier integration aims to be user-friendly, fully featured, and accessible to a wider audience.

When possible, devices are automatically discovered for an easy and seamless setup, and their firmware/software can be directly updated from Home Assistant.

All provided devices and entities are named logically and fully translatable, and they have been properly categorized and enabled for long-term statistical use.

The documentation for these integrations is extensive, and primarily aimed toward end-users and understandable by non-technical consumers. Besides providing general information on the integration, the documentation provides possible example use cases, a list of compatible devices, a list of described entities the integration provides, and extensive descriptions and usage examples of available actions provided by the integration. The use of example automations, dashboards, available Blueprints, and links to additional external resources, is highly encouraged as well.

The integration provides means for debugging issues, including downloading diagnostic information and documenting troubleshooting instructions. If needed, the integration can be reconfigured via the UI.

From a technical perspective, the integration needs to have full automated test coverage of its codebase to ensure the set integration quality is maintained now and in the future.

All integrations that have devices in the Works with Home Assistant program are at least required to have this tier.

The gold tier has the following characteristics:

- Provides everything the *Silver* tier has.
- Has the best end-user experience an integration can offer; streamlined and intuitive.
- Can be automatically discovered, simplifying the integration setup.
- Integration can be reconfigured and adjusted.
- Supports translations.
- Extensive documentation, aimed at non-technical users.
- It supports updating the software/firmware of devices through Home Assistant when possible.
- The integration has automated tests covering the entire integration.
- Required level for integrations providing devices in the Works with Home Assistant program.

## Platinum

Platinum is the highest tier an integration can reach, the epitome of quality within Home Assistant. It not only provides the best user experience but also achieves technical excellence by adhering to the highest standards, supreme code quality, and well-optimized performance and efficiency.

The platinum tier has the following characteristics:

- Provides everything the *Gold* tier has.
- All source code follows all coding and Home Assistant integration standards and best practices and is fully typed with type annotations and clear code comments for better code clarity and maintenance.
- A fully asynchronous integration code base ensures efficient operation.
- Implements efficient data handling, reducing network and CPU usage.



## Special tiers

---

There are 4 special tiers that are used for integrations that don't have a place on the scaled tier list. This is because they are either an internal part of Home Assistant Core, they are not in Home Assistant Core at all, or they don't meet the minimum requirements to be graded against the scaled tiers.

The special tiers are defined as follows.

### ? No score

These integrations can be set up through the Home Assistant user interface. The *No score* designation doesn't imply that they are bad or buggy, instead, it indicates that they haven't been assessed according to the quality scale or that they need some maintenance to reach the now-considered minimum *Bronze* standard.

The *No score* tier cannot be assigned to new integrations, as they are required to have at least a *Bronze* level when introduced. The Home Assistant project encourages the community to help update these integrations without a score to meet at least the *Bronze* level requirements.

Characteristics:

- Not yet scored or lacks sufficient information for scoring.
- Can be set up via the UI, but may need enhancements for a better experience.
- May function correctly, but hasn't been verified against current standards.
- Documentation most often provides only basic setup steps.

### Internal

The internal tier is assigned to integrations used internally by Home Assistant. These integrations provide basic components and building blocks for the Home Assistant Core program or for other integrations to build on top of it.

Internal integrations are maintained by the Home Assistant project and subjected to strict architectural design procedures.

Characteristics:

- Internal, built-in building blocks of the Home Assistant Core program.
- Provides building blocks for other integrations to use and build on top of.
- Maintained by the Home Assistant project.

### Legacy

Legacy integrations are older integrations that have been part of Home Assistant for many years, possibly since its inception. They can only be configured through YAML files and often lack active maintainers (code owners). These integrations might be complex to set up and do not adhere to current/modern end-user expectations in their use and features.

The Home Assistant project encourages the community to help migrate these integrations to the UI and update them to meet modern standards, making these integrations accessible to everyone.

Characteristics:

- Complex setup process; only configurable via YAML, without UI-based setup.
- May lack active code ownership and maintenance.
- Could be missing recent updates or bug fixes.
- Documentation may still be aimed at developers.

## Custom

Custom integrations are developed and distributed by the community, and offer additional functionalities and support for devices and services to Home Assistant. These integrations are not included in the official Home Assistant releases and can be installed manually or via third-party tools like HACS (Home Assistant Community Store).

The Home Assistant project does not review, security audit, maintain, or support third-party custom integrations. Users are encouraged to exercise caution and review the custom integration's source and community feedback before installation.

Developers are encouraged and invited to contribute their custom integration to the Home Assistant project by aligning them with the integration quality scale and submitting them for inclusion.

Characteristics:

- Not included in the official Home Assistant releases.
  - Manually installable or installable via community tools, like [HACS](#).
  - Maintained by individual developers or community members.
  - User experience may vary widely.
  - Functionality, security, and stability can vary widely.
  - Documentation may be limited.
-

## Docs/Scene/Editor

---

From the UI choose **Settings** which is located in the sidebar, then select **Automations & scenes** to go to the scene editor. Press the **Add scene** button in the lower right corner to get started.

Choose a meaningful name for your scene.

### Scene editor

Select all the devices (or entities when advanced mode is enabled on your user profile) you want to include in your scene. The state of your devices will be saved, so it can be restored when you are finished creating your scene. Set the state of the devices to how you want them to be in your scene, this can be done by selecting it and edit the state from the popup, or any other method that changes the state. On the moment you save the scene, all the states of your devices are stored in the scene. When you leave the editor the states of the devices are restored to the state from before you started editing. The menu on the top-right has options to **Duplicate scene** and **Delete scene**.

A scene can be called in automation action and scripts using a turn on scene action:

```
action: scene.turn_on
target:
 entity_id: scene.my_unique_id
```

## Updating your configuration to use the editor

---

First, check that you have activated the configuration editor.

```
Activate the configuration editor
config:
```

The scene editor reads and writes to the file `scenes.yaml` in the root of your `configuration` folder. Currently, both the name of this file and its location are fixed. Make sure that you have set up the scene integration to read from it:

```
Configuration.yaml example
scene: !include scenes.yaml
```

If you still want to use your old scene section, add a label to the old entry:

```
scene old:
 - name: ...
```

You can use the `scene:` and `scene old:` sections at the same time:

- `scene old:` to keep your manual designed scenes
- `scene:` to save the scene created by the online editor

```
scene: !include scenes.yaml
scene old: !include_dir_merge_list scenes
```

## Migrating your scenes to `scenes.yaml`

---

If you want to migrate your old scenes to use the editor, you'll have to copy them to `scenes.yaml`. Make sure that `scenes.yaml` remains a list! For each scene that you copy over, you'll have to add an `id`. This can be any string as long as it's unique.

For example:

```
Example scenes.yaml entry
- id: my_unique_id # <-- Required for editor to work.
 name: Romantic
 entities:
 light.tv_back_light: on
 light.ceiling:
 state: on
 xy_color: [0.33, 0.66]
 brightness: 200
```



**Note:** Any comments in the YAML file will be lost and templates will be reformatted when you update a scene via the editor.

## Docs/Scene

---

You can create scenes that capture the states you want certain entities to be. For example, a scene can specify that light A should be turned on and light B should be bright red. Scenes are available as an entity through the standalone [Scene integration](#) but can also be embedded in automations and scripts.

```
Example configuration.yaml entry
scene:
 - name: Romantic
 entities:
 light.tv_back_light: "on"
 light.ceiling:
 state: "on"
 xy_color: [0.33, 0.66]
 brightness: 200
 - name: Movies
 entities:
 light.tv_back_light:
 state: "on"
 brightness: 125
 light.ceiling: off
 media_player.sony_bravia_tv:
 state: "on"
 source: HDMI 1
```

## How to configure your scene

In the scene you define in your YAML files, please ensure you use all required parameters as listed below.

Option	Type	Required	Default	Description
<code>name</code>	string	Yes		Friendly name of the scene.
<code>entities</code>	list	Yes		Entities to control and their desired state.

As you can see, there are two ways to define the states of each `entity_id`:

- Define the `state` directly with the entity. Be aware, that `state` needs to be defined.
- Define a complex state with its attributes. You can see all attributes available for a particular entity under `developer-tools -> state`.

Scenes can be activated using the action `scene.turn_on` (there is no 'scene.turn\_off' action).

```
Example automation
automation:
 triggers:
 - trigger: state
 entity_id: device_tracker.sweetheart
 from: "not_home"
 to: "home"
 actions:
 - action: scene.turn_on
 target:
 entity_id: scene.romantic
```

## Applying a scene without defining it

---

With the `scene.apply` action you are able to apply a scene without first defining it via configuration. Instead, you pass the states as part of the data. The format of the data is the same as the `entities` field in a configuration.

```
Example automation
automation:
 triggers:
 - trigger: state
 entity_id: device_tracker.sweetheart
 from: "not_home"
 to: "home"
 actions:
 - action: scene.apply
 data:
 entities:
 light.tv_back_light:
 state: "on"
 brightness: 100
 light.ceiling: off
 media_player.sony_bravia_tv:
 state: "on"
 source: "HDMI 1"
```



## Using scene transitions

---

Both the `scene.apply` and `scene.turn_on` actions support setting a transition, which enables you to smoothen the transition to the scene.

This is an example of an automation that sets a romantic scene, in which the light will transition to the scene in 2.5 seconds.

```
Example automation
automation:
 triggers:
 - trigger: state
 entity_id: device_tracker.sweetheart
 from: "not_home"
 to: "home"
 actions:
 - action: scene.turn_on
 target:
 entity_id: scene.romantic
 data:
 transition: 2.5
```

Transitions are currently only support by lights, which in their turn, have to support it as well. However, the scene itself does not have to consist of only lights to have a transition set.

## Reloading scenes

---

Whenever you make a change to your scene configuration, you can call the `scene.reload` action to reload the scenes.

---

## Docs/Scripts/Conditions

---

Conditions can be used within a script or automation to prevent further execution. When a condition evaluates true, the script or automation will be executed. If any other value is returned, the script or automation stops executing. A condition will look at the system at that moment. For example, a condition can test if a switch is currently turned on or off.

Unlike a trigger, which is always `or`, conditions are `and` by default - all conditions have to be true.

All conditions support an optional `alias`.

## Logical conditions

---

### AND condition

Test multiple conditions in one condition statement. Passes if all embedded conditions are true.

```
conditions:
- alias: "Paulus home AND temperature below 20"
 condition: and
 conditions:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20
```

If you do not want to combine AND and OR conditions, you can list them sequentially.

The following configuration works the same as the one listed above:

```
conditions:
- condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
- condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20
```

Currently you need to format your conditions like this to be able to edit them using the [automations editor](#).

The AND condition also has a shorthand form. The following configuration works the same as the ones listed above:

```
conditions:
 alias: "Paulus home AND temperature below 20"
 - and:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20
```

### OR condition

Test multiple conditions in one condition statement. Passes if any embedded condition is true.

```

conditions:
- alias: "Paulus home OR temperature below 20"
 condition: or
 conditions:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20

```

The OR condition also has a shorthand form. The following configuration works the same as the one listed above:

```

conditions:
- alias: "Paulus home OR temperature below 20"
 or:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20

```

## Mixed AND and OR conditions

Test multiple AND and OR conditions in one condition statement. Passes if any embedded condition is true. This allows you to mix several AND and OR conditions together.

```

conditions:
- condition: and
 conditions:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - condition: or
 conditions:
 - condition: state
 entity_id: sensor.weather_precip
 state: "rain"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20

```

Or in shorthand form:

```

conditions:
- and:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - or:
 - condition: state
 entity_id: sensor.weather_precip
 state: "rain"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20

```

## NOT condition

Test multiple conditions in one condition statement. Passes if all embedded conditions are **not** true.

```

conditions:
- alias: "Paulus not home AND alarm not disarmed"
 condition: not
 conditions:
 - condition: state
 entity_id: device_tracker.paulus
 state: "home"
 - condition: state
 entity_id: alarm_control_panel.home_alarm
 state: "disarmed"

```

The NOT condition also has a shorthand form. The following configuration works the same as the one listed above:

```

conditions:
 alias: "Paulus not home AND alarm not disarmed"
 not:
 - condition: state
 entity_id: device_tracker.paulus
 state: "home"
 - condition: state
 entity_id: alarm_control_panel.home_alarm
 state: disarmed

```

## Numeric state condition

---

This type of condition attempts to parse the state of the specified entity or the attribute of an entity as a number, and triggers if the value matches the thresholds (strictly below/above, so equal excluded).

If both `below` and `above` are specified, both tests have to pass.

```
conditions:
- alias: "Temperature between 17 and 25 degrees"
 condition: numeric_state
 entity_id: sensor.temperature
 above: 17
 below: 25
```

You can optionally use a `value_template` to process the value of the state before testing it.

```
conditions:
- condition: numeric_state
 entity_id: sensor.temperature
 above: 17
 below: 25
 # If your sensor value needs to be adjusted
 value_template: "{{ float(state.state) + 2 }}"
```

It is also possible to test the condition against multiple entities at once. The condition will pass if **all** entities match the thresholds.

```
conditions:
- condition: numeric_state
 entity_id:
 - sensor.kitchen_temperature
 - sensor.living_room_temperature
 below: 18
```

Alternatively, the condition can test against a state attribute. The condition will pass if the attribute value of the entity matches the thresholds.

```
conditions:
- condition: numeric_state
 entity_id: climate.living_room_thermostat
 attribute: temperature
 above: 17
 below: 25
```

Number helpers (`input_number` entities), `number`, `sensor`, and `zone` entities that contain a numeric value, can be used in the `above` and `below` options to make the condition more dynamic.

```
conditions:
- condition: numeric_state
 entity_id: climate.living_room_thermostat
 attribute: temperature
 above: input_number.temperature_threshold_low
 below: input_number.temperature_threshold_high
```



## State condition

---

Tests if an entity has a specified state.

```
conditions:
- alias: "Paulus not home for an hour and a bit"
 condition: state
 entity_id: device_tracker.paulus
 state: "not_home"
 # optional: Evaluates to true only if state was this for last X time.
 for:
 hours: 1
 minutes: 10
 seconds: 5
```

It is also possible to test the condition against multiple entities at once. The condition will pass if **all** entities match the state.

```
conditions:
- condition: state
 entity_id:
 - light.kitchen
 - light.living_room
 state: "on"
```

Instead of matching all, it is also possible if one of the entities matches. In the following example the condition will pass if **any** entity matches the state.

```
conditions:
- condition: state
 entity_id:
 - binary_sensor.motion_sensor_left
 - binary_sensor.motion_sensor_right
 match: any
 state: "on"
```

Testing if an entity is matching a set of possible conditions; The condition will pass if the entity matches one of the states given.

```
conditions:
- condition: state
 entity_id: alarm_control_panel.home
 state:
 - "armed_away"
 - "armed_home"
```

Or, combine multiple entities with multiple states. In the following example, both media players need to be either paused or playing for the condition to pass.

```

conditions:
- condition: state
 entity_id:
 - media_player.living_room
 - media_player.kitchen
 state:
 - "playing"
 - "paused"

```

Alternatively, the condition can test against a state attribute. The condition will pass if the attribute matches the given state.

```

conditions:
- condition: state
 entity_id: climate.living_room_thermostat
 attribute: fan_mode
 state: "auto"

```

Finally, the `state` option accepts helper entities (also known as `input_*` entities). The condition will pass if the state of the entity matches the state of the given helper entity.

```

conditions:
- condition: state
 entity_id: alarm_control_panel.home
 state: input_select.guest_mode

```

You can also use templates in the `for` option.

```

conditions:
- condition: state
 entity_id: device_tracker.paulus
 state: "home"
 for:
 minutes: "{{ states('input_number.lock_min')|int }}"
 seconds: "{{ states('input_number.lock_sec')|int }}"

```

The `for` template(s) will be evaluated when the condition is tested.

## Sun condition

### Sun state condition

The sun state can be used to test if the sun has set or risen.

```

conditions:
- alias: "Sun up"
 condition: state # 'day' condition: from sunrise until sunset
 entity_id: sun.sun
 state: "above_horizon"

```

```
conditions:
- alias: "Sun down"
 condition: state # from sunset until sunrise
 entity_id: sun.sun
 state: "below_horizon"
```

## Sun elevation condition

The sun elevation can be used to test if the sun has set or risen, it is dusk, or it is night when a trigger occurs. For an in-depth explanation of sun elevation, see [sun elevation trigger](#).

```
conditions:
- condition: and # 'twilight' condition: dusk and dawn, in typical locations
 conditions:
 - condition: template
 value_template: "{{ state_attr('sun.sun', 'elevation') < 0 }}"
 - condition: template
 value_template: "{{ state_attr('sun.sun', 'elevation') > -6 }}"
```

```
conditions:
 condition: template # 'night' condition: from dusk to dawn, in typical locati
 value_template: "{{ state_attr('sun.sun', 'elevation') < -6 }}"
```

## Sunset/sunrise condition

The sun condition can also test if the sun has already set or risen when a trigger occurs. The `before` and `after` keys can only be set to `sunset` or `sunrise`. They have a corresponding optional offset value (`before_offset`, `after_offset`) that can be added, similar to the [sun trigger](#).

Note that if only `before` key is used, the condition will be true *from midnight* until sunrise/sunset. If only `after` key is used, the condition will be true from sunset/sunrise *until midnight*. If both `before: sunrise` and `after: sunset` keys are used, the condition will be true *from midnight* until sunrise **and** from sunset *until midnight*. If both `after: sunrise` and `before: sunset` keys are used, the condition will be true from sunrise until sunset.

 **Tip:** The sunset/sunrise conditions do not work in locations inside the polar circles, and also not in locations with a highly skewed local time zone. In those cases it is advised to use conditions evaluating the solar elevation instead of the before/after sunset/sunrise conditions.

This is an example of 1 hour offset before sunset:

```
conditions:
- condition: sun
 after: sunset
 after_offset: "-01:00:00"
```

This is 'when dark' - equivalent to a state condition on `sun.sun` of `below_horizon` :

```
conditions:
- condition: sun
 after: sunset
 before: sunrise
```

This is 'when light' - equivalent to a state condition on `sun.sun` of `above_horizon` :

```
conditions:
- condition: sun
 after: sunrise
 before: sunset
```

A visual timeline is provided below, showing an example of when these conditions are true. In this chart, sunrise is at 6:00, and sunset is at 18:00 (6:00 PM). The green areas of the chart indicate when the specified conditions are true.

Graphic showing an example of sun conditions

## Template condition

The template condition tests if the [given template](#) renders a value equal to true. This is achieved by having the template result in a true boolean expression or by having the template render `True`.

```
conditions:
- alias: "Iphone battery above 50%"
 condition: template
 value_template: "{{ (state_attr('device_tracker.iphone', 'battery_level')|in
```

Within an automation, template conditions also have access to the `trigger` variable as [described here](#).

## Template condition shorthand notation

The template condition has a shorthand notation that can be used to make your scripts and automations shorter.

For example:

```
conditions: "{{ (state_attr('device_tracker.iphone', 'battery_level')|int) > 50
```

Or in a list of conditions, allowing to use existing conditions as described in this chapter and one or more shorthand template conditions

```
conditions:
- "{{ (state_attr('device_tracker.iphone', 'battery_level')|int) > 50 }}"
- condition: state
 entity_id: alarm_control_panel.home
 state: armed_away
- "{{ is_state('device_tracker.iphone', 'away') }}"
```

This shorthand notation can be used everywhere in Home Assistant where conditions are accepted. For example, in `and`, `or` and `not` conditions:

```
conditions:
- condition: or
 conditions:
 - "{{ is_state('device_tracker.iphone', 'away') }}"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20
```

It's also supported in the `repeat` action's `while` or `until` option, or in a `choose` action's `conditions` option:

```
- while: "{{ is_state('sensor.mode', 'Home') and repeat.index < 10 }}"
 sequence:
 - ...
```

```
- choose:
 - conditions: "{{ is_state('sensor.mode', 'Home') and repeat.index < 10 }}"
 sequence:
 - ...
```

It's also supported in script or automation `condition` actions:

```
- condition: "{{ is_state('device_tracker.iphone', 'away') }}"
```

## Time condition

The time condition can test if it is after a specified time, before a specified time or if it is a certain day of the week.

```
conditions:
- alias: "Time 15~02"
 condition: time
 # At least one of the following is required.
 after: "15:00:00"
 before: "02:00:00"
 weekday:
 - mon
 - wed
 - fri
```

Valid values for `weekday` are `mon`, `tue`, `wed`, `thu`, `fri`, `sat`, `sun`. Note that if only `before` key is used, the condition will be `true` *from midnight* until the specified time. If only `after` key is used, the condition will be `true` from the specified time *until midnight*.

Time condition windows can span across the midnight threshold if **both** `after` and `before` keys are used. In the example above, the condition window is from 3pm to 2am.

The after times are inclusive while before are exclusive. In the example above, if the time was at 3pm (15:00:00) then it meets the after time condition. If the time was at 2am (2:00:00), it would fail the condition because it will only be valid up to 1:59:59.

 **Tip:** A better weekday condition could be by using the [Workday Binary Sensor](#).

For the `after` and `before` options a time helper ( `input_datetime` entity), a `time` entity, or another `sensor` entity containing a timestamp with the "timestamp" device class, can be used instead.

```
conditions:
- alias: "Example referencing a time helper"
 condition: time
 after: input_datetime.house_silent_hours_start
 before: input_datetime.house_silent_hours_end

- alias: "Example referencing a time entity"
 before: time.dnd_start

- alias: "Example referencing another sensor"
 after: sensor.groceries_delivery_time
```

 **Note:** Note that the time condition only takes the time into account. If a referenced sensor or helper entity contains a timestamp with a date, the date part is fully ignored.

## Trigger condition

---

The trigger condition can test if an automation was triggered by a certain trigger, identified by the trigger's `id`.

```
conditions:
 - condition: trigger
 id: event_trigger
```

For a trigger identified by its index, both a string and integer is allowed:

```
conditions:
 - condition: trigger
 id: "0"
```

```
conditions:
 - condition: trigger
 id: 0
```

It is possible to give a list of triggers:

```
conditions:
 - condition: trigger
 id:
 - event_1_trigger
 - event_2_trigger
```



## Zone condition

---

Zone conditions test if an entity is in a certain zone. For zone automation to work, you need to have set up a device tracker platform that supports reporting GPS coordinates.

```
conditions:
 - alias: "Paulus at home"
 condition: zone
 entity_id: device_tracker.paulus
 zone: zone.home
```

It is also possible to test the condition against multiple entities at once. The condition will pass if all entities are in the specified zone.

```
conditions:
 - condition: zone
 entity_id:
 - device_tracker.frenck
 - device_tracker.daphne
 zone: zone.home
```

Testing if an entity is matching a set of possible zones; The condition will pass if the entity is in one of the zones.

```
conditions:
 - condition: zone
 entity_id: device_tracker.paulus
 state:
 - zone.home
 - zone.work
```

Or, combine multiple entities with multiple zones. In the following example, both entities need to be either in the home or the work zone for the condition to pass.

```
conditions:
 condition: zone
 entity_id:
 - device_tracker.frenck
 - device_tracker.daphne
 state:
 - zone.home
 - zone.work
```

## Examples

---

```
conditions:
- condition: numeric_state
 entity_id: sun.sun
 value_template: "{{ state.attributes.elevation }}"
 below: 1
- condition: state
 entity_id: light.living_room
 state: "off"
- condition: time
 before: "23:00:00"
 after: "14:00:00"
- condition: state
 entity_id: script.light_turned_off_5min
 state: "off"
```

## Disabling a condition

---

Every individual condition can be disabled, without removing it. To do so, add `enabled: false` to the condition configuration.

This can be useful if you want to temporarily disable a condition, for example, for testing. A disabled condition will behave as if it were removed.

For example:

```
This condition will always pass, as it is disabled.
conditions:
 - enabled: false
 condition: state
 entity_id: sun.sun
 state: "above_horizon"
```

Conditions can also be disabled based on limited templates or blueprint inputs.

```
blueprint:
 input:
 input_boolean:
 name: Boolean
 selector:
 boolean:
 input_number:
 name: Number
 selector:
 number:
 min: 0
 max: 100

 trigger_variables:
 _enable_number: !input input_number

 conditions:
 - condition: state
 entity_id: sun.sun
 state: "above_horizon"
 enabled: !input input_boolean
 - condition: state
 entity_id: sun.sun
 state: "below_horizon"
 enabled: "{{ _enable_number < 50 }}"
```

## Docs/Scripts/Perform Actions

---

Various integrations allow performing actions when a certain event occurs. The most common one is performing an action when an automation trigger happens. But an action can also be called from a script, a dashboard, or via voice command devices such as Amazon Echo.

The configuration options to perform action are the same between all integrations and are described on this page.

Examples on this page will be given as part of an automation integration configuration but different approaches can be used for other integrations too.

 **Tip:** Use the **Actions** tab under **Settings** > **Developer tools** > **Actions** to discover available actions.

### The basics

Perform the action `homeassistant.turn_on` on the entity `group.living_room`. This will turn all members of `group.living_room` on. You can also use `entity_id: all` and it will turn on all possible entities.

```
action: homeassistant.turn_on
target:
 entity_id: group.living_room
```

### Targeting areas and devices

Instead of targeting an entity, you can also target an area or device. Or a combination of these. This is done with the `target` key.

A `target` is a map that contains at least one of the following: `area_id`, `device_id`, `entity_id`. Each of these can be a list. The values should be lower-cased.

The following example uses a single action to turn on the lights in the living room area, 2 additional light devices and 2 additional light entities:

```
action: light.turn_on
target:
 area_id: living_room
 device_id:
 - ff22a1889a6149c5ab6327a8236ae704
 - 52c050ca1a744e238ad94d170651f96b
 entity_id:
 - light.hallway
 - light.landing
```

## Passing data to the action

You can also specify other parameters beside the entity to target. For example, the `light.turn_on` action allows specifying the brightness.

```
action: light.turn_on
target:
 entity_id: group.living_room
data:
 brightness: 120
 rgb_color: [255, 0, 0]
```

A full list of the parameters for an action can be found on the documentation page of each integration, in the same way as it's done for the `light.turn_on` action.

## Use templates to decide which action to perform

You can use [templating](#) support to dynamically choose which action to perform. For example, you can perform a certain action based on if a light is on.

```
action: >
 {% if states('sensor.temperature') | float > 15 %}
 switch.turn_on
 {% else %}
 switch.turn_off
 {% endif %}
entity_id: switch.ac
```

## Using the Actions developer tool

You can use the **Actions** developer tool to test data to pass in an action. For example, you may test turning on or off a 'group' (See [groups](#) for more info)

To turn a group on or off, pass the following info:

- Domain: `homeassistant`
- Action: `turn_on`
- Action data: `{ "entity_id": "group.kitchen" }`

## Use templates to determine the attributes

Templates can also be used for the data that you pass to the action.

```

action: thermostat.set_temperature
target:
 entity_id: >
 {% if is_state('device_tracker.paulus', 'home') %}
 thermostat.upstairs
 {% else %}
 thermostat.downstairs
 {% endif %}
data:
 temperature: "{{ 22 - distance(states.device_tracker.paulus) }}"

```

You can use a template returning a native dictionary as well, which is useful if the attributes to be set depend on the situation.

```

action: climate.set_temperature
data: >
 {% if states('sensor.temperature_living') < 19 %}
 {"hvac_mode": "heat", "temperature": 19 }
 {% else %}
 {"hvac_mode": "auto" }
 {% endif %}

```

## Use templates to handle response data

Some actions may respond with data that can be used in automation. This data is called *action response data*. Action response data is typically used for data that is dynamic or large and which may not be suited for use in entity state. Examples of action response data are upcoming calendar events for the next week or detailed driving directions.

Templates can also be used for handling response data. The action can specify a `response_variable`. This is the `variable` that contains the response data. You can define any name for your `response_variable`. This example performs an action and stores the response in the variable called `agenda`.

```

action: calendar.get_events
target:
 entity_id: calendar.school
data:
 duration:
 hours: 24
 response_variable: agenda

```

You may then use the response data in the variable `agenda` in another action in the same script. The example below sends a notification using the response data.

 **Important:** Which data fields can be used in an action depends on the type of notification that is used.

```
action: notify.gmail_com
data:
 target: "gduser1@workspacesamples.dev"
 title: "Daily agenda for {{ now().date() }}"
 message: >-
 Your agenda for today:
 <p>
 {% for event in agenda['calendar.school'].events %}
 {{ event.start}}: {{ event.summary }}

 {% endfor %}
 </p>
```

## homeassistant actions

There are four `homeassistant` actions that aren't tied to any single domain, these are:

- `homeassistant.turn_on` - Turns on an entity (that supports being turned on), such as an `automation` or `switch`.
- `homeassistant.turn_off` - Turns off an entity (that supports being turned off), such as an `automation` or `switch`.
- `homeassistant.toggle` - Turns off an entity that is on, or turns on an entity that is off (that supports being turned on and off)
- `homeassistant.update_entity` - Request the update of an entity, rather than waiting for the next scheduled update, for example [Google travel time](#) sensor, a [template sensor](#), or a [light](#)

Complete action details and examples can be found on the [Home Assistant integration](#) page.

---

## Docs/Scripts

---

A script is a sequence of steps that Home Assistant runs from top to bottom whenever you call it. Think of it as a small recipe: "turn on the porch light, wait 30 seconds, then send me a notification". Once you have written a script, you can run it from a button on your dashboard, from Assist, from inside an automation, or from anywhere else that calls actions.

Scripts and automations are very closely related. The only real difference is that an automation runs by itself when something triggers it, and a script runs when you call it.

When the script runs as part of an automation, the `trigger` variable is also available. See [Available-Trigger-Data](#).



## Script syntax

---

The script syntax basic structure is a list of key/value maps that contain actions. If a script contains only 1 action, the wrapping list can be omitted.

All actions support an optional `alias`.

```
Example script integration containing script syntax
script:
 example_script:
 sequence:
 # This is written using the Script Syntax
 - alias: "Turn on ceiling light"
 action: light.turn_on
 target:
 entity_id: light.ceiling
 - alias: "Notify that ceiling light is turned on"
 action: notify.notify
 data:
 message: "Turned on the ceiling light!"
```

## Perform an action

---

Performing an action can be done in various ways. For all the different possibilities, have a look at the [actions page](#).

```
- alias: "Bedroom lights on"
 action: light.turn_on
 target:
 entity_id: group.bedroom
 data:
 brightness: 100
```

## Activate a scene

Scripts may also use a shortcut syntax for activating scenes instead of calling the `scene.turn_on` action.

```
- scene: scene.morning_living_room
```

## Variables

---

The variables action allows you to set/override variables that will be accessible by templates in action after it. See also [script variables](#) for how to define variables accessible in the entire script.

```
- alias: "Set variables"
 variables:
 entities:
 - light.kitchen
 - light.living_room
 brightness: 100
- alias: "Control lights"
 action: light.turn_on
 target:
 entity_id: "{{ entities }}"
 data:
 brightness: "{{ brightness }}"
```

Variables can be templated.

```
- alias: "Set a templated variable"
 variables:
 blind_state_message: "The blind is {{ states('cover.blind') }}."
- alias: "Notify about the state of the blind"
 action: notify.mobile_app_iphone
 data:
 message: "{{ blind_state_message }}"
```

## Scope of variables

The `variables` action assigns the values to previously defined variables with the same name. If a variable was not previously defined, it is assigned in the top-level (script run) scope.

```

sequence:
 # Set the people variable to a default value
 - variables:
 people: 0
 # Try to increment people if Paulus is home
 - if:
 - condition: state
 entity_id: device_tracker.paulus
 state: "home"
 then:
 - variables:
 people: "{{ people + 1 }}"
 paulus_home: true
 - action: notify.notify
 data:
 message: "There are {{ people }} people home" # "There are 1 people home"
 # Variable value is now updated
 - action: notify.notify
 data:
 message: "There are {{ people }} people home {% if paulus_home is defined %}
 # "There are 1 people home (including Paulus)"

```

## Test a condition

---

While executing a script you can add a condition in the main sequence to stop further execution. When a condition does not return `true`, the script will stop executing. For documentation on the many different conditions refer to the [conditions page](#).

 **Note:** The `condition` action only stops executing the current sequence block. When it is used inside a `repeat` action, only the current iteration of the `repeat` loop will stop. When it is used inside a `choose` action, only the actions within that `choose` will stop.

```
If paulus is home, continue to execute the script below these lines
- alias: "Check if Paulus is home"
 condition: state
 entity_id: device_tracker.paulus
 state: "home"
```

`condition` can also be a list of conditions and execution will then only continue if ALL conditions return `true`.

```
- alias: "Check if Paulus is home AND temperature is below 20"
 conditions:
 - condition: state
 entity_id: "device_tracker.paulus"
 state: "home"
 - condition: numeric_state
 entity_id: "sensor.temperature"
 below: 20
```

## Wait for time to pass (delay)

---

Delays are useful for temporarily suspending your script and start it at a later moment. We support different syntaxes for a delay as shown below.

```
Seconds
Waits 5 seconds
- alias: "Wait 5s"
 delay: 5
```

```
HH:MM
Waits 1 hour
- delay: "01:00"
```

```
HH:MM:SS
Waits 1.5 minutes
- delay: "00:01:30"
```

```
Supports milliseconds, seconds, minutes, hours, days
Can be used in combination, at least one required
When using milliseconds, consider that delay as *at least* X milliseconds. It
Waits 1 minute
- delay:
 minutes: 1
```

All forms accept templates.

```
Waits however many minutes input_number.minute_delay is set to
- delay: "{{ states('input_number.minute_delay') | multiply(60) | int }}"
```

## Wait

---

These actions allow a script to wait for entities in the system to be in a certain state as specified by a template, or some event to happen as expressed by one or more triggers.

### Wait for a template

This action evaluates the template, and if true, the script will continue. If not, then it will wait until it is true.

The template is re-evaluated whenever an entity ID that it references changes state. If you use non-deterministic functions like `now()` in the template it will not be continuously re-evaluated, but only when an entity ID that is referenced is changed. If you need to periodically re-evaluate the template, reference a sensor from the [Time and Date](#) integration that will update minutely or daily.

```
Wait until media player is stopped
- alias: "Wait until media player is stopped"
 wait_template: "{{ is_state('media_player.floor', 'stop') }}"
```

### Wait for a trigger

This action can use the same triggers that are available in an automation's `trigger` section. See [Automation Trigger](#). The script will continue whenever any of the triggers fires. All previously defined [trigger variables](#), [variables](#) and [script variables](#) are passed to the trigger.

```
Wait for a custom event or light to turn on and stay on for 10 sec
- alias: "Wait for MY_EVENT or light on"
 wait_for_trigger:
 - trigger: event
 event_type: MY_EVENT
 - trigger: state
 entity_id: light.LIGHT
 to: "on"
 for: 10
```

### Wait timeout

With both types of waits it is possible to set a timeout after which the script will continue its execution if the condition/event is not satisfied. Timeout has the same syntax as `delay`, and like `delay`, also accepts templates.

```
Wait for sensor to change to 'on' up to 1 minute before continuing to execute.
- wait_template: "{{ is_state('binary_sensor.entrance', 'on') }}"
 timeout: "00:01:00"
```

You can also get the script to abort after the timeout by using optional `continue_on_timeout: false`.

```
Wait for IFTTT event or abort after specified timeout.
- wait_for_trigger:
 - trigger: event
 event_type: ifttt_webhook_received
 event_data:
 action: connected_to_network
 timeout:
 minutes: "{{ timeout_minutes }}"
 continue_on_timeout: false
```

Without `continue_on_timeout: false` the script will always continue since the default for `continue_on_timeout` is `true`.

## Wait variable

After each time a wait completes, either because the condition was met, the event happened, or the timeout expired, the variable `wait` will be created/updated to indicate the result.

Variable	Description
<code>wait.completed</code>	<code>true</code> if the condition was met, <code>false</code> otherwise
<code>wait.remaining</code>	Timeout remaining, or <code>none</code> if a timeout was not specified
<code>wait.trigger</code>	Exists only after <code>wait_for_trigger</code> . Contains information about which trigger fired. (See <a href="#">Available-Trigger-Data</a> .) Will be <code>none</code> if no trigger happened before timeout expired

This can be used to take different actions based on whether or not the condition was met, or to use more than one wait sequentially while implementing a single timeout overall.



```

Take different actions depending on if condition was met.
- wait_template: "{{ is_state('binary_sensor.door', 'on') }}"
 timeout: 10
- if:
 - "{{ not wait.completed }}"
 then:
 - action: script.door_did_not_open
 else:
 - action: script.turn_on
 target:
 entity_id:
 - script.door_did_open
 - script.play_fanfare

Wait a total of 10 seconds.
- wait_template: "{{ is_state('binary_sensor.door_1', 'on') }}"
 timeout: 10
 continue_on_timeout: false
- action: switch.turn_on
 target:
 entity_id: switch.some_light
- wait_for_trigger:
 - trigger: state
 entity_id: binary_sensor.door_2
 to: "on"
 for: 2
 timeout: "{{ wait.remaining }}"
 continue_on_timeout: false
- action: switch.turn_off
 target:
 entity_id: switch.some_light

```

## Fire an event

---

This action allows you to fire an event. In the GUI (Graphical User Interface) for actions it is found under "Fire Manual Event". Events can be used for many things. It could trigger an automation or indicate to another integration that something is happening. For instance, in the below example it is used to create an entry in the **Activity** panel.

```
- alias: "Fire LOGBOOK_ENTRY event"
 event: LOGBOOK_ENTRY
 event_data:
 name: Paulus
 message: is waking up
 entity_id: device_tracker.paulus
 domain: light
```

You can also use event\_data to fire an event with custom data. This could be used to pass data to another script awaiting an event trigger.

The `event_data` accepts templates.

```
- event: MY_EVENT
 event_data:
 name: myEvent
 customData: "{{ myCustomVariable }}"
```

## Raise and consume custom events

The following automation example shows how to raise a custom event called `event_light_state_changed` with `entity_id` as the event data. The action part could be inside a script or an automation.

```
- alias: "Fire Event"
 triggers:
 - trigger: state
 entity_id: switch.kitchen
 to: "on"
 actions:
 - event: event_light_state_changed
 event_data:
 state: "on"
```

The following automation example shows how to capture the custom event `event_light_state_changed` with an [Event Automation Trigger](#), and retrieve corresponding `entity_id` that was passed as the event trigger data, see [Available-Trigger-Data](#) for more details.

```
- alias: "Capture Event"
 triggers:
 - trigger: event
 event_type: event_light_state_changed
 actions:
 - action: notify.notify
 data:
 message: "kitchen light is turned {{ trigger.event.data.state }}"
```

## Repeat a group of actions

---

This action allows you to repeat a sequence of other actions. Nesting is fully supported. There are three ways to control how many times the sequence will be run.

### Counted repeat

This form accepts a count value. The value may be specified by a template, in which case the template is rendered when the repeat step is reached.

```
script:
 flash_light:
 mode: restart
 sequence:
 - action: light.turn_on
 target:
 entity_id: "light.{{ light }}"
 - alias: "Cycle light 'count' times"
 repeat:
 count: "{{ count|int * 2 - 1 }}"
 sequence:
 - delay: 2
 - action: light.toggle
 target:
 entity_id: "light.{{ light }}"
 flash_hallway_light:
 sequence:
 - alias: "Flash hallway light 3 times"
 action: script.flash_light
 data:
 light: hallway
 count: 3
```

### For each

This repeat form accepts a list of items to iterate over. The list of items can be a pre-defined list, or a list created by a template.

The sequence is run for each item in the list, and current item in the iteration is available as `repeat.item`.

The following example will turn a list of lights:

```
repeat:
 for_each:
 - "living_room"
 - "kitchen"
 - "office"
 sequence:
 - action: light.turn_off
 target:
 entity_id: "light.{{ repeat.item }}"
```

Other types are accepted as list items, for example, each item can be a template, or even a mapping of key/value pairs.

```
repeat:
 for_each:
 - language: English
 message: Hello World
 - language: Dutch
 message: Hallo Wereld
 sequence:
 - action: notify.phone
 data:
 title: "Message in {{ repeat.item.language }}"
 message: "{{ repeat.item.message }}!"
```

## While loop

This form accepts a list of conditions (see [conditions page](#) for available options) that are evaluated *before* each time the sequence is run. The sequence will be run *as long as* the condition(s) evaluate to true.

```
script:
 do_something:
 sequence:
 - action: script.get_ready_for_something
 - alias: "Repeat the sequence AS LONG AS the conditions are true"
 repeat:
 while:
 - condition: state
 entity_id: input_boolean.do_something
 state: "on"
 # Don't do it too many times
 - condition: template
 value_template: "{{ repeat.index <= 20 }}"
 sequence:
 - action: script.something
```

The `while` also accepts a [shorthand notation of a template condition](#). For example:

```
- repeat:
 while: "{{ is_state('sensor.mode', 'Home') and repeat.index < 10 }}"
 sequence:
 - ...
```

## Repeat until

This form accepts a list of conditions that are evaluated *after* each time the sequence is run. Therefore the sequence will always run at least once. The sequence will be run *until* the condition(s) evaluate to true.

```

automation:
- triggers:
 - trigger: state
 entity_id: binary_sensor.xyz
 to: "on"
 conditions:
 - condition: state
 entity_id: binary_sensor.something
 state: "off"
 actions:
 - alias: "Repeat the sequence UNTIL the conditions are true"
 repeat:
 sequence:
 # Run command that for some reason doesn't always work
 - action: shell_command.turn_something_on
 # Give it time to complete
 - delay:
 milliseconds: 200
 until:
 # Did it work?
 - condition: state
 entity_id: binary_sensor.something
 state: "on"

```

`until` also accepts a [shorthand notation of a template condition](#). For example:

```

- repeat:
 until: "{{ is_state('device_tracker.iphone', 'home') }}"
 sequence:
 - ...

```

## Repeat loop variable

A variable named `repeat` is defined within the repeat action, meaning it is available inside `sequence`, `while`, and `until`. It contains the following fields:

field	description
<code>first</code>	True during the first iteration of the repeat sequence
<code>index</code>	The iteration number of the loop: 1, 2, 3, ...
<code>last</code>	True during the last iteration of the repeat sequence, which is only valid for counted loops

## If-then

---

This action allows you to conditionally ( `if` ), based on or more `conditions` (which are `and` combined), run a sequence of actions ( `then` ) and optionally supports running other sequence when the condition didn't pass ( `else` ).

```
script:
 - if:
 - alias: "If no one is home"
 condition: state
 entity_id: zone.home
 state: 0
 then:
 - alias: "Then start cleaning already!"
 action: vacuum.start
 target:
 area_id: living_room
 # The `else` is fully optional and can be omitted
 else:
 - action: notify.notify
 data:
 message: "Skipped cleaning, someone is home!"
```

This action supports nesting, however, if you find yourself using nested if-then actions in the `else` part, you may want to consider using `choose` instead.

## Choose a group of actions

This action allows you to select a sequence of other actions from a list of sequences. Nesting is fully supported.

Each sequence is paired with a list of conditions. (See the [conditions page](#) for available options and how multiple conditions are handled.) The first sequence whose conditions are all true will be run. An *optional* `default` sequence can be included which will be run only if none of the sequences from the list are run.

An *optional* `alias` can be added to each of the sequences, excluding the `default` sequence.

The `choose` action can be used like an "if/then/elseif/then.../else" statement. The first `conditions` / `sequence` pair is like the "if/then", and can be used just by itself. Or additional pairs can be added, each of which is like an "elif/then". And lastly, a `default` can be added, which would be like the "else."

```
Example with "if", "elif" and "else"
automation:
 - triggers:
 - trigger: state
 entity_id: input_boolean.simulate
 to: "on"
 mode: restart
 actions:
 - choose:
 # IF morning
 - conditions:
 - condition: template
 value_template: "{{ now().hour < 9 }}"
 sequence:
 - action: script.sim_morning
 # ELIF day
 - conditions:
 - condition: template
 value_template: "{{ now().hour < 18 }}"
 sequence:
 - action: light.turn_off
 target:
 entity_id: light.living_room
 - action: script.sim_day
 # ELSE night
 default:
 - action: light.turn_off
 target:
 entity_id: light.kitchen
 - delay:
 minutes: "{{ range(1, 11)|random }}"
 - action: light.turn_off
 target:
 entity_id: all
```

`conditions` also accepts a [shorthand notation of a template condition](#). For example:



```

automation:
- triggers:
 - trigger: state
 entity_id: input_select.home_mode
 actions:
 - choose:
 - conditions: >
 {{ trigger.to_state.state == 'Home' and
 is_state('binary_sensor.all_clear', 'on') }}
 sequence:
 - action: script.arrive_home
 data:
 ok: true
 - conditions: >
 {{ trigger.to_state.state == 'Home' and
 is_state('binary_sensor.all_clear', 'off') }}
 sequence:
 - action: script.turn_on
 target:
 entity_id: script.flash_lights
 - action: script.arrive_home
 data:
 ok: false
 - conditions: "{{ trigger.to_state.state == 'Away' }}"
 sequence:
 - action: script.left_home

```

More `choose` can be used together. This is the case of an IF-IF.

The following example shows how a single automation can control entities that aren't related to each other but have in common the same trigger.

When the sun goes below the horizon, the `porch` and `garden` lights must turn on. If someone is watching the TV in the living room, there is a high chance that someone is in that room, therefore the living room lights have to turn on too. The same concept applies to the `studio` room.

```

Example with "if" and "if"
automation:
 - alias: "Turn lights on when the sun gets dim and if some room is occupied"
 triggers:
 - trigger: numeric_state
 entity_id: sun.sun
 attribute: elevation
 below: 4
 actions:
 # This must always apply
 - action: light.turn_on
 data:
 brightness: 255
 color_temp: 366
 target:
 entity_id:
 - light.porch
 - light.garden
 # IF a entity is ON
 - choose:
 - conditions:
 - condition: state
 entity_id: binary_sensor.livingroom_tv
 state: "on"
 sequence:
 - action: light.turn_on
 data:
 brightness: 255
 color_temp: 366
 target:
 entity_id: light.livingroom
 # IF another entity not related to the previous, is ON
 - choose:
 - conditions:
 - condition: state
 entity_id: binary_sensor.studio_pc
 state: "on"
 sequence:
 - action: light.turn_on
 data:
 brightness: 255
 color_temp: 366
 target:
 entity_id: light.studio

```

## Grouping actions

---

The `sequence` action allows you to group multiple actions together. Each action will be executed in order, meaning the next action will only be executed after the previous action has been completed.

Grouping actions in a sequence can be useful when you want to be able to collapse related groups in the user interface for organizational purposes.

Combined with the `parallel` action, it can also be used to run multiple groups of actions in a sequence in parallel.

In the example below, two separate groups of actions are executed in sequence, one for turning on devices, the other for sending notifications. Each group of actions is executed in order, this includes the actions in each group and the groups themselves. In total, four actions are executed, one after the other.

```
automation:
- triggers:
 - trigger: state
 entity_id: binary_sensor.motion
 to: "on"
 actions:
 - alias: "Turn on devices"
 sequence:
 - action: light.turn_on
 target:
 entity_id: light.ceiling
 - action: siren.turn_on
 target:
 entity_id: siren.noise_maker
 - alias: "Send notifications"
 sequence:
 - action: notify.person1
 data:
 message: "The motion sensor was triggered!"
 - action: notify.person2
 data:
 message: "Oh oh, someone triggered the motion sensor..."
```

## Parallelizing actions

By default, all sequences of actions in Home Assistant run sequentially. This means the next action is started after the current action has been completed.

This is not always needed, for example, if the sequence of actions doesn't rely on each other and order doesn't matter. For those cases, the `parallel` action can be used to run the actions in the sequence in parallel, meaning all the actions are started at the same time.

The following example shows sending messages out at the same time (in parallel):

```
automation:
- triggers:
 - trigger: state
 entity_id: binary_sensor.motion
 to: "on"
 actions:
 - parallel:
 - action: notify.person1
 data:
 message: "These messages are sent at the same time!"
 - action: notify.person2
 data:
 message: "These messages are sent at the same time!"
```

It is also possible to run a group of actions sequentially inside the parallel actions. The example below demonstrates that:

```
script:
 example_script:
 sequence:
 - parallel:
 - sequence:
 - wait_for_trigger:
 - trigger: state
 entity_id: binary_sensor.motion
 to: "on"
 - action: notify.person1
 data:
 message: "This message awaited the motion trigger"
 - action: notify.person2
 data:
 message: "I am sent immediately and do not await the above action!"
```

 **Warning:** Running actions in parallel can be helpful in many cases, but use it with caution and only if you need it.

There are some caveats (see below) when using parallel actions.

While it sounds attractive to parallelize, most of the time, just the regular sequential actions will work just fine.

Some of the caveats of running actions in parallel:

- There is no order guarantee. The actions will be started in parallel, but there is no guarantee that they will be completed in the same order.
- If one action fails or errors, the other actions will keep running until they too have finished or errored.
- Variables created/modified in one parallelized action can conflict with variables from another parallelized action. Make sure to give them distinct names to prevent that.

## Stopping a script sequence

---

It is possible to halt a script sequence at any point and return script responses using the `stop` action.

The `stop` action takes a text as input explaining the reason for halting the sequence. This text will be logged and shows up in the automations and script traces.

`stop` can be useful to halt a script halfway through a sequence when, for example, a condition is not met.

```
- stop: "Stop running the rest of the sequence"
```

To return a response from a script, use the `response_variable` option. This option expects the name of the variable that contains the data to return. The response data must contain a mapping of key/value pairs.

```
- stop: "Stop running the rest of the sequence"
 response_variable: "my_response_variable"
```

There is also an `error` option, to indicate we are stopping because of an unexpected error. It stops the sequence as well, but marks the automation or script as failed to run.

```
- stop: "Well, that was unexpected!"
 error: true
```

## Continuing on error

---

By default, a sequence of actions will be halted when one of the actions in that sequence encounters an error. The automation or script will be halted, an error is logged, and the automation or script run is marked as errored.

Sometimes these errors are expected, for example, because you know the action you perform can be problematic at times, and it doesn't matter if it fails. You can set `continue_on_error` for those cases on such an action.

The `continue_on_error` is available on all actions and is set to `false`. You can set it to `true` if you'd like to continue the action sequence, regardless of whether that action encounters an error.

The example below shows the `continue_on_error` set on the first action. If it encounters an error; it will continue to the next action.

```
- alias: "If this one fails..."
 continue_on_error: true
 action: notify.super_unreliable_service_provider
 data:
 message: "I'm going to error out..."

- alias: "This one will still run!"
 action: persistent_notification.create
 data:
 title: "Hi there!"
 message: "I'm fine..."
```

Please note that `continue_on_error` will not suppress/ignore misconfiguration or errors that Home Assistant does not handle.

## Disabling an action

---

Every individual action in a sequence can be disabled, without removing it. To do so, add `enabled: false` to the action. For example:

```
Example script with a disabled action
script:
 example_script:
 sequence:
 # This action will not run, as it is disabled.
 # The message will not be sent.
 - enabled: false
 alias: "Notify that the ceiling light is being turned on"
 action: notify.notify
 data:
 message: "Turning on the ceiling light!"

 # This action will run, as it is not disabled
 - alias: "Turn on the ceiling light"
 action: light.turn_on
 target:
 entity_id: light.ceiling
```

Actions can also be disabled based on limited templates or blueprint inputs.

```
blueprint:
 input:
 input_boolean:
 name: Boolean
 selector:
 boolean:

 actions:
 - delay: 0:35
 enabled: !input input_boolean
```



## Respond to a conversation

---

The `set_conversation_response` script action allows returning a custom response when an automation is triggered by a conversation engine, for example a voice assistant. The conversation response can be templated.

```
Example of a templated conversation response resulting in "Testing 123"
- variables:
 my_var: "123"
- set_conversation_response: "{{ 'Testing ' ~ my_var }}"
```

The response is handed to the conversation engine when the automation finishes. If the `set_conversation_response` is executed multiple times, the most recent response will be handed to the conversation engine. To clear the response, set it to `None` :

```
Example of a clearing a conversation response
set_conversation_response: ~
```

If the automation was not triggered by a conversation engine, the response will not be used by anything.

---

## Docs/Templating/Custom Templates

---

When you find yourself writing the same template in two or three places, it is time to save it once and reuse it. Home Assistant has a feature for that: the `custom_templates` folder. Drop a template file in there, and every automation, script, and template entity in your configuration can call it.

This page is optional. If you are starting out with templates, you can skip it. Come back when you notice yourself copy-pasting the same logic around.

## Setting up the folder

---

Create a folder called `custom_templates` inside your Home Assistant configuration directory (the one that holds `configuration.yaml`). Inside it, create files with the `.jinja` extension. The name before the extension is what you will import by.

```
config/
├── configuration.yaml
├── automations.yaml
├── custom_templates/
│ └── formatter.jinja
```

Each file must be under 5 MiB. Home Assistant loads everything in this folder at startup.

## Reloading without a restart

When you edit a file in `custom_templates`, Home Assistant does not pick up the change automatically. You need to tell it to reload in one of two ways:

- Call the `developer_call_service` action.
- Go to **Developer tools** > **Actions** and run the `homeassistant.reload_custom_templates` action.

Home Assistant does not need a restart for custom template changes.

## Sharing a macro

---

A macro is a reusable piece of template that takes arguments and produces a result. Macros are perfect for recipes you apply over and over, like formatting an entity in a consistent way.

Define the macro in a file under `custom_templates`. For example, `config/custom_templates/formatter.jinja`:

```
{% macro format_entity(entity_id) %}
{{ state_attr(entity_id, 'friendly_name') }}: {{ states(entity_id) }}
{% endmacro %}
```

Then, in any template elsewhere in your configuration, import the macro and use it:

### template

```
template: |
 {% from 'formatter.jinja' import format_entity %}
 {{ format_entity('sensor.outdoor_temperature') }}
 {{ format_entity('sensor.indoor_temperature') }}
output: |
 Outdoor temperature: 22.5
 Indoor temperature: 21.0
```

If you change the format (say, you want to put the state first), you only update `formatter.jinja`, and every automation that imports it picks up the new format on the next reload.

## Importing vs including

---

Home Assistant supports both of the usual ways to pull in a template file:

- `{% include <file> %}` loads a specific macro from a file. Use this when you want to call a macro by name.
- `[Include not found: 'file.jinja']` runs the entire file as if its contents were at the current spot. Use this for drop-in content.

For most reuse cases, `import` is cleaner because it keeps the macro names explicit.

## Macros that return values

---

A regular macro produces text as its result. When you call it, you get back whatever the macro writes. That is fine for formatting strings, but sometimes you want a macro that hands back a number, a list, or a yes-or-no answer.

The trick is to define a macro that takes a special argument called `returns`, then call it through the `as_function` filter. That makes the macro behave like a normal function call.

### template

```
template: |
 {% macro macro_is_switch(entity_name, returns) -%}
 {% do returns(entity_name.startswith('switch.')) %}
 {% endmacro -%}

 {% set is_switch = macro_is_switch | as_function -%}
 {% set result = "It's a switch!" if is_switch('switch.my_switch')
 else "Not a switch!" -%}
 {{ result }}
output: "It's a switch!"
```

Read that out loud: "define a macro that answers whether an entity ID starts with `switch.`, then turn it into a function, then use that function inside an `if`". This pattern lets you build up your own library of small utility functions that return actual values.

## When to use custom templates

---

Reach for the `custom_templates` folder when:

- You have a formatting snippet you repeat across notifications, dashboards, or template entities.
- You have a calculation (like a complicated condition check) that shows up in many automations.
- You want your own library of helper macros that do specific things.

For one-off templates, this is overkill. Write the template inline where you need it.

## Next steps

---

- Review the [template functions reference](#) for existing filters and functions that might already do what you need.
- See `as_function` for details on macros that return values.
- For the conceptual basics of templates, see [Template syntax](#) and [Loops and conditions](#).



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

---

## Docs/Templating/Dates And Times

---

Dates and times come up constantly in Home Assistant templates. "How long ago did the door open?" "Is it past sunset?" "How many days until my next bin collection?" This page gathers the tools and patterns you need, in one place.

If you only need the current moment, start with `now()`. If you're converting or formatting, jump to the [formatting section](#). If you're working with a timestamp from a sensor, head to the [conversion section](#).

## Getting the current time

---

### now() and utcnow()

`now()` returns the current date and time in your configured time zone. `utcnow()` returns it in UTC.

#### template

```
template: |
 Local: {{ now() }}
 UTC: {{ utcnow() }}
output: |
 Local: 2026-04-04 14:30:00.123456+02:00
 UTC: 2026-04-04 12:30:00.123456+00:00
```

**⚠ Important:** Templates that use `now()` or `utcnow()` re-run once per minute. This is how clock-based templates stay current. If you need something to refresh more often, base it on a state change instead.

### today\_at

`today_at` gives you a specific time on the current day, handy for "is it past bedtime?" checks.

#### template

```
template: |
 {{ today_at('22:00') }}
output: "2026-04-04 22:00:00+02:00"
```

## Accessing parts of a datetime

---

Once you have a datetime, you can pull out individual parts with dots.

### template

```
template: |
 Year: {{ now().year }}
 Month: {{ now().month }}
 Day: {{ now().day }}
 Hour: {{ now().hour }}
 Minute: {{ now().minute }}
 Weekday: {{ now().weekday() }}
output: |
 Year: 2026
 Month: 4
 Day: 4
 Hour: 14
 Minute: 30
 Weekday: 5
```

`weekday()` is `0` for Monday through `6` for Sunday. If you prefer the other convention, `isoweekday()` gives you `1` for Monday through `7` for Sunday.

## Formatting with strftime

---

`strftime` turns a datetime into text, shaped the way you want. This is the single most useful method for displaying dates and times.

### template

```
template: |
 24-hour: {{ now().strftime('%H:%M') }}
 12-hour: {{ now().strftime('%I:%M %p') }}
 Weekday: {{ now().strftime('%A') }}
 Short date: {{ now().strftime('%Y-%m-%d') }}
 Long date: {{ now().strftime('%A, %B %-d, %Y') }}
 Sentence: {{ now().strftime('It is %A at %H:%M') }}
output: |
 24-hour: 14:30
 12-hour: 02:30 PM
 Weekday: Saturday
 Short date: 2026-04-04
 Long date: Saturday, April 4, 2026
 Sentence: It is Saturday at 14:30
```

## Common format codes

Here are the ones you will use most often:

- `%Y` : four-digit year (for example, `2026` )
- `%m` : zero-padded month (for example, `04` )
- `%d` : zero-padded day (for example, `04` )
- `%-d` : day without leading zero (for example, `4` )
- `%H` : hour in 24-hour format (for example, `14` )
- `%M` : minute (for example, `30` )
- `%S` : second (for example, `00` )
- `%I` : hour in 12-hour format (for example, `02` )
- `%p` : `AM` or `PM`
- `%A` : full weekday name (for example, `Saturday` )
- `%a` : short weekday name (for example, `Sat` )
- `%B` : full month name (for example, `April` )
- `%b` : short month name (for example, `Apr` )

The Python documentation has the [full list of format codes](#) if you need something unusual.

## Parsing text into a datetime with `strptime`

---

`strptime` is the reverse of `strftime`. It takes a piece of text and a format string, and gives you back a datetime.

### template

```
template: |
 {% set event = strptime('2026-12-25 10:30', '%Y-%m-%d %H:%M') %}
 {{ event }}
output: "2026-12-25 10:30:00"
```

See the `strptime` reference for the full signature.

## Converting between types

---

Sensors, APIs, and MQTT messages deliver timestamps in many different shapes. These functions convert between them.

### UNIX timestamp to datetime

A **UNIX timestamp** is the number of seconds since January 1, 1970. Many APIs use them.

#### template

```
template: |
 {% set ts = 1710510600 %}
 Local: {{ ts | timestamp_local }}
 UTC: {{ ts | timestamp_utc }}
 Custom: {{ ts | timestamp_custom('%H:%M on %B %d') }}
output: |
 Local: 2024-03-15 14:30:00+01:00
 UTC: 2024-03-15 13:30:00+00:00
 Custom: 14:30 on March 15
```

See `timestamp_local`, `timestamp_utc`, and `timestamp_custom`.

### Datetime to UNIX timestamp

Use `as_timestamp`:

#### template

```
template: "{{ as_timestamp(now()) | int }}"
output: "1743768600"
```

### Text to datetime

`as_datetime` is a forgiving version of `strptime` that tries to figure out common formats on its own.

#### template

```
template: |
 {{ as_datetime('2026-04-04 14:30:00') }}
output: "2026-04-04 14:30:00+02:00"
```

## Time differences

---

When you subtract two datetimes, you get a **timedelta** that represents the duration between them. Timedeltas support their own calculations and methods.

### How long ago did something happen?

Every state has a `last_changed` timestamp. Subtract it from `now()` to get a timedelta.

#### template

```
template: |
 {% set since = now() - states.binary_sensor.front_door.last_changed %}
 Total seconds: {{ since.total_seconds() | int }}
 Total minutes: {{ (since.total_seconds() / 60) | int }}
output: |
 Total seconds: 900
 Total minutes: 15
```

For a human-readable version, reach for `time_since` instead:

#### template

```
template: |
 {{ time_since(states.binary_sensor.front_door.last_changed) }} ago
output: "15 minutes ago"
```

### Is it more than X minutes?

A common pattern: trigger an alert when something has been in a state too long.

#### template

```
template: |
 {{
 (now() - states.binary_sensor.front_door.last_changed)
 .total_seconds() > 600
 }}
output: "True (when the door has been in its state for more than 10 minutes)"
```

### Counting down to a future date

`time_until` turns a future datetime into a human-readable duration.

#### template



```
template: |
 {% set birthday = strtotime('2026-12-25', '%Y-%m-%d') %}
 Christmas: {{ time_until(birthday) }}
output: "Christmas: 8 months"
```

## Time zones

---

Home Assistant stores state timestamps ( `last_changed` , `last_updated` ) in your configured time zone. `now()` also returns your local time zone. `utcnow()` returns UTC.

If you need to compare datetimes, both sides need to be in the same time zone. `as_datetime` and `strptime` return datetimes without a time zone by default. Apply the matching conversion before comparing, or stick to `timestamp_local` and `timestamp_utc` which handle this for you.

## Common gotchas

---

- **Text comparison vs time comparison.** Comparing date-looking text with `<` or `>` compares alphabetically, not chronologically. Convert both sides to datetimes first.
- **`timestamp_custom` is not the only option.** For datetime values (like `now()`), use `.strftime(...)` directly. `timestamp_custom` is specifically for UNIX timestamps.
- **Templates using `now()` re-run every minute.** Don't use it for things that should update more often.
- **Subtraction gives you a `timedelta`, not seconds.** Call `.total_seconds()` on the result when you need a number.
- **`.replace(hour=..., minute=...)` can be wrong around daylight saving time.** Replacing the hour field on a timezone-aware datetime can land in a skipped or repeated local hour, which silently produces the wrong absolute time. Prefer `today_at('HH:MM')` for "today at a specific local time", and add or subtract full days with `timedelta(days=1)` only when you accept 24 exact hours, not one calendar day.

## Next steps

---

- See the [full list of date and time functions](#) in the reference.
  - For common date/time recipes, see [Common template patterns](#).
  - The [Python methods](#) page covers `.strftime()`, `.weekday()`, and similar datetime methods.
-

## Docs/Templating/Debugging

---

Every template author has stared at a template that refuses to work and wondered if they're losing it. You are not. Templates are fiddly, and even experienced people make the same mistakes over and over. This page walks through the tools and habits that turn "why isn't this working?" into "ah, that's the problem".

## The template editor

---

Home Assistant has a built-in template editor that shows the result of a template while you type. Open it from **Settings > Developer tools > Template**.

The editor is the fastest feedback loop you have. It:

- Shows the output of your template live.
- Points at errors with a red message.
- Has access to all your entities, which means you can try the template against real data.

When a template misbehaves in an automation or template entity, copy it into the editor, adjust until it produces what you want, then paste it back.

## Read the error message

---

When a template has a mistake, Home Assistant shows an error. The message is often more helpful than it looks.

**UndefinedError: 'foo' is undefined** Something in your template refers to a variable that does not exist. Check the spelling and make sure anything you `` is defined before you use it.

**TypeError: unsupported operand type(s)** You are trying to do something to a value of the wrong type, like adding a number to some text. Convert with `float` or `int` first, and remember to add a fallback.

**TemplateSyntaxError** A typo in the template itself. Count your brackets, check that every `has` `an`, and that every ``.

**No first item, sequence was empty** You used `first` or `last` on an empty list. Add an `if` check, or use the `default` filter.

## Narrow down the problem

---

When a long template does not work, the trick is to take it apart. Start with the smallest piece you can and add one thing at a time. The moment the result goes wrong, you know exactly which piece broke. This sounds slow but it is always faster than staring at the whole thing and hoping the problem jumps out.

### template

```
template: |
 {# Start here #}
 {{ states('sensor.outdoor_temperature') }}
output: "22.5"
```

### template

```
template: |
 {# Add the conversion #}
 {{ states('sensor.outdoor_temperature') | float(0) }}
output: "22.5"
```

### template

```
template: |
 {# Add the rounding #}
 {{ states('sensor.outdoor_temperature') | float(0) | round(1) }}
output: "22.5"
```

### template

```
template: |
 {# Add the full sentence #}
 It is {{ states('sensor.outdoor_temperature')
 | float(0) | round(1) }}°C outside.
output: "It is 22.5°C outside."
```



## Check what a value actually is

---

When a template gives the wrong result, show the raw value to see what you are working with.

### template

```
template: |
 {{ states('sensor.outdoor_temperature') }}
output: "22.5"
```

That result looks like a number, but it is actually text. Many sensors return text even when the content looks numeric. Use `typeof` to be sure:

### template

```
template: |
 {{ states('sensor.outdoor_temperature') | typeof }}
output: "str"
```

`str` is short for "string", which is how templates refer to text. If you see `str` where you expected a number, that's your clue to convert with `float` or `int`. `int` and `float` in the output mean the value is already a whole number or decimal number.

## Inspect attributes

---

When you are not sure what attributes an entity has, show them all with `tojson`:

### template

```
template: |
 {{ state_attr('media_player.living_room', 'source_list') | tojson }}
output: '["Spotify", "Bluetooth", "TV", "Radio"]'
```

Or list every attribute:

### template

```
template: |
 {% for key, value in states.media_player.living_room.attributes.items() %}
 {{ key }}: {{ value }}
 {% endfor %}
output: |
 friendly_name: Living room
 volume_level: 0.35
 source: Spotify
 source_list: ['Spotify', 'Bluetooth', 'TV', 'Radio']
```

## When does my template run?

---

A common source of confusion is "why isn't my template updating?", or the opposite: "why is it running so often?". Home Assistant decides when to re-evaluate a template based on what is *inside* it.

**Entity states.** When your template reads a state, like `states('sensor.temperature')`, Home Assistant watches that entity. The template runs again every time the entity's state changes.

`now()` and `utcnow()`. Templates that use `now()` or `utcnow()` re-run once per minute.

**Other variables.** Templates that depend on `this`, `trigger`, or other context variables only re-run when the context changes (a new trigger fires, the entity updates).

If you write a template that does not read any state or use `now()`, it runs *once* at startup and never again. That is fine for constant values, but it's a common trap when you want a template to react to something.

## Why does it work in Developer Tools but not in my automation?

The **Template editor** runs your template once, right when you open it, and shows the result. There is no ongoing re-evaluation. That makes it great for testing, but a working template in the editor does not guarantee it works in an automation.

Two things commonly differ:

- **`this` and `trigger` are not available** in the editor. If your template refers to them, the editor shows an error. In a template entity or automation, they exist.
- **Automations re-evaluate on change.** A template trigger only fires when the template's result goes from false to true. If you load an automation while the condition is already true, nothing fires.

When a template works in the editor but not in an automation, check whether you are using `this`, `trigger`, or expecting a specific "on-change" behavior.

## Common mistakes

---

- **Sensor gives text, not a number.** Use `| float(0)` or `| int(0)`.
- **Entity is `unknown` or `unavailable`.** Add a fallback with `| default(...)` or an `if has_value(...)` check.
- **Variable changes inside a loop are lost.** Use `namespace`.
- **YAML refuses to load your template.** Check quoting. See [Templates in YAML](#).
- **Template appears as literal text in output.** The field does not support templating, or the template is inside a `` / `` block.
- **Template evaluates at the wrong time.** Triggers and conditions are checked on change, not continuously. A template condition that depends on `now()` only re-checks when something else triggers.
- **Comparing text to a number.** `'6' < '10'` is `False` because text is compared alphabetically, not numerically. Convert with `| float(0)` or `| int(0)` on both sides first.

## Common Python mistakes

---

Templates use [Jinja2](#), a templating engine built on top of Python, but the syntax is not the same as Python. A few things that work in Python don't work in templates:

- **No `print()`**. Write ``` to show a value. There is no `print` statement.
- **No `import`**. You cannot import modules. The functions available to you are the ones in the [template functions reference](#) plus the standard Jinja filters.
- **No `=` inside `{{ }}` for assignment**. Use ``` to create a variable. Inside `}`, `=`` does not exist.
- **`None` must be capitalized**. Python writes `None`, `True`, `False` with a capital first letter. Template tests and literals use lowercase `none`, `true`, `false`. Both work in most places, but be consistent.
- **No f-strings**. Python's `f"hello {name}"` is not template syntax. Use `"hello " ~ name` (the `~` joins text) or ``` directly.
- **No `while` loops**. Templates only support `for` loops.
- **List and dict methods are limited**. Read-only methods like `.items()`, `.values()`, `.keys()`, `.get()`, `.split()`, `.lower()`, and `.upper()` work fine. Methods that mutate a value in place, like `.append()`, `.pop()`, or `.update()`, are blocked as unsafe. To build up a list across loop iterations, use a `namespace` instead.

## Next steps

---

- Looking up a specific error message? See [Error messages and fixes](#).
- Need a working example to compare against? Browse [Common template patterns](#).
- Want to refresh the basics? Review [Template syntax](#).

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit](#). When you ask, include these four things and the answer usually comes within minutes:

- The template you are using (copied from the editor, where you can see what it runs against).
- What you expected the result to be.
- What the actual result or error was.
- The entity IDs involved (from **Settings > Developer tools > States**).

 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

---

## Docs/Templating/Errors

---

This page lists template error messages you might run into, what each one means in plain language, and how to fix it. If the message you are seeing does not match exactly, look for the closest one. The names of types and variables change, but the shape of each error stays the same.

For a general debugging workflow, see [Debugging templates](#).



## UndefinedError: 'foo' is undefined

---

**What it means.** The template is trying to use a variable named `foo` that does not exist. Either the name is misspelled, or the variable was never set.

### How to fix it.

- Check the spelling of every name in the template. `states` and `state` are different; `trigger.to_state` and `trigger.tostate` are different.
- If you use a variable with `` , make sure the set`` runs before the variable is used.
- If you expect the variable to come from an automation trigger ( `trigger.*` ) or a template entity ( `this.*` ), remember these only exist in those contexts. They are not available in the Template editor.

## UndefinedError: 'dict object' has no attribute 'foo'

---

**What it means.** You tried to read a key named `foo` from a dictionary, but that key does not exist.

**How to fix it.**

- Use `.get("foo", default_value)` to return a default when the key is missing:  
`data.get("foo", 0)` .
- Or check for the key first with `if "foo" in data` .
- If the dictionary comes from a JSON response, print it with `` to see exactly which keys are there.

## TypeError: unsupported operand type(s) for +: 'str' and 'int'

---

**What it means.** You are trying to do math between a piece of text and a number. This happens most often with entity states, because every state is stored as text. `"22.5" + 5` does not work.

**How to fix it.** Convert the text to a number first with `| float(0)` or `| int(0)` :

**template**

```
template: |
 {{ states('sensor.temperature') | float(0) + 5 }}
output: "27.5"
```

The `0` is a fallback used when the conversion fails (for example, when the sensor is `unavailable`). See [Types and conversion](#).

## TypeError: float() argument must be a string or a real number, not 'NoneType'

---

**What it means.** You tried to convert `None` to a number, and `None` is not a number. This usually comes from reading an attribute that does not exist, or calling `state_attr` for an entity that has not been set up yet.

**How to fix it.** Add a default value to `float` or `int` :

### template

```
template: |
 {{ state_attr('light.kitchen', 'brightness') | float(0) }}
output: "0"
```

Or skip the calculation when the value is missing using `has_value` .

## TypeError: object of type 'generator' has no len()

---

**What it means.** You tried to count or measure an iterable directly. Filters like `map`, `select`, `reject`, `selectattr`, and `rejectattr` return iterables, not lists. Iterables cannot be counted until you materialize them.

**How to fix it.** Add `| list` to turn the iterable into a list first:

### template

```
template: |
 {{ states.light | selectattr('state', 'eq', 'on') | list | count }}
output: "3"
```

See [Types and conversion](#) for more on iterables.

## TemplateSyntaxError: expected token 'end of statement block', got 'X'

---

**What it means.** There is a typo or an unexpected character inside a `{% ... %}` block. Something is written that the template engine doesn't recognize.

**How to fix it.**

- Look at the template line number in the error.
- Check for missing commas, brackets, or quotes.
- Verify that operators are spelled right ( `==` , `!=` , `and` , `or` , `not` , `in` ).
- Python comparisons like `is not None` work; make sure you used `is not` , not `not is` .

## TemplateSyntaxError: Unexpected end of template

---

**What it means.** A `,`, `{% endset %}`, or `{% endmacro %}`.

**How to fix it.**

- Count the opening and closing tags. If you have two `,` you need two.
- Indent the template in the editor so you can see the structure.

## TemplateSyntaxError: tag name expected

---

**What it means.** You have a `{% %}` block with nothing inside, or with something the engine cannot parse as a statement.

**How to fix it.** Check the line. Remove empty `{% %}` markers. If you intended a comment, use `{# ... #}` instead.



## TemplateAssertionError: no test named 'foo'

---

**What it means.** After `is`, you used a test name the engine does not know.

**How to fix it.** Check the test name against the [template functions reference](#). Common tests are `defined`, `none`, `number`, `string`, `boolean`, `iterable`, `mapping`, `even`, `odd`, `eq`, `gt`, `lt`, and `in`. Aliases like `equalto`, `greaterthan`, and `lessthan` also work.

## UndefinedError: 'states' has no attribute 'sensor'

---

**What it means.** When using dot notation like `states.sensor.temperature.state`, one of the pieces in the chain does not exist. Usually this means the entity ID is wrong, or the entity has not been set up yet.

**How to fix it.**

- Use `states('sensor.temperature')` instead. The function version returns the text `'unknown'` for missing entities instead of raising an error, which is safer.
- Verify the entity ID in **Settings > Developer tools > States**.

## No first item, sequence was empty

---

**What it means.** You used `first` or `last` on a list that turned out to be empty. There is nothing to return.

**How to fix it.** Check the list length first, or use `default` :

**template**

```
template: |
 {% set items = ['a', 'b', 'c'] %}
 {{ items | first | default('nothing') }}
output: "a"
```

## YAML error: could not find expected ':'

---

**What it means.** Your YAML file contains a template with unquoted braces ( `{{` or `{%}` ). YAML tries to parse `{` as the start of a flow-style mapping and fails.

**How to fix it.** Wrap the single-line template in quotes, or use a multi-line block scalar:

```
Correct: quoted
value_template: "{{ states('sensor.temperature') }}"

Correct: multi-line
value_template: >
 {{ states('sensor.temperature') }}
```

See [Templates in YAML](#) for the full set of quoting rules.

## Next steps

---

- For a systematic approach to narrowing down any template problem, see [Debugging templates](#).
- If the error came from a quoting or indentation issue, head to [Templates in YAML](#).
- If the error involves state values being text, see [Working with states](#).

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and the exact error message, or share on [our subreddit](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language. Paste in your template and the error message.

---

## Docs/Templating/Introduction

---

When you want a notification that reads "It is 22°C in the living room" instead of a fixed message, or an automation that fires only when more than three doors are open, you write a template.

A template is a short snippet of code that Home Assistant runs every time it needs a value. Instead of writing a fixed piece of text or a fixed number, you write instructions for how the value should be computed from your home's current data.

Home Assistant's templating is powered by [Jinja2](#), a widely used template engine in the Python world. That means you can search the web for Jinja2 examples and most of what you find will work in Home Assistant too. We add many of our own functions on top for reading states, finding entities, working with areas, and similar tasks specific to smart homes.

Because templates are code, this is one of the more technical corners of Home Assistant. It is fair to think of it as a light form of programming. You will write small calculations, learn how to handle text and numbers together, and sometimes run into puzzling error messages. Don't worry, the [Debugging templates](#) page has your back when that happens.

## You probably don't need templates

---

Home Assistant is designed to be used through its interface. You can set up devices, build automations, create dashboards, and manage your whole smart home without ever looking at a configuration file or writing a single line of code. The automation editor is powerful enough to handle nearly every real-world scenario, and it is getting better with every release.

So before you invest any time on this page: templates are *not* required to use Home Assistant. If the visual editors do what you need, you are done. Skip this section with a clear conscience.

Templates are for when you want to go further than the interface alone allows. You might reach for them when:

- You want a notification to say something dynamic, like "The living room is 22°C and the basement is 14°C", using live values from your sensors.
- You need an automation condition that depends on a calculation across several entities, such as "only run if more than three doors are open".
- You are creating a [template entity](#), a sensor whose value is computed from other entities.
- You are processing raw data from a REST API, MQTT topic, or command-line output, and need to reshape it into something Home Assistant can use.

If any of those sound like problems you actually have, keep reading. If not, bookmark this page and come back when you need it.



## An example

---

Imagine you want your phone to tell you the temperature when you come home. A plain notification can only say one thing:

```
It is warm outside.
```

With a template, you can include the actual temperature from your outdoor sensor:

### template

```
template: |
 It is {{ states('sensor.outdoor_temperature') }}°C outside.
```

Home Assistant replaces the bit between `{{` and `}}` with the current value every time the notification is sent. You might get:

### output

```
output: "It is 22.5°C outside."
```

Later that day, when it is cooler, the same template produces:

### output

```
output: "It is 14.8°C outside."
```

## What you can do with templates

---

Once you are comfortable with the basics, templates can help you:

- **Read your home's data.** Get the state of any entity, its attributes, or information about devices, areas, and floors.
- **Make decisions.** Show one message when someone is home and a different one when nobody is. Trigger an automation only when several conditions line up.
- **Do calculations.** Average values from multiple sensors, convert units, count how many lights are on, or format a number the way you want to see it.
- **Shape text.** Turn raw values into friendly sentences, lists, or table-like summaries for notifications and dashboards.
- **Process incoming data.** Parse JSON from a web API, clean up messy sensor readings, or extract only the bits you need.

## How templates are written

---

Every template mixes normal text with small pieces of code inside special markers:

- `{{ ... }}` calculates something and inserts the result in the output.
- `{% ... %}` runs logic like `if` and `for` without adding to the output.
- `{# ... #}` is a note for yourself that does not appear anywhere.

The [Template syntax](#) page explains these markers in detail.

## Try it yourself

---

Home Assistant has a built-in **Template editor** that shows the result of a template while you type. It is the fastest way to experiment.

Open it from **Settings > Developer tools > Template**. Try pasting this in:

**template**

```
template: |
 It is {{ now().strftime("%A") }}, and the time is {{
 now().strftime("%H:%M") }}.
output: "It is Saturday, and the time is 14:32."
```

You should see a sentence with the current day of the week and time. Change what is inside the quotes to experiment with different formats, and watch the result update as you type.

## Next steps

---

Pick up from whichever topic is most useful to you:

- Curious where templates show up? Read [Where to use templates](#).
  - Want to understand the building blocks? Start with [Template syntax](#).
  - Looking for ready-made examples? Head to [Common template patterns](#).
  - Stuck on something? The [Debugging templates](#) page can help.
-

## Docs/Templating/Loops And Conditions

---

So far, the templates we've looked at have been straightforward: read a value, show it, maybe do a small calculation. But the moment your template needs to decide something ("is it cold outside?") or go through a list of things ("show me every light that is on"), you need a couple more tools.

That is what this page is about. We will cover two ideas that work together to make templates much more useful:

- A **condition** is a question with a yes-or-no answer that decides what happens next. "Is the front door open?" "Is the temperature below zero?"
- A **loop** is a way to do the same thing once for each item in a list. "For every light in the house, show its name and state."

Both live inside `{% ... %}` markers, because they run logic without adding anything to the output by themselves. And both will feel familiar if you've ever written a shopping list ("if I'm out of milk, add it to the list") or followed a recipe ("for each onion, chop it finely").

## Conditions with if

The `if` statement runs the text inside it only when its condition is true. When the condition is false, that text is skipped.

### template

```
template: |
 {% if is_state('sun.sun', 'above_horizon') %}
 The sun is up.
 {% else %}
 The sun is down.
 {% endif %}
output: "The sun is up."
```

Read that out loud: "If the sun is above the horizon, show 'The sun is up'. Otherwise, show 'The sun is down'." That is exactly what the template does.

Every `if` must be closed with `{% endif %}`. The ```` part is optional but often useful.

## Multiple paths with elif

When you have more than two outcomes, use `{% elif %}` ("else if") to chain conditions together. Home Assistant checks them in order and runs the first one that is true.

### template

```
template: |
 {% set temp = states('sensor.outdoor_temperature') | float(0) %}
 {% if temp < 0 %}
 Freezing.
 {% elif temp < 15 %}
 Cold.
 {% elif temp < 25 %}
 Comfortable.
 {% else %}
 Warm.
 {% endif %}
output: "Comfortable."
```

You can add as many `elif` branches as you need.

## Inline if for one-liners

If you only need to pick between two values, you can write the `if` on one line. This is often handier inside `{{ ... }}` than writing a whole `if/else` block.

### template

```
template: |
 {% set temp = 22 %}
 It is {{ 'warm' if temp > 20 else 'cool' }}.
output: "It is warm."
```

Read it left to right: "warm, if the temperature is over 20, otherwise cool". Natural English.



## Loops with for

A `for` loop repeats whatever is inside it once for each item in a list. The word right after `for` is a name you pick; each time the loop runs, it holds the current item.

### template

```
template: |
 {% for light in states.light %}
 {{ light.name }}: {{ light.state }}
 {% endfor %}
output: |
 Kitchen: on
 Living Room: off
 Bedroom: off
```

In plain English: "for each light in the list of lights, show its name and state". The name `light` is only a label; you could call it `x` or `item` or `thingy` and it would still work.

Every `for` must be closed with `{% endfor %}`.

## Filtering with if inside a for

You can tell a `for` loop to skip items that don't match a condition by adding `if` at the end:

### template

```
template: |
 Lights that are on:
 {% for light in states.light if light.state == 'on' %}
 - {{ light.name }}
 {% endfor %}
output: |
 Lights that are on:
 - Kitchen
 - Hallway
```

That reads as: "for each light in the list of lights, where its state is 'on', show a bullet with the name". Items that don't match are skipped.

## Knowing where you are: the loop variable

Inside a loop, templates give you a special variable called `loop` that tells you where you are. These are the fields you'll use most:

- `loop.first`: `True` on the first time through, `False` otherwise.
- `loop.last`: `True` on the last time through.
- `loop.index`: the current position, starting at 1.
- `loop.index0`: the current position, starting at 0 (handy if you're used to programming).
- `loop.length`: the total number of items.

That last-item check is useful when you want to format a list neatly:

#### template

```
template: |
 {% for person in states.person %}
 {{ loop.index }}. {{ person.name }}{% if not loop.last %},{% endif %}
 {% endfor %}
output: |
 1. Frenck,
 2. Paulus,
 3. Zack
```

The comma is added for every item except the last one.

## Variables with set

Templates can get long, and typing the same thing twice makes them harder to read. `set` lets you give a value a name so you can reuse it.

### template

```
template: |
 {% set temp = states('sensor.outdoor_temperature') | float(0) %}
 {% set unit = state_attr('sensor.outdoor_temperature',
 'unit_of_measurement') %}
 It is {{ temp | round(1) }} {{ unit }} outside.
output: "It is 22.5 °C outside."
```

Now `temp` and `unit` can be used anywhere in the template. If the name of the sensor changes, you only need to update it once.

## A common gotcha: variables and loops

Here is something that catches everyone at least once. A variable changed inside a loop does not stick around after the loop ends.

### template

```
template: |
 {% set count = 0 %}
 {% for light in states.light if light.state == 'on' %}
 {% set count = count + 1 %}
 {% endfor %}
 {{ count }}
output: "0"
```

You'd think `count` would end up at 3 (or however many lights are on), but it doesn't. The `set count = count + 1` inside the loop creates a brand new `count` each time, one that only exists inside that loop iteration. The outer `count` never changes.

The fix is to use a `namespace`. Think of a namespace as a small box that holds values. Changes to what's inside the box persist, because you're updating the box, not replacing it.

### template

```
template: |
 {% set ns = namespace(count=0) %}
 {% for light in states.light if light.state == 'on' %}
 {% set ns.count = ns.count + 1 %}
 {% endfor %}
 {{ ns.count }}
output: "3"
```

This looks weird the first time, but it becomes natural quickly. Whenever you need to count something or build up a result inside a loop, reach for `namespace`.

## Breaking out of loops early

---

Sometimes you want a loop to stop before reaching the end of the list. Home Assistant has two extra statements for that:

- `{% break %}` stops the loop right away.
- `{% continue %}` skips to the next item without finishing the current one.

### template

```
template: |
 {# Show the first three lights that are on #}
 {% set ns = namespace(shown=0) %}
 {% for light in states.light %}
 {% if light.state != 'on' %}
 {% continue %}
 {% endif %}
 {% if ns.shown >= 3 %}
 {% break %}
 {% endif %}
 {{ light.name }}
 {% set ns.shown = ns.shown + 1 %}
 {% endfor %}
output: |
 Kitchen
 Hallway
 Desk
```

This one skips lights that are off (with `continue`), and stops entirely once three have been shown (with `break`).

## The do statement

---

You may see `mentioned in Jinja documentation elsewhere`. It runs an expression without printing anything, which in plain Jinja is useful for things like `to` mutate a list.

Home Assistant's template environment is sandboxed. Mutation methods like `.append()`, `.pop()`, and `.update()` are blocked for safety, so `is rarely needed in Home Assistant templates`. To build up a list or counter across loop iterations, use a `[`namespace`] (/template-functions/namespace/)` with `instead`.

## Next steps

---

- For the full list of filters and tests you can use inside `if` conditions, see the [template functions reference](#).
  - Common counting and aggregation patterns live on the [Common template patterns](#) page.
  - When using these statements inside YAML, keep the [Templates in YAML](#) page handy for quoting rules.
-

## Docs/Templating/Patterns

---

Templates are most useful when you can reach for an example that already does what you need. This page collects recipes for the situations that come up again and again: counting things, summarizing data, building nice-sounding sentences, and dealing with values that might be missing.

Each recipe explains what it does, shows you a working template, and links to the reference pages for the functions it uses. Copy the template you need, swap in your own entity IDs, and adjust the wording.

## Counting entities

---

Counting is one of the most common things you do with templates. "How many lights are on?" "How many windows are open?" These templates follow the same shape every time: list the entities, filter them, count the result.

 **Tip:** If all you need is a count on a dashboard, a [Group helper](#) can gather the entities and its state shows how many are active. No template needed.

### Count how many lights are on:

#### template

```
template: |
 {{ states.light | selectattr('state', 'eq', 'on') | list | count }}
output: "3"
```

Read the template left to right: take every light, keep only the ones whose state is `on`, turn that into a list, count the items in the list.

### Count how many doors are open:

#### template

```
template: |
 {{
 states.binary_sensor
 | selectattr('attributes.device_class', 'eq', 'door')
 | selectattr('state', 'eq', 'on')
 | list | count
 }}
output: "2"
```

Here the filter is stricter. First keep only binary sensors whose device class is `door`, then only the ones that are `on` (binary sensors that represent doors report `on` when the door is open).

See `selectattr`, `list`, and `count`.



## Finding the highest or lowest value

When you have several sensors of the same kind, you often want to pick out the extreme. The warmest room. The coldest fridge. The lowest battery. The heaviest power draw.

💡 **Tip:** A [Group helper](#) can pick the minimum, maximum, or mean from a group of sensors and expose it as a regular sensor. No template needed if you only need the value.

### The lowest battery:

#### template

```
template: |
 {{
 states.sensor
 | selectattr('attributes.device_class', 'eq', 'battery')
 | selectattr('entity_id', 'has_value')
 | map(attribute='state')
 | map('float')
 | min
 | round(0)
 }}%
output: "18%"
```

We gather all battery sensors, use `has_value` to remove any that are `unknown` or `unavailable`, convert their states to numbers, then `min` picks the smallest. The `| float` conversion is important because sensor states are text, and comparing text alphabetically gives wrong results ( `"9"` would beat `"23"` because `"9"` comes after `"2"` ).

### The warmest room (with the room name):

The previous example gives you the value but not which sensor it came from. When you need both, loop through the sensors and track the winner in a `namespace` :

#### template

```
template: |
 {% set temps = states.sensor
 | selectattr('attributes.device_class', 'eq', 'temperature')
 | selectattr('entity_id', 'has_value')
 | list %}
 {% set ns = namespace(warmest=temps[0]) %}
 {% for sensor in temps %}
 {% if sensor.state | float > ns.warmest.state | float %}
 {% set ns.warmest = sensor %}
 {% endif %}
 {% endfor %}
 {{ ns.warmest.name }}: {{ ns.warmest.state }}°C
output: "Kitchen: 23.5°C"
```

Same filtering as before, but instead of extracting the values and losing track of which entity they belong to, we keep the full entity reference. The loop compares each sensor's state as a number and remembers the winner. At the end, we can show both the name and the value.

See also `map` and `namespace` .

## Safe numbers from a sensor

---

This one deserves its own section because it comes up constantly. Sensors return their state as text, even when the content looks like a number. If you try to do math on that text directly, you get errors. The fix is always the same: convert with a fallback.

### template

```
template: |
 {% set temp = states('sensor.outdoor_temperature') | float(0) %}
 {{ temp | round(1) }}°C
output: "22.5°C"
```

Use `float(0)` for decimals and `int(0)` for whole numbers. The `0` inside the parentheses is the fallback value: if the sensor is offline or reports something that cannot be converted to a number, the template uses `0` instead of crashing.

Pick a fallback value that makes sense for your situation. For a temperature, `0` might be misleading. For a counter, `0` is fine. For a power draw, `0` is accurate. Think about what should happen when things go wrong.

## Building a sentence from a list of names

---

When you're sending a notification, writing "Kitchen, Hallway, Desk" looks robotic. "Kitchen, Hallway, and Desk are on." reads much better. Here is a template that does that formatting with a proper Oxford comma:

**All lights that are on, in one sentence:**

**template**

```
template: |
 {% set lights = states.light
 | selectattr('state', 'eq', 'on')
 | map(attribute='name') | list %}
 {% if lights | count == 0 %}
 No lights are on.
 {% elif lights | count == 1 %}
 {{ lights[0] }} is on.
 {% elif lights | count == 2 %}
 {{ lights[0] }} and {{ lights[1] }} are on.
 {% else %}
 {{ lights[:-1] | join(', ') }}, and {{ lights[-1] }} are on.
 {% endif %}
output: "Kitchen, Hallway, and Desk are on."
```

The template handles three cases: nothing, exactly one thing, or more than one. The tricky part is `lights[:-1]` which means "all items except the last", and `lights[-1]` which means "the last item". That lets us put "and" before the final name.

See `join` and `map`.

## Picking the right word: pluralization

---

"1 door is open" and "3 doors are open" both need to sound right. Templates handle this with a small `inline if`.

### template

```
template: |
 {% set count = states.binary_sensor
 | selectattr('attributes.device_class', 'eq', 'door')
 | selectattr('state', 'eq', 'on') | list | count %}
 {{ count }} door{{ 's' if count != 1 }}
 {{- ' is' if count == 1 else ' are' }} open.
output: "2 doors are open."
```

Two small tricks there: `{{ 's' if count != 1 }}` adds an `s` when the count is not 1 (so "doors" instead of "door"), and `{{ 'is' if count == 1 else 'are' }}` picks the right verb.

## Time differences

---

Home Assistant records when each state last changed, and you can reach that timestamp from any state in a template. That opens up "how long ago" questions.

### How long ago did the front door change?

#### template

```
template: |
 The front door changed {{
 time_since(states.binary_sensor.front_door.last_changed)
 }} ago.
output: "The front door changed 15 minutes ago."
```

`time_since` produces nice human-readable text like "15 minutes" or "2 hours".

### Has it been more than 10 minutes?

#### template

```
template: |
 {{
 (now() - states.binary_sensor.front_door.last_changed)
 .total_seconds() > 600
 }}
output: "True"
```

`now()` gives you the current moment. Subtracting two moments gives you the duration between them. `.total_seconds()` turns that duration into a number of seconds, and `600` is ten minutes. This pattern is handy for conditions like "only notify me if the door has been open for more than 10 minutes".

See `now` and `time_since`.

## Formatting timestamps

---

Humans read dates and times in all sorts of ways. Templates use `strftime` (the standard date-formatting tool) to turn a timestamp into whichever shape you want.

### template

```
template: |
 {{ now().strftime('%A %B %-d at %H:%M') }}
output: "Saturday April 4 at 14:30"
```

The mysterious characters ( `%A` , `%B` , `%H` , `%M` ) are format codes. Each one stands for a piece of the date or time. You can mix them freely with plain text.

Common codes:

- `%H:%M` for 24-hour time ( `14:30` ).
- `%I:%M %p` for 12-hour time with AM/PM ( `02:30 PM` ).
- `%A` for the full weekday name ( `Saturday` ).
- `%B` for the full month name ( `April` ).
- `%Y-%m-%d` for ISO-style dates ( `2026-04-04` ).

The Python documentation has the [full list of format codes](#) if you need something unusual.

## Summing values across sensors

---

When you have several sensors measuring the same thing, summing them gives you a household total. Total power draw, total water usage, total number of people home.

💡 **Tip:** A [Group helper](#) with a `sum` type adds up its members and exposes the result as a sensor. For energy specifically, the [Energy dashboard](#) handles totals automatically.

**Total power draw across all power sensors:**

**template**

```
template: |
 {{ states.sensor
 | selectattr('attributes.device_class', 'eq', 'power')
 | selectattr('entity_id', 'has_value')
 | map(attribute='state') | map('float') | sum | round(1) }} W
output: "427.3 W"
```

The steps: keep only power sensors, skip offline ones, pull out their states, convert to numbers, add them up, round the total. See `sum`.



## Parsing JSON

---

REST sensors, MQTT topics, and command-line sensors often return their data as JSON.

`from_json` turns JSON text into something you can read with dots and brackets.

### template

```
template: |
 {% set data = '{"temp": 22.5, "humidity": 60}' | from_json %}
 Temperature: {{ data.temp }}°C
 Humidity: {{ data.humidity }}%
output: |
 Temperature: 22.5°C
 Humidity: 60%
```

In real use, you'd pass a variable (like `value` from an MQTT payload) instead of a hardcoded piece of text.

## Handling missing values

---

Home Assistant entities can go missing, go offline, or report `unknown`. Templates that don't handle these cases produce noisy output or outright errors. Add a safety net.

With `default`:

**template**

```
template: |
 {{ states('sensor.might_not_exist') | default('not available') }}
output: "not available"
```

The `default` filter replaces a missing value with one you choose.

With `has_value`:

**template**

```
template: |
 {% if has_value('sensor.outdoor_temperature') %}
 {{ states('sensor.outdoor_temperature') }}°C
 {% else %}
 Temperature is not available.
 {% endif %}
output: "22.5°C"
```

Use `has_value` when you want a completely different branch for missing data, not only a replacement word.

## Finding entities by device class

The **device class** tells Home Assistant what kind of thing a sensor measures: temperature, humidity, battery, power, door, motion, and many others. Filtering by device class is the most reliable way to pick out "all the X sensors" without listing them by name.

 **Tip:** On dashboards, a grid of [Tile cards](#) shows individual entities cleanly. Templates are the right tool when you need the result as text inside an automation or notification.

### All temperature sensors in the house:

#### template

```
template: |
 {% for sensor in states.sensor
 | selectattr('attributes.device_class', 'eq', 'temperature')
 | selectattr('entity_id', 'has_value') %}
 {{ sensor.name }}: {{ sensor.state }}°C
 {% endfor %}
output: |
 Living room: 22.5°C
 Kitchen: 23.5°C
 Bedroom: 20.1°C
```

### All open doors and windows:

#### template

```
template: |
 {{
 states.binary_sensor
 | selectattr('attributes.device_class', 'in', ['door', 'window'])
 | selectattr('state', 'eq', 'on')
 | map(attribute='name') | join(', ')
 }}
output: "Front door, Kitchen window"
```

The pattern is always the same: take a list of entities, keep only those with the device class you care about, drop the offline ones, then do whatever you need with the remaining list. Swap `temperature` for `humidity`, `battery`, `power`, or any other device class to cover a different kind of sensor.

### All temperature sensors on a specific floor:

#### template

```

template: |
 {% set floor_sensors = floor_entities('first_floor') %}
 {% for entity in floor_sensors %}
 {% if entity.startswith('sensor.')
 and is_state_attr(entity, 'device_class', 'temperature') %}
 {{ state_attr(entity, 'friendly_name') }}: {{ states(entity) }}°C
 {% endif %}
 {% endfor %}
output: |
 Living room: 22.5°C
 Kitchen: 23.5°C

```

This combines `floor_entities` with a device-class check. Use `area_entities` or `label_entities` in the same shape when you want to filter by area or label instead.

## Aggregating entities by label

---

Labels let you group entities in ways that don't match your area/floor structure. A classic use is collecting everything you want to control together, like all TRVs or all radiators in a "Heating Zone".

**Any radiator calling for heat (boiler demand signal):**

**template**

```
template: |
 {{
 label_entities('Heating Zone')
 | select('is_state_attr', 'hvac_action', 'heating')
 | list | count > 0
 }}
output: "True"
```

This template returns `True` whenever at least one climate entity with the `Heating Zone` label has its `hvac_action` attribute set to `heating`. Drop this into a template binary sensor with `device_class: heat` and you have a boiler demand signal. Add it as a template trigger and you can turn the boiler on exactly when heat is needed.

## Which entity changed most recently

---

When you have a group of similar sensors (doors, motion, windows), you sometimes want to know which one was active last. Every state tracks a `last_changed` timestamp, so you can sort by it.

### template

```
template: |
 {% set doors = states.binary_sensor
 | selectattr('attributes.device_class', 'eq', 'door')
 | list %}
 {% set latest = doors | max(attribute='last_changed') %}
 {{ latest.name }} changed {{ time_since(latest.last_changed) }} ago.
output: "Front door changed 3 minutes ago."
```

## Hold-last-value (ignore intermittent dropouts)

---

A sensor that goes `unknown` or `unavailable` for a moment can wreck your automations. A trigger-based template sensor lets you freeze the value during dropouts so downstream templates see a stable number.

### automation

```
automation: |
 template:
 - triggers:
 - trigger: state
 entity_id: sensor.outdoor_temperature
 not_to:
 - unknown
 - unavailable
 sensor:
 - name: "Outdoor temperature (held)"
 unique_id: outdoor_temperature_held
 state: "{{ states('sensor.outdoor_temperature') }}"
 device_class: temperature
 state_class: measurement
 unit_of_measurement: "°C"
 output: "22.5 (even when the original sensor briefly goes unavailable)"
```

The trigger uses `not_to` so the sensor only updates when the source reports a real value. Between updates, the held sensor keeps whatever it had last. Pair this with a long-term alarm on the original sensor if you want to be notified about extended outages.

## Latch: keep a condition on until another clears it

---

Sometimes a condition needs to "latch" on: once it becomes true, it stays true until a different condition clears it. Heating control with hysteresis is a common example. Turn the heater on when the temperature drops below 18°, keep it on until the temperature rises above 20°.

💡 **Tip:** For heating specifically, the [Generic thermostat](#) integration has hysteresis built in. For a generic two-threshold switch, see the [Threshold helper](#).

### template

```
template: |
 {{
 states('sensor.temperature') | float(20) < 18
 or (this.state == 'on'
 and states('sensor.temperature') | float(20) < 20)
 }}
output: "True"
```

Reading it aloud: "the heater is on if the temperature is below 18, OR if the heater is already on AND the temperature is still below 20". Put this in a template binary sensor; the `this.state` reference reads the binary sensor's own previous value, creating the latching behavior.



## Loop over a dictionary

---

Dictionaries come from JSON responses, entity attributes, action responses, and many other places. Iterate through one with `.items()`:

### template

```
template: |
 {% set data = {"temperature": 22.5, "humidity": 54, "pressure": 1013} %}
 {% for key, value in data.items() %}
 {{ key }}: {{ value }}
 {% endfor %}
output: |
 temperature: 22.5
 humidity: 54
 pressure: 1013
```

To list all attributes of an entity, call `.items()` on its `attributes`:

### template

```
template: |
 {% for name, value in states.sensor.outdoor_temperature.attributes.items() %}
 {{ name }}: {{ value }}
 {% endfor %}
output: |
 unit_of_measurement: °C
 device_class: temperature
 friendly_name: Outdoor temperature
```

# Data navigation cheat sheet

---

Templates read data using three tools: dots, brackets, and iteration. Here are the common shapes you will run into.

## Attribute with a dot:

### template

```
template: '{{ states.sensor.outdoor_temperature.state }}'
output: "22.5"
```

## Dictionary key with a bracket:

### template

```
template: |
 {% set data = {"temp": 22.5, "unit": "°C"} %}
 {{ data['temp'] }}{{ data['unit'] }}
output: "22.5°C"
```

## List item by position (starting at 0):

### template

```
template: |
 {% set sources = ["Spotify", "Bluetooth", "TV"] %}
 First: {{ sources[0] }}
 Last: {{ sources[-1] }}
output: |
 First: Spotify
 Last: TV
```

## Nested access:

### template

```
template: |
 {% set payload = {"sensor": {"readings": [22.5, 23.0, 22.8]}} %}
 First reading: {{ payload['sensor']['readings'][0] }}
 Via dots: {{ payload.sensor.readings[0] }}
output: |
 First reading: 22.5
 Via dots: 22.5
```

## List of dicts (common in action responses):

### template

```
template: |
 {% set events = [
 {"name": "Meeting", "start": "09:00"},
 {"name": "Lunch", "start": "12:00"}
] %}
 {% for event in events %}
 {{ event.start }}: {{ event.name }}
 {% endfor %}
output: |
 09:00: Meeting
 12:00: Lunch
```

When you don't know the shape of your data, pipe it through `tojson` in the template editor. That gives you a JSON view of exactly what you're working with.

## Next steps

---

- If you need to understand a pattern better, look up its functions in the [template functions reference](#).
- When a template does not work, see [Debugging templates](#).

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit](#). The forum has thousands of community templates for all kinds of situations.

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

---

## Docs/Templating/Python Methods

---

Templates are built on top of Python, and many standard Python methods are available to you. You call them the same way you would in Python: put a dot after the value, then the method name and parentheses. These methods are not listed in the [template functions reference](#) because they come from Python itself, but they solve a lot of common tasks.

This page is a cheat sheet of the ones that come up most often in Home Assistant templates. When in doubt, open the template editor and try it.

## String methods

---

Everything Home Assistant stores as text supports these. Since entity states are always text (until you convert them), these work directly on `states()` results.

### split

Splits a piece of text into a list at each occurrence of a separator.

#### template

```
template: |
 {{ "light.living_room".split(".") }}
output: ["light", "living_room"]
```

Useful for breaking entity IDs into their domain and name, splitting comma-separated strings, or pulling words out of a sentence.

### replace

Replaces one piece of text with another.

#### template

```
template: '{{ "light.living_room".replace("_", " ") }}'
output: "light.living room"
```

### lower, upper, title, capitalize

Change the case of a piece of text.

#### template

```
template: |
 Lower: {{ "Living Room".lower() }}
 Upper: {{ "Living Room".upper() }}
 Title: {{ "living room".title() }}
 Capitalize: {{ "living room".capitalize() }}
output: |
 Lower: living room
 Upper: LIVING ROOM
 Title: Living Room
 Capitalize: Living room
```

### strip

Removes whitespace from the start and end of a piece of text.

#### template

```
template: "{{ ' hello world ' .strip() }}"
output: "'hello world'"
```

## startswith, endswith

Check whether a piece of text begins or ends with another piece of text.

### template

```
template: |
 {{ "sensor.outdoor_temperature".startswith("sensor.") }}
 {{ "image.jpg".endswith(".jpg") }}
output: |
 True
 True
```

These are very handy for filtering entity IDs by domain.

## find, index, count

`find` returns the position of a piece of text, or `-1` if not found. `index` does the same but raises an error when not found. `count` counts how many times it appears.

### template

```
template: |
 {{ "sensor.outdoor_temperature".find("outdoor") }}
 {{ "hello world hello".count("hello") }}
output: |
 7
 2
```

## join

Joins a list into a single piece of text using a separator.

### template

```
template: "{{ ' ', '.join(['apples', 'oranges', 'pears']) }}"
output: "apples, oranges, pears"
```

Note the order: you call `.join()` on the separator, passing the list in. There is also a `join` filter that reads more naturally: `['apples', 'oranges', 'pears'] | join(', ')`.

## format

Inserts values into a piece of text using placeholders.

### template



```
template: '{{ "Temperature: {} {}".format(22.5, "°C") }}'
output: "Temperature: 22.5 °C"
```

## Dictionary methods

---

Dictionaries show up in entity attributes, JSON responses, and action responses.

### items

Iterate over a dictionary's key-value pairs.

#### template

```
template: |
 {% for key, value in {"a": 1, "b": 2}.items() %}
 {{ key }} = {{ value }}
 {% endfor %}
output: |
 a = 1
 b = 2
```

### keys, values

Get only the keys or only the values.

#### template

```
template: |
 {% set data = {"temp": 22.5, "humidity": 54} %}
 Keys: {{ data.keys() | list }}
 Values: {{ data.values() | list }}
output: |
 Keys: ['temp', 'humidity']
 Values: [22.5, 54]
```

### get

Fetch a value by key, with a fallback if the key is missing. This is safer than bracket lookup, which errors on missing keys.

#### template

```
template: |
 {% set data = {"name": "Frenck"} %}
 Name: {{ data.get("name", "unknown") }}
 Age: {{ data.get("age", "unknown") }}
output: |
 Name: Frenck
 Age: unknown
```

## When a key name conflicts with a dict method

If a dictionary has a key with the same name as a dict method (like `values`, `keys`, `items`, or `get`), dot notation returns the method, not the value. This commonly happens when parsing API responses.

### template

```
template: |
 {% set response = {"status": "ok", "values": [1, 2, 3]} %}
 {{ response['values'] }}
output: "[1, 2, 3]"
```

Use bracket notation ( `response['values']` ) when a key might collide with a method. It always reaches the dictionary value first.

## Datetime methods

---

When you have a datetime (for example, from `now()`), you can reach into its parts or format it.

### Accessing parts

#### template

```
template: |
 Hour: {{ now().hour }}
 Minute: {{ now().minute }}
 Weekday: {{ now().weekday() }}
 Day: {{ now().day }}
output: |
 Hour: 14
 Minute: 30
 Weekday: 5
 Day: 4
```

`weekday()` returns Monday as `0` through Sunday as `6`. Use `isoweekday()` if you prefer Monday as `1` through Sunday as `7`.

### Formatting with strftime

`strftime` formats a datetime using format codes. It is covered in detail on the [Working with dates and times](#) page, but here is the short version:

#### template

```
template: |
 {{ now().strftime('%A, %B %-d, %Y') }}
output: "Saturday, April 4, 2026"
```

### Parsing with strptime

The reverse of `strftime`. Parses a piece of text into a datetime using a format string. See `strptime`.

## Number methods

---

Numbers have a few helpful methods.

### **is\_integer**

Check whether a floating-point number has no decimal part.

#### **template**

```
template: |
 {{ (10.0).is_integer() }}
 {{ (10.5).is_integer() }}
output: |
 True
 False
```

## When to use methods vs filters

---

Both work for many tasks. Pick whichever reads better:

- **Methods** (with a dot) come from Python. `"text".upper()` , `data.get("key")` .
- **Filters** (with a pipe) come from the template engine and Home Assistant. `"text" | upper` , `data | default("key")` .

Filters chain more naturally when you have several transformations. Methods can be clearer for a single transformation or when the method returns something unusual. The [template functions reference](#) lists all the filters Home Assistant provides.

## Next steps

---

- If you're looking for a specific transformation, check the [template functions reference](#) first.
  - For template debugging, see [Debugging templates](#).
  - For date and time formatting in particular, read [Working with dates and times](#).
-

## Docs/Templating/States

---

Home Assistant keeps track of everything in your home as a collection of entities. Each entity has a **state** (its current value) and often a few **attributes** (extra details about it). When you write a template, you are almost always reading state or attributes from one or more entities.

This page explains how to get at that information inside a template.



## First, go look at your states

---

Before you write a single line of template, spend five minutes at **Settings > Developer tools > States**. This is where Home Assistant shows you every entity it knows about, its current state, and all of its attributes.

For example, you might see something like this for your outdoor thermometer:

```
Entity: sensor.outdoor_temperature
State: 22.5
Attributes:
 unit_of_measurement: °C
 device_class: temperature
 friendly_name: Outdoor temperature
```

This one entity has:

- An **entity ID** ( `sensor.outdoor_temperature` ). This is the name you use to look it up.
- A **state** ( `22.5` ). That is the entity's main value.
- Several **attributes** (unit of measurement, device class, friendly name). These are extra pieces of information that go along with the state.

When you write ``,` Home Assistant looks up that exact entity ID and gives you back its state. It is that direct.

 **Tip:** Keep the Developer Tools > States page open in a separate browser tab while you write templates. You can search for entities, see exactly what state and attributes they have, and make sure you are spelling entity IDs correctly. It is the single most useful debugging habit you can build.

## Reading a state

The `states` function gives you the current state of an entity. You pass it the entity ID as text.

### template

```
template: |
 The kitchen light is {{ states('light.kitchen') }}.
output: "The kitchen light is on."
```

## Every state is text

Here is something that catches *everyone* at least once. **Home Assistant stores every entity state as text.** Even when a sensor looks like it's giving you a number, `states()` hands it back as a piece of text. So `states('sensor.outdoor_temperature')` returns the text `'22.5'`, not the number `22.5`.

Why does that matter? Because you can't do math with text, and comparing text to a number gives surprising results. For example, `'6' < '10'` is `False` (text is sorted alphabetically, so `'6'` comes after `'1'`). If you try to do math directly, you will get an error or the wrong answer.

### template

```
template: |
 {{ states('sensor.outdoor_temperature') | float(0) + 5 }}
output: "27.5"
```

The `| float(0)` part converts the text to a number. The `0` is a fallback: if the conversion fails (maybe the sensor is offline), the template uses `0` instead of crashing.

**Rule of thumb:** whenever you do math or number comparisons on a sensor state, add `| float(0)` or `| int(0)` first. It is not optional, it is how templates work.

## States have a 255 character limit

The text stored in an entity's state can be at most 255 characters long. Home Assistant enforces this limit so the state can fit in the database and dashboards. If your template sensor needs to produce something longer (say, a list of names, a formatted table, or a long paragraph), store it in an attribute instead. Attributes don't have the same limit.

## When an entity is missing or unavailable

Home Assistant has two special state values for when things go wrong:

- `unknown` means the entity exists but Home Assistant does not know its value right now.
- `unavailable` means the entity cannot be reached at all. Maybe a device is offline or an integration failed to load.

And if you ask for an entity that does not exist at all, you get the text `unknown` back as well.

Templates that depend on live values should handle these cases gracefully. Adding a number fallback ( `| float(0)` ) fixes most math problems. For decisions, use `has_value` (covered below).

## Reading an attribute

---

Attributes carry extra details about an entity. A light has attributes for brightness and color. A weather entity has attributes for forecast data. A media player has attributes for the current track.

To read one, use `state_attr`:

### template

```
template: |
 The kitchen light is at {{ state_attr('light.kitchen', 'brightness') }}.
output: "The kitchen light is at 192."
```

Like with states, if the entity does not exist or the attribute is not set, you get nothing back (`none`). For math, add a fallback:

### template

```
template: |
 {{ state_attr('light.kitchen', 'brightness') | int(0) }}
output: "192"
```

You can find an entity's attribute names by looking at Developer Tools > States.

## Checking a state

---

You can compare a state with `==`, but there is a dedicated function that is cleaner and handles missing entities without surprises: `is_state`.

### template

```
template: |
 {{ is_state('light.kitchen', 'on') }}
output: "True"
```

This reads naturally: "is the state of `light.kitchen` equal to `on` ?". The answer is `True` or `False`.

There is a matching function for attributes, `is_state_attr`:

### template

```
template: |
 {{ is_state_attr('media_player.living_room', 'source', 'Spotify') }}
output: "True"
```

And `has_value` checks whether an entity has a usable state at all (not `unknown` or `unavailable`):

### template

```
template: |
 {% if has_value('sensor.outdoor_temperature') %}
 It is {{ states('sensor.outdoor_temperature') }}°C outside.
 {% else %}
 The outdoor sensor is unavailable.
 {% endif %}
output: "It is 22.5°C outside."
```

Use `has_value` whenever you want to fall back to a friendly message instead of showing "unavailable" on a dashboard or in a notification.

## Getting a list of entities

---

`states.domain` gives you every entity in that domain (the first part of an entity ID, like `light.` or `sensor.`). This is how you count, filter, and iterate over groups of entities.

### template

```
template: |
 There are {{ states.light | count }} lights in total.
output: "There are 12 lights in total."
```

Combine it with `selectattr` to filter the list down to what you care about. `selectattr` reads as "select entities where this attribute equals this value":

### template

```
template: |
 Lights that are on:
 {% for light in states.light | selectattr('state', 'eq', 'on') %}
 - {{ light.name }}
 {% endfor %}
output: |
 Lights that are on:
 - Kitchen
 - Hallway
 - Desk
```

## Finding entities by area, device, label, or floor

---

Home Assistant comes with a family of functions for finding entities grouped by how you've organized them:

- `area_entities` returns every entity in an area.
- `device_entities` returns every entity tied to a device.
- `label_entities` returns every entity carrying a given label.
- `floor_entities` returns every entity on a floor.
- `integration_entities` returns every entity created by a given integration.

Each has matching functions for going the other way (for example, `area_devices` lists the devices in an area). Browse the [Areas](#), [Devices](#), [Floors](#), and [Labels](#) categories in the reference for the full set.

Here is how you'd list the bedroom lights that are on:

### template

```
template: |
 Bedroom lights on:
 {% for entity in area_entities('bedroom') %}
 {% if entity.startswith('light.') and is_state(entity, 'on') %}
 - {{ state_attr(entity, 'friendly_name') }}
 {% endif %}
 {% endfor %}
output: |
 Bedroom lights on:
 - Bedroom ceiling
 - Bedside lamp
```

## The `this` variable (in template entities)

---

When you write a template that defines a [template entity](#), `this` refers to the entity itself. That is useful when the entity needs to read its own state or attributes without hardcoding its entity ID.

### automation

```
automation: |
 template:
 - sensor:
 - name: "Kitchen helper"
 state: "{{ this.attributes.get('counter', 0) + 1 }}"
 attributes:
 counter: "{{ this.state | int(0) + 1 }}"
 output: "1 (the sensor increments its own value each time it updates)"
```

`this` only exists where Home Assistant knows which entity the template belongs to. That means template entities and some automation contexts, but not the Developer Tools template editor.



## The `trigger` variable (in automations)

---

When an automation runs, it receives a `trigger` variable with details about what caused it. The fields depend on the trigger type; the [Automation trigger variables](#) page lists them all.

### automation

```
automation: |
 - trigger: state
 entity_id: binary_sensor.front_door
 to: "on"
 action:
 - action: notify.mobile_app
 data:
 message: >
 {{ trigger.to_state.name }} was opened at
 {{ trigger.to_state.last_changed.strftime('%H:%M') }}.
output: "Front door was opened at 14:32."
```

## Next steps

---

- Ready-made examples that combine these tools live on the [Common template patterns](#) page.
  - The full list of state and entity functions is in the [template functions reference](#).
  - When a template does not give you what you expected, see [Debugging templates](#).
-

## Docs/Templating/Syntax

---

Templates are made of a handful of pieces that show up everywhere. Once you've seen them a few times, you will recognize them instantly and templates will stop looking like gibberish. This page introduces each piece one at a time.

Don't try to memorize everything at once. Read through, keep this page open while you experiment in the [template editor](#), and come back when you need a refresher. That is honestly how most people learn this.

## Code lives inside markers

---

When Home Assistant reads a template, it gives you back the regular text exactly as you wrote it. But when it sees certain markers, it knows the text in between is code that needs to be calculated. Those markers are called **delimiters** (a fancy word for "the thing that marks where something starts and stops").

Templates have three delimiters. You will use the first one most of the time:

- `{{ ... }}` says "calculate this and show me the result".
- `{% ... %}` says "run this logic, but don't add anything to the output directly".
- `{# ... #}` says "this is a note for me, ignore it".

Here is all three working together:

### template

```
template: |
 {# Say hello to whoever is home #}
 Hello, {{ states('person.frenck') }}.
 {% if is_state('sun.sun', 'below_horizon') %}
 It is dark outside.
 {% endif %}
output: |
 Hello, home.
 It is dark outside.
```

Notice what happened:

- The comment ( `{# ... #}` ) was removed from the output.
- The ```` was replaced with the person's current state.
- The ```` decided whether to include the "It is dark outside." line.

That is the whole trick. Everything else on this page is details about what you can put inside those markers.

## Expressions: anything that produces a value

---

When you write `{{ ... }}`, whatever goes inside is called an **expression**. An expression is "something that can be turned into a value". The value could be a number, a piece of text, a list, or anything else.

These are all valid expressions:

- Text: `'hello'` or `"hello"` (both kinds of quotes work).
- Numbers: `42`, `3.14`.
- True and false: `true`, `false`.
- Nothing at all: `none`.
- A list of things: `[1, 2, 3]` or `['kitchen', 'bedroom']`.
- A calculation: `10 + 5`.
- A call to a function: `states('sensor.temperature')` or `now()`.
- A variable you defined earlier (more on those in [loops and conditions](#)).

When a value has parts inside it (like a state that carries attributes), you reach them with a dot or with square brackets:

### template

```
template: |
 Temperature: {{ states.sensor.outdoor.state }}
 Friendly name: {{ states.sensor.outdoor.attributes.friendly_name }}
 Same thing: {{ states.sensor.outdoor.attributes['friendly_name'] }}
output: |
 Temperature: 22.5
 Friendly name: Outdoor temperature
 Same thing: Outdoor temperature
```

Dots look cleaner. Brackets are needed when the name has a space or other character that dots don't handle. The most common place you'll hit this is with entity IDs that start with a number, like `device_tracker.2008_gmc`. Writing `states.device_tracker.2008_gmc` fails because names cannot start with a digit, so use `states.device_tracker['2008_gmc']` instead.

## Operators: doing things with values

---

An **operator** is a symbol that combines or compares values. You already know these from arithmetic at school. They work exactly the same here.

### Math

#### template

```
template: |
 Addition: {{ 10 + 5 }}
 Subtraction: {{ 10 - 5 }}
 Multiplication: {{ 10 * 5 }}
 Division: {{ 10 / 3 }}
 Integer division: {{ 10 // 3 }}
 Remainder: {{ 10 % 3 }}
 Power: {{ 10 ** 2 }}
output: |
 Addition: 15
 Subtraction: 5
 Multiplication: 50
 Division: 3.3333333333333335
 Integer division: 3
 Remainder: 1
 Power: 100
```

The last three are less common. `//` is division that throws away the decimals (so `10 // 3` is `3`, not `3.33`). `%` gives you what is left over after division. `**` raises one number to the power of another.

### Comparison

Comparison operators ask "how does this value relate to that one?". They always answer with `True` or `False`.

#### template

```
template: |
 Equal: {{ 10 == 10 }}
 Not equal: {{ 10 != 5 }}
 Greater than: {{ 10 > 5 }}
 Less than or equal: {{ 5 <= 5 }}
output: |
 Equal: True
 Not equal: True
 Greater than: True
 Less than or equal: True
```

Watch out for the double equals in `==`. A single `=` is used for assignment (giving a name to a value), while `==` is the question "are these two the same?".

## Logic

Logic operators combine multiple true-or-false answers into one. `and` needs both to be true. `or` needs at least one. `not` flips the answer. `in` checks whether something is inside a list or a piece of text.

### template

```
template: |
 Both true: {{ true and false }}
 Either true: {{ true or false }}
 Flipped: {{ not false }}
 Contains: {{ 'bedroom' in 'bedroom light' }}
output: |
 Both true: False
 Either true: True
 Flipped: True
 Contains: True
```

## Filters: changing a value

A **filter** takes a value and transforms it into something new. Filters use the `|` symbol (called a pipe). The pipe points at the filter, like handing the value over to it.

You can read a filter chain out loud left to right: "take `hello`, make it upper case" or "take these numbers, sort them, then join them with commas".

### template

```
template: |
 Upper case: {{ 'hello' | upper }}
 Rounded: {{ 3.14159 | round(2) }}
 Chained: {{ [3, 1, 2] | sort | join(',') }}
output: |
 Upper case: HELLO
 Rounded: 3.14
 Chained: 1, 2, 3
```

Home Assistant has dozens of filters for converting between types, formatting text, calculating with numbers, and working with lists. The [template functions reference](#) lists them all, each with examples.

## Many filters are also functions

A lot of Home Assistant's template functions can be used either as a filter (with `|`) or as a regular function call. These two are exactly the same:

### template

```
template: |
 As a function: {{ float(states('sensor.outdoor_temperature')) }}
 As a filter: {{ states('sensor.outdoor_temperature') | float }}
output: |
 As a function: 22.5
 As a filter: 22.5
```

The filter form reads more naturally when you are chaining several steps together. The function form can be clearer when you need an explicit fallback: `float(value, 0)` versus `value | float(0)`. Use whichever feels more readable in each situation. The reference page for each function notes which forms it supports.

## Watch out: filters run before math

Here is a gotcha that catches everyone at least once. The `|` symbol binds tighter than `+`, `-`, `*`, `/`, and all the other math operators. That means this template does not do what it looks like it should:

### template



```
template: |
 {{ 10 / 3 | round(2) }}
output: "3.3333333333333335"
```

You might read that as "ten divided by three, rounded to two decimals", which should be `3.33`. But because filters take priority, it actually runs as "ten divided by (three rounded to two decimals)", which is `10 / 3.0 = 3.3333333333333335`.

The gotcha bites hardest when the filter changes the value meaningfully. When in doubt, add parentheses so the order you want is clear:

### template

```
template: |
 With parentheses: {{ (10 / 3) | round(2) }}
 Without parentheses: {{ 10 / 3 | round(2) }}
output: |
 With parentheses: 3.33
 Without parentheses: 3.3333333333333335
```

In the "without parentheses" version, `3 | round(2)` runs first (rounding `3` gives `3`), then `10 / 3` divides as normal. The rounding had no effect. Parentheses force `10 / 3` to happen first, then round.

Whenever a template looks right but gives an unexpected result, suspect operator precedence and reach for parentheses.

## Tests: asking questions about a value

---

A **test** is a yes-or-no question you ask about a value. Tests are written with the word `is`, which reads naturally.

### template

```
template: |
 Is a number: {{ 42 is number }}
 Is text: {{ 'hello' is string }}
 Is even: {{ 6 is even }}
 Is in a list: {{ 3 is in [1, 2, 3] }}
output: |
 Is a number: True
 Is text: True
 Is even: True
 Is in a list: True
```

To ask the opposite, write `is not`:

### template

```
template: |
 {{ 42 is not number }}
output: "False"
```

Tests are most useful inside `if` statements, where you want to react differently based on what kind of value you have.

## Whitespace: trimming unwanted spaces

---

Templates keep every space and newline you write. That includes the line breaks around `and` tags. This is often fine for notifications, but when you need clean output (like a sensor value), those extra spaces become a problem.

### The problem

#### template

```
template: |
 {% set temp = 22 %}
 {% if temp > 20 %}
 Warm
 {% else %}
 Cool
 {% endif %}
 outside.
output: |

Warm

outside.
```

The output has blank lines everywhere. Each `{% %}` tag occupies a line, and that line break stays in the output even though the tag itself produces nothing visible.

### What trimming does

Adding a `-` inside a tag marker removes all whitespace (spaces, tabs, and line breaks) on that side, up to the next non-whitespace character.

- `{% ... %}` trims nothing (default).
- `{%- ... %}` trims the left side: everything before the tag.
- `{% ... -%}` trims the right side: everything after the tag.
- `{%- ... -%}` trims both sides.

The same works for expressions: `{{- ... }}`, `{{ ... -}}`, `{{- ... -}}`.

### Trimming one side

Trimming the right side (`-%`) removes the line break after a tag. This is the most common form because the blank lines come from the line break that follows each tag:

#### template

```

template: |
 {% set temp = 22 -%}
 {% if temp > 20 -%}
 Warm
 {% else -%}
 Cool
 {% endif -%}
 outside.
output: |
 Warm
 outside.

```

The `-%` on each tag eats the line break after it, so the tag and the content after it end up on the same line. "Warm" and "outside." each get their own line, which is clean output.

Trimming the left side (`{%-`) removes whitespace before the tag. This pulls the tag toward whatever came before it:

### template

```

template: |
 The door is
 {%- if true %} open{% endif %}.
output: "The door is open."

```

The `{%-` on the `if` tag eats the line break and spaces before it, so "is" and "open" connect without a gap. Without the `-`, there would be a line break between "is" and "open".

## Trimming both sides

When every tag trims both sides, the output becomes as tight as possible:

### template

```

template: |
 {%- set temp = 22 -%}
 {%- if temp > 20 -%}
 Warm
 {%- else -%}
 Cool
 {%- endif -%}
 output: "Warm"

```

Everything collapses onto one line. The `{%- ... -%}` on every tag removes all surrounding whitespace.

## Mixing trimmed and untrimmed

You do not have to trim every tag. Choose based on what the output should look like:

### template

```

template: |
 {%- set temp = 22 -%}
 {%- set humidity = 65 -%}
 Temperature: {{ temp }}°C
 Humidity: {{ humidity }}%
output: |
 Temperature: 22°C
 Humidity: 65%

```

The `{%- set ... -%}` tags are fully trimmed (they are setup, not output). The `{{ }}` expressions are not trimmed because we want each reading on its own line.

## Expressions can trim too

The `-` works inside `{{ }}` the same way:

### template

```

template: |
 Status:
 {{- ' OK' if true else ' Error' }}
output: "Status: OK"

```

The `{{-` eats the line break between "Status:" and the expression, so the result is one line. The space before "OK" is inside the string itself, so it survives.

## When to trim

- **Notifications and messages:** Usually no trimming needed. Extra blank lines are harmless and can improve readability.
- **Sensor values:** Trim aggressively. A template sensor's state should be a clean value like `22.5`, not `22.5\n` with extra whitespace.
- **Building values on one line:** Trim both sides of every `set`, `if`, and `endif` tag.
- **Multi-line output:** Trim the `set` and logic tags but leave the output expressions untrimmed so each result gets its own line.

## Putting it all together

---

Most real templates mix several of these pieces. Here is one you might see in an automation:

### template

```
template: |
 {% set temp = states('sensor.outdoor_temperature') | float(0) %}
 It is {{ temp | round(1) }}°C outside,
 which is {{ 'warm' if temp > 20 else 'cool' }}.
output: "It is 22.5°C outside, which is warm."
```

Let's read it piece by piece:

1. `**`**` reads the outdoor temperature, converts it to a number (with `0` as a fallback in case the sensor is offline), and stores the result in a variable named `temp`.
2. ```` shows that number rounded to one decimal place.
3. `{{ 'warm' if temp > 20 else 'cool' }}` picks between two words. If `temp` is more than 20, it picks "warm"; otherwise, "cool".

If that feels like a lot, read the [Loops and conditions](#) page next. It explains `set`, `if`, and `for` in detail.

## Next steps

---

- Learn about [loops and conditions](#) to make your templates smarter.
  - Writing templates inside automations or scripts? Read [Templates in YAML](#) for the quoting rules.
  - Browse the [template functions reference](#) for the full list of filters, tests, and functions.
  - Try every example on this page in the [template editor](#). Experimenting is the fastest way to learn.
-

## Docs/Templating/Tutorial Average Temperature

---

In this tutorial, you will build a template sensor that averages all temperature readings in your home into a single number. It's ideal for seeing your whole-house temperature at a glance, or for triggering automations based on overall comfort. It shows up on your dashboard like any other sensor, and it updates automatically whenever one of the underlying temperature readings changes.

This is a classic first template sensor. You will learn how to gather readings from many entities, average them, round the result, and wire the whole thing into Home Assistant as a real sensor you can use anywhere else.



## What you will build

---

A sensor called `sensor.home_average_temperature` that you can place on any dashboard. It shows a single number like `21.4` °C, calculated from all the temperature sensors in your home.

You get a real sensor entity, so you can also use it in automations, chart it in history, or reference it from other templates.

## Before you start

---

You need:

- At least two temperature sensors in Home Assistant. They should have `device_class: temperature` in their attributes. Most climate integrations set this automatically.
- Five minutes with the [Developer tools template editor](#) open.

If you want your result in Fahrenheit, see the [Going further](#) section at the bottom.

## Step 1: See your temperature sensors

---

Open **Settings** > **Developer tools** > **States** and filter on `temperature` .

You should see a list of entities with `device_class: temperature` in their attributes. The state column shows the current reading as text, like `22.5` .

These are the sensors your new sensor will average. Make a mental note of how many you have.

## Step 2: Write the averaging template

---

In **Developer tools** > **Template**, paste this:

**template**

```
template: |
 {{
 states.sensor
 | selectattr('attributes.device_class', 'eq', 'temperature')
 | rejectattr('state', 'in', ['unknown', 'unavailable'])
 | map(attribute='state') | map('float') | average | round(1)
 }}
output: "21.4"
```

That is one long line. Read it left to right as one sentence:

1. Start with every `sensor` entity.
2. Keep only the ones whose `device_class` is `temperature`.
3. Drop any that are `unknown` or `unavailable`.
4. From each remaining entity, pull out only the `state` value.
5. Convert each state from text to a number.
6. Average them.
7. Round the result to one decimal.

Your result should be a single number close to what you feel in your home.

 **Tip:** If you see an error, try running the template one piece at a time. Remove the `| round(1)`, then `| average`, then `| map('float')`, and see where it breaks. Step-by-step narrowing is explained in [Debugging templates](#).

## Step 3: Make it a real sensor

---

A one-off template in the editor is great for testing. To use the result on a dashboard, you need to turn it into an actual entity. You do this with a **template helper**, which Home Assistant creates for you from the user interface.

1. Go to **Settings > Devices & services > Helpers**.
2. Select **Create helper** in the bottom-right.
3. Pick **Template**.
4. Pick **Template a sensor**.

### Fill in the form

You will see a form with several fields. Fill them out like this:

- **Name:** `Home average temperature`. This is what you will see in dashboards and the app, and it is used to generate the entity ID ( `sensor.home_average_temperature` ).
- **State template:** paste the template you tested in step 2. Paste it exactly, including the `{{ ... }}` delimiters.
- **Unit of measurement:** `°C` (or `°F` if your sensors report Fahrenheit).
- **Device class:** `Temperature`. This tells Home Assistant this sensor represents a temperature so dashboards and voice assistants pick the right icon and handle units correctly.
- **State class:** `Measurement`. This tells Home Assistant the value changes continuously over time, which is what makes the sensor chartable in history.

Select **Submit**. Your new sensor is ready to use immediately, no restart needed.

 **Tip:** If you later rename the helper, its **unique ID** stays the same. That is the internal identifier Home Assistant uses to track this sensor across renames, moves, and configuration tweaks. History, dashboards, and automations keep working because they reference the unique ID behind the scenes. You don't have to do anything with it, but it's good to know it exists if you ever see it mentioned elsewhere.

## Step 4: Check it

---

Your new helper now appears on the **Settings > Devices & services > Helpers** page. Find `Home average temperature` in the list and select it to open the details dialog. The current value shows there.

If the value looks wrong, use the three-dot menu in the dialog to open the helper's settings and adjust the template. Changes save immediately.

## Step 5: Add it to your dashboard

---

1. Open a dashboard in edit mode.
2. Select **Add card**.
3. Pick a **Tile** card (clean display with optional extras) or a **Gauge** card (visual dial).
4. Choose `sensor.home_average_temperature` from the entity list.
5. Save the dashboard.

The card updates automatically whenever any of your temperature sensors changes.

## Going further

---

A few ways to extend this:

- **Fahrenheit.** If your sensors report in Fahrenheit, open the sensor's settings from the entity and set **Unit of measurement** to °F. No changes to the template are needed.
- **By area.** Instead of averaging the whole house, limit it to a single area. Replace the `states.sensor` starting point with `area_entities('living_room')` and add a step to filter for temperature sensors. See `area_entities`.
- **Highest and lowest too.** Add two more sensors that use `max` and `min` instead of `average`. Great for seeing the spread.
- **Weighted average.** If you want the bedroom thermostat to count more than the garage sensor, you will need a bit more template logic. A `for` loop with manual weighting handles that.
- **Skip sensors reporting unrealistic values.** If a broken sensor reports `999`, it will skew your average. Filter out values outside a reasonable range with an additional step before `| average`.
- **Group by type.** You can build one for humidity, one for pressure, one for CO<sub>2</sub>. Same template shape, different `device_class` value.



## Next steps

---

- The [Working with states](#) page explains the `selectattr`, `rejectattr`, and `map` filters used here.
  - The [Common template patterns](#) page has more aggregation recipes.
  - If this is the first tutorial you did, try the [Tutorial: get notified when a device needs a new battery](#) next to learn about templates in automations.
-

## Docs/Templating/Tutorial Battery Alerts

---

In this tutorial, you will build a daily notification that tells you which devices need new batteries. It looks at every battery sensor in your home, picks out the ones below a threshold you choose, and sends you a message listing each device with its location and current percentage.

This is one of the most popular templates in the Home Assistant community, and it is a great first real-world template. You will learn how to loop over a collection of sensors, filter by state, build up a list, and format a message. All of these skills carry over to many other templates you will write later.

## What you will build

---

A notification that arrives each morning with a message like this:

```
2 devices need new batteries: Front door lock in Hallway (15%), Motion sensor in
```

If everything is healthy, you get a reassuring message instead:

```
All batteries are healthy.
```

## Before you start

---

You need:

- At least one battery-powered device reporting a percentage through Home Assistant. Zigbee, Z-Wave, and Matter devices usually do this automatically.
- A notification service set up on your phone, usually the [Home Assistant Companion app](#). The examples below use `notify.mobile_app_your_phone`. Replace that with your own `notify` action when you get there.
- Five minutes with the [Developer tools template editor](#) open.

## Step 1: See your battery sensors

---

Before you write a single template, it helps to see what you are working with. Open **Settings** > **Developer tools** > **States** and filter on `battery`.

You should see a list of sensors with `device_class: battery` in their attributes and a number (like `85` or `12`) in the state column. Those are the entities this automation will watch.

If you do not see any, your devices might be using the old `binary_sensor` battery class (which reports `on` / `off` instead of a percentage) or they do not report battery at all. This tutorial focuses on the numeric `sensor` kind.

## Step 2: List the low batteries

Open **Developer tools** > **Template** and paste this in:

**template**

```
template: |
 {% set low = namespace(batteries=[]) %}
 {% for sensor in states.sensor
 | selectattr('attributes.device_class', 'eq', 'battery')
 | rejectattr('state', 'in', ['unknown', 'unavailable']) %}
 {% if sensor.state | float(100) < 20 %}
 {% set low.batteries = low.batteries + [device_name(sensor.entity_id)] %}
 {% endif %}
 {% endfor %}
 {{ low.batteries }}
output: "['Front door lock', 'Motion sensor', 'Bedroom sensor']"
```

Read it left to right:

1. Create a namespace called `low` with an empty `batteries` list inside.
2. For each sensor whose `device_class` is `battery`, skipping any that are `unknown` or `unavailable` ...
3. If its state (converted to a number) is below `20` ...
4. Add the name of the **device** that sensor belongs to to `low.batteries`.

Why the device name? With modern Home Assistant naming, a battery sensor's own name is often only "Battery", which is not very helpful in a notification. The `device_name` function gives you back the friendly name of the device the sensor is attached to (like "Front door lock" or "Motion sensor").

You should see a list of device names with low batteries. If the list is empty, you can temporarily raise the threshold (change `20` to `100`) to confirm the template is working.

 **Note:** Variables set inside a `for` loop do not survive outside the loop, so `low` is created with a `namespace`. That is the object whose attribute (`low.batteries`) can be updated inside the loop and still hold the full list afterwards. See [Loops and conditions](#) for more on this quirk.

## Step 3: Format the message

A bare list is not what you want to send to your phone. Add the area the device is in, the current battery level, and turn the result into a friendly sentence. Replace the template with this:

### template

```
template: |
 {% set low = namespace(batteries=[]) %}
 {% for sensor in states.sensor
 | selectattr('attributes.device_class', 'eq', 'battery')
 | rejectattr('state', 'in', ['unknown', 'unavailable']) %}
 {% if sensor.state | float(100) < 20 %}
 {% set device = device_name(sensor.entity_id) %}
 {% set area = area_name(sensor.entity_id) %}
 {% set label = device ~ (' in ' ~ area if area else '')
 ~ ' (' ~ sensor.state ~ '%)' %}
 {% set low.batteries = low.batteries + [label] %}
 {% endif %}
 {% endfor %}
 {% if low.batteries | count == 0 %}
 All batteries are healthy.
 {% elif low.batteries | count == 1 %}
 1 device needs a new battery: {{ low.batteries[0] }}.
 {% else %}
 {{ low.batteries | count }} devices need new batteries: {{ low.batteries | join }}
 {% endif %}
output: |-
 3 devices need new batteries: Front door lock in Hallway (15%),
 Motion sensor in Kitchen (12%), Window sensor in Bedroom (8%).
```

A few things changed from the previous step:

- Two helper variables: `device_name` gives you the device's friendly name, and `area_name` gives you the name of the area it is assigned to.
- `label` builds one formatted piece of text per device, combining the device name, the area (only if it is set), and the battery percentage. The `~` symbol glues text together.
- An `if / elif / else` picks between three messages based on how many items are in the list. No low batteries get a friendly "all healthy" message. One low battery uses singular wording. Two or more get the full list joined with commas.

Play with the template. Change the threshold, remove the `%`, reword the messages. The template editor updates as you type, so you can see the result of every change right away.

## Step 4: Create the automation

---

Now turn the template into an actual automation.

1. Go to **Settings > Automations & scenes**.
2. Select **Create automation**, then **Create new automation**.
3. Give it a name: `Low battery notification`.

### Add the trigger

You want this to run once every morning.

1. Under **When**, select **Add trigger**.
2. Choose **Time**.
3. Set the time to something reasonable, like `09:00:00`.

### Add the action

1. Under **Then do**, select **Add action**.
2. Choose **Perform Action**, then pick your `notify` action (like `notify.mobile_app_your_phone`).
3. Fill in the message field with the template you built in step 3.

In YAML, the action looks like this:

**action**



```

action: |
- action: notify.mobile_app_your_phone
data:
 title: "Battery check"
 message: >
 {% set low = namespace(batteries=[]) %}
 {% for sensor in states.sensor
 | selectattr('attributes.device_class', 'eq', 'battery')
 | rejectattr('state', 'in', ['unknown', 'unavailable']) %}
 {% if sensor.state | float(100) < 20 %}
 {% set device = device_name(sensor.entity_id) %}
 {% set area = area_name(sensor.entity_id) %}
 {% set label = device ~ (' in ' ~ area if area else '')
 ~ ' (' ~ sensor.state ~ '%)' %}
 {% set low.batteries = low.batteries + [label] %}
 {% endif %}
 {% endfor %}
 {% if low.batteries | count == 0 %}
 All batteries are healthy.
 {% elif low.batteries | count == 1 %}
 1 device needs a new battery: {{ low.batteries[0] }}.
 {% else %}
 {{ low.batteries | count }} devices need new batteries: {{ low.batteries }}
 {% endif %}

```

1. Save the automation.

## Step 5: Test it

---

You do not want to wait until tomorrow morning to find out if it works.

1. Open your automation.
2. Select the **Run** button (or use the three-dots menu and choose **Run actions**).

The notification should arrive on your phone within a few seconds. If it does not, check that the `notify` action name matches what you have on your setup, and look at the automation's trace for clues.

## Going further

---

A few ways to take this further:

- **Quiet "healthy" days.** Skip the notification entirely when the list is empty. Put the "all healthy" message in an `if` branch that does nothing, or use a template condition before the notification action.
- **Different thresholds.** Replace `20` with a helper input so you can tune the threshold from the dashboard without editing the automation.
- **Weekly instead of daily.** Change the time trigger to a more specific schedule like "every Monday at 9am".
- **Include unavailable devices.** The current template skips sensors that are `unknown` or `unavailable`, but those might indicate a completely dead battery. You can add a second list for offline devices and mention them in the message.
- **Sort by percentage.** If you have many devices, sort the list with the most-drained device first so you know what to replace first.

## Next steps

---

- The [Loops and conditions](#) page explains the `for` loop and `if / elif / else` used here.
  - The [Common template patterns](#) page has more cookbook-style recipes.
  - Try the [Tutorial: show the average home temperature on your dashboard](#) next to learn how to build a template sensor.
-

## Docs/Templating/Types

---

Templates work with several kinds of values: text, numbers, lists, and a few others. Each kind is called a **type**, and knowing which type you have matters because most operations only work with specific types. Trying to do math on text, or count items in an iterable, gives you surprising results until you convert first.

This page walks through the types you will meet, how they interact, and when you need to convert between them.

## The types you will meet

---

- **Text** ( `str` or `string` ): a piece of text, written in single or double quotes. Every entity state in Home Assistant is stored as text.
- **Integer** ( `int` or `integer` ): a whole number without a decimal point. Examples: `0` , `42` , `-7` .
- **Float** ( `float` ): a number with a decimal point. Examples: `3.14` , `-0.5` , `22.0` .
- **Boolean** ( `bool` or `boolean` ): either `True` or `False` . The answer to yes-or-no questions.
- **None** ( `NoneType` or `None` ): the absence of a value. Returned when something does not exist.
- **List**: an ordered collection of values, written with square brackets. Example: `[1, 2, 3]` or `['kitchen', 'bedroom']` . Also called an array or sequence.
- **Dictionary** ( `dict` ): a set of key/value pairs, written with curly braces. Example: `{'temp': 22, 'humidity': 54}` . Also called a map or mapping.
- **Iterable**: a sequence of values you can loop over **once**. Also called a generator. Filters like `map` , `select` , and `selectattr` return iterables. You cannot count them, index them, or use them twice until you turn them into a list. Covered in detail [below](#).
- **Datetime**: a moment in time with a date, time, and time zone. Returned by `now()` and the `last_changed` attribute on states.

## Why types matter

---

Most functions and operators only work with specific types. Here are some examples:

- Math operators ( `+` , `-` , `*` , `/` ) need **numbers**. Using them on text fails.
- `| count` or `| length` needs a **list** or similar countable. It does not work on an iterable.
- Comparison with `<` , `>` works on numbers correctly, but on text it compares alphabetically ( `'6' > '10'` is `True` because `'6'` comes after `'1'` ).
- The `in` operator works on lists, text, and dicts, but the behavior is different for each.

When a template produces an unexpected result, the type of the values involved is usually the first thing to check.

## Every state is text

---

This is the single most important type fact in Home Assistant templates: **every entity state is stored as text**. Even when a sensor looks like it reports a number, `states('sensor.temperature')` returns the text `'22.5'`, not the number `22.5`.

### template

```
template: "{{ states('sensor.temperature') | typeof }}"
output: "str"
```

This is why you see `| float(0)` or `| int(0)` everywhere in template examples. Before you can do math, compare numerically, or average a bunch of values, you have to convert them to numbers.

### template

```
template: |
 {# This looks like math but is text concatenation #}
 {{ states('sensor.a') + states('sensor.b') }}
output: "22.518.5"
```

### template

```
template: |
 {# This does real math #}
 {{ states('sensor.a') | float(0) + states('sensor.b') | float(0) }}
output: "41.0"
```



## Converting between types

---

Each type has a matching filter (and function) to convert a value to that type:

- `| float(default)` : converts to a decimal number. If the value cannot be converted, the default is used.
- `| int(default)` : converts to a whole number. Decimals are dropped.
- `| string` : converts anything to text.
- `| bool(default)` : converts to `True` or `False` .
- `| list` : materializes an iterable into a list.

The `default` arguments for `float` , `int` , and `bool` are fallbacks used when the conversion fails (for example, a sensor that reports `unavailable` ).

### template

```
template: |
 {{ "3.14" | float(0) }}
 {{ "seven" | float(0) }}
 {{ "42" | int(0) }}
 {{ 3.7 | int }}
 {{ 1 | string }}
 {{ "yes" | bool }}
output: |
 3.14
 0.0
 42
 3
 1
 True
```

**Rule of thumb:** whenever you read a state and need to do math with it, add `| float(0)` or `| int(0)` with a sensible fallback.

## Iterables look like lists, but are not

---

Some filters return an iterable instead of a list. An iterable is a lazy sequence. It gives you values one at a time when you loop over it, but it is not a list. You cannot count it, index it, or use it twice.

The Home Assistant filters that return iterables are:

- `map`
- `select`
- `reject`
- `selectattr`
- `rejectattr`

If you try to count an iterable directly, you get an error:

### template

```
template: |
 {# This fails #}
 {{ states.light | selectattr('state', 'eq', 'on') | count }}
output: "Error: object of type 'generator' has no len()"
```

The fix is to add `| list` to turn the iterable into a list first:

### template

```
template: |
 {# This works #}
 {{ states.light | selectattr('state', 'eq', 'on') | list | count }}
output: "3"
```

You will see `| list` at the end of filter chains all over Home Assistant templates. It is not optional decoration, it is the step that makes the result countable, sortable, and reusable. A good habit: if you feed the result of `map`, `select`, `reject`, `selectattr`, or `rejectattr` into anything that needs a list, add `| list`.

## Getting items from lists and dictionaries

---

Collections give you several ways to reach the items inside them.

### From a list

Use the `first` and `last` filters to grab the ends, or square brackets with an index to reach a specific position. Indices start at `0`, and negative numbers count from the end.

#### template

```
template: |
 {% set rooms = ['kitchen', 'bedroom', 'garage'] %}
 First: {{ rooms | first }}
 Last: {{ rooms | last }}
 Index 0: {{ rooms[0] }}
 Index 1: {{ rooms[1] }}
 Index -1: {{ rooms[-1] }}
output: |
 First: kitchen
 Last: garage
 Index 0: kitchen
 Index 1: bedroom
 Index -1: garage
```

### From a dictionary

Dictionaries support two ways to read a value: bracket notation ( `data['key']` ) and dot notation ( `data.key` ). Both usually work.

#### template

```
template: |
 {% set data = {'temp': 22.5, 'humidity': 54} %}
 Bracket: {{ data['temp'] }}
 Dot: {{ data.temp }}
output: |
 Bracket: 22.5
 Dot: 22.5
```

**Use bracket notation when a key name could conflict with a dict method.** If a key is named `values`, `keys`, `items`, `get`, or any other built-in dict method, dot notation returns the method instead of your value. This is common with API responses.

#### template

```
template: |
 {% set response = {'status': 'ok', 'values': [1, 2, 3]} %}
 {{ response['values'] }}
output: "[1, 2, 3]"
```

When in doubt, reach for bracket notation. It always looks up the dictionary value first.

## Checking what type you have

---

When you are not sure what type a value is, use `typeof` to inspect it:

### template

```
template: |
 {{ 42 | typeof }}
 {{ 3.14 | typeof }}
 {{ "hello" | typeof }}
 {{ [1, 2, 3] | typeof }}
 {{ {"a": 1} | typeof }}
 {{ None | typeof }}
output: |
 int
 float
 str
 list
 dict
 NoneType
```

The names you get back are the Python type names: `str` for text, `int` for whole numbers, `float` for decimals, `bool` for booleans, `list`, `dict`, `NoneType` for `None`.

## Testing types with tests

---

Tests let you check a type inside an `if` condition, without calling `typeof` :

### template

```
template: |
 {{ 42 is number }}
 {{ "hello" is string }}
 {{ [1, 2, 3] is iterable }}
 {{ {"a": 1} is mapping }}
 {{ None is none }}
output: |
 True
 True
 True
 True
 True
```

Useful type tests: `number` , `string` , `boolean` , `integer` , `float` , `iterable` , `mapping` , `none` , `defined` . See the [Comparison](#) category in the reference for the full list.

## Common type mistakes

---

- **Doing math on a sensor state without** `| float(0)`. Always convert first.
- **Comparing text to a number.** `states('sensor.temp') < 20` compares text to a number. Always `| float(0)` first.
- **Using `<` or `>` on text.** Text compares alphabetically, which rarely does what you want for numbers.
- **Forgetting `| list` after** `map`, `select`, `reject`, `selectattr`, `rejectattr`. The result is an iterable, not a list, so `| count` and `| length` fail.
- **Expecting `None` or `unknown` to be a number.** Both break math operations. Use a fallback.

## Next steps

---

- For hands-on debugging, see [Debugging templates](#).
  - For more on working with states, see [Working with states](#).
  - For recipes that use these conversions, see [Common template patterns](#).
-



## Docs/Templating/Where To Use

---

Once you know what templates are, the next question is usually "where do I actually put one?". You'll reach for a template when you write a notification with live sensor values, trigger an automation based on the current state of your home, or build a dashboard number from several entities at once. Templates show up in a lot of places in Home Assistant, and this page walks through the most common ones so you can recognize each spot when you meet it.

You don't need to memorize this. Come back when you're building something and need to remember which field accepts a template.

## Automation and script actions

---

Most actions accept templates for their data values. You can use them to compose messages, pick targets, or tailor behavior based on the current state of your home.

### automation

```
automation: |
 - alias: "Goodnight message"
 triggers:
 - trigger: time
 at: "23:00:00"
 actions:
 - action: notify.mobile_app
 data:
 message: >
 Goodnight.
 {{ states('sensor.front_door') | capitalize }} front door, and
 {{ states.light | selectattr('state', 'eq', 'on') | list | count }}
 lights still on.
 output: "Goodnight. Closed front door, and 3 lights still on."
```

## Automation conditions

---

A [template condition](#) lets an automation decide whether to continue based on any test you can write. The template needs to end up either `True` or `False`, and the automation only continues when the answer is `True`.

### condition

```
condition: |
 conditions:
 - condition: template
 value_template: >
 {{ states('sensor.outdoor_temperature') | float(0) < 5 }}
 output: "True (when the outdoor temperature is below 5°C)"
```

## Automation triggers

---

A [template trigger](#) fires when the template changes from false to true. This lets you react to conditions that no single entity represents.

### trigger

```
trigger: |
 triggers:
 - trigger: template
 value_template: >
 {{
 states('sensor.washing_machine_power') | float(0) < 5
 and states('sensor.washing_machine_power') | float(0) > 0
 }}
 output: "True (fires when the washing machine power drops between 0 and 5 W)"
```

 **Note: Limited templates.** Some trigger types, and the `trigger_variables` section of an automation, only support a reduced set of template features. This is called a "limited template". Most of Home Assistant's state-reading functions (like `states()`, `state_attr()`, and the area/device/label helpers) are not available there. If a template works in the editor but fails in a trigger configuration, the limited-template scope is a likely cause. Check the specific trigger's documentation for the details.

## Template entities

---

The [Template integration](#) lets you create entities whose value is computed from other entities. Template sensors, binary sensors, switches, and more are defined entirely with templates.

### automation

```
automation: |
 template:
 - sensor:
 - name: "Lights on"
 state: >
 {{ states.light | selectattr('state', 'eq', 'on') | list | count }}
 output: "5 (the sensor reports the number of lights currently on)"
```

## Notifications

---

The [notify actions](#) accept templates for the title, message, and often other fields. This is one of the most popular places to use templates, because it turns fixed notifications into personal, context-aware messages.

### action

```
action: |
 - action: notify.mobile_app
 data:
 title: "Door alert"
 message: >
 {{ trigger.to_state.name }} was opened at
 {{ now().strftime('%H:%M') }}.
output: "Front door was opened at 14:32."
```

## REST and command-line sensors

---

The `rest` and `command_line` sensors use templates to pull values out of the raw response they receive. This is how you turn a JSON reply or a command's output into a usable sensor.

### automation

```
automation: |
 sensor:
 - platform: rest
 resource: https://api.example.com/weather
 name: "Outside humidity"
 value_template: "{{ value_json.current.humidity }}"
 unit_of_measurement: "%"
 output: "72 (extracted from the JSON field current.humidity)"
```

## MQTT

---

The [MQTT integration](#) accepts templates for topics, payloads, and value extraction. Incoming payloads are often JSON, so a template extracts the field you need.

### automation

```
automation: |
 mqtt:
 sensor:
 - name: "Garden moisture"
 state_topic: "garden/sensor/moisture"
 value_template: "{{ value_json.moisture }}"
 unit_of_measurement: "%"
 output: "45 (extracted from an MQTT payload like {\"moisture\": 45})"
```

MQTT uses two kinds of templates. A **value template** transforms an incoming payload into an entity state or attribute. A **command template** turns an action into the outgoing payload that the device expects. Both get the usual `value` and `value_json` variables, plus three extras specific to MQTT:

- `entity_id`: the entity ID that the template belongs to.
- `name`: the friendly name of the entity.
- `this`: the entity's own state object (the same one you meet in template entities).

The [MQTT integration documentation](#) has the full list of where each template type applies and which fields on each entity support templating.



## Processing incoming data

---

The REST, MQTT, and command-line examples above use two special variables that need a word of introduction: `value` and `value_json`. You will run into them anywhere Home Assistant pulls data from an outside source and hands it to a template for shaping.

When raw data comes in, the template that processes it has these extras available:

- `value` holds the raw incoming data as text. It is always there.
- `value_json` holds the same data parsed as JSON. It is only there when the data actually is valid JSON.

So when an MQTT payload arrives like this:

```
{ "state": "ON", "temperature": 21.9, "humidity": 54 }
```

You can reach the fields with dot notation:

### template

```
template: |
 Temperature: {{ value_json.temperature }}
 Humidity: {{ value_json.humidity }}
 State: {{ value_json.state }}
output: |
 Temperature: 21.9
 Humidity: 54
 State: ON
```

## Nested JSON

Real-world payloads often have nested structures. Use square brackets or more dots to reach deeper:

### template

```
template: |
 {% set value_json = {
 "sensor": {"type": "air", "id": "12345"},
 "values": {"temp": 26.09, "hum": 56.73}
 } %}
 Sensor ID: {{ value_json['sensor']['id'] }}
 Temperature: {{ value_json.values.temp }}
output: |
 Sensor ID: 12345
 Temperature: 26.09
```

Square brackets become necessary when a field name contains characters that dots don't handle, like a dash or a space.

## JSON arrays

If the data is a list, index into it with square brackets (starting at zero):

### template

```
template: |
 {% set value_json = {"primes": [2, 3, 5, 7, 11, 13]} %}
 First prime: {{ value_json.primes[0] }}
 Third prime: {{ value_json.primes[2] }}
output: |
 First prime: 2
 Third prime: 5
```

## When the data is not JSON

If the incoming data is plain text or a number (say, from a command-line sensor that outputs `42.5`), use `value` directly:

### template

```
template: |
 {% set value = "42.5" %}
 {{ value | float(0) + 1 }}
output: "43.5"
```

Remember: `value` is always text, so convert with `| float(0)` or `| int(0)` before doing math.

## Testing an incoming-data template

The template editor does not know what `value_json` or `value` would be in a real incoming payload, because there is no live payload at that moment. To test a template that uses these variables, define them yourself at the top with ``:

### template

```
template: |
 {% set value_json = {"name": "Outside",
 "data": {"temp": "24C", "hum": "35%"} } %}
 Humidity reading: {{ value_json.data.hum[: -1] }}%
output: "Humidity reading: 35%"
```

This lets you work out the right template in the editor, then paste the finished version into your REST sensor, MQTT entity, or command-line sensor without needing the real device to send data.

## Dashboards

---

Most dashboard cards accept templates for titles, names, and other text fields. Support varies by card, and some cards need `state_template` or similar fields to make templates work. The [Markdown card](#) is the most flexible, as its entire content is a template.

## Not everywhere

---

Not every field accepts templates. As a rule:

- Text, numbers, and lists in automation actions, conditions, triggers, and scripts can usually be templated.
- Entity IDs and structural YAML keys usually cannot.
- Dashboards support templates in specific fields only. Check the card's documentation.

If a template is not being evaluated, it is most likely in a field that does not support templating. The [Debugging templates](#) page has more tips.

## Next steps

---

Now that you know where templates live, learn how to write them:

- Start with [Template syntax](#) for the building blocks.
  - Read [Templates in YAML](#) for the quoting rules that trip everyone up.
  - Browse [Common template patterns](#) for recipes you can copy.
  - When something does not work, head to [Debugging templates](#).
-

## Docs/Templating/Yaml

---

Most of the time, templates live inside YAML files. YAML has its own rules about quoting and multi-line text, and those rules interact with templates in ways that trip everyone up at least once. This page covers the traps and the patterns that get around them.

Don't feel bad when YAML bites you. It bites everyone. Come back to this page whenever Home Assistant refuses to load your config and the error message makes no sense.

## The quoting rule

---

A single-line template *must* be wrapped in quotes. Either single ( `' '` ) or double ( `" "` ) will do.

### automation

```
automation: |
 # Correct: template is quoted
 value_template: "{{ states('sensor.temp') | float > 20 }}"
 output: "True (when the sensor reports a value above 20)"
```

Without the quotes, YAML tries to interpret `{{` as the start of a flow-style mapping and fails. The error message you get in that case can be confusing, so if something refuses to load, check your quotes first.

There is no difference in behavior between single and double quotes as far as Home Assistant is concerned. Pick one style and stick with it.

## Quotes inside the template

---

If the template itself contains quotes, wrap it in the other kind to avoid clashes.

### **automation**

```
automation: |
 # Template contains single quotes, so wrap in double quotes
 value_template: "{{ is_state('light.kitchen', 'on') }}"

 # Template contains double quotes, so wrap in single quotes
 value_template: '{{ states["sensor.temp"] }}'
output: "Both templates evaluate correctly without quote conflicts"
```

If you need both kinds of quotes in the same template, switch to a multi-line string (see below).



## Multi-line templates

When a template grows beyond a single line, quoting gets painful. YAML has two styles for writing text across multiple lines without needing quotes at all: the `>` style and the `|` style. Pick whichever matches what you want the output to look like.

The `>` style (**folded**) joins your lines back into one, with a single space between them. Use this when your template is really one long piece of code that you broke into multiple lines to fit on the page nicely.

### automation

```
automation: |
 value_template: >
 {{
 states('sensor.temp') | float(0) > 20
 and states('sensor.humidity') | float(0) < 60
 }}
 output: "True (when both conditions are true)"
```

The `|` style (**literal**) keeps every line break exactly as you wrote it. Use this when your template output needs the line breaks preserved, like a multi-line notification message.

### automation

```
automation: |
 message: |
 Today's summary:
 Temperature: {{ states('sensor.temp') }}°C
 Humidity: {{ states('sensor.humidity') }}%
 output: |
 Today's summary:
 Temperature: 22.5°C
 Humidity: 54%
```

## The `>-` and `|-` variants

A trailing `-` strips the final newline. You will see this used often for `value_template` where you do not want a trailing blank line.

### automation

```
automation: |
 value_template: >-
 {% if states('sensor.outdoor_temp') | float(0) < 0 %}
 freezing
 {% else %}
 not freezing
 {% endif %}
 output: "not freezing"
```

## Indentation inside multi-line blocks

---

Inside a `|` or `>` block, YAML figures out the indentation from the first non-empty line. Every following line must be indented at least as far. If you mix indents, YAML will cut the block off at the less-indented line, and the rest will look like a new key to YAML.

### automation

```
automation: |
 # Wrong: second line is less indented than the first
 message: |
 Status Report
 Temperature: 22
```

### automation

```
automation: |
 # Correct: all lines share the same base indent
 message: |
 Status Report
 Temperature: 22
 output: |
 Status Report
 Temperature: 22
```

If you need more indentation on specific lines (for example, centered text), add it with the template itself rather than with extra YAML indentation.

## Common mistakes

---

- **Unquoted single-line template.** YAML will complain about the `{{ }}`. Add quotes.
- **Starting with a quote but forgetting to close it.** A common slip with long templates. Switch to a `>-` multi-line block when in doubt.
- **Using `>` when you need line breaks preserved.** The output becomes one long line. Use `|` instead.
- **Inconsistent indentation inside `|` or `>` blocks.** YAML truncates the block at the lowest indent.

When a template refuses to work, the [Debugging templates](#) page walks through how to narrow the problem down.

## Next steps

---

- For the core language itself, see [Template syntax](#).
- When a template does not behave, the [Debugging templates](#) page walks through the process.
- For ready-made examples you can adapt, see [Common template patterns](#).

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

---

## Docs/Templating

---

Templates are short snippets of code you can use wherever Home Assistant needs to figure something out for you. Instead of typing a fixed message or value, you write a small instruction that reads your data and produces the right result.

For example, instead of a notification that always says "Someone is home", a template can say "Frenck is home" or "Nobody is home, they're at the gym" depending on what's actually happening.

## Quick example

---

### action

```
action: |
 action: notify.mobile_app
 data:
 message: >
 {% if is_state('device_tracker.frenck', 'home') %}
 Frenck is home.
 {% else %}
 Frenck is at {{ states('device_tracker.frenck') }}.
 {% endif %}
 output: "Frenck is at gym."
```

The text between `{%, %}` and `{{, }}` is the template. When the action runs, Home Assistant replaces it with the right message.

## Learning guide

---

Start here if you are new to templating in Home Assistant. The pages below walk you through the concepts step by step.

### **`mdi:rocket-launch` Introduction**

What templates are and why you would use them.

### **`mdi:map-marker-radius` Where to use templates**

The places in Home Assistant where templates show up.

### **`mdi:code-braces` Template syntax**

The building blocks of every template, explained in plain language.

### **`mdi:sync` Loops and conditions**

Making decisions and repeating work with `if`, `for`, and friends.

### **`mdi:file-code-outline` Templates in YAML**

Quoting, multi-line strings, and other YAML gotchas.

### **`mdi:toggle-switch` Working with states**

Reading states, attributes, and entity information.

### **`mdi:swap-horizontal` Types and conversion**

Understand data types and how to convert between them.

### **`mdi:clock-outline` Dates and times**

Format, compare, and calculate with dates and times.



## **`mdi:language-python` Python methods**

Cheat sheet of Python methods available in templates.

## **`mdi:book-open-page-variant` Common patterns**

Recipes for the things you actually want to do.

## **`mdi:bug-outline` Debugging templates**

Fixing mistakes with the template editor.

## **`mdi:alert-circle-outline` Error messages**

What each error means and how to fix it.

## **`mdi:puzzle-outline` Custom templates and macros**

Share templates across your configuration.

## Tutorials

---

Two step-by-step walkthroughs that show templating in practice.

### ``mdi:battery-alert`` **Notify me about low batteries**

Build a daily notification that lists devices with low batteries.

### ``mdi:thermometer`` **Average home temperature**

Create a template sensor that averages all your temperature sensors.

## Reference

---

Home Assistant provides hundreds of template functions, filters, and tests for working with your data. Each one has its own page with explanations and examples.

- [Template functions reference](#). Browse all functions, filters, and tests by category.
-

## Docs/Tools/Check Config

---

Test any changes to your `configuration.yaml` file before launching Home Assistant. This script allows you to test changes without the need to restart Home Assistant.

```
hass --script check_config
```

The script has further options like checking configuration files which are not located in the default directory or showing your secrets for debugging.

```
$ hass --script check_config -h
usage: hass [-h] [--script {check_config}] [-c CONFIG] [-i [INFO]] [-f] [-s] [--

Check Home Assistant configuration.

optional arguments:
 -h, --help show this help message and exit
 --script {check_config}
 -c CONFIG, --config CONFIG
 Directory that contains the Home Assistant
 configuration
 -i [INFO], --info [INFO]
 Show a portion of the config
 -f, --files Show used configuration files
 -s, --secrets Show secret information
 --json Output JSON format
 --fail-on-warnings Exit non-zero if warnings are present
```

## Docs/Tools/Dev Tools

---

The dashboard contains a section called **Developer tools**.

Screenshot of Home Assistant's developer tools.

Section	Description
YAML	Lets you validate the configuration and trigger a reload or restart
States	Sets the representation of an entity
Actions	Performs actions from integrations
Template	Renders templates
Events	Fires events
Statistics	Shows a list of long-term statistic entities
Assist	Lets you see how Home Assistant Assist processes a sentence

## What can I do with Developer tools?

---

The Developer tools is meant for **all** (not just for the developers) to quickly try out things - like performing actions, updating states, raising events, and publishing messages in MQTT). It is also a necessary tool for those who write custom automations and scripts by hand. The following describes each of the sections in detail.

## YAML tab

---

The YAML tab provides buttons to trigger a check of configuration files and to reload the configuration. Reloading is needed to apply changes that you've made to the configuration.

It is almost the same as the option under **Settings** > three dots `mdi:dots-vertical` menu (top right) > **Restart Home Assistant** > **Quick reload**. The only difference is that **Quick reload** reloads all the configuration, whereas this YAML tab allows you to only reload one specific configuration at a time.

### Reloading the YAML configuration

For configuration changes to become effective, the configuration must be reloaded. Most integrations in Home Assistant (that do not interact with devices or services) can reload changes made to their configuration in `configuration.yaml` without needing to restart Home Assistant.

1. Go to **Developer tools** > **YAML** and scroll down to the YAML configuration reloading section (alternatively, hit "c" anywhere in the UI and search for "reload").
2. You are presented with a list of integrations, such as **Automations** or **Conversation**.  
Reload configuration changes
3. Depending on what you find in the list, you can proceed with either reloading or you need to restart Home Assistant:
4. If the integration is listed, select it to reload the settings.
  - For example, if you've changed the [General settings](#), you can select **Location & customizations** to apply those changes.
5. If the integration is not listed, you need to **Restart** Home Assistant for changes to take effect.

## States tab

---

This section shows all the available entities, their corresponding state and the attribute values. The state and the attribute information is what Home Assistant sees at run time. To update the entity with a new state, or a new attribute value, select the entity, scroll to the top, and modify the values, and select the **SET STATE** button.

Note that this is the state representation of a device within Home Assistant. That means, it is what Home Assistant sees, and it does not communicate with the actual device in any manner. The updated information can still be used to trigger events, and state changes. To communicate with the actual device, it is recommended to perform actions in the **Actions** section above, instead of updating state.

For example, changing the `light.bedroom` state from `off` to `on` does not turn on the light. If there is an automation that triggers on the `state` change of the `light.bedroom`, it will be triggered – even though the actual bulb has not turned on. Also, when the bulb state changes – the state information will be overridden (the refresh icon can be used to retrieve the latest information that Home Assistant has). In other words, the changes that are made through the **States** section are temporary, and are recommended to use for testing purposes only.

The table containing all entities can be filtered for each column. The used search is a wildcard search meaning that if you input "office" in the entity column filter, every entity whose ID matches "\*office\*" will be shown. You can also add your own wildcards in the search input (such as "office\*light"). The attribute filter supports separate filters for attribute names and values, separated by a colon ":". So the filter "location:3" will result in the table showing all entities that have an attribute name that contains "location" and whose attribute value contains "3".



## Actions tab

---

This section is used to perform actions that are available in Home Assistant.

The list of actions in the **Actions** dropdown are automatically populated based on the integrations that are found in the configuration, automation and script files. If a desired action does not exist, it means either the integration is not configured properly or not defined in the configuration, automation or script files.

When an action is selected, and if that action requires an `entity_id` to be passed, the **Entity** dropdown will automatically be populated with corresponding entities.

An action may also require additional input to be passed. It is commonly referred to as “action data”. The action data is accepted in YAML format, and it may be optional depending on the action.

When an entity is selected from the Entity dropdown, it automatically populates action data with the corresponding `entity_id`. The action data YAML can then be modified to pass additional [optional] parameters. The following is an illustration on how to perform a `light.turn_on` action.

To turn on a light bulb, use the following steps:

1. Select `light.turn_on` from the **Action** dropdown.
2. Select the entity (typically the light bulb) from the Entity dropdown (if no `entity_id` is selected, it turns on ALL lights)
3. If an entity is selected, the action data is populated with basic YAML that will be passed to the action. Additional data can also be passed by updating the YAML as below.

```
entity_id: light.bedroom
brightness: 255
rgb_color: [255, 0, 0]
```

## Template editor tab

---

The template editor provides a way to quickly test templates prior to placing them into automations and scripts. A code editor is on the left side and your real-time output is displayed in the preview on the right side.

By default, this will contain sample code that illustrates how templates can be written and tested. This sample code can be removed and replaced with your own. You can restore the default example by pressing the **Reset to Demo Template** button beneath the code editor.

For more information about Jinja2, visit [Jinja2 documentation](#), and also read templating document [here](#).

## Events tab

---

In the Events section, you can either fire an event on the event bus or subscribe to an event type in order to view the event data JSON.

### Fire an event

To fire an event, simply type the name of the event, and pass the event data in JSON format. For example, to fire a custom event, enter the `event_type` as `event_light_state_changed` and the event data JSON as

```
state: on
```

If there is an automation that handles that event, it will be automatically triggered. See below:

```
- alias: "Capture Event"
 triggers:
 - trigger: event
 event_type: event_light_state_changed
 actions:
 - action: notify.notify
 data:
 message: "Light is turned {{ trigger.event.data.state }}"
```

### Subscribe to an event

To subscribe to an event, enter the event type under **Listen to events** and select **Start listening**. Some events types are listed in the **Events** section under **Active listeners**. You can usually find information about event types for a particular integration in its documentation. You can then examine the event data JSON to find the correct parameters for your automations.

For example, subscribing to the event type `shelly.click` of the Shelly integration, returns event data JSON similar to the following on a button press.

Event 0 fired 9:53 AM:

```
{
 "event_type": "shelly.click",
 "data": {
 "device_id": "e09c64a22553484d804353ef97f6fcd6",
 "device": "shellybutton1-A4C12A45174",
 "channel": 1,
 "click_type": "single"
 },
 "origin": "LOCAL",
 "time_fired": "2021-04-28T08:53:12.755729+00:00",
 "context": {
 "id": "e0f379706563aaa0c2c1fda5174b5a0e",
 "parent_id": null,
 "user_id": null
 }
}
```

## Statistics tab

---

The **Statistics** tab shows a list of long-term statistic entities. If the long term statistics is not working for an entity, a **Fix issue** link is shown. Select it to view a description of the issue. There might also be an option to fix the issue.

Statistics issue message

Another use of the statistics developer tool is to correct any measurements. Select the  icon. Use date & time to search for the incorrect data point and adjust the value.

Screenshot showing adjusting the long-term statistic history value

## Assist tab

---

The **Assist** tab lets you see how Home Assistant's Assist processes a sentence.

If no matching intent is found, then Assist is unable to interpret the sentence. If a matching intent was found, information is provided on the action that will be performed on which entities. The example below shows how the following sentence was parsed: *what lights are on in the office*.

- Assist found a matching intent: *HassGetState*.
- It found entities matching the domain: *lights*.
- The lights have the state *on*.
- The lights are in the area *office*.
- The targets are the narrowed-down entities in scope.

Example use of assist developer tools

---

## Docs/Tools/Quick Search

---

The **Quick search** allows you to find entities and run commands without needing to navigate away from your current view.

It can be launched from anywhere in the frontend using [hotkeys](#). Use `ctrl` + `k` on Windows and `cmd` + `k` on macOS to open the Quick search.

Quick search dialog Use ``ctrl`` + ``k`` on Windows and ``cmd`` + ``k`` on macOS to open the Quick search for accessing entities and running commands

## Hotkeys

Type these from anywhere in the application to launch the dialog.

Mode	Hotkey	Description
Quick search	<code>ctrl + k</code> (Windows) / <code>cmd + k</code> (macOS)	Opens the Quick search dialog.
Entity filter	<code>e</code>	Opens the entity filter in the Quick search.
Command palette	<code>c</code>	Opens the command palette in the Quick search.
Device filter	<code>d</code>	Opens the device filter in the Quick search.
Create <code>my</code> link	<code>m</code>	Opens a new tab to create a my link to the page you are on.
Assist	<code>a</code>	Opens the Home Assistant Assist dialog.

 **Tip:** The application must have focus for the hotkey to register. If the dialog doesn't launch, try selecting an empty part of the main content area of Home Assistant and type it again.



## Entity filter

---

Hotkey:  e

Similar to **Settings** > **Devices & services** > **Entities**, but more lightweight and accessible from anywhere in the frontend.

Quick search entity filter mode Press E to filter for entities in the Quick search's entity filter mode

Once launched, start typing your entity ID (or "[bits and pieces](#)" of your entity ID) to get back a filtered list of entities. Selecting an entity (or hitting  enter when the desired entity is highlighted) will open the **More info** dialog for that entity.

This is helpful when, for example, you are in the middle of writing an automation and need some quick insight about an entity but don't want to navigate away to Developer tools.

## Device filter

---

Hotkey: **d**

Similar to **Settings > Devices & services > Devices**, but accessible from anywhere in the frontend.

Once launched, start typing your device name to get back a filtered list of your devices. Selecting a device (or hitting **enter** when the desired device is highlighted) will open the selected device detail page.

This is helpful when you need to quickly access a device's detail page without navigating your way through the menu.

Press D to search for devices Press D to start a quick search for devices

## Command palette

---

Hotkey: 

Run various commands from anywhere without having to go to another view.

Quick search command mode Run commands in the **Quick search**'s "command palette"

### Currently-supported commands

- **Navigate:** All entries in the sidebar and settings
- **Reload:** All currently-supported "Reload {domain}" actions (for example, "Reload Scripts")
- **Server:** Restart/Stop

## My links

---

Hotkey: m

Create my links from any supported page in the user interface. When invoked on a supported page, it will open a new tab that will allow you to share the link in different formats.

## Assist

---

*Hotkey:* 

Opens the Assist dialog to interact with Home Assistant using your voice or by text. This feature is only available if you have set up a voice assistant.

Learn more about [voice assistants](#).

## Disabling shortcuts

---

You can enable or disable all of Home Assistant's keyboard shortcuts by going to your User Profile and selecting the **Keyboard shortcuts** toggle button.

Toggle for enabling or disabling keyboard shortcuts Toggle button for enabling/disabling keyboard shortcuts added by Home Assistant.

## Tips

---

### Search by "bits and pieces" rather than an exact substring

We know something like **"light.ch"** should match **"light.chandelier"**. Similarly, **"telev"** should match **"media\_player.television"**.

But with **Quick search**, **"lich"** would also match **"light.chandelier"**, and **"plyrtv"** would also match **"media\_player.television"**. It checks letter *sequences* rather than exact substrings.

One nice use case for this is that you can quickly filter out an entire domain of entities with just a couple letters and a period. For example, **"li."** will match any **"light.\*"** entities. Continuing with **"li.ch"** would bring up the chandelier right away.

### Filters work against friendly name too

If **"light.hue\_ceiling\_light"** has been named **"Chandelier"**, you can type either **"hue\_ceil"** or **"chand"** to find it.

### Use the enter key any time to open the top result in the list

As soon as the item you wanted shows up at the top of your filtered results, just hit **"enter"** to activate it. No need to arrow down to the item, or select with your mouse.

### Use arrow keys to move around the list

When in the text field, use the down arrow  to navigate down the item list. Hit  to activate the currently highlighted row.

When in the item list, use the up arrow  to navigate up the item list, and to get back into the text field.

### Typing more letters will always add to your filter string

Say you've just used arrow keys to navigate halfway down the list, and want to add more text to your filter. You don't need to select back into the text field, just start typing new letters and they'll append to your filter.

## Troubleshooting

---

### Dialog doesn't launch using hotkeys

There are a few possible reasons why the **Quick search** dialog won't launch:

1. Your user is not an admin.
2. The application lost focus. Try selecting the main content area of the application and typing the shortcut again.
3. You have disabled keyboard shortcuts in your User Profile settings.
4. Shortcut is marked by browser as non-overridable. Firefox does this with some shortcuts, for example. But this shouldn't be a problem with single-key shortcuts currently used by the **Quick search**.
5. Some other application or browser extension is using or overriding the shortcut. Try disabling the extension.

### A command is missing

The command list only shows commands that are available to you based on your user settings, and loaded integrations.

For example, if you don't have `automations:` in your config, then you won't see the **Reload Automations** command.

If **Advanced Mode** is turned off in User Settings, then any command related to advanced mode will not appear in the list.

If a command is missing that you feel is in error, please create an issue on GitHub.

### Shortcuts interfere with accessibility tools, browser extensions, or are otherwise annoying

You can [disable shortcuts](#) in your User settings.

Please consider submitting an issue explaining why the shortcut was disruptive to you. Keyboard shortcuts are new to Home Assistant, and getting them right is a challenge for any Web application. We rely on user feedback to ensure the experience is minimally disruptive.

---



## Docs/Tools

---

Home Assistant ships a couple of helpers for the command-line and the frontend which simplify common tasks, are helping with migrations, and ensure that Home Assistant runs properly.

---

## Docs/Troubleshooting General

---

This page provides some information about more generic troubleshooting topics.

# Home Assistant went into recovery mode

---

## Symptom: Home Assistant is in recovery mode

On top of the page you see a red banner. On the **Overview** page, you see a **Recovery mode** notification.

image

## Description

When Home Assistant is in recovery mode, there was an issue with the configuration.

Recovery mode loads a minimum set of integrations to allow troubleshooting the configuration. Recovery mode will use the parts of the configuration that was used the last time Home Assistant started successfully. You can still see the user interface, the settings, and apps.

## Resolution

You need to identify the issue in the configuration files and fix it there. The issue could be caused by something as simple as an invalid YAML file.

- If you are running Home Assistant Operating System, you can install an app such as Studio Code Server to edit the configuration file if needed.
- If you are still logged in, you can [edit your configuration](#).
- In the Home Assistant user interface, open the app you usually use and edit the configuration file.
- Restart Home Assistant.
- If you are locked out because you forgot your password, you cannot edit the configuration file from the user interface. Follow the steps to [reset your password](#).

## Restarting Home Assistant in safe mode

---

If your Home Assistant is acting up and you cannot identify a root cause, you can use **Safe mode** to narrow down the number of possible causes. **Safe mode** loads Home Assistant Core, but no custom integrations, no custom cards, and no custom themes. If the issue does not persist in **Safe mode**, the issue is not with Home Assistant Core. Before reporting an issue, check if the issue persists in **Safe mode**.

You can enable Safe mode in several ways:

- From the UI:
- Go to **Settings** > **System** > **Restart Home Assistant** (top right) > **Restart Home Assistant in safe mode**.
- From the [command line](#):
- Run: `bash ha core restart --safe-mode`
- By creating a file in the configuration directory:
- Create an empty file named `safe-mode` in your Home Assistant configuration directory. Home Assistant will detect this file on startup and automatically boot into Safe mode.

## I don't see any updates

---

Typically, updates are shown at the top of the **Settings** page. If you don't see them there, the **Visibility** option might be disabled.

### Resolution

1. On the **System** page, in the top-right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
  2. Go to **System > Updates**.
    - Select the update notification.
    - Select the cogwheel `mdi:cog-outline` , then set **Visible** to active.
-

## Docs/Z Wave/Controllers

---

To use Z-Wave with Home Assistant, you need a compatible Z-Wave adapter.

## Recommended Z-Wave adapter

---

The [Home Assistant Connect ZWA-2](#) is an 800 series Z-Wave adapter specifically developed to work with Home Assistant.

## Other supported Z-Wave adapters

---

This section lists devices that have been confirmed to work with Z-Wave JS.

A few recommendations if you are new to Z-Wave:

- Use an [800 series adapter](#) (with firmware updated to  $\geq 7.23.2$ ).
- The 800 series adapters are the most future-proof and offer the best RF performance.
- Opt for a USB connection, not a module.
- Passing a module through Docker is more complicated than passing a USB connector through.

### 800 series USB adapters

Before connecting the Z-Wave 800 series adapter to Home Assistant, make sure the adapter uses a compatible firmware and SDK version. Some 800 series Z-Wave adapters have bugs which impact the stability of the mesh and can cause the adapter to become unresponsive.

Upgrade the firmware of the 800 series adapter to a recommended version.

- Because there is no known firmware version that is completely fixed, it is recommended to choose a firmware based on the following criteria:
- prefer SDK versions 7.23.x and newer
- SDK versions 7.22.x are okay
- SDK versions 7.17.2 to 7.19.x are okay
- avoid SDK versions before 7.17.2
- avoid SDK versions 7.20 to 7.21.3
- **Note:** The SDK version does not have to match the firmware version.
- If you are unsure which SDK versions a firmware is based on, contact the manufacturer of your device.

#### List of supported 800 series adapters

The following 800 series USB adapters have been reported to work with Home Assistant if using the SDK and firmware versions mentioned above.

- [Home Assistant Connect ZWA-2](#) (officially recommended adapter)
- HomeSeer SmartStick G8
- Zooz 800 Series Z-Wave Long Range S2 Stick (ZST39 LR)

### 700 series USB adapters

In general, using a 700 series USB adapter is not recommended.

Before connecting the Z-Wave 700 series adapter to Home Assistant, make sure the adapter uses a compatible firmware and SDK version. Some 700 series Z-Wave adapters have bugs which impact the stability of the mesh and can cause the adapter to become unresponsive.



Upgrade the firmware of the 700 series adapter to a recommended version:

- Because there is no known firmware version that is completely fixed, it is recommended to choose a firmware based on the following criteria:
- prefer SDK versions 7.17.2 to 7.18.x or 7.21.6 and newer
- SDK versions 7.19.x are okay
- avoid SDK versions before 7.17.2
- avoid SDK versions 7.20 to 7.21.5
- **Note:** The SDK version does not have to match the firmware version.
- If you are unsure which SDK versions a firmware is based on, contact the manufacturer of your device.
- To upgrade the firmware, search for the instructions that match your system.
- For Linux, the [Upgrade instructions from kpin](#) can be helpful.

### List of supported 700 series USB adapters

The following 700 series USB adapters have been reported to work with Home Assistant if using the SDK and firmware versions mentioned above:

- Aeotec Z-Stick 7 USB stick (ZWA010) (the EU version is not recommended due to RF performance issues)
- HomeSeer SmartStick+ G3
- HomeSeer Z-NET G3
- Silicon Labs UZB-7 USB Stick (Silabs SLUSB7000A / SLUSB001A)
- Zooz S2 Stick 700 (ZST10 700)
- Z-Wave.Me Z-Station

### 500 series USB adapters

The following 500 series USB adapters have been reported to work with Home Assistant:

- Aeotec Z-Stick Gen5 (see note below)
- Everspring USB stick - Gen 5
- GoControl HUSBZB-1 stick
- Sigma Designs UZB stick
- Vision USB stick - Gen5
- Z-Wave.Me UZB1 stick (see Aeotec Z-Stick note below)
- HomeSeer SmartStick+ G2
- HomeSeer Z-NET G2

### Raspberry Pi HAT adapters

- Aeotec Z-Pi 7 Raspberry Pi HAT/Shield (ZWA025, 700 series)
- Z-Wave.Me RaZberry 7 (ZME\_RAZBERRY7, 700 series)
- Z-Wave.Me RaZberry 7 Pro (ZMEERAZBERRY7\_PRO or ZMEURAZBERRY7\_PRO, 700 series)

- Z-Wave.Me Razberry 2 (500 series)
- Z-Wave.Me Razberry 1 (300 series)

## Third-party hubs

---

For the best experience, it is recommended to use an adapter directly with Home Assistant. If this doesn't work for you, you can use a hub that supports Z-Wave. Home Assistant supports the following third-party hubs with Z-Wave support:

- [Vera](#)
- [Fibaro](#)
- [SmartThings](#)
- [Z-Wave.Me Z-Way](#)

## Adapter notes

---

### Aeotec Z-Stick

 **Note:** The Aeotec Z-Stick and some of its variants (such as the Z-Wave.Me UZB1) are known to have compatibility issues with the Linux kernel because of their [non-compliant behavior](#). Plugging these adapters through a USB hub can serve as a workaround that sometimes mitigates the issue.

It's totally normal for your Z-Wave stick to cycle through its LEDs (Yellow, Blue and Red) while plugged into your system.

---

## Includes/Actions/More Examples

---

## More examples

---

Real scenarios where this action shows up in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

---

## Includes/Actions/Related

---

## Related actions

---

These actions work well alongside this one:

{% endif %}

---



## Includes/Actions/Stuck

---

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the action you're calling and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain actions or suggest the right one when you describe what you want in plain language.

---

## Includes/Actions/Targets

---

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the action

---

This action requires a target. The target is the object of the action. You can point the action at a single entity, a device, an area, a floor, or a label, and Home Assistant will run the action on every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one action. For example, you can add a specific entity and an area as targets in the same action to run the action on both of them at once.

---

## Includes/Actions/Try It

---

## Try it yourself

---

Ready to test this? Open **Developer tools > Actions**, search for this action, fill in the fields, and select **Perform action**. You see what happens on your actual entities without writing a line of YAML.

---



## Using this action from the user interface

---

If you prefer building automations and scripts visually, Home Assistant walks you through this action step by step. You pick what to target, tweak a few options, and save. No YAML knowledge required.

---



## Includes/Actions/Yaml Header

---

## Using this action in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

---

## Includes/Common Tasks/Backups

---

# Backups

---

It is important to regularly back up your Home Assistant setup. You may have spent many hours configuring your system and creating automations. Keep your configurations safe so that you can [restore](#) a system or migrate your Home Assistant to new hardware.

Backups are encrypted and stored in a compressed archive file (.tar) and by default, stored locally in the `/backup` directory.

A full backup includes the following directories:

- `config`
- `share`
- `addons` (only manually installed or created apps, not those installed from the store)
- `ssl`
- `media`

A partial backup consists of any number of the above default directories and installed apps.

## Preparing for a backup

Before creating a backup, check if you can reduce the size of the backup. This is especially important if you want to use the backup to migrate the system to new hardware, for example from a Raspberry Pi Compute Module 4 to a Raspberry Pi Compute Module 5.

1. Check if your configuration directory contains a large database file:
2. Go to **Settings > System > Repairs**.
3. From the three dots `mdi:dots-vertical` menu, select **System information** and under the **Recorder** section, look for the **Estimated Database Size (MiB)**.
4. By default, the data is kept for 10 days. If you have modified that to a longer period, check the `recorder` integration page for options to keep your database data down to a size that won't cause issues.
5. Note the keep days, purge interval, and include/exclude options.
6. To check how much space you've used in total, go to **Settings > System > Repairs**.
7. From the three dots `mdi:dots-vertical` menu, select **System information**, and check under **Home Assistant Supervisor > Disk used**.
8. If you have apps installed that you no longer use, uninstall those apps. Some apps require quite a bit of space.
9. If you want to store the backup on your network storage instead of just locally on your system, follow the steps on [adding a new network storage](#) and select the **Backup** option.

## Setting up an automatic backup process

The automatic backup process creates a backup on a predefined schedule and also deletes old, redundant backups.

1. Go to **Settings > System > Backups**.

2. Under **Set up backups**, select **Set up backups**.
3. Download the emergency kit and store it somewhere safe.
4. You need it to restore encrypted backups.
5. To learn more about backup encryption, refer to the documentation on the [backup emergency kit](#).
6. Define the backup schedule.
7. It is recommended to back up **Daily**, but you can also choose to back up on specific days.
8. Define the time:
  - **System optimal** sets a time in a predefined time window as shown in the UI.
  - **Custom** lets you pick the time when you want the backup to start.
  - Make sure you pick a time when all your backup locations are up and running and available. Otherwise, the backup will fail for locations which are not available.
9. Define if you want to back up automatically before updating.
10. This sets a default. But you can change this setting each time before updating.
11. For large installations, backups might take a while.
12. Your update might start later than you expected.
13. Define how many backups you want to keep.
14. Older backups will be automatically deleted.
15. For example: if you back up daily, and select 7 backups, then the backup from 8 days ago and older will be deleted.
16. Define the data you want to back up.
17. It is recommended to disable media and the shared folder to reduce the size of the backup.
18. A large backup also takes longer to restore.
19. Some apps may also be quite large.
20. [Define the location for backups](#).

## Defining backup locations

You might need a backup in case your system has crashed. If you only store backups on the device itself, you won't be able to access them easily. It is recommended to keep a copy on another system (outside of Home Assistant) and ideally also one off-site.

 **Note:** You will find an overview of integrations which provide a backup location [here](#).

### About the backup storage on Home Assistant Cloud

If you have Home Assistant Cloud, you can store a backup of maximum 5 GB on Home Assistant Cloud. This cloud storage space is available for all existing and new Home Assistant Cloud subscribers without additional cost. It stores one backup file: the backup that was last saved to Home Assistant Cloud. These backups are always encrypted. To restore encrypted backups, you need the encryption key stored in the [backup emergency kit](#).

## To define the backup location for automatic backups

1. Go to **Settings > System > Backups** and under **Automatic backups**, select **Configure automatic backups**.
2. Under **Locations**, use the toggle to enable all the backup locations you want to use.
3. If you don't see Home Assistant Cloud in the list, you are not [logged in](#).
4. If you want to back up to your NAS (such as [Synology](#)) or a cloud provider (such as [Google Drive](#) or [Microsoft OneDrive](#)), check their integration documentation for specific instructions on setting up a Home Assistant backup.
5. If you don't see a network storage, you haven't added one. Follow the steps on [adding a new network storage](#) and select the **Backup** option. Define the backup locations
6. For each enabled location, select the cog `mdi:cog-outline` to enable/disable encryption.
7. **Info:** The backup stored on Home Assistant Cloud is always encrypted.

## Creating a backup automation using the backup action

If the backup automation settings provided in the UI do not match your use case, you can manually configure your own backup automation using the `backup.create_automatic` action.

Using the `developer_call_service` action in your own automation allows you to create automated backups on any schedule you like, or even add conditions and actions around it. For example, you could make an automation that triggers on a calendar, turns on your NAS, waits until it is online, and then triggers a backup.

## Creating a manual backup

This creates a backup instantly. You can create a manual backup at any time, irrespective of any automatic backups you may have defined.

1. Go to **Settings > System > Backups**.
2. In the lower-right corner, select **Backup now** and select **Manual backup**.
3. Define the data you want to back up.
4. It is recommended to disable media and the share folder to reduce the size of the backup.
5. A large backup also takes longer to restore.
6. Some apps may also be quite large.
7. Provide a name for the backup.
8. Choose the backup locations.
9. To learn more about the locations, refer to the section on [defining the backup location](#).
10. Download the [backup emergency kit](#) and store it somewhere safe. Make sure you take note of the backup name it belongs to.
11. To start the backup process, select **Create backup**.

## Downloading your local backups

When downloading the backup from the Home Assistant backup page, it is decrypted on the fly so that you can view the data using your favorite archive tool. This is done for all backup locations and also when you download from Home Assistant Cloud.

There are multiple ways to download your local backup from your Home Assistant instance and store it on another device:

**Option 1:** Download from the backup page:

1. Under **Settings** > **System** > **Backups**, select **Show all backups**.
2. To select multiple backups, select the `mdi:order-checkbox-ascending` button.
3. Select the three dots `mdi:dots-vertical` menu and select **Download backup**.
4. Result: The selected backup is stored in the **Downloads** folder of your computer.
5. If a backup is stored on multiple locations, you can select where you download it from:
6. Select the backup, and under **Locations**, select the three dots `mdi:dots-vertical` and select **Download from this location**.

**Option 2:** Copy backups from the backups folder:

1. If you haven't already done so, [configure access to files on Home Assistant](#), using one of the methods listed there.
2. For example, [use the samba app](#).
3. In your file explorer, access Home Assistant, open the `backup` folder and copy the file to your computer.

## Downloading a backup from Home Assistant Cloud

If you were logged in to Home Assistant Cloud and had Cloud backup enabled when creating a backup, your last backup is stored on Home Assistant Cloud.

There are two ways to download the backup from Home Assistant Cloud:

- **Option 1:** From the backups page
  - Go to **Settings** > **System** > **Backups** and select **Show all backups**.
  - Select the backup from the list.
  - Under **Locations**, select the three dots `mdi:dots-vertical` and select **Download from this location**.
- **Option 2:** From your Home Assistant Cloud account
  - Log in to your [Home Assistant Cloud account](#).
  - Under **Stored files**, you can see the latest available backup file. Select the **Download** button.

## Deleting obsolete backups

If you defined an automatic backup and backup purge schedule, old backups are deleted automatically. However, you might still have some old backups around that you want to delete.

To delete old backups, follow these steps:

1. In Home Assistant, under **Settings** > **System** > **Backups**, select **Show all backups**.
2. To **delete one backup**: from the list, select the backup of interest.
3. Select the three dots `mdi:dots-vertical` menu and select **Delete**.
4. To **delete multiple backups**: select the `mdi:order-checkbox-ascending` button.
5. From the list of backups, select all the ones you want to delete and select **Delete selected**.
6. `mdi:information-outline` Consider keeping at least one recent backup for recovery purposes.
7. To **delete a backup that is stored on Home Assistant Cloud**, you have 2 options:
8. **Option 1**: Trigger backup deletion from within Home Assistant
  - Follow steps 1 and 2 from above.
  - Even though you select **Delete** in Home Assistant, it will be deleted from Home Assistant Cloud storage.
9. **Option 2**: Delete the backup from the Nabu Casa account page.
  - Log in to your [Nabu Casa account](#).
  - Under **Backups**, delete the backup.

## Restoring a backup

There are two ways to use a backup:

- On your current system to recover your settings.
- During onboarding, to migrate your setup to a new device or to a device on which you performed a factory reset.

### Estimated duration

The time it takes to restore a backup depends on your installation. Home Assistant Core and all apps are being reinstalled. For a larger installation, this process can take about 45 minutes.

### Restoring a backup during onboarding

You can use a backup during the onboarding process to restore your configuration.

**Migration:** This procedure also works if you want to migrate from one device to another. In that case, use the backup of the old device on the new device. The target device can be a different device type. For example, you can migrate from a Raspberry Pi to another device.

### Prerequisites

- This procedure assumes you have already completed the [installation](#) procedure on your target device and are now viewing the welcome screen as part of the [onboarding](#).
- The login credentials of the device from which you made the backup.
- The [backup emergency kit](#) that contains the key needed to decrypt the backup.
- **Required storage capacity:** If you migrate the installation to a new device, make sure the new device has more storage capacity than the existing device.



- Before migrating, on the old system, check how much storage you used.
- Go to **Settings > System > Repairs > ... > System Information**, and under **Home Assistant Supervisor**, look at the **Disk used** value.
  - The target device must have more free space than the source device.
  - If your target device is a Home Assistant Yellow, note that it is the size of the eMMC that is relevant.
  - The restore process mainly uses the eMMC, not the NVMe.
  - The size of the backup file is no indication of the size of your installation. To know the size of your installation, you need to check the **Disk used** value mentioned above.
- If you are migrating to a new device:
- You do not need to transfer the backup to a USB or SD card to bring it to your device.
- You will be able to upload the backup file from the device you are accessing the onboarding from.

#### To restore a backup during onboarding

1. If you are migrating to a new device and you had controllers or radios connected (such as a Z-Wave stick or a Connect ZBT-2):
2. make sure to plug them into the new device.
3. You can either restore a backup from your local machine or a backup stored on Home Assistant Cloud:
4. **Option 1:** restoring from a local backup.
  - On the welcome screen, select **Upload backup**.
  - Select **Select backup file**.
  - The file explorer opens on the device on which you are viewing the Home Assistant User interface.
  - You can access any connected network drive from there.
  - Select the backup file.
5. **Option 2:** restoring from a Home Assistant Cloud backup.
  - On the welcome screen, select **Home Assistant Cloud**.
  - Sign in to Home Assistant Cloud.
6. In the dialog, select all the parts you want to restore.
  - Your current system will be overwritten with the parts that you choose to restore.
  - If you want to restore the complete configuration with all directories and apps, select everything.
7. Enter the encryption key stored in the [backup emergency kit](#).
8. To start the process, select **Restore backup**.
9. The restore may take a while, depending on the amount of data.
10. Don't refresh the page. Just wait.
  - If you refresh the page, you will see a "Not found" message. This is because the system is shutdown, wiped, and reinstalled from the backup. During that time, it won't be reachable.

11. If your previous installation had certificates enabled directly for the `http` integration, when the restore is complete, it will no longer respond to `http://` requests. In this case, use `https://` (added `s`) instead.
12. On the login screen, enter the credentials of the system from which you took the backup.
13. The login password and username must match the ones you used at the time the backup was taken.
14. Your dashboard should show all the elements as they were when you created the backup.
15. If some devices are shown as unavailable, you may need to wake the battery-powered devices.
16. If you had `network storage` connected on the previous system, you may need to reconnect those.
17. If you had Zigbee devices, and you migrated to a new device with its own Zigbee radio built-in:
18. Because this is now a different Zigbee radio, you need to [migrate Zigbee](#).

### To restore a backup on your current system

1. Go to **Settings > System > Backups**.
  2. From the list of backups, select the backup from which you want to restore.
  3. Select what to restore:
  4. Your current system will be overwritten with the parts that you choose to restore.
  5. If you want to restore the complete configuration with all directories and apps, select everything.
  6. If you only want to restore specific elements, only select the folders and apps you want to restore.
  7. Select **Restore**.
  8. This may take a while, depending on how much there is to compress or decompress.
  9. Once the restore is complete, Home Assistant restarts to apply the new settings.
  10. You will lose the connection to the UI and it will return once the restart is completed.
  11. On the login screen, enter the password and username as they were at the time the backup was taken.
-

## Includes/Common Tasks/Beta Version

---

### Running a beta version

If you would like to test next release before anyone else, you can install the beta version.

{% tabbed\_block %}

• title: From the UI content: |

1. In Home Assistant, go to **System > Updates**.
2. In the top-right corner, select the three dots `mdi:dots-vertical` menu.
3. Select **Join beta**.
4. Go to the **Configuration** panel.
5. Install the update that is presented to you.
6. Troubleshooting: If you do not see that notification:
  - In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
  - Go to **System > Updates**.
  - Select the update notification.
  - Select the cogwheel `mdi:cog-outline`, then set **Visible** to active.

• title: From the CLI content: |

1. Join the beta channel.

```
bash ha supervisor options --channel beta
```

2. Reload Home Assistant Supervisor.

```
bash ha supervisor reload
```

3. Update Home Assistant Core to the latest beta version.

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

{% endtabbed\_block %}

```
docker pull {{ site.installation.container }}:beta
```

**You then need to recreate the container with the new image.**

---

## Includes/Common Tasks/Commandline

---

## Home Assistant via the command line

---

On the [SSH command line](#), you can use the `ha` command to retrieve logs, check the details of connected hardware, and more.

### Home Assistant

```
ha core check
ha core info
ha core logs
ha core options
ha core rebuild
ha core restart
ha core restart --safe-mode
ha core start
ha core stats
ha core stop
ha core update
```

### Supervisor

```
ha supervisor info
ha supervisor logs
ha supervisor reload
ha supervisor update
```

### Host

```
ha host reboot
ha host shutdown
ha host update
```

### Hardware

```
ha hardware info
ha hardware audio
```

### Usage examples

To update Home Assistant to a specific version, use the command:

```
ha core update --version x.y.z
```

Replace x.y.z with the desired version like `--version`

You can get a better description of the CLI capabilities by typing `ha help`:

```
The Home Assistant CLI is a small and simple command line utility that allows
you to control and configure different aspects of Home Assistant
```

```
Usage:
 ha [command]
```

Available Commands:

addons	Install, update, remove and configure Home Assistant apps
audio	Audio device handling.
authentication	Authentication for Home Assistant users.
backups	Create, restore and remove backups
banner	Prints the CLI Home Assistant banner along with some useful info
cli	Get information, update or configure the Home Assistant cli backend
core	Provides control of the Home Assistant Core
dns	Get information, update or configure the Home Assistant DNS server
docker	Docker backend specific for info and OCI configuration
hardware	Provides hardware information about your system
help	Help about any command
host	Control the host/system that Home Assistant is running on
info	Provides a general Home Assistant information overview
jobs	Get information and manage running jobs
multicast	Get information, update or configure the Home Assistant Multicast
network	Network specific for updating, info and configuration imports
observer	Get information, update or configure the Home Assistant observer
os	Operating System specific for updating, info and configuration
resolution	Resolution center of Supervisor, show issues and suggest solutions
supervisor	Monitor, control and configure the Home Assistant Supervisor

Flags:

<code>--api-token</code>	string	Home Assistant Supervisor API token
<code>--config</code>	string	Optional config file (default is \$HOME/.homeassistant)
<code>--endpoint</code>	string	Endpoint for Home Assistant Supervisor (default is 'supervisor')
<code>-h, --help</code>		help for ha
<code>--log-level</code>	string	Log level (defaults to Warn)
<code>--no-progress</code>		Disable the progress spinner
<code>--raw-json</code>		Output raw JSON from the API

Use "ha [command] --help" for more information about a command.

## Console access

You can also access the Home Assistant Operating System via a directly connected keyboard and monitor, the console.

### Wiping the data disk from the command line

In Home Assistant Operating System, the `ha os datadisk wipe` command wipes the data disk. The command deletes all user data as well as Home Assistant Core, Supervisor, and any installed apps.

The command `ha os datadisk wipe` marks the data partition (either internal on the eMMC or the SD card, or on an external attached data disk) as to be cleared on the next reboot. The command automatically reboots the system. Upon reboot, the data is cleared. Then the system continues to boot and reinstalls the latest version of all Home Assistant components.

The `ha os datadisk wipe` command can only be run from the local terminal. Connect a display and keyboard and use the terminal.

Note, some systems have a reset button you can use to clear the data disk, instead of using the command line:

- If you have a Home Assistant Yellow with a Raspberry Pi Compute Module 5, use the command line steps described above.
- If you have a Home Assistant Yellow with a Raspberry Pi Compute Module 4, there is a red hardware button to wipe the data disk. Follow the procedure on [resetting the Home Assistant Yellow](#).
- If you have a Home Assistant Green, there is a black hardware button to wipe the data disk. Follow the procedure on [resetting the Home Assistant Green](#).

### Listing all users from the command line

In Home Assistant Operating System, the `ha auth list` command lists all users that are registered on your Home Assistant.

The `ha auth list` command can only be run from the local terminal. Connect a display and keyboard and use the terminal.

---

## Includes/Common Tasks/Configuration Check

---



## Configuration check

---

After changing configuration or automation files, check if the configuration is valid before restarting Home Assistant Core.

### Running a configuration check from the UI

1. Go to your user profile and enable **Advanced Mode**.
2. Go to **Settings > Developer tools > YAML** and in the **Configuration validation** section, select the **Check configuration** button.
3. This is to make sure there are no syntax errors before restarting Home Assistant.
4. It checks for valid YAML and valid config structures.
5. If you need to do a more comprehensive configuration check, [run the check from the CLI](#).

### Running a configuration check from the CLI

Use the following command to check if the configuration is valid. The command line configuration check validates the YAML files and checks for valid config structures, as well as some other elements.

```
ha core check
```

After changing configuration files, check if the configuration is valid before restarting Home Assistant Core.

*If your container name is something other than `homeassistant`, change that part in the examples below.*

Run the full check:

```
docker exec homeassistant python -m homeassistant --script check_config --config
```

Listing all loaded files:

```
docker exec homeassistant python -m homeassistant --script check_config --files
```

Viewing an integration's configuration ( `light` in this example):

```
docker exec homeassistant python -m homeassistant --script check_config --info light
```

Or all integrations' configuration

```
docker exec homeassistant python -m homeassistant --script check_config --info all
```

You can get help from the command line using:

```
docker exec homeassistant python -m homeassistant --script check_config --help
```

---

## Includes/Common Tasks/Data Disk

---

## Using external data disk

---

Home Assistant Operating System supports storing data on a secondary storage medium. For example, this can be a second internal SSD or HDD, or a USB-attached SSD or HDD. This data disk contains not only user data but also most of the Home Assistant software, including Home Assistant Core and Apps. This means a fast data disk will make the system much faster overall.

Graphics showing the architecture of the data disk feature

The data disk feature can be used on an existing installation without losing data: The system will move existing data to the external data disk automatically. However, it is recommended to create and download a full [Backup](#) before proceeding!

◆ **Caution:** All data on the target disk will be overwritten!

⚠ **Important:** The storage capacity of the external data disk must be larger than the storage capacity of the existing (boot) disk.

⚠ **Important:** If you have been using a data disk previously with Home Assistant Operating System, you need to use your host computer to delete all partitions *before* using it as a data disk again.

## Using UI to move the data partition

1. Connect the data disk to your system.
2. Go to **Settings > System > Storage** in the UI.
3. Select **Move data disk**.
4. Select the data disk from the list of available devices.
5. Select **Move**.
6. Depending on the amount of data, this may take a while.

## Using CLI to move the data partition

To see the current data disk use:

```
$ ha os info
...
data_disk: /dev/mmcblk1p4
...
```

To get a list of potential targets which can be used by `datadisk` :

```
ha os datadisk list
```

To initiate the move to the new data disk use the `move` command:

```
ha os datadisk move /dev/sdx
```

The system will prepare the data disk and immediately reboot. The reboot will take 10 minutes or more depending on the speed of the new data disk; please be patient!

 **Warning:** Using an USB attached SSD can draw quite some power. For instance on Raspberry Pi 3 the official Raspberry Pi power supply (PSU) only provides 2.5A which can be too tight. Use a more powerful power supply if you experience issues. Alternatively use a powered USB hub. Connect the Hub to one of the USB slots of your Raspberry Pi, and connect the SSD to the Hub. In this setup the power supply of the Hub will power the attached device(s).

## Migrating an external data disk to another system

This section shows how to move an external data disk from one system to another. This can be an option if the following elements apply to your use case:

- You already have a functioning Home Assistant instance (system 1) that is using an external data disk.
- You have another, new, Home Assistant instance (system 2).
- You now want to use the data disk of system 1 on system 2 instead.

The aim is to migrate the data from system 1 to system 2. One way to do this is by [restoring a backup](#). The other way is to move the data disk. This can be an interesting option if you have a large amount of data on your external disk or if your external disk has more storage capacity than your new system.

### Prerequisites

- A Home Assistant instance using an external data disk (system 1)
- A Home Assistant instance to which you want to move the external data disk (system 2)

### To migrate an external data disk to another system

To migrate an external data disk from one system to another, follow these steps:

1. [Create a backup](#) of both systems and store these backups on another system (not strictly necessary, but recommended just in case, at least for the important data).
2. Shut down system 1 and remove the data disk.
3. Make sure system 2 has Home Assistant OS installed, and Home Assistant is up and running. Home Assistant is using the data disk (partition) on the boot drive, such as the SD card, at this point.
4. Make sure system 2 has completed the basic [onboarding](#) steps, including the last steps where devices are discovered automatically.
5. Plug the external disk into system 2 and go to the **Settings > System**. Select the three dots `mdi:dots-vertical` menu, and **Restart Home Assistant > Reboot system**. Result: A repair issue is displayed **Multiple data disks detected**.

6. The repair issue comes up because system 2 now sees two file systems with an identical name. During a reboot, there is a name conflict with the existing data disk as it is undefined which file system should be used. This can lead to a random selection of the system you end up with. Hence you must make a decision.
  7. Open the repair issue and choose one of the options:
  8. Select **Use the detected data disk instead of the current system**.
    - This will override the current file system of system 2 and use your external data disk instead.
    - You won't have access anymore to the current Home Assistant data. It will be marked as inactive data disk.
  9. If you changed your mind about using the external data disk:
    - Unplug the external data disk.
    - If you cannot unplug the external data disk for some reason, select **Mark as inactive data disk (rename file system)**.
    - This makes sure that there is no name conflict after rebooting.
    - It also means you cannot use the file system that is stored on your external disk.
    - You keep using the current file system of system 1.
-

## Includes/Common Tasks/Define Custom Polling

---

If you want to define a specific interval at which your device is being polled for data, you can disable the default polling interval and create your own polling automation.

To add the automation:

1. Go to **Settings > Devices & services**, and select your integration.
  2. On the integration entry, select the `mdi:dots-vertical`.
  3. Then, select **System options** and toggle the button to disable polling. Disable polling for updates
  4. To define your custom polling interval, create an automation.
  5. Go to **Settings > Automations & scenes** and create a new automation.
  6. Define any trigger and condition you like.
  7. Select **Add action**, then select **Other actions**.
  8. Select **Perform action**, and from the list, select the `homeassistant.update_entity` action.
  9. Add the entities you want to poll to the **Entity** field. The `homeassistant.update_entity` action only supports targeting by entity. Selecting an area, device, or label is not supported.  
Update entity
  10. Save your new automation to poll for data.
-

## Includes/Common Tasks/Development Version

---

### Running a development version

If you want to stay on the bleeding-edge Home Assistant Core development branch, you can upgrade to `dev`.

◆ **Caution:** The `dev` branch is likely to be unstable. Potential consequences include loss of data and instance corruption.

1. Join the dev channel.

```
bash ha supervisor options --channel dev
```

2. Reload the Home Assistant Supervisor.

```
bash ha supervisor reload
```

3. Update Home Assistant Core to the latest dev version.

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

```
docker pull {{ site.installation.container }}:dev
```

You then need to recreate the container with the new image.

---



## Includes/Common Tasks/Enable Entities

---

## Enabling or disabling entities

---

Some entities are disabled by default. Whether a particular entity of a device is disabled or enabled by default, depends on the integration. Diagnostic entities for example are often disabled by default so that they don't clutter Home Assistant. For example, the RSSI entity (representing the RF signal strength) provided by the ZHA integration for each Zigbee device is disabled by default.

There are different ways to enable entities. You can enable a single entity in the entity settings, or you can enable multiple entities at once from the list of entities.

### To enable or disable a single entity

1. Go to **Settings > Devices & services** and select your integration card.
2. Select the device.
3. To see all the entities, you may need to expand the **entity not shown** section.
4. Select the entity of interest, select the cogwheel `mdi:cog-outline`, then select the toggle for **Enable**.
5. Select **Update**.
6. Confirm the notification stating that it will take about 30 seconds for the entity to become enabled. Select **OK**.

Screencast showing how to enable a single entity

### To enable or disable multiple entities

1. In Home Assistant, open the table of interest.
2. To enable or disable entities, go to **Settings > Devices & services > Entities**.
3. To enable or disable automations, go to **Settings > Automations & Scenes**.
4. [Enable multiselect](#) and select all the entities you want to enable or disable.
5. In the top right corner, select the three dots `mdi:dots-vertical` menu, then select **Enable** or **Disable**.

Screenshot showing how to enable or disable multiple automations

### Related topics

- [Customizing entities](#)
- [Grouping your assets](#)
- [Working with tables](#)



## Enable I2C

---

Home Assistant using the Home Assistant Operating System which is a managed environment, which means you can't use existing methods to enable the I2C bus on a Raspberry Pi. In order to use I2C devices you will have to - Enable I2C for the Home Assistant Operating System - Set up I2C devices, such as sensors

### Enable I2C with an SD card reader

#### Access the boot partition

You will need: - SD card reader - SD card with Home Assistant Operating System flashed on it

Shutdown/turn-off your Home Assistant installation and unplug the SD card. Plug the SD card into an SD card reader and find a drive/file system named `hassos-boot`. The file system might be shown/mounted automatically. If not, use your operating systems disk management utility to find the SD card reader and make sure the first partition is available.

#### Add files to enable I2C

- In the root of the `hassos-boot` partition, add a new folder called `CONFIG`.
- In the `CONFIG` folder, add another new folder called `modules`.
- Inside the `modules` folder add a text file called `rpi-i2c.conf` with the following content:  

```
txt i2c-dev
```
- In the root of the `hassos-boot` partition, edit the file called `config.txt` add two lines to it:  

```
txt dtparam=i2c_vc=on dtparam=i2c_arm=on
```

#### Start with the new OS configuration

- Insert the SD card back into your Raspberry Pi.
- On startup, the `hassos-config.service` will automatically pickup the new `rpi-i2c.conf` configuration.
- Another reboot might be necessary to make sure the just imported `rpi-i2c.conf` is present at boot time.

### Enable I2C via Home Assistant Operating System Terminal

Alternatively, by attaching a keyboard and screen to your device, you can access the physical terminal to the Home Assistant Operating System.

You can enable I2C via this terminal:

- Login as `root`.
- Type `login` and press enter to access the shell:

```
shell mkdir -p /mnt/boot/CONFIG/modules echo i2c-dev > /mnt/boot/CONFIG/modules/
rpi-i2c.conf echo dtparam=i2c_vc=on >> /mnt/boot/config.txt echo
dtparam=i2c_arm=on >> /mnt/boot/config.txt sync reboot
```

## Troubleshooting

After rebooting the host there should be `i2c-0` and similar device files in `/dev`. If such device files are missing, enabling I2C failed for some reason. You can check the status of I2C kernel modules by using `lsmod | grep i2c` in the terminal. If they are loaded, you should find at least the entry `i2c_dev`. Active usage of the modules is indicated by a number. For example, `i2c_dev 20480 2` would indicate two active I2C device files.

An active I2C can also be checked with a multi meter showing 3.3 V on the I2C pins GPIO2 and GPIO3.

---

## Includes/Common Tasks/File Access

---

## Configuring access to files

---

Your Home Assistant Operating server includes two repositories by default: The official core app repository, and the community app repository. All of the apps mentioned here can be installed by navigating to the app store using **Settings > Apps > Install app** in the UI.

One of the first things to take care of after installing Home Assistant OS is to provide yourself access to files. There are several apps commonly used for this, and most users employ a mix of various apps. Default directories on the host are mapped to the apps so that they can be accessed by the services any particular app might provide. On the host system these directories exist on the `/data` partition at `/mnt/data/supervisor/`.

Using any of the apps listed below, the following directories are made available for access:

- `addons`
- `backup`
- `config`
- `media`
- `share`
- `ssl`

---

## Installing and using the Samba app

The **Samba** app allows you to share the directories on Home Assistant with other systems on your network. After installing the app, you can then also edit files using the editor of your preference from your client computer. This app can be installed from the app store's official repository.

To install the app, follow these steps:

1. Go to **Settings > Apps > Samba share** and select **Install**.
2. On the **Configuration** tab, define **Username** and **Password**, store them in a safe place, and save your changes.
3. You can specify any username and password.
4. They are not related to the login credentials you use to log in to Home Assistant or to log in to the computer from which you are accessing the files.
5. The app won't start if username and password are not defined.
6. For further configuration information, refer to the **Documentation** tab.
7. To start the app, on the **Information** tab, select **Start**.

To access the Home Assistant directories from the other device, follow these steps:

1. Go to **Settings > System > Network** and take note of the **Host name**.
2. Alternatively, you can look up the host name or IP address of your Home Assistant on your router.
3. How you connect from another device to Home Assistant depends on your system. Use one of the following options:

4. **On Windows:** Open **File Explorer** and in the address bar, enter the IP address or hostname with two backslashes as `\\your.ha.ip.address` or `\\hostname`.

Screenshot of File Explorer displaying the navigation to a file share using an IP address  
Screenshot of File Explorer displaying the navigation to a file share using an IP address

5. **On OS X:** Open **Finder** and select **Go > Connect to Server...** and enter the IP address or hostname as `smb://your.ha.ip.address` or `smb://hostname`.
6. **On Ubuntu:** Open **Files** and in the address bar, enter the IP address or hostname as `smb://your.ha.ip.address` or `smb://hostname`.
7. Enter the credentials you entered in the **Samba** app configuration.
8. You also have the option of having the credentials stored so that you do not need to enter them again.
9. Done! You now have access to the directories which you can then mount as a drive or pin to Quick Access.

---

## Installing and using the Visual Studio Code (VSC) app

The **Studio Code Server** app provides access through a feature-packed web-based version of the Visual Studio Code editor. It currently only supports AMD64 and aarch64/ARM64 machines. The app also provides access to the Home Assistant Command Line Interface (CLI) using VSC's built-in terminal, which allows for checking logs, stopping, and starting Home Assistant and apps, creating/restoring backups, and more. (See [Home Assistant via Command Line](#) for further info).

Screenshot of an example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation. Example of a configuration.yaml file, accessed using the Studio Code Server app on a Home Assistant Operating System installation.

To install and use the **Studio Code Server** in Home Assistant, follow these steps:

1. To install the app, go to **Settings > Apps > Studio Code Server** and install the app.
2. Once you have the app installed, if you want, select the **Show in sidebar** option. Then, select **Start**.
3. For information on configuration settings, open the **Documentation** tab.
4. To start browsing, on the **Info** tab, select **Open Web UI**.

---

## Installing and using the File Editor app

The **File Editor** app is a web-based file system browser and text editor. It is a more basic and light weight alternative to Visual Studio Code. YAML files are automatically checked for syntax errors while editing.

Screenshot of an example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation. Example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation.



To install and use the File Editor in Home Assistant, follow these steps:

1. To install the app, go to **Settings > Apps** and select **Install app**.
  2. Search for **File editor**, select it and then select **Install**.
  3. Once you have the app installed, you can edit files within your `/config` directory.
  4. If you want to be able to access directories outside the `/config` directory, you need to disable an option in the app configuration.
  5. Go to the app under **Settings > Apps > File editor**.
  6. Open the **Configuration** tab.
  7. In the configuration settings, disable the **Enforce basepath** option.
  8. Note: The **Enforce basepath** option is intended to protect you from inadvertently making changes to settings files.
  9. For information on other configuration settings, open the **Documentation** tab.
  10. To confirm your changes, select **Save**.
  11. To start browsing, on the **Info** tab, select **Open Web UI**.
- 

## Installing and using the SSH app

If you want to use the Home Assistant command line or an SSH client, you can do this through the **Terminal & SSH** app.

The **Terminal & SSH** app provides the following functionalities:

- It provides a web terminal that you can access from the Home Assistant user interface.
- It allows you to use the Home Assistant Command Line Interface (CLI) which provides custom commands for checking logs, stopping and starting Home Assistant and apps, creating/restoring backups, and more.
- For a list of command line commands, refer to [Home Assistant via Command Line](#).
- It allows connecting to your system using an SSH client.
- It also includes common tools like nano and vi editors.
- The Terminal & SSH app does **not provide** access to the underlying host file system.

To get started with the **Terminal & SSH** app, follow these steps:

1. In the bottom left, select your user to open the **Profile** page. Make sure **Advanced Mode** is enabled.
  2. To install the app, go to the app store under **Settings > Apps** and install the **Terminal & SSH** app.
  3. To use the web terminal, **start** the app, then select **Open Web UI**.
  4. You can now start typing your [commands](#).
  5. If you want to access from an ssh client, you need to enter credentials:
  6. Open the **Configuration** page.
  7. Enter a password or authorized Keys.
  8. Then save and start the app.
-

## Includes/Common Tasks/Filters

---

Filters are applied as follows:

1. No filter
  - All entities included
2. Only includes
  - Entity listed in entities include: include
  - Otherwise, entity matches domain include: include
  - Otherwise, entity matches glob include: include
  - Otherwise: exclude
3. Only excludes
  - Entity listed in exclude: exclude
  - Otherwise, entity matches domain exclude: exclude
  - Otherwise, entity matches glob exclude: exclude
  - Otherwise: include
4. Domain and/or glob includes (may also have excludes)
  - Entity listed in entities include: include
  - Otherwise, entity listed in entities exclude: exclude
  - Otherwise, entity matches glob include: include
  - Otherwise, entity matches glob exclude: exclude
  - Otherwise, entity matches domain include: include
  - Otherwise: exclude
5. Domain and/or glob excludes (no domain and/or glob includes)
  - Entity listed in entities include: include
  - Otherwise, entity listed in exclude: exclude
  - Otherwise, entity matches glob exclude: exclude
  - Otherwise, entity matches domain exclude: exclude
  - Otherwise: include
6. No Domain and/or glob includes or excludes
  - Entity listed in entities include: include
  - Otherwise: exclude

The following characters can be used in entity globs:

 - The asterisk represents zero, one, or multiple characters  - The question mark represents zero or one character

---

## **Includes/Common Tasks/Flashing M1S Otg**

---

# Flashing an ODROID-M1S

---

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

**Warning:** Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

## What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

## Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S\\_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

## Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

### HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

---

## **Includes/Common Tasks/Flashing N2 Otg**

---

## Flashing an ODROID-N2+

---

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

*All these instructions work the same for the ODROID-N2 (non-plus version).*

### What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

### Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

### Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



**Note:** When using the command line, it may return the following message:  
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

## Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
  2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
  3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos\_odroid-n2-.img.xz).
  4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
  5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
  6. Remove the power cable.
  7. Remove the USB and HDMI cables.
  8. Make sure to toggle the boot mode switch back to MMC.
  9. Put the ODROID back in its case.
  10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
  11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
  12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
  13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
  14. Continue with [onboarding](#).
-



## Includes/Common Tasks/Lost Password

---

## Lost Password and password reset

---

Please refer to the [I'm locked out!](#) documentation page.

---

## Includes/Common Tasks/Network Storage

---

## Network storage

You can configure both Network File System (NFS) and Samba/Windows (CIFS) targets to be used within Home Assistant and apps. To list all your currently connected network storages, go to **Settings > System > Storage** in the UI.

**⚠ Important:** You need to update to Home Assistant Operating System 10.2 before you can use this feature.

Screenshot of the list of network shares inside the storage panel.

### Add a new network storage

1. Go to **Settings > System > Storage** in the UI.
2. Select **Add network storage**.
3. Fill out all the information for your network storage.
4. Select **Connect**.

Screenshot of connecting a new network storage.

### Network storage configuration

{% configuration\_basic "hassio.network\_share" %} Name: description: This is the name that will be used for the mounted directory on your system. Usage: description: Here, you select how the target should be used. [See usage types below](#) Server: description: The IP/hostname of the server running NFS/CIFS. "Protocol"<sup>3</sup>: description: The service the server is using for the network storage. "[NFS]"<sup>1</sup> Remote share path": description: The path used to connect to the remote storage server. "[CIFS]"<sup>2</sup> Username": description: "The username to use when connecting to the storage server. Use User Principal Name for domain accounts. For example: `user@domain.com`." "[CIFS]"<sup>2</sup> Password": description: The password to use when connecting to the storage server. "[CIFS]"<sup>2</sup> Share": description: The share to connect to on the storage server.

<sup>1</sup> Options prefixed with `[NFS]` are only available for NFS targets.

<sup>2</sup> Options prefixed with `[CIFS]` are only available for CIFS targets.

<sup>3</sup> For the `CIFS` option, only version 2.1+ is supported.

### Usage types

{% configuration\_basic "hassio.network\_share.usage" %} Backup: description: This will become a target. You can use it when creating an automatic or manual backup. The first storage you add of this type becomes your new default target. If you want to change the default target, [check out the documentation below](#). Media: description: A new directory with the name you gave your network storage will be created under `/media`. This directory can be accessed by Home Assistant and apps. Share: description: A new directory with the name you gave your network storage will be created under `/share`. This directory can be accessed by Home Assistant and apps.

## Change default local backup location

By default, the first network storage of type **Backup** that you add is used as your local default backup location.

If you want to change the local network storage that is used to store your backups, follow these steps:

1. Go to **Settings > System > Backups**.
  2. Select **Settings and history**.
  3. In the top-right corner, select the three dots `mdi:dots-vertical` menu and select **Change default action location**.
  4. Select your preferred network location and save your changes. Select default location used for local backup
  5. Troubleshooting: Don't see your external storage location? This list contains only the network storage targets you have added of type **Backup**.
-

## Includes/Common Tasks/Specific Version

---

### Running a specific version

To see which version your system is running, go to **Settings > About**.

In the event that a Home Assistant Core version doesn't play well with your hardware setup, you can downgrade to a previous release. In this example `` is used as the target version but you can choose the version you desire to run.

To upgrade to a specific version, you can use the command line (CLI). The example below shows how to upgrade to ``. To learn how to get started with the command line in Home Assistant, refer to the [SSH app setup instructions](#).

```
ha core update --version {{current_version}} --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade later.

To downgrade your installation, do a [partial restore of a backup](#) instead.

```
docker pull {{ site.installation.container }}:{{current_version}}
```

You then need to recreate the container with the new image.

---

## Includes/Common Tasks/Third Party Addons

---

## Installing a third-party app repository

---

Home Assistant allows anyone to create an app repository to share their own apps with the community.

 **Warning:** Home Assistant cannot guarantee the quality or security of third-party apps. Use at your own risk.

To add an app repository, follow these steps:

1. Copy the URL of the repository.
2. The URL is the git repository clone URL (on GitHub, use the Code button and copy the HTTPS URL).
3. This documentation uses an example app repository. It is not practically useful but follows the same steps.
4. If you are interested in app development, refer to our [Home Assistant app development documentation](#).

```
```text
https://github.com/home-assistant/hassio-addons-example
```
```

1. Go to **Settings > Apps** and select **App store**. Screenshot of the Home Assistant app store
2. In the top-right corner, select the three dots `mdi:dots-vertical` menu, and select **Repositories**.
3. Add the URL of the repository and select **Add**. Screenshot of the app store
4. Result: A new card for the third-party repository will appear. Screenshot of the app store

## Troubleshooting: Repository is not showing up

If you have added an app repository, but it's not showing up, make sure to refresh your browser. If it still doesn't show up, the app repository may contain invalid configuration data.

1. Go to **Settings > System > Logs** and select Supervisor in the top right corner to get the Supervisor log.
  2. It should tell you what went wrong.
  3. Report this information to the app repository author.
-



## Includes/Common Tasks/Update

---

### Prerequisites

1. [Back up your installation](#) and store the backup and the [backup emergency kit](#) somewhere safe.
2. This ensures that you can [restore your installation from backup](#) if needed.
3. Check the release notes for backward-incompatible changes on [Home Assistant release notes](#). Be sure to check all release notes between the version you are running and the one you are upgrading to. Use the search function in your browser ( `CTRL + f` / `CMD + f` ) and search for **Backward-incompatible changes**.

### To update Home Assistant Core

To update Home Assistant Core, choose one of the following options.

{% tabbed\_block %}

• title: Using the UI content: |

1. Open your Home Assistant UI.
2. Go to the **Settings** panel.
3. On the top you will be presented with an update notification.
  - Troubleshooting: If you do not see that notification:
  - In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
  - Go to **System > Updates**.
    - Select the update notification.
    - Select the cogwheel `mdi:cog-outline`, then set **Visible** to active.
4. Open the notification for the component you want to update.
5. If you want to backup the system first (recommended), enable the backup toggle.
6. Select **Update**.
7. After the update completed, check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

• title: Using the CLI content: |

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

{% endtabbed\_block %}

{% tabbed\_block %}

• title: Docker CLI content: |

**First start with pulling the new container.**

```
bash docker pull {{ site.installation.container }}:stable
```

**You then need to recreate the container with the new image.**

- title: Docker Compose content: |

```
bash docker compose pull homeassistant docker compose up -d
```

{% endtabbed\_block %}

After the update, check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

---

## Includes/Conditions/Behavior

---

### Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Condition passes if** option controls how the results combine:

- **Any** (default): the condition passes if at least one of the targeted entities matches. For example, if you check three smoke sensors and only one of them detects smoke, the condition still passes. This is useful for questions like "is there smoke anywhere in the house?"
  - **All**: the condition passes only when every targeted entity matches. For example, if you check the same three smoke sensors, the condition passes only once all three report cleared. This is useful for "is the entire house safe now?" checks, so your automation does not send an all-clear while one room still has a reading.
-

## Includes/Conditions/More Examples

---

## More examples

---

Real scenarios where this condition gates an automation. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

---

## Includes/Conditions/Related

---

## Related conditions

---

These conditions work well alongside this one:

{% endif %}

---

## Includes/Conditions/Stuck

---



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the condition you're using and what you expected to happen, or share on our [subreddit /r/homeassistant](#).

 **Tip:** AI assistants like ChatGPT or Claude can also explain conditions or suggest the right one when you describe what you want in plain language.

---

## Includes/Conditions/Targets

---

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the condition

---

This condition requires a target. The target is the object that Home Assistant will check. You can point the condition at a single entity, a device, an area, a floor, or a label, and Home Assistant will evaluate every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one condition. For example, you can add a specific entity and an area as targets in the same condition to check both of them at once.

---

## Includes/Conditions/Try It

---

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, open an automation, and add this condition. Trigger the automation with and without the condition met, and watch whether it continues or stops.

---



## Using this condition from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this condition step by step. You pick what to check, tweak a few options, and save. No YAML knowledge required.

---





## Using this condition in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

---

## Includes/Dashboard/Add Picture To Card

---

{% endcapture %} {% endcapture %}

## Adding a to your dashboard

---

1. To add a card, follow steps 1-4 on [adding a card from a view](#).
  2. In step 2, on the **By card** tab, select .
  3. Add a picture:
  4. **Upload picture** lets you pick an image from the system used to show your Home Assistant UI.
  5. **Local path** lets you pick an image stored on Home Assistant. For example:  
`/homeassistant/images/lights_view_background_image.jpg` .
    - To store an image on Home Assistant, you need to [configure access to files](#), for example via [Samba](#) or the [Studio Code Server](#) app (formerly known as add-on).
  6. **web URL** let you use an image from the web. For example `https://www.home-assistant.io/images/frontpage/assist_wake_word.png` .
  7. Define the parameters that are specific to the .
  8. For a description of the specific settings, refer to the description under YAML configuration.
  9. They also apply to the UI.
  10. Save your changes.
-

## Includes/Dashboard/Edit Dashboard

---

{% endcapture %} {% endcapture %}

## Adding the to a dashboard

---

1. In the top right of the screen, select the edit `mdi:edit` button.
  2. If this is your first time editing a dashboard, the **Edit dashboard** dialog appears.
    - By editing the dashboard, you are taking over control of this dashboard.
    - This means that it is no longer automatically updated when new dashboard elements become available.
    - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
    - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
  3. Add a card and customize actions and features to your dashboard.
-

## Includes/Docs/Paste Yaml Tip

---

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

---

## Includes/Energy/Ct Clamp

---

Current transformer (CT) clamp sensors measure your energy usage by looking at the current passing through an electrical wire. This makes it possible to calculate the energy usage. In Home Assistant we have support for off-the-shelf CT clamp sensors or you can build your own.

- The off-the-shelf solution that we advise is the [Shelly EM](#). The device has a local API, updates are pushed to Home Assistant and it has a high quality [integration](#).
- You can build your own using ESPHome's [CT Clamp Current sensor](#) or energy meter sensors like the [ATM90E32](#). For the DIY route, check out [this video by digiblur](#) to get started.
- Using a Raspberry Pi, you can use a CT clamp HAT from LeChacal called [RPICT hats](#). They can be stacked to expand the number of lines to monitor. They also provide Active, Apparent, and Reactive power and power factor for single-phase and three-phase installations. They integrate with Home Assistant using MQTT.

*Attention! Installing CT clamp sensor devices requires opening your electrical cabinet. This work should be done by someone familiar with electrical wiring and may require a licensed professional in some regions. Your qualified installer will know how to do this.*

*Disclaimer: Some links in this section are affiliate links.*

---

## Includes/Energy/Rtl Sdr

---

In the United States and Canada, many electricity, gas, and water meters use [AMR \(Automatic Meter Reading\)](#) or [ERT \(Encoder Receiver Transmitter\)](#) protocols to wirelessly broadcast their readings. You can receive these broadcasts using an inexpensive [RTL-SDR](#) USB dongle and decode them with [rtlamr](#), an open-source receiver for these protocols.

The community project [rtlamr2mqtt](#) packages this into a Home Assistant add-on that automatically publishes your meter readings via MQTT discovery, making them available in Home Assistant without any physical connection to the meter.

This approach can work with many Itron, Badger, and other AMR-compatible meters commonly deployed by US and Canadian utilities, but compatibility varies by meter model and region, and some meters use encryption. Check the [rtlamr wiki](#) to confirm that your specific meter type is supported.

---



## Includes/Installation/Container/Alternative

---

### Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
  - Choose a container-name you want (for example, `homeassistant`).
  - Set **Enable auto-restart** if you like.
  - Select **Next**.
- On the **Advanced Settings** page:
  - In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config`), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
  - To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London`). Timezones can be found [here](#).
  - In the **Network** section, set the Network dropdown as `host`.
  - Select **Next**.
  - Ensure **Run this container after the wizard is finished** is checked and select **Done**.
  - Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123`).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least

privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

[https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system\\_terminal](https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal)

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `./config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.
- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

## QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see [https://www.qnap.com/solution/container\\_station/en/index.php](https://www.qnap.com/solution/container_station/en/index.php).

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing **your correct timezone**.
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

## Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

---

## Includes/Installation/Container/Cli

---

{% tabbed\_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/
dbus:/run/dbus:ro \ --network=host \ {{ site.installation.container }}:
{{ include.tag | default: 'stable' }}
```

- title: Update content: |

```
```bash
```

if this returns "Image is up to date" then you can stop here

```
docker pull {{ site.installation.container }}:{{ include.tag | default: 'stable' }}
```

```
```bash
```

## stop the running container

---

```
docker stop homeassistant
```

```
```bash
```

remove it from Docker's list of containers

```
docker rm homeassistant
```

```
```bash
```

## finally, start a new one

---

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e
TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --
network=host \ {{ site.installation.container }}:{{ include.tag | default: 'stable' }}
```

{% endtabbed\_block %}

---

## Includes/Installation/Container/Compose

---

```
services:
 homeassistant:
 container_name: homeassistant
 image: "{{ include.image | default: site.installation.container }}:{{ incl
 volumes:
 - /PATH_TO_YOUR_CONFIG:/config
 - /etc/localtime:/etc/localtime:ro
 - /run/dbus:/run/dbus:ro
 restart: unless-stopped
 privileged: true
 network_mode: host
 environment:
 TZ: Europe/Amsterdam
```

## Includes/Installation/Container

---

# Install Home Assistant Container

These below instructions are for an installation of Home Assistant Container running in your own container environment, which you manage yourself. Any [OCI](#) compatible runtime can be used, however this guide will focus on installing it with Docker.

 **Note:** This installation type **does not have access to apps**. If you want to use apps, you need to use another installation type. The recommended type is Home Assistant Operating System. Checkout the [overview table of installation types](#) to see the differences.

 **Important: Prerequisites** This guide assumes that you already have an operating system setup and a container runtime installed (like Docker).

If you are using Docker, you need Docker Engine 23.0.0 or later. Docker *Desktop* will not work; you must use Docker *Engine*.

## Platform installation

Installation with Docker is straightforward. Adjust the following command so that:

- `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration and run it. Make sure that you keep the `:/config` part.
- `MY_TIME_ZONE` is a [tz database name](#), like `TZ=America/Los_Angeles`.
- D-Bus is optional but required if you plan to use the [Bluetooth integration](#).

## Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
  - Choose a container-name you want (for example, `homeassistant`).
  - Set **Enable auto-restart** if you like.
- Select **Next**.
- On the **Advanced Settings** page:



- In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config` ), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
- To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London` ). Timezones can be found [here](#).
- In the **Network** section, set the Network dropdown as `host` .
- Select **Next**.
- Ensure **Run this container after the wizard is finished** is checked and select **Done**.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123` ).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

[https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system\\_terminal](https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal)

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `:/config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.

- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

## QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see [https://www.qnap.com/solution/container\\_station/en/index.php](https://www.qnap.com/solution/container_station/en/index.php).

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing [your correct timezone](#).
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

## Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

{% tabbed\_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/
dbus:/run/dbus:ro \ --network=host \ :
```

- title: Update content: |

```
```bash
```

if this returns "Image is up to date" then you can stop here

```
docker pull : ``
```

```
```bash
```

## stop the running container

---

```
docker stop homeassistant ``
```

```
```bash
```

remove it from Docker's list of containers

```
docker rm homeassistant ""  
""bash
```

finally, start a new one

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e  
TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --  
network=host \ : ""
```

{% endtabbed_block %}

Once the Home Assistant Container is running Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Restart Home Assistant

If you change the configuration, you have to restart the server. To do that you have 3 options.

1. In your Home Assistant UI, go to **Settings** > **System** and in the top-right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Restart Home Assistant**.
2. Go to **Settings** > **Developer tools** > **Actions**, select `homeassistant.restart` and select **Perform action**.
3. Restart it from a terminal.

{% tabbed_block %}

- title: Docker CLI content: |

```
bash docker restart homeassistant
```

- title: Docker Compose content: |

```
bash docker compose restart
```

{% endtabbed_block %}

Docker compose

 **Tip:** `docker compose` should already be installed on your system. If not, you can manually install it.

As the Docker command becomes more complex, switching to `docker compose` can be preferable and support automatically restarting on failure or system restart. Create a `compose.yaml` file:

```
services:
  homeassistant:
    container_name: homeassistant
    image: ":"
    volumes:
      - /PATH_TO_YOUR_CONFIG:/config
      - /etc/localtime:/etc/localtime:ro
      - /run/dbus:/run/dbus:ro
    restart: unless-stopped
    privileged: true
    network_mode: host
    environment:
      TZ: Europe/Amsterdam
```

Start it by running:

```
docker compose up -d
```

Once the Home Assistant Container is running, Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Exposing devices

In order to use Zigbee or other integrations that require access to devices, you need to map the appropriate device into the container. Ensure the user that is running the container has the correct privileges to access the `/dev/tty*` file, then add the device mapping to your container instructions:

{% tabbed_block %}

- title: Docker CLI content: |

```
bash docker run ... --device /dev/ttyUSB0:/dev/ttyUSB0 ...
```

- title: Docker Compose content: |

```
yaml services: homeassistant: ... devices: - /dev/ttyUSB0:/dev/ttyUSB0
```

{% endtabbed_block %}

Optimizations

The Home Assistant Container is using an alternative memory allocation library `jemalloc` for better memory management and Python runtime speedup.

As the `jemalloc` configuration used can cause issues on certain hardware featuring a page size larger than 4K (like some specific ARM64-based SoCs), it can be disabled by passing the environment variable `DISABLE_JEMALLOC` with any value, for example:

{% tabbed_block %}

- title: Docker CLI content: |

```
bash docker run ... -e "DISABLE_JEMALLOC=true" ...
```

- title: Docker Compose content: |

```
yml services: homeassistant: ... environment: DISABLE_JEMALLOC: true
```

{% endtabbed_block %}

The error message `<jemalloc>: Unsupported system page size` is one known indicator.

Includes/Installation/Operating System

Install Home Assistant Operating System

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.

Suggested hardware

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

These are affiliated links. We get commissions for purchases made through links in this post.

Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

Configure the BIOS on your x86-64 hardware

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

Write HAOS onto your x86-64 hardware

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

Method 1 (recommended): Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

Method 2: With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
 - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
 - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
 - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
 - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
 - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
 - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

Required material

- Computer

- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

Write the image to your boot medium

Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

{% endtabbed_block %}

```
text          {%          assign          board_key          =
site.installation.types[page.installation_type].variants[0].key          %}
{{release_url}}/{{site.data.version_data.hassos[board_key]}}/
haos_{{ board_key }}-{{site.data.version_data.hassos[board_key]}}.img.xz
```

{% endif %}

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.
- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at homeassistant.local:8123.

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at homeassistant:8123 or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

Download the appropriate image

- [VirtualBox \(Intel chip\) \(.vdi\)](#)
- [VirtualBox \(Apple Silicon chip\) \(.vdi\)](#)
- [KVM \(.qcow2\)](#)
- [KVM/Proxmox \(.qcow2\)](#)
- [VMware ESXi/vSphere \(.ova\)](#)
- [VMware Workstation \(.vmdk\)](#)
- [Hyper-V \(.vhdx\)](#)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

All these can be extended if your usage calls for more resources.

Hypervisor specific configuration

{% tabbed_block %}

- title: VirtualBox content: |

Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).

2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager**, **Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.

6. Select the CPU cores you want to let the VM use and give it some memory.
 7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
 8. Select your keyboard language under **VM Console Keyboard**.
 9. Select **br0** under **Network Source**.
 10. Select **virtio** under **Network model**.
 11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
 12. Deselect **Start VM after creation**.
 13. Select **Create**.
 14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
 15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
 1. Create a new virtual machine in `virt-manager`.
 2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
 3. Choose **Generic Default** for the operating system.
 4. Check the box for **Customize configuration before install**.
 5. Under **Network Selection**, select your bridge.
 6. Under customization select **Overview > Firmware > UEFI x86_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
 7. Select **Add Hardware** (bottom left), and select **Channel**.
 8. Select device type: **unix**.
 9. Select name: **org.qemu.guest_agent.0**.
 10. Finally, select **Begin Installation** (upper left corner).
 - title: KVM (virt-install) content: |


```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
 - Select the one for Ethernet.
 - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
 - If you haven't unzipped the archive, unzip it.
 - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
 - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.

6. Now continue with the next step to start your VM.

- If you see a message about side channel mitigations, select **OK**.
 - If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
 - title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
 1. Create a new virtual machine.
 2. Select **Generation 2**.
 3. Select **Connection** > **Your Virtual Switch that is bridged**.
 4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings** > **Security** and deselect **Enable Secure Boot**.

{% endtabbed_block %}

Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Flashing an ODROID-N2+

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

All these instructions work the same for the ODROID-N2 (non-plus version).

What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



Note: When using the command line, it may return the following message:
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).

Flashing an ODROID-M1S

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

Warning: Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

Includes/Integrations/Actions

{%- assign domain = include.domain | default: page.ha_domain -%}

List of actions

The integration provides the following actions. Each link below opens a dedicated page with examples, parameters, and a step-by-step UI walkthrough.

For an overview of every action across all integrations, see the [actions reference](#).

This integration does not provide any documented actions yet. See the [actions reference](#) for actions from other integrations.

Includes/Integrations/Building Block Integration

{% endcapture %}

Note: Building block integration

This is a building block integration that cannot be added to your Home Assistant directly but is used and provided by other integrations.

A building block integration differs from the typical integration that connects to a device or service. Instead, other integrations that do integrate a device or service into Home Assistant use this building block to provide entities, services, and other functionality that you can use in your automations or dashboards.

If one of your integrations features this building block, this page documents the functionality the building block offers.

Includes/Integrations/Conditions

{%- assign domain = include.domain | default: page.ha_domain -%}

List of conditions

The integration provides the following conditions. Each link below opens a dedicated page with examples, fields, and a step-by-step UI walkthrough.

For an overview of every condition across all integrations, see the [conditions reference](#).

This integration does not provide any documented conditions yet. See the [conditions reference](#) for conditions from other integrations.

Includes/Integrations/Config Flow

{% endcapture %}

Configuration

To add the **** integration{% endunless %} to your Home Assistant instance, use this My button:

`config_flow_start`

can be auto-discovered by Home Assistant. If an instance was found, it will be shown as **Discovered**. You can then set it up right away.

{% details "Manual configuration steps" icon="mdi:cursor-hand" %} If it wasn't discovered automatically, don't worry! You can set up a manual integration entry:

If the above My button doesn't work, you can also perform the following steps manually:

- Browse to your Home Assistant instance.
- Go to **Settings > Devices & services**.
- At the top of the screen, select the tab: **helpers**.
- In the bottom right corner, select the **Create helper** button.
- In the bottom right corner, select the **config_flow_start** button.
- From the list, select ****.
- Follow the instructions on screen to complete the setup.

{% enddetails %}

Includes/Integrations/Device Class Intro

A device class is a measurement categorization in Home Assistant. It influences how the entity is represented in the [dashboard](#). This can be modified in the [customize section](#). For example, different states may be represented by different icons, colors, or text.

Includes/Integrations/Google Api

{% endcapture %} {% endcapture %} {% endcapture %} {% endcapture %}

1. Go the Google Developers Console and .
 2. Confirm the project and **Enable** the API.
-

Includes/Integrations/Google Client Secret

{% endcapture %} {% endcapture %} {% endcapture %} {% endcapture %}

Scenario 1: You already have credentials

In this case, all you need to do is enable the API:

1. Go the Google Developers Console and .
2. Confirm the project and **Enable** the API.
3. Continue with the steps described in the [Configuration](#) section.

Scenario 2: You do not have credentials set up yet

In this case, you need to generate a client secret first:

To generate client ID and client secret

This section explains how to generate a client ID and client secret on [Google Developers Console](). 1. First, go to the Google Developers Console to enable []() and [](). 2. Select **Create project**, enter a project name and select **Create**. 3. **Enable** the . 4. Navigate to **APIs & Services** (left sidebar) > [Credentials](https://console.cloud.google.com/apis/credentials). 5. In the left sidebar, select **OAuth consent screen**. 6. It will take you to the Overview page and ask for **Project Configuration**: - Complete the App Information: - Set the **App name** (the name of the application asking for consent) to anything you want, for example, *Home Assistant*. - For a **Support email**, choose your email address from the dropdown menu. - Select **Next**. - For Audience, select **External** then select **Next**. - Under Contact Information, enter your email address (the same as above is fine). Select **Next**. - Read the policy and check the box if you agree. Select **Continue**. - Select **Create**. 7. In the left sidebar, select **Audience**: - Under **Publishing status** > **Testing**, select **Publish app**. > Otherwise, your credentials will expire every 7 days. 8. In the left sidebar, select **Clients**: - Select **+ Create Client**. - For Application type, choose **Web Application** and give this client ID a name (like "Home Assistant Client"). - Add `https://my.home-assistant.io/redirect/oauth` to **Authorized redirect URIs** then select **Create**. > **Note**: This is not a placeholder. It is the URI that must be used. 9. From the resulting dialog take a note of **Client ID** and **Client Secret** you **can not retrieve it again** after closing the dialog. - Once you have noted these strings, select **Close**. - Congratulations! You are now the keeper of a client secret. Guard it in your treasure box. In most cases, your new credentials will be active within a few minutes. However, Google states that activation may take up to five hours in some circumstances.

Includes/Integrations/Google Oauth

{% endcapture %} {% endcapture %} {% endcapture %} {% endcapture %} {% endcapture %}

The integration setup will next give you instructions to enter the [Application Credentials](#) (OAuth Client ID and Client Secret) and authorize Home Assistant to connect to .

OAuth and device authorization steps

1. Continue through the steps of selecting the account you want to authorize. 2. ****NOTE****: You may get a message telling you that the app has not been verified and you will need to acknowledge that in order to proceed. 3. You can now see the details of what you are authorizing Home Assistant to access with two options at the bottom. Select ****Continue****. 4. The page will now display ****Link account to Home Assistant?****, note ****Your instance URL****. If this is not correct, refer to [My Home Assistant](/integrations/my). If everything looks good, select ****Link Account****. 5. You may close the window, and return back to Home Assistant where you should see a ****Success!**** message from Home Assistant.

Includes/Integrations/Labs Entity Triggers Note

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Includes/Integrations/Option Flow

{% endcapture %}

Options

To define options for , follow these steps:

1. In Home Assistant, go to **Settings > Devices & services**.
2. If multiple instances of are configured, choose the instance you want to configure.
3. On the card, select the cogwheel `mdi:cog-outline` .
4. If the card does not have a cogwheel, the integration does not support options for this device.

Screenshot showing the cogwheel icon on an integration card in the Settings > Devices & services page

1. Edit the options, then select **Submit** to save the changes.
-

Includes/Integrations/Remove Device Service

To remove an integration instance from Home Assistant

1. Go to **Settings** > **Devices & services** and select the integration card.
 2. From the list of devices, select the integration instance you want to remove.
 3. Next to the entry, select the three dots `mdi:dots-vertical` menu. Then, select **Delete**.
-

Includes/Integrations/Remove Device Service Steps

1. Go to **Settings** > **Devices & services** and select the integration card.
 2. From the list of devices, select the integration instance you want to remove.
 3. Next to the entry, select the three dots `mdi:dots-vertical` menu. Then, select **Delete**.
-

Includes/Integrations/Restart Ha After Config Inclusion

After changing the `configuration.yaml` file, restart Home Assistant to apply the changes. The integration is now shown on the integrations page under **Settings > Devices & services**. Its entities are listed on the integration card itself and on the **Entities** tab. {% endif %}

Includes/Integrations/Supported Brand

{% endcapture %}

Support for devices by in Home Assistant is provided by the integration.

devices are either rebranded devices or devices that share a common communication protocol, making it possible to use them with the integration.

[Include not found: integrations/config_flow.md domain=page.ha_supporting_domain]

Usage information

For more documentation on how to use in Home Assistant, please refer to the [integration documentation page](#).

Includes/Integrations/Triggers

{%- assign domain = include.domain | default: page.ha_domain -%}

List of triggers

The integration provides the following triggers. Each link below opens a dedicated page with examples, fields, and a step-by-step UI walkthrough.

For an overview of every trigger across all integrations, see the [triggers reference](#).

This integration does not provide any documented triggers yet. See the [triggers reference](#) for triggers from other integrations.

Includes/Integrations/Triggers Conditions Actions

{%- assign domain = include.domain | default: page.ha_domain -%}

List of triggers

The integration provides the following triggers. Each link below opens a dedicated page with examples, fields, and a step-by-step UI walkthrough.

For an overview of every trigger across all integrations, see the [triggers reference](#).

This integration does not provide any documented triggers yet. See the [triggers reference](#) for triggers from other integrations.

{%- assign domain = include.domain | default: page.ha_domain -%}

List of conditions

The integration provides the following conditions. Each link below opens a dedicated page with examples, fields, and a step-by-step UI walkthrough.

For an overview of every condition across all integrations, see the [conditions reference](#).

This integration does not provide any documented conditions yet. See the [conditions reference](#) for conditions from other integrations.

{%- assign domain = include.domain | default: page.ha_domain -%}

List of actions

The integration provides the following actions. Each link below opens a dedicated page with examples, parameters, and a step-by-step UI walkthrough.

For an overview of every action across all integrations, see the [actions reference](#).

This integration does not provide any documented actions yet. See the [actions reference](#) for actions from other integrations.

Includes/Integrations/Using Templates

Using templates

For incoming data, a value template translates incoming JSON or raw data to a valid payload. Incoming payloads are rendered with possible JSON values, so when rendering, the `value_json` can be used to access the attributes in a JSON based payload, otherwise the `value` variable can be used for non-json based data.

Additional, the `this` can be used as variables in the template. The `this` attribute refers to the current [entity state](#) of the entity. Further information about `this` variable can be found in the [template documentation](#)

 **Note:** Example value template with json:

With given payload:

```
``json
```

```

>
> Template ``{% endraw %} renders to `21.9`.

---

## Includes/Integrations/Wwha

{% endcapture %}
{% endcapture %}

[]() is a member of the Works with Home Assistant partner program for their pro

Z-Wave devices work locally and integrate seamlessly with the Z-Wave integratio

add_zwave_device

[Learn more about Z-Wave in Home Assistant.](/integrations/zwave_js/)

Zigbee devices work locally and integrate seamlessly with the Zigbee integratio

add_zigbee_device

[Learn more about Zigbee in Home Assistant.](/integrations/zha/)

Matter devices work locally and integrate seamlessly with the Matter integratio

add_matter_device

[Learn more about Matter in Home Assistant.](/integrations/matter/)

---

## Includes/Organizing/Reorder Areas

Follow these steps to rearrange floors and areas on the built-in dashboards (suc

1. Go to Settings > Areas, labels & zones.
2. There are 2 options to rearrange items:
   - Option 1: Use drag-and-drop.
   - Option 2: In the top-right corner, select the three dots mdi:dots-vert
     - In the dialog, move the floors or areas you want to rearrange:

   ![Reorder floors and areas](/images/organizing/reorder-areas-menu.png)

```

```

---

## Includes/Template Functions/More Examples

## More examples

Real scenarios where this function comes up in automations and templates. Copy a

---

## Includes/Template Functions/Related

## Related template functions

These functions work well alongside this one:

{% endif %}
{% endif %}

---

## Includes/Template Functions/Signatures

## Function signature

The signature is a technical summary of this template function. It shows the nam
Function parameters that have a `=` with a value after them are optional. If you

---

## Includes/Template Functions/Stuck

## Still stuck?

The Home Assistant community is quick to help: join [Discord](https://discord.gg

> **💡 Tip:**
> AI assistants like ChatGPT or Claude can also explain or fix templates when yo

---

## Includes/Template Functions/Try It

```

```
## Try it yourself
```

Ready to test this? Open **Developer tools** > **Template**, paste the example i

```
---
```

```
## Includes/Template Functions/Usage
```

```
## Usage
```

Here's how to use this template function. Copy any example and adjust it to your

```
---
```

```
## Includes/Triggers/Behavior
```

```
### Behavior with multiple targets
```

When you target more than one entity (or select an area, floor, or label that co

- **Each** (``any`` in YAML, default): the trigger fires every time any one of the
- **First** (``first`` in YAML): the trigger fires only on the first transition in
- **All** (``last`` in YAML): the trigger fires only after the last targeted entit

```
---
```

```
## Includes/Triggers/More Examples
```

```
## More examples
```

Real scenarios where this trigger fires in automations and scripts. Copy any exa

> **💡 Tip:**

> You don't need to edit YAML to use these examples. Copy a YAML snippet from th

```
---
```

```
## Includes/Triggers/Related
```

```
## Related triggers
```

These triggers work well alongside this one:

```
{% endif %}
{% endif %}
```

```
## Includes/Triggers/Stuck
```

```
## Still stuck?
```

The Home Assistant community is quick to help: join [Discord] (<https://discord.gg>

> **💡 Tip:**

> AI assistants like ChatGPT or Claude can also explain triggers or suggest the

```
## Includes/Triggers/Targets
```

```
{%- assign target_domain = include.domain | default: page.domain -%}
```

```
<a id="targets"></a>
```

```
<a id="targets-of-the-trigger"></a>
```

```
## Targets of the trigger
```

This trigger requires a target. The target is the object that Home Assistant wil

- **Entity**: one specific entity, such as `.living_room`.`
- **Device**: every entity that belongs to a device.
- **Area**: every entity in a room or area.
- **Floor**: every entity on a floor.
- **Label**: every entity that shares a label.

You can also select different target types in one trigger. For example, you can

```
## Includes/Triggers/Try It
```

```
## Try it yourself
```

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new

```

## Includes/Triggers/Ui Header

## Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through th

---

## Includes/Triggers/Yaml Header

## Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant do

---

## Template Functions/Abs

The `abs` filter returns the absolute value of a number, which is its distance f
This is useful when you care about the magnitude of a difference but not its dir

## Usage

Here's how to use this template function. Copy any example and adjust it to your

{% template_function_usage %}
filter: '{{ -5 | abs }}'
type: integer
output: "5"
{% endtemplate_function_usage %}

## Function signature

The signature is a technical summary of this template function. It shows the nam
Function parameters that have a `=` with a value after them are optional. If you

```signature
value | abs() -> number

```

## Function parameters

The following parameters can be provided to this filter.

```
{% function_parameters %} value: description: > The number to get the absolute value of.
Negative values become positive, positive values remain unchanged. required: true type: float
```

## Absolute value of a float

---

The filter works with both integers and floating-point numbers.

### template

```
template: '{{ -21.5 | abs }}'
type: float
output: "21.5"
```

## Good to know

---

- Raises an error when the value is not a number, so convert with `float` or `int` first if you're working with state strings.
- Works on integers and floats and preserves the original numeric type.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Calculate temperature deviation

Find how far the current temperature is from a target, regardless of direction.

#### template

```
template: |
 {% set current = states("sensor.temperature") | float(0) %}
 {% set target = 22.0 %}
 {{ (current - target) | abs | round(1) }} degrees from target
type: string
output: "1.5 degrees from target"
```

### Compute the difference between two sensors

Calculate the absolute difference between two sensor readings.

#### template

```
template: |
 {% set indoor = states("sensor.indoor_temp") | float(0) %}
 {% set outdoor = states("sensor.outdoor_temp") | float(0) %}
 Difference: {{ (indoor - outdoor) | abs | round(1) }} degrees
type: string
output: "Difference: 8.3 degrees"
```

### Use in an automation condition

Only trigger when the power consumption change exceeds a threshold, regardless of direction.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{
 (states("sensor.power_now") | float(0)
 - states("sensor.power_previous") | float(0))
 | abs > 500
 }}
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Acos

---

The `acos` template function returns the arc cosine (inverse cosine) of a value. The result is in radians, in the range  $[0, \pi]$ . The input must be between -1 and 1.

This is useful when you need to convert a ratio back into an angle. For example, you could determine the angle between two direction vectors derived from sensor data.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "1.0471975511965979"

---

filter: '' type: float output: "1.0471975511965979"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
acos(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to calculate the arc cosine of. Must be numeric and between -1 and 1. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is out of range or not numeric). If not provided, an error is raised instead. required: false type: any

## Good to know

---

- Inputs outside the range -1 to 1 raise an error unless you supply a default.
- The result is in radians. Multiply by `180 / pi` to convert to degrees.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Using a default value

If the input might be out of range or non-numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

#### template

```
template: '{{ acos(states("sensor.ratio"), default=0) }}'
type: float
output: "0"
```

### Convert arc cosine to degrees

Convert the result from radians to degrees to make it more readable.

#### template

```
template: '{{ (acos(0.5) * 180 / pi) | round(1) }}'
type: float
output: "60.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Add

---

The `add` filter converts a value to a float and adds a specified amount to it. If the value cannot be converted to a number, it returns the default you provide instead of raising an error.

This is a convenient shorthand for arithmetic on sensor values. Instead of converting to a float first and then adding, you can do it in a single step. It is especially useful when you need to apply a fixed offset to a reading, such as adjusting a temperature sensor by a calibration value, adding a base cost to a calculated price, or shifting a time value. For multiplication, see `multiply`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: float output: "24.0"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
add(
 value: Any,
 amount: float,
 default: Any = _SENTINEL,
) -> float | Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The value to convert to a float and add to. Must be a number or a string that can be converted to a float. required: true type: any amount: description: > The amount to add to the value. Can be negative to subtract. required: true type: float default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. required: false type: any

## Subtracting values

---

To subtract, pass a negative amount.

### template

```
template: '{{ states("sensor.temperature") | add(-5) }}'
title: Subtract 5 from a value
type: float
output: "16.5"
```



## Using a default value

---

If the sensor might be unavailable, provide a default to prevent errors.

### template

```
template: '{{ states("sensor.temperature") | add(2.5, default=0) }}'
title: Safe addition with default
type: float
output: "24.0"
```

## Good to know

---

- There is no subtract filter. Pass a negative amount to subtract.
- The result is always a float, even when both inputs are integers.
- Without a default, a non-numeric input raises an error. Always pass a default when working with state values.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Apply a calibration offset

Correct a temperature sensor reading by a known offset.

#### template

```
template: '{{ states("sensor.outdoor_temperature") | add(-1.2) | round(1) }}'
title: Apply a -1.2 degree calibration offset
type: float
output: "17.1"
```

### Calculate a total with a base fee

Add a fixed base cost to a calculated usage charge.

#### template

```
template: |
 {{
 (states("sensor.energy_today") | float(0) * 0.25)
 | add(5.0) | round(2)
 }}
title: Energy cost with a base fee
type: float
output: "8.47"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Apply

---

The `apply` template function calls another function with a value as its first argument, plus any extra arguments you give it. It lets you hand a function to places that normally expect a filter or test, like `map`, `select`, or `reject`.

You reach for `apply` when you want to run a custom function (one you built with `as_function`) or a built-in function across every item of a list. Most templates don't need it. It comes in handy when you have reusable logic you want to apply to many values without writing a full `for` loop.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "5.0"

---

filter: '' type: float output: "5.0"

---

test: | Less than 10

type: string output: "Less than 10"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
apply(
 value: Any,
 fn: Callable,
 *args: Any,
 **kwargs: Any,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to pass as the first argument to the function (for example, a number from a sensor, or an item from a list). required: true type: any fn: description: > The function to call. Can be any built-in template function, filter, or a macro wrapped with `as_function`. required: true type: any args: description: > Additional positional arguments to pass to the function after the value. For example, `apply(5, float, 0)` calls `float(5, 0)`. required: false type: any kwargs: description: > Additional keyword arguments to pass to the function, written as `name=value` pairs. required: false type: any

## Using apply with map

---

`apply` is most useful when combined with `map` to transform lists of values. You pass any function to `map("apply", fn)` and it runs on each item in the list.

### template

```
template: '{{ [1, 2, 3] | map("apply", float) | list }}'
title: Convert list items to floats
type: list
output: "[1.0, 2.0, 3.0]"
```

## Using apply as a test with select

---

When used as a test, `apply` lets you filter items using any function as a predicate.

### template

```
template: '{{ [3, 7, 2, 9, 5] | select("apply", gt, 4) | list }}'
title: Select values greater than 4
type: list
output: "[7, 9, 5]"
```

## Good to know

---

- Extra arguments are passed after the value, so `apply(5, float, 0)` calls `float(5, 0)` and uses `0` as the default.
- Works with built-in template functions, filters, and macros wrapped with `as_function`, but not arbitrary Python callables.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Apply a custom function across sensor values

Use `apply` with `as_function` to run a custom macro over a list of values.

#### template

```
template: |
 {% macro macro_celsius_to_f(value, returns) %}
 {{ returns(value * 9 / 5 + 32) }}
 {% endmacro %}
 {% set c_to_f = as_function(macro_celsius_to_f) %}
 {{ [0, 20, 100] | map("apply", c_to_f) | list }}
type: list
output: "[32.0, 68.0, 212.0]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Area Devices

---

The `area_devices` template function returns a list of device IDs that belong to a given area. You can specify the area by its name or by its internal ID. While `area_entities` gives you individual entity IDs, `area_devices` gives you the devices themselves.

This is useful when you need to work at the device level rather than the entity level. For example, you might want to count how many physical devices are in a room, check if a specific device is assigned to an area, or loop through all devices to find their attributes. Each device can have multiple entities, so the device list is typically shorter than the entity list for the same area.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["a1b2c3d4e5f6a1b2c3d4e5f6",
"f6e5d4c3b2a1f6e5d4c3b2a1",]
```

---

```
filter: '{{ "Living Room" | area_devices }}' type: list output: | ["a1b2c3d4e5f6a1b2c3d4e5f6",
"f6e5d4c3b2a1f6e5d4c3b2a1",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
area_devices(
 area_name_or_id: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} area\_name\_or\_id: description: > The name or ID of the area. You can find area IDs in **Settings > Areas, labels & zones**. required: true type: string

## Good to know

---

- Returns an empty list when the area has no devices or when the name does not match. It does not raise an error.
- A device can only belong to one area. Entities assigned directly to an area but not through a device are not included here.
- The list contains device IDs, not entity IDs. Use `device_entities` to get entities from each device.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many devices are in a room

Find out how many physical devices are assigned to an area.

#### template

```
template: '{{ area_devices("Kitchen") | count }}'
type: integer
output: "5"
```

### Get the name of each device in an area

Loop through all devices in an area and display their names using `device_name`.

#### template

```
template: |
 {% for device_id in area_devices("Living Room") %}
 {{ device_name(device_id) }}
 {% endfor %}
```

### Check if any device in an area has a firmware update

Combine `area_devices` with entity lookups to see if any device in a room has a pending update.

#### template

```
template: |
 {{
 area_devices("Office")
 | map("device_entities")
 | sum(start=[])
 | select("match", "update.")
 | select("is_state", "on")
 | list
 | count > 0
 }}
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Area Entities

---

The `area_entities` template function returns a list of entity IDs that belong to a given area. You can specify the area by its name (like *"Living Room"*) or by its internal ID. It gives you all entities that have been assigned to that area in Home Assistant.

This is useful when you want to work with all entities in a room at once, without having to list each one by hand. For example, you could turn off every light in the bedroom at bedtime, check if any window in the house is open, or count how many motion sensors are currently active on a floor. As you add or remove devices from an area in Home Assistant, the list automatically updates, so your automations and templates always stay in sync with your actual setup.

 **Tip:** Automation actions can target an entire area through the visual editor, no template needed. Reach for `area_entities()` when you need to loop over or filter the entities inside a template expression.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["light.living_room_ceiling",
"sensor.living_room_temperature", "binary_sensor.living_room_motion",]
```

---

```
filter: '{{ "Living Room" | area_entities }}' type: list output: | ["light.living_room_ceiling",
"sensor.living_room_temperature", "binary_sensor.living_room_motion",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
area_entities(
 area_name_or_id: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} area\_name\_or\_id: description: > The name or ID of the area. You can find area IDs in **Settings > Areas, labels & zones**. required: true type: string

## Good to know

---

- Returns an empty list when the area has no entities or when the name does not match. It does not raise an error.
- Includes both entities assigned directly to the area and entities inherited from devices in that area.
- The names used in the lookup are case-sensitive, so `"Living Room"` and `"living room"` are not the same.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count the lights that are on in an area

Want to know how many lights are currently on in a room? This combines `area_entities` with filters to narrow down to lights and count the ones that are on.

#### template

```
template: |
 {{
 area_entities("Kitchen")
 | select("match", "light.")
 | select("is_state", "on")
 | list
 | count
 }}
type: integer
output: "3"
```

### Check if any motion sensor is active in an area

This checks whether any motion sensor in the hallway is currently detecting motion. Useful as a condition in automations.

#### template

```
template: |
 {{
 area_entities("Hallway")
 | select("match", "binary_sensor.")
 | select("is_state", "on")
 | list
 | count > 0
 }}
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---



## Template Functions/Area Id

---

The `area_id` template function returns the unique area ID for a given area name, entity ID, or device ID. Every area in Home Assistant has an internal ID that stays the same even if you rename the area, and this function lets you look it up.

This is useful when you need the area ID to pass to other area functions like `area_entities` or `area_devices`, or when you want to find out which area a particular entity or device belongs to. For example, you could use it in an automation to determine which room triggered a motion sensor and then act on all devices in that room.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "living\_room"

---

filter: '{{ "Living Room" | area\_id }}' type: string output: "living\_room"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
area_id(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The area name, entity ID, or device ID to look up. If an area name is given, the matching area ID is returned. If an entity ID or device ID is given, the area ID of that entity or device is returned. required: true type: string

## Good to know

---

- Returns `None` when no matching area is found for the lookup value.
- The area ID is stable and does not change when you rename the area, so it is safe to hard-code in templates.
- When an entity or device has no area assigned, the result is `None` even if the entity itself exists.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find the area of an entity

Look up which area a specific entity belongs to.

#### template

```
template: '{{ area_id("sensor.living_room_temperature") }}'
type: string
output: "living_room"
```

### Find the area of a device

Look up which area a device is assigned to by passing its device ID.

#### template

```
template: '{{ area_id("deadbeefdeadbeefdeadbeef") }}'
type: string
output: "kitchen"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---



## Template Functions/Area Name

---

The `area_name` template function returns the friendly, human-readable name of an area. You can give it an area ID, an entity ID, or a device ID, and it tells you the name of the area it belongs to.

This is especially useful for building dynamic messages and notifications. Instead of showing a technical area ID like `living_room`, you can show the actual name "*Living Room*" that you (or whoever receives the message) recognize. For example, when motion is detected, your notification can say "*Motion detected in the Living Room*" by looking up the area name of the sensor that triggered.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "Living Room"

---

filter: '{{ "living\_room" | area\_name }}' type: string output: "Living Room"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
area_name(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The area ID, entity ID, or device ID to look up. Returns the friendly name of the area, or `None` if no matching area is found. required: true type: string

## Good to know

---

- Returns `None` when the lookup value does not match an area, entity, or device.
- When an entity or device has no area assigned, the result is `None`.
- The name reflects the current area label, so renaming an area updates the output immediately.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get the area name from an entity

Find out which room a sensor is in by passing its entity ID.

#### template

```
template: '{{ area_name("sensor.living_room_temperature") }}'
type: string
output: "Living Room"
```

### Use in a notification with the trigger area

A very common pattern: when a sensor triggers an automation, include the room name in the notification. This works with any trigger that provides `trigger.entity_id`, such as state or motion triggers.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 title: >
 {{ area_name(trigger.entity_id) }}
 message: >
 Motion detected in {{ area_name(trigger.entity_id) }}
```

### List all area names

Loop through all areas and display their names.

#### template

```
template: |
 {% for area_id in areas() %}
 {{ area_name(area_id) }}
 {% endfor %}
type: string
output: |
 Living Room
 Kitchen
 Bedroom
 Hallway
```

## List rooms with open windows

A common use case: build a message listing which rooms have open windows or doors. This uses `area_name` as a filter to convert each open sensor's entity ID into a room name, then removes duplicates.

### template

```
template: |
 {{
 states.binary_sensor
 | selectattr('attributes.device_class', 'in',
 ('window', 'door'))
 | map(attribute='entity_id')
 | select('is_state', 'on')
 | map('area_name')
 | select
 | unique
 | list
 | join(', ')
 }}
type: string
output: "Kitchen, Bedroom"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Areas

---

The `areas` template function returns a list of all area IDs in your Home Assistant instance. Each area you've created in Home Assistant has a unique ID, and this function gives you all of them.

This is useful when you want to loop through every area in your home and do something with it. For example, you could check all areas for motion, count how many rooms have lights on, or build a dynamic dashboard that adapts to your areas. Since areas can be added or removed at any time, using `areas()` ensures your templates always reflect your current setup.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: list output: | ["living_room", "kitchen",
"bedroom", "hallway",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
areas() -> list[str]
```

## Good to know

---

- Returns area IDs, not human-readable names. Pair with `area_name` to get names for display.
- The list is unordered. Apply `| sort` if you need a consistent order.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many areas you have

A convenient way to see how many areas are set up in your Home Assistant instance.

#### template

```
template: '{{ areas() | count }}'
type: integer
output: "4"
```

### Check if any area has motion

Loop through all areas and check if any has an active motion sensor.

#### template

```
template: |
 {% for area_id in areas() %}
 {% if area_entities(area_id)
 | select("match", "binary_sensor.")
 | select("is_state", "on")
 | list
 | count > 0 %}
 Motion in {{ area_name(area_id) }}!
 {% endif %}
 {% endfor %}
```

### Count lights on per room

Build a summary of how many lights are on in each area. This loops through all areas and counts the active lights in each one.

#### template

```
template: |
 {% for id in areas() %}
 {% set lights = area_entities(id)
 | select("match", "light.")
 | select("is_state", "on")
 | list %}
 {% if lights | count > 0 %}
 {{ area_name(id) }}: {{ lights | count }} lights on
 {% endif %}
 {% endfor %}
type: string
output: |
 Living Room: 3 lights on
 Kitchen: 1 lights on
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/As Datetime

---

The `as_datetime` template function converts a date/time string or a UNIX timestamp into a datetime object. Give it an ISO 8601 formatted string like "2024-03-15T14:30:00" or a numeric timestamp, and it returns a proper datetime you can work with in your templates.

Many sensors and integrations provide dates and times as plain text strings or UNIX timestamps. To compare those values with `now`, calculate time differences, or format them for display, you first need to turn them into datetime objects. `as_datetime` handles this conversion for you. It accepts ISO 8601 formatted strings (the most common format in Home Assistant) as well as numeric UNIX timestamps.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: datetime output: "2024-03-15 14:30:00+01:00"

---

filter: '{{ "2024-03-15T14:30:00+01:00" | as\_datetime }}' type: datetime output: "2024-03-15 14:30:00+01:00"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
as_datetime(
 value: str | int | float,
 default: Any = None,
) -> datetime | Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to convert to a datetime. Can be an ISO 8601 formatted string or a numeric UNIX timestamp. required: true type: [string, float] default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. required: false type: any

## Converting from a UNIX timestamp

---

You can also convert a numeric UNIX timestamp (seconds since January 1, 1970) into a datetime object.

### template

```
template: '{{ as_datetime(1710510600) }}'
type: datetime
output: "2024-03-15 14:30:00+00:00"
```

## Using a default value

---

If the input string might be invalid, provide a default value to avoid errors.

### template

```
template: '{{ as_datetime("not a date", default="unknown") }}'
type: string
output: "unknown"
```

## Good to know

---

- Strings without a time zone are treated as local time; strings with an offset or `Z` suffix keep the original zone.
- UNIX timestamps are interpreted in UTC, so chain with `as_local` if you need local time.
- Without a default, an unparseable input raises an error.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Convert a sensor value to a datetime

Many sensors store dates as strings. Convert them to datetime objects to perform comparisons or calculations.

#### template

```
template: '{{ as_datetime(states("sensor.next_appointment")) }}'
type: datetime
output: "2024-03-15 14:30:00+01:00"
```

### Calculate time until a future event

Convert a date string from a sensor and calculate how many hours remain until that event.

#### template

```
template: |
 {% set event = as_datetime(states("sensor.next_appointment")) %}
 {{ ((event - now()).total_seconds() / 3600) | round(1) }}
type: float
output: "3.5"
```

### Chain with as\_local

Convert a UTC datetime string to your local time zone by chaining `as_datetime` with `as_local`.

#### template

```
template: '{{ as_datetime("2024-03-15T13:30:00+00:00") | as_local }}'
type: datetime
output: "2024-03-15 14:30:00+01:00"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/As Function

---

The `as_function` template function takes a template macro and turns it into a reusable function that produces a value. A macro on its own can only output text, which limits what you can do with its result. Wrapping it with `as_function` gives you a proper function you can pass to `map`, `select`, `reject`, or store in a variable for later use.

You reach for `as_function` when you've written a piece of reusable logic (usually in a macro) and want to run it across a list of values, or use it as a custom test. For most templates, built-in filters and functions are enough. The macro you wrap must take a special `returns` argument. Inside the macro, you call `returns(value)` to say "this is what the function should hand back".

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: |

type: integer output: "10"

---

filter: |

type: callable output: ""

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
as_function(
 macro: Macro,
) -> Callable
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} macro: description: > A template macro that takes a `returns` argument. Inside the macro, call `returns(value)` with whatever you want the resulting function to hand back. required: true type: any

## Writing a compatible macro

---

The macro must accept a `returns` parameter. Call `returns()` with the value you want the function to produce. The macro name is conventionally prefixed with `macro_`; this prefix is stripped from the resulting function's name.

### template

```
template: |
 {% macro macro_add_ten(value, returns) %}
 {{ returns(value + 10) }}
 {% endmacro %}
 {% set add_ten = as_function(macro_add_ten) %}
 {{ add_ten(5) }}
title: Basic usage
type: integer
output: "15"
```



## Using with map

---

Once converted, the function works seamlessly with `map` to transform lists.

### template

```
template: |
 {% macro macro_format_temp(value, returns) %}
 {{ returns(value | round(1) ~ "°C") }}
 {% endmacro %}
 {% set format_temp = as_function(macro_format_temp) %}
 {{ [21.456, 19.8, 22.123] | map("apply", format_temp) | list }}
title: Format temperatures
type: list
output: "['21.5°C', '19.8°C', '22.1°C']"
```

## Good to know

---

- The macro must include a `returns` parameter and end by calling `returns(value)` .  
Without that call, the function produces `None` .
- The conventional `macro_` name prefix is stripped from the resulting function, so `macro_double` becomes `double` .

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Custom filter for select

Create a function that tests whether a value is within a range, then use it with `select` to filter a list.

#### template

```
template: |
 {% macro macro_in_range(value, low, high, returns) %}
 {{ returns(low <= value <= high) }}
 {% endmacro %}
 {% set in_range = as_function(macro_in_range) %}
 {{ [15, 22, 30, 18, 25] | select("apply", in_range, 20, 26) | list }}
type: list
output: "[22, 25]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/As Local

---

The `as_local` template function converts a datetime object to your local time zone, as configured in Home Assistant. If you have a datetime in UTC or any other time zone, `as_local` shifts it to your local time so it displays correctly for you.

Many entities in Home Assistant store their timestamps in UTC. When you display these values on a dashboard or use them in a notification, you usually want to show the time as it appears on your clock, not in UTC. `as_local` handles this conversion. For example, if a sensor reports its last update as 13:30 UTC and you live in CET (UTC+1), `as_local` converts it to 14:30. It also correctly handles daylight saving time changes.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: datetime output: "2024-03-15 14:30:00.123456+01:00"
```

---

```
filter: " type: datetime output: "2024-03-15 14:30:00.123456+01:00"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
as_local(
 value: datetime,
) -> datetime
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The datetime object to convert to the local time zone. Must be a datetime object, not a string. Use `as_datetime` first if you have a string.  
required: true type: datetime

## Converting UTC entity timestamps

---

Entity attributes like `last_changed` and `last_updated` are stored in UTC. Convert them to local time for display.

### template

```
template: '{{ states.binary_sensor.front_door.last_changed | as_local }}'
type: datetime
output: "2024-03-15 14:30:00.123456+01:00"
```

## Good to know

---

- The input must be a datetime object, not a string. Pipe through `as_datetime` first if you have a string.
- Naive datetimes (without time zone info) are treated as UTC before the conversion.
- Daylight saving transitions are handled automatically based on the Home Assistant configured time zone.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display last changed time in local format

Show when a sensor last changed, formatted in local time.

#### template

```
template: |
 {{ (states.sensor.temperature.last_changed | as_local).strftime("%H:%M") }}
type: string
output: "14:30"
```

### Convert a UTC datetime string to local time

First parse the string with `as_datetime`, then convert to local time.

#### template

```
template: '{{ as_datetime("2024-03-15T13:30:00+00:00") | as_local }}'
type: datetime
output: "2024-03-15 14:30:00+01:00"
```

### Show the local time of the next scheduled event

Convert a UTC event time from a sensor to your local time zone for display in a notification.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 Next appointment at
 {{
 (as_datetime(states("sensor.next_appointment")) | as_local)
 .strftime("%H:%M")
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/As Timedelta

---

The `as_timedelta` template function parses an ISO 8601 duration string and returns a Python timedelta object. Give it a string like "PT1H30M" (1 hour and 30 minutes) or "P2DT6H" (2 days and 6 hours), and it returns a timedelta you can use in time calculations.

Some integrations and external services provide durations in ISO 8601 format. To use these values in calculations, such as adding a duration to the current time or comparing how long something will take, you need to convert them into timedelta objects first. `as_timedelta` does this for you. If you need to create a timedelta from numeric values (days, hours, minutes) rather than parsing a string, use `timedelta` instead.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: timedelta output: "1:30:00"

---

filter: '{{ "PT1H30M" | as\_timedelta }}' type: timedelta output: "1:30:00"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
as_timedelta(
 value: str,
) -> timedelta | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > An ISO 8601 duration string to parse. Common formats include "PT1H" (1 hour), "PT30M" (30 minutes), "P1D" (1 day), and "P2DT6H30M" (2 days, 6 hours, 30 minutes). Returns `None` if the string cannot be parsed. required: true type: string

## ISO 8601 duration format

---

The ISO 8601 duration format uses **P** as a prefix, followed by date components and **T** to separate time components:

- **P1D** - 1 day
- **PT1H** - 1 hour
- **PT30M** - 30 minutes
- **PT45S** - 45 seconds
- **P2DT6H30M** - 2 days, 6 hours, 30 minutes

### template

```
template: '{{ as_timedelta("P2DT6H30M") }}'
type: timedelta
output: "2 days, 6:30:00"
```

## Good to know

---

- Returns `None` when the string cannot be parsed, so guard against missing data before arithmetic.
- Only the ISO 8601 duration format is accepted. A plain string like `"1h30m"` will not work. For numeric inputs, use `timedelta`.
- Year and month components ( `Y` , `M` before `T` ) are not supported because their length in seconds is not fixed.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Add a duration to the current time

Calculate when an event will end by adding a duration from a sensor to `now`.

#### template

```
template: '{{ now() + as_timedelta("PT2H30M") }}'
type: datetime
output: "2024-03-15 17:00:00.123456+01:00"
```

### Get total seconds from a duration

Convert an ISO 8601 duration string to a total number of seconds for use in numeric comparisons.

#### template

```
template: '{{ as_timedelta("PT1H30M").total_seconds() | int }}'
type: integer
output: "5400"
```

### Check if a remaining duration is short

Determine whether a timer or countdown is about to finish by comparing the duration to a threshold.

#### template

```
template: |
 {{
 as_timedelta(states("sensor.washer_remaining"))
 < as_timedelta("PT10M")
 }}
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/As Timestamp

---

The `as_timestamp` template function converts a datetime object or a date/time string into a UNIX timestamp. The result is a floating-point number representing the number of seconds since January 1, 1970 (the UNIX epoch).

UNIX timestamps are useful when you need to perform arithmetic with dates and times, since they are plain numbers that can be added, subtracted, and compared. For example, you can subtract two timestamps to find the number of seconds between two events, or add a number of seconds to find a future time. Many external services and APIs also expect timestamps in this format. You can convert the result back to a readable date using `timestamp_custom`, `timestamp_local`, or `as_datetime`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "1710510600.0"

---

filter: '{{ "2024-03-15T14:30:00+01:00" | as\_timestamp }}' type: float output: "1710510600.0"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
as_timestamp(
 value: datetime | str,
 default: Any = None,
) -> float | Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The datetime object or date/time string to convert. Strings must be in a recognized date/time format (for example, ISO 8601). required: true type: [string, datetime] default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. required: false type: any

## Converting the current time

---

You can convert `now` or `utcnow` to get the current UNIX timestamp.

### template

```
template: '{{ as_timestamp(now()) }}'
type: float
output: "1710510600.0"
```

## Using a default value

---

If the input might be invalid, provide a default to avoid errors.

### template

```
template: '{{ as_timestamp("not a date", default=0) }}'
type: float
output: "0"
```

## Good to know

---

- The result is a UNIX timestamp in seconds (a float), not milliseconds.
- Without a default, unparseable strings raise an error.
- Subtracting two timestamps gives a duration in seconds, which can be compared directly with numeric thresholds.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Seconds since an entity last changed

Calculate how many seconds have passed since a sensor last changed state.

#### template

```
template: |
 {{
 (as_timestamp(now())
 - as_timestamp(states.binary_sensor.front_door.last_changed))
 | int
 }}
type: integer
output: "3847"
```

### Check if something happened in the last 5 minutes

Determine whether the front door opened in the last 300 seconds.

#### template

```
template: |
 {{
 (as_timestamp(now())
 - as_timestamp(states.binary_sensor.front_door.last_changed))
 < 300
 }}
type: boolean
output: "true"
```

### Format a timestamp for display

Convert a datetime string to a UNIX timestamp and then format it using `timestamp_custom`.

#### template

```
template: |
 {{
 as_timestamp("2024-03-15T14:30:00+01:00")
 | timestamp_custom("%H:%M")
 }}
type: string
output: "14:30"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Asin

---

The `asin` template function returns the arc sine (inverse sine) of a value. The result is in radians, in the range  $[-\pi/2, \pi/2]$ . The input must be between -1 and 1.

This is useful when you need to convert a ratio back into an angle. For example, given a normalized sensor reading you could recover the original angle used to produce it.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "1.5707963267948966"

---

filter: '' type: float output: "1.5707963267948966"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
asin(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to calculate the arc sine of. Must be numeric and between -1 and 1. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is out of range or not numeric). If not provided, an error is raised instead. required: false type: any

## Good to know

---

- Inputs outside the range -1 to 1 raise an error unless you supply a default.
- The result is in radians. Multiply by `180 / pi` to convert to degrees.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Using a default value

If the input might be out of range or non-numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

#### template

```
template: '{{ asin(states("sensor.ratio"), default=0) }}'
type: float
output: "0"
```

### Convert arc sine to degrees

Convert the result from radians to degrees to make it more readable.

#### template

```
template: '{{ (asin(0.5) * 180 / pi) | round(1) }}'
type: float
output: "30.0"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Atan

---

The `atan` template function returns the arc tangent (inverse tangent) of a value. The result is in radians, in the range  $[-\pi/2, \pi/2]$ . Unlike `atan2`, this function takes a single value and cannot distinguish between quadrants.

This is useful for converting a slope or ratio back into an angle, for example when calculating an incline angle from rise-over-run sensor data.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "0.7853981633974483"

---

filter: '' type: float output: "0.7853981633974483"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
atan(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to calculate the arc tangent of. Must be numeric. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is not numeric). If not provided, an error is raised instead. required: false type: any

## Good to know

---

- Cannot tell quadrants apart because the sign information is lost. Use `atan2` when you need the correct angle in all four quadrants.
- The result is in radians. Multiply by `180 / pi` to convert to degrees.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Using a default value

If the input might not be numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

#### template

```
template: '{{ atan(states("sensor.slope"), default=0) }}'
type: float
output: "0"
```

### Convert arc tangent to degrees

Turn a slope ratio (rise over run) into an angle in degrees.

#### template

```
template: '{{ (atan(1) * 180 / pi) | round(1) }}'
type: float
output: "45.0"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Atan2

---

The `atan2` template function returns the four-quadrant arc tangent of  $y/x$  in radians. Unlike `atan`, which takes a single ratio, `atan2` takes two separate values ( $y$  and  $x$ ) and correctly determines the angle in all four quadrants. The result is in the range  $[-\pi, \pi]$ .

This is useful when you need to compute a true bearing or direction angle from two coordinate components. For example, you could calculate the angle to a point on a map, or determine wind direction from separate north-south and east-west wind speed sensors.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "0.7853981633974483"

---

filter: '{{ (1, 1) | atan2 }}' type: float output: "0.7853981633974483"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
atan2(
 y: Any,
 x: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} y: description: > The y-coordinate (vertical component). Must be numeric. required: true type: float x: description: > The x-coordinate (horizontal component). Must be numeric. required: true type: float default: description: > Value to return if the calculation fails (for example, if either input is not numeric). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If either input might not be numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{
 atan2(
 states("sensor.wind_ns") | float(0),
 states("sensor.wind_ew") | float(0),
 default=0
)
 }}
type: float
output: "0"
```

## Good to know

---

- The argument order is `y, x`, not `x, y`. Swapping them flips your angle.
- The result is in radians and covers all four quadrants, unlike `atan`.
- When `x` and `y` are both zero, the result is `0.0`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Passing values as a list

You can also pass the y and x values as a list or tuple, which is convenient when working with coordinate pairs.

#### template

```
template: |
 {{ atan2([1, 0]) }}
type: float
output: "1.5707963267948966"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Attr

---

The `attr` filter reads an attribute from an object by name, the same way a dot does. Writing `foo | attr("bar")` is exactly like writing `foo.bar`. The difference is that with `attr`, the attribute name can come from a variable or be built at the moment the template runs, instead of being written into the template as fixed text.

This is useful when you don't know in advance which attribute you need to read. You might store an attribute name in a variable and look it up on a state object, or loop over a list of attribute names and read each one in turn. For most everyday templates, a regular dot (`states.sensor.temperature.state`) is simpler and should be preferred.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "21.5"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
attr(
 value: Any,
 name: str,
) -> Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The object to read from. Most often a state object like `states.sensor.temperature`. required: true type: any name: description: > The attribute to read, as text. Common values include `state`, `entity_id`, `last_changed`, `last_updated`, or any attribute your entity reports (for example, `brightness` on a light or `temperature` on a climate entity). required: true type: string

## Using a variable name

---

Use a variable to decide which attribute to read from an object at runtime.

### template

```
template: |
 {% set field = "state" %}
 {{ states.sensor.temperature | attr(field) }}
type: string
output: "21.5"
```

## Good to know

---

- This reads object attributes, not entity state attributes. For entity attributes, use `state_attr` instead.
- When the attribute does not exist, it produces an undefined value, which pairs well with `| default(value)`.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Access entity attributes dynamically

Retrieve different attributes from an entity state object based on a variable.

#### template

```
template: |
 {% set prop = "last_changed" %}
 {{ states.light.living_room | attr(prop) }}
type: string
output: "2026-04-03 10:15:00+00:00"
```

### Iterate over multiple attributes

Loop over a list of attribute names and retrieve each one from a state object.

#### template

```
template: |
 {% set fields = ["entity_id", "state"] %}
 {% for field in fields %}
 {{ field }}: {{ states.sensor.temperature | attr(field) }}
 {% endfor %}
type: string
output: |
 entity_id: sensor.temperature
 state: 21.5
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Average

---

The `average` template function calculates the arithmetic mean of a collection of values. Give it a list of numbers and it returns their average. This is the same as adding all the values together and dividing by how many there are, but without having to do that math yourself.

This is useful whenever you have multiple sensors measuring the same thing and want a single combined value. For example, you might have temperature sensors in the living room, bedroom, and kitchen, and want to know the average temperature across your home. Or you might want to average humidity readings from several rooms to decide whether to turn on a dehumidifier. You can also use it to smooth out fluctuating sensor readings or calculate average energy usage over a set of devices.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "21.1"

---

filter: '{{ [21.5, 22.0, 19.8] | average }}' type: float output: "21.1"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
average(
 *args: list | float,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} values: description: > The values to average. Can be a list or multiple separate arguments. All values must be numeric. required: true type: list default: description: > Value to return if the calculation fails (for example, if the list is empty or contains non-numeric values). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the list might be empty or contain invalid values, provide a default to avoid errors. This prevents your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{
 [states("sensor.maybe_broken") | float(none)]
 | reject("none")
 | list
 | average(default=0)
 }}
type: float
output: "0"
```



## Good to know

---

- An empty list raises an error unless you supply a default.
- All items must be numeric. Convert state strings with `float` before averaging.
- This calculates the arithmetic mean. For the middle value, use `median` ; for the most common value, use `statistical_mode` .

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Average of sensor values

Calculate the average temperature across multiple rooms by passing each sensor value as a separate argument.

#### template

```
template: |
 {{
 average(
 states("sensor.living_room_temperature") | float,
 states("sensor.bedroom_temperature") | float,
 states("sensor.kitchen_temperature") | float
)
 }}
type: float
output: "21.4"
```

### Average across a group of entities

If you have a group of temperature sensors, you can expand the group and average all their values in one expression.

#### template

```
template: |
 {{
 expand("group.indoor_temperatures")
 | map(attribute="state")
 | map("float")
 | average
 }}
type: float
output: "21.4"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Base64 Decode

---

The `base64_decode` filter takes a base64-encoded string and decodes it back to its original form. By default, it returns a UTF-8 string, but you can set the encoding to `none` to get raw bytes instead.

This is useful when you receive base64-encoded data from external services or sensors. For example, some MQTT messages include base64-encoded payloads, certain APIs return data in base64 format, or you might receive encoded credentials that need to be decoded. The optional `encoding` parameter lets you control how the decoded bytes are interpreted. Set it to `none` when working with binary data that is not a text string.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "SGVsbG8sIFdvcmxklQ==" | base64_decode }}' type:
string output: "Hello, World!"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | base64_decode(
 encoding: str | None = "utf-8",
) -> str | bytes
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The base64-encoded string to decode. required: true type: string encoding: description: > The character encoding to use when converting the decoded bytes to a string. Defaults to `utf-8`. Set to `none` to return raw bytes instead of a string. required: false default: "utf-8" type: string



## Decoding to raw bytes

---

Set `encoding` to `none` to get the raw bytes instead of a string. This is useful when the encoded data is not text.

### template

```
template: '{{ "AQIDBA==" | base64_decode(encoding=none) }}'
type: bytes
output: "b'\\x01\\x02\\x03\\x04'"
```

## Good to know

---

- By default, the decoded bytes are interpreted as UTF-8. Non-UTF-8 binary data raises an error unless you pass `encoding=None`.
- Setting `encoding=None` returns raw bytes, which display with a `b'...'` prefix and cannot be concatenated with strings directly.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Decode an MQTT payload

Decode a base64-encoded message received from an MQTT topic.

#### template

```
template: '{{ states("sensor.mqtt_encoded") | base64_decode }}'
type: string
output: "Temperature: 21.5C"
```

### Decode and parse JSON

Decode a base64 string that contains JSON data, then parse it.

#### template

```
template: |
 {% set decoded = "eyJ0ZWlwIjogMjEuNX0=" | base64_decode %}
 {{ decoded | from_json }}
type: dict
output: "{{ 'temp': 21.5 }}"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Base64 Encode

---

The `base64_encode` filter converts a string or bytes value into its base64-encoded representation. Base64 encoding transforms binary data into a text format that uses only printable ASCII characters, making it safe to include in places that only support text.

This is useful when you need to send binary data or credentials through text-only channels. For example, some REST APIs require authentication headers to be base64-encoded, certain MQTT payloads need binary data encoded as text, or you might need to embed image data in a notification. The filter accepts both strings and raw bytes, encoding them into a standard base64 string.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Hello, World!" | base64_encode }}' type: string output:
"SGVsbG8sIFdvcmxklQ=="
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | base64_encode() -> str
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The string or bytes value to encode. Strings are first converted to UTF-8 bytes before encoding. required: true type: string

## Good to know

---

- Strings are encoded as UTF-8 before being base64-encoded.
- The result includes `=` padding characters when the input length is not a multiple of three.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Create a basic auth header

Build an HTTP Basic Authentication header by base64-encoding credentials.

#### template

```
template: '{{ ("username:password") | base64_encode }}'
type: string
output: "dXNlcm5hbWU6cGFzc3dvcmQ="
```

### Encode sensor data for transmission

Encode a sensor value for inclusion in a base64-only payload.

#### template

```
template: '{{ states("sensor.temperature") | base64_encode }}'
type: string
output: "MjEuNQ=="
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Batch

---

The `batch` filter splits a list into smaller sub-lists (batches) of a given size. If the last batch has fewer items than the requested size, you can optionally provide a fill value to pad it.

This is useful when you need to process or display items in fixed-size groups. For example, you might want to display entities in rows of three on a dashboard, create notification messages that include a limited number of items per message, or iterate through a large list of devices in manageable chunks.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [1, 2, 3, 4, 5] | batch(2) | list }}' type: list output: "[[1, 2], [3, 4], [5]]"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
batch(
 value: list,
 linecount: int,
 fill_with: Any = None,
) -> list[list]
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list to split into batches. required: true type: list  
linecount: description: > The number of items per batch. required: true type: integer  
fill\_with: description: > A value to use for padding the last batch if it has fewer items than the batch size. If not provided, the last batch may be shorter than the others. required: false type: any

## Padding the last batch

---

Use the `fill_with` parameter to pad the last batch so all batches have the same number of items.

### template

```
template: '{{ [1, 2, 3, 4, 5] | batch(3, "N/A") | list }}'
type: list
output: '[[1, 2, 3], [4, 5, "N/A"]]'
```

## Good to know

---

- Returns a generator, so add `| list` before using it with `length` or iterating twice.
- Without `fill_with`, the last batch can be smaller than the requested size.
- The batch size must be a positive integer. Passing zero raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display entities in rows

Split a list of entity states into rows of three for a formatted display.

#### template

```
template: |
 {% set lights = ["light.kitchen", "light.bedroom", "light.hall",
 "light.porch", "light.garage"] %}
 {% for row in lights | batch(3) %}
 Row {{ loop.index }}: {{ row | join(", ") }}
 {% endfor %}
type: string
output: |
 Row 1: light.kitchen, light.bedroom, light.hall
 Row 2: light.porch, light.garage
```

### Process devices in chunks

Iterate through a large list of items in manageable batches.

#### template

```
template: |
 {% set items = range(10) | list %}
 {% for chunk in items | batch(4) %}
 Batch {{ loop.index }}: {{ chunk | join(", ") }}
 {% endfor %}
type: string
output: |
 Batch 1: 0, 1, 2, 3
 Batch 2: 4, 5, 6, 7
 Batch 3: 8, 9
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Bitwise And

---

The `bitwise_and` template function performs a bitwise AND operation on two integer values. Each bit in the result is set to 1 only if both corresponding bits in the input values are 1.

This is useful when working with devices that communicate status or configuration using bitmasks. For example, some sensors report multiple flags packed into a single integer value, and you can use `bitwise_and` to check whether a specific flag bit is set. It is also helpful for masking out specific bits from a register value.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: int output: "3"
```

---

```
filter: '' type: int output: "3"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
bitwise_and(
 first_value: Any,
 second_value: Any,
) -> int
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} first\_value: description: > The first integer value for the AND operation. required: true type: integer second\_value: description: > The second integer value for the AND operation. required: true type: integer

## Good to know

---

- Both inputs must be integers. Convert state strings with `int` first.
- To test whether a specific bit is set, compare the result to the bit mask or to zero.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Checking a status bit

Check if bit 2 (value 4) is set in a sensor's status register.

#### template

```
template: |
 {% set status = states("sensor.device_status") | int %}
 {{ bitwise_and(status, 4) > 0 }}
type: boolean
output: "True"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Bitwise Or

---

The `bitwise_or` template function performs a bitwise OR operation on two integer values. Each bit in the result is set to 1 if at least one of the corresponding bits in the input values is 1.

This is useful when you need to combine multiple status flags into a single value or set specific bits in a bitmask. For example, if different sensors each report a single flag bit and you want to combine them into a single status register, `bitwise_or` lets you merge them together.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: int output: "7"

---

filter: '' type: int output: "7"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
bitwise_or(
 first_value: Any,
 second_value: Any,
) -> int
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} first\_value: description: > The first integer value for the OR operation. required: true type: integer second\_value: description: > The second integer value for the OR operation. required: true type: integer

## Good to know

---

- Both inputs must be integers. Convert state strings with `int` first.
- Binary literals like `0b0010` work and can make bit flags more readable.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Combining status flags

Combine two flag values into a single bitmask.

#### template

```
template: |
 {% set flag_a = 0b0010 %}
 {% set flag_b = 0b0100 %}
 {{ bitwise_or(flag_a, flag_b) }}
type: int
output: "6"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Bitwise Xor

---

The `bitwise_xor` template function performs a bitwise XOR (exclusive OR) operation on two integer values. Each bit in the result is set to 1 only if exactly one of the corresponding bits in the input values is 1.

This is useful for toggling specific bits in a bitmask or comparing two status values to find which bits differ. For example, you could compare the current and previous status registers of a device to detect which flags have changed.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: int output: "6"
```

---

```
filter: '' type: int output: "6"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
bitwise_xor(
 first_value: Any,
 second_value: Any,
) -> int
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} first\_value: description: > The first integer value for the XOR operation. required: true type: integer second\_value: description: > The second integer value for the XOR operation. required: true type: integer

## Good to know

---

- Both inputs must be integers. Convert state strings with `int` first.
- XOR-ing a value with itself produces zero, which is a quick way to detect whether two values are identical.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Detecting changed bits

Compare two status values to find which bits changed.

#### template

```
template: |
 {% set old_status = 0b1010 %}
 {% set new_status = 0b1100 %}
 {{ bitwise_xor(old_status, new_status) }}
type: int
output: "6"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Bool

---

The `bool` template function converts a value to a boolean (`true` or `false`). It recognizes common truthy values like `true`, `yes`, `on`, `enable`, and `1`, as well as falsy values like `false`, `no`, `off`, `disable`, and `0`. If the value cannot be recognized as either, it returns the default you provide instead of raising an error.

This is useful when working with entity states or attributes that represent on/off or yes/no values as strings. For example, some sensors report `on` or `off` as their state, and you might need to convert that to a proper boolean for use in a condition or calculation. The `bool` function understands all the common representations of true and false that appear throughout Home Assistant.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: boolean output: "true"

---

filter: '{{ "off" | bool }}' type: boolean output: "false"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
bool(
 value: Any,
 default: Any = _SENTINEL,
) -> bool | Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to convert to a boolean. Recognizes `true / false`, `yes / no`, `on / off`, `enable / disable`, `1 / 0`. Other values raise an error unless a default is provided. required: true type: any default: description: > Value to return if the conversion fails. If not provided, an error is raised on unrecognized input. required: false type: any

## Recognized values

---

The filter converts the following values to `true`: `true`, `yes`, `on`, `enable`, `1`.

It converts the following values to `false`: `false`, `no`, `off`, `disable`, `0`.

All string comparisons are case-insensitive, so `True`, `TRUE`, and `true` all work.

## Using a default value

---

If the input might be an unexpected value, provide a default to prevent errors.

### template

```
template: '{{ bool("maybe", default=false) }}'
title: Unrecognized value with default
type: boolean
output: "false"
```

## Good to know

---

- Only specific words are recognized as boolean. `"unavailable"` or `"unknown"` are not, so pass a default when reading sensor states.
- Comparisons are case-insensitive, so `"YES"`, `"Yes"`, and `"yes"` all convert to `true`.
- Integers other than `0` and `1` are not recognized and raise an error unless a default is given.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Convert a state for use in a condition

Check whether a switch-like sensor reports a truthy value.

#### template

```
template: |
 {% if states("sensor.night_mode") | bool(false) %}
 Night mode is active
 {% else %}
 Normal mode
 {% endif %}
type: string
output: "Night mode is active"
```

### Use with iif for display text

Combine `bool` with `iif` to convert a state into a human-readable label.

#### template

```
template: |
 {{
 states("input_boolean.guest_mode") | bool(false)
 | iif("Guests expected", "No guests")
 }}
type: string
output: "Guests expected"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Boolean Test

---

The `boolean` test checks whether a value is a boolean ( `true` or `false` ). It returns `true` if the value is of boolean type and `false` for any other type, including strings like `true` or integers like `1` .

This is useful when you need to verify the type of a value before processing it. In Home Assistant, entity states are always strings, but attributes can be actual booleans. If you need to distinguish between a real boolean and a string that looks like one, this test lets you check the actual type.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is a boolean
```

```
type: string output: "It is a boolean"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
boolean(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is of boolean type ( `true` or `false` ). required: true type: any

## Good to know

---

- Only the actual boolean values `true` and `false` pass this test. Strings like `"true"` and integers like `1` do not.
- Entity states are always strings, so this test will never succeed on a raw `states(...)` result.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distinguish boolean from string

Check whether a value is actually a boolean and not a string representation.

#### template

```
template: |
 {{ true is boolean }}
 {{ "true" is boolean }}
type: boolean
output: |
 true
 false
```

### Validate an attribute type

Verify that an entity attribute is a real boolean before using it directly.

#### template

```
template: |
 {% set val = state_attr("switch.device", "is_locked") %}
 {% if val is boolean %}
 Lock state: {{ val }}
 {% else %}
 Unexpected type
 {% endif %}
type: string
output: "Lock state: True"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Callable

---

The `callable` test checks whether a value is callable, meaning it can be invoked as a function. It returns `true` for functions, methods, and other callable objects, and `false` for regular values like strings, numbers, and lists.

In typical Home Assistant template usage, you rarely need to check if something is callable. However, this test can be useful in templates where you work with template macros or need to verify that a value is a function before attempting to call it.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | range is callable
```

type: string output: "range is callable"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
callable(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value can be called as a function. required: true type: any

## Good to know

---

- Most template values are not callable. This test is primarily useful when working with macros wrapped via `as_function`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various values

Functions are callable, but regular values are not.

#### template

```
template: |
 {{ range is callable }}
 {{ "hello" is callable }}
 {{ 42 is callable }}
type: boolean
output: |
 true
 false
 false
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Capitalize

---

The `capitalize` filter converts the first character of a string to uppercase and makes all remaining characters lowercase. This is useful when you want to ensure a consistent sentence-case format for display purposes. For example, you might want to normalize a sensor state like `OPEN` or `closed` to a cleaner `Open` or `Closed` format for use in notifications or dashboard cards.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "hello WORLD" | capitalize }}' type: string output: Hello world
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
capitalize(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to capitalize. The first character is uppercased and the rest are lowercased. required: true type: string

## Good to know

---

- Only the very first character is uppercased. The rest of the string is always lowercased, even if it contained capitals before.
- Multi-word values become sentence case, not title case. Use `title` when you want every word capitalized.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Normalize a sensor state for display

Clean up an entity state so it looks presentable in a notification.

#### template

```
template: |
 {{ "The door is " ~ states("binary_sensor.front_door") | capitalize }}
type: string
output: "The door is Off"
```

### Format a name from user input

Normalize a name entered through a text helper to sentence case.

#### template

```
template: '{{ states("input_text.greeting") | capitalize }}'
type: string
output: "Hello world"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Center

---

The `center` filter centers a string within a field of a specified width by padding it with spaces on both sides. If the string is already longer than the given width, it is returned unchanged. This can be useful when you need to align text for fixed-width displays or when building formatted text output. For example, you might want to center a title in a notification or format text for a display that expects a fixed number of characters.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "hello" | center(20) }}' type: string output: " hello "
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
center(
 value: str,
 width: int = 80,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to center within the field. required: true type: string width: description: > The total width of the field. The string is padded with spaces on both sides to fill this width. Defaults to `80`. required: false default: "80" type: integer

## Good to know

---

- The default width is 80 characters, not the length of the input.
- If the string is already as long or longer than the width, it is returned unchanged with no padding or truncation.
- When the remaining space is odd, one extra space is added on the right side.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Center a heading in formatted text

Create a centered heading within a block of formatted text for a notification.

#### template

```
template: '{{ "hello" | center(20) }}'
type: string
output: ' hello '
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Clamp

---

The `clamp` template function constrains a value to lie within a given range. If the value is below the minimum it returns the minimum; if it is above the maximum it returns the maximum; otherwise it returns the value unchanged. The range is inclusive on both ends.

This is useful whenever you need to ensure a sensor value stays within safe bounds before using it in an automation. For example, you could clamp a brightness percentage to a range your light supports, limit a thermostat setpoint to a safe temperature range, or ensure a volume level never exceeds a certain maximum.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "100.0"
```

---

```
filter: '' type: float output: "100.0"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
clamp(
 value: Any,
 min_value: Any,
 max_value: Any,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to constrain. Must be numeric. required: true type: float min\_value: description: > The minimum allowed value (inclusive lower bound). Must be numeric. required: true type: float max\_value: description: > The maximum allowed value (inclusive upper bound). Must be numeric. required: true type: float

## Good to know

---

- Both bounds are inclusive, so a value equal to the minimum or maximum is returned unchanged.
- The result is always a float, even when all inputs are integers.
- If `min_value` is greater than `max_value`, the result will always be `min_value`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Limiting a thermostat setpoint

Ensure a calculated setpoint stays within a safe range.

#### template

```
template: |
 {% set desired = states("sensor.target_temperature") | float %}
 {{ clamp(desired, 16, 28) }}
type: float
output: "22.0"
```

### Clamping brightness

Keep a brightness value within the 0-255 range that a light expects.

#### template

```
template: |
 {% set raw = states("sensor.ambient_light") | float * 2.5 %}
 {{ clamp(raw, 0, 255) }}
type: float
output: "255.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Closest

---

The `closest` template function finds the entity that is geographically closest to a given location. By default, it compares against your home location, but you can also specify coordinates or another entity as the reference point.

This is useful for location-based automations. For example, you could find out which family member is closest to home, which store is nearest to your current position, or which of your device trackers is closest to a specific zone. The function returns a full state object, so you can access the entity's name, state, attributes, and ID.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: " type: string output: Phone

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
closest(
 *args: str | State | list,
) -> State | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} args: description: > Can be called in several ways. With just entities or groups, it finds the closest to home. With coordinates or a zone followed by entities, it finds the closest to that point. Returns a state object, or `None` if no entity has location data. required: true type: any

## Different ways to call closest

---

Find closest to home:

### template

```
template: '{{ closest(states.device_tracker).name }}'
title: Closest device tracker to home
type: string
output: Phone
```

Find closest to a specific zone:

### template

```
template: '{{ closest("zone.work", states.device_tracker).name }}'
title: Closest device tracker to work
type: string
output: Tablet
```

Find closest to specific coordinates:

### template

```
template: '{{ closest(52.5, 13.4, states.device_tracker).name }}'
title: Closest to coordinates
type: string
output: Phone
```

## Good to know

---

- Returns `None` when no entity has usable GPS coordinates, so check before accessing `.name` or other attributes.
- Only entities with latitude and longitude attributes are considered. Entities without coordinates are silently skipped.
- Returns a full state object, not just an entity ID or name.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Show who is closest to home and their distance

Combine `closest` with `distance` to show both who is nearest and how far away they are.

#### template

```
template: |
 {% set nearest = closest(states.person) %}
 {{ nearest.name }} is closest at
 {{ distance(nearest) | round(1) }} km away
type: string
output: Paulus is closest at 2.3 km away
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Combine

---

The `combine` template function merges multiple dictionaries into a single dictionary. When the same key appears in more than one dictionary, the value from the last dictionary wins. This lets you build up complex data structures by combining smaller pieces.

This is useful when you need to assemble data from multiple sources into a single dictionary. For example, you might want to combine default settings with overrides, merge attributes from several entities into one payload, or build up a notification data dictionary from multiple parts. The optional `recursive` parameter enables deep merging, where nested dictionaries are merged rather than replaced, which is helpful when working with deeply structured data.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '{{ combine({"a": 1}, {"b": 2}, {"c": 3}) }}' type: dict
output: '{"a": 1, "b": 2, "c": 3}'
```

---

```
filter: '{{ {"a": 1} | combine({"b": 2}) }}' type: dict output: '{"a": 1, "b": 2}'
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
combine(
 *args: dict,
 recursive: bool = False,
) -> dict
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} args: description: > One or more dictionaries to merge. When keys overlap, later dictionaries take precedence. required: true type: map recursive: description: > If `true`, nested dictionaries are merged recursively instead of being replaced. Defaults to `false`. required: false default: "false" type: boolean

## Key overlap behavior

---

When the same key appears in multiple dictionaries, the value from the last dictionary wins.

### template

```
template: '{{ combine({"color": "red", "size": "large"}, {"color": "blue"}) }}'
type: dict
output: '{"color": "blue", "size": "large"}'
```

## Recursive merging

---

With `recursive=true`, nested dictionaries are merged together instead of one replacing the other.

### template

```
template: |
 {{
 combine(
 {"settings": {"brightness": 100, "color": "warm"}},
 {"settings": {"color": "cool", "mode": "auto"}},
 recursive=true
)
 }}
type: dict
output: '{"settings': {'brightness': 100, 'color': 'cool', 'mode': 'auto'}}"
```

## Good to know

---

- When the same key appears in multiple dictionaries, the value from the later dictionary wins.
- Without `recursive=true`, nested dictionaries are replaced outright, not merged.
- The original dictionaries are not modified; a new dictionary is returned.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build a notification payload

Combine default notification settings with dynamic content.

#### action

```
action: |
 action:
 - action: notify.mobile
 data: >
 {{
 combine(
 {"title": "Home Assistant", "priority": "normal"},
 {"message": "Front door is " ~ states("binary_sensor.front_door")}
)
 }}
```

### Merge settings with overrides

Start with default settings and override specific values.

#### template

```
template: |
 {% set defaults = {"mode": "auto", "brightness": 128, "color_temp": 300} %}
 {% set overrides = {"brightness": 255} %}
 {{ combine(defaults, overrides) }}
type: dict
output: '{"mode": "auto", "brightness": 255, "color_temp": 300}'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Config Entry Attr

---

The `config_entry_attr` template function returns a specific attribute from a config entry, identified by its config entry ID. You can retrieve attributes like `domain`, `title`, `state`, `source`, `disabled_by`, and `pref_disable_polling`. It returns `None` if the config entry does not exist.

This is useful for inspecting the configuration behind your entities. For example, you might want to check if a config entry is still loaded and working, find out which integration domain it belongs to, or display the title of the config entry in a notification. Combined with `config_entry_id`, you can trace any entity back to its config entry and read details about it.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "Living Room Hue Bridge"

---

filter: '{{ "01234567890abcdef01234567890abcd" | config\_entry\_attr("title") }}' type: string output: "Living Room Hue Bridge"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
config_entry_attr(
 config_entry_id: str,
 attr_name: str,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} `config_entry_id`: description: > The config entry ID to look up. You can get this from `config_entry_id`. required: true type: string `attr_name`: description: > The attribute to retrieve. Must be one of: `domain`, `title`, `state`, `source`, `disabled_by`, or `pref_disable_polling`. required: true type: string

## Available attributes

---

The following attributes can be retrieved from a config entry:

- `domain`: The integration domain (for example, `hue`, `zwave_js`).
- `title`: The user-defined title of the config entry.
- `state`: The current state of the config entry (for example, `loaded`, `setup_error`, `not_loaded`).
- `source`: How the config entry was created (for example, `user`, `discovery`).
- `disabled_by`: Why the config entry is disabled, or `None` if it is not disabled.
- `pref_disable_polling`: Whether polling has been disabled for this config entry.



## Good to know

---

- Returns `None` when the config entry does not exist or when the attribute name is not one of the supported ones.
- Only the fixed attribute names listed above are allowed. Arbitrary field names are rejected.
- The input is the config entry ID, not an entity ID. Resolve from an entity with `config_entry_id` first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check the state of a config entry from an entity

Look up the config entry behind an entity and check its state to see if the integration is running.

#### template

```
template: |
 {% set entry_id = config_entry_id("light.living_room") %}
 {{ config_entry_attr(entry_id, "state") }}
type: string
output: "loaded"
```

### Get the integration domain for an entity

Find out which integration is responsible for a specific entity.

#### template

```
template: |
 {% set entry_id = config_entry_id("sensor.power_meter") %}
 {{ config_entry_attr(entry_id, "domain") }}
type: string
output: "shelly"
```

### Check if a config entry was discovered automatically

Determine whether a config entry was set up manually or discovered automatically.

#### template

```
template: |
 {% set entry_id = config_entry_id("light.bedroom") %}
 {{ config_entry_attr(entry_id, "source") }}
type: string
output: "discovery"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Config Entry Id

---

The `config_entry_id` template function returns the config entry ID that an entity belongs to. Every entity that is created by an integration is tied to a config entry, and this function lets you look up that connection. It returns `None` if the entity does not exist in the registry.

This is useful when you need to look up details about the config entry behind an entity using `config_entry_attr`. For example, you might want to find out which integration domain an entity came from, check whether the config entry is still active, or group entities by their config entry. It is also helpful for debugging, letting you trace an entity back to the configuration that created it.

# Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: "" type: string output:
"01234567890abcdef01234567890abcd"
```

---

```
filter: '{{ "light.living_room" | config_entry_id }}' type: string output:
"01234567890abcdef01234567890abcd"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
config_entry_id(
 entity_id: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: > The entity ID to look up. Returns the config entry ID as a string, or `None` if the entity does not exist in the registry. required: true type: string



## Good to know

---

- Returns `None` for helper entities and YAML-configured entities, which have no config entry.
- Returns `None` when the entity does not exist in the registry yet, which can happen right after creation.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find the integration domain for an entity

Combine `config_entry_id` with `config_entry_attr` to find out which integration domain an entity belongs to.

#### template

```
template: |
 {{ config_entry_attr(config_entry_id("light.living_room"), "domain") }}
type: string
output: "hue"
```

### Check if an entity's config entry is loaded

Verify that the config entry behind an entity is in the `loaded` state, which means the integration is running normally.

#### template

```
template: |
 {{
 config_entry_attr(
 config_entry_id("sensor.power_meter"), "state"
)
 }}
type: string
output: "loaded"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Contains

---

The `contains` template function checks whether one value is found inside another. It works with strings (substring search), lists (membership check), and dictionaries (key lookup). It uses Python's `in` operator under the hood.

This is useful when you need to check if a sensor value includes a specific word, if a list of items includes a particular entry, or if a dictionary has a certain key. For example, you might want to check if a weather condition string contains "rain", if a list of active devices includes a specific one, or filter entities whose states contain a particular substring.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "hello world" | contains("world") }}' type: boolean output: "true"
```

---

test: | Found it

type: string output: "Found it"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
contains(
 value: str | list | dict,
 item: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to search in. Can be a string (checks for substring), a list (checks for membership), or a dictionary (checks for key). required: true type: any item: description: > The value to search for within the first value. required: true type: any



## Different container types

---

The `contains` function adapts to the type of value being searched.

### template

```
template: |
 String: {{ "automation running" | contains("running") }}
 List: {{ [1, 2, 3] | contains(2) }}
 Dict: {{ {"name": "test"} | contains("name") }}
title: Works with strings, lists, and dicts
type: string
output: |
 String: true
 List: true
 Dict: true
```

## Using as a test with select

---

Use `contains` as a test to filter lists of values.

### template

```
template: |
 {{
 ["sunny", "rainy day", "cloudy", "light rain"]
 | select("contains", "rain") | list
 }}
title: Filter strings containing "rain"
type: list
output: "['rainy day', 'light rain']"
```

## Good to know

---

- On dictionaries, this checks keys, not values. Use `value in my_dict.values()` when you need to match a value.
- String matching is case-sensitive. Pipe through `| lower` first to make it case-insensitive.
- Unlike `in` as a test, the argument order here is container first, item second.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if a weather condition includes rain

Test the weather state to decide whether to send a reminder to bring an umbrella.

#### template

```
template: |
 {% if states("weather.home") | contains("rain") %}
 Don't forget your umbrella!
 {% else %}
 No rain expected
 {% endif %}
type: string
output: "Don't forget your umbrella!"
```

### Filter entities by state content

Find all sensors whose state contains a specific keyword.

#### template

```
template: |
 {{
 ["online", "offline", "online - active", "standby"]
 | select("contains", "online") | list
 }}
title: Find online items
type: list
output: "['online', 'online - active']"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Cos

---

The `cos` template function returns the cosine of a value given in radians. It wraps Python's `math.cos`, so the input is expected to be an angle measured in radians rather than degrees.

This is useful for trigonometric calculations based on sensor data, such as computing horizontal distance from an angle, projecting solar elevation into usable values, or creating smooth cyclic patterns in automations.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "1.0"
```

---

```
filter: '' type: float output: "1.0"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
cos(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The angle in radians to calculate the cosine of. Must be numeric. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is not numeric). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the input value might not be numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

### **template**

```
template: |
 {{ cos(states("sensor.angle"), default=1) }}
type: float
output: "1"
```

## Good to know

---

- The input must be in radians, not degrees. Multiply by `pi / 180` to convert degrees to radians.
- Due to floating-point precision, `cos(pi / 2)` returns a very small number instead of exactly zero.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Cosine of a known angle

Calculate the cosine of 60 degrees by first converting to radians.

#### template

```
template: |
 {{ cos(60 * (pi / 180)) }}
type: float
output: "0.5"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Cycler

---

The `cycler` template function creates an object that cycles through a sequence of values. You initialize it with the values you want to cycle through, then call `.next()` each iteration to advance to the next value. The `.current` property holds the current value, and the `.reset()` method returns to the beginning.

This can be handy when you need to alternate between values in a loop. For example, you could alternate between two labels, rotate through a set of icons, or cycle through status indicators. While less commonly used in typical Home Assistant templates, it can simplify patterns where you need repeating sequences.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: |

type: string output: | odd even odd even

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
cycler(
 *items: Any,
) -> Cycler
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} items: description: > Two or more values to cycle through. The cycler advances to the next value each time `.next()` is called and wraps around when it reaches the end. required: true type: any

## Good to know

---

- Needs at least two values. Calling it with one item or none raises an error.
- `.current` peeks at the current value without advancing; `.next()` moves to the next one.
- The cycler is tied to the template run. A fresh call to the template starts from the first value again.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Alternate labels in a list

Cycle through labels while iterating over items.

#### template

```
template: |
 {% set priority = cycler("high", "medium", "low") %}
 {% set rooms = ["Living Room", "Kitchen", "Bedroom"] %}
 {% for room in rooms %}
 {{ room }}: {{ priority.next() }}
 {% endfor %}
type: string
output: |
 Living Room: high
 Kitchen: medium
 Bedroom: low
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Default

---

The `default` filter returns a fallback value when the variable it is applied to is undefined. If the optional `boolean` parameter is set to `true`, it also replaces values that are falsy (empty strings, `0`, `false`, `none`, and empty collections). This is one of the most commonly used template filters.

In Home Assistant templates, you frequently work with variables that may or may not exist depending on the context. For instance, trigger variables are only available when the automation actually fires, and custom variables from `{% jinja %}{% endjinja %}` blocks may be conditionally defined. The `default` filter lets you write robust templates that always produce a usable value, even when some inputs are missing or empty.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "fallback"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | default(
 default_value: Any = "",
 boolean: bool = false,
) -> Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} default\_value: description: > The value to return if the input is undefined (or falsy, when `boolean` is `true`). Defaults to an empty string. required: false default: "" type: any boolean: description: > If `true`, the filter also replaces falsy values (`false`, `0`, `none`, empty strings, and empty collections) with the default. If `false` (the default), only undefined variables are replaced. required: false default: "false" type: boolean

## Handling undefined variables

---

When a variable has not been defined, the `default` filter returns the fallback value instead of raising an error.

### template

```
template: '{{ undefined_var | default("no value") }}'
title: Undefined variable gets fallback
type: string
output: "no value"
```

## Replacing falsy values with boolean mode

---

When `boolean` is set to `true`, the filter also replaces empty strings, `0`, `false`, `none`, and empty lists.

### template

```
template: '{{ "" | default("empty!", true) }}'
title: Empty string replaced in boolean mode
type: string
output: "empty!"
```

### template

```
template: '{{ 0 | default(42, true) }}'
title: Zero replaced in boolean mode
type: integer
output: "42"
```

## Good to know

---

- Without `boolean=true`, the default only kicks in for undefined variables. `None`, empty strings, and `0` pass through unchanged.
- The default value for `default_value` is an empty string, not `None`.
- This does not catch `unavailable` or `unknown` entity states, since those are defined strings. Combine with `has_value` or `float` for those cases.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Safe access to trigger data

Trigger variables are only available when the automation fires. Use `default` to provide a fallback when testing templates outside an automation context.

#### template

```
template: '{{ trigger.to_state.state | default("unknown") }}'
title: Safely access trigger state
type: string
output: "unknown"
```

### Provide a default sensor value

When a sensor might not have a specific attribute, provide a sensible default.

#### template

```
template: |
 {{
 state_attr("climate.living_room", "preset_mode")
 | default("none")
 }}
type: string
output: "comfort"
```

### Chain with other filters

The `default` filter is often combined with `float` or `int` to ensure calculations do not fail.

#### template

```
template: |
 {{ states("sensor.temperature") | default("0") | float(0) + 5 }}
type: float
output: "26.5"
```

### Use in an automation action

Set a notification message with a fallback greeting when the name is not set.

#### action



```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 Hello {{ states("input_text.user_name") | default("friend") }}!
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Defined

---

The `defined` test checks whether a variable has been defined in the current template context. It returns `true` if the variable exists and `false` if it is undefined.

This test is commonly used to guard against errors when working with variables that may or may not be present. For example, trigger variables are only available inside an automation context, and optional variables passed to scripts may not always be set. Testing with `defined` before accessing a variable prevents "undefined" errors and lets you branch your logic accordingly.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: | Variable exists:

type: string output: "Variable exists: hello"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
defined(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The variable to test. Returns `true` if the variable is defined in the current context. required: true type: any

## Good to know

---

- Checks whether a variable exists in the current context, not whether it has a non-empty value.
- A variable set to `None` passes this test. Use `is not none` to filter those out too.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Guard trigger variable access

Only access trigger data when it is available.

#### template

```
template: |
 {% if trigger is defined %}
 Triggered by: {{ trigger.entity_id }}
 {% else %}
 No trigger context
 {% endif %}
type: string
output: "No trigger context"
```

### Check for optional script variables

Handle optional variables in a script gracefully.

#### template

```
template: |
 {% if message is defined %}
 {{ message }}
 {% else %}
 Default notification text
 {% endif %}
type: string
output: "Default notification text"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Device Attr

---

The `device_attr` template function returns the value of a specific attribute from a device in the device registry. You can pass either a device ID or an entity ID along with the attribute name you want to retrieve. If the device or attribute doesn't exist, it returns `None`.

This is useful when you need device-level information that isn't available through entity states or attributes. For example, you might want to check the manufacturer, model, software version, or serial number of a device. You could use this to track which devices need firmware updates, build a hardware inventory, or include model information in a support request notification. Common attribute names include `manufacturer`, `model`, `sw_version`, `hw_version`, `serial_number`, and `identifiers`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output: "Philips"
```

---

```
filter: '{{ "a1b2c3d4e5f6a1b2c3d4e5f6" | device_attr("manufacturer") }}' type: string output:
"Philips"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
device_attr(
 device_or_entity_id: str,
 attr_name: str,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} `device_or_entity_id`: description: > The device ID or entity ID to look up. When using an entity ID, the function first resolves the device the entity belongs to. required: true type: string `attr_name`: description: > The name of the device attribute to retrieve. Common attributes include `manufacturer`, `model`, `sw_version`, `hw_version`, `serial_number`, and `identifiers`. required: true type: string

 **Tip:** To see all available device attributes, go to **Developer Tools > States** and look at the device registry entries, or use the **Devices** page in **Settings > Devices & services** to inspect a specific device.

## Good to know

---

- Returns `None` when the device or attribute does not exist, so chain with `| default(value)` for display fallbacks.
- This reads from the device registry, not the entity state. Attributes like `manufacturer` and `sw_version` live here, not in `state_attr`.
- Accepts either a device ID or an entity ID. When given an entity ID, it resolves to the device first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get the model of a device from an entity ID

You can pass an entity ID directly. The function automatically resolves the device behind the entity.

#### template

```
template: '{{ device_attr("light.living_room", "model") }}'
type: string
output: "LCA001"
```

### Check the firmware version of a device

Read the software version to see if a device is up to date.

#### template

```
template: '{{ device_attr("a1b2c3d4e5f6a1b2c3d4e5f6", "sw_version") }}'
type: string
output: "1.104.2"
```

### Include device info in a notification

Send a notification with the manufacturer and model when a device goes offline.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 {{ device_name("sensor.garage_door_battery") }} is offline.
 Device:
 {{ device_attr("sensor.garage_door_battery", "manufacturer") }}
 {{ device_attr("sensor.garage_door_battery", "model") }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Device Entities

---

The `device_entities` template function returns a list of entity IDs that belong to a given device. You pass it a device ID, and it gives you every entity that is tied to that device in Home Assistant.

This is useful when you want to work with all the entities that a single device provides. For example, a smart plug might expose a switch entity, an energy sensor, and a signal strength sensor. With `device_entities`, you can loop through all of those at once. You could use it to check if any entity on a device is unavailable, build a summary card for a specific device, or find all sensors on a particular piece of hardware.

💡 **Tip:** The device's page in **Settings > Devices & services** lists all entities that belong to it. Reach for `device_entities()` when you need that list inside a template expression.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["switch.smart_plug",
"sensor.smart_plug_energy", "sensor.smart_plug_signal",]
```

---

```
filter: '{{ "a1b2c3d4e5f6a1b2c3d4e5f6" | device_entities }}' type: list output: | ["switch.smart_plug",
"sensor.smart_plug_energy", "sensor.smart_plug_signal",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
device_entities(
 device_id: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} device\_id: description: > The ID of the device. You can find device IDs by using the `device_id` function with an entity ID or device name. required: true type: string

## Good to know

---

- Returns an empty list when the device ID does not match. It does not raise an error.
- Only accepts a device ID, not an entity ID or device name. Use `device_id` to resolve those first.
- Hidden and disabled entities are included in the result.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count entities on a device

Find out how many entities a specific device has registered.

#### template

```
template: '{{ device_entities("a1b2c3d4e5f6a1b2c3d4e5f6") | count }}'
type: integer
output: "3"
```

### Check if any entity on a device is unavailable

This checks whether any entity belonging to a device is currently in an unavailable state. Useful for monitoring device health.

#### template

```
template: |
 {{
 device_entities("a1b2c3d4e5f6a1b2c3d4e5f6")
 | select("is_state", "unavailable")
 | list
 | count > 0
 }}
type: boolean
output: "false"
```

### Get all sensor entities from a device

Filter the device's entities to only include sensors.

#### template

```
template: |
 {{
 device_entities("a1b2c3d4e5f6a1b2c3d4e5f6")
 | select("match", "sensor.")
 | list
 }}
type: list
output: |
 [
 "sensor.smart_plug_energy",
 "sensor.smart_plug_signal",
]
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Device Id

---

The `device_id` template function returns the device ID associated with an entity ID or a device name. If you pass an entity ID, it looks up which device that entity belongs to. If you pass a device name, it searches for a device with that exact name. It returns `None` if no matching device is found.

This is the starting point for most device-related template work. Since many device functions require a device ID, you'll often use `device_id` to get that ID from something more human-readable. For example, you might know the entity ID of your thermostat sensor but need the device ID to look up the manufacturer or firmware version with `device_attr`. Or you might want to find all entities on the same device as a particular sensor using `device_entities`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "a1b2c3d4e5f6a1b2c3d4e5f6"

---

filter: '{{ "sensor.living\_room\_temperature" | device\_id }}' type: string output:  
"a1b2c3d4e5f6a1b2c3d4e5f6"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
device_id(
 entity_id_or_device_name: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id\_or\_device\_name: description: > The entity ID or the name of the device. When using a device name, it matches against the custom name you set first, then falls back to the default device name. required: true type: string

## Good to know

---

- Returns `None` when no device matches or when the entity does not belong to a device.
- Name lookups need an exact match, including capitalization and spaces.
- When you set a custom device name, that name takes precedence over the default device name.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Look up a device ID by device name

You can also pass a device name instead of an entity ID. This is handy when you know the name of the device but not its ID.

#### template

```
template: '{{ device_id("Living Room Thermostat") }}'
type: string
output: "a1b2c3d4e5f6a1b2c3d4e5f6"
```

### Chain with `device_entities` to find sibling entities

Find all entities that share the same device as a known entity. This is useful when you know one entity on a device and want to discover the rest.

#### template

```
template: |
 {{
 device_id("sensor.living_room_temperature")
 | device_entities
 }}
type: list
output: |
 [
 "sensor.living_room_temperature",
 "sensor.living_room_humidity",
 "binary_sensor.living_room_battery",
]
```

### Get the manufacturer of a device from an entity ID

Combine `device_id` with `device_attr` to look up device attributes starting from an entity ID.

#### template

```
template: |
 {{
 device_attr(
 device_id("sensor.living_room_temperature"),
 "manufacturer"
)
 }}
type: string
output: "Aqara"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Device Name

---

The `device_name` template function returns the human-readable name of a device. You can pass it either a device ID or an entity ID. If a user-set name exists for the device, that name is returned; otherwise, the default device name is used. It returns `None` if no matching device is found.

This is useful whenever you want to display or include a device's name in a notification, dashboard card, or log message. For example, you could loop through all devices in an area and list their names, or include the device name in an alert so you know exactly which device reported a problem. Since it accepts both device IDs and entity IDs, you can use whichever you happen to have available.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output: "Living Room Thermostat"
```

---

```
filter: '{{ "a1b2c3d4e5f6a1b2c3d4e5f6" | device_name }}' type: string output: "Living Room Thermostat"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
device_name(
 device_id_or_entity_id: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} device\_id\_or\_entity\_id: description: > The device ID or entity ID to look up. When using an entity ID, the function finds the device the entity belongs to and returns that device's name. required: true type: string

## Good to know

---

- Returns `None` when no device matches the lookup value.
- Uses the custom name if you set one, otherwise returns the default name the integration provided.
- Renaming a device updates the output immediately, so the result can change over time.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get a device name from an entity ID

Pass an entity ID to find out the name of the device it belongs to.

#### template

```
template: '{{ device_name("sensor.living_room_temperature") }}'
type: string
output: "Living Room Thermostat"
```

### Include device name in a notification

Send a notification that includes the name of the device reporting low battery.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 {{ device_name("binary_sensor.front_door_battery") }}
 has a low battery!
```

### List all device names in an area

Combine with `area_devices` to display the names of all devices in an area.

#### template

```
template: |
 {% for dev_id in area_devices("Kitchen") %}
 {{ device_name(dev_id) }}
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Dict

---

The `dict` template function creates a dictionary (also known as a mapping) from keyword arguments. Each keyword becomes a key in the dictionary, and its value becomes the corresponding value. This is equivalent to Python's `dict()` constructor and provides a clean way to create dictionaries inline in your templates.

This is useful when you need to build a dictionary on the fly, for example when constructing data payloads for actions, passing structured data to scripts, or creating lookup tables. While you can also create dictionaries using the `{"key": "value"}` literal syntax, the `dict()` function can be easier to read when you have many keys or want to avoid quoting key names.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: dict output: '{"name': 'Living Room',
'brightness': 255, 'color': 'warm'}"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
dict(
 **kwargs: Any,
) -> dict
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} kwargs: description: > Any number of keyword arguments. Each keyword becomes a key in the resulting dictionary, and its argument becomes the corresponding value. required: false type: any

## Good to know

---

- Only accepts keyword arguments. Keys that contain spaces, dashes, or reserved words cannot be created with this function. Use literal syntax `{"my key": value}` instead.
- Keys are always strings, even though they look like identifiers in the call.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build action data

Create a dictionary to pass as data to an action call.

#### action

```
action: |
 action:
 - action: notify.mobile
 data: >
 {{
 dict(
 title="Alert",
 message="Motion detected in " ~ states("sensor.room"),
 priority="high"
)
 }}
```

### Create a lookup table

Build a mapping from keys to values for quick lookups.

#### template

```
template: |
 {% set icons = dict(
 clear="☀️",
 cloudy="☁️",
 rainy="🌧️"
) %}
 {{ icons.get(states("weather.home"), "?") }}
type: string
output: "☀️"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Dictsort

---

The `dictsort` filter sorts a dictionary and returns a list of `(key, value)` tuples in the sorted order. By default, it sorts by key in ascending order, but you can sort by value instead, control case sensitivity, and reverse the sort direction.

This is useful when you want to display or process dictionary data in a predictable order. For example, you might have a dictionary of entity attributes and want to display them sorted alphabetically by name, or you might want to sort a mapping of room names to temperatures so the warmest room appears first.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ {"b": 2, "a": 1, "c": 3} | dictsort }}' type: list output: "[('a', 1), ('b', 2), ('c', 3)]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
dictsort(
 value: dict,
 case_sensitive: bool = False,
 by: str = "key",
 reverse: bool = False,
) -> list[tuple]
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The dictionary to sort. required: true type: map  
case\_sensitive: description: > Whether the sort should be case-sensitive. Defaults to `false`, meaning uppercase and lowercase letters are treated as equal. required: false default: "false" type: boolean  
by: description: > Whether to sort by `key` or `value`. Defaults to `key`. required: false default: "key" type: string  
reverse: description: > If `true`, sorts in descending order. Defaults to `false`. required: false default: "false" type: boolean

## Sort by value

---

Sort dictionary entries by their values instead of keys.

### template

```
template: |
 {{
 {"kitchen": 22.5, "bedroom": 19.8, "living_room": 21.3}
 | dictsort(by="value")
 }}
type: list
output: "[('bedroom', 19.8), ('living_room', 21.3), ('kitchen', 22.5)]"
```



## Reverse sort order

---

Sort in descending order to see the highest values first.

### template

```
template: |
 {{
 {"kitchen": 22.5, "bedroom": 19.8, "living_room": 21.3}
 | dictsort(by="value", reverse=true)
 }}
type: list
output: "[('kitchen', 22.5), ('living_room', 21.3), ('bedroom', 19.8)]"
```

## Good to know

---

- Returns a list of `(key, value)` tuples, not a dictionary. Access items with `result[0][0]` for the first key.
- Case-insensitive by default, which differs from `sort`. Pass `case_sensitive=true` to match exact case.
- Sorting mixed-type values (strings and numbers together) raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display sorted entity attributes

Sort and display all attributes of a sensor in alphabetical order.

#### template

```
template: |
 {% for key, value in state_attr("sensor.weather", "forecast")[0] | dictsort %}
 {{ key }}: {{ value }}
 {% endfor %}
type: string
output: |
 condition: sunny
 temperature: 24
 wind_speed: 12
```

### Find the warmest room

Sort a temperature mapping by value in descending order and pick the first entry.

#### template

```
template: |
 {% set temps = {"Kitchen": 22.5, "Bedroom": 19.8, "Living room": 21.3} %}
 {% set sorted = temps | dictsort(by="value", reverse=true) %}
 The warmest room is {{ sorted[0][0] }} at {{ sorted[0][1] }}°C
type: string
output: "The warmest room is Kitchen at 22.5°C"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Difference

---

The `difference` template function returns a list of items that are in the first list but not in the second. This is the set difference operation: it removes from the first list any items that also appear in the second list.

This is useful when you want to exclude certain items from a collection. For example, you might have all the entities in a room and want to remove the ones that are already off, or you might have a list of all family members and want to exclude those who are currently home. You can also use it to find which devices have been removed from a group, or to filter out known items from a detection list. Duplicates are removed from the result since it operates as a set operation.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: list output: "[1, 2, 5]"

---

filter: '{{ [1, 2, 3, 4, 5] | difference([3, 4]) }}' type: list output: "[1, 2, 5]"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
difference(
 value: list,
 other: list,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The first list (the base set). Must be a list or collection. required: true type: list other: description: > The second list (items to remove). Must be a list or collection. required: true type: list

## Good to know

---

- Duplicates are removed in the result because this works as a set operation.
- Order is not preserved. Add `| sort` if you need a consistent order.
- This is one-sided. Items only in the second list do not appear in the result. Use `symmetric_difference` to get items unique to either side.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find entities not in a specific group

Determine which lights are in one group but not another.

#### template

```
template: |
 {% set all_lights = expand("group.all_lights")
 | map(attribute="entity_id") | list %}
 {% set automated = expand("group.automated_lights")
 | map(attribute="entity_id") | list %}
 {{ difference(all_lights, automated) }}
type: list
output: '["light.porch", "light.garage"]'
```

### Exclude unavailable entities

Filter out entities that are currently unavailable from a list.

#### template

```
template: |
 {% set all_sensors = ["sensor.temp_a", "sensor.temp_b", "sensor.temp_c"] %}
 {% set unavailable = all_sensors | select("is_state", "unavailable") | list %}
 {{ difference(all_sensors, unavailable) }}
type: list
output: '["sensor.temp_a", "sensor.temp_c"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Distance

---

The `distance` template function calculates the distance in kilometers between two locations. If you only provide one location, it calculates the distance from your home to that point. Locations can be specified as entity IDs (for entities with GPS coordinates), zone names, or latitude/longitude pairs.

This is useful for proximity-based automations. You could send a notification when someone is within a certain distance of home, calculate driving distances between family members, or determine how far away an entity is from a specific zone. The result is a number in kilometers that you can use in conditions and comparisons.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: 5.2



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
distance(
 *args: str | State | float,
) -> float | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} args: description: > One or two locations. A location is either a text identifier (an entity ID with GPS data, a zone name, or a state object) or two numbers in a row for latitude and longitude. Examples of valid calls: `distance("device_tracker.phone")` (distance from home to the entity), `distance(52.5, 13.4)` (distance from home to a lat/lon point), `distance("device_tracker.a", "device_tracker.b")` (distance between two entities), `distance("device_tracker.phone", 52.5, 13.4)` (distance between an entity and a lat/lon point). required: true type: any

## Good to know

---

- Returns `None` when any location lacks GPS coordinates, so guard your comparisons for missing data.
- The result is always in kilometers, regardless of the unit system configured in Home Assistant.
- With a single location argument, distance is measured from your Home Assistant home location.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distance from home

Calculate how far a device tracker is from home.

#### template

```
template: '{{ distance("device_tracker.phone") | round(1) }}'
type: float
output: 5.2
```

### Distance between two entities

Calculate the distance between two people.

#### template

```
template: |
 {{
 distance("device_tracker.phone_a",
 "device_tracker.phone_b") | round(1)
 }}
type: float
output: 12.7
```

### Distance using coordinates

Calculate the distance from home to a specific latitude/longitude.

#### template

```
template: '{{ distance(52.5, 13.4) | round(1) }}'
type: float
output: 847.3
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Divisibleby

---

The `divisibleby` test checks whether a number is evenly divisible by another number (the remainder is zero). It returns `true` if the division has no remainder and `false` otherwise. Use `value is divisibleby(num)` to perform the check.

This is useful for creating patterns in loops, such as applying different formatting to every third or fifth item. It can also be used with `select` and `reject` to filter lists of numbers based on divisibility.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Divisible by 5
```

type: string output: "Divisible by 5"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
divisibleby(
 value: int,
 num: int,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The number to test. required: true type: integer  
num: description: > The divisor. The test checks if `value` divided by `num` has zero remainder.  
required: true type: integer

## Good to know

---

- Both inputs must be integers. Floats raise an error unless converted with `int` first.
- Dividing by zero raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various values

Test whether numbers are divisible by a given divisor.

#### template

```
template: |
 {{ 12 is divisibleby(3) }}
 {{ 12 is divisibleby(5) }}
 {{ 100 is divisibleby(10) }}
type: boolean
output: |
 true
 false
 true
```

### Filter multiples from a list

Use `select` to find all numbers in a list that are multiples of a given number.

#### template

```
template: '{{ range(1, 21) | select("divisibleby", 4) | list }}'
type: list
output: "[4, 8, 12, 16, 20]"
```

### Apply alternating row styles in a loop

Use `divisibleby` to create a pattern every N iterations in a loop.

#### template

```
template: |
 {% for i in range(1, 7) %}
 {{ i }}{% if i is divisibleby(3) %} (group boundary){% endif %}
 {% endfor %}
type: string
output: |
 1
 2
 3 (group boundary)
 4
 5
 6 (group boundary)
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/E

---

The `e` template constant provides the mathematical constant  $e$  (Euler's number), approximately 2.71828. This is the base of the natural logarithm and appears throughout mathematics and science.

This is useful as the base for natural logarithm and exponential calculations in your templates. For example, you might use it with the `log` function for natural logarithm calculations, or in exponential growth and decay formulas applied to sensor data.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "2.718281828459045"
```



## Good to know

---

- This is a mathematical constant approximately equal to 2.71828.
- Use it without parentheses. `e` is a value, not a function you call.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Natural logarithm verification

Verify that the natural logarithm of  $e$  is 1.

#### template

```
template: |
 {{ log(e) }}
type: float
output: "1.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Entity Name

---

The `entity_name` template function returns the friendly name of an entity given its entity ID. It looks up the name from the entity registry first, falling back to the state object if the entity is not registered. If the entity does not exist at all, it returns `None`.

This is helpful whenever you want to display a human-readable name instead of a raw entity ID. For example, you might want to build a notification that says "The Living Room Light is on" instead of "light.living\_room is on". It is also useful when looping over a list of entity IDs and you need to present their names to a user in a readable format.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output: "Living Room Temperature"
```

---

```
filter: '{{ "sensor.living_room_temperature" | entity_name }}' type: string output: "Living Room Temperature"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
entity_name(
 entity_id: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: > The entity ID to look up. Returns the friendly name as a string, or `None` if the entity does not exist. required: true type: string



## Good to know

---

- Returns `None` when the entity does not exist. Chain with `| default(value)` for a fallback.
- Uses the friendly name from the entity registry first, then falls back to the state object's friendly name.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Use entity names in a notification

Send a notification with the friendly name of a sensor instead of its entity ID.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 {{ entity_name("binary_sensor.front_door") }} is
 {{ states("binary_sensor.front_door") }}
```

### List names of all lights that are on

Loop through all light entities and display the friendly name of each one that is currently on.

#### template

```
template: |
 {% for light in states.light | selectattr("state", "eq", "on") %}
 {{ entity_name(light.entity_id) }}
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Eq

---

The `eq` test checks whether two values are equal. It is also available under the aliases `equalto` and `==`. When used with `is`, it reads naturally: `value is eq(other)`.

While you can always use `==` directly in `{% jinja %}{% endjinja %}` conditions, the `eq` test becomes essential when working with `selectattr`, `rejectattr`, `select`, and `reject`. These filters require a test name as a string, and `eq` lets you filter collections based on equality without writing a loop.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: |

```
Equal
```

type: string output: Equal

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
eq(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the comparison. required: true type: any other: description: > The value to compare against. required: true type: any



## Aliases

---

This test can also be used as: - `equalto` - `value is equalto(other)` - `==` - `value is == (other)`

## Using with selectattr

---

The most common use of `eq` is filtering lists of objects by an attribute value.

### template

```
template: |
 {{
 states.light
 | selectattr("state", "eq", "on")
 | map(attribute="name") | list
 }}
title: Find all lights that are on
type: list
output: "['Living Room', 'Kitchen']"
```

## Good to know

---

- Numbers and strings compare as unequal even when they look the same. `"5" is eq(5)` is `false`.
- When used with `selectattr`, comparisons against entity states always compare strings, so wrap numbers in quotes or convert with `float` first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Filter entities by state

Find all binary sensors that are currently off.

#### template

```
template: |
 {{
 states.binary_sensor
 | selectattr("state", "eq", "off")
 | map(attribute="entity_id") | list
 }}
type: list
output: "['binary_sensor.front_door', 'binary_sensor.garage']"
```

### Compare values in a list

Use `select` to find items matching a specific value.

#### template

```
template: '{{ [1, 2, 3, 2, 1] | select("eq", 2) | list }}'
type: list
output: "[2, 2]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Escape

---

The `escape` filter converts HTML special characters ( `&`, `<`, `>`, `"`, `'` ) into their HTML entity equivalents so they are displayed as literal text instead of being interpreted as HTML. This filter is also available under the alias `e`. This is important when displaying user-provided or dynamic text in contexts where HTML is rendered, such as Markdown cards or notification messages. For example, if a sensor state contains characters like `<` or `&`, escaping ensures they appear correctly instead of being treated as HTML markup.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Hello & welcome" | escape }}' type: string output:
"Hello & welcome"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
escape(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to escape. Characters `&`, `<`, `>`, `"`, and `'` are replaced with their HTML entity equivalents. required: true type: string

## Using the e alias

---

The `escape` filter can also be used with its shorter alias `e`.

### template

```
template: '{{ "Tom & Jerry" | e }}'
title: Using the e alias
type: string
output: "Tom & Jerry"
```

## Good to know

---

- Only the five HTML-special characters are replaced. Other characters like accented letters or emoji pass through unchanged.
- The result is marked as safe HTML, so a later `safe` or escape call will not double-escape it.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Safely display a sensor value containing special characters

Escape a sensor state so that any special characters are rendered literally in a Markdown card.

#### template

```
template: '{{ states("sensor.device_status") | escape }}'
type: string
output: "Running <OK>"
```

### Escape dynamic content in a notification

Ensure that user-provided text does not break HTML formatting in a notification message.

#### template

```
template: |
 The device reported: {{ states("sensor.error_message") | escape }}
type: string
output: "The device reported: Error code <5> & retry"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Even

---

The `even` test checks whether an integer is even (divisible by 2 with no remainder). It returns `true` for even numbers and `false` for odd numbers.

This is commonly used in loops to apply alternating styles or logic to every other item. For example, you might use it to alternate row colors in a Markdown table or to apply different formatting to even-numbered items in a list displayed on a dashboard.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is even
```

type: string output: "It is even"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
even(
 value: int,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The integer to test. Returns `true` if the value is divisible by 2. required: true type: integer

## Good to know

---

- Zero is considered even.
- The input must be an integer. Passing a float or string raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various numbers

#### template

```
template: |
 {{ 4 is even }}
 {{ 7 is even }}
 {{ 0 is even }}
type: boolean
output: |
 true
 false
 true
```

### Filter even numbers from a list

Use `select` to extract only even numbers from a range.

#### template

```
template: '{{ range(1, 11) | select("even") | list }}'
type: list
output: "[2, 4, 6, 8, 10]"
```

### Alternating logic in a loop

Apply different formatting to even and odd iterations.

#### template

```
template: |
 {% for i in range(1, 5) %}
 {{ i }}: {{ "even" if i is even else "odd" }}
 {% endfor %}
type: string
output: |
 1: odd
 2: even
 3: odd
 4: even
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---



## Template Functions/Expand

---

The `expand` template function takes groups, zones, or lists of entity IDs and expands them into a flat, sorted list of individual entity state objects. If you pass in a group, it gives you all the individual entities that belong to that group, with duplicates removed.

This is extremely useful when you want to work with a collection of entities. For example, you might have a group of temperature sensors and want to calculate the average, find the highest reading, or count how many are above a threshold. The result is sorted alphabetically by entity ID and contains full state objects, so you can access `.state`, `.entity_id`, `.attributes`, and `.last_changed` on each one.

💡 **Tip:** A [Group helper](#) can combine entities and expose aggregated values (count, sum, min, max, mean) without any template. Reach for `expand()` when you need to process the individual entities inside a template.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["light.living_room_ceiling",
"light.living_room_lamp", "light.living_room_strip",]
```

---

```
filter: '{{ "group.living_room_lights" | expand | map(attribute="entity_id") | list }}' type: list output: |
["light.living_room_ceiling", "light.living_room_lamp", "light.living_room_strip",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
expand(
 *args: str | State | list,
) -> list[State]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} args: description: > One or more entity IDs, state objects, groups, or lists of these. Groups are expanded into their individual entities. Duplicates are removed and the result is sorted by entity ID. required: true type: [string, list]

## Good to know

---

- Returns full state objects, not entity IDs. Add `| map(attribute="entity_id")` to get IDs.
- Entities that do not exist are silently dropped from the result.
- The result is sorted by entity ID and has duplicates removed.
- Nested groups are expanded recursively, so you get every individual entity in the end.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many lights are on in a group

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | list
 | count
 }}
type: integer
output: "5"
```

### Calculate average temperature from a group

Combine `expand` with `average` to get the mean temperature across a group of sensors.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | map(attribute="state")
 | map("float")
 | average
 }}
type: float
output: "21.3"
```

### Find the entity with the highest value

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | sort(attribute="state", reverse=true)
 | map(attribute="entity_id")
 | first
 }}
type: string
output: "sensor.kitchen_temperature"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/False Test

---

The `false` test checks whether a value is `false`. It performs a strict identity check, meaning only the boolean value `false` passes. Values like `0`, `""`, `none`, and the string `"false"` do not match.

This is useful when you need to verify that a value is specifically the boolean `false` and not merely a falsy value. In Home Assistant, entity attributes can be actual booleans, and this test lets you check for an exact `false` value without accidentally matching other falsy types like `0` or an empty string.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is false
```

type: string output: "It is false"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
false(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` only if the value is the boolean `false`. required: true type: any

## Good to know

---

- This is a strict identity test. Only the boolean `false` passes, while `0`, empty strings, and `None` do not.
- Entity states are strings, so `states("switch.x") is false` is always `false`. Use `is_state("switch.x", "off")` instead.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Strict boolean check

Only the boolean `false` passes; falsy values like `0` or `""` do not.

#### template

```
template: |
 {{ false is false }}
 {{ 0 is false }}
 {{ "" is false }}
type: boolean
output: |
 true
 false
 false
```

### Verify a boolean attribute

Check that an entity attribute is specifically `false`.

#### template

```
template: |
 {% set charging = state_attr("sensor.phone", "is_charging") %}
 {% if charging is false %}
 Phone is not charging
 {% endif %}
type: string
output: "Phone is not charging"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Filesizeformat

---

The `filesizeformat` filter converts a number of bytes into a human-readable file size string, such as `13.2 kB` or `4.1 MB`. It automatically chooses the appropriate unit (bytes, kB, MB, GB, and larger). By default, it uses decimal (SI) units (1 kB = 1000 bytes), but you can switch to binary units (1 KiB = 1024 bytes) with the `binary` parameter. This is useful when displaying storage-related information from sensors, such as disk usage, download sizes, or memory consumption. For example, a sensor might report a value in bytes, and you can use this filter to display it in a more readable format on a dashboard card or in a notification.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "1.2 GB"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
filesizeformat(
 value: int | float,
 binary: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The number of bytes to format as a human-readable file size. required: true type: integer binary: description: > If `true`, uses binary units (KiB, MiB, GiB) where 1 KiB = 1024 bytes. If `false` (the default), uses decimal units (kB, MB, GB) where 1 kB = 1000 bytes. required: false default: "false" type: boolean

## Using binary units

---

Set `binary` to `true` to use binary (IEC) units instead of decimal (SI) units.

### template

```
template: '{{ 1048576 | filesizeformat(binary=true) }}'
title: Display as binary file size
type: string
output: "1.0 MiB"
```

## Good to know

---

- The default is decimal units (1 kB = 1000 bytes), not binary. Pass `binary=true` for KiB/MiB/GiB.
- The input must be a number of bytes. Convert state strings with `int` or `float` first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display disk usage from a sensor

Format a disk usage sensor value that reports bytes into a readable size.

#### template

```
template: '{{ states("sensor.disk_used") | int | filesizeformat }}'
type: string
output: "42.3 GB"
```

### Show download size in a notification

Format a file size for display in a notification message.

#### template

```
template: |
 {{
 "Update available: "
 ~ (states("sensor.update_size") | int | filesizeformat)
 }}
type: string
output: "Update available: 156.2 MB"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/First

---

The `first` filter returns the first item of a list or the first character of a string. It provides a clean, readable way to access the beginning of a sequence.

This is commonly used at the end of a filter chain to pick the top result after sorting, selecting, or otherwise narrowing down a list of entities. For example, after sorting temperature sensors by value, you can use `first` to grab the coldest reading. After filtering lights by state, use `first` to get the first match. It is one of the most frequently used filters when working with entity collections.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ [10, 20, 30] | first }}' type: integer output: "10"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
first(
 value: list,
) -> Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list or string to get the first item from. Must be a non-empty sequence. required: true type: any

## Good to know

---

- Raises an error when the sequence is empty. Chain with `| default(value)` if emptiness is possible.
- Works on any sequence, including strings (returns the first character) and generators.
- When used after `selectattr` or `select`, add `| list` first or the result will only consume part of the iterable.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get the entity with the lowest temperature

Sort temperature sensors and pick the first (lowest) one.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | sort(attribute="state")
 | map(attribute="entity_id")
 | first
 }}
type: string
output: "sensor.bedroom_temperature"
```

### Get the first active light

Find the first light that is currently on.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | map(attribute="entity_id")
 | first
 }}
type: string
output: "light.kitchen"
```

### First character of a string

The `first` filter also works on strings, returning the first character.

#### template

```
template: '{{ "Hello" | first }}'
type: string
output: "H"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Flatten

---

The `flatten` template function takes a list that contains other lists (nested lists) and flattens them into a single, flat list. All items from all levels of nesting are brought up to the top level, which is useful for working with deeply nested data structures.

This is useful when you collect data from multiple sources that each return a list, and you end up with a list of lists. For example, you might gather the attributes from several sensors that each return a list of values, or you might combine multiple groups and want a single flat list of all entities. The optional `levels` parameter lets you control how many levels of nesting to flatten, so you can partially flatten a deeply nested structure if needed.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: list output: "[1, 2, 3, 4, 5]"

---

filter: '{{ [[1, 2], [3, [4, 5]]] | flatten }}' type: list output: "[1, 2, 3, 4, 5]"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
flatten(
 value: list,
 levels: int | None = None,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The nested list to flatten. Must be a list or collection. required: true type: list levels: description: > The number of levels of nesting to flatten. If not provided, all levels are flattened completely. Set to `1` to only flatten one level deep. required: false type: integer

## Controlling flatten depth

---

By default, `flatten` removes all nesting. Use the `levels` parameter to control how deep the flattening goes.

### template

```
template: '{{ [1, [2, 3]], [4, [5, 6]] | flatten(levels=1) }}'
type: list
output: "[1, [2, 3], 4, [5, 6]]"
```

## Good to know

---

- Without `levels`, every layer of nesting is removed, which can produce surprising results on deeply structured data.
- Only nested lists are unpacked. Dictionaries and other collections inside the list are kept as-is.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Flatten sensor attributes from multiple entities

When multiple sensors each return a list in an attribute, combine and flatten them into one list.

#### template

```
template: |
 {{
 [
 state_attr("sensor.room_a", "device_list"),
 state_attr("sensor.room_b", "device_list")
] | flatten
 }}
type: list
output: '["device_1", "device_2", "device_3", "device_4"]'
```

### Flatten and count unique items

Flatten a nested structure and then count the unique values.

#### template

```
template: |
 {{ [[1, 2, 3], [2, 3, 4], [4, 5]] | flatten | set | list | count }}
type: integer
output: "5"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Float

---

The `float` template function converts a value to a floating-point number. If the value cannot be converted, it returns the default you provide instead of raising an error. This "forgiving" behavior makes it safe to use with sensor values that might sometimes be `unavailable` or `unknown`.

In Home Assistant, all entity states are stored as strings, even when they represent numbers. This means that `states("sensor.temperature")` returns `21.5` (a string), not `21.5` (a number). Before you can do any math with a state value, such as comparing it to a threshold, adding values together, or using it in a calculation, you must convert it to a number first. The `float` function (or filter) is the standard way to do this.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "21.5"

---

filter: '' type: float output: "21.5"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
float(
 value: Any,
 default: Any = _SENTINEL,
) -> float | Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to convert to a float. Strings that represent numbers (like `21.5`) are converted. Non-numeric values raise an error unless a default is provided. required: true type: any default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. It is strongly recommended to always provide a default. required: false type: any

## Using a default value

---

Since sensor states can be `unavailable` or `unknown`, always provide a default to prevent your template from breaking.

### template

```
template: '{{ states("sensor.temperature") | float(0) }}'
title: Safe conversion with default
type: float
output: "21.5"
```

If the sensor state were `unavailable`, this would return `0` instead of raising an error.

## Why conversion is needed

---

All entity states in Home Assistant are strings. Without conversion, comparisons and math produce unexpected results or errors:

### template

```
template: '{{ states("sensor.temperature") | float > 25 }}'
title: Compare a sensor value to a threshold
type: boolean
output: "false"
```

## Good to know

---

- Without a default, conversion failures raise an error that can break your template. Always pass a default when reading from entity states.
- The default is returned when the input is `None`, `unavailable`, `unknown`, or any value that cannot be converted.
- The default does not have to be a number. Passing a string like `"unknown"` is valid when you want a readable fallback.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Calculate a temperature difference

Calculate the difference between indoor and outdoor temperatures.

#### template

```
template: |
 {{
 (states("sensor.indoor_temperature") | float(0))
 - (states("sensor.outdoor_temperature") | float(0))
 }}
type: float
output: "5.3"
```

### Use in an automation trigger

Trigger when the power usage exceeds a threshold. The `float` filter ensures the string state is compared as a number.

#### automation

```
automation: |
 trigger:
 - trigger: template
 value_template: >
 {{ states("sensor.power_usage") | float(0) > 3000 }}
```

### Convert with a fallback in a notification

Send a notification that includes a sensor value, using a readable fallback if the sensor is unavailable.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 The humidity is
 {{ states("sensor.humidity") | float("unknown") }}%
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Float Test

---

The `float` test checks whether a value is a floating-point number. It returns `true` if the value is of float type and `false` for any other type, including integers and numeric strings.

This is useful when you need to verify the exact type of a numeric value. In Home Assistant, calculations often produce floats, and some entity attributes are specifically floating-point values. This test lets you distinguish between integers and floats when the difference matters for your template logic.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is a float
```

type: string output: "It is a float"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
float(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is of float type. required: true type: any

## Good to know

---

- Only true float values pass. Integers, numeric strings, and booleans do not.
- To accept any numeric input, use the `number` test instead.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distinguish float from integer

Check whether a numeric value is a float rather than an integer.

#### template

```
template: |
 {{ 21.5 is float }}
 {{ 42 is float }}
 {{ "21.5" is float }}
type: boolean
output: |
 true
 false
 false
```

### Check calculation result type

Verify the type of a calculation result before formatting it.

#### template

```
template: |
 {% set result = 10 / 3 %}
 {% if result is float %}
 Result: {{ result | round(2) }}
 {% endif %}
type: string
output: "Result: 3.33"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Floor Areas

---

The `floor_areas` template function returns a list of area IDs that belong to a given floor. You can specify the floor by its name (like "*Ground Floor*") or by its internal ID. It gives you all areas that have been assigned to that floor in Home Assistant.

This is useful when you want to work with all areas on a particular floor at once. For example, you could turn off every light on the ground floor at bedtime, check if any room upstairs has an open window, or count how many rooms are on each floor. As you add or remove areas from a floor in Home Assistant, the list automatically updates, so your automations and templates always stay in sync with your actual setup.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: list output: | ["living_room", "kitchen",
"hallway",]
```

---

```
filter: '{{ "Ground Floor" | floor_areas }}' type: list output: | ["living_room", "kitchen", "hallway",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
floor_areas(
 floor_id_or_name: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} floor\_id\_or\_name: description: > The name or ID of the floor. You can find floor IDs in **Settings > Areas, labels & zones**. required: true type: string

## Good to know

---

- Returns an empty list when the floor does not exist or has no areas assigned.
- Returns area IDs, not human-readable names. Use `area_name` to get the names.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many areas are on a floor

Find out how many areas are assigned to a given floor.

#### template

```
template: '{{ floor_areas("Ground Floor") | count }}'
type: integer
output: "3"
```

### List the area names on a floor

Loop through all areas on a floor and display their names using `area_name`.

#### template

```
template: |
 {% for area_id in floor_areas("Ground Floor") %}
 {{ area_name(area_id) }}
 {% endfor %}
type: string
output: |
 Living Room
 Kitchen
 Hallway
```

### Check if any area on a floor has motion

Loop through all areas on a floor and check if any has an active motion sensor. This uses `area_entities` to get the entities for each area.

#### template

```
template: |
 {% for area_id in floor_areas("First Floor") %}
 {% if area_entities(area_id)
 | select("match", "binary_sensor.")
 | select("is_state", "on")
 | list
 | count > 0 %}
 Motion in {{ area_name(area_id) }}!
 {% endif %}
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Floor Entities

---

The `floor_entities` template function returns a list of entity IDs for all entities on a given floor. You can specify the floor by its name (like "Ground Floor") or by its internal ID. It gathers entities from every area assigned to that floor, giving you a single flat list.

This is useful when you want to work with all entities across an entire floor at once, without having to loop through each area individually. For example, you could turn off every light on the upstairs floor at bedtime, count how many sensors are active on the ground floor, or check if any window on a particular floor is open. As you add or remove devices and areas from a floor in Home Assistant, the list automatically updates, so your automations and templates always stay in sync.

💡 **Tip:** Automation actions can target an entire floor through the visual editor, no template needed. Reach for `floor_entities()` when you need to loop over or filter the entities inside a template expression.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["light.living_room_ceiling",
"sensor.living_room_temperature", "light.kitchen_counter", "sensor.kitchen_humidity",
"light.hallway",]
```

---

```
filter: '{{ "Ground Floor" | floor_entities }}' type: list output: | ["light.living_room_ceiling",
"sensor.living_room_temperature", "light.kitchen_counter", "sensor.kitchen_humidity",
"light.hallway",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
floor_entities(
 floor_id_or_name: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} floor\_id\_or\_name: description: > The name or ID of the floor. You can find floor IDs in **Settings > Areas, labels & zones**. required: true type: string

## Good to know

---

- Entities without an area (or whose area is not assigned to a floor) are not included.
- The result aggregates entities across every area on the floor, including entities inherited from devices in those areas.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count the lights that are on across a floor

Want to know how many lights are currently on across an entire floor? This filters the floor's entities down to lights and counts the ones that are on.

#### template

```
template: |
 {{
 floor_entities("Ground Floor")
 | select("match", "light.")
 | select("is_state", "on")
 | list
 | count
 }}
type: integer
output: "3"
```

### Check if any window is open on a floor

This checks whether any window sensor on the first floor is currently open. Useful as a condition in automations, for example to prevent turning on the heating if a window is open upstairs.

#### template

```
template: |
 {{
 floor_entities("First Floor")
 | select("match", "binary_sensor.")
 | select("is_state", "on")
 | list
 | count > 0
 }}
type: boolean
output: "true"
```

### Turn off all lights on a floor

Use `floor_entities` in an automation action to turn off every light on a given floor. This is handy for a "goodnight" routine that shuts down an entire floor.

#### action

```
action: |
 action:
 - action: light.turn_off
 target:
 entity_id: >
 {{
 floor_entities("First Floor")
 | select("match", "light.")
 | list
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Floor Id

---

The `floor_id` template function returns the unique floor ID for a given floor name, area name, device ID, or entity ID. Every floor in Home Assistant has an internal ID that stays the same even if you rename the floor, and this function lets you look it up.

This is useful when you need the floor ID to pass to other floor functions like `floor_areas` or `floor_entities`, or when you want to find out which floor a particular entity, device, or area belongs to. For example, you could use it in an automation to determine which floor a motion sensor is on and then act on all devices on that floor.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "ground\_floor"

---

filter: '{{ "Ground Floor" | floor\_id }}' type: string output: "ground\_floor"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
floor_id(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The floor name, area name, entity ID, or device ID to look up. If a floor name is given, the matching floor ID is returned. If an area name, entity ID, or device ID is given, the floor ID of the floor that area, entity, or device belongs to is returned. required: true type: string



## Good to know

---

- Returns `None` when the lookup value does not match a floor, or when the area, entity, or device is not assigned to a floor.
- An entity gets its floor through its area, so an entity with no area returns `None`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find the floor of an entity

Look up which floor a specific entity belongs to.

#### template

```
template: '{{ floor_id("sensor.living_room_temperature") }}'
type: string
output: "ground_floor"
```

### Find the floor of an area

Look up which floor a specific area is assigned to by passing its name.

#### template

```
template: '{{ floor_id("Kitchen") }}'
type: string
output: "ground_floor"
```

### Group areas by floor

Combine `floor_id` with `areas` to find out which areas belong to each floor.

#### template

```
template: |
 {% for area_id in areas() %}
 {{ area_name(area_id) }} is on {{ floor_name(floor_id(area_id)) }}
 {% endfor %}
type: string
output: |
 Living Room is on Ground Floor
 Kitchen is on Ground Floor
 Bedroom is on First Floor
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Floor Name

---

The `floor_name` template function returns the friendly, human-readable name of a floor. You can give it a floor ID, an area name, an entity ID, or a device ID, and it tells you the name of the floor it belongs to.

This is especially useful for building dynamic messages and notifications. Instead of showing a technical floor ID like `ground_floor`, you can show the actual name "Ground Floor" that you (or whoever receives the message) recognize. For example, when a leak sensor triggers, your notification can say "Water detected on the Ground Floor" by looking up the floor name of the sensor that triggered.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "Ground Floor"

---

filter: '{{ "ground\_floor" | floor\_name }}' type: string output: "Ground Floor"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
floor_name(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The floor ID, area name, entity ID, or device ID to look up. Returns the friendly name of the floor, or `None` if no matching floor is found.  
required: true type: string



## Good to know

---

- Returns `None` when the lookup value does not match a floor.
- The name changes when you rename the floor, so the output can shift over time.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get the floor name from an entity

Find out which floor a sensor is on by passing its entity ID.

#### template

```
template: '{{ floor_name("sensor.living_room_temperature") }}'
type: string
output: "Ground Floor"
```

### Use in a notification with the trigger floor

A common pattern: when a sensor triggers an automation, include the floor name in the notification. This works with any trigger that provides `trigger.entity_id`, such as state or leak triggers.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 title: >
 {{ floor_name(trigger.entity_id) }}
 message: >
 Alert on {{ floor_name(trigger.entity_id) }}
```

### List all floor names

Loop through all floors and display their names.

#### template

```
template: |
 {% for id in floors() %}
 {{ floor_name(id) }}
 {% endfor %}
type: string
output: |
 Ground Floor
 First Floor
 Basement
```

## Show the floor of each area

Build a summary showing which floor each area belongs to, using `floor_name` as a way to resolve the human-readable floor for each area.

### template

```
template: |
 {% for id in areas() %}
 {{ area_name(id) }}: {{ floor_name(id) }}
 {% endfor %}
type: string
output: |
 Living Room: Ground Floor
 Kitchen: Ground Floor
 Bedroom: First Floor
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Floors

---

The `floors` template function returns a list of all floor IDs in your Home Assistant instance. Each floor you've created in Home Assistant has a unique ID, and this function gives you all of them.

This is useful when you want to loop through every floor in your home and do something with it. For example, you could check which floors have lights on, count how many areas are on each floor, or build a summary of activity across your entire home. Since floors can be added or removed at any time, using `floors()` ensures your templates always reflect your current setup.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["ground_floor", "first_floor",
"basement",]
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
floors() -> list[str]
```

## Good to know

---

- Returns floor IDs, not names. Pair with `floor_name` to display names.
- The order is not guaranteed, so sort before display if you need a stable ordering.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many floors you have

A convenient way to see how many floors are set up in your Home Assistant instance.

#### template

```
template: '{{ floors() | count }}'
type: integer
output: "3"
```

### List all floor names

Loop through all floors and display their names using `floor_name`.

#### template

```
template: |
 {% for floor_id in floors() %}
 {{ floor_name(floor_id) }}
 {% endfor %}
type: string
output: |
 Ground Floor
 First Floor
 Basement
```

### Count lights on per floor

Build a summary of how many lights are on across each floor. This loops through all floors, gathers the entities on each one using `floor_entities`, and counts the active lights.

#### template

```
template: |
 {% for id in floors() %}
 {% set lights = floor_entities(id)
 | select("match", "light.")
 | select("is_state", "on")
 | list %}
 {% if lights | count > 0 %}
 {{ floor_name(id) }}: {{ lights | count }} lights on
 {% endif %}
 {% endfor %}
type: string
output: |
 Ground Floor: 5 lights on
 First Floor: 2 lights on
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Forceescape

---

The `forceescape` filter escapes HTML special characters ( `&`, `<`, `>`, `"`, `'` ) just like the `escape` filter, but it enforces escaping even on strings that have already been marked as safe. Normally, once a string is marked safe with the `safe` filter, `escape` will not escape it again. The `forceescape` filter overrides that behavior. This is useful when you need to guarantee that a value is escaped regardless of how it was produced earlier in a template chain. For example, if a value was marked as safe somewhere upstream but you want to display it as literal text rather than rendered HTML, `forceescape` ensures it gets escaped.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "bold" | safe | forceescape }}' type: string output:
"bold"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
forceescape(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to escape. All HTML special characters are converted to their entity equivalents, even if the string was previously marked as safe. required: true type: string

## Good to know

---

- Use this instead of `escape` when an upstream `safe` call has already marked a string as trusted HTML.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Escape a pre-marked safe string

Show the raw HTML markup of a value that was previously marked as safe.

#### template

```
template: |
 {% set html_content = "Important" | safe %}
 {{ html_content | forceescape }}
type: string
output: "Important"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Format

---

The `format` filter applies printf-style string formatting to a template string, replacing placeholders like `%s` (string), `%d` (integer), and `%f` (float) with the provided values. This is the same formatting style used in many programming languages. This is useful when you want to build a formatted string with multiple dynamic values inserted at specific positions. For example, you might want to format a notification message with a sensor name and its value, or create a display string with a number formatted to a specific number of decimal places.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Hello %s, you have %d messages" | format("Alice",
5) }}' type: string output: "Hello Alice, you have 5 messages"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
format(
 value: str,
 *args: Any,
 **kwargs: Any,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The format string containing printf-style placeholders such as `%s` (string), `%d` (integer), or `%f` (float). required: true type: string args: description: > The values to insert into the placeholders, in order. required: true type: any

## Formatting numbers

---

Use `%d` for integers and `%f` for floats. You can control decimal places with `%.Nf`.

### template

```
template: '{{ "Temperature: %.1f degrees" | format(22.456) }}'
title: Format a float to one decimal place
type: string
output: "Temperature: 22.5 degrees"
```

## Good to know

---

- Uses printf-style specifiers, not Python f-strings or `{0}` positional placeholders.
- Type must match: `%d` with a non-integer raises an error. Cast with `int` or `float` first.
- A literal percent sign in the output requires `%%` in the format string.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build a notification message with sensor data

Create a formatted notification message combining multiple sensor values.

#### template

```
template: |
 {{
 "The %s is %.1f°C with %d%% humidity"
 | format("living room", 22.5, 65)
 }}
type: string
output: "The living room is 22.5°C with 65% humidity"
```

### Format a zero-padded number

Use `%03d` to pad a number with leading zeros.

#### template

```
template: '{{ "Sensor ID: S-%03d" | format(7) }}'
type: string
output: "Sensor ID: S-007"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/From Hex

---

The `from_hex` filter converts a hexadecimal string into a bytes object. Each pair of hex characters in the input string is converted to a single byte. For example, "48656c6c6f" decodes to the bytes representing "Hello".

This is useful when working with IoT devices and protocols that represent binary data as hex strings. Many BLE (Bluetooth Low Energy) sensors, Zigbee devices, and serial protocols report their data as hexadecimal strings. The `from_hex` filter converts these hex strings into bytes that you can then process further with `unpack` to extract numeric values, or decode to a text string.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "48656c6c6f" | from_hex }}' type: bytes output: "b'Hello'"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | from_hex() -> bytes
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The hexadecimal string to decode. Must contain an even number of hex characters (0-9, a-f, A-F). required: true type: string

## Good to know

---

- The input must have an even number of hex characters. Odd-length strings raise an error.
- The result is raw bytes. Pipe to `string` for text, or `unpack` for numeric values.
- A `0x` prefix on the input raises an error; strip it first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Decode a hex sensor value to an integer

Convert a hex string from a sensor into an integer using `unpack`.

#### template

```
template: '{{ "04d2" | from_hex | unpack(">H") }}'
type: integer
output: "1234"
```

### Decode hex to a text string

Convert a hex-encoded string to readable text by decoding the resulting bytes.

#### template

```
template: '{{ "486f6d65" | from_hex | string }}'
type: string
output: "Home"
```

### Process BLE advertisement data

Extract a temperature value from a BLE sensor that reports data as a hex string. The first two bytes (4 hex characters) contain the temperature as a big-endian signed short.

#### template

```
template: |
 {% set hex_data = states("sensor.ble_advertisement") %}
 {{ hex_data[:4] | from_hex | unpack(">h") / 100 }}
type: float
output: "21.5"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/From Json

---

The `from_json` filter takes a JSON-formatted string and parses it into a Python object (dictionary, list, string, number, or boolean). This is the opposite of `to_json`, which converts an object into a JSON string.

This is essential when working with data received from external services. Many integrations store JSON data as strings in entity states or attributes. For example, MQTT messages, REST sensor responses, and webhook payloads often arrive as JSON strings that you need to parse before you can access individual values. Once parsed, you can use standard dictionary and list operations to extract the data you need. The optional `default` parameter lets you provide a fallback value if the JSON is invalid, preventing errors in your templates.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ {"temperature": 21.5, "humidity": 45}' | from_json }}'
type: dict output: '{"temperature": 21.5, "humidity": 45}'
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | from_json(
 default: Any = <raises error>,
) -> Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} default: description: > Value to return if the JSON string is invalid or cannot be parsed. If not provided, an error is raised on invalid input. required: false type: any

## Using a default value

---

If the input might not be valid JSON, provide a default to avoid errors.

### template

```
template: '{{ "not valid json" | from_json(default={}) }}'
type: dict
output: "{}"
```

## Good to know

---

- Without a default, invalid JSON raises an error that can break your template.
- JSON uses double quotes only. Single-quoted strings like `'{"a": 1}'` need to wrap a string that itself uses double quotes inside.
- Returns the matching Python type: an object becomes a dict, an array becomes a list, `null` becomes `None`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Parse an MQTT message

Extract a value from a JSON-formatted MQTT sensor payload.

#### template

```
template: |
 {% set payload = states("sensor.mqtt_sensor") | from_json %}
 Temperature: {{ payload.temperature }}
 Humidity: {{ payload.humidity }}
type: string
output: |
 Temperature: 21.5
 Humidity: 45
```

### Parse a REST sensor response

Access nested data from a REST API response stored as a JSON string.

#### template

```
template: |
 {% set data = states("sensor.api_response") | from_json %}
 {{ data.results | map(attribute="name") | list }}
type: list
output: '["Item A", "Item B", "Item C"]'
```

### Safe parsing with a default

When processing data that might not always be valid JSON, use a default to keep your template working.

#### template

```
template: |
 {% set data = states("sensor.maybe_json")
 | from_json(default={"status": "unknown"}) %}
 Status: {{ data.status }}
type: string
output: "Status: unknown"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Ge

---

The `ge` test checks whether a value is greater than or equal to another value. It is also available under the alias `>=`. When used with `is`, it reads naturally: `value is ge(other)`.

While you can always use `>=` directly in conditions, the `ge` test is essential when working with `selectattr`, `select`, and similar filters that require a test name as a string. This lets you filter collections by comparing attribute values to a minimum threshold without writing explicit loops.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Greater or equal
```

type: string output: Greater or equal

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
ge(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the comparison. required: true type: any other: description: > The value to compare against. required: true type: any

## Aliases

---

This test can also be used as: - `>=` - `value is >=(other)`

## Using with selectattr

---

Filter entities whose attributes meet or exceed a minimum value.

### template

```
template: |
 {{
 states.sensor
 | selectattr("attributes.battery_level", "ge", 50)
 | map(attribute="name") | list
 }}
title: Devices with at least 50% battery
type: list
output: "['Phone', 'Tablet']"
```

## Good to know

---

- When comparing entity state strings with numbers, convert first or wrap the threshold in quotes. String comparison puts "9" above "10".
- Comparisons between incompatible types (a number and a string, for instance) raise an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find values at or above a minimum

Use `select` to find all numbers at or above a threshold.

#### template

```
template: '{{ [10, 25, 30, 15, 40] | select("ge", 25) | list }}'
type: list
output: "[25, 30, 40]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Groupby

---

The `groupby` filter groups items in a list by a common attribute, producing a list of `(grouper, list)` pairs where `grouper` is the attribute value and `list` contains all items that share that value.

This is extremely useful when you want to organize entities by a shared property. For example, you can group lights by their area, sensors by their state, or devices by their manufacturer. The result lets you iterate over each group and display or process the items together. It works especially well with `expand` to organize large collections of entities into meaningful categories.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: |

type: string output: | off: light.bedroom, light.garage on: light.kitchen, light.living\_room

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
groupby(
 value: list,
 attribute: str,
 default: Any = None,
) -> list[tuple]
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of items to group. required: true type: list  
attribute: description: > The attribute to group by. Supports dotted notation for nested attributes (for example, `attributes.device_class`). required: true type: string default: description: > The default value to use for items that do not have the specified attribute. If not provided, items without the attribute are excluded. required: false type: any

## Grouping with dotted attributes

---

Use dotted notation to group by nested attributes.

### template

```
template: |
 {% for device_class, entities in expand("group.all_sensors")
 | groupby("attributes.device_class") %}
 {{ device_class }}: {{ entities | length }} sensors
 {% endfor %}
title: Group sensors by device class
type: string
output: |
 humidity: 3 sensors
 temperature: 5 sensors
```

## Default value for missing attributes

---

Use the `default` parameter to include items that lack the grouping attribute.

### template

```
template: |
 {% for area, entities in expand("group.all_lights")
 | groupby("attributes.area", default="Unknown") %}
 {{ area }}: {{ entities | map(attribute="entity_id") | join(", ") }}
 {% endfor %}
type: string
output: |
 Kitchen: light.kitchen_ceiling, light.kitchen_counter
 Living room: light.living_room_main
 Unknown: light.unnamed_bulb
```



## Good to know

---

- The input must be sorted by the grouping attribute already. This filter only groups adjacent items with matching values.
- Items without the attribute are dropped unless you supply a `default`.
- Returns a list of `(grouper, items)` tuples, which you unpack in a `for` loop.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Group entities by state

Group all entities in a group by their current state value.

#### template

```
template: |
 {% for state, entities in expand("group.all_lights") | groupby("state") %}
 {{ state | upper }}: {{ entities | map(attribute="name") | join(", ") }}
 {% endfor %}
type: string
output: |
 OFF: Bedroom light, Garage light
 ON: Kitchen light, Living room light
```

### Count entities per group

Show a summary of how many entities are in each group.

#### template

```
template: |
 {% for state, entities in expand("group.all_lights") | groupby("state") %}
 {{ entities | length }} light(s) are {{ state }}
 {% endfor %}
type: string
output: |
 2 light(s) are off
 2 light(s) are on
```

### Group sensors by area and find averages

Group temperature sensors by their area and calculate the average for each.

#### template

```
template: |
 {% for area, sensors in expand("group.temperature_sensors")
 | groupby("attributes.area") %}
 {{ area }}: {{ sensors | map(attribute="state") | map("float")
 | average | round(1) }}°C
 {% endfor %}
type: string
output: |
 Bedroom: 19.8°C
 Kitchen: 22.1°C
 Living room: 21.3°C
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Gt

---

The `gt` test checks whether a value is strictly greater than another value. It is also available under the aliases `greaterthan` and `>`. When used with `is`, it reads naturally: `value is gt(other)`.

While you can always use `>` directly in conditions, the `gt` test is essential when working with `selectattr`, `select`, and similar filters that require a test name as a string. This lets you filter collections by comparing attribute values without writing explicit loops.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Greater
```

type: string output: Greater



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
gt(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the comparison. required: true type: any other: description: > The value to compare against. required: true type: any

## Aliases

---

This test can also be used as: - `greaterthan` - `value is greaterthan(other)` - `>` - `value is >(other)`

## Using with selectattr

---

Filter entities whose numeric attributes exceed a threshold.

### template

```
template: |
 {{
 states.sensor
 | selectattr("state", "gt", "25")
 | map(attribute="name") | list
 }}
title: Sensors above 25
type: list
output: "['Outdoor Temperature']"
```

## Good to know

---

- When comparing entity state strings with numbers, convert first or wrap the threshold in quotes. String comparison puts `"9"` above `"10"`.
- This is strict. A value equal to the threshold does not pass. Use `ge` for greater-or-equal.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Filter values above a threshold

Use `select` to find numbers exceeding a limit.

#### template

```
template: '{{ [10, 25, 30, 15, 40] | select("gt", 20) | list }}'
type: list
output: "[25, 30, 40]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Has Value

---

The `has_value` template function checks whether an entity exists and has a meaningful state. It returns `false` if the entity doesn't exist, is `unavailable`, or is `unknown`. Otherwise it returns `true`.

This is essential for building robust templates that don't break when a sensor goes offline or hasn't reported yet. Sensors can temporarily become `unavailable` (for example, when a battery-powered device goes to sleep) or `unknown` (during Home Assistant startup). By checking with `has_value` first, you can avoid errors in calculations or show a fallback message instead of displaying confusing values.

 **Tip:** Automation triggers and conditions let you require an entity to have a valid state through the visual editor. Reach for `has_value()` when you need the check inside a template expression.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: boolean output: "true"

---

filter: '{{ "sensor.temperature" | has\_value }}' type: boolean output: "true"

---

test: | Sensor is available

type: string output: "Sensor is available"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
has_value(
 entity_id: str,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: > The entity ID to check. Returns `true` if the entity exists and its state is not `unavailable` or `unknown`. required: true type: string

 **Tip:** You can check if an entity is `unavailable` or `unknown` by looking it up in **Developer Tools > States**. The state column shows the current value.

## Good to know

---

- Returns `false` for missing entities, so use this before reading a state with `states` to avoid chasing `unknown` or `unavailable` downstream.
- Treats both `unavailable` and `unknown` the same. If you only want to exclude one, compare to the specific state with `is_state`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Safely read a sensor value with a fallback

Only use the sensor value if it's available, otherwise show a default message.

#### template

```
template: |
 {% if has_value("sensor.outdoor_temperature") %}
 It's {{ states("sensor.outdoor_temperature") }}°C outside
 {% else %}
 Temperature sensor is not available
 {% endif %}
type: string
output: "It's 18.5°C outside"
```

### Guard a calculation

Prevent errors by checking that a sensor has a value before doing math with it.

#### template

```
template: |
 {% if has_value("sensor.power_usage") %}
 {{ states("sensor.power_usage") | float * 0.25 }}
 {% else %}
 0
 {% endif %}
type: float
output: "62.5"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/iif

---

The `iif` template function is a shorthand for if/else logic. Give it a condition and two values, and it returns the first when true, the second when false. Think of it as asking a yes/no question: *"Is this true? If yes, give me this. If no, give me that."*

In many places throughout Home Assistant, you'll want to change what is displayed or sent based on the current state of something. Maybe you want a notification to say *"The garage is open"* or *"The garage is closed"* depending on the actual state. Or you want to set a light brightness to 100 when you're home and 30 when you're away. Or show *"Armed"* or *"Disarmed"* on your dashboard. All of these are choices between two values based on a condition, and `iif` is designed exactly for that.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: " type: string output: warm

---

filter: " type: string output: open

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
iif(
 condition: Any,
 if_true: Any = True,
 if_false: Any = False,
 if_none: Any = None,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} condition: description: > The value to evaluate as a boolean. Truthy values return `if_true`, falsy values return `if_false`. required: true type: any if\_true: description: Value to return when the condition is truthy. required: false default: "True" type: any if\_false: description: Value to return when the condition is falsy. required: false default: "False" type: any if\_none: description: > Value to return when the condition is `None`. If not provided, `None` is treated as falsy and `if_false` is returned. required: false type: any

## Compared to an if block

---

`iif` replaces a multi-line `{% jinja %}{% endjinja %}` block with a single expression. These two templates produce the same result:

### template

```
template: |
 {% if is_state("binary_sensor.front_door", "on") %}
 open
 {% else %}
 closed
 {% endif %}
title: Using an if block
type: string
output: open
```

### template

```
template: |
 {{ is_state("binary_sensor.front_door", "on") | iif("open", "closed") }}
title: Using iif
type: string
output: open
```

Use `iif` when you need to choose between two values. For more complex logic with multiple branches, use `{% jinja %}{% endjinja %} / {% jinja %}{% elif %}{% endjinja %} / {% jinja %}{% endjinja %}` instead.

## Good to know

---

- All arguments are evaluated, even the branch that is not returned. Use an `if / else` block when one branch could raise an error.
- `None` is only handled separately when you pass `if_none`. Otherwise it is treated as falsy.
- Values like `0`, empty strings, and empty lists are falsy, so they return `if_false`, not `if_none`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## Caveats

---

### All arguments are always evaluated

Unlike an `{% jinja %}{% endjinja %}` block, `iif` evaluates *all* its arguments before deciding which one to return. This means both the `if_true` and `if_false` values are computed regardless of the condition.

In practice, this is rarely a problem since most arguments are plain strings or numbers. But be aware of it if your arguments contain expressions that could fail:

#### template

```
template: |
 {{
 has_value("sensor.temperature")
 | iif(
 states("sensor.temperature") | float,
 "unavailable"
)
 }}
title: This may cause an error
```

In this case, use an `{% jinja %}{% endjinja %}` block instead:

#### template

```
template: |
 {% if has_value("sensor.temperature") %}
 {{ states("sensor.temperature") | float }}
 {% else %}
 unavailable
 {% endif %}
title: Safer alternative
```

### None vs other falsy values

Values like `0`, empty strings, and `false` are all falsy. They return the `if_false` value, not the `if_none` value. Only a literal `None` triggers the `if_none` parameter. If you need to distinguish between `0` and `None`, you must use the `if_none` parameter.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Dynamic notification message

Send a notification that adapts its message based on the state of the garage door.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 The garage door is
 {{ is_state("cover.garage", "open") | iif("open", "closed") }}.
 {{
 is_state("cover.garage", "open")
 | iif("You might want to close it.", "")
 }}
```

### Handling None values

By default, `None` is treated as falsy and returns the `if_false` value. Use the third argument to handle `None` separately. This is useful when a sensor attribute might not exist.

#### template

```
template: |
 {{
 state_attr("sensor.weather", "temperature")
 | iif("has temp", "no temp", "sensor unavailable")
 }}
```

This returns: - `has temp` when the attribute has a truthy value (like `21.5`) - `no temp` when the attribute is a falsy value (like `0`) - `sensor unavailable` when the attribute is `None` (does not exist)



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/In Test

---

The `in` test checks whether a value is contained in a given sequence (list, tuple, or string). It returns `true` if the value is found and `false` otherwise. Use `value is in(sequence)` to perform the check.

This test is useful when you want to check if an entity state matches one of several possible values, or when filtering collections. While you can use Python's `in` operator directly in conditions ( `if state in ["on", "off"]` ), the `in` test can be used with filters like `select` and `reject` that expect a test name.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Found in list
```

type: string output: "Found in list"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
in(
 value: Any,
 seq: Sequence,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to search for in the sequence. required: true type: any seq: description: > The sequence (list, tuple, or string) to search in. required: true type: list

## Good to know

---

- String membership is case-sensitive. `"ON" is in(["on", "off"])` is `false`.
- On a string, this checks for substring presence rather than whole-word matching.
- Unlike Python's `in` operator, this is a Jinja test, so it works as a filter argument in `select` and `reject`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if state is one of several values

Verify that a sensor state matches an expected set of values.

#### template

```
template: |
 {% set state = states("climate.thermostat") %}
 {% if state is in(["heat", "cool", "auto"]) %}
 Climate is active: {{ state }}
 {% else %}
 Climate is idle
 {% endif %}
type: string
output: "Climate is active: heat"
```

### Check for substring in a string

The `in` test also works with strings, checking for substring membership.

#### template

```
template: '{{ "error" is in("Connection error occurred") }}'
type: boolean
output: "true"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Indent

---

The `indent` filter adds a specified number of spaces to the beginning of each line in a string. By default, the first line and blank lines are not indented, but you can change this behavior with the `first` and `blank` parameters. This is useful when building multi-line text output that needs proper formatting. For example, you might want to indent a block of YAML data in a notification, format a list of entity states for readable display, or align text within a larger multi-line message.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "line 1\nline 2\nline 3" | indent(4) }}' type: string output:
"line 1\n line 2\n line 3"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
indent(
 value: str,
 width: int = 4,
 first: bool = False,
 blank: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to indent. Each line (except optionally the first and blank lines) will be prefixed with the specified number of spaces. required: true type: string width: description: > The number of spaces to add to the beginning of each indented line. Defaults to `4`. required: false default: "4" type: integer first: description: > Whether to indent the first line as well. Defaults to `false`. required: false default: "false" type: boolean blank: description: > Whether to indent blank lines. Defaults to `false`. required: false default: "false" type: boolean

## Indenting the first line too

---

Set `first` to `true` to also indent the first line.

### template

```
template: '{{ "line 1\nline 2" | indent(2, first=true) }}'
title: Indent all lines including the first
type: string
output: " line 1\n line 2"
```

## Good to know

---

- The first line is not indented by default. Pass `first=true` to indent it along with the rest.
- Blank lines are also skipped by default. Pass `blank=true` to pad them too.
- The width is in spaces, not tab characters.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Format a multi-line notification body

Indent a block of status information within a notification message.

#### template

```
template: |
 Home status:
 {{ "Temperature: 22°C\nHumidity: 65%\nPressure: 1013 hPa" | indent(4) }}
type: string
output: |
 Home status:
 Temperature: 22°C
 Humidity: 65%
 Pressure: 1013 hPa
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Int

---

The `int` template function converts a value to an integer (whole number). If the value cannot be converted, it returns the default you provide instead of raising an error. Like `float`, it is "forgiving" and safe to use with sensor values that might sometimes be invalid.

In Home Assistant, all entity states are stored as strings. When you need a whole number for math, comparisons, or passing to an action, you need to convert first. The `int` function also supports a `base` parameter for converting hexadecimal, octal, or binary strings. If you need decimal precision, use `float` instead.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: integer output: "42"

---

filter: '' type: integer output: "87"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
int(
 value: Any,
 default: Any = _SENTINEL,
 base: int = 10,
) -> int | Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to convert to an integer. Strings representing whole numbers (like `42`) are converted. Non-numeric values raise an error unless a default is provided. required: true type: any default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. It is strongly recommended to always provide a default. required: false type: any base: description: > The numeric base to use for conversion. Defaults to `10` (decimal). Use `16` for hexadecimal, `8` for octal, or `2` for binary. required: false default: "10" type: integer

## Using a default value

---

Since sensor states can be `unavailable` or `unknown`, always provide a default to prevent your template from breaking.

### template

```
template: '{{ states("sensor.battery_level") | int(0) }}'
title: Safe conversion with default
type: integer
output: "87"
```

## Converting from other bases

---

Use the `base` parameter to convert hexadecimal, octal, or binary strings.

### template

```
template: '{{ int("ff", base=16) }}'
title: Convert hexadecimal to integer
type: integer
output: "255"
```

### template

```
template: '{{ int("0b1010", base=2) }}'
title: Convert binary to integer
type: integer
output: "10"
```



## Good to know

---

- Floats are truncated toward zero, not rounded. `2.9 | int` is `2`, and `-2.9 | int` is `-2`. Use `round` first if you want rounding.
- A string with decimals like `"21.5"` fails without a default. Pipe through `float` first or supply a default.
- The `base` parameter only applies to string inputs. Passing a number with `base=16` does not convert it.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Set a light brightness from a sensor

Use an entity state as a brightness value for a light. The `int` filter converts the string state to a whole number suitable for the brightness attribute.

#### action

```
action: |
 action:
 - action: light.turn_on
 target:
 entity_id: light.living_room
 data:
 brightness: >
 {{ states("sensor.ambient_light") | int(128) }}
```

### Count items with integer math

Calculate how many full hours have passed since a counter was last reset.

#### template

```
template: |
 {{
 ((as_timestamp(now())
 - as_timestamp(states.counter.runtime.last_changed)) / 3600)
 | int(0)
 }}
type: integer
output: "3"
```

### Use in a condition

Only proceed if the battery level is above a threshold.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ states("sensor.phone_battery") | int(0) > 20 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Integer Test

---

The `integer` test checks whether a value is an integer. It returns `true` if the value is of integer type and `false` for any other type, including floats and strings that contain numeric characters.

This test is useful when you need to verify that a value is specifically an integer rather than a float or string. For example, some entity attributes return integers for discrete counts (like the number of connected devices) while others return floats for measurements. This test lets you confirm the type before proceeding with type-specific logic.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is an integer
```

type: string output: "It is an integer"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
integer(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is of integer type. required: true type: any



## Good to know

---

- Booleans pass this test because `True` and `False` are integers in Python. Guard with `is not boolean` if that matters.
- Numeric strings like `"42"` do not pass. Convert with `int` first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distinguish integer from float

Check whether a numeric value is specifically an integer.

#### template

```
template: |
 {{ 42 is integer }}
 {{ 42.0 is integer }}
 {{ "42" is integer }}
type: boolean
output: |
 true
 false
 false
```

### Validate before integer operations

Ensure a value is an integer before using it in an operation that requires whole numbers.

#### template

```
template: |
 {% set count = state_attr("sensor.devices", "count") %}
 {% if count is integer %}
 {{ count }} devices connected
 {% else %}
 Invalid count
 {% endif %}
type: string
output: "5 devices connected"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Integration Entities

---

The `integration_entities` template function returns a list of entity IDs that belong to a specific integration or config entry. You can pass either a domain name (like `hue`) to get all entities for that integration, or a config entry title (like `Living Room Hue Bridge`) to narrow it down to a specific instance.

This is especially useful when you have multiple instances of the same integration and need to work with entities from only one of them. For example, if you have two Hue bridges, you can get the entities for each one separately by using their config entry titles. You can also use this to count how many entities an integration has created, or to check the state of all entities that belong to a particular integration at once.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: list output: | ["light.living_room",
"light.bedroom", "sensor.hue_motion",]
```

---

```
filter: '{{ "hue" | integration_entities }}' type: list output: | ["light.living_room", "light.bedroom",
"sensor.hue_motion",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
integration_entities(
 entry_name: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entry\_name: description: > The integration domain name (for example, `hue`) or config entry title (for example, `Living Room Hue Bridge`). When a config entry title matches, only entities from that specific config entry are returned. Otherwise, all entities for the domain are returned. required: true type: string



## Good to know

---

- A domain name returns entities from every config entry of that integration. Use a config entry title to narrow down to one instance.
- Config entry title matches take precedence over domain matches, so a title that looks like a domain name is matched first as a title.
- Returns an empty list when nothing matches, rather than raising an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count entities for an integration

Find out how many entities a specific integration has registered.

#### template

```
template: '{{ integration_entities("zwave_js") | count }}'
type: integer
output: "42"
```

### Check if any entity from an integration is unavailable

Loop through all entities from an integration and check if any are in the `unavailable` state.

#### template

```
template: |
 {{
 integration_entities("mqtt")
 | select("is_state", "unavailable")
 | list
 | count > 0
 }}
type: boolean
output: "false"
```

### Get entities from a specific config entry

When you have multiple instances of the same integration, use the config entry title to get entities for only one of them.

#### template

```
template: '{{ integration_entities("Living Room Hue Bridge") }}'
type: list
output: |
 [
 "light.living_room_ceiling",
 "light.living_room_lamp",
]
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Intersect

---

The `intersect` template function returns a list of items that appear in both of the two lists you provide. This is the set intersection operation: only items that exist in both lists are included in the result.

This is useful when you want to find what two collections have in common. For example, you might have a list of entities that are on and a list of entities in a specific room, and you want to know which entities in that room are currently on. Or you might want to find which devices appear in two different groups, or determine which family members are in both the "home" zone and the "allowed" list. Duplicates are removed from the result since it operates as a set operation.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: list output: "[3, 4]"

---

filter: '{{ [1, 2, 3, 4] | intersect([3, 4, 5, 6]) }}' type: list output: "[3, 4]"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
intersect(
 value: list,
 other: list,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The first list to compare. Must be a list or collection. required: true type: list other: description: > The second list to compare against. Must be a list or collection. required: true type: list



## Good to know

---

- Duplicates are removed in the result because this works as a set operation.
- Order is not preserved. Add `| sort` if you need a consistent order.
- Returns an empty list when the two lists share nothing.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find entities that are in both groups

Determine which entities appear in two different groups.

#### template

```
template: |
 {% set group_a = expand("group.downstairs_lights")
 | map(attribute="entity_id") | list %}
 {% set group_b = expand("group.automated_lights")
 | map(attribute="entity_id") | list %}
 {{ intersect(group_a, group_b) }}
type: list
output: '["light.kitchen", "light.hallway"]'
```

### Find common items between two sensor lists

When two sensors each report a list of detected items, find which items are detected by both.

#### template

```
template: |
 {% set cam1 = state_attr("sensor.camera_1", "detected_objects") %}
 {% set cam2 = state_attr("sensor.camera_2", "detected_objects") %}
 {{ intersect(cam1, cam2) }}
type: list
output: '["person", "car"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Is Defined

---

The `is_defined` filter checks that a value is not undefined and forces a template error if it is. If the value is defined, it passes through unchanged. This is a safety mechanism to catch typos and missing variables early rather than having them silently produce empty strings.

By default, Home Assistant is configured to be lenient with undefined variables, meaning a misspelled variable name produces an empty string instead of an error. While this is convenient for smaller templates, it can make debugging very difficult. If you write `{% jinja %}{% endjinja %}` instead of `{% jinja %}{% endjinja %}`, you get a blank value with no indication that anything is wrong. The `is_defined` filter lets you opt in to strict checking for specific variables, so you get an immediate error that points to the problem.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: any output: "the value of my_variable"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
is_defined(
 value: Any,
) -> Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The value to check. If the value is an `Undefined` object, an error is raised. Otherwise, the value is returned unchanged. required: true type: any



## How it works

---

When you reference a variable that does not exist in a template, it becomes an `Undefined` object. Normally, this silently renders as an empty string. The `is_defined` filter detects this and raises an error instead, making the problem visible.

### template

```
template: '{{ my_var | is_defined }}'
title: "Raises an error if my_var does not exist"
```

💡 **Tip:** This filter is most useful in scripts and automations where variables are passed in from outside. It helps you catch cases where a required variable was not provided.

## Good to know

---

- This raises an error on undefined input, which is the opposite of most filters. Use it intentionally to enforce a variable's presence.
- A value set to `None` is considered defined and passes through without error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Validate a script variable

Ensure that a required variable was passed into a script before using it.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 Reminder: {{ message | is_defined }}
```

If the `message` variable was not provided when calling the script, this raises a clear error instead of sending an empty notification.

### Guard multiple variables

Apply `is_defined` to each variable you depend on at the start of a template.

#### template

```
template: |
 {% set name = target_name | is_defined %}
 {% set room = target_room | is_defined %}
 {{ name }} is in the {{ room }}
type: string
output: "Alice is in the Kitchen"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Is Device Attr

---

The `is_device_attr` template function checks whether a specific attribute of a device matches a given value. It returns `true` or `false`. This is the device-level equivalent of `is_state_attr`, working with the device registry instead of entity states.

This is useful when you want to make decisions in automations or templates based on device properties. For example, you could check if a device is made by a specific manufacturer, runs a particular firmware version, or matches a certain model. You might use it to apply different logic depending on the hardware vendor, or to trigger an alert when a device's software version doesn't match the expected value after an update.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: boolean output: "true"
```

---

test: | This is a Philips device!

type: string output: "This is a Philips device!"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
is_device_attr(
 device_or_entity_id: str,
 attr_name: str,
 attr_value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} `device_or_entity_id`: description: > The device ID or entity ID to check. When using an entity ID, the function first resolves the device the entity belongs to. required: true type: string `attr_name`: description: > The name of the device attribute to test. Common attributes include `manufacturer`, `model`, `sw_version`, `hw_version`, and `serial_number`. required: true type: string `attr_value`: description: The value to compare the attribute against. required: true type: any

## Good to know

---

- Returns `false` (not an error) when the device or attribute does not exist.
- String comparison is case-sensitive. `is_device_attr(id, "manufacturer", "philips")` does not match `"Philips"`.
- Accepts either a device ID or an entity ID. With an entity ID, the device is resolved first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if a device is a specific model

Verify that a device matches an expected model, for example to apply model-specific logic in an automation.

#### template

```
template: '{{ is_device_attr("light.living_room", "model", "LCA001") }}'
type: boolean
output: "true"
```

### Use in an automation condition

Only run the rest of the automation if the device is from a specific manufacturer.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ is_device_attr("sensor.front_door", "manufacturer", "Aqara") }}
```

### Check firmware version across devices in an area

Combine with `area_devices` to find devices in an area that are running a specific firmware version.

#### template

```
template: |
 {% for dev_id in area_devices("Living Room") %}
 {% if is_device_attr(dev_id, "sw_version", "1.104.2") %}
 {{ device_name(dev_id) }}
 {% endif %}
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Is Hidden Entity

---

The `is_hidden_entity` template function checks whether an entity has been marked as hidden in the entity registry. It returns `true` if the entity exists and is hidden, and `false` otherwise.

Hidden entities are ones you have chosen to hide from the default user interface. They still exist and still track state, but they are not shown on dashboards by default. This function is useful when you are looping over entities in an area or domain and want to skip the ones that have been hidden. For example, you might want to count how many visible sensors are in a room, or build a dynamic dashboard card that only lists entities you have not hidden.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: boolean output: "true"
```

---

test: | This entity is hidden

type: string output: "This entity is hidden"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
is_hidden_entity(
 entity_id: str,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: > The entity ID to check. Returns `true` if the entity exists in the registry and is hidden. Returns `false` if the entity is not hidden or does not exist. required: true type: string

## Good to know

---

- Returns `false` for entities that do not exist in the registry, not just for visible ones.
- This checks the registry "hidden" flag only. It does not detect entities filtered out by dashboard cards or custom visibility.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count visible entities in an area

Count the number of entities in an area that are not hidden, so you know how many are actually shown on the dashboard.

#### template

```
template: |
 {{
 area_entities("Living Room")
 | reject("is_hidden_entity")
 | list
 | count
 }}
type: integer
output: "8"
```

### Filter out hidden entities from a list

Build a list of only the visible sensor entities in the kitchen.

#### template

```
template: |
 {% for entity_id in area_entities("Kitchen")
 | select("match", "sensor.")
 | reject("is_hidden_entity") %}
 {{ entity_id }}
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Is Number

---

The `is_number` template function tests whether a value can be converted to a valid, finite floating-point number. It returns `true` if the value is numeric and `false` otherwise. Values like `infinity` and `NaN` are not considered numbers and return `false`.

This is a safety check you should use before performing math on values that might not be numeric. Entity states in Home Assistant can be `unavailable`, `unknown`, or other non-numeric strings at any time. Trying to do math on these values without checking first causes errors. By testing with `is_number` first, you can branch your logic to handle numeric and non-numeric cases separately. It is also useful as a template test in `{% jinja %}{% endjinja %}` blocks.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: boolean output: "true"
```

---

```
filter: '' type: boolean output: "true"
```

---

```
test: |
```

```
It is a number!
```

```
type: string output: "It is a number!"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
is_number(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value can be converted to a finite float. Returns `false` for non-numeric strings, `None`, `infinity`, and `NaN`. required: true type: any

## What counts as a number

---

The function attempts to convert the value to a float. If the conversion succeeds and the result is a finite number (not `infinity` or `NaN`), it returns `true`.

### template

```
template: |
 {{ is_number(42) }}
 {{ is_number("21.5") }}
 {{ is_number("unavailable") }}
 {{ is_number("inf") }}
title: Various inputs
type: boolean
output: |
 true
 true
 false
 false
```

## Good to know

---

- Returns `false` for `unavailable`, `unknown`, `infinity`, and `NaN`, which catches common unsafe cases for math.
- Booleans pass this test because Python treats `True` and `False` as `1` and `0`.
- Numeric strings like `"21.5"` pass. This checks convertibility, not the current type.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Guard a calculation

Only perform math if the sensor value is actually a number.

#### template

```
template: |
 {% if states("sensor.power_usage") | is_number %}
 {{ states("sensor.power_usage") | float * 0.25 }}
 {% else %}
 Sensor is not reporting a number
 {% endif %}
type: string
output: "62.5"
```

### Filter a list to only numeric values

When working with multiple sensors, filter out any that are not reporting numbers before calculating an average.

#### template

```
template: |
 {{
 ["21.5", "unavailable", "19.8", "unknown"]
 | select("is_number")
 | map("float")
 | list
 | average
 }}
type: float
output: "20.65"
```

### Use in an automation condition

Only proceed if the sensor is reporting a valid number.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ states("sensor.wind_speed") | is_number }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Is State

---

The `is_state` template function tests if an entity is in a specific state. It returns `true` or `false`. This is probably the template function you'll use most often, and it's the recommended way to check what state something is in.

Whenever you need to make a decision based on the current state of a device or sensor, `is_state` is the way to do it. Is the alarm armed? Is someone home? Is a door open? Is the washing machine still running? These are all yes/no questions about the state of an entity, and `is_state` answers them. It's also safer than comparing states directly, because it will never throw an error if the entity doesn't exist yet (for example, during Home Assistant startup). It returns `false`.

 **Tip:** Automation conditions and triggers already let you check an entity's state through the visual editor, no template needed. Reach for `is_state()` when you need the result inside a template expression, notification message, or template sensor.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: boolean output: "true"
```

---

test: | Phone is home!

type: string output: "Phone is home!"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
is_state(
 entity_id: str,
 state: str | list[str],
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: The entity ID to check. required: true type: string  
state: description: > The state value to compare against. Can be a single string or a list of strings to match against any of them. required: true type: [string, list]

## Checking against multiple states

---

You can pass a list of states to check against. The function returns `true` if the entity's state matches any value in the list.

### template

```
template: '{{ is_state("vacuum.roborock", ["cleaning", "returning"]) }}'
type: boolean
output: "true"
```

## Good to know

---

- Returns `false` (not an error) when the entity does not exist, making it safer than string comparison during startup.
- The state to compare against is always a string. Numeric states need to be wrapped in quotes, like `is_state("sensor.x", "25")`.
- Passing a list of states returns `true` if any of them match, acting like an OR check.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Use in an automation condition

Only run the rest of the automation if the alarm is armed away.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ is_state("alarm_control_panel.home", "armed_away") }}
```

### Combine with iif in a notification

Send a notification that includes the current state of the front door, using `iif` to convert the state into a human-readable word.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 The front door is
 {{
 iif(is_state("binary_sensor.front_door", "on"),
 "open", "closed")
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---



## Template Functions/Is State Attr

---

The `is_state_attr` template function checks whether a specific attribute of an entity matches a given value. It returns `true` or `false`. This is the attribute equivalent of `is_state`, which checks the main state.

You'll use this when you need to make decisions based on an entity's attributes rather than its main state. For example, checking if a light's color mode is set to `color_temp`, if a media player's source is a specific input, or if a device's battery level is at a certain value. Like `is_state`, it safely returns `false` if the entity doesn't exist.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: boolean output: "true"
```

---

test: | Light is in color temperature mode

type: string output: "Light is in color temperature mode"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
is_state_attr(
 entity_id: str,
 attribute: str,
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: The entity ID to check. required: true type: string  
attribute: description: The name of the attribute to test. required: true type: string value:  
description: The value to compare the attribute against. required: true type: any

 **Tip:** To find the exact attribute names and values for an entity, go to **Developer Tools > States** and select the entity.

## Good to know

---

- Returns `false` (not an error) when the entity or attribute does not exist.
- The comparison is type-strict. `is_state_attr("light.x", "brightness", "255")` is `false` when the attribute is the integer `255`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if a media player is playing from a specific source

#### template

```
template: '{{ is_state_attr("media_player.tv", "source", "HDMI 1") }}'
type: boolean
output: "true"
```

### Use in an automation condition

Only proceed if the climate system is in heating mode.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ is_state_attr("climate.living_room", "hvac_action", "heating") }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Issue

---

The `issue` template function looks up a specific repair issue by its domain and issue ID. Repair issues are problems that Home Assistant has detected and flagged for your attention. This function returns the issue as a dictionary if it exists and is active, or `None` if no such issue is found.

This is useful when you want to check whether a specific known issue is currently active. For example, you might want to monitor whether a particular integration has a configuration problem, display the status of a specific repair on your dashboard, or trigger an automation only when a specific issue is present.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: dict output: '{"domain': 'hue', 'issue_id': 'deprecated_bridge', 'severity': 'warning', ...}'"
```

---

```
filter: '{{ "deprecated_bridge" | issue("hue") }}' type: dict output: '{"domain': 'hue', 'issue_id': 'deprecated_bridge', 'severity': 'warning', ...}'"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
issue(
 domain: str,
 issue_id: str,
) -> dict[str, Any] | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} domain: description: > The integration domain that created the issue (for example, `hue`, `zwave`, or `mqtt`). required: true type: string issue\_id: description: > The unique identifier for the specific issue within the domain. required: true type: string

## Checking if an issue exists

---

Since the function returns `None` when the issue is not found, you can use it directly in conditions.

### template

```
template: |
 {% if issue("hue", "deprecated_bridge") %}
 Hue bridge issue is active
 {% else %}
 No Hue bridge issue
 {% endif %}
title: Check for a specific issue
type: string
output: "Hue bridge issue is active"
```

## Good to know

---

- Returns `None` when the issue does not exist or has been dismissed, so checks like `if issue(...)` work directly.
- Only active (non-ignored) issues are returned.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display issue severity

Look up a specific issue and show its severity level.

#### template

```
template: |
 {% set hue_issue = issue("hue", "deprecated_bridge") %}
 {% if hue_issue %}
 Hue issue severity: {{ hue_issue.severity }}
 {% else %}
 No issue found
 {% endif %}
type: string
output: "Hue issue severity: warning"
```

### Conditional notification based on issue

Send a notification only when a specific repair issue is active.

#### action

```
action: |
 action:
 - condition: template
 value_template: '{{ issue("zwave", "config_error") is not none }}'
 - action: notify.mobile
 data:
 message: "Z-Wave configuration error detected. Check repairs."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Issues

---

The `issues` template function returns all currently active repair issues in Home Assistant. Repair issues are problems that Home Assistant has detected and flagged for your attention, such as deprecated integrations, configuration errors, or devices that need manual intervention. Each issue is returned as a dictionary keyed by a tuple of `(domain, issue_id)`.

This is useful for building dashboard cards or automations that monitor the health of your Home Assistant installation. For example, you might want to display a count of open issues on your dashboard, send a notification when new issues appear, or create a template sensor that tracks whether any critical repairs are pending.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: integer output: "3"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
issues() -> dict[tuple[str, str], dict[str, Any]]
```

## Function parameters

This function takes no parameters. It returns all currently active repair issues.

## Understanding repair issues

---

Repair issues are created by integrations when they detect a problem that requires user attention. Each issue contains information such as:

- `domain` - The integration that created the issue
- `issue_id` - A unique identifier for this specific issue
- `severity` - How critical the issue is ( `error` , `warning` , or `other` )
- `translation_key` - A key used to look up the translated issue description

### template

```
template: |
 {% for key, issue in issues().items() %}
 {{ key }}: {{ issue.severity }}
 {% endfor %}
title: List all issues with severity
type: string
output: |
 ('hue', 'deprecated_bridge'): warning
 ('zwave', 'config_error'): error
```

## Good to know

---

- Returns a dictionary keyed by a `(domain, issue_id)` tuple, not a plain list.
- Severity values are `error`, `warning`, or `other`. Filter by severity when you only care about a subset.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count open issues

Display the total number of active repair issues on a dashboard.

#### template

```
template: |
 {% set all_issues = issues() %}
 {{ all_issues | length }} repair issue({ "s" if all_issues | length != 1 })
type: string
output: "3 repair issues"
```

### Check for critical issues

Look for any issues with error severity and notify if found.

#### template

```
template: |
 {% set errors = issues().values()
 | selectattr("severity", "eq", "error") | list %}
 {% if errors | length > 0 %}
 {{ errors | length }} critical issue({ "s" if errors | length != 1 }) found
 {% else %}
 All clear
 {% endif %}
type: string
output: "1 critical issue found"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Items

---

The `items` filter returns the key/value pairs from a dictionary as a list of `(key, value)` tuples. This makes it possible to iterate over a dictionary's entries in a template, accessing both the key and the value in each iteration.

This is useful when you need to loop over the attributes of an entity or any other dictionary structure. For example, you might want to display all attributes of a sensor along with their values, process configuration options stored as a dictionary, or transform a dictionary into another format. It converts a dictionary into a format that works with the template `for` loop unpacking.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ {"a": 1, "b": 2, "c": 3} | items | list }}' type: list output:
"[(('a', 1), ('b', 2), ('c', 3))]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
items(
 value: dict,
) -> list[tuple]
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The dictionary to extract key/value pairs from.  
required: true type: map

## Good to know

---

- Returns an iterable of `(key, value)` tuples, not a list. Wrap with `| list` when counting or reusing.
- Order depends on how the dictionary was built. Use `dictsort` for a guaranteed sort order.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display all entity attributes

Iterate over all attributes of a sensor and display them.

#### template

```
template: |
 {% for key, value in state_attr("climate.living_room", "hvac_modes")
 | default({}) | items %}
 {{ key }}: {{ value }}
 {% endfor %}
type: string
output: ""
```

### Iterate over a configuration dictionary

Loop over a dictionary of settings and display each one.

#### template

```
template: |
 {% set config = {"mode": "heat", "target_temp": 22, "fan": "auto"} %}
 {% for key, value in config | items %}
 {{ key }} = {{ value }}
 {% endfor %}
type: string
output: |
 mode = heat
 target_temp = 22
 fan = auto
```

### Filter dictionary entries

Combine `items` with `selectattr` or conditional logic to filter dictionary entries.

#### template



```
template: |
 {% set temps = {"kitchen": 22.5, "bedroom": 19.8, "living_room": 21.3} %}
 {% for room, temp in temps | dictsort %}
 {% if temp > 20 %}
 {{ room }}: {{ temp }}°C
 {% endif %}
 {% endfor %}
type: string
output: |
 kitchen: 22.5°C
 living_room: 21.3°C
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Iterable

---

The `iterable` test checks whether a value can be iterated over. It returns `true` for lists, tuples, strings, dictionaries, generators, and other iterable types. It returns `false` for non-iterable types like numbers, booleans, and `None`.

This is useful when you need to verify that a value can be looped over with `{% jinja %}{% endjinja %}` before attempting to do so. Some entity attributes might be a list in some cases and a single value in others. Testing with `iterable` prevents errors when the value turns out not to be something you can loop over.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is iterable
```

```
type: string output: "It is iterable"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
iterable(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value can be iterated over. required: true type: any

## Good to know

---

- Strings pass this test because they are iterable character by character.
- Use `sequence` if you want to exclude dictionaries or use `mapping` to match only dictionaries.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various types

Lists, strings, and dictionaries are iterable. Numbers and booleans are not.

#### template

```
template: |
 {{ [1, 2] is iterable }}
 {{ "hello" is iterable }}
 {{ 42 is iterable }}
type: boolean
output: |
 true
 true
 false
```

### Guard a loop

Only attempt to iterate over an attribute if it is actually iterable.

#### template

```
template: |
 {% set items = state_attr("sensor.device", "features") %}
 {% if items is iterable %}
 {% for item in items %}
 - {{ item }}
 {% endfor %}
 {% else %}
 No features available
 {% endif %}
type: string
output: |
 - wifi
 - bluetooth
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Join

---

The `join` filter concatenates a list of values into a single string, placing a separator between each element. You can also join a specific attribute of a list of objects. This is useful whenever you need to turn a list of items into a readable string. For example, you might want to list the names of entities that are currently on, build a comma-separated summary for a notification, or join a list of room names for display on a dashboard.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ ["living room", "kitchen", "bedroom"] | join(", ") }}' type:
string output: "living room, kitchen, bedroom"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
join(
 value: list,
 separator: str = "",
 attribute: str | None = None,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The list of values to join into a string. Each element is converted to a string before joining. required: true type: list separator: description: > The string to place between each element. Defaults to an empty string (no separator). required: false default: "" type: string attribute: description: > If the list contains objects, join by this attribute instead of the object itself. required: false type: string

## Joining without a separator

---

By default, elements are concatenated directly with no separator between them.

### template

```
template: '{{ ["a", "b", "c"] | join }}'
title: Concatenate letters
type: string
output: abc
```

## Good to know

---

- The default separator is an empty string, so without an argument the items are concatenated directly.
- Each item is converted to a string automatically, so you can join numbers, booleans, and other types.
- Every item must be concrete. Iterables produced by `select` or `selectattr` need no conversion, but generators should be materialized first.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### List active lights in a notification

Build a human-readable list of lights that are currently on.

#### template

```
template: |
 {% set lights = ["Living room", "Kitchen", "Porch"] %}
 Lights on: {{ lights | join(", ") }}
type: string
output: "Lights on: Living room, Kitchen, Porch"
```

### Build a path from parts

Join path segments together with a separator.

#### template

```
template: '{{ ["home", "user", "config"] | join("/") }}'
type: string
output: home/user/config
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Joiner

---

The `joiner` template function creates a small helper object for joining items in a loop with a separator. When you call the joiner, it returns an empty string the first time and the separator string on every subsequent call. This avoids the common problem of getting a leading or trailing separator when building a delimited string in a loop.

This provides an alternative to the `join` filter when you need more control over how items are separated. For example, if you are conditionally including items in a loop, the `join` filter may not work cleanly because it operates on a full list. A `joiner` only emits the separator between items that are actually rendered.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: |

type: string output: "apple, banana, cherry"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
joiner(
 separator: str = ", ",
) -> Joiner
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} separator: description: > The string to insert between items. Defaults to `", "`. required: false default: `","` type: string

## Good to know

---

- The first call returns an empty string, so you can place it before each item without worrying about a leading separator.
- The default separator is `" , "` with a trailing space.
- A fresh `joiner()` must be created per loop. Reusing one across loops keeps counting from where it left off.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Conditionally join active sensors

Build a string of only the sensors that are on, properly separated.

#### template

```
template: |
 {% set sep = joiner(" | ") %}
 {% for entity in ["binary_sensor.front_door", "binary_sensor.back_door",
 "binary_sensor.garage"] %}
 {% if is_state(entity, "on") %}
 {{ sep() }}{{ state_attr(entity, "friendly_name") }}
 {% endif %}
 {% endfor %}
type: string
output: "Front Door | Garage"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Label Areas

---

The `label_areas` template function returns a list of area IDs that have a specific label assigned. You can specify the label by its name or by its internal ID. This gives you all areas tagged with that label.

This is useful when you organize your areas using labels and want to act on groups of rooms. For example, if you label certain areas as "*Outdoor*", you could use `label_areas` to find all outdoor areas and then turn off their lights at night, or check the temperature in all rooms labeled "*Heated*". As you add or remove labels from areas, the list automatically updates, so your automations and templates always stay in sync.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["patio", "garden", "driveway",]
```

---

```
filter: '{{ "Outdoor" | label_areas }}' type: list output: | ["patio", "garden", "driveway",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
label_areas(
 label_id_or_name: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} label\_id\_or\_name: description: > The label name or ID to look up. Returns the area IDs that have this label assigned. You can find labels in **Settings > Areas, labels & zones**. required: true type: string

## Good to know

---

- Returns an empty list when the label does not exist or no areas carry it.
- Accepts either the label name or the label ID, matching by name first.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many areas have a label

Find out how many areas are tagged with a specific label.

#### template

```
template: '{{ label_areas("Outdoor") | count }}'
type: integer
output: "3"
```

### List area names with a specific label

Get the friendly names of all areas that have a given label by combining `label_areas` with `area_name`.

#### template

```
template: |
 {{
 label_areas("Outdoor")
 | map("area_name")
 | join(", ")
 }}
type: string
output: "Patio, Garden, Driveway"
```

### Turn off lights in all areas with a label

Use in an automation to find all areas tagged with a label and act on their entities. This example collects all light entities from areas labeled "Outdoor".

#### template

```
template: |
 {% for area_id in label_areas("Outdoor") %}
 {% for entity_id in area_entities(area_id)
 | select("match", "light.") %}
 {{ entity_id }}
 {% endfor %}
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Label Description

---

The `label_description` template function returns the description text of a label from its internal ID. When you create a label in Home Assistant, you can optionally give it a description that explains its purpose, and this function lets you retrieve that description.

This is useful when you want to display additional context about a label beyond its name. For example, if you have a label called *"Critical"* with a description like *"Sensors that trigger emergency alerts"*, you could display that description in a dashboard or use it in notifications to provide more context. If the label has no description set, the function returns `None`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output: "Sensors that trigger emergency alerts"
```

---

```
filter: '{{ "critical" | label_description }}' type: string output: "Sensors that trigger emergency alerts"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
label_description(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The label ID to look up. Returns the description of the label, or `None` if no label with that ID exists or the label has no description.  
required: true type: string

## Good to know

---

- Returns `None` when the label does not exist or has no description.
- Only accepts a label ID, not a label name. Resolve a name with `label_id` first.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a label's description

Retrieve the description text for a specific label.

#### template

```
template: '{{ label_description("energy_monitoring") }}'
type: string
output: "Devices and sensors used for tracking energy usage"
```

### List labels with their descriptions

Loop through all labels and display their names along with descriptions, using `labels` and `label_name`.

#### template

```
template: |
 {% for lbl_id in labels() %}
 {{ label_name(lbl_id) }}: {{ label_description(lbl_id) }}
 {% endfor %}
type: string
output: |
 Outdoor: Items located outside the house
 Critical: Sensors that trigger emergency alerts
 Energy Monitoring: Devices and sensors used for tracking energy usage
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Label Devices

---

The `label_devices` template function returns a list of device IDs that have a specific label assigned. You can specify the label by its name or by its internal ID. This gives you all devices tagged with that label.

This is useful when you organize your devices using labels and want to act on or inspect a specific group. For example, if you label battery-powered devices as "*Battery*", you could use `label_devices` to find them all and check their battery levels, or if you label certain devices as "*Guest Room*", you could quickly find and control all devices for your guests. As you add or remove labels from devices, the list automatically updates, so your automations and templates always reflect the current state.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["a1b2c3d4e5f6a1b2c3d4e5f6",
"f6e5d4c3b2a1f6e5d4c3b2a1",]
```

---

```
filter: '{{ "Battery" | label_devices }}' type: list output: | ["a1b2c3d4e5f6a1b2c3d4e5f6",
"f6e5d4c3b2a1f6e5d4c3b2a1",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
label_devices(
 label_id_or_name: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} label\_id\_or\_name: description: > The label name or ID to look up. Returns the device IDs that have this label assigned. You can find labels in **Settings > Areas, labels & zones**. required: true type: string

## Good to know

---

- Only returns devices with the label assigned directly. Labels applied to entities or areas do not roll up to their devices.
- Returns an empty list when the label does not match any devices.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count devices with a specific label

Find out how many devices are tagged with a given label.

#### template

```
template: '{{ label_devices("Battery") | count }}'
type: integer
output: "2"
```

### Get device names for a label

List the friendly names of all devices that have a specific label by combining `label_devices` with `device_name`.

#### template

```
template: |
 {% for device_id in label_devices("Battery") %}
 {{ device_name(device_id) }}
 {% endfor %}
type: string
output: |
 Living Room Motion Sensor
 Front Door Sensor
```

### Check battery levels of labeled devices

Find all devices with a *"Battery"* label and check their battery sensor entities. This collects the battery-level entities for all labeled devices.

#### template

```

template: |
 {% for device_id in label_devices("Battery") %}
 {% set batteries = device_entities(device_id)
 | select("match", "sensor.")
 | select("search", "battery")
 | list %}
 {% for entity_id in batteries %}
 {{ device_name(device_id) }}: {{ states(entity_id) }}%
 {% endfor %}
 {% endfor %}
type: string
output: |
 Living Room Motion Sensor: 85%
 Front Door Sensor: 42%

```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Label Entities

---

The `label_entities` template function returns a list of entity IDs that have a specific label assigned. You can specify the label by its name or by its internal ID. This gives you all entities tagged with that label.

This is useful when you want to act on a group of entities that share a common label, regardless of which area or device they belong to. For example, if you label certain sensors as *"Critical"*, you could use `label_entities` to monitor all of them at once, or if you label energy-related entities as *"Energy Monitoring"*, you could build a dashboard that automatically includes every labeled entity. As you add or remove labels from entities, the list automatically updates, so your automations and templates always stay current.

 **Tip:** Automation actions can target entities by label through the visual editor, no template needed. Reach for `label_entities()` when you need to loop over or filter the entities inside a template expression.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["sensor.living_room_temperature",
"binary_sensor.front_door", "binary_sensor.smoke_detector",]
```

---

```
filter: '{{ "Critical" | label_entities }}' type: list output: | ["sensor.living_room_temperature",
"binary_sensor.front_door", "binary_sensor.smoke_detector",]
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
label_entities(
 label_id_or_name: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} label\_id\_or\_name: description: > The label name or ID to look up. Returns the entity IDs that have this label assigned. You can find labels in **Settings > Areas, labels & zones**. required: true type: string



## Good to know

---

- Only returns entities with the label assigned directly. Labels applied to devices or areas do not roll up to their entities.
- Returns an empty list when the label does not match any entities.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count entities with a specific label

Find out how many entities are tagged with a given label.

#### template

```
template: '{{ label_entities("Critical") | count }}'
type: integer
output: "3"
```

### Check if any critical entity is unavailable

Monitor all entities with a specific label and report if any of them are unavailable.

#### template

```
template: |
 {{
 label_entities("Critical")
 | select("is_state", "unavailable")
 | list
 | count > 0
 }}
type: boolean
output: "false"
```

### List the states of all labeled entities

Display the current state of every entity that has a given label.

#### template

```
template: |
 {% for entity_id in label_entities("Energy Monitoring") %}
 {{ state_attr(entity_id, "friendly_name") }}: {{ states(entity_id) }}
 {% endfor %}
type: string
output: |
 Living Room Power: 120
 Kitchen Power: 85
 Dryer Power: 2400
```

## Sum up energy usage from labeled entities

Calculate the total value across all entities with a specific label. This is useful for energy monitoring dashboards.

### template

```
template: |
 {{
 label_entities("Energy Monitoring")
 | map("states")
 | map("float", default=0)
 | sum
 }}
type: float
output: "2605.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Label Id

---

The `label_id` template function returns the unique label ID for a given label name. Every label in Home Assistant has an internal ID that stays the same even if you rename the label, and this function lets you look it up from the human-readable name.

This is useful when you need the label ID to pass to other label functions like `label_areas`, `label_devices`, or `label_entities`. For example, you could use it in an automation to dynamically find all entities or devices with a particular label by name, without needing to know the internal ID ahead of time.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output: "critical"
```

---

```
filter: '{{ "Critical" | label_id }}' type: string output: "critical"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
label_id(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The name of the label to look up. Returns the matching label ID, or `None` if no label with that name exists. You can find labels in **Settings** > **Areas, labels & zones**. required: true type: string

## Good to know

---

- Returns `None` when no label matches the name.
- The match is case-sensitive. `"critical"` does not match a label named `"Critical"`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Look up a label ID by name

Find the internal ID of a label using its friendly name.

#### template

```
template: '{{ label_id("Energy Monitoring") }}'
type: string
output: "energy_monitoring"
```

### Use label ID to find all entities with that label

Combine `label_id` with `label_entities` to find all entities that have a specific label, using the label's friendly name.

#### template

```
template: '{{ label_entities(label_id("Critical")) }}'
type: list
output: |
 [
 "sensor.living_room_temperature",
 "binary_sensor.front_door",
]
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Label Name

---

The `label_name` template function returns the friendly, human-readable name of a label from its internal ID. Every label in Home Assistant has an ID (like `energy_monitoring`) and a display name (like *"Energy Monitoring"*), and this function converts the ID to the name.

This is especially useful for building dynamic messages and notifications. Instead of showing a technical label ID, you can display the actual name you (or whoever receives the message) recognize. For example, when listing the labels applied to an entity or device, you could convert each label ID returned by `labels` into its friendly name for a more readable output.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output: "Critical"
```

---

```
filter: '{{ "critical" | label_name }}' type: string output: "Critical"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
label_name(
 lookup_value: str,
) -> str | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > The label ID to look up. Returns the friendly name of the label, or `None` if no label with that ID exists. required: true type: string

## Good to know

---

- Returns `None` when the label does not exist.
- Only takes a label ID, not a name. The name comes back as the display string you see in the UI.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a label's friendly name

Convert a label ID into its human-readable name.

#### template

```
template: '{{ label_name("energy_monitoring") }}'
type: string
output: "Energy Monitoring"
```

### List all label names

Loop through all labels and display their friendly names using `labels` combined with `label_name`.

#### template

```
template: |
 {% for lbl_id in labels() %}
 {{ label_name(lbl_id) }}
 {% endfor %}
type: string
output: |
 Outdoor
 Critical
 Energy Monitoring
```

### Show the labels on an entity as names

Get the labels assigned to an entity and display their friendly names instead of IDs.

#### template

```
template: |
 {{
 labels("sensor.living_room_temperature")
 | map("label_name")
 | join(", ")
 }}
type: string
output: "Critical, Energy Monitoring"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Labels

---

The `labels` template function returns a list of all label IDs in your Home Assistant instance when called without an argument. When called with an entity ID, device ID, or area ID, it returns only the labels assigned to that specific item.

Labels are tags you can assign to entities, devices, and areas for organization. This function is useful when you want to work with your labels programmatically. For example, you could list all labels in your system, check which labels are applied to a particular sensor, or build automations that act on items based on their label assignments. Since labels can be added or removed at any time, using `labels()` ensures your templates always reflect your current setup.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: | ["outdoor", "critical",
"energy_monitoring",]
```

---

```
filter: '{{ "light.living_room" | labels }}' type: list output: | ["outdoor", "critical",]
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
labels() -> list[str]
labels(
 lookup_value: str,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} lookup\_value: description: > An entity ID, device ID, or area ID to look up. When provided, only the labels assigned to that item are returned. When omitted, all label IDs are returned. required: false type: string

## Good to know

---

- Without an argument, returns all label IDs. With an ID, returns only labels assigned to that specific entity, device, or area.
- Labels on a device or area do not roll up into the labels of their entities.
- Returns an empty list when the lookup value does not match or has no labels.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many labels you have

A convenient way to see how many labels are defined in your Home Assistant instance.

#### template

```
template: '{{ labels() | count }}'
type: integer
output: "3"
```

### Get labels for a specific entity

Find out which labels have been assigned to a particular entity.

#### template

```
template: '{{ labels("sensor.living_room_temperature") }}'
type: list
output: |
 [
 "critical",
 "energy_monitoring",
]
```

### Get labels for an area

Check which labels are assigned to a specific area by passing its area ID.

#### template

```
template: '{{ labels("living_room") }}'
type: list
output: |
 [
 "outdoor",
]
```

### Check if an entity has a specific label

Determine whether a particular label has been applied to an entity.

#### template

```
template: '{{ "critical" in labels("sensor.living_room_temperature") }}'
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Last

---

The `last` filter returns the last item of a list or the last character of a string. It provides a clean, readable way to access the end of a sequence.

This is commonly used to get the final result from a sorted or filtered list. For example, after sorting sensors by their state value, `last` gives you the highest reading. After filtering a list of recent events, `last` gives you the most recent one. It pairs naturally with `first` to access both ends of a sequence.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ [10, 20, 30] | last }}' type: integer output: "30"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
last(
 value: list,
) -> Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list or string to get the last item from. Must be a non-empty sequence. required: true type: any

## Good to know

---

- Raises an error when the sequence is empty. Chain with `| default(value)` for a fallback.
- After a generator-producing filter like `select` or `map`, add `| list` first. Otherwise the entire sequence is consumed just to find the end.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get the entity with the highest temperature

Sort temperature sensors in ascending order and pick the last (highest) one.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | sort(attribute="state")
 | map(attribute="entity_id")
 | last
 }}
type: string
output: "sensor.kitchen_temperature"
```

### Get the most recently changed entity

Sort entities by their last changed time and pick the last one.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | sort(attribute="last_changed")
 | map(attribute="entity_id")
 | last
 }}
type: string
output: "light.bedroom"
```

### Last character of a string

The `last` filter also works on strings, returning the last character.

#### template

```
template: '{{ "Hello" | last }}'
type: string
output: "o"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Le

---

The `le` test checks whether a value is less than or equal to another value. It is also available under the alias `<=`. When used with `is`, it reads naturally: `value is le(other)`.

While you can always use `<=` directly in conditions, the `le` test is essential when working with `selectattr`, `select`, and similar filters that require a test name as a string. This lets you filter collections by comparing values to a maximum threshold without writing explicit loops.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: | Less or equal

type: string output: Less or equal

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
le(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the comparison. required: true type: any other: description: > The value to compare against. required: true type: any

## Aliases

---

This test can also be used as: - `<=` - `value is <= (other)`

## Using with selectattr

---

Filter entities at or below a maximum value.

### template

```
template: |
 {{
 states.sensor
 | selectattr("attributes.brightness", "le", 128)
 | map(attribute="name") | list
 }}
title: Dimly lit rooms
type: list
output: "['Hallway', 'Bedroom']"
```

## Good to know

---

- When comparing entity state strings with numbers, convert first or wrap the threshold in quotes. String comparison puts "9" above "10".
- Comparisons between incompatible types (a number and a string, for instance) raise an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find values at or below a maximum

Use `select` to find all numbers at or below a limit.

#### template

```
template: '{{ [10, 25, 30, 15, 40] | select("le", 25) | list }}'
type: list
output: "[10, 25, 15]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Length

---

The `length` filter returns the number of items in a collection. For lists, it returns the number of elements. For strings, it returns the number of characters. For dictionaries, it returns the number of keys. It is also available under the alias `count`.

This is one of the most commonly used filters when working with entity collections. You will frequently use it to count how many lights are on, how many doors are open, how many sensors are above a threshold, or how many devices are in a particular area. It typically appears at the end of a filter chain after `select`, `selectattr`, or `expand`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ [1, 2, 3, 4, 5] | length }}' type: integer output: "5"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
length(
 value: Sized,
) -> int
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list, string, or dictionary to measure. Any object that supports `len()` is accepted. required: true type: any

## The count alias

---

The `count` filter is an alias for `length`. They behave identically.

### template

```
template: '{{ ["a", "b", "c"] | count }}'
type: integer
output: "3"
```

## Good to know

---

- Cannot count generators directly. Add `| list` after `select`, `map`, or `selectattr` first.
- For strings, it counts characters, not words or bytes.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count how many lights are on

Count the number of lights currently in the "on" state.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | list
 | length
 }}
type: integer
output: "5"
```

### Count open doors

Count how many door sensors report an "on" (open) state.

#### template

```
template: |
 {{
 expand("group.door_sensors")
 | selectattr("state", "eq", "on")
 | list
 | count
 }}
type: integer
output: "2"
```

### Get the number of attributes on an entity

Count how many attributes a particular entity has.

#### template

```
template: '{{ states.sensor.weather.attributes | length }}'
type: integer
output: "8"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Lipsum

---

The `lipsum` template function generates lorem ipsum placeholder text. By default, it produces 5 paragraphs of HTML-formatted text with sentences of varying length. You can control the number of paragraphs, whether to output HTML or plain text, and the minimum and maximum number of words per sentence.

This is mainly useful for testing and prototyping. If you are developing a custom dashboard card or testing how a notification handles long text, `lipsum` provides a quick way to generate filler content without writing it yourself.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: string output: "Lorem ipsum dolor sit amet, consectetur adipiscing elit."
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
lipsum(
 n: int = 5,
 html: bool = true,
 min: int = 20,
 max: int = 100,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} n: description: > The number of paragraphs to generate. Defaults to 5. required: false default: "5" type: integer html: description: > If true, wraps each paragraph in <p> tags. If false, separates paragraphs with double newlines. Defaults to true. required: false default: "true" type: boolean min: description: > The minimum number of words per sentence. Defaults to 20. required: false default: "20" type: integer max: description: > The maximum number of words per sentence. Defaults to 100. required: false default: "100" type: integer

## Good to know

---

- HTML output is the default and wraps paragraphs in `<p>` tags. Pass `html=false` for plain text.
- The sentence length range is 20-100 words by default, which is longer than typical sentences.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Generate plain text paragraphs

Produce two paragraphs of plain text without HTML tags.

#### template

```
template: '{{ lipsum(2, html=false, min=5, max=15) }}'
type: string
output: |
 Lorem ipsum dolor sit amet, consectetur adipiscing elit.

 Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/List

---

The `list` filter converts a value to a list. When applied to a string, it splits the string into a list of individual characters. When applied to a dictionary, it returns a list of the dictionary's keys. When applied to other collections like generators or sets, it materializes them into a list. This filter is frequently used at the end of a filter chain to materialize the result of `select`, `reject`, `map`, `selectattr`, and similar filters into an actual list. These filters return generators (lazy sequences), and converting them to a list is necessary before you can use the result in further operations or display it.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "hello" | list }}' type: list output: "['h', 'e', 'l', 'l', 'o']"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | list() -> list
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The value to convert to a list. Strings become lists of characters, dictionaries become lists of keys, and other collections are materialized into lists. required: true type: any

## Materializing filter chains

---

Most template selection filters like `select`, `reject`, and `map` return generators. Use the `list` filter to convert them to an actual list.

### template

```
template: '{{ [1, 2, 3, 4, 5] | select("gt", 3) | list }}'
title: Materialize a select filter result
type: list
output: "[4, 5]"
```

## Good to know

---

- Use this after `select`, `reject`, `map`, `selectattr`, and `rejectattr` so you can count, loop, or slice the result.
- Applied to a string, this produces a list of single characters. Use `split(" ")` for words.
- Applied to a dictionary, this produces a list of keys, not values or pairs.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get dictionary keys as a list

Extract the keys from a dictionary into a list.

#### template

```
template: '{{ {"name": "Kitchen", "temperature": 21.5} | list }}'
type: list
output: "['name', 'temperature']"
```

### Collect entity names into a list

Gather all light names that are currently on into a list for display.

#### template

```
template: |
 {{
 states.light
 | selectattr("state", "eq", "on")
 | map(attribute="name")
 | list
 }}
type: list
output: "['Living Room', 'Kitchen']"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Log

---

The `log` template function returns the logarithm of a value. By default it computes the natural logarithm (base  $e$ ). You can specify a different base, such as 10 for common logarithm or 2 for binary logarithm.

This is useful for scaling sensor data that spans a wide range into something more manageable. For example, sound level sensors often benefit from logarithmic scaling, or you might use a log scale to visualize energy consumption data that varies by orders of magnitude.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "1.0"

---

filter: '' type: float output: "2.0"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
log(
 value: Any,
 base: Any = e,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to calculate the logarithm of. Must be a positive number. required: true type: float base: description: > The base of the logarithm. Defaults to `e` (Euler's number, ~2.718) for the natural logarithm. Use 10 for common logarithm or 2 for binary logarithm. required: false type: float default: description: > Value to return if the calculation fails (for example, if the input is not a positive number). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the input value might not be a positive number, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{ log(states("sensor.sound_level") | float(0), 10, default=0) }}
type: float
output: "0"
```

## Good to know

---

- The default base is `e`, which gives the natural logarithm. Pass `10` for common log or `2` for binary log.
- Zero or negative inputs raise an error unless you supply a default.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Natural logarithm

When no base is specified, the natural logarithm (base  $e$ ) is used.

#### template

```
template: |
 {{ log(e) }}
type: float
output: "1.0"
```

### Binary logarithm

Use base 2 for binary logarithm calculations.

#### template

```
template: |
 {{ log(256, 2) }}
type: float
output: "8.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Lower

---

The `lower` filter converts all characters in a string to lowercase. This is useful when you need to normalize text for comparison or display. For example, sensor states may come in mixed case like `On` or `OFF`, and converting to lowercase lets you compare them consistently. It is also handy for formatting text before using it in entity IDs or other contexts that expect lowercase values.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ "Hello WORLD" | lower }}' type: string output: hello world

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
lower(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to convert to lowercase. required: true  
type: string

## Good to know

---

- Entity states in Home Assistant are already lowercase (for example, `on`, `off`, `heating`), so this filter is most useful for attributes, user input, or text from external sources.
- Non-letter characters like digits and punctuation pass through unchanged.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Normalize a state for comparison

Convert a sensor state to lowercase so comparisons work regardless of how the value is capitalized.

#### template

```
template: |
 {% if states("sensor.status") | lower == "online" %}
 Device is online
 {% else %}
 Device is offline
 {% endif %}
type: string
output: "Device is online"
```

### Format text for display

Ensure a label is consistently lowercase for a clean look on a dashboard.

#### template

```
template: '{{ "MOTION DETECTED" | lower }}'
type: string
output: motion detected
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Lt

---

The `lt` test checks whether a value is strictly less than another value. It is also available under the aliases `lessthan` and `<`. When used with `is`, it reads naturally: `value is lt(other)`.

While you can always use `<` directly in conditions, the `lt` test is essential when working with `selectattr`, `select`, and similar filters that require a test name as a string. This lets you filter collections by comparing values to an upper limit without writing explicit loops.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: | Less than

type: string output: Less than

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
lt(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the comparison. required: true type: any other: description: > The value to compare against. required: true type: any

## Aliases

---

This test can also be used as: - `lessthan` - `value is lessthan(other)` - `<` - `value is <(other)`



## Using with selectattr

---

Filter entities below a certain threshold, for example to find low battery devices.

### template

```
template: |
 {{
 states.sensor
 | selectattr("attributes.battery_level", "lt", 20)
 | map(attribute="name") | list
 }}
title: Low battery devices
type: list
output: "['Remote Control']"
```

## Good to know

---

- When comparing entity state strings with numbers, convert first or wrap the threshold in quotes. String comparison puts `"9"` above `"10"`.
- This is strict. A value equal to the threshold does not pass. Use `le` for less-or-equal.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Filter values below a limit

Use `select` to find numbers below a threshold.

#### template

```
template: '{{ [10, 25, 30, 15, 40] | select("lt", 20) | list }}'
type: list
output: "[10, 15]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Map

---

The `map` filter applies a transformation to each item in a list and returns the results. It can be used in two primary ways: extracting an attribute from each item using `attribute=`, or applying another filter to each item by passing the filter name as a string. This is a Home Assistant override of the standard `map` filter, extended to support additional Home Assistant-specific filters and type conversions.

This is one of the most heavily used filters in Home Assistant templates. You will find it in almost every template that works with a collection of entities. The `attribute=` form extracts values like `.state`, `.entity_id`, or `.name` from entity state objects returned by `expand`. The filter form lets you apply conversions like `float`, `int`, or `round` to each item in a list, so you can build filter chains that transform raw entity data into useful results.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: list output: ['21.5', '19.8', '22.3']
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
map(
 value: list,
 *args: str,
 attribute: str | None = None,
 default: Any = None,
) -> iterable
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of items to transform. required: true type: list args: description: > When extracting attributes, omit this. When applying a filter, pass the filter name as a string (for example, `float`, `int`, `upper`), optionally followed by arguments for that filter. required: false type: string attribute: description: > The name of the attribute to extract from each item. Supports dotted notation for nested attributes (for example, `attributes.brightness`). required: false type: string default: description: > A default value to use when an item does not have the specified attribute. Only used with `attribute=`. required: false type: any

## Extract an attribute from each item

---

Use `attribute=` to pull a specific property from each item in the list.

### template

```
template: |
 {{
 expand("group.all_lights")
 | map(attribute="entity_id")
 | list
 }}
title: Get entity IDs from expanded group
type: list
output: '["light.bedroom", "light.kitchen", "light.living_room"]'
```

## Apply a filter to each item

---

Pass a filter name as a string to apply it to every item in the list.

### template

```
template: |
 {{
 ["21.5", "19.8", "22.3"]
 | map("float")
 | list
 }}
title: Convert string values to floats
type: list
output: "[21.5, 19.8, 22.3]"
```

## Nested attribute access

---

Use dotted notation to access nested attributes.

### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | map(attribute="attributes.brightness")
 | list
 }}
title: Get brightness of all lights that are on
type: list
output: "[255, 128, 64]"
```

## Good to know

---

- Returns an iterable, not a list. Add `| list` before using it with `length`, `first`, or looping twice.
- `attribute=` extracts a property; a filter name as the first positional argument applies that filter to each item. You can use one or the other, not both.
- Missing attributes default to `None` unless you pass `default=`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Calculate average temperature from a group

Extract state values, convert to floats, and compute the average.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | map(attribute="state")
 | map("float")
 | average
 }}
type: float
output: "21.2"
```

### Get friendly names of all entities in a group

#### template

```
template: |
 {{
 expand("group.all_lights")
 | map(attribute="name")
 | list
 }}
type: list
output: '["Bedroom light", "Kitchen light", "Living room light"]'
```

### Chain map with round for clean display

Apply multiple transformations by chaining `map` calls.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | map(attribute="state")
 | map("float")
 | map("round", 1)
 | list
 }}
type: list
output: "[21.5, 19.8, 22.3]"
```

## Apply string filters to a list

Use `map` to apply string transformations to every item.

### template

```
template: |
 {{
 ["hello", "world", "template"]
 | map("upper")
 | list
 }}
type: list
output: '["HELLO", "WORLD", "TEMPLATE"]'
```

## Default value for missing attributes

Use `default=` to handle items that may not have the requested attribute.

### template

```
template: |
 {{
 expand("group.all_lights")
 | map(attribute="attributes.brightness", default=0)
 | list
 }}
title: Brightness with 0 default for off lights
type: list
output: "[255, 0, 128]"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Mapping

---

The `mapping` test checks whether a value is a mapping type, such as a Python dictionary. It returns `true` if the value supports key-value access and `false` for lists, strings, numbers, and other types.

This is useful when working with entity attributes or parsed JSON data that might be either a dictionary or a list. For example, some API responses return a dictionary for a single result and a list for multiple results. Testing with `mapping` lets you handle both cases correctly in your template.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is a mapping
```

```
type: string output: "It is a mapping"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
mapping(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is a mapping (dictionary) type. required: true type: any

## Good to know

---

- Only dictionary-like values pass. Lists, tuples, and strings do not, even though they may look similar.
- Use this to guard key access: dictionary access on a non-mapping raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various types

Dictionaries are mappings, but lists and strings are not.

#### template

```
template: |
 {{ {"a": 1} is mapping }}
 {{ [1, 2, 3] is mapping }}
 {{ "hello" is mapping }}
type: boolean
output: |
 true
 false
 false
```

### Handle parsed JSON data

When parsing JSON that could be a dictionary or a list, check the type before accessing keys.

#### template

```
template: |
 {% set data = '{"status": "ok"}' | from_json %}
 {% if data is mapping %}
 Status: {{ data.status }}
 {% else %}
 Unexpected format
 {% endif %}
type: string
output: "Status: ok"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Match

---

The `match` template test checks whether a string matches a regular expression (regex) pattern at the beginning. A regular expression is a special text pattern that lets you describe what you are looking for, such as "starts with a number" or "begins with the word sensor". As a template test, it is used with the `is` keyword, making your templates read more naturally.

This is useful in conditions and `{% jinja %}{% endjinja %}` blocks where you want to check if a value starts with a certain pattern. For example, you might test whether an entity ID begins with a particular domain, or whether a sensor value starts with a specific prefix. Because it is a test rather than a filter, it fits naturally into conditional expressions. It checks only the beginning of the string; use the `search` test to look for a pattern anywhere.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: |

```
It's a light entity
```

type: string output: "It's a light entity"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
match(
 value: str,
 find: str = "",
 ignorecase: bool = False,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to test against the regex pattern. required: true type: string find: description: > The regular expression pattern to match at the beginning of the string. required: true type: string ignorecase: description: > Set to `true` to make the match case-insensitive. required: false default: "false" type: boolean

## Match vs search test

---

The `match` test only checks the **beginning** of the string. Use the `search` test to find a pattern **anywhere** in the string.

### template

```
template: |
 {{ "Room: Living Room" is match("Living") }}
 {{ "Room: Living Room" is search("Living") }}
title: "match checks the start, search checks anywhere"
type: boolean
output: |
 false
 true
```

## Good to know

---

- Only checks the start of the string. Use `search` when the pattern can appear anywhere.
- A literal dot ( `.` ) in the pattern must be escaped with a backslash, since `.` matches any character in regex.
- Case-sensitive by default. Pass `ignorecase=true` to match regardless of case.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Filter entities by domain in a loop

Use `match` inside a loop to select only entities from a specific domain.

**template**

```
template: |
 {% for entity in states if entity.entity_id is match("binary_sensor\\.") %}
 {{ entity.entity_id }}
 {% endfor %}
```

### Conditional automation based on entity pattern

Check if a triggering entity belongs to a specific domain using the `match` test in a condition.

**template**

```
template: |
 {% if trigger.entity_id is match("sensor\\.temperature_") %}
 Temperature sensor triggered: {{ trigger.entity_id }}
 {% endif %}
type: string
output: "Temperature sensor triggered: sensor.temperature_living_room"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Max

---

The `max` template function returns the largest value from a collection. This is a Home Assistant override of the built-in `max` filter that also works as a function, allowing you to pass values either as a list or as separate arguments.

This is useful whenever you need to find the highest reading among multiple sensors. For example, you might want to find the warmest room in your house, the highest power consumption across all your devices, or the peak value in a set of readings. You can also use it with `expand` to work with groups of entities.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "22.0"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
max(
 *args: list | float,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} values: description: > The values to find the maximum of. Can be a list or multiple separate arguments. required: true type: list

## Good to know

---

- An empty list raises an error, so convert or filter out missing sensors first.
- Mixing strings and numbers in the input raises an error. Convert state strings with `float` first.
- After a generator-producing filter like `map`, add `| list` before using `max` as a filter.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Maximum sensor value

Find the highest temperature across multiple rooms.

#### template

```
template: |
 {{
 max(
 states("sensor.living_room_temperature") | float,
 states("sensor.bedroom_temperature") | float,
 states("sensor.kitchen_temperature") | float
)
 }}
type: float
output: "22.0"
```

### Maximum from a list

Pass a list directly to the function.

#### template

```
template: |
 {{ max([21.5, 22.0, 19.8]) }}
type: float
output: "22.0"
```

### Maximum across a group of entities

If you have a group of sensors, expand the group and find the largest value.

#### template

```
template: |
 {{
 expand("group.indoor_temperatures")
 | map(attribute="state")
 | map("float")
 | list
 | max
 }}
type: float
output: "22.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Md5

---

The `md5` template function calculates the MD5 hash of a string and returns the result as a hexadecimal string. MD5 produces a 128-bit (32-character hex) hash value from any input string.

This is useful when you need to generate a consistent, fixed-length identifier from a variable-length string. For example, you might hash an entity ID to create a unique filename, generate a cache key, or create a deterministic identifier for use with external services. Note that MD5 is not considered cryptographically secure for password hashing or security purposes. For stronger hashing, use `sha256` or `sha512`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: string output:
"65a8e27d8879283831b664bd8b7f0ad4"
```

---

filter: '{{ "Hello, World!" | md5 }}' type: string output: "65a8e27d8879283831b664bd8b7f0ad4"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
md5(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to hash. The string is encoded to UTF-8 before hashing. required: true type: string

## Good to know

---

- MD5 is not secure for passwords or sensitive data. Use `sha256` or `sha512` for those.
- The input is encoded as UTF-8 before hashing, and the output is lowercase hexadecimal, always 32 characters long.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Generate a unique identifier

Create a short, consistent identifier from an entity ID.

#### template

```
template: '{{ "light.living_room_ceiling" | md5 }}'
type: string
output: "a3c2f8b1d4e5f6a7b8c9d0e1f2a3b4c5"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Median

---

The `median` template function calculates the statistical median of a collection of values. Give it a list of numbers and it returns the middle value when they are sorted. If the list has an even number of elements, it returns the average of the two middle values.

This is useful when you want a representative value from multiple sensors that is less affected by outliers than `average`. For example, if you have five temperature sensors and one is reading incorrectly high, the median will ignore that outlier and give you a more accurate picture of the actual temperature. It works well for any situation where a single extreme value should not skew the result.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "21.5"

---

filter: '{{ [21.5, 22.0, 19.8] | median }}' type: float output: "21.5"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
median(
 *args: list | float,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} values: description: > The values to find the median of. Can be a list or multiple separate arguments. All values must be numeric. required: true type: list default: description: > Value to return if the calculation fails (for example, if the list is empty or contains non-numeric values). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the list might be empty or contain invalid values, provide a default to avoid errors. This prevents your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{
 [states("sensor.maybe_broken") | float(none)]
 | reject("none")
 | list
 | median(default=0)
 }}
type: float
output: "0"
```

## Good to know

---

- An empty list raises an error unless you supply a default.
- With an even number of values, the result is the average of the two middle values.
- Less sensitive to outliers than `average`, which makes it better for noisy sensor data.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Median of sensor values

Calculate the median temperature across multiple rooms.

#### template

```
template: |
 {{
 median(
 states("sensor.living_room_temperature") | float,
 states("sensor.bedroom_temperature") | float,
 states("sensor.kitchen_temperature") | float
)
 }}
type: float
output: "21.5"
```

### Median across a group of entities

If you have a group of temperature sensors, you can expand the group and find the median of all their values in one expression.

#### template

```
template: |
 {{
 expand("group.indoor_temperatures")
 | map(attribute="state")
 | map("float")
 | median
 }}
type: float
output: "21.5"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Merge Response

---

The `merge_response` template function takes the response dictionary from a multi-entity action call and merges it into a single flat list. When you call an action that targets multiple entities, the response is a dictionary keyed by entity ID, where each value contains the entity's response data. `merge_response` flattens this structure into a single list, adding the `entity_id` to each item for reference.

This is useful when you call actions like `calendar.get_events` or `weather.get_forecasts` that target multiple entities at once. Without `merge_response`, you would need to manually loop through the response dictionary and extract items from each entity. This function handles that for you and produces a unified list that is straightforward to sort, filter, and display. Each item in the result includes an `entity_id` field so you can still tell which entity it came from.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: |

type: list output: | [ {"start": "2024-03-15", "summary": "Meeting", "entity\_id": "calendar.work"},  
{"start": "2024-03-15", "summary": "Dentist", "entity\_id": "calendar.personal"}, ]

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
merge_response(
 value: dict,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The action response dictionary to merge. This must be a dictionary where each key is an entity ID and each value is a dictionary containing the entity's response data. required: true type: map

## Good to know

---

- Adds an `entity_id` field to each item, so you can tell which entity each response came from after merging.
- Only works on responses shaped as `{entity_id: {key: [items]}}`. Responses with a different shape need manual handling.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Merge calendar events from multiple calendars

Call `calendar.get_events` targeting multiple calendars and merge all events into a single sorted list.

#### action

```
action: |
 action:
 - action: calendar.get_events
 target:
 entity_id:
 - calendar.work
 - calendar.personal
 data:
 duration:
 hours: 24
 response_variable: agenda
 - action: notify.mobile
 data:
 message: >
 Today's events:
 {% for event in merge_response(agenda) | sort(attribute="start") %}
 - {{ event.summary }} ({{ event.entity_id.split(".")[1] }})
 {% endfor %}
```

### Merge weather forecasts

Combine forecasts from multiple weather entities into one list for comparison.

#### action

```
action: |
 action:
 - action: weather.get_forecasts
 target:
 entity_id:
 - weather.home
 - weather.office
 data:
 type: daily
 response_variable: forecasts
 - action: notify.mobile
 data:
 message: >
 {% for item in merge_response(forecasts) %}
 {{ item.entity_id }}: {{ item.temperature }}C
 {% endfor %}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Min

---

The `min` template function returns the smallest value from a collection. This is a Home Assistant override of the built-in `min` filter that also works as a function, allowing you to pass values either as a list or as separate arguments.

This is useful whenever you need to find the lowest reading among multiple sensors. For example, you might want to find the coldest room in your house, the lowest battery level across all your devices, or the minimum energy usage in a set of readings. You can also use it with `expand` to work with groups of entities.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "19.8"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
min(
 *args: list | float,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} values: description: > The values to find the minimum of. Can be a list or multiple separate arguments. required: true type: list



## Good to know

---

- An empty list raises an error, so convert or filter out missing sensors first.
- Mixing strings and numbers in the input raises an error. Convert state strings with `float` first.
- After a generator-producing filter like `map`, add `| list` before using `min` as a filter.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Minimum sensor value

Find the lowest temperature across multiple rooms.

#### template

```
template: |
 {{
 min(
 states("sensor.living_room_temperature") | float,
 states("sensor.bedroom_temperature") | float,
 states("sensor.kitchen_temperature") | float
)
 }}
type: float
output: "19.8"
```

### Minimum from a list

Pass a list directly to the function.

#### template

```
template: |
 {{ min([21.5, 22.0, 19.8]) }}
type: float
output: "19.8"
```

### Minimum across a group of entities

If you have a group of sensors, expand the group and find the smallest value.

#### template

```
template: |
 {{
 expand("group.indoor_temperatures")
 | map(attribute="state")
 | map("float")
 | list
 | min
 }}
type: float
output: "19.8"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Multiply

---

The `multiply` filter converts a value to a float and multiplies it by a specified amount. If the value cannot be converted to a number, it returns the default you provide instead of raising an error.

This is a convenient shorthand for scaling sensor values. Instead of converting to a float first and then multiplying, you can do it in a single step. Common uses include converting units (watts to kilowatts, Celsius to Fahrenheit), applying a cost rate to an energy reading, or scaling a percentage to a different range. For addition, see `add`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: float output: "1.5"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
multiply(
 value: Any,
 amount: float,
 default: Any = _SENTINEL,
) -> float | Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The value to convert to a float and multiply. Must be a number or a string that can be converted to a float. required: true type: any amount: description: > The amount to multiply the value by. required: true type: float default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. required: false type: any



## Using a default value

---

If the sensor might be unavailable, provide a default to prevent errors.

### template

```
template: '{{ states("sensor.power_watts") | multiply(0.001, default=0) }}'
title: Safe multiplication with default
type: float
output: "1.5"
```

## Good to know

---

- There is no division filter. Pass a fractional amount, like `0.001` to divide by 1000.
- The result is always a float, even when both inputs are integers.
- Without a default, non-numeric input raises an error. Always pass a default when working with state values.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Convert watts to kilowatts

Scale a power reading from watts to kilowatts for cleaner display.

#### template

```
template: '{{ states("sensor.power_usage") | multiply(0.001) | round(2) }}'
type: float
output: "1.50"
```

### Calculate energy cost

Multiply today's energy consumption by the price per kWh.

#### template

```
template: '{{ states("sensor.energy_today") | multiply(0.25) | round(2) }}'
title: Energy cost at 0.25 per kWh
type: float
output: "3.47"
```

### Scale a percentage to a brightness value

Convert a 0-100 percentage to a 0-255 brightness scale for a light.

#### action

```
action: |
 action:
 - action: light.turn_on
 target:
 entity_id: light.desk_lamp
 data:
 brightness: >
 {{ states("sensor.ambient_light_pct") | multiply(2.55) | int(128) }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Namespace

---

When you try to count something inside a `for` loop or track a running total, you quickly hit a wall: variables changed inside the loop do not survive outside of it. The `namespace` template function solves this. It creates a special object whose attributes you can modify from any scope, so your counter, total, or tracker keeps its value after the loop ends.

This is one of the most important functions to understand when writing Home Assistant templates that involve loops. Without `namespace`, you cannot accumulate a total, track a maximum, or build a result inside a `for` loop and use it afterward. You create a namespace with initial values like `namespace(count=0, total=0)`, then modify its attributes with ``` inside the loop. The changes persist after the loop ends.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: |

type: integer output: "3"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
namespace(
 **kwargs: Any,
) -> Namespace
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} kwargs: description: > Any number of keyword arguments that become attributes on the namespace object. For example, `namespace(total=0, name="")` creates a namespace with a `total` attribute set to `0` and a `name` attribute set to an empty string. required: false type: any

## Why namespace is needed

---

Without a namespace, variables set inside a loop do not persist outside the loop. This is a common source of confusion.

### template

```
template: |
 {# This does NOT work as expected #}
 {% set count = 0 %}
 {% for i in range(3) %}
 {% set count = count + 1 %}
 {% endfor %}
 {{ count }}
title: Without namespace (broken)
type: integer
output: "0"
```

### template

```
template: |
 {# This works correctly with namespace #}
 {% set ns = namespace(count=0) %}
 {% for i in range(3) %}
 {% set ns.count = ns.count + 1 %}
 {% endfor %}
 {{ ns.count }}
title: With namespace (correct)
type: integer
output: "3"
```

## Good to know

---

- Without a namespace, a `` inside a loop does not persist outside of it. This is the main reason to reach for this function.
- Access attributes with dot notation ( `ns.count` ), and update them with ``.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Sum energy consumption across entities

Accumulate a total from multiple sensors inside a loop.

#### template

```
template: |
 {% set ns = namespace(total=0) %}
 {% for entity in states.sensor
 | selectattr("attributes.device_class", "eq", "energy")
 | list %}
 {% set ns.total = ns.total + entity.state | float(0) %}
 {% endfor %}
 {{ ns.total | round(1) }} kWh
type: string
output: "47.3 kWh"
```

### Find the hottest room

Track the maximum temperature and which room it belongs to.

#### template

```
template: |
 {% set ns = namespace(max_temp=-999, room="unknown") %}
 {% for sensor in states.sensor
 | selectattr("attributes.device_class", "eq", "temperature")
 | list %}
 {% if sensor.state | float(0) > ns.max_temp %}
 {% set ns.max_temp = sensor.state | float(0) %}
 {% set ns.room = sensor.name %}
 {% endif %}
 {% endfor %}
 {{ ns.room }}: {{ ns.max_temp }}°C
type: string
output: "Kitchen: 23.5°C"
```

### Build a comma-separated list of active lights

Collect names of lights that are on into a single string.

#### template

```

template: |
 {% set ns = namespace(names=[]) %}
 {% for light in states.light | selectattr("state", "eq", "on") | list %}
 {% set ns.names = ns.names + [light.name] %}
 {% endfor %}
 {{ ns.names | join(", ") }}
type: string
output: "Living Room, Kitchen, Hallway"

```

## Count entities matching a condition

Count how many doors are currently open using a namespace counter.

### template

```

template: |
 {% set ns = namespace(open=0) %}
 {% for entity in states.binary_sensor
 | selectattr("attributes.device_class", "eq", "door")
 | selectattr("state", "eq", "on")
 | list %}
 {% set ns.open = ns.open + 1 %}
 {% endfor %}
 {{ ns.open }} door{{ "s" if ns.open != 1 }} open
type: string
output: "2 doors open"

```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Ne

---

The `ne` test checks whether two values are not equal. It is also available under the alias `!=`. When used with `is`, it reads naturally: `value is ne(other)`.

While you can always use `!=` directly in conditions, the `ne` test is particularly useful with `selectattr`, `rejectattr`, `select`, and `reject`. These filters require a test name as a string, and `ne` lets you filter out items matching a specific value without writing a loop.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Not equal
```

type: string output: Not equal

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
ne(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the comparison. required: true type: any other: description: > The value to compare against. required: true type: any

## Aliases

---

This test can also be used as: - `!=` - `value is !=(other)`

## Using with selectattr

---

Filter entities whose state is not a specific value. This is useful for finding entities that are in an unexpected or non-default state.

### template

```
template: |
 {{
 states.light
 | selectattr("state", "ne", "off")
 | map(attribute="name") | list
 }}
title: Find lights that are not off
type: list
output: "['Living Room', 'Kitchen']"
```

## Good to know

---

- Numbers and strings are not equal even when they look the same. `"5" is ne(5) is true`.
- Especially useful with `selectattr` to skip entities in a specific state like `unavailable`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Exclude unavailable entities

Filter out entities that are unavailable from a list.

#### template

```
template: |
 {{
 states.sensor
 | selectattr("state", "ne", "unavailable")
 | map(attribute="entity_id") | list
 }}
type: list
output: "['sensor.temperature', 'sensor.humidity']"
```

### Remove specific values from a list

Use `reject` to remove items equal to a specific value.

#### template

```
template: '{{ [1, 0, 3, 0, 5] | reject("eq", 0) | list }}'
type: list
output: "[1, 3, 5]"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/None

---

The `none` test checks whether a value is `None`. It returns `true` if the value is `None` and `false` otherwise.

In Home Assistant, `None` can appear in various situations: an attribute that does not exist on an entity, a function that returns nothing, or a variable that has been explicitly set to `None`. Testing for `None` is different from testing for `undefined`: a variable can be defined but have a value of `None`. This test helps you distinguish between "not set" and "set to nothing."

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Value is None
```

type: string output: "Value is None"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
none (
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is `None`. required: true type: any

## Good to know

---

- This is different from `undefined`. A variable can be defined and still be `None`.
- The string `"None"` does not pass this test, only the actual `None` value does.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check for missing attributes

Test whether an entity attribute exists before using it.

#### template

```
template: |
 {% if state_attr("climate.thermostat", "preset_mode") is none %}
 No preset mode available
 {% else %}
 Preset: {{ state_attr("climate.thermostat", "preset_mode") }}
 {% endif %}
type: string
output: "No preset mode available"
```

### Guard against None in calculations

Ensure a value is not `None` before performing math.

#### template

```
template: |
 {% set temp = state_attr("weather.home", "temperature") %}
 {% if temp is none %}
 Temperature unavailable
 {% else %}
 {{ temp | round(1) }} degrees
 {% endif %}
type: string
output: "21.5 degrees"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Now

---

The `now` template function returns the current date and time in your local time zone. It gives you a full date and time object that you can use to make decisions, format for display, or calculate time differences.

Many automations and templates need to know what time it is right now. You might want to send a different greeting in the morning versus the evening, only turn on lights after sunset, check if it's a weekday before triggering your work routine, or display how long ago something happened.

`now()` is your starting point for all of these. From the result, you can pull out the current hour, minute, day, month, day of the week, and more. Using `now()` in a template also causes it to be re-evaluated at the start of every new minute, so time-dependent values stay up to date automatically.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: datetime output: "2024-03-15
14:30:00.123456+01:00"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
now() -> datetime
```

## Working with the result

---

`now()` returns a datetime object. You can access individual components and format the output:

### template

```
template: |
 The time is {{ now().hour }}:{{ now().minute }}
 Today is day {{ now().day }} of month {{ now().month }}
type: string
output: |
 The time is 14:30
 Today is day 15 of month 3
```

## Formatting with strftime

You can format the date and time in any way you like using Python's `strftime` method.

### template

```
template: '{{ now().strftime("%H:%M") }}'
type: string
output: "14:30"
```

### template

```
template: '{{ now().strftime("%A, %B %d") }}'
type: string
output: "Friday, March 15"
```

## Good to know

---

- Templates using `now()` re-evaluate once per minute, not continuously. Second-level precision in conditions won't react immediately.
- The returned datetime is timezone-aware and uses the Home Assistant configured time zone. Use `utcnow()` when you need UTC.
- `weekday()` returns 0 for Monday through 6 for Sunday, not starting at 1.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if it's a weekday

Use this in a condition to only run automations on weekdays. The `weekday()` method returns 0 for Monday through 6 for Sunday.

#### template

```
template: '{{ now().weekday() < 5 }}'
type: boolean
output: "true"
```

### Only run between certain hours

Limit an automation to only run during daytime hours.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ 8 <= now().hour < 22 }}
```

### Calculate seconds since an event

Find out how many seconds have passed since the front door last changed state. Useful for checking if something happened recently.

#### template

```
template: |
 {{
 (now() - states.binary_sensor.front_door.last_changed)
 .total_seconds() | int
 }}
type: integer
output: "3847"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Number Test

---

The `number` test checks whether a value is a number, meaning either an integer or a float. It returns `true` for both types and `false` for anything else, including numeric strings.

This test checks the Python type of the value, not whether a string can be parsed as a number. If you need to check whether a string like `21.5` can be converted to a number, use `is_number` instead. The `number` test is useful when you already have a value from a calculation or attribute and want to confirm it is numeric before proceeding.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is a number
```

type: string output: "It is a number"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
number(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is an integer or float. required: true type: any

## Good to know

---

- This checks the Python type. Numeric strings like `"21.5"` do not pass.
- To test whether a value can be parsed as a number, use `is_number` instead.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various types

Both integers and floats pass the `number` test, but strings do not.

#### template

```
template: |
 {{ 42 is number }}
 {{ 21.5 is number }}
 {{ "42" is number }}
type: boolean
output: |
 true
 true
 false
```

### Validate attribute values

Check that an entity attribute is a numeric type before doing math.

#### template

```
template: |
 {% set brightness = state_attr("light.living_room", "brightness") %}
 {% if brightness is number %}
 Brightness: {{ (brightness / 255 * 100) | round(0) }}%
 {% else %}
 Brightness unknown
 {% endif %}
type: string
output: "Brightness: 75%"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Odd

---

The `odd` test checks whether an integer is odd (not divisible by 2). It returns `true` for odd numbers and `false` for even numbers.

Like `even`, this test is commonly used in loops to apply alternating styles or logic. It can also be used with `select` and `reject` to filter lists of numbers based on whether they are odd.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is odd
```

type: string output: "It is odd"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
odd(
 value: int,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The integer to test. Returns `true` if the value is not divisible by 2. required: true type: integer

## Good to know

---

- Zero is even, not odd.
- The input must be an integer. Floats or strings raise an error unless converted first.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various numbers

#### template

```
template: |
 {{ 3 is odd }}
 {{ 8 is odd }}
 {{ 1 is odd }}
type: boolean
output: |
 true
 false
 true
```

### Filter odd numbers from a list

Use `select` to extract only odd numbers from a range.

#### template

```
template: '{{ range(1, 11) | select("odd") | list }}'
type: list
output: "[1, 3, 5, 7, 9]"
```

### Count odd-valued sensors

Count how many sensors in a group have odd integer values.

#### template

```
template: |
 {% set values = [
 states("sensor.counter_a") | int(0),
 states("sensor.counter_b") | int(0),
 states("sensor.counter_c") | int(0)
] %}
 {{ values | select("odd") | list | count }} sensors have odd values
type: string
output: "2 sensors have odd values"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Ord

---

The `ord` filter returns the Unicode code point (an integer) for a single character. It wraps Python's built-in `ord()` function. For example, `A` becomes `65`, and `a` becomes `97`.

This is useful when you need to work with character codes in templates. For instance, you might need to convert characters to their numeric representation for protocol communication, compare characters numerically, or perform calculations based on character positions in the alphabet.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ "A" | ord }}' type: integer output: "65"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
ord(
 character: str,
) -> int
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} character: description: > A single character to convert to its Unicode code point. Must be a string of length 1. required: true type: string

## Common character codes

---

A few commonly used character codes for reference:

### template

```
template: |
 A = {{ "A" | ord }}, Z = {{ "Z" | ord }}
 a = {{ "a" | ord }}, z = {{ "z" | ord }}
 0 = {{ "0" | ord }}, 9 = {{ "9" | ord }}
title: ASCII character codes
type: string
output: |
 A = 65, Z = 90
 a = 97, z = 122
 0 = 48, 9 = 57
```

## Good to know

---

- The input must be exactly one character. A string longer than one character raises an error.
- Works with any Unicode character, not just ASCII. The code point can be a large number for emoji or non-Latin scripts.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if a character is uppercase

Use the code point to determine if a character is an uppercase letter.

#### template

```
template: |
 {% set char = "H" %}
 {% set code = char | ord %}
 {{ code >= 65 and code <= 90 }}
title: Is uppercase check
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Ordinal

---

The `ordinal` template filter converts a number into its ordinal string representation. Give it `1` and it returns `1st`, give it `2` and it returns `2nd`, give it `3` and it returns `3rd`, and so on. It correctly handles the special cases for 11th, 12th, and 13th.

This is useful for creating human-readable messages that include rankings or positions. For example, you might want to display "This is the 3rd time the door opened today" in a notification, or show "Floor 2nd" on a dashboard. Anywhere you want a number to read naturally in English text, this filter handles it.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: 1st
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
ordinal(
 value: int | str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The number to convert to an ordinal string. Can be an integer or a string representation of a number. required: true type: [integer, string]

## Good to know

---

- English suffixes only. Other languages are not supported.
- Numbers ending in 11, 12, and 13 get `th`, not `st`, `nd`, or `rd`.
- The result includes the number and the suffix together, like `"21st"`, not just the suffix.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a counter with ordinal suffix

Show how many times an event has occurred today using a counter helper, with a natural-sounding ordinal format.

#### template

```
template: |
 {{
 "This is the " ~ (states("counter.door_opens") | int | ordinal)
 ~ " time the door opened today."
 }}
type: string
output: "This is the 5th time the door opened today."
```

### Show the day of the month as an ordinal

Display the current date in a friendly format with an ordinal day number.

#### template

```
template: |
 {{
 "Today is the " ~ (now().day | ordinal)
 ~ " of " ~ now().strftime("%B") ~ "."
 }}
type: string
output: "Today is the 15th of March."
```

### Handle special cases

The filter correctly handles the special English ordinal cases for 11, 12, and 13.

#### template

```
template: |
 {{ 11 | ordinal }}, {{ 12 | ordinal }}, {{ 13 | ordinal }},
 {{ 21 | ordinal }}, {{ 22 | ordinal }}, {{ 23 | ordinal }}
type: string
output: "11th, 12th, 13th, 21st, 22nd, 23rd"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Pack

---

The `pack` template function converts a value into a bytes object using a Python struct format string. This is the same as Python's `struct.pack` function, allowing you to convert numbers and other values into their binary representations according to a specified format.

This function is primarily useful for IoT and low-level device communication. Many IoT devices communicate using binary protocols where you need to send data in a specific byte format. For example, you might need to send a temperature value as a big-endian 16-bit integer to a Bluetooth device, or pack a command byte sequence for an MQTT-connected microcontroller. The format strings follow [Python's struct module syntax](#).

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: bytes output: "b'\x04\xd2'"
```

---

```
filter: '' type: bytes output: "b'\x04\xd2'"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
pack(
 value: Any,
 format_string: str,
) -> bytes | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to pack into bytes. The type must match what the format string expects (for example, an integer for `>H`). required: true type: any  
format\_string: description: > A Python struct format string that describes the byte layout. Common formats include `>H` (big-endian unsigned short), `<i` (little-endian signed int), `>f` (big-endian float). See the [Python struct documentation](#) for all options. required: true type: string

## Common format strings

---

Here are the format strings you will reach for most often. The `>` prefix means big-endian (most significant byte first) and `<` means little-endian (least significant byte first).

- `>B` : big-endian unsigned byte (1 byte)
- `>H` : big-endian unsigned short (2 bytes)
- `>I` : big-endian unsigned int (4 bytes)
- `>f` : big-endian float (4 bytes)
- `<H` : little-endian unsigned short (2 bytes)
- `<i` : little-endian signed int (4 bytes)



## Good to know

---

- The value type must match the format specifier, or packing fails. `>H` expects an integer between 0 and 65535.
- Endianness matters. `>` is big-endian (network order) and `<` is little-endian.
- Returns raw bytes. For transmission over text channels, combine with `base64_encode`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Pack a temperature for a BLE device

Pack a temperature reading as a big-endian 16-bit integer for transmission to a Bluetooth Low Energy device.

#### template

```
template: |
 {{
 states("sensor.temperature") | float | round(0) | int
 | pack(">H") | base64_encode
 }}
type: string
output: "ABU="
```

### Pack a float value

Pack a floating-point number into its binary representation.

#### template

```
template: '{{ 3.14 | pack(">f") }}'
type: bytes
output: "b'@H\xcf5\xc3'"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Pi

---

The `pi` template constant provides the mathematical constant pi, approximately 3.14159. This is the ratio of a circle's circumference to its diameter and is fundamental to trigonometric and geometric calculations.

This is commonly used for converting between degrees and radians in trigonometric calculations. For example, you might multiply a degree-based sensor value by `pi / 180` before passing it to `sin`, `cos`, or `tan`. It is also useful for circle-related geometry in your templates.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "3.141592653589793"
```

## Good to know

---

- This is a mathematical constant approximately equal to 3.14159.
- Use it without parentheses. `pi` is a value, not a function you call.
- Trigonometric functions like `sin`, `cos`, and `tan` expect radians, so multiply degree values by `pi / 180` first.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Converting degrees to radians

Convert a sensor value in degrees to radians for use with trigonometric functions.

#### template

```
template: |
 {% set degrees = states("sensor.wind_direction") | float %}
 {{ degrees * (pi / 180) }}
type: float
output: "3.141592653589793"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Pprint

---

The `pprint` filter formats a value in a human-readable way, making it easier to inspect complex data structures like dictionaries and lists. This is primarily useful for debugging templates. When you are trying to understand the structure of data returned by a sensor or another template function, piping the value through `pprint` displays it in a nicely formatted, indented layout. This makes it much easier to read than the default single-line representation.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ {"name": "Living Room", "temperature": 22.5} | pprint }}'
```

type: string output: '{"name": "Living Room", "temperature": 22.5}'

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
pprint(
 value: Any,
 verbose: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to pretty-print. Works with any data type including dictionaries, lists, strings, and numbers. required: true type: any verbose: description: > If `true`, includes additional type information in the output. Defaults to `false`. required: false default: "false" type: boolean

## Good to know

---

- The output is intended for debugging. For structured data you plan to use elsewhere, use `to_json`.
- With `verbose=true`, the output also includes the Python type name, which can help identify unexpected types.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Inspect entity attributes

View all attributes of an entity in a readable format for debugging.

#### template

```
template: '{{ state_attr("climate.living_room", "hvac_modes") | pprint }}'
type: string
output: "['off', 'heat', 'cool', 'auto']"
```

### Debug a complex data structure

Pretty-print a nested dictionary to understand its structure.

#### template

```
template: |
 {{
 {"rooms": {"living_room": {"temp": 22}, "bedroom": {"temp": 19}}}
 | pprint
 }}
type: string
output: '{"rooms": {"bedroom": {"temp": 19}, "living_room": {"temp": 22}}}'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Random

---

The `random` filter chooses a random item from a list. This is Home Assistant's override of the built-in `random` filter, with one important difference: it produces a new random value every time the template is evaluated, rather than caching the result.

This is useful when you want to add variety to your automations. For example, you might want to pick a random color for a light, choose a random greeting message for a notification, or select a random playlist to play on a media player. Because the value changes with each evaluation, you get a genuinely different result each time the template runs.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ ["red", "green", "blue"] | random }}' type: string output:
green
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
random(
 values: list,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} values: description: > The list of values to randomly choose from. Must contain at least one item. required: true type: list

## Context-dependent evaluation

---

Unlike many other template functions, `random` is context-dependent. This means the result is not cached between evaluations. Each time the template is rendered, a new random choice is made.

### template

```
template: '{{ ["morning", "afternoon", "evening"] | random }}'
title: Random greeting period
type: string
output: afternoon
```



## Good to know

---

- A new random value is picked every time the template is rendered, so template entities using `random` re-evaluate and change often.
- Calling `random` twice in a single template returns two independent picks, which can surprise you if you expect the same value.
- An empty list raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Random light color

Pick a random color for a light from a predefined list.

#### action

```
action: |
 action:
 - action: light.turn_on
 target:
 entity_id: light.living_room
 data:
 color_name: >
 {{ ["red", "blue", "green", "purple", "orange"] | random }}
```

### Random notification message

Send a different motivational message each time an automation triggers.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 {{
 [
 "Have a great day!",
 "Remember to stay hydrated!",
 "You're doing awesome!",
 "Time to stretch!"
] | random
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Range

---

The `range` template function generates a sequence of numbers. It works just like Python's built-in `range()` function. You can call it with only a stop value to get numbers from 0 up to (but not including) that value, or provide start, stop, and step values for full control over the sequence.

This is one of the most commonly used functions in Home Assistant templates, especially when you need to loop a specific number of times or generate a series of numbered values. For example, you might want to check a set of numbered entities, repeat an action a certain number of times, or build a list of values at regular intervals. Since `range` produces values lazily, you may need to pipe it through `| list` to see the full output.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: "[0, 1, 2, 3, 4]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
range(
 stop: int,
) -> generator[int]

range(
 start: int,
 stop: int,
 step: int = 1,
) -> generator[int]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} start: description: > The starting number of the sequence. Defaults to `0` when only a stop value is given. required: false default: "0" type: integer stop: description: > The end of the sequence. The stop value is not included in the output. required: true type: integer step: description: > The increment between each number in the sequence. Defaults to `1`. Can be negative to count downward. required: false default: "1" type: integer



## Specifying start and stop

---

Generate numbers starting from a value other than 0.

### template

```
template: '{{ range(1, 6) | list }}'
title: Numbers 1 through 5
type: list
output: "[1, 2, 3, 4, 5]"
```

## Using a step value

---

Generate every other number or count in custom increments.

### template

```
template: '{{ range(0, 10, 2) | list }}'
title: Even numbers under 10
type: list
output: "[0, 2, 4, 6, 8]"
```

## Counting downward

---

Use a negative step to count backward.

### template

```
template: '{{ range(5, 0, -1) | list }}'
title: Countdown
type: list
output: "[5, 4, 3, 2, 1]"
```

## Good to know

---

- The stop value is exclusive, so `range(1, 5)` produces `1, 2, 3, 4`.
- Add `| list` to materialize the range for length checks or printing. A generator cannot be reused.
- A negative step is required to count down. `range(5, 0)` with the default step returns an empty sequence.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check a set of numbered entities

Loop through a series of numbered sensors to find which are active.

#### template

```
template: |
 {% for i in range(1, 5) %}
 {% set entity = "binary_sensor.zone_" ~ i %}
 {% if is_state(entity, "on") %}
 Zone {{ i }} is active
 {% endif %}
 {% endfor %}
type: string
output: |
 Zone 2 is active
 Zone 4 is active
```

### Count open windows

Use `range` with a `namespace` to count how many numbered window sensors are open.

#### template

```
template: |
 {% set ns = namespace(count=0) %}
 {% for i in range(1, 7) %}
 {% if is_state("binary_sensor.window_" ~ i, "on") %}
 {% set ns.count = ns.count + 1 %}
 {% endif %}
 {% endfor %}
 {{ ns.count }} windows open
type: string
output: "2 windows open"
```

### Build a percentage bar

Create a text-based progress indicator from a sensor value.

#### template

```
template: |
 {% set level = states("sensor.battery_level") | int(0) %}
 {% set filled = (level / 10) | round(0) | int %}
 [
 {%- for i in range(10) -%}
 {% if i < filled %}#{% else %}-{% endif %}
 {%- endfor -%}
] {{ level }}%
type: string
output: "[#####----] 60%"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Regex Findall

---

The `regex_findall` template filter finds all occurrences of a regular expression (regex) pattern in a string and returns them as a list. A regular expression is a special text pattern that lets you describe what you are looking for. While `regex_search` just tells you whether a pattern exists, `regex_findall` collects every match and gives them all back to you.

This is useful when you need to extract multiple pieces of data from a text value. For example, you might want to pull all numbers out of a sensor's state string, extract all IP addresses from a log message, or collect all entity IDs mentioned in a text attribute. The result is a list, so you can count the matches, loop through them, or pick a specific one by index.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Living room 23.5C, Bedroom 19.8C" | regex_findall("\d+\\.\\d+") }}' type: list output: "['23.5', '19.8']"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
regex_findall(
 value: str,
 find: str = "",
 ignorecase: bool = False,
) -> list[str]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to search within for all matches of the regex pattern. required: true type: string find: description: > The regular expression pattern to search for. All non-overlapping matches are returned. required: true type: string ignorecase: description: > Set to `true` to make the search case-insensitive. required: false default: "false" type: boolean

## Counting matches

---

Since the result is a list, you can use the `length` filter to count how many matches were found.

### template

```
template: |
 {{
 "error at 10:15, error at 11:30, error at 12:45"
 | regex_findall("error") | length
 }}
title: Count the number of errors
type: integer
output: "3"
```

## Good to know

---

- Returns an empty list when nothing matches, not `None` or an error.
- Capturing groups change the output: with one group, you get strings; with multiple groups, you get tuples.
- A literal dot in the pattern must be escaped as `\.`, since `.` matches any character in regex.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Extract all numbers from a sensor state

Pull every number out of a text-based sensor value for processing.

#### template

```
template: |
 {% set values = "Power: 150W, Voltage: 230V, Current: 0.65A"
 | regex_findall("\\d+\\.?\\d*") %}
 {{ values }}
type: list
output: "['150', '230', '0.65']"
```

### List all matching words

Find all words in a string that start with a capital letter.

#### template

```
template: '{{ "The Living Room Light is On" | regex_findall("[A-Z][a-z]+") }}'
type: list
output: "['The', 'Living', 'Room', 'Light', 'On']"
```

### Loop through extracted values

Extract all temperature values from a multi-sensor string and display them.

#### template

```
template: |
 {% set temps = "Kitchen 21C, Hall 19C, Office 22C"
 | regex_findall("(\\w+) (\\d+)C") %}
 {% for room, temp in temps %}
 {{ room }}: {{ temp }}C
 {% endfor %}
type: string
output: |
 Kitchen: 21C
 Hall: 19C
 Office: 22C
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Regex Findall Index

---

The `regex_findall_index` template filter finds all occurrences of a regular expression (regex) pattern in a string and returns the match at a specific position (index). A regular expression is a special text pattern that describes what you are looking for. This filter is a shorthand for using `regex_findall` and then picking one result from the list by its index number.

This is useful when you know a sensor value or text contains multiple matches but you only need one specific one. For example, a sensor might report "23.5C / 45% humidity" and you want only the first number (temperature) or the second number (humidity). The index starts at 0 for the first match, 1 for the second, and so on.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "23.5C / 45% humidity" | regex_findall_index("\d+\.\d*") }}' type: string output: "23.5"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
regex_findall_index(
 value: str,
 find: str = "",
 index: int = 0,
 ignorecase: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to search within for matches of the regex pattern. required: true type: string find: description: > The regular expression pattern to search for. All non-overlapping matches are found, and the one at the specified index is returned. required: true type: string index: description: > The position of the match to return, starting from 0 for the first match. Defaults to 0 (the first match). required: false default: "0" type: integer ignorecase: description: > Set to `true` to make the search case-insensitive. required: false default: "false" type: boolean

## Selecting different matches

---

Use the `index` parameter to pick which match you want from the results.

### template

```
template: |
 {{
 "10:15 - 23.5C, 11:30 - 24.1C"
 | regex_findall_index("\\d+\\.\\d+", index=0)
 }}
 {{
 "10:15 - 23.5C, 11:30 - 24.1C"
 | regex_findall_index("\\d+\\.\\d+", index=1)
 }}
title: First and second decimal number
type: string
output: |
 23.5
 24.1
```

## Good to know

---

- Indexing starts at 0 for the first match. Out-of-range indexes raise an error.
- Returns the string form of the match. For numbers, convert with `float` or `int` afterward.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Extract a specific value from a multi-part sensor

When a sensor returns a combined string with multiple values, pick the one you need.

#### template

```
template: |
 {% set state = "Power: 150W | Voltage: 230V | Current: 0.65A" %}
 Voltage: {{ state | regex_findall_index("\\d+\\.?\\d*", index=1) }}V
type: string
output: "Voltage: 230V"
```

### Get the second word in a string

Extract the second capitalized word from a status message.

#### template

```
template: |
 {{
 "Status: Device Online Ready"
 | regex_findall_index("[A-Z][a-z]+", index=1)
 }}
type: string
output: Device
```

### Parse a formatted date component

Extract a specific part from a formatted date string.

#### template

```
template: |
 {% set date_str = "2024-03-15" %}
 Month: {{ date_str | regex_findall_index("\\d+", index=1) }}
type: string
output: "Month: 03"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Regex Match

---

The `regex_match` template filter tests whether a string matches a regular expression (regex) pattern at the beginning of the string. A regular expression is a special text pattern that lets you describe what you are looking for, for example "any number" or "a word followed by a space". It returns `true` if the beginning of the string matches the pattern, and `false` otherwise.

This is useful when you need to check whether a sensor value, entity ID, or other text starts with a specific pattern. For example, you might want to verify that a sensor value looks like a valid IP address, check if an entity ID starts with a certain prefix, or validate that an input follows an expected format. Because it only checks from the start of the string, use `regex_search` if you need to find a pattern anywhere in the text.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "light.living_room" | regex_match("light\\.") }}' type:
boolean output: "true"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
regex_match(
 value: str,
 find: str = "",
 ignorecase: bool = False,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to test against the regex pattern. required: true type: string find: description: > The regular expression pattern to match at the beginning of the string. required: true type: string ignorecase: description: > Set to `true` to make the match case-insensitive. required: false default: "false" type: boolean

## Match vs search

---

`regex_match` only checks if the pattern matches at the **beginning** of the string. If you need to find a pattern **anywhere** in the string, use `regex_search` instead.

### template

```
template: |
 {{ "Room: Living Room" | regex_match("Living") }}
 {{ "Room: Living Room" | regex_search("Living") }}
title: "match checks the start, search checks anywhere"
type: boolean
output: |
 false
 true
```

## Case-insensitive matching

---

Set `ignorecase` to `true` to match regardless of upper or lowercase letters.

### template

```
template: '{{ "Light.living_room" | regex_match("light", ignorecase=true) }}'
type: boolean
output: "true"
```



## Good to know

---

- Only checks the start of the string. Use `regex_search` to match anywhere.
- A literal dot ( `.` ) must be escaped as `\.` because `.` matches any character in regex.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if an entity ID belongs to a domain

Verify that an entity ID starts with a specific domain prefix.

#### template

```
template: |
 {% set entity = "sensor.outdoor_temperature" %}
 {% if entity | regex_match("sensor\\.") %}
 This is a sensor entity
 {% endif %}
type: string
output: "This is a sensor entity"
```

### Validate a sensor value format

Check if a sensor value looks like a valid number with optional decimal point.

#### template

```
template: '{{ states("sensor.temperature") | regex_match("^-?\\d+\\.?\\d*$") }}'
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Regex Replace

---

The `regex_replace` template filter replaces all occurrences of a regular expression (regex) pattern in a string with a replacement value. A regular expression is a special text pattern that describes what you are looking for; `regex_replace` finds every part of the string that matches and swaps it out. This works like a find-and-replace tool, but with the power of patterns instead of fixed text.

This is useful when you need to clean up or transform text from sensors and other sources. For example, you might want to strip unwanted characters from a sensor value, reformat a phone number, remove units from a measurement string so you can convert it to a number, or clean up device names for display on a dashboard.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ "Hello 123 World 456" | regex\_replace("\d+", "N") }}' type: string output: Hello N World N

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
regex_replace(
 value: str,
 find: str = "",
 replace: str = "",
 ignorecase: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string in which to perform the replacement. required: true type: string find: description: > The regular expression pattern to search for. All matches will be replaced. required: true type: string replace: description: > The string to replace each match with. Can be empty to remove matches entirely. required: true type: string ignorecase: description: > Set to `true` to make the pattern matching case-insensitive. required: false default: "false" type: boolean



## Removing matched text

---

Pass an empty string as the replacement to remove all matches from the string.

### template

```
template: '{{ "Temperature: 23.5 C" | regex_replace("[^0-9.]", "") }}'
title: Strip everything except numbers and dots
type: string
output: "23.5"
```

## Case-insensitive replacement

---

Set `ignorecase` to `true` to match and replace regardless of upper or lowercase letters.

### template

```
template: |
 {{
 "Error: DEVICE offline"
 | regex_replace("error", "Warning", ignorecase=true)
 }}
type: string
output: "Warning: DEVICE offline"
```

## Good to know

---

- Replaces every occurrence, not just the first one.
- Backreferences in the replacement use `\1`, `\2`, and so on, to refer to capturing groups from the pattern.
- Passing an empty replacement deletes every match from the string.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Strip units from a sensor value

Remove non-numeric characters from a sensor state so you can use it as a number.

#### template

```
template: |
 {{ states("sensor.wind_speed") | regex_replace("[^0-9.]", "") | float }}
type: float
output: "15.3"
```

### Clean up a device name for display

Remove unwanted prefixes or suffixes from a device name.

#### template

```
template: |
 {{
 "MQTT - Living Room Sensor [v2]"
 | regex_replace("^MQTT - ", "")
 | regex_replace("\s*\\[.*\]$", "")
 }}
type: string
output: Living Room Sensor
```

### Mask sensitive information

Replace digits in a phone number or account number for display purposes.

#### template

```
template: '{{ "+1-555-123-4567" | regex_replace("\\d(?:=\\d{4})", "**") }}'
type: string
output: "+*-***-***-4567"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Regex Search

---

The `regex_search` template filter searches for a regular expression (regex) pattern anywhere in a string. A regular expression is a special text pattern that describes what you are looking for, such as "a sequence of digits" or "the word error". It returns `true` if the pattern is found anywhere in the string, and `false` otherwise.

This is useful when you need to check whether some text contains a particular pattern, regardless of where it appears. For example, you might want to check if a sensor's state contains a number, see if an error message mentions a specific keyword, or test whether a device's firmware version string includes a certain format. Unlike `regex_match`, which only checks the beginning of the string, `regex_search` scans the entire string.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Temperature is 23.5 degrees" | regex_search("\d+\.\d+") }}' type: boolean output: "true"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
regex_search(
 value: str,
 find: str = "",
 ignorecase: bool = False,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to search within for the regex pattern. required: true type: string find: description: > The regular expression pattern to search for anywhere in the string. required: true type: string ignorecase: description: > Set to `true` to make the search case-insensitive. required: false default: "false" type: boolean

## Case-insensitive searching

---

Set `ignorecase` to `true` to find matches regardless of upper or lowercase letters.

### template

```
template: |
 {{ "Error: Device Offline" | regex_search("error", ignorecase=true) }}
type: boolean
output: "true"
```

## Good to know

---

- Returns `true` or `false`, not the matched text. Use `regex_findall` to extract the match.
- Matches anywhere in the string. Use `regex_match` to require a match at the start.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if a sensor value contains a number

Test whether a sensor state that returns text includes a numeric value.

#### template

```
template: '{{ states("sensor.status_message") | regex_search("\\d+") }}'
type: boolean
output: "true"
```

### Filter entities by pattern in state

Use `regex_search` in a condition to check if a sensor's state mentions a specific keyword.

#### template

```
template: |
 {% if states("sensor.weather_report")
 | regex_search("rain|storm|thunder", ignorecase=true) %}
 Bad weather expected
 {% else %}
 Weather looks fine
 {% endif %}
type: string
output: "Bad weather expected"
```

### Check for a version number format

Verify that a firmware version string contains a semantic version pattern.

#### template

```
template: '{{ "Firmware v2.4.1-beta" | regex_search("v\\d+\\.\\.\\d+\\.\\.\\d+") }}'
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Reject

---

The `reject` filter is the opposite of `select`. It iterates over a list and removes items that pass the given test, keeping only those that fail it.

This is useful when it is easier to describe what you want to exclude rather than what you want to keep. For example, you might want to remove all zero values from a list, exclude unavailable states, or filter out empty strings. Instead of writing a complex `select` with a negated condition, `reject` lets you express the exclusion directly and clearly.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [1, 2, 3, 4, 5] | reject("greaterthan", 3) | list }}' type: list
output: "[1, 2, 3]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
reject(
 value: list,
 *args: str,
) -> iterable
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of items to filter. required: true type: list  
args: description: > The test name as a string, optionally followed by arguments for the test. If no test is provided, items that are truthy are removed. Common tests include `equalto`, `greaterthan`, `lessthan`, `string`, `number`, `contains`, and `is_number`. required: false  
type: string

## Reject by truthiness

---

When no test is specified, `reject` removes items that are truthy, keeping only falsy values.

### template

```
template: '{{ [0, 1, "", "hello", none, true] | reject | list }}'
type: list
output: "[0, '', None]"
```

## Exclude specific values

---

### template

```
template: |
 {{
 ["on", "off", "unavailable", "on", "unknown"]
 | reject("equalto", "unavailable") | list
 }}
title: Remove unavailable states
type: list
output: "['on', 'off', 'on', 'unknown']"
```

## Good to know

---

- Returns an iterable, not a list. Add `| list` before using it with `length`, `first`, or looping twice.
- Without a test, truthy items are removed. Zero, empty strings, and `None` are kept because they are falsy.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Remove zero values before averaging

Exclude zero readings that might skew an average calculation.

#### template

```
template: |
 {{
 [21.5, 0, 19.8, 22.3, 0]
 | reject("equalto", 0)
 | list
 | average
 | round(1)
 }}
type: float
output: "21.2"
```

### Exclude non-numeric values

Remove values that are not valid numbers before performing calculations.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | map(attribute="state")
 | reject("in", ["unavailable", "unknown"])
 | map("float")
 | list
 }}
type: list
output: "[21.5, 19.8, 22.3]"
```

### Remove empty strings

Clean up a list by removing empty or blank entries.

#### template

```
template: |
 {% set items = ["kitchen", "", "bedroom", "", "hall"] %}
 {{ items | reject("equalto", "") | list }}
type: list
output: '["kitchen", "bedroom", "hall"]'
```



## Exclude outlier values

Remove temperature readings that are outside a reasonable range.

### template

```
template: |
 {{
 [21.5, 19.8, -40.0, 22.3, 99.9]
 | reject("greaterthan", 50)
 | reject("lessthan", -20)
 | list
 }}
type: list
output: "[21.5, 19.8, 22.3]"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Rejectattr

---

The `rejectattr` filter is the opposite of `selectattr`. It iterates over a list of objects and removes those where a specified attribute passes the given test, keeping only those that fail it.

This is useful when you want to exclude entities based on an attribute rather than select them. For example, you might want to remove all unavailable entities from a list, exclude entities that are off, or filter out sensors with a specific device class. It provides a cleaner, more readable alternative to writing `selectattr` with a negated condition.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: |

type: list output: ["light.kitchen", "light.living\_room"]

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
rejectattr(
 value: list,
 attribute: str,
 *args: str,
) -> iterable
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of objects to filter. required: true type: list  
attribute: description: > The attribute to test on each item. Supports dotted notation for nested attributes (for example, `attributes.device_class`). required: true type: string args: description: > The test name and optional arguments. If only an attribute is provided (no test), items are removed when the attribute is truthy. Common tests include `eq`, `ne`, `gt`, `lt`, `contains`, and `in`. required: false type: string

## Reject by truthiness

---

When no test is specified, items are removed if the attribute value is truthy.

### template

```
template: |
 {{
 expand("group.all_lights")
 | rejectattr("attributes.brightness")
 | map(attribute="entity_id")
 | list
 }}
title: Lights without a brightness value set
type: list
output: '["light.hall", "light.porch"]'
```

## Good to know

---

- Returns an iterable, not a list. Add `| list` before using it with `length`, `first`, or looping twice.
- Without a test, items are removed when the attribute is truthy.
- Numeric comparisons against state strings compare alphabetically unless you convert first.  
`"9"` is greater than `"10"` as strings.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Exclude unavailable entities

Remove entities that are unavailable or unknown before processing.

#### template

```
template: |
 {{
 expand("group.all_sensors")
 | rejectattr("state", "in", ["unavailable", "unknown"])
 | map(attribute="entity_id")
 | list
 }}
title: Only available sensors
type: list
output: '["sensor.temperature", "sensor.humidity", "sensor.pressure"]'
```

### Remove entities that are off

Keep only entities that are not in the "off" state.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | rejectattr("state", "eq", "off")
 | map(attribute="name")
 | join(", ")
 }}
type: string
output: "Kitchen light, Living room light"
```

### Exclude a specific device class

Remove sensors of a particular device class from a collection.

#### template

```

template: |
 {{
 expand("group.all_sensors")
 | rejectattr("attributes.device_class", "eq", "battery")
 | map(attribute="entity_id")
 | list
 }}
title: All sensors except battery sensors
type: list
output: '["sensor.temperature", "sensor.humidity", "sensor.pressure"]'

```

## Chain rejectattr with selectattr

Combine both filters to precisely control which entities are included.

### template

```

template: |
 {{
 expand("group.all_sensors")
 | rejectattr("state", "in", ["unavailable", "unknown"])
 | selectattr("attributes.device_class", "eq", "temperature")
 | map(attribute="state")
 | map("float")
 | average
 | round(1)
 }}
title: Average temperature excluding unavailable sensors
type: float
output: "21.2"

```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Relative Time

---

The `relative_time` template function returns a human-readable string describing how long ago a datetime occurred. Give it a datetime in the past, and it returns something like "2 hours" or "3 days".

This function has been deprecated in favor of `time_since`, which provides the same functionality with the added ability to control precision. You should use `time_since` for all new templates. `relative_time` remains available for backward compatibility, but may be removed in a future release.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: string output: "2 hours"
```

---

```
filter: " type: string output: "2 hours"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
relative_time(
 value: datetime,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > A datetime object representing a point in the past. The function calculates the time elapsed between this datetime and now. required: true type: datetime



## Migrating to time\_since

---

Replace `relative_time` with `time_since` in your templates. The basic usage is identical:

### template

```
template: '{{ time_since(states.binary_sensor.front_door.last_changed) }}'
title: Using time_since instead
type: string
output: "2 hours"
```

The advantage of `time_since` is the optional precision parameter, which lets you control how many time components to include in the output:

### template

```
template: '{{ time_since(states.binary_sensor.front_door.last_changed, 2) }}'
title: time_since with precision
type: string
output: "2 hours and 30 minutes"
```

## Good to know

---

- Deprecated. Use `time_since` for new templates, which supports a precision argument.
- Only returns the largest time unit. `"2 hours"` is given, not `"2 hours 15 minutes"`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Show how long ago something happened

Display the elapsed time since a sensor last changed state.

#### template

```
template: |
 {{
 "Last updated "
 ~ relative_time(states.sensor.temperature.last_changed) ~ " ago"
 }}
type: string
output: "Last updated 45 minutes ago"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Remap

---

The `remap` template function maps a value from one numeric range to another. Give it a value and its original range, plus the target range, and it returns the proportionally mapped result. For example, a value of 50 in the range 0-100 maps to 127.5 in the range 0-255.

This is useful whenever you need to convert a sensor reading from one scale to another. For example, you might remap a 0-100% brightness value to the 0-255 range a light expects, convert a temperature from one unit range to another, or map a 0-1023 analog sensor reading to a meaningful percentage. The optional `steps` and `edges` parameters give you fine-grained control over quantization and out-of-bounds behavior.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "127.5"

---

filter: '' type: float output: "127.5"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
remap(
 value: Any,
 in_min: Any,
 in_max: Any,
 out_min: Any,
 out_max: Any,
 *,
 steps: int = 0,
 edges: str = "none",
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to remap. Must be numeric. required: true type: float in\_min: description: > The minimum of the input range. Must be numeric. required: true type: float in\_max: description: > The maximum of the input range. Must be numeric. required: true type: float out\_min: description: > The minimum of the output range. Must be numeric. required: true type: float out\_max: description: > The maximum of the output range. Must be numeric. required: true type: float steps: description: > If greater than 0, quantizes the output into the specified number of discrete steps. For example, `steps=4` divides the output range into 4 equal intervals. Defaults to 0 (continuous output). required: false type: integer edges: description: > Controls how input values outside the input range are handled. Accepts one of four options: `none` (default) performs no special handling, allowing out-of-range values to be extrapolated linearly into the output range. `clamp` constrains out-of-range input values to the nearest boundary before mapping. `wrap` wraps out-of-range input values cyclically within the input range. `mirror` reflects out-of-range input values back into the input range. required: false type: string

## Edge handling options

---

The `edges` parameter determines what happens when the input value falls outside the input range.

- `none` (default): Values outside the input range are extrapolated linearly. A value of 120 in a 0-100 input range maps proportionally beyond the output range.
- `clamp`: Values are constrained to the input range boundaries before mapping. A value of 120 in a 0-100 input range is treated as 100.
- `wrap`: Values wrap cyclically. A value of 120 in a 0-100 input range wraps to 20 before mapping.
- `mirror`: Values reflect back into the range. A value of 120 in a 0-100 input range mirrors to 80 before mapping.

### template

```
template: |
 {{ remap(120, 0, 100, 0, 255, edges="clamp") }}
type: float
output: "255.0"
```

## Using steps for quantized output

---

The `steps` parameter divides the output into discrete intervals. This is useful when you need the result to snap to specific levels rather than being a continuous value.

### template

```
template: |
 {{ remap(33, 0, 100, 0, 255, steps=4) }}
type: float
output: "63.75"
```

## Good to know

---

- By default, out-of-range inputs are extrapolated beyond the output range. Pass `edges="clamp"` to limit the result to the output range.
- The output ranges can go in reverse, which produces an inverted mapping.
- With `steps`, the output is quantized to discrete intervals, which is useful for dimmer steps or discrete sensor levels.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Remapping a sensor percentage to brightness

Convert a 0-100% sensor value to the 0-255 brightness range a light expects.

#### template

```
template: |
 {{
 remap(
 states("sensor.ambient_light_pct") | float,
 0, 100, 0, 255
)
 }}
type: float
output: "127.5"
```

### Remapping with wrap edge handling

Wrap out-of-range values cyclically. This keeps the output cycling smoothly.

#### template

```
template: |
 {{ remap(450, 0, 360, 0, 100, edges="wrap") }}
type: float
output: "25.0"
```

### Remapping with mirror edge handling

Mirror out-of-range values back into the range for a ping-pong effect.

#### template

```
template: |
 {{ remap(120, 0, 100, 0, 255, edges="mirror") }}
type: float
output: "204.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Replace

---

The `replace` filter replaces all occurrences of a substring within a string with a new value. You can optionally limit the number of replacements. This is useful when you need to clean up or transform text from sensors and other sources. For example, you might want to remove units from a sensor value, replace underscores with spaces for a friendlier display name, or substitute specific words in a message before sending a notification.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Hello World" | replace("World", "Home Assistant") }}'
```

type: string output: Hello Home Assistant

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
replace(
 value: str,
 old: str,
 new: str,
 count: int | None = None,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string in which to perform the replacement. required: true type: string old: description: > The substring to search for. required: true type: string new: description: > The string to replace each occurrence with. required: true type: string count: description: > The maximum number of replacements to make. If not provided, all occurrences are replaced. required: false type: integer

## Limiting the number of replacements

---

Pass a `count` to only replace the first N occurrences.

### template

```
template: '{{ "a-b-c-d" | replace("-", " ", 2) }}'
title: Replace only the first two hyphens
type: string
output: "a b c-d"
```

## Good to know

---

- Replaces every occurrence by default. Pass a `count` to limit the number of replacements.
- Only matches literal text. For patterns, use `regex_replace`.
- Case-sensitive. `"ON"` does not match `"on"`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Make an entity ID more readable

Replace underscores with spaces to create a friendly name from an entity ID.

#### template

```
template: '{{ "living_room_temperature" | replace("_", " ") | title }}'
type: string
output: Living Room Temperature
```

### Remove unwanted text from a sensor value

Strip a unit suffix from a sensor state so it can be processed as a number.

#### template

```
template: '{{ "23.5 °C" | replace(" °C", "") | float }}'
type: float
output: "23.5"
```

### Clean up a notification message

Replace placeholder text in a message template.

#### template

```
template: '{{ "Welcome home, {name}!" | replace("{name}", "Alice") }}'
type: string
output: "Welcome home, Alice!"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Reverse

---

The `reverse` filter reverses the order of items in a list or characters in a string. When applied to a list, the last item becomes the first and vice versa. When applied to a string, the characters are reversed.

This is useful when you need to display items in the opposite order from how they are stored. For example, you might have a list of recent events sorted oldest-first and want to show them newest-first, or you might want to reverse the order of entities returned by `expand` after sorting. It provides a convenient way to flip any sequence without needing to re-sort with `reverse=true`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [1, 2, 3, 4, 5] | reverse | list }}' type: list output: "[5, 4, 3, 2, 1]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
reverse(
 value: list | str,
) -> list | str
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list or string to reverse. For lists, the result is a reverse iterator (use `| list` to convert back to a list). For strings, a reversed string is returned directly. required: true type: any

## Reversing a string

---

When applied to a string, the characters are reversed directly.

### template

```
template: '{{ "hello" | reverse }}'
type: string
output: "olleh"
```

## Good to know

---

- Returns a reverse iterator for lists. Add `| list` before counting or accessing by index.
- On strings, the characters are reversed, which can break multi-byte sequences like emoji in some cases.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display entities in reverse order

Reverse the alphabetical order of expanded entities.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | sort(attribute="entity_id")
 | reverse
 | map(attribute="entity_id")
 | list
 }}
type: list
output: '["light.porch", "light.kitchen", "light.bedroom"]'
```

### Show most recent events first

Reverse a chronologically sorted list so the most recent items appear first.

#### template

```
template: |
 {% set events = ["06:00 Motion detected", "07:30 Door opened",
 "08:15 Light turned on"] %}
 {% for event in events | reverse %}
 {{ event }}
 {% endfor %}
type: string
output: |
 08:15 Light turned on
 07:30 Door opened
 06:00 Motion detected
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Round

---

The `round` filter rounds a numeric value to a specified number of decimal places. It is a Home Assistant override of the standard `round` filter that adds support for a `default` parameter and the `half` rounding method. When rounding to zero decimals, it returns an integer instead of a float.

Rounding is essential whenever you display sensor values on a dashboard or in a notification. A temperature of `21.456789` is not useful on a dashboard; `21.5` is much better. Similarly, you might want to round energy costs to two decimal places, or display a percentage as a whole number. The `round` filter supports four rounding methods to cover different use cases, from standard rounding to always rounding up or down.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: float output: "21.5"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
round(
 value: Any,
 precision: int = 0,
 method: str = "common",
 default: Any = _SENTINEL,
) -> float | int | Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The value to round. Must be a number or a string that can be converted to a float. required: true type: any precision: description: > The number of decimal places to round to. Defaults to `0`, which returns an integer. required: false default: "0" type: integer method: description: > The rounding method to use. One of `common`, `ceil`, `floor`, or `half`. Defaults to `common`. required: false default: "common" type: string default: description: > Value to return if the conversion fails. If not provided, an error is raised on invalid input. required: false type: any

## Rounding methods

---

The `round` filter supports four rounding methods:

### Common (default)

The default method uses Python's built-in rounding, which follows the "round half to even" (banker's rounding) strategy. Values exactly halfway between two numbers are rounded to the nearest even number.

#### template

```
template: '{{ 21.456 | round(1) }}'
title: Round to 1 decimal
type: float
output: "21.5"
```

#### template

```
template: '{{ 2.5 | round(0) }}'
title: "Banker's rounding: 2.5 rounds to even"
type: integer
output: "2"
```

### Ceil (always round up)

Always rounds up to the next value at the given precision.

#### template

```
template: '{{ 21.1 | round(0, "ceil") }}'
title: Always round up
type: integer
output: "22"
```

### Floor (always round down)

Always rounds down to the previous value at the given precision.

#### template

```
template: '{{ 21.9 | round(0, "floor") }}'
title: Always round down
type: integer
output: "21"
```

## Half (round to nearest 0.5)

Rounds the value to the nearest 0.5 increment. This is useful for thermostats and other devices that operate in half-degree steps.

### template

```
template: '{{ 21.3 | round(0, "half") }}'
type: float
output: "21.5"
```

## Good to know

---

- The default `common` method uses banker's rounding, where 0.5 rounds to the nearest even number. So `2.5 | round` gives `2`, not `3`.
- With `precision=0` (the default), the result is an integer. With a nonzero precision, the result is a float.
- The `half` method rounds to the nearest 0.5 rather than the nearest whole number.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a clean temperature value

Round a temperature sensor to one decimal place for display on a dashboard.

#### template

```
template: '{{ states("sensor.outdoor_temperature") | float(0) | round(1) }}'
type: float
output: "18.3"
```

### Round up for resource planning

When calculating how many items you need, always round up so you don't run short.

#### template

```
template: |
 {{ (states("sensor.paint_area") | float(0) / 10) | round(0, "ceil") }}
title: Cans of paint needed (each covers 10 sqm)
type: integer
output: "4"
```

### Round a cost to two decimals

Display an energy cost with exactly two decimal places.

#### template

```
template: '{{ (states("sensor.energy_today") | float(0) * 0.25) | round(2) }}'
type: float
output: "3.47"
```

### Set a thermostat to the nearest half degree

Round a calculated target temperature to the nearest 0.5 so it matches what the thermostat accepts.

#### action

```
action: |
 action:
 - action: climate.set_temperature
 target:
 entity_id: climate.living_room
 data:
 temperature: >
 {{
 states("sensor.desired_temperature")
 | float(20) | round(0, "half")
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Safe

---

The `safe` filter marks a string as safe HTML, which means it will not be automatically escaped when rendered in an HTML context. Without this filter, HTML characters like `<` and `>` would be converted to their entity equivalents to prevent accidental HTML injection. This is useful when you intentionally want to include HTML markup in your output. For example, you might want to include bold text, links, or line breaks in a Markdown card or an HTML notification message. Use this filter with care and only on content you trust, since it bypasses the automatic escaping that protects against malformed or unexpected HTML.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Important message" | safe }}' type: string output:
"Important message"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
safe(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to mark as safe. It will not be escaped when rendered in an HTML context. required: true type: string



## Good to know

---

- Only apply this to content you trust. Untrusted input marked safe can inject HTML.
- Use `forceescape` downstream to re-escape a value that was marked safe.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Include HTML formatting in a notification

Mark a string as safe to include bold text in an HTML notification.

#### template

```
template: |
 {{
 "Alert: Motion detected in the "
 ~ states("sensor.last_motion_room") | safe
 }}
type: string
output: "Alert: Motion detected in the living room"
```

### Add a line break in output

Include an HTML line break in your template output.

#### template

```
template: '{{ ("Temperature: 22°C
Humidity: 65%") | safe }}'
type: string
output: "Temperature: 22°C
Humidity: 65%"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sameas

---

The `sameas` test checks whether two values are the exact same object in memory (identity), not merely equal in value. It is equivalent to Python's `is` operator. Use `value is sameas(other)` to perform the check.

In most Home Assistant templates, you will use `eq` or `==` for value comparisons. The `sameas` test is primarily useful when you need to distinguish between identity and equality, such as checking if a value is specifically `true`, `false`, or `none` as singletons rather than merely equal to them.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | Same object
```

```
type: string output: "Same object"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sameas(
 value: Any,
 other: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. This is the left-hand side of the identity check. required: true type: any other: description: > The object to compare against. The test checks if both refer to the same object. required: true type: any



## Good to know

---

- Identity, not equality. `1 is sameas(true)` is `false` even though `1 == true`.
- Mostly useful for checking singletons like `true`, `false`, and `none`. For normal value comparison, use `eq`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Identity vs equality

Two values can be equal but not the same object.

#### template

```
template: |
 {{ true is sameas(true) }}
 {{ 1 is sameas(true) }}
type: boolean
output: |
 true
 false
```

### Check for specific singletons

Verify that a value is the actual `none` singleton rather than something that equals `None`.

#### template

```
template: |
 {% set val = none %}
 {% if val is sameas(none) %}
 Value is None
 {% endif %}
type: string
output: "Value is None"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Search

---

The `search` template test checks whether a string contains a regular expression (regex) pattern anywhere within it. A regular expression is a special text pattern that lets you describe what you are looking for, such as "any number" or "the word offline". As a template test, it is used with the `is` keyword, making your templates read like natural language.

This is useful in conditions and `{% jinja %}{% endjinja %}` blocks where you want to check if a value contains a certain pattern. For example, you might test whether a sensor's state contains the word "error", whether a notification message includes a phone number, or whether a device attribute mentions a particular model. Unlike the `match` test, which only checks the beginning of the string, `search` scans the entire string for the pattern.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: |

```
Contains a decimal number
```

type: string output: "Contains a decimal number"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
search(
 value: str,
 find: str = "",
 ignorecase: bool = False,
) -> bool
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to search within for the regex pattern. required: true type: string find: description: > The regular expression pattern to search for anywhere in the string. required: true type: string ignorecase: description: > Set to `true` to make the search case-insensitive. required: false default: "false" type: boolean



## Good to know

---

- Returns `true` or `false` only. Use `regex_findall` to get the matched substrings.
- Case-sensitive by default. Pass `ignorecase=true` to match regardless of case.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if a state contains a keyword

Test whether a sensor's state text contains a specific word, regardless of case.

#### template

```
template: |
 {% if states("sensor.system_status") is search("offline", ignorecase=true) %}
 System is offline!
 {% endif %}
type: string
output: "System is offline!"
```

### Filter entities with a pattern in their state

Use the `search` test to find entities whose state contains a particular pattern.

#### template

```
template: |
 {% for entity in states.sensor
 if entity.state is search("error|fault|fail", ignorecase=true) %}
 {{ entity.entity_id }}: {{ entity.state }}
 {% endfor %}
```

### Check for a numeric value in text

Test whether a text-based sensor state includes any number.

#### template

```
template: |
 {% if states("sensor.status_message") is search("\\d+") %}
 Status contains a number
 {% else %}
 Status is text only
 {% endif %}
type: string
output: "Status contains a number"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Select

---

The `select` filter iterates over a list and keeps only the items that pass a given test. Each item is tested individually, and only those for which the test returns true are included in the result.

This is one of the most powerful filters for working with lists of values in Home Assistant templates. You can use it to filter numeric values (keep only those above a threshold), filter strings (keep only those matching a pattern), or apply any of the built-in tests. It works on the items themselves; to filter by an attribute of each item, use `selectattr` instead. The `select` filter is commonly combined with `map` to first extract values, then filter them.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [1, 2, 3, 4, 5] | select("greaterthan", 3) | list }}' type: list
output: "[4, 5]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
select(
 value: list,
 *args: str,
) -> iterable
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of items to filter. required: true type: list  
args: description: > The test name as a string, optionally followed by arguments for the test. If no test is provided, items are tested for truthiness. Common tests include `equalto`, `greaterthan`, `lessthan`, `string`, `number`, `contains`, `is_state`, and `is_number`. required: false type: string



## Filter by truthiness

---

When no test is specified, `select` keeps items that are truthy (not `false`, `0`, `none`, or empty).

### template

```
template: '{{ [0, 1, "", "hello", none, true] | select | list }}'
type: list
output: "[1, 'hello', True]"
```

## Common tests

---

### Greater than / less than

#### template

```
template: |
 {{
 [18.5, 21.3, 25.0, 19.8, 22.1]
 | select("greaterthan", 20)
 | list
 }}
title: Temperatures above 20
type: list
output: "[21.3, 25.0, 22.1]"
```

### Equal to

#### template

```
template: |
 {{ ["on", "off", "on", "off", "on"] | select("equalto", "on") | list }}
title: Keep only "on" values
type: list
output: "['on', 'on', 'on']"
```

### Contains

#### template

```
template: |
 {{
 ["sunny", "light rain", "cloudy", "heavy rain"]
 | select("contains", "rain")
 | list
 }}
title: Weather conditions with rain
type: list
output: "['light rain', 'heavy rain']"
```

## Good to know

---

- Returns an iterable, not a list. Add `| list` before using it with `length`, `first`, or looping twice.
- Without a test, truthy items are kept. Zero, empty string, and `None` are dropped because they are falsy.
- To filter by an attribute of each item, use `selectattr` instead.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Filter numeric values from sensor data

After extracting state values, keep only those that are valid numbers.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | map(attribute="state")
 | select("is_number")
 | map("float")
 | list
 }}
type: list
output: "[21.5, 19.8, 22.3]"
```

### Find temperatures above a threshold

Extract values, convert to float, and filter for those above a target.

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | map(attribute="state")
 | map("float")
 | select("greaterthan", 21)
 | list
 }}
type: list
output: "[21.5, 22.3]"
```

### Count values matching a condition

Combine `select` with `list` and `length` to count matching items.

#### template

```
template: |
 {{
 expand("group.all_doors")
 | map(attribute="state")
 | select("equalto", "on")
 | list
 | length
 }}
title: Number of open doors
type: integer
output: "2"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Selectattr

---

The `selectattr` filter iterates over a list of objects and keeps only those where a specified attribute passes a given test. It is the attribute-based counterpart of `select`: while `select` tests the item itself, `selectattr` tests an attribute of each item.

This is one of the most frequently used filters in Home Assistant templates. It is the primary way to filter entity state objects returned by `expand`. You can filter lights that are on, sensors above a threshold, devices in a specific area, or entities matching any condition based on their attributes. It is incredibly versatile and appears in the vast majority of templates that work with entity collections.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: |

type: list output: ["light.kitchen", "light.living\_room"]

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
selectattr(
 value: list,
 attribute: str,
 *args: str,
) -> iterable
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of objects to filter. required: true type: list  
attribute: description: > The attribute to test on each item. Supports dotted notation for nested attributes (for example, `attributes.device_class`). required: true type: string args: description: > The test name and optional arguments. If only an attribute is provided (no test), items are kept when the attribute is truthy. Common tests include `eq`, `ne`, `gt`, `lt`, `ge`, `le`, `contains`, `is_state`, and `in`. required: false type: string

## Filter by truthiness

---

When no test is specified, items are kept if the attribute value is truthy.

### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("attributes.brightness")
 | map(attribute="entity_id")
 | list
 }}
title: Lights that have a brightness attribute set
type: list
output: '["light.kitchen", "light.living_room"]'
```

## Common tests

---

### Equal to (eq)

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | map(attribute="name")
 | join(", ")
 }}
title: Names of lights that are on
type: string
output: "Kitchen light, Living room light"
```

### Not equal to (ne)

#### template

```
template: |
 {{
 expand("group.all_sensors")
 | selectattr("state", "ne", "unavailable")
 | map(attribute="entity_id")
 | list
 }}
title: Available sensors
type: list
output: '["sensor.temperature", "sensor.humidity", "sensor.pressure"]'
```

### Greater than (gt)

#### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | selectattr("state", "gt", "22")
 | map(attribute="entity_id")
 | list
 }}
title: Sensors above 22
type: list
output: '["sensor.kitchen_temp"]'
```

## Good to know

---

- Returns an iterable, not a list. Add `| list` before using it with `length`, `first`, `last`, or looping twice.
- Without a test name, items are kept when the attribute value is truthy. Zero, empty string, and `None` are treated as false.
- Numeric comparisons compare state strings alphabetically unless you convert first. `"9"` is greater than `"10"` as strings. Apply `| map(attribute='state') | map('float')` before comparing, or filter with a test that converts on the fly.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count lights on in a specific area

Filter entities by state and count the results.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | list
 | length
 }}
type: integer
output: "3"
```

### Filter by device class

Use dotted notation to filter by nested attributes like device class.

#### template

```
template: |
 {{
 expand("group.all_sensors")
 | selectattr("attributes.device_class", "eq", "temperature")
 | map(attribute="entity_id")
 | list
 }}
type: list
output: |
 ["sensor.bedroom_temp", "sensor.kitchen_temp",
 "sensor.living_room_temp"]
```

### Find entities with state containing a substring

Use the `contains` test to match partial state values.

#### template



```

template: |
 {{
 expand("group.media_players")
 | selectattr("state", "contains", "play")
 | map(attribute="name")
 | join(", ")
 }}
type: string
output: "Living room speaker, Bedroom speaker"

```

## Chain multiple selectattr filters

Apply multiple filters in sequence to narrow down results.

### template

```

template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | selectattr("attributes.brightness", "gt", 100)
 | map(attribute="name")
 | join(", ")
 }}
title: Bright lights that are on
type: string
output: "Kitchen light, Living room light"

```

## Filter unavailable entities

Find entities that are unavailable or unknown.

### template

```

template: |
 {{
 expand("group.all_sensors")
 | selectattr("state", "in", ["unavailable", "unknown"])
 | map(attribute="entity_id")
 | list
 }}
title: Sensors that need attention
type: list
output: '["sensor.outdoor_temp", "sensor.garage_humidity"]'

```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sequence

---

The `sequence` test checks whether a value is a sequence type, which includes lists, tuples, and strings. Unlike the `iterable` test, it returns `false` for dictionaries and generators. Unlike `mapping`, it returns `false` for dictionaries.

This is useful when you need to distinguish between sequences (ordered, indexed collections) and mappings (key-value collections). For example, when processing entity attributes that could be either a list or a dictionary, testing with `sequence` (and confirming the value is not a `mapping`) lets you determine the correct way to access the data.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is a sequence
```

```
type: string output: "It is a sequence"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sequence(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is a sequence (list, tuple, or string). required: true type: any

## Good to know

---

- Dictionaries pass this test in Jinja, so pair with `is not mapping` when you need to exclude them.
- Strings pass as sequences because they behave like sequences of characters.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check various types

Lists and strings are sequences, but dictionaries and numbers are not.

#### template

```
template: |
 {{ [1, 2, 3] is sequence }}
 {{ "hello" is sequence }}
 {{ {"a": 1} is sequence }}
 {{ 42 is sequence }}
type: boolean
output: |
 true
 true
 true
 false
```

### Determine how to process data

Check whether an attribute is a list-like sequence before indexing into it.

#### template

```
template: |
 {% set data = state_attr("sensor.forecast", "list") %}
 {% if data is sequence and data is not mapping %}
 First item: {{ data[0] }}
 {% endif %}
type: string
output: "First item: sunny"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Set

---

The `set` template function converts a collection (like a list) into a set. A set is an unordered collection of unique values, so any duplicate items in the input are automatically removed. This gives you a quick way to deduplicate a list.

This is useful when you need to work with unique values only. For example, if you collect entity states from multiple sources and some appear more than once, converting to a set removes the duplicates. You can also use it to count the number of distinct values, or to prepare data for set operations like `intersect`, `union`, and `difference`. Note that sets are unordered, so the items may not be in the same order as the original list.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: set output: "{1, 2, 3}"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
set(
 value: list,
) -> set
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The collection to convert to a set. Duplicate values are removed. required: true type: list

## Good to know

---

- Sets are unordered, so add `| list | sort` when you need a stable ordering.
- All items must be hashable. Lists and dicts inside the input raise an error.
- A quick way to count distinct values is `| set | list | count`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Count unique values

Find out how many distinct states exist among a group of entities.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | map(attribute="state")
 | set
 | list
 | count
 }}
type: integer
output: "2"
```

### Deduplicate a list

Remove duplicates from a list of detected objects.

#### template

```
template: '{{ set(["person", "car", "person", "dog", "car"]) | list }}'
type: list
output: '["person", "car", "dog"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sha1

---

The `sha1` template function calculates the SHA1 hash of a string and returns the result as a hexadecimal string. SHA1 produces a 160-bit (40-character hex) hash value from any input string.

This is useful when you need a longer hash than MD5 for generating identifiers or checksums. SHA1 provides a 40-character hex output compared to MD5's 32 characters. Like MD5, it is not recommended for security-sensitive purposes. For cryptographic use cases, prefer `sha256` or `sha512`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: string output:
"0a0a9f2a6772942557ab5355d76af442f8f65e01"
```

---

```
filter: '{{ "Hello, World!" | sha1 }}' type: string output:
"0a0a9f2a6772942557ab5355d76af442f8f65e01"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sha1(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to hash. The string is encoded to UTF-8 before hashing. required: true type: string

## Good to know

---

- SHA1 is no longer considered secure for cryptographic use. Prefer `sha256` or `sha512` for anything sensitive.
- The result is always a 40-character lowercase hexadecimal string.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Generate a unique identifier

Create a consistent identifier from a combination of entity ID and date.

#### template

```
template: '{{ sha1("sensor.temperature_" ~ now().date() | string) }}'
type: string
output: "b2e5f8a1c3d4e6f7a8b9c0d1e2f3a4b5c6d7e8f9"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sha256

---

The `sha256` template function calculates the SHA256 hash of a string and returns the result as a hexadecimal string. SHA256 produces a 256-bit (64-character hex) hash value and is part of the SHA-2 family of cryptographic hash functions.

This is a good general-purpose hash function suitable for generating secure identifiers, verifying data integrity, or creating unique keys. SHA256 is widely used and considered secure for most applications. For an even longer hash, use `sha512`. For shorter (but less secure) hashes, see `md5` or `sha1`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: "" type: string output:
"dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f"
```

---

```
filter: '{{ "Hello, World!" | sha256 }}' type: string output:
"dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sha256(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to hash. The string is encoded to UTF-8 before hashing. required: true type: string

## Good to know

---

- The result is always a 64-character lowercase hexadecimal string.
- The input is encoded as UTF-8 before hashing. Identical strings always produce the same hash.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Create a verification hash

Generate a hash to verify that a message has not been tampered with.

#### template

```
template: '{{ sha256("important_data_" ~ states("sensor.temperature")) }}'
type: string
output: "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sha512

---

The `sha512` template function calculates the SHA512 hash of a string and returns the result as a hexadecimal string. SHA512 produces a 512-bit (128-character hex) hash value and is part of the SHA-2 family of cryptographic hash functions.

This is the strongest hash function available in Home Assistant templates. It produces the longest hash output, which minimizes the chance of collisions. Use it when you need maximum hash strength, or when an external service specifically requires SHA512. For most purposes, `sha256` is sufficient. For shorter hashes where security is not critical, see `md5` or `sha1`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: string output:
"374d794a95cdcfd8b35993185fef9ba368f160d8daf432d08ba9f1ed1e5abe6cc69291e0fa2fe0006a52570ef18c19def
```

---

```
filter: '{{ "Hello, World!" | sha512 }}' type: string output:
"374d794a95cdcfd8b35993185fef9ba368f160d8daf432d08ba9f1ed1e5abe6cc69291e0fa2fe0006a52570ef18c19def
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sha512(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to hash. The string is encoded to UTF-8 before hashing. required: true type: string

## Good to know

---

- The result is always a 128-character lowercase hexadecimal string.
- SHA512 is cryptographically strong but produces the longest output. Use `sha256` when you need a shorter but still secure hash.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Generate a long unique identifier

Create a strong hash from an entity ID for use as a unique key.

#### template

```
template: '{{ sha512("binary_sensor.front_door") }}'
type: string
output: "a1b2c3d4e5f6..."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Shuffle

---

The `shuffle` template function takes a list and returns a new list with the items in a random order. Every time it is evaluated, the order can be different, giving you a way to introduce randomness into your templates.

This is useful when you want to pick a random item from a collection or display things in a random order. For example, you might shuffle a list of greetings to get a different welcome message each time, randomize the order of automations to stagger their execution, or select a random playlist. You can also pass a `seed` value to get a reproducible shuffle, which can be helpful for testing or when you want the same random order for a given input.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: list output: '["c", "a", "d", "b"]'

---

filter: '{{ ["a", "b", "c", "d"] | shuffle }}' type: list output: '["c", "a", "d", "b"]'

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
shuffle(
 *args: list | Any,
 seed: Any = None,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The list of items to shuffle. Can be provided as a list or as multiple separate arguments. required: true type: list seed: description: > An optional seed value for reproducible shuffling. When the same seed is used, the same order is produced every time. required: false type: any

## Reproducible shuffle with a seed

---

Pass a seed to get the same shuffled order each time. This is useful for testing or when you want consistent randomization based on a known value.

### template

```
template: '{{ shuffle(["a", "b", "c", "d"], seed="fixed") }}'
type: list
output: '["b", "d", "a", "c"]'
```

## Good to know

---

- A new random order is returned each time the template renders. Template entities using `shuffle` re-evaluate and change often.
- Pass a `seed` to get a reproducible order, which is useful for testing or stable randomness tied to a date.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Pick a random greeting

Shuffle a list and take the first item to select a random value.

#### template

```
template: |
 {{ shuffle(["Hello!", "Hi there!", "Welcome!", "Good day!"]) | first }}
type: string
output: "Welcome!"
```

### Shuffle individual arguments

You can also pass items as separate arguments instead of a list.

#### template

```
template: '{{ shuffle("red", "green", "blue") }}'
type: list
output: '["green", "red", "blue"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sin

---

The `sin` template function returns the sine of a value given in radians. It wraps Python's `math.sin`, so the input is expected to be an angle measured in radians rather than degrees.

This is helpful when you need to create smooth cyclic animations, compute positional offsets for automations, or perform trigonometric calculations based on sensor data such as wind direction or solar elevation.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "1.0"

---

filter: '' type: float output: "1.0"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sin(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The angle in radians to calculate the sine of. Must be numeric. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is not numeric). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the input value might not be numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{ sin(states("sensor.wind_direction"), default=0) }}
type: float
output: "0"
```



## Good to know

---

- The input must be in radians, not degrees. Multiply by `pi / 180` to convert degrees to radians.
- Due to floating-point precision, `sin(pi)` returns a very small number instead of exactly zero.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Sine of a known angle

Calculate the sine of 90 degrees by first converting to radians.

#### template

```
template: |
 {{ sin(90 * (pi / 180)) }}
type: float
output: "1.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Slice

---

The `slice` filter divides a list into a specified number of sub-lists of approximately equal size. If the items do not divide evenly, the earlier sub-lists will have one more item than the later ones. You can optionally provide a fill value to pad the shorter sub-lists.

This is useful when you need to distribute items evenly across a fixed number of groups. For example, you might want to split a list of entities into columns for display, distribute tasks across multiple workers, or divide a collection into a set number of pages. Note that `slice` differs from `batch`: `batch` creates groups of a fixed size, while `slice` creates a fixed number of groups.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [1, 2, 3, 4, 5] | slice(3) | list }}' type: list output: "[[1, 2], [3, 4], [5]]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
slice(
 value: list,
 slices: int,
 fill_with: Any = None,
) -> list[list]
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list to slice into sub-lists. required: true type: list  
slices: description: > The number of sub-lists to create. required: true type: integer  
fill\_with: description: > A value to use for padding the shorter sub-lists so they all have the same length. If not provided, sub-lists may have different lengths. required: false type: any



## Padding with fill\_with

---

Use the `fill_with` parameter so all sub-lists have the same number of items.

### template

```
template: '{{ [1, 2, 3, 4, 5] | slice(3, "-") | list }}'
type: list
output: '[[1, 2], [3, 4], [5, "-"]]'
```

## Good to know

---

- You specify the number of sub-lists, not the size of each one. Use `batch` when you want a fixed size per group.
- Returns a generator, so add `| list` before iterating twice.
- Earlier sub-lists get the extra items when the count does not divide evenly.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distribute entities across columns

Split a list of entities into three columns for a balanced display.

#### template

```
template: |
 {% set lights = ["light.kitchen", "light.bedroom", "light.hall",
 "light.porch", "light.garage", "light.office",
 "light.bathroom"] %}
 {% for column in lights | slice(3) %}
 Column {{ loop.index }}: {{ column | join(", ") }}
 {% endfor %}
type: string
output: |
 Column 1: light.kitchen, light.bedroom, light.hall
 Column 2: light.porch, light.garage, light.office
 Column 3: light.bathroom
```

### Split items into two halves

Divide a list into two roughly equal parts.

#### template

```
template: |
 {% set items = range(8) | list %}
 {% set halves = items | slice(2) | list %}
 First half: {{ halves[0] | join(", ") }}
 Second half: {{ halves[1] | join(", ") }}
type: string
output: |
 First half: 0, 1, 2, 3
 Second half: 4, 5, 6, 7
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Slugify

---

The `slugify` template function converts a string into a slug format, which is a URL-friendly, lowercase version of the text with special characters removed and spaces replaced by a separator. By default it uses underscores as the separator, but you can choose any character you like, such as a hyphen.

Slugs are the format Home Assistant uses for entity IDs, so this function is handy whenever you need to build or match entity IDs dynamically. For example, you might want to turn a room name from a sensor into an entity ID format, or create consistent naming for use in automations and scripts. It is also useful when generating filenames or identifiers that should not contain spaces or special characters.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: hello\_world

---

filter: '{{ "Hello World!" | slugify }}' type: string output: hello\_world



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
slugify(
 value: str,
 separator: str = "_",
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to convert into a slug. Special characters are removed, spaces are replaced by the separator, and the result is lowercased. required: true type: string separator: description: > The character to use between words. Defaults to an underscore ( `_` ). required: false default: `"_"` type: string

## Using a custom separator

---

You can use a hyphen or any other character as the separator instead of the default underscore.

### **template**

```
template: '{{ slugify("Living Room Light", "-") }}'
title: Using a hyphen separator
type: string
output: living-room-light
```

## Good to know

---

- The default separator is an underscore, which matches the Home Assistant entity ID format.
- The result is always lowercase, regardless of the input case.
- Accented characters are stripped to plain ASCII equivalents where possible.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build an entity ID from a room name

Turn a human-readable room name into an entity ID format that can be used to look up entities.

#### template

```
template: '{{ "light." ~ slugify("Kitchen Ceiling") }}'
type: string
output: light.kitchen_ceiling
```

### Create a consistent identifier from user input

When processing input from a text helper or other source, convert it to a clean slug for use in templates.

#### template

```
template: |
 {% set room = states("input_text.room_name") %}
 sensor.temperature_{{ slugify(room) }}
type: string
output: sensor.temperature_living_room
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sort

---

The `sort` filter sorts the items of a list in ascending order by default. You can reverse the order, control case sensitivity, and sort by a specific attribute of each item.

This is one of the most frequently used filters when working with entity collections. It allows you to sort sensors by their state value to find the highest or lowest reading, sort lights by brightness, or sort entities alphabetically by name. Combined with `first` or `last`, you can quickly find the minimum or maximum value from any group of entities.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [3, 1, 4, 1, 5, 9] | sort | list }}' type: list output: "[1, 1, 3, 4, 5, 9]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sort(
 value: list,
 reverse: bool = False,
 case_sensitive: bool = False,
 attribute: str | None = None,
) -> list
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list to sort. required: true type: list reverse: description: > If `true`, sorts in descending order. Defaults to `false`. required: false default: "false" type: boolean case\_sensitive: description: > If `true`, uppercase letters sort before lowercase. Defaults to `false`, which treats uppercase and lowercase as equal. required: false default: "false" type: boolean attribute: description: > Sort by this attribute of each item instead of the item itself. Useful for sorting lists of objects by a specific property. required: false type: string

## Sort in descending order

---

Use `reverse=true` to sort from highest to lowest.

### template

```
template: '{{ [3, 1, 4, 1, 5, 9] | sort(reverse=true) | list }}'
type: list
output: "[9, 5, 4, 3, 1, 1]"
```

## Sort by attribute

---

Sort a list of objects by a specific attribute. This is particularly powerful with entity state objects.

### template

```
template: |
 {{
 expand("group.temperature_sensors")
 | sort(attribute="state")
 | map(attribute="entity_id")
 | list
 }}
title: Sort sensors by state value
type: list
output: |
 ["sensor.bedroom_temp", "sensor.living_room_temp",
 "sensor.kitchen_temp"]
```

## Good to know

---

- The default sort is case-insensitive. Pass `case_sensitive=true` for exact-case ordering.
- Sorting a list of entity states sorts strings alphabetically, so `"9"` comes after `"10"`. Convert values to numbers first if you need numeric ordering.
- Mixing types (strings and numbers together) raises an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find the brightest light

Sort lights by brightness in descending order and get the first one.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | sort(attribute="attributes.brightness", reverse=true)
 | map(attribute="entity_id")
 | first
 }}
type: string
output: "light.living_room"
```

### Sort entities alphabetically by friendly name

Sort a group of entities by their friendly name attribute for display.

#### template

```
template: |
 {% for entity in expand("group.all_lights")
 | sort(attribute="name") %}
 {{ entity.name }}: {{ entity.state }}
 {% endfor %}
type: string
output: |
 Bedroom light: off
 Kitchen light: on
 Living room light: on
```

### Get the coldest and warmest temperatures

Use `sort` with `first` and `last` to find both extremes.

#### template

```
template: |
 {% set sensors = expand("group.temperature_sensors")
 | sort(attribute="state") %}
 Coldest: {{ sensors | first | attr("entity_id") }}
 Warmest: {{ sensors | last | attr("entity_id") }}
type: string
output: |
 Coldest: sensor.bedroom_temperature
 Warmest: sensor.kitchen_temperature
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Sqrt

---

The `sqrt` template function returns the square root of a value. It wraps Python's `math.sqrt`, so the input must be a non-negative number.

This is useful for distance calculations, root-mean-square computations, or any template where you need to take the square root of a sensor value. For example, you could calculate the Euclidean distance between two points or convert power readings.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "4.0"
```

---

```
filter: '' type: float output: "4.0"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sqrt(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to calculate the square root of. Must be a non-negative number. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is negative or not numeric). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the input value might be negative or non-numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{ sqrt(states("sensor.area") | float(-1), default=0) }}
type: float
output: "0"
```

## Good to know

---

- Negative inputs raise an error unless you supply a default.
- The result is always a float, even when the input is a perfect square.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distance calculation

Calculate the straight-line distance between two points using the Pythagorean theorem.

#### template

```
template: |
 {% set dx = 3 %}
 {% set dy = 4 %}
 {{ sqrt(dx**2 + dy**2) }}
type: float
output: "5.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/State Attr

---

The `state_attr` template function returns the value of a specific attribute from an entity's state. Entities in Home Assistant often have more information than their main state. For example, a climate entity's state might be `heating`, but it also has attributes like `temperature`, `current_temperature`, and `hvac_modes`.

You'll use this whenever you need information beyond the main state of an entity. Common examples include reading the brightness of a light, the battery level of a device, the friendly name of an entity, or the list of options available on a select entity. If the entity or attribute doesn't exist, it returns `None` instead of causing an error.

💡 **Tip:** For dashboards, a [Tile card](#) can show attributes as badges without a template. Reach for `state_attr()` when you need the value inside an automation, a notification message, or another template.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "21.0"

---

filter: '{{ "climate.living\_room" | state\_attr("temperature") }}' type: float output: "21.0"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
state_attr(
 entity_id: str,
 attribute: str,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: The entity ID to get the attribute from. required: true type: string attribute: description: > The name of the attribute to retrieve. Returns `None` if the entity or attribute does not exist. required: true type: string

 **Tip:** To see all available attributes for an entity, go to **Developer Tools > States** and select the entity. The attributes are listed below the state value.

## Good to know

---

- Returns `None` when the entity or attribute does not exist. Chain with `| default(value)` for a fallback.
- Attribute values keep their original type, so numeric attributes like `brightness` come back as numbers without conversion.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Get the brightness of a light

Read how bright a light is currently set.

#### template

```
template: '{{ state_attr("light.living_room", "brightness") }}'
type: integer
output: "178"
```

### Get battery level

Check the battery percentage of a device.

#### template

```
template: '{{ state_attr("sensor.phone_battery", "battery_level") }}'
type: integer
output: "85"
```

### Use an attribute in a condition

Check if the current temperature is below the target temperature.

#### template

```
template: |
 {{
 state_attr("climate.living_room", "current_temperature")
 < state_attr("climate.living_room", "temperature")
 }}
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/State Attr Translated

---

The `state_attr_translated` template function returns the value of an entity's attribute translated into your configured language. While `state_attr` returns the raw attribute value, this function returns the localized version shown in the interface.

This is useful when you display attribute values that have been translated by integrations, such as HVAC modes, fan modes, or preset names. For example, if your Home Assistant is set to French, `state_attr_translated("climate.living_room", "hvac_action")` might return *Chauffage* instead of `heating`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: Heating

---

filter: '{{ "climate.living\_room" | state\_attr\_translated("hvac\_action") }}' type: string output: Heating

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
state_attr_translated(
 entity_id: str,
 attribute: str,
) -> Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: The entity ID to get the translated attribute from.  
required: true type: string attribute: description: > The name of the attribute to retrieve. Returns the translated value if a translation is available, otherwise returns the raw value. Returns `None` if the entity or attribute does not exist. required: true type: string

## Good to know

---

- Returns `None` when the entity or attribute does not exist.
- Only attributes that integrations provide translations for are localized. Unsupported attributes fall back to the raw value.
- The output depends on the Home Assistant user language, not a fixed locale.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Show the translated HVAC action

Display the current action of a climate entity in your language.

#### template

```
template: |
 The heater is currently: {{ state_attr_translated(
 "climate.living_room", "hvac_action") }}
type: string
output: "The heater is currently: Heating"
```

### Show the translated fan mode

Show a media player's or climate device's mode in a friendly, localized way.

#### template

```
template: 'Fan mode: {{ state_attr_translated("climate.bedroom", "fan_mode") }}'
type: string
output: "Fan mode: Automatic"
```

### Build a localized status message

Combine the translated state with the translated attribute to build a status line in your language.

#### template

```
template: |
 {% set entity = "climate.living_room" %}
 The thermostat is {{ state_translated(entity) }} and {{
 state_attr_translated(entity, "preset_mode") | lower }}.
type: string
output: "The thermostat is Heat and eco."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/State Translated

---

The `state_translated` template function returns the state of an entity translated into the language configured in Home Assistant. While `states` returns the raw English state value (like `on`, `off`, `heating`), this function returns the localized version shown in the interface.

This is useful when you want to include entity states in notifications or dashboard text that should match your configured language. For example, if your Home Assistant is set to German, `state_translated("climate.living_room")` might return *"Heizen"* instead of `heating`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: Heating

---

filter: '{{ "climate.living\_room" | state\_translated }}' type: string output: Heating

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
state_translated(
 entity_id: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: > The entity ID to get the translated state from. Returns the state in your configured language. Returns `unknown` if the entity does not exist.  
required: true type: string

## Good to know

---

- The returned text depends on the language configured in Home Assistant, so comparing the result against a fixed English word like `"heating"` will fail once the language changes. Use `states` for comparisons and reserve `state_translated` for display.
- Translations come from the entity's integration. Entities without translated states fall back to the raw state value.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Use in a notification

Send a notification with the climate state in your configured language.

#### **action**

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 The living room climate is
 {{ state_translated("climate.living_room") }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/States

---

The `states` template function returns the current state of an entity as a text value. For example, a light might return `on` or `off`, a temperature sensor might return `21.5`, and a person might return `home` or `away`.

This is the most fundamental template function in Home Assistant. Every time you want to read what a sensor is showing, check if a device is on, or use a value in a notification or calculation, you'll use `states()`. It's also the safest way to read a state because it returns `unknown` if the entity doesn't exist, instead of causing an error.

 **Tip:** For dashboards, a [Tile card](#) shows an entity's state without any template. Reach for `states()` when you need the value inside an automation, a notification message, or another template.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "21.5"

---

filter: '{{ "sensor.living\_room\_temperature" | states }}' type: string output: "21.5"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
states(
 entity_id: str,
 rounded: bool = False,
 with_unit: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} entity\_id: description: > The entity ID to get the state from. Returns the state as a string, or `unknown` if the entity does not exist. required: true type: string rounded: description: > When `true`, numeric states are rounded according to the entity's display precision (the same rounding shown in the UI). When omitted, it follows `with_unit`. required: false default: "false" type: boolean with\_unit: description: > When `true`, appends the entity's unit of measurement to the state. Implies `rounded` unless `rounded` is set explicitly. required: false default: "false" type: boolean

## Iterating over states

---

You can also use `states` without an argument to loop through all entities, or use `states.domain` to loop through entities of a specific domain.

### template

```
template: |
 {% for state in states.sensor %}
 {{ state.entity_id }}: {{ state.state }}
 {% endfor %}
```

 **Tip:** Not sure what state an entity has? Go to **Developer Tools > States** to see the current state and all attributes of every entity in your system.

## Important: states are always strings

---

The value returned by `states()` is always a text string, even for numeric sensors. If you need to do math with it, convert it using `| float` or `| int` first.

### template

```
template: '{{ states("sensor.temperature") | float + 1 }}'
type: float
output: "22.5"
```



## Good to know

---

- Returns the text `unknown` when the entity does not exist, not `None` and not an error.
- The result is always a string, even for numeric sensors. Convert with `float` or `int` before doing math.
- Using `states.sensor.name` (dotted notation) raises an error when the entity is missing, while `states("sensor.name")` does not.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Use a sensor value in a notification

Include the current temperature reading in a notification message. Use `with_unit=true` to automatically append the entity's unit of measurement, so the output uses whatever unit the sensor reports (°C, °F, K) without hardcoding it.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 The temperature is {{ states("sensor.outside_temperature", with_unit=t
```

### Check a value before acting

Read a sensor value, convert it to a number, and use it in a condition.

#### template

```
template: '{{ states("sensor.humidity") | float > 70 }}'
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Statistical Mode

---

The `statistical_mode` template function returns the most commonly occurring value in a collection. Give it a list of values and it returns the one that appears most often. If multiple values share the highest frequency, it returns the first one encountered.

This is useful when you want to find the most frequent reading from a set of sensors rather than a numeric average. For example, if several motion sensors each report a room as either "occupied" or "empty", the mode tells you what the majority of sensors agree on. It also works with numeric values, such as finding the most common brightness setting across a group of lights.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "21"

---

filter: '{{ [21, 22, 21, 23, 21] | statistical\_mode }}' type: float output: "21"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
statistical_mode(
 *args: list | float,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} values: description: > The values to find the mode of. Can be a list or multiple separate arguments. required: true type: list default: description: > Value to return if the calculation fails (for example, if the list is empty). If not provided, an error is raised instead. required: false type: any



## Using a default value

---

If the list might be empty or contain invalid values, provide a default to avoid errors. This prevents your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{
 [states("sensor.maybe_broken") | float(none)]
 | reject("none")
 | list
 | statistical_mode(default=0)
 }}
type: float
output: "0"
```

## Good to know

---

- When multiple values are tied for the highest frequency, the first one encountered wins.
- Works with any hashable type, not only numbers. Strings like `"on"` and `"off"` are valid inputs too.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Most common temperature reading

Find the most frequently reported temperature across multiple sensors.

#### template

```
template: |
 {{
 statistical_mode(
 states("sensor.living_room_temperature") | float,
 states("sensor.bedroom_temperature") | float,
 states("sensor.kitchen_temperature") | float,
 states("sensor.hallway_temperature") | float
)
 }}
type: float
output: "21.0"
```

### Mode across a group of entities

If you have a group of sensors, you can expand the group and find the most common value in one expression.

#### template

```
template: |
 {{
 expand("group.indoor_temperatures")
 | map(attribute="state")
 | map("float")
 | statistical_mode
 }}
type: float
output: "21.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/String

---

The `string` filter converts a value to its string representation. Numbers, booleans, lists, and other types are all converted to their textual form. While most values in Home Assistant templates are already strings (especially entity states), you may encounter numbers or booleans from calculations or attributes that need to be explicitly converted to strings before concatenation or comparison. The `string` filter makes this conversion explicit and clear.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "42"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | string() -> str
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The value to convert to a string. Any type is accepted. required: true type: any

## Good to know

---

- Booleans become `"True"` and `"False"` with a capital letter, matching Python's output.
- Lists and dictionaries are rendered using Python's `repr` format, so a list shows up as `"[1, 2, 3]"` with single quotes around strings.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Convert a number for display

Ensure a numeric result is treated as a string when building a message.

#### template

```
template: '{{ "Battery level: " ~ (87 | string) ~ "%" }}'
type: string
output: "Battery level: 87%"
```

### Convert a boolean to string

Convert a boolean value to its string form for use in a notification or label.

#### template

```
template: '{{ true | string }}'
type: string
output: "True"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/String Test

---

The `string` test checks whether a value is a string. It returns `true` if the value is of string type and `false` for any other type, including numbers and booleans.

In Home Assistant, entity states are always strings, but attributes can be various types including numbers, booleans, and lists. This test is useful when you need to verify that a value is a string before performing string operations like splitting, replacing, or pattern matching.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is a string
```

type: string output: "It is a string"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
string(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` if the value is of string type. required: true type: any

## Good to know

---

- A numeric-looking string such as `"42"` still passes this test. The test looks at the type, not the content.
- All entity states are strings in Home Assistant, so this test is most useful for attributes or values built inside a template.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Distinguish strings from numbers

Check whether a value is a string or a numeric type.

#### template

```
template: |
 {{ "hello" is string }}
 {{ 42 is string }}
 {{ "42" is string }}
type: boolean
output: |
 true
 false
 true
```

### Conditionally process based on type

Handle an attribute differently depending on whether it is a string or another type.

#### template

```
template: |
 {% set val = state_attr("sensor.device", "info") %}
 {% if val is string %}
 Info: {{ val }}
 {% else %}
 Info: {{ val | tojson }}
 {% endif %}
type: string
output: "Info: Ready"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Striptags

---

The `striptags` filter removes all HTML and XML tags from a string, leaving only the plain text content. It also normalizes whitespace by collapsing multiple spaces into one and trimming the result. This is useful when you receive HTML-formatted content from a sensor or external source and need only the plain text. For example, some weather services or RSS feed sensors provide descriptions with HTML markup, and you might want to strip the tags before displaying the content in a notification or on a dashboard card.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "
```

Hello **World**

```
" | striptags }}' type: string output: Hello World
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
striptags(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string from which to remove all HTML/XML tags. Whitespace is normalized in the result. required: true type: string

## Good to know

---

- Whitespace is normalized: runs of spaces, tabs, and newlines are collapsed into single spaces, and the result is trimmed.
- Only tags are removed. HTML entities like `&amp;` or `&nbsp;` remain as-is. Combine with `replace` if you need to decode them.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Extract plain text from an HTML sensor

Strip HTML from a sensor that provides formatted content.

#### template

```
template: '{{ states.sensor.news_headline.attributes.description | striptags }}'
type: string
output: "Breaking news: severe weather warning issued for the area."
```

### Clean up HTML content for a notification

Remove HTML tags from a message before sending it as a plain text notification.

#### template

```
template: |
 {{
 "<h1>Alert</h1><p>Motion detected in the backyard.</p>"
 | striptags
 }}
type: string
output: "Alert Motion detected in the backyard."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Strptime

---

The `strptime` template function parses a date/time string according to a format you specify and returns a datetime object. Give it a string like "15/03/2024 14:30" along with the matching format "%d/%m/%Y %H:%M", and it converts the text into a proper datetime you can work with.

While `as_datetime` handles ISO 8601 formatted strings automatically, many sensors and external data sources provide dates in non-standard formats. A weather service might return "March 15, 2024", a calendar integration might give you "15/03/2024", or a custom sensor might report "2024.03.15 14:30". `strptime` lets you parse any date format by telling it exactly how the string is structured. The format codes follow Python's strftime/strptime conventions.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: "" type: datetime output: "2024-03-15 14:30:00"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
strptime(
 string: str,
 format: str,
 default: Any = None,
) -> datetime | Any
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} string: description: The date/time string to parse. required: true type: string format: description: > The format string describing the structure of the date/time string. Uses Python strftime format codes such as `%Y` (year), `%m` (month), `%d` (day), `%H` (hour), `%M` (minute), `%S` (second). required: true type: string default: description: > Value to return if the string cannot be parsed with the given format. If not provided, an error is raised on invalid input. required: false type: any

## Common format codes

---

Here are the most frequently used format codes and what each one stands for:

- `%Y` : four-digit year (for example, `2024` )
- `%m` : zero-padded month (for example, `03` )
- `%d` : zero-padded day (for example, `15` )
- `%H` : hour in 24-hour format (for example, `14` )
- `%M` : minute (for example, `30` )
- `%S` : second (for example, `00` )
- `%B` : full month name (for example, `March` )
- `%A` : full weekday name (for example, `Friday` )
- `%I` : hour in 12-hour format (for example, `02` )
- `%p` : AM or PM

## Using a default value

---

If the string might not match the expected format, provide a default to avoid errors.

### template

```
template: '{{ strptime("invalid", "%Y-%m-%d", default="unknown") }}'
type: string
output: "unknown"
```

## Good to know

---

- The returned datetime has no time zone attached unless the format string includes one (for example, with `%z`). Compare it to another naive datetime or add a time zone before comparing with `now`.
- Missing components default to their lowest value. A format that only parses a time gives you a datetime on `1900-01-01`.
- The format string must match the input exactly. Extra spaces, different separators, or a mismatched case for month names causes parsing to fail.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Parse a date with month name

Parse a date string that uses the full month name.

#### template

```
template: '{{ strptime("March 15, 2024", "%B %d, %Y") }}'
type: datetime
output: "2024-03-15 00:00:00"
```

### Parse a 12-hour time format

Parse a time that uses AM/PM notation.

#### template

```
template: '{{ strptime("2:30 PM", "%I:%M %p") }}'
type: datetime
output: "1900-01-01 14:30:00"
```

### Parse and compare with now

Parse a date string from a sensor and check if it is in the past.

#### template

```
template: |
 {% set event_date = strptime(states("sensor.next_event"), "%Y-%m-%d %H:%M") %}
 {{ event_date < now().replace(tzinfo=None) }}
type: boolean
output: "false"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Sum

---

The `sum` filter adds up all the values in a list and returns the total. You can optionally specify an `attribute` to sum a specific property from each item, and a `start` value to begin the summation from.

This is useful whenever you need to total up numeric values from sensors or entities. For example, you might want to calculate total energy consumption across multiple meters, sum up the brightness values of all lights, or add up the number of items detected by multiple sensors. It works well at the end of a filter chain after extracting and converting values to numbers.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [10, 20, 30] | sum }}' type: integer output: "60"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
sum(
 value: list,
 attribute: str | None = None,
 start: int | float = 0,
) -> int | float
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list of values to sum. Items should be numeric. required: true type: list attribute: description: > If provided, this attribute is extracted from each item and summed. Useful for summing a property from a list of objects without needing a separate `map` call. required: false type: string start: description: > A starting value for the sum. Defaults to `0`. required: false default: "0" type: float

## Good to know

---

- An empty list returns `0` (or the `start` value if you provided one) rather than raising an error.
- Every item in the list needs to be numeric. If states or attributes might be strings, pipe them through `map("float")` or `map("int")` first.

## Sum with attribute

---

Instead of using `map(attribute=...)` followed by `sum`, you can pass the attribute name directly to `sum`.

### template

```
template: |
 {% set items = [{"name": "a", "value": 10}, {"name": "b", "value": 20}] %}
 {{ items | sum(attribute="value") }}
type: integer
output: "30"
```

## Sum with a start value

---

Use the `start` parameter to add an offset to the total.

### template

```
template: '{{ [10, 20, 30] | sum(start=100) }}'
type: integer
output: "160"
```

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Total energy consumption across meters

Sum the state values from multiple energy sensors.

#### template

```
template: |
 {{
 expand("group.energy_meters")
 | map(attribute="state")
 | map("float")
 | sum
 | round(2)
 }}
type: float
output: "47.83"
```

### Total brightness of all lights that are on

Sum the brightness attribute across active lights.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | selectattr("state", "eq", "on")
 | map(attribute="attributes.brightness")
 | sum
 }}
type: integer
output: "447"
```

### Calculate total daily cost

Multiply each sensor's consumption by a rate and sum the results.

#### template



```
template: |
 {% set rate = 0.25 %}
 {{
 expand("group.energy_meters")
 | map(attribute="state")
 | map("float")
 | sum
 | multiply(rate)
 | round(2)
 }}
title: Total energy cost
type: float
output: "11.96"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Symmetric Difference

---

The `symmetric_difference` template function returns a list of items that are in either of the two lists, but not in both. This is the set symmetric difference operation: it gives you the items that are unique to each list, excluding any items that the two lists share.

This is useful when you want to find what has changed between two collections. For example, you might compare today's list of active devices with yesterday's to see which devices were added or removed. Or you might compare the members of two groups to find which entities are exclusive to each group. It effectively combines the results of two `difference` calls in both directions. Duplicates are removed from the result since it operates as a set operation.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: list output: "[1, 2, 5, 6]"

---

filter: '{{ [1, 2, 3, 4] | symmetric\_difference([3, 4, 5, 6]) }}' type: list output: "[1, 2, 5, 6]"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
symmetric_difference(
 value: list,
 other: list,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The first list. Must be a list or collection. required: true type: list other: description: > The second list. Must be a list or collection. required: true type: list

## Good to know

---

- Duplicates within either input list are collapsed: the result treats both inputs as sets.
- The order of items in the result is not guaranteed, so do not rely on a specific ordering.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find devices exclusive to each group

Determine which entities are in one group but not the other, and vice versa.

#### template

```
template: |
 {% set morning = ["light.kitchen", "light.hallway", "light.bedroom"] %}
 {% set evening = ["light.hallway", "light.bedroom",
 "light.porch", "light.living_room"] %}
 {{ symmetric_difference(morning, evening) }}
type: list
output: '["light.kitchen", "light.porch", "light.living_room"]'
```

### Detect changes in a list

Compare a previous and current list to find items that were added or removed.

#### template

```
template: |
 {% set previous = ["sensor.temp_a", "sensor.temp_b", "sensor.temp_c"] %}
 {% set current = ["sensor.temp_b", "sensor.temp_c", "sensor.temp_d"] %}
 {{ symmetric_difference(previous, current) }}
type: list
output: '["sensor.temp_a", "sensor.temp_d"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Tan

---

The `tan` template function returns the tangent of a value given in radians. It wraps Python's `math.tan`, so the input is expected to be an angle measured in radians rather than degrees.

This can be useful in calculations involving slopes, gradients, or angles derived from sensor data. For example, you might use it to calculate a projected shadow length from a solar elevation angle.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "1.0"

---

filter: '' type: float output: "1.0"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
tan(
 value: Any,
 default: Any = None,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The angle in radians to calculate the tangent of. Must be numeric. required: true type: float default: description: > Value to return if the calculation fails (for example, if the input is not numeric). If not provided, an error is raised instead. required: false type: any

## Using a default value

---

If the input value might not be numeric, provide a default to avoid errors. This keeps your template from breaking when a sensor is temporarily unavailable.

### template

```
template: |
 {{ tan(states("sensor.slope_angle"), default=0) }}
type: float
output: "0"
```

## Good to know

---

- The input is expected in radians, not degrees. Multiply degree values by `pi / 180` before passing them in.
- Tangent is undefined at 90, 270 degrees (and so on). Inputs near these angles return very large numbers due to floating-point behavior.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Tangent of a known angle

Calculate the tangent of 45 degrees by first converting to radians.

#### template

```
template: |
 {{ tan(45 * (pi / 180)) }}
type: float
output: "1.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Tau

---

The `tau` template constant provides the mathematical constant tau, which equals  $2 * \pi$ , approximately 6.28318. Tau represents one full turn in radians, making some circular and angular calculations more intuitive.

This is useful when working with full rotations or cyclic values. For example, instead of writing  $2 * \pi$  to represent a full circle, you can use `tau`. This can make templates involving angular sensor data or periodic calculations cleaner and easier to read.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: float output: "6.283185307179586"
```

## Good to know

---

- This is a mathematical constant approximately equal to 6.28318, the same as `2 * pi`.
- Use it without parentheses. `tau` is a value, not a function you call.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Fraction of a full rotation

Calculate one quarter of a full rotation (90 degrees in radians).

#### template

```
template: |
 {{ tau / 4 }}
type: float
output: "1.5707963267948966"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Time Since

---

The `time_since` template function returns a human-readable string describing how much time has elapsed since a given datetime. Give it a datetime in the past, and it returns something like "2 hours" or "3 days and 5 hours" instead of a raw number of seconds.

This is useful whenever you want to display elapsed time in a natural, readable format. For example, showing "last seen 45 minutes ago" on a dashboard, displaying how long a door has been open, or reporting how long ago a sensor last updated. The function only works with datetimes in the past. For future datetimes, use `time_until` instead. You can control the level of detail with the optional precision parameter, which determines how many time components (years, months, days, hours, minutes, seconds) to include.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: " type: string output: "2 hours"

---

filter: " type: string output: "2 hours"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
time_since(
 value: datetime,
 precision: int = 1,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > A datetime object representing a point in the past. The function calculates the time elapsed between this datetime and now. required: true type: datetime precision: description: > The number of time components to include in the output, from 1 to 6. A precision of 1 might return "2 hours", while a precision of 2 might return "2 hours and 30 minutes". Higher precision gives more detail. required: false default: "1" type: integer

## Controlling precision

---

By default, `time_since` returns only the largest time component. Increase the precision to include more detail.

### template

```
template: '{{ time_since(states.binary_sensor.front_door.last_changed, 1) }}'
type: string
output: "2 hours"
```

### template

```
template: '{{ time_since(states.binary_sensor.front_door.last_changed, 3) }}'
type: string
output: "2 hours, 30 minutes and 15 seconds"
```

## Good to know

---

- Only works with datetimes in the past. For future datetimes, use `time_until` instead.
- Durations shorter than one second return `"0 seconds"`.
- The input datetime must be time zone aware. Pass something like `states.sensor.foo.last_changed` or `now()`, not a naive datetime.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Show how long a door has been open

Display the elapsed time since a door opened on your dashboard.

#### template

```
template: |
 {{
 "Open for "
 ~ time_since(states.binary_sensor.front_door.last_changed)
 }}
type: string
output: "Open for 2 hours"
```

### Detailed elapsed time in a notification

Send a notification with a more detailed breakdown of elapsed time.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 The washing machine has been running for
 {{ time_since(states.sensor.washer_start.last_changed, 2) }}.
```

### Show when something was last seen

Create a "last seen" display for a person tracker.

#### template

```
template: |
 {{
 "Last seen "
 ~ time_since(states.person.paulus.last_changed) ~ " ago"
 }}
type: string
output: "Last seen 45 minutes ago"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Time Until

---

The `time_until` template function returns a human-readable string describing how much time remains until a given datetime in the future. Give it a future datetime, and it returns something like "3 hours" or "2 days and 6 hours" instead of a raw number of seconds.

This is useful whenever you want to display a countdown or remaining time in a natural format. For example, showing how long until the next alarm goes off, displaying "arrives in 45 minutes" for a delivery, or reporting how much time is left on a timer. The function only works with datetimes in the future. For past datetimes, use `time_since` instead. You can control the level of detail with the optional precision parameter, which determines how many time components (years, months, days, hours, minutes, seconds) to include.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: " type: string output: "6 hours"

---

filter: " type: string output: "6 hours"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
time_until(
 value: datetime,
 precision: int = 1,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > A datetime object representing a point in the future. The function calculates the time remaining between now and this datetime. required: true type: datetime precision: description: > The number of time components to include in the output, from 1 to 6. A precision of 1 might return "6 hours", while a precision of 2 might return "6 hours and 15 minutes". Higher precision gives more detail. required: false default: "1" type: integer

## Controlling precision

---

By default, `time_until` returns only the largest time component. Increase the precision to include more detail.

### template

```
template: '{{ time_until(as_datetime(states("sensor.next_alarm")), 1) }}'
type: string
output: "6 hours"
```

### template

```
template: '{{ time_until(as_datetime(states("sensor.next_alarm")), 3) }}'
type: string
output: "6 hours, 15 minutes and 30 seconds"
```



## Good to know

---

- Only works with datetimes in the future. For past datetimes, use `time_since` instead.
- Durations shorter than one second return `"0 seconds"`.
- The input datetime must be time zone aware.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Show time until next alarm

Display a countdown to the next alarm on your dashboard.

#### template

```
template: |
 {{ "Alarm in " ~ time_until(as_datetime(states("sensor.next_alarm"))) }}
type: string
output: "Alarm in 6 hours"
```

### Countdown to a scheduled event

Show how long until a calendar event starts, with more detail.

#### template

```
template: |
 {% set event_time = as_datetime(states("sensor.next_appointment")) %}
 Starts in {{ time_until(event_time, 2) }}
type: string
output: "Starts in 2 hours and 30 minutes"
```

### Time remaining in an automation condition

Only run the rest of an automation if a scheduled event is less than 1 hour away. Combine `time_until` with a check on the remaining time using `today_at` and `now`.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{
 as_datetime(states("sensor.next_event")) - now()
 < timedelta(hours=1)
 }}
 - condition: template
 value_template: >
 {{
 as_datetime(states("sensor.next_event")) - now()
 < timedelta(hours=1)
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Timedelta

---

The `timedelta` template function creates a `timedelta` object from numeric values like days, hours, minutes, and seconds. This gives you a duration that you can add to or subtract from `datetime` objects to calculate future or past times.

Whenever you need to offset a time by a specific amount, `timedelta` is how you express that amount. For example, you might want to check if a sensor changed in the last 10 minutes, calculate what time it will be in 2 hours, or determine if an event happened more than 3 days ago. You create a `timedelta` with the desired duration and then add or subtract it from a `datetime` like `now`. If you need to parse a duration from an ISO 8601 string instead of numeric values, use `as_timedelta`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: timedelta output: "2:30:00"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
timedelta(
 days: float = 0,
 seconds: float = 0,
 microseconds: float = 0,
 milliseconds: float = 0,
 minutes: float = 0,
 hours: float = 0,
 weeks: float = 0,
) -> timedelta
```

## Function parameters

The following parameters can be provided to this function. All parameters are optional and default to 0. You can combine multiple parameters to create any duration.

{% function\_parameters %} days: description: Number of days. required: false default: "0" type: float seconds: description: Number of seconds. required: false default: "0" type: float microseconds: description: Number of microseconds. required: false default: "0" type: float milliseconds: description: Number of milliseconds. required: false default: "0" type: float minutes: description: Number of minutes. required: false default: "0" type: float hours: description: Number of hours. required: false default: "0" type: float weeks: description: Number of weeks. required: false default: "0" type: float



## Good to know

---

- All arguments are keyword-only. Call it as `timedelta(hours=2)`, not `timedelta(2)`.
- Values can be negative and can exceed their usual range. `timedelta(hours=36)` and `timedelta(days=1, hours=12)` produce the same duration.
- There is no `months` or `years` argument. Use `days=30` for an approximate month, or calculate a future date by adjusting `now()` components directly.

## Adding and subtracting durations

---

The most common use is adding or subtracting a `timedelta` from `now` to calculate a time in the past or future.

### template

```
template: '{{ now() - timedelta(hours=1) }}'
type: datetime
output: "2024-03-15 13:30:00.123456+01:00"
```

### template

```
template: '{{ now() + timedelta(days=7) }}'
type: datetime
output: "2024-03-22 14:30:00.123456+01:00"
```

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if something changed recently

Determine whether a sensor changed state in the last 10 minutes.

**template**

```
template: |
 {{
 now() - states.binary_sensor.motion.last_changed
 < timedelta(minutes=10)
 }}
type: boolean
output: "true"
```

### Check if something is overdue

Determine whether a plant was last watered more than 3 days ago.

**template**

```
template: |
 {{
 now() - states.sensor.plant_last_watered.last_changed
 > timedelta(days=3)
 }}
type: boolean
output: "false"
```

### Calculate a future time for display

Show what time it will be in 45 minutes, formatted for a notification.

**template**

```
template: '{{ (now() + timedelta(minutes=45)).strftime("%H:%M") }}'
type: string
output: "15:15"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Timestamp Custom

---

The `timestamp_custom` filter formats a UNIX timestamp into a human-readable string using a format pattern you specify. Give it a timestamp and a format string like "%H:%M" or "%Y-%m-%d", and it returns the formatted date and time.

This is useful whenever you have a UNIX timestamp (a number of seconds since January 1, 1970) and want to display it in a specific format. Many sensors expose timestamps as attributes, and external APIs often return times as UNIX timestamps. With `timestamp_custom`, you can turn those numbers into readable dates and times in exactly the format you need. By default, the timestamp is converted to your local time zone, but you can set the second parameter to `false` to keep it in UTC.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "14:30"
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | timestamp_custom(
 format_string: str = "%Y-%m-%d %H:%M:%S",
 local_time: bool = True,
 default: Any = None,
) -> str | Any
```

## Function parameters

The following parameters can be provided to this filter.

`{% function_parameters %}` `format_string`: description: > The format string describing the desired output. Uses Python strftime format codes such as `%Y` (year), `%m` (month), `%d` (day), `%H` (hour), `%M` (minute), `%S` (second). required: false default: `"%Y-%m-%d %H:%M:%S"` type: string  
`local_time`: description: > Whether to convert the timestamp to the local time zone before formatting. Set to `false` to format in UTC. required: false default: `"true"` type: boolean default:  
description: > Value to return if the formatting fails. If not provided, an error is raised on invalid input. required: false type: any

## Formatting in UTC

---

By default, `timestamp_custom` converts the timestamp to your local time zone. Pass `false` as the second argument to format in UTC instead.

### template

```
template: '{{ 1710510600 | timestamp_custom("%H:%M", false) }}'
type: string
output: "13:30"
```

## Common format patterns

---

Here are some patterns that cover the formats you need most often:

- `%Y-%m-%d` produces `2024-03-15`
- `%H:%M` produces `14:30`
- `%H:%M:%S` produces `14:30:00`
- `%d/%m/%Y` produces `15/03/2024`
- `%A, %B %d` produces `Friday, March 15`
- `%I:%M %p` produces `02:30 PM`

## Good to know

---

- The input must be a UNIX timestamp (a number of seconds). Pass a datetime object through `as_timestamp` first.
- The timestamp is converted to your Home Assistant time zone by default. Pass `false` as the second argument to format in UTC instead.
- Without a `default`, the filter raises an error for non-numeric inputs. Provide one to keep templates from breaking on unavailable sensors.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Format last changed time

Convert an entity's last changed time to a UNIX timestamp with `as_timestamp` and then format it.

#### template

```
template: |
 {{
 as_timestamp(states.sensor.temperature.last_changed)
 | timestamp_custom("%H:%M")
 }}
type: string
output: "14:30"
```

### Display a full date and time

Format a timestamp as a complete, human-readable date and time string.

#### template

```
template: '{{ as_timestamp(now()) | timestamp_custom("%A, %B %d at %H:%M") }}'
type: string
output: "Friday, March 15 at 14:30"
```

### Using a default value

If a sensor might provide an invalid timestamp, use a default to avoid errors.

#### template

```
template: |
 {{
 states("sensor.last_event") | float(0)
 | timestamp_custom("%Y-%m-%d", default="unknown")
 }}
type: string
output: "2024-03-15"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Timestamp Local

---

The `timestamp_local` filter formats a UNIX timestamp as a datetime string in your local time zone. It returns the date and time in the standard ISO-like format, automatically converting from UTC to the time zone configured in Home Assistant.

This is a quick way to turn a UNIX timestamp into a readable local date and time without needing to specify a format string. If you need a custom format, use `timestamp_custom` instead. If you want the UTC representation, use `timestamp_utc`. This filter is particularly useful for displaying sensor attributes or API responses that provide times as UNIX timestamps.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "2024-03-15 14:30:00+01:00"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | timestamp_local(
 default: Any = None,
) -> str | Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} default: description: > Value to return if the formatting fails. If not provided, an error is raised on invalid input. required: false type: any

## Using a default value

---

If the input might not be a valid timestamp, provide a default to avoid errors.

### template

```
template: '{{ "invalid" | float(0) | timestamp_local(default="unknown") }}'
type: string
output: "unknown"
```

## Good to know

---

- The input must be a UNIX timestamp (a number of seconds). Pass a datetime object through `as_timestamp` first.
- The result is a string, not a datetime object. To do datetime math, use `as_local` instead.
- Without a `default`, the filter raises an error for non-numeric inputs.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a sensor's last changed time

Convert the last changed timestamp of an entity to a readable local time string.

#### template

```
template: |
 {{ as_timestamp(states.sensor.temperature.last_changed) | timestamp_local }}
type: string
output: "2024-03-15 14:30:00+01:00"
```

### Show the current time as a formatted string

Convert the current UNIX timestamp to a local datetime string.

#### template

```
template: '{{ as_timestamp(now()) | timestamp_local }}'
type: string
output: "2024-03-15 14:30:00+01:00"
```

### Use in a notification

Include a formatted local timestamp in a notification message.

#### action

```
action: |
 action:
 - action: notify.mobile
 data:
 message: >
 Motion detected at
 {{
 as_timestamp(states.binary_sensor.motion.last_changed)
 | timestamp_local
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Timestamp Utc

---

The `timestamp_utc` filter formats a UNIX timestamp as a datetime string in UTC (Coordinated Universal Time). It returns the date and time in the standard ISO-like format, without any local time zone conversion.

This is useful when you need to display or log times in UTC, share timestamps with external services that expect UTC, or compare times across different time zones without ambiguity. If you want the local time representation instead, use `timestamp_local`. If you need a custom format, use `timestamp_custom` with the `local_time` parameter set to `false`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '' type: string output: "2024-03-15 13:30:00+00:00"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | timestamp_utc(
 default: Any = None,
) -> str | Any
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} default: description: > Value to return if the formatting fails. If not provided, an error is raised on invalid input. required: false type: any

## Using a default value

---

If the input might not be a valid timestamp, provide a default to avoid errors.

### template

```
template: '{{ "invalid" | float(0) | timestamp_utc(default="unknown") }}'
type: string
output: "unknown"
```

## Good to know

---

- The input must be a UNIX timestamp (a number of seconds). Pass a datetime object through `as_timestamp` first.
- The result is a string, not a datetime object.
- Without a `default`, the filter raises an error for non-numeric inputs.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a sensor timestamp in UTC

Convert the last changed timestamp of an entity to a UTC datetime string.

#### template

```
template: |
 {{ as_timestamp(states.sensor.temperature.last_changed) | timestamp_utc }}
type: string
output: "2024-03-15 13:30:00+00:00"
```

### Compare local and UTC representations

Display the same timestamp in both local and UTC format to see the difference.

#### template

```
template: |
 {% set ts = as_timestamp(now()) %}
 Local: {{ ts | timestamp_local }}
 UTC: {{ ts | timestamp_utc }}
type: string
output: |
 Local: 2024-03-15 14:30:00+01:00
 UTC: 2024-03-15 13:30:00+00:00
```

### Log an event time in UTC

Use UTC timestamps in automations for consistent logging regardless of time zone.

#### action

```
action: |
 action:
 - action: notify.log
 data:
 message: >
 Event occurred at
 {{ as_timestamp(now()) | timestamp_utc }}
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Title

---

The `title` filter converts a string to title case, where the first letter of each word is capitalized and the rest are lowered. This is useful when you want to present text in a polished, headline-like format. For example, you might want to format a room name or device name nicely for a notification or a dashboard display, turning values like `living room light` into `Living Room Light`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "living room light" | title }}' type: string output: Living Room Light
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
title(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to convert to title case. The first letter of each word is capitalized. required: true type: string

## Good to know

---

- Words are split on any non-letter character, so names like `"o'brien"` become `"O'Brien"` and values with numbers may capitalize unexpectedly.
- Every word is capitalized, including short words like "of", "the", and "in". This is not the same as English title-case conventions.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Format an entity name for display

Turn a raw entity name into a nicely formatted title for a notification.

#### template

```
template: '{{ "front door sensor" | title ~ " is active" }}'
type: string
output: "Front Door Sensor is active"
```

### Create a friendly room name

Format a room name for use in a dashboard card heading.

#### template

```
template: '{{ "guest bedroom" | title }}'
type: string
output: Guest Bedroom
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/To Json

---

The `to_json` filter converts a Python object (like a dictionary, list, string, or number) into a JSON-formatted string. This is the opposite of `from_json`, which parses a JSON string back into an object.

This is essential when you need to send structured data to external services. Many integrations like MQTT, REST, and webhooks expect JSON-formatted payloads. For example, you might build a dictionary with sensor data and convert it to JSON for an MQTT publish, or format a payload for a REST API call. The optional parameters let you control the output format: `pretty_print` adds indentation for readability, `sort_keys` orders the keys alphabetically, and `ensure_ascii` escapes non-ASCII characters.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ {"temperature": 21.5, "humidity": 45} | to\_json }}' type:  
string output: '{"temperature":21.5,"humidity":45}'

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | to_json(
 ensure_ascii: bool = False,
 pretty_print: bool = False,
 sort_keys: bool = False,
) -> str
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} ensure\_ascii: description: > If `true`, all non-ASCII characters are escaped in the output. Defaults to `false`, which allows Unicode characters to pass through unchanged. required: false default: "false" type: boolean pretty\_print: description: > If `true`, the output is formatted with indentation for readability. Useful for debugging or displaying structured data. required: false default: "false" type: boolean sort\_keys: description: > If `true`, dictionary keys are sorted alphabetically in the output. required: false default: "false" type: boolean

## Pretty-printed output

---

Use `pretty_print` to format the JSON with indentation, making it easier to read in logs or on dashboards.

### template

```
template: |
 {{
 {"name": "Living Room", "temperature": 21.5}
 | to_json(pretty_print=true)
 }}
type: string
output: |
 {
 "name": "Living Room",
 "temperature": 21.5
 }
```

## Good to know

---

- The output has no spaces between keys and values by default, which is ideal for MQTT or API payloads but less readable. Turn on `pretty_print` for debugging.
- Values that are not JSON-serializable (like datetime objects) raise an error. Convert them to strings or numbers first.
- Dictionary keys are preserved in their original order unless you set `sort_keys=true` .

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Publish sensor data via MQTT

Build a JSON payload from sensor values and publish it to an MQTT topic.

#### action

```
action: |
 action:
 - action: mqtt.publish
 data:
 topic: "home/sensors/living_room"
 payload: >
 {{
 {
 "temperature": states("sensor.living_room_temperature") | float,
 "humidity": states("sensor.living_room_humidity") | float,
 "timestamp": now() | as_timestamp | int
 } | to_json
 }}
```

### Send data to a REST API

Format a dictionary as JSON for a RESTful notification or webhook.

#### action

```
action: |
 action:
 - action: rest_command.send_data
 data:
 payload: >
 {{
 {
 "event": "door_opened",
 "entity": "binary_sensor.front_door",
 "state": states("binary_sensor.front_door")
 } | to_json
 }}
```

### Sort keys for consistent output

Use `sort_keys` when you need deterministic output, for example when comparing JSON strings.

#### template

```
template: '{{ { "z": 1, "a": 2, "m": 3 } | to_json(sort_keys=true) }}'
type: string
output: '{"a":2,"m":3,"z":1}'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Today At

---

The `today_at` template function returns a datetime object for today's date at a specific time you provide. Give it a time string like "08:00" or "14:30:00" and it returns a full datetime representing that time today, in your local time zone.

This is especially useful in automations when you need to check whether the current time is before or after a specific point in the day. For example, you might want to turn on lights only after 18:00, send a morning summary at 07:00, or check if it is still before bedtime. Instead of manually constructing a datetime from the current date and your target time, `today_at` handles it in a single call.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: datetime output: "2024-03-15 08:00:00+01:00"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
today_at(time_str: str = "00:00") -> datetime
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} time\_str: description: > A time string in "HH:MM" or "HH:MM:SS" format. If not provided, defaults to midnight ("00:00"). required: false default: "00:00" type: string

## Comparing with the current time

---

A common use is to check if the current time is before or after a specific point in the day, using `now` for comparison.

### template

```
template: '{{ now() > today_at("18:00") }}'
type: boolean
output: "false"
```



## Good to know

---

- The returned datetime is in your Home Assistant time zone, making it safe to compare directly with `now`.
- "Today" is determined when the template runs, so a template scheduled at 23:59 and compared at 00:01 will use two different dates.
- Accepts strings in `"HH:MM"` or `"HH:MM:SS"` format. Other formats raise an error.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Only run an automation in the evening

Use `today_at` in a condition to limit an automation to run only between 18:00 and 23:00.

#### automation

```
automation: |
 condition:
 - condition: template
 value_template: >
 {{ today_at("18:00") <= now() < today_at("23:00") }}
```

### Calculate minutes until a time

Find out how many minutes remain until a specific time today.

#### template

```
template: '{{ (today_at("17:00") - now()).total_seconds() / 60 | int }}'
type: integer
output: "150"
```

### Set a time-based brightness

Adjust light brightness based on how close it is to bedtime.

#### template

```
template: |
 {% set bedtime = today_at("22:00") %}
 {% set hours_left = ((bedtime - now()).total_seconds() / 3600)
 | float | round(1) %}
 {{ [(hours_left / 4) * 100] | int, 10] | max }}
type: integer
output: "75"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/ToJson

---

The `toJson` filter converts a Python object (like a dictionary, list, string, or number) into a JSON-formatted string. This is a Home Assistant override of the standard `toJson` filter. It accepts an optional `indent` parameter to pretty-print the output with indentation.

This filter is useful when you need to produce JSON output for external services or for debugging. If you need more control over the serialization (such as sorting keys or controlling ASCII escaping), consider using the `to_json` filter instead, which offers additional options.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} filter: '{{ {"temperature": 21.5, "unit": "C"} | tojson }}' type: string  
output: '{"temperature": 21.5, "unit": "C"}'

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
value | tojson(
 indent: int | None = None,
) -> str
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} indent: description: > Number of spaces to use for indentation when pretty-printing. When set to `none` (the default), the output is compact with no extra whitespace.  
required: false default: "none" type: integer



## Pretty-printed output

---

Use the `indent` parameter to format the JSON with indentation for readability.

### template

```
template: |
 {{ {"name": "Living Room", "temperature": 21.5} | toJson(indent=2) }}
title: Pretty-print with 2-space indent
type: string
output: |
 {
 "name": "Living Room",
 "temperature": 21.5
 }
```

## Good to know

---

- The default output includes a space after each comma and colon, unlike `to_json`, which produces a compact string.
- If you need sorted keys or control over ASCII escaping, use `to_json` instead.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Send a JSON payload via MQTT

Build a dictionary from sensor values and serialize it for an MQTT publish action.

#### action

```
action: |
 action:
 - action: mqtt.publish
 data:
 topic: "home/sensors"
 payload: >
 {{
 {
 "temperature": states("sensor.temperature") | float,
 "humidity": states("sensor.humidity") | float
 } | tojson
 }}
```

### Serialize a list

Convert a list of values to a JSON array string.

#### template

```
template: '{{ ["on", "off", "on"] | tojson }}'
type: string
output: '["on", "off", "on"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Trim

---

The `trim` filter removes leading and trailing whitespace from a string. You can optionally specify which characters to strip instead of whitespace. This is useful when working with sensor values or other data that may include extra spaces or unwanted characters at the beginning or end. For example, some devices report states with trailing spaces, or you may need to remove surrounding quotes or other characters before processing a value further.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ " hello world " | trim }}' type: string output: hello world
```



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
trim(
 value: str,
 chars: str | None = None,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to strip. Leading and trailing whitespace (or specified characters) will be removed. required: true type: string chars: description: > A string of characters to strip from both ends. If not provided, whitespace is stripped. required: false type: string

## Stripping specific characters

---

Pass a string of characters to remove from both ends instead of whitespace.

### template

```
template: '{{ "---hello---" | trim("-") }}'
title: Strip dashes from both ends
type: string
output: hello
```

## Good to know

---

- Only affects the beginning and end of the string. Whitespace between words is kept intact. Use `replace` for inner cleanup.
- When you pass the `chars` argument, every character in that string is treated as a candidate to strip. `trim("-_")` removes both dashes and underscores from each end.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Clean up a sensor value

Remove extra whitespace from a sensor state before using it in a comparison.

#### template

```
template: |
 {% set status = states("sensor.status") | trim %}
 {% if status == "ready" %}
 System is ready
 {% endif %}
type: string
output: "System is ready"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/True Test

---

The `true` test checks whether a value is `true`. It performs a strict identity check, meaning only the boolean value `true` passes. Values like `1`, `true`, or non-empty strings do not match.

This is useful when you need to verify that a value is specifically the boolean `true` and not merely a truthy value. In Home Assistant, entity attributes can be actual booleans, and this test lets you check for an exact `true` value without accidentally matching other truthy types.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} test: | It is true
```

type: string output: "It is true"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
true(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The value to test. Returns `true` only if the value is the boolean `true`. required: true type: any

## Good to know

---

- This is a strict identity check. Truthy values like `1`, `"yes"`, or a non-empty list do not pass. Use `boolean` if you want looser matching.
- The string `"true"` is not the boolean `true`. Entity states are always strings, so comparing a state with this test always returns `false`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Strict boolean check

Only the boolean `true` passes; truthy values like `1` or `true` do not.

#### template

```
template: |
 {{ true is true }}
 {{ 1 is true }}
 {{ "true" is true }}
type: boolean
output: |
 true
 false
 false
```

### Verify a boolean attribute

Check that an entity attribute is specifically `true`.

#### template

```
template: |
 {% set locked = state_attr("lock.front_door", "is_jammed") %}
 {% if locked is true %}
 Door is jammed!
 {% endif %}
type: string
output: "Door is jammed!"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Truncate

---

The `truncate` filter shortens a string to a specified maximum length and appends an end marker (by default `...`) if the string was truncated. By default, it tries to break at a word boundary so words are not cut in half. This is useful when you need to limit the length of text for display in notifications, dashboard cards, or other contexts with limited space. For example, a sensor that provides a long description or error message can be truncated to a manageable length so it does not overflow your display.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "This is a very long string that should be truncated" | truncate(20) }}' type: string output: "This is a very..."
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
truncate(
 value: str,
 length: int = 255,
 killwords: bool = False,
 end: str = "...",
 leeway: int = 5,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to truncate. required: true type: string  
length: description: > The maximum length of the output string, including the end marker. Defaults to `255`. required: false default: "255" type: integer  
killwords: description: > If `true`, words may be cut in half at the exact length. If `false` (the default), the string is truncated at the last word boundary before the length limit. required: false default: "false" type: boolean  
end: description: > The string to append when the text is truncated. Defaults to `...`. required: false default: "..."  
leeway: description: > If the string is only this many characters longer than the length, it is not truncated. Defaults to `5`. required: false default: "5" type: integer

## Cutting at exact length

---

Set `killwords` to `true` to truncate at the exact character position, even if it is in the middle of a word.

### template

```
template: '{{ "Hello beautiful world" | truncate(12, true) }}'
title: Cut at exact length
type: string
output: "Hello bea..."
```

## Custom end marker

---

Use a different end marker instead of the default ellipsis.

### template

```
template: |
 {{ "A very long status message here" | truncate(20, end=" [more]") }}
title: Custom end marker
type: string
output: "A very long [more]"
```

## Good to know

---

- The `length` limit includes the end marker. Truncating to 20 characters with the default `...` gives you 17 characters of text plus the three-dot ending.
- Strings within `leeway` characters of the limit are returned unchanged. Set `leeway=0` if you want a strict cutoff every time.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Shorten a notification message

Truncate a long sensor message so it fits in a mobile notification.

#### template

```
template: '{{ states("sensor.error_log") | truncate(80) }}'
type: string
output: |
 Connection timed out while attempting to reach the remote server.
 Please check...
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Tuple

---

The `tuple` template function converts a collection (like a list) into a tuple. A tuple is similar to a list but is immutable, meaning its contents cannot be changed after creation. This can be useful when you need a hashable, fixed sequence of values.

Tuples are useful in specific situations where you need an immutable sequence. For example, some template operations or comparisons work better with tuples, and tuples can be used as dictionary keys because they are hashable (unlike lists). In most day-to-day Home Assistant templates, lists work fine, but `tuple` is available when you need this specific data type.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: tuple output: "(1, 2, 3)"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
tuple(
 value: list,
) -> tuple
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The collection to convert to a tuple. Can be a list, set, or any other collection. required: true type: list

## Good to know

---

- Tuples are immutable. Once created, you cannot append to or modify them. Use a list if you need to change items later.
- Most Home Assistant template work does not need tuples. Reach for them when you need a hashable value to use as a dictionary key or set member.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Convert a list of coordinates to a tuple

Create a fixed coordinate pair from a list.

#### template

```
template: '{{ tuple([52.5, 13.4]) }}'
type: tuple
output: "(52.5, 13.4)"
```

### Use a tuple for comparison

Tuples can be compared directly, which is useful for checking if a set of values matches an expected combination.

#### template

```
template: '{{ tuple([1, 2, 3]) == tuple([1, 2, 3]) }}'
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.



## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Typeof

---

The `typeof` template function returns the type name of a value as a string. It tells you what kind of data you are working with, such as `str`, `int`, `float`, `list`, `dict`, or `bool`. This is the Python class name of the value.

This is primarily useful for debugging templates. When a template does not behave as expected, it is often because a value is a different type than you assumed. For example, you might think a sensor value is a number, but it is actually a string. Or an attribute you expected to be a list is actually `None`. Using `typeof` lets you inspect the actual types at runtime so you can figure out what is going on and write the correct conversion or comparison.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: string output: "int"

---

filter: '' type: string output: "str"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
typeof(
 value: Any,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value whose type name to return. Returns the Python class name as a string. required: true type: any

## Common type names

---

Here are the type names you will encounter most often:

- `str` for text (for example, `hello` or the result of `states("...")` )
- `int` for whole numbers (for example, `42` or `states("...") | int` )
- `float` for decimal numbers (for example, `21.5` or `states("...") | float` )
- `bool` for `true` or `false`
- `list` for a list of values (for example, `[1, 2, 3]` )
- `dict` for a mapping (for example, `{"a": 1}` )
- `NoneType` for `None`

## Good to know

---

- Results are Python class names like `str`, `int`, or `NoneType`. They are not Jinja or YAML type names.
- All entity states are `str`, regardless of what they look like. A temperature sensor returning `21.5` still reports as `str` until you convert it.
- Intended mainly for debugging. In production templates, prefer type tests like `is number` or conversion functions like `float`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Debug a sensor state type

Check what type a state value is. Since all entity states are strings, this confirms the need for type conversion before math.

#### template

```
template: |
 State type: {{ states("sensor.temperature") | typeof }}
 After float: {{ (states("sensor.temperature") | float) | typeof }}
type: string
output: |
 State type: str
 After float: float
```

### Check the type of an attribute

Entity attributes can be various types. Use `typeof` to inspect what you are working with.

#### template

```
template: '{{ state_attr("climate.living_room", "hvac_modes") | typeof }}'
type: string
output: "list"
```

### Conditional logic based on type

Handle different types differently in a template.

#### template

```
template: |
 {% set val = state_attr("sensor.data", "reading") %}
 {% if val | typeof == "list" %}
 Got {{ val | length }} items
 {% else %}
 Single value: {{ val }}
 {% endif %}
type: string
output: "Got 3 items"
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Undefined

---

The `undefined` test checks whether a variable has not been defined in the current template context. It returns `true` if the variable does not exist and `false` if it has been assigned a value. It is the opposite of `defined`.

This is useful when you want to explicitly detect missing variables and handle the absence case. For instance, you might want to set a default value or skip a section of the template when a particular variable has not been provided. While `default` is often more concise for fallback values, `undefined` gives you full control with `{% jinja %}{% endjinja %}` blocks.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} test: |

```
Variable is missing
```

type: string output: "Variable is missing"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
undefined(
 value: Any,
) -> bool
```

## Function parameters

The following parameters can be provided to this test.

{% function\_parameters %} value: description: > The variable to test. Returns `true` if the variable is not defined in the current context. required: true type: any

## Good to know

---

- A variable explicitly set to `none` is still defined, so this test returns `false` for it. Use `is none` to check for that case.
- Useful for template macros or scripts where callers may or may not pass an argument.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Provide fallback logic for missing data

When an optional variable is not provided, supply alternative content.

#### template

```
template: |
 {% if custom_greeting is undefined %}
 Welcome home!
 {% else %}
 {{ custom_greeting }}
 {% endif %}
type: string
output: "Welcome home!"
```

### Skip optional sections

Only render a section when the required data is available.

#### template

```
template: |
 Status report:
 {% if details is undefined %}
 No additional details available.
 {% else %}
 {{ details }}
 {% endif %}
type: string
output: |
 Status report:
 No additional details available.
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Union

---

The `union` template function combines two lists and returns all unique items from both. This is the set union operation: every item that appears in either list is included once in the result, with duplicates removed.

This is useful when you want to merge two collections without getting duplicate entries. For example, you might want to combine entities from two different groups into one list, merge detected objects from multiple sensors, or build a complete list of all devices across multiple rooms. Since it operates as a set operation, each item appears only once in the result regardless of how many times it appeared in the input lists.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: list output: "[1, 2, 3, 4, 5]"

---

filter: '{{ [1, 2, 3] | union([3, 4, 5]) }}' type: list output: "[1, 2, 3, 4, 5]"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
union(
 value: list,
 other: list,
) -> list
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The first list. Must be a list or collection. required: true type: list other: description: > The second list. Must be a list or collection. required: true type: list

## Good to know

---

- Duplicates within either input list are removed in the result.
- The order of items is not guaranteed, so do not rely on a specific ordering.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Combine entities from multiple groups

Build a complete list of all unique entities across two groups.

#### template

```
template: |
 {% set group_a = expand("group.downstairs_lights")
 | map(attribute="entity_id") | list %}
 {% set group_b = expand("group.upstairs_lights")
 | map(attribute="entity_id") | list %}
 {{ union(group_a, group_b) }}
type: list
output: '["light.kitchen", "light.hallway", "light.bedroom", "light.bathroom"]'
```

### Merge detected objects from multiple cameras

Combine detection lists to get a complete picture of what has been detected across all cameras.

#### template

```
template: |
 {% set cam1 = state_attr("sensor.camera_1", "detected_objects") %}
 {% set cam2 = state_attr("sensor.camera_2", "detected_objects") %}
 {{ union(cam1, cam2) }}
type: list
output: '["person", "car", "dog", "cat"]'
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Unique

---

The `unique` filter removes duplicate values from a list, keeping only the first occurrence of each value. Unlike converting to a `set`, the `unique` filter preserves the original order of items. You can also deduplicate based on a specific attribute of each item.

This is useful when you have a list that may contain duplicates and you want to remove them while keeping the order intact. For example, you might collect states from multiple entities and want to know the distinct states that exist, or you might merge multiple lists of devices and need to eliminate duplicates. The `attribute` parameter is especially helpful when working with lists of objects, letting you deduplicate by a specific property.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ [1, 2, 2, 3, 1, 4] | unique | list }}' type: list output: "[1, 2, 3, 4]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
unique(
 value: list,
 case_sensitive: bool = False,
 attribute: str | None = None,
) -> list
```

## Function parameters

The following parameters can be provided to this filter.

{% function\_parameters %} value: description: > The list to remove duplicates from. required: true type: list case\_sensitive: description: > If `true`, "On" and "on" are treated as different values. Defaults to `false`. required: false default: "false" type: boolean attribute: description: > Deduplicate based on this attribute of each item. Only the first item with each unique attribute value is kept. required: false type: string

## Unique by attribute

---

Deduplicate a list of objects based on a specific attribute, keeping only the first object with each unique attribute value.

### template

```
template: |
 {{
 expand("group.all_sensors")
 | unique(attribute="state")
 | map(attribute="entity_id")
 | list
 }}
title: One sensor per unique state value
type: list
output: '["sensor.kitchen_temp", "sensor.bedroom_temp"]'
```

## Good to know

---

- Unlike Python's standard behavior, this filter is case-insensitive by default. Pass `case_sensitive=True` if you want `"On"` and `"on"` treated as different values.
- Returns a generator, so pipe the result through `list` if you need to loop over it more than once or display it directly.
- The first occurrence of each value wins, preserving the original list order.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.



## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Find distinct states across entities

Get the unique state values from a group of entities.

#### template

```
template: |
 {{
 expand("group.all_lights")
 | map(attribute="state")
 | unique
 | list
 }}
type: list
output: '["on", "off"]'
```

### Deduplicate a merged list

When combining entities from multiple sources, remove any that appear more than once.

#### template

```
template: |
 {% set list1 = ["light.kitchen", "light.bedroom", "light.hall"] %}
 {% set list2 = ["light.bedroom", "light.porch", "light.kitchen"] %}
 {{ (list1 + list2) | unique | list }}
type: list
output: '["light.kitchen", "light.bedroom", "light.hall", "light.porch"]'
```

### Case-sensitive deduplication

By default, `unique` is case-insensitive. Use `case_sensitive=true` to treat different cases as distinct values.

#### template

```
template: |
 {{ ["On", "on", "OFF", "off"] | unique(case_sensitive=true) | list }}
type: list
output: '["On", "on", "OFF", "off"]'
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Unpack

---

The `unpack` template function converts bytes into a native Python value using a struct format string. This is the reverse of `pack` and works like Python's `struct.unpack_from` function, extracting a single value from a byte sequence according to the specified format.

This function is primarily useful for IoT and low-level device communication. Many IoT devices send data as raw bytes that need to be interpreted according to a specific format. For example, a Bluetooth sensor might send temperature as a 16-bit big-endian integer, or an MQTT-connected microcontroller might report values in a packed binary format. The optional `offset` parameter lets you skip bytes at the beginning of the data, which is useful when the value you need is not at the start of the byte sequence. The format strings follow [Python's struct module syntax](#).

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: " type: integer output: "1234"

---

filter: " type: integer output: "1234"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
unpack(
 value: bytes,
 format_string: str,
 offset: int = 0,
) -> Any | None
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The bytes object to unpack. Must contain enough bytes for the specified format string. required: true type: string format\_string: description: > A Python struct format string that describes how to interpret the bytes. Common formats include `>H` (big-endian unsigned short), `<i` (little-endian signed int), `>f` (big-endian float). See the [Python struct documentation](#) for all options. required: true type: string offset: description: > The number of bytes to skip from the beginning of the data before unpacking. Defaults to `0` (start at the beginning). required: false default: "0" type: integer

## Using an offset

---

When the value you need is embedded within a larger byte sequence, use the `offset` parameter to skip to the right position.

### template

```
template: '{{ b"\x00\x00\x04\xd2" | unpack(">H", offset=2) }}'
type: integer
output: "1234"
```

## Good to know

---

- Only unpacks a single value. If your format string describes multiple fields, only the first is returned. Call `unpack` multiple times with different offsets when you need all of them.
- The byte sequence must contain enough bytes for the format string starting at `offset`, otherwise the function returns `None`.
- Format strings follow [Python's struct module syntax](#), where `<` and `>` set little-endian or big-endian byte order.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Decode a BLE sensor reading

Decode a temperature value from a Bluetooth Low Energy sensor that sends data as base64-encoded big-endian bytes.

#### template

```
template: |
 {% set raw = states("sensor.ble_raw") | base64_decode(encoding=None) %}
 {{ raw | unpack(">h") / 100 }}
type: float
output: "21.5"
```

### Decode a hex-encoded sensor value

Combine `from_hex` with `unpack` to decode a hex string containing a packed integer.

#### template

```
template: '{{ "04d2" | from_hex | unpack(">H") }}'
type: integer
output: "1234"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Upper

---

The `upper` filter converts all characters in a string to uppercase. This is useful when you need to display text in all capitals for emphasis, or when normalizing text for comparison purposes. For example, you might want to display a status message in uppercase on a dashboard, or convert a value to uppercase before comparing it against a known constant.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "hello world" | upper }}' type: string output: HELLO
WORLD
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
upper(
 value: str,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to convert to uppercase. required: true  
type: string

## Good to know

---

- Non-letter characters like digits and punctuation pass through unchanged.
- For case-insensitive comparisons, convert both sides with the same filter ( `upper` or `lower` ) rather than mixing them.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Display a status label in uppercase

Show a sensor state in all capitals for a prominent display.

#### template

```
template: '{{ states("sensor.alarm_status") | upper }}'
type: string
output: ARMED
```

### Normalize text for comparison

Convert to uppercase to ensure a case-insensitive match.

#### template

```
template: |
 {% if states("sensor.mode") | upper == "AUTO" %}
 System is in automatic mode
 {% endif %}
type: string
output: "System is in automatic mode"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Urlencode

---

The `urlencode` template function converts a dictionary of key-value pairs into a URL-encoded query string. This is Home Assistant's override of the default `urlencode` filter, specifically designed to handle dictionaries for building HTTP request parameters.

This is useful when constructing URLs for REST commands, webhooks, or API calls from within your automations and scripts. For example, you might need to send data to an external service with properly encoded parameters, or build a query string for a webhook URL. The function ensures that special characters in your values are properly escaped so they are transmitted correctly.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '{{ urlencode({"name": "Living Room", "value": "23.5"}) }}' type: string output: name=Living+Room&value=23.5
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
urlencode(
 value: dict,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > A dictionary of key-value pairs to encode into a URL query string format. Keys and values are joined with `=` and pairs are separated with `&`.  
required: true type: map

## Good to know

---

- Spaces in values are encoded as `+`, not `%20`. Most servers accept both, but if you need `%20` specifically, replace it afterwards.
- Only accepts a dictionary. To encode a single string (for example, a URL path segment), handle it manually or use `slugify` where appropriate.



## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build a webhook URL with parameters

Construct a full URL with encoded query parameters for calling an external webhook.

#### template

```
template: |
 {{
 "https://example.com/webhook?" ~ urlencode({
 "token": "abc123",
 "message": "Front door opened",
 "room": "Living Room"
 })
 }}
type: string
output: "https://example.com/webhook?token=abc123&message=Front+door+opened&room=Living+Room"
```

### Send sensor data to an external service

Encode sensor values into a query string for use in a REST command.

#### action

```
action: |
 action:
 - action: rest_command.send_data
 data:
 payload: >
 {{
 urlencode({
 "temperature": states("sensor.temperature"),
 "humidity": states("sensor.humidity"),
 "location": "home"
 })
 }}
 }}
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Urlize

---

The `urlize` filter finds plain text URLs in a string and converts them into HTML anchor ( `<a>` ) tags. It recognizes URLs starting with `http://` , `https://` , and `www.` . This is useful when you have text that contains URLs and you want to turn them into links in an HTML context, such as a Markdown card or an HTML notification. For example, a sensor might report a URL as plain text, and you can use this filter to turn it into a real link.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "Visit https://www.home-assistant.io for more info" |
urlize }}' type: string output: 'Visit https://www.home-assistant.io for more info'
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
urlize(
 value: str,
 trim_url_limit: int | None = None,
 nofollow: bool = False,
 target: str | None = None,
 rel: str | None = None,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string in which to find and convert URLs into HTML links. required: true type: string trim\_url\_limit: description: > If set, the displayed URL text is shortened to this many characters. The link itself is not affected. required: false type: integer nofollow: description: > If `true`, adds `rel="nofollow"` to the generated links. Defaults to `false`. required: false default: "false" type: boolean target: description: > Sets the `target` attribute on the generated links (for example, `_blank` to open in a new tab). required: false type: string rel: description: > Sets a custom `rel` attribute on the generated links. required: false type: string

## Shortening long URLs

---

Use `trim_url_limit` to shorten the displayed URL text while keeping the full link.

### template

```
template: |
 {{
 "See https://www.example.com/docs/templating/"
 | urlize(trim_url_limit=30)
 }}
title: Shorten displayed URL text
type: string
output: |-
 See <a href="https://www.example.com/docs/templating/"
 rel="noopener">https://www.example.com/docs...
```



## Good to know

---

- Only URLs starting with `http://`, `https://`, or `www.` are recognized. Protocols like `ftp://` or bare domains are left as plain text.
- The output contains HTML. Use it in contexts that render HTML, like Markdown cards or HTML-capable notifications.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Turn a sensor URL into a link

Convert a URL reported by a sensor into a real link for a Markdown card.

#### template

```
template: '{{ states("sensor.update_url") | urlize(target="_blank") }}'
type: string
output: |-
 <a href="https://example.com/update" rel="noopener"
 target="_blank">https://example.com/update
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

`{% endif %}`

---

## Template Functions/Utcnow

---

The `utcnow` template function returns the current date and time in UTC (Coordinated Universal Time). It gives you a full datetime object representing the present moment in the UTC time zone, regardless of your local time zone setting.

When you need to compare times across time zones, store timestamps in a universal format, or perform calculations that should not be affected by daylight saving time changes, `utcnow()` is the right choice. It is also useful when interacting with external services that expect UTC timestamps. Like `now`, using `utcnow()` in a template causes it to be re-evaluated at the start of every new minute, keeping time-dependent values up to date automatically.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: " type: datetime output: "2024-03-15
13:30:00.123456+00:00"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
utcnow() -> datetime
```



## Working with the result

---

`utcnow()` returns a datetime object. You can access individual components and format the output, just like with `now`.

### template

```
template: |
 The UTC time is {{ utcnow().hour }}:{{ utcnow().minute }}
 Today is day {{ utcnow().day }} of month {{ utcnow().month }}
type: string
output: |
 The UTC time is 13:30
 Today is day 15 of month 3
```

## Formatting with strftime

You can format the date and time in any way you like using Python's `strftime` method.

### template

```
template: '{{ utcnow().strftime("%Y-%m-%dT%H:%M:%SZ") }}'
type: string
output: "2024-03-15T13:30:00Z"
```

## Good to know

---

- Templates that reference `utcnow()` are re-evaluated at the start of every minute, the same as `now`.
- The returned datetime is time zone aware and always in UTC, so adding a `timedelta` or subtracting from a local datetime works without conversion.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Calculate the UTC offset

Compare `now` with `utcnow()` to determine your current UTC offset in hours.

#### template

```
template: '{{ (now() - utcnow()).total_seconds() / 3600 | round(0) | int }}'
type: integer
output: "1"
```

### ISO 8601 formatted UTC timestamp

Create a standard ISO 8601 timestamp string, useful for logging or sending to external APIs.

#### template

```
template: '{{ utcnow().isoformat() }}'
type: string
output: "2024-03-15T13:30:00.123456+00:00"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Version

---

The `version` template function converts a string into a version object that can be compared with other version objects. Instead of comparing version strings as plain text (where `9.0` would incorrectly sort after `10.0`), version objects understand semantic versioning and compare correctly.

This is useful when you have devices or software that report their firmware or software version as a sensor value. For example, you might want to check if a device's firmware is outdated, trigger an automation when a new version is detected, or filter a list of devices by their version number. The version object handles common version formats like `1.2.3`, `2024.1.0`, and `1.0.0-beta`.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: boolean output: "true"

---

filter: '{{ "2.1.0" | version > "2.0.0" | version }}' type: boolean output: "true"



## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
version(
 value: str,
) -> AwesomeVersion
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The version string to parse. Supports common formats like `1.2.3`, `2024.1.0`, or `1.0-beta`. required: true type: string

## Comparing versions

---

Version objects support all comparison operators. This is much safer than comparing version strings directly, because string comparison would consider `9.0` greater than `10.0`.

### template

```
template: |
 {{ version("10.0") > version("9.0") }}
title: Correct version comparison
type: boolean
output: "true"
```

### template

```
template: |
 {{ "10.0" > "9.0" }}
title: Incorrect string comparison
type: boolean
output: "false"
```

## Good to know

---

- Comparing two version objects returns a boolean. Comparing a version object with a plain string raises an error, so convert both sides first.
- Version parsing is lenient. Unusual formats may parse in ways you do not expect, so test your comparisons against real values.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check if firmware is up to date

Compare a device's current firmware version against the minimum required version.

#### template

```
template: |
 {% set current = states("sensor.device_firmware") | version %}
 {% set required = version("3.2.0") %}
 {% if current >= required %}
 Firmware is up to date
 {% else %}
 Firmware update required
 {% endif %}
type: string
output: "Firmware is up to date"
```

### Notify on version change

Detect whether a software version has increased, which could indicate an update was installed.

#### template

```
template: |
 {{ version("2024.4.0") > version("2024.3.1") }}
title: Check for newer version
type: boolean
output: "true"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Wordcount

---

The `wordcount` filter counts the number of words in a string, splitting on whitespace. This is useful when you need to measure the length of text content. For example, you might want to check if a sensor message is too long before sending a notification, or display a word count for text entered through an input helper.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "The quick brown fox jumps" | wordcount }}' type: integer
output: "5"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
wordcount(
 value: str,
) -> int
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string in which to count the words. Words are split on whitespace. required: true type: string

## Good to know

---

- Words are anything separated by whitespace. Punctuation stays attached to its neighboring word and does not add to the count.
- An empty string returns `0`.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Check message length before sending

Only send a notification if the message is not too long.

#### template

```
template: |
 {% set msg = states("sensor.latest_message") %}
 {% if msg | wordcount <= 50 %}
 {{ msg }}
 {% else %}
 Message too long to display
 {% endif %}
type: string
output: "Front door was opened at 14:30"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Template Functions/Wordwrap

---

The `wordwrap` filter inserts line breaks into a string so that each line is no longer than the specified width. By default, it wraps at word boundaries so words are not split. This is useful when you need to format long text for display in contexts that do not automatically wrap, such as fixed-width notification templates or text-based displays. For example, you might wrap a long status message to a specific width for a text-only notification.



## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ "The quick brown fox jumps over the lazy dog" | wordwrap(20) }}' type: string output: "The quick brown fox\njumps over the lazy\ndog"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
wordwrap(
 value: str,
 width: int = 79,
 break_long_words: bool = True,
 break_on_hyphens: bool = True,
 wrapstring: str | None = None,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The string to wrap. required: true type: string  
width: description: > The maximum number of characters per line. Defaults to `79`. required: false  
default: "79" type: integer break\_long\_words: description: > If `true`, words longer than the width  
will be broken. If `false`, long words are kept intact and may exceed the width. Defaults to `true`.  
required: false default: "true" type: boolean break\_on\_hyphens: description: > If `true`, wrapping  
can occur at hyphens in compound words. Defaults to `true`. required: false default: "true" type:  
boolean wrapstring: description: > The string to use for line breaks. Defaults to a newline  
character. required: false type: string

## Using a custom wrap string

---

Use an HTML `<br>` tag instead of a newline for wrapping in HTML contexts.

### template

```
template: |
 {{
 "A long message that needs wrapping for HTML display"
 | wordwrap(25, wrapstring="
")
 }}
title: Wrap with HTML line breaks
type: string
output: "A long message that needs
wrapping for HTML
display"
```

## Good to know

---

- Existing newlines in the input are preserved and treated as paragraph breaks.
- With `break_long_words` set to `false` , words longer than the width stay intact on their own line, potentially exceeding the target width.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Format a notification for a text display

Wrap a message so it fits within a fixed-width display.

#### template

```
template: |
 {{ states("sensor.daily_summary") | wordwrap(40) }}
type: string
output: |-
 Today was sunny with a high of 28
 degrees. Expected to cool down
 overnight.
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Wrap

---

The `wrap` template function wraps a value cyclically within a given range. When the value goes past the maximum, it wraps back around to the minimum, and vice versa. The minimum is inclusive and the maximum is exclusive, so the result is always in the range  $[\text{min}, \text{max})$ .

This is useful for working with cyclic quantities like angles, hours of the day, or any value that should loop around rather than be clamped. For example, you could wrap a calculated heading to stay within 0-360 degrees, or keep an hour-of-day offset within a 24-hour cycle.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

{% template\_function\_usage %} function: '' type: float output: "10.0"

---

filter: '' type: float output: "10.0"

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
wrap(
 value: Any,
 min_value: Any,
 max_value: Any,
) -> float
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > The value to wrap. Must be numeric. required: true type: float min\_value: description: > The minimum of the range (inclusive). Must be numeric. required: true type: float max\_value: description: > The maximum of the range (exclusive). Must be numeric and greater than min\_value. required: true type: float

## Good to know

---

- The result is always a float, even when all inputs are integers.
- The range is half-open: `min_value` is included, `max_value` is not. Wrapping a value equal to `max_value` returns `min_value`.
- Unlike `clamp`, out-of-range values cycle around instead of being pinned to the nearest edge.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Wrapping a compass heading

Keep a calculated compass heading within the standard 0-360 degree range.

#### template

```
template: |
 {% set heading = states("sensor.wind_direction") | float + 45 %}
 {{ wrap(heading, 0, 360) }}
type: float
output: "35.0"
```

### Wrapping negative values

Values below the minimum wrap back from the maximum end.

#### template

```
template: |
 {{ wrap(-10, 0, 360) }}
type: float
output: "350.0"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Xmlattr

---

The `xmlattr` filter converts a dictionary into a string of XML/HTML attributes. Each key-value pair becomes an attribute in `key="value"` format, with values automatically escaped. Keys with `None` or `undefined` values are omitted. This is useful when dynamically building HTML elements with variable attributes. For example, you might want to generate an HTML tag with attributes that come from sensor data or template variables. By default, a leading space is added before the attributes so you can place the result directly after a tag name.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} filter: '{{ {"class": "alert", "id": "msg1"} | xmlattr }}' type: string
output: ' class="alert" id="msg1"'
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
xmlattr(
 value: dict,
 autospace: bool = True,
) -> str
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} value: description: > A dictionary of attribute names and values to convert into an HTML/XML attribute string. Keys with `None` values are omitted. required: true type: map autospace: description: > If `true`, a leading space is added before the first attribute. This lets you append the result directly after a tag name without worrying about spacing. Defaults to `true`. required: false default: "true" type: boolean

## Good to know

---

- Keys whose value is `None` or undefined are skipped, so you can build attribute dictionaries conditionally without extra template logic.
- Values are HTML-escaped automatically, making the result safe to drop directly into a tag.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build an HTML element with dynamic attributes

Generate an HTML tag with attributes set from template variables.

#### template

```
template: |
 {% set attrs = {"style": "color: red", "title": "Warning"} %}
 <span{{ attrs | xmlattr }}>Alert
type: string
output: 'Alert'
```

### Create a link with dynamic attributes

Build an anchor tag with attributes from a dictionary.

#### template

```
template: |
 {% set link_attrs = {"href": "https://www.home-assistant.io",
 "target": "_blank"} %}
 <a{{ link_attrs | xmlattr }}>Home Assistant
type: string
output: |-
 Home Assistant
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---



## Template Functions/Zip

---

The `zip` template function combines two or more lists (or other collections) element by element, producing a list of tuples where each tuple contains one item from each input list. It works just like Python's built-in `zip()` function. If the lists are different lengths, the result is truncated to the length of the shortest list.

This is useful when you have related data in separate lists that you need to pair up. For example, you might have a list of room names and a list of temperature readings, and you want to combine them so each room is paired with its temperature. Or you might want to iterate over two lists in parallel to build a formatted output.

## Usage

---

Here's how to use this template function. Copy any example and adjust it to your setup.

```
{% template_function_usage %} function: '' type: list output: "[('a', 1), ('b', 2), ('c', 3)]"
```

## Function signature

---

The signature is a technical summary of this template function. It shows the name of the function, the values (called parameters) it accepts, and what type of data each parameter expects (for example, a piece of text or a number).

Function parameters that have a `=` with a value after them are optional. If you leave them out, the default value shown is used automatically. Function parameters without a default are required.

```
zip(
 *iterables: Iterable,
) -> list[tuple]
```

## Function parameters

The following parameters can be provided to this function.

{% function\_parameters %} iterables: description: > Two or more iterables to zip together. Each iterable contributes one element per tuple in the result. required: true type: list

## Pairing related data

---

When you have data split across multiple lists, `zip` brings them together.

### template

```
template: |
 {% set rooms = ["Living room", "Bedroom", "Kitchen"] %}
 {% set temps = [21.5, 19.8, 22.3] %}
 {% for room, temp in zip(rooms, temps) %}
 {{ room }}: {{ temp }}°C
 {% endfor %}
title: Pair rooms with temperatures
type: string
output: |
 Living room: 21.5°C
 Bedroom: 19.8°C
 Kitchen: 22.3°C
```

## Unequal length lists

---

When lists have different lengths, `zip` stops at the shortest one.

### template

```
template: '{{ zip([1, 2, 3], ["a", "b"]) | list }}'
title: Shorter list wins
type: list
output: "[(1, 'a'), (2, 'b')]"
```

## Good to know

---

- Returns an iterator. Pipe the result through `list` if you need to reuse it or display it directly.
- Input lists of different lengths are silently truncated to the shortest. Extra items from the longer lists are dropped without warning.

## Try it yourself

---

Ready to test this? Open **Developer tools** > **Template**, paste the example into the **Template editor**, and watch the result update on the right. Edit the values to see how the function adapts to your own entities.

## More examples

---

Real scenarios where this function comes up in automations and templates. Copy any example and adapt it to your setup.

### Build a formatted summary

Combine entity names with their states to create a summary string.

#### template

```
template: |
 {% set names = ["Front door", "Back door", "Garage"] %}
 {% set statuses = ["closed", "open", "closed"] %}
 {% for name, status in zip(names, statuses) %}
 {{ name }} is {{ status }}
 {% endfor %}
type: string
output: |
 Front door is closed
 Back door is open
 Garage is closed
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with your template and expected result, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain or fix templates when you describe what you want in plain language.

## Related template functions

---

These functions work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Co2 Changed

---

The **Carbon dioxide level changed** trigger fires after the carbon dioxide (CO2) reading on one or more air quality sensors changes by a meaningful amount. Carbon dioxide builds up naturally in occupied rooms from breathing, cooking, and heating. A bedroom with the door closed overnight or a packed meeting room after lunch are classic spots where CO2 creeps up without anyone noticing. Rising CO2 is one of the clearest signs that a space needs fresh air.

Imagine your bedroom ventilation fan spinning up automatically in the middle of the night because CO2 climbed while you slept, so you wake up feeling refreshed instead of groggy. Use this trigger to automate ventilation, log indoor air quality trends, or remind household members to open a window when CO2 shifts noticeably.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your CO2 sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Carbon dioxide level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the carbon dioxide level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.co2_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.co2_changed
 target:
 entity_id: sensor.bedroom_co2
 options:
 threshold: 50
```

This fires whenever the bedroom CO2 sensor reading changes by at least 50 ppm.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the carbon dioxide level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- Indoor CO2 levels typically range from about 400 ppm (well-ventilated) to over 1,000 ppm (stuffy room). A threshold of 50 to 100 ppm works well for most ventilation automations.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when CO2 crosses a specific level in one direction, use [Carbon dioxide level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: ventilate when bedroom CO2 rises

Sleeping in a closed bedroom builds up CO2 overnight, leaving you groggy in the morning. This automation turns on the bedroom ventilation fan whenever the CO2 level shifts significantly, keeping the air fresh so you wake up feeling rested.

- **Trigger:** Carbon dioxide level changed
- **Target:** Bedroom CO2 sensor
- **Threshold type:** 100
- **Action:** Turn on fan

#### YAML example for CO2-based bedroom ventilation

**\*\*automation\*\***

```
automation: |
 alias: "Ventilate bedroom on CO2 change"
 triggers:
 - trigger: air_quality.co2_changed
 target:
 entity_id: sensor.bedroom_co2
 options:
 threshold: 100
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.bedroom_ventilation
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Co2 Crossed Threshold

---

The **Carbon dioxide level crossed threshold** trigger fires when a carbon dioxide (CO2) reading on one or more air quality sensors crosses a specific level. CO2 builds up naturally in occupied rooms from breathing, cooking, and heating. Once levels climb above 1,000 ppm, a room feels stuffy, concentration drops, and it is time to let in fresh air.

Picture your bedroom ventilation fan switching on automatically the moment CO2 crosses 1,000 ppm while you sleep, keeping the air fresh without you lifting a finger. Or getting a gentle reminder on your phone to crack a window when the living room gets stuffy during a gathering. This trigger lets your home respond to stale air the instant it becomes a problem.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Carbon dioxide level crossed threshold**.
6. Under **Threshold type**, set the carbon dioxide level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The carbon dioxide level (in ppm) the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.co2_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.co2_crossed_threshold
 target:
 entity_id: sensor.bedroom_co2
 options:
 threshold: 1000
 behavior: any
```

This fires whenever the bedroom CO2 sensor crosses 1,000 ppm in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The carbon dioxide level (in ppm) the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current CO2 level is above or below your threshold.
- Indoor CO2 typically ranges from about 400 ppm (well-ventilated) to over 1,000 ppm (stuffy room). A threshold around 800 to 1,000 ppm works well for most ventilation automations.
- Pair this trigger with [Carbon dioxide level changed](#) if you also want to track smaller fluctuations between crossings.



## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: keep the bedroom fresh while you sleep

Stuffy air makes for restless nights. This automation turns on the bedroom ventilation fan when CO2 crosses 1,000 ppm, so you wake up feeling refreshed instead of groggy.

- **Trigger:** Carbon dioxide level crossed threshold
- **Target:** Bedroom CO2 sensor
- **Threshold type:** 1000
- **Trigger when:** Each
- **Action:** Turn on fan

#### YAML example for CO2-based bedroom ventilation

**\*\*automation\*\***

```
automation: |
 alias: "Ventilate bedroom on high CO2"
 triggers:
 - trigger: air_quality.co2_crossed_threshold
 target:
 entity_id: sensor.bedroom_co2
 options:
 threshold: 1000
 behavior: any
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.bedroom_ventilation
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Co Changed

---

The **Carbon monoxide level changed** trigger fires after the carbon monoxide (CO) reading on one or more air quality sensors changes by a meaningful amount. Carbon monoxide is a colorless, odorless gas produced by incomplete combustion of fuels like gas, oil, wood, and charcoal. A faulty furnace, a blocked chimney, or a car left running in an attached garage all release CO without any visible warning. Even small increases in CO levels indoors deserve your attention, because prolonged exposure is dangerous.

Imagine getting an instant phone alert the moment your garage CO sensor picks up a shift, giving you time to ventilate before the situation becomes serious. Use this trigger to kick off a ventilation routine, send a safety notification, or log concentration changes whenever your CO sensor reports a significant shift.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Carbon monoxide level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the carbon monoxide level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value. required: true

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.co_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.co_changed
 target:
 entity_id: sensor.living_room_co
 options:
 threshold: 5
```

This fires whenever the living room CO sensor reading changes by at least 5 ppm.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the carbon monoxide level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.



## Good to know

---

- Carbon monoxide is produced by gas stoves, fireplaces, furnaces, and running vehicles in attached garages. A sudden rise in CO is a safety concern.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when CO crosses a specific level in one direction, use [Carbon monoxide level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: alert when CO rises in the garage

Maybe someone left the car idling or the workshop heater is acting up. This automation sends a notification to your phone whenever the carbon monoxide level in the garage shifts noticeably, so you know to open the garage door or investigate the source right away.

- **Trigger:** Carbon monoxide level changed
- **Target:** Garage CO sensor
- **Threshold type:** 10
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a garage CO alert

**\*\*automation\*\***

```
automation: |
 alias: "Alert on garage CO change"
 triggers:
 - trigger: air_quality.co_changed
 target:
 entity_id: sensor.garage_co
 options:
 threshold: 10
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Carbon monoxide change"
 message: "CO level in the garage changed significantly. Check ventila
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Co Cleared

---

The **Carbon monoxide cleared** trigger fires after a carbon monoxide sensor entity stops detecting carbon monoxide, confirming that the air in your home is safe again. After the urgency of a CO alarm, knowing exactly when the danger has passed brings real peace of mind. Use this trigger to silence a siren, send an all-clear notification to everyone in the household, or restore your home to its normal state so you and your family feel safe resuming everyday life.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your CO sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Carbon monoxide cleared**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, set how long the sensor must stay in the cleared state before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor clears, **First** to fire only when the first sensor in a group clears, or **All** to fire only after every targeted sensor has cleared. For at least: description: How long the sensor must stay in the cleared state before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.co_cleared`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.co_cleared
 target:
 entity_id: binary_sensor.hallway_co
```

This fires every time `binary_sensor.hallway_co` transitions to the cleared state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}



## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a sensor transitions from a known, valid state. If a sensor comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Use the **For at least** option to confirm the air has truly cleared before taking action. A delay of ten or fifteen minutes helps avoid premature all-clear alerts.
- To react to the opposite transition, use `Carbon monoxide detected`.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: silence the siren and let everyone know it is safe

After a carbon monoxide alarm, a blaring siren and anxious waiting are the last things your family needs once the danger is over. This automation waits until every CO sensor in the house has been clear for fifteen minutes, then silences the siren and sends a reassuring all-clear notification to your phone. No more wondering whether it is truly safe to go back to sleep or return home.

- **Trigger:** Carbon monoxide cleared
- **Target:** All CO sensors (by label)
- **Trigger when:** All
- **For at least:** 00:15:00
- **Action:** Siren: Turn off
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for silencing the siren after CO clears

```
automation
```

```
automation: |
 alias: "Silence siren after CO clears"
 triggers:
 - trigger: air_quality.co_cleared
 target:
 label_id: co_sensors
 options:
 behavior: last
 for: "00:15:00"
 actions:
 - action: siren.turn_off
 target:
 entity_id: siren.home_alarm
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "All CO sensors are clear."
 title: "CO all-clear"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Co Crossed Threshold

---

The **Carbon monoxide level crossed threshold** trigger fires when a carbon monoxide (CO) reading on one or more air quality sensors crosses a specific level. Carbon monoxide is a colorless, odorless gas produced by incomplete combustion of fuels, and it is life-threatening at elevated concentrations. Most residential CO alarms activate around 35 to 70 ppm, but you deserve to know the moment levels start climbing, not just when an alarm goes off.

Imagine getting an urgent alert on your phone the second CO reaches a dangerous level in the garage, even in the middle of the night. Or having your exhaust fan kick on automatically when a sensor detects rising CO while you are away. This trigger gives you that early warning, so your home protects your family before a situation becomes critical.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Carbon monoxide level crossed threshold**.
6. Under **Threshold type**, set the carbon monoxide level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The carbon monoxide level (in ppm) the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.co_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.co_crossed_threshold
 target:
 entity_id: sensor.living_room_co
 options:
 threshold: 35
 behavior: any
```

This fires whenever the living room CO sensor crosses 35 ppm in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The carbon monoxide level (in ppm) the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a numeric condition that checks whether the current CO level is above or below your threshold.
- Most residential CO alarms activate around 35 to 70 ppm depending on exposure time. A threshold in that range is a good starting point for safety automations.
- Pair this trigger with [Carbon monoxide level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: send a safety alert when garage CO gets dangerous

Nobody wants to discover a CO problem too late. This automation sends an urgent notification straight to your phone the moment the garage sensor crosses 35 ppm, giving you time to open the door and check for running engines or faulty appliances.

- **Trigger:** Carbon monoxide level crossed threshold
- **Target:** Garage CO sensor
- **Threshold type:** 35
- **Trigger when:** Each
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a garage CO threshold alert

**\*\*automation\*\***

```
automation: |
 alias: "Garage CO threshold alert"
 triggers:
 - trigger: air_quality.co_crossed_threshold
 target:
 entity_id: sensor.garage_co
 options:
 threshold: 35
 behavior: any
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Carbon monoxide warning"
 message: >
 CO in the garage crossed 35 ppm.
 Open the garage door and check
 for running engines or appliances.
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Air Quality.Co Detected

---

The **Carbon monoxide detected** trigger fires the moment a carbon monoxide sensor entity starts detecting carbon monoxide. Carbon monoxide is colorless and odorless, which makes it one of the most dangerous household hazards because you simply cannot sense it on your own. This trigger gives Home Assistant the ability to warn you immediately, whether your family is sleeping, the kids are playing downstairs, or you are away at work. Pair it with a loud siren and an urgent phone notification for the strongest possible safety net.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your CO sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Carbon monoxide detected**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, set how long the sensor must stay in the detected state before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor detects carbon monoxide, **First** to fire only when the first sensor in a group detects carbon monoxide, or **All** to fire only after every targeted sensor detects carbon monoxide. For at least: description: How long the sensor must stay in the detected state before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.co_detected`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.co_detected
 target:
 entity_id: binary_sensor.hallway_co
```

This fires every time `binary_sensor.hallway_co` transitions to the detected state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a sensor transitions from a known, valid state. If a sensor comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Carbon monoxide is odorless and colorless, making automated alerts especially important. Pair this trigger with a loud notification or siren action for maximum safety.
- To react to the opposite transition, use [Carbon monoxide cleared](#).

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: sound the alarm and alert the whole household

Imagine everyone in your home is fast asleep and carbon monoxide starts building up from a faulty furnace. This automation triggers every siren in the house and sends an urgent notification to your phone the instant any sensor picks up carbon monoxide. Those extra seconds of warning protect the people who matter most to you.

- **Trigger:** Carbon monoxide detected
- **Target:** All CO sensors (by label)
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Siren: Turn on
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a carbon monoxide alarm

```
automation
```

```

automation: |
 alias: "CO alarm and notification"
 triggers:
 - trigger: air_quality.co_detected
 target:
 label_id: co_sensors
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: siren.turn_on
 target:
 entity_id: siren.home_alarm
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "Carbon monoxide detected!"
 title: "CO alert"

```

## Automation: ventilate the garage automatically when CO builds up

A car left idling or a gas-powered tool running in the garage produces carbon monoxide that builds up fast in an enclosed space. This automation turns on the exhaust fan after a confirmed one-minute reading, helping clear the air before the situation becomes dangerous. You could also combine this with a notification so you know to check on what caused the buildup.

- **Trigger:** Carbon monoxide detected
- **Target:** Garage CO sensor
- **Trigger when:** Each
- **For at least:** 00:01:00
- **Action:** Fan: Turn on

### YAML example for ventilation on CO detection

**\*\*automation\*\***

```

automation: |
 alias: "Garage ventilation on CO detection"
 triggers:
 - trigger: air_quality.co_detected
 target:
 entity_id: binary_sensor.garage_co
 options:
 behavior: any
 for: "00:01:00"
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.garage_exhaust

```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Gas Cleared

---

The **Gas cleared** trigger fires after a gas sensor entity stops detecting gas, letting you know the danger has passed and the air is safe again. After the stress of a gas alert, there is real comfort in getting a clear, automatic confirmation that everything is back to normal. Use this trigger to send an all-clear notification, re-open a gas valve that was shut off during the alarm, or restore your home to its everyday state.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your gas sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Gas cleared**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, set how long the sensor must stay in the cleared state before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor clears, **First** to fire only when the first sensor in a group clears, or **All** to fire only after every targeted sensor has cleared. For at least: description: How long the sensor must stay in the cleared state before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.gas_cleared`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.gas_cleared
 target:
 entity_id: binary_sensor.kitchen_gas
```

This fires every time `binary_sensor.kitchen_gas` transitions to the cleared state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a sensor transitions from a known, valid state. If a sensor comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Use the **For at least** option to make sure the gas has truly cleared. Setting a delay of five or ten minutes confirms the reading stays clear before you take action.
- To react to the opposite transition, use `Gas detected`.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: get peace of mind with an all-clear notification

After a gas alarm, the last thing you want is to keep wondering whether the situation is truly resolved. This automation waits until every gas sensor in the house has been clear for at least ten minutes, then sends a reassuring notification to your phone. No more checking the sensors yourself or second-guessing whether it is safe to go back inside.

- **Trigger:** Gas cleared
- **Target:** All gas sensors (by label)
- **Trigger when:** All
- **For at least:** 00:10:00
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a gas all-clear notification

**\*\*automation\*\***

```
automation: |
 alias: "Gas all-clear notification"
 triggers:
 - trigger: air_quality.gas_cleared
 target:
 label_id: gas_sensors
 options:
 behavior: last
 for: "00:10:00"
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "All gas sensors are clear."
 title: "Gas all-clear"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Gas Detected

---

The **Gas detected** trigger fires the moment a gas sensor entity starts detecting gas in your home, whether it is a natural gas leak near the stove or a combustible gas buildup in the basement. A gas leak is one of those situations where every second of early warning matters. With this trigger, Home Assistant alerts you instantly so you and your family have time to react, even in the middle of the night or while you are away from home.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your gas sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Gas detected**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, set how long the sensor must stay in the detected state before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor detects gas, **First** to fire only when the first sensor in a group detects gas, or **All** to fire only after every targeted sensor detects gas. For at least: description: How long the sensor must stay in the detected state before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.gas_detected`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.gas_detected
 target:
 entity_id: binary_sensor.kitchen_gas
```

This fires every time `binary_sensor.kitchen_gas` transitions to the detected state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a sensor transitions from a known, valid state. If a sensor comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Use the **For at least** option to avoid false alarms from brief sensor spikes. Setting a short delay (like one or two minutes) helps confirm the reading is genuine.
- To react to the opposite transition, use `Gas cleared`.



## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: get an urgent phone alert the moment gas is detected in the kitchen

Imagine you are upstairs or out running errands and a burner valve is leaking in the kitchen. This automation sends an urgent notification straight to your phone the instant your kitchen gas sensor picks something up, giving you the earliest possible warning to take action.

- **Trigger:** Gas detected
- **Target:** Kitchen gas sensor
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a gas detection notification

**\*\*automation\*\***

```
automation: |
 alias: "Notify on kitchen gas detection"
 triggers:
 - trigger: air_quality.gas_detected
 target:
 entity_id: binary_sensor.kitchen_gas
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "Gas detected in the kitchen!"
 title: "Gas alert"
```

## Automation: automatically shut off the gas valve to protect your home

When a gas leak happens while everyone is asleep or nobody is home, you want the house to protect itself. This automation watches every gas sensor in the house and, after a confirmed 30-second reading, shuts off the main gas valve automatically. That extra layer of safety means you never have to rely on being awake and nearby to prevent a dangerous situation.

- **Trigger:** Gas detected
- **Target:** All gas sensors (by label)
- **Trigger when:** Each
- **For at least:** 00:00:30
- **Action:** Valve: Close

### YAML example for automatic gas shutoff

**\*\*automation\*\***

```
automation: |
 alias: "Shut gas valve on detection"
 triggers:
 - trigger: air_quality.gas_detected
 target:
 label_id: gas_sensors
 options:
 behavior: any
 for: "00:00:30"
 actions:
 - action: valve.close_valve
 target:
 entity_id: valve.gas_main
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.N2O Changed

---

The **Nitrous oxide level changed** trigger fires after the nitrous oxide (N2O) reading on one or more air quality sensors changes by a meaningful amount. Nitrous oxide is a potent greenhouse gas produced by agricultural activities, industrial processes, and combustion of fossil fuels. While it is less commonly monitored at home than other pollutants, specialized sensors track it in greenhouses, workshops near agricultural operations, and laboratory or medical settings.

Imagine your greenhouse ventilation fans spinning up automatically when N2O shifts after a round of fertilizing, keeping the growing environment healthy without an extra trip outside. Use this trigger to log environmental data, activate ventilation, or send alerts whenever your N2O sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Nitrous oxide level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the nitrous oxide level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.n2o_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.n2o_changed
 target:
 entity_id: sensor.greenhouse_n2o
 options:
 threshold: 5
```

This fires whenever the greenhouse N2O sensor reading changes by at least 5 ppb.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the nitrous oxide level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}



## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- Nitrous oxide is a potent greenhouse gas with roughly 300 times the warming potential of carbon dioxide. Monitoring it is valuable for environmental tracking.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when N<sub>2</sub>O crosses a specific concentration in one direction, use [Nitrous oxide level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: ventilate the greenhouse on N2O change

After fertilizing, N2O levels in a greenhouse tend to climb. This automation turns on the ventilation fans when N2O levels shift, keeping the growing environment healthy for your plants without an extra trip outside to check.

- **Trigger:** Nitrous oxide level changed
- **Target:** Greenhouse N2O sensor
- **Threshold type:** 5
- **Action:** Turn on fan

#### YAML example for N2O greenhouse ventilation

**\*\*automation\*\***

```
automation: |
 alias: "Ventilate greenhouse on N2O change"
 triggers:
 - trigger: air_quality.n2o_changed
 target:
 entity_id: sensor.greenhouse_n2o
 options:
 threshold: 5
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.greenhouse_ventilation
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.N2O Crossed Threshold

---

The **Nitrous oxide level crossed threshold** trigger fires when the nitrous oxide (N<sub>2</sub>O) reading on one or more air quality sensors crosses a specific level. Nitrous oxide is a potent greenhouse gas released by agricultural practices, industrial processes, and certain combustion sources. While less common in typical household monitoring, specialized sensors track N<sub>2</sub>O for environmental research, greenhouse management, and agricultural applications.

If you manage a greenhouse or monitor environmental conditions, this trigger keeps you informed without constant manual checks. Get a notification on your phone the moment N<sub>2</sub>O crosses a concerning level, or have your ventilation system respond automatically to keep conditions within a healthy range.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Nitrous oxide level crossed threshold**.
6. Under **Threshold type**, set the nitrous oxide level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The nitrous oxide concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.n2o_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.n2o_crossed_threshold
 target:
 entity_id: sensor.greenhouse_n2o
 options:
 threshold: 350
 behavior: any
```

This fires whenever the greenhouse N2O sensor crosses 350 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The nitrous oxide concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current N2O level is above or below your threshold.
- Ambient outdoor N2O levels sit around 330 ppb. If you are monitoring for environmental purposes, small threshold offsets above that baseline work well.
- Pair this trigger with [Nitrous oxide level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: stay on top of greenhouse air quality

Keeping N2O in check helps you maintain healthy growing conditions and track environmental trends. This automation sends a notification to your phone when the greenhouse reading crosses 350, so you know right away when ventilation needs attention.

- **Trigger:** Nitrous oxide level crossed threshold
- **Target:** Greenhouse N2O sensor
- **Threshold type:** 350
- **Trigger when:** Each
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for greenhouse N2O alert

**\*\*automation\*\***

```
automation: |
 alias: "Greenhouse N2O threshold alert"
 triggers:
 - trigger: air_quality.n2o_crossed_threshold
 target:
 entity_id: sensor.greenhouse_n2o
 options:
 threshold: 350
 behavior: any
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "N2O threshold crossed"
 message: >
 Nitrous oxide in the greenhouse
 crossed 350. Check ventilation.
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.No2 Changed

---

The **Nitrogen dioxide level changed** trigger fires after the nitrogen dioxide (NO<sub>2</sub>) reading on one or more air quality sensors changes by a meaningful amount. Nitrogen dioxide is a reddish-brown gas with a sharp odor, produced mainly by traffic, power plants, and gas stoves. It irritates the airways and contributes to smog and acid rain. Indoors, your gas stove is often the biggest source. Every time you fire up a burner, NO<sub>2</sub> levels in the kitchen rise.

Imagine your range hood turning on automatically when you start cooking, clearing combustion byproducts before they spread through the house. Use this trigger to start ventilation, close windows facing a busy road, or send a health alert whenever your NO<sub>2</sub> sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Nitrogen dioxide level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the nitrogen dioxide level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.no2_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.no2_changed
 target:
 entity_id: sensor.street_side_no2
 options:
 threshold: 10
```

This fires whenever the street-side NO2 sensor reading changes by at least 10 ppb.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the nitrogen dioxide level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- Indoor gas stoves and heaters are common sources of NO<sub>2</sub> inside the home. A kitchen NO<sub>2</sub> sensor combined with this trigger helps you automate range hood ventilation.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when NO<sub>2</sub> crosses a specific concentration in one direction, use [Nitrogen dioxide level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn on the range hood when cooking produces NO2

Every time you fire up the gas stove, NO2 levels in the kitchen rise. This automation turns on the range hood as soon as the kitchen NO2 sensor detects a shift, clearing combustion byproducts before they spread through the house.

- **Trigger:** Nitrogen dioxide level changed
- **Target:** Kitchen NO2 sensor
- **Threshold type:** 15
- **Action:** Turn on switch

#### YAML example for NO2-driven range hood

**\*\*automation\*\***

```
automation: |
 alias: "Range hood on NO2 change"
 triggers:
 - trigger: air_quality.no2_changed
 target:
 entity_id: sensor.kitchen_no2
 options:
 threshold: 15
 actions:
 - action: switch.turn_on
 target:
 entity_id: switch.range_hood
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Air Quality.No2 Crossed Threshold

---

The **Nitrogen dioxide level crossed threshold** trigger fires when the nitrogen dioxide (NO<sub>2</sub>) reading on one or more air quality sensors crosses a specific level. Nitrogen dioxide is a reddish-brown gas with a sharp odor, produced primarily by vehicle traffic and gas appliances. Elevated NO<sub>2</sub> irritates the airways and worsens respiratory conditions like asthma, so the WHO recommends keeping short-term exposure below 25 micrograms per cubic meter.

If you live near a busy road, this trigger is a game-changer. Have your ventilation system close its fresh-air intake automatically when street-side NO<sub>2</sub> rises past your limit, keeping traffic pollution out of the house. Or get an alert on your phone during rush hour so you know to keep the kids inside until levels drop. Your home watches the air for you and reacts the instant conditions change.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Nitrogen dioxide level crossed threshold**.
6. Under **Threshold type**, set the nitrogen dioxide level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The nitrogen dioxide concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.no2_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.no2_crossed_threshold
 target:
 entity_id: sensor.street_side_no2
 options:
 threshold: 40
 behavior: any
```

This fires whenever the street-side NO2 sensor crosses 40 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The nitrogen dioxide concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current NO2 level is above or below your threshold.
- The WHO guideline for short-term NO2 exposure is 25. If you live near a busy road, a threshold in that range helps protect indoor air quality.
- Pair this trigger with [Nitrogen dioxide level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: block traffic pollution during rush hour

Living near a busy road means NO<sub>2</sub> spikes every rush hour. This automation closes the fresh-air intake on your ventilation system the moment your street-side sensor crosses 40, keeping traffic fumes out of the house while you go about your evening.

- **Trigger:** Nitrogen dioxide level crossed threshold
- **Target:** Street-side NO<sub>2</sub> sensor
- **Threshold type:** 40
- **Trigger when:** Each
- **Condition:** NO<sub>2</sub> is above 40
- **Action:** Close cover (fresh-air intake)

#### YAML example for closing intake on high NO<sub>2</sub>

**\*\*automation\*\***

```
automation: |
 alias: "Close intake on high NO2"
 triggers:
 - trigger: air_quality.no2_crossed_threshold
 target:
 entity_id: sensor.street_side_no2
 options:
 threshold: 40
 behavior: any
 conditions:
 - condition: numeric_state
 entity_id: sensor.street_side_no2
 above: 40
 actions:
 - action: cover.close_cover
 target:
 entity_id: cover.fresh_air_intake
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.



## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.No Changed

---

The **Nitrogen monoxide level changed** trigger fires after the nitrogen monoxide (NO) reading on one or more air quality sensors changes by a meaningful amount. Nitrogen monoxide is a reactive gas produced mainly by vehicle engines and combustion processes. It quickly converts to nitrogen dioxide in the atmosphere and plays a key role in smog formation. If you live near a busy road or intersection, rush-hour traffic sends NO levels climbing, and that pollution easily seeps indoors through open windows.

Imagine your home automatically logging pollution patterns near the driveway so you know which hours to keep the windows shut. Use this trigger to track pollution data, activate ventilation, or send alerts whenever your NO sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Nitrogen monoxide level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the nitrogen monoxide level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.no_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.no_changed
 target:
 entity_id: sensor.driveway_no
 options:
 threshold: 10
```

This fires whenever the driveway NO sensor reading changes by at least 10 ppb.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the nitrogen monoxide level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- Nitrogen monoxide levels spike near busy roads and during rush hour. Monitoring helps you time ventilation to avoid peak traffic pollution.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when NO crosses a specific concentration in one direction, use [Nitrogen monoxide level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: log roadside pollution changes

Rush-hour traffic sends NO levels spiking near the road, and knowing the pattern helps you decide when to open or close the windows. This automation records a log entry whenever nitrogen monoxide levels near the driveway shift, helping you spot pollution trends over time.

- **Trigger:** Nitrogen monoxide level changed
- **Target:** Driveway NO sensor
- **Threshold type:** 10
- **Action:** Log entry via logbook

#### YAML example for NO pollution logging

**\*\*automation\*\***

```
automation: |
 alias: "Log NO level changes"
 triggers:
 - trigger: air_quality.no_changed
 target:
 entity_id: sensor.driveway_no
 options:
 threshold: 10
 actions:
 - action: logbook.log
 data:
 name: "Air quality"
 message: "Nitrogen monoxide level near the driveway changed."
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.No Crossed Threshold

---

The **Nitrogen monoxide level crossed threshold** trigger fires when the nitrogen monoxide (NO) reading on one or more air quality sensors crosses a specific level. Nitrogen monoxide is produced by combustion engines, gas stoves, and industrial processes. It quickly converts to nitrogen dioxide (NO<sub>2</sub>) in the presence of oxygen, meaning a rising NO reading is often an early warning of broader air quality issues.

Think about your garage after starting a car on a cold morning. This trigger lets you turn on the exhaust fan the second NO crosses a safe limit, clearing combustion fumes before they drift into the rest of the house. You also get a notification on your phone so you know ventilation is running, giving you confidence that the air indoors stays healthy even when engines or gas appliances are in use.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Nitrogen monoxide level crossed threshold**.
6. Under **Threshold type**, set the nitrogen monoxide level the reading must cross for the trigger to fire.
7. Under **Trigger when**, pick **Each** to fire every time a targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The nitrogen monoxide concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.no_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.no_crossed_threshold
 target:
 entity_id: sensor.garage_no
 options:
 threshold: 50
 behavior: any
```

This fires whenever the garage NO sensor crosses 50 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The nitrogen monoxide concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current NO level is above or below your threshold.
- Nitrogen monoxide quickly converts to nitrogen dioxide (NO<sub>2</sub>) in the presence of oxygen. Monitoring both pollutants together gives a more complete picture of combustion-related air quality.
- Pair this trigger with [Nitrogen monoxide level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: clear the garage after starting a car

Cold starts produce a burst of combustion fumes that linger in an enclosed garage. This automation turns on the exhaust fan the moment NO crosses 50, pulling those fumes out before they seep into the house through a connecting door.

- **Trigger:** Nitrogen monoxide level crossed threshold
- **Target:** Garage NO sensor
- **Threshold type:** 50
- **Trigger when:** Each
- **Action:** Turn on fan (garage exhaust)

#### YAML example for garage NO ventilation

**\*\*automation\*\***

```
automation: |
 alias: "Garage exhaust on high NO"
 triggers:
 - trigger: air_quality.no_crossed_threshold
 target:
 entity_id: sensor.garage_no
 options:
 threshold: 50
 behavior: any
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.garage_exhaust
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Ozone Changed

---

The **Ozone level changed** trigger fires after the ozone (O<sub>3</sub>) reading on one or more air quality sensors changes by a meaningful amount. Ground-level ozone forms when sunlight reacts with pollutants from vehicles and industry. It is the primary ingredient in smog and tends to peak on hot, sunny afternoons. If you enjoy opening the windows on a warm day, ozone monitoring helps you decide whether the outdoor air is actually better than what's already inside.

Imagine your HVAC switching to recirculation mode on a scorching summer afternoon because ozone levels shifted, keeping your family comfortable without pulling smoggy air indoors. Use this trigger to close windows, pause outdoor ventilation, or send a health reminder whenever ozone readings shift noticeably.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Ozone level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the ozone level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.ozone_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.ozone_changed
 target:
 entity_id: sensor.backyard_ozone
 options:
 threshold: 10
```

This fires whenever the backyard ozone sensor reading changes by at least 10 ppb.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the ozone level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- Ozone levels are highest on warm, sunny days with little wind. Monitoring helps you decide when to keep windows closed and rely on indoor filtration.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when ozone crosses a specific concentration in one direction, use [Ozone level crossed threshold](#) instead.



## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: pause fresh-air intake on ozone change

On a scorching summer afternoon, ground-level ozone peaks just when you want fresh air the most. This automation switches the HVAC system to recirculation mode when the outdoor ozone reading shifts significantly, so you stop pulling smoggy air indoors.

- **Trigger:** Ozone level changed
- **Target:** Outdoor ozone sensor
- **Threshold type:** 10
- **Action:** Select HVAC preset

#### YAML example for ozone-driven HVAC recirculation

**\*\*automation\*\***

```
automation: |
 alias: "Recirculate on ozone change"
 triggers:
 - trigger: air_quality.ozone_changed
 target:
 entity_id: sensor.backyard_ozone
 options:
 threshold: 10
 actions:
 - action: climate.set_preset_mode
 target:
 entity_id: climate.hvac
 data:
 preset_mode: "recirculate"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Ozone Crossed Threshold

---

The **Ozone level crossed threshold** trigger fires when the ozone (O3) reading on one or more air quality sensors crosses a specific level. Ground-level ozone forms when sunlight reacts with pollutants from vehicles and industry, and it tends to peak on hot, sunny afternoons. High ozone levels irritate the lungs and are especially risky during outdoor exercise.

Imagine getting a notification before your afternoon run telling you ozone is too high to exercise outside today. Or having your ventilation system close its fresh-air intake automatically when ozone spikes, so your indoor air stays clean without you thinking about it. This trigger watches the sky for you and lets your home take action the instant conditions become unhealthy.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Ozone level crossed threshold**.
6. Under **Threshold type**, set the ozone level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The ozone concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.ozone_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.ozone_crossed_threshold
 target:
 entity_id: sensor.backyard_ozone
 options:
 threshold: 100
 behavior: any
```

This fires whenever the backyard ozone sensor crosses 100 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The ozone concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.



## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current ozone level is above or below your threshold.
- Ground-level ozone peaks on hot, sunny afternoons. Automations that close fresh-air intakes or notify household members before outdoor activities are common use cases.
- Pair this trigger with [Ozone level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: remind yourself to exercise indoors on high-ozone days

On hot afternoons, ground-level ozone spikes without any visible sign. This automation sends a notification when your backyard ozone sensor crosses 100, giving you a friendly heads-up to take your workout inside.

- **Trigger:** Ozone level crossed threshold
- **Target:** Backyard ozone sensor
- **Threshold type:** 100
- **Trigger when:** Each
- **Condition:** Ozone is above 100
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for ozone exercise advisory

```
automation
```

```
automation: |
 alias: "Ozone exercise advisory"
 triggers:
 - trigger: air_quality.ozone_crossed_threshold
 target:
 entity_id: sensor.backyard_ozone
 options:
 threshold: 100
 behavior: any
 conditions:
 - condition: numeric_state
 entity_id: sensor.backyard_ozone
 above: 100
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "High ozone outside"
 message: >
 Ozone crossed 100. Consider
 exercising indoors today.
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm10 Changed

---

The **PM10 level changed** trigger fires after the PM10 (particulate matter 10 micrometers or smaller) reading on one or more air quality sensors changes by a meaningful amount. PM10 includes dust, pollen, mold spores, and other coarse particles that are stirred up by wind, traffic, construction, and household activities like vacuuming or sweeping. Spring pollen, a windy day, or a renovation project next door all send PM10 readings climbing.

Imagine your robot vacuum automatically heading out for a cleanup once the dust from a nearby construction site settles, without you having to remember. Use this trigger to start an air purifier, close windows, or send a reminder whenever PM10 readings shift noticeably.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM10 level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the PM10 level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm10_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm10_changed
 target:
 entity_id: sensor.hallway_pm10
 options:
 threshold: 20
```

This fires whenever the hallway PM10 sensor reading changes by at least 20 micrograms per cubic meter.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the PM10 level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- PM10 levels tend to spike during spring pollen season, construction work nearby, or windy days. A threshold of 10 to 25  $\mu\text{g}/\text{m}^3$  works well for most home automations.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when PM10 crosses a specific concentration in one direction, use [PM10 level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: run the robot vacuum after dust settles

A construction project nearby or a windy spring day sends dust everywhere. This automation dispatches the robot vacuum when PM10 levels in the hallway shift by a large amount, cleaning up the dust so you don't have to think about it.

- **Trigger:** PM10 level changed
- **Target:** Hallway PM10 sensor
- **Threshold type:** 25
- **Action:** Start vacuum

#### YAML example for PM10-triggered vacuuming

**\*\*automation\*\***

```
automation: |
 alias: "Vacuum on PM10 change"
 triggers:
 - trigger: air_quality.pm10_changed
 target:
 entity_id: sensor.hallway_pm10
 options:
 threshold: 25
 actions:
 - action: vacuum.start
 target:
 entity_id: vacuum.robot
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm10 Crossed Threshold

---

The **PM10 level crossed threshold** trigger fires when the PM10 (particulate matter 10 micrometers or smaller) reading on one or more air quality sensors crosses a specific level. PM10 includes coarser particles like dust, pollen, and mold spores that irritate the nose, throat, and airways. Levels tend to spike during construction work, dry windy days, and seasonal pollen peaks.

Get a heads-up on your phone the moment outdoor PM10 crosses 50, so you know to keep the windows shut on a high-pollen day. Or have your smart windows close automatically when a dust storm rolls in. This trigger is especially helpful during allergy season, letting your home shield you from airborne irritants before they become a problem.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM10 level crossed threshold**.
6. Under **Threshold type**, set the PM10 level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The PM10 concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm10_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm10_crossed_threshold
 target:
 entity_id: sensor.patio_pm10
 options:
 threshold: 50
 behavior: any
```

This fires whenever the patio PM10 sensor crosses 50 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The PM10 concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current PM10 level is above or below your threshold.
- The WHO guideline for 24-hour average PM10 exposure is 45. A threshold between 45 and 100 is a reasonable starting point depending on your local environment.
- Pair this trigger with [PM10 level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: get a pollen season heads-up on your phone

Allergy season is tough enough without guessing whether the air outside is safe. This automation sends a notification to your phone when outdoor PM10 crosses 50, so you know to keep the windows shut and stay comfortable indoors.

- **Trigger:** PM10 level crossed threshold
- **Target:** Patio PM10 sensor
- **Threshold type:** 50
- **Trigger when:** Each
- **Condition:** PM10 is above 50
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for PM10 pollen season alert

```
automation
```

```
automation: |
 alias: "PM10 pollen season alert"
 triggers:
 - trigger: air_quality.pm10_crossed_threshold
 target:
 entity_id: sensor.patio_pm10
 options:
 threshold: 50
 behavior: any
 conditions:
 - condition: numeric_state
 entity_id: sensor.patio_pm10
 above: 50
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "High PM10 outside"
 message: >
 Outdoor PM10 crossed 50.
 Consider keeping windows closed.
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.



## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm1 Changed

---

The **PM1 level changed** trigger fires after the PM1 (particulate matter 1 micrometer or smaller) reading on one or more air quality sensors changes by a meaningful amount. PM1 particles are ultrafine and originate from combustion, vehicle exhaust, and industrial emissions. Because of their tiny size, they penetrate deep into the lungs and bloodstream. Cooking on a gas stove, nearby traffic, or a wildfire miles away all push PM1 levels up inside your home.

Imagine your nursery air purifier ramping up the moment particle levels shift, keeping the air as clean as possible for little ones without you doing anything. Use this trigger to start an air purifier, adjust HVAC filtration, or log particle count changes when your PM1 sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM1 level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the PM1 level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pml_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pml_changed
 target:
 entity_id: sensor.living_room_pml
 options:
 threshold: 5
```

This fires whenever the living room PM1 sensor reading changes by at least 5 micrograms per cubic meter.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the PM1 level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- PM1 particles are the smallest commonly measured particulate matter. They are especially relevant for respiratory health.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when PM1 crosses a specific concentration in one direction, use [PM1 level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: boost the air purifier when PM1 shifts

Ultrafine particles from cooking or traffic drift into every room, and tiny lungs are the most vulnerable. This automation increases your air purifier speed whenever PM1 levels in the nursery shift noticeably, keeping the air as clean as possible for little ones.

- **Trigger:** PM1 level changed
- **Target:** Nursery PM1 sensor
- **Threshold type:** 5
- **Action:** Fan: Set speed

#### YAML example for PM1-driven air purifier boost

**\*\*automation\*\***

```
automation: |
 alias: "Boost purifier on PM1 change"
 triggers:
 - trigger: air_quality.pm1_changed
 target:
 entity_id: sensor.nursery_pm1
 options:
 threshold: 5
 actions:
 - action: fan.set_percentage
 target:
 entity_id: fan.nursery_air_purifier
 data:
 percentage: 80
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm1 Crossed Threshold

---

The **PM1 level crossed threshold** trigger fires when the PM1 (particulate matter 1 micrometer or smaller) reading on one or more air quality sensors crosses a specific level. PM1 particles are ultrafine, small enough to pass deep into the lungs and even enter the bloodstream. Everyday activities like burning candles, cooking, and using a fireplace produce these tiny particles.

With this trigger, your air purifier switches on the second PM1 crosses your chosen limit, clearing the air before you even notice a difference. You also get the option to send a notification to your phone when candle smoke or cooking pushes ultrafine particles past a safe level, giving you peace of mind that the air your family breathes is being watched around the clock.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM1 level crossed threshold**.
6. Under **Threshold type**, set the PM1 level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The PM1 concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm1_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm1_crossed_threshold
 target:
 entity_id: sensor.living_room_pm1
 options:
 threshold: 25
 behavior: any
```

This fires whenever the living room PM1 sensor crosses 25 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The PM1 concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current PM1 level is above or below your threshold.
- PM1 is one of the finest fractions of particulate matter. If your sensor also reports PM2.5 or PM10, you get a fuller picture of air quality by combining triggers for each size.
- Pair this trigger with [PM1 level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: start the air purifier when ultrafine particles rise

Candles, cooking, and fireplaces all release ultrafine particles that are invisible but still matter. This automation starts the living room air purifier the moment PM1 crosses 25, clearing the air before you notice a thing.

- **Trigger:** PM1 level crossed threshold
- **Target:** Living room PM1 sensor
- **Threshold type:** 25
- **Trigger when:** Each
- **Action:** Turn on switch (air purifier)

#### YAML example for PM1-based air purifier

**\*\*automation\*\***

```
automation: |
 alias: "Air purifier on high PM1"
 triggers:
 - trigger: air_quality.pm1_crossed_threshold
 target:
 entity_id: sensor.living_room_pm1
 options:
 threshold: 25
 behavior: any
 actions:
 - action: switch.turn_on
 target:
 entity_id: switch.living_room_air_purifier
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm25 Changed

---

The **PM2.5 level changed** trigger fires after the PM2.5 (particulate matter 2.5 micrometers or smaller) reading on one or more air quality sensors changes by a meaningful amount. PM2.5 is the most widely tracked particle size for indoor and outdoor air quality. Sources include cooking, candles, wildfires, traffic exhaust, and dust. These fine particles are small enough to reach deep into the lungs, making them especially relevant during wildfire season, allergy season, or anytime you want to keep tabs on what your family is breathing.

Imagine your windows closing automatically the moment outdoor particle levels jump during a nearby wildfire, keeping smoke outside where it belongs. Use this trigger to turn on an air purifier, close windows, or send an alert whenever your PM2.5 sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM2.5 level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the PM2.5 level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm25_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm25_changed
 target:
 entity_id: sensor.living_room_pm25
 options:
 threshold: 10
```

This fires whenever the living room PM2.5 sensor reading changes by at least 10 micrograms per cubic meter.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the PM2.5 level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- PM2.5 is the particle size most commonly referenced in air quality indexes worldwide. A threshold of 5 to 15  $\mu\text{g}/\text{m}^3$  suits most home automations.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when PM2.5 crosses a specific concentration in one direction, use [PM2.5 level crossed threshold](#) instead.



## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: close windows during wildfire season

When wildfire smoke rolls in, the last thing you want is to leave the windows open. This automation closes your motorized windows as soon as PM2.5 levels shift significantly, keeping smoke and fine particles outside where they belong.

- **Trigger:** PM2.5 level changed
- **Target:** Outdoor PM2.5 sensor
- **Threshold type:** 15
- **Action:** Close cover

#### YAML example for closing windows on PM2.5 change

**\*\*automation\*\***

```
automation: |
 alias: "Close windows on PM2.5 change"
 triggers:
 - trigger: air_quality.pm25_changed
 target:
 entity_id: sensor.outdoor_pm25
 options:
 threshold: 15
 actions:
 - action: cover.close_cover
 target:
 entity_id: cover.living_room_windows
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm25 Crossed Threshold

---

The **PM2.5 level crossed threshold** trigger fires when the PM2.5 (particulate matter 2.5 micrometers or smaller) reading on one or more air quality sensors crosses a specific level. PM2.5 is one of the most widely tracked air quality metrics because these fine particles travel deep into the lungs and affect your health. When PM2.5 rises above 25 µg/m³, air quality is already poor enough to bother sensitive groups, including children and anyone with asthma or allergies.

Have your air purifier start the second PM2.5 crosses the safe limit, or close your windows automatically when wildfire smoke pushes outdoor readings into unhealthy territory. You also get a notification on your phone so you always know what is happening, whether you are at home or away. This trigger helps your home react to air quality changes faster than you ever could on your own.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM2.5 level crossed threshold**.
6. Under **Threshold type**, set the PM2.5 level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The PM2.5 concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm25_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm25_crossed_threshold
 target:
 entity_id: sensor.outdoor_pm25
 options:
 threshold: 35
 behavior: any
```

This fires whenever the outdoor PM2.5 sensor crosses 35 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The PM2.5 concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.



## Good to know

---

- The trigger fires on any crossing, up or down. If you want to monitor only one direction, add a condition that checks whether the current PM2.5 level is above or below your threshold.
- The WHO guideline for 24-hour average PM2.5 exposure is 15. Many people use a threshold between 25 and 50 for automations depending on their sensitivity and local conditions.
- Pair this trigger with [PM2.5 level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: close windows when outdoor pollution rises

Wildfire smoke, traffic exhaust, and smog all push PM2.5 to unhealthy levels. This automation closes your smart windows the moment outdoor PM2.5 crosses 35, sealing your home off from pollution before it drifts inside.

- **Trigger:** PM2.5 level crossed threshold
- **Target:** Outdoor PM2.5 sensor
- **Threshold type:** 35
- **Trigger when:** Each
- **Condition:** PM2.5 is above 35
- **Action:** Close cover (windows)

#### YAML example for closing windows on high PM2.5

**\*\*automation\*\***

```
automation: |
 alias: "Close windows on high PM2.5"
 triggers:
 - trigger: air_quality.pm25_crossed_threshold
 target:
 entity_id: sensor.outdoor_pm25
 options:
 threshold: 35
 behavior: any
 conditions:
 - condition: numeric_state
 entity_id: sensor.outdoor_pm25
 above: 35
 actions:
 - action: cover.close_cover
 target:
 entity_id: cover.living_room_windows
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm4 Changed

---

The **PM4 level changed** trigger fires after the PM4 (particulate matter 4 micrometers or smaller) reading on one or more air quality sensors changes by a meaningful amount. PM4 particles come from sources like pollen, mold spores, dust, and certain industrial processes. They sit between the finer PM2.5 and the coarser PM10 range, giving you an additional view of the particles floating in your air. If you deal with seasonal allergies or live near a dusty road, tracking PM4 helps you spot irritants before your nose does.

Imagine your home office air purifier starting as soon as pollen counts shift on a spring afternoon, so you stay focused instead of reaching for tissues. Use this trigger to start filtration, log changes, or notify household members whenever PM4 readings shift noticeably.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM4 level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the PM4 level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm4_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm4_changed
 target:
 entity_id: sensor.office_pm4
 options:
 threshold: 10
```

This fires whenever the office PM4 sensor reading changes by at least 10 micrograms per cubic meter.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the PM4 level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}



## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- PM4 fills the gap between PM2.5 and PM10 measurements, which is useful if your sensor reports this size range separately.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when PM4 crosses a specific concentration in one direction, use [PM4 level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: notify when office PM4 shifts

Allergy season and dusty days make it hard to focus when you work from home. This automation sends a notification when PM4 levels in your home office change noticeably, so you know exactly when to turn on the air purifier or crack a window.

- **Trigger:** PM4 level changed
- **Target:** Office PM4 sensor
- **Threshold type:** 10
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for PM4 office notification

**\*\*automation\*\***

```
automation: |
 alias: "Notify on office PM4 change"
 triggers:
 - trigger: air_quality.pm4_changed
 target:
 entity_id: sensor.office_pm4
 options:
 threshold: 10
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "PM4 change detected"
 message: "PM4 levels in the office changed. Consider turning on the a:
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Pm4 Crossed Threshold

---

The **PM4 level crossed threshold** trigger fires when the PM4 (particulate matter 4 micrometers or smaller) reading on one or more air quality sensors crosses a specific level. PM4 sits between the finest particles (PM2.5) and the coarser dust and pollen (PM10), capturing a range of irritants that affect breathing and comfort. Sources include household dust, pollen, mold spores, and cooking emissions.

Think of a nursery where clean air really matters. This trigger lets you boost the air filter to high speed the moment PM4 levels rise, or send a notification to your phone when spring pollen pushes particle counts past your comfort level. You stay one step ahead, keeping the air cleaner for young children and anyone with allergies.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **PM4 level crossed threshold**.
6. Under **Threshold type**, set the PM4 level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The PM4 concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.pm4_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.pm4_crossed_threshold
 target:
 entity_id: sensor.nursery_pm4
 options:
 threshold: 30
 behavior: any
```

This fires whenever the nursery PM4 sensor crosses 30 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The PM4 concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current PM4 level is above or below your threshold.
- PM4 sits between PM2.5 and PM10 in particle size. If your sensor reports multiple particulate sizes, combining thresholds for each gives you a more complete picture of air quality.
- Pair this trigger with [PM4 level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: protect the nursery by boosting the air filter

Young children are more sensitive to airborne particles. This automation switches the nursery air filter to high speed the moment PM4 crosses 30, keeping the room cleaner so your children breathe easier.

- **Trigger:** PM4 level crossed threshold
- **Target:** Nursery PM4 sensor
- **Threshold type:** 30
- **Trigger when:** Each
- **Action:** Turn on fan (air filter)

#### YAML example for PM4-based nursery air filter

**\*\*automation\*\***

```
automation: |
 alias: "Nursery air filter on high PM4"
 triggers:
 - trigger: air_quality.pm4_crossed_threshold
 target:
 entity_id: sensor.nursery_pm4
 options:
 threshold: 30
 behavior: any
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.nursery_air_filter
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Smoke Cleared

---

The **Smoke cleared** trigger fires after a smoke sensor entity stops detecting smoke, letting your home confirm that the danger has passed and it is safe to breathe easy again. After the chaos of a smoke alarm, an automatic all-clear brings real relief. Use this trigger to re-lock doors that were unlocked during evacuation, send a reassuring notification to your family, or restore your home to its normal routine.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your smoke sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Smoke cleared**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, set how long the sensor must stay in the cleared state before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor clears, **First** to fire only when the first sensor in a group clears, or **All** to fire only after every targeted sensor has cleared. For at least: description: How long the sensor must stay in the cleared state before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.smoke_cleared`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.smoke_cleared
 target:
 entity_id: binary_sensor.living_room_smoke
```

This fires every time `binary_sensor.living_room_smoke` transitions to the cleared state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a sensor transitions from a known, valid state. If a sensor comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Use the **For at least** option to confirm the smoke has truly cleared. A delay of ten or fifteen minutes helps avoid premature all-clear actions.
- To react to the opposite transition, use `Smoke detected`.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: secure your home again once the smoke clears

After a smoke event, your front door was unlocked to help everyone evacuate safely. Once every smoke sensor has been clear for fifteen minutes, this automation locks the door again and sends a reassuring notification to your phone. Your home goes back to being secure, and you know the situation is truly resolved without having to check every sensor yourself.

- **Trigger:** Smoke cleared
- **Target:** All smoke sensors (by label)
- **Trigger when:** All
- **For at least:** 00:15:00
- **Action:** Lock: Lock
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for re-locking after smoke clears

```
automation
```

```
automation: |
 alias: "Re-lock door after smoke clears"
 triggers:
 - trigger: air_quality.smoke_cleared
 target:
 label_id: smoke_sensors
 options:
 behavior: last
 for: "00:15:00"
 actions:
 - action: lock.lock
 target:
 entity_id: lock.front_door
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "All smoke sensors are clear."
 title: "Smoke all-clear"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.



## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Smoke Detected

---

The **Smoke detected** trigger fires the moment a smoke sensor entity starts detecting smoke, giving you the earliest possible warning of a potential fire. Whether you are deep asleep at 3 AM, away on vacation, or simply in another part of the house, this trigger makes sure Home Assistant reacts on your behalf. Flash the lights to wake sleeping children, unlock doors to speed up evacuation, or send an urgent alert to your phone so you always know what is happening at home.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your smoke sensor is in (like your kitchen or garage). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Smoke detected**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, set how long the sensor must stay in the detected state before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor detects smoke, **First** to fire only when the first sensor in a group detects smoke, or **All** to fire only after every targeted sensor detects smoke. For at least: description: How long the sensor must stay in the detected state before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.smoke_detected`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.smoke_detected
 target:
 entity_id: binary_sensor.living_room_smoke
```

This fires every time `binary_sensor.living_room_smoke` transitions to the detected state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a sensor transitions from a known, valid state. If a sensor comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- For fire safety, keep the **For at least** delay short or at zero. Every second counts when smoke is involved.
- To react to the opposite transition, use `Smoke cleared`.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: flash the lights and alert everyone in the household

Picture this: it is the middle of the night and a smoke sensor activates in the hallway. A standard alarm beeps, but someone wearing earplugs or a heavy sleeper might not notice. This automation flashes every light in the living room and sends an urgent notification to your phone at the same time, making sure the alert reaches everyone through both sight and sound.

- **Trigger:** Smoke detected
- **Target:** All smoke sensors (by label)
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Light: Turn on light (flash)
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a smoke detection alert

```
automation
```



```

automation: |
 alias: "Smoke alert with lights and notification"
 triggers:
 - trigger: air_quality.smoke_detected
 target:
 label_id: smoke_sensors
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: light.turn_on
 target:
 area_id: living_room
 data:
 flash: long
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "Smoke detected in the house!"
 title: "Smoke alert"

```

## Automation: unlock the front door so your family gets out faster

In a fire, fumbling with a lock in the dark costs precious seconds. This automation unlocks the front door after smoke has been confirmed for 30 seconds, removing one obstacle from the evacuation path. Your family gets a clear exit, and you get the peace of mind that your home is actively looking out for their safety.

- **Trigger:** Smoke detected
- **Target:** All smoke sensors (by label)
- **Trigger when:** Each
- **For at least:** 00:00:30
- **Action:** Lock: Unlock

### YAML example for unlocking the door on smoke detection

```

automation

```

```
automation: |
 alias: "Unlock front door on smoke detection"
 triggers:
 - trigger: air_quality.smoke_detected
 target:
 label_id: smoke_sensors
 options:
 behavior: any
 for: "00:00:30"
 actions:
 - action: lock.unlock
 target:
 entity_id: lock.front_door
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

`{% endif %}`

---

## Triggers/Air Quality.S02 Changed

---

The **Sulphur dioxide level changed** trigger fires after the sulphur dioxide (SO<sub>2</sub>) reading on one or more air quality sensors changes by a meaningful amount. Sulphur dioxide is a sharp-smelling gas released by burning fossil fuels (especially coal and oil), volcanic activity, and certain industrial processes. It irritates the respiratory system and contributes to acid rain. If you live near an industrial area, a power plant, or in a volcanically active region, SO<sub>2</sub> levels are worth watching closely.

Imagine your outdoor vents sealing automatically when an industrial plume drifts your way, keeping that acrid air out of your home. Use this trigger to activate air filtration, close outdoor vents, or send notifications whenever your SO<sub>2</sub> sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Sulphur dioxide level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the sulphur dioxide level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.so2_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.so2_changed
 target:
 entity_id: sensor.rooftop_so2
 options:
 threshold: 5
```

This fires whenever the rooftop SO2 sensor reading changes by at least 5 ppb.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the sulphur dioxide level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.



## Good to know

---

- Sulphur dioxide levels are most relevant near industrial areas, power plants, or regions with volcanic activity. Even moderate exposure irritates the throat and lungs.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when SO<sub>2</sub> crosses a specific concentration in one direction, use [Sulphur dioxide level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: seal the house when SO2 shifts

If you live near an industrial site or in a volcanically active area, SO2 plumes drift in without warning. This automation closes the motorized vents when your outdoor SO2 sensor detects a significant change, keeping that acrid air out of your home.

- **Trigger:** Sulphur dioxide level changed
- **Target:** Rooftop SO2 sensor
- **Threshold type:** 5
- **Action:** Close cover

#### YAML example for SO2-driven vent closure

**\*\*automation\*\***

```
automation: |
 alias: "Close vents on SO2 change"
 triggers:
 - trigger: air_quality.so2_changed
 target:
 entity_id: sensor.rooftop_so2
 options:
 threshold: 5
 actions:
 - action: cover.close_cover
 target:
 entity_id: cover.fresh_air_vents
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.S02 Crossed Threshold

---

The **Sulphur dioxide level crossed threshold** trigger fires when the sulphur dioxide (SO<sub>2</sub>) reading on one or more air quality sensors crosses a specific level. Sulphur dioxide is a sharp-smelling gas produced by burning fossil fuels that contain sulphur, volcanic activity, and some industrial processes. The WHO recommends keeping 24-hour SO<sub>2</sub> exposure below 40 micrograms per cubic meter, because elevated levels irritate the respiratory system and worsen conditions like asthma.

If you live near industrial areas or in a region with volcanic activity, this trigger is especially valuable. Have your smart windows close automatically the moment outdoor SO<sub>2</sub> crosses your safety limit, or get an alert on your phone so you know to stay indoors until the air clears. Your home reacts to changing conditions in real time, keeping irritating fumes outside where they belong.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Sulphur dioxide level crossed threshold**.
6. Under **Threshold type**, set the sulphur dioxide level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The sulphur dioxide concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.so2_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.so2_crossed_threshold
 target:
 entity_id: sensor.outdoor_so2
 options:
 threshold: 40
 behavior: any
```

This fires whenever the outdoor SO2 sensor crosses 40 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The sulphur dioxide concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}



## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current SO2 level is above or below your threshold.
- The WHO guideline for 24-hour SO2 exposure is 40. If you live near industrial areas or in regions with volcanic activity, a threshold around that value is a good starting point.
- Pair this trigger with [Sulphur dioxide level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: seal the house when industrial or volcanic SO2 spikes

If you live downwind of a factory or in a volcanic region, SO2 levels change fast. This automation closes your smart windows the moment outdoor SO2 crosses 40, keeping that sharp, irritating gas out of the house so your family breathes easy.

- **Trigger:** Sulphur dioxide level crossed threshold
- **Target:** Outdoor SO2 sensor
- **Threshold type:** 40
- **Trigger when:** Each
- **Condition:** SO2 is above 40
- **Action:** Close cover (windows)

#### YAML example for closing windows on high SO2

**\*\*automation\*\***

```
automation: |
 alias: "Close windows on high SO2"
 triggers:
 - trigger: air_quality.so2_crossed_threshold
 target:
 entity_id: sensor.outdoor_so2
 options:
 threshold: 40
 behavior: any
 conditions:
 - condition: numeric_state
 entity_id: sensor.outdoor_so2
 above: 40
 actions:
 - action: cover.close_cover
 target:
 entity_id: cover.living_room_windows
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Voc Changed

---

The **Volatile organic compounds level changed** trigger fires after the VOC reading on one or more air quality sensors changes by a meaningful amount. Volatile organic compounds are gases released by paints, cleaning products, adhesives, new furniture, cooking, and many building materials. That "new furniture smell" or the sharp scent of a freshly cleaned bathroom? Those are VOCs. Elevated levels affect indoor air quality and comfort, and prolonged exposure is a health concern.

Imagine the exhaust fan in your freshly painted room switching on automatically as fumes build up, clearing the air so you don't have to keep checking. Use this trigger to boost ventilation, turn on an air purifier, or log air quality changes whenever your VOC sensor reports a significant shift.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Volatile organic compounds level changed**.
6. Under **Threshold type**, set how much the level has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the volatile organic compounds level has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.voc_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.voc_changed
 target:
 entity_id: sensor.kitchen_voc
 options:
 threshold: 50
```

This fires whenever the kitchen VOC sensor reading changes by at least 50 micrograms per cubic meter.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the volatile organic compounds level must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- VOC sensors are especially useful in kitchens, bathrooms, and newly renovated rooms where off-gassing is common.
- The trigger fires on any change that meets the threshold, whether the level goes up or down.
- To react only when VOC levels cross a specific concentration in one direction, use [Volatile organic compounds level crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: ventilate after painting

A freshly painted room smells exciting at first, but those fumes are VOCs you don't want to breathe for hours. This automation turns on the exhaust fan when VOC levels shift, helping clear the fumes faster so the room is ready to enjoy sooner.

- **Trigger:** Volatile organic compounds level changed
- **Target:** Workshop VOC sensor
- **Threshold type:** 100
- **Action:** Turn on fan

#### YAML example for VOC-driven ventilation

**\*\*automation\*\***

```
automation: |
 alias: "Ventilate on VOC change"
 triggers:
 - trigger: air_quality.voc_changed
 target:
 entity_id: sensor.workshop_voc
 options:
 threshold: 100
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.workshop_exhaust
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Voc Crossed Threshold

---

The **Volatile organic compounds level crossed threshold** trigger fires when the VOC reading on one or more air quality sensors crosses a specific level. VOCs are invisible gases released by paints, cleaning products, new furniture, adhesives, and many everyday household items. When VOC levels climb above comfortable limits, you might notice headaches, eye irritation, or a general feeling that something is "off" about the air.

With this trigger, your ventilation starts automatically the moment VOC readings cross your chosen limit, whether that spike comes from mopping the floor or painting a room. You also get a notification on your phone right away, so you know exactly when to step outside for fresh air. Your home takes care of indoor air quality in the background, so you do not have to keep checking a sensor yourself.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Volatile organic compounds level crossed threshold**.
6. Under **Threshold type**, set the VOC level the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the level must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The VOC concentration the reading has to cross for the trigger to fire. Can be a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.voc_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.voc_crossed_threshold
 target:
 entity_id: sensor.office_voc
 options:
 threshold: 300
 behavior: any
```

This fires whenever the office VOC sensor crosses 300 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The VOC concentration the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current VOC level is above or below your threshold.
- VOC readings often spike after cleaning, painting, or cooking. A threshold that matches your normal baseline helps avoid false alarms during everyday activities.
- Pair this trigger with [Volatile organic compounds level changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: keep your home office comfortable during long work sessions

Hours of focused work in a closed room lets VOCs build up from furniture, carpet, and electronics. This automation opens the office ventilation when the VOC reading crosses 300, keeping the air fresh so you stay focused and headache-free.

- **Trigger:** Volatile organic compounds level crossed threshold
- **Target:** Office VOC sensor
- **Threshold type:** 300
- **Trigger when:** Each
- **Action:** Turn on fan (office ventilation)

#### YAML example for VOC-based office ventilation

**\*\*automation\*\***

```
automation: |
 alias: "Office ventilation on high VOC"
 triggers:
 - trigger: air_quality.voc_crossed_threshold
 target:
 entity_id: sensor.office_voc
 options:
 threshold: 300
 behavior: any
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.office_ventilation
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Air Quality.Voc Ratio Changed

---

The **Volatile organic compounds ratio changed** trigger fires after the VOC ratio reading on one or more air quality sensors changes by a meaningful amount. Some sensors express VOC concentration as a ratio (typically a percentage) rather than an absolute concentration. This ratio indicates how the current VOC level compares to a baseline, giving you a quick sense of whether air quality is improving or getting worse. It is particularly handy for tracking off-gassing from new furniture, fresh paint, or cleaning products over time.

Imagine getting a notification the moment your living room air quality starts dipping after you've finished cleaning, so you know to crack a window. Use this trigger to automate ventilation, send alerts, or log trends whenever your VOC ratio reading shifts noticeably.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Volatile organic compounds ratio changed**.
6. Under **Threshold type**, set how much the ratio has to change before the trigger fires.
7. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: How much the volatile organic compounds ratio has to change before the trigger fires. Can be a fixed number, or reference a helper entity that provides the value.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.voc_ratio_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.voc_ratio_changed
 target:
 entity_id: sensor.living_room_voc_ratio
 options:
 threshold: 5
```

This fires whenever the living room VOC ratio changes by at least 5 percent.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The minimum amount the volatile organic compounds ratio must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- The VOC ratio provides a relative reading. A rising ratio means air quality is degrading compared to the sensor's baseline.
- The trigger fires on any change that meets the threshold, whether the ratio goes up or down.
- To react only when the VOC ratio crosses a specific value in one direction, use [Volatile organic compounds ratio crossed threshold](#) instead.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: alert when living room air quality shifts

Maybe someone just sprayed cleaning solution, or the new couch is off-gassing. This automation sends a notification when the VOC ratio in the living room changes significantly so you know to investigate the cause and open a window.

- **Trigger:** Volatile organic compounds ratio changed
- **Target:** Living room VOC ratio sensor
- **Threshold type:** 5
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for VOC ratio alert

**\*\*automation\*\***

```
automation: |
 alias: "Alert on VOC ratio change"
 triggers:
 - trigger: air_quality.voc_ratio_changed
 target:
 entity_id: sensor.living_room_voc_ratio
 options:
 threshold: 5
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "VOC ratio changed"
 message: "The VOC ratio in the living room shifted. Check ventilation"
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.



## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Air Quality.Voc Ratio Crossed Threshold

---

The **Volatile organic compounds ratio crossed threshold** trigger fires when the VOC ratio reading on one or more air quality sensors crosses a specific level. While the VOC level measures an absolute concentration, the VOC ratio expresses the reading as a proportion of a reference baseline, making it easier to spot relative changes. Some sensors report this as a percentage or index value.

Picture your kitchen extractor fan turning on the instant cooking fumes push the VOC ratio past its normal baseline, clearing the air before odors spread through the house. Or getting an alert on your phone when cleaning products cause a spike so you know to open a window. This trigger reacts to relative shifts in air quality, which is ideal for catching sudden changes even when absolute readings vary from sensor to sensor.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your air quality sensor is in (like your living room or bedroom). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Volatile organic compounds ratio crossed threshold**.
6. Under **Threshold type**, set the VOC ratio the reading must cross for the trigger to fire.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Under **For at least**, set how long the reading must stay past the threshold before the trigger fires. Leave at the default to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: The VOC ratio the reading has to cross for the trigger to fire. You must enter a fixed number, or reference a helper entity that provides the value. Trigger when: description: When multiple sensors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted sensor crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted sensors have crossed the threshold. For at least: description: How long the reading must remain past the threshold before the trigger fires. Defaults to firing immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `air_quality.voc_ratio_crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: air_quality.voc_ratio_crossed_threshold
 target:
 entity_id: sensor.kitchen_voc_ratio
 options:
 threshold: 50
 behavior: any
```

This fires whenever the kitchen VOC ratio sensor crosses 50 in either direction.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: > The VOC ratio the reading has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: true type: any behavior: description: > When multiple sensors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger fires on any crossing, up or down. If you only want one direction, add a condition that checks whether the current VOC ratio is above or below your threshold.
- The VOC ratio is different from the absolute VOC level. Check your sensor's documentation to understand what scale or baseline it uses.
- Pair this trigger with [Volatile organic compounds ratio changed](#) if you also want to track smaller fluctuations between crossings.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: clear cooking fumes before they spread

Frying and searing create a burst of fumes that quickly fills the kitchen. This automation turns on the extractor fan the moment the VOC ratio crosses 50, pulling cooking odors out before they drift into the rest of the house.

- **Trigger:** Volatile organic compounds ratio crossed threshold
- **Target:** Kitchen VOC ratio sensor
- **Threshold type:** 50
- **Trigger when:** Each
- **Action:** Turn on fan (kitchen extractor)

#### YAML example for VOC ratio kitchen extractor

**\*\*automation\*\***

```
automation: |
 alias: "Kitchen extractor on VOC ratio spike"
 triggers:
 - trigger: air_quality.voc_ratio_crossed_threshold
 target:
 entity_id: sensor.kitchen_voc_ratio
 options:
 threshold: 50
 behavior: any
 actions:
 - action: fan.turn_on
 target:
 entity_id: fan.kitchen_extractor
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Alarm Control Panel.Armed

---

The **Alarm armed** trigger fires after an alarm control panel entity becomes armed, regardless of the arming mode. It covers away, home, night, vacation, and any other armed state your alarm supports. Use it when you want a single automation to respond the moment the house is secured, like turning off all the lights, locking the front door, or sending a quick confirmation to your phone that the alarm is set.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm armed**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay armed before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel arms, **First** to fire only when the first panel in a group arms, or **All** to fire only after every targeted panel is armed. For at least: description: How long the alarm must stay armed before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.armed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.armed
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` transitions to any armed state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- This trigger responds to *any* armed mode. If you only want to react to a specific mode, use [Alarm armed away](#), [Alarm armed home](#), [Alarm armed night](#), or [Alarm armed vacation](#) instead.
- Pair this trigger with a notification action to confirm that the alarm is set whenever anyone arms it, no matter the mode.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: lock the front door and turn off the living room lights when the alarm is armed

When someone arms the alarm in any mode, the house locks down. The front door locks, the living room lights turn off, and you get a quick confirmation that everything is secured.

- **Trigger:** Alarm armed
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Lock the front door
- **Action:** Turn off the living room lights

#### YAML example for locking down on arm

**\*\*automation\*\***

```
automation: |
 alias: "Lock down when alarm is armed"
 triggers:
 - trigger: alarm_control_panel.armed
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: lock.lock
 target:
 entity_id: lock.front_door
 - action: light.turn_off
 target:
 area_id: living_room
```

## Automation: send a confirmation notification when any alarm is armed

You want to know the alarm is set, especially when someone else in the household arms it. This automation sends a quick notification to your phone the instant any alarm panel in the house becomes armed.

- **Trigger:** Alarm armed
- **Target:** All alarm panels (by label)
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

### YAML example for arm confirmation notification

**\*\*automation\*\***

```
automation: |
 alias: "Confirm alarm armed"
 triggers:
 - trigger: alarm_control_panel.armed
 target:
 label_id: alarm_panels
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "The alarm has been armed."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Alarm Control Panel.Armed Away

---

The **Alarm armed away** trigger fires after an alarm control panel entity switches to the armed away state. Away mode is typically the full-protection mode you set when everyone leaves the house. Use this trigger to automate tasks that should only happen when the home is completely empty, like turning off the HVAC to save energy, closing the garage door, or starting a security camera recording schedule.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm armed away**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay armed away before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel arms away, **First** to fire only when the first panel in a group arms away, or **All** to fire only after every targeted panel is armed away. For at least: description: How long the alarm must stay armed away before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.armed_away`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.armed_away
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` transitions to the armed away state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.



## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Away mode typically means all sensors are active, including interior motion sensors. If you only want perimeter protection while you are home, look at [Alarm armed home](#) instead.
- To react when the alarm is disarmed after being away, use [Alarm disarmed](#).

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn off the HVAC and close the garage when armed away

When the alarm is armed in away mode, nobody is home. Save energy by turning off the climate system and make sure the garage door is closed.

- **Trigger:** Alarm armed away
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn off the thermostat
- **Action:** Close the garage door

#### YAML example for energy saving on away arm

**\*\*automation\*\***

```
automation: |
 alias: "Save energy when armed away"
 triggers:
 - trigger: alarm_control_panel.armed_away
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: climate.turn_off
 target:
 entity_id: climate.thermostat
 - action: cover.close_cover
 target:
 entity_id: cover.garage_door
```

## Automation: start security camera recording when everyone leaves

When the alarm switches to away mode, start recording on all security cameras so you have footage of everything that happens while the house is empty.

- **Trigger:** Alarm armed away
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on security cameras

### YAML example for camera recording on away

**\*\*automation\*\***

```
automation: |
 alias: "Record cameras when armed away"
 triggers:
 - trigger: alarm_control_panel.armed_away
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: switch.turn_on
 target:
 label_id: security_cameras
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Alarm Control Panel.Armed Home

---

The **Alarm armed home** trigger fires after an alarm control panel entity switches to the armed home state. Home mode typically activates perimeter sensors (doors and windows) while leaving interior motion sensors inactive, so you move around freely inside. Use this trigger to run automations that should start when you are home but want the exterior secured, like locking exterior doors, dimming the porch lights, or sending a confirmation that the perimeter is protected.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm armed home**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay armed home before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel arms home, **First** to fire only when the first panel in a group arms home, or **All** to fire only after every targeted panel is armed home. For at least: description: How long the alarm must stay armed home before the trigger fires. Set to zero to fire immediately.



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.armed_home`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.armed_home
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` transitions to the armed home state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Home mode protects the perimeter while allowing interior movement. If you want full protection with all sensors active, use [Alarm armed away](#) instead.
- To react when the alarm is disarmed from home mode, use [Alarm disarmed](#).

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: lock exterior doors when armed home

When you arm the alarm in home mode for the evening, make sure every exterior door locks automatically. You settle in for the night knowing the perimeter is secure.

- **Trigger:** Alarm armed home
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Lock all exterior doors

#### YAML example for locking doors on home arm

**\*\*automation\*\***

```
automation: |
 alias: "Lock doors when armed home"
 triggers:
 - trigger: alarm_control_panel.armed_home
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: lock.lock
 target:
 label_id: exterior_doors
```

### Automation: dim the porch light when the perimeter is armed

When you arm the alarm in home mode, dim the porch light to a gentle glow. It stays on enough to light the walkway, but signals that the house is locked up for the evening.

- **Trigger:** Alarm armed home
- **Target:** Home alarm panel

- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Dim porch light to 20%

#### YAML example for dimming porch lights

**\*\*automation\*\***

```
automation: |
 alias: "Dim porch lights on home arm"
 triggers:
 - trigger: alarm_control_panel.armed_home
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: light.turn_on
 target:
 entity_id: light.porch
 data:
 brightness_pct: 20
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Alarm Control Panel.Armed Night

---

The **Alarm armed night** trigger fires after an alarm control panel entity switches to the armed night state. Night mode is designed for sleeping hours, keeping perimeter sensors and select interior zones active while allowing movement in bedrooms and bathrooms. Use this trigger to kick off a bedtime routine: turn off downstairs lights, lower the thermostat, and send a goodnight confirmation so you drift off knowing the house is secure.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm armed night**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay armed night before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel arms night, **First** to fire only when the first panel in a group arms night, or **All** to fire only after every targeted panel is armed night. For at least: description: How long the alarm must stay armed night before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.armed_night`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.armed_night
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` transitions to the armed night state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Night mode is ideal for sleeping hours. If you need perimeter-only protection while still active during the day, [Alarm armed home](#) is typically a better fit.
- To react when the alarm is disarmed in the morning, use [Alarm disarmed](#).

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: start a bedtime routine when the alarm is armed for the night

When you arm the alarm in night mode, the house prepares for sleep. Downstairs lights turn off, and the thermostat lowers to a comfortable sleeping temperature.

- **Trigger:** Alarm armed night
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn off downstairs lights
- **Action:** Lower the thermostat

#### YAML example for a bedtime routine

**\*\*automation\*\***

```
automation: |
 alias: "Bedtime routine on night arm"
 triggers:
 - trigger: alarm_control_panel.armed_night
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: light.turn_off
 target:
 area_id: downstairs
 - action: climate.set_temperature
 target:
 entity_id: climate.thermostat
 data:
 temperature: 18
```

## Automation: turn on a hallway night light when night mode activates

A dim hallway light makes midnight trips to the bathroom safer. When the alarm switches to night mode, a soft glow lights the way without disturbing anyone.

- **Trigger:** Alarm armed night
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on hallway light at 5%

### YAML example for a hallway night light

**\*\*automation\*\***

```
automation: |
 alias: "Hallway night light on night arm"
 triggers:
 - trigger: alarm_control_panel.armed_night
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: light.turn_on
 target:
 entity_id: light.hallway
 data:
 brightness_pct: 5
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Alarm Control Panel.Armed Vacation

---

The **Alarm armed vacation** trigger fires after an alarm control panel entity switches to the armed vacation state. Vacation mode is for extended absences when you want maximum protection and the appearance that someone is still home. Use this trigger to start routines that simulate occupancy, like cycling lights on and off on a random schedule, pausing mail delivery notifications, or lowering the thermostat to save energy while you are away for days or weeks.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm armed vacation**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay armed vacation before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel arms vacation, **First** to fire only when the first panel in a group arms vacation, or **All** to fire only after every targeted panel is armed vacation. For at least: description: How long the alarm must stay armed vacation before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.armed_vacation`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.armed_vacation
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` transitions to the armed vacation state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- Vacation mode is designed for extended absences. For day-to-day departures, [Alarm armed away](#) is usually the right choice.
- Pair this with automations that simulate occupancy to deter break-ins while you are traveling.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: simulate occupancy when vacation mode is armed

When you arm the alarm in vacation mode, turn on a helper that your other automations use for vacation-mode behavior. The house looks lived in even when you are hundreds of miles away.

- **Trigger:** Alarm armed vacation
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on the vacation mode helper

#### YAML example for occupancy simulation

**\*\*automation\*\***

```
automation: |
 alias: "Simulate occupancy on vacation"
 triggers:
 - trigger: alarm_control_panel.armed_vacation
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: input_boolean.turn_on
 target:
 entity_id: input_boolean.vacation_mode
```

### Automation: lower the thermostat and notify the household when leaving for vacation

When vacation mode activates, drop the thermostat to save energy and send a message to every family member confirming that the house is in vacation lockdown.

- **Trigger:** Alarm armed vacation

- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Set the thermostat to 15 degrees
- **Action:** Notify all household members

#### YAML example for vacation energy saving

**\*\*automation\*\***

```
automation: |
 alias: "Vacation energy saving"
 triggers:
 - trigger: alarm_control_panel.armed_vacation
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: climate.set_temperature
 target:
 entity_id: climate.thermostat
 data:
 temperature: 15
 - action: notify.notify
 data:
 message: >
 Vacation mode activated. The thermostat
 is set to 15 degrees and the alarm is
 fully armed.
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Alarm Control Panel.Disarmed

---

The **Alarm disarmed** trigger fires after an alarm control panel entity switches to the disarmed state. Use it to start welcome-home routines the moment the alarm is turned off: turn on the entryway lights, set the thermostat to a comfortable temperature, unlock the front door, or play your favorite playlist. Whether you disarm from a keypad, the app, or an automation, this trigger responds instantly.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm disarmed**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay disarmed before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel disarms, **First** to fire only when the first panel in a group disarms, or **All** to fire only after every targeted panel is disarmed. For at least: description: How long the alarm must stay disarmed before the trigger fires. Set to zero to fire immediately.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.disarmed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.disarmed
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` transitions to the disarmed state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.



## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- This trigger fires regardless of which armed mode the alarm was previously in. It responds when the alarm changes to disarmed from another valid state, including armed away, armed custom bypass, armed home, armed night, armed vacation, and triggered.
- To react when the alarm is armed instead of disarmed, use [Alarm armed](#).

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: welcome home with lights and comfortable temperature

When the alarm is disarmed, someone just walked in. Turn on the entryway light and set the thermostat to a comfortable temperature so the house feels warm and inviting from the first step through the door.

- **Trigger:** Alarm disarmed
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on entryway lights
- **Action:** Set the thermostat to 21 degrees

#### YAML example for a welcome-home routine

**\*\*automation\*\***

```
automation: |
 alias: "Welcome home routine"
 triggers:
 - trigger: alarm_control_panel.disarmed
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: light.turn_on
 target:
 area_id: entryway
 - action: climate.set_temperature
 target:
 entity_id: climate.thermostat
 data:
 temperature: 21
```

## Automation: stop occupancy simulation when disarming after vacation

When you arrive home from vacation and disarm the alarm, turn off the occupancy simulation helper so lights stop cycling randomly. Your house returns to normal operation.

- **Trigger:** Alarm disarmed
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn off the vacation mode helper

### YAML example for ending vacation simulation

**\*\*automation\*\***

```
automation: |
 alias: "End vacation mode on disarm"
 triggers:
 - trigger: alarm_control_panel.disarmed
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: input_boolean.turn_off
 target:
 entity_id: input_boolean.vacation_mode
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Alarm Control Panel.Triggered

---

The **Alarm triggered** trigger fires the moment an alarm control panel entity enters the triggered state. This is the state that means something set off the alarm, whether a door was opened, a window was broken, or motion was detected in a protected zone. Use this trigger to take immediate action: flash all the lights red, sound a siren, send an urgent notification to every household member, or start recording on your security cameras.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your alarm panel is in (like your hallway or entryway). You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Alarm triggered**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple alarm panels are targeted.
7. Under **For at least**, set how long the alarm must stay triggered before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple alarm panels are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted panel triggers, **First** to fire only when the first panel in a group triggers, or **All** to fire only after every targeted panel is triggered. For at least: description: How long the alarm must stay triggered before the trigger fires. Set to zero to fire immediately.



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `alarm_control_panel.triggered`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: alarm_control_panel.triggered
 target:
 entity_id: alarm_control_panel.home_alarm
```

This fires every time `alarm_control_panel.home_alarm` enters the triggered state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple alarm panels are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any  
for: description: > Duration the state must hold before firing. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when an alarm panel transitions from a known, valid state. If an alarm panel comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- For maximum safety, keep the **For at least** duration at zero so the automation fires without delay. Every second counts when the alarm goes off.
- To react when the alarm is disarmed after a triggered event, use [Alarm disarmed](#).

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: flash lights and send an urgent notification when the alarm triggers

The alarm just went off. Every light in the house flashes red and your phone gets a critical notification that cuts through silent mode. Whether you are asleep upstairs or away at work, you know immediately that something triggered the alarm.

- **Trigger:** Alarm triggered
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Flash all lights red
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for alarm triggered alert

```
automation
```

```

automation: |
 alias: "Alert on alarm trigger"
 triggers:
 - trigger: alarm_control_panel.triggered
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: light.turn_on
 target:
 label_id: all_lights
 data:
 color_name: red
 flash: long
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Alarm triggered!"
 message: >
 Your home alarm has been triggered.
 Check your cameras immediately.

```

## Automation: sound the siren and start recording cameras

When the alarm triggers, activate the siren and start recording on all security cameras. The siren deters intruders and the camera footage gives you evidence of what happened.

- **Trigger:** Alarm triggered
- **Target:** Home alarm panel
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on the siren
- **Action:** Start camera recording

### YAML example for siren and camera recording

```

automation

```

```
automation: |
 alias: "Siren and cameras on alarm trigger"
 triggers:
 - trigger: alarm_control_panel.triggered
 target:
 entity_id: alarm_control_panel.home_alarm
 options:
 behavior: any
 for: "00:00:00"
 actions:
 - action: siren.turn_on
 target:
 entity_id: siren.home_alarm
 - action: switch.turn_on
 target:
 label_id: security_cameras
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.



## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Button.Pressed

---

Use this trigger when you want an automation to run every time a button entity is pressed. This is useful when a button starts a task on a device and you also want Home Assistant to take a follow-up action.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), select the area, floor, device, label, or entity you want to monitor.
5. From the triggers shown for that target, select **Button pressed**.
6. Select **Save**.

### Options in the UI

This trigger has no additional options beyond the target.

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `button.pressed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: button.pressed
target:
 entity_id: button.air_purifier_reset_filter
```

This fires every time `button.air_purifier_reset_filter` is pressed.

## Options in YAML

This trigger has no additional YAML options beyond the target.

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- This trigger fires when Home Assistant detects a button press from the button entity.
- Changes to `unavailable` or `unknown` do not count as button presses.
- If you only need to press a button from an automation, use the related **Press button** action instead.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: send a notification when a filter reset button is pressed

Use this automation to keep a record of when a maintenance button was pressed.

- **Trigger:** Button pressed
- **Target:** Air purifier filter reset button
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a filter reset button notification

**\*\*automation\*\***

```
automation: |
 - alias: "Notify when the filter reset button is pressed"
 triggers:
 - trigger: button.pressed
 target:
 entity_id: button.air_purifier_reset_filter
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "The air purifier filter reset button was pressed."
```

### Automation: turn on a light when an identify button is pressed

Use this automation when a button helps you locate a device and you want a nearby light to turn on at the same time.

- **Trigger:** Button pressed
- **Target:** Front door camera identify button
- **Action:** Light: Turn on

#### YAML example for turning on a light from an identify button



**\*\*automation\*\***

```
automation: |
 - alias: "Turn on the porch light when the camera identify button is pressed"
 triggers:
 - trigger: button.pressed
 target:
 entity_id: button.front_door_camera_identify
 actions:
 - action: light.turn_on
 target:
 entity_id: light.porch
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Calendar.Event Ended

---

The **Calendar event ended** trigger fires when a calendar event ends. You can also set up the trigger to fire before or after the end of the event.

Use it to automate actions based on the end of a calendar event.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Calendar event ended**.
5. Under **Targets** (see [Targets](#)), select **Add target** and pick what to watch. Select the calendar entity with the event that you want to watch. You can also select a device or a label, for example.
6. Under **Offset**, you can enter the time from the end of the event when the trigger will fire. If you want the trigger to fire at the ending time of the event, skip this step and the next one.
7. If you entered an offset, under **Offset type**, select one of the following:
8. **Before** if you want the trigger to fire before the end of the event.
9. **After** if you want the trigger to fire after the end of the event.
10. Select **Save**.

### Options in the UI

{% options\_ui %} Offset: description: The length of time from the end of the event in days, hours, minutes, and seconds. Offset type: description: Whether to trigger before or after the end of the event, if an offset is defined. default: before

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `calendar.event_ended`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: calendar.event_ended
 target:
 entity_id: calendar.personal
 options:
 offset:
 minutes: 30
 offset_type: after
```

This fires 30 minutes after the end of an event in `calendar.personal`.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} offset: description: > The length of time from the end of the event. Accepts a time period mapping in `hours`, `minutes`, `seconds`, and `days`. Also accepts a duration string in `HH:MM:SS` format, for example. required: true type: time offset\_type: description: > Whether to trigger before or after the end of the event, if an offset is defined. required: false default: before type: string

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Good to know

---

- Note that calendars are read once every 15 minutes. When testing, make sure you do not plan events less than 15 minutes away from the current time, or your trigger might not fire.
- You can also create an automation based on the state of a calendar entity.
- A calendar trigger should not generally use automation mode `single` to ensure the trigger can fire when multiple events end at the same time. For example, use `queued` or `parallel` instead. For details about these modes, refer to the [Automation modes](#) page.
- In YAML, you can also set up other variables for calendar triggers. See [Automation Trigger Variables: Calendar](#) to check the available trigger data.



## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn off lights after a family gathering

After a scheduled family gathering takes place at home, this automation turns off specific lights in the living room.

- **Trigger:** Calendar event ended
- **Action:** Turn off light

#### YAML example for turning off specific lights after a family gathering

**\*\*automation\*\***

```
automation: |
 alias: "Turn off lights after a family gathering"
 triggers:
 - trigger: calendar.event_ended
 target:
 entity_id: calendar.my_family_events
 actions:
 - action: light.turn_off
 target:
 label_id: cozy_lights_living_room
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

`{% endif %}`

---

## Triggers/Calendar.Event Started

---

The **Calendar event started** trigger fires when a calendar event starts. You can also set up the trigger to fire before or after the start of the event.

Use it to automate actions based on the start of a calendar event.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Calendar event started**.
5. Under **Targets** (see [Targets](#)), select **Add target** and pick what to watch. Select the calendar entity with the event that you want to watch. You can also select a device or a label, for example.
6. Under **Offset**, you can enter the time from the start of the event when the trigger will fire. If you want the trigger to fire at the starting time of the event, skip this and the next option and select **Save**.
7. If you entered an offset, under **Offset type**, select one of the following:
8. **Before** if you want the trigger to fire before the start of the event.
9. **After** if you want the trigger to fire after the start of the event.
10. Select **Save**.

### Options in the UI

{% options\_ui %} Offset: description: The length of time from the start of the event in days, hours, minutes, and seconds. Offset type: description: Whether to trigger before or after the start of the event, if an offset is defined. default: before

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `calendar.event_started`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: calendar.event_started
 target:
 entity_id: calendar.personal
 options:
 offset:
 hours: 1
 minutes: 15
 seconds: 5
 days: 1
 offset_type: before
```

This fires 1 day, 1 hour, 15 minutes and 5 seconds before the start of an event in `calendar.personal`.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} offset: description: > The length of time from the start of the event. Accepts a time period mapping in `hours`, `minutes`, `seconds`, and `days`. Also accepts a duration string in `HH:MM:SS` format, for example. required: true type: time offset\_type: description: > Whether to trigger before or after the start of the event, if an offset is defined. required: false type: string default: before

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.



## Good to know

---

- Note that calendars are read once every 15 minutes. When testing, make sure you do not plan events less than 15 minutes away from the current time, or your trigger might not fire.
- You can also create an automation based on the state of a calendar entity.
- A calendar trigger should not generally use automation mode `single` to ensure the trigger can fire when multiple events start at the same time. For example, use `queued` or `parallel` instead. For details about these modes, refer to the [Automation modes](#) page.
- In YAML, you can also set up other variables for calendar triggers. See [Automation Trigger Variables: Calendar](#) to check the available trigger data.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: send notification when a calendar event starts

For the calendar entity `calendar.my_calendar`, at the start of any calendar event, this automation sends a notification that is visible in the notification panel of Home Assistant. This automation allows the start of multiple events at the same time.

- **Trigger:** Calendar event started
- **Action:** Send a persistent notification

#### YAML example for sending a calendar event notification

**\*\*automation\*\***

```
automation: |
 alias: "Calendar notification"
 triggers:
 - trigger: calendar.event_started
 target:
 entity_id: calendar.my_calendar
 actions:
 - action: notify.persistent_notification
 data:
 message: Event started!
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Climate.Hvac Mode Changed

---

The **Thermostat mode changed** trigger fires after the HVAC mode of a climate entity changes. Climate entities include thermostats, air conditioners, heat pumps, and evaporative coolers. The HVAC mode determines what the device is set to do: **Off**, **Heat**, **Cool**, **Auto**, **Dry**, **Fan only**, or **Heat/Cool**. Use this trigger when you want to react to the user changing the device's operational mode, regardless of whether it is actively heating, cooling, or idle.

Note: The UI labels this trigger as "Thermostat," but it works with all climate entities.

You can optionally filter the trigger to fire only when the thermostat switches to one or more specific modes. Leave the mode option empty to fire on any mode change.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Thermostat mode changed** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your thermostat is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Thermostat mode changed**.
6. Optionally, under **Modes**, select one or more modes you want to watch for. Leave it empty to trigger on any mode change.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple thermostats are targeted.
8. Under **For at least**, set how long the thermostat must remain in the new mode before the trigger fires. Leave it at zero to fire immediately.
9. Select **Save**.

### Options in the UI

{% options\_ui %} Modes: description: The HVAC mode or modes the thermostat must switch to for the trigger to fire. Typical modes include **Off**, **Heat**, **Cool**, **Auto**, **Dry**, **Fan only**, and **Heat/Cool**, though the exact modes available depend on your device. Default is empty, which fires on any mode change. Trigger when: description: | When multiple thermostats are targeted, controls when the trigger fires:

- **\*\*Each\*\*** (``any`` in YAML, default): fires every time any targeted thermostat changes mode.
- **\*\*First\*\*** (``first`` in YAML): fires only on the first mode change.
- **\*\*All\*\*** (``last`` in YAML): fires only after every targeted thermostat changes mode.

For at least: description: How long the thermostat must remain in the new mode before the trigger fires. Useful to ignore brief mode changes. Default is  (fires immediately).

## Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Thermostat mode changed** is referred to as `climate.hvac_mode_changed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: climate.hvac_mode_changed
 target:
 entity_id: climate.living_room
```

This fires every time the HVAC mode of `climate.living_room` changes to any mode.

To fire only when the thermostat switches to a specific mode:

### trigger

```
trigger: |
 trigger: climate.hvac_mode_changed
 target:
 entity_id: climate.living_room
 options:
 hvac_mode: "heat"
```

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} hvac\_mode: description: > The HVAC mode or modes the thermostat must switch to for the trigger to fire. Accepts a single mode string or a list of modes. Typical modes include `off`, `heat`, `cool`, `auto`, `dry`, `fan_only`, and `heat_cool`, though the exact modes available depend on your device. Omit to fire on any mode change. required: false type: string default: (empty, fires on any mode change) behavior: description: | When multiple thermostats are targeted, controls when the trigger fires:

- ``any`` (\*\*Each\*\* in the UI, default): fires every time any targeted thermostat
- ``first`` (\*\*First\*\* in the UI): fires only on the first mode change.
- ``last`` (\*\*All\*\* in the UI): fires only after every targeted thermostat changes

required: false type: string default: any for: description: | How long the thermostat must remain in the new mode before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:00:10` fires only after the thermostat has stayed in the new mode for 10 seconds, which is useful to ignore brief mode changes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}





## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The available modes depend entirely on the device. Check your climate entity's documentation or the entity's attributes to see which modes are supported.
- If you filter by mode, the trigger only fires when the device *enters* that mode, not when it leaves it.
- The HVAC mode is different from the `hvac_action`. The mode is what you set the device to do, while the action is what the device is currently doing (heating, cooling, or idle).

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press **Ctrl+V** (or **Cmd+V** on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: adjust lights when first thermostat switches to auto mode

When the first thermostat in the bedroom area switches to auto mode and stays in that mode for at least 30 seconds, dim all lights to create a comfortable ambiance for sleep. The delay prevents lights from dimming during accidental mode changes, and firing on the first thermostat avoids multiple light adjustments.

- **Trigger:** Thermostat mode changed
- **Target:** Bedroom area
- **Modes:** Auto
- **Trigger when:** First
- **For at least:** 30 seconds
- **Action:** Turn on light

#### YAML example for adjusting lights on auto mode

**\*\*automation\*\***

```
automation: |
 alias: "Dim lights on auto mode"
 triggers:
 - trigger: climate.hvac_mode_changed
 target:
 area_id: bedroom
 options:
 hvac_mode: "auto"
 behavior: first
 for: "00:00:30"
 actions:
 - action: light.turn_on
 target:
 entity_id: light.bedroom_ceiling
 data:
 brightness_pct: 30
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Climate.Turned Off

---

The **Thermostat turned off** trigger fires after a climate entity turns off. Climate entities include thermostats, air conditioners, heat pumps, and evaporative coolers. Use it to react the moment the device is shut down, whether it was switched off manually, by a schedule, through a automation, or by a voice command.

Note: The UI labels this trigger as "Thermostat," but it works with all climate entities.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Thermostat turned off** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your thermostat is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Thermostat turned off**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple thermostats are targeted.
7. Under **For at least**, set how long the thermostat must stay off before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: | When multiple thermostats are targeted, controls when the trigger fires:

- **Each** (``any`` in YAML, default): fires every time any targeted thermostat turns off.
- **First** (``first`` in YAML): fires only when the first of a group turns off.
- **All** (``last`` in YAML): fires only after every targeted thermostat is off.

For at least: description: How long the thermostat must stay off before the trigger fires. Default is  (fires immediately).

## Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Thermostat turned off** is referred to as `climate.turned_off`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: climate.turned_off
 target:
 entity_id: climate.living_room
```

This fires every time `climate.living_room` transitions to the off state.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: | When multiple thermostats are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fires every time any targeted thermostat
- ``first`` (**First** in the UI): fires only when the first thermostat turns off.
- ``last`` (**All** in the UI): fires only after every targeted thermostat is off.

required: false type: string default: any for: description: | How long the thermostat must stay off before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:00:10` fires only after the thermostat has stayed off for 10 seconds, which helps ignore accidental toggles or brief power losses. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- Climate entities include thermostats, air conditioners, heat pumps, evaporative coolers, and other HVAC devices.
- This trigger fires when the HVAC mode changes to **Off**. It does not fire when the device is idle (for example, when a thermostat is in heat mode but not actively heating).
- Use this trigger to save energy by turning off supplementary systems like fans or space heaters when climate control stops.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn off fans when all thermostats are off

When all thermostats in the living room area turn off and stay off for 5 minutes, turn off the ceiling fans to save energy and reduce noise. The delay prevents the fans from turning off during brief manual adjustments, and waiting for all thermostats ensures complete shutdown.

- **Trigger:** Thermostat turned off
- **Target:** Living room area
- **Trigger when:** All
- **For at least:** 5 minutes
- **Action:** Turn off fan

#### YAML example for turning off fans when all thermostats stop

**\*\*automation\*\***

```
automation: |
 alias: "Turn off fans when all thermostats off"
 triggers:
 - trigger: climate.turned_off
 target:
 area_id: living_room
 options:
 behavior: last
 for: "00:05:00"
 actions:
 - action: fan.turn_off
 target:
 entity_id: fan.living_room_ceiling
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Climate.Turned On

---

The **Thermostat turned on** trigger fires after a climate entity turns on, entering any operational mode (such as **Heat**, **Cool**, or **Auto**). Climate entities include thermostats, air conditioners, heat pumps, and evaporative coolers. The trigger doesn't care which specific mode the device switches to. It only checks that it transitions from **Off** to any active mode. Use this trigger when you want to react as soon as the climate entity becomes active, regardless of whether it's heating, cooling, or in another mode.

Note: The UI labels this trigger as "Thermostat," but it works with all climate entities.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Thermostat turned on** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your thermostat is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Thermostat turned on**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple thermostats are targeted.
7. Under **For at least**, set how long the thermostat must stay on before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: | When multiple thermostats are targeted, controls when the trigger fires:

- ```
- **Each** (`any` in YAML, default): fires every time any targeted thermostat tu
- **First** (`first` in YAML): fires only when the first of a group turns on.
- **All** (`last` in YAML): fires only after every targeted thermostat is on.
```

For at least: description: How long the thermostat must stay on before the trigger fires. Default is (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Thermostat turned on** is referred to as `climate.turned_on`. A basic example looks like this:

trigger

```
trigger: |
  trigger: climate.turned_on
  target:
    entity_id: climate.living_room
```

This fires every time `climate.living_room` transitions from off to any active mode.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple thermostats are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fire every time any targeted thermostat t
- ``first`` (**First** in the UI): fire only when the first thermostat turns on.
- ``last`` (**All** in the UI): fire only after every targeted thermostat is on.

required: false type: string default: any for: description: | How long the thermostat must stay on before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:00:10` fires only after the thermostat has stayed on for 10 seconds. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- This trigger fires when the device transitions from **Off** to any operational mode (**Heat**, **Cool**, **Auto**, **Dry**, **Fan only**, or **Heat/Cool**).
- The trigger does not fire when switching between active modes. For example, changing from **Heat** to **Cool** will not fire this trigger.
- To react to specific mode changes or when the device switches between modes, use **Thermostat mode changed** instead.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on fans when first evaporative cooler starts

When the first of multiple evaporative coolers turns on and stays on for 10 seconds, start a ceiling fan on low speed to help distribute the cooled air more evenly throughout the room. The short delay ensures the cooler has fully started, and firing on the first one prevents multiple fan activations.

- **Trigger:** Thermostat turned on
- **Target:** Multiple evaporative coolers
- **Trigger when:** First
- **For at least:** 10 seconds
- **Action:** Turn on fan

YAML example for running fans when first cooler starts

****automation****

```
automation: |
  alias: "Start fan when first cooler turns on"
  triggers:
    - trigger: climate.turned_on
      target:
        entity_id:
          - climate.bedroom_evaporative_cooler
          - climate.living_room_evaporative_cooler
      options:
        behavior: first
        for: "00:00:10"
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom_ceiling
      data:
        percentage: 30
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Door.Closed

The **Door closed** trigger fires when a targeted door changes to closed. Use it when you want an automation to run only after a door is shut, like locking up, turning lights off, or resuming another routine.

This trigger is especially useful for routines that should wait for a clear end state, such as a garage door finishing its close cycle or a back door being fully shut before you lock it.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Door closed**.
5. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your door is in, like your entryway or garage. You can also select a floor, a device, a specific entity, or a label.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All**.
7. Under **For at least**, enter how long the door must stay closed before the trigger fires. Leave it at zero to fire right away.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple doors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted door closes, **First** to fire only when the first targeted door closes, or **All** to fire only after every targeted door is closed. For at least: description: How long the door must stay closed before the trigger fires. Set it to zero to fire immediately.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `door.closed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: door.closed
  target:
    entity_id: binary_sensor.back_door
```

This fires when `binary_sensor.back_door` closes.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple doors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the door must stay closed before the trigger fires. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- This trigger works with door contact sensors and door covers, like garage doors, as long as they use the `door` device class.
- If an entity comes back from `unavailable` or `unknown`, that recovery does not count as the door closing.
- The `for` option only fires the automation if the door stays closed for the entire time you set.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: lock the back door after it has been closed for 30 seconds

This automation waits until the back door has stayed closed for 30 seconds, then locks it. That short delay gives you time to come inside without the lock engaging immediately.

- **Trigger:** Door closed
- **Target:** Back door sensor
- **For at least:** 00:00:30
- **Action:** Lock: Lock

YAML example for locking the back door after it closes

****automation****

```
automation: |
  alias: "Lock back door after it closes"
  triggers:
    - trigger: door.closed
      target:
        entity_id: binary_sensor.back_door
      options:
        for: "00:00:30"
  actions:
    - action: lock.lock
      target:
        entity_id: lock.back_door
```

Automation: turn off the hallway light after the garage door closes

If you turn on a nearby hallway light while bringing things in from the car, there is no reason to leave it on once the garage is shut again. This automation turns the light off after the garage door has stayed closed for a minute.

- **Trigger:** Door closed
- **Target:** Garage door
- **For at least:** 00:01:00

- **Action:** Light: Turn off

YAML example for turning off the hallway light

****automation****

```
automation: |
  alias: "Turn off hallway light after garage door closes"
  triggers:
    - trigger: door.closed
      target:
        entity_id: cover.garage_door
      options:
        for: "00:01:00"
  actions:
    - action: light.turn_off
      target:
        entity_id: light.hallway
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Door.Opened

The **Door opened** trigger fires when a targeted door changes to open. Use it when you want Home Assistant to respond the moment someone opens a front door, patio door, or garage door.

This trigger is useful for entry lighting, arrival notifications, security checks, and automations that should start as soon as access to a room or building changes.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Door opened**.
5. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your door is in, like your entryway or garage. You can also select a floor, a device, a specific entity, or a label.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All**.
7. Under **For at least**, enter how long the door must stay open before the trigger fires. Leave it at zero to fire right away.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple doors are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted door opens, **First** to fire only when the first targeted door opens, or **All** to fire only after every targeted door is open. For at least: description: How long the door must stay open before the trigger fires. Set it to zero to fire immediately.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `door.opened`. A basic example looks like this:

trigger

```
trigger: |
  trigger: door.opened
  target:
    entity_id: binary_sensor.front_door
```

This fires when `binary_sensor.front_door` opens.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple doors are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the door must stay open before the trigger fires. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- This trigger works with door contact sensors and door covers, like garage doors, as long as they use the `door` device class.
- If an entity comes back from `unavailable` or `unknown`, that recovery does not count as opening the door.
- The `for` option only fires the automation if the door stays open for the entire time you set.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the entry light when the front door opens after dark

If someone comes home after sunset, this automation turns on the entry light as soon as the front door opens. It gives you light right where you need it instead of leaving the hallway dark.

- **Trigger:** Door opened
- **Target:** Front door sensor
- **Action:** Light: Turn on

YAML example for entry lighting on arrival

****automation****

```
automation: |
  alias: "Turn on entry light when front door opens after dark"
  triggers:
    - trigger: door.opened
      target:
        entity_id: binary_sensor.front_door
  conditions:
    - condition: numeric_state
      entity_id: sun.sun
      attribute: elevation
      below: 0
  actions:
    - action: light.turn_on
      target:
        entity_id: light.entryway
```

Automation: notify you if the garage door opens after you leave home

If the garage door opens after you leave home, you probably want to know right away. This automation waits 30 seconds, then sends a notification so brief movement does not alert you unnecessarily.

- **Trigger:** Door opened

- **Target:** Garage door
- **For at least:** 00:00:30
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a garage-door alert after leaving home

****automation****

```
automation: |
  alias: "Notify when garage door opens after leaving home"
  triggers:
    - trigger: door.opened
      target:
        entity_id: cover.garage_door
      options:
        for: "00:00:30"
  conditions:
    - condition: state
      entity_id: person.frenck
      state: "not_home"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        title: "Garage door opened"
        message: "The garage door has been open for 30 seconds after you left"
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Fan.Turned Off

The **Fan turned off** trigger is useful when you want to react after a fan stops. Use it to end a related routine, restore another device to its normal state, or send a reminder if a fan shuts down when you did not expect it to.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the fan you want to monitor. You can also select an area, a floor, a device, or a label.
5. From the triggers shown for that target, select **Fan turned off**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All**.
7. Under **For at least**, set how long the fan must stay off before the trigger fires.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple fans are targeted, controls whether the trigger fires for **Each** fan, only the **First** fan, or after **All** targeted fans are off. required: false For at least: description: How long the fan must stay off before the trigger fires. required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `fan.turned_off`. A basic example looks like this:

trigger

```
trigger: |
  trigger: fan.turned_off
  target:
    entity_id: fan.kitchen
  options:
    behavior: any
    for: "00:10:00"
```

This fires when `fan.kitchen` has been off for 10 minutes.

Options in YAML

{% options_yaml %} behavior: description: When multiple fans are targeted, controls whether the trigger fires for `any`, `first`, or `last`. required: false type: string default: any for: description: How long the fan must stay off before the trigger fires. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- A fan in the `unknown` or `unavailable` state does not count as turned off.
- If the fan turns on again before the **For at least** time finishes, the timer resets.
- To react when a fan starts instead, use [Fan turned on](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: switch the hallway light back to normal after the fan stops

You might raise the hallway light while the fan is running to make a late-night bathroom trip easier. When the fan stops, this automation restores the light to its usual setting.

- **Trigger:** Fan turned off
- **Target:** Bathroom fan
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on light

YAML example for restoring the hallway light

****automation****

```
automation: |
  alias: "Restore hallway light after bathroom fan"
  triggers:
    - trigger: fan.turned_off
      target:
        entity_id: fan.bathroom
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.hallway
      data:
        brightness_pct: 30
```

Automation: notify if the nursery fan stops overnight

If you rely on airflow for comfort at night, a notification can tell you when the nursery fan has stopped for a few minutes.

- **Trigger:** Fan turned off

- **Target:** Nursery fan
- **Trigger when:** Each
- **For at least:** 00:05:00
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a nursery fan alert

****automation****

```
automation: |
  alias: "Nursery fan stopped overnight"
  triggers:
    - trigger: fan.turned_off
      target:
        entity_id: fan.nursery
      options:
        behavior: any
        for: "00:05:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "The nursery fan has been off for 5 minutes."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Fan.Turned On

The **Fan turned on** trigger is useful when you want something else to happen as soon as a fan starts running. Use it to send a reminder, start a related device, or begin a timed routine after a fan has been on for a while.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the fan you want to monitor. You can also select an area, a floor, a device, or a label.
5. From the triggers shown for that target, select **Fan turned on**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All**.
7. Under **For at least**, set how long the fan must stay on before the trigger fires.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple fans are targeted, controls whether the trigger fires for **Each** fan, only the **First** fan, or after **All** targeted fans are on. required: false For at least: description: How long the fan must stay on before the trigger fires. required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `fan.turned_on`. A basic example looks like this:

trigger

```
trigger: |
  trigger: fan.turned_on
  target:
    entity_id: fan.bedroom
  options:
    behavior: any
    for: "00:05:00"
```

This fires when `fan.bedroom` has been on for 5 minutes.

Options in YAML

{% options_yaml %} behavior: description: When multiple fans are targeted, controls whether the trigger fires for `any`, `first`, or `last`. required: false type: string default: any for: description: How long the fan must stay on before the trigger fires. Accepts a duration string like `00:05:00` for five minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- A fan in the `unknown` or `unavailable` state does not count as turned on.
- If the fan turns off before the **For at least** time finishes, the timer resets.
- To react when a fan stops instead, use [Fan turned off](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: notify when the bathroom fan has been left on

If the bathroom fan has been running for a while, you may want a reminder to turn it off after the room has cleared.

- **Trigger:** Fan turned on
- **Target:** Bathroom fan
- **Trigger when:** Each
- **For at least:** 00:20:00
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a bathroom fan reminder

****automation****

```
automation: |
  alias: "Bathroom fan reminder"
  triggers:
    - trigger: fan.turned_on
      target:
        entity_id: fan.bathroom
      options:
        behavior: any
        for: "00:20:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "The bathroom fan has been on for 20 minutes."
```

Automation: turn on the bedside lamp when the bedroom fan starts

When you start the bedroom fan in the evening, you can also turn on a dim bedside lamp for a calmer bedtime routine.

- **Trigger:** Fan turned on
- **Target:** Bedroom fan
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on light

YAML example for a bedroom bedtime routine

****automation****

```
automation: |
  alias: "Bedroom fan bedtime routine"
  triggers:
    - trigger: fan.turned_on
      target:
        entity_id: fan.bedroom
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.bedside
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Gate.Closed

The **Gate closed** trigger fires when a targeted gate changes to closed. Use it when you want an automation to wait until access is secured again before it runs.

This trigger is useful for turning lights back off, confirming that a gate finished closing, or starting routines that should happen only after the gate is shut.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Gate closed**.
5. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your gate is in, like your driveway or courtyard. You can also select a floor, a device, a specific entity, or a label.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All**.
7. Under **For at least**, enter how long the gate must stay closed before the trigger fires. Leave it at zero to fire right away.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple gates are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted gate closes, **First** to fire only when the first targeted gate closes, or **All** to fire only after every targeted gate is closed. For at least: description: How long the gate must stay closed before the trigger fires. Set it to zero to fire immediately.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `gate.closed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: gate.closed
  target:
    entity_id: cover.driveway_gate
```

This fires when `cover.driveway_gate` closes.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple gates are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the gate must stay closed before the trigger fires. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- This trigger works only with `cover` entities that use the `gate` device class.
- If a gate comes back from `unavailable` or `unknown`, that recovery does not count as the gate closing.
- The `for` option only fires the automation if the gate stays closed for the entire time you set.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn off the driveway lights after the gate has been closed for 2 minutes

If your driveway lights turn on when the gate opens, this automation turns them back off after the gate has stayed closed for 2 minutes. That gives you enough time to park and walk inside.

- **Trigger:** Gate closed
- **Target:** Driveway gate
- **For at least:** 00:02:00
- **Action:** Light: Turn off light

YAML example for turning off driveway lights after the gate closes

****automation****

```
automation: |
  alias: "Turn off driveway lights after the gate closes"
  triggers:
    - trigger: gate.closed
      target:
        entity_id: cover.driveway_gate
      options:
        for: "00:02:00"
  actions:
    - action: light.turn_off
      target:
        entity_id: light.driveway
```

Automation: lock the side door after the courtyard gate closes

If you use the courtyard gate as your usual way in, you might want the side door to lock only after the gate is shut again. This automation waits for the gate to close, then locks the door.

- **Trigger:** Gate closed
- **Target:** Courtyard gate
- **Action:** Lock lock

YAML example for locking the side door after the gate closes

****automation****

```
automation: |
  alias: "Lock the side door after the courtyard gate closes"
  triggers:
    - trigger: gate.closed
      target:
        entity_id: cover.courtyard_gate
  actions:
    - action: lock.lock
      target:
        entity_id: lock.side_door
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Gate.Opened

The **Gate opened** trigger fires when a targeted gate changes to open. Use it when you want Home Assistant to react as soon as a driveway, courtyard, or community gate opens.

This trigger is useful for arrival lighting, security notifications, and routines that should begin the moment a gate gives access to your property.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Gate opened**.
5. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your gate is in, like your driveway or courtyard. You can also select a floor, a device, a specific entity, or a label.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All**.
7. Under **For at least**, enter how long the gate must stay open before the trigger fires. Leave it at zero to fire right away.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple gates are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted gate opens, **First** to fire only when the first targeted gate opens, or **All** to fire only after every targeted gate is open. For at least: description: How long the gate must stay open before the trigger fires. Set it to zero to fire immediately.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `gate.opened`. A basic example looks like this:

trigger

```
trigger: |
  trigger: gate.opened
  target:
    entity_id: cover.driveway_gate
```

This fires when `cover.driveway_gate` opens.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple gates are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the gate must stay open before the trigger fires. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- This trigger works only with `cover` entities that use the `gate` device class.
- If a gate comes back from `unavailable` or `unknown`, that recovery does not count as the gate opening.
- The `for` option only fires the automation if the gate stays open for the entire time you set.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the driveway lights when the gate opens after dark

If you come home after sunset, this automation turns on the driveway lights as soon as the gate opens. That gives you light where you need it without leaving the lights on all evening.

- **Trigger:** Gate opened
- **Target:** Driveway gate
- **Action:** Light: Turn on light

YAML example for driveway lights when the gate opens

****automation****

```
automation: |
  alias: "Turn on driveway lights when the gate opens after dark"
  triggers:
    - trigger: gate.opened
      target:
        entity_id: cover.driveway_gate
  conditions:
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id: light.driveway
```

Automation: notify you if the gate opens while you are away

If the gate opens after everyone has left home, this automation sends a notification right away. It helps you spot an unexpected visitor or a gate that someone forgot to secure.

- **Trigger:** Gate opened
- **Target:** Driveway gate
- **For at least:** 00:00:10
- **Action:** Send a notification message

- **Target:** Mobile app

YAML example for a gate-opened alert while away

****automation****

```
automation: |
  alias: "Notify when the gate opens while I am away"
  triggers:
    - trigger: gate.opened
      target:
        entity_id: cover.driveway_gate
      options:
        for: "00:00:10"
  conditions:
    - condition: state
      entity_id: person.frenck
      state: "not_home"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_mobile_app
      data:
        title: "Gate opened"
        message: "The driveway gate opened while you were away."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidifier.Mode Changed

The **Humidifier mode changed** trigger fires after the operating mode of a humidifier entity changes. Modes are device-specific and typically include options like **Normal**, **Eco**, **Sleep**, **Auto**, or **Baby**, though the exact modes available depend on your device. Use **Humidifier mode changed** to react when the mode changes, for example to automatically lower the target humidity on all your humidifiers when one of them switches to **Eco** mode, keeping your whole home in sync with a single mode change.

You can optionally filter the trigger to fire only when the humidifier switches to a specific mode. Leave the mode option empty to fire on any mode change.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#). Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier mode changed** in an automation, follow these steps:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Humidifier mode changed**.
6. Optionally, under **Mode**, select one or more modes you want to watch for. Leave it empty to trigger on any mode change.
7. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple humidifiers are targeted.
8. Under **For at least**, set how long the humidifier must remain in the new mode before the trigger fires. Leave it at zero to fire immediately.
9. Select **Save**.

Options in the UI

{% options_ui %} Mode: description: The mode or modes the humidifier must switch to for the trigger to fire. Typical modes include **Normal**, **Eco**, **Away**, **Boost**, **Comfort**, **Home**, **Sleep**, **Auto**, and **Baby**, though the exact modes available depend on your device. Default is empty, which fires on any mode change. Trigger when: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- **Each** (``any`` in YAML, default): fire every time any targeted humidifier changes mode.
- **First** (``first`` in YAML): fire only on the first mode change.
- **All** (``last`` in YAML): fire only after every targeted humidifier changes mode.

For at least: description: How long the humidifier must remain in the new mode before the trigger fires. Useful to ignore brief transitional modes some devices cycle through during startup. If you set a short delay of a few seconds, it prevents your automation from firing on that momentary blip. Default is (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier mode changed** is referred to as `humidifier.mode_changed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidifier.mode_changed
  target:
    entity_id: humidifier.bedroom
```

This fires every time the bedroom humidifier switches to a different mode.

To fire only when the humidifier switches to a specific mode:

trigger

```
trigger: |
  trigger: humidifier.mode_changed
  target:
    entity_id: humidifier.bedroom
  options:
    mode: "sleep"
```

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} mode: description: > The mode or modes the humidifier must switch to for the trigger to fire. Accepts a single mode string or a list of modes. Typical modes include `normal`, `eco`, `away`, `boost`, `comfort`, `home`, `sleep`, `auto`, and `baby`, though the exact modes available depend on your device. Omit to fire on any mode change. required: false type: string default: (empty, fires on any mode change) behavior: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fire every time any targeted humidifier changes
- ``first`` (**First** in the UI): fire only on the first mode change.
- ``last`` (**All** in the UI): fire only after every targeted humidifier changes

required: false type: string default: any for: description: | How long the humidifier must remain in the new mode before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:00:10` fires only after the humidifier has stayed in the new mode for 10 seconds, which is useful to ignore brief transitional modes some devices cycle through during startup. required: false type: string default: "00:00:00"


```
{%- assign target_domain = include.domain | default: page.domain -%}
```

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The available modes depend entirely on the device. Check your humidifier's documentation or the Home Assistant entity's attributes to see which modes are supported.
- If you filter by mode, the trigger only fires when the humidifier *enters* that mode, not when it leaves it.
- To check whether the humidifier is currently in a specific mode during a condition step, use the **Humidifier is in mode** condition.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: set the scene when the humidifier enters sleep mode

When the bedroom humidifier switches to sleep mode, dim the lights and activate the night scene so the room feels ready for rest.

- **Trigger:** Humidifier mode changed
- **Target:** Bedroom humidifier
- **Mode:** sleep
- **Trigger when:** Each
- **Action:** Light: Turn on (night scene)

YAML example for a sleep-mode scene

****automation****

```
automation: |
  alias: "Activate night scene on sleep mode"
  triggers:
    - trigger: humidifier.mode_changed
      target:
        entity_id: humidifier.bedroom
      options:
        mode: "sleep"
        behavior: any
  actions:
    - action: scene.turn_on
      target:
        entity_id: scene.bedroom_night
```

Automation: notify when any humidifier switches to Eco mode

When a humidifier in the house switches to Eco mode, send a notification confirming that energy-saving operation has started.

- **Trigger:** Humidifier mode changed
- **Target:** All humidifiers (by label)

- **Mode:** Eco
- **Trigger when:** Each
- **Action:** Send a notification message
- **Target:** My device (`notify.my_device`)

YAML example for an Eco mode notification

****automation****

```
automation: |
  alias: "Notify on Eco mode"
  triggers:
    - trigger: humidifier.mode_changed
      target:
        label_id: all_humidifiers
      options:
        mode: "eco"
        behavior: any
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "A humidifier switched to eco mode."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidifier.Started Drying

The **Humidifier started drying** trigger fires when a humidifier entity begins actively removing moisture from the air. This typically applies to dehumidifiers and devices with a dehumidification device class that pause once the target humidity is reached and then resume when the air becomes too humid again.

Use this trigger to track dehumidification cycles, send alerts when the air becomes too humid, or coordinate other actions that should happen while the device is actively removing moisture.

When you target more than one humidifier, the **Trigger when** option controls when it fires.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier started drying** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your dehumidifier is in (like your basement or bathroom). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Humidifier started drying**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple devices are targeted.
7. Under **For at least**, set how long the device must be actively drying before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: | When multiple devices are targeted, controls when the trigger fires:

- ****Each**** (``any`` in YAML, default): fire every time any targeted device starts
- ****First**** (``first`` in YAML): fire only on the first device that starts drying.
- ****All**** (``last`` in YAML): fire only after every targeted device starts drying.

For at least: description: How long the device must be actively drying before the trigger fires. Default is (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier started drying** is referred to as `humidifier.started_drying`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidifier.started_drying
  target:
    entity_id: humidifier.basement_dehumidifier
```

This fires every time `humidifier.basement_dehumidifier` starts actively removing moisture from the air.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple devices are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fire every time any targeted device start
- ``first`` (**First** in the UI): fire only on the first device that starts dryin
- ``last`` (**All** in the UI): fire only after every targeted device starts dryin

required: false type: string default: any for: description: | How long the device must be actively drying before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:05:00` fires only after the device has been actively drying for 5 minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- **Humidifier started drying** fires independently of [Humidifier turned on](#). A dehumidifier can be on but idle (the air is already dry enough), and **Humidifier started drying** fires only when active drying begins.
- **Humidifier started drying** is most useful with devices that have the dehumidifier device class, but it also applies to multi-mode devices that can switch between humidifying and drying.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: alert when the basement gets too humid

When the basement dehumidifier starts running again, it means the air has become too humid. Send a notification so you can check whether there is a moisture problem that needs attention.

- **Trigger:** Humidifier started drying
- **Target:** Basement dehumidifier
- **Trigger when:** Each
- **For at least:** 00:05:00
- **Action:** Send a notification message
- **Target:** My device (`notify.my_device`)

YAML example for a basement humidity alert

****automation****

```
automation: |
  alias: "Alert when basement dehumidifier starts"
  triggers:
    - trigger: humidifier.started_drying
      target:
        entity_id: humidifier.basement_dehumidifier
      options:
        behavior: any
        for: "00:05:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Basement dehumidifier started drying. Humidity may be high."
```

Automation: close the windows when the dehumidifier kicks in

When the dehumidifier starts drying, close any open motorized windows automatically to prevent more humid air from coming in and making the device work harder. Motorized windows are supported in Home Assistant through integrations like [Velux](#), [Somfy](#), and [KNX](#).

- **Trigger:** Humidifier started drying
- **Target:** Basement dehumidifier
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Cover: Close cover

YAML example for closing windows on dehumidification start

****automation****

```
automation: |
  alias: "Close windows when dehumidifier starts"
  triggers:
    - trigger: humidifier.started_drying
      target:
        entity_id: humidifier.basement_dehumidifier
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: cover.close_cover
      target:
        area_id: basement
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidifier.Started Humidifying

The **Humidifier started humidifying** trigger fires when a humidifier entity begins actively adding moisture to the air. A humidifier that is turned on does not necessarily humidify continuously. It pauses once the target humidity is reached and then resumes when the air dries out again. **Humidifier started humidifying** fires whenever it moved from idle back to active humidification.

Use **Humidifier started humidifying** to track active humidification cycles, send notifications when the air is dry enough that the device kicks back in, or coordinate other devices that should run alongside it.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier started humidifying** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Humidifier started humidifying**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple humidifiers are targeted.
7. Under **For at least**, set how long the humidifier must be actively humidifying before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: | When multiple humidifiers are targeted, controls when the trigger fires:

```
- **Each** (`any` in YAML, default): fire every time any targeted humidifier sta
- **First** (`first` in YAML): fire only on the first humidifier that starts hum
- **All** (`last` in YAML): fire only after every targeted humidifier starts hum
```

For at least: description: How long the humidifier must be actively humidifying before the trigger fires. Default is (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier started humidifying** is referred to as `humidifier.started_humidifying`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidifier.started_humidifying
  target:
    entity_id: humidifier.bedroom
```

This fires every time `humidifier.bedroom` starts actively adding moisture to the air.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fire every time any targeted humidifier s
- ``first`` (**First** in the UI): fire only on the first humidifier that starts h
- ``last`` (**All** in the UI): fire only after every targeted humidifier starts h

required: false type: string default: any for: description: | How long the humidifier must be actively humidifying before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:05:00` fires only after the humidifier has been actively humidifying for 5 minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- **Humidifier started humidifying** fires independently of [Humidifier turned on](#). A humidifier can be on but idle, and **Humidifier started humidifying** fires only when it moves from idle to active.
- To react to the opposite transition on a dehumidifier, use [Humidifier started drying](#).
- If your device is a dehumidifier, it removes moisture rather than adds it. Use [Humidifier started drying](#) instead.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: notify when the nursery needs more moisture

When the nursery humidifier starts humidifying again after a pause, it means the air has dried out. Send a gentle notification so you're aware the cycle has restarted overnight.

- **Trigger:** Humidifier started humidifying
- **Target:** Nursery humidifier
- **Trigger when:** Each
- **Condition:** Time is between 22:00 and 07:00
- **Action:** Send a notification message
- **Target:** My device (`notify.my_device`)

YAML example for a nursery humidity cycle alert

****automation****

```
automation: |
  alias: "Alert when nursery starts humidifying at night"
  triggers:
    - trigger: humidifier.started_humidifying
      target:
        entity_id: humidifier.nursery
      options:
        behavior: any
        for: "00:00:00"
  conditions:
    - condition: time
      after: "22:00:00"
      before: "07:00:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Nursery humidifier started humidifying."
```

Automation: turn on a fan to help distribute moisture

When the bedroom humidifier starts humidifying, turn on a low-speed fan to distribute the moisture more evenly throughout the room.

- **Trigger:** Humidifier started humidifying
- **Target:** Bedroom humidifier
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Fan: Turn on

YAML example for running a fan when humidifying starts

****automation****

```
automation: |
  alias: "Run fan when bedroom humidifies"
  triggers:
    - trigger: humidifier.started_humidifying
      target:
        entity_id: humidifier.bedroom
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom
      data:
        percentage: 30
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidifier.Turned Off

The **Humidifier turned off** trigger fires after a humidifier entity turns off. Use it to react the moment the device shuts down, whether it was switched off manually, by a schedule, or because the target humidity was met and the device powered down automatically.

When you target more than one humidifier, the **Trigger when** option controls when it fires. You can have it fire the first time any targeted humidifier turns off, only after all targeted humidifiers have turned off, or every single time any of them turn off.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier turned off** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Humidifier turned off**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple humidifiers are targeted.
7. Under **For at least**, set how long the humidifier must stay off before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- ****Each**** (``any`` in YAML, default): fire every time any targeted humidifier turns off.
- ****First**** (``first`` in YAML): fire only when the first of a group turns off.
- ****All**** (``last`` in YAML): fire only after every targeted humidifier is off.

For at least: description: How long the humidifier must stay off before the trigger fires. Default is (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier turned off** is referred to as `humidifier.turned_off`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidifier.turned_off
  target:
    entity_id: humidifier.bedroom
```

This fires every time `humidifier.bedroom` transitions from on to off.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fire every time any targeted humidifier t
- ``first`` (**First** in the UI): fire only when the first of a group turns off.
- ``last`` (**All** in the UI): fire only after every targeted humidifier is off.

required: false type: string default: any for: description: | How long the humidifier must stay off before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:05:00` fires only after the humidifier has stayed off for 5 minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The trigger only fires when a humidifier transitions from a known, valid state. Transitions from the **Unavailable** or **Unknown** state to off do not count.
- To react to the opposite transition, use [Humidifier turned on](#).
- Pair **Humidifier turned off** with the `last` behavior to do something only after every humidifier in an area has stopped. For example, send a single notification when all humidifiers in the house are off.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: send an alert when the bedroom humidifier turns off unexpectedly

If the bedroom humidifier turns off during the night, send a notification so you can check whether it ran out of water or was switched off by accident.

- **Trigger:** Humidifier turned off
- **Target:** Bedroom humidifier
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Condition:** Time is between 22:00 and 07:00
- **Action:** Send a notification message
- **Target:** My device (`notify.my_device`)

YAML example for an overnight humidifier-off alert

```
**automation**
```

```

automation: |
  alias: "Alert when bedroom humidifier turns off at night"
  triggers:
    - trigger: humidifier.turned_off
      target:
        entity_id: humidifier.bedroom
      options:
        behavior: any
        for: "00:00:00"
  conditions:
    - condition: time
      after: "22:00:00"
      before: "07:00:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Bedroom humidifier turned off."

```

Automation: turn off the fan when the last humidifier stops

When all humidifiers in the house turn off, turn off the ventilation fan as well, since there is nothing left to support.

- **Trigger:** Humidifier turned off
- **Target:** All humidifiers (by label)
- **Trigger when:** All
- **For at least:** 00:00:00
- **Action:** Fan: Turn off

YAML example for turning off the fan when all humidifiers stop

****automation****

```

automation: |
  alias: "Turn off fan when all humidifiers off"
  triggers:
    - trigger: humidifier.turned_off
      target:
        label_id: all_humidifiers
      options:
        behavior: last
        for: "00:00:00"
  actions:
    - action: fan.turn_off
      target:
        entity_id: fan.ventilation

```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidifier.Turned On

The **Humidifier turned on** trigger fires after a humidifier entity turns on. Use it to start an automation the moment the device powers up, whether you turned it on manually, through the app, or via another automation.

When you target more than one humidifier, the **Trigger when** option controls when it fires. You can have it fire the first time any targeted humidifier turns on, only after all targeted humidifiers have turned on, or every single time any of them turn on.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Humidifier turned on** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your humidifier is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Humidifier turned on**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple humidifiers are targeted.
7. Under **For at least**, set how long the humidifier must stay on before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- ****Each**** (``any`` in YAML, default): fire every time any targeted humidifier turns on.
- ****First**** (``first`` in YAML): fire only when the first of a group turns on.
- ****All**** (``last`` in YAML): fire only after every targeted humidifier is on.

For at least: description: How long the humidifier must stay on before the trigger fires. Default is (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Humidifier turned on** is referred to as `humidifier.turned_on`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidifier.turned_on
  target:
    entity_id: humidifier.bedroom
```

This fires every time `humidifier.bedroom` transitions from off to on.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple humidifiers are targeted, controls when the trigger fires:

- ``any`` (**Each** in the UI, default): fire every time any targeted humidifier t
- ``first`` (**First** in the UI): fire only when the first of a group turns on.
- ``last`` (**All** in the UI): fire only after every targeted humidifier is on.

required: false type: string default: any for: description: | How long the humidifier must stay on before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:05:00` fires only after the humidifier has stayed on for 5 minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The trigger only fires when a humidifier transitions from a known, valid state. If a humidifier comes back from the **Unavailable** or **Unknown** state, the trigger does not fire for that recovery.
- Turning on a humidifier does not necessarily mean it starts actively humidifying immediately. To react when humidification actually begins, use [Humidifier started humidifying](#) instead.
- To react to the opposite transition, use [Humidifier turned off](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on a fan when the bedroom humidifier powers on

When the bedroom humidifier turns on, start a low-speed fan to help distribute the moisture more evenly throughout the room.

- **Trigger:** Humidifier turned on
- **Target:** Bedroom humidifier
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Fan: Turn on

YAML example for running a fan when the humidifier turns on

****automation****

```
automation: |
  alias: "Start fan when bedroom humidifier turns on"
  triggers:
    - trigger: humidifier.turned_on
      target:
        entity_id: humidifier.bedroom
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bedroom
      data:
        percentage: 30
```

Automation: notify when the nursery humidifier turns on overnight

If the nursery humidifier turns on during the night, send a quiet notification to your phone so you know air quality is being maintained without having to check the app.

- **Trigger:** Humidifier turned on

- **Target:** Nursery humidifier
- **Trigger when:** Each
- **Condition:** Time is between 22:00 and 07:00
- **Action:** Send a notification message
- **Target:** My device (`notify.my_device`)

YAML example for a nighttime nursery humidifier notification

****automation****

```
automation: |
  alias: "Notify on nursery humidifier at night"
  triggers:
    - trigger: humidifier.turned_on
      target:
        entity_id: humidifier.nursery
      options:
        behavior: any
        for: "00:00:00"
  conditions:
    - condition: time
      after: "22:00:00"
      before: "07:00:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Nursery humidifier turned on."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidity.Changed

The **Relative humidity changed** trigger fires after a humidity reading changes. Humidity creeps up slowly in a bathroom after a shower, climbs in a greenhouse overnight, or drops when the sun beats down on a dry afternoon. Use the threshold type to filter which changes matter to your automation.

The threshold type controls where the new reading must land for the trigger to fire. You can require the new value to be above a level, below a level, within a range, or outside a range. You can also select **Any change** to fire on any change at all.

Use **Relative humidity changed** to log humidity trends, trigger a fan when the air in a room becomes noticeably more humid, or alert you when a sensor reading shifts in a way that might signal a problem.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Relative humidity changed** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your humidity sensor is in (like your bathroom or bedroom). You can also select a device, a specific entity, or a label. When you target multiple entities (via area, label, or multiple entity selections), the trigger fires whenever any of them changes.
5. From the triggers shown for that target, select **Relative humidity changed**.
6. Under **Threshold type**, configure what kind of change fires the trigger:
7. Select **Any change** to fire on any change, regardless of direction or new value.
8. Select **Above** or **Below** and enter a value to fire only when the new reading is above or below that value.
9. Select **In range** and enter a lower and upper bound to fire only when the new reading falls inside the range.
10. Select **Outside range** and enter a lower and upper bound to fire only when the new reading is outside the range.
11. For each option, you can enter a fixed percentage (0-100%), pick a sensor entity or a [number helper](#) entity as the threshold.
 - If you don't have a number helper, you can create one by selecting **Create a new number helper**.
12. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: | Controls which changes fire the trigger:

- ****Any change****: fires on any change, regardless of direction or new value.
- ****Above**** or ****Below****: enter a value to fire only when the new reading is above or below that value.
- ****In range****: enter a lower and upper bound to fire only when the new reading falls inside the range.
- ****Outside range****: enter a lower and upper bound to fire only when the new reading is outside the range.

For each mode you can enter a fixed percentage (0-100%), reference a sensor entity or a [number helper](#) entity.

{% endoptions_ui %}

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Relative humidity changed** is referred to as `humidity.changed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidity.changed
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: above
      value:
        number: 5
```

This fires whenever the bedroom humidity sensor reading moves to a value above 5%. To fire on any change regardless of direction or value, use `type: any` and omit `value`.

To fire only when the new reading is within a comfort range:

trigger

```
trigger: |
  trigger: humidity.changed
  target:
    entity_id:
      - sensor.bedroom_humidity
      - sensor.bathroom_humidity
  options:
    threshold:
      type: between
      value_min:
        number: 40
      value_max:
        number: 60
```

This fires whenever any of the humidity sensors changes to a value within the comfort range (40-60%).

To fire only when the new reading is outside a comfort range:

trigger

```
trigger: |
  trigger: humidity.changed
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: outside
      value_min:
        number: 40
      value_max:
        number: 60
```

To use a number helper as a dynamic threshold that you can adjust without editing the automation:

trigger

```
trigger: |
  trigger: humidity.changed
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: above
      value:
        entity: input_number.humidity_alert_threshold
```

To monitor all humidity sensors in an area and trigger when any changes outside the comfort range:

trigger

```
trigger: |
  trigger: humidity.changed
  target:
    area_id: basement
  options:
    threshold:
      type: outside
      value_min:
        number: 40
      value_max:
        number: 60
```

This fires whenever any humidity sensor in the basement area changes to a value outside the 40-60% range.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: | A mapping that defines which kind of change fires the trigger:

- ``type: any``: Fires on any change (no additional keys needed).
- ``type: above`` or ``type: below``: Provide ``value`` with a ``number`` key (for a list).
- ``type: between`` or ``type: outside``: Provide ``value_min`` and ``value_max``, each with a ``number`` key.

required: true type: map

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Good to know

- The threshold type controls both the direction and the landing zone of the change. Use **Above** or **Below** to filter by direction, **In range** to fire only when the new value is inside a range, and **Outside range** to fire only when it escapes a range.
- Use **Any change** to fire on every change regardless of direction or where the new value lands.
- To react only when humidity first crosses a specific level, use [Relative humidity crossed threshold](#) instead.
- Pair this trigger with [Relative humidity](#) in follow-up conditions to verify the reading meets a threshold before continuing the automation.
- The trigger works with climate entities, humidifier entities, weather entities, and sensors with the humidity device class.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: run the bathroom fan after a shower

After a shower, humidity in a bathroom can spike quickly. This automation turns on the bathroom fan whenever either bathroom humidity sensor rises above 70%, keeping the room from getting damp.

- **Trigger:** Relative humidity changed
- **Target:** Bathroom and shower humidity sensors
- **Threshold type:** Above 70%
- **Action:** Turn on fan
- **Target:** fan.bathroom

YAML example for a post-shower bathroom fan

****automation****

```
automation: |
  alias: "Run bathroom fan on humidity spike"
  triggers:
    - trigger: humidity.changed
      target:
        entity_id:
          - sensor.bathroom_humidity
          - sensor.shower_humidity
      options:
        threshold:
          type: above
          value:
            number: 70
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bathroom
```

Automation: log humidity changes in the greenhouse

Track how much the humidity in your greenhouse shifts throughout the day by sending a notification whenever the reading changes.

- **Trigger:** Relative humidity changed
- **Target:** Greenhouse humidity sensor
- **Threshold type:** Any change
- **Action:** Send a notification
- **Target:** My Device (`mobile.my_device`)

YAML example for greenhouse humidity logging

****automation****

```
automation: |
  alias: "Log greenhouse humidity changes"
  triggers:
    - trigger: humidity.changed
      target:
        entity_id: sensor.greenhouse_humidity
      options:
        threshold:
          type: any
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Greenhouse humidity changed significantly."
```

Automation: alert when humidity changes above comfort level

Send a notification whenever the bedroom humidity changes to a level above your personal comfort threshold. Use a number helper as the threshold so you can easily adjust your preferred level through the UI.

- **Trigger:** Relative humidity changed
- **Target:** Bedroom humidity sensor
- **Threshold type:** Above (entity: comfort humidity threshold)
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for using a number helper as threshold

****automation****

```
automation: |
  alias: "Alert when humidity changes above comfort level"
  triggers:
    - trigger: humidity.changed
      target:
        entity_id: sensor.bedroom_humidity
      options:
        threshold:
          type: above
          value:
            entity: input_number.comfort_humidity_threshold
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: >-
          Bedroom humidity is now {{ trigger.to_state.state }}%, above
          your comfort threshold.
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Humidity.Crossed Threshold

The **Relative humidity crossed threshold** trigger fires when a humidity reading crosses into a zone you define. A bathroom sensor crossing above 70% after a shower, a basement sensor dipping below 30% in a dry winter, a reading entering a target comfort range, or a reading escaping that range are all supported.

Use **Relative humidity crossed threshold** to automate ventilation when the air becomes too humid, alert you when conditions in a sensitive room drift out of range, or coordinate devices that respond to specific humidity levels.

When you target more than one entity, the trigger's **Trigger when** option controls when it fires.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Relative humidity crossed threshold** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your humidity sensor is in (like your bathroom or basement). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Relative humidity crossed threshold**.
6. Under **Threshold type**, configure the zone the reading must enter for the trigger to fire:
7. Select **Above** or **Below** and enter a value to fire when the reading crosses that level.
8. Select **In range** and enter a lower and upper bound to fire when the reading enters the range from outside.
9. Select **Outside range** and enter a lower and upper bound to fire when the reading leaves the range (crosses past either bound). For each option, you can enter a fixed percentage (0-100%), pick a sensor entity or a [number helper](#) entity as the threshold. If you don't have a number helper, you can create one by selecting **Create a new number helper**.
10. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple entities are targeted.
11. Under **For at least**, set how long the reading must stay past the threshold before the trigger fires. Leave it at zero to fire immediately.
12. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: | Controls the zone the reading must enter for the trigger to fire:

- **Above** or **Below**: enter a value to fire when the reading crosses that level.
- **In range**: enter a lower and upper bound to fire when the reading enters the range from outside.
- **Outside range**: enter a lower and upper bound to fire when the reading leaves the range (crosses past either bound).

For each mode you can enter a fixed percentage (0-100%), reference a sensor entity or a [number helper](#) entity as the threshold.

Trigger when: description: | When multiple entities are targeted, controls when the trigger fires:

- **Each**: fires every time any targeted entity crosses the threshold.
- **First**: fires only on the first crossing.
- **All**: fires only after every targeted entity crosses the threshold.

This corresponds to the `behavior` field in YAML. Default is **Each**.

For at least: description: How long the reading must remain past the threshold before the trigger fires. Useful to avoid triggering on brief fluctuations. For example, set it to `0:05:00` to fire only after the reading has stayed past the threshold for 5 minutes. Default is `0` (fires immediately).

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Relative humidity crossed threshold** is referred to as `humidity.crossed_threshold`. A basic example looks like this:

trigger

```
trigger: |
  trigger: humidity.crossed_threshold
  target:
    entity_id: sensor.bedroom_humidity
  options:
    threshold:
      type: between
      value_min:
        number: 40
      value_max:
        number: 60
```

This fires whenever the bedroom humidity sensor enters the comfort range (40% to 60%).

To fire when the reading leaves a comfort range (escapes above 60% or below 40%):

trigger

```
trigger: |
  trigger: humidity.crossed_threshold
  target:
    entity_id:
      - sensor.bedroom_humidity
      - sensor.bathroom_humidity
  options:
    threshold:
      type: outside
      value_min:
        number: 40
      value_max:
        number: 60
    behavior: any
```

This fires whenever any of the humidity sensors crosses outside the comfort range.

To use a number helper as a dynamic threshold that you can adjust without editing the automation:

trigger

```

trigger: |
  trigger: humidity.crossed_threshold
  target:
    label_id: humidity_sensors
  options:
    threshold:
      type: above
      value:
        entity: input_number.humidity_alert_threshold
    behavior: first

```

This fires when the first humidity sensor with the `humidity_sensors` label crosses above the threshold set in the number helper.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: | A mapping that defines the zone the reading must enter for the trigger to fire. Set `type` to one of:

- ``above`` or ``below``: provide ``value`` with a ``number`` key or an ``entity`` key.
- ``between`` or ``outside``: provide ``value_min`` and ``value_max``, each with a ``numb`

For example:

```

```yaml
threshold:
 type: above
 value:
 number: 70
```

```

required: true type: map behavior: description: | When multiple entities are targeted, controls when the trigger fires. Accepts:

- ``any``: fire every time any targeted entity crosses the threshold.
- ``first``: fire only on the first crossing.
- ``last``: fire only after every targeted entity crosses the threshold.

required: false type: string default: any for: description: | How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:05:00` fires only after the reading has stayed past the threshold for 5 minutes. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- **Above** and **Below** fire on the crossing moment only. Once the reading is above the threshold, the trigger does not fire again until the reading dips back below it and then crosses above again.
- **In range** (**between**) fires when the reading moves from outside the bounds into the bounds. **Outside range** (**outside**) fires when the reading moves from inside the bounds past either bound.
- A comfortable indoor humidity range is typically 40% to 60%. Use **Outside range** with those bounds to fire the moment conditions drift out of that comfort zone.
- Pair this trigger with [Relative humidity changed](#) if you also want to react to smaller fluctuations between crossings.
- Pair this trigger with [Relative humidity](#) in follow-up conditions to double-check the final state.
- The trigger works with climate entities, humidifier entities, weather entities, and sensors with the humidity device class.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the bathroom fan when it gets too humid

After a shower, bathroom humidity can climb fast. This automation turns on the bathroom fan the moment humidity crosses 70%.

- **Trigger:** Relative humidity crossed threshold
- **Target:** Bathroom humidity sensor
- **Threshold type:** Above 70%
- **Trigger when:** Each
- **Action:** Turn on fan
- **Target:** fan.bathroom

YAML example for bathroom humidity ventilation

****automation****

```
automation: |
  alias: "Ventilate bathroom on high humidity"
  triggers:
    - trigger: humidity.crossed_threshold
      target:
        entity_id: sensor.bathroom_humidity
      options:
        threshold:
          type: above
          value:
            number: 70
  actions:
    - action: fan.turn_on
      target:
        entity_id: fan.bathroom
```

Automation: alert when basement humidity goes out of range

Keep your basement at a healthy humidity level by sending a notification whenever the sensor crosses a level that may indicate a moisture problem.

- **Trigger:** Relative humidity crossed threshold
- **Target:** Basement humidity sensors
- **Threshold type:** Above 60%
- **Trigger when:** Each
- **For at least:** 00:10:00
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a basement humidity alert

****automation****

```
automation: |
  alias: "Alert on basement humidity"
  triggers:
    - trigger: humidity.crossed_threshold
      target:
        area_id: basement
      options:
        threshold:
          type: above
          value:
            number: 60
        behavior: any
        for: "00:10:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Basement humidity crossed 60%."
```

Automation: trigger humidifier based on adjustable comfort level

Trigger the humidifier when humidity crosses below your personal comfort threshold. Use a number helper as the threshold so you can easily adjust it through the UI without editing the automation.

- **Trigger:** Relative humidity crossed threshold
- **Target:** Bedroom humidity sensor
- **Threshold type:** Below (entity: comfort humidity threshold)
- **Action:** Turn on switch
- **Target:** switch.bedroom_humidifier

YAML example for using a number helper as threshold

****automation****

```
automation: |
  alias: "Turn on humidifier when crossing below comfort threshold"
  triggers:
    - trigger: humidity.crossed_threshold
      target:
        entity_id: sensor.bedroom_humidity
      options:
        threshold:
          type: below
          value:
            entity: input_number.comfort_humidity_threshold
  actions:
    - action: switch.turn_on
      target:
        entity_id: switch.bedroom_humidifier
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Light.Brightness Changed

The **Light brightness changed** trigger fires after the brightness of a light entity changes by a meaningful amount. Use it to react to fine-grained dimming, like adjusting a fan speed as you brighten the room, or logging changes to a dashboard graph.

The **threshold** field tells Home Assistant how big a change counts. The trigger only fires when the light's brightness moves by at least that much, which keeps it from firing on every tiny tick from a smooth fade.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Light: Light brightness changed**.
5. Under **Targets**, select the light entity, an area, a floor, or a label.
6. Under **Threshold type**, set how much the brightness has to change before the trigger fires.
7. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: How much the brightness has to change before the trigger fires, as a percentage of full brightness. Can be a fixed number, or reference a helper entity that provides the value.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `light.brightness_changed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: light.brightness_changed
  target:
    entity_id: light.living_room
  options:
    threshold: 10
```

This fires whenever the living room light's brightness changes by at least ten percent.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The minimum amount (in percent) the brightness must change before the trigger fires. Accepts a number, or a reference to an `input_number`, `number`, or `sensor` entity with a percent unit. required: false type: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Good to know

- The trigger uses absolute percentage change, so a jump from 20% to 30% is the same size as a drop from 80% to 70%.
- To react only when brightness crosses a fixed line in one direction, use [Light brightness crossed threshold](#) instead.
- The trigger only fires when the light is on. It does not fire when the light transitions from off to on.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: sync a ceiling fan speed to the ceiling light

When you dim the ceiling light down, slow the fan down too. A classic "scene mood" automation that keeps the room coordinated.

- **Trigger:** Light brightness changed
- **Target:** Living room ceiling light
- **Threshold type:** 10
- **Action:** Fan: Set speed

YAML example for a ceiling-light-linked fan

****automation****

```
automation: |
  alias: "Match fan to ceiling light"
  triggers:
    - trigger: light.brightness_changed
      target:
        entity_id: light.living_room_ceiling
      options:
        threshold: 10
  actions:
    - action: fan.set_percentage
      target:
        entity_id: fan.living_room
      data:
        percentage: "{{ state_attr('light.living_room_ceiling', 'brightness_p
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Light.Brightness Crossed Threshold

The **Light brightness crossed threshold** trigger fires when a light entity crosses a specific brightness level. Use it to react to a light passing a particular value in either direction, like starting an automation only once brightness passes 50%.

Unlike [Light brightness changed](#), which fires on any sizable change, this trigger only fires when the brightness moves across the exact threshold you pick. It fires once per crossing, in whichever direction.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Light: Light brightness crossed threshold**.
5. Under **Targets**, select the light entity, an area, a floor, or a label.
6. Under **Threshold type**, set the brightness percentage you want the trigger to watch for.
7. Under **Trigger when**, pick **Each**, **First**, or **All** to control how multiple targets interact.
8. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: The brightness level the light has to cross for the trigger to fire. Expressed as a percentage of full brightness. Trigger when: description: When multiple lights are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted light crosses the threshold, **First** to fire only on the first crossing, or **All** to fire only after all targeted lights have crossed the threshold.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `light.brightness_crossed_threshold`. A basic example looks like this:

trigger

```
trigger: |
  trigger: light.brightness_crossed_threshold
  target:
    entity_id: light.living_room
  options:
    threshold: 50
    behavior: any
```

This fires whenever the living room light crosses 50% brightness in either direction.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: > The brightness percentage the light has to cross for the trigger to fire. Accepts a number or a reference to an `input_number`, `number`, or `sensor` entity. required: false type: any behavior: description: > When multiple lights are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Good to know

- The trigger fires on any crossing, up or down. If you only want one direction, gate the automation with a numeric condition that checks whether the light's brightness is above or below the threshold.
- The threshold is measured against the light's brightness percentage. A light at 0% (off) and the same light at 1% are both below a 50% threshold.
- Pair this trigger with [Light is brightness](#) in follow-up conditions to double-check the final state.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: start mood lighting when the ceiling light dims below 40%

When you dim the ceiling light below 40% in the evening, turn on the accent lighting so the room feels layered instead of dim.

- **Trigger:** Light brightness crossed threshold
- **Target:** Living room ceiling light
- **Threshold type:** 40
- **Trigger when:** Each
- **Condition:** Sun is below horizon
- **Condition:** Ceiling light brightness is below 40%
- **Action:** Light: Turn on (accent lights)

YAML example for mood lighting on dim

****automation****

```
automation: |
  alias: "Mood lighting on dim"
  triggers:
    - trigger: light.brightness_crossed_threshold
      target:
        entity_id: light.living_room_ceiling
      options:
        threshold: 40
        behavior: any
  conditions:
    - condition: sun
      after: sunset
    - condition: light.is_brightness
      target:
        entity_id: light.living_room_ceiling
      options:
        threshold: 40
        behavior: any
  actions:
    - action: light.turn_on
      target:
        entity_id: light.living_room_accent
      data:
        brightness_pct: 60
        color_name: warm_white
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Light.Turned Off

The **Light turned off** trigger fires after a light entity turns off. Use it to start an automation the moment a light goes dark, whether someone flipped a physical switch, pressed a button in the UI, or called an action.

When you target more than one light, the trigger's **behavior** option controls when it fires. You can have it fire the first time any targeted light turns off, the last time the final targeted light turns off, or every single time any of them turn off.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Light: Light turned off**.
5. Under **Targets**, choose what to watch:
 - To watch a specific light, select the entity.
 - To watch every light in a room, select an area.
 - To watch every light on a floor, select a floor.
 - To watch lights sharing a tag, select a label.
6. Under **Trigger when**, pick **Each**, **First**, or **All**.
7. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple lights are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted light turns off, **First** to fire only when the first of a group of on lights turns off, or **All** to fire only after every targeted light is off.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `light.turned_off`. A basic example looks like this:

trigger

```
trigger: |
  trigger: light.turned_off
  target:
    entity_id: light.kitchen
```

This fires every time `light.kitchen` transitions from on to off.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple lights are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Good to know

- The trigger only fires when a light transitions from a known, valid state. Transitions from being unavailable (`unavailable`) or having an unknown state (`unknown`) to off do not count.
- To react to the opposite transition, use [Light turned on](#).
- Pair this trigger with the `last` behavior to run something once every light in an area is off, like turning off the TV when every light in the living room has been switched off.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: arm the alarm when the last light goes off at night

When the last light in the house turns off late at night, arm the alarm automatically. A great way to forget a manual step without forgetting your security.

- **Trigger:** Light turned off
- **Target:** All lights (by label)
- **Trigger when:** All
- **Condition:** Time is after 22:30
- **Action:** Alarm control panel: Arm away

YAML example for an arm-on-last-light automation

****automation****

```
automation: |
  alias: "Arm alarm when last light off"
  triggers:
    - trigger: light.turned_off
      target:
        label_id: all_lights
      options:
        behavior: last
  conditions:
    - condition: time
      after: "22:30:00"
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home
```

Automation: turn off the media player when the living room goes dark

When every light in the living room is off, stop whatever is playing on the living room speaker. Saves energy and silence at the same time.

- **Trigger:** Light turned off

- **Target:** Living room area
- **Trigger when:** All
- **Action:** Media player: Turn off

YAML example for auto-pausing media when the room goes dark

****automation****

```
automation: |
  alias: "Stop media when living room dark"
  triggers:
    - trigger: light.turned_off
      target:
        area_id: living_room
      options:
        behavior: last
  actions:
    - action: media_player.turn_off
      target:
        entity_id: media_player.living_room
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Light.Turned On

The **Light turned on** trigger fires after a light entity turns on. Use it to start an automation the moment the light lights up, whether someone flipped a physical switch, pressed a button in the UI, or called an action in another automation.

When you target more than one light, the trigger's **behavior** option controls when it fires. You can have it fire the first time any targeted light turns on, the last time the final targeted light turns on, or every single time any of them turn on.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Light: Light turned on**.
5. Under **Targets**, choose what to watch:
 - To watch a specific light, select the entity.
 - To watch every light in a room, select an area.
 - To watch every light on a floor, select a floor.
 - To watch lights sharing a tag, select a label.
6. Under **Trigger when**, pick **Each**, **First**, or **All** to control how the trigger behaves when multiple lights are targeted.
7. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple lights are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted light turns on, **First** to fire only when the first of a group of off lights turns on, or **All** to fire only after every targeted light is on.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `light.turned_on`. A basic example looks like this:

trigger

```
trigger: |
  trigger: light.turned_on
  target:
    entity_id: light.kitchen
```

This fires every time `light.kitchen` transitions from off to on.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple lights are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Good to know

- The trigger only fires when a light transitions from a known, valid state. If a light comes back from being unavailable (`unavailable`) or having an unknown state (`unknown`), the trigger does not fire for that recovery.
- Pair this trigger with `Light is off` to make sure the automation only runs when a specific target is expected to have been off first.
- To react to the opposite transition, use `Light turned off`.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: notify when the hallway light comes on at night

When the hallway light turns on after sunset, send a phone notification so you know someone's moving around the house.

- **Trigger:** Light turned on
- **Target:** Hallway light
- **Trigger when:** Each
- **Condition:** Sun is below horizon
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a nighttime hallway notification

****automation****

```
automation: |
  alias: "Notify on hallway light at night"
  triggers:
    - trigger: light.turned_on
      target:
        entity_id: light.hallway
      options:
        behavior: any
  conditions:
    - condition: sun
      after: sunset
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        message: "Hallway light just turned on."
```

Automation: play welcome music when any downstairs light comes on in the morning

When the first downstairs light turns on in the morning, start a gentle playlist so you walk into a lively room.

- **Trigger:** Light turned on
- **Target:** Downstairs area
- **Trigger when:** First
- **Condition:** Time is between 06:00 and 10:00
- **Action:** Media player: Play media

YAML example for a morning welcome playlist

****automation****

```
automation: |
  alias: "Welcome music on first downstairs light"
  triggers:
    - trigger: light.turned_on
      target:
        area_id: downstairs
      options:
        behavior: first
  conditions:
    - condition: time
      after: "06:00:00"
      before: "10:00:00"
  actions:
    - action: media_player.play_media
      target:
        entity_id: media_player.kitchen
      data:
        media_content_id: "spotify:playlist:37i9dQZF1DXdwmD5Q7Gxah"
        media_content_type: music
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Lock.Jammed

The **Lock jammed** trigger helps you react when a lock cannot finish its movement. Use it when you want Home Assistant to warn you about a problem at the door, like a misaligned bolt or something blocking the lock.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Lock jammed**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay jammed before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple locks are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted lock jams, **First** to fire only when the first targeted lock jams, or **All** to fire only after every targeted lock is jammed. required: false For at least: description: How long the lock must stay jammed before the trigger fires. Set to zero to fire immediately. required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `lock.jammed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: lock.jammed
  target:
    entity_id: lock.front_door
```

This fires when `lock.front_door` enters the jammed state.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the lock must stay jammed before the trigger fires. Accepts a duration like `00:01:00` for one minute. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The trigger fires only when a lock changes into the jammed state from a known state. If a lock comes back from `unavailable` or `unknown`, that recovery does not fire this trigger.
- A jammed lock often needs attention right away, so keep **For at least** short unless you want to ignore brief reports.
- To confirm that the lock later reaches its normal state, use [Lock locked](#).

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: notify you when the front door lock jams

If the front door lock jams while someone is leaving, you want to know right away so the door is not left unsecured. This automation sends a phone notification as soon as the lock reports a jam.

- **Trigger:** Lock jammed
- **Target:** Front door lock
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a jammed lock alert

****automation****

```
automation: |
  alias: "Notify when the front door lock jams"
  triggers:
    - trigger: lock.jammed
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        title: "Front door lock jammed"
        message: "Check the front door lock. It may be blocked."
```


Automation: turn on the porch light if any outside door lock jams at night

If a lock at an outside entry jams after dark, extra light can help you see what is wrong. This automation turns on the porch light when any targeted outside lock stays jammed for 10 seconds.

- **Trigger:** Lock jammed
- **Target:** Outside door locks (by label)
- **Trigger when:** Each
- **For at least:** 00:00:10
- **Condition:** Sun is below the horizon
- **Action:** Turn on

YAML example for lighting the entry when a lock jams

****automation****

```
automation: |
  alias: "Turn on the porch light for a jammed lock"
  triggers:
    - trigger: lock.jammed
      target:
        label_id: outside_locks
      options:
        behavior: any
        for: "00:00:10"
  conditions:
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id: light.porch
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Lock.Locked

The **Lock locked** trigger helps you react when a lock reaches the locked state. Use it when you want Home Assistant to confirm that a door is secured before turning off lights, arming an alarm, or sending a status update.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Lock locked**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay locked before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple locks are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted lock locks, **First** to fire only when the first targeted lock locks, or **All** to fire only after every targeted lock is locked. required: false For at least: description: How long the lock must stay locked before the trigger fires. Set to zero to fire immediately. required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `lock.locked`. A basic example looks like this:

trigger

```
trigger: |
  trigger: lock.locked
  target:
    entity_id: lock.front_door
```

This fires when `lock.front_door` changes to the locked state.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the lock must stay locked before the trigger fires. Accepts a duration like `00:05:00` for five minutes. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The trigger fires only when a lock changes into the locked state from a known state. If a lock comes back from `unavailable` or `unknown`, that recovery does not fire this trigger.
- Use **For at least** if you want to wait until the lock has stayed secure for a while before acting.
- To react when a door is no longer secured, use [Lock unlocked](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn off the entry light after the front door locks

When you lock the front door after coming in, you may not need the bright entry light anymore. This automation waits 30 seconds after the door locks, then turns the light off.

- **Trigger:** Lock locked
- **Target:** Front door lock
- **Trigger when:** Each
- **For at least:** 00:00:30
- **Action:** Turn off

YAML example for turning off the entry light

****automation****

```
automation: |
  alias: "Turn off the entry light after locking"
  triggers:
    - trigger: lock.locked
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:00:30"
  actions:
    - action: light.turn_off
      target:
        entity_id: light.entry
```

Automation: arm the alarm when all the outside doors are locked

If you want one final check before arming the house, wait until every outside door lock reports locked. This automation arms the alarm only after all targeted locks are secure.

- **Trigger:** Lock locked
- **Target:** Outside door locks (by label)

- **Trigger when:** All
- **For at least:** 00:00:00
- **Action:** Arm alarm away

YAML example for arming the alarm after all locks are secure

****automation****

```
automation: |
  alias: "Arm the alarm after all locks are locked"
  triggers:
    - trigger: lock.locked
      target:
        label_id: outside_locks
      options:
        behavior: last
        for: "00:00:00"
  actions:
    - action: alarm_control_panel.alarm_arm_away
      target:
        entity_id: alarm_control_panel.home_alarm
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Lock.Opened

The **Lock opened** trigger helps you react when a lock reports that it is open. Use it when you want Home Assistant to respond to a door that has been opened, like turning on lights, pausing an alarm workflow, or sending a message that an entry point is no longer closed.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Lock opened**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay open before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple locks are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted lock opens, **First** to fire only when the first targeted lock opens, or **All** to fire only after every targeted lock is open. required: false For at least: description: How long the lock must stay open before the trigger fires. Set to zero to fire immediately. required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `lock.opened`. A basic example looks like this:

trigger

```
trigger: |
  trigger: lock.opened
  target:
    entity_id: lock.front_door
```

This fires when `lock.front_door` changes to the open state.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the lock must stay open before the trigger fires. Accepts a duration like `00:00:30` for 30 seconds. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The trigger fires only when a lock changes into the open state from a known state. If a lock comes back from `unavailable` or `unknown`, that recovery does not fire this trigger.
- Not every lock reports an open state. Use this trigger only with locks that support open-state reporting.
- To react when the same door becomes secure again, use [Lock locked](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the hall light when the front door opens at night

When you arrive home after dark, it helps if the hallway is already lit. This automation turns on the hall light when the front door lock reports open after sunset.

- **Trigger:** Lock opened
- **Target:** Front door lock
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Condition:** Sun is below the horizon
- **Action:** Turn on

YAML example for lighting the hall when the door opens

****automation****

```
automation: |
  alias: "Turn on the hall light when the front door opens"
  triggers:
    - trigger: lock.opened
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:00:00"
  conditions:
    - condition: sun
      after: sunset
  actions:
    - action: light.turn_on
      target:
        entity_id: light.hall
```

Automation: pause the alarm countdown when the patio door opens

If you have created a script to handle alarm entry steps, opening the patio door can start a different response. This automation runs that script when the patio door lock reports open.

- **Trigger:** Lock opened
- **Target:** Patio door lock
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Script: Turn on script

YAML example for running a user-created script

****automation****

```
automation: |
  alias: "Run the patio entry script when the door opens"
  triggers:
    - trigger: lock.opened
      target:
        entity_id: lock.patio_door
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: script.patio_entry_sequence
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Lock.Unlocked

The **Lock unlocked** trigger helps you react when a lock reaches the unlocked state. Use it when you want Home Assistant to welcome someone home, keep track of entry events, or adjust security devices after a door is no longer locked.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).
Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your lock is in, like your front door or garage entry. You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Lock unlocked**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple locks are targeted.
7. Under **For at least**, set how long the lock must stay unlocked before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: When multiple locks are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted lock unlocks, **First** to fire only when the first targeted lock unlocks, or **All** to fire only after every targeted lock is unlocked. required: false For at least: description: How long the lock must stay unlocked before the trigger fires. Set to zero to fire immediately. required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `lock.unlocked`. A basic example looks like this:

trigger

```
trigger: |
  trigger: lock.unlocked
  target:
    entity_id: lock.front_door
```

This fires when `lock.front_door` changes to the unlocked state.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: > When multiple locks are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: false type: string default: any for: description: > How long the lock must stay unlocked before the trigger fires. Accepts a duration like `00:00:10` for 10 seconds. required: false type: time default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- The trigger fires only when a lock changes into the unlocked state from a known state. If a lock comes back from `unavailable` or `unknown`, that recovery does not fire this trigger.
- Use **For at least** if you want to ignore very brief unlock events.
- To react when the door is secured again, use [Lock locked](#).

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on the hallway light when the front door unlocks

If you often arrive home with your hands full, it helps when the light is already on. This automation turns on the hallway light when the front door unlocks.

- **Trigger:** Lock unlocked
- **Target:** Front door lock
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Turn on

YAML example for turning on the hallway light

****automation****

```
automation: |
  alias: "Turn on the hallway light when the front door unlocks"
  triggers:
    - trigger: lock.unlocked
      target:
        entity_id: lock.front_door
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.hallway
```

Automation: send a message when any storage door lock unlocks

If several storage areas use smart locks, you may want a quick record when one of them is unlocked. This automation sends a phone notification when any targeted storage lock unlocks.

- **Trigger:** Lock unlocked
- **Target:** Storage locks (by label)

- **Trigger when:** Each
- **For at least:** 00:00:00
- **Action:** Send a notification message
- **Target:** My Device (`notify.my_device`)

YAML example for a storage lock notification

****automation****

```
automation: |
  alias: "Notify when a storage lock unlocks"
  triggers:
    - trigger: lock.unlocked
      target:
        label_id: storage_locks
      options:
        behavior: any
        for: "00:00:00"
  actions:
    - action: notify.send_message
      target:
        entity_id: notify.my_device
      data:
        title: "Storage door unlocked"
        message: "One of the storage door locks has been unlocked."
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Motion.Cleared

The **Motion cleared** trigger fires when one or more motion sensors stop detecting motion.

Use it to automate actions, such as turning devices on or off, or sending notifications, based on inactivity in an area of the house. Use a single sensor to detect motion in specific spots and a group of sensors for larger areas.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Motion cleared**.
5. Select **Add target** (see [Targets](#)) and pick the motion sensor that you want to watch. You can also select an area, a floor, a device, or a label.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, you can set how long the sensor must remain without detecting motion before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: | When multiple motion sensors are targeted, controls when the trigger fires:

- ****Each**** (default): fires every time any targeted sensor stops detecting motion.
- ****First****: fires only when the first sensor stops detecting motion.
- ****All****: fires only after every targeted sensor stops detecting motion.

required: false For at least: description: How long the sensor or sensors must remain without detecting motion before the trigger fires. The default is hours, minutes and seconds (fires immediately). required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `motion.cleared`. A basic example looks like this:

trigger

```
trigger: |
  trigger: motion.cleared
  target:
    entity_id: binary_sensor.movement_backyard
  options:
    for:
      hours: 1
      minutes: 5
      seconds: 2
```

This fires 1 hour, 5 minutes and 2 seconds after the sensor entity `binary_sensor.movement_backyard` stops detecting motion.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple motion sensors are targeted, controls when the trigger fires:

- ``any``: fires every time any targeted sensor stops detecting motion.
- ``first``: fires only when the first sensor stops detecting motion.
- ``last``: fires only after every targeted sensor stops detecting motion.

required: false type: string default: any for: description: | How long the sensor or sensors must remain without detecting motion before the trigger fires. Accepts a duration string in `HH:MM:SS` format or a time period mapping in hours, minutes and seconds. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- Use single sensors for motion detection in areas of passage, such as hallways or entrances, or very specific spots like an office desk.
- Add the **For at least** option to your automation to avoid turning off devices too quickly if someone is still in the room.
- For reliable motion detection in larger areas, you can use grouped motion sensors and [input boolean](#) helpers.

Try it yourself

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn off HVAC when there is no motion in the office for 15 minutes

When no movement is detected at the office for 15 minutes, this automation turns off the office HVAC (Heating, Ventilating, and Air Conditioning) entities.

- **Trigger:** Motion cleared
- **Target:** Office motion sensors (by label)
- **Action:** Turn off HVAC (in the office)

YAML example for turning off office HVAC after inactivity

****automation****

```
automation: |
  alias: "Turn off office HVAC after inactivity"
  triggers:
    - trigger: motion.cleared
      target:
        label_id: motion_sensors_office
      options:
        for:
          minutes: 15
  actions:
    - action: climate.set_hvac_mode
      target:
        entity_id: climate.hvac_office
      data:
        hvac_mode: "off"
```


Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Motion.Detected

The **Motion detected** trigger fires when one or more motion sensors start detecting motion.

Use it to automate actions, such as turning devices on or off, or sending notifications, based on motion detection in an area of the house. Use a single motion sensor to detect motion in specific spots and a group of sensors for larger areas.



Labs: Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.

Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Motion detected**.
5. Select **Add target** (see [Targets](#)) and pick the motion sensor that you want to watch. You can also select an area, a floor, a device, or a label.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple sensors are targeted.
7. Under **For at least**, you can set how long the sensor must remain detecting motion before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

Options in the UI

{% options_ui %} Trigger when: description: | When multiple motion sensors are targeted, controls when the trigger fires:

- ****Each**** (default): fires every time any targeted sensor starts detecting motion.
- ****First****: fires only when the first sensor starts detecting motion.
- ****All****: fires only after every targeted sensor starts detecting motion.

required: false For at least: description: How long the sensor or sensors must remain detecting motion before the trigger fires. The default is hours, minutes and seconds (fires immediately). required: false

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `motion.detected`. A basic example looks like this:

trigger

```
trigger: |
  trigger: motion.detected
  target:
    entity_id: binary_sensor.movement_backyard
  options:
    for:
      hours: 1
      minutes: 5
      seconds: 2
```

This fires 1 hour, 5 minutes and 2 seconds after the sensor entity `binary_sensor.movement_backyard` starts detecting motion.

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} behavior: description: | When multiple motion sensors are targeted, controls when the trigger fires:

- ``any``: fires every time any targeted sensor starts detecting motion.
- ``first``: fires only when the first sensor starts detecting motion.
- ``last``: fires only after every targeted sensor starts detecting motion.

required: false type: string default: any for: description: | How long the sensor or sensors must remain detecting motion before the trigger fires. Accepts a duration string in `HH:MM:SS` format or a time period mapping in hours, minutes and seconds. required: false type: string default: "00:00:00"

{%- assign target_domain = include.domain | default: page.domain -%}

Targets of the trigger

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** (`any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** (`first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** (`last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

Good to know

- Use single sensors for motion detection in areas of passage, such as hallways or entrances, or very specific spots like an office desk.
- Add the **For at least** option to your automation to avoid turning off devices too quickly if someone is still in the room.
- For reliable motion detection in larger areas, you can use grouped motion sensors and [input boolean](#) helpers.
- When you automate lights turning on, combine motion with ambient light sensors, or time conditions, and only turn on lights if the room is dark enough. This avoids unnecessary activations.

Try it yourself

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

Automation: turn on light when motion is detected and it is dark

When motion is detected at the entrance of the hallway and if it is dark, this automation turns on the light there.

- **Trigger:** Motion detected
- **Target:** Entrance binary sensor
- **Condition:** Time (after 21:30:00 and before 07:00:00)
- **Action:** Turn on light (in hallway)

YAML example for turning hallway light if it is dark and motion is detected

****automation****

```
automation: |
  alias: "Turn on light in hallway if dark and motion is detected"
  triggers:
    - trigger: motion.detected
      target:
        entity_id: binary_sensor.movement_hallway
  conditions:
    - condition: time
      after: "21:30:00"
      before: "07:00:00"
  actions:
    - action: light.turn_on
      target:
        entity_id: light.hallway_entrance_light
```

Still stuck?

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

Related triggers

These triggers work well alongside this one:

{% endif %}

Triggers/Temperature.Changed

The **Temperature changed** trigger fires after a temperature reading changes. Temperature shifts gradually as heating or cooling systems cycle, rises when the sun heats a room in the afternoon, or drops overnight. Use the threshold type to filter which changes matter to your automation.

Use **Temperature changed** to log temperature trends, trigger heating or cooling when the temperature in a room changes noticeably, or alert you when a sensor reading shifts in a way that might signal a problem.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Temperature changed** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your temperature sensor is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Temperature changed**.
6. Under **Threshold type**, configure what kind of change fires the trigger:
7. Select **Any change** to fire on any change, regardless of direction or new value.
8. Select **Above** or **Below** and enter a value to fire only when the new reading is above or below that value.
9. Select **In range** and enter a lower and upper bound to fire only when the new reading falls inside the range.
10. Select **Outside range** and enter a lower and upper bound to fire only when the new reading is outside the range.
11. For each option, you can enter a fixed temperature, pick a sensor entity or a [number helper](#) entity as the threshold.
 - If you don't have a number helper, you can create one by selecting **Create a new number helper**.
12. Under **Unit**, select the temperature unit (°C or °F) to use for the threshold comparison.
13. Select **Save**.

Options in the UI

{% options_ui %} Threshold type: description: | Controls which changes fire the trigger:

- ****Any change****: fires on any change, regardless of direction or new value.
- ****Above**** or ****Below****: enter a value to fire only when the new reading is above or below that value.
- ****In range****: enter a lower and upper bound to fire only when the new reading falls inside the range.
- ****Outside range****: enter a lower and upper bound to fire only when the new reading is outside the range.

For each mode you can enter a fixed temperature, reference a sensor entity or a

Unit: description: The temperature unit to use for threshold comparison. Accepts **°C** or **°F**. Required when using numerical thresholds (not required when using entity references). Default is **°C**.

Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Temperature changed** is referred to as `temperature.changed`. A basic example looks like this:

trigger

```
trigger: |
  trigger: temperature.changed
  target:
    entity_id: sensor.living_room_temperature
  options:
    threshold:
      type: above
      value:
        number: 20
        unit_of_measurement: "°C"
```

This fires whenever the living room temperature sensor reading moves to a value above 20°C. To fire on any change regardless of direction or value, use `type: any` and omit `value`.

To fire only when the new reading is within a comfort range:

trigger

```
trigger: |
  trigger: temperature.changed
  target:
    entity_id: sensor.living_room_temperature
  options:
    threshold:
      type: between
      value_min:
        number: 20
        unit_of_measurement: "°C"
      value_max:
        number: 22
        unit_of_measurement: "°C"
```

To fire only when the new reading is outside a comfort range:

trigger

```

trigger: |
  trigger: temperature.changed
  target:
    entity_id: sensor.living_room_temperature
  options:
    threshold:
      type: outside
      value_min:
        number: 20
        unit_of_measurement: "°C"
      value_max:
        number: 22
        unit_of_measurement: "°C"

```

Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options_yaml %} threshold: description: | A mapping that defines which kind of change fires the trigger:

- `type: any`: Fires on any change (no additional keys needed).
- `type: above` or `type: below`: Provide `value` with a `number` key (for a lit
- `type: between` or `type: outside`: Provide `value\_min` and `value\_max`, each

When using the `number` key, you must also include `unit\_of\_measurement` to spec

For example:

```

```yaml
threshold:
 type: outside
 value_min:
 entity: input_number.comfort_temperature_min
 value_max:
 number: 24
 unit_of_measurement: °C
...

```

A `sensor` entity's current reading is used as the threshold, which lets you com

required: true type: map

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.



## Good to know

---

- The threshold type controls both the direction and the landing zone of the change. Use **Above** or **Below** to filter by direction, **In range** to fire only when the new value is inside a range, and **Outside range** to fire only when it escapes a range.
- Use **Any change** to fire on every change regardless of direction or where the new value lands.
- To react only when temperature first crosses a specific level, use [Temperature crossed threshold](#) instead.
- The trigger works with [climate](#) entities, [water heater](#) entities, [weather](#) entities, and sensors with the temperature device class.
- Climate, water heater, and weather entities that don't report a current temperature attribute are automatically excluded from the trigger. Only entities with a valid temperature value can fire the trigger.
- All temperature values are automatically converted to the unit you specify. For example, if your sensor reports in Fahrenheit but you configure the trigger in Celsius, the conversion happens automatically.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn on heating or cooling when temperature leaves comfort range

When the living room temperature changes to a value outside the comfort range (20 to 22°C), this automation turns on heating or cooling to restore comfortable conditions.

- **Trigger:** Temperature changed
- **Target:** Living room temperature sensor
- **Threshold type:** Outside range (20-22°C)
- **Action:** Set thermostat HVAC mode (state: cool)

#### YAML example for climate control when outside comfort range

```
automation
```

```

automation: |
 alias: "Adjust climate when living room is uncomfortable"
 triggers:
 - trigger: temperature.changed
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: outside
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
 actions:
 - if:
 - condition: template
 value_template: "{{ trigger.to_state.state | float < 20 }}"
 then:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.living_room
 data:
 hvac_mode: heat
 else:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.living_room
 data:
 hvac_mode: cool

```

## Automation: alert when temperature leaves comfort range

This automation sends a notification when any room temperature drifts outside the comfort range of 20 to 22°C, helping you maintain consistent conditions throughout your home.

- **Trigger:** Temperature changed
- **Target:** All temperature sensors (label)
- **Threshold type:** Outside range (20-22°C)
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

### YAML example for comfort range alert

```

automation

```

```

automation: |
 alias: "Alert when temperature leaves comfort range"
 triggers:
 - trigger: temperature.changed
 target:
 label_id: temperature_sensors
 options:
 threshold:
 type: outside
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: >
 Temperature in {{ trigger.to_state.name }} is {{ trigger.to_state.state }}
 Comfortable range is 20-22°C.

```

## Automation: alert when temperature enters comfort range

Send a notification whenever the bedroom temperature changes to a level within your personal comfort range. Use number helpers for the range bounds so you can easily adjust your preferred temperatures through the UI.

- **Trigger:** Temperature changed
- **Target:** Bedroom temperature sensor
- **Threshold type:** In range (entity: comfort temperature min and max)
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

### YAML example for using number helpers as threshold

```

automation

```

```

automation: |
 alias: "Alert when temperature enters comfort range"
 triggers:
 - trigger: temperature.changed
 target:
 entity_id: sensor.bedroom_temperature
 options:
 threshold:
 type: between
 value_min:
 entity: input_number.comfort_temperature_min
 value_max:
 entity: input_number.comfort_temperature_max
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 message: "Bedroom temperature is now {{ trigger.to_state.state }}°C, v

```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Temperature.Crossed Threshold

---

The **Temperature crossed threshold** trigger fires when a temperature reading crosses into a zone you define. A bedroom sensor crossing below 18°C on a cold night, a living room sensor climbing above 24°C in summer, a reading entering a target comfort range, or a reading escaping that range are all supported.

Use **Temperature crossed threshold** to automate heating or cooling when the temperature becomes uncomfortable, alert you when conditions in a room drift out of range, or coordinate devices that respond to specific temperature levels.

When you target more than one entity, the trigger's **Trigger when** option controls when it fires.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature. Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use **Temperature crossed threshold** in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area your temperature sensor is in (like your bedroom or living room). You can also select a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Temperature crossed threshold**.
6. Under **Threshold type**, configure the zone the reading must enter for the trigger to fire:
7. Select **Above** or **Below** and enter a value to fire when the reading crosses that level.
8. Select **In range** and enter a lower and upper bound to fire when the reading enters the range from outside.
9. Select **Outside range** and enter a lower and upper bound to fire when the reading leaves the range (crosses past either bound).
10. For each option, you can enter a fixed temperature or pick a sensor entity or a [number helper](#) entity as the threshold.
  - If you don't have a number helper, you can create one by selecting **Create a new number helper**.
11. Under **Unit**, select the temperature unit (°C or °F) to use for the threshold comparison.
12. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple entities are targeted.
13. Under **For at least**, set how long the reading must stay past the threshold before the trigger fires. Leave it at zero to fire immediately.
14. Select **Save**.

### Options in the UI

{% options\_ui %} Threshold type: description: | Controls the zone the reading must enter for the trigger to fire:

```
- Above or Below: enter a value to fire when the reading crosses that level
- In range: enter a lower and upper bound to fire when the reading enters the range
- Outside range: enter a lower and upper bound to fire when the reading leaves the range

For each mode you can enter a fixed temperature or reference a sensor entity or
```

Unit: description: The temperature unit to use for threshold comparison. Accepts **°C** or **°F**. Required when using numerical thresholds (not required when using entity references). Default is

°C. Trigger when: description: | When multiple entities are targeted, controls when the trigger fires:

- **\*\*Each\*\***: fires every time any targeted entity crosses the threshold.
- **\*\*First\*\***: fires only on the first crossing.
- **\*\*All\*\***: fires only after every targeted entity crosses the threshold.

This corresponds to the ``behavior`` field in YAML. Default is **\*\*Each\*\***.

For at least: description: How long the reading must remain past the threshold before the trigger fires. Useful to avoid triggering on brief fluctuations. For example, set it to `0:05:00` to fire only after the reading has stayed past the threshold for 5 minutes. Default is `0` (fires immediately).

## Using this trigger in YAML

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, **Temperature crossed threshold** is referred to as `temperature.crossed_threshold`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: temperature.crossed_threshold
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: between
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
```

This fires whenever the living room temperature sensor enters the comfort range (20 to 22°C).

To fire when the reading leaves a comfort range (escapes above 22°C or below 20°C):

### trigger

```
trigger: |
 trigger: temperature.crossed_threshold
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: outside
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
```

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} threshold: description: | A mapping that defines the zone the reading must enter for the trigger to fire:

- ``type: above`` or ``type: below``: Provide ``value`` with a ``number`` key (for a literal value).
- ``type: between`` or ``type: outside``: Provide ``value_min`` and ``value_max``, each with a ``number`` key.

When using the ``number`` key, you must also include ``unit_of_measurement`` to specify the unit.

For example:

```
```yaml
threshold:
  type: between
  value_min:
    number: 18
    unit_of_measurement: °C
  value_max:
    entity: input_number.max_comfort_temperature
```
```

A ``sensor`` entity's current reading is used as the threshold, which lets you compare the current reading to a target value.

required: true type: map behavior: description: | When multiple entities are targeted, controls when the trigger fires. Accepts:

- ``any``: fires every time any targeted entity crosses the threshold.
- ``first``: fires only on the first crossing.
- ``last``: fires only after every targeted entity crosses the threshold.

required: false type: string default: any for: description: | How long the reading must remain past the threshold before the trigger fires. Accepts a duration string in `HH:MM:SS` format. For example, `00:05:00` fires only after the reading has stayed past the threshold for 5 minutes. required: false type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- **Above** and **Below** fire on the crossing moment only. Once the reading is above the threshold, the trigger does not fire again until the reading dips back below it and then crosses above again.
- **In range** ( **between** ) fires when the reading moves from outside the bounds into the bounds. **Outside range** ( **outside** ) fires when the reading moves from inside the bounds past either bound.
- A comfortable indoor temperature range is typically 20 to 22°C (68 to 72°F). Use **Outside range** with those bounds to fire the moment conditions drift out of that comfort zone.
- Pair this trigger with **Temperature changed** if you also want to react to smaller fluctuations between crossings.
- The trigger works with **climate** entities, **water heater** entities, **weather** entities, and sensors with the temperature device class.
- Climate, water heater, and weather entities that don't report a current temperature attribute are automatically excluded from the trigger. Only entities with a valid temperature value can fire the trigger.
- All temperature values are automatically converted to the unit you specify. For example, if your sensor reports in Fahrenheit but you configure the trigger in Celsius, the conversion happens automatically.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press **Ctrl+V** (or **Cmd+V** on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn off climate when temperature enters comfort range

This automation turns off the living room climate system the moment the temperature crosses into the comfort range (20 to 22°C), saving energy once comfortable conditions are achieved.

- **Trigger:** Temperature crossed threshold
- **Target:** Living room temperature sensor
- **Threshold type:** In range (20-22°C)
- **Action:** Set thermostat HVAC mode (state: off)

#### YAML example for turning off climate when comfortable

**\*\*automation\*\***

```
automation: |
 alias: "Turn off climate when living room is comfortable"
 triggers:
 - trigger: temperature.crossed_threshold
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: between
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
 actions:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.living_room
 data:
 hvac_mode: "off"
```

## Automation: alert when bedroom temperature enters comfort range

After opening windows to cool down a stuffy bedroom, this automation alerts you the moment the temperature enters your preferred comfort range so you can close the windows.

- **Trigger:** Temperature crossed threshold
- **Target:** Bedroom temperature sensor
- **Threshold type:** In range (20-22°C)
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

### YAML example for comfort range entry alert

**\*\*automation\*\***

```
automation: |
 alias: "Alert when bedroom temperature is comfortable"
 triggers:
 - trigger: temperature.crossed_threshold
 target:
 entity_id: sensor.bedroom_temperature
 options:
 threshold:
 type: between
 value_min:
 number: 20
 unit_of_measurement: "°C"
 value_max:
 number: 22
 unit_of_measurement: "°C"
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Temperature Alert"
 message: >-
 Bedroom temperature reached {{ trigger.to_state.state }}°C.
 You can now close the windows.
```

## Automation: prevent false triggers with a delay

To avoid false triggers from brief temperature fluctuations when opening a door or window, add a **For at least** delay. This automation only fires after the temperature has been below 18°C for 5 minutes.

- **Trigger:** Temperature crossed threshold
- **Target:** Living room temperature sensor
- **Threshold type:** Below (18°C)
- **For at least:** 5 minutes
- **Action:** Set thermostat HVAC mode (state: heat)

### YAML example with delay to prevent false triggers

**\*\*automation\*\***

```
automation: |
 alias: "Turn on heating when consistently cold"
 triggers:
 - trigger: temperature.crossed_threshold
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: below
 value:
 number: 18
 unit_of_measurement: "°C"
 for: "00:05:00"
 actions:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.living_room
 data:
 hvac_mode: heat
```

### Automation: trigger heating based on adjustable comfort temperature

Trigger the heating when temperature crosses below your personal comfort threshold. Use a number helper as the threshold so you can easily adjust it through the UI without editing the automation.

- **Trigger:** Temperature crossed threshold
- **Target:** Living room temperature sensor
- **Threshold type:** Below (entity: comfort temperature threshold)
- **Action:** Set thermostat HVAC mode (state: heat)

### YAML example for using a number helper as threshold

**\*\*automation\*\***

```
automation: |
 alias: "Turn on heating when crossing below comfort threshold"
 triggers:
 - trigger: temperature.crossed_threshold
 target:
 entity_id: sensor.living_room_temperature
 options:
 threshold:
 type: below
 value:
 entity: input_number.comfort_temperature_threshold
 actions:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.living_room
 data:
 hvac_mode: heat
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Vacuum.Docked

---

The **Vacuum returned to dock** trigger fires when your vacuum docks at its charging station. Use it to automate notifications or actions when cleaning has finished.

This is a good fit when you want to send a completion message, reset a cleaning status helper, or start another task only after the robot is safely back on the charger.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Vacuum: Vacuum returned to dock**.
5. Under **Targets**, choose a single vacuum, an area, floor, or multiple vacuums.
6. Under **Trigger when**, pick **Each**, **First**, or **All** to control how the trigger fires when more than one vacuum is targeted.
7. Under **For at least**, enter how long the vacuum must stay docked before the trigger fires.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When monitoring more than one vacuum, controls when the trigger fires. Pick **Each** to fire every time any targeted vacuum returns to dock, **First** to fire only on the first dock event, or **All** to fire only after all targeted vacuums have returned to dock. required: true For at least: description: The time the vacuum must remain docked before the trigger fires. required: false



## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `vacuum.docked`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: vacuum.docked
target:
 entity_id:
 - vacuum.upstairs
 - vacuum.downstairs
options:
 behavior: last
```

This example fires after both vacuums have docked.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls when the trigger fires. Options: `any` (every time any targeted vacuum docks), `first` (only when the first returns), or `last` (only after all have docked). required: true type: string default: any for: description: The time the vacuum must remain docked before the trigger fires. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- This trigger fires only on a real transition to docked.
- If a vacuum comes back online from `unavailable` or `unknown`, that does not count as docking.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: send a notification when cleaning is finished

When the vacuum docks, the cleaning run is usually complete. This automation sends a quick message so you know the robot is done and back on the charger.

- **Trigger:** Vacuum returned to dock
- **Target:** Downstairs vacuum
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a cleaning finished notification

**\*\*automation\*\***

```
automation: |
 alias: "Vacuum finished cleaning"
 triggers:
 - trigger: vacuum.docked
 target:
 entity_id: vacuum.downstairs
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Vacuum finished"
 message: "The downstairs vacuum returned to its dock."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Vacuum.Errorred

---

The **Vacuum encountered an error** trigger fires as soon as your vacuum reports an error state. You can use it to create alerts, notifications, or proactive automations when something interrupts a cleaning session.

This is useful when you want to know right away that the robot is tangled, blocked, out of water, or needs another kind of manual help before it can continue.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.



## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Vacuum: Vacuum encountered an error**.
5. Under **Targets**, pick the vacuum entities (or an area/floor) you want to monitor.
6. Under **Trigger when**, pick **Each**, **First**, or **All** to control group behavior.
7. Under **For at least**, enter how long the vacuum must remain in the error state before the trigger fires.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When more than one vacuum is targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted vacuum reports an error, **First** to fire only on the first error event, or **All** to fire only after all targeted vacuums have reported an error. required: true For at least: description: The time the vacuum must remain in the error state before the trigger fires. required: false

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `vacuum.errorred`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: vacuum.errorred
 target:
 entity_id:
 - vacuum.upstairs
 - vacuum.downstairs
 options:
 behavior: first
```

This fires after the first vacuum reports an error.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: The time the vacuum must remain in the error state before the trigger fires. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- This trigger fires only when a vacuum actually enters an error state.
- If a vacuum comes back online from `unavailable` or `unknown`, that does not count as an error event.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: alert when the vacuum reports an error

When the vacuum enters an error state, send a notification right away so you can clear the problem before the cleaning run is abandoned for the rest of the day.

- **Trigger:** Vacuum encountered an error
- **Target:** Upstairs vacuum
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a vacuum error alert

**\*\*automation\*\***

```
automation: |
 alias: "Vacuum error alert"
 triggers:
 - trigger: vacuum.errorred
 target:
 entity_id: vacuum.upstairs
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Vacuum error"
 message: "The upstairs vacuum reported an error."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---



## Triggers/Vacuum.Paused Cleaning

---

The **Vacuum cleaner paused cleaning** trigger fires when a vacuum interrupts its cleaning session by pausing. Use this to send reminders, alert for stuck devices, or to chain additional automations.

Use it to send a message when the robot needs help, turn on a nearby light so you can find it, or record how often it gets stuck in the same place.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview](#) feature.  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. In the **When** section, select **Add trigger**.
4. From the search box, search for and select **Vacuum: Vacuum cleaner paused cleaning**.
5. Under **Targets**, pick the vacuums, area, or group you want.
6. Under **Trigger when**, pick **Each**, **First**, or **All**.
7. Under **For at least**, enter how long the vacuum must stay paused before the trigger fires.
8. Save the automation.

### Options in the UI

{% options\_ui %} Trigger when: description: When monitoring more than one vacuum, controls when the trigger fires. Pick **Each** to fire every time any targeted vacuum pauses, **First** to fire only on the first pause event, or **All** to fire only after all targeted vacuums have paused. required: true  
For at least: description: The time the vacuum must remain paused before the trigger fires. required: false

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `vacuum.paused_cleaning`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: vacuum.paused_cleaning
 target:
 entity_id:
 - vacuum.upstairs
 - vacuum.downstairs
 options:
 behavior: first
```

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: The time the vacuum must remain paused before the trigger fires. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- This trigger fires only when a vacuum actually pauses.
- If a vacuum comes back online from `unavailable` or `unknown`, that does not count as a pause event.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: send a notification when the vacuum pauses

If the vacuum pauses unexpectedly, it may be stuck under furniture, wrapped in a cable, or waiting for you to empty the bin. This automation sends a phone alert so you can check on it.

- **Trigger:** Vacuum paused cleaning
- **Target:** Hallway vacuum
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a paused vacuum alert

**\*\*automation\*\***

```
automation: |
 alias: "Vacuum paused alert"
 triggers:
 - trigger: vacuum.paused_cleaning
 target:
 entity_id: vacuum.hallway
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Vacuum paused"
 message: "The hallway vacuum paused and may need attention."
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.



## Related triggers

---

These triggers work well alongside this one:

`{% endif %}`

---

## Triggers/Vacuum.Started Cleaning

---

The **Vacuum cleaner started cleaning** trigger fires when the vacuum begins a new cleaning run. Use it for automations that need to respond when cleaning starts, like announcements, status changes, or notifications.

If you want to mark the house as being cleaned, pause other noisy routines, or let someone know the robot has started, this trigger gives you a reliable starting point.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. In the **When** section, select **Add trigger**.
4. Search for **Vacuum: Vacuum cleaner started cleaning**.
5. Select targets (area, floor, or vacuums).
6. Pick **Trigger when: Each, First, or All** as needed.
7. Under **For at least**, enter how long the vacuum must keep cleaning before the trigger fires.
8. Save the automation.

### Options in the UI

{% options\_ui %} Trigger when: description: If targeting multiple vacuums, determines when the trigger fires. Pick **Each** to fire every time any targeted vacuum starts cleaning, **First** to fire only on the first start event, or **All** to fire only after all targeted vacuums have started cleaning. required: true For at least: description: The time the vacuum must keep cleaning before the trigger fires. required: false

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `vacuum.started_cleaning`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: vacuum.started_cleaning
 target:
 entity_id:
 - vacuum.upstairs
 - vacuum.downstairs
 options:
 behavior: first
```

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: The time the vacuum must keep cleaning before the trigger fires. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- This trigger fires only when cleaning actually starts.
- If a vacuum comes back online from `unavailable` or `unknown`, that does not count as starting a cleaning run.

## Try it yourself

---

Ready to test this? Go to **Settings > Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

---

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: mark cleaning as in progress

When the downstairs vacuum starts, turn on a helper that other automations can use to avoid interrupting the cleaning run.

- **Trigger:** Vacuum started cleaning
- **Target:** Downstairs vacuum
- **Action:** Turn on input boolean

#### YAML example for tracking an active cleaning run

**\*\*automation\*\***

```
automation: |
 alias: "Track active vacuum cleaning"
 triggers:
 - trigger: vacuum.started_cleaning
 target:
 entity_id: vacuum.downstairs
 actions:
 - action: input_boolean.turn_on
 target:
 entity_id: input_boolean.vacuum_cleaning
```



## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Vacuum.Started Returning

---

The **Vacuum cleaner started returning to dock** trigger fires when the vacuum finishes its current activity and sets out for its charging station. Use this to automate post-cleanup events, notifications, or dock preparation routines.

Use it to prepare for the robot to pass through a dark hallway, announce that cleaning is almost done, or turn off modes that only matter while active cleaning is in progress.



**Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).

Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open or create an automation.
3. In the **When** section, select **Add trigger**.
4. Search for **Vacuum: Vacuum cleaner started returning to dock**.
5. Select targets (individual/group, area, or floor).
6. Choose **Trigger when: Each, First, or All** as needed.
7. Under **For at least**, enter how long the vacuum must keep returning before the trigger fires.
8. Save the automation.

### Options in the UI

{% options\_ui %} Trigger when: description: When monitoring more than one vacuum, controls when the trigger fires. Pick **Each** to fire every time any targeted vacuum starts returning, **First** to fire only on the first return event, or **All** to fire only after all targeted vacuums have started returning. required: true For at least: description: The time the vacuum must keep returning before the trigger fires. required: false

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `vacuum.started_returning`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: vacuum.started_returning
 target:
 entity_id:
 - vacuum.upstairs
 - vacuum.downstairs
 options:
 behavior: first
```

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple vacuums are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: The time the vacuum must keep returning before the trigger fires. required: false type: time

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- This trigger fires only when a vacuum actually starts returning to its dock.
- If a vacuum comes back online from `unavailable` or `unknown`, that does not count as a return-to-dock event.

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.



## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn on the hallway light while the vacuum returns

If your vacuum docks in a darker part of the house, you can turn on a nearby light when it starts heading back so it can finish its route with a clear path.

- **Trigger:** Vacuum started returning to dock
- **Target:** Downstairs vacuum
- **Condition:** Sun is below horizon
- **Action:** Turn on hallway light

#### YAML example for lighting the path back to the dock

**\*\*automation\*\***

```
automation: |
 alias: "Light path for returning vacuum"
 triggers:
 - trigger: vacuum.started_returning
 target:
 entity_id: vacuum.downstairs
 conditions:
 - condition: sun
 after: sunset
 actions:
 - action: light.turn_on
 target:
 entity_id: light.hallway
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Window.Closed

---

The **Window closed** trigger fires when a targeted window closes. Use it to restore heating after airing out a room, confirm that windows are shut before bedtime, or start an automation only after a window has stayed closed for a while.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area the window is in, like your bedroom or kitchen. You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Window closed**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple windows are targeted.
7. Under **For at least**, set how long the window must stay closed before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple windows are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted window closes, **First** to fire only when the first targeted window closes, or **All** to fire only after every targeted window is closed. required: true For at least: description: How long the window must stay closed before the trigger fires. Set to zero to fire immediately. required: true

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `window.closed`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: window.closed
 target:
 entity_id: binary_sensor.bedroom_window
```

This fires when `binary_sensor.bedroom_window` closes.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple windows are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the window must stay closed before the trigger fires. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}

## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a window transitions from a known, valid state. If a window comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- This trigger works with binary sensors and covers that use the `window` device class.
- To react when a window opens instead, use [Window opened](#).



## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

💡 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: turn heating back on when bedroom window has been closed for 2 minutes

After you air out a room, it can help to wait until the window is fully closed before heating the room again.

- **Trigger:** Window closed
- **Target:** Bedroom window sensor
- **Trigger when:** Each
- **For at least:** 00:02:00
- **Action:** Climate: Set HVAC mode to heat

#### YAML example for restoring heating after a window closes

**\*\*automation\*\***

```
automation: |
 alias: "Resume bedroom heating when the window closes"
 triggers:
 - trigger: window.closed
 target:
 entity_id: binary_sensor.bedroom_window
 options:
 behavior: any
 for: "00:02:00"
 actions:
 - action: climate.set_hvac_mode
 target:
 entity_id: climate.bedroom
 data:
 hvac_mode: heat
```

## Automation: lock patio door after all ground floor windows are closed

When everyone finishes airing out the house, you can wait until every ground floor window is shut before locking the patio door.

- **Trigger:** Window closed
- **Target:** Ground-floor windows label
- **Trigger when:** All
- **For at least:** 00:01:00
- **Action:** Lock: Lock

### YAML example for locking a door after all windows close

**\*\*automation\*\***

```
automation: |
 alias: "Lock the patio door when all ground-floor windows are closed"
 triggers:
 - trigger: window.closed
 target:
 label_id: ground_floor_windows
 options:
 behavior: last
 for: "00:01:00"
 actions:
 - action: lock.lock
 target:
 entity_id: lock.patio_door
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---

## Triggers/Window.Opened

---

The **Window opened** trigger fires when a targeted window opens. Use it when you want Home Assistant to react right away, like sending an alert when a window opens after dark or pausing heating when fresh air starts coming in.

 **Labs:** Requires the **Purpose-specific triggers and conditions** [Labs preview feature](#).  
Enable it at **Settings > System > Labs**.

## Using this trigger from the user interface

---

If you prefer building automations visually, Home Assistant walks you through this trigger step by step. You pick what to watch, tweak a few options, and save. No YAML knowledge required.

To use this trigger in an automation:

1. Go to **Settings > Automations & scenes**.
2. Open an existing automation, or select **Create automation > Create new automation**.
3. In the **When** section, select **Add trigger**.
4. Select what you want to monitor. Under **By target** (see [Targets](#)), pick the area the window is in, like your bedroom or kitchen. You can also select a floor, a device, a specific entity, or a label.
5. From the triggers shown for that target, select **Window opened**.
6. Under **Trigger when** (see [Behavior](#)), pick **Each**, **First**, or **All** to control how the trigger behaves when multiple windows are targeted.
7. Under **For at least**, set how long the window must stay open before the trigger fires. Leave it at zero to fire immediately.
8. Select **Save**.

### Options in the UI

{% options\_ui %} Trigger when: description: When multiple windows are targeted, controls when the trigger fires. Pick **Each** to fire every time any targeted window opens, **First** to fire only when the first targeted window opens, or **All** to fire only after every targeted window is open. required: true For at least: description: How long the window must stay open before the trigger fires. Set to zero to fire immediately. required: true

## Using this trigger in YAML

---

If you work directly in YAML, or you want to know exactly what Home Assistant does under the hood, this section has the technical reference. It lists the field names you use in YAML, their types, and which ones are required.

In YAML, refer to this trigger as `window.opened`. A basic example looks like this:

### trigger

```
trigger: |
 trigger: window.opened
 target:
 entity_id: binary_sensor.kitchen_window
```

This fires when `binary_sensor.kitchen_window` opens.

## Options in YAML

YAML sometimes provides additional options for more complex use cases that are not available through the UI.

{% options\_yaml %} behavior: description: > When multiple windows are targeted, controls when the trigger fires. Accepts `any`, `first`, or `last`. required: true type: string default: any for: description: > Duration the window must stay open before the trigger fires. Accepts a duration string like `00:05:00` for five minutes. required: true type: string default: "00:00:00"

{%- assign target\_domain = include.domain | default: page.domain -%}



## Targets of the trigger

---

This trigger requires a target. The target is the object that Home Assistant will watch. You can select a single entity, a device, an area, a floor, or a label as a target, and Home Assistant will watch every matching entity behind that target.

- **Entity:** one specific entity, such as `.living_room`.
- **Device:** every entity that belongs to a device.
- **Area:** every entity in a room or area.
- **Floor:** every entity on a floor.
- **Label:** every entity that shares a label.

You can also select different target types in one trigger. For example, you can add a specific entity and an area as targets in the same trigger to monitor both of them at once.

## Behavior with multiple targets

When you target more than one entity (or select an area, floor, or label that contains several), the **Trigger when** option controls how the trigger responds:

- **Each** ( `any` in YAML, default): the trigger fires every time any one of the targeted entities transitions. For example, if you monitor three motion sensors in the living room and someone walks past sensor 1, the automation fires. When they walk past sensor 2 a moment later, it fires again. Every individual event counts.
- **First** ( `first` in YAML): the trigger fires only on the first transition in the targeted group, then waits until all targeted entities have reset before it fires again. For example, if you monitor the same three motion sensors, the automation fires when the first one picks up movement (someone entered the room). The other two firing afterward are ignored, so you get one notification per "someone walked in" event instead of three.
- **All** ( `last` in YAML): the trigger fires only after the last targeted entity in the group has fired, meaning all of them are now in the expected state. For example, if you monitor the lights in the living room, bedroom, and hallway, the automation fires only once all three have turned off. This is useful for scenarios like "start the robot vacuum only after every light on the floor is off," so you know the room is truly empty.

## Good to know

---

- The trigger only fires when a window transitions from a known, valid state. If a window comes back from being unavailable ( `unavailable` ) or having an unknown state ( `unknown` ), the trigger does not fire for that recovery.
- This trigger works with binary sensors and covers that use the `window` device class.
- To react when a window closes instead, use [Window closed](#).

## Try it yourself

---

Ready to test this? Go to **Settings** > **Automations & scenes**, create a new automation, and add this trigger. Save the automation, then change the state of the targeted entity to watch the trigger fire on your actual entities.

## More examples

Real scenarios where this trigger fires in automations and scripts. Copy any example and adapt it to your setup.

 **Tip:** You don't need to edit YAML to use these examples. Copy a YAML snippet from this page, open the automation editor in Home Assistant, and press `Ctrl+V` (or `Cmd+V` on Mac). Home Assistant automatically converts the pasted YAML into the visual editor format, whether it's a full automation, a single trigger, a condition, or an action.

### Automation: notify when kitchen window opens after sunset

If a kitchen window opens after dark, a quick notification can help you notice it before you lock up for the night.

- **Trigger:** Window opened
- **Target:** Kitchen window sensor
- **Trigger when:** Each
- **For at least:** 00:00:00
- **Condition:** Sun is below horizon
- **Action:** Send a notification message
- **Target:** My Device ( `notify.my_device` )

#### YAML example for a nighttime window-open notification

**\*\*automation\*\***

```
automation: |
 alias: "Notify when a kitchen window opens after sunset"
 triggers:
 - trigger: window.opened
 target:
 entity_id: binary_sensor.kitchen_window
 options:
 behavior: any
 for: "00:00:00"
 conditions:
 - condition: sun
 after: sunset
 actions:
 - action: notify.send_message
 target:
 entity_id: notify.my_device
 data:
 title: "Kitchen window opened"
 message: "The kitchen window was opened after sunset."
```

## Automation: pause cooling when skylight opens

When a motorized skylight opens, there is little point in keeping the air conditioning running. This automation turns cooling off after the skylight has been open for 2 minutes.

- **Trigger:** Window opened
- **Target:** Hallway skylight cover
- **Trigger when:** Each
- **For at least:** 00:02:00
- **Action:** Climate: Turn off

### YAML example for pausing cooling when a skylight opens

**\*\*automation\*\***

```
automation: |
 alias: "Turn off cooling when the skylight opens"
 triggers:
 - trigger: window.opened
 target:
 entity_id: cover.hallway_skylight
 options:
 behavior: any
 for: "00:02:00"
 actions:
 - action: climate.turn_off
 target:
 entity_id: climate.upstairs
```

## Still stuck?

---

The Home Assistant community is quick to help: join [Discord](#) for real-time chat, post on the [community forum](#) with the trigger you're using and what you expected to happen, or share on [our subreddit /r/homeassistant](#).

💡 **Tip:** AI assistants like ChatGPT or Claude can also explain triggers or suggest the right one when you describe what you want in plain language.

## Related triggers

---

These triggers work well alongside this one:

{% endif %}

---





## Cloud/Alexa

---

Home Assistant Cloud is a subscription service provided by our partner Nabu Casa, Inc. Check out [the Nabu Casa website](#) for help with the Amazon Alexa integration via Home Assistant Cloud.

Logo of Nabu Casa, Inc

---

## Cloud/Google Assistant

---

Home Assistant Cloud is a subscription service provided by our partner Nabu Casa, Inc. Check out [the Nabu Casa website](#) for help with the Google Assistant integration via Home Assistant Cloud.

Logo of Nabu Casa, Inc

---

## Cloud/Index

---

Home Assistant Cloud is an optional subscription service provided by our partner [Nabu Casa](#). It adds secure remote access to your Home Assistant from anywhere, integrations with Google Assistant and Amazon Alexa, and the text-to-speech and speech-to-text engines used by [Assist](#). Subscriptions also help fund the development of Home Assistant itself.

Home Assistant works fully without it. Home Assistant Cloud is there for the things that genuinely benefit from a cloud connection, such as remote access and the voice assistant integrations, while everything else continues to run locally on your own hardware.

For full details on features, pricing, and setup instructions, see the [Nabu Casa website](#).

[Logo of Nabu Casa, Inc](#)

---

## Cloud/Troubleshooting

---

Home Assistant Cloud is a subscription service provided by our partner Nabu Casa, Inc. Check out [their website](#) for help with troubleshooting.

Logo of Nabu Casa, Inc

---

## Common Tasks/Container

---

## Backup

---

To learn how to back up the system or how to restore a system from a backup, refer to the backup documentation under [common tasks](#).

# Update

---

## Prerequisites

1. [Back up your installation](#) and store the backup and the [backup emergency kit](#) somewhere safe.
2. This ensures that you can [restore your installation from backup](#) if needed.
3. Check the release notes for backward-incompatible changes on [Home Assistant release notes](#). Be sure to check all release notes between the version you are running and the one you are upgrading to. Use the search function in your browser ( `CTRL + f` / `CMD + f` ) and search for **Backward-incompatible changes**.

## To update Home Assistant Core

To update Home Assistant Core, choose one of the following options.

{% tabbed\_block %}

• title: Using the UI content: |

1. Open your Home Assistant UI.
2. Go to the **Settings** panel.
3. On the top you will be presented with an update notification.
  - Troubleshooting: If you do not see that notification:
  - In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
  - Go to **System > Updates**.
    - Select the update notification.
    - Select the cogwheel `mdi:cog-outline` , then set **Visible** to active.
4. Open the notification for the component you want to update.
5. If you want to backup the system first (recommended), enable the backup toggle.
6. Select **Update**.
7. After the update completed, check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

• title: Using the CLI content: |

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

{% endtabbed\_block %}

{% tabbed\_block %}

• title: Docker CLI content: |

**First start with pulling the new container.**

```
bash docker pull :stable
```

You then need to recreate the container with the new image.

- title: Docker Compose content: |

```
bash docker compose pull homeassistant docker compose up -d
```

{% endtabbed\_block %}

After the update, check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

## Running a beta version

If you would like to test next release before anyone else, you can install the beta version.

{% tabbed\_block %}

- title: From the UI content: |

1. In Home Assistant, go to **System > Updates**.
2. In the top-right corner, select the three dots `mdi:dots-vertical` menu.
3. Select **Join beta**.
4. Go to the **Configuration** panel.
5. Install the update that is presented to you.
6. Troubleshooting: If you do not see that notification:
  - In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
  - Go to **System > Updates**.
  - Select the update notification.
  - Select the cogwheel `mdi:cog-outline`, then set **Visible** to active.

- title: From the CLI content: |

1. Join the beta channel.

```
bash ha supervisor options --channel beta
```

2. Reload Home Assistant Supervisor.

```
bash ha supervisor reload
```

3. Update Home Assistant Core to the latest beta version.

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

{% endtabbed\_block %}

```
docker pull :beta
```

You then need to recreate the container with the new image.



## Running a development version

If you want to stay on the bleeding-edge Home Assistant Core development branch, you can upgrade to `dev`.

◆ **Caution:** The `dev` branch is likely to be unstable. Potential consequences include loss of data and instance corruption.

1. Join the dev channel.

```
bash ha supervisor options --channel dev
```

2. Reload the Home Assistant Supervisor.

```
bash ha supervisor reload
```

3. Update Home Assistant Core to the latest dev version.

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

```
docker pull :dev
```

You then need to recreate the container with the new image.

## Running a specific version

To see which version your system is running, go to **Settings > About**.

In the event that a Home Assistant Core version doesn't play well with your hardware setup, you can downgrade to a previous release. In this example `` is used as the target version but you can choose the version you desire to run.

To upgrade to a specific version, you can use the command line (CLI). The example below shows how to upgrade to ``. To learn how to get started with the command line in Home Assistant, refer to the [SSH app setup instructions](#).

```
ha core update --version `` --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade later.

To downgrade your installation, do a [partial restore of a backup](#) instead.

```
docker pull :
```

You then need to recreate the container with the new image.

## Configuration check

---

After changing configuration or automation files, check if the configuration is valid before restarting Home Assistant Core.

### Running a configuration check from the UI

1. Go to your user profile and enable **Advanced Mode**.
2. Go to **Settings > Developer tools > YAML** and in the **Configuration validation** section, select the **Check configuration** button.
3. This is to make sure there are no syntax errors before restarting Home Assistant.
4. It checks for valid YAML and valid config structures.
5. If you need to do a more comprehensive configuration check, [run the check from the CLI](#).

### Running a configuration check from the CLI

Use the following command to check if the configuration is valid. The command line configuration check validates the YAML files and checks for valid config structures, as well as some other elements.

```
ha core check
```

After changing configuration files, check if the configuration is valid before restarting Home Assistant Core.

*If your container name is something other than `homeassistant`, change that part in the examples below.*

Run the full check:

```
docker exec homeassistant python -m homeassistant --script check_config --config
```

Listing all loaded files:

```
docker exec homeassistant python -m homeassistant --script check_config --files
```

Viewing an integration's configuration ( `light` in this example):

```
docker exec homeassistant python -m homeassistant --script check_config --info light
```

Or all integrations' configuration

```
docker exec homeassistant python -m homeassistant --script check_config --info all
```

You can get help from the command line using:

```
docker exec homeassistant python -m homeassistant --script check_config --help
```

---

## Common Tasks/General

---

This section provides tasks that do not depend on a specific Home Assistant installation type or a specific integration. They may be referenced in other procedures.

# Backups

---

It is important to regularly back up your Home Assistant setup. You may have spent many hours configuring your system and creating automations. Keep your configurations safe so that you can [restore](#) a system or migrate your Home Assistant to new hardware.

Backups are encrypted and stored in a compressed archive file (.tar) and by default, stored locally in the `/backup` directory.

A full backup includes the following directories:

- `config`
- `share`
- `addons` (only manually installed or created apps, not those installed from the store)
- `ssl`
- `media`

A partial backup consists of any number of the above default directories and installed apps.

## Preparing for a backup

Before creating a backup, check if you can reduce the size of the backup. This is especially important if you want to use the backup to migrate the system to new hardware, for example from a Raspberry Pi Compute Module 4 to a Raspberry Pi Compute Module 5.

1. Check if your configuration directory contains a large database file:
2. Go to **Settings > System > Repairs**.
3. From the three dots `mdi:dots-vertical` menu, select **System information** and under the **Recorder** section, look for the **Estimated Database Size (MiB)**.
4. By default, the data is kept for 10 days. If you have modified that to a longer period, check the `recorder` integration page for options to keep your database data down to a size that won't cause issues.
5. Note the keep days, purge interval, and include/exclude options.
6. To check how much space you've used in total, go to **Settings > System > Repairs**.
7. From the three dots `mdi:dots-vertical` menu, select **System information**, and check under **Home Assistant Supervisor > Disk used**.
8. If you have apps installed that you no longer use, uninstall those apps. Some apps require quite a bit of space.
9. If you want to store the backup on your network storage instead of just locally on your system, follow the steps on [adding a new network storage](#) and select the **Backup** option.

## Setting up an automatic backup process

The automatic backup process creates a backup on a predefined schedule and also deletes old, redundant backups.

1. Go to **Settings > System > Backups**.

2. Under **Set up backups**, select **Set up backups**.
3. Download the emergency kit and store it somewhere safe.
4. You need it to restore encrypted backups.
5. To learn more about backup encryption, refer to the documentation on the [backup emergency kit](#).
6. Define the backup schedule.
7. It is recommended to back up **Daily**, but you can also choose to back up on specific days.
8. Define the time:
  - **System optimal** sets a time in a predefined time window as shown in the UI.
  - **Custom** lets you pick the time when you want the backup to start.
  - Make sure you pick a time when all your backup locations are up and running and available. Otherwise, the backup will fail for locations which are not available.
9. Define if you want to back up automatically before updating.
10. This sets a default. But you can change this setting each time before updating.
11. For large installations, backups might take a while.
12. Your update might start later than you expected.
13. Define how many backups you want to keep.
14. Older backups will be automatically deleted.
15. For example: if you back up daily, and select 7 backups, then the backup from 8 days ago and older will be deleted.
16. Define the data you want to back up.
17. It is recommended to disable media and the shared folder to reduce the size of the backup.
18. A large backup also takes longer to restore.
19. Some apps may also be quite large.
20. [Define the location for backups](#).

## Defining backup locations

You might need a backup in case your system has crashed. If you only store backups on the device itself, you won't be able to access them easily. It is recommended to keep a copy on another system (outside of Home Assistant) and ideally also one off-site.

 **Note:** You will find an overview of integrations which provide a backup location [here](#).

### About the backup storage on Home Assistant Cloud

If you have Home Assistant Cloud, you can store a backup of maximum 5 GB on Home Assistant Cloud. This cloud storage space is available for all existing and new Home Assistant Cloud subscribers without additional cost. It stores one backup file: the backup that was last saved to Home Assistant Cloud. These backups are always encrypted. To restore encrypted backups, you need the encryption key stored in the [backup emergency kit](#).

## To define the backup location for automatic backups

1. Go to **Settings > System > Backups** and under **Automatic backups**, select **Configure automatic backups**.
2. Under **Locations**, use the toggle to enable all the backup locations you want to use.
3. If you don't see Home Assistant Cloud in the list, you are not [logged in](#).
4. If you want to back up to your NAS (such as [Synology](#)) or a cloud provider (such as [Google Drive](#) or [Microsoft OneDrive](#)), check their integration documentation for specific instructions on setting up a Home Assistant backup.
5. If you don't see a network storage, you haven't added one. Follow the steps on [adding a new network storage](#) and select the **Backup** option. Define the backup locations
6. For each enabled location, select the cog `mdi:cog-outline` to enable/disable encryption.
7. **Info:** The backup stored on Home Assistant Cloud is always encrypted.

## Creating a backup automation using the backup action

If the backup automation settings provided in the UI do not match your use case, you can manually configure your own backup automation using the `backup.create_automatic` action.

Using the `developer_call_service` action in your own automation allows you to create automated backups on any schedule you like, or even add conditions and actions around it. For example, you could make an automation that triggers on a calendar, turns on your NAS, waits until it is online, and then triggers a backup.

## Creating a manual backup

This creates a backup instantly. You can create a manual backup at any time, irrespective of any automatic backups you may have defined.

1. Go to **Settings > System > Backups**.
2. In the lower-right corner, select **Backup now** and select **Manual backup**.
3. Define the data you want to back up.
4. It is recommended to disable media and the share folder to reduce the size of the backup.
5. A large backup also takes longer to restore.
6. Some apps may also be quite large.
7. Provide a name for the backup.
8. Choose the backup locations.
9. To learn more about the locations, refer to the section on [defining the backup location](#).
10. Download the [backup emergency kit](#) and store it somewhere safe. Make sure you take note of the backup name it belongs to.
11. To start the backup process, select **Create backup**.

## Downloading your local backups

When downloading the backup from the Home Assistant backup page, it is decrypted on the fly so that you can view the data using your favorite archive tool. This is done for all backup locations and also when you download from Home Assistant Cloud.

There are multiple ways to download your local backup from your Home Assistant instance and store it on another device:

**Option 1:** Download from the backup page:

1. Under **Settings** > **System** > **Backups**, select **Show all backups**.
2. To select multiple backups, select the `mdi:order-checkbox-ascending` button.
3. Select the three dots `mdi:dots-vertical` menu and select **Download backup**.
4. Result: The selected backup is stored in the **Downloads** folder of your computer.
5. If a backup is stored on multiple locations, you can select where you download it from:
6. Select the backup, and under **Locations**, select the three dots `mdi:dots-vertical` and select **Download from this location**.

**Option 2:** Copy backups from the backups folder:

1. If you haven't already done so, [configure access to files on Home Assistant](#), using one of the methods listed there.
2. For example, [use the samba app](#).
3. In your file explorer, access Home Assistant, open the `backup` folder and copy the file to your computer.

## Downloading a backup from Home Assistant Cloud

If you were logged in to Home Assistant Cloud and had Cloud backup enabled when creating a backup, your last backup is stored on Home Assistant Cloud.

There are two ways to download the backup from Home Assistant Cloud:

- **Option 1:** From the backups page
  - Go to **Settings** > **System** > **Backups** and select **Show all backups**.
  - Select the backup from the list.
  - Under **Locations**, select the three dots `mdi:dots-vertical` and select **Download from this location**.
- **Option 2:** From your Home Assistant Cloud account
  - Log in to your [Home Assistant Cloud account](#).
  - Under **Stored files**, you can see the latest available backup file. Select the **Download** button.

## Deleting obsolete backups

If you defined an automatic backup and backup purge schedule, old backups are deleted automatically. However, you might still have some old backups around that you want to delete.



To delete old backups, follow these steps:

1. In Home Assistant, under **Settings** > **System** > **Backups**, select **Show all backups**.
2. To **delete one backup**: from the list, select the backup of interest.
3. Select the three dots `mdi:dots-vertical` menu and select **Delete**.
4. To **delete multiple backups**: select the `mdi:order-checkbox-ascending` button.
5. From the list of backups, select all the ones you want to delete and select **Delete selected**.
6. `mdi:information-outline` Consider keeping at least one recent backup for recovery purposes.
7. To **delete a backup that is stored on Home Assistant Cloud**, you have 2 options:
8. **Option 1**: Trigger backup deletion from within Home Assistant
  - Follow steps 1 and 2 from above.
  - Even though you select **Delete** in Home Assistant, it will be deleted from Home Assistant Cloud storage.
9. **Option 2**: Delete the backup from the Nabu Casa account page.
  - Log in to your [Nabu Casa account](#).
  - Under **Backups**, delete the backup.

## Restoring a backup

There are two ways to use a backup:

- On your current system to recover your settings.
- During onboarding, to migrate your setup to a new device or to a device on which you performed a factory reset.

### Estimated duration

The time it takes to restore a backup depends on your installation. Home Assistant Core and all apps are being reinstalled. For a larger installation, this process can take about 45 minutes.

### Restoring a backup during onboarding

You can use a backup during the onboarding process to restore your configuration.

**Migration:** This procedure also works if you want to migrate from one device to another. In that case, use the backup of the old device on the new device. The target device can be a different device type. For example, you can migrate from a Raspberry Pi to another device.

### Prerequisites

- This procedure assumes you have already completed the [installation](#) procedure on your target device and are now viewing the welcome screen as part of the [onboarding](#).
- The login credentials of the device from which you made the backup.
- The [backup emergency kit](#) that contains the key needed to decrypt the backup.
- **Required storage capacity:** If you migrate the installation to a new device, make sure the new device has more storage capacity than the existing device.

- Before migrating, on the old system, check how much storage you used.
- Go to **Settings > System > Repairs > ... > System Information**, and under **Home Assistant Supervisor**, look at the **Disk used** value.
  - The target device must have more free space than the source device.
  - If your target device is a Home Assistant Yellow, note that it is the size of the eMMC that is relevant.
  - The restore process mainly uses the eMMC, not the NVMe.
  - The size of the backup file is no indication of the size of your installation. To know the size of your installation, you need to check the **Disk used** value mentioned above.
- If you are migrating to a new device:
- You do not need to transfer the backup to a USB or SD card to bring it to your device.
- You will be able to upload the backup file from the device you are accessing the onboarding from.

#### To restore a backup during onboarding

1. If you are migrating to a new device and you had controllers or radios connected (such as a Z-Wave stick or a Connect ZBT-2):
2. make sure to plug them into the new device.
3. You can either restore a backup from your local machine or a backup stored on Home Assistant Cloud:
4. **Option 1:** restoring from a local backup.
  - On the welcome screen, select **Upload backup**.
  - Select **Select backup file**.
  - The file explorer opens on the device on which you are viewing the Home Assistant User interface.
  - You can access any connected network drive from there.
  - Select the backup file.
5. **Option 2:** restoring from a Home Assistant Cloud backup.
  - On the welcome screen, select **Home Assistant Cloud**.
  - Sign in to Home Assistant Cloud.
6. In the dialog, select all the parts you want to restore.
  - Your current system will be overwritten with the parts that you choose to restore.
  - If you want to restore the complete configuration with all directories and apps, select everything.
7. Enter the encryption key stored in the [backup emergency kit](#).
8. To start the process, select **Restore backup**.
9. The restore may take a while, depending on the amount of data.
10. Don't refresh the page. Just wait.
  - If you refresh the page, you will see a "Not found" message. This is because the system is shutdown, wiped, and reinstalled from the backup. During that time, it won't be reachable.

11. If your previous installation had certificates enabled directly for the `http` integration, when the restore is complete, it will no longer respond to `http://` requests. In this case, use `https://` (added `s`) instead.
12. On the login screen, enter the credentials of the system from which you took the backup.
13. The login password and username must match the ones you used at the time the backup was taken.
14. Your dashboard should show all the elements as they were when you created the backup.
15. If some devices are shown as unavailable, you may need to wake the battery-powered devices.
16. If you had `network storage` connected on the previous system, you may need to reconnect those.
17. If you had Zigbee devices, and you migrated to a new device with its own Zigbee radio built-in:
18. Because this is now a different Zigbee radio, you need to [migrate Zigbee](#).

### To restore a backup on your current system

1. Go to **Settings > System > Backups**.
2. From the list of backups, select the backup from which you want to restore.
3. Select what to restore:
4. Your current system will be overwritten with the parts that you choose to restore.
5. If you want to restore the complete configuration with all directories and apps, select everything.
6. If you only want to restore specific elements, only select the folders and apps you want to restore.
7. Select **Restore**.
8. This may take a while, depending on how much there is to compress or decompress.
9. Once the restore is complete, Home Assistant restarts to apply the new settings.
10. You will lose the connection to the UI and it will return once the restart is completed.
11. On the login screen, enter the password and username as they were at the time the backup was taken.

## Enabling or disabling entities

---

Some entities are disabled by default. Whether a particular entity of a device is disabled or enabled by default, depends on the integration. Diagnostic entities for example are often disabled by default so that they don't clutter Home Assistant. For example, the RSSI entity (representing the RF signal strength) provided by the ZHA integration for each Zigbee device is disabled by default.

There are different ways to enable entities. You can enable a single entity in the entity settings, or you can enable multiple entities at once from the list of entities.

### To enable or disable a single entity

1. Go to **Settings > Devices & services** and select your integration card.
2. Select the device.
3. To see all the entities, you may need to expand the **entity not shown** section.
4. Select the entity of interest, select the cogwheel `mdi:cog-outline`, then select the toggle for **Enable**.
5. Select **Update**.
6. Confirm the notification stating that it will take about 30 seconds for the entity to become enabled. Select **OK**.

Screencast showing how to enable a single entity

### To enable or disable multiple entities

1. In Home Assistant, open the table of interest.
2. To enable or disable entities, go to **Settings > Devices & services > Entities**.
3. To enable or disable automations, go to **Settings > Automations & Scenes**.
4. [Enable multiselect](#) and select all the entities you want to enable or disable.
5. In the top right corner, select the three dots `mdi:dots-vertical` menu, then select **Enable** or **Disable**.

Screenshot showing how to enable or disable multiple automations

### Related topics

- [Customizing entities](#)
- [Grouping your assets](#)
- [Working with tables](#)

## Defining a custom polling interval

---

If you want to define a specific interval at which your device is being polled for data, you can disable the default polling interval and create your own polling automation.

### What is data polling?

Data polling is the process of querying a device or service at regular intervals to check for updates or retrieve data. By defining a custom polling interval, you can control how frequently your system checks for new data, which can help optimize performance and reduce unnecessary network traffic.

### Why use an automation instead of changing the integration's polling configuration?

Creating an automation for polling gives you more flexibility on when to poll:

1. Not all integrations have a configurable polling interval. The `homeassistant.update_entity` service, on the other hand, works with most of the integrations; no code changes are required.
2. An automation allows you to poll whenever you want. For example, if you have a rate-limited solar panel provider with a maximum number of requests per day, you may want to lower/stop the polling at night but poll more frequently during the day.

If you want to define a specific interval at which your device is being polled for data, you can disable the default polling interval and create your own polling automation.

To add the automation:

1. Go to **Settings > Devices & services**, and select your integration.
2. On the integration entry, select the `mdi:dots-vertical`.
3. Then, select **System options** and toggle the button to disable polling. Disable polling for updates
4. To define your custom polling interval, create an automation.
5. Go to **Settings > Automations & scenes** and create a new automation.
6. Define any trigger and condition you like.
7. Select **Add action**, then select **Other actions**.
8. Select **Perform action**, and from the list, select the `homeassistant.update_entity` action.
9. Add the entities you want to poll to the **Entity** field. The `homeassistant.update_entity` action only supports targeting by entity. Selecting an area, device, or label is not supported.  
Update entity
10. Save your new automation to poll for data.

## Removing an integration instance

---

If you no longer want to use a device or service in Home Assistant, you can remove the integration instance including the device or service with all its entities.

The following steps describe the general steps needed to remove an integration instance. Depending on the integration, additional steps can be needed, such as resetting the device or to delete credentials. Refer to the integration documentation to see if additional steps are needed.

### To remove an integration instance from Home Assistant

1. Go to **Settings** > **Devices & services** and select the integration card.
2. From the list of devices, select the integration instance you want to remove.
3. Next to the entry, select the three dots `mdi:dots-vertical` menu. Then, select **Delete**.

Screenshot showing how to remove an integration instance

---

## Common Tasks/Os

---

This section will provide guides to some common tasks and information which you will need in order to run, maintain, and edit your Home Assistant OS system. For further details on any particular subject, make sure to refer to the documentation for specific apps (formerly known as add-ons) or topics listed here.

## Configuring access to files

---

Your Home Assistant Operating server includes two repositories by default: The official core app repository, and the community app repository. All of the apps mentioned here can be installed by navigating to the app store using **Settings > Apps > Install app** in the UI.

One of the first things to take care of after installing Home Assistant OS is to provide yourself access to files. There are several apps commonly used for this, and most users employ a mix of various apps. Default directories on the host are mapped to the apps so that they can be accessed by the services any particular app might provide. On the host system these directories exist on the `/data` partition at `/mnt/data/supervisor/`.

Using any of the apps listed below, the following directories are made available for access:

- `addons`
- `backup`
- `config`
- `media`
- `share`
- `ssl`

---

## Installing and using the Samba app

The **Samba** app allows you to share the directories on Home Assistant with other systems on your network. After installing the app, you can then also edit files using the editor of your preference from your client computer. This app can be installed from the app store's official repository.

To install the app, follow these steps:

1. Go to **Settings > Apps > Samba share** and select **Install**.
2. On the **Configuration** tab, define **Username** and **Password**, store them in a safe place, and save your changes.
3. You can specify any username and password.
4. They are not related to the login credentials you use to log in to Home Assistant or to log in to the computer from which you are accessing the files.
5. The app won't start if username and password are not defined.
6. For further configuration information, refer to the **Documentation** tab.
7. To start the app, on the **Information** tab, select **Start**.

To access the Home Assistant directories from the other device, follow these steps:

1. Go to **Settings > System > Network** and take note of the **Host name**.
2. Alternatively, you can look up the host name or IP address of your Home Assistant on your router.
3. How you connect from another device to Home Assistant depends on your system. Use one of the following options:



4. **On Windows:** Open **File Explorer** and in the address bar, enter the IP address or hostname with two backslashes as `\\your.ha.ip.address` or `\\hostname`.

Screenshot of File Explorer displaying the navigation to a file share using an IP address  
Screenshot of File Explorer displaying the navigation to a file share using an IP address

5. **On OS X:** Open **Finder** and select **Go > Connect to Server...** and enter the IP address or hostname as `smb://your.ha.ip.address` or `smb://hostname`.
6. **On Ubuntu:** Open **Files** and in the address bar, enter the IP address or hostname as `smb://your.ha.ip.address` or `smb://hostname`.
7. Enter the credentials you entered in the **Samba** app configuration.
8. You also have the option of having the credentials stored so that you do not need to enter them again.
9. Done! You now have access to the directories which you can then mount as a drive or pin to Quick Access.

---

## Installing and using the Visual Studio Code (VSC) app

The **Studio Code Server** app provides access through a feature-packed web-based version of the Visual Studio Code editor. It currently only supports AMD64 and aarch64/ARM64 machines. The app also provides access to the Home Assistant Command Line Interface (CLI) using VSC's built-in terminal, which allows for checking logs, stopping, and starting Home Assistant and apps, creating/restoring backups, and more. (See [Home Assistant via Command Line](#) for further info).

Screenshot of an example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation. Example of a configuration.yaml file, accessed using the Studio Code Server app on a Home Assistant Operating System installation.

To install and use the **Studio Code Server** in Home Assistant, follow these steps:

1. To install the app, go to **Settings > Apps > Studio Code Server** and install the app.
2. Once you have the app installed, if you want, select the **Show in sidebar** option. Then, select **Start**.
3. For information on configuration settings, open the **Documentation** tab.
4. To start browsing, on the **Info** tab, select **Open Web UI**.

---

## Installing and using the File Editor app

The **File Editor** app is a web-based file system browser and text editor. It is a more basic and light weight alternative to Visual Studio Code. YAML files are automatically checked for syntax errors while editing.

Screenshot of an example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation. Example of a configuration.yaml file, accessed using the File editor app on a Home Assistant Operating System installation.

To install and use the File Editor in Home Assistant, follow these steps:

1. To install the app, go to **Settings > Apps** and select **Install app**.
  2. Search for **File editor**, select it and then select **Install**.
  3. Once you have the app installed, you can edit files within your `/config` directory.
  4. If you want to be able to access directories outside the `/config` directory, you need to disable an option in the app configuration.
  5. Go to the app under **Settings > Apps > File editor**.
  6. Open the **Configuration** tab.
  7. In the configuration settings, disable the **Enforce basepath** option.
  8. Note: The **Enforce basepath** option is intended to protect you from inadvertently making changes to settings files.
  9. For information on other configuration settings, open the **Documentation** tab.
  10. To confirm your changes, select **Save**.
  11. To start browsing, on the **Info** tab, select **Open Web UI**.
- 

## Installing and using the SSH app

If you want to use the Home Assistant command line or an SSH client, you can do this through the **Terminal & SSH** app.

The **Terminal & SSH** app provides the following functionalities:

- It provides a web terminal that you can access from the Home Assistant user interface.
- It allows you to use the Home Assistant Command Line Interface (CLI) which provides custom commands for checking logs, stopping and starting Home Assistant and apps, creating/restoring backups, and more.
- For a list of command line commands, refer to [Home Assistant via Command Line](#).
- It allows connecting to your system using an SSH client.
- It also includes common tools like nano and vi editors.
- The Terminal & SSH app does **not provide** access to the underlying host file system.

To get started with the **Terminal & SSH** app, follow these steps:

1. In the bottom left, select your user to open the **Profile** page. Make sure **Advanced Mode** is enabled.
2. To install the app, go to the app store under **Settings > Apps** and install the **Terminal & SSH** app.
3. To use the web terminal, **start** the app, then select **Open Web UI**.
4. You can now start typing your [commands](#).
5. If you want to access from an ssh client, you need to enter credentials:
6. Open the **Configuration** page.
7. Enter a password or authorized Keys.
8. Then save and start the app.

# Backup

---

To learn how to back up the system or how to restore a system from a backup, refer to the backup documentation under [common tasks](#).

## Alternative: Creating a backup using the Home Assistant Command Line Interface

In general, to create or restore from a backup, follow the steps described under [common tasks](#). However, If you have the Home Assistant Operating System installed, you can also create a backup from the command line. Follow these steps:

1. `ha backups list` - lists backups and their slugnames
2. `ha backups restore slugname` - restores a specific backup
3. `ha backups new --name nameofbackup` - create a backup

For additional information about command line usage, use the `ha help` command or refer to the [Home Assistant Command Line documentation](#).

# Updating Home Assistant

---

If you have the Home Assistant Operating System installed, you receive update notifications from different components:

- Home Assistant Operating System
- Home Assistant Supervisor
- Home Assistant Core
- Apps, if you have any installed

Each of these components needs to be updated separately.

## Updating the Home Assistant Operating System

Updates of the Home Assistant Operating System are independent of other updates. They do not trigger repair issues and are usually backward-compatible.

### Prerequisites

- [Backup your installation](#).
- Make sure the backup is stored on a [backup location](#) outside of the device where Home Assistant is installed.
  - For example, if Home Assistant is installed on [Home Assistant Green](#), make sure it is stored on [Home Assistant Cloud](#) or another location.
- So that you can [restore from that backup](#) in case there is an issue with the system.

### To update the Home Assistant Operating System

{% tabbed\_block %}

- title: Using the UI content: |
  1. Open the **Settings** panel.
  2. On the top you will be presented with an update notification.
  3. Troubleshooting: If you do not see that notification:
    - In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
    - Go to **System > Updates**.
    - Select the update notification.
    - Select the cogwheel `mdi:cog-outline`, then set **Visible** to active.
  4. Open the notification for the component you want to update.
  5. If you want to update the system first (recommended), enable the backup toggle.
  6. Select **Update**.
  7. Check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

- title: Using the CLI content: |

```
bash ha os update
```

*This updates to the latest version. If you want to update to a specific version instead, use `ha os update --version 15.2`.*

{% endtabbed\_block %}

#### Advanced: changing the boot slot used during the update

#### About boot slots used during the update The Home Assistant Operating System uses two boot slots. On first installation, boot slot A is used. After that, on each Operating System update, the other boot slot is updated and reboot is triggered. On that reboot, the system boots from the other boot slot (A → B → A,...). When booting fails, the system automatically uses the previous boot slot, so that it boots from the last working operating system. #### Changing the boot slot used You can manually define that the previous boot slot is used. This can be useful in cases where the system boots but something still seems wrong. For example, when the device is no longer correctly detected or you see another issue that might be related to the latest update of the operating system. 1. To check which boot slot is currently in use and what OS versions are installed in the individual slots, in the Home Assistant command line, enter the following command: `bash ha os info` 2. To change the boot slot, enter the following command: `bash ha os boot-slot other` - This will boot into the other (previous) OS version. Alternatively, if the Operating Systems runs on a platform that uses the GRUB bootloader, a boot menu is presented early in the boot. The alternative boot slot can be selected here, marking it active for future boots if the following boot attempt is successful.

## Updating Home Assistant Core

### Prerequisites

1. [Back up your installation](#) and store the backup and the [backup emergency kit](#) somewhere safe.
2. This ensures that you can [restore your installation from backup](#) if needed.
3. Check the release notes for backward-incompatible changes on [Home Assistant release notes](#). Be sure to check all release notes between the version you are running and the one you are upgrading to. Use the search function in your browser ( `CTRL + f` / `CMD + f` ) and search for **Backward-incompatible changes**.

### To update Home Assistant Core

To update Home Assistant Core, choose one of the following options.

{% tabbed\_block %}

- title: Using the UI content: |

1. Open your Home Assistant UI.
2. Go to the **Settings** panel.
3. On the top you will be presented with an update notification.
  - Troubleshooting: If you do not see that notification:

- In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.

- Go to **System > Updates**.

- Select the update notification.
- Select the cogwheel `mdi:cog-outline`, then set **Visible** to active.

4. Open the notification for the component you want to update.

5. If you want to backup the system first (recommended), enable the backup toggle.

6. Select **Update**.

7. After the update completed, check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

- title: Using the CLI content: |

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

{% endtabbed\_block %}

{% tabbed\_block %}

- title: Docker CLI content: |

**First start with pulling the new container.**

```
bash docker pull :stable
```

**You then need to recreate the container with the new image.**

- title: Docker Compose content: |

```
bash docker compose pull homeassistant docker compose up -d
```

{% endtabbed\_block %}

After the update, check if there are any repair issues and check the logs to see if there are any issues with your configuration that need to be addressed.

## Network storage

You can configure both Network File System (NFS) and Samba/Windows (CIFS) targets to be used within Home Assistant and apps. To list all your currently connected network storages, go to **Settings > System > Storage** in the UI.

**⚠ Important:** You need to update to Home Assistant Operating System 10.2 before you can use this feature.

Screenshot of the list of network shares inside the storage panel.

### Add a new network storage

1. Go to **Settings > System > Storage** in the UI.
2. Select **Add network storage**.
3. Fill out all the information for your network storage.
4. Select **Connect**.

Screenshot of connecting a new network storage.

### Network storage configuration

{% configuration\_basic "hassio.network\_share" %} Name: description: This is the name that will be used for the mounted directory on your system. Usage: description: Here, you select how the target should be used. [See usage types below](#) Server: description: The IP/hostname of the server running NFS/CIFS. "Protocol"<sup>3</sup>: description: The service the server is using for the network storage. "[NFS]"<sup>1</sup> Remote share path": description: The path used to connect to the remote storage server. "[CIFS]"<sup>2</sup> Username": description: "The username to use when connecting to the storage server. Use User Principal Name for domain accounts. For example: `user@domain.com`." "[CIFS]"<sup>2</sup> Password": description: The password to use when connecting to the storage server. "[CIFS]"<sup>2</sup> Share": description: The share to connect to on the storage server.

<sup>1</sup> Options prefixed with `[NFS]` are only available for NFS targets.

<sup>2</sup> Options prefixed with `[CIFS]` are only available for CIFS targets.

<sup>3</sup> For the `CIFS` option, only version 2.1+ is supported.

### Usage types

{% configuration\_basic "hassio.network\_share.usage" %} Backup: description: This will become a target. You can use it when creating an automatic or manual backup. The first storage you add of this type becomes your new default target. If you want to change the default target, [check out the documentation below](#). Media: description: A new directory with the name you gave your network storage will be created under `/media`. This directory can be accessed by Home Assistant and apps. Share: description: A new directory with the name you gave your network storage will be created under `/share`. This directory can be accessed by Home Assistant and apps.

## Change default local backup location

By default, the first network storage of type **Backup** that you add is used as your local default backup location.

If you want to change the local network storage that is used to store your backups, follow these steps:

1. Go to **Settings > System > Backups**.
2. Select **Settings and history**.
3. In the top-right corner, select the three dots `mdi:dots-vertical` menu and select **Change default action location**.
4. Select your preferred network location and save your changes. Select default location used for local backup
5. Troubleshooting: Don't see your external storage location? This list contains only the network storage targets you have added of type **Backup**.



## Lost Password and password reset

---

Please refer to the [I'm locked out!](#) documentation page.

## Installing a third-party app repository

Home Assistant allows anyone to create an app repository to share their own apps with the community.

 **Warning:** Home Assistant cannot guarantee the quality or security of third-party apps. Use at your own risk.

To add an app repository, follow these steps:

1. Copy the URL of the repository.
2. The URL is the git repository clone URL (on GitHub, use the Code button and copy the HTTPS URL).
3. This documentation uses an example app repository. It is not practically useful but follows the same steps.
4. If you are interested in app development, refer to our [Home Assistant app development documentation](#).

```
```text
https://github.com/home-assistant/hassio-addons-example
```
```

1. Go to **Settings > Apps** and select **App store**. Screenshot of the Home Assistant app store
2. In the top-right corner, select the three dots `mdi:dots-vertical` menu, and select **Repositories**.
3. Add the URL of the repository and select **Add**. Screenshot of the app store
4. Result: A new card for the third-party repository will appear. Screenshot of the app store

## Troubleshooting: Repository is not showing up

If you have added an app repository, but it's not showing up, make sure to refresh your browser. If it still doesn't show up, the app repository may contain invalid configuration data.

1. Go to **Settings > System > Logs** and select Supervisor in the top right corner to get the Supervisor log.
2. It should tell you what went wrong.
3. Report this information to the app repository author.

## Configuration check

---

After changing configuration or automation files, check if the configuration is valid before restarting Home Assistant Core.

### Running a configuration check from the UI

1. Go to your user profile and enable **Advanced Mode**.
2. Go to **Settings > Developer tools > YAML** and in the **Configuration validation** section, select the **Check configuration** button.
3. This is to make sure there are no syntax errors before restarting Home Assistant.
4. It checks for valid YAML and valid config structures.
5. If you need to do a more comprehensive configuration check, [run the check from the CLI](#).

### Running a configuration check from the CLI

Use the following command to check if the configuration is valid. The command line configuration check validates the YAML files and checks for valid config structures, as well as some other elements.

```
ha core check
```

After changing configuration files, check if the configuration is valid before restarting Home Assistant Core.

*If your container name is something other than `homeassistant`, change that part in the examples below.*

Run the full check:

```
docker exec homeassistant python -m homeassistant --script check_config --config
```

Listing all loaded files:

```
docker exec homeassistant python -m homeassistant --script check_config --files
```

Viewing an integration's configuration ( `light` in this example):

```
docker exec homeassistant python -m homeassistant --script check_config --info light
```

Or all integrations' configuration

```
docker exec homeassistant python -m homeassistant --script check_config --info all
```

You can get help from the command line using:

```
docker exec homeassistant python -m homeassistant --script check_config --help
```

# Home Assistant versions

---

To see which version your system is running, go to **Settings > About**.

## Feature preview

If you want to preview upcoming features, you can enable preview under **Settings > System > Labs**.

**Labs** allows you to preview selected features that are stable but are still being fine-tuned. Preview is different from installing a beta or development version, which are used for development and testing.

For more information, refer to the [Labs documentation](#).

## Running a beta version

If you would like to test next release before anyone else, you can install the beta version.

{% tabbed\_block %}

• title: From the UI content: |

1. In Home Assistant, go to **System > Updates**.
2. In the top-right corner, select the three dots `mdi:dots-vertical` menu.
3. Select **Join beta**.
4. Go to the **Configuration** panel.
5. Install the update that is presented to you.
6. Troubleshooting: If you do not see that notification:
  - In the top right corner, select the three dots `mdi:dots-vertical` menu and select **Check for updates**.
  - Go to **System > Updates**.
  - Select the update notification.
  - Select the cogwheel `mdi:cog-outline`, then set **Visible** to active.

• title: From the CLI content: |

1. Join the beta channel.

```
bash ha supervisor options --channel beta
```

2. Reload Home Assistant Supervisor.

```
bash ha supervisor reload
```

3. Update Home Assistant Core to the latest beta version.

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

{% endtabbed\_block %}

```
docker pull :beta
```

You then need to recreate the container with the new image.

## Running a development version

If you want to stay on the bleeding-edge Home Assistant Core development branch, you can upgrade to `dev`.

◆ **Caution:** The `dev` branch is likely to be unstable. Potential consequences include loss of data and instance corruption.

1. Join the dev channel.

```
bash ha supervisor options --channel dev
```

2. Reload the Home Assistant Supervisor.

```
bash ha supervisor reload
```

3. Update Home Assistant Core to the latest dev version.

```
bash ha core update --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade.

```
docker pull :dev
```

You then need to recreate the container with the new image.

## Running a specific version

To see which version your system is running, go to **Settings > About**.

In the event that a Home Assistant Core version doesn't play well with your hardware setup, you can downgrade to a previous release. In this example `` is used as the target version but you can choose the version you desire to run.

To upgrade to a specific version, you can use the command line (CLI). The example below shows how to upgrade to ``. To learn how to get started with the command line in Home Assistant, refer to the [SSH app setup instructions](#).

```
ha core update --version `` --backup
```

The `--backup` flag here ensures that you have a partial backup of your current setup in case you need to downgrade later.

To downgrade your installation, do a [partial restore of a backup](#) instead.

```
docker pull :
```

**You then need to recreate the container with the new image.**

## Using external data disk

---

Home Assistant Operating System supports storing data on a secondary storage medium. For example, this can be a second internal SSD or HDD, or a USB-attached SSD or HDD. This data disk contains not only user data but also most of the Home Assistant software, including Home Assistant Core and Apps. This means a fast data disk will make the system much faster overall.

Graphics showing the architecture of the data disk feature

The data disk feature can be used on an existing installation without losing data: The system will move existing data to the external data disk automatically. However, it is recommended to create and download a full [Backup](#) before proceeding!

◆ **Caution:** All data on the target disk will be overwritten!

⚠ **Important:** The storage capacity of the external data disk must be larger than the storage capacity of the existing (boot) disk.

⚠ **Important:** If you have been using a data disk previously with Home Assistant Operating System, you need to use your host computer to delete all partitions *before* using it as a data disk again.

## Using UI to move the data partition

1. Connect the data disk to your system.
2. Go to **Settings > System > Storage** in the UI.
3. Select **Move data disk**.
4. Select the data disk from the list of available devices.
5. Select **Move**.
6. Depending on the amount of data, this may take a while.

## Using CLI to move the data partition

To see the current data disk use:

```
$ ha os info
...
data_disk: /dev/mmcblk1p4
...
```

To get a list of potential targets which can be used by `datadisk` :

```
ha os datadisk list
```

To initiate the move to the new data disk use the `move` command:



```
ha os datadisk move /dev/sdx
```

The system will prepare the data disk and immediately reboot. The reboot will take 10 minutes or more depending on the speed of the new data disk; please be patient!

 **Warning:** Using an USB attached SSD can draw quite some power. For instance on Raspberry Pi 3 the official Raspberry Pi power supply (PSU) only provides 2.5A which can be too tight. Use a more powerful power supply if you experience issues. Alternatively use a powered USB hub. Connect the Hub to one of the USB slots of your Raspberry Pi, and connect the SSD to the Hub. In this setup the power supply of the Hub will power the attached device(s).

## Migrating an external data disk to another system

This section shows how to move an external data disk from one system to another. This can be an option if the following elements apply to your use case:

- You already have a functioning Home Assistant instance (system 1) that is using an external data disk.
- You have another, new, Home Assistant instance (system 2).
- You now want to use the data disk of system 1 on system 2 instead.

The aim is to migrate the data from system 1 to system 2. One way to do this is by [restoring a backup](#). The other way is to move the data disk. This can be an interesting option if you have a large amount of data on your external disk or if your external disk has more storage capacity than your new system.

### Prerequisites

- A Home Assistant instance using an external data disk (system 1)
- A Home Assistant instance to which you want to move the external data disk (system 2)

### To migrate an external data disk to another system

To migrate an external data disk from one system to another, follow these steps:

1. [Create a backup](#) of both systems and store these backups on another system (not strictly necessary, but recommended just in case, at least for the important data).
2. Shut down system 1 and remove the data disk.
3. Make sure system 2 has Home Assistant OS installed, and Home Assistant is up and running. Home Assistant is using the data disk (partition) on the boot drive, such as the SD card, at this point.
4. Make sure system 2 has completed the basic [onboarding](#) steps, including the last steps where devices are discovered automatically.
5. Plug the external disk into system 2 and go to the **Settings > System**. Select the three dots `mdi:dots-vertical` menu, and **Restart Home Assistant > Reboot system**. Result: A repair issue is displayed **Multiple data disks detected**.

6. The repair issue comes up because system 2 now sees two file systems with an identical name. During a reboot, there is a name conflict with the existing data disk as it is undefined which file system should be used. This can lead to a random selection of the system you end up with. Hence you must make a decision.
7. Open the repair issue and choose one of the options:
8. Select **Use the detected data disk instead of the current system**.
  - This will override the current file system of system 2 and use your external data disk instead.
  - You won't have access anymore to the current Home Assistant data. It will be marked as inactive data disk.
9. If you changed your mind about using the external data disk:
  - Unplug the external data disk.
  - If you cannot unplug the external data disk for some reason, select **Mark as inactive data disk (rename file system)**.
  - This makes sure that there is no name conflict after rebooting.
  - It also means you cannot use the file system that is stored on your external disk.
  - You keep using the current file system of system 1.

## Home Assistant via the command line

---

On the [SSH command line](#), you can use the `ha` command to retrieve logs, check the details of connected hardware, and more.

### Home Assistant

```
ha core check
ha core info
ha core logs
ha core options
ha core rebuild
ha core restart
ha core restart --safe-mode
ha core start
ha core stats
ha core stop
ha core update
```

### Supervisor

```
ha supervisor info
ha supervisor logs
ha supervisor reload
ha supervisor update
```

### Host

```
ha host reboot
ha host shutdown
ha host update
```

### Hardware

```
ha hardware info
ha hardware audio
```

### Usage examples

To update Home Assistant to a specific version, use the command:

```
ha core update --version x.y.z
```

Replace x.y.z with the desired version like `--version`

You can get a better description of the CLI capabilities by typing `ha help`:

```
The Home Assistant CLI is a small and simple command line utility that allows
you to control and configure different aspects of Home Assistant
```

```
Usage:
 ha [command]
```

Available Commands:

|                |                                                                     |
|----------------|---------------------------------------------------------------------|
| addons         | Install, update, remove and configure Home Assistant apps           |
| audio          | Audio device handling.                                              |
| authentication | Authentication for Home Assistant users.                            |
| backups        | Create, restore and remove backups                                  |
| banner         | Prints the CLI Home Assistant banner along with some useful info    |
| cli            | Get information, update or configure the Home Assistant cli backend |
| core           | Provides control of the Home Assistant Core                         |
| dns            | Get information, update or configure the Home Assistant DNS server  |
| docker         | Docker backend specific for info and OCI configuration              |
| hardware       | Provides hardware information about your system                     |
| help           | Help about any command                                              |
| host           | Control the host/system that Home Assistant is running on           |
| info           | Provides a general Home Assistant information overview              |
| jobs           | Get information and manage running jobs                             |
| multicast      | Get information, update or configure the Home Assistant Multicast   |
| network        | Network specific for updating, info and configuration imports       |
| observer       | Get information, update or configure the Home Assistant observer    |
| os             | Operating System specific for updating, info and configuration      |
| resolution     | Resolution center of Supervisor, show issues and suggest solutions  |
| supervisor     | Monitor, control and configure the Home Assistant Supervisor        |

Flags:

|                            |        |                                                                  |
|----------------------------|--------|------------------------------------------------------------------|
| <code>--api-token</code>   | string | Home Assistant Supervisor API token                              |
| <code>--config</code>      | string | Optional config file (default is \$HOME/.homeassistant)          |
| <code>--endpoint</code>    | string | Endpoint for Home Assistant Supervisor (default is 'supervisor') |
| <code>-h, --help</code>    |        | help for ha                                                      |
| <code>--log-level</code>   | string | Log level (defaults to Warn)                                     |
| <code>--no-progress</code> |        | Disable the progress spinner                                     |
| <code>--raw-json</code>    |        | Output raw JSON from the API                                     |

Use "ha [command] --help" for more information about a command.

## Console access

You can also access the Home Assistant Operating System via a directly connected keyboard and monitor, the console.

### Wiping the data disk from the command line

In Home Assistant Operating System, the `ha os datadisk wipe` command wipes the data disk. The command deletes all user data as well as Home Assistant Core, Supervisor, and any installed apps.

The command `ha os datadisk wipe` marks the data partition (either internal on the eMMC or the SD card, or on an external attached data disk) as to be cleared on the next reboot. The command automatically reboots the system. Upon reboot, the data is cleared. Then the system continues to boot and reinstalls the latest version of all Home Assistant components.

The `ha os datadisk wipe` command can only be run from the local terminal. Connect a display and keyboard and use the terminal.

Note, some systems have a reset button you can use to clear the data disk, instead of using the command line:

- If you have a Home Assistant Yellow with a Raspberry Pi Compute Module 5, use the command line steps described above.
- If you have a Home Assistant Yellow with a Raspberry Pi Compute Module 4, there is a red hardware button to wipe the data disk. Follow the procedure on [resetting the Home Assistant Yellow](#).
- If you have a Home Assistant Green, there is a black hardware button to wipe the data disk. Follow the procedure on [resetting the Home Assistant Green](#).

### Listing all users from the command line

In Home Assistant Operating System, the `ha auth list` command lists all users that are registered on your Home Assistant.

The `ha auth list` command can only be run from the local terminal. Connect a display and keyboard and use the terminal.

## Enable duplicate log file

---

By default, Home Assistant Core logs are sent to the Systemd Journal, which can be read using the `ha core logs` command. If you need logs to also be written to a file ( `/config/home-assistant.log` ), you can enable the duplicated log file option using the [command line](#):

```
ha core options --duplicate-log-file=true
ha core rebuild
ha core restart
```

To disable it:

```
ha core options --duplicate-log-file=false
ha core rebuild
ha core restart
```

## Enable I2C

---

Home Assistant using the Home Assistant Operating System which is a managed environment, which means you can't use existing methods to enable the I2C bus on a Raspberry Pi. In order to use I2C devices you will have to - Enable I2C for the Home Assistant Operating System - Set up I2C devices, such as sensors

### Enable I2C with an SD card reader

#### Access the boot partition

You will need: - SD card reader - SD card with Home Assistant Operating System flashed on it

Shutdown/turn-off your Home Assistant installation and unplug the SD card. Plug the SD card into an SD card reader and find a drive/file system named `hassos-boot`. The file system might be shown/mounted automatically. If not, use your operating systems disk management utility to find the SD card reader and make sure the first partition is available.

#### Add files to enable I2C

- In the root of the `hassos-boot` partition, add a new folder called `CONFIG`.
- In the `CONFIG` folder, add another new folder called `modules`.
- Inside the `modules` folder add a text file called `rpi-i2c.conf` with the following content:  

```
txt i2c-dev
```
- In the root of the `hassos-boot` partition, edit the file called `config.txt` add two lines to it:  

```
txt dtparam=i2c_vc=on dtparam=i2c_arm=on
```

#### Start with the new OS configuration

- Insert the SD card back into your Raspberry Pi.
- On startup, the `hassos-config.service` will automatically pickup the new `rpi-i2c.conf` configuration.
- Another reboot might be necessary to make sure the just imported `rpi-i2c.conf` is present at boot time.

### Enable I2C via Home Assistant Operating System Terminal

Alternatively, by attaching a keyboard and screen to your device, you can access the physical terminal to the Home Assistant Operating System.

You can enable I2C via this terminal:

- Login as `root`.
- Type `login` and press enter to access the shell:

```
shell mkdir -p /mnt/boot/CONFIG/modules echo i2c-dev > /mnt/boot/CONFIG/modules/
rpi-i2c.conf echo dtparam=i2c_vc=on >> /mnt/boot/config.txt echo
dtparam=i2c_arm=on >> /mnt/boot/config.txt sync reboot
```

## Troubleshooting

After rebooting the host there should be `i2c-0` and similar device files in `/dev`. If such device files are missing, enabling I2C failed for some reason. You can check the status of I2C kernel modules by using `lsmod | grep i2c` in the terminal. If they are loaded, you should find at least the entry `i2c_dev`. Active usage of the modules is indicated by a number. For example, `i2c_dev 20480 2` would indicate two active I2C device files.

An active I2C can also be checked with a multi meter showing 3.3 V on the I2C pins GPIO2 and GPIO3.

---



## Dashboards/Actions

---

Some cards have support for tap actions. These actions define what will happen when you tap or hold on an object within a card.

Actions can be enabled on the following cards:

- Button
- Entities
- Gauge
- Glance
- Light
- Markdown
- Picture
- Picture element
- Picture entity
- Picture glance
- Shortcut
- Tile
- Weather forecast

## Tap action

---

Action that will be performed when an object on a card is tapped.

```
tap_action:
 action: toggle
```

{% configuration tap-action %} tap\_action: required: false description: Action taken on tap. type: map keys: action: required: true description: "Action to perform ( `more-info` , `toggle` , `perform-action` , `navigate` , `url` , `assist` , `none` )" type: string default: " `toggle` (some cards overwrite default to `more-info` if the provided entity cannot be toggled)" navigation\_path: required: false description: "Path to navigate to (e.g., `/lovelace/0/` ) when the `action` is defined as `navigate` " type: string default: none navigation\_replace: required: false description: "Whether to replace the current page in the the history with the new URL when the `action` is defined as `navigate` " type: boolean default: none url\_path: required: false description: "Path to navigate to (e.g., `https://www.home-assistant.io` ) when the `action` is defined as `url` " type: string default: none perform\_action: required: false description: "Action to perform (e.g., `media_player.media_play_pause` ) when the `action` is defined as `perform-action` " type: string default: none data: required: false description: "Action data to include (e.g., `brightness: 100` ) when the `action` is defined as `perform-action` " type: string default: none target: required: false description: "Action target to user (e.g., `entity_id: media_player.bedroom` ) when the `action` is defined as `perform-action` " type: string default: none confirmation: required: false description: "Present a confirmation dialog to confirm the action. See `confirmation` object below" type: [boolean, map] default: "false" pipeline\_id: required: false description: "Assist pipeline to use when the `action` is defined as `assist` . It can be either `last_used` , `preferred` , or a pipeline id." type: string default: " `last_used` " start\_listening: required: false description: "If supported, listen for voice commands when opening the assist dialog and the `action` is defined as `assist` " type: boolean default: none entity: required: false description: "Overrides the default entity to show when the `action` is defined as `more-info` " type: string default: none

## Hold action

---

Action that will be performed when an object on a card is tapped, held for at least half a second and then released. Action will only be triggered once, not continuously during hold.

```
hold_action:
 action: toggle
```

{% configuration hold\_action %} hold\_action: required: false description: Action taken on tap-and-hold type: map keys: action: required: true description: "Action to perform ( `more-info` , `toggle` , `perform-action` , `navigate` , `url` , `assist` , `none` )" type: string default: "`more-info`" navigation\_path: required: false description: "Path to navigate to (e.g., `/lovelace/0/` ) when the `action` is defined as `navigate`" type: string default: none navigation\_replace: required: false description: "Whether to replace the current page in the the history with the new URL when the `action` is defined as `navigate`" type: boolean default: none url\_path: required: false description: "Path to navigate to (e.g., `https://www.home-assistant.io` ) when the `action` is defined as `url`" type: string default: none perform\_action: required: false description: "Action to perform (e.g., `media_player.media_play_pause` ) when the `action` is defined as `perform-action`" type: string default: none data: required: false description: "Action data to include (e.g., `brightness: 100` ) when the `action` is defined as `perform-action`" type: string default: none target: required: false description: "Action target to user (e.g., `entity_id: media_player.bedroom` ) when the `action` is defined as `perform-action`" type: string default: none confirmation: required: false description: "Present a confirmation dialog to confirm the action. See `confirmation` object below" type: [boolean, map] default: "false" pipeline\_id: required: false description: "Assist pipeline id to use when the `action` is defined as `assist`" type: string default: none start\_listening: required: false description: "If supported, listen for voice commands when opening the assist dialog and the `action` is defined as `assist`" type: boolean default: none entity: required: false description: "Overrides the default entity to show when the `action` is defined as `more-info`" type: string default: none

## Double tap action

---

Action that will be performed when an object on a card is double-tapped.

```
double_tap_action:
 action: toggle
```

{% configuration double\_tap\_action %} double\_tap\_action: required: false description: Action taken on double tap type: map keys: action: required: true description: "Action to perform ( `more-info` , `toggle` , `perform-action` , `navigate` , `url` , `assist` , `none` )" type: string default: "`more-info`" navigation\_path: required: false description: "Path to navigate to (e.g., `/lovelace/0/` ) when the `action` is defined as `navigate` " type: string default: none navigation\_replace: required: false description: "Whether to replace the current page in the the history with the new URL when the `action` is defined as `navigate` " type: boolean default: none url\_path: required: false description: "Path to navigate to (e.g., `https://www.home-assistant.io` ) when the `action` is defined as `url` " type: string default: none perform\_action: required: false description: "Action to perform (e.g., `media_player.media_play_pause` ) when the `action` is defined as `perform-action` " type: string default: none data: required: false description: "Action data to include (e.g., `brightness: 100` ) when the `action` is defined as `perform-action` " type: string default: none target: required: false description: "Action target to user (e.g., `entity_id: media_player.bedroom` ) when the `action` is defined as `perform-action` " type: string default: none confirmation: required: false description: "Present a confirmation dialog to confirm the action. See `confirmation` object below" type: [boolean, map] default: "false" pipeline\_id: required: false description: "Assist pipeline id to use when the `action` is defined as `assist` " type: string default: none start\_listening: required: false description: "If supported, listen for voice commands when opening the assist dialog and the `action` is defined as `assist` " type: boolean default: none entity: required: false description: "Overrides the default entity to show when the `action` is defined as `more-info` " type: string default: none

## Options for confirmation

---

If you define confirmation as an object instead of boolean, you can add more customization and configurations.

```
double_tap_action:
 action: perform-action
 confirmation:
 text: Are you sure you want to restart?
 perform_action: script.restart
hold_action:
 action: perform-action
 confirmation: true
 perform_action: script.do_other_thing
```

{% configuration confirmation%} text: required: false description: Text to present in the confirmation dialog. type: string title: required: false description: Title text of the confirmation dialog. type: string default: "Are you sure? (translated)" confirm\_text: required: false description: Confirmation button text of the confirmation dialog. type: string default: "OK (translated)" dismiss\_text: required: false description: Dismiss button text of the confirmation dialog. type: string default: "Cancel (translated)" exemptions: required: false description: "List of exemption objects. See below" type: list

## Options for exemptions

---

{% configuration exemptions %} user: required: true description: User ID for which the confirmation dialog will **not** be shown. type: string

```
double_tap_action:
 action: perform-action
 confirmation:
 text: Are you sure you want to restart?
 exemptions:
 - user: x9405b8c64ee49bb88c42000e0a9dfa8
 - user: 88bcfbdc39155d16c3b2d09cbf8b0367
 perform_action: script.restart
```

## Examples

---

Tap action implemented on an entity button card:

```
type: button
tap_action:
 action: toggle
hold_action:
 action: more-info
```

## Limitations

---

It is not possible to use templates for actions. But calling a [script](#) is a good alternative.

---



## Dashboards/Badges

---

Badges are small widgets that sit at the top of a dashboard view, above all the cards. They give you an at-a-glance overview of entity states. This is perfect for things like temperatures, open windows, or who is home.

Badges Badges at the top of a panel.

## Adding a badge to your dashboard

---

1. Go to **Settings** > **Dashboards**.
2. If you have multiple [views](#), open the view to which you want to add a badge.
3. In the top right of the screen, select the edit `mdi:edit` button.
4. To add a badge, select the plus `mdi:plus` button.

Screenshot showing how to add a badge

1. Select the entity for which you want to display a badge.
2. Configure your badge.
3. The available options depend on the entity.
4. Add the states you want to see.
5. If you want, add a **Name**.

Screenshot showing how to configure a badge 7. Under **Interactions**, you can define the tap behavior. 8. If you want this badge to be visible only to specific users or under a certain condition, open the **Visibility** tab to [define those conditions](#). - The [available conditions](#) are the same as the ones for the conditional card. 9. Select **Save**.

screencast showing how to add a badge to a dashboard Adding a badge to a dashboard.

## Removing a badge

---

1. Go to the dashboard and follow steps 1–3 in [adding a badge](#).
2. Hover over the badge to reveal the three dots `mdi:dots-vertical` menu. Screenshot showing edit buttons
3. Select the three dots `mdi:dots-vertical` menu and select **Delete**.

Screenshot showing the three dots menu

## Entity badge

---

The Entity badge allows you to display the state of an entity. This badge supports [actions](#).

```
type: entity
entity: light.living_room
```

{% configuration entity %} type: required: true description: "`entity`" type: string entity: required: true description: Entity ID. type: string name: required: false description: Overwrites friendly name. Can be a string, or a name configuration object. See [naming documentation](#). type: [string, map, list] icon: required: false description: Overwrites the entity icon. type: string color: required: false description: Set the color when the entity is active. By default, the color is based on `state`, `domain`, and `device_class` of your entity. It accepts [color token](#) or hex color code. type: string default: state show\_entity\_picture: required: false description: If your entity has a picture, it will replace the icon. type: boolean default: false show\_name: required: false description: Show the name. type: boolean default: "false" show\_icon: required: false description: Show the icon. type: boolean default: "true" show\_state: required: false description: Show the state. type: boolean default: "true" state\_content: required: false description: > Content to display for the state. Can be `state`, `last_changed`, `last_updated`, or any attribute of the entity. Can be either a string with a single item, or a list of string items. Default depends on the entity domain. type: [string, list] tap\_action: required: false description: Action taken on card tap. See [action documentation](#). By default, it will show the "more-info" dialog. type: map hold\_action: required: false description: Action taken on tap-and-hold. See [action documentation](#). type: map double\_tap\_action: required: false description: Action taken on double tap. See [action documentation](#). type: map

## Shortcut badge

---

The shortcut badge gives you a quick way to trigger an action from your dashboard. You can use it to navigate to another page, open a URL, launch the voice assistant, or perform an action.

The label, icon, and color are automatically resolved from the action you configure. For example, if you navigate to a dashboard view, the shortcut picks up the view's title and icon. You can override any of these values if you want something different.

```
type: shortcut
tap_action:
 action: navigate
 navigation_path: "/lovelace/kitchen"
```

{% configuration shortcut %} type: required: true description: " `shortcut` " type: string text: required: false description: The text displayed on the badge. If not set, the text is resolved automatically from the configured `tap_action` . type: string icon: required: false description: The icon displayed on the badge. If not set, the icon is resolved automatically from the configured `tap_action` . type: string color: required: false description: The color of the icon and background accent. Accepts a [color token](#) or hex color code. type: string default: primary tap\_action: required: true description: The action taken on badge tap. For more information, see the [action documentation](#). type: map hold\_action: required: false description: The action taken on badge tap-and-hold. For more information, see the [action documentation](#). type: map double\_tap\_action: required: false description: The action taken on badge double tap. For more information, see the [action documentation](#). type: map

## Examples

Open an external URL with a custom label and icon:

```
type: shortcut
text: "Home Assistant docs"
icon: mdi:book-open-variant
tap_action:
 action: url
 url_path: "https://www.home-assistant.io"
```

Launch the voice assistant:

```
type: shortcut
tap_action:
 action: assist
```

Perform an action with a custom color:

```
type: shortcut
text: "Good night"
color: indigo
tap_action:
 action: perform-action
 perform_action: script.good_night
```

## Entity filter badge

---

This badge allows you to define a list of entities that you want to track only when in a certain state. It's great for showing lights that you forgot to turn off, or for displaying a list of people only when they're at home.

{% configuration filter\_badge %} type: required: true description: "`entity-filter`" type: string  
entities: required: true description: A list of entity IDs or `entity` objects, see below. type: list  
conditions: required: false description: List of conditions to check. See [available conditions](#). type: list  
*state\_filter: required: false description: (legacy) List of strings representing states or filters to check. See [available legacy filters](#).* type: list

\*one is required ( `conditions` or `state_filter` )

### Options for entities

If you define entities as objects instead of strings (by adding `entity:` before entity ID), you can add more customization and configurations:

{% configuration entities %} type: required: false description: "Sets a custom badge type: `custom:my-custom-badge`" type: string  
entity: required: true description: Entity ID. type: string  
name: required: false description: Overwrites friendly name. Can be a string, or a name configuration object. See [naming documentation](#). type: [string, map, list]  
icon: required: false description: Overwrites icon or entity picture. You can use any icon from [Material Design Icons \(MDI\)](#). Prefix the icon name with `mdi:`, for example `mdi:home`. type: string  
image: required: false description: URL of an image to display instead of the icon. type: string  
conditions: required: false description: List of conditions to check. See [available conditions](#). type: list  
*state\_filter: required: false description: (legacy) List of strings representing states or filters to check. See [available legacy filters](#).* type: list

\*only one filter will be applied: `conditions` or `state_filter` if `conditions` is not present

You may also add any additional configuration options to an entity which are supported by the chosen badge type ( `Entity` badge type if no type is chosen).

## Conditions options

---

You can specify multiple `conditions`, in which case the entity will be displayed if it matches all conditions.

### State

Tests if an entity has a specified state.

```
type: entity-filter
entities:
 - climate.thermostat_living_room
 - climate.thermostat_bed_room
conditions:
 - condition: state
 state: heat
```

```
type: entity-filter
entities:
 - climate.thermostat_living_room
 - climate.thermostat_bed_room
conditions:
 - condition: state
 state_not: "off"
```

```
type: entity-filter
entities:
 - sensor.gas_station_1
 - sensor.gas_station_2
 - sensor.gas_station_3
conditions:
 - condition: state
 state: sensor.gas_station_lowest_price
```

{% configuration condition\_state %} condition: required: true description: "`state`" type: string  
state: required: false description: Entity state or ID to be equal to this value. Can contain an array of states. type: *[list, string]* state\_not: required: false description: Entity state or ID to not be equal to this value. Can contain an array of states. type: *[list, string]*

\*one is required ( `state` or `state_not` )

### Numeric state

Tests if an entity state matches the thresholds.



```

type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: numeric_state
 above: 10
 below: 20

```

{% configuration condition\_numeric\_state %} condition: required: true description: " **numeric\_state** " type: string above: required: false description: Entity state or ID to be above this value. *type: string below: required: false description: Entity state or ID to be below this value.* type: string

\*at least one is required ( **above** or **below** ), both are also possible for values between.

## Screen

Specify the visibility of the entity per screen size. Some screen size presets are available in the UI, but you can use any CSS media query you want in YAML.

```

type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: screen
 media_query: "(min-width: 1280px)"

```

{% configuration condition\_screen %} condition: required: true description: " **screen** " type: string media\_query: required: true description: Media query to check which screen size are allowed to display the entity. type: string

## User

Specify the visibility of the entity per user.

```

type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04

```

{% configuration condition\_user %} condition: required: true description: " **user** " type: string users: required: true description: User ID that can see the entity (unique hex value found on the Users configuration page). type: list

## And

Specify that both conditions must be met.

```
type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: and
 conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

{% configuration condition\_and %} condition: required: true description: "and" type: string  
conditions: required: false description: List of conditions to check. See [available conditions](#). type: list

## Or

Specify that at least one of the conditions must be met.

```
type: entity-filter
entities:
 - sensor.outside_temperature
 - sensor.living_room_temperature
 - sensor.bed_room_temperature
conditions:
 - condition: or
 conditions:
 - condition: numeric_state
 above: 0
 - condition: user
 users:
 - 581fca7fdc014b8b894519cc531f9a04
```

{% configuration condition\_or %} condition: required: true description: "or" type: string  
conditions: required: false description: List of conditions to check. See [available conditions](#). type: list

## Legacy state filters

---

### String filter

Show only active switches or lights in the house.

```
type: entity-filter
entities:
 - entity: light.bed_light
 name: Bed
 - light.kitchen_lights
 - light.ceiling_lights
state_filter:
 - "on"
```

You can also specify multiple `state_filter` conditions, in which case the entity will be displayed if it matches any condition.

If you define `state_filter` as objects instead of strings, you can add more customization to your filter, as described below.

### Operator filter

Tests if an entity state corresponds to the applied `operator`.

{% configuration condition\_operator %} value: required: true description: String representing the state. type: string operator: required: true description: Operator to use in the comparison. Can be `==`, `<=`, `<`, `>=`, `>`, `!=`, `in`, `not in`, or `regex`. type: string attribute: required: false description: Attribute of the entity to use instead of the state. type: string

### Examples

Displays everyone who is at home or at work.

```
type: entity-filter
entities:
 - device_tracker.demo_paulus
 - device_tracker.demo_anne_thereise
 - device_tracker.demo_home_boy
state_filter:
 - operator: "=="
 value: home
 - operator: "=="
 value: work
```

You can also specify a filter for a single entity.

```
type: entity-filter
state_filter:
 - "on"
 - operator: ">"
 value: 90
entities:
 - sensor.water_leak
 - sensor.outside_temp
 - entity: sensor.humidity_and_temp
 state_filter:
 - operator: ">"
 value: 50
 attribute: humidity
```

You can also use a regex filter against entity attributes. The example below looks for expressions that are 1 digit in length and where the number is between 0–7. This shows holidays occurring today or within the next 7 days as entities in the Entity filter badge.

```
type: entity-filter
state_filter:
 - operator: regex
 value: "^[0-7]{1})$"
 attribute: eta
entities:
 - entity: sensor.upcoming_ical_holidays_0
 - entity: sensor.upcoming_ical_holidays_1
 - entity: sensor.upcoming_ical_holidays_2
 - entity: sensor.upcoming_ical_holidays_3
 - entity: sensor.upcoming_ical_holidays_4
```

---

## Dashboards/Cards

---

Each dashboard is made up of cards. Cards display information and let you control your devices. You can arrange them freely across views, resize them in the [Sections view](#) layout, and configure actions, features, and conditional visibility.

Screenshot of the masonry view with different types of cards Screenshot of the masonry view with different types of cards.

## Card categories

---

There are several different card types, each with their own configuration options. They can be categorized in terms of their function:

- **Specific to a device type or service:** alarm, light, humidifier, thermostat, plant status, media control, weather forecast, to-do list, shopping list, map, activity, calendar
- **Grouping other cards:** vertical stack, horizontal stack, grid
- **Logic function:** conditional, entity filter
- **Display generic data:** sensor, history graph, statistic, statistics graph, energy, gauge, clock, Markdown, webpage
- **Control devices and entities:** button, entity, shortcut
- **Display data and control entities:** tile, heading, entities, glance, area, picture, picture entity, picture elements, picture glance

## Adding cards to your dashboard

---

A card can be added to a dashboard directly [from the view](#) where you want to add it, or from the [device page](#).

### To add a card from a view

1. Depending on your dashboard view layout:
2. For Sections layout, select `mdi:plus` in the section (bottom left). Add card in sections layout
3. For other layout types (such as Masonry, Panel, or Sidebar), in the bottom right of the view, select **Add card**. Add card in masonry layout
4. There are two methods to add a card:
5. If you have an idea of what card type you want to use for an entity, add **By card** type: - Browse the list of available cards. - If you are using the **Sections** view, try the **Tile** card in the **Suggested cards** section. Add card by Card type dialog
6. If you want to add cards for multiple entities at once, select **By entity**. - Select the entities from the list. - Then, select **Continue**. Screenshot add cards by entity
7. If you want this card to be visible only to specific users or under a certain condition, you can [define those conditions](#).
8. If you are adding this card to a [sections view](#), on the **Layout** tab, you can [resize the card](#).
9. Customize your card:
10. [define card actions](#).
11. [define header and footer](#)
12. [customize features](#).
13. Not all cards support these elements. Refer to the documentation of the specific card type.
14. Select **Add to dashboard**.

Screenshot card suggestions

### To add a card from the device page

This method is useful if you are on the **Device** page and want to create a card directly from there.

1. Go to **Settings > Devices & services**.
2. On the integration card of interest, select **Devices**.
3. If there are multiple devices, select the device from the list.
4. Add the card:
5. Find the area of interest, for example **Sensors**, or **Events** and select **Add to Dashboard**. Add to Dashboard button on the device page
6. In the dialog, select the target dashboard and if you have multiple views, select the view.
7. Select **Next**.
8. If you like the card proposal, select **Add to dashboard**.

9. If you want to change the card, **Pick different card** and choose your card type.
10. **Result:** The card is added to the selected view. Add to Dashboard button on the device page
11. To edit the card configuration, open the view to which you added the card.
12. Select **Edit card**.
13. If you want this card to be visible only to specific users or under a certain condition, you can [define those conditions](#).
14. If you are adding this card to a [sections view](#), on the **Layout** tab, you can [resize the card](#).
15. Customize your card:
16. [define card actions or header and footer widgets](#).
17. [define header and footer](#)
18. [customize features](#).
19. Not all cards support these elements. Refer to the documentation of the specific card type.



## Showing or hiding a card or badge conditionally

---

You can choose to show or hide certain cards or [badges](#) based on different conditions. The [available conditions](#) are the same as the ones for the conditional card.

1. On the **Visibility** tab, select **Add condition**.
2. Troubleshooting: Don't see a **Visibility** tab?
  - It is not available inside nested cards: vertical stack, horizontal stack, and grid card
3. Select the type of condition, and enter the parameters.
4. The [available conditions](#) are the same as the ones for the conditional card.
5. If you define multiple conditions, the card or badge is only shown when all conditions are met.
6. If you did not define any conditions, the card or badge is always shown to all users.
7. Select **Save**.

## Resizing a card

---

In [sections view](#), you can resize cards. Follow these steps:

1. On the **Layout** tab, use the grid size picker to adjust the number of columns and rows the card occupies. Screenshot of the Layout tab in the card dialog
2. Troubleshooting: Don't see a **Layout** tab?
  - It is not available inside nested cards: vertical stack, horizontal stack, and grid card
  - It is not available on the picture elements card.
3. To make the card span the full width of the section, regardless of section size, enable **Full width card**.
4. If you want a finer grid to size your card, enable **Precise mode**. The last row was done using precise mode. Screenshot of a section using precise mode for some cards
5. Select **Save**.

## Revert resizing of a card

---

If you previously [resized](#) a card in the [sections view](#), and you don't like the new size, you can revert back to the card's default size. Follow these steps:

1. On the **Layout** tab, select the `mdi:restore` icon. Screenshot of the Layout tab in the card dialog
2. Select **Save**.

## Card actions, features, header and footer widgets

---

Some cards have support for [tap actions](#). These actions define what will happen when you tap or hold on an object within a card.

Some cards have support for [features](#). These widgets add quick controls to the card. Supported features depend on the card and entity capabilities. Multiple features can be added to a single card.

Screenshot of tile cards with features. Screenshot of tile cards with features.

Some cards have support for [header and footer widgets](#). These widgets fill up the entire available space in a card.

Screenshot of an entities card with a picture header. Screenshot of an entities card with a picture header and buttons footer.

---

## Dashboards/Dashboards

---

You can define multiple dashboards in Home Assistant. Each dashboard can be added to the sidebar. This makes it possible to create separate control dashboards for each individual part of your house.

Under **Settings** > **Dashboards**, you can see your own dashboards as well as the predefined, built-in dashboards.

Screenshot of the dashboard list Screenshot of the Dashboard list.

## Home Assistant built-in dashboards

---

**Built-in** dashboards are available out of the box, cannot be deleted, and there are limited options on how much you can edit them.

### Dashboards shown in the sidebar by default

Predefined dashboards that are available in the sidebar by default:

- [Home dashboard](#). Category: built-in. It is shown in the sidebar only while it is set as your default dashboard. If you set another dashboard as default, that dashboard appears in the sidebar instead.
- [Activity dashboard](#). Category: built-in.
- [Energy dashboard](#). Category: built-in.
- [History dashboard](#). Category: built-in.
- [Map dashboard](#). Category: User created. The Map dashboard is an exception: it is available out of the box, but you can edit it freely. This is why it is categorized as a **User created** dashboard.
- [To-do lists dashboard](#). Category: built-in.

Screenshot of the dashboard list on a new installation Screenshot of the Dashboard list on a new installation.

### Dashboards only shown in the dashboard list by default

Some of the built-in dashboards are not shown in the sidebar by default, but are listed under **Settings > Dashboards**.

- **Lights** dashboards: Overview of your lights, [grouped by floors](#) and [areas](#).
- **Security** dashboards: Overview of your security-related devices, [grouped by floors](#) and [areas](#). The security-related devices include devices such as alarm, lock, camera, doors/covers, motion sensors, and binary sensor.
- **Climate** dashboards: Overview of your climate devices, [grouped by floors](#) and [areas](#). The climate dashboard includes devices such as heating and cooling devices, windows, and fans.
- **Energy** dashboards: Allows you to visualize your energy consumption and production, if you have such entities available. This includes electricity from grid and from solar panels, gas and water consumption, and the status of your battery storage.

Not all of the predefined dashboards are listed under **Settings > Dashboards**. The **Activity** and **History** dashboards for example are powered by their respective integrations.

### Home dashboard

The **Home** dashboard is an entry point to open other built-in dashboards based on areas or topics such as lights, climate, or media players.

The **Home** dashboard is prepopulated by default and shows your entities [grouped by areas](#).

- It uses the [sections view](#) type and [tile cards](#).
- The first view shows all your areas and the entities that are [assigned to those areas](#).
- In addition, the dashboard provides a separate view for each area.
- entities Entities, such as lights, covers, and cameras are automatically grouped by domain.

Screenshot of the built-in Home dashboard Screenshot of the built-in Home dashboard.

### Editing the Home dashboard

1. Make sure all the entities are [assigned to an area](#).
2. In the top-right corner, select the `mdi:pencil` icon.
3. You can add entities to customize which items appear on your dashboard.
4. If you don't like how the cards are arranged, [you can reorder floors and areas](#) under **Settings** > **Areas, labels & zones**.

### Troubleshooting: entity is not showing

1. Not all devices or entity types are automatically added to the **Home** dashboard.
2. Make sure the entity is [assigned to an area](#) and check the dashboard again.

## History dashboard

The predefined **History** dashboard is powered by the [History integration](#). To learn about the data sources used and how to export data, refer to the documentation of the History integration.

## Activity dashboard

The predefined **Activity** dashboard is powered by the [Activity integration](#). To control which events to show or filter out, refer to the documentation of the Activity integration.

## Map dashboard

The predefined **Map** dashboard is populated by the [Map card](#). You can edit this dashboard like any other dashboard. For example, you can edit the [view](#) to use the **Sidebar** instead of the default **Panel** view type if you like.

### Maps and presence detection

If you see a [person](#) on the map, it means you have connected a device that allows [presence detection](#). This is the case for example if you have the [Home Assistant Companion App](#) on your phone and allowed location tracking.

## To-do lists dashboard

The predefined **To-do lists** dashboard is powered by the [To-do integration](#). To learn how to use to-do and shopping lists, refer to the documentation of the to-do list integration.

## Webpage dashboard

---

Another available (but not default) dashboard is the webpage dashboard. The webpage dashboard allows you to add and embed a webpage to your dashboard. This could be a web page from the internet or a local web page from a local machine or device like your router or NAS. The webpage dashboard uses the [webpage card](#).

Screenshots showing addition of a new webpage dashboard to Home Assistant, embedding the Home Assistant website.

This dashboard replaces the old iFrame panel ( `iframe_panel` ). If you have existing panels configured in your YAML configuration, Home Assistant will automatically migrate them to the new webpage dashboard on upgrade.

Screenshot showing the Home Assistant website embedded into the Home Assistant frontend using a webpage dashboard.

Note that not every webpage can be embedded due to security restrictions that some sites have in place. These restrictions are enforced by your browser and prevent embedding them into a Home Assistant dashboard.



## Setting a default dashboard

---

The default dashboard is the dashboard that is shown when you open Home Assistant. It is listed on top of the sidebar.

- If you have administrator rights, you can set an initial default dashboard for all users.
- Go to **Settings > Dashboards**.
- In the list of dashboards, find the dashboard of interest and select the `mdi:dots-vertical` menu.
- Select **Set as default**.

Setting a default dashboard for all users

- **Result:** This dashboard is shown to all users when they open Home Assistant.
- To change your personal default dashboard, you don't need administrator rights.
- Go to **User profile**.
- On the **General** tab, under **Dashboard**, select your default dashboard.

Changing your own default dashboard 3. If you want your wall tablet to use a different dashboard than your other devices, use a separate user profile for your wall tablet. - If you set your phone to one dashboard and your wall tablet to another, using the same user, they'll both revert to the default dashboard.

## Creating a new dashboard

---

The built-in dashboards update themselves when you add new devices. If you want a customized dashboard, it is recommended not to change the **Overview** dashboard, but to create a new dashboard instead.

This will leave the default dashboard intact.

1. Go to **Settings > Dashboards**.
2. Select **Add dashboard**.
3. In the dialog, choose one of the options:
4. If you want to start with a pre-populated dashboard, choose **Overview (Legacy)** or one of the suggested ones, such as the **Map** dashboard.
5. If you want to start with a completely empty dashboard, choose **New dashboard from scratch**.
6. In the **Add new dashboard** dialog, enter a name and select an icon.
7. Define if this dashboard should be visible only to the admin user.
8. Define if you want the dashboard to be listed in the sidebar.
9. Select **Create**.
10. Result: The dashboard is added.

## Editing a new dashboard

---

1. Open your new dashboard and in the top right of the screen, select the `mdi:pencil` button.
  2. Result: The **Edit dashboard** dialog appears.
  3. Select the areas you want to show on this new dashboard and select **Save**.
  4. If you want to have more detailed control over the dashboard, you need to take control:
    - This means that this dashboard is no longer automatically updated when new dashboard elements become available.
    - Once you've taken control, you can't get this specific dashboard back to update automatically. However, you can create a new default dashboard.
    - To continue, in the dialog, select the three dots `mdi:dots-vertical` menu, then select **Take control**.
  5. You can now [add a badge](#), [add a card](#), or [add a view](#).
  6. To **undo** or **redo** a change, select the buttons on top of the dashboard.
- Screenshot of the undo and redo buttons on top of the dashboard

## Deleting a dashboard

---

If you do not use one of the predefined dashboards, or created a dashboard you no longer need, you can delete that dashboard. It will then no longer show in the sidebar.

1. Go to **Settings > Dashboards**.
2. From the list of dashboards, select the dashboard you want to delete.
3. In the dialog, select **Delete**. Deleting a dashboard

## Adding YAML dashboards

You can use YAML to define dashboards. Each YAML dashboard is loaded from its own YAML file.

If it is the first time you edit the `configuration.yaml` file, refer to [Editing configuration.yaml](#) to know how to install a file editor and find the file.

To add YAML dashboards, in your `configuration.yaml` file, create a `dashboards:` section under the top-level `lovelace:` key.

```
lovelace:
 # Use resource_mode to load resources from YAML
 resource_mode: yaml
 # Include external resources (requires resource_mode: yaml)
 resources:
 - url: /local/my-custom-card.js
 type: module
 - url: /local/my-webfont.css
 type: css
 # Add YAML dashboards
 dashboards:
 my-home: # Needs to contain a hyphen (-)
 mode: yaml
 filename: my-home.yaml
 title: My home
 icon: mdi:home-outline
 show_in_sidebar: true
 dashboard-hidden:
 mode: yaml
 filename: hidden.yaml
 title: Hidden
 show_in_sidebar: false
 dashboard-admin:
 mode: yaml
 title: Admin
 icon: mdi:tools
 show_in_sidebar: true
 require_admin: true
 filename: admin.yaml
```

{% configuration dashboards %} resource\_mode: required: false description: "Controls how resources are loaded. Set to `yaml` to load resources from the `resources` key in YAML configuration. Set to `storage` to manage resources through the UI." type: string default: storage resources: required: false description: "List of resources that should be loaded. Requires `resource_mode: yaml` to take effect. After changing the YAML configuration, select the three dots `mdi:dots-vertical` menu (top-right) and select **Reload resources** to pick up changes without restarting Home Assistant. You can also call the `lovelace.reload_resources` action directly." type: list keys: url: required: true description: The URL of the resource to load. type: string type: required: true description: "The type of resource, this should be either `module` for a JavaScript module or `css` for a StyleSheet." type: string dashboards: required: false description: "Additional YAML dashboards. The key is used for the URL and should contain a hyphen (-), except for `lovelace`, which is allowed for backward compatibility." type: map keys: mode:

required: true description: "The mode of the dashboard, this should always be `yaml`. Dashboards in `storage` mode can be created in the configuration panel." type: string filename: required: true description: "The file in your `config` directory where the configuration for this panel is." type: string title: required: true description: "The title of the dashboard, will be used in the sidebar." type: string icon: required: false description: The icon to show in the sidebar. You can use any icon from [Material Design Icons](#). Prefix the icon name with `mdi:`, ie `mdi:home`. type: string show\_in\_sidebar: required: false description: Should this dashboard be shown in the sidebar. type: boolean default: true require\_admin: required: false description: Should this dashboard be only accessible for admin users. type: boolean default: false

As a super minimal example of a dashboard config, here's the bare minimum you will need for it to work:

```
views:
 # View tab title.
 - title: Example
 cards:
 # The markdown card will render markdown text.
 - type: markdown
 title: Dashboard
 content: >
 Welcome to your dashboard.
```

A slightly more advanced example:

```

views:
 # View tab title.
 - title: Example
 # Unique path for direct access /lovelace/${path}
 path: example
 # Each view can have a different theme applied. Theme should be defined in the
 theme: dark-mode
 # The cards to show on this view.
 cards:
 # The filter card will filter entities for their state
 - type: entity-filter
 entities:
 - device_tracker.paulus
 - device_tracker.anne_there
 state_filter:
 - 'home'
 card:
 type: glance
 title: People that are home

 # The picture entity card will represent an entity with a picture
 - type: picture-entity
 image: https://www.home-assistant.io/images/default-social.png
 entity: light.bed_light

 # Specify a tab icon if you want the view tab to be an icon.
 - icon: mdi:home-assistant
 # Title of the view. Will be used as the tooltip for tab icon
 title: Second view
 cards:
 # Entities card will take a list of entities and show their state.
 - type: entities
 # Title of the entities card
 title: Example
 # The entities here will be shown in the same order as specified.
 # Each entry is an entity ID or a map with extra options.
 entities:
 - light.kitchen
 - switch.ac
 - entity: light.living_room
 # Override the name to use
 name: LR Lights

 # The markdown card will render markdown text.
 - type: markdown
 title: Dashboard
 content: >
 Welcome to your dashboard.

```

---

## Dashboards/Features

---

Some dashboard cards have support for features. These widgets add quick controls to the card. Supported features depend on the card and entity capabilities. Multiple features can be added to a single card.

Screenshot of tile cards with features. Screenshot of tile cards with features.

Features can be enabled on the following cards:

- [Humidifier](#)
- [Thermostat](#)
- [Tile](#)
- [Area](#)



## Customizing features

---

1. Edit the card and open the **Features** section.
2. To add an additional feature to your card, select **Add feature**.
3. **Info:** The available options for a feature depend on the entity and type of feature.
  - For example, not all entities have a [toggle](#) or a [counter-action](#).
4. On tile cards, you can adjust the feature position.
5. Under **Features > Feature position**, select **Bottom** or **Inline**:

Screen recording showing how you can now reorder the HVAC modes on the thermostat shown in a tile card.

1. Reordering features:
2. Some features of the tile card, such as the presets or the HVAC modes of a thermostat, can show buttons.
3. To reorder the buttons, enable **Customize** and drag and drop the buttons into position.
4. If you don't like the buttons, you can replace them by a **Dropdown** instead.
  - Under **Style**, select the **Dropdown** option.

Screen recording showing how you can now reorder the HVAC modes on the thermostat shown in a tile card.

## Alarm modes

---

Widget that displays buttons to arm and disarm an [alarm](#).

Screenshot of the tile card with alarm modes feature Screenshot of the tile card with alarm modes feature

```
features:
- type: "alarm-modes"
 modes:
 - armed_home
 - armed_away
 - armed_night
 - armed_vacation
 - armed_custom_bypass
 - disarmed
```

{% configuration features %} type: required: true description: " `alarm-modes` " type: string modes: required: false description: List of modes to show on the card. The list can contain `armed_home` , `armed_away` , `armed_night` , `armed_vacation` , `armed_custom_bypass` , and `disarmed` . If not set, all modes supported by the entity are shown. type: list

## Bar gauge

---

Widget that displays the state of a numeric [sensor](#) as a horizontal bar.

Screenshot of the tile card with the bar gauge feature Screenshots of the tile card with the bar gauge feature

```
features:
- type: "bar-gauge"
 min: 0
 max: 100
```

{% configuration features %} type: required: true description: "[bar-gauge](#)" type: string min: required: false description: Minimum value for the gauge range. type: integer default: 0 max: required: false description: Maximum value for the gauge range. type: integer default: 100

## Button

---

Widget that displays buttons to control [button](#), [input\\_button](#), [scene](#), or [script](#).

Screenshot of the tile card with the button feature Screenshot of the tile card with the button feature

```
features:
- type: "button"
 action_name: "Press"
 data:
 variable: some_value
```

{% configuration features %} type: required: true description: "[button](#)" type: string action\_name: required: false type: string description: Text inside the button. type: string data: required: false description: Additional data to be passed when the action is executed. Only applies to script. type: map

## Climate fan modes

---

Widget that displays buttons or icons to control the fan mode for a [climate](#) device.

Screenshot of the tile card with the climate fan modes feature Screenshot of the tile card with the climate fan modes feature

```
features:
- type: "climate-fan-modes"
 style: "icons"
 fan_modes:
 - "off"
 - low
 - medium
 - high
```

{% configuration features %} type: required: true description: " `climate-fan-modes` " type: string style: required: false description: "How the fan modes should be displayed. It can be either `dropdown` or `icons` ." type: string default: dropdown fan\_modes: required: false description: List of fan modes to show on the card. The list can contain `on` , `off` , `auto` , `low` , `medium` , `high` , `middle` , `focus` and `diffuse` or any other custom fan mode. If not set, all fan modes supported by the entity are shown. type: list

## Climate HVAC modes

---

Widget that displays buttons to control the HVAC mode for a [climate](#).

Screenshot of the tile card with the climate HVAC modes feature Screenshot of the tile card with the climate HVAC modes feature

```
features:
- type: "climate-hvac-modes"
 hvac_modes:
 - auto
 - heat_cool
 - heat
 - cool
 - dry
 - fan_only
 - "off"
```

{% configuration features %} type: required: true description: " `climate-hvac-modes` " type: string style: required: false description: "How the modes should be displayed. It can be either `dropdown` or `icons` ." type: string default: icons hvac\_modes: required: false description: List of modes to show on the card. The list can contain `auto` , `heat_cool` , `heat` , `cool` , `dry` , `fan_only` , and `off` . If not set, all HVAC modes supported by the entity are shown. type: list

## Climate preset modes

---

Widget that displays buttons or icons to control the preset mode for a [climate](#).

Screenshot of the tile card with the climate preset modes feature Screenshot of the tile card with the climate preset modes feature

```
features:
- type: "climate-preset-modes"
 style: "icons"
 preset_modes:
 - home
 - eco
```

{% configuration features %} type: required: true description: "[climate-preset-modes](#)" type: string style: required: false description: "How the preset modes should be displayed. It can be either [dropdown](#) or [icons](#)." type: string default: dropdown preset\_modes: required: false description: List of preset modes to show on the card. The list can contain [eco](#), [away](#), [boost](#), [comfort](#), [home](#), [sleep](#), and [activity](#) or any other custom preset mode. If not set, all preset modes supported by the entity are shown. type: list

## Climate swing modes

---

Widget that displays a dropdown or icons to control the swing mode for a [climate](#).

Screenshot of the tile card with the climate swing modes feature Screenshot of the tile card with the climate swing modes feature

```
features:
- type: "climate-swing-modes"
 style: "icons"
 swing_modes:
 - "on"
 - "off"
```

{% configuration features %} type: required: true description: " `climate-swing-modes` " type: string style: required: false description: "How the swing modes should be displayed. It can be either `dropdown` or `icons` ." type: string default: dropdown swing\_modes: required: false description: List of swing modes to show on the card. The list can contain `on` , `off` , or any other custom swing mode supported by your climate device. If not set, all swing modes supported by the entity are shown. type: list



## Climate swing horizontal modes

---

Widget that displays a dropdown or icons to control the horizontal swing mode for a [climate](#).

Screenshot of the tile card with the climate swing horizontal modes feature Screenshot of the tile card with the climate swing horizontal modes feature

```
features:
- type: "climate-swing-horizontal-modes"
 style: "dropdown"
 swing_horizontal_modes:
 - "on"
 - "off"
```

{% configuration features %} type: required: true description: " `climate-swing-horizontal-modes` " type: string style: required: false description: "How the horizontal swing modes should be displayed. It can be either `dropdown` or `icons` ." type: string default: dropdown swing\_horizontal\_modes: required: false description: List of horizontal swing modes to show on the card. The list can contain `on` , `off` , or any other custom horizontal swing mode supported by your climate device. If not set, all horizontal swing modes supported by the entity are shown. type: list

## Counter actions

---

Widget that displays buttons to increment, decrement, and reset a [counter](#).

Screenshot of the tile card with counter actions feature Screenshot of the tile card with counter actions feature

```
features:
- type: "counter-actions"
 actions:
 - increment
 - decrement
 - reset
```

{% configuration features %} type: required: true description: "[counter-actions](#)" type: string  
actions: required: false description: List of actions to show on the card. The list can contain [increment](#) , [decrement](#) , and [reset](#) . If not set, all actions supported by the entity are shown.  
type: list

## Cover open/close

---

Widget that displays buttons to open, close, or stop a [cover](#).

Screenshot of the tile card with open/close feature Screenshot of the tile card with cover open/close feature

```
features:
 - type: "cover-open-close"
```

{% configuration features %} type: required: true description: " `cover-open-close` " type: string

## Cover position

---

Widget that displays a slider to control the position for a [cover](#).

Screenshot of the tile card with the cover position feature Screenshot of the tile card with the cover position feature

```
features:
 - type: "cover-position"
```

{% configuration features %} type: required: true description: " `cover-position` " type: string

## Cover favorite positions

---

Widget that displays a dropdown with favorite positions for a [cover](#).

You can customize favorites in a cover's **More info** dialog. To edit them, press and hold a favorite.

Screenshot of the tile card with the cover favorite positions feature Screenshot of the tile card with the cover favorite positions feature

```
features:
 - type: "cover-position-favorite"
```

{% configuration features %} type: required: true description: " `cover-position-favorite` " type: string

## Cover tilt

---

Widget that displays buttons to open, close, or stop a [cover](#).

Screenshot of the tile card with tilt feature Screenshot of the tile card with cover tilt feature

```
features:
 - type: "cover-tilt"
```

{% configuration features %} type: required: true description: " `cover-tilt` " type: string

## Cover favorite tilt positions

---

Widget that displays a dropdown with favorite tilt positions for a [cover](#).

You can customize favorites in a cover's **More info** dialog. To edit them, press and hold a favorite.

Screenshot of the tile card with the cover favorite tilt positions feature Screenshot of the tile card with the cover favorite tilt positions feature

```
features:
 - type: "cover-tilt-favorite"
```

{% configuration features %} type: required: true description: "[cover-tilt-favorite](#)" type: string

## Cover tilt position

---

Widget that displays a slider to control the tilt position for a [cover](#).

Screenshot of the tile card with the cover tilt position feature Screenshot of the tile card with the cover tilt position feature

```
features:
 - type: "cover-tilt-position"
```

{% configuration features %} type: required: true description: "`cover-tilt-position`" type: string



## Date

---

Widget that displays a button to select a date using the date picker dialog for the [date](#), [datetime](#), and [input datetime](#) entities.

Screenshot of the tile card with the date feature Screenshot of the tile card with the date feature

```
features:
 - type: "date-set"
```

{% configuration features %} type: required: true description: " `date-set` " type: string

## Fan direction

---

Widget that displays controls to change direction for a [fan](#).

Screenshot of the tile card with the fan direction feature Screenshot of the tile card with the fan direction feature

```
features:
 - type: "fan-direction"
```

{% configuration features %} type: required: true description: " `fan-direction` " type: string

## Fan oscillate

---

Widget that displays controls to change oscillation state for a [fan](#).

Screenshot of the tile card with the fan oscillate feature Screenshot of the tile card with the fan oscillate feature

```
features:
 - type: "fan-oscillate"
```

{% configuration features %} type: required: true description: " `fan-oscillate` " type: string

## Fan preset modes

---

Widget that displays buttons or icons to control the preset mode for a [fan](#).

Screenshot of the tile card with the fan preset modes feature Screenshot of the tile card with the fan preset modes feature

```
features:
- type: "fan-preset-modes"
 style: "icons"
 preset_modes:
 - auto
 - smart
 - sleep
 - 'on'
```

{% configuration features %} type: required: true description: "[fan-preset-modes](#)" type: string style: required: false description: "How the preset modes should be displayed. It can be either [dropdown](#) or [icons](#)." type: string default: dropdown preset\_modes: required: false description: List of preset modes to show on the card. The list can contain any supported preset modes. If not set, all preset modes supported by the entity are shown. type: list

## Fan speed

---

Widget that displays speed controls for a [fan](#).

Screenshot of the tile card with fan speed feature Screenshot of the tile card with fan speed feature

```
features:
 - type: "fan-speed"
```

{% configuration features %} type: required: true description: " `fan-speed` " type: string

## Humidifier modes

---

Widget that displays buttons or icons to control the mode for a [humidifier](#).

Screenshot of the tile card with the humidifier modes feature Screenshot of the tile card with the humidifier modes feature

```
features:
- type: "humidifier-modes"
 style: "icons"
 modes:
 - home
 - eco
```

{% configuration features %} type: required: true description: " `humidifier-modes` " type: string style: required: false description: "How the modes should be displayed. It can be either `dropdown` or `icons` ." type: string default: dropdown modes: required: false description: List of modes to show on the card. The list can contain `normal` , `eco` , `away` , `boost` , `comfort` , `home` , `sleep` , `auto` , and `baby` or any other custom mode. If not set, all modes supported by the entity are shown. type: list

## Humidifier toggle

---

Widget that displays buttons to turn on or off a [humidifier](#).

Screenshot of the tile card with the humidifier toggle feature Screenshot of the tile card with the humidifier toggle feature

```
features:
 - type: "humidifier-toggle"
```

{% configuration features %} type: required: true description: " `humidifier-toggle` " type: string

## Lawn mower commands

---

Widget that displays buttons to control a [lawn mower](#).

Screenshot of the tile card with the lawn mower commands feature Screenshot of the tile card with the lawn mower commands feature

```
features:
- type: "lawn-mower-commands"
 commands:
 - start_pause
 - dock
```

{% configuration features %} type: required: true description: "[lawn-mower-commands](#)" type: string commands: required: true description: List of commands to show on the card. The list can contain [start\\_pause](#) and [dock](#). type: list



## Light brightness

---

Widget that displays a slider to select the brightness for a [light](#).

Screenshot of the tile card with light brightness feature Screenshot of the tile card with light brightness feature

```
features:
 - type: "light-brightness"
```

{% configuration features %} type: required: true description: " `light-brightness` " type: string

## Light color favorites

---

Widget that displays a set of buttons to select a color for a [light](#) from a list of favorites.

You can customize favorites in a light's more-info dialog. The feature shows as many favorites as fit in the available width, following the favorites' sort order.

Screenshot of the tile card with the light color favorites feature Screenshot of the tile card with the light color favorites feature

```
features:
 - type: "light-color-favorites"
```

{% configuration features %} type: required: true description: " `light-color-favorites` " type: string

## Light color temp

---

Widget that displays a slider to select the color temperature for a [light](#).

Screenshot of the tile card with the light color temperature feature Screenshot of the tile card with the light color temperature feature

```
features:
 - type: "light-color-temp"
```

{% configuration features %} type: required: true description: " `light-color-temp` " type: string

## Lock commands

---

Widget that displays buttons to lock or unlock a [lock](#).

Screenshot of the tile card with the lock commands feature Screenshot of the tile card with the lock commands feature

```
features:
 - type: "lock-commands"
```

{% configuration features %} type: required: true description: " `lock-commands` " type: string

## Lock open door

---

Widget that displays a button to [open a door](#).

Screenshot of the tile card with the lock open door feature Screenshot of the tile card with the lock open door feature

```
features:
 - type: "lock-open-door"
```

{% configuration features %} type: required: true description: " `lock-open-door` " type: string

## Media player playback controls

---

Widget that displays playback controls for a [media player](#).

Screenshot of the tile card with media player playback feature Screenshot of the tile card with media player playback feature

```
features:
 - type: "media-player-playback"
```

{% configuration features %} type: required: true description: " `media-player-playback` " type: string

## Media player sound mode

---

Widget that displays a dropdown to select the sound mode for a [media player](#).

Screenshot of the tile card with media player sound mode feature Screenshot of the tile card with media player sound mode feature

```
features:
 - type: "media-player-sound-mode"
```

{% configuration features %} type: required: true description: " `media-player-sound-mode` " type: string

## Media player source

---

Widget that displays a dropdown to select the source for a [media player](#).

Screenshot of the tile card with media player source feature Screenshot of the tile card with media player source feature

```
features:
 - type: "media-player-source"
```

{% configuration features %} type: required: true description: "[media-player-source](#)" type: string



## Media player volume buttons

---

Widget that displays buttons to control the volume for a [media player](#).

Screenshot of the tile card with media player volume buttons feature Screenshot of the tile card with media player volume buttons feature

```
features:
 - type: "media-player-volume-buttons"
```

{% configuration features %} type: required: true description: " `media-player-volume-buttons` "  
type: string step: required: false description: "The step size of the volume. The default is 5%."  
type: integer default: 5

## Media player volume slider

---

Widget that displays a slider to control the volume for a [media player](#).

Screenshot of the tile card with media player volume slider feature Screenshot of the tile card with media player volume slider feature

```
features:
 - type: "media-player-volume-slider"
```

{% configuration features %} type: required: true description: "`media-player-volume-slider`"  
type: string

## Numeric input

---

Widget that displays a slider or buttons to set the value for a [number](#) or [input number](#).

Screenshot of the tile card with the numeric input feature Screenshot of the tile card with the numeric input feature

```
features:
- type: "numeric-input"
 style: "buttons"
```

{% configuration features %} type: required: true description: " `numeric-input` " type: string style: required: false description: "Which style of control to display. It can be either `buttons` or `slider` ." type: string default: slider

## Select options

---

Widget that displays a dropdown to select an option for a [select](#) or [input select](#).

Screenshot of the tile card with the select options feature Screenshot of the tile card with the select options feature

```
features:
 - type: "select-options"
```

{% configuration features %} type: required: true description: "[select-options](#)" type: string  
options: required: false description: List of options to show on the card. If not specified, all available options from the entity are displayed. type: list

## Target humidity

---

Widget that displays a slider to select the target humidity for a [humidifier](#).

Screenshot of the tile card with the target humidity feature Screenshot of the tile card with the target humidity feature

```
features:
 - type: "target-humidity"
```

{% configuration features %} type: required: true description: " `target-humidity` " type: string

## Target temperature

---

Widget that displays buttons to select the target temperature for a [climate](#) or a [water heater](#).

Screenshot of the tile card with the target temperature feature Screenshot of the tile card with the target temperature feature

```
features:
 - type: "target-temperature"
```

{% configuration features %} type: required: true description: " `target-temperature` " type: string

## Toggle

---

Widget that displays a button to toggle a [switch](#) or [input boolean](#) entity on or off.

Screenshot of the tile card with the toggle feature Screenshot of the tile card with the toggle feature

```
features:
 - type: "toggle"
```

{% configuration features %} type: required: true description: " `toggle` " type: string

## Trend graph

---

Widget that displays a trend of the history for a numeric [sensor](#).

Screenshot of the tile card with the trend graph feature Screenshot of the tile card with the trend graph feature

```
features:
- type: "trend-graph"
 hours_to_show: 24
 detail: true
```

{% configuration features %} type: required: true description: " `trend-graph` " type: string  
hours\_to\_show: required: false description: Hours to show in graph. Minimum is 1 hour. Big values can result in delayed rendering, especially if the selected entities have a lot of state changes. type: integer default: 24 detail: required: false description: Show more detail in the graph. When enabled, samples to 1 point per 5 pixels. When disabled, samples to 1 point per hour using mean values for a smoother graph. type: boolean default: true

 **Note:** The `hours_to_show` option controls the time range of historical data shown in the graph. The amount of history available depends on the Recorder's `purge_keep_days` setting. By default, the Recorder purges data older than 10 days. See the [Recorder integration documentation](#) for more information.



## Update actions

---

Widget that displays actions to install or skip an [update](#).

Screenshot of the tile card with update actions feature Screenshot of the tile card with update actions feature

```
features:
- type: "update-actions"
 backup: "ask"
```

{% configuration features %} type: required: true description: " `update-actions` " type: string  
backup: required: false description: Whether a backup should be done before updating. The value can be `ask` , `yes` , or `no` . `ask` will open a dialog to ask if a backup should be done. type: list  
default: ask

## Vacuum commands

---

Widget that displays buttons to control a [vacuum](#).

Screenshot of the tile card with vacuum commands feature Screenshot of the tile card with vacuum commands feature

```
features:
- type: "vacuum-commands"
 commands:
 - start_pause
 - stop
 - clean_spot
 - locate
 - return_home
```

{% configuration features %} type: required: true description: "[vacuum-commands](#)" type: string  
commands: required: true description: List of commands to show on the card. The list can contain [start\\_pause](#), [stop](#), [clean\\_spot](#), [locate](#), and [return\\_home](#). type: list

## Valve open/close

---

Widget that displays buttons to open, close, or stop a [valve](#).

Screenshot of the tile card with open/close feature Screenshot of the tile card with valve open/close feature

```
features:
 - type: "valve-open-close"
```

{% configuration features %} type: required: true description: " `valve-open-close` " type: string

## Valve position

---

Widget that displays a slider to control the position for a [valve](#).

Screenshot of the tile card with the valve position feature Screenshot of the tile card with the valve position feature

```
features:
 - type: "valve-position"
```

{% configuration features %} type: required: true description: " `valve-position` " type: string

## Valve favorite positions

---

Widget that displays a dropdown with favorite positions for a [valve](#).

You can customize favorites in a valve's **More info** dialog. To edit them, press and hold a favorite.

Screenshot of the tile card with the valve favorite positions feature Screenshot of the tile card with the valve favorite positions feature

```
features:
 - type: "valve-position-favorite"
```

{% configuration features %} type: required: true description: " `valve-position-favorite` " type: string

## Water heater operation modes

---

Widget that displays buttons to control the operation mode of a [water heater](#).

Screenshot of the tile card with the water heater operation modes feature Screenshot of the tile card with the water heater operation modes feature

```
features:
- type: "water-heater-operation-modes"
 operation_modes:
 - electric
 - gas
 - heat_pump
 - eco
 - performance
 - high_demand
 - "off"
```

{% configuration features %} type: required: true description: " `water-heater-operation-modes` " type: string operation\_modes: required: false description: List of modes to show on the card. The list can contain `electric` , `gas` , `heat_pump` , `eco` , `performance` , `high_demand` , and `off` . If not set, all operation modes supported by the entity are shown. type: list

## Area control

---

Widget that displays buttons to control different types of entities in your area. You can control all entities of a specific domain or select individual entities.

Screenshot of the area card with the area controls feature Screenshot of the area card with the area controls feature

```
features:
 - type: "area-controls"
 controls:
 - light
 - fan
 - switch
 - entity_id: light.kitchen_counter
```

{% configuration features %} type: required: true description: " `area-controls` " type: string  
controls: required: false description: List of controls to show on the card. The list can contain domain names like `light` , `fan` , and `switch` , or mappings that specify a particular entity by using the `entity_id` key, as shown in the example above. If not set, the default set of controls supported in the area is shown. type: list

---

## Dashboards/Header Footer

---

Some dashboard cards have support for header and footer widgets. These widgets fill up the whole available space in a card.

Screenshot of an entities card with a picture header. Screenshot of an entities card with a picture header and buttons footer.

Header and footer can be used on the following cards:

- [Entity](#)
- [Entities](#)
- [Statistic](#)



## Picture header & footer

---

Widget to show a picture as a header or a footer. A picture can have touch actions associated with it.

```
header:
 type: picture
 image: "https://www.home-assistant.io/images/dashboards/header-footer/balloons"
```

{% configuration header-footer %} type: required: true description: " **picture** " type: string image: required: true description: The URL of an image. type: string alt\_text: required: false description: Alternative text for the image. This is necessary for users of assistive technology. The [W3C images tutorial](#) provides simple guidance for writing alternative text. type: string tap\_action: required: false description: Action taken on card tap. See [action documentation](#). type: map hold\_action: required: false description: Action taken on tap-and-hold. See [action documentation](#). type: map double\_tap\_action: required: false description: Action taken on double tap. See [action documentation](#). type: map

## Buttons header & footer

---

Widget to show entities as buttons in the header or footer.

```
footer:
 type: buttons
 entities:
 - script.launch_confetti
 - entity: script.swirl_lights
 icon: "mdi:track-light"
 - entity: script.run_siren
 icon: "mdi:alarm-light"
```

{% configuration header-footer %} entities: required: true description: A list of entities to show. Each entry is either an entity ID or a map. type: list keys: entity: required: true description: The entity ID to render. type: string icon: required: false description: Override the entity icon. You can use any icon from [Material Design Icons](#). Prefix the icon name with `mdi:`, ie `mdi:home`. type: string image: required: false description: Override the entity image. type: string name: required: false description: Label for the button. type: string show\_icon: required: false description: Show entity icon. type: boolean default: "true"

show\_name: required: false description: Show entity name. type: boolean default: "false"

tap\_action: required: false description: Action taken on button tap. See [action documentation](#). type: map hold\_action: required: false description: Action taken on tap-and-hold. See [action documentation](#). type: map double\_tap\_action: required: false description: Action taken on double tap. See [action documentation](#). type: map

{% endconfiguration %}

## Graph header & footer

---

Widget to show an entity in the sensor domain as a graph in the header or footer.

Screenshot of an entities card with a graph footer. Screenshot of an entities card with a graph footer.

```
footer:
 type: graph
 entity: sensor.outside_temperature
 hours_to_show: 24
 detail: 1
```

{% configuration header-footer %} entity: required: true description: Entity ID of `sensor` domain. type: string detail: required: false description: "Detail level of the graph: `1` or `2` (`1` = one point/hour, `2` = six points/hour)" type: integer default: 1 hours\_to\_show: required: false description: Hours to show in graph. Minimum is 1 hour. Big values can result in delayed rendering, especially if the selected entities have a lot of state changes. type: integer default: 24

 **Note:** The `hours_to_show` option controls the time range of historical data shown in the graph. The amount of history available depends on the Recorder's `purge_keep_days` setting. By default, the Recorder purges data older than 10 days. See the [Recorder integration documentation](#) for more information.

## Dashboards/Index

---

Your dashboard is what you and your family see when you open the Home Assistant app or website. It is the at-a-glance view of your home: which lights are on, what the temperature is in each room, who is home, and what is happening right now. From the same dashboard, a single tap dims the lights, starts your robot vacuum, or arms the alarm.

Home Assistant comes with a dashboard out of the box, automatically generated from the devices you add. From there, you can build your own dashboards visually, with drag and drop, no coding required. Make a focused dashboard for the kitchen tablet, a simple one for guests, and a detailed one for yourself.

You can customize your dashboard using various options:

- Different card types to visualize your data and control your smart home devices.
- [Themes](#) (even at a per card basis).
- Override names and icons of entities.
- Use custom cards from our amazing community.

Screencast showing how to edit a dashboard in section view and how to rearrange cards

Screencast showing how to edit a dashboard in section view and how to rearrange cards

## Explore the interactive demo dashboard

---

Try it yourself with [the interactive demo](#).

## Get started with your own dashboard

---

To create your own dashboard, follow the steps on [creating a new dashboard](#).

## Discuss dashboard

---

- Suggestions are welcome in the [frontend repository](#)
  - For help with dashboards, join the `#frontend` channel on [our chat](#) or [our forums](#)
-

## Dashboards/Naming

---

All dashboard cards and badges that use an entity support flexible name configuration. This allows you to display custom text, show contextual information like area or device names, or use the entity name instead of its friendly name.

The name configuration helps you avoid redundant information (like repeating device names on every card) and create more contextual, easier-to-understand dashboards.



## Simple string name

---

The simplest way to set a custom name is to use a string to override the entity's display name.

```
name: "My custom name"
```

{% configuration name-string %} name: required: false description: A custom string to display as the name. type: string

## Name object

---

The name can also be configured as an object with a `type` property to display contextual information about the entity.

### Entity name

Displays the entity name instead of the friendly name. The entity name is the specific function or data point the entity represents, without the device or area prefix.

```
name:
 type: entity
```

This is useful when you want to avoid repeating device or area names that are already included in the friendly name.

{% configuration name-entity %} name: required: false description: Name configuration object. type: map keys: type: required: true description: "Type of name to display. Use `entity` to show the entity name." type: string

### Device name

Displays the name of the device the entity belongs to.

```
name:
 type: device
```

{% configuration name-device %} name: required: false description: Name configuration object. type: map keys: type: required: true description: "Type of name to display. Use `device` to show the device name." type: string

### Area name

Displays the name of the area the entity is assigned to.

```
name:
 type: area
```

{% configuration name-area %} name: required: false description: Name configuration object. type: map keys: type: required: true description: "Type of name to display. Use `area` to show the area name." type: string

### Floor name

Displays the name of the floor where the entity's area is located.

```
name:
 type: floor
```

{% configuration name-floor %} name: required: false description: Name configuration object. type: map keys: type: required: true description: "Type of name to display. Use `floor` to show the floor name." type: string

## Custom text

Displays custom static text using the object syntax. This is useful when combining with other name types in a list.

```
name:
 type: text
 text: "My label"
```

{% configuration name-text %} name: required: false description: Name configuration object. type: map keys: type: required: true description: "Type of name to display. Use `text` to show custom text." type: string text: required: true description: The custom text to display. type: string

## Combining multiple name parts

---

You can combine multiple name components by using a list. This allows you to create hybrid names with contextual information. A space is automatically added between each component.

If a name component does not have a value, like when an entity has no area assigned, Home Assistant will automatically skip that component. This means only the components with values will be shown, and you will not see empty spaces or errors in the card name.

```
name:
 - type: area
 - type: device
```

This example would display something like "Living Room Light Switch".

{% configuration name-list %} name: required: false description: A list of names to combine. Each item must be a name object. type: list

## Examples

---

### Simple string name

```
type: tile
entity: light.living_room_ceiling_light
name: "Ceiling light"
```

This is the simplest way to set a custom name, overriding the entity's display name with a static string.

### Using entity name to avoid repetition

```
type: tile
entity: sensor.living_room_sensor_temperature
name:
 type: entity
```

This displays the entity name "Temperature" instead of the full friendly name "Living room sensor Temperature", avoiding repetition when the card is already grouped by area.

### Combining device and area

```
type: tile
entity: media_player.living_room_tv
name:
 - type: area
 - type: device
```

This combines the area name with the device name, displaying something like "Living room TV".

### Using custom text with other name types

```
type: button
entity: switch.kitchen_lights
name:
 - type: area
 - type: text
 text: "lights"
```

This would display "Kitchen lights" by combining the area name with custom text.

---

## Dashboards/Views

---

A view is a tab inside a dashboard. For example, the screenshot below shows a separate view for lights on the Overview dashboard.

Screenshot of a light view tab on the Overview dashboard A lights view tab on the Overview dashboard

Views control the layout.

Three of the four view types: Panel, Sidebar, and Masonry Three of the four view types are shown: Panel, Sidebar, and Masonry. The default Sections view is not pictured.

There are four different view types:

- **Sections (default):** Arranges cards in a grid system and lets you group them in sections.
- **Masonry:** Arranges cards in columns based on their card size.
- **Panel:** Displays one card in full width. For example a map or an image.
- **Sidebar:** Arranges cards in 2 columns, a wide one and a smaller one on the right.

## Adding a view to a dashboard

---

1. To add a view to your dashboard, in the top right corner, select the pencil icon.

2. Select the  button in the top menu bar.

Views toolbar

3. Define the view settings:

4. If you want a view title, enter the **Title**.

5. If you want to see an icon, select the [view icon](#).

- If an icon is defined, only the icon is shown. The text title is shown as a tooltip. To show both the icon and the text title, enable the **Show icon and title** toggle.

- We use [Material icons](#).

6. If you want to link to another view, define the [URL](#).

7. If you want to use a previously defined theme, select the [theme](#).

8. Select the [view type](#).

9. If this view is meant to be used as a [subview](#) only, enable the **Subview** toggle.

10. If you are using **Sections view**, choose the number of columns you want to use, and, if you want to let the system fill gaps between cards, enable **Dense section placement**.

The create new view configuration dialog

1. To use a background image, on the **Background** tab, select an image and customize the background settings. [Read more about these options](#).

2. On the **Badges** tab, select the entities you want to be represented by a badge.

- Sidebar and panel views do not support badges.

3. By default, the new section is visible to all users. On the **Visibility** tab, you can disable the view for users.

## Migrating a view into a sections view

---

If you have already defined a view but you would now like to have it in a section view type, you can migrate that content. For example, you can migrate from a masonry to a sections view. Currently, you cannot migrate a sections view type into another view type.

Migrating does not affect the current view. It will stay as is, and a new, additional view is created.

To migrate a view into a sections view type, follow these steps:

1. Open the view you want to migrate, and go into edit mode.
2. In the configuration dialog, select the new view type.
3. If the new view type offers additional settings, define those settings.
4. For more information on those settings, refer to the documentation of that view type.
5. In the top-right corner, select **Convert**.
6. Result: A new, additional view is created.
7. Your current view will stay untouched.
8. A new tab opens, and all your cards are imported to the new view.
9. In the **Imported cards** section, pick each of the cards, and drag them into the sections.
10. To edit and customize the view, follow the steps in the [sections view](#) documentation.  
  
Move cards from imported cards section onto your dashboard 6. To save your changes, select **Done**. - Result: Your new dashboard is shown. - If you have cards that were not yet integrated, you can still add them later. They are still available in the Edit mode, in the **Imported cards** section.



## URL of a view

---

You can link to one view from a card in another view when using cards that support navigation (`navigation_path`). The string supplied here will be appended to the string `/lovelace/` to create the path to the view. Do not use special characters in paths. Do not begin a path with a number. This will cause the parser to read your path as a view index.

### Example

View configuration:

```
- title: Living room
 # the final path is /lovelace/living_room
 path: living_room
```

Picture card configuration:

```
- type: picture
 image: /local/living_room.png
 tap_action:
 action: navigate
 navigation_path: /lovelace/living_room
```

## View icon

---

If a view icon is defined, only the icon is shown. The text title is shown as a tooltip. To show both the icon and the text title, enable the **Show icon and title** toggle.

### Examples

```
- title: Garden
 icon: mdi:flower
```

```
- title: Garden
 icon: mdi:flower
 show_icon_and_title: true
```

## Visible

---

You can specify the visibility of views as a whole or per-user. (Note: This is only for the display of the tabs. The URL path is still accessible)

### Example

```
views:
- title: Ian
 visible:
 - user: 581fca7fdc014b8b894519cc531f9a04
 cards:
 ...
- title: Chelsea
 visible:
 - user: 6e690cc4e40242d2ab14cf38f1882ee6
 cards:
 ...
- title: Admin
 visible: db34e025e5c84b70968f6530823b117f
 cards:
 ...
```

### Options for visible objects

If you define `visible` as objects instead of a boolean to specify conditions for displaying the view tab:

{% configuration badges %} user: required: true description: User ID that can see the view tab (unique hex value found on the Users configuration page). type: string

## Changing the view type in YAML

---

You can change the layout of a view in YAML by using a different view type. The default is `section`.

### Example

```
- title: Map
 type: panel
 cards:
 - type: map
 entities:
 - device_tracker.demo_paulus
 - zone.home
```

## Theme

---

Set a separate `theme` for the view and its cards.

### Example

```
- title: Home
 theme: happy
```

## Background

---

The background settings of a view can be customized to display a background. Alternatively, a theme variable can be used to customize the background of all views.

### View-specific background settings

**Image** - Sets the background image to use behind the view: - **Upload picture** lets you pick an image from the system used to show your Home Assistant UI. - **Local path** lets you pick an image stored on Home Assistant. For example: `/homeassistant/images/lights_view_background_image.jpg`. - To store an image on Home Assistant, you need to [configure access to files](#), for example via [Samba](#) or the [Studio Code Server](#) app. - **web URL** let you pick an image from the web. For example `https://www.home-assistant.io/images/frontpage/assist_wake_word.png`.

{% configuration views %} background: required: false description: Customize the view's background with options for image, transparency, size, alignment, repeat, and attachment. type: map keys: image: required: false description: Sets the background image to use behind the view. type: string opacity: required: false description: Adjust the background's opacity, from fully opaque to transparent. type: integer default: 100 size: required: false description: Choose how the background fits the space. Defaults to the original picture size, fill view (`cover` in YAML) fills the view with cropping if necessary and fits view (`contain` in YAML) fits the image within the view, maintaining aspect ratio. type: string default: auto alignment: required: false description: Precisely position the background. Valid options can be anything between top left and bottom right, with center being the default. type: string default: center repeat: required: false description: Controls whether the background repeats across the view. Repeating is useful when a tiled background is being used. type: string default: no-repeat attachment: required: false description: Controls whether a background image's position is fixed within the view, or scrolls. type: string default: scroll

### Example

```
Example background section in view yaml
background:
 image: /local/background.png
 opacity: 50 # any percentage between 0 and 100
 size: auto # auto, cover, contain
 alignment: center # top left, top center, top right, center left, center, center right, bottom left, bottom center, bottom right
 repeat: no-repeat # repeat, no-repeat
 attachment: scroll # scroll, fixed
```

### Background theme variable

You can style the background of all your views with a [theme](#). You can use the CSS variable `lovelace-background`. For wallpapers you probably want to use the example below, more options can be found [here](#).

## Example

```
Example configuration.yaml entry
frontend:
 themes:
 example:
 lovelace-background: center / cover no-repeat url("/local/background.png")
```

## Subview

---

A "View" can be marked as "Subview". Subviews won't show up in the navigation bar on top of the sidebar. Subviews can, for instance, be used to show detailed information; you could link to this subview from a page with a clean look with only basic information (by using [cards that support the `navigate` action](#)). Think of a view with a few thermostats and a subview with status information on the heating/cooling device.

When on the subview, the navigation bar only shows the name of the subview and a back button (no icon is shown). By default, clicking on back button will navigate to the previous view but a custom back path (`back_path`) can be set.

You can access subviews from other parts of your dashboard by using [cards that support the `navigate` action](#).

### Example

Simple subview:

```
- title: Map
 subview: true
```

Subview with custom back path:

```
- title: Map
 subview: true
 back_path: /lovelace/home
```

{% configuration views %} views: required: true description: A list of view configurations. type: list keys: type: required: false description: The type of the view. type: string default: masonry title: required: true description: The title or name. type: string badges: required: false description: List of entities IDs or `badge` objects to display as badges. Note that badges do not show when view is in panel mode. type: list cards: required: false description: Cards to display in this view. type: list path: required: false description: Paths are used in the URL. type: string default: view index icon: required: false description: Icon-name from Material Design Icons. You can use any icon from [Material Design Icons](#). Prefix the icon name with `mdi:`, ie `mdi:home`. Only for "View", not for "Subview". type: string show\_icon\_and\_title: required: false description: Show both icon and text title. Only for "View", not for "Subview". type: boolean default: false background: required: false description: Style the background behind the view. type: map theme: required: false description: Themes view and cards. type: string visible: required: false description: "Hide/show the view tab from all users or a list of individual `visible` objects." type: [boolean, list] default: true subview: required: false description: Mark the view as "Subview". type: boolean default: false back\_path: required: false description: Only for "Subview". Path to navigate when clicking on back button. type: string



## Example

View configuration:

```
- title: Living room
 badges:
 - device_tracker.demo_paulus
 - entity: light.ceiling_lights
 name: Ceiling Lights
 icon: mdi:bulb
 - entity: switch.decorative_lights
 image: /local/lights.png
```

Subview configuration:

```
- title: "Energieprijzen"
 path: "energieprijzen"
 subview: true
 back_path: "/ui-data/climate"

 cards:
 - type: entities
 entities:
 - sensor.today_avg_price
```

---

# Contributor License Agreement

---

By making a contribution to this project, I certify that:

- (a) The contribution was created in whole or in part by me and I have the right to submit it under the Apache 2.0 license; or
- (b) The contribution is based upon previous work that, to the best of my knowledge, is covered under an appropriate open source license and I have the right under that license to submit that work with modifications, whether created in whole or in part by me, under the Apache 2.0 license; or
- (c) The contribution was provided directly to me by some other person who certified (a), (b) or (c) and I have not modified it.
- (d) I understand and agree that this project and the contribution are public and that a record of the contribution (including all personal information I submit with it) is maintained indefinitely and may be redistributed consistent with this project or the open source license(s) involved.

## Attribution

---

The text of this license is available under the [Creative Commons Attribution-ShareAlike 3.0 Unported License](#). It is based on the Linux [Developer Certificate Of Origin](#), but is modified to explicitly use the Apache 2.0 license and not mention sign-off.

## Signing

---

If you have not signed the CLA and you submit a pull request to a repository under the Home Assistant organization, a link will be automatically generated. Just follow the link and the instructions in the link.

## Adoption

---

This Contributor License Agreement (CLA) was first announced on January 21st, 2017 in [this](#) blog post and adopted January 28th, 2017.

---

## Developers/Credits

---

This page contains a list of people who have contributed in one way or another to Home Assistant. Hover over a username to see their contributions.

### Author

- Paulus Schoutsen (@balloob)

### Contributors

(in alphabetical order)

- 0bmay (@0bmay)
- 100ferhas (@100ferhas)
- 15goudreau (@15goudreau)
- 1DontEx1st (@1DontEx1st)
- 1v0dev (@1v0dev)
- 235816 (@235816)
- 333ryan18 (@333ryan18)
- 4lloyd (@4lloyd)
- 50m3rer0 (@50m3rer0)
- 53Industries (@53Industries)
- 5mauggy (@5mauggy)
- 7even (@hwikene)
- 9R (@9R)
- 9rpp (@9rpp)
- [p]øs (@pyos)
- [Security] (@smartechru)
- A C++ MaNong (@zhuqf)
- A Gomes (@ambgomes)
- a1ex4 (@a1ex4)
- a-andre (@a-andre)
- a-r-j-a-n (@a-r-j-a-n)
- aaamoeder (@aaamoeder)
- Aalian Khan (@AalianKhan)
- aapjeisbaas (@aapjeisbaas)
- Aarni Koskela (@akx)
- Aaron (@mcd1992)
- Aaron (@xeanhort)
- Aaron (@aaroncmills)

- Aaron Bach (@bachya)
- Aaron Cunnington (@azcn2503)
- Aaron David Schneider (@AaronDavidSchneider)
- Aaron Godfrey (@boralyl)
- Aaron Linville (@linville)
- Aaron Malone (@aaroncm)
- Aaron Morris (@Morrisai)
- Aaron Parecki (@aaronpk)
- Aaron Polley (@xarnze)
- Aaron Sierra (@aaron-sierra)
- Aaron Ten Clay (@aarontc)
- Aaron Wolen (@aaronwolen)
- Aaron Wood (@aaronjwood)
- aaronpace (@aaronpace)
- aaska (@aaska)
- Abadede (@Abadede)
- ABdataman (@ABdataman)
- Abdelmalek Benelouezzane (@malekbene)
- Abdul Hussain (@hussain-abdul)
- Abe Wiersma (@Snuggert)
- Abel Matser (@abelmatser)
- Abhimanyu Vishwakarma (@abhimanyuv-img)
- Abhishek Anand (@aa755)
- ablack89 (@ablack89)
- abondoe (@abondoe)
- ABOTlegacy (@ABOTlegacy)
- abrlox (@abrlox)
- absurdist81 (@absurdist81)
- Abílio Costa (@abmantis)
- acbee (@acbee)
- aceindy (@aceindy)
- Achilleas Pipinellis (@axilleas)
- acshef (@acshef)
- actionpotato (@actionpotato)
- ActuallyRuben (@RvanBaarle)
- Adam (@SilvrrGIT)
- Adam Allport (@AAllport)
- Adam Baxter (@voltageX)
- Adam Belebczuk (@sqldiablo)
- Adam Bogdał (@bogdal)

- Adam Brin (@abrin)
- Adam Cheng (@adamchengtkc)
- Adam Chyb (@chybby)
- Adam Cooper (@GenericStudent)
- Adam Dullage (@Dullage)
- Adam Duskett (@aduskett)
- Adam Duskett (@retoroot-linux)
- Adam Ernst (@adamjernst)
- Adam Feldman (@adamfeldman)
- Adam Garcia (@pancho-villa)
- Adam Goodbar (@adamgoodbar)
- Adam Gregory (@adamgreg)
- Adam Griffiths (@aogriffiths)
- Adam Heinrich (@adamheinrich)
- Adam James (@atj)
- Adam Knight (@ahknight)
- Adam Kropp (@akropp)
- Adam Król (@adamkrol93)
- Adam Liddell (@aaliddell)
- Adam Michaleski (@prairieapps)
- Adam Outler (@adamoutler)
- Adam Reznecek (@adreznec)
- Adam Starbuck (@starbuck93)
- Adam Stone (@astone123)
- Adam Whittingham (@AdamWhittingham)
- Adam Wujek (@awujek)
- Adam Žurek (@adaamz)
- adamaze (@adamaze)
- adamomg (@adamomg)
- adamp237 (@adamp237)
- Adde Lovein (@addelovein)
- Addison Lynch (@addisonlynch)
- Addo Solutions (@AddoSolutions)
- Adduq (@ifelsq)
- adebeun (@adebeun)
- ADeeds (@ADeeds)
- AdeZwart (@AdeZwart)
- Adi Roiban (@adiroiban)
- AdighaLogic (@AdighaLogic)
- adipose (@adipose)



- AdithyanI (@AdithyanI)
- Aditya Shevade (@adibis)
- AdmiralStipe (@AdmiralStipe)
- Adorem (@Adorem)
- adr29truck (@adr29truck)
- Adriaan Peeters (@apeeters)
- AdriaanIO (@AdriaanIO)
- Adrian (@adriankaylor)
- Adrian Campos (@adriancampos)
- Adrian Goins (@oskapt)
- Adrian Huber (@Adi146)
- Adrian Perez (@aperezdc)
- Adrian Popa (@mad-ady)
- Adrian Schröter (@adrianschroeter)
- Adrian Scillato (@ascillato)
- Adrian Sterr (@adriansterr)
- Adrian Suwała (@adriansuwala)
- Adrian Sweet (@asweet-thegoodpenguin)
- Adrian Yee (@brewt)
- adrian-vlad (@adrian-vlad)
- adrianmihalko (@adrianmihalko)
- Adrien Brault (@adrienbrault)
- Adrien Béraud (@aberaud)
- Adrien Decostre (@Wallamit)
- Adrien Foulon (@Tofandel)
- Adrien Gallouët (@angt)
- Adrien Ricciardi (@RICCIARDI-Adrien)
- Adrià Vilanova Martínez (@avm99963)
- Adrián López (@adrianlzt)
- Adrián Moreno (@adrianmo)
- aenea (@aenea)
- AEnterprise (@AEnterprise)
- Aeotec-ccheng (@Aeotec-ccheng)
- Aephir (@Aephir)
- aetolus (@aetolus)
- aex351 (@aex351)
- Agneev Mukherjee (@agneevX)
- agrieco (@agrieco)
- AGSPhoenix (@AGSPhoenix)
- aguedob (@aguedob)

- ahertz (@ahertz)
- Ahmet BARIŞ (@barisahmet)
- ahobsonsayers (@ahobsonsayers)
- aidbish (@aidbish)
- Aijay Adams (@aijayadams)
- ain-ver (@ain-ver)
- airthusiast (@airthusiast)
- aisbergde (@aisbergde)
- Aivaras Sevelevicius (@asev)
- aizerin (@aizerin)
- AJ Schmidt (@ajschmidt8)
- ajayjohn (@ajayjohn)
- Ajinkya Bawaskar (@ajinkyabawaskar)
- ajobbins (@ajobbins)
- AJStubbsy (@AJStubbsy)
- Ajurna (@ajurna)
- akargl (@akargl)
- akasma74 (@akasma74)
- akennedy-adtran (@akennedy-adtran)
- Akihiko Odaki (@akihikodaki)
- akloeckner (@akloeckner)
- Akriti Chadda (@akriticg)
- Akin Ömeroğlu (@akinomeroglu)
- Aladin (@aladin2000)
- Alain Tavan (@alain57)
- Alain Turbide (@Dilbert66)
- AlainH (@AlainH)
- alakdae (@alakdae)
- Alan Bowman (@alanbowman)
- Alan Byrne (@burnsie-la)
- Alan Fischer (@alanfischer)
- Alan Murray (@atmurray)
- Alan Ott (@signal11)
- Alan Ott (@alan-softiron-limited)
- Alan Quinby (@Alan-K2)
- Alan Tse (@alandtse)
- Alan Yaniger (@ayaniger)
- AlanLane1 (@AlanLane1)
- Alasdair Nicol (@alsdairnicol)
- Alastair D'Silva (@deece)

- Alba Mendez (@mildsunrise)
- Albatross (@DyingAlbatross)
- Albert Gouws (@KiLLeRRaT)
- Albert Lee (@trisk)
- Alberto Arias Maestro (@albertoarias)
- Alberto Geniola (@albertogeniola)
- AlCalzone (@AlCalzone)
- aldot (@aldot)
- Alec Holmes (@clockworkant)
- Alec Rust (@AlecRust)
- Aleix Murtra (@alemuro)
- Alejandro Almazán (@aalmazanarbs)
- Alejandro Del Rincón López (@alexdr1)
- Alejandro González (@AlexTMjugador)
- Alejandro Rivera (@AlejandroRivera)
- Aleks (@IrealiTY)
- Aleksandar Todorović (r3bl) (@inputsh)
- Aleksander Morgado (@aleksander0m)
- Aleksander Żarczyński (@o-l-o)
- Aleksandr Smirnov (@jaxer)
- Aleksey Gureiev (@alg)
- Aleksey Jurchenko (@alekseyjurchenko)
- alekslyse (@alekslyse)
- Alessandro Del Prete (@alexdelprete)
- Alessandro Di Felice (@difelice)
- Alessandro Ghedini (@ghedo)
- Alessandro Mogavero (@alexmogavero)
- Alessandro Pilotti (@alexpilotti)
- Alessandro Staniscia (@Odyno)
- Alessio Margelli (@alex9446)
- Alessio Papi (@alessio-papi)
- Alex (@shr00mie)
- Alex (@asbach)
- Alex (@newAM)
- Alex (@nnmalex)
- Alex (@buhralex)
- Alex (@MungoRae)
- Alex (@r-xela)
- Alex Bahm (@techfreak)
- Alex Baldwin (@rampartisan)

- Alex Barcelo (@alexbarcelo)
- Alex Calderon (@AlexCalderon02)
- Alex Cragg (@epicalex)
- Alex Crawford (@crawford)
- Alex Dobrynin (@adobrynin)
- Alex Fung (@paraselene)
- Alex Groleau (@awgneo)
- Alex Harvey (@infamy)
- Alex Henry (@Hyralex)
- Alex Iribarren (@alexiri)
- Alex Kaplan (@kaplan2539)
- Alex Kretzschmar (@ironicbadger)
- Alex Lauerman (@alexlauerman)
- Alex Loret de Mola (@EdgarVerona)
- Alex Mekkerling (@AlexMekkerling)
- Alex MF (@adsmf)
- Alex Molodoj (@Molodax)
- Alex Muller (@alexmuller)
- Alex Osadchyy (@aosadchyy)
- Alex Peters (@Lx)
- Alex Popoutsis (@apop880)
- Alex S (@asleeis)
- Alex S. Glomsaas (@SuperManifolds)
- Alex Solomaha (@CyanoFresh)
- Alex Suykov (@arsv)
- Alex Szlavik (@AlexSzlavik)
- Alex Thompson (@apt-itude)
- Alex Tzonkov (@attzonko)
- Alex van den Hoogen (@alex3305)
- Alex Vermulst (@alexdepalex)
- Alex Ward (@alxwrld)
- Alex X (@AlexxIT)
- Alex Yao (@alex Yao2015)
- Alex1234 (@Alex1234)
- alex6480 (@alex6480)
- Alex-Klein (@Alex-Klein)
- alex.bennee@linaro.org (@stsquad)
- Alexander (@ualex73)
- Alexander (@vtochq)
- Alexander 'z33ky' Hirsch (@z33ky)

- Alexander Bandukwala (@7h3kk1d)
- Alexander Clouter (@jimdigriz)
- Alexander Dahl (@LeSpocky)
- Alexander Egorenkov (@eaibmz)
- Alexander Egorenkov (@egorenar)
- Alexander Eisele (@dereisele)
- Alexander Foxleigh (@foxleigh81)
- Alexander Goldstone (@alexgoldstone)
- Alexander Groß (@agross)
- Alexander Hardwicke (@alexhardwicke)
- Alexander Hradetzky (@blitzkneisser)
- Alexander Kurz (@alexanderkurz)
- Alexander Leisentritt (@Alex9779)
- Alexander Lukichev (@alukichev)
- Alexander Lyon (@arlyon)
- Alexander Mukhin (@aimukhin)
- Alexander Petrov (@meanmail)
- Alexander Pitkin (@peleccom)
- Alexander Reinert (@alexreinert)
- Alexander Rey (@alexander0042)
- Alexander Ryazanov (@alryaz)
- Alexander Schneider (@alexschneider)
- Alexander Shiyani (@shcggit)
- Alexander Slansky (@aslansky)
- Alexander Stefanov (@fedya)
- Alexander Varnin (@fenixk19)
- Alexandre Belloni (@alexandrebelloni)
- Alexandre Esse (@ahresse)
- Alexandre Leites (@alexandre-leites)
- Alexandre Macabies (@zopieux)
- Alexandre Pereira da Silva (@aletes)
- Alexandre Perrin (@kaworu)
- Alexandre Prates Dias (@pratesbh)
- Alexandru Ardelean (@commodo)
- Alexandru Branza (@jaruba)
- alexanv1 (@alexanv1)
- Alexei Chetroi (@Adminiuga)
- Alexey (@spirit-x)
- Alexey 'Cluster' Avdyukhin (@ClusterM)
- Alexey Brodtkin (@abrodtkin)

- Alexey Kardashevskiy (@aik)
- Alexey Kustov (@papajojo)
- Alexey Mednyy (@swex)
- Alexey Neyman (@stilor)
- Alexey Pristavkin (@Pristavkin)
- Alexey Roslyakov (@e-yes)
- Alexey Savin (@savin-alexey)
- Alexey Zimarev (@alexeyzimarev)
- Alexis Iglauer (@ax42)
- Alexis Tyler (@OmgImAlexis)
- AlexSchmitz222 (@AlexSchmitz222)
- Alexxander0 (@Alexxander0)
- Alfie Day (@Azelpur)
- Alfiegener (@Alfiegener)
- Alfonso Sorrentino (@BigNocciolino)
- AlfredJKwack (@AlfredJKwack)
- Algirdas Č. (@algirdasc)
- Aliaksandr (@minchik)
- Alifer Moraes (@alifermoraes)
- alim4r (@alim4r)
- Alistair Francis (@alistair23)
- Alistair Galbraith (@alistairg)
- Alistair Young (@cerebrate)
- Allan Clark (@chickenandpork)
- Allan Glen (@allanglen)
- Allan Klein (@allanak)
- Allan P. (@allanpersson)
- Allan W. Nielsen (@allannielsen)
- Allen Derusha (@aderusha)
- Allen Porter (@allenporter)
- Allison (@Leapo)
- allserv (@allserv)
- Almog Moyal (@almogmoyal)
- Almost Engineer (@almostengr)
- almostserious (@almostserious)
- Alok Saboo (@arsaboo)
- Alon (@0xAlon)
- Alone (@al-one)
- Alpha Chen (@kejadlen)
- Alpha Tango (@alphetangoalpha)

- AlucardZero (@chennin)
- Alvaro Duarte (@ad-ha)
- Alvaro G. M. (@agamez)
- Alvi Mahadi (@amahadi)
- alvinchen1 (@alvinchen1)
- alxrdn (@alxrdn)
- amadeo-alex (@amadeo-alex)
- amahlaka (@amahlaka)
- Aman Gupta Karmani (@tmm1)
- ambarkhuizen (@ambarkhuizen)
- amigian74 (@amigian74)
- Amir Hanan (@Amir974)
- Amir hossein Hossein Zadeh Karimi (@AmirHosseinKarimi)
- Amit Keret (@amitkeret)
- amitfin (@amitfin)
- Amol Katdare (@amolkatdare)
- Amos Yuen (@amosyuen)
- Amran Haroon (@haroon3rd)
- ams123ios (@ams123ios)
- Ana Paula Gomes (@anapaulagomes)
- Anand Borkar (@anandb235)
- Anand Radhakrishnan (@anand-p-r)
- Anarion (@anarion80)
- Anastasia A (@Sacret)
- Anatoly (@dontbug)
- Anatoly Borodin (@anatolyborodin)
- Anders Darander (@darander)
- Anders Einar Hilden (@Kagee)
- Anders Fogh Eriksen (@Fogh)
- Anders Gjendem (@agjendem)
- Anders Liljekvist (@thrawnarn)
- Anders Ljusberg (@andlju)
- Anders Melchiorson (@amelchio)
- Anders Norås (@anoras)
- Anders Roxell (@roxell)
- Andi (@h4de5)
- andig (@andig)
- Andre (@andre68723)
- Andre Lengwenus (@alengwenus)
- Andre Renaud (@AndreRenaud)

- Andre Richter (@andre-richter)
- Andrea (@andker87)
- Andrea (@andreabusi)
- Andrea Barbaresi (@barban-dev)
- Andrea Campi (@andreacampi)
- Andrea Cioccarelli (@cioccarellia)
- Andrea Donno (@adonno)
- Andrea G (@Muffo)
- Andrea Tosatto (@andtos90)
- Andreas (@a529987659852)
- Andreas (@andreas-amlabs)
- Andreas Billmeier (@onkelbeh)
- Andreas Björshammar (@abjorshammar)
- Andreas Brett (@andreasbrett)
- Andreas Cambitsis (@acambitsis)
- Andreas Ehn (@ehn)
- Andreas Franz (@andreasfranz)
- Andreas Hartl (@nd-net)
- Andreas Jacobsen (@andreasjacobsen93)
- Andreas Klinger (@it-klinger)
- Andreas Larsson (@andreas-gaisler)
- Andreas Lindhé (@lindhe)
- Andreas Oberritter (@mtdcr)
- Andreas Oetken (@oetken)
- Andreas Rammhold (@andir)
- Andreas Rehn (@DAMEK86)
- Andreas Renberg (IQAndreas) (@IQAndreas)
- Andreas Riddering (@Dunstkreis)
- Andreas Rydbrink (@easink)
- Andreas Sansano (@asansano)
- Andreas Setterlind (@Gamester17)
- Andreas Willich (@SabotageAndi)
- Andreas Wolter (@a-wolter)
- Andreas Wrede (@awrede)
- Andreas2430 (@Andreas2430)
- andreasfelder (@andreasfelder)
- Andreea-Daniela Ene (@AndreeaEne)
- Andrei (@gipnokote)
- Andrei Costescu (@cosandr)
- Andrei Pop (@andreipop2005)



- Andrei Popovici (@andreipopovici)
- Andrej Friesen (@ajfriesen)
- Andrej Shadura (@andrewshadura)
- Andrejs (@tlpbu)
- Andrew (@aneisch)
- Andrew (@aoakeson)
- Andrew (@adpriebe)
- Andrew (@a005)
- Andrew Ash (@ash211)
- Andrew Barnes (@UmbraMalison)
- Andrew Berry (@deviantintegral)
- Andrew Beveridge 🍷 (@beveradb)
- Andrew Bullock (@trullock)
- Andrew Chatham (@achatham)
- Andrew Cockburn (@acockburn)
- Andrew Donnellan (@ajdlinux)
- Andrew Dunham (@andrew-d)
- Andrew Fahrenholtz (@PlasmaEye)
- Andrew Garrett (@werdnum)
- Andrew Grimberg (@tykeal)
- Andrew Hall (@FattusMannus)
- Andrew Hayworth (@ahayworth)
- Andrew Jackson (@andrew-codechimp)
- Andrew LeCody (@aceat64)
- Andrew Loe (@loe)
- Andrew Ma (@ajma)
- Andrew Marks (@ajmarks)
- Andrew McRae (@aamcrae)
- Andrew Onyshchuk (@oandrew)
- Andrew Patton (@acusti)
- Andrew Rabert (@nvlsvm)
- Andrew Rice (@riceman85)
- Andrew Riley (@andrewcr7)
- Andrew Rowson (@growse)
- Andrew Ruder (@aeruder)
- Andrew Sayre (@andrewsayre)
- Andrew Scheller (@lurch)
- Andrew Simmons (@agsimmons)
- Andrew Smith (@andrewmichaelsmith)
- Andrew Soback (@asoback)

- Andrew Starr-Bochicchio (@andrewsomething)
- Andrew Stock (@watchforstock)
- Andrew Thigpen (@andythigpen)
- Andrew Webster (@andrwwbstr)
- Andrew Wedgbury (@sconemad)
- Andrew Williams (@nikdoof)
- Andrew Wong (@featherbear)
- Andrew Ying (@andrewying)
- Andrew55529 (@Andrew55529)
- andrew-curtis (@andrew-curtis)
- andrewdolphin (@andrewdolphin)
- andrewfoster (@andrewfoster)
- andrews-tech (@andrewstech)
- Andrey (@andrey-git)
- Andrey (@divanikus)
- Andrey "Limych" Khrolenok (@Limych)
- Andrey Gorbunov (@gorbunovav)
- Andrey Kupreychik (@foxel)
- Andrey Mikhaylov (lolmaus) (@lolmaus)
- Andrey Petrov (@anpetrov)
- Andrey Skvortsov (@AndreySV)
- Andrey Smirnov (@ndreys)
- Andrey Ulanov (@aulanov)
- Andrey Yurovsky (@yurovsky)
- Andrius Štikonas (@stikonas)
- androidemil (@androidemil)
- Andrzej (@andriej)
- Andrzej Chmielewski (@andrzejchm)
- Andrzej Pichliński (@apichlinski)
- Andrzej Raczkowski (@araczkowski)
- András Rutkai (@rutkai)
- André Bação (@abacao)
- André Hahn (@ahahn94)
- André Lemos (@avlemos)
- André Lobo (@andrewolfy)
- André Matthies (@matthiez)
- André Zwing (@AndreRH)
- Andréas Lundgren (@adevade)
- andvikt (@andvikt)
- Andy Allsopp (@arallsopp)

- Andy Castille (@Klikini)
- Andy Cordill (@acordill)
- Andy Gibbs (@andyg1001)
- Andy Grunwald (@andygrunwald)
- Andy Jackson (@andybjackson)
- Andy Kittner (@andkit)
- Andy Lindeman (@alindeman)
- Andy Loughran (@andylockran)
- Andy Piper (@andypiper)
- Andy Shevchenko (@andy-shev)
- Andy9FromSpace (@Andy9FromSpace)
- andyat (@andyat)
- andyboeh (@andyboeh)
- aneeshd (@aneeshd)
- angel12 (@angel12)
- Angeliki Papadopoulou (@apapadopoulou)
- Angelo Compagnucci (@angeloc)
- Angelo Gagliano (@TheGardenMonkey)
- Angie1313 (@Angie1313)
- Anglac (@Anglac)
- Ani Betts (@anaisbetts)
- Aniket (@HandyHat)
- Anil Daoud (@AnilDaoud)
- Anisse Astier (@anisse)
- Anna Prosvetova (@aprosvetova)
- Anna Shipman (@annashipman)
- Anna Tikhomirova (@anyuta1166)
- annuges (@annuges)
- anotherjulien (@anotherjulien)
- anotherthomas (@anotherthomas)
- Anrolosia (@Anrolosia)
- anrudolph (@anrudolph)
- Ansgar Mertens (@ansgarm)
- Anssi Hannula (@anssih)
- Antetokounpo (@Antetokounpo)
- AnthiasD (@AnthiasD)
- Anthony Arnaud (@aarnaud)
- Anthony Carbone (@ant-car)
- Anthony Hughes (@tony2nite)
- antihate8 (@antihate8)

- antipiot (@antipiot)
- Antoine (@Twanislas)
- Antoine Bertin (@Diaoul)
- Antoine GRÉA (@grea09)
- Antoine Meillet (@inetAnt)
- Antoine Pierlot-Garcin (@bok)
- Antoine Ténart (@atenart)
- Anton Averkiev (@wowgamr)
- Anton Babenko (@antonbabenko)
- Anton Blanchard (@antonblanchard)
- Anton Chakirov (@achakirov)
- Anton Glukhov (@toxxin)
- Anton Johansson (@anton-johansson)
- Anton Kolesov (@anthony-kolesov)
- Anton Lundin (@glance-)
- Anton Malko (@ANMalko)
- Anton Palgunov (@Toxblh)
- Anton Sarukhanov (@antsar)
- Anton Tolchanov (@knyar)
- Anton Verburg (@antonverburg)
- Anton-Juul-Naber (@atjn)
- Antongiaco Polimeno (@antongiaco)
- Antoni Czaplicki (@Antoni-Czaplicki)
- Antoni K (@antoni-k)
- Antoni Róžański (@arozans)
- Antonino Piazza (@a-p-z)
- Antonio Larrosa (@antlarr)
- Antonio Párraga Navarro (@aparraga)
- Antonio Pérez (@skarcha)
- Antony Messerli (@antonym)
- Antony Pavlov (@frantony)
- Antti K. Koskela (@koskila)
- Anubhaw Arya (@aarya123)
- anugs (@anugs)
- Anurag El Dorado (@aedorado)
- aomann (@aomann)
- apaperclip (@apaperclip)
- apastuszek (@apastuszek)
- apetrycki (@apetrycki)
- apo-mak (@apo-mak)

- Appleguru (@appleguru)
- apworks1 (@apworks1)
- aquarium (@theaquarium)
- aque0us (@aque0us)
- aquesnel (@aquesnel)
- AR (@sp3c73r2038)
- arantes555 (@arantes555)
- Aras Vaichas (@mrstinky)
- Arbuzov Sergey (@Arbuzov)
- arcsi42 (@arcsi42)
- arcsur (@arcsur)
- Arda ŞEREMET (@ArdaSeremet)
- Ardetus (@ardzeus)
- ardevd (@ardevd)
- Ardi Mehist (@omgapuppy)
- arg0nik (@arg0nik)
- Ari (@arigit)
- Ari Lotter (@arilotter)
- Ari Simonen (@asimonen)
- Ari Sosnovsky (@asosnovsky)
- Ariana Hlavaty (@Ariana-Hlavaty-i2)
- aribarreto (@aribarreto)
- Ariel D'Alessandro (@adalessandro)
- Ariel Voskov (@Voskov)
- Arielpod (@Arielpod)
- Arif Widi Nugroho (@arifwn)
- arigilder (@arigilder)
- Arik Yavilevich (@ayavilevich)
- Arjan van Balken (@vanbalken)
- arjenfvellinga (@arjenfvellinga)
- arjenvrh (@arjenvrh)
- Arman Taherian (@armantaherian)
- Arnaud (@aaujon)
- Arne Mauer (@arnemauer)
- Arno (@aetjansen)
- Arno M (@arnom-ms)
- ArnoGit (@ArnoGit)
- Arnold Bloemert (@abloemert)
- arnout (@arnout)
- Aron Hafner (@hafffe)

- ArrayLabs (@arraylabs)
- Art M. Gallagher (@artmg)
- arteck (@arteck)
- Artekus (@Artekus)
- Artem (@ArtHome12)
- Artem Draft (@Drafteed)
- Artem Poliukhovych (@nergal)
- Artem Senichev (@artemsen)
- Artem Sorokin (@dext0r)
- Artem Tokarev (@RealArtemiy)
- Arthur (@arthurjdam)
- Arthur LAMBERT (@Evanok)
- Arthur Leonard Andersen (@leoc)
- Arthur Lutz (@arthurlutz)
- Arthur Zapparoli (@arthurgeek)
- Arto Jantunen (@viiru-)
- Artur 'Wodor' Wielogorski (@wodor)
- Artur L (@lorek123)
- arturdo (@arturdo)
- Artuto (@Artuto)
- Arun S. Sekher (@arunsekher)
- arunderwood (@arunderwood)
- Arvid Hahné (@maanrijp)
- Arvind Prasanna (@apasanna)
- Asaf Levy (@asaf400)
- asafkahlon (@asafkahlon)
- aschamberger (@aschamberger)
- aschmitz (@aschmitz)
- Asgeir Bjarni Ingvarsson (@asgeir)
- Ash Charles (@ashcharles)
- ash7777 (@ash7777)
- ashev (@ashev)
- Ashley 'DrToxic' Devine (@DrToxic)
- Ashley Cawley (@ashleycawley)
- Ashton Campbell (@AshtonCampbell)
- Ashton Lafferty (@cogneato)
- Askarov Rishat (@rishatik92)
- asowlowl (@asowlowl)
- Aspers (@Aspers)
- Assaf Inbal (@shmuelzon)

- astronaut (@rsegers)
- Atanas Palavrov (@palavrov)
- Atharva Lele (@atharvalele)
- atlflyer (@atlflyer)
- atorralba (@atorralba)
- Atticus Maximus (@amaximus)
- Attie Grande (@attie)
- Attila Lukács (@LouiS22)
- Aubin Paul (@outlyer)
- Audrey Motheron (@Audmo)
- Audric Schiltknecht (@chemicalstorm)
- Audun Ytterdal (@auduny)
- aufano (@aufano)
- augustas2 (@augustas2)
- Augustin Thiercelin (@n1tsu)
- auhlie (@auhlie)
- Auke Bruinsma (@air2)
- Aurelijus Rožėnas (@aurelijusrozenas)
- Aurélien Chabot (@trishika)
- ausserirdischesindgesund (@ausserirdischesindgesund)
- Austin (@trainman419)
- Austin Brunkhorst (@AustinBrunkhorst)
- Austin Drummond (@adrum)
- Austin Mroczek (@austinmroczek)
- austinlg96 (@austinlg96)
- automaton82 (@automaton82)
- avee87 (@avee87)
- Avi Miller (@Djelibeybi)
- Avi Schwab (@froboy)
- Avi Shukron (@avrahamshukron)
- Avishay Orpaz (@avishorp)
- avocadio (@avocadio)
- Avraham David Gelbfish (@adgelbfish)
- awkwardDuck (@awkwardDuck)
- Axel (@axel8viii)
- Axel Lin (@AxelLin)
- Axel Sepúlveda (@chepo92)
- ayatoy (@ayatoy)
- azeroth12 (@azeroth12)
- Azimoth (@Azimoth)

- [azrdev \(@azrdev\)](#)
- [b1g1an \(@bobbynobble\)](#)
- [B1ue-W01f \(@B1ue-W01f\)](#)
- [b3nj1 \(@b3nj1\)](#)
- [b4dpxl \(@b4dpxl\)](#)
- [B-Hartley \(@B-Hartley\)](#)
- [b-pass \(@b-pass\)](#)
- [backcountrymountains \(@backcountrymountains\)](#)
- [BackSlasher \(@BackSlasher\)](#)
- [badele \(@badele\)](#)
- [badguy99 \(@badguy99\)](#)
- [Bagira \(@BagiraHun\)](#)
- [Bai Yingjie \(@Bai-Yingjie\)](#)
- [bailz \(@bailz\)](#)
- [Bakkoda \(@Bakkoda\)](#)
- [Balamurugan P \(@balamurugan1603\)](#)
- [Balazs Keresztury \(@belidzs\)](#)
- [Balazs Sandor \(@sanyatuning\)](#)
- [balk77 \(@balk77\)](#)
- [Balázs Suhajda \(@suhajdab\)](#)
- [BamBamBam996 \(@TFhdKi95ae8L\)](#)
- [bangom \(@bangom\)](#)
- [Baptiste Candellier \(@outadoc\)](#)
- [Baptiste Lecocq \(@tiste\)](#)
- [Baptiste Moraine \(@bmoraine\)](#)
- [Baptiste Poirriez \(@bpoirriez\)](#)
- [Baran Kaynak \(@barankaynak\)](#)
- [barleybobs \(@barleybobs\)](#)
- [barney \(@barneyman\)](#)
- [BarrettLowe \(@BarrettLowe\)](#)
- [Barry Quiel \(@quielb\)](#)
- [Barry Williams \(@bazwilliams\)](#)
- [Barrysv \(@Barrysv\)](#)
- [Bart Janssens \(@barche\)](#)
- [Bart S \(@zBart\)](#)
- [Bart274 \(@Bart274\)](#)
- [bart-roos \(@bart-roos\)](#)
- [Bartek Celary \(@bcelary\)](#)
- [Bartosz Biłas \(@bbilas\)](#)
- [Bartosz Fenski \(@fenio\)](#)



- Bartosz Gołaszewski (@brgl)
- Baruch Rothkoff (@baruchiro)
- Baruch Siach (@baruchsiach)
- Bas (@basbl)
- Bas Delfos (@basdelfos)
- Bas Nijholt (@basnijholt)
- Bas Schipper (@basschipper)
- Bas Stottelaar (@basilfx)
- Bas Veeling (@basveeling)
- Bascht74 (@Bascht74)
- Bashir (@Brahmah)
- bashworth (@bash-worth)
- basst22778 (@basst22778)
- Bastian Stegmann (@stegmannb)
- bastien (@cbastienbaron)
- Bastien Gautier (@bastgau)
- bastshoes (@bastshoes)
- battlemoose (@battlemoose)
- Batári Balázs László (@bayi)
- baurandr (@baurandr)
- BazaJayGee66 (@BazaJayGee66)
- bchastain (@bchastain)
- bcl1713 (@bcl1713)
- Beat (@bdurrer)
- Beau Breeden (@BeauBreedem)
- Beau Simensen (@simensen)
- beavis9k (@beavis9k)
- Beddie (@Beddie)
- beepmill (@beepmill)
- beestree (@beestree)
- Bejje (@Bejje)
- bejoyjaison (@bejoyjaison)
- belgianrubs (@belgianrubs)
- Ben (@benleb)
- Ben (@benj-n)
- Ben (@unixben)
- Ben Aylott (@beaylott)
- Ben Bangert (@bbangert)
- Ben Bennett (@knobunc)
- Ben Boeckel (@mathstuf)

- Ben Dews (@bendews)
- Ben Doerr (@bendoerr)
- Ben Edmunds (@Tigger2014)
- Ben Felton (@itjedi42)
- Ben Hale (@nebhole)
- Ben Krajancic (@fantasmos)
- Ben McClure (@bmccclure)
- Ben Mckeown (@bmck8)
- Ben Menchaca (@bmenchaca)
- Ben Nelson (@nelsonblaha)
- Ben Nuttall (@bennuttall)
- Ben Origas (@borigas)
- Ben Pirt (@bjpirt)
- Ben Randall (@veleek)
- Ben Schattinger (@lights0123)
- Ben Shaner (@bens545)
- Ben Shelton (@beshelto)
- Ben Spoon (@spoonben)
- Ben Stolovitz (@citelao)
- Ben Thomas (@wazoo)
- Ben Van Mechelen (@benvm)
- ben wal (@benwalio)
- ben423423n32j14e (@ben423423n32j14e)
- benborra (@benborra)
- BenDaMAN303 (@BenDaMAN303)
- Bendik Brenne (@bendikrb)
- Benedict Aas (@Shou)
- Benedikt Böhm (@hollow)
- Benjamin (@bbr111)
- Benjamin Bryan (@ahnnoie)
- Benjamin Calderon (@benjcal)
- Benjamin Carlsson (@glacials)
- Benjamin Granzow (@Dysman)
- Benjamin Kamath (@kamathba)
- Benjamin Paap (@BenjaminPaap)
- Benjamin Parzella (@bparzella)
- Benjamin RAIBAUD (@braibaud)
- Benjamin Richter (@kilrogg)
- Benjamin Salchow (@benjamin-salchow)
- Benjamin Schmid (@bentolor)

- Benjamin Staffin (@benley)
- Benjamin Stürmer (@thebino)
- Benjamin Tripp (@BTripp1986)
- Benji (@bbbenji)
- Bennett Kanuka (@bkanuka)
- Benny (@Benny-Git)
- Benny de Leeuw (@leeuwte)
- Benoit Anastay (@BenoitAnastay)
- Benoit BESSET (@bbeset)
- Benoit Louy (@benoitlouy)
- Benoît Mauduit (@be-neth)
- Benoît Thébaudeau (@bthebaudeau)
- BenPru (@BenPru)
- bepsoccer (@bepsoccer)
- beren12 (@beren12)
- bergemalm (@bergemalm)
- berightback-dev (@berightback-dev)
- Bernardo Loureiro (@bernardolm)
- Bernardus Jansen (@bajansen)
- Bernd Amend (@BerndAmend)
- Bernd Schlapsi (@brot)
- Bernhard B (@bbernhard)
- Bernhard Hecker (@bairnhard)
- Berni Moses (@bernimoses)
- Bert Melis (@bertmelis)
- Bert Outtier (@bertouttier)
- Bert Roos (@Bert-R)
- Bertbert (@bertbert72)
- Bertie Blackman (@covertbert)
- bestlibre (@bestlibre)
- Beta-Computer (@Beta-Computer)
- bevosangryghost (@bevosangryghost)
- bforbird (@bforbird)
- Bhaaf (@Bhaaf)
- bheading (@bheading)
- Biagio Montaruli (@biagiom)
- bifurcated (@bifurcated)
- Big Mike (@mikelawrence)
- bigbadblunt (@bigbadblunt)
- biggms (@biggms)

- BigMoby (@bigmoby)
- bigwoof (@bigwoof)
- Bike Dude (@hackacad)
- Bilal Wasim (@bwasim)
- Bill (William) O'Neill (@woneill)
- Bill Church (@billchurch)
- Bill Durr (@billyburly)
- billsq (@billsq)
- Billy Stevenson (@Sotolotl)
- Bin Meng (@lbmeng)
- BingKn0s (@BingKn0s)
- binli71 (@binli71)
- binsentsu (@binsentsu)
- BioSehnsucht (@BioSehnsucht)
- biver0 (@biver0)
- Bjarne Riis (@briis)
- Bjarni Ivarsson (@bjarniivarsson)
- bjohnson8949 (@bjohnson8949)
- Björn Fredriksson (@Aangbaeck)
- Björn Lindahl (@blindahl)
- Björn Olsson Jarl (@frangiz)
- Björn Orri (@bjornorri)
- Björn Victor (@bictorv)
- Bjørn Forsman (@bjornfor)
- Bjørn Snoen (@bjornsnoen)
- bkcberry (@bkcberry)
- bkuhls (@bkuhls)
- bl8rnr (@bl8rnr)
- blackdog70 (@blackdog70)
- blackmesataiwan (@blackmesataiwan)
- blackray12 (@blackray12)
- blacktirion (@blacktirion)
- blackwind (@blackwind)
- blair (@blairun)
- blakadder (@blakadder)
- Blake (@sreknob)
- Blake Blackshear (@blakeblackshear)
- Blanyal D'Souza (@blanyal)
- blastoise186 (@blastoise186)
- blawford (@blawford)

- bleader (@bleader)
- Blear (@Blear)
- Blender3D (@Blender3D)
- blhoward2 (@blhoward2)
- blindlight86 (@blindlight86)
- blissarts (@blissarts)
- Blomme Maarten (@mblfir)
- Blue Charm Beacons (@BlueCharmBeacons)
- bluestripe (@bluestripe)
- bmaier-col (@bmaier-col)
- bmcccluskey (@bmcccluskey)
- Bo (@bohmandan)
- Bob Anderson (@rwa)
- Bob Clough (@thinkl33t)
- Bob Igo (@Human)
- Bob Matcuk (@bmatcuk)
- Bob van Mierlo (@bobvmierlo)
- Bob van Oijen (@bobvanoijen)
- bobbrowsers7 (@bobbrowsers7)
- bobnwk (@bobnwk)
- boc-the-git (@boc-the-git)
- Boked66 (@boked66)
- Bogdan Alexe (@bogdanalexe90)
- Bogdan Radulescu (@bogdanr)
- Bogdan Vlaicu (@bvlaicu)
- boltgolt (@boltgolt)
- bonterra (@bonterra)
- boojew (@boojew)
- boolangery (@boolangery)
- boradwell (@boradwell)
- Boris Gulay (@BoresXP)
- Boris K (@bokub)
- Boris Kaplounovsky (@bskaplou)
- borky (@borky)
- bottomquark (@bottomquark)
- bouni (@Bouni)
- Bouwe Westerdijk (@bouwew)
- Boyi C (@fanthos)
- bqstony (@bqstony)
- br0nd (@br0nd)

- Br4veSt4rr (@Br4veSt4rr)
- braam (@braam)
- Brad (@BradleyFord)
- brad (@oakbrad)
- Brad Ackerman (@backerman)
- Brad Buran (@bburan)
- Brad Choate (@bradchoate)
- Brad Dixon (@rbdixon)
- Brad Downey (@magic7s)
- Brad Fitzpatrick (@bradfitz)
- Brad Johnson (@bradsk88)
- Brad Keifer (@bradkeifer)
- Brad Love (@b-rad-NDi)
- Brad Sanders (@thebradleysanders)
- Brad Simmons (@easyas314)
- bradarndt (@digdugg)
- braddparker (@braddparker)
- bradford barr (@bradfordbarr)
- Bradley Nelson (@BCNelson)
- Bradley Wehmeier (@bradleywehmeier)
- BradleyDHobbs (@BradleyDHobbs)
- Brady Holt (@bradymholt)
- Brady Rosino (@BradyRosino)
- Brahma Fear (@brahmafear)
- Brainik (@Brainik)
- Bram Gerritsen (@bramstroker)
- Bram Goolaerts (@bollewolle)
- Bram Kragten (@bramkragten)
- Bram Mittendorff (@brammittendorff)
- Bram van Dartel (@xirixiz)
- Brandon (@bgulla)
- Brandon Chothia (@chothia)
- Brandon Corry (@brandoncorry)
- Brandon Davidson (@brandond)
- Brandon Mathis (@imathis)
- Brandon Rothweiler (@bdr99)
- Brandon Valentine (@bval)
- Brandon Weeks (@brandonweeks)
- Brandon West (@bwest1711)
- brandondb1 (@brandondb1)

- bransjr (@bransjr)
- Brant Knudson (@brantlk)
- Brazen00 (@Brazen00)
- BreakingBread0 (@BreakingBread0)
- breakthestatic (@breakthestatic)
- brefra (@brefra)
- Breina (@Breina)
- bremor (@bremor)
- Brenda Wallace (@Br3nda)
- Brendan Berg (@captainnapalm)
- Brendan Dahl (@dahlb)
- Brendan M. Sleight (@bmsleight)
- Brendan Tobolaski (@btobolaski)
- Brendan Ward (@brendanpward)
- Brenden Crowie (@bcrowie)
- Brendon Baumgartner (@bbrendon)
- Brendon Go (@brendongo)
- Breno Lima (@brenolimaxp)
- Brent Generous (@bgenerous-impinj)
- Brent Kerlin (@bkerlin)
- Brent Petit (@bjpetit)
- Brent Saltzman (@brent20)
- Brenton (@rbrenton)
- Brenton Zillins (@bzillins)
- Brett Adams (@Bre77)
- Brett Hiltbrand (@bjhiltbrand)
- Brett Lammers (@BALSwim50W)
- Brett T. Warden (@bwarden)
- Brian (@imbrianj)
- Brian Baidal (@drakhart)
- Brian Beattie (@beattie)
- Brian Cribbs (@cribbstechnologies)
- Brian Dunlay (@bdunlay)
- Brian Egge (@brianegge)
- Brian Fitzgerald (@Brianfit)
- Brian Gehrich (@bgehrich)
- Brian Gianforcaro (@bgianfo)
- Brian Hanifin (@brianhanifin)
- Brian Harwell (@brianharwell)
- Brian Hopkins (@btotharye)

- Brian J King (@brianjking)
- Brian Jinwright (@bjinwright)
- Brian Karani Ndwiga (@rayrayndwiga)
- Brian Kim (@bkrepo)
- Brian Kramer (@B-Kramer)
- Brian Low (@brianlow)
- Brian Norman (@gingemonster)
- Brian O'Connor (@btoconnor)
- Brian Orpin (@borpin)
- Brian Redbeard (@brianredbeard)
- Brian Retterer (@bretterer)
- Brian Rogers (@brg468)
- Brian Thorne (@hardbyte)
- Brian Torres-Gil (@btorresgil)
- Brian Towles (@wonderslug)
- BrianWithAHat (@BrianWithAHat)
- Brig Lamoreaux (@briglx)
- Brigham Brown (@brigham)
- Brint E. Kriebel (@bekriebel)
- Britton Clapp (@britton-clapp)
- brkr19 (@brkr19)
- broadcasttechie (@broadcasttechie)
- Brock Allen (@brockallen)
- Brock Williams (@cotcomsol)
- brode (@brode)
- bromosky (@bromosky)
- Brooke Hedrick (@hedrickbt)
- brouwer (@brouwer)
- brubaked (@brubaked)
- Bruce (@trux66)
- Bruce Howells (@BruceHowells)
- Bruce Sheplan (@bshep)
- brucemiranda (@brucemiranda)
- BRUH Automation (@bruhautomation)
- Bruno (@HVR88)
- Bruno Binet (@bbinet)
- Bruno Filipe (@clapbr)
- Bruno Furtado (@bmfurtado)
- Bruno Maia (@queimadus)
- Bryan Berg (@berg)



- Bryan Van de Ven (@bryevdv)
- Bryan York (@bryanyork)
- BryanJacobs (@BryanJacobs)
- Bryant Lee (@bryantlee)
- Bryce Edwards (@hoopty)
- Brynley McDonald (@ZephireNZ)
- Bryton Hall (@hall)
- bsmappee (@bsmappee)
- Burningstone91 (@Burningstone91)
- butako (@butako)
- BUUT! (@buut-vrij)
- bvansambeek (@bvansambeek)
- bvweerd (@bvweerd)
- bw3 (@bw3)
- bwduncan (@bwduncan)
- byte-bender (@byte-bender)
- Béla Becker (@micsurin)
- Børge Nordli (@bnordli)
- Błażej Sowa (@bjsowa)
- c0ffee7 (@c0ffee7)
- c727 (@c727)
- c-soft (@c-soft)
- Cadi (@cadihowley)
- Cadsters (@Cadsters)
- cadwal (@cadwal)
- cagnulein (@cagnulein)
- caiosweet (@caiosweet)
- caius (@caiuspb)
- Caleb (@calebhfaber)
- Caleb Dunn (@finish06)
- Caleb Mah (@calebmah)
- Calin Crisan (@ccrisan)
- callifo (@callifo)
- Callum Powell (@Callum0x50)
- Calvin (@VR29)
- Calvin Vette (@calvinvette)
- Cam Hutchison (@camh-)
- Cam Mannett (@cmannett85)
- Cameron Bullock (@cbullock)
- Cameron Llewellyn (@camrun91)

- Cameron Maclean (@BowsiePup)
- Cameron McEfee (@cameronmcefee)
- Cameron Morris (@cameronrmorris)
- Cameron Smith (@camcs1)
- Can Geliş (@cangelis)
- candreacchio (@candreacchio)
- cansl (@cansl)
- CantankerousBullMoose (@CantankerousBullMoose)
- capstan1 (@capstan1)
- Captain Haddock (@ca-haddock)
- Captone Habiyaremye (@Liopun)
- Carl Chan (@carlchan)
- Carl Johnson (@carlivar)
- Carl Sutton (@dogmatic69)
- Carlo Caione (@carlocaione)
- Carlo Costanzo (@CCOSTAN)
- Carlos Duarte Do Nascimento (Chester) (@chesterbr)
- Carlos Garcia Saura (@CarlosGS)
- Carlos Giraldo (@cgiraldo)
- Carlos Gomes (@cgomesu)
- Carlos Gustavo Sarmiento (@carlos-sarmiento)
- Carlos J. Aliaga (@cjaliaga)
- Carlos Quijano (@cquijano)
- Carlos Rodríguez (@crguez)
- Carlos Santos (@casantos)
- carlosmgr (@carlosmgr)
- Carmen Sanchez (@soundch3z)
- carras (@carras)
- Carsten Hiort (@katter)
- Carsten Igel (@carstencodes)
- Carsten Schoenert (@tijuca)
- carstenschroeder (@carstenschroeder)
- Carter (@BluGeni)
- casey (@csjo)
- Casper (@casperklein)
- Casper Smits (@cmitz)
- Caswell1000 (@Caswell1000)
- Cathy M (@AddMoreLimes)
- caz0075 (@caz0075)
- cbhiii (@cbhiii)

- cburgess (@cburgess)
- cby016 (@cby016)
- cdheiser (@cdheiser)
- cdnninja (@cdnninja)
- cdpuk (@cdpuk)
- Cedric (@Kleinrotti)
- Cedric Gatay (@CedricGatay)
- Cedric Van Goethem (@Zepheus)
- Cedrick Flocon (@CedrickFlocon)
- ceejii (@ceejii)
- Celedhrim (@Celedhrim)
- celeroll (@celeroll)
- celestinjr (@celestijnr)
- Cello Spring (@cellerich)
- Cem Sinan Kaynar (@cskaynar)
- Cenk Gündoğan (@cgundogan)
- Cezar Sá Espinola (@cezarsa)
- cfreeze (@cfreeze)
- Chad (@chadlyy)
- Chad Condon (@thetic)
- Chad Parry (@chadparry)
- ChadCurvin (@Curvin777)
- Chaim Turkel (@chaimt)
- Chaly Flavour (@ChalyFlavour)
- chammp (@chammp)
- Chandan Rai (@bhageena)
- chanders (@chanders)
- Changming Huang (@huangcm0101)
- Changwoo Ryu (@changwoo)
- Chao (@chaoranxie)
- CHAPELLE Quentin (@quentinchap)
- chaptergy (@chaptergy)
- CharIB (@CharlieBailly)
- Charles Blonde (@CharlesBlonde)
- Charles Duffy (@charles-dyfis-net)
- Charles Garwood (@cgarwood)
- Charles Hardin (@ckhardin)
- Charles Manning (@cdhmanning)
- Charles Spirakis (@srcLurker)
- CharlesGillanders (@CharlesGillanders)

- Charlie Fish (@fishcharlie)
- Charlie Turner (@charlie-ht)
- Chase (@ab3lson)
- ChaseCares (@ChaseCares)
- ChaYoung You (@yous)
- CHAZICLE (@CHAZICLE)
- Chema García (@sch3m4)
- chemaaa (@chemaaa)
- Chen-IL (@Chen-IL)
- cheng2wei (@cheng2wei)
- Cheong Yip (@wingy3181)
- ChevySSinSD (@ChevySSinSD)
- chewbh (@chewbh)
- Chia-liang Kao (@clkao)
- chiefdragon (@chiefdragon)
- chierichetto (@erlendsellie)
- Chih-Min Chao (@cmchao)
- chises (@chises)
- chknetsc (@chknetsc)
- chocomega (@chocomega)
- ChopperRob (@ChopperRob)
- choss (@choss)
- Chouffy (@Chouffy)
- chpego (@chpego)
- Chris (@firstof9)
- Chris (@pandabear41)
- Chris (@darthsebulba04)
- Chris (@digitallyserviced)
- Chris Adams (@mrchrisadams)
- Chris Aljoudi (@chrisaljoudi)
- Chris Aloï (@ctaloi)
- Chris Baumgartner (@mchrisb03)
- Chris Brandt (@seebe)
- Chris Browet (@koying)
- Chris Caron (@caronc)
- Chris Colohan (@colohan)
- Chris Cowart (@cpcowart)
- Chris Crowe (@chriscrowe)
- Chris Drackett (@chrisdrackett)
- Chris Feist (@chris-feist)

- Chris Frederick (@cdf123)
- Chris Halls (@challs)
- Chris Hammond (@ChrisHammond)
- Chris Hasenpflug (@chrishas35)
- Chris Heath (@Nedlinin)
- Chris Helming (@CWhits)
- Chris Helming (@chelming)
- Chris Johnston (@Chris-Johnston)
- Chris Jones (@fezfox)
- Chris Kacerguis (@chriskacerguis)
- Chris Kankiewicz (@PHLAK)
- Chris Kemp (@chriskemp)
- Chris LaRose (@cjlarose)
- Chris LeBlanc (@spacesuitdiver)
- Chris Lesiak (@ChrisLesiak)
- Chris Mandich (@ChrisMandich)
- Chris McCurdy (@chrismccurdy)
- Chris Miller (@mysteriouspants)
- Chris Monteiro (@cmonteiro128)
- Chris Mulder (@chrisvis)
- Chris Nesbitt-Smith (@chrisns)
- Chris Packham (@cpackham)
- Chris Petersen (@ex-nerd)
- Chris Reichel (@xu-chris)
- Chris Romp (@ChrisRomp)
- Chris Sims (@jcsims)
- Chris Smart (@csmart)
- Chris Soyars (@ctso)
- Chris Talkington (@ctalkington)
- Chris Thorn (@chris-thorn)
- Chris Thornton (@cj-thornton)
- Chris Turra (@cturra)
- Chris Van Humbeeck (@humbeeck)
- Chris van Marle (@qistoph)
- Chris Vanclercq (@Vanclec)
- Chris Vick (@cvick)
- Chris Weiss (@chrwei)
- Chris Zankel (@czankel)
- chris669 (@chris669)
- chriscla (@chriscla)

- ChrisHaPunkt (@ChrisHaPunkt)
- chrismcneil (@chrismcneil)
- chriss158 (@chriss158)
- ChrisS85 (@ChrisS85)
- ChrisTBarnes (@ChrisTBarnes)
- Christer Vestermark (@crallian)
- christerr (@christerr)
- Christiaan Blom (@Deinara)
- Christiaan Schriel (@cschriel)
- Christian (@cwildt)
- Christian Artin (@gravufo)
- Christian Biamont (@chrillux)
- Christian Bilevits (@chrillebile)
- Christian Brædstrup (@LinuxChristian)
- Christian Clauss (@cclauss)
- Christian Ferbar (@ferbar)
- Christian Häussler (@cniweb)
- Christian Knittl-Frank (@lcnittl)
- Christian Kuhn (@lolli42)
- Christian Lasarczyk (@ChrisLasar)
- Christian Manivong (@chrismanivong)
- Christian Muck (@cmuck)
- Christian Müller (@kitsunet)
- Christian Müller (@chphmu)
- Christian Rodriguez (@chrodriguez)
- Christian Schneider (@christianschneider89)
- Christian Schröter (@cschroeter)
- Christian Schwinne (@Aircoookie)
- Christian Soltenborn (@csoltenborn)
- Christian Stangier (@c-st)
- Christian Stewart (@paralin)
- Christian Stigen Larsen (@cslarsen)
- Christian Studer (@cstuder)
- Christian Winther (@jippi)
- ChristianKuehnel (@ChristianKuehnel)
- Christofer Hellqvist (@hellqvist)
- Christoffer Graversen (@ChristofferG)
- Christoffer Kylvåg (@christoe)
- Christoph Engelbert (@noctarius)
- Christoph Gerneth (@c7h)

- Christoph Kisfeld (@chk1)
- Christoph Müllner (@cmuellner)
- Christoph Settgast (@csett86)
- Christoph Spörk (@DarwinsBuddy)
- Christoph Wagner (@Christoph-Wagner)
- Christophe Coevoet (@stof)
- Christophe Fergeau (@cfergeau)
- Christophe Painchaud (@cpainchaud)
- Christophe Trefois (@Trefex)
- Christophe Vu-Brugier (@cvubrugier)
- christopheBfr (@christopheBfr)
- Christopher Bailey (@AngellusMortis)
- Christopher Cavey (@cicavey)
- Christopher Gozdziwski (@Chrisgozd)
- Christopher Gramberg (@chrisgramberg678)
- Christopher Hoage (@chrishoage)
- Christopher Kochan (@crkochan)
- Christopher Loessl (@hashier)
- Christopher Masto (@masto)
- Christopher McCrory (@chrismcc-gmail)
- Christopher Nethercott (@chriscn)
- Christopher Pelloux (@chp-io)
- Christopher Rosset (@chrisrosset)
- Christopher Sacca (@csacca)
- Christopher Svensson (@stoffus)
- Christopher Thornton (@cthornton)
- Christopher Toth (@ctoth)
- Christopher Vella (@chrisvella)
- Christopher Viel (@Chris-V)
- chrom3 (@chrom3)
- chrysn (@chrysn)
- Chsldz (@Chsldz)
- Chuang Zheng (@frogkind)
- Chun-wei Kuo (@Domon)
- cHunter789 (@cHunter789)
- chz^3 (@chzchzchz)
- Ciquattro (@CiquattroFPV)
- Ciro Santilli (@cirosantilli)
- citruz (@citruz)
- ckesc (@ckesc)

- cklagenberg (@cklagenberg)
- clach04 (@clach04)
- Claes Hallström (@claha)
- clamel77 (@clamel77)
- Clark Rawlins (@crawlins)
- clarkewd (@clarkewd)
- clarkwmcd (@clarkwmcd)
- Claudio Barca (@clabnet)
- Claudio Catterina (@ccatterina)
- Claudio Heckler (@heckler)
- Claudiu Bucur (@clau-bucur)
- Claudiu Farcas (@farcasclaudiu)
- claughinghouse (@claughinghouse)
- claurita (@claurita)
- Claus F. Strasburger (@cfstras)
- clayton craft (@craftyguy)
- Clayton Nummer (@claytonjn)
- Clayton O'Neill (@claytono)
- Clayton Shotwell (@clshotwe)
- Clemens (@cvaliente)
- Clemens @ Elektor (@ClemensAtElektor)
- Clemens Wolff (@c-w)
- Cliffano Subagio (@cliffano)
- cliffjao (@cliffjao)
- Clifford Roche (@cmroche)
- Clifford W. Hansen (@cliffordwhansen)
- clipman (@clipman)
- clssn (@clssn)
- Cláudio Ribeiro (@DailyMatters)
- Clément Leger (@clementleger)
- Clément P. (@clementprevot)
- Clément Péron (@clementperon)
- Clément TOCHE (@ClementToche)
- cnico (@cnico)
- cnstudio (@cnstudio)
- Coby Fleener (@coby)
- cockroach (@cockroach)
- coco (@k0rventen)
- code-review-doctor (@code-review-doctor)
- codeavenger07 (@codeavenger07)



- codedmind (@codedmind)
- Codepadawan (@Codepadawan)
- Cody Schafer (@jmesmon)
- codypet (@codypet)
- Colby Rome (@cisasteelersfan)
- colemand (@colemand)
- Colin (@cln-io)
- Colin Cachia (@colincachia)
- Colin Dunn (@colindunn)
- Colin Frei (@colinfrei)
- Colin Harrington (@ColinHarrington)
- Colin O'Dell (@colinodell)
- Colin Robbins (@ColinRobbins)
- Colin Teubner (@netopiex)
- Colleen (@ctwitty)
- Comic Chang (@comicchang)
- ComputerCandy (@HexF)
- Conall O'Brien (@conallob)
- conflipper (@conflipper)
- Conrad Juhl Andersen (@cnrd)
- Constantine Grantcharov (@conz27)
- coolguymatt (@coolguymatt)
- Cooper Dale (@Cooper-Dale)
- copykatze (@copykatze)
- Corban Mailloux (@corbanmailloux)
- Corbeno (@Corbeno)
- coreGreenberet (@coreGreenberet)
- Corey Edwards (@heytsai)
- Corey Pauley (@devspacenine)
- Corneels de Waard (@corneels)
- Cornelius Mund (@cornim)
- corneyl (@corneyl)
- Costas (@costastf)
- Cougar (@Cougar)
- Courtenay (@hdsheena)
- Courtney Strachan (@cstrachan88)
- cpgifford (@cpgifford)
- cpolhout (@cpolhout)
- cpopp (@cpopp)
- cpriouzeau (@cpriouzeau)

- cpw (@cpw)
- CraftyKoala (@CraftyKoala)
- Craig Barratt (@craigbarratt)
- Craig Hills (@chills42)
- Craig Huckabee (@huckabeec)
- Craig J. Midwinter (@craigmidwinter)
- craiggenner (@craiggenner)
- craigmcgowan (@craigmcgowan)
- Crash (@CrashWorksLLC)
- CrazYoshi (@CrazYoshi)
- Creasol (@CreasolTech)
- Cristian Asenjo (@casenjo)
- cristian-vescan (@cristian-vescan)
- croghostrider (@croghostrider)
- Cromefire\_ (@cromefire)
- Crowbar Z (@crowbarz)
- crunch (@simonk83)
- cryptelli (@cryptelli)
- csgitmeup (@csgitmeup)
- ctborg (@ctborg)
- CTLS (@CTLS)
- CtrlZvi (@CtrlZvi)
- culssw (@culssw)
- CupertinoGeek (@CupertinoGeek)
- CupricReki (@CupricReki)
- curlydingo (@curlydingo)
- Curtis Gibby (@curtisgibby)
- Curtis Kennington (@Curtisjk)
- CV (@dagobert)
- cvroque (@cvroque)
- cvwillegen (@cvwillegen)
- cweakland (@cweakland)
- cxlwill (@cxlwill)
- CyberDave17 (@CyberDave17)
- cydia2020 (@cydia2020)
- Cyril Bur (@cyrilbur-ibm)
- Cyril Perrin (@cyrilperrin)
- Cyro (@cyrosy)
- czechmark (@czechmark)
- Cédric Chépied (@chep)

- d0nni3q84 (@d0nni3q84)
- D34DC3N73R (@D34DC3N73R)
- D3v01dZA (@D3v01dZA)
- d3wy (@d3wy)
- d6e (@d6e)
- d8ahazard (@d8ahazard)
- d-walsh (@d-walsh)
- D. Olsson (@dickolsson)
- da-anda (@da-anda)
- DaCoD (@dacod)
- dacwe (@dacwe)
- Dagg Stompler (@daggs1)
- dailow (@dailow)
- Dale Higgs (@dale3h)
- daleye (@daleye)
- Dallas Opelt (@DallasO)
- damarco (@damarco)
- Damian Nowak (@Nowaker)
- Damien Duboeuf (@Smeagolworms4)
- Damien Le Moal (@damien-lemoal)
- Damien Levin (@damienlevin)
- Damien Riegel (@d-k-c)
- Dan (@danieljkemp)
- Dan (@dcs8)
- Dan Burke (@danburke)
- Dan C Williams (@dancwilliams)
- Dan Chen (@djchen)
- Dan Cinnamon (@Cinntax)
- Dan Faulknor (@danielfaulknor)
- Dan Ferrante (@dferrante)
- Dan Ford (@dpford)
- Dan Jackson (@e28eta)
- Dan Jenkins (@danjenkins)
- Dan Jones (@djj211)
- Dan Klaffenbach (@klada)
- Dan Lehman (@DeltaTangoLima)
- Dan Loewenherz (@dlo)
- Dan Lowe (@tangledhelix)
- Dan Nixon (@DanNixon)
- Dan Olson (@danielolson13)

- Dan Pattison (@SirDan1963)
- Dan Ponte (@amigan)
- Dan Ports (@drkp)
- Dan Riegsecker (@danriegsecker)
- Dan Sarginson (@dansarginson)
- Dan Smith (@kk7ds)
- Dan Sullivan (@dansullivan86)
- Dan Trevino (@dantrevino)
- Dan Walkes (@dwalkes)
- danaues (@danaues)
- danbishop (@danbishop)
- Dane (@Xiol)
- Dane Berryman (@daneberryman)
- Dane Peterson (@peterson-dane)
- dangyuluo (@t0ny-peng)
- Dani (@danichispa)
- Daniel (@da-snap)
- Daniel (@danieldotnl)
- Daniel (@Danieldiazi)
- Daniel (@djansen1987)
- Daniel (@azrael783)
- Daniel (@delneet)
- Daniel (@danielbrunt57)
- Daniel (@ConsoleCriminal)
- Daniel (@danielkihlgren)
- Daniel (@danimtb)
- Daniel (@sMauldaeschle)
- Daniel (@Danielinte)
- Daniel (@GhostNr1)
- Daniel Anner (@danner26)
- Daniel Baulig (@DanielBaulig)
- Daniel Bowman (@vrih)
- Daniel Chesterton (@dchesterton)
- Daniel Claes (@daenny)
- Daniel Correa Lobato (@dclobato)
- Daniel D'Abate (@danieldabate)
- Daniel de Jong (@daniel-jong)
- Daniel Dietzler (@danieldietzler)
- Daniel Escoz (@Darkhogg)
- Daniel Fahlgren (@fahlgren)

- Daniel Gangl (@killer0071234)
- Daniel García (@dani-garcia)
- Daniel Hyles (@DotNetDann)
- Daniel J. Leach (@DanielJLeach)
- Daniel Jost (@PxIBuzzard)
- Daniel Kalmar (@kalimaul)
- Daniel Kucera (@danielkucera)
- Daniel Lando (@robertsLando)
- Daniel Lashua (@dlashua)
- Daniel Lintott (@TwoWheelDev)
- Daniel Mack (@zonque)
- Daniel Martin Gonzalez (@danimart1991)
- Daniel Matuschek (@usul27)
- Daniel Mühlbachler-Pietrzykowski (@muhlba91)
- Daniel Nelson (@sigpwr)
- Daniel Nicoletti (@dantti)
- Daniel O'Connor (@CloCkWeRX)
- Daniel Pereira (@kriansa)
- Daniel Perna (@danielperna84)
- Daniel Pervan (@danielpervan)
- Daniel Peukert (@dpeukert)
- Daniel Powell (@danpowell88)
- Daniel Price (@danielbprice)
- Daniel Reimer (@dreimer1986)
- Daniel Rheinbay (@danielrheinbay)
- Daniel Rivard (@danielrivard)
- Daniel Sack (@DanielTheCoder)
- Daniel Sangué (@mrdasa)
- Daniel Schaal (@schaal)
- Daniel Schröder (@MesserschmittX)
- Daniel Serpell (@dmsc)
- Daniel Shokouhi (@dshokouhi)
- Daniel Stockhausen (@daniel-stockhausen)
- Daniel Stone (@daniel-stoneuk)
- Daniel Sörlöv (@DSorlov)
- Daniel Watkins (@OddBloke)
- Daniel Welch (@danielwelch)
- Daniel Wiberg (@dannew)
- Daniel Winks (@DanielWinks)
- Daniele Salvatore Albano (@danielealbano)

- Danielhiversen (@Danielhiversen)
- danielpodwysocki (@danielpodwysocki)
- DanielV (@dala318)
- DanielXYZ2000 (@DanielXYZ2000)
- Danijel Stojnic (@danijelst)
- Danilo Bargaen (@dbrgn)
- Daniyar Yeralin (@yeralin)
- Daniël van de Giessen (@DvdGiessen)
- Danny Fullerton (@northox)
- Danny Gershman (@dgershman)
- Danny Murphy (@Dmurph24)
- Danny Tsang (@dannysang)
- DannyHg (@DannyHg)
- danomi (@danomi)
- danomimanchego123 (@danomimanchego123)
- DanPatten (@DanPatten)
- DanskerUS (@DanskerUS)
- danu5 (@danu5)
- daphatty (@daphatty)
- Daphne Lin (@daphotron)
- dapowers87 (@dapowers87)
- Dara Adib (@daradib)
- Dario Binacchi (@passgat)
- Dario Iacampo (@JackNova)
- Darius Augulis (@adarius)
- DarkPark (@DarkPark)
- Darrell (@DTTerastar)
- Darren Foo (@stonith)
- Darren Reynolds (@reynos)
- Darryl Ross (@daemondazz)
- Darío Hereñú (@kant)
- dasadi (@dasadi)
- Dash Winterson (@dashdanw)
- dasos (@dasos)
- DataBitz (@DataBitz)
- Dav0815 (@Dav0815)
- dav76df (@dav76df)
- Dave (@davidrhunt)
- Dave (@d4v3d)
- Dave (@DeeVeX)

- Dave Atherton (@cofeedave)
- Dave Banks (@djbanks)
- Dave Clarke (@clarkd)
- Dave Code (@dave-code-ruiz)
- Dave Eddy (@bahamas10)
- Dave Finlay (@dfinlay)
- Dave Hylands (@dhylands)
- Dave J (@kxtcd950)
- Dave Lowper (@davelowper)
- Dave Pearce (@davecpearce)
- Dave Pearce (@UrbanDave)
- Dave Skok (@blanco-ether)
- Dave T (@davet2001)
- DAVe3283 (@DAVe3283)
- DaveSergeant (@dethpickle)
- davesmeghead (@davesmeghead)
- Davey Chu (@daveychu)
- David (@fanaticDavid)
- David (@dr1rrb)
- David (@vasqued2)
- David (@DaveCo1701)
- David (@dschoorisse)
- David A. Bell (@dabell-cc)
- David Barbion (@david-barbion)
- David Barnett (@dbarnett)
- David Barrera (@davidbb)
- David Barshow (@barshow)
- David Baumann (@daBONDi)
- David Beitey (@davidjb)
- David Bender (@codehero)
- David Bezuidenhout (@tinuva)
- David Bilay (@dYalib)
- David Bonnes (@zxdavb)
- David Boslee (@dboslee)
- David Broadfoot (@dlbroadfoot)
- David Brückmann (@MrGde)
- David Byrne (@David-Byrne)
- David Conley (@conleydg)
- David Cramer (@dcramer)
- David Danssaert (@ddanssaert)

- David De Grave (Essensium/Mind) (@ledav-net)
- David De Sloovere (@DavidDeSloovere)
- David Dix (@WizBangCrash)
- David du Colombier (@0intro)
- David Edmondson (@dme)
- David Edmundson (@davidedmundson)
- David F. Mulcahey (@dmulcahey)
- David Fiel (@dfiel)
- David Gibbons (@davidgibbons)
- David Glessner (@dg-rc)
- David Grant (@davegravy)
- David Graziano (@davidgraz)
- David Jackson (@David-Jackson)
- David K (@neffs)
- David K Turner (@dkt01)
- David Keijser (@keis)
- David Kendall (@BottlecapDave)
- David Kessler (@DJKessler)
- David Langerman | Onyx Zero Software (@dlangerm)
- David Le Brun (@davidlb)
- David Lechner (@dlech)
- David Lie (@davidlie)
- David Lloyd (@davlloyd)
- David Luhmer (@David-Development)
- David Martin (@3ative)
- David Martínez (@vaites)
- David McClosky (@dmcc)
- David McNett (@nugget)
- David Nielsen (@dcnielsen90)
- David Noren (@dcnoren)
- David Noyes (@davidnoyes)
- David Ohayon (@ohayon)
- David Peterson (@dippysan)
- David Pierret (@Galadrin)
- David Postle (@DMailMan)
- David Purdy (@davygravy)
- David Radcliffe (@dwradcliffe)
- David Raeman (@draeman)
- David Ramiro (@davidramiro)
- david reid (@zathras777)



- David Roberts (@drobtravels)
- David Rocharz (@davidrocharz)
- David Rosca (@nowrep)
- David Ryan (@ptcryan)
- David Shanske (@dshanske)
- David Steele (@davesteele)
- David Straub (@DavidMStraub)
- David Tchepak (@dtchepak)
- David Thomas (@synth3tk)
- David Thulke (@dthulke)
- David Vega Fontelos (@repocho)
- David VINET (@DavZero)
- David Wang (@dcwangmit01)
- David Winn (@qypea)
- David Woodhouse (@dwmw2)
- David Worsham (@arbreng)
- David Young (@dmyoung9)
- David Zhu (@Dethada)
- David Zidar (@DavidZidar)
- david81 (@david81)
- David-Leon Pohl (@DavidLP)
- davida72 (@davida72)
- Davide Setti (@vad)
- Davide Varricchio (@bannhead)
- Davide Viti (@zinosat)
- DavidFW1960 (@DavidFW1960)
- davidlang42 (@davidlang42)
- davidm84 (@davidm84)
- davidw3591 (@davidw3591)
- davaloko (@davaloko)
- Davor Val Vega (@slovenec88)
- Davy Landman (@DavyLandman)
- Dawid Chyrzyński (@dawidchyrzynski)
- Dawid Wróbel (@wrobelda)
- dayofdoom (@dayofdoom)
- dbachelart (@dbachelart)
- DBCL (@DB-CL)
- dbrowndan (@dbrowndan)
- dburnsii (@dburnsii)
- dcharbonnier (@dcharbonnier)

- dckiller51 (@dckiller51)
- dcmeglio (@dcmeglio)
- dcrusader (@dcrusader)
- DDanii (@DDanii)
- DeadEnd (@DeadEnded)
- Dean (@FreekingDean)
- Dean Camera (@abcmিনিuser)
- Dean Kroker (@deankroker)
- deddc23efb (@deddc23efb)
- DeepCoreSystem (@DeepCoreSystem)
- Deez73 (@Deez73)
- definitio (@definitio)
- deftdawg (@deftdawg)
- degorius (@degorius)
- deisi (@deisi)
- Dejan Dakić (@dejax)
- Delio Castillo (@jangeador)
- Delper (@Delper)
- DelusionalAI (@DelusionalAI)
- Demkes (@r2xud)
- demonspork (@demonspork)
- denes44 (@denes44)
- Denilson Sá Maia (@denilsonsa)
- Denis Bodor (@0xDRRB)
- Denis Generalov (@giantlock)
- Denis Milanović (@denismilanovic)
- Denis Mingulov (@mingulov)
- Denis Osterland (@OsterlaD)
- Denis Paavilainen (@denpamusic)
- Denise Yu (@deniseyu)
- Denix (@denics)
- Dennis (@MuppetOwl)
- Dennis (@dennis-bell)
- Dennis de Greef (@dennisdegreef)
- Dennis Gilmore (@ausil)
- Dennis Ham (@denniswham)
- Dennis Karpienski (@TheRealLink)
- Dennis Keitzel (@cybe)
- Dennis Modig (@techdude200)
- Dennis Schroer (@dennisschroer)

- Dennis Sutch (@sutch)
- dennisaion (@dennisaion)
- Denny Korsukéwitz (@dennykorsukewitz)
- Denver P (@DrTexx)
- denverpilot (@denverpilot)
- Denys Dovhan (@denysdovhan)
- deosrc (@deosrc)
- Department G33k (@department-g33k)
- depasseg (@depasseg)
- derandiunddasbo (@derandiunddasbo)
- Derek (@KrunchMuffin)
- Derek Atkins (@derekatkins)
- Derek Brooks (@broox)
- Dermot Duffy (@dermotduffy)
- Derrick Lyndon Pallas (@pallas)
- dersger (@dersger)
- Desausoi Laurent (@lade-odoo)
- Destix (@Destix)
- DetroitEE (@DetroitEE)
- devbf (@devbf)
- devdelay (@dcrypt3d)
- devianceluka (@devianceluka)
- DeviantEng (@DeviantEng)
- Devin Sevilla (@dasevilla)
- DeviousPenguin (@DeviousPenguin)
- devnill (@devnill)
- Devon Kerkhove (@TASSDevon)
- Devon Peet (@dpeet)
- dewdrop (@dewdropawoo)
- Dewet Diener (@dewet22)
- dewi-ny-je (@dewi-ny-je)
- dewo1988 (@dewo1988)
- Dezorian (@Dezorian)
- dfigus (@dfigus)
- dfournie (@dfournie)
- dgorti (@dgorti)
- dgouarin (@dgouarin)
- dgta1 (@dgta1)
- dgthomson (@dgthomson)
- dhaas (@dhaas)

- dhzl84 (@dhzl84)
- Dick Swart (@DickSwart)
- dickesW (@dickesW)
- Didgeridrew (@Didgeridrew)
- Diede van Marle (@Cyanogenbot)
- Diederik van den Burger (@DiederikvandenB)
- Diefferson Koderer Môro (@djpremier)
- Diego Ambrosanio (@snakuzzo)
- Diego Elio Pettenò (@Flameeyes)
- digiblur (@digiblur)
- Dilepa (@Dilepa)
- dilruacs (@dilruacs)
- Dima Boger (@b0g3r)
- Dima Goltsman (@dimagoltsman)
- Dima Zavin (@thecynic)
- Dimiter Geelen (@Didel)
- dimitri (@magicbuffer)
- Dimitri Prybysh (@dimp-gh)
- Dimitrij Kotrev (@nooploop)
- Dimitrios Siganos (@dsiganos)
- dimitripb (@dimitripb)
- Dimitris (@karate)
- Dmitry Golubovsky (@dmgolubovsky)
- Dimtuhop (@Dimtuhop)
- dinoaus (@dinoaus)
- Diogo Alves (@killercode)
- Diogo F. Andrade Murteira (@murtas)
- Diogo Gomes (@dgomes)
- Diogo Soares (@diogos88)
- diplx (@diplx)
- Dirk (@dm82m)
- DJ Adams (@qmacro)
- djaydev (@djaydev)
- djerik (@djerik)
- DJJo14 (@DJJo14)
- djm300 (@djm300)
- DjMoren (@DjMoren)
- Djowie (@Djowie)
- djtimca (@djtimca)
- DK (@poldim)

- DKAutomater (@DKAutomater)
- dklemm (@dklemm)
- dkramer74 (@Swanky-Bubbles)
- Dmitriy (@marcellus00)
- Dmitriy Shafranskiy (@shafr)
- Dmitry (@4ybaka)
- Dmitry Avramenko (@B1tMaster)
- Dmitry Katsubo (@dmak)
- Dmitry Kosenkov (@Junker)
- Dmitry Krasnoukhov (@krasnoukhov)
- Dmitry Mamontov (@dmamontov)
- Dmitry Tretyakov (@dtretyakov)
- Dmitry Tsyrlnikov (@Dubzer)
- Dmitry Vasilyev (@slydiman)
- dmonego (@dmonego)
- dmoranf (@dmoranf)
- dmschlab (@dmschlab)
- dmunozv04 (@dmunozv04)
- Dmytro Kytsmen (@Kietzmann)
- Dmytro Milinevskyy (@niamster)
- Dmytro Shvaika (@DmytryS)
- Dmytro Vasylenko (@odi-um)
- dnaphreak (@dnaphreak)
- dnguyen800 (@dnguyen800)
- DoctorLOT (@DoctorLOT)
- DoloresHA (@DoloresHA)
- Dom (@domwillcode)
- Domantas Mauruča (@Dohxis)
- domier (@domier)
- Dominik Bruhn (@theomega)
- Dominik Moritz (@domoritz)
- Dominik Palo (@DominikPalo)
- Donal Phipps (@Phippsy)
- DonHugo (@DonHugo)
- Donnie (@donkawechico)
- donoghdb (@donoghdb)
- doogstar (@doogstar)
- Doomic (@Doomic)
- dotsoltecti (@dotsoltecti)
- Doug (@douglasbeck)

- Doug Hoffman (@doug-hoffman)
- Doug Ollerenshaw (@dougollerenshaw)
- Doug Schaapveld (@djschaap)
- Dougal Matthews (@d0ugal)
- dougiteixeira (@dougiteixeira)
- Douglas RAILLARD (@DouglasRaillard)
- dozminic (@dozminic)
- dpressle (@dpressle)
- dpryor (@dpryor)
- Dr John Tunnicliffe (@DrJohnT)
- Dr. Daniel Alexander Smith (@danielsmith-eu)
- drahdiwaberl (@drahdiwaberl)
- Drake Loud (@drakeloud)
- dramamoose (@dramamoose)
- DrBlokmeister (@DrBlokmeister)
- dreed47 (@dreed47)
- dreizehnelf (@dreizehnelf)
- drentsemoi (@drentsemoi)
- Drew Budwin (@dbudwin)
- Drew Fustini (@pdp7)
- Drew Mares (@dmares01)
- Drew Skwiers-Koballa (@dzsquared)
- Drew Wells (@drewwells)
- drfinn15 (@drfinn15)
- Dries De Peuter (@NoUseFreak)
- drinfernoo (@drinfernoo)
- drivin (@drivin)
- drizzle1 (@drizzle1)
- drjared88 (@drjared88)
- drogfeld (@drogfild)
- droopanu (@droopanu)
- drop table USERS; -- (@hudashot)
- Dror Eiger (@deiger)
- drosoCode (@drosoCode)
- drpeppershaker (@drpeppershaker)
- drphungky (@drphungky)
- DrZoid (@docteurzoidberg)
- Drzony (@drzony)
- DrZzs (@Snipercaine)
- dtorner (@dtorner)

- Dubh Ad (@DubhAd)
- DuchkPy (@DuchkPy)
- dugurs (@dugurs)
- dummylabs (@dummylabs)
- Duncan Elliot (@dmelliot)
- Duncan Findlay (@duncf)
- Duncan Leo (@duncanleo)
- duncanvanzyl (@duncanvanzyl)
- dunkelz (@dunkelz)
- dunstad (@dunstad)
- Duoxilian (@Duoxilian)
- DUPONCHEEL Sébastien (@sduponch)
- dupondje (@dupondje)
- Dusan Cervenka (@Hadatko)
- dusan-ivanco (@dusan-ivanco)
- Dushara Jayasinghe (@nidujay)
- Dustin Essington (@aetaric)
- Dustin Johnson (@drjson)
- Dustin Rue (@dustinrue)
- Dustin S (@texnofobix)
- Dustin Wyatt (@dmwyatt)
- dvermd (@dvermd)
- dvz (@dvz)
- Dwain Scheeren (@dwainscheeren)
- Dwight Holman (@anonfunc)
- dwinnnspeaks (@dwinnnspeaks)
- Dycelll (@Dycelll)
- Dylan (@DSAAutomations)
- Dylan Barlett (@dbarlett)
- Dylan Do Amaral (@dylandoamaral)
- Dylan Gore (@DylanGore)
- dynasticorpheus (@dynasticorpheus)
- dywisor (@dywisor)
- dzukero (@dzukero)
- Eamonn O'Connell (@halfbaked)
- earaya (@earaya)
- ec-blaster (@ec-blaster)
- echoromeo (@echoromeo)
- ecksun (@ecksun)
- Ed Boal (@edwork)

- Ed Coen (@ecoen66)
- Ed Grau (@eddiegrau)
- Ed Marshall (@logic)
- Ed Spiridonov (@edo1)
- ed-blake1 (@ed-blake1)
- edAndy (@Vip0r)
- Eddie Chiang (@eddie-chiang)
- Eddy G (@eddyg)
- EddyK69 (@EddyK69)
- eddyliao (@eddyliao)
- Edgar Bonet (@edgar-bonet)
- Edgardo Ramírez (@SoldierCorp)
- edgimar (@edgimar)
- edif30 (@edif30)
- Edmundo Ferreira (@edmundoferreira)
- edomat (@edomat)
- Edu\_Coder (@jptrsn)
- Eduard van Valkenburg (@eavanvalkenburg)
- Eduardo Fonseca (@ebfio)
- Eduardo Roldan (@eroldan)
- Edward (@EdJoJob)
- Edward Knight (@Edward-Knight)
- Edward Loveall (@edwardloveall)
- Edward Romano (@oudeismetis)
- Edward Thomson (@ethomson)
- Edwin Martin (@edwinm)
- Edwin Smulders (@Dutchy-)
- EdwinEngelen (@EdwinEngelen)
- Ee Durbin (@ewdurbin)
- Eelco Bode (@eelcob)
- eelcohn (@eelcohn)
- Eerovil (@Eerovil)
- EetuRasilainen (@EetuRasilainen)
- efirestone (@efirestone)
- efp1 (@efp1)
- EgonMarmol (@EgonMarmol)
- Egor Romanko (@egor-romanko)
- Egor Shilyaev (@Fancy-dcm)
- Egor Tsinko (@etsinko)
- ehendrix23 (@ehendrix23)



- eieste (@eieste)
- Eiko Wagenknecht (@eikowagenknecht)
- Einar Jón (@einarjon)
- einarhauks (@einarhauks)
- einschmidt (@einschmidt)
- Eirik H (@eithe)
- Eirik Z (@atxbyea)
- Eiríkur Haraldsson (@eiki25)
- Eitan Mosenkis (@emosenkis)
- ejars (@ejars)
- ejaviga (@ejaviga)
- ekutner (@ekutner)
- Elad Bar (@elad-bar)
- Elahd Bar-Shai (@elahd)
- ElBalseiro (@ElBalseiro)
- elBoz73 (@elBoz73)
- Eldan Ben-Haim (@eldanb)
- Eleftherios Chamakiotis (@lexam79)
- Elena Rogleva (@erogleva)
- Elias Hunt (@raetha)
- Elias Karakoulakis (@ekarak)
- Eliot Wong (@eliotw)
- Eliran Turgeman (@VirtualL)
- Eliseo Martelli (@eliseomartelli)
- elithe1 (@elithe1)
- Elliot Morales Solé (@elliotmoso)
- Ellis Michael (@emichael)
- Ellis Percival (@flyte)
- ElmaxSrl (@ElmaxSrl)
- elmurato (@elmurato)
- Eloi Bail (@ebail)
- Eloston (@Eloston)
- elschnert (@elschnert)
- eltoro81 (@mvjt)
- Elvis (@mu3)
- elwing00 (@elwing00)
- elyesa (@ssl)
- elyobelyob (@elyobelyob)
- Em (@esciara)
- Emacee (@Emacee)

- Emanuel Winblad (@ManneW)
- Emanuele (@elax46)
- Emanuele (@ema987)
- Emanuele Palombo (@elbowz)
- emartec-ca (@emartec-ca)
- Emeric (@Mryck)
- emes30 (@emes30)
- Emil Horpen Hetty (@emilhetty)
- Emil Hørlyck (@eHorlyck)
- Emil Nildersen (@voxic)
- Emil Stjerneman (@bratanon)
- emil-e (@emil-e)
- emilgil (@emilgil)
- Emilv2 (@Emilv2)
- Emily Cassandra Meeker (@ecmeeker)
- Emily Love Mills (@emlove)
- EmitKiwi (@EmitKiwi)
- emkay82 (@emkay82)
- emlt (@emlt)
- Emmanuel Krebs (@e-krebs)
- Emmanuel Mwangi (@cloudbring)
- EmmanuelLM (@EmmanuelLM)
- Emory Penney (@ejpenney)
- Emre Saglam (@emresaglam)
- emufan (@emufan)
- endor (@endor-force)
- enegaard (@enegaard)
- Enrico Battistella (@battistaar)
- Enrico Berndt (@treehoof)
- Enrique Gonzalez (@enriquegh)
- Enrique Ocaña González (@eocanha)
- Enu Rist (@enurist)
- eparla774 (@eparla774)
- epenet (@epenet)
- eracknaphobia (@eracknaphobia)
- Erdem MEYDANLI (@meerd)
- eresonance (@eresonance)
- erffrfez (@erffrfez)
- Eric Clymer (@ericwclymer)
- Eric Coffman (@ericbrian)

- Eric Hagan (@ehagan)
- Eric Harris (@ericmharris)
- Eric Hiller (@erichiller)
- Eric Hosford (@HosfordDotMe)
- Eric J. Smith (@ejsmith)
- Eric Jansen (@ej81)
- Eric Jarrige (@jorasse)
- Eric Le Bihan (@elebihan)
- Eric Mai (@ericmai624)
- Eric Matte (@ericmatte)
- Eric Miller (@ericmillersfbay)
- Eric Nagley (@marchingphoenix)
- Eric Nelson (@ericnelsonaz)
- Eric Oosting (@eoosting)
- Eric Petway (@ebpetway)
- Eric Pignet (@ericpignet)
- Eric Rolf (@xrolfex)
- Eric Severance (@esev)
- Eric Stern (@Stormalong)
- Eric Svård (@ulmerkott)
- Eric Thomas (@et)
- Eric Thompson (@er0ck)
- ericgingras (@e850205)
- Erico Nunes (@enunes)
- ericvb (@ericvb)
- ericvitale (@ericvitale)
- Erik (@edekeijzer)
- Erik A (@erikarenhill)
- Erik B Andersen (@ErikBAndersen)
- Erik Bent (@erikbent)
- Erik Eriksson (@molobrakos)
- Erik Flodin (@erijo)
- Erik Gustavsson (@cyr123)
- Erik J. Olson (@arychj)
- Erik Kastelec (@erikkastelec)
- Erik Larsson (@ortogonal)
- Erik Montnemery (@emontnemery)
- Erik Raae (@mr-raw)
- Erik Seglem (@eseglem)
- Erik Stromdahl (@erstrom)

- Erik van Paassen (@evpaassen)
- Erik-jan Riemers (@riemers)
- erizhang (@erizhang)
- Erkka Tahvanainen (@tahvane1)
- erlendmoland (@erlendmoland)
- Ermanno Baschiera (@ebaschiera)
- Ernst Klamer (@Ernst79)
- ErnstEeldert (@ErnstEeldert)
- ERovirosa (@ERovirosa)
- Erwan Gautron (@ErwanGa)
- Erwin B (@eblekkenhorst)
- Erwin Oldenkamp (@Eernie)
- Esben Damgaard (@Ebbe)
- Esben Haabendal (@esben)
- escoand (@escoand)
- Esouder (@Esouder)
- EspenT (@EspenT)
- Ethan Madden (@jetpacktuxedo)
- etheralm (@etheralm)
- Etienne Carriere (@etienne-lms)
- Etienne G (@egguy)
- EtienneMD (@EtienneMD)
- Ettienne Gous (@ettiennegous)
- Etzion Bar-Noy (@ezaton)
- Eu (@covrig)
- Eugen Hristev (@ehristev)
- Eugene Kuzin (@kuzin2006)
- Eugene Pirogov (@gmile)
- Eugene Prystupa (@prystupa)
- Eugene Tarassov (@eugene-tarassov)
- eugeneniemand (@eugeneniemand)
- Eugenio Panadero (@azogue)
- Evan (@bigun27)
- Evan Bruhn (@evanjd)
- Evan Morse (@RowdyDog12)
- Evan Zelkowitz (@ezelkow1)
- evantaur (@evantaur)
- everix1992 (@everix1992)
- Evert Van den Bruel (@Evertvandenbruel)
- Everything Smart Home (@EverythingSmartHome)

- EvgeniiDidin (@EvgeniiDidin)
- Evgeniy (@evgeniy-khatko)
- Evgeny (@freekode)
- evoblice (@evoblice)
- EvolvingFuture (@EvTheFuture)
- ewgast (@ewgast)
- ews99 (@erwin-willems)
- explosivo22 (@explosivo22)
- eXtgmA (@eXtgmA)
- eyager1 (@eyager1)
- Eyal (@smhgit)
- Eyal Cohen (@eyalcha)
- Ezequiel Garcia (@ezequielgarcia)
- f8cfe@free.fr (@fillods)
- Fabian Affolter (@fabaff)
- Fabian Fischer (@nodomain)
- Fabian Heredia Montiel (@fabianhjr)
- Fabian Mewes (@archi-tekt)
- Fabian Peter Hammerle (@fphammerle)
- Fabian Rodriguez (@MagicFab)
- Fabian Seitz (@FaserF)
- Fabian Zimmermann (@devfaz)
- Fabien Marteau (@Martoni)
- Fabien Parent (@Fabo)
- Fabien Piuze (@reefab)
- Fabio (@pagadrift85)
- Fabio Fantoni (@Fantu)
- Fabio Porcedda (@fabio-porcedda)
- Fabio Porcedda (@telit-fabiopo)
- Fabio Urquiza (@fabiorush)
- fabiocastagnino (@fabiocastagnino)
- fabioestevam (@fabioestevam)
- Fabrice Fontaine (@ffontaine)
- Fabrice Pipart (@fabricepipart)
- Fabricio Avila (@fabricioavil)
- Fabrizio Furnari (@fabfurnari)
- Fabrizio Leonelli (@leofabri)
- Fabrizio Tarizzo (@bluviolin)
- fabtesta (@fabtesta)
- Fairesoimeme (@fairecasoimeme)

- fakezeta (@fakezeta)
- Falco Hyfing (@hyfing)
- fanvyr (@fanvyr)
- fapfaff (@fapfaff)
- Fares Rihani (@anchepiece)
- Farid (@ooii)
- Farzad Noorian (@fnoorian)
- faster777 (@faster777)
- Fatih Aşıcı (@fatih-asici)
- Faucogney Anthony (@afaucogney)
- Fazli Sapuan (@fuzzie360)
- fb22 (@fb22)
- Federico Ariel Castagnini (@facastagnini)
- Federico Granata (@Edo78)
- Federico Leoni (@effelle)
- Federico Marani (@fmarani)
- Federico Pellegrin (@fedepell)
- Federico Zivolo (@FezVrasta)
- Feliksas The Lion (@Feliksas)
- Felipe Contreras (@felipec)
- Felipe Cypriano (@fcy)
- Felipe Martins Diel (@felipediel)
- Felipe Santos (@felipecrs)
- Felix (@0xFelix)
- Felix (@vidschofelix)
- Felix Barbalet (@xlfe)
- Felix Breidenstein (@fleaz)
- Felix Eckhofer (@tribut)
- Felix Fischer (@felixfischer)
- Felix Ippolitov (@fellu)
- Felix Krause (@KrauseFx)
- Felix Stupp (@Zocker1999NET)
- Felix Stürmer (@weltenwort)
- Felix Vollmer (@fvollmer)
- fenner (@fenner)
- Ferdinand (@Smitplaza)
- ferdinand (@ferdinand)
- ferdydek (@ferdydek)
- Fergus (@fergus)
- Fermulator (@fermulator)

- Fernando (@ferazambuja)
- FeromonDE (@FeromonDE)
- Ferry van Zeelst (@ProtoThis)
- fhoekstra (@fhoekstra)
- fi-sch (@fi-sch)
- FidgetyRat (@FidgetyRat)
- Fietspomp86 (@Fietspomp86)
- fignew (@fignew)
- figurcoe (@figurcoe)
- Filip Bednárík (@drndos)
- Filip Pytloun (@fpytloun)
- Filip Skoneczny (@fsky3)
- Filip Van Ham (@filipvh)
- Filip van Hoeckel (@filip-van-hoeckel)
- Filipe Pina (@fopina)
- Filippo Sironi (@filipposironi)
- fillefilip8 (@fillefilip8)
- filmgarage (@filmgarage)
- Finbarr Brady (@fbradyirl)
- fineemb (@fineemb)
- finity69x2 (@finity69x2)
- Finn (@F1nnM)
- finnysamuel (@finnysamuel)
- Firestorm2010 (@Firestorm2010)
- firstmentoring (@firstmentoring)
- fiskhest (@fiskhest)
- FL550 (@FL550)
- flashoftheblades (@TheSwert)
- Flavien Charlon (@Flavien)
- Flavien Norindr (@Plumillon)
- Flavio Barisi (@flavio20002)
- Flavio Castelli (@flavio)
- FlavorFx (@FlavorFx)
- flebourse (@flebourse)
- Flemming Sørvollen Skaret (@flemmingss)
- FletcherAU (@FletcherAU)
- flfue (@flfue)
- flinkebernt (@flinkebernt)
- Flip Hess (@fliphess)
- flixhsw (@flixhsw)

- Flo (@hiFlo)
- floatiepen (@floatiepen)
- Flop2006 (@Flop2006)
- flopp999 (@flopp999)
- Florent Thoumie (@flz)
- florent valette (@fvalette)
- Florian (@1technophile)
- Florian (@gador)
- Florian Bachmann (@baflo)
- Florian Bernd (@flob Bernd)
- Florian Chauveau (@fchauveau)
- Florian Egner (@egnerfl)
- Florian Fainelli (@ffainelli)
- Florian Gareis (@TheZoker)
- Florian Harr (@caffeineflo)
- Florian Harwoeck (@harwoeck)
- Florian Heilmann (@FHeilmann)
- Florian Holzapfel (@florianholzapfel)
- Florian Klien (@flowolf)
- Florian Lagg (@LaggAt)
- Florian Ludwig (@FlorianLudwig)
- Florian Rüchel (@Javex)
- Florian Werner (@flo-wer)
- florianj1 (@florianj1)
- Florijan Hamzic (@cinatic)
- florincosta (@florincosta)
- Floris Van der kriecken (@florisvdk)
- Floyd Pink (@floydpink)
- Flsab (@flsabourin)
- fluzz (@fluzz)
- flyize (@flyize)
- fmartens (@fmartens)
- fmartinou (@fmartinou)
- FMKaiba (@FMKaiba)
- fnurgel (@fnurgel)
- fodi666 (@fodi666)
- fOmey (@fOmey)
- Fonta (@Fonta)
- foreign-sub (@foreign-sub)
- Forte (@fortepc)



- fotoetienne (@fotoetienne)
- fotvoren (@fotvoren)
- Foxler2010 (@foxler2010)
- foxy82 (@foxy82)
- fprokop (@fprokop)
- fran1987 (@fran1987)
- Francesco (@Leosirth)
- Francesco (@frasim)
- Francesco Negri (@dhinus)
- Francesco Nwokeka (@nwoki)
- Franchie (@Franchie)
- Francis Mendes (@francismendes)
- Francis Reyes (@fmr)
- Francisco Gonzalez Morell (@gzmorell)
- francispoisson (@francispoisson)
- Franck Nijhof (@frenck)
- Francois Chagnon (@EiNSTeiN-)
- Francois Gervais (@DC-fgervais)
- Frank (@syphernl)
- Frank (@BraveChicken1)
- Frank (@fcfort)
- Frank Bakker (@FrankBakkerNL)
- Frank Bergmann (@frajasalo)
- Frank Burmo (@FrankBurmo)
- Frank Hunleth (@fhunleth)
- Frank Niesten (@Frankniesten)
- Frank van Ierland (@fierland)
- Frank Vanbever (@frankvanbever)
- Frank Wickström (@frwickst)
- Frankster-NL (@Frankster-NL)
- Frankwin Hooglander (@Frankwin)
- Frans Saris (@fsaris)
- Frantz (@rofrantz)
- François (@mockersf)
- François LASSERRE (@ChoiZ)
- François Perrad (@fperrad)
- François-Olivier Devaux (@fodevaux)
- frasskungin (@mlauhalu)
- frazierjason (@frazierjason)
- Fred (@smokingdev)

- Fred Taylor-Young (@Fr3d)
- Freddie Leeman (@freddieleeman)
- Frederic Bor (@f-bor)
- Frederic Hemberger (@fhemberger)
- Frederic Seiler (@fredericseiler)
- Frederick Henderson (@frederickjh)
- fredericks1982 (@fredericks1982)
- FredericMa (@FredericMa)
- fredericvl (@fredericvl)
- Frederik Bolding (@FrederikBolding)
- Frederik Gladhorn (@gladhorn)
- Frederik Vannoote (@frederikvannoote)
- fredespi (@fredespi)
- Fredric Palmgren (@Sha-Darim)
- Fredrik (@fohlso2)
- Fredrik Andersson (@glyph-se)
- Fredrik Baberg (@fredrikbaberg)
- Fredrik Erlandsson (@fredrike)
- Fredrik Fjeld (@fredrikfjeld)
- Fredrik Haglund (@PetitCircuitLab)
- Fredrik Oterholt (@k1rd3rf)
- Fredrik Rambris (@fredrik-rambris)
- Fredrik Tuomas (@tuomaz)
- Fredrik Wendt (@FredrikWendt)
- FredrikFornstad (@FredrikFornstad)
- freekeys (@freekeys)
- freol35241 (@freol35241)
- Friedrieck (@Friedrieck)
- Fritiof Hedman (@zyberzero)
- frittes (@frittes)
- Fritz Mueller (@fritzm)
- frodcab (@frodcab)
- froz (@froz)
- Frédéric Kinnaer (@Censored3)
- Frédéric Sagnes (@ndfred)
- fuga2136 (@fuga2136)
- functionpointer (@functionpointer)
- Funtastix (@funtastix)
- FutureTense (@FutureTense)
- Fuzzy (@FuzzyMistborn)

- fwestenberg (@fwestenberg)
- fxxer (@fxxer)
- G Johansson (@gjohansson-ST)
- g00dd00d (@g00dd00d)
- G. Hussain Chinoy (@ghchinoy)
- Gabe (@gives1976)
- Gabe Cook (@gabe565)
- Gabor SZOLLOSI (@szogi)
- gabrialdestruir (@gabrialdestruir)
- Gabriel Barceló Soteras (@gabrielgbs97)
- Gabriel Oliveira (@gabrielboliveira)
- Gabriel Potter (@gpotter2)
- Gabriel Rauter (@rautesamtr)
- Gabriel Visser (@gvssr)
- gadget-man (@gadget-man)
- gadgetmobile (@gadgetmobile)
- GadgetReactor (@GadgetReactor)
- Gaetan Semet (@gsemet)
- Gage Benne (@gagebenne)
- Galen Richards (@tangerinehuge)
- galex80 (@galex80)
- galsom (@galsom)
- Galtwise (@Galtwise)
- Ganesh Hegde (@xgt001)
- Ganey (@ganey)
- Gao Liang (@gaoliang)
- Gao Xiang (@hsiangkao)
- Garen Torikian (@gjtorkian)
- Gareth Cooper (@gaco79)
- Gareth Ellis (@gareth-ellis)
- Garret Heaton (@powdahound)
- Garret Kelly (@gkelly)
- Garrett (@G-Two)
- Garrett Welson (@garrettwelson)
- garrettbeachy (@garrettbeachy)
- Gary Barclay (@Bodge-IT)
- Gary Bisson (@gibsson)
- Gary Cobb (@gcobb321)
- Gary Coulbourne (@ursine)
- Gary Yendell (@GDYendell)

- GaryOkie (@GaryOkie)
- Gaston Dombiak (@gdombiak)
- Gaurav Kulkarni (@gauravkulkarni96)
- Gautham Varma K (@GauthamVarmaK)
- Gauthier B (@redkite127)
- Gautier Vanderslyen (@jtbgroup)
- Gavin Mogan (@halkeye)
- Gavin Ni (@widewing)
- Gavin Staniforth (@gsdevme)
- gazoodle (@gazoodle)
- gazoscalvertos (@gazoscalvertos)
- Gaël PORTAY (@gportay)
- GBP (@CubicPill)
- gde79 (@gde79)
- Gedde (@raygoat)
- geduxas (@geduxas)
- geekman2 (@geekman2)
- geekofweek (@geekofweek)
- Geert (@ge2rt)
- Geert van Horrik (@GeertvanHorrik)
- Geert Visser (@GeertVisser)
- geftactics (@geftactics)
- geirra (@geirra)
- Geliras (@Geliras)
- gemolnar (@gemolnar)
- gentoo-root (@gentoo-root)
- Geo Maciolek (@GeoMaciolek)
- Geoff (@gapple)
- Geoff Davis (@geoffdavis)
- Geoff Levand (@glevand)
- Geoff Norton (@kangaroo)
- Geoff Pursell (@geoffp)
- GeoffAtHome (@GeoffAtHome)
- Geoffrey (@maschel)
- Geoffrey Lagaisse (@geoffreylagaisse)
- Geoffrey Ragot (@gfyrag)
- Geoffrey Westhoff (@GeoffreyWesthoff)
- George Marshall (@georgemarshall)
- George Redivo (@redivo)
- George Vedamanickam (@intractive)

- George Zhao (@georgezhao2010)
- George.M (@nodinosaur)
- Georges Savoundararadj (@manoj23)
- Georgi Gardev (@GeorgeSG)
- Georgi Kirichkov (@kirichkov)
- Georgi Todorov (@TeraHz)
- Georgi Yanev (@jumpalottahigh)
- Georgii Staroselskii (@staroselskii)
- georgroehl (@georgroehl)
- Gerald Hansen (@geraldhansen)
- Gerard (@gerard33)
- Gerardo Castillo (@altersis)
- Gerben Meijer (@infernix)
- Gerben ten Hove (@gurbyz)
- Gergely Imreh (@imrehg)
- Gergely Peidl (@geripgeri)
- gerliczky (@gerliczky)
- Geronimo2015 (@Geronimo2015)
- Gert (@Gerto)
- Gert Jansen van Rensburg (@gertjvr)
- Gert-Jan van de Streek (@keerts)
- gertdb (@gertdb)
- gevans (@gevans)
- Ghannes (@Ghannes)
- ghermant (@ghermant)
- Gian Klug (@gianklug)
- giangvo (@giangvo)
- Gianluca Barbaro (@MrMep)
- Giannie (@Giannie)
- Giannuz (@Giannuz)
- Gianpaolo Macario (@gmacario)
- Gianpiero (@jumping2000)
- gibman (@gibman)
- gibwar (@gibwar)
- Gido (@GidoHakvoort)
- giefca (@giefca)
- Giel Janssens (@gieljnssns)
- Giel van Schijndel (@muggenhor)
- gigatexel (@gigatexel)
- Gijs Reichert (@GGeezes)

- Gil Peeters (@grillp)
- Gilad Ben-Yossef (@gby)
- Gilad Peleg (@pgilad)
- Gilles Margerie (@Gilles95)
- Gilles TALIS (@gtalis)
- gilles-chanteperdrix (@gilles-chanteperdrix)
- Gilson Júnior (@gilsonmandalogo)
- Giorgio Aresu (@GiorgioAresu)
- Giorgio Ravera (@xraver)
- Giorgos Logiotatidis (@glogiotatidis)
- Giovanni Campagna (@gcampax)
- Giovanni Iachello (@giachello)
- GitBook Bot (@gitbook-bot)
- GitHub Action (@GitHub-Action)
- githubbuh (@githubbuh)
- GitHubGoody (@GitHubGoody)
- gitmirko (@gitmirko)
- gitmopp (@gitmopp)
- gitolicious (@gitolicious)
- gitvb (@gitvb)
- gitwazza (@gitwazza)
- Giulio Benetti (@giuliobenetti)
- Giuseppe (@glpatcern)
- Giuseppe Iannello (@giannello)
- gizmocuz (@gizmocuz)
- GJH (@OpenMyDog)
- gjong (@gjong)
- glcos (@glcos)
- Gleb Mazovetskiy (@glebm)
- Gleb Sinyavskiy (@zhulik)
- Gleekop (@Gleekop)
- Glen Buktenica (@gbuktenica)
- Glen Eustace (@geustace)
- Glen Takahashi (@glentakahashi)
- Glenn Morrison (@atomicpapa)
- Glenn Waters (@gwww)
- glenn20 (@glenn20)
- Gluek (@mrgluek)
- Glyn Hudson (@glynhudson)
- glynaunac (@glynaunac)

- Glyph (@glyph)
- GMFalka (@GMFalka)
- gmlupatelli (@gmlupatelli)
- gngj (@gngj)
- Goir (@goir)
- goldminenine (@goldminenine)
- GoNzCiD (@GoNzCiD)
- Gonçalo Salazar (@glbsalazar)
- Gopal (@gopalkildoliya)
- Gordon Allott (@gordallott)
- GoSpursGoNL (@GoSpursGoNL)
- goto100 (@goto100)
- gozasc (@gozasc)
- GP8x (@GP8x)
- gpatkinson (@gpatkinson)
- gpgmailencrypt (@gpgmailencrypt)
- gppanayotov (@gppanayotov)
- Gradyn Wursten (@GNUGradyn)
- Graeme Smith (@Instagraeme)
- Graham Arthur Blair (@grablair)
- Graham Christensen (@grahamc)
- Graham Holland (@gmholland)
- Graham Rogers (@TastyPi)
- Graham Wetzler (@grahamwetzler)
- grandwolf (@grandwolf)
- Grant Edwards (@GrantEdwards)
- Grant Emsley (@grantemsley)
- Grant Russell (@ukgrant)
- Grayson Adams (@GraysonCAdams)
- grdnkln (@grdnkln)
- Green Lightning (@GreenLightning)
- GreenTurtwig (@GreenTurtwig)
- Greg (@gtdiehl)
- Greg (@oziee)
- Greg (@theCMack)
- Greg Badros (@gjbados)
- Greg Barker (@fletchowns)
- Greg Dowling (@pavoni)
- Greg Hesp (@greghesp)
- Greg Johnson (@notgwj)

- Greg Kroah-Hartman (@gregkh)
- Greg Laabs (@OverloadUT)
- Greg MacLellan (@gregmac)
- Greg Martin (@gregorymartin)
- Greg Rapp (@gdrapp)
- Greg Schwartz (@gregschwartz)
- Greg Sheremeta (@gregsheremeta)
- Greg Stevenson (@gstevenson)
- Greg Thornton (@xdissent)
- Greg Troxel (@gdt)
- Greg Wildman (@GregWildman)
- Greg. A (@gautric)
- gregod (@gregod)
- Gregor Gruener (@ggruner)
- Gregory Benner (@Klathmon)
- Gregory Clement (@gclement)
- Gregory D (@gregd72002)
- Gregory Dosh (@GregoryDosh)
- Gregory Knox (@Knoxie)
- gregwis (@gregwis)
- gremblin (@e2m32)
- griddyadmin (@griddyadmin)
- grimlock (@milezmilez)
- Grodesch (@Grodesch)
- groot406 (@groot406)
- groth-its (@groth-its)
- gryzli133 (@gryzli133)
- Grzegorz Blach (@gblach)
- Grzegorz Matyszczyk (@gmatyszczyk)
- Grégoire Delattre (@gregdel)
- Grégoire M. (@M-Gregoire)
- Grégoire Seux (@kamaradclimber)
- gstorer (@gstorer)
- GSzabados (@GSzabados)
- GTH (@gunnarhelgason)
- gth (@gth)
- gudata (@gudata)
- Guess (@mynameisdaniel32)
- Guglielmo De Concini (@cr0wbar)
- GuGu927 (@GuGu927)



- Guido Schmitz (@Shutgun)
- Guilherme Conti Teixeira (@guiconti)
- Guillaume DELVIT (@guiguid)
- Guillaume Duveau (@guix77)
- Guillaume Gardet (@ggardet)
- Guillaume Rischard (@grischard)
- Guillaume SABBE (@gsabbe)
- Guillaume Smith (@gusmith)
- Guillaume Zin (@guillaumezin)
- guillaume1410 (@guillaume1410)
- guillempages (@guillempages)
- Guillermo A. Amaral (@gamaral)
- Guillermo González (@GuilleGF)
- Guillermo Ruffino (@glmnet)
- GuISm0 (@GuISm0)
- GuitarSkater (@GuitarSkater)
- Guliver (@Guliver)
- Gummientchen (@Gummientchen)
- Gunnar Klauberg (@Aeroid)
- Guo Ren (@guoren83)
- guozi7788 (@guozi7788)
- Gustav Ahlberg (@Gyran)
- Gustav Alerby (@gbyx3)
- Gustavo Pimentel (@gustavoSNPS)
- Gustavo Stor (@godely)
- Gustavo Sverzut Barbieri (@barbieri)
- Gustavo Zacarias (@gustavoz)
- Guus Hutschemaekers (@Guus-H)
- Guy Khmelnitsky (@GuyKh)
- Guy Lewin (@GuyLewin)
- Guy Parisi (@Guyanthalas)
- Guy Sheffer (@guysoft)
- Guy Sie (@GuySie)
- guygma (@guyest)
- Guyohms (@Guyohms)
- gwbres (@gwbres)
- gwendalg (@gwendalg)
- Gwenhael (@trabucayre)
- gwhiteCL (@gwhiteCL)
- gwmullin (@gwmullin)

- Gyosa3 (@Gyosa3)
- Gábor Frank (@frankyhun)
- Gábor Gyenge (@gyengus)
- Gábor Kiss (@g-kiss)
- h1tch007 (@h1tch007)
- h2zero (@h2zero)
- h3ndrik (@h3ndrik)
- h4ckNinja (@h4ckninja)
- H. Árkosi Róbert (@nagyrobi)
- H.Abraham (@abrahamh)
- ha0y (@ha0y)
- ha14937 (@clandestine-avocado)
- haakon storm heen (@haakonstorm)
- Hackashaq666 (@Hackashaq666)
- hackerESQ (@hackerESQ)
- Hadi Victorya (@hadipsy27)
- Hadrien B (@hbouttev)
- Hadrien Milano (@hmil)
- HAEdwin (@HAEdwin)
- Haemish Kyd (@haemishkyd)
- hagenuck1 (@hagenuck1)
- hahn-th (@hahn-th)
- Haim Gelfenbeyn (@haimgel)
- Halász Dávid (@skateman)
- Hamid Elaosta (@hamid-elaosta)
- Hans Bulfone (@bulf1)
- Hans Knöchel (@hansemannn)
- Hans Kröner (@hanskroner)
- Hans Oischinger (@oischinger)
- Hans Svedåker (@svedaker)
- Hans-Christian Otto (@hco)
- hansgoed (@hansgoed)
- hanzoh (@hanzoh)
- hanzukun (@hanzukun)
- happyleavesaoc (@happyleavesaoc)
- Harald Klein (@haklein)
- Hareesh M U (@hareeshmu)
- Harm-Jan Roskam (@harmjanr)
- Haroen Viaene (@Haroenv)
- Harris Borawski (@hborawski)

- Harrison Cook (@HCookie)
- Harrison Pace (@thehaxxa)
- Harry Kantas (@harrykantas)
- Harryjholmes (@Harryjholmes)
- Harsh Kevadia (@harshjk)
- Harshit Sanghvi (@sanghviharshit)
- HarvsG (@HarvsG)
- Harvtronix (@Harvtronix)
- hasscasts (@hasscasts)
- Hate-Usernames (@Hate-Usernames)
- havmind (@havmind)
- hawk259 (@hawk259)
- haXs (@JayBkr)
- Hayley McIlldoon (@shmooley)
- HBDK (@HBDK)
- hblaschka (@hblaschka)
- hboulakhrif (@hboulakhrif)
- hcoohb (@hcoohb)
- hcooper (@hcooper)
- He Chunhui (@hchunhui)
- Heath Paddock (@heathbar)
- Hebezo (@Kannix2005)
- hecon5 (@hecon5)
- Hector Kesari (@hkesari)
- Hector Oron (@zumbi)
- Hedda (@Hedda)
- Hedgehog57 (@Hedgehog57)
- Heiko Helmle (@netzwer2)
- Heiko Rothe (@mKeRix)
- Heiko Stuebner (@mmind)
- Heiko Thiery (@hthiery)
- HeimdallMidgard (@HeimdallMidgard)
- Heine Furubotten (@hfurubotten)
- Heinrich Dahms (@htdahms)
- Heisenberg (@Heisenberg2980)
- hello-world-dot-c (@hello-world-dot-c)
- Helmut Januschka (@hjanuschka)
- Henco Appel (@hencoappel)
- Hendrik Schröter (@Rikorose)
- Henne (@HennieLP)

- Henning Dickten (@hensing)
- Henri Bragge (@hbragge)
- Henri Roosen (@roosen)
- Henrik (@werkstrom)
- Henrik Aronsson (@heennkkee)
- Henrik Carlioth (@carlioth)
- Henrik Nicolaisen (@hmn)
- Henrik Sozzi (@energywave)
- Henrique Camargo (@henriqueqc)
- Henry Ou (@henryouly)
- henryk (@henryk)
- henryptung (@henryptung)
- HepoH3 (@HepoH3)
- Heriberto Madrigal (@magic-madrigal)
- Hermann Kraus (@herm)
- Hernán (@hmronline)
- Herr Frei (@herrfrei)
- Hessel Bierma (@hesje)
- hesselonline (@hesselonline)
- hexxter (@hexxter)
- heyajohnny (@heyajohnny)
- heydonms (@heydonms)
- heymoe (@heymoe)
- heytcass (@heytcass)
- HFeenstra (@HFeenstra)
- hg1337 (@hg1337)
- hieptv (@hieptv2902)
- hif2k1 (@hif2k1)
- HighOnMikey (@HighOnMikey)
- Hikaru Wada (@sugoi-wada)
- Hilbrand Bouwkamp (@Hilbrand)
- Hillary Fraley (@hillaryfraley)
- hioctane61 (@hioctane61)
- hitokiri8x (@hitokiri8x)
- hiveai (@hiveai)
- Hmmbob (@hmmbob)
- Hoang Tran (@hoangtran)
- hojland (@Hojland)
- hokagegano (@hokagegano)
- holdestmade (@holdestmade)

- holelattanuttin (@holelattanuttin)
- Hollis Blanchard (@slightlyunconventional)
- HollyFredD (@HollyFredD)
- holysoles (@holysoles)
- Home Assistant Bot (@homeassistant)
- honcheng (@honcheng)
- Honza Slesinger (@slesinger)
- Hoobie7 (@Hoobie7)
- hoopsta1423 (@hoopsta1423)
- HoppingMonk (@HoppingMonk)
- HoratiuVultur (@HoratiuVultur)
- horga83 (@horga83)
- hotswapster (@hotswapster)
- Hovo (Luke) (@lhovo)
- Hoytron (@Hoytron)
- Hristo Atanasov (@hristo-atanasov)
- hSATAC (@hSATAC)
- htcheroportugal (@htcheroportugal)
- htmltiger (@htmltiger)
- Hu Hao (@howiehu)
- HUANG da (@huangda1982)
- huangyupeng (@huangyupeng)
- hubbergit (@hubbergit)
- Hubert Lacote (@hubert-lacote-yv)
- hubertbanas (@hubertbanas)
- Hugh Eaves (@hugheaves)
- Hugo Dupras (@jabesq)
- Hugo Gresse (@HugoGresse)
- Hugo Herter (@hoh)
- Hugo Hromic (@hhromic)
- Hugues Granger (@Wohlraj)
- huimang2 (@huimang2)
- hulkhaugen (@hulkhaugen)
- hultenvp (@hultenvp)
- Hung Le (@hungle)
- Hung Nguyen (@hungnguyenm)
- hung2kgithub (@aes-alienrip)
- Hunter Horsman (@kariudo)
- HunterX86 (@HunterX86)
- Huon Wilson (@huonw)

- Hurzelchen (@hurzelchen)
- husky-koglhof (@husky-koglhof)
- Huw Davies (@beardedgeek)
- Huwee2 (@Huwee2)
- HWL1968 (@HWL1968)
- hwmland (@hwmland)
- hybridparadigm (@hybridparadigm)
- Hydreliox (@HydrelioxGitHub)
- Hypnos (@Hypnos3)
- hypnosiss (@hypnosiss)
- Håkan Åkerberg (@hakana)
- Håvard Elnan (@havardelnan)
- i-am-shodan (@i-am-shodan)
- Iain Matchett (@matchett808)
- IainPHay (@IainPHay)
- iainsmacleod (@iainsmacleod)
- Ian (@ViViDboarder)
- Ian (@Apocrathia)
- Ian (@ianperrin)
- Ian (@TheBassEngineer)
- Ian Byrne (@ianByrne)
- Ian Copp (@icopp)
- Ian Darwin (@IanDarwin)
- Ian Day (@iandday)
- Ian Duffy (@imduffy15)
- Ian Foster (@ianrat)
- Ian Gibbs (@realflash)
- Ian Harcombe (@MrHarcombe)
- Ian Haylock (@haylocki)
- Ian Richardson (@iantrich)
- Ian Slinger (@ianjs)
- Ian T. Wright (@IanTWright)
- Ian Tewksbury (@itewk)
- iangregory (@iangregory)
- ianj001 (@ianj001)
- iAutom8 (@iAutom8)
- ic3cool (@ic3cool)
- IceBotYT (@IceBotYT)
- IceEyz (@IceEyz)
- icemanch (@icemanch)

- IceOnly (@IceOnly)
- Icexist (@Icexist)
- icovada (@icovada)
- idfxken (@idfxken)
- iDVB (@iDVB)
- IGastel (@IGastel)
- Ignacio Hernandez-Ros (@IgnacioHR)
- Ignacy Gawędzki (@iazz)
- Igor (@IATkachenko)
- Igor Bernstein (@igorbernstein2)
- Igor Gocalinski (@grogI)
- Igor Loborec (@Bikonja)
- Igor Magès (@igormages)
- Igor Motov (@imotov)
- Igor Pakhomov (@Kirmas)
- Igor Santos (@igorsantos07)
- Igor Shults (@ishults)
- Ihor Kostiuk (@ki0ki0)
- Ikko Ashimine (@eltoclear)
- ikonixx (@ikonixx)
- Iku (@yokattana)
- ikucuze (@ikucuze)
- ildar170975 (@ildar170975)
- Ilia Sotnikov (@hostcc)
- Ilias Apalodimas (@apalos)
- iliketoprogram14 (@iliketoprogram14)
- Ilja Leiko (@leikoilja)
- Illia Grybkov (@igrybkov)
- Ilya Averyanov (@a1ien)
- Ilya Kuzmich (@ikv)
- Ilya Lipnitskiy (@lipnitsk)
- iLyas Bakouch (@isbkch)
- im-boots (@im-boots)
- indexample (@indexample)
- Indu Prakash (@iprak)
- indykoning (@indykoning)
- infefeeee (@infefeeee)
- Ing. Jaroslav Šafka (@jedi7)
- Ingmar Stein (@IngmarStein)
- Ingo Theiss (@itn3rd77)

- Inovelli (@InovelliUSA)
- inputd (@inputd)
- insajd (@insajd)
- Ioan Loosley (@ioangogo)
- ioctl2 (@ioctl2)
- ionred (@ionred)
- Ionut Popovici (@intelroman)
- Ionuț Neagu (@IonutNeagu)
- IoTmessenger (@IoTmessenger)
- ioull (@ioull)
- ipetrovits (@ipetrovits)
- ipreston (@ipreston)
- Irad Aharoni (@irad100)
- Irfaq Syed (@irazasyed)
- irvingwa (@irvingwa)
- Isaac I.Y. Saito (@130s)
- Isabella Gross Alström (@isabellaalstrom)
- isk0001y (@isk0001y)
- Ismael Luceno (@ismaell)
- Issac (@issacg)
- Issac Kelly (@issackelly)
- iStitch07 (@iStitch07)
- itairaz1 (@itairaz1)
- Itamar Dori (@itamARBareket)
- itchannel (@itchannel)
- itineric (@itineric)
- iTitou (@titouanc)
- Itzik Ephraim (@oranja)
- Iulian Onofrei (@revolter)
- Iulius (@tuxbox)
- Ivan Belokobylskiy (@devbis)
- Ivan Bessarabov (@bessarabov)
- Ivan Dyedov (@idyedov)
- Ivan Herrera (@Klievan)
- Ivan Puddu (@dn0sar)
- IVEEL (@iveelco)
- Ivo Slanina (@buben19)
- Ivo Wever (@Confusion)
- Ivo-tje (@Ivo-tje)
- Iván Cea Fontenla (@ivancea)



- J Evans (@g40)
- j james (@julijames)
- J M H (@jay-em-Ach)
- J3n50m4t (@J3n50m4t)
- J4nsen (@J4nsen)
- j-a-n (@j-a-n)
- J-CMartin (@J-CMartin)
- J-Lindvig (@J-Lindvig)
- J. B. Rainsberger (@jbrains)
- J. Nick Koston (@bdraco)
- J. Tang (@devnull42)
- J.A.P. Klessens (@JKlessens)
- J.C. Woltz (@jcwoltz)
- J.P. Hutchins (@JPHutchins)
- J.P. Krauss (@asymworks)
- Jaak Laineste (@jaakla)
- Jaap Crezee (@jaapcrezee)
- jaburges (@jaburges)
- JaccoR (@JaccoR)
- Jacek Kończewski (@jacekk015)
- jacekpaszkowski (@jacekpaszkowski)
- Jacen (@jacen92)
- Jack (@jackjohnsonuk)
- jack (@jackmakesthings)
- Jack (@iamjackg)
- Jack A (@wasteofusername)
- Jack Boswell (@boswelja)
- Jack Chapple (@jbroadice)
- Jack Fan (@JackWindows)
- Jack Kao (@jackzzjack)
- Jack Minardi (@jminardi)
- Jack Wilsdon (@jackwilsdon)
- Jack Yin (@jkyin)
- jack1142 (@jack1142)
- jack5mikemotown (@jack5mikemotown)
- Jackie Yang (@valksaaa)
- Jaco Kok (@jacokok)
- Jacob Donenfeld (@jacobdonenfeld)
- Jacob Mansfield (@girlpunk)
- Jacob McSwain (@USA-RedDragon)

- Jacob Minnis (@jminn)
- Jacob Penderworth (@Penderworth)
- Jacob Schwartz (@schwartzpub)
- Jacob Shufro (@jshufro)
- Jacob Siverskog (@jsiverskog)
- Jacob Snyder (@JacobSnyder)
- Jacob Southard (@thevoltagesource)
- Jacob Tomlinson (@jacobtomlinson)
- Jacob Weisz (@ocdtrekkie)
- Jacques-D. Piguet (@jdpiguet)
- Jadson Santos (@gtjadsonsantos)
- Jafar Atili (@jafar-atili)
- Jagan Teki (@openedev)
- jagjordi (@jagjordi)
- Jaimyn Mayer (@jabelone)
- Jake McCrary (@jakemcc)
- jakecolman (@jakecolman)
- Jakob Reiter (@jakommo)
- Jakob Ruhe (@jakeru)
- Jakob Schlyter (@jschlyter)
- Jakub (@lenisko)
- Jakub Bartkowiak (@gralin)
- Jakub Bednář (@bednar)
- Jakub Cabak (@jcabak)
- Jakub Dąbrowski (@dabku)
- Jakub Kolář (@LordBoos)
- Jakub Piasecki (@zaporylie)
- Jakub Schmidtke (@sjakub)
- Jakub Wilk (@jwilk)
- jakubradziwon (@jakubradziwon)
- jambees (@jambees)
- James (@jimbob1001)
- James (@EnochPrime)
- James Alseth (@jalseth)
- James Baker (@j-baker)
- James Bolean (@slicen)
- James Callaghan (@jcallaghan)
- James Chaloupka (@SirGoodenough)
- James Cole (@jamespcole)
- James Crook (@cooljimmy84)

- James Goodhouse (@jamesgoodhouse)
- James Grant (@jamesg-nz)
- James Hebden (@devec0)
- James Hewitt (@Jamstah)
- James Hilliard (@jameshilliard)
- James Hodgkinson (@yaleman)
- James Kiefer (@zinefer)
- James Knight (@jdknight)
- James Marsh (@doctorjames)
- James Milligan (@nightowlengineer)
- James Myatt (@jamesmyatt)
- James Nimmo (@jnimmo)
- James Ruan (@jamesruan)
- James Stroud (@strouja)
- James Sun (@sun16)
- James Szalay (@jtszalay)
- James Taylor (@jt-nti)
- James Warne (@bebleo)
- James Yox (@yoxjames)
- james-fry (@james-fry)
- jamesking420 (@jamesking420)
- JamesMcClelland (@JamesMcClelland)
- JamesT44 (@JamesT44)
- jamesthomas128 (@jamesthomas128)
- JamesW-AU (@JamesW-AU)
- JamesWong9 (@JamesWong9)
- Jamie Macdonald (@sidequestboy)
- Jamie Shaw (@jamieshaw)
- Jamie van Dyke (@fearoffish)
- Jamie Weston (@jlweston)
- Jamin Collins (@jaminscollins)
- jaminh (@jaminh)
- Jan (@jevermeister)
- Jan (@jnxxx)
- Jan (@js94x)
- Jan (@climblinne)
- Jan Almeroth (@jalmeroth)
- Jan Bouwhuis (@jbouwh)
- Jan C (@infradom)
- Jan Castermans (@pprazzi)

- Jan Collijs (@visibilityspots)
- Jan De Luyck (@jdeluyck)
- Jan Dohl (@polygon)
- Jan Harkes (@jaharkes)
- Jan Heylen (@heyleke)
- jan iversen (@janiversen)
- Jan Jedelský (@jjedelsky)
- Jan Jurča (@janjurca)
- Jan Keith Darunday (@jkcdarunday)
- Jan Kiszka (@jan-kiszka)
- Jan Kraval (@jkraval)
- Jan Kunderát (@jktjkt)
- Jan Losinski (@janLo)
- Jan Löffler (@jan-loeffler)
- Jan Nylund (@jannylund)
- Jan Olbrecht (@olbjan)
- Jan Pedersen (@jp-embedded)
- Jan Peer Stöcklmair (@JPeer264)
- Jan Pobořil (@iBobik)
- Jan Seidl (@jseidl)
- Jan Sepke (@jansepke)
- Jan Stienstra (@j-stienstra)
- Jan van Helvoort (@janvanhelvoort)
- Jan Viktorin (@jviki)
- Jan Willem Janssen (@jawi)
- Jan Willhaus (@janw)
- Jan-Philipp Litza (@jplitza)
- Jan-Preben Mossin (@jpmossin)
- Jan-Willem Mulder (@jwnmulder)
- Janick Bergeron (@janick)
- Janis Jansons (@Janhouse)
- Janitha Karunaratne (@janitha)
- janlugt (@janlugt)
- Janne Grunau (@jannau)
- Jannik Beyerstedt (@jbeyerstedt)
- Jannis Göing (@jannis3005)
- Janos Racz (@rczjns)
- Jordi Martinez (@jardiamj)
- Jared Agee (@jagee23)
- Jared Beckham (@jtbeckha)

- Jared Bents (@jmbents)
- Jared Hobbs (@jaredhobbs)
- Jared J (@jjensn)
- Jared Quinn (@jaredquinn)
- Jari Ylimäinen (@Jarilnc)
- Jarle B. Hjortand (@jarlebh)
- Jarmo van Lenthe (@jarmovanlenthe)
- Jarod Wilson (@jarodwilson)
- Jaroslav Hanslík (@kukulich)
- Jarosław Salwa (@armata007)
- Jaryl Chng (@jarylcn)
- Jason (@zachowj)
- Jason A. Donenfeld (@zx2c4)
- Jason Albert (@thejta)
- Jason Antman (@jantman)
- Jason Carter (@JasonCarter80)
- Jason Cheatham (@jason0x43)
- Jason Cronquist (@jcronq)
- Jason Fowler (@jsnfwlr)
- Jason Heddings (@jheddings)
- Jason Hines (@jasonehines)
- Jason Hu (@awarecan)
- Jason Hunter (@hunterjm)
- Jason Kingsbury (@relvacode)
- Jason Knott (@jayknott)
- Jason Kölker (@jkoelker)
- Jason Lachowsky (@dajo)
- Jason Lawrence (@jjlawren)
- Jason Madigan (@jasonmadigan)
- Jason Maners (@jlmaners)
- Jason Nader (@ammgws)
- Jason Pelzer (@jpelzer)
- Jason Pruitt (@jrspruitt)
- Jason Rebelo (@igotinfected)
- Jason Ross (@csfreak)
- Jason Rumney (@make-all)
- Jason Schollenberger (@jschollenberger)
- Jason Scurtu (@xarbit)
- Jason Swails (@swails)
- Jason Wells (@sentientapp)

- Jason White (@jdwhite)
- Jason Woodford (@jwood55812)
- Jason Woodward (@woodwardjd)
- Jason Wragg (@jasonwragg)
- Jason-nz (@Jason-nz)
- jasonbuechler (@jasonbuechler)
- jasondefuria (@jasondefuria)
- Jasper Slits (@jasperslits)
- Jasper Smulders (@JasperS2307)
- Jasper van der Neut - Stulen (@jvanderneutstulen)
- JasperPlant (@JasperPlant)
- jasperro (@jasperro)
- Jatin Desai (@jdesai61)
- javicalle (@javicalle)
- Javier Domingo Cansino (@txomon)
- Javier Gonel (@graffic)
- Javier González Calleja (@gonzalezcalleja)
- Javier Lopez (@muniter)
- Javier Martínez (@JavierMartinz)
- Javier Viguera (@jviguera)
- javixuwi (@javixuwi)
- Jaxom Nutt (@JaxomCS)
- Jay (@jshridha)
- Jay Love (@jslove)
- Jay Newstrom (@JayNewstrom)
- Jay Stevens (@Jay2645)
- Jay Tuckey (@jay-tuckey)
- jaydeehitop (@jaydeehitop)
- Jayden (@lukyjay)
- Jayden Litolff (@JayBigGuy10)
- jaydesl (@jaydesl)
- Jaydev Shiroya (@jaydevs)
- JAYMAN-ATX (@JAYMAN-ATX)
- JayOne73 (@JayOne73)
- jazzaj (@jazzaj)
- jazzy-r (@jazzy-r)
- jbcodemonkey (@jbcodemonkey)
- JBelinchon (@JBelinchon)
- jbrody17 (@jbrody17)
- JBS (@JBS5)

- [jc \(@gobanza\)](#)
- [JC Connell \(@jcconnell\)](#)
- [Jc2k \(@Jc2k\)](#)
- [jcEcaSinters \(@jcEcaSinters\)](#)
- [jchasey \(@jchasey\)](#)
- [jchus-dot-id \(@jchus-id\)](#)
- [jcoetsie \(@jcoetsie\)](#)
- [jcrowegitHu8 \(@jcrowegitHu8\)](#)
- [jdeath \(@jdeath\)](#)
- [jdelaney72 \(@jdelaney72\)](#)
- [jduquennoy \(@jduquennoy\)](#)
- [Jean Gauthier \(@SupremeSports\)](#)
- [Jean Regisser \(@jeanregisser\)](#)
- [Jean Swart \(@swartjean\)](#)
- [Jean-Christophe DUBOIS \(@jcdubois\)](#)
- [Jean-Christophe PLAGNIOL-VILLARD \(@plagnioj\)](#)
- [Jean-François Auger \(@nechry\)](#)
- [Jean-François Paris \(@jfparis\)](#)
- [Jean-François Roy \(@jfroy\)](#)
- [Jean-Michel Julien \(@KurdyMalloy\)](#)
- [Jean-Michel Ruiz \(@coolcow\)](#)
- [Jean-Paul Etienne \(@fractalclone\)](#)
- [Jean-Paul van Ravensberg \(@DevSecNinja\)](#)
- [Jean-Philippe Bouillot \(@Jypy\)](#)
- [Jean-Philippe Jodoin \(@jpjodoin\)](#)
- [Jean-pierre Cartal \(@jeanpierrecartal\)](#)
- [Jean-Yves Avenard \(@jyavenard\)](#)
- [Jed Lippold \(@jlippold\)](#)
- [Jedmeng \(@jedmeng\)](#)
- [JeeCee1 \(@JeeCee1\)](#)
- [Jeef \(@jeefftor\)](#)
- [Jeena Paradies \(@jeena\)](#)
- [Jef D \(@danielsjf\)](#)
- [Jeff \(@jumbledbytes\)](#)
- [Jeff Bailey \(@kaladron\)](#)
- [Jeff Basso \(@Jjbasso\)](#)
- [Jeff Boothe \(@jboothe\)](#)
- [Jeff Cutsinger \(@jeffcutsinger\)](#)
- [Jeff H \(@jeffhentschel\)](#)
- [Jeff Irion \(@JeffLlirion\)](#)

- Jeff Koenig (@TheJefe)
- Jeff Lewis (@Jeff-Lewis)
- Jeff McGehee (@jlmcgehee21)
- Jeff Rescignano (@JeffResc)
- Jeff Schroeder (@SEJeff)
- Jeff Sloyer (@jsloyer)
- jeff tapia (@jtmmoderate876)
- Jeff Terrace (@jterrace)
- Jeff Wilson (@jawilson)
- Jeff Zignego (@HED-jzignego)
- jeff-h (@jeff-h)
- Jefferson Bledsoe (@JeffersonBledsoe)
- Jeffery To (@jefferyto)
- jeffh0821 (@jeffh0821)
- Jeffrey (@Sjeff)
- Jeffrey (@theglus)
- Jeffrey Lin (@linjef)
- jeffsanicola (@jeffsanicola)
- Jelle Raaijmakers (@GMTA)
- Jelle Sels (@jellesels)
- Jelle Spijker (@jellespijker)
- Jelmer Tiete (@JelmerT)
- Jelmer van der Stel (@steljwagh)
- jelmerkk (@jelmerkk)
- Jelte Zeilstra (@J3173)
- Jenny (@pinkywafer)
- Jens (@jensihnow)
- Jens (@jhoepken)
- Jens Kohl (@jk)
- Jens Maus (@jens-maus)
- Jens Mazzanti-Myretyr (@jm-73)
- Jens Nistler (@lociii)
- Jens Vanhooydonck (@JensVanhooydonck)
- Jens Østergaard Nielsen (@dingusdk)
- jensjakob (@jensjakob)
- Jenya Y (@jenyayel)
- Jeppe Ladefoged (@jladefoged)
- Jerad Meisner (@jeradM)
- Jeremiah Wuenschel (@jer)
- Jeremy (@Wutname1)



- Jeremy (@jeremyowens)
- Jeremy Bunting (@qbunt)
- Jeremy Cook (@jeremycook61)
- Jeremy Gray (@grayjeremy)
- Jeremy Hettenhouser (@borland502)
- Jeremy Kerr (@jk-ozlabs)
- Jeremy Klein (@jeremydk)
- Jeremy Roe (@jeremyroe)
- Jeremy Schlatter (@jeremyschlatter)
- Jeremy Volkman (@jvolkman)
- Jeremy Williams (@jwillaz)
- jeremysv (@jeremysv)
- jeremywillans (@jeremywillans)
- Jerin Jacob (@jerinjacobk)
- Jerod Santo (@jerodsanto)
- Jeroen Laverman (@LavermanJJ)
- Jeroen Peters (@Heronimonimo)
- Jeroen Seegers (@jeroenseegers)
- Jeroen ter Heerdt (@jeroenterheerdt)
- Jeroen Van den Keybus (@vdkeybus)
- JeroenTuinstra (@JeroenTuinstra)
- Jerold Albertson (@jeroldalbertson-wf)
- JeromeHXP (@JeromeHXP)
- Jerrod Lankford (@jerrod-lankford)
- Jerry Chong (@zanglang)
- Jerry Workman (@JerryWorkman)
- Jerônimo Lopes (@lopesjeronimo)
- Jess (@castaway)
- Jesse Campbell (@jcam)
- Jesse Hills (@jesserockz)
- Jesse Newland (@jnewland)
- Jesse O'Connor (@jessexoc)
- Jesse Rizzo (@jesserizzo)
- Jesse Ruiters (@jesseruiters)
- Jesse van Leth (@jessevl)
- JessePrater (@JessePrater)
- JesseWebDotCom (@JesseWebDotCom)
- jessyjones (@jessyjones)
- Jevgeni Kiski (@yozik04)
- Jewel Andraia Darger-Sacher (@jewel-andraia)

- jey burrows (@jeyrb)
- jezcooke (@jezcooke)
- Jezza (@Jezza34000)
- jfearon (@jfearon)
- jfette (@jfette)
- jgriff2 (@jgriff2)
- jgrob1 (@jgrob1)
- jh (@realjax)
- jheling (@jheling)
- jhemzal (@jhemzal)
- Jianhui Zhao (@zhaojh329)
- Jim Brennan (@jbrennan-impinj)
- Jim Ekman (@blejdfist)
- Jim Rollenhagen (@jimrollenhagen)
- Jim Shank (@jshank)
- jim-granger (@jim-granger)
- Jimbo.Automates (@jimboca)
- Jimmy Everling (@sockless-coding)
- Jimmy Tang (@jcftang)
- Jin (@saury)
- jingsno (@jingsno)
- Jiri Cincura (👤) (@cincuranet)
- Jiri Novotny (@jiri-novotny)
- Jiri Urbasek (@ibru)
- JJdeVries (@JJdeVries)
- jkuettner (@jkuettner)
- jlijssen (@jlijssen)
- jlmaroto (@jlmaroto)
- jlrgraham (@jlrgraham)
- jlvaillant (@jlvaillant)
- jma89 (@jma89)
- jmtatsch (@jmtatsch)
- jmvermeulen (@jmvermeulen)
- Jo De Boeck (@grimpy)
- Jo Liss (@joliss)
- Jo Verbeeck (@joverbee)
- jo89lin (@invalidarg)
- Joachim Wiberg (@troglobit)
- Joakim af Sandeberg (@jotunacorn)
- Joakim Lindbom (@JoakimLindbom)

- Joakim Olsson (@argoyle)
- Joakim Plate (@elupus)
- Joakim Syk (@jockesyk)
- Joakim Sørensen (@ludeeus)
- joakim-tjernlund (@joakim-tjernlund)
- Joao Carreira (@jfmcarreira)
- Joao Pedro Pinto (@joaopmpinto)
- Joaquín (@joa-queen)
- Job van Koeveringe (@jobvk)
- jobhh (@jobhh)
- Jochen Baltes (@jbaltes)
- Jochen Martin Eppler (@jougs)
- Jochen Mehlhorn (@jo-me)
- Jochen Schalanda (@joschi)
- jodur (@jodur)
- Joe Francis (@lostapathy)
- Joe Garcia CISSP (@infamousjoeg)
- Joe Gross (@joegross)
- Joe Lee (@xnoodle)
- Joe Lin (@xlcwu)
- Joe Lu (@snjoetw)
- Joe McMonagle (@joemcmonagle)
- Joe Rocklin (@joerocklin)
- Joe Rouvier (@jr Rouvier)
- Joe Trabulsy (@webdjoe)
- joe248 (@joe248)
- Joeboyc2 (@Joeboyc2)
- Joel (@JoelKle)
- Joel Asher Friedman (@joelash)
- Joel Brun (@jobr97)
- Joel Carlson (@JoelsonCarl)
- Joel Clermont (@joelclermont)
- Joel Fernandes (@jrfernandes)
- Joel Fischer (@joeljfisher)
- Joel Midstjärna (@midstar)
- Joel Spiers (@joelspiers15)
- Joel Stanley (@shenki)
- Joeri (@yurnih)
- Joeri Colman (@depuits)
- joggs (@joggs)

- Johan Bloemberg (@aequitas)
- Johan Carlquist (@theseal)
- Johan Derycke (@jhdr-barco)
- Johan Eliasson (@jeliasson)
- Johan Elmerfjord (@JohanElmis)
- Johan Haals (@jhaals)
- Johan Hammar (@johanhammar)
- Johan Henkens (@jhenkens)
- Johan Isacsson (@CJNE)
- Johan Josua Storm (@Melantrix)
- Johan Klintberg (@moogblob)
- Johan Lindell (@lindell)
- Johan Lindström (@JohanLeirnes)
- Johan Nenzén (@JohNan)
- Johan Oudinet (@joudinet)
- Johan Smits (@johansmitsnl)
- Johan Ström (@stromnet)
- Johan Svensson (@jsvensson)
- Johan Thelin (@e8johan)
- Johan van der Kuijl (@johanvanderkuijl)
- Johan von Forstner (@johan12345)
- johanf85 (@johanf85)
- Johann Bauer (@bauerj)
- Johann Kellerman (@kellerza)
- Johann Vanackere (@vanackej)
- Johannes Dahlgren (@jdahlgren)
- Johannes Innerbichler (@jinnerbichler)
- Johannes Jonker (@JohJonker)
- Johannes K (@roadrash2108)
- Johannes la Poutre (@squio)
- Johannes Schmitz (@johschmitz)
- Johannes Truschnigg (@jtru)
- John (@j0hnby)
- John (@J-C-B)
- John (@CircuitSetup)
- John (@john30)
- John Allen (@jra3)
- John Arild Berentsen (@turbokongen)
- John Boiles (@johnboiles)
- John Coggeshall (@coogle)

- John Dyer (@johntdyer)
- John Evans (@GrandadEvans)
- John Faith (@jfaith-impinj)
- John Hollowell (@jhollowe)
- John Howard (@howardjohn)
- John James Jacoby (@JJJ)
- John K. Luebs (@jkl1337)
- John Keeping (@johnkeeping)
- John L (@jplafonta)
- John Levermore (@zhibek)
- John Luetke (@johnluetke)
- John Males (@johnmales)
- John McLaughlin (@loghound)
- John O'Nolan (@JohnONolan)
- John Parchem (@jparchem)
- John Pavlecich (@jmpavlec)
- John Raahauge (@AZDane)
- John Shahawy (@JohnShahawy)
- John Stile (@johstile)
- John W. Long (@jlong)
- John Warne (@johnwarne)
- John Williams (@Jaidan)
- JohnClay (@JohnClay)
- Johnny (@Johnny-Malizia)
- Johnny Bretz (@FindingJohnny)
- Johnny Chadda (@joch)
- Johnny Moore (@JXGA)
- Johnny Walker (@jwalker343)
- Johnny Willemsen (@jwillemsen)
- JohnnyCAPSLOCK (@JohnnyCAPSLOCK)
- johnnychicago (@johnnychicago)
- johnnyletrois (@johnnyletrois)
- Jon (@JonMurphy)
- Jon (@VdkaShaker)
- Jon (@jon-riches)
- Jon Banafato (@jonafato)
- Jon Benson (@hastarin)
- Jon Bloom (@jonbloom)
- Jon Calderín Goñi (@jcalderin)
- Jon Caruana (@joncar)

- Jon Evans (@evansj)
- Jon Evans (@craftyjon)
- Jon Gerdes (@gerdesj)
- Jon Gilmore (@JonGilmore)
- Jon Gilmore (@jon102034050)
- Jon Griffith (@arretx)
- Jon Kristian (@jonkristian)
- Jon Maddox (@maddox)
- Jon Shier (@jshier)
- Jon Travis (@trav)
- Jonas (@jonasem)
- Jonas (@JonasJoKuJonas)
- Jonas (@k4ds3)
- Jonas Brauer (@jonas-brauer)
- Jonas Bröms (@jonasbroms)
- Jonas Diemer (@jonasdiemer)
- Jonas Janz (@PixelJonas)
- Jonas Karlsson (@jonasbkarlsson)
- Jonas Kohlbrenner (@cepresso)
- Jonas Lundberg (@lundberg)
- Jonas Pedersen (@JonasPed)
- Jonas Skoogh (@hAmpzter)
- Jonas Thuresson (@jthure)
- JonasClever (@JonasClever)
- jonasgustavsson (@jonasgustavsson)
- jonasrickert (@jonasrickert)
- Jonatan Castro (@jcastro)
- Jonathan (@jlauwers)
- Jonathan (@jmw6773)
- Jonathan Adams (@jonathanadams)
- Jonathan Batchelor (@jmb)
- Jonathan Ben-Avraham (@benavrh)
- Jonathan DEKHTIAR (@DEKHTIARJonathan)
- Jonathan Haddock (@joncojonathan)
- Jonathan Herlin (@Jonher937)
- Jonathan Jefferies (@jjok)
- Jonathan Keljo (@jkeljo)
- Jonathan Keslin (@decompil3d)
- Jonathan Laliberte (@JonLaliberte)
- Jonathan Liu (@net147)

- Jonathan Martens (@jmartens)
- Jonathan McDowell (@u1f35c)
- Jonathan Tran (@jonathanmtran)
- Jonathan Weinberg (@jonathanweinberg)
- Jonathan Wukitsch (@insleep)
- Jonathan Østrup (@TechnicallyJoe)
- jonne013 (@jonne013)
- Jonny Bergdahl (@bergdahl)
- JonnyaiR (@jonnyair)
- jonudewux (@jonudewux)
- jonwaland (@jonwaland)
- JoomlaStats (@JoomlaStats)
- joopster (@joopert)
- Joost Boomkamp (@increddibelly)
- Joost D (@jmjdamen)
- Joost Lekkerkerker (@joostlek)
- Jordan Hotmann (@jhotmann)
- Jordan Keith (@zeddD1abl0)
- Jordan Kueh (@jkueh)
- Jordan Speicher (@uSpike)
- Jordan Yelloz (@jyelloz)
- jordaosoft (@jordaosoft)
- Jordi (@hokus15)
- jordibsk10 (@jordibsk10)
- Jordy (@jbarrancos)
- Jorge Cruz-Lambert (@jscruz)
- Jorge Martínez López (@jorgeml)
- Jorijn Schrijvershof (@Jorijn)
- Jorim Tielemans (@tjorim)
- Joris Offouga (@jorisoffouga)
- Joris Pelgröm (@jpelgrom)
- Joris Roovers (@jorisroovers)
- joris1989 (@joris1989)
- jorisc90 (@jorisc90)
- Josa Gesell (@josa42)
- Jose Juan Montes (@jjmontesl)
- Jose Motta Lopes (@josemotta)
- Jose Ramirez (@jramirez857)
- Josef Schlehofer (@BKPepe)
- Josep del Rio (@joseprio)

- Joseph Albert (@jcalbert)
- Joseph Amalfitano (@CanDoAnything)
- Joseph Carter (@knghtbrd)
- Joseph Hassell (@poster983)
- Joseph Kogut (@jakogut)
- Joseph Madamba (@Joehuu)
- Joseph Perera (@jpere039)
- Joseph Piron (@wookiesh)
- josephnad (@josephnad)
- Josh (@Joshfindit)
- Josh (@karlw00t)
- Josh (@space-pope)
- Josh Anderson (@andersonshatch)
- Josh Asplund (@joshuata)
- Josh Bendavid (@bendavid)
- Josh Laseter (@JoshLaseter)
- Josh McCarty (@joshmcrtty)
- Josh Moore (@tycrek)
- Josh Nichols (@technicalpickles)
- Josh Ross (@spinydelta)
- Josh Sharpe (@josh-m-sharpe)
- Josh Shoemaker (@shoejosh)
- Josh Soref (@jsoref)
- Josh Willox (@jcwillox)
- Josh Wright (@JshWright)
- Josh Wyse (@jwyse)
- Josh.Wu (@JoshWu)
- Joshi (@Joshi425)
- josh85 (@josh85)
- Joshua (@JoshuaGarrison27)
- Joshua Henderson (@joshua-henderson)
- Joshua Henderson (@digitalpeer)
- Joshua Jack (@joshuaja)
- Joshua M. Boniface (@joshuaboniface)
- Joshua Mulliken (@JoshuaMulliken)
- Joshua Roys (@roysjosh)
- joshua stein (@jcs)
- Josias Montag (@josiasmontag)
- Jossie (@jossie67)
- José A. Jiménez (@jcampoy)



- José Luis Salvador Rufo (@jlsalvador)
- joth76 (@joth76)
- jowieweb (@jowieweb)
- João Gabriel (@joogps)
- João Henriques (@jfhenriques)
- João Pedro Hickmann (@joaophi)
- jpcomtois (@jpcomtois)
- jpcornil-git (@jpcornil-git)
- jpearl (@jpearl)
- Jpsy (@Jpsy)
- JQWeb (@JQWeb)
- jrel17 (@jrel17)
- jrester (@jrester)
- JRK (@jr-k)
- jroble98 (@jroble98)
- jschwalbe (@jschwalbe)
- jshore1296 (@jshore1296)
- JSprengard (@JSprengard)
- jtaseff (@jtaseff)
- jtbnz (@jtbnz)
- JTimNolan (@JTimNolan)
- jtitley (@jtitley)
- JTruj110923 (@JTruj110923)
- jtscott (@jtscott)
- ju (@delphiki)
- Juan (@JuanMTech)
- Juan Carlos Guzmán Islas (@JuanCarlosGI)
- Juan Carlos Mendez-Garcia (@jcmendez)
- Juan Jesús García de Soria (@skandalfo)
- Juan Manuel Rey (@bulju)
- Juan Martin (@tinchox5)
- Juan Martín (@nauj27)
- Juan María Laó Ramos (@juanlao)
- Juan Pablo de Castro (@juacas)
- JudgeDredd (@JudgeDreddKLC)
- Juggels (@bjw-s)
- juggie (@juggie)
- jugla (@jugla)
- Jugurtha BELKALEM (@jugurthab)
- JuGuSm (@JuGuSm)

- Juha Niemi (@juhaniemi)
- Juha Rantanen (@RanzQ)
- juho20 (@juho20)
- juicejuice (@juicejuice)
- Jules Maselbas (@jmaselbas)
- julesverhaeren (@julesverhaeren)
- Julian (@thejacko12354)
- Julian Einwag (@jeinwag)
- Julian Engelhardt (@oxygen0211)
- Julian Groß (@JulianGro)
- Julian Kaffke (@jaykay)
- Julian Kahnert (@JulianKahnert)
- Julian Knauer (@jappikay)
- Julian Lunz (@jlunz)
- Julian Löhr (@jloehr)
- Julian Scheel (@julianscheel)
- Julian Schiavo (@julianschiavo)
- Julian Waller (@Julusian)
- Julien (@weimdall)
- Julien (@julesxxl)
- Julien "\_FrnchFrgg\_" Rivaud (@FrnchFrgg)
- Julien Boibessot (@artemysfr)
- Julien Brochet (@aerials)
- Julien Danjou (@jd)
- Julien Debaru (@Debaru)
- Julien Ehrhart (@Mincka)
- Julien Floret (@jfloret)
- Julien Grossholtz (@JGrossholtz)
- Julien Olivain (@jolivain)
- Julien Roy (@royto)
- Julien Tant (@JulienTant)
- Julien Viard de Galbert (@JulienVdG)
- Julien Wajsberg (@julienw)
- Julius Kriukas (@fln)
- Julius Mittenzwei (@Julius2342)
- Julius Vitkauskas (@jvitkauskas)
- JumpmanJunior (@JumpmanJunior)
- Junian Triajianto (@junian)
- junktrunk202 (@junktrunk202)
- Jupred (@Jupred)

- [jurafxp \(@jurafxp\)](#)
- [Juraj Liso \(@LiJu09\)](#)
- [Juraj Nyíri \(@JurajNyiri\)](#)
- [Juri Calleri \(@Nihvel\)](#)
- [Jurriaan Puijs \(@jurriaan\)](#)
- [Jussi Vattjus-Anttila \(@jupe\)](#)
- [Justin \(@ItsAGeekThing\)](#)
- [Justin Bassett \(@JBassett\)](#)
- [Justin Bassett-Green \(@jtbgr\)](#)
- [Justin Berstler \(@jberstler\)](#)
- [Justin Dray \(@justin8\)](#)
- [Justin Edelson \(@justinedelson\)](#)
- [Justin Goette \(@jcggoette\)](#)
- [Justin Good \(@justingood\)](#)
- [Justin Hammond \(@Fishwaldo\)](#)
- [Justin Hayes \(@GussyH\)](#)
- [Justin Huff \(@jjhuff\)](#)
- [Justin Loutsenhizer \(@jloutsenhizer\)](#)
- [Justin Maggard \(@jmaggard10\)](#)
- [Justin Otherguy \(@justinotherguy\)](#)
- [Justin Paupore \(@blueshiftlabs\)](#)
- [Justin Scott \(@JustinScott\)](#)
- [Justin Sherman \(@jsheerman256\)](#)
- [Justin Vallelonga \(@jlvallelonga\)](#)
- [Justin Vanderhooft \(@vanstinator\)](#)
- [Justin Weberg \(@justweb1\)](#)
- [JustinDwy \(@JustinDwy\)](#)
- [Justyn Shull \(@justyns\)](#)
- [jvannoyx4 \(@jvannoyx4\)](#)
- [jvimont \(@jvimont\)](#)
- [JW \(@jw-00000\)](#)
- [jwater7 \(@jwater7\)](#)
- [jwildman16 \(@jwildman16\)](#)
- [jwin32 \(@jwin32\)](#)
- [jxwolstenholme \(@jxwolstenholme\)](#)
- [jyrki69 \(@jyrki69\)](#)
- [jyz0501 \(@jyz0501\)](#)
- [J       Klein \(@grm\)](#)
- [J       Lemaire \(@jeremy-lemaire\)](#)
- [J       Rosen \(@boucman\)](#)

- Jérôme Oufella (@joufellafl)
- Jérôme Pouiller (@jerome-pouiller)
- Jérôme W (@RomRider)
- József Kertész (@zentale)
- Jörg Krause (@joerg-krause)
- Jörg Thalheim (@Mic92)
- Jørgen Rørvik (@jorgror)
- Jürgen Haas (@jurgenhaas)
- Jürgen W (@jurgenweber)
- k2v1n58 (@k2v1n58)
- k3mpaxl (@k3mpaxl)
- Ka (@KaSt)
- kaareseras (@kaareseras)
- Kacper Krupa (@pagenoare)
- Kaelig Deloumeau-Prigent (@kaelig)
- Kai (@luxus)
- Kai Bepperling (@Lyr3x)
- Kai Willadsen (@kaiw)
- KaiboshOz (@KaiboshOz)
- kaioomatico (@kaioomatico)
- Kal42 (@Kal42)
- Kalen Johnson (@kalenjohnson)
- Kalle Møller (@KalleDK)
- Kallum Burgin (@Kallb123)
- KalooxD (@KalooxD)
- Kamahat (@kamahat)
- KAMAL AWASTHI (@KamalAwasthi)
- Kame (@tobiasgraf)
- Kamel Bouhara (@kamel-bouhara)
- Kamil (@luntain)
- Kamil Doległo (@kamildoleglo)
- Kamil Warguła (@quamilek)
- Kanga-Who (@Kanga-Who)
- Kanishka Samarasinghe (@Kanishkaz)
- Kapernicus (@Kapernicus)
- Kaptensanders (@Kaptensanders)
- Kareem Straker (@KptnKMan)
- Kareem Sultan (@kareem613)
- Karen Goode (@kfgoode)
- Karim Geiger (@KarimGeiger)

- Karim Roukoz (@kkr16)
- Karl (@KarlKI)
- Karl Kihlström (@kalkih)
- Karl Möller (@FreshlyBrewedCode)
- karliemeads (@karliemeads)
- karlkar (@karlkar)
- Karol Babioch (@kbabioch)
- Karol Będkowski (@carlo497)
- Karol Stosik (@KarolStosik)
- Karsten Nerdinger (@Piratonym)
- Karthik T (@ktaragorn)
- Karthikeyan Singaravelan (@tirkarthi)
- Kasper Kirkegaard (@KasperLK)
- Kasper Malfroid (@malfroid)
- kaustubhphatak (@kaustubhphatak)
- kayge (@kayge)
- kcd83 (@kcd83)
- KD4SIR (@scotthibbs)
- Kdemontf (@Kdemontf)
- kdvlr (@kdvlr)
- Keaton Taylor (@keatontaylor)
- keeperofdakeys (@keeperofdakeys)
- Kees Schollaart (@keesschollaart81)
- keesak (@cdkonecny)
- Keilin Bickar (@kbickar)
- Keiran S (@keirans)
- Keith (@seedzero)
- keith (@afex)
- Keith Bentrup (@keithbentrup)
- Keith Lamprecht (@Nixon506E)
- Keith Mok (@ek9852)
- Kelly (@SpudGunMan)
- Kelly Atkinson (@kellyatkinson)
- Kelly Burke (@klyburke)
- Kelvin Cheung (@kelvincheung)
- Ken Bannister (@kb2ma)
- Ken Carpenter (@mindjuice)
- Ken Davidson (@kwdavidson)
- Ken Guest (@kenguest)
- Kendell R (@KTibow)

- kennedyshead (@kennedyshead)
- Kenneth Henderick (@khenderick)
- Kenneth J. Miller (@KloudJack)
- Kenneth Lavrsen (@KennethLavrsen)
- Kenny Millington (@kmdm)
- Kent Fenwick (@kent)
- Kent Hua (@kenthua)
- kentloving (@kentloving)
- kepath (@kepath)
- kernehed (@kernehed)
- kerta1n (@kerta1n)
- Kerwood (@Kerwood)
- keстераernoudt (@keстераernoudt)
- Ketil Moland Olsen (@ketilmo)
- kevank (@kevank)
- kevdliu (@kevdliu)
- Kevin (@roofuskit)
- Kevin (@Mister-Espria)
- Kevin Anthony (@kdanthony)
- Kevin Cathcart (@KevinCathcart)
- Kevin Cernekee (@cernekee)
- Kevin Christensen (@nivekmai)
- Kevin Delaney (@kevindelaney)
- Kevin Duong (@kevinduong)
- Kevin Fowlks (@kfowlks)
- Kevin Fronczak (@fronzbot)
- Kevin Goff (@kevindgoff)
- Kevin Gottsman (@gottsman)
- Kevin Haffner (@khaffner)
- Kevin Hellemun (@OGKevin)
- Kevin Händel (@KevinHaendel)
- Kevin Köck (@kevinkk525)
- Kevin Lee (@kevineriklee)
- Kevin McCormack (@HarlemSquirrel)
- Kevin Raddatz (@raddatzk)
- Kevin Schlosser (@kdschlosser)
- Kevin Siml (@appzer)
- Kevin Stillhammer (@eifinger)
- Kevin T. Berstene (@KBerstene)
- Kevin Tawaststjerna (@ktawaststjerna)

- Kevin Tuhumury (@kevintuhumury)
- Kevin Van den Abeele (@beelee)
- Kevin W. Gisi (@gisikw)
- Kevin Worrel (@dieselrabbit)
- kevinpanaro (@kevinpanaro)
- kevjs1982 (@kevjs1982)
- Kevyn Bruyere (@kevynb)
- Keyasha Brothorn (@KMBrothorn)
- kezziny (@kezziny)
- kfcook (@kfcook)
- kg333 (@kg333)
- kgalilio (@kgalilio)
- Khaled Attia (@khal3d)
- Khalid (@GotoCode)
- Khem Raj (@kraj)
- Khole (@KJonline)
- Kiall Mac Innes (@kiall)
- Kieran Bingham (@kbingham)
- kifort (@kifort)
- Kilian Lackhove (@crabmanX)
- Kim De Mey (@kdeme)
- Kim Frellsen (@kimfrellsen)
- kimvonmullen (@kimvonmullen)
- KingJ (@KingJ)
- kingy444 (@kingy444)
- Kip (@kipwittchen)
- kireyeu (@kireyeu)
- Kiril Peyanski (@peyanski)
- kirilldem (@kirilldem)
- Kit Klein (@kit-klein)
- kitcorey (@kitcorey)
- Kittl L (@Kiaaz)
- Kjetil (@kjetilmjos)
- kkasza (@kkasza)
- Klaas Hoekema (@KlaasH)
- Klaas Neirinck (@kneirinck)
- Klaas Schoute (@klaasnicolaas)
- Klaudiusz Staniek (@kstaniek)
- Klaus Aengenvoort (@klacol)
- Klaus Schwarzkopf (@Klaus-schwarzkopf)

- kllngtme (@kllngtme)
- kloggy (@kloggy)
- Knapoc (@Knapoc)
- Knodd (@Knodd)
- knottyau75 (@knottyau75)
- knwpsk (@knwpsk)
- KNXBroker (@KNXBroker)
- Kocsen Chung (@kocsenc)
- kodsnutten (@kodsnutten)
- Koen Beckers (@K-4U)
- Koen Ekelschot (@koenekelschot)
- Koen Hendriks (@koenhendriks)
- Koen Kanters (@Koenkk)
- Koen Martens (@sonologic)
- Koen van Zuijlen (@kvanzuijlen)
- koen01 (@koen01)
- Kole (@kktrum)
- Kolia56 (@Kolia56)
- Kolja Windeler (@KoljaWindeler)
- kongo09 (@kongo09)
- Konrad Ponichtera (@kponichtera)
- Konrad Schwarz (@konrad-schwarz)
- Konstantin (@konstantib)
- Konstantin Antselovich (@kantselovich)
- Konstantin Belyalov (@belyalov)
- Konsts (@Konsts)
- koolsb (@koolsb)
- koomik (@koomik)
- koreth (@koreth)
- Korn (@k-korn)
- Kory Maincent (@kmaincent)
- Kory Prince (@korylprince)
- Kosh42 (@Kosh42)
- Kostas Chatzikokolakis (@chatziko)
- kotborealis (@kotborealis)
- kounelii (@kounelii)
- kozerskil (@kozerskil)
- kozoke (@kozoke)
- kpine (@kpine)
- kplachecki (@kplachecki)



- kpoppel (@kpoppel)
- krahabb (@krahabb)
- Krasimir Chariyski (@Chariyski)
- Krasimir Zhelev (@zhelev)
- krazos (@krazos)
- kreegahbundolo (@kreegahbundolo)
- kreene1987 (@kreene1987)
- Kris Bennett (@i00)
- Kris Molendyke (@krismolendyke)
- Kris Noble (@krisnoble)
- Kris Skarbo (@Skarbo)
- Krisjanis Lejejs (@klejejs)
- Krisjanis Rijnieks (@kr15h)
- Kristian (@krisnyse)
- Kristian Heljas (@kristianheljas)
- KristjanLaane (@KristjanLaane)
- Kristján Bjarni (@kristjanbjarni)
- Kristof Krenn (@KrennKristof)
- Kriszta Matyi (@matyikriszta)
- krygal (@krygal)
- Kryštof Korb (@krystof-k)
- Krzysztof Karol (@KrzysztofKarol)
- Krzysztof Kotlenga (@pocek)
- ktnrg45 (@ktnrg45)
- ktownsend-personal (@ktownsend-personal)
- Kuba Orasiński (@korasinski)
- Kuba Wolanin (@kubawolanin)
- Kumar Gala (@galak)
- Kumar Gaurav Pandey (@gaurav1911)
- kunago (@kunago)
- kurniawan77 (@kurniawan77)
- Kurt McKee (@kurtmckee)
- Kurt Van Dijck (@kurt-vd)
- Kuzj (@Kuzj)
- kvanhoorn (@kvanhoorn)
- kyak (@kyak)
- kyangk (@kyangk)
- Kyle Gill (@gillkyle)
- Kyle Gordon (@kylegordon)
- Kyle Hasegawa (@kylehase)

- Kyle Hendricks (@kylehendricks)
- Kyle Hildebrandt (@kylehild)
- Kyle Hoermann (@kdhoermann)
- Kyle McNally (@Sparticuz)
- Kyle Niewiada (@aav7fl)
- Kyle Pinette (@sowelie)
- Kyle R (@kylmp)
- Kyle Rector (@KyleARector)
- kyclerw (@kyclerw)
- L (@lance36)
- l3nticular (@l3nticular)
- L-I-Am (@L-I-Am)
- LAB02 Administrator (@LAB02-Admin)
- labrunning (@labrunning)
- Lado Kumsiashvili (@herrlado)
- Lahoudere Fabien (@aragua)
- Laine Walker-Avina (@lwalkera)
- lambtho (@lambtho12)
- lamiskin (@lamiskin)
- Lance Haynie (@lancehaynie)
- Lance McCarthy (@LanceMcCarthy)
- Lance Moss (@mossyhub)
- lancer73 (@lancer73)
- Landrash (@Landrash)
- lapy (@lapy)
- Laqoore (@Laqoore)
- larena1 (@larena1)
- Larry (@larry-wong)
- Larry Gregory (@legrego)
- Lars (@flabbamann)
- Lars (@LarsMichelsen)
- Lars Alexander Blumberg (@larsblumberg)
- Lars Hagström (@DonOregano)
- Lars Kerick (@Brice187)
- Lars Lydersen (@larsvinc)
- Lars Marowsky-Brée (@l-mb)
- Lars-P (@Lars-P)
- Larsen Vallecillo (@larsenv)
- Lasath Fernando (@shocklateboy92)
- Lasse Bang Mikkelsen (@lassebm)

- Lasse Korpela (@bobotus)
- Lasse Rosenow (@LasseRosenow)
- Laszlo Jakab (@laszlojakab)
- Laszlo Magyar (@Imagyar)
- Lau1406 (@Lau1406)
- LaurensBot (@LaurensBot)
- Laurent Bouriez (@lbouriez)
- Laurent Charpentier (@laurentchar)
- laurent laffont (@lolgzs)
- LaurentTrk (@LaurentTrk)
- LavaGlass (@LavaGlass)
- Lawrence (@mindmelting)
- lawtancool (@lawtancool)
- LayerCake (@LayerCakeMakes)
- Laz (@lazdavila)
- Lazar Obradovic (@lobradov)
- lbdpy (@lbdpy)
- lcersly (@lcersly)
- lclc98 (@lclc98)
- ldvc86 (@ldvc86)
- LE LAY Olivier (@liollury)
- Leach Daniel J (@djleach-belcan)
- leaen (@mossbanay)
- Leah Oswald (@leahoswald)
- Leandro (@leofuscaldi)
- Leandro Loureiro (@lealoureiro)
- Lechu (@ShadeByLesio)
- Lecopzer Chen (@lecopzer)
- led-spb (@led-spb)
- Ledernacken6 (@Ledernacken6)
- Lee Jones (@lag-linaro)
- lee-js (@lee-js)
- legacycode (@legacycode)
- Leif (@leifclaesson)
- LeighCurran (@LeighCurran)
- lekobob (@lekobob)
- lenardteri (@lenardteri)
- lendy007 (@lendy007)
- Lennard Scheibel (@lscheibel)
- Lennart (@lennart-k)

- Lennart Henke (@L-Henke)
- LeoCal (@LeoCal)
- LeoDJ (@LeoDJ)
- Leon (@leonxi)
- Leon Anavi (@leon-anavi)
- Leon Knauer (@reuank)
- Leon Richard (@leonDataNut)
- Leon99 (@Leon99)
- Leonard (@leonardlcl)
- Leonardo Bellettini (@leobel96)
- Leonardo Brondani Schenkel (@lbschenkel)
- Leonardo Figueiro (@leofig-rj)
- Leonardo Merza (@ljmerza)
- Leonardo Saraiva (@vyper)
- LeonCB (@LeonCB)
- Leonid Yuriev (@erthink)
- Leonidas Loucas (@merc1031)
- leopal77 (@leopal77)
- Leothlon (@leothlon)
- leoujz (@leoujz)
- leranp (@leranp)
- Lerosen (@Lerosen)
- Leroy Shirto (@leroyshirto)
- leroyboerefijn (@leroyboerefijn)
- leroyloren (@leroyloren)
- leschekfm (@leschekfm)
- Lester Lo (@lesterlo)
- Leszek Swirski (@LeszekSwirski)
- Lev Aronsky (@aronsky)
- Levi Govaerts (@legovaer)
- lewei50 (@lewei50)
- Lewis Juggins (@lwis)
- Lewis Roberts (@lwsrbrts)
- Lewys Martin (@CountParadox)
- leytpapas (@leytpapas)
- Ifelten (@Ifelten)
- LFPoulain (@lfpoulain)
- lh-128 (@lh-128)
- lhanneus (@lhanneus)
- LHarta (@LHarta)

- Li-Huai (Allan) Lin (@qqaatw)
- Li-Wen Yip (@liwenyip)
- liaanvdm (@liaanvdm)
- Liam (@SkiingIsFun123)
- Liang Sun (@1e0ng)
- liangleslie (@liangleslie)
- libots (@libots)
- lich (@mu4yu3)
- Lieven Hollevoet (@hollie)
- lifeisafractal (@lifeisafractal)
- lightbullet (@lightbullet)
- LightIsLife (@LightIsLife)
- likeablob (@likeablob)
- Lily Williams (@runt2pb)
- limpens (@limpens)
- Lincoln Kirchoff (@Kirchoff)
- Lindsay Ward (@lindsaymarkward)
- linebp (@linebp)
- linkztream (@linkztream)
- linlicai0110 (@linlicai0110)
- linsvensson (@linsvensson)
- lintaba (@lintaba)
- Linus H (@linus-ha)
- linvinus (@linvinus)
- Lionel Flandrin (@simias)
- Lionel Landwerlin (@djdeath)
- Lionel Orry (@chickenkiller)
- liquidbear99 (@liquidbear99)
- Liran BG (@liranbg)
- Liran Tal (@lirantal)
- Littlejon (@littlejon)
- lizaoreo (@lizaoreo)
- LJU (@LEJOUI)
- Ljzd-PRO (@Ljzd-PRO)
- Ik3de (@Ik3de)
- Ilevering (@Ilevering)
- loeffelpa (@loeffelpa)
- Loek Sangers (@LoekSangers)
- Logan Marchione (@loganmarchione)
- logan893 (@logan893)

- [loganrector1 \(@loganrector1\)](#)
- [logicallyanime \(@logicallyanime\)](#)
- [lolouk44 \(@lolouk44\)](#)
- [LonePurpleWolf \(@LonePurpleWolf\)](#)
- [longman391 \(@longman391\)](#)
- [Lopton \(@Matt-PMCT\)](#)
- [lordneon \(@lordneon\)](#)
- [Lorenz Schmid \(@lorenzschmid\)](#)
- [Lorenzo \(@lorenzoraimondi\)](#)
- [Lorenzo Brescanzin \(@br3sc4\)](#)
- [Lorenzo Catucci \(@lmctv\)](#)
- [Lorenzo Milesi \(@maxxer\)](#)
- [lorenzofattori \(@lorenzofattori\)](#)
- [LorenzoVasi \(@LorenzoVasi\)](#)
- [Loryan Strant \(@loryanstrant\)](#)
- [Louis Aussedat \(@aussedatlo\)](#)
- [Louis Laureys \(@louis-lau\)](#)
- [Louis Matthijssen \(@LouisMT\)](#)
- [Louis Orleans \(@dudeofawesome\)](#)
- [Louis-Dominique Dubeau \(@Iddubeau\)](#)
- [Louis-Etienne \(@ledor473\)](#)
- [LouisP \(@lp35\)](#)
- [Lovro Oreskovic \(@oresk\)](#)
- [Irmate \(@Irmate\)](#)
- [LRubin \(@sanscontext\)](#)
- [Itloopy \(@Itloopy\)](#)
- [Itx4jay \(@Itx4jay\)](#)
- [Luar Roji \(@cyberplant\)](#)
- [luar123 \(@luar123\)](#)
- [LuBeDa \(@lubeda\)](#)
- [Lubomir Rintel \(@lkundrak\)](#)
- [Luc Touraille \(@stilllman\)](#)
- [Luca Adrian Lindhorst \(@lal12\)](#)
- [Luca Angemi \(@luca-angemi\)](#)
- [Luca Cavalli \(@lcavalli\)](#)
- [Luca Ceresoli \(@lucaceresoli\)](#)
- [Luca De Rosa \(@lrzdeveloper\)](#)
- [Luca Mannella \(@Sarcares\)](#)
- [Luca Simonetti @Nohup \(@luca-simonetti\)](#)
- [Luca Soldi \(@LucaSoldi\)](#)

- Luca Zimmermann (@soundstorm)
- lucagiove (@lucagiove)
- Lucas De Marchi (@lucasdemarchi)
- Lucas Mindêllo de Andrade (@rokam)
- Lucas Nussbaum (@lnussbaum)
- Luciano Colosio (@unlucio)
- Lucien Guimier (@guimier)
- Ludovic (@ldvc)
- Ludovic Desroches (@ldesroches)
- Ludovico de Nittis (@RyuzakiKK)
- Ludwig Hubert (@lud-hu)
- Ludy (@Ludy87)
- Luis Alberto Pérez García (@luixal)
- Luis Araneda (@luaraneda)
- Luis Martinez de Bartolome Izquierdo (@lasote)
- luismonge1192 (@luismonge1192)
- Lukas (@lukashass)
- Lukas Bachschwell (@s00500)
- Lukas Barth (@tinloaf)
- Lukas Ecklmayr (@outrun0506)
- Lukas Hetzenecker (@lukas-hetzecker)
- Lukas Kempf (@lkempf)
- Lukas Koch (@ast0815)
- Lukas Porubcan (@Luc3as)
- Lukasz Pulka (@luke4oxnet)
- Lukasz Tekieli (@ltekieli)
- Luke (@sunshinejr)
- Luke Armstrong (@lukearmstrong)
- Luke Fritz (@lukiffer)
- Luke Karrys (@lukekarrys)
- Luke Pomfrey (@lpomfrey)
- Luke Waite (@lukewaite)
- lukie80 (@lukie80)
- Lukáš Fuchs (@panzelva)
- Lukáš Polívka (@spike411)
- lunar-consultancy (@lunar-consultancy)
- lundan (@lundan)
- lunmay (@lunmay)
- LuPa (@lupa18)
- Lupin Demid (@lupin-de-mid)

- Luuk (@luukd)
- Luuk (@Maharball1)
- Luuk Loeffen (@luuloe)
- LvivEchoes (@LvivEchoes)
- lwad (@lwad)
- LWD (@kb6673)
- lyghtnox (@lyghtnox)
- Lyle Franklin (@ljfranklin)
- lymanepp (@lymanepp)
- Lén (@renaiku)
- Lévai Tamás (@levaitamas)
- m1ckyb (@m1ckyb)
- m4dmin (@m4dmin)
- m4tek (@m4tek)
- M4v3r1cK87 (@M4v3r1cK87)
- M-J-W-H (@M-J-W-H)
- M. Frister (@mfrister)
- M. R. (@m-roberts)
- M. Schrauwen (@yoflipp)
- M.J. Wydra (Jay) (@Tediore)
- Maarten (@mvandek)
- Maarten Damen (@maartendamen)
- Maarten Groeneweg (@lazytesting)
- Maarten Lakerveld (@mlakerveld)
- Maarten ter Huurne (@mthuurne)
- Mac\_Zhou (@mac-zhou)
- Maciej Bieniek (@bieniu)
- Maciej Sokołowski (@matemaciek)
- Maciej Wasilak (@mwasilak)
- Maciej Wilczyński (@mLupine)
- MacKaarstad (@MacKaarstad)
- Macley (@Macleykun)
- macooper (@macooper)
- macxq (@macxq)
- Madd.is (@EyMaddis)
- Madelaide (@Madelaide)
- Madhan Sundaram (@madhan5000)
- Madison Bullard (@madisonbullard)
- Mads Sørensen (@mads03dk)
- Magas (@magas0)



- Maggi Trymbill (@trymbill)
- MagicalTrev89 (@MagicalTrev89)
- MagieMalone (@MagieMalone)
- Magnus Brange (@brange)
- Magnus Eldén (@elden1337)
- Magnus Hedemark (@magnus919)
- Magnus Ihse Bursie (@magicus)
- Magnus Månsson (@magma1447)
- Magnus Nordseth (@mnordseth)
- Magnus Øverli (@magnusoverli)
- MagnusKnutas (@MagnusKnutas)
- magreen24 (@magreen24)
- Mahasri Kalavala (@skalavala)
- Mahesh Subramaniya (@msubra)
- maheus (@maheus)
- Mahyar Koshkouei (@deltabeard)
- Maido Käära (@v3rm0n)
- Maikel Punie (@Cereal2nd)
- Maikel Wever (@maikelwever)
- maikukun (@maikukun)
- MajestyIV (@MajestyIV)
- majstermod (@majstermod)
- majuss (@majuss)
- MakeMeASandwich (@MakeMeASandwich)
- MaKin211 (@MaKin211)
- Makrand Gupta (@makrandgupta)
- Maksym D (@dudyn5ky1)
- makyul (@makyul)
- Mal Curtis (@snikch)
- Malachi Soord (@inverse)
- Malcolm (@malcolmc Dixon)
- Malene Trab (@MTrab)
- Malte Franken (@exxamalte)
- Mamy Razafintsialonina (@nyandrianinamamy)
- mandelflanders (@mandelflanders)
- mannte (@mannte)
- Manoj (@vmulpuru)
- Manolis Chalkiadakis (@mxalk)
- manonstreet (@manonstreet)
- Mans Matulewicz (@MansM)

- Manuel de la Rosa (@manuel-jrs)
- Manuel Durán (@mduran80)
- Manuel Díez (@manutenfruits)
- Manuel Pietschmann (@manup)
- Manuel Vögele (@manuelVo)
- Manuel Zubieta (@iautom8things)
- manzenbeck (@manzenbeck)
- Maor (@maorcc)
- MapGuy11 (@MapGuy11)
- Mara Sophie Grosch (@LittleFox94)
- Marat Levit (@mlevit)
- marawan31 (@marawan31)
- MarBra (@MarBra)
- Marc (@B5r1oJ0A9G)
- Marc (@mandrebern)
- Marc (@marc-gist)
- Marc Billow (@mbillow)
- Marc Egli (@frog32)
- Marc Goodner (@robotdad)
- Marc Hörsken (@mback2k)
- Marc Khouri (@mnkhouri)
- Marc Kolly (@makuser)
- Marc Mueller (@cdce8p)
- Marc Plano-Lesay (@Kernald)
- Marc Randolph (@mrand)
- Marc Seeger (@rb2k)
- Marc Smith (@mrmarcsmith)
- Marc Vila (@LaQuay)
- Marcel (@MTRNord)
- Marcel Brückner (@marcelbrueckner)
- Marcel de Haas (@mdehaas)
- Marcel Herd (@marcelherd)
- Marcel Holle (@multiholle)
- Marcel Hoppe (@hobbypunk90)
- Marcel Neumann (@selmaohneh)
- Marcel Patzlaff (@mcpat)
- Marcel Steinbach (@mst)
- Marcel van der Veldt (@marcelveldt)
- Marcel van Peer (@marcelvanpeer)
- Marcel030nl (@Marcel030nl)

- Marcelo Moreira de Mello (@tchellomello)
- Marcin (@merdok)
- Marcin Bis (@marcinbis)
- Marcin Ciupak (@mciupak)
- Marcin Domański (@kabturek)
- Marcin Jabrzyk (@bzyx)
- Marcin Niestroj (@mniestroj)
- Marcin Nowakowski (@MJNowakowski)
- Marcin Sędkak-Jakubowski (@fdmarcin)
- Marcio Granzotto Rodrigues (@marciogranzotto)
- MarcJenningsUK (@MarcJenningsUK)
- Marco (@marconett)
- Marco (@Marco98)
- Marco Bakera (@pintman)
- Marco Colombo (@marcomow)
- Marco Gazzola (@marcogazzola)
- Marco H (@myxor)
- Marco Kamner (@ProfessorLogout)
- Marco Kreeft (@marcokreeft87)
- Marco M (@ToRvaLDz)
- Marco Nuñez (@setterlee)
- Marco Orovecchia (@Oro)
- Marco Palazzetti (@marcopal74)
- Marco Sirabella (@mjsir911)
- Marco Sousa (@marcomsousa)
- Marco Trevisan (@3v1n0)
- marcolertora (@marcolertora)
- marconfus (@marconfus)
- MarcSN311 (@MarcSN311)
- Marcus (@marcusds)
- Marcus (@dgraye)
- Marcus Folkesson (@marcusfolkesson)
- Marcus Fredlund (@mr-awk)
- Marcus Lönnberg (@marcuslonnberg)
- Marcus R. Brown (@marcusrbrown)
- Marcus Schmidt (@mar-schmidt)
- Marcus Young (@myoung34)
- marecabo (@marecabo)
- Marek Belisko (@nandra)
- Marek Lewandowski (@mlewand)

- Marek Skrobacki (@skrobul)
- marengaz (@marengaz)
- Marianne Hval (@mahval)
- Mariano Faraco (@mfaraco)
- Marijn Giesen (@marijngiesen)
- Marijn Pool (@lcyPalm)
- Mario Di Raimondo (@mario-tux)
- Mario Fink (@mario-fink)
- Mario Lang (@mlang)
- Mario Limonciello (@superm1)
- Mario Mintel (@mintelm)
- Mario Rugiero (@Oppen)
- Mario Villavecchia (@lichtteil)
- Mario Wenzel (@maweki)
- Marius (@Mariusthvd)
- Marius (@ciotlosm)
- Marius Balčytis (@mariusbalcytis)
- Marius Flage (@mflage)
- Marius Kotlarz (@kotlarz)
- Marius Oei (@mariusoei)
- Marius Retegan (@mretegan)
- Mariusz Kryński (@mrk-its)
- Mariusz Łuciów (@mariuszluciw)
- mariwing (@mariwing)
- Mark (@mwunderling)
- Mark (@markus99)
- Mark (@marima84)
- Mark (@scmmmh)
- Mark Adkins (@funkybunch)
- Mark Allanson (@markallanson)
- Mark Bergsma (@markbergsma)
- Mark Breen (@markvader)
- Mark Corbin (@markcorbinuk)
- Mark Dietzer (@Doridian)
- Mark Grimes (@mvgrimes)
- Mark Grosen (@mgsb)
- Mark Harrison (@marksharrison)
- Mark Hirota (@sanseihappa)
- Mark Hofmann (@MarkHofmann11)
- Mark Huson (@mehuman)

- Mark Ireland (@m4rkireland)
- Mark Jozefiak (@ImEmJay)
- Mark King (@vemek)
- Mark Kovacs (@kovacsm)
- Mark Kuchel (@kuchel77)
- Mark Lagendijk (@marklagendijk)
- Mark Lambert (@dude0001)
- Mark Leenaerts (@mleenaerts)
- Mark Lopez (@Silvenga)
- Mark LUCAS (@marco10024)
- Mark Mäkinen (@Z1ni)
- Mark Nichols (@zanshin)
- Mark Oude Veldhuis (@markoudev)
- Mark Parker (@msp1974)
- Mark Perdue (@markperdue)
- Mark Petereit (@petereit)
- Mark Remijn (@markisoke)
- Mark Zachmann (@MZachmann)
- markcarline (@markcarline)
- markferry (@markferry)
- markharleman (@markharleman)
- Marko Korhonen (@FunctionalHacker)
- markterm (@markterm)
- Markus (@iMarkus)
- Markus (@mb-software)
- Markus Becker (@markushx)
- Markus Bong (@2Fake)
- Markus Breitenberger (@breiti)
- Markus Friedli (@frimtec)
- Markus Haack (@mhaack)
- Markus Heidelberg (@marcows)
- Markus Ijäs (@mtijas)
- Markus Jankowski (@SukramJ)
- Markus Kaindl (@kaindl)
- Markus Landgren (@MarkusLandgren)
- Markus Meissner (@daringer)
- Markus Nigbur (@mnigbur)
- Markus Peter (@bimbar)
- Markus Pöschl (@Poeschl)
- Markus Ressel (@markusressel)

- Markus Steinhilber (@Markus-St)
- Markus Stenberg (@fingon)
- Markus Thiel (@mackelito)
- Markus Winkler (@mawinkler)
- markuspalme (@markuspalme)
- Marley Jaffe (@marleyjaffe)
- Marouane Felja (@maroil)
- marthoc (@marthoc)
- Martidjen (@Martidjen)
- Martijn Baay (@baaym)
- Martijn de Gouw (@martijndegouw)
- Martijn Russchen (@martijnrusschen)
- Martijn van Dijk (@martijnvandijk)
- Martijn van Zal (@Martijn02)
- martijnvanduijneveldt (@martijnvanduijneveldt)
- Martin (@crazyfx1)
- Martin (@martin3000)
- Martin (@CapricornZ)
- Martin (@xtools-at)
- Martin (@mrvanes)
- Martin (@mtandersson)
- Martin Ayotte (@martinayotte)
- Martin Banas (@bnsn)
- Martin Banky (@MartinBanky)
- Martin Bark (@martinbark)
- Martin Berg (@mbrrg)
- Martin Bjeldbak Madsen (@martinbjeldbak)
- Martin Brooksbank (@shutupflanders)
- Martin Donlon (@wickerwaka)
- Martin Eberhardt (@DarkFox)
- Martin Elshuber (@melshuber)
- Martin Elwin (@melwin)
- Martin Finkenflügel (@MartinFinkenflugel)
- Martin Fuchs (@fucm)
- Martin Gafner (@mgafner)
- Martin Gross (@pc-coholic)
- Martin Hicks (@mohicks)
- Martin Hjelmare (@MartinHjelmare)
- Martin Ilievski (@mxilievski)
- Martin J. Laubach (@mjl)

- Martin Kepplinger (@merge)
- Martin Long (@martinlong1978)
- Martin Mois (@siom79)
- Martin Nöhrer (@matrixx567)
- Martin Otzen (@otzen)
- Martin Rowan (@shortbloke)
- Martin Tremblay (@MartyTremblay)
- Martin Treml (@Munsio)
- Martin Vacula (@MatoKafkac)
- Martin Weber (@martinweu)
- Martin Weinelt (@mweinelt)
- Martin Zaloudek (@ma-zal)
- martindybal (@martindybal)
- martinfrancois (@martinfrancois)
- Martino Papero (@martymarty004)
- MartinP (@mplinuxgeek)
- Martokk (@martokk)
- martst (@martst)
- Marty Cochrane (@martycochrane)
- Marty Zalega (@evilmarty)
- marutanm (@marutanm)
- Marvin Gaube (@margau)
- Marvin ROGER (@marvinroger)
- Marvin Wichmann (@marvin-w)
- Mas2112 (@Mas2112)
- Masahiro Kamata (@kamatari)
- Masahiro Yamada (@masahir0y)
- Mask3007 (@Mask3007)
- Mason Garrison (@smasongarrison)
- Mason Stewart (@mason-stewart)
- MasonCrawford (@MasonCrawford)
- Massimiliano Cannarozzo (@maxcanna)
- Massimo Callegari (@mcallegari)
- mastakebob (@mastakebob)
- Masteroshi430 (@Masteroshi430)
- masukomi (@masukomi)
- Mat Strange (@matstrange)
- mat1990dj (@mat1990dj)
- matchett808-gh (@matchett808-gh)
- Matej Drobnič (@matejdro)

- Matej Plavevski (@MatejMecka)
- Matej Sekoranja (@msekoranja)
- Mateus (@ma-te-us)
- Mateusz Drab (@mateuszdab)
- Mateusz Korniak (@matkor)
- Mateusz Soszyński (@TheLastGimbus)
- matgad (@matgad)
- Matheson Steplock (@ikifar2012)
- matheus2308 (@matheus2308)
- Mathew Peterson (@mathewpeterson)
- Mathew Verdouw (@madmat777)
- Mathias De Mare (@Mathiasdm)
- Mathieu Frappier (@matfra)
- Mathieu Maret (@mmaret)
- Mathieu Maret (@mmaret-geny)
- Mathieu Velten (@MatMaul)
- mathieu-mp (@mathieu-mp)
- matlimatli (@matlimatli)
- Matouš Bečvář (@MattXcz)
- Matrix (@matrixd2)
- MatsNI (@MatsNI)
- MATSUZAKI Tsuyoshi (@matsuza)
- Matt (@mattmattmatt)
- Matt (@mdawsonuk)
- Matt Bilodeau (@mattbilodeau)
- Matt Black (@mafrosis)
- Matt Blaha (@mattblaha)
- Matt Burke (@burkemw3)
- Matt Cahill (@matt-cahill)
- Matt Caminiti (@mcaminiti)
- Matt Colyer (@mcolyer)
- Matt D (@matt1)
- Matt Emerick-Law (@emericklaw)
- Matt Enright (@wickedshimmy)
- Matt F (@flamingm0e)
- Matt Farmer (@m14t)
- Matt Flax (@flatmax)
- Matt Galligan (@galligan)
- Matt Hall (@Neko22)
- Matt Hamilton (@Eriner)



- Matt Hamrick (@diminishedprime)
- Matt Johnson (@mattjohnsonpint)
- Matt Kasa (@mattkasa)
- Matt Krasowski (@mkrasowski)
- Matt LeBrun (@mlebrun)
- Matt N. (@mnoorenberghe)
- Matt Olson (@olsoma13)
- Matt Rayner (@matrrayner)
- Matt Robinson (@brimstone)
- Matt Rogers (@rogersmj)
- Matt Schmitt (@schmittx)
- Matt Snyder (@oblogic7)
- Matt Swan (@surlymatt)
- Matt White (@matt-FFFFFF)
- Matt White (@mw-white)
- Matt Wood (@mattwood2000)
- Matt Zimmerman (@mdz)
- matt2005 (@matt2005)
- Matte23 (@Matte23)
- MatteGary (@MatteGary)
- Matteo Agnoletto (@EPMatt)
- Matteo Gheza (@MatteoGheza)
- Matteo Lampugnani (@t30)
- Mattheus (@tioan)
- Matthew (@Gaff)
- Matthew (@Mattat01)
- Matthew Bowen (@mgbowen)
- Matthew Breedlove (@sirmalloc)
- Matthew Byington (@matthewcbyington)
- Matthew Donoughe (@mdonoughe)
- Matthew Dornquast (@dornquast)
- Matthew Garrett (@mjg59)
- Matthew Gottlieb (@matthewgottlieb)
- Matthew Grimes (@cybergrimes)
- Matthew James Kraai (@kraai)
- Matthew LeMay (@mplemay)
- Matthew Miller (@mattdm)
- Matthew Parlane (@Parlane)
- Matthew R. (@mjrouser)
- Matthew Rollings (@stealthcopter)

- Matthew Schick (@mattsch)
- Matthew Scoville (@Chaotic)
- Matthew Simpson (@typhoon2099)
- Matthew T. Kelley (@mkelley88)
- Matthew Treinish (@mtreinish)
- Matthew Turney (@pho3nixf1re)
- Matthew Wegner (@mwegner)
- Matthew Wire (@mattwire)
- matthewcky2k (@matthewcky2k)
- MatthewFlamm (@MatthewFlamm)
- Matthias Alphart (@farmio)
- Matthias de Baat (@matthiasdebaat)
- Matthias Dötsch (@brainexe)
- Matthias Grawinkel (@Meatz)
- Matthias Gutjahr (@mattsches)
- Matthias Lohr (@MatthiasLohr)
- Matthias Merz (@matt-gnu)
- Matthias Pfefferle (@pfefferle)
- Matthias Urlichs (@smurfix)
- Matthias Wahl (@mfelsche)
- Matthias Weiss (@weissm)
- Matthias Weisser (@modbw)
- Matthieu (@d9pouces)
- Matthieu (@Aohzan)
- Matthieu DUVAL (@optimatth)
- Matthieu Houdebine (@mathoudebine)
- Matthijs (@pasibun)
- Matthijs Logemann (@matthijs2704)
- Mattias Persson (@matt8707)
- Mattias Ryrlén (@mattiasr)
- Mattias Welponer (@worm-ee)
- MattiasC (@MattiasC)
- mattie47 (@mattie47)
- matlongman (@matlongman)
- mattn (@mattn)
- MattWestb (@MattWestb)
- mattwing (@mattwing)
- mattyteds (@mattyteds)
- maurerle (@maurerle)
- Maurice Kok (@mories76)

- Maurice Makaay (@mmakaay)
- Maurice van der Pot (@Griffon26)
- Mauricio Bonani (@bonanitech)
- Mauro Condarelli (@mcondarelli)
- Mauro Meneghin (@mauro-youview)
- Mauro Midolo (@mauro-midolo)
- mavrikkk (@mavrikkk)
- Max (@max-te)
- Max (@maxjoehnk)
- Max Chodorowski (@maxwroc)
- Max Filippov (@jcmvbkbc)
- Max Gashkov (@maxgashkov)
- Max Messner (@mamesner)
- Max Mudde (@makzdot)
- Max Muth (@mammuth)
- Max Prokhorov (@mcspr)
- Max Roeleveld (@doenietzomoeilijk)
- Max Rosin (@ekeih)
- Max Rumpf (@Maxr1998)
- Max Rydahl Andersen (@maxandersen)
- Max Savard (@TravelinMax)
- Max Thirouin (@MoOx)
- Max von Weibel (@343max)
- Max Winterstein (@MaxWinterstein)
- maxclaey (@maxclaey)
- MaxG88 (@MaxG88)
- Maxim Kochetkov (@fidomax)
- Maxim Krušina (@maximkrusina)
- maxim2405 (@maxim2405)
- Maxime Brunelle (@madxime)
- Maxime Chevallier (@minimaxwell)
- Maxime Coquelin (@mcoquelin-stm32)
- Maxime Hadjinlian (@maximeh)
- Maxime Petazzoni (@mpetazzoni)
- Maxime Ripard (@mripard)
- Maximilian (@DeerMaximum)
- Maximilian Bösing (@boesing)
- Maximilian Ertl (@Sirs0ri)
- Maximilian Früh (@mfrueh)
- Maximilien Cuony (@the-glu)

- maxlaverse (@maxlaverse)
- MaxMac\_STN (@maxmacstn)
- maxnet (@maxnet)
- Maël Kimmerlin (@maelk)
- mbo18 (@mbo18)
- mboeru (@mboeru)
- mburget (@mburget)
- mcc05 (@mcc05)
- mcfrojd (@mcfrojd)
- mchen6 (@mchen6)
- mckochan (@mckochan)
- mclem (@mclem)
- McNutty195 (@McNutty195)
- McYars (@McYars)
- mdallaire (@mdallaire)
- MDW (@mdeweerd)
- meatheadmike (@meatheadmike)
- medgar2048 (@medgar2048)
- Megachip (@Megachip)
- Megamannen (@Megamannen)
- MeganerdNL (@MeganerdNL)
- Mehmet Sami Tok (@samitok)
- Melchthys (@meichthys)
- meijerwynand (@meijerwynand)
- Mel Riffe (@melriffe)
- Melissa LeBlanc-Williams (@makermelissa)
- Melvin (@MBlokhuijzen)
- melyux (@melyux)
- Menco Bolt (@macmenco)
- mendhak (@mendhak)
- Menno Blom (@b10m)
- Meow (@GrumpyMeow)
- Meowcino (@Puppercino)
- Meredith (@shieldmaiden95)
- Merlin Büge (@camoz)
- mertenats (@smrtnt)
- mezz64 (@mezz64)
- mgiako (@mgiako)
- MGWGIT (@MGWGIT)
- mh-daedalus (@mh-daedalus)

- mhorst314 (@mhorst314)
- MHV33 (@vdheijden)
- Micah Neal (@noxlux)
- micarex (@micarex)
- Mich-b (@Mich-b)
- Micha LaQua (@milaq)
- micha91 (@micha91)
- Micha-Btz (@Micha-Btz)
- Michael (@mib1185)
- Michael (@hartmms)
- Michael (@mischsa)
- Michael (@splunty)
- Michael A. Alderete (@alderete)
- Michael Auchter (@auchter)
- Michael Bachmaier (@bachtron)
- Michael Ball (@cycomachhead)
- Michael Baudino (@michaelbaudino)
- Michael Bisbjerg (@LordMike)
- Michael Bravo (@mbravorus)
- Michael Buffington (@elbowdonkey)
- Michael Chang (@micbase)
- Michael Chisholm (@chishm)
- Michael Cicogna (@miccico)
- Michael Davie (@michaeldavie)
- Michael Dokolin (@dokmic)
- Michael Drake (@tlsa)
- Michael Dubno (@dubnom)
- Michael Durrant (@michaeldurrant)
- Michael Farrell (@micolous)
- Michael Fester (@michaelfester)
- Michael Friis (@friism)
- Michael Furtak (@mfurtak)
- Michael G. Schwern (@schwern)
- Michael Gorven (@mgorven)
- Michael Haas (@mhaas)
- Michael Hansen (@synesthesiam)
- Michael Hertig (@hertg)
- Michael Irigoyen (@mrigoyen)
- Michael J. Hammel (@mjhammel)
- Michael J. Kidd (@linuxkidd)

- Michael Jäger (@EmJay276)
- Michael Kane (@thisIsMikeKane)
- Michael Klamming (@m1ch)
- Michael Kowalchuk (@mkowalchuk)
- Michael Kutý (@michaelkuty)
- Michael L. (@mike-jl)
- Michael Liu (@icefalcn)
- Michael Luggen (@l00mi)
- Michael Lunzer (@michaellunzer)
- Michael Meinel (@led02)
- Michael Meli (@mjmel)
- Michael Mior (@michaelmior)
- Michael Molisani (@molisani)
- Michael Nosthoff (@heinemml)
- Michael Pereira (@MichaelPereira)
- Michael Pfister (@pfista)
- Michael Pollett (@pollett)
- Michael Prokop (@mika)
- Michael Pusterhofer (@feanor12)
- Michael R. Davis (@mrdvt92)
- Michael Requeny (@requenym)
- Michael Rommel (@michaelrommel)
- Michael Roth (@mrothNET)
- Michael Scherer (@schmic)
- Michael Schoonmaker (@Schoonology)
- Michael Schulze (@michsch)
- Michael Senn (@Lavode)
- Michael Shim (@shimeez)
- Michael Stovenour (@mstovenour)
- Michael Thingnes (@thimic)
- Michael Trimarchi (@panicking)
- Michael Varrieur (@mvarrieur)
- Michael Vetter (@jubalh)
- Michael Walle (@mwalle)
- Michael Wei (@no2chem)
- Michael Wheeler (@TheSkorm)
- Michael Writhe (@pironic)
- michael1413 (@michael1413)
- MichaelSprague (@MichaelSprague)
- Michal (@Dinth)

- Michal Bartak (@michalk-k)
- Michal Knizek (@macrosak)
- Michal Ziemski (@misialq)
- Michal Čihař (@nijel)
- Michaël Arnauts (@michaelarnauts)
- Michaël Bitard (@MichaelBitard)
- Michaël Burtin (@mburtin-netgem)
- Michał Huryn (@riokuu)
- Michał Kalbarczyk (@fazibear)
- Michał Leśniewski (@mlesniew)
- Michał Mrozek (@Michsior14)
- Michał Stańczyk (@michalstanczyk)
- Michał Węgrzynek (@mwegrzynek)
- Michał Łyszczek (@mlyszczek)
- Michel Marti (@matoxp)
- Michel Settembrino (@Michel-Settembrino)
- Michel Stempin (@Squonk42)
- Michel van de Wetering (@mvdwetering)
- Michele Porelli (@porelli)
- Michelle Fuchs (@MichelleFuchs)
- Michiel Karnebeek (@mkarnebeek)
- Mick Dekkers (@mickdekkers)
- Mick Vleeshouwer (@iMicknl)
- Mickaël Cornière (@bnounours)
- Mickaël Le Baillif (@demikl)
- Mickaël Schoentgen (@BoboTiG)
- mickeydarrenlau (@mickeydarrenlau)
- micronen (@micronen)
- micw (@micw)
- Midbin (@Midbin)
- migromao (@migromao)
- migube (@migube)
- Miguel Camba (@cibernox)
- Miguel Gil Martínez (@miguelgilmartinez)
- Miguel Sánchez Villafán (@mig8447)
- Miha Lunar (@SmilyOrg)
- Mihail Morosan (@morosanmihail)
- mihailescu2m (@mihailescu2m)
- mihalski (@mihalski)
- Mika Hiltunen (@saaste)

- Mikael Eliasson (@mikaerobomagi)
- Mikael Svensson (@Nossnevs)
- Mikayla Hutchinson (@mhutch)
- Mike (@mike391)
- Mike (@mikedm139)
- Mike (@C64ever)
- Mike (@mikenolet)
- Mike (@thegame3202)
- Mike Ballou (@ballou88)
- Mike Christianson (@MikeChristianson)
- Mike Cousins (@mikecousins)
- Mike Crowe (@mikecrowe)
- Mike Dahlgren (@mikedahlgren)
- Mike Degatano (@mdegat01)
- Mike Edmunds (@medmunds)
- Mike Eriksson (@swedishmike)
- Mike Frampton (@mikeframpo)
- Mike Frysinger (@vapier)
- Mike Fugate (@mfugate1)
- Mike G Chambers (@mikegchambers)
- Mike Harmony (@MikeHarmony)
- Mike Hennessy (@henworth)
- Mike Hershberger (@gentoosu)
- Mike Keesey (@mkeesey)
- Mike Knudson (@mtgeekman)
- Mike Kormendy (@mkormendy)
- Mike Meessen (@netmikey)
- Mike Megally (@cmsimike)
- Mike Miller (@mikeage)
- Mike Morrison (@bikemike)
- Mike Nabhan (@mikenabhan)
- Mike Nicholson (@mikenicholson)
- Mike O'Driscoll (@mikeodr)
- Mike Shaver (@shaver)
- Mike Stangel (@stangel)
- Mike Thomas (@mt0321)
- Mike Williams (@mikebwilliams)
- mikebarris (@mikebarris)
- mikebaz (@mikebaz)
- mikehole (@mikehole)



- [mikekreisher \(@mikekreisher\)](#)
- [mikemc35 \(@mikemc35\)](#)
- [Miken Valabdas \(@MValabdas\)](#)
- [MikeTsenatek \(@MikeTsenatek\)](#)
- [Mikhail Boiko \(@kurtis99\)](#)
- [Mikhail Diatchenko \(@muxa\)](#)
- [Mikhail Dronov \(@dronov\)](#)
- [Mikkel Hoegh \(@mikl\)](#)
- [Mikkel Pilehave Jensen \(@pilehave\)](#)
- [Mikko Tapionlinna \(@Arkkimaagi\)](#)
- [Miklos Bagi \(@miklosbagi\)](#)
- [Miklos Szanyi \(@swingerman\)](#)
- [Miklós \(@kmikiy\)](#)
- [Mikołaj Chwalisz \(@mchwalisz\)](#)
- [Milan Meulemans \(@milanmeu\)](#)
- [Milan V \(@milanvo\)](#)
- [Milas Bowman \(@milas\)](#)
- [millallo \(@millallo\)](#)
- [millerkyle72 \(@InTheDaylight14\)](#)
- [milothomas \(@milothomas\)](#)
- [Miloš Bunčić \(@psyhomb\)](#)
- [Milton Soares Filho \(@msoares1979\)](#)
- [Mimi \(@stevenjoezhang\)](#)
- [Mimiix \(@Mimiix\)](#)
- [MinchinWeb \(@MinchinWeb\)](#)
- [mindakas \(@mindakas\)](#)
- [Minecraftchest1 \(@minecraftchest1\)](#)
- [ming.z \(@zvving\)](#)
- [miniconfig \(@miniconfig\)](#)
- [minida28 \(@miq28\)](#)
- [Minims \(@Minims\)](#)
- [Miquel Raynal \(@miquelraynal\)](#)
- [Mirko Lenz \(@mirkolenz\)](#)
- [mirkster \(@mirkster\)](#)
- [Miroslav Ždrale \(@mzdrale\)](#)
- [miroslawkrol \(@miroslawkrol\)](#)
- [Mirza Krak \(@mirzak\)](#)
- [Mischa Gruber \(@GruberMischa\)](#)
- [Mischa Jonker \(@mischajonker\)](#)
- [Misha Komarovskiy \(@zombah\)](#)

- MishManners®™ (@mishmanners)
- Mislav Mandarić (@MislavMandarin)
- MissingDLL (@MissingDLL)
- Mister Wil (@MisterWil)
- misterbenj34 (@misterbenj34)
- misza-do (@misza-do)
- Mitch (@mitch-dc)
- Mitch Dempsey (@webdestroya)
- Mitch Schwenk (@yombo)
- Mitchell Cash (@MitchellCash)
- Mitchell Cohen (@mitchchn)
- Mitchell Currie (@mitchins)
- Mitesh Patel (@gurumitts)
- MithrasPan (@MithrasPan)
- Mitko Masarliev (@masarliev)
- miumida (@miumida)
- MizterB (@MizterB)
- mje-nz (@mje-nz)
- MJJ (@mjj4791)
- mjoshd (@mjoshd)
- Mk4242 (@Mk4242)
- mk-maddin (@mk-maddin)
- mkfink (@mkfink)
- mkmer (@mkmer)
- mlambert-zotec (@mlambert-zotec)
- mlemainque (@mlemainque)
- mletenay (@mletenay)
- mmcnama4 (@mmcnama4)
- mmillmor (@mmillmor)
- mn (@nolstedt)
- mnaggatz (@mnaggatz)
- mnestor (@mnestor)
- mnl1121 (@mnl1121)
- mnorrskén (@mnorrskén)
- Mo Lawson (@molawson)
- mobcdi (@mobcdi)
- modem158 (@modem158)
- MoellerDi (@MoellerDi)
- mogsub (@mogsub)
- Mohamad Tarbin (@meauxt)

- Mohammed Chotia (@mcchots)
- Mohana Datta Yelugoti (@ymdatta)
- mohmacht (@mohmacht)
- moinmoin-sh (@moinmoin-sh)
- mojtaker (@mojtaker)
- Moksh Mridul (@mokshmridul)
- Momotica (@Momotica)
- Monkey • D • Code (@shaonianzhentan)
- monte-monte (@monte-monte)
- mooninite (@mooninite)
- Moonshot (@moonshot)
- Moos (@moos)
- moox-it (@moox-it)
- Morgan Kesler (@keslerm)
- MORITA Hajime (@omo)
- Moritz Fey (@Mofeywalker)
- Moritz Glöckl (@moritzgloeckl)
- Moritz Sternemann (@moritzsternemann)
- moritzbeck01 (@moritzbeck01)
- Morten Lied Johansen (@mortenlj)
- Morten Lüneborg (@mopolus)
- Morten Rugaard (@rugaard)
- mortenmathiasen (@mortenmathiasen)
- mosburgerr (@mosburgerr)
- Moshe Kaplan (@moshekaplan)
- Moshe Levi (@moshe010)
- MosheTzvi (@MosheTzvi)
- moskovskiy82 (@moskovskiy82)
- mountainsandcode (@mountainsandcode)
- MPopti0n (@MPopti0n)
- mptei (@mptei)
- Mr-Groch (@Mr-Groch)
- Mr. Snyds (@mrsnyds)
- MrAliFu (@MrAliFu)
- MrBartusek (@MrBartusek)
- MrDadoo (@MrDadoo)
- mreiling (@mreiling)
- mrosseel (@mrosseel)
- mrpraline (@mrpraline)
- MrRemmers (@MrRemmers)

- MrRimmer (@MrRimmer)
- Mrten (@Mrten)
- mrtncis (@mrtncis)
- mruss (@mruss)
- mrwhite31 (@mrwhite31)
- msvinth (@msvinth)
- mtannertdev (@mtannertdev)
- mtarjoianu (@mtarjoianu)
- MtK (@kochen)
- mtl010957 (@mtl010957)
- muchtall (@muchtall)
- muerzi (@muerzi)
- mueslo (@mueslo)
- Muhammad Sheraz Lodhi (@sherazlodhi)
- Muhammed Saho (@muhasaho)
- Muhd Hakim (@Codestian)
- mukundv (@mukundv)
- mullerdavid (@mullerdavid)
- muppet3000 (@muppet3000)
- Murat Demirten (@demirten)
- mutantmonkey (@mutantmonkey)
- MvdDonk (@MvdDonk)
- mvillarejo (@mvillarejo)
- mvn23 (@mvn23)
- mvnetbiz (@mvnetbiz)
- mweimerskirch (@mweimerskirch)
- myhomeiot (@myhomeiot)
- Myles (@FormallyMyles)
- Myles Eftos (@madpilot)
- Myles K (@myleskeeffe)
- myzhang1029 (@myzhang1029)
- myztillx (@myztillx)
- Márton Maráz (@marazmarci)
- Mårten Seiplax (@seiplax)
- Miguel Ángel Mulero Martínez (@McGiverGim)
- Mārtiņš Bruņenieks (@papuass)
- n0dyjeff (@n0dyjeff)
- N1nja98 (@N1nja98)
- Nacho Barrientos (@nbarrientos)
- NachtaktiverHalbaffe (@NachtaktiverHalbaffe)

- nagubal (@nagubal)
- NanoSector (@NanoSector)
- Naomi C. Bush (@naomicbush)
- Naren Salem (@naren8642)
- narfight (@narfight)
- Narmishka (@Narmishka)
- Nash Kaminski (@nkaminski)
- nashant (@nashant)
- Nasser Afshin (@nasafix-nasser)
- Nate (@BillyNate)
- Nate (@nateww)
- Nate Clark (@heythisisnate)
- Nate Harris (@nwithan8)
- Nate Kleven (@nkleven)
- Nate Robinson (@NateRobinsonS)
- natemason (@natemason)
- Nathan (@nemccarthy)
- Nathan Broadbent (@ndbroadbent)
- Nathan Flynn (@eperdeme)
- Nathan Ford (@wpt-nathan)
- Nathan Freitas (@n8fr8)
- Nathan Henrie (@n8henrie)
- Nathan Knotts (@nknotts)
- Nathan Long (@nathanl)
- Nathan Lynch (@nathanlynch)
- Nathan Orick (@cnorick)
- Nathan Spencer (@natekspencer)
- Nathan Tilley (@ntilley905)
- Nathan W (@itsamenathan)
- nau-mll (@nau-tic)
- Naveen (@naveensrinivasan)
- Nayna Jain (@naynajain)
- ndilieto (@ndilieto)
- Neil Armstrong (@superna9999)
- Neil Crosby (@NeilCrosby)
- Neil du Toit (@NeilDuToit92)
- Neil Lathwood (@laf)
- neillt (@neillt)
- Nelis Willers (@NelisW)
- NeLLyMerC (@NeLLyMerC)

- Nelson Chen (@nelsonjchen)
- Nelson Osacky (@runningcode)
- Nemanja Stefanovic (@nemik)
- Nemec (@nemec)
- Nenad Bogojevic (@nbogojevic)
- neonbunny (@neonbunny)
- Neonicus (@neopilou)
- Nephiel (@Nephiel)
- nepozs (@nepozs)
- Nerdix (@N3rdix)
- Ness (@Xx-Ness-xX)
- Netzwerkfehler (@Netzwerkfehler)
- newbee112 (@newbee112)
- nhendin (@nhendin)
- niai (@niai)
- Niall Donegan (@ndonegan)
- Nic Eggert (@neggert)
- Nic Jansma (@nicjansma)
- Niccolo Zapponi (@nzapponi)
- Niccolò Maggioni (@nmaggioni)
- niceboygithub (@niceboygithub)
- Nicholas Amadori (@namadori)
- Nicholas Hadler (@nhadler)
- Nicholas Hawkes (@hawkesn)
- Nicholas Sielicki (@sielicki)
- Nicholas Westerhausen (@nwesterhausen)
- Nick (@nkm8)
- Nick (@GreenTentacle)
- Nick Adams (@Nick-Adams-AU)
- Nick Chamberlin (@NickChamberlin)
- Nick Daria (@nickdaria)
- Nick Dawson (@Neonkoala)
- Nick Ellering (@nellering)
- Nick Hammond (@nickhammond)
- Nick Horvath (@nhorvath)
- Nick Iacullo (@DuckyCrayfish)
- Nick Leverton (@nickleverton)
- Nick Nothom (@NickNothom)
- Nick Oliver (@PixnBits)
- Nick Peoples (@NickPeoples)

- Nick Sabinske (@quadportnick)
- Nick Touran (@partofthething)
- Nick Waring (@nickwaring)
- Nick Whyte (@nickw444)
- Nick Zelei (@nickzelei)
- nick2525 (@nick2525)
- Nick4-1 (@Nick4-1)
- nick-seward (@nick-seward)
- nickidw (@nickidw)
- NicklasMD (@NicklasMD)
- NickMa (@nickma82)
- Nicko van Someren (@nickovs)
- Nickolaos C (@NickolaosC)
- nickrout (@nickrout)
- Niclas (@nicmar)
- Nico (@N1c093)
- Nico (@NicoHood)
- Nico Domino (@ndom91)
- Nico Hirsch (@noxhirsch)
- Nico Müller (@niecore)
- Nico Vromans (@nico-vromans)
- Nico0084 (@Nico0084)
- nicogramm (@nicogramm)
- Nicolae Vlădescu (@nicolaevlădescu)
- nicolalandro (@nicolalandro)
- Nicolas Bougues (@nbougues)
- Nicolas Bourasseau (@lmbuzi)
- Nicolas Braem (@nbraem)
- Nicolas Dechesne (@ndechesne)
- Nicolas Dichtel (@NicolasDichtel)
- Nicolas Ferre (@noglitch)
- Nicolas Graziano (@ngraziano)
- Nicolas Lhomme (@nlhomme)
- Nicolas Martignoni (@martignoni)
- Nicolas Mowen (@NickM-27)
- Nicolas Quiénot (@niQo)
- Nicolas Serafini (@nicolas-s)
- Nicolas Toromanoff (@toromanoSTM)
- Nicole Zeckner (@PurelyNicole)
- Nicolò Ribaudo (@nicolo-ribaudo)

- nicootje55 (@nicootje55)
- nicop4 (@nicop4)
- nicx (@nicx)
- nicxvan (@nicxvan)
- Niek Beernink (@nbeernink)
- Niels AD (@nielsAD)
- Niels Faber (@nielsfaber)
- Niels Keurentjes (@curry684)
- Niels Klumper (@Klumper)
- Niels Mündler (@nielstron)
- Niels Skou Olsen (@nsolsen)
- nierob (@nierob)
- Nigel Kukard (@nkukard)
- Nigel Rook (@NigelRook)
- Nihaal Sangha (@nihaals)
- Nik Klever (@nklever)
- Nik Nyby (@nikolas)
- Nik Rolls (@nicole-ashley)
- Nikfinn99 (@Nikfinn99)
- nikita (@nox10)
- Nikita Chernozipunnikov (@thatguynikita)
- nikito7 (@nikito7)
- Niklas (@niklaswa)
- Niklas (@ngdio)
- Niklas Grebe (@ThYpHo0n)
- Niklas Janz (@Alphakilo)
- Niklas Morberg (@morberg)
- Niklas Plakolb (@plakna)
- Niklas V (@Nicxe)
- Niklas Wagner (@Skaronator)
- Niklas Åström (@niklasastrom)
- Nikola (@0nikola1)
- Nikolai (@NikoM87)
- Nikolai Rahimi (@nikolairahimi)
- Nikolai Shcheglov (@shcheglovnd)
- Nikolaos Stamatopoulos (@nickstamat)
- Nikolas Beutler (@biacz)
- Nikolay Dimitrov (@picmaster)
- Nikolay Kasyanov (@nikolaykasyanov)
- Nikolay Vasilchuk (@Anonym-tsk)



- Nils Jacobs (@Revilum)
- Nils Kristian Brekke (@Brekjkjern)
- Nils Larsgård (@nilsmagnus)
- Nils Strelow (@nstrelow)
- Nils Uliczka (@darookee)
- Nils154 (@Nils154)
- nilzen (@nilzen)
- Nima A (@Nimzr)
- Nimai Mahajan (@nimaim)
- niobos (@niobos)
- Nippey (@Nippey)
- Niranjan Reddy (@kniranjanreddy)
- nirnachmani (@nirnachmani)
- Nishchith Shetty (@inishchith)
- Nito Buendia (@nitobuendia)
- Nityananda Padhan (@ntneitin)
- nivnoach (@nivnoach)
- Nix (@nickalcock)
- nklebedev (@nklebedev)
- nkrauss (@nkrauss)
- nldroid (@nldroid)
- NMA (@nma83)
- nmenegale (@nicolas-owi)
- Noam Okman (@noamokman)
- noam148 (@noam148)
- NobleKangaroo (@NobleKangaroo)
- nojocodex (@nojocodex)
- Nolan Darilek (@ndarilek)
- Nolan Gilley (@nkgilley)
- nolange (@nolange)
- nollbit (@nollbit)
- noodlemctwoodle (@noodlemctwoodle)
- nordeep (@nordeep)
- nordlead2005 (@nordlead2005)
- Norien (@Norien)
- normakm (@normakm)
- north3221 (@north3221)
- NotoriousBDG (@notoriousbdg)
- NotSomeBot (@mariusdkm)
- NovapaX (@NovapaX)

- Noé Rubinstein (@Phlogistique)
- nragon (@nragon)
- nroach44 (@nroach44)
- nsimb (@nsimb)
- ntalekt (@ntalekt)
- nuhi (@noohi)
- nullpixel (@nullpixel)
- nullstalgia (@nullstalgia)
- Numa Perez (@npres83)
- Nuno Nunes (@nunonunes)
- Nuno Sousa (@nunofgs)
- nunojusto (@nunojusto)
- NWiBGRsK (@NWiBGRsK)
- Nyro (@nyroDev)
- oa- (@oa-)
- obelix05 (@obelix05)
- obgm (@obgm)
- ochlocracy (@ochlocracy)
- Oderik (@Oderik)
- Odin Hørthe-Omdal Urdland (@odinho)
- Odin Ugedal (@odinuge)
- odysseas.eth (@odyslam)
- Ofek (@ofekp)
- Ofek Ashery (@ofekashery)
- Ofer Heifetz (@oferhz)
- officiallybob (@officiallybob)
- Ofir Petrushka (@hatkyinc2)
- OGINO Masanori (@omasanori)
- Ohad Benita (@ohadbenita)
- Ohad Levy (@ohadlevy)
- ohm (@nohm)
- ohmer1 (@ohmer1)
- Okke Garling (@ogarling)
- Ola Lidholm (@olalid)
- Olaf Conradi (@oohlaf)
- Olaf Rempel (@orempel)
- Olaf van Zandwijk (@olafz)
- Olav Alexander Mjelde (@olavxxx)
- OldSurferDude (@OldSurferDude)
- Ole-Kenneth (@olekenneth)

- Ole-Martin Heggen (@Swampen)
- Oleg (@5LICK)
- Oleg Kitain (@BanHammer)
- Oleg Kurapov (@2sheds)
- Oleh (@chomupashchuk)
- Oleh Hordiienko (@gordio)
- Oleksandr Kapshuk (@skynetua)
- Oleksandr Omelchuk (@sashao)
- Oleksandr Zhadan (@ozhlab)
- OleksandrBerchenko (@OleksandrBerchenko)
- Oleksii Serdiuk (@leppa)
- Olen (@Olen)
- Olifant1990 (@Olifant1990)
- olijouve (@olijouve)
- Olimpiu Rob (@olimpiurob)
- Oliv3rDog (@Oliv3rDog)
- Oliver (@ol-iver)
- Oliver Acevedo (@Oliver84)
- Oliver Dippel (@multigcs)
- Oliver Dunk (@oliverdunk)
- Oliver Großkloß (@Progaros)
- Oliver Gruß (@CubieMedia)
- Oliver Ou (@zlinoliver)
- Oliver van Porten (@mcdeck)
- Oliver Voelker (@magenbrot)
- oliverhg1 (@oliverhg1)
- OliverRepo (@OliverRepo)
- oliverscu (@oliverscu)
- Olivier B (@hobbe)
- Olivier Bossaer (@olibos)
- Olivier Cloirec (@clook)
- Olivier DEBAUCHE (@Smartappli)
- Olivier Guerriat (@olivierguerriat)
- Olivier Matz (@olivier-matz-6wind)
- Olivier Schonken (@olivierschonken)
- Olivier Singla (@osingla)
- Olivér Falvai (@ofalvai)
- Olli2807 (@Olli2807)
- olli.dev (@olli-dot-dev)
- Olliver Schinagl (@oliv3r)

- ollo69 (@ollo69)
- Olof Hellqvist (@hellqvio86)
- Omar Ali (@omarali)
- Omar Usman (@oozman)
- omegacore (@omegacore)
- Omen Wild (@OmenWild)
- Omer Efrat (@efratomer)
- omriasta (@omriasta)
- On Freund (@OnFreund)
- onagurna (@onagurna)
- Oncleben31 (@oncleben31)
- ondras12345 (@ondras12345)
- ondrejbaranek (@ondrejbaranek)
- Ong Vairoj (@ejel)
- Ongy (@Ongy)
- onkytonk (@onkytonk)
- onlymejosh (@onlymejosh)
- onsmam (@onsmam)
- Onur-d (@Onur-d)
- Open Home Automation (@open-homeautomation)
- OpenDave15 (@OpenDave15)
- optama (@optama)
- Orad SA (@oradsa)
- orb9 (@orb9)
- Orce MARINKOVSKI (@orcema)
- Oren (@baget)
- Ormund Williams (@ormundw)
- orrpan (@orrpan)
- Orson (@orson1282)
- Oscar (@oscfor)
- Oscar Calvo (@ocalvo)
- Oscar Carlsson (@oscarcarlsson)
- Oscar Hellström (@oscarh)
- Oscar Stenqvist (@Tickaren)
- Oscar Tin Lai (@soraxas)
- OscarHanzely (@OscarHanzely)
- osirisinferi (@osirisinferi)
- osk2 (@osk2)
- osono-design (@osono-design)
- Othou (@Othou)

- Ottavio Campana (@ocampana)
- ottersen (@ottersen)
- Otto Winter (@OttoWinter)
- Otto-G (@Otto-G)
- overkill32 (@overkill32)
- Ovidiu Sas (@ovidiusas)
- ovidiulaz7 (@ovidiulaz7)
- Owen Johnson (@owen2)
- Owen Voke (@owenvoke)
- Owen Walpole (@owenthewizard)
- Oxan van Leeuwen (@oxan)
- Oxbef (@Oxbef)
- oywino (@oywino)
- OzGav (@OzGav)
- oznu (@oznu)
- P0L0 (@p0l0)
- p4p3r (@p4p3r)
- P-Verbrugge (@P-Verbrugge)
- paalex (@paalex)
- pablito1755 (@pablito1755)
- pablitofernandez (@pablitofernandez)
- Pablo Mellado (@mellado)
- Pablo Ovelleiro Corral (@pinpox)
- Paddy0174 (@Paddy0174)
- Pdraig James Connolly (@padraigconnolly)
- pailloM (@pailloM)
- pakwan (@pakwan)
- panache67 (@panache67)
- Panagiotis Panagiotopoulos (@ppanagiotis)
- pander8828 (@pander8828)
- PanicRide (@PanicRide)
- panosmz (@panosmz)
- Paolo Antinori (@paoloantinori)
- Paolo Bonzini (@bonzini)
- Paolo Tagliaferri (@Vortexmind)
- Paolo Tuninetto (@tulindo)
- paolog89 (@paolog89)
- papperone (@papperone)
- Parham Ghazanfari (@pghazanfari)
- Parker Moore (@parkr)

- Parker Wahle (@regulad)
- Parnell Springmeyer (@ixmatus)
- partytimeexcellent (@partytimeexcellent)
- Pascal (@p4sl)
- Pascal Bach (@bachp)
- Pascal de Bruijn (@pmjdebruijn)
- Pascal de Ladurantaye (@pascal-de-ladurantaye)
- Pascal Hahn (@pascalhahn)
- Pascal Huerst (@pascalhuerst)
- Pascal Reeb (@pree)
- Pascal Roeleven (@pascallj)
- Pascal Vizeli (@pvizeli)
- Pascal Winters (@pascalwinters)
- pascalsaul (@pascalsaul)
- Pat Freeman (@patfreeman)
- Pat Rooney (@sotatech)
- pataar (@pataar)
- patagona (@patagonaa)
- patatman (@patatman)
- patmann03 (@patmann03)
- Patrick (@tradiuz)
- Patrick Aikens (@duckpuppy)
- Patrick Clery (@patrickclery)
- Patrick Connelly (@pcon)
- Patrick D. (@pavax)
- Patrick de Mooij (@patrickdemooij)
- Patrick Decat (@pdecat)
- Patrick Easters (@patrickeasters)
- Patrick Georgi (@pgeorgi)
- Patrick Hilker (@patrickhilker)
- Patrick Hobusch (@pathob)
- Patrick Hofmann (@PH89)
- Patrick Kishino (@pkishino)
- Patrick Langendoen (@pathia)
- Patrick Ribbing (@patrickribbing)
- Patrick Roozen (@pcroozen)
- Patrick Smits (@Pater100)
- Patrick T.C (@ptc)
- Patrick Toft Steffensen (@PTST)
- Patrick White (@pw)

- Patrick ZAJDA (@Nardol)
- Patrik Ekström (@patrik3k)
- Patrik Hermansson (@bphermansson)
- Patrik Lindgren (@ggravlingen)
- Patryk (@C6H6)
- Patryk Duda (@semihalf-duda-patryk)
- Patryk Gałczyński (@evemorgen)
- PatSki123 (@patski123)
- Pau Marin (@Pau-Marin)
- Paul (@TakesTheBiscuit)
- Paul Annekov (@PaulAnnekov)
- Paul Archer (@geek65535)
- Paul B. Henson (@pbhenson)
- Paul Beckcom (@pbeckcom)
- Paul Biester (@isonet)
- Paul Bottein (@piitaya)
- Paul Burton (@paulburton)
- Paul Cercueil (@pcercuei)
- Paul Daumlechner (@pawlizio)
- Paul Davis (@paulbdavis)
- Paul Dee (@systemcrash)
- Paul Deen (@PaulAntonDeen)
- Paul Dugas (@pdugas)
- Paul Enright (@penright)
- Paul Frank (@pail23)
- Paul Ganssle (@pganssle)
- Paul Hendry (@polendri)
- Paul Hobbs (@SolidElectronics)
- Paul Jimenez (@pjz)
- Paul Jolly (@myitcv)
- Paul Jones (@PeeJay)
- Paul Kehrer (@reaperhulk)
- Paul Kline (@paul-kline)
- Paul Klingelhuber (@NoUsername)
- Paul Krischer (@SqyD)
- Paul Madden (@maddenp)
- Paul Manzotti (@manzanotti)
- Paul Monigatti (@paulmonigatti)
- Paul Mundt (@pmundt)
- Paul Nicholls (@pauln)

- Paul Owen (@Drae)
- Paul Philippov (@themactep)
- Paul Rabahy (@PRabahy)
- Paul Romkes (@Romkabouter)
- Paul Sinclair (@sinclairpaul)
- Paul Sokolovsky (@pfalcon)
- Paul Stenius (@stenius)
- Paul Swartz (@paulswartz)
- Paul Tarjan (@ptarjan)
- paulcram (@paulcram)
- PaulDalyROI (@PaulDalyROI)
- Pauli Sundberg (@susundberg)
- Paulius Zaleckas (@pauliuszaleckas)
- Paulo Matos (@pmatots)
- Paulo Serra Filho (@pauloserrafh)
- PaulParis1 (@PaulParis1)
- pavanagrawal123 (@pavanagrawal123)
- Pavel Moukhataev (@mpashka)
- Pavel Pletenev (@ASMfreak)
- Pavel Popov (@tolkonepiu)
- Pavel Roslovets (@roslovets)
- Pavel S (@pilotak)
- Pavel Skuratovich (@Chupaka)
- pavelplus (@pavelplus)
- Pavlo Ponomarov (@awsum)
- Pavol Babinčák (@scrool)
- Pavol Holes (@pavolholes)
- Pawel (@pszafer)
- Pawel Sikora (@pawelsikora)
- Pawel Winogrodzki (@PawelWMS)
- Paweł Hulek (@pawelhulek)
- Paweł Kobylak (@noose)
- Paweł Krupa (@paulfantom)
- Paweł Stankowski (@ak-ambi)
- pawkakol1 (@pawkakol1)
- Paxy (@Paxy)
- pbalogh77 (@pbalogh77)
- pbassa (@pbassa)
- PCR-2020 (@PCR-2020)
- pcragone (@pcragone)



- [pdanilew \(@pdanilew\)](#)
- [PDarkTemplar \(@PDarkTemplar\)](#)
- [pdcemulator \(@pdcemulator\)](#)
- [pdeelen \(@pdeelen\)](#)
- [PDekker \(@PDekker\)](#)
- [peca89 \(@peca89\)](#)
- [peddamat \(@peddamat\)](#)
- [Pedro Aguilar \(@paguilar\)](#)
- [Pedro Jardim \(@pejardim\)](#)
- [Pedro Lamas \(@pedrolamas\)](#)
- [Pedro Miguel Fernandes \(@pmfernandes\)](#)
- [Pedro Navarro \(@pedronavf\)](#)
- [Pedro Ribeiro \(@peribeir\)](#)
- [Pedro Rodriguez Tavaréz \(@pjrt\)](#)
- [PedroKTFC \(@PedroKTFC\)](#)
- [PeeJay \(@pejotigrek\)](#)
- [Peeter N \(@petslane\)](#)
- [pepeEL \(@pepsonEL\)](#)
- [Per Osbäck \(@perosb\)](#)
- [Per Sandström \(@persandstrom\)](#)
- [Per Thomas Jahr \(@perja12\)](#)
- [Per Öberg \(@droberg\)](#)
- [Per-Arne Andersen \(@perara\)](#)
- [Per-Øyvind Bruun \(@peroyvind\)](#)
- [pereiraru \(@pereiraru\)](#)
- [perfalk \(@perfalk\)](#)
- [pergolafabio \(@pergolafabio\)](#)
- [perjury \(@perjury\)](#)
- [Peronia \(@Peronia\)](#)
- [Perry Naseck \(@DaAwesomeP\)](#)
- [Pertti Roitto \(@roitto\)](#)
- [Petar Petrov \(@MindFreeze\)](#)
- [Pete \(@peteh\)](#)
- [Pete Cooper \(@petecoop\)](#)
- [Pete LePage \(@petele\)](#)
- [Pete Peterson \(@petey\)](#)
- [PeteBa \(@PeteBa\)](#)
- [PeteCondliffe \(@PeteCondliffe\)](#)
- [PetePriority \(@PetePriority\)](#)
- [Peter \(@pyrrolizin\)](#)

- Peter (@AnderssonPeter)
- Peter A. Bigot (@pabigot)
- Peter Armstrong (@peteretep)
- Peter Bainbridge (@BushmanPete)
- Peter Clarke (@peteclarkez)
- Peter Cooney (@PeteCooney)
- Peter Ebenezer (@Coedy)
- Peter Epley (@epleypa)
- Peter Galantha (@peterg79)
- Peter Gebruers (@petergebruers)
- Peter Golm (@pgolm)
- Peter Huewe (@PeterHuewe)
- Peter Korsgaard (@jacmet)
- Peter Krempa (@pipo)
- Peter Kyrkos (@KmanOz)
- Peter Kümmel (@syntheticpp)
- Peter Macleod Thompson (@PeterMacleodThompson)
- Peter Maguire (@Peter-Maguire)
- Peter Meerwald (@pmeerw)
- Peter Nijssen (@peternijssen)
- Peter Rosin (@peda-r)
- Peter Sarossy (@psarossy)
- Peter Seiderer (@pseiderer)
- Peter Tisovčík (@mienkofax)
- Peter van Dijk (@Habbie)
- Peter Weidenkaff (@weidenka)
- Peter Zsak (@wroadd)
- PeteRager (@PeteRager)
- petergridge (@petergridge)
- petewill (@petewill)
- petkov (@petkov)
- Petr Kotek (@petrkotek)
- Petr Vorel (@pevik)
- Petr Vraník (@konikvranik)
- Petro31 (@Petro31)
- Petru Paler (@ppetru)
- pettertho (@pettertho)
- pezinek (@pezinek)
- pfunkmallone (@pfunkmallone)
- pgenera (@pgenera)

- Ph-Wagner (@Ph-Wagner)
- phardy (@phardy)
- phfix (@phfix)
- Phi Dong (@pdong)
- PhiBo (@phibos)
- Phil (@pnbruckner)
- Phil (@godloth)
- Phil Cole (@filcole)
- Phil Eichinger (@philenotfound)
- Phil Elson (@pelson)
- Phil Freo (@philfreo)
- Phil Frost (@bitglue)
- Phil Haack (@haacked)
- Phil Hansen (@Hansen8601)
- Phil Hawthorne (@philhawthorne)
- Phil Hollenback (@tels7ar)
- Phil Kates (@philk)
- Phil Lavin (@phil-lavin)
- Phil Nelson (@phil-nelson)
- Phil Wareham (@philwareham)
- phileaton (@phileaton)
- Phileep (@Phileep)
- Philip Allgaier (@spacegaier)
- Philip Hofstetter (@pilif)
- Philip Howard (@Gadgetoid)
- Philip Kleimeyer (@philklei)
- Philip Lundrigan (@philipbl)
- Philip Molloy (@pamolloy)
- Philip Rosenberg-Watt (@PhilRW)
- Philip Sørenberg (@philipsoeberg)
- Philipp Danner (@dannerph)
- Philipp Richter (@popsUlfr)
- Philipp Schmitt (@pschmitt)
- Philipp Temminghoff (@phil65)
- Philipp Wagner (@imphil)
- Philipp Wensauer (@ultrara1n)
- Philippe Delodder (@phdelodder)
- Philippe Pepiot (@philpep)
- Philippe Proulx (@eepp)
- Philippe Schenker (@philschenker)

- philipperequile (@philipperequile)
- Phill (pssc) (@pssc)
- Phill Price (@phillprice)
- Phillip Howard (@PhillipHoward)
- phispi (@phispi)
- phithor (@phithor)
- phlet (@phlet)
- Phongsakorn Wisetthorn (@boyphongsakorn)
- photinus (@photinus)
- PhractedBlue (@PhractedBlue)
- phrfpeixoto (@phrfpeixoto)
- Pier-Luc Charbonneau (@plcharbonneau)
- Piero (@pierosavi)
- Pierre (@bemble)
- Pierre (@pinaraf)
- Pierre (@BaQs)
- Pierre Courbin (@pcourbin)
- Pierre CROKAERT (@crookies)
- Pierre Gronlier (@ticapix)
- Pierre Sicot (@psicot)
- Pierre Ståhl (@postlund)
- Pierre Wilken (@pwilken)
- Pierre-Jean Leger (@Caligone)
- Pierre-jean Texier (@texierp)
- Pierre-Louis Bossart (@plbossart)
- Pierre-Luc Milord (@plmilord)
- Piers Dawson-Damer (@piersdd)
- Piet van Dongen (@pietvandongen)
- Pieter De Baets (@javache)
- Pieter De Gendt (@pdgendt)
- Pieter Ennes (@skion)
- Pieter Frenssen (@pfrenssen)
- Pieter Goetschalckx (@314eter)
- Pieter Hamels (@phamels)
- Pieter Mulder (@inytar)
- Pieter Ouwerkerk (@pouwerkerk)
- Pieter Rautenbach (@parautenbach)
- Pieter Ronsijn (@piro-barco)
- Pieter Ronsijn (@pieter-dev)
- Pieter Smith (@smipi1)

- Pieterjan Camerlynck (@pieterjanc)
- Pigotka (@Pigotka)
- Pim (@PimDoos)
- pinksocks (@pinksocks)
- Piotr Dobrowolski (@Informatic)
- Piotr Kubiak (@piotr-kubiak)
- Piotr Machowski (@PiotrMachowski)
- Piotr Majkrzak (@majkrzak)
- Piotr Miazga (@polishdeveloper)
- Piotr Potulski (@piotrpo)
- Piotr Witek (@piowit)
- Piotr Żuralski (@piotr-zuralski)
- pip (@pipip)
- Pipiche (@pipiche38)
- Pirionfr (@Pirionfr)
- pixel::doc (@pixeldoc2000)
- pixelasticity (@pixelasticity)
- pizzaboy192 (@pizzaboy192)
- pjv (@pjv)
- pkeymeruu (@pkeymeruu)
- pkininja (@pkininja)
- pkonnekermetrics (@pkonnekermetrics)
- plafü (@plafue)
- plamish (@plamish)
- PlanetJ (@PlanetJ)
- plasma16 (@plasma16)
- pleerja (@pleerja)
- plomosits (@plomosits)
- PlusPlus-ua (@PlusPlus-ua)
- plyblu (@plyblu)
- pmmcmullen94 (@pmmcmullen94)
- pnguyen-tyro (@pnguyen-tyro)
- pniewiadowski (@pniewiadowski)
- PollieKrismis (@PollieKrismis)
- Poltorak Serguei (@PoltoS)
- PoofyTeddy (@poofyteddy)
- popboxgun (@popboxgun)
- popoviciri (@popoviciri)
- posixx (@posixx)
- pothirak (@pothirak)

- Povl H. Pedersen (@povlhp)
- pp81381 (@pp81381)
- pplucky (@pplucky)
- prairiesnpr (@prairiesnpr)
- Pranit Sirsat (@pranit-sirsat-imgtec)
- Prasad Bankar (@psbankar)
- Prasetyo Joko Lelono (@xilense)
- Prathamesh Gawas (@Prathamesh010)
- Pratyush Mohapatra (@Ativerc)
- presslab-us (@presslab-us)
- Preston A Elder (@corporategoth)
- priiduonu (@priiduonu)
- PriMachVisSys (@PriMachVisSys)
- printzlau (@printzlau)
- ProfDrYoMan (@ProfDrYoMan)
- proferabg (@proferabg)
- prokon (@prokon)
- PromInc (@PromInc)
- prophit987 (@prophit987)
- ProudElm (@ProudElm)
- prwood80 (@prwood80)
- Przemek Więch (@PeWu)
- Przemyslaw (Pem) Lasota (@plasota)
- Przemyslaw Wrzos (@calyptech)
- ps-m (@ps-m)
- psike (@psike)
- psixilambda (@psixilambda)
- Pteranodon (@Pteranodon)
- PuckStar (@PuckStar)
- puddly (@puddly)
- PulsarFX (@PulsarFX)
- punch (@jonthebastard)
- Punita Ojha (@punitaojha)
- purcell-lab (@purcell-lab)
- Purplecarrot (@purplecarrot)
- pvianag (@pvianag)
- pvmil (@pvmil)
- pwesters (@pwesters)
- pyitphyoaung (@pyitphyoaung)
- Pär Svanström (@pp-svanstrom)

- Qais Patankar (@qaisjp)
- qaiser1996 (@qaiser1996)
- QbaF (@QbaF)
- qinghuangchan (@qinghuangchan)
- qiz-li (@qiz-li)
- qqgg231 (@qqgg231)
- qrionic labs (@qrioniclabs)
- quaec (@quaec)
- Quentame (@Quentame)
- Quentin Stafford-Fraser (@quentinsf)
- Questler (@Questler)
- quiglag (@quiglag)
- Quinn Casey (@qcasey)
- Quinn Hosler (@quinnhosler)
- Quint Burkley (@qwow5)
- quthla (@quthla)
- Qwertee (@jberrend)
- QxIkdr (@QxIkdr)
- R Huish (@genestealer)
- R0nd (@R0nd)
- r24-IT (@r24-IT)
- r4nd0mbr1ck (@r4nd0mbr1ck)
- r-jordan (@r-jordan)
- R. de Veen (@rdeveen)
- Raa'Shaun Hunter (@CardcaptorRLH85)
- Racailleux (@Racailleux)
- Radek Rada (@rrada)
- RadekHvizados (@RadekHvizados)
- radinsky (@radinsky)
- radovanbauer (@radovanbauer)
- Radu (@mercenaruss)
- Radu Cotescu (@raducotescu)
- Rafael Alencar (@rafaeldca)
- Rafael Gil (@cybervoid)
- Rafael Goes (@rgriffogoes)
- Rafael Uguina (@crazystress)
- Rafal Fabich (@R2F)
- Rafal Susz (@rsusz)
- rafale77 (@rafale77)
- Rafhaan Shah (@RafhaanShah)

- Rafi Wiener (@rafiw)
- rahul-bedarkar (@rahul-bedarkar)
- rahul\_5409 (@RahulRavishankar)
- Raiford (@raiford)
- rainlake (@rainlake)
- Raj Laud (@rajlaud)
- rako77 (@rako77)
- ral (@leitonp)
- Ralf Muehlen (@muehlen)
- Ralph (@bberg115)
- Ralph Hopman (@rhopman)
- Raman Gupta (@raman325)
- Rami Mosleh (@engrbm87)
- Ramon (@jush)
- Ramon Fried (@mellowcandle)
- Ramshield (@Ramshield)
- Randall Mason (@ClashTheBunny)
- Randall Wessel (@randyxxl)
- randellhodes (@randellhodes)
- rangulvers (@rangulvers)
- Raoul Teeuwen (@raoulteeuwen)
- Raphael (@sdrapha)
- Raphaël Beamonte (@XaF)
- Raphaël Mélotte (@rmelotte)
- Raphaël Poggi (@raphui)
- rappenze (@rappenze)
- Rashmi Yadav (@raysrashmi)
- rasmuoy (@rasmuoy)
- Rasmus (@rasmusbe)
- ratcash (@ratcashdev)
- Raukze (@Raukze)
- Raul Hidalgo Caballero (@deinok)
- Ravi K (@shreram)
- Ray (@raymck)
- Ray Cielencki (@rayslinky)
- Ray Kinsella (@mdr78)
- ray0711 (@ray0711)
- raymccarthy (@raymccarthy)
- Raymon de Looff (@raymondellooff)
- Raymond Ha (@Shraymonks)



- Raymond Ng (@RaymondNg2)
- Raz Luvaton (@rluvaton)
- Raúl Sánchez Siles (@kebianizao)
- rbflurry (@rbflurry)
- RBHR (@rbhr)
- rc-matthew-l-weber (@rc-matthew-l-weber)
- rccoleman (@rccoleman)
- rcmdnk (@rcmdnk)
- rctgamer3 (@rctgamer3)
- RDFurman (@rdfurman)
- rdkayman (@rdkayman)
- rdkehn (@rdkehn)
- realPy (@realPy)
- realthk (@realthk)
- reaper7 (@reaper7)
- Rebecca Cran (@bcran)
- Rechner Fox (@rechner)
- Redah (@redahb)
- redbar0n (@redbar0n)
- redgryphon (@redgryphon)
- reductio (@reductio)
- Reed Nightingale (@rnighting)
- Reed Riley (@reedriley)
- reef-actor (@reef-actor)
- Reese (@reesericci)
- refinedcranberry (@refinedcranberry)
- RefineryX (@RefineryX)
- Regev Brody (@regevbr)
- reggit96 (@reggit96)
- reichley (@reichley)
- Reinder Reinders (@reinder83)
- Reinhard Tartler (@siretart)
- remc0 (@remc0)
- Remi Pommarel (@repk)
- Remy (@Pouzor)
- remy33 (@remy33)
- Renat Nurgaliyev (@rnurgaliyev)
- Renat Sibgatulin (@Sibgatulin)
- renat85 (@renat85)
- Renaud AUBIN (@r3n4ud)

- Renaud Martinet (@karouf)
- Renaud Morvan (@nel)
- Rendili (@Rendili)
- Rene Lehfeld (@rlehfeld)
- Rene Nulsch (@ReneNulschDE)
- Rene Tode (@ReneTode)
- RenierM26 (@RenierM26)
- René (@rretsiem)
- René Kliment (@renekliment)
- René Klomp (@rklomp)
- René-Marc Simard (@renemarc)
- resistr (@resistr)
- Reuben Bijl (@reubenbijl)
- Reuben Dowle (@reubendowle)
- Reuben Gow (@geuben)
- Rev Michael Greb (@mikegrb)
- rexcze (@rexcze)
- Reza Moallemi (@moallemi)
- rforro (@rforro)
- rgc99 (@rgc99)
- rgenoud (@rgenoud)
- rgruebel (@rgruebel)
- rhadamantys (@rhadamantys)
- rhino53150 (@rhino53150)
- rhooper (@rhooper)
- rhpijnacker (@rhpijnacker)
- Rhys Williams (@wilberforce)
- rianadon (@rianadon)
- ricardcc (@ricardcc)
- Ricardo JL Rufino (@ricardojlrufino)
- Ricardo Martincoski (@ricardo-martincoski)
- Ricardo Steijn (@RicArch97)
- Riccardo (@safepay)
- Riccardo Canta (@commento)
- Riccardo Massari (@maxdrift)
- Rich Schumacher (@richid)
- Richard (@NextNebula)
- Richard (@ainen)
- Richard Annand (@ohheyjrj)
- Richard Arends (@Mosibi)

- Richard Benson (@rkben)
- Richard Braun (@richardbraun)
- Richard Cox (@Khabi)
- Richard Cunningham (@rythie)
- Richard de Boer (@rigrig)
- Richard Evans (@rmevans9)
- Richard Jones (@RJ)
- Richard Kunze (@rkunze)
- Richard Lucas (@lucasweb78)
- Richard Meyer (@meyerrij)
- Richard Mitchell (@mitchellrij)
- Richard Niemand (@rniemand)
- Richard Orr (@rorr73)
- Richard Patel (@terorie)
- Richard Powell (@richardpowellus)
- Richard T. Schaefer (@r-t-s)
- Richard van Duijn (@TheNr1Guest)
- Richard van Paasen (@strangeflower)
- RichardUUU (@RichardUUU)
- RichieFrame (@RichieFrame)
- richo (@richo)
- Rick (@rickdoesdev)
- Rick (@rickvdl)
- Rick (@teleksterling)
- Rick Fletcher (@rfletcher)
- Rick Sharp (@ricksharp)
- Rick Sherman (@shermdog)
- Rick van Hattem (@wolph)
- Rick van Hattem (@WoLpH)
- RickW (@wltng)
- rickylee64 (@rickylee64)
- RickyTaterSalad (@RickyTaterSalad)
- ricmik (@ricmik)
- Ridicule80 (@Ridicule80)
- Rihan9 (@Rihan9)
- rik-v (@rik-v)
- rikroe (@rikroe)
- rishabmehta7 (@rishabmehta7)
- RJ Ascani (@rascani)
- rjulius23 (@rjulius23)

- rkabadi (@rkabadi)
- rkuijer (@rkuijer)
- rlippmann (@rlippmann)
- rmacklin (@rmacklin)
- rmogstad (@rmogstad)
- rnizametdinov (@rnizametdinov)
- Rob (@robvanuden)
- Rob Bierbooms (@RobBie1221)
- Rob Borkowski (@rborkow)
- Rob Capellini (@capellini)
- Rob Chandhok (@robchandhok)
- Rob Connolly (@webworxshop)
- Rob Cranfill (@RobCranfill)
- Rob Jackson (@rjackson)
- Rob Johnson (@robjohnson189)
- Rob Landley (@landley)
- Rob McCann (@rob-mccann)
- Rob Migchels (@DhrRob)
- Rob Slifka (@rslifka)
- Rob VDM (@rvdm82)
- Rob Wolinski (@trekie86)
- Rob Zwissler (@robzr)
- Rob\_R (@Robbot)
- robadenshi (@robadenshi)
- Robbert Müller (@mj rider)
- Robbert van Markus (@rvanmarkus)
- Robbie (@superjunky)
- Robbie Page (@rorpage)
- Robbie Trencheny (@robbiet480)
- Robby Grossman (@freerobby)
- Robert (@metbril)
- Robert (@sqrtroot)
- Robert (@robehem)
- Robert "DocSalvager" Watson (@DocSalvager)
- Robert Accettura (@raccettura)
- Robert Beal (@robertbeal)
- Robert Blomqvist (@roqvist)
- Robert Chmielowiec (@chmielowiec)
- Robert Delpout (@robertdelpout)
- Robert Dunmire III (@slackr31337)

- Robert Fischer (@foundbobby)
- Robert Hancock (@robhancocksed)
- Robert Hibbeler (@kventil)
- Robert Hillis (@tkdrob)
- Robert J. Heywood (@robjh)
- Robert Kingston (@kingo55)
- Robert Kiss (@kepten)
- Robert Kowalski (@robertkowalski)
- Robert Marklund (@trollkarlen)
- Robert Meijers (@RobertMe)
- Robert P. J. Day (@rpjday)
- Robert Resch (@edenhaus)
- Robert Rose (@robertroyrose)
- Robert Schütz (@dotlambda)
- Robert Sohn (@grepper)
- Robert Svensson (@Kane610)
- Robert Teller (@r-teller)
- Robert Van Gorkom (@vangorra)
- RobertD502 (@RobertD502)
- Roberto Cc (@marotoweb)
- Roberto Tyley (@rtyley)
- robhuls (@robhuls)
- Robin (@robmarkcole)
- Robin (@Escape)
- Robin (@Derkades)
- Robin (@rbnis)
- Robin Clarke (@robin13)
- Robin Dupont (@N0ciple)
- Robin Harmsen (@reharmsen)
- Robin Jarry (@rjarry)
- Robin Kolk (@kloknibor)
- Robin Laurén (@llauren)
- Robin Malik (@robinmalik)
- Robin Migalski (@RobinMglsk)
- Robin Pronk (@rfpronk)
- Robin Wittebol (@robinwittebol)
- Robin Wohlers-Reichel (@squishykid)
- Robk (@230delphi)
- roblandry (@roblandry)
- RoboMagus (@RoboMagus)

- robot256 (@robot256)
- robybob64 (@robybob64)
- Rocik (@Rocik)
- RockBomber (@RockBomber)
- Rod Payne (@rodpayne)
- Roddie Hasan (@eiddor)
- Rodrigo Pérez (@rodripf)
- Rodrigo Rebello (@rrebello)
- Roel van der Ark (@roelvanderark)
- Roelof Schuiling (@rschuiling)
- Roger Gammans (@rgammans)
- Roger Pettett (@rmp)
- RogerSelwyn (@RogerSelwyn)
- Rogue136198 (@Rogue136198)
- Rogério Ribeiro (@zroger49)
- Rohan Kapoor (@rohankapoorcom)
- Rohit (@bogusfocused)
- Roi Dayan (@roidayan)
- roiff (@roiff)
- Rokas Brazdžionis (@dzonatan)
- Roland Beck (@Data-Monkey)
- Roland Franke (@Roland-F)
- roleo (@roleoroleo)
- Rolf Berkenbosch (@rolfberkenbosch)
- Rolf K (@abstrakct)
- Rolf Schäuble (@rschaeuble)
- rolfikr1 (@rolfikr1)
- rollbrettler (@rollbrettler)
- Romain (@Moumoustache)
- Romain Izard (@romain-izard-pro)
- Romain Naour (@RomainNaour)
- Romain Reignier (@romainreignier)
- romain38 (@romain38)
- Roman (@HerrHofrat)
- Roman (@Roemer)
- Roman Shtylman (@defunctzombie)
- Romel Williams (@omerome83)
- Ron Heald (@tallcuss)
- Ron Heft (@ronaldheft)
- Ron Klinkien (@cyberjunky)

- Ron Mervine (@rmervine)
- Ron Miller (@StreamOfRon)
- Ron Schaeffer (@ronschaeffer)
- Ron Šmeral (@rsmeral)
- Ronak Desai (@ronakadesai)
- Ronald Dehuysser (@rdehuyss)
- Ronald Evers (@ronaldevers)
- Ronaldo Lima (@ronal2do)
- Ronan Murray (@ronanmu)
- Roncoa (@roncoa)
- rondoal (@rondoal)
- Ronen Hayun (@rhayun)
- Ronnie (@rroller)
- Ronnie Garcia (@ronniegarcia)
- Ronny Eia (@eiaro)
- Ronny Winkler (@RonnyWinkler)
- RonSpawns (@RonSpawns)
- ronsum232 (@ronsum232)
- roqeer (@roqeer)
- Rosemary Orchard (@RosemaryOrchard)
- rospogrigio (@rospogrigio)
- Ross Cullen (@rosscullen)
- Ross Dargan (@rossdargan)
- Ross Patterson (@rpatterson)
- Ross Schulman (@rschulman)
- Ross Troha (@rosstroha)
- rourke (@rourke)
- Roustem Karimov (@roustem)
- Roy (@twoez)
- Roy Duineveld (@royduin)
- Roy Lee (@roylee17)
- Roy Tomeij (@roytomeij)
- rozie (@rozie)
- rpbx (@rpbx)
- rpitera (@rpitera)
- rpr69 (@rpr69)
- rrubin0 (@rrubin0)
- rssalerno (@rssalerno)
- Ruben (@freakinruben)
- Ruben (@rubenvandeven)

- Ruben Andrist (@andrist)
- Ruben J. Jongejan (@rvben)
- rubenbe (@rubenbe)
- RubenKelevra (@RubenKelevra)
- Rubens Panfili (@rpanfili)
- rubenverhoef (@rubenverhoef)
- rubund (@rubund)
- Rubén Infante (@ruben0909)
- Rudertier (@Rudertier)
- Rudi Middel (@mrBussy)
- Rudolf Offereins (@Sholofly)
- Rui Dias (@netsoft-ruidias)
- runningman84 (@runningman84)
- RunOnGitHub (@RunOnGitHub)
- runraid (@runraid)
- ruohan.chen (@crhan)
- Rupert (@dublowduck)
- Ruslan Sayfutdinov (@KapJI)
- Russ Kubes (@rkubes)
- Russ Nelson (@RussNelson)
- russdan (@russdan)
- Russell Cloran (@rcloran)
- rwinjanssen (@rwinjanssen)
- rxwen (@rxwen)
- Ryan (@rsnodgrass)
- Ryan Bahm (@rdbahm)
- Ryan Barnett (@rjbarnet)
- Ryan Barnett (@rjbarnet-work)
- Ryan Bateman (@ryanbateman)
- Ryan Bray (@rbray89)
- Ryan Claussen (@rtclauss)
- Ryan Coe (@nismoryco)
- Ryan Daigle (@rwdaiGLE)
- Ryan Davies (@PrimusNZ)
- Ryan Ewen (@RyanEwen)
- Ryan Feigenbaum (@royalfig)
- Ryan Fleming (@rfleming71)
- Ryan Garcia (@rgarcia6520)
- Ryan Gibbons (@rtgibbons)
- Ryan Hunt (@nayrnet)



- Ryan Hurst (@rmhrisk)
- Ryan Jarvis (@Cabalist)
- Ryan Johnson (@ryanjohnsontv)
- Ryan Kladar (@Kladar)
- Ryan Kraus (@rmkraus)
- Ryan Macdonald (@ryanmac8)
- Ryan Mallon (@RyanMallon)
- Ryan McLean (@ryanm101)
- Ryan Meek (@maykar)
- Ryan Meulenkamp (@RyanMeulenkamp)
- Ryan Miguel (@renegaderyu)
- Ryan Mounce (@rmounce)
- Ryan Nazaretian (@ryannazaretian)
- Ryan Nowakowski (@tubaman)
- Ryan Parrish (@stickstyle)
- Ryan Sandridge (@dissolved)
- Ryan Spicer (@alterscape)
- Ryan Steckler (@rsteckler)
- Ryan Turner (@turnrye)
- Ryan Wagoner (@rwagoner)
- Ryan Warner (@rwarner)
- Ryan Wilkins (@rwilkins74)
- Ryan Winchester (@ryanwinchester)
- Ryan Winter (@ryanwinter)
- RyanRoberts210 (@RyanRoberts210)
- ryansun96 (@ryansun96)
- RybA (@freedomsarge)
- ryborg (@ryborg)
- ryddler (@ryddler)
- ryman1 (@ryman1)
- ryoku-cha (@ryoku-cha)
- Ryota Kinukawa (@pojiro)
- ryqiem (@MartinBernstorff)
- Ryszard Trojnacki (@Ryszard-Trojnacki)
- Rémi Rérolle (@rrerolle)
- Róbert Nagy (@vrnagy)
- Sabesto (@Sabesto)
- sabuto (@sabuto)
- Sacha Telgenhof (@stelgenhof)
- saepfle (@saepfle)

- SafetySigning (@SAFEcert)
- Sagaert Johan (ON5DI) (@sagaertj)
- Sagi Bernstein (@sagioto)
- sagilo (@sagilo)
- SaintMalik (@saintmalik)
- saintman23 (@saintman23)
- Salamandar (@Salamandar)
- SalexSun (@SalexSun)
- Salim B (@salim-b)
- Salman Shah (@salman-bhai)
- salsaman (@salsaman)
- Salvatore Cordiano (@salvatorecordiano)
- Salvatore Mazzarino (@mazzy89)
- Sam (@samuelalexmclean)
- Sam Birch (@hotplot)
- Sam Debruyn (@sdebruyn)
- Sam Hanley (@sphanley)
- Sam Holmes (@sam3d)
- Sam Jongenelen (@SamJongenelen)
- Sam Lancia (@nerd2)
- Sam Mendoza-Jonas (@sammj)
- Sam Neely (@LedPighp)
- Sam Pierce Lolla (@sampl)
- Sam Reed (@reedy)
- Sam Riley (@samriley)
- Sam Steele (@c99koder)
- Sam Voss (@smvoss-collins)
- Sam Voss (@smvoss)
- Sam Welek (@tiberiushunter)
- Sam Whited (@SamWhited)
- sam-io (@sam-io)
- sam-wright (@sam-wright)
- Samantha (@Samantha-uk)
- Samar Dhwoj Acharya (@techgaun)
- sambltc (@sambltc)
- Sami Heino (@sampod)
- Samprit JC (@Nekros1712)
- samtygier (@samtygier)
- Samuel Attard (@MarshallOfSound)
- Samuel Bétrisey (@betrisey)

- Samuel Dumont (@samueldumont)
- Samuel Maggs (@samuelmaggs)
- Samuel MARTIN (@tSed)
- Samuel Progin (@Arduous)
- Samuel Rau (@samrdev)
- Samuel Tardieu (@samueltardieu)
- Samuel Yun (@samyun)
- Samuele Illuminati (@samugi)
- samuelsson86 (@samuelsson86)
- SamyGarib (@SamyGarib)
- Sander (@golles)
- Sander (@spmvooss)
- Sander Cox (@sandercox)
- Sander de Leeuw (@sdeleeuw)
- Sander Geerts (@Devqon)
- Sander Huisman (@Sanderhuisman)
- Sander Jochems (@Sander0542)
- Sander Kromwijk (@raiju)
- Sander Zumbrink (@zumitnl)
- sander76 (@sander76)
- Sangwon Kim (@scon-io)
- Santobert (@Santobert)
- Santosh Multhalli (@santoshmulthalli)
- sapph42 (@sapph42)
- Sarabveer Singh (@sarabveer)
- Sarah Mount (@snim2)
- sarakha63 (@sarakha63)
- saruter (@saruter)
- Sascha Arthur (@saschaarthur)
- Sascha Kühndel (@InuSasha)
- Sascha Sander (@SaSa1983)
- Sasha Shyrovkov (@alexander-shyrovkov)
- sasso0101 (@Sasso0101)
- SaturnusDJ (@SaturnusDJ)
- Saumya Balodi (@saumya1906)
- Saurabh Sharma (@saurabhsharma001)
- Sava Tshontikidis (@stshontikidis)
- Save me (@Cyr-ius)
- Sbaa1 (@Sbaa1)
- sbabcock23 (@sbabcock23)

- sbilly (@sbilly)
- sbombay (@sbombay)
- sbYm (@GongT)
- Schachar Levin (@schachar)
- Schenk Michael (@schemic)
- scheric (@scheric)
- schferbe (@schferbe)
- schiermi (@schiermi)
- schlac (@schlac)
- Schmackos (@Schmackos)
- Schmidsfeld (@Schmidsfeld)
- schmjop (@schmjop)
- schmyd (@schmyd)
- schnabel (@schnabel)
- schneefux (@schneefux)
- schnoetz (@schnoetz)
- Scholli (@ScholliYT)
- schreyack (@schreyack)
- schuelles (@schuelles)
- SchumyHao (@SchumyHao)
- Scott (@lostage)
- Scott (@SonicMagna)
- Scott (@talltechdude)
- Scott Albertson (@salbertson)
- Scott Bartuska (@sxb1n9)
- Scott Bradshaw (@swbradshaw)
- Scott Bressler (@sbressler)
- Scott Ellis (@scottellis)
- Scott Fan (@vancepym)
- Scott Gauche (@sgauche)
- Scott Giminiani (@ScottG489)
- Scott Griffin (@scottocs11)
- Scott Gruby (@sgruby)
- Scott Henning (@shenning00)
- Scott O'Neil (@americanwookie)
- Scott Phillips (@scottyphillips)
- Scott Prive (@sprive)
- Scott Reston (@ih8gates)
- Scott Rodgers (@thegunslingers)
- Scott Walsh (@invisiblethreat)

- Scott Watermasysk (@scottwater)
- Scotte Zinn (@szinn)
- scottjones4k (@scottjones4k)
- scpotter (@scpotter)
- ScratMan (@ScratMan)
- screenagerbe (@screenagerbe)
- scyto (@scyto)
- Sdahl1234 (@Sdahl1234)
- sdekker90 (@sdekker90)
- sdelliot (@sdelliot)
- sdotter (@sdotter)
- Sean (@mcnovy)
- Sean (@mitchese)
- Sean Dague (@sdague)
- Sean Danischevsky (@seaniedan)
- Sean Gillies (@sgillies)
- Sean Gollschewsky (@gollo)
- Sean Hoyt (@deadman96385)
- Sean Kelly (@xconverge)
- Sean Kerr (@seankerr)
- Sean Leonard (@MeanderingCode)
- Sean Nyekjær (@sknsean)
- Sean Straus (@scstraus)
- sean tearney (@ispysoftware)
- Sean Vig (@flacjacket)
- Sean Wilson (@swilson)
- seanauff (@seanauff)
- seanb-uk (@seanb-uk)
- seanodell (@seanodell)
- SeanPM5 (@SeanPM5)
- seanvictory (@seanvictory)
- Seb Ruiz (@sebr)
- Sebastiaan (@sebastiaandegeus)
- Sebastiaan (@sebastiaanwezenberg)
- Sebastian (@sgso)
- Sebastian (@sebk-666)
- Sebastian Höffner (@shoeffner)
- Sebastian Kügler (@sebasje)
- Sebastian Lövdahl (@slovdahl)
- Sebastian Muszynski (@syssi)

- Sebastian Nohn (@nohn)
- Sebastian Ovide (@sebastianovide)
- Sebastian Rodriguez (@sebasrp)
- Sebastian Rutofski (@SebRut)
- Sebastian Spaeth (@spaetz)
- Sebastian von Minckwitz (@teodoc)
- sebastian-misztal (@sebastian-misztal)
- Sebastien Roy (@Sebastein)
- Sebastien Van Cauwenberghe (@svancau)
- sebdoan (@sebdoan)
- Sebest (@sebest)
- sebfortier2288 (@sebfortier2288)
- Sefan Rommel (@StefanRommel)
- sekavatar (@sekavatar)
- semenak94 (@semenak94)
- Semir Patel (@analogue)
- Senne (@Sennevds)
- sentinel-23 (@sentinel-23)
- serenewaffles (@serenewaffles)
- Sergei Vishnikin (@armicron)
- Sergej (@Sergej-Popov)
- Sergey (@volshebniks)
- Sergey Alyoshin (@alyoshin)
- Sergey Avdeev (@kasitoru)
- Sergey Isachenko (@zabuldon)
- Sergey Lanzman (@sergeylanzman)
- Sergey Lysov (@sergeylysov)
- Sergey Matyukevich (@geomatsi)
- Sergey Morozik (@morozsm)
- Sergey Rymsha (@rymsha)
- Serghei Iakovlev (@sergeyklay)
- Sergi Granell (@xerpi)
- Sergio Conde Gómez (@skgsergio)
- Sergio Gutierrez Alvarez (@arksega)
- Sergio Mayoral Martínez (@sermayoral)
- Sergio Oller (@zeehio)
- Sergio Prado (@sergioprado)
- Sergio Viudes (@sjvc)
- Sergiy Maysak (@sergeymaysak)
- Serhii Sakhno (@sdongles)

- Serj Kalichev (@pkun)
- Seth (@WillCodeForCats)
- Seth Jackson (@sethjackson)
- Sev (@collse)
- Seweryn Zeman (@cadavre)
- Sezer K (@darkson95)
- sfalkman (@sfalkman)
- sfam (@sfam)
- sfjes (@sfjes)
- sfroberg (@sfroberg)
- SgtBatten (@SgtBatten)
- sh00t2kill (@sh00t2kill)
- ShadowBr0ther (@ShadowBr0ther)
- Shai Ungar (@shaiu)
- shammysha (@shammysha)
- shanbs (@shanbs)
- Shane (@SmbKiwi)
- Shane Liesegang (@sjml)
- Shane M (@shanem2004)
- Shane Madden (@shanemadden)
- Shane Qi (@ShaneQi)
- Shane Taylor (@GrizzlyAK)
- Shantanu Tushar (@shaan7)
- Sharif Nassar (@mrwacky42)
- sharukins (@sharukins)
- Shaun McCloud (@smccloud)
- Shawn (@shawnsyhguy)
- Shawn J. Goff (@shawnjgoff)
- Shawn Oster (@shawnoster)
- Shawn Saenger (@ssaenger)
- Shawn Wilsher (@sdwilsh)
- Shawna (@cherrykoda)
- Shay G (@ShayGus)
- Shay Levy (@thecode)
- shbatm (@shbatm)
- Shiwigy (@Shiwigy)
- shker (@shker-tt)
- Shlomi Vaknin (@shlomow)
- shou72 (@shou72)
- shred86 (@shred86)

- [shrung \(@shrung\)](#)
- [shuaiger \(@shuaiger\)](#)
- [Shubham mittal \(@upgoingstar\)](#)
- [Shulyaka \(@Shulyaka\)](#)
- [Shyam Saini \(@elshyam\)](#)
- [Sian \(@Sian-Lee-SA\)](#)
- [siberx \(@siberx\)](#)
- [Sicco van Sas \(@siccovansas\)](#)
- [siebert \(@siebert\)](#)
- [Siemon Geeroms \(@siemon-geeroms\)](#)
- [SigmaPic \(@SigmaPic\)](#)
- [signaleleven \(@signaleleven\)](#)
- [Signed-off-by: Alex Xu \(@Hello71\)](#)
- [Siim Talvik \(@simpss\)](#)
- [silfa718 \(@silfa718\)](#)
- [SiliconAvatar \(@SiliconAvatar\)](#)
- [sillyfrog \(@sillyfrog\)](#)
- [silsha fux \(@silsha\)](#)
- [silversword411 \(@silversword411\)](#)
- [Simao \(@simaosimao\)](#)
- [simbaja \(@simbaja\)](#)
- [simeon-simsoft \(@simeon-simsoft\)](#)
- [simieski \(@simieski\)](#)
- [Simon \(@echox\)](#)
- [Simon \(@chatainsim\)](#)
- [Simon \(@Deadolus\)](#)
- [Simon \(@Makin-Things\)](#)
- [Simon Bumm \(@codingcyclist\)](#)
- [Simon Dawson \(@spdawson\)](#)
- [Simon Elsbrock \(@elsbrock\)](#)
- [Simon Engelhardt \(@simonengelhardt\)](#)
- [Simon Hansen \(@DurgNomis-drol\)](#)
- [Simon Hellbe \(@hellbe\)](#)
- [Simon Holzmayer \(@sholzmayer\)](#)
- [Simon Hörrle \(@CM000n\)](#)
- [Simon L. B. Nielsen \(@simonlbn\)](#)
- [Simon Lepla \(@Platzii\)](#)
- [Simon Maes \(@SimonMaes\)](#)
- [Simon Marchi \(@simark\)](#)
- [Simon Nørager Sørensen \(@simse\)](#)



- Simon Opelt (@sopelt)
- Simon Sander (@sim-san)
- Simon Szustkowski (@simonszu)
- Simon Tegelid (@simontegelid)
- Simon Vallières (@sisimomo)
- Simon van der Veldt (@simonvanderveldt)
- Simon Wüllhorst (@descilla)
- Simone (@simonewebdesign)
- Simone Chemelli (@chemelli74)
- simonmcmahon4 (@simonmcmahon4)
- SimonThoustrup (@SimonThoustrup)
- Simply Synced (@SimplySynced)
- Sindre Hansen (@sindrehan)
- sindudas (@sindudas)
- sirvictory444 (@sirvictory444)
- Sissi91 (@Sissi91)
- siyuan-nz (@siyuan-nz)
- sjabby (@sjabby)
- Sjack-Sch (@Sjack-Sch)
- sjee105 (@sjee105)
- Sjoerd (@Sjoerdfc)
- Sjors (@Sjorsa)
- sjoshi10 (@sjoshi10)
- skanab (@skanab)
- SkechyWolf (@SkechyWolf)
- skif-web (@skif-web)
- Skipperro (@Skipperro)
- Skyborg (@skyborgff)
- skycryer (@skycryer)
- Skyler Riley (@skylerisskyler)
- slamp (@slamp)
- slashback100 (@slashback100)
- Slava (@vaceslav)
- Slava Zanko (@slavaz)
- Sleeps (@MrSleeps)
- slimatic (@slimatic)
- Sly Gryphon (@sgryphon)
- sly1111 (@sly1111)
- smaggard (@smaggard)
- SmaginPV (@SmaginPV)

- smallship (@elliott-100)
- Smart Home Junkie (@smarthomejunkie)
- smarthomelawyer (@smarthomelawyer)
- smega (@smega)
- smoldaner (@smoldaner)
- smolz (@smolz)
- smonesi (@smonesi)
- smugleafdev (@smugleafdev)
- sn0oz (@sn0oz)
- snagytX (@snagytX)
- Snailboy2 (@Snailboy2)
- snarky-snark (@snarky-snark)
- sndcr (@sndcr)
- SneakSnackSnake (@SneakSnackSnake)
- snis (@snis)
- snizzleorg (@snizzleorg)
- SNoof85 (@SNoof85)
- Snuffy2 (@Snuffy2)
- so3n (@nickneos)
- SoCalix (@Socalix)
- sOckhamSter (@sOckhamSter)
- Sodbileg Gansukh (@minimaluminium)
- Sofiane Gargouri (@sofianegargouri)
- SoftXperience (@SoftXperience)
- sol (@s-ol)
- solarx (@solarx)
- Soloam (@soloam)
- Solomon Sklash (@SolomonSklash)
- soluga (@soluga)
- some-guy-in-oz (@some-guy-in-oz)
- sonicz (@sonicz)
- sophof (@sophof)
- Soren Brinkmann (@sorenb-xlnx)
- Sorin Sbarnea (@ssbarnea)
- sorinyo2004 (@sorinyo2004)
- sorryusernameisalreadytaken (@sorryusernameisalreadytaken)
- Souradip Mookerjee (@souramoo)
- Soós Péter (@soosp)
- spacemanspiff2007 (@spacemanspiff2007)
- spahlimi (@spahlimi)

- sparkydave1981 (@sparkydave1981)
- Spartan-II-117 (@Spartan-II-117)
- speedmann (@speedmann)
- spektren (@spektren)
- Spencer Oberstadt (@soberstadt)
- Spencer Owen (@spuder)
- Spencer Salisbury (@smsalisbury)
- Spencer Williams (@spencerwi)
- Spenser Gilliland (@Spenser309)
- Spiffo (@Spiffo)
- spinside (@spinside)
- splerman (@splerman)
- springstan (@springstan)
- sprocket-9 (@sprocket-9)
- spycle (@spycle)
- square99 (@square99)
- squidwardy (@villanyibalint)
- squirtbrnr (@squirtbrnr)
- Squixx (@Squixx)
- srappan (@srappan)
- srg74 (@srg74)
- Sriram Vaidyanathan (@vaidyasr)
- srirams (@srirams)
- ssenart (@ssenart)
- St. John Johnson (@stjohnjohnson)
- staa10 (@staa10)
- Stackie Jia (@stackia)
- Stanislas Bach (@sbach)
- Stanislav Bogatyrev (@realloc)
- Stanislav Vasic (@Stane1983)
- stanvx (@stanvx)
- Stany MARCEL (@ynsta)
- Staphylea (@Staphylea)
- stappel (@stappel)
- staraxis (@staraxis)
- starkillerOG (@starkillerOG)
- Stavros Korokithakis (@skorokithakis)
- stboch (@stboch)
- Steaff (@steffenslavetinsky)
- Stealth Hacker (@stealthhacker)

- steckenpferd (@steckenpferd)
- Stef Smeets (@stefsmets)
- Stefan (@stefanroelofs)
- Stefan Agner (@agners)
- Stefan Becker (@stefanb2)
- Stefan Burke (@boot-ini)
- Stefan de Lange (@langestefan)
- Stefan Fejes (@fejes713)
- Stefan Jaroschek (@jaroschek)
- Stefan Jonasson (@stefan-jonasson)
- Stefan Lehmann (@stlehmann)
- Stefan Nyffenegger (@nyffchanium)
- Stefan Rado (@kroimon)
- Stefan Sørensen (@ssorensen)
- stefan.nickl@gmail.com (@snickl)
- StefanIacobLivisi (@StefanIacobLivisi)
- stefanlod (@Talismanian)
- Stefano Scipioni (@scipioni)
- Stefano0042 (@Stefano0042)
- stefano055415 (@stefano055415)
- Steffen Dirkwinkel (@sdirkwinkel)
- Steffen Fredriksen (@Hellowlol)
- Steffen Ronalter (@ronalterde)
- Steffen Rusitschka (@rusitschka)
- Steffen Uhlig (@suhlig)
- Steffen Zimmermann (@mampfes)
- Stefán Jökull Sigurðarson (@stebet)
- stegm (@stegm)
- Steltek (@Steltek)
- Sten Spans (@sspans)
- Stepan Mazurov (@smazurov)
- Stephan (@steho)
- Stephan Auerhahn (@mpstephana)
- Stephan Beier (@stbkde)
- Stephan Grobler (@stephangrobler)
- Stephan Hoffmann (@Stephan-H)
- Stephan Thamm (@thammi)
- Stephan Traub (@sbody)
- Stephan Uhle (@StephanU)
- stephan192 (@stephan192)

- [stephanerosi \(@stephanerosi\)](#)
- [stephanfevrier \(@stephanfevrier\)](#)
- [StephanJoubert \(@StephanJoubert\)](#)
- [stephanmiehe \(@stephanmiehe\)](#)
- [StephanVinkenburg \(@StephanVinkenburg\)](#)
- [Stephen Beechen \(@sabeechen\)](#)
- [Stephen Benjamin \(@stbenjam\)](#)
- [Stephen Coogan \(@coogie\)](#)
- [Stephen Edgar \(@ntwb\)](#)
- [Stephen Eisenhauer \(@BHSPitMonkey\)](#)
- [Stephen Foskett \(@SFoskett\)](#)
- [Stephen Hoekstra \(@shoekstra\)](#)
- [Stephen Jones \(@skipishere\)](#)
- [Stephen Littman \(@anarchking\)](#)
- [Stephen Papierski \(@stephenpapierski\)](#)
- [Stephen Vanterpool \(@blackgold9\)](#)
- [Stephen Yeargin \(@stephenyeargin\)](#)
- [StephenWetzel \(@StephenWetzel\)](#)
- [Steve \(@kabongsteve\)](#)
- [Steve Bauer \(@stevejbauer\)](#)
- [Steve Brandt \(@SteveBrandt\)](#)
- [Steve Dougherty \(@Thynix\)](#)
- [Steve Dwyer \(@kangaroomadman\)](#)
- [Steve Easley \(@SteveEasley\)](#)
- [Steve Edson \(@SteveEdson\)](#)
- [Steve Herrell \(@twrecked\)](#)
- [Steve HOLWEG \(@PoppyPop\)](#)
- [Steve James \(@stevejames\)](#)
- [Steve Luzynski \(@sluzynsk\)](#)
- [Steve M \(@shmick\)](#)
- [Steve Pomeroy \(@xxv\)](#)
- [Steve Repsher \(@steverep\)](#)
- [Steve Rhoades \(@steverhoades\)](#)
- [Steve Scott \(@thewishy\)](#)
- [Steve Simms \(@ssimms\)](#)
- [Steve Thompson \(@stetho\)](#)
- [Steve9F \(@Steve9F\)](#)
- [steve-gombos \(@steve-gombos\)](#)
- [SteveDinn \(@SteveDinn\)](#)
- [Steven "NemesisRE" Koeberich \(@NemesisRE\)](#)

- Steven Adams (@daKuleMune)
- Steven Barnes (@salt-lick)
- Steven Barth (@sbyx)
- Steven Conaway (@SConaway)
- Steven D. Lander (@stevendlander)
- Steven Hosking (@onpremcloudguy)
- Steven J. Hill (@sjhill71)
- Steven Looman (@StevenLooman)
- Steven Noonan (@tycho)
- Steven Onorato (@SteveOnorato)
- Steven Rollason (@gadgetchnnel)
- Steven Webb (@cy1701)
- stevenp (@stevenp)
- Stevie Robinson (@stevietv)
- StevusPrimus (@StevusPrimus)
- Stewart Smith (@stewartsmith)
- stickpin (@stickpin)
- Stijn Tintel (@stintel)
- StopMotionCuber (@StopMotionCuber)
- strelniece (@strelniece)
- Strixx76 (@Strixx76)
- Stu Gott (@stu-gott)
- stu247 (@stu247)
- Stuart (@schford)
- Stuart Clark (@stuart-c)
- Stuart McCroden (@McCroden)
- Stuart Mumford (@Cadair)
- Stuart Pook (@stuart12)
- Stuart Summers (@stuartsummers)
- StuartW (@stuartwishart)
- stuiow (@stuiow)
- Ståle Storø Hauknes (@LaStrada)
- Stéphane BERTHELOT (@sberthelot)
- Stéphane Bidoul (ACSONE) (@sbidoul)
- Stéphane Raimbault (@stephane)
- Stéphane Veyret (@sveyret)
- Suhun Han (@ssut)
- Sumit Garg (@b49020)
- Suniel Mahesh (@sunielmahesh)
- superpuffin (@superpuffin)

- [surfermarty \(@surfermarty\)](#)
- [sustah \(@sustah\)](#)
- [sv99 \(@sv99\)](#)
- [svedese \(@svedese\)](#)
- [sveip \(@sveip\)](#)
- [Sven \(@svendroid\)](#)
- [Sven Fischer \(@svenihoney\)](#)
- [Sven Klomp \(@avanc\)](#)
- [Sven Naumann \(@sVnsation\)](#)
- [Sven Serlier \(@wrt54g\)](#)
- [Sven-Hendrik Haase \(@svenstaro\)](#)
- [sverdlin \(@sverdlin\)](#)
- [sverleysen \(@sverleysen\)](#)
- [svh1985 \(@svh1985\)](#)
- [sviau \(@sviau\)](#)
- [Swamp-Ig \(@Swamp-Ig\)](#)
- [swanwila \(@swanwila\)](#)
- [swhaat \(@swhaat\)](#)
- [Sy \(@sym0nd0\)](#)
- [Sybas \(@Sybas\)](#)
- [sycx2 \(@sycx2\)](#)
- [sylvaincherrier \(@sylvaincherrier\)](#)
- [Sylvia van Os \(@TheLastProject\)](#)
- [Synaptic149 \(@Synaptic149\)](#)
- [Syntox \(@Syntoxr\)](#)
- [System Tester \(@systemtester\)](#)
- [Sytone \(@sytone\)](#)
- [szaroubi \(@szaroubi\)](#)
- [Szymon Sakowicz \(@sakowicz\)](#)
- [Sébastien GALLET \(@bib21000\)](#)
- [Sébastien Lorber \(@slorber\)](#)
- [Sébastien Marchand \(@sebmarchand\)](#)
- [Sébastien RAMAGE \(@doudz\)](#)
- [Sérgio \(@sergioisidoro\)](#)
- [Sören \(@pattyland\)](#)
- [Sören Beye \(@Hypfer\)](#)
- [Sören Krollmann \(@Kr0llx\)](#)
- [Sören Oldag \(@soldag\)](#)
- [Søren Christian Aarup \(@scaarup\)](#)
- [Søren Dam Pedersen \(@Pengman\)](#)

- t0ny (@t0ny)
- Tabakhase (@tabakhase)
- Tadas Danielius (@tadasdanielius)
- tadly (@tadly)
- Tais Hedegaard Holland (@taisholland)
- taiyeoguns (@taiyeoguns)
- Tal Salmona (@talsalmona)
- Tal Shorer (@talshorer)
- tamueller (@tamueller)
- Tamás Nagy (@T-bond)
- Tamás Vörös (@vorostamas)
- tanderson1992 (@tanderson1992)
- TangoAlpha (@TangoAlpha)
- Tanny (@tannyl)
- tantecky (@tantecky)
- Tarunpreet Ubhi (@UbhiTS)
- Tasshack (@Tasshack)
- Tatham Oddie (@tathamoddie)
- Tathar (@Tathar)
- Tatsuyuki Ishi (@ishitatsuyuki)
- tausen (@tausen)
- Taylor Peet (@RePeet13)
- Taylor Silva (@taylorsilva)
- Taylor Vierrether (@viertaxa)
- tbergo (@tbergo)
- tbertonatti (@tbertonatti)
- tbrock47 (@tbrock47)
- tchentchen (@tchentchen)
- TD-er (@TD-er)
- tdejneka (@tdejneka)
- tdorsey (@tdorsey)
- tduffy83 (@tduffy83)
- Teagan Glenn (@Teagan42)
- Team Super Panda (@teamsuperpanda)
- technofreak74 (@technofreak74)
- techtrails (@TechTrails)
- Ted Drain (@TD22057)
- Ted Sluis (@tedsluis)
- Ted van den Brink (@tedvdb)
- tedstriker (@tedstriker)



- teejay-87 (@teejay-87)
- Teemu Mikkonen (@T3m3z)
- Teemu Patja (@tpatja)
- Teemu R. (@rytilahti)
- Teguh Sobirin (@tjstyle)
- tehbrd (@tehbrd)
- TehRobot (@TehRobot)
- Teis Angel Clausen (@AngelFreak)
- Tejpal Sahota (@GrandNewbien)
- TekniskSupport (@TekniskSupport)
- teldri (@teldri)
- teliov (@teliov)
- temap (@temap)
- temeteke (@temeteke)
- Tentoe (@Tentoe)
- Teppo Rekola (@sytem)
- Terence Eden (@edent)
- Terence Foxcroft (@tfoxcroft)
- terminet85 (@terminet85)
- terminusxx (@terminusxx)
- Terry Carlin (@terrycarlin)
- Tertius (@Tertiush)
- terual (@terual)
- tfitts (@tfitts)
- tflack (@tflack)
- tfyoung (@tfyoung)
- tggm (@tggm)
- tguerena (@tguerena)
- Thanasis (@drthanwho)
- thaohttp (@thaohttp)
- Tharsan Bhuvanendran (@thizzle)
- ThaSiouL (@ThaSiouL)
- ThaStealth (@ThaStealth)
- THATDONFC (@THATDONFC)
- The Gitter Badger (@gitter-badger)
- The Louie (@the-louie)
- The PapaMaan (@thepapamaan)
- The Witty Coder (@wittycoder)
- the-real-wizhack (@the-real-wizhack)
- thealexproctor (@thealexproctor)

- Theb-1 (@Theb-1)
- TheBelgarion (@TheBelgarion)
- Thebuz (@Thebuz)
- TheByteStuff (@TheByteStuff)
- TheCellMC (@TheCellMC)
- thecem (@thecem)
- TheDK (@TheDK)
- TheFes (@TheFes)
- TheGroundZero (@TheGroundZero)
- TheHolyRoger (@TheHolyRoger)
- thehookup (@thehookup)
- TheJulianJES (@TheJulianJES)
- thelittlefireman (@thelittlefireman)
- themanieldaniel (@themanieldaniel)
- themoffatt (@themoffatt)
- theneweinstein (@theneweinstein)
- TheNogl (@TheNogl)
- Theo Arends (@arendst)
- Theo Debrouwere (@tdebrouw)
- TheOgre (@TheOneOgre)
- theolind (@theolind)
- thepotoo (@thepotoo)
- therealryanbonham (@therealryanbonham)
- TheRedBull205 (@TheRedBull205)
- ThermIQ (@ThermIQ)
- Thiago A. Correa (@correa)
- Thiago Oliveira (@chilicheech)
- Thibault Cohen (@titilambert)
- Thibault Maekelbergh (@thibmaek)
- Thibaut (@tetienne)
- Thierry Bellocchi (@tbeloc)
- Thierry Bultel (@tbultel)
- Thijs de Jong (@thijsdejong)
- Thijs Putman (@thijsputman)
- Thijs Vermeir (@lovebug356)
- Thijs Walcarius (@wlcrs)
- thinkelastic (@thinkelastic)
- ThinkPadNL (@ThinkPadNL)
- Thom Stricklin (@thomshouse)
- Thom Troy (@ttroy50)

- Thomas (@ttuffin)
- Thomas (@a-tom-s)
- Thomas (@l3d00m)
- Thomas Augustinus (@taugusti)
- Thomas Aunvik (@ThomasAunvik)
- Thomas Barnekov (@tbarnekov)
- Thomas Baxter (@bacco007)
- Thomas Bobby (@tbobby)
- Thomas De Schampheleire (@patrickdepinguin)
- Thomas Deblock (@deblockt)
- Thomas Delaet (@thomasdelaet)
- Thomas Dietrich (@ThomDietrich)
- Thomas Ehrhardt (@tehrhardt)
- Thomas Faivre (@ThomasFaivre)
- Thomas Friedel (@tfriedel)
- Thomas G (@tomgie)
- Thomas Germain (@thomasgermain)
- Thomas Geymayer (@tomgey)
- Thomas Hervé (@therve)
- Thomas Hollstegge (@Tho85)
- Thomas Huth (@huth)
- Thomas Jaggi (@backflip)
- Thomas Johanns (@jtommi)
- Thomas Klingbeil (@thomasklingbeil)
- Thomas Krüger (@thomaskr)
- Thomas Le Gentil (@kifeo)
- Thomas Lemberger (@lemlbergerth)
- Thomas Linde (@tellerbop)
- Thomas Lovén (@thomasloven)
- Thomas Maurer (@thomasmaurer)
- Thomas Passer Jensen (@tomatpasser)
- Thomas Pedersen (@twpedersen)
- Thomas Petazzoni (@tpetazzoni)
- Thomas Prior (@ThomasPrior)
- Thomas Purchas (@thomaspurchas)
- Thomas Pötsch (@thp-comnets)
- Thomas Redmer (@Skorfulose)
- Thomas Rix (@rixth)
- Thomas Rudin (@thomasrudin)
- Thomas Ruschival (@truschival)

- Thomas Schamm (@tschamm)
- Thomas Sigmund (@ThoSig)
- Thomas Svedberg (@ThomasSvedberg)
- Thomas Wilson (@novirium)
- Thomas55555 (@Thomas55555)
- thomas-svrts (@thomas-svrts)
- ThomasADavis (@ThomasADavis)
- thomaslian (@thomaslian)
- thomasvs (@thomasvs)
- thomkaufmann (@thomkaufmann)
- Thor (@Only1Thor)
- Thorbjørn Bruarøy (@The2rB)
- Thorjan Knudsvik (@knudsvik)
- Thorsten Alteholz (@alteholz)
- Thorsten Lachmann (@tlachmann)
- Thorsten Rinne (@thorsten)
- thoscut (@thoscut)
- thrust15 (@thrust15)
- Thuan Ho (@sandwichdoge)
- ThUnD3r|Gr33n (@thundergreen)
- thunfischbrot (@thunfischbrot)
- thursday (@xthursdayx)
- tiagofreire-pt (@tiagofreire-pt)
- Tibault Damman (@DarthBo)
- Tiemoowh (@Tiemoowh)
- Tiernan (@nvx)
- Tierney Cyren (@bnb)
- Ties de Kock (@ties)
- Ties42 (@Ties42)
- tigattack (@tigattack)
- Tiger Oakes (@NotWoods)
- Tihomir Heidelberg (@9a4gl)
- Tiit Rätsep (@ratsept)
- Tijs Verkoyen (@tijsverkoyen)
- tikismoke (@tikismoke)
- Till (@microraptor)
- Till Schulte-Coerne (@tillsc)
- Till Skrodzki (@Grennith)
- tilphousia (@tilphousia)
- Tim (@tinglis1)

- Tim (@tmyoungjr)
- Tim Bailey (@TimBailey-pnk)
- Tim Clephas (@Timple)
- Tim Coninx (@timconinx)
- Tim de Boer (@tim427)
- Tim Fisher (@asktimfisher)
- Tim Gates (@tingates42)
- Tim Gerla (@tgerla)
- Tim Gray (@tgray)
- Tim Harton (@timharton)
- Tim Hordern (@mence)
- Tim Lyakhovetskiy (@tlyakhov)
- Tim McCormick (@timmccor)
- Tim Messerschmidt (@SeraphimSerapis)
- Tim Niemueller (@timn)
- Tim Rightnour (@garbled1)
- Tim Riker (@timriker)
- Tim Soderstrom (@m00dawg)
- Tim Stallmann (@tinstallmann)
- Tim Stanley (@tim-devel)
- Tim van Cann (@timvancann)
- Tim van der Heide (@timmmmmmmmm)
- Tim Werner (@tim-werner)
- Tim Wilbrink (@TWilb)
- Tim Wilde (@twilde)
- timfosse (@networkpotato)
- timkoers (@timkoers)
- Timm Schäuble (@tymm)
- Timmo (@timmo001)
- Timo J. Rinne (@rinne)
- Timo Matthias (@BudBundi)
- Timo S (@sti0)
- Timothy Kist (@Kisty)
- Timothy Lee (@timothytylee)
- Timothy Macdonald (@tsmacdonald)
- TimothyLeeAdams (@TimothyLeeAdams)
- TimV (@vroomfonde1)
- TimVa (@TimVa)
- tizzen33 (@tizzen33)
- TJ Horner (@tjhorner)

- TJ Rana (@tjrana)
- tjerkruyter (@tjerkruyter)
- tkacikdominik (@tkacikdominik)
- TKTF50 (@TKTF50)
- tktino (@tktino)
- tleegaard (@tleegaard)
- tlpeter (@tlpeter)
- tmartinez (@tmartinez69009)
- tmd224 (@tmd224)
- tmechen (@tmechen)
- tmjo (@tmjo)
- Toast (@swetoast)
- Tobi (@PepperPhil)
- Tobias (@tlindener)
- Tobias Bielohlawek (@rngtng)
- Tobias Bieniek (@Turbo87)
- Tobias Brenner (@brenner-tobias)
- Tobias Efinger (@tefinger)
- Tobias Haber (@KartoffelToby)
- Tobias Hoff (@ToSa27)
- Tobias Hutterer (@tobiashutterer)
- Tobias Klauser (@tklauser)
- Tobias Kündig (@tobias-kuendig)
- Tobias Nordahl Kristensen (@exetico)
- Tobias Perschon (@tofuSCHNITZEL)
- Tobias Sauerwein (@cgtobi)
- tobiasz256 (@tobiasz256)
- Tobie Booth (@tobiebooth)
- tobim8 (@tobim8)
- tobkim (@tobkim)
- Toby Gray (@tobygray)
- Toby Matejovsky (@tobym)
- tocklime (@tocklime)
- Tod Schmidt (@tschmidty69)
- Todd McNeal (@tmcneal)
- toddeye (@toddeye)
- Toke Høiland-Jørgensen (@tohojo)
- tokenize47 (@tokenize47)
- Tom (@CoMPaTech)
- TOM (@franc6)

- Tom (@T0mWz)
- Tom (@uphillbattle)
- Tom Barbette (@tbarbette)
- Tom Behets (@betz)
- Tom Brien (@TomBrien)
- Tom Chapin (@tomchapin)
- Tom Dickman (@tdickman)
- Tom Duijf (@tomduijf)
- Tom Harris (@teharris1)
- Tom Hennigan (@tomhennigan)
- Tom Hoover (@tomhoover)
- Tom Howe (@tomh05)
- Tom Ivar Helbekkmo (@tih)
- Tom J Nowell (@tomjn)
- Tom Kay (@TomK)
- Tom L (@Qu3uk)
- Tom Matheussen (@Tommatheussen)
- Tom Monck JR (@tmonck)
- Tom Most (@twm)
- Tom Parker-Shemilt (@palfrey)
- Tom Puttemans (@Glodenox)
- Tom Raithel (@tomraithel)
- Tom Rini (@trini)
- Tom Robinson (@tlobinson)
- Tom Schneider (@vigonotion)
- Tom Sparks (@tomasparks)
- Tom Toor (@currentoor)
- Tom Waters (@tomwaters)
- tom-winkler (@tom-winkler)
- Tomaee (@tomaee)
- Tomas Hellström (@helto4real)
- Tomas Kislán (@tkislán)
- Tomas Klouda (@johny-mnemonic)
- Tomas McGuinness (@tomasmcguinness)
- Tomasz (@Misiu)
- Tomasz Pieczykolan (@tomaszpieczykolan)
- Tomasz Wieczorek (@OldShaterhan)
- tomaszduda23 (@tomaszduda23)
- tomaszstrejczek (@tomaszstrejczek)
- tombbo (@tombbo)

- Tomek Modrzyński (@modrzew)
- Tomek985 (@Tomek985)
- Tomer (@tomer-w)
- Tomer Figenblat (@TomerFi)
- Tomi Blinnikka (@docBliny)
- Tomi Lehto (@tomilehto)
- Tomi Salmi (@tomppasalmi)
- Tomi Tuhkanen (@ttu)
- tomlut (@tomlut)
- Tommaso Marchionni (@tommasomarchionni)
- Tommi Pääkkö (@Glenf)
- TomMini (@TomMini)
- Tommy Goode (@airdrummingfool)
- Tommy Jonsson (@quazzie)
- Tommy Larsson (@larssont)
- Tommy Long (@tommyjlong)
- tomtzeng (@tomtzeng)
- Tomáš Bedřich (@tomasbedrich)
- tonire1702 (@tonire1702)
- Tontze (@Tontze)
- Tony Apuzzo (@TonyApuzzo)
- Tony Brobston (@TonyBrobston)
- Tony Carmichael (@aicarmic)
- tony chang (@idealisms)
- Tony Lindgren (@tmlind)
- Tony McDonald (@technocoffee)
- Tony Mitchell (@tonymitchell)
- Tony Nichols (@JranZu)
- Tony Phan (@phan-t)
- Tony Roman (@ronytomen)
- Tony763 (@Tony763)
- Tony-Munich (@Tony-Munich)
- Toon Willems (@nudded)
- TopdRob (@TopdRob)
- Tor Arne Vestbø (@torarnv)
- Tor Inge Redalen (@toringer)
- Tor Magnus (@tcastberg)
- Tore Amundsen (@toreamun)
- tortfeaser (@tortfeaser)
- Toshik (@Toshik)



- Totoo (@htotoo)
- Touliloup (@RiRomain)
- towerhand (@towerhand)
- travellingkiwi (@travellingkiwi)
- Travis Carr (@tmcarr)
- Travis Collins (@travatomic)
- Travis Glenn Hansen (@travisghansen)
- travislreno (@travislreno)
- trdischat (@trdischat)
- Trekky12 (@Trekky12)
- trentjw (@trentjw)
- Trenton H (@stumpylog)
- Trevin (@tmchow)
- Trevor (@tboyce021)
- Trevor Joynson (@akatrevorjay)
- Trevor North (@trvrnrth)
- Trevor Woerner (@twoerner)
- trevormiller6 (@trevormiller6)
- Trey Hunner (@treyhunner)
- Trey Sheldon (@tsheldon)
- treylok (@treylok)
- trilu2000 (@trilu2000)
- tringler (@tringler)
- Trinnik (@Trinnik)
- Tristan (@trilleplay)
- Tristan Caulfield (@Bahnburner)
- Tristan Lelong (@blunderer)
- Troels Agergaard Jacobsen (@tkjacobsen)
- Troon (@Troon)
- Troy Kelly (@troykelly)
- Troy Prelog (@tprelog)
- Trung Lê (@runlevel5)
- trvqhuy (@trvqhuy)
- Try2Fly (@Try2Fly)
- Trygve Veia (@kvisle)
- Triona (@TriploidTree)
- tsat-psv (@tsat-psv)
- tschnilo (@tschnilo)
- tscibilia (@tscibilia)
- tslpre (@tslpre)

- tstabrawa (@tstabrawa)
- tsunglung (@tsunglung)
- Tsvi Mostovicz (@tsvi)
- tubalainen (@tubalainen)
- tube0013 (@tube0013)
- tucoti (@tucoti)
- Tudor Holton (@h6w)
- Tuen Lee (@leeyuentuen)
- Tully Foote (@tfoote)
- tumik (@tumik)
- Tuncay (@Tuncay-Ayhan)
- Tungsteno74 (@Tungsteno74)
- tux2000 (@tux2000)
- Twan Coenraad (@tcoenraad)
- twdkeule (@twdkeule)
- twendt (@twendt)
- Twit123 (@Twit123)
- twodice (@twodice)
- txNginer (@txNginer)
- TychoWerner (@TychoWerner)
- tyjtyj (@tyjtyj)
- Tyler (@flamechair)
- Tyler (@TFenby)
- Tyler Bigler (@tyler-8)
- Tyler Britten (@tybritten)
- Tyler Carberry (@TylerCarberry)
- Tyler Crumpton (@tylercrompton)
- Tyler Gibson (@tylergibson)
- Tyler Shaw! (@tylershaw)
- Tyler Straub (@tylerstraub)
- Tyler Szabo (@tylerszabo)
- tzagim (@tzagim)
- tzapu (@tzapu)
- Tõnis Tobre (@tobre6)
- u8915055 (@u8915055)
- Ubaldo Porcheddu (@ubaldus)
- ubergeek801 (@ubergeek801)
- ubnt-marc-khouri (@ubnt-marc-khouri)
- uchagani (@uchagani)
- uDude (@uDude)

- ufodone (@ufodone)
- UgaitzEtxebarria (@UgaitzEtxebarria)
- Ugo Viti (@ugoviti)
- UiGuy (@jjanderson)
- Ulf Magnusson (@ulfalizer)
- Uli (@uehler)
- Ullrich Neiss (@slowflyer)
- ulnic (@ulnic)
- Ulrich Dobramysl (@ulido)
- Umer Salman (@umer936)
- Unai (@unaiur)
- Underknowledge (@Underknowledge)
- Unilogic B.V. (@unilogicbv)
- unixko (@unixko)
- UnrealKazu (@UnrealKazu)
- Unsigus (@DCSBL)
- upsert (@upsert)
- Urja Rannikko (@urjaman)
- user2684 (@user2684)
- uSlackr (@uSlackr)
- uvjim (@uvjim)
- uvjustin (@uvjustin)
- Uwe Kindler (@githubuser0xFFFF)
- Vaarlion (@Vaarlion)
- Vaclav (@bruxy70)
- vacumet (@vacumet)
- Vadim Petrov (@imposibrus)
- Vadim Titov (@vadimtitov)
- Vadym Kochan (@vkochan)
- Vadym Kochan (@vkochan-plv)
- vaidyasm (@vaidyasm)
- Valentin Alexeev (@valentinalexeev)
- Valentin Petkov (@skyval)
- Valentin VALCIU (@axiac)
- Valerio Baudo (@vabbb)
- Valiceemo (@Valiceemo)
- validev (@validev)
- vanwiegen (@vanwiegen)
- Vanya Sergeev (@vsergeev)
- Varga Tamas (@tamasv)

- Variour (@Variour)
- Vasiley (@Vasiley)
- Vasilis Koulis (@bkbilly)
- Vasily Khoruzhick (@anarsoul)
- vatir (@vatir)
- Vaughan Zeng (@vaughan-zeng)
- vauriga (@vauriga)
- Vaz Allen (@vaz)
- Vc (@Valcob)
- VDRainer (@VDRainer)
- Vedant Bhamare (@Dark-Knight11)
- Vedeneb (@Vedeneb)
- Vedran Pavic (@vpavic)
- vegascom-jeff (@vegascom-jeff)
- Vegetto (@angelnu)
- veista (@veista)
- Veldkornet (@Veldkornet)
- Veli Veromann (@velijv)
- Venkateswara Rao Mandela (@vmandela-ti)
- Ventilix (@Ventilix)
- vermium-sifell (@VermiumSifell)
- vetegrodd (@vetegrodd)
- vexofp (@vexofp)
- Vicente Bergas (@vicencb)
- Victor Cerutti (@victorcerutti)
- Victor Guimarães (@guimaraes13)
- Victor Vostrikov (@gorynychzmey)
- victorclaessen (@victorclaessen)
- Vidar Tyldum (@tyldum)
- viegelinsch (@viegelinsch)
- Viet Dzung (@dzungpv)
- Vighnesh Kadam (@vighnesh-kadam)
- Vignesh Venkat (@vickyg3)
- Vikram Gorla (@vikramgorla)
- viktak (@viktak)
- Viktor Paltsiuk (@vipals)
- Viktor Lindgren (@ViktorLindgren95)
- Viktor45 (@Viktor45)
- Ville Skyttä (@scop)
- villevirtanen (@villevirtanen)

- Villhellm (@Villhellm)
- Vilppu Vuorinen (@vilppuvuorinen)
- Vincent Dehors (@vdehors)
- Vincent Etter (@Wookai)
- Vincent Fazio (@vfazio)
- Vincent Filby (@vfilby)
- Vincent Giorgi (@vincegio)
- Vincent KHERBACHE (@vincent-k)
- Vincent Knoop Pathuis (@vpathuis)
- Vincent Le Bourlot (@vlebourl)
- Vincent Masselis (@VincentMasselis)
- Vincent Miceli (@vincemic)
- Vincent Ollivier (@vinc)
- Vincent Palmer (@shift)
- Vincent Prince (@nefethael)
- Vincent Saluzzo (@vincentsaluzzo)
- Vincent Stehlé (@vstehle)
- Vincent Teo (@vinteo)
- Vincent Van Den Berghe (@vandenberghv)
- Vincenzo Chianese (@XVincentX)
- vindaalex (@vindaalex)
- Vinicius Tinti (@tinti)
- Vinilox (@vinilox)
- Vinny Furia (@vinnyfuria)
- Viorel Stirbu (@viores)
- viperk1 (@viperk1)
- virukita (@virukita)
- viswa-swami (@viswa-swami)
- Vitalii Martyniak (@VDigital)
- Vitaly Bogdanov (@vsbogd)
- Vittorio Monaco (@vittoriom)
- VivantSenior (@VivantSenior)
- Vivien Didelot (@vivien)
- vkorenblit (@vkorenblit)
- vlack (@v1ack)
- Vlad (@vladm)
- Vlad Korniev (@vkorn)
- Vladimir Dronnikov (@dvv)
- Vladimir Eremin (@yottatsa)
- Vladimír (@machv)

- Vladimír Záhradník (@vzahradnik)
- vladonemo (@vladonemo)
- vllungu (@vllungu)
- vlumikero (@vlumikero)
- vMeph (@vMeph)
- vocweb (@vocweb)
- VoiceOfSoftware (@VoiceOfSoftware)
- Volker Krause (@vkrause)
- Volker Thiel (@riker09)
- Vova-SH (@Vova-SH)
- Voydz (@voydz)
- vrs01 (@vrs01)
- vwir (@vwir)
- vwt12eh8 (@vwt12eh8)
- Václav Kubernát (@syyyr)
- Víctor Manuel Jáquez Leal (@ceyusa)
- Víctor Martínez (@knoopx)
- w00den (@w00den)
- w35l3y (@w35l3y)
- w-marco (@w-marco)
- Wade Berrier (@wberrier)
- Wade Dorrell (@waded)
- Wadih Zaatar (@wzaatar)
- Wagner Sartori Junior (@trunet)
- Waldemar Brodkorb (@wbx-github)
- Waldemar Tomme (@WiiPlayer2)
- Walker Boyle (@walkerdb)
- Walter Huf (@hufman)
- walthowd (@walthowd)
- walzert (@walzert)
- warcanoid (@warcanoid)
- Ward Boumans (@wardboumans)
- WaRFaiR (@warfair1337)
- Warren Konkel (@wkonkel)
- Warwick Davison (@Waz-Cpt)
- wasper17 (@wasper17)
- WAStaggs (@StagasaurusRex)
- Watchfox (@Watchfox)
- waxhell (@waxhell)
- WayneZ68 (@WayneZ68)

- wbradmoore (@wbradmoore)
- wchan-ranelagh (@wchan-ranelagh)
- weado (@weado)
- web-dc (@web-dc)
- webeling67 (@webeling67)
- WebSpider (@WebSpider)
- weidi (@weidi)
- Welby McRoberts (@welbymcroberts)
- Werner Pieterse (@wernerhp)
- Wesley Chow (@wesc)
- Wesley Vos (@Wesley-Vos)
- Wesley Young (@wesdyoung)
- whhsw (@whhsw)
- WhimsySpoon (@WhimsySpoon)
- WhiteNight121 (@WhiteNight121)
- Whiterat (@Wh1terat)
- WhiteSource Renovate (@renovate-bot)
- Whytey (@Whytey)
- Wibias (@Wibias)
- Wictor (@wicol)
- wieteschmitt (@wieteschmitt)
- wiggitamoo (@wiggitamoo)
- Wilco Land (@Wilco89)
- Wilco van Duinkerken (@Sparkboxx)
- wildcomputations (@wildcomputations)
- Will Adler (@wtadler)
- Will Boyce (@wrboyce)
- Will Freeman (@TheRealWaldo)
- Will Hargrave (@will-h)
- Will Hayslett (@willhayslett)
- Will Heid (@bassclarinet12)
- Will Marler (@wmarler)
- Will Newton (@willnewton)
- Will Norris (@willnorris)
- Will Pimblett (@wjdp)
- Will W (@tiktok7)
- Will Wagner (@willw-carallon)
- Willem Burgers (@wburgers)
- Willem-Jan (@liudger)
- Willems Davy (@joyrider3774)

- [willgreenberg \(@willgreenberg\)](#)
- [willholdoway \(@willholdoway\)](#)
- [William \(@willyb321\)](#)
- [William Comartin \(@wcomartin\)](#)
- [William Hughes \(@insertjokehere\)](#)
- [William Johansson \(@radhus\)](#)
- [William Sutton \(@zombielinux\)](#)
- [William Wennerström \(@wstrm\)](#)
- [williambherman \(@williambherman\)](#)
- [williamlehman \(@williamlehman\)](#)
- [willidh74 \(@willidh74\)](#)
- [wills106 \(@wills106\)](#)
- [willscottuk \(@willscottuk\)](#)
- [Wim \(@wimb0\)](#)
- [Wim Fournier \(@hsmade\)](#)
- [Wim Haanstra \(@depl0y\)](#)
- [wind-rider \(@hansmbakker\)](#)
- [winterscar \(@winterscar\)](#)
- [Witold Sowa \(@wsowa\)](#)
- [wizmo2 \(@wizmo2\)](#)
- [wmn79 \(@wmn79\)](#)
- [WofWca \(@WofWca\)](#)
- [wogri \(@wogri\)](#)
- [Wojciech Bederski \(@wuub\)](#)
- [Wojciech M. Zabolotny \(@wzab\)](#)
- [Wojciech Mamak \(@atomic7777\)](#)
- [Wojciech Niziński \(@niziak\)](#)
- [Wojtek \(@wiuuiu\)](#)
- [wokar \(@wokar\)](#)
- [Wolf-Bastian Pöttner \(@BastianPoe\)](#)
- [Wolfgang Ettlinger \(@ettisan\)](#)
- [Wolfgang Malgadey \(@wmalgadey\)](#)
- [WolfRevo \(@WolfRevo\)](#)
- [Wopalecki \(@Wopalecki\)](#)
- [Wouter \(@wouterpotters\)](#)
- [Wouter Schoot \(@wschoot\)](#)
- [Wouter van der Wal \(@wjtje\)](#)
- [Wouter van Os \(@Wouter0100\)](#)
- [Wouter Wolkers \(@wwolkers\)](#)
- [wouterbaake \(@wouterbaake\)](#)



- [wovka88 \(@wovka88\)](#)
- [wranglatang \(@wranglatang\)](#)
- [wrkn4alivn \(@wrkn4alivn\)](#)
- [Wuppie007 \(@Wuppie007\)](#)
- [wyapx \(@wyapx\)](#)
- [wyzbv \(@wyzbv\)](#)
- [wyznerd \(@wyznerd\)](#)
- [xabivaz \(@spanishkangaroo\)](#)
- [xander2 \(@xander2\)](#)
- [xannor \(@xannor\)](#)
- [Xavi Moreno \(@xaviml\)](#)
- [Xavier Damman \(@xdamman\)](#)
- [Xavier Decuyper \(@Savjee\)](#)
- [Xavier Ruppen \(@zkrx\)](#)
- [xdite \(@xdite\)](#)
- [XECDesign \(@XECDesign\)](#)
- [Xeevis \(@Xeevis\)](#)
- [xelprep \(@xelprep\)](#)
- [xfceKris \(@xfceKris\)](#)
- [Xiaobing Luo \(@0lxb\)](#)
- [Xiaonan Shen \(@shenxn\)](#)
- [xifle \(@xifle\)](#)
- [Xinglong Liao \(@LiaoXinglong\)](#)
- [xLarry \(@xLarry\)](#)
- [xlcnd \(@xlcnd\)](#)
- [xonestonex \(@xonestonex\)](#)
- [Xor \(@tgl0be\)](#)
- [xorbital \(@xorbital\)](#)
- [Xorso \(@Xorso\)](#)
- [xpac1985 \(@xpac1985\)](#)
- [xpavli44 \(@xpavli44\)](#)
- [xPsIXx \(@xPsIXx\)](#)
- [xs4 \(@xs4\)](#)
- [xTCx \(@asafbiton96\)](#)
- [Xuefer \(@xuefer\)](#)
- [xuminready \(@xuminready\)](#)
- [Xus Badia \(@XusBadia\)](#)
- [y34hbuddy \(@y34hbuddy\)](#)
- [Yair Ben Avraham \(@yairbenavraham\)](#)
- [yakirk \(@yakirk\)](#)

- yangqian (@yangqian)
- yankees9920 (@wagnerbenh)
- Yann Droneaud (@ydroneaud)
- Yann E. MORIN (@yann-morin-1998)
- Yann E. MORIN (@ymorin-orange)
- Yann Jajkiewicz (@yjajkiew)
- Yann Sionneau (@fallen)
- Yannic-HAW (@Yannic-HAW)
- Yannick Brosseau (@greenscientist)
- Yannick KERMAREC (@YanK-fr)
- Yannick Kiekens (@YannickKiekens)
- Yannick POLLART (@ypollart)
- Yannick Simard (@TheRaven)
- Yannik Ache Eicher (@thegnuu)
- Yannik25 (@Yannik25)
- yanuino (@yanuino)
- Yariv Amar (@yarix)
- Yarmo Mackenbach (@YarmoM)
- Yaron de Leeuw (@jarondl)
- Yaroslav (@Yarikx)
- Yaroslav Syrytsia (@joy4eg)
- yasin (@yasinS)
- Yasser Saleemi (@yasn77)
- Yaw Anokwa (@yanokwa)
- yayitazale (@yayitazale)
- YcKe (@YcKe)
- yeah (@yeah)
- yeahme49 (@yeahme49)
- Yegor Vialov (@estevez-dev)
- Yegor Yefremov (@yegorich)
- Yehuda Davis (@hudcap)
- Yeon Vinzenz Varapragasam (@YeonV)
- YesterKo (@YesterKo)
- Yevgeniy (@Yevgenium)
- Yevhenii Vaskivskyi (@Vaskivskyi)
- Yi Zheng (@goodmenzy)
- Yien Xu (@yienxu)
- yllar (@yllar)
- Yllelder Bamir (@Yllelder)
- yniezink (@yniezink)

- [yoedf \(@yoedf\)](#)
- [Yogotech's GitHub account \(@YogoGit\)](#)
- [yosilevy \(@yosilevy\)](#)
- [Yossi Frances \(@yfrans\)](#)
- [youdroid \(@youdroid\)](#)
- [youknowjack0 \(@youknowjack0\)](#)
- [Younes Manton \(@ymanton\)](#)
- [Yu \(@GuryYu\)](#)
- [Yuchen Ying \(@yegle\)](#)
- [Yue Kang \(@kangyue92\)](#)
- [YujiTFD \(@YujiTFD\)](#)
- [Yuki Ueda \(@Ikuyadeu\)](#)
- [Yukon Vinecki \(@Yukon\)](#)
- [yulongying \(@yulongying\)](#)
- [Yum \(@goofz\)](#)
- [Yuri Dogandjiev \(@ydogandjiev\)](#)
- [Yurii Monakov \(@monakov-y\)](#)
- [Yuriy Sannikov \(@yury-sannikov\)](#)
- [Yury Kotlyarov \(@yura505\)](#)
- [Yuval \(@rt400\)](#)
- [Yuval Aboulafia \(@yuvalabou\)](#)
- [Yuxiang Zhu \(@vfreex\)](#)
- [Yuxin Wang \(@yuxincs\)](#)
- [Z \(@zanerv\)](#)
- [z00nx 0 \(@z00nx\)](#)
- [z0p \(@z0mbieprocess\)](#)
- [Zac \(@zacs\)](#)
- [Zac West \(@zacwest\)](#)
- [Zac-HD \(@Zac-HD\)](#)
- [Zach \(@ZacheryThomas\)](#)
- [Zach \(@snowzach\)](#)
- [Zach Berger \(@zachberger\)](#)
- [Zachary Priddy \(@zpriddy\)](#)
- [Zack Arnett \(@zsarnett\)](#)
- [Zack Lalanne \(@zlalanne\)](#)
- [Zack R \(@OspreyPrey\)](#)
- [zackbcom \(@zackbcom\)](#)
- [zacpotts \(@zacpotts\)](#)
- [Zadkiel Aharonian \(@aslafy-z\)](#)
- [zajnic \(@zajnic\)](#)

- Zak (@zemerick1)
- Zane Riley (@zaneriley)
- Zapfmeister (@Zapfmeister)
- zaubererty (@zaubererty)
- zbeky (@zbeky)
- Zeb Palmer (@zebpalmer)
- ZeGuigui (@ZeGuigui)
- Zellux Wang (@zellux)
- zeltom (@zeltom)
- Zen Tormey (@xehn)
- Zen3515 (@Zen3515)
- zeng5200 (@zeng5200)
- Zenichi Amano (@crow-misia)
- Zep Fietje (@zepfietje)
- Zephyr (@JeromeZephyr)
- ZERBIB Mickael (@Hellorheaven)
- Zero King (@l2dy)
- zetvio (@zetvio)
- zewelor (@zewelor)
- zgmknv (@zgmknv)
- zgyarmati (@zgyarmati)
- Zhao (@zhaokoh)
- Zhao Lü (@zlu)
- Zhong Jianxin (@azuwis)
- zhoubest (@zhoubest)
- zhujisheng (@zhujisheng)
- zhumuht (@zhumuht)
- zigul (@zigul)
- zimmra (@zimmra)
- zinxes (@zinxes)
- Zio Tibia (@ziotibia81)
- zipperten (@zipperten)
- Zippit (@zippit)
- ZiroNL (@ZiroNL)
- Ziv (@ziv1234)
- Zixuan Wang (@TheNetAdmin)
- zmarties (@zmarties)
- zmrowicki@hotmail.com (@zmrow)
- Zoe 🌟 (@zoeisnowooze)
- Zoltán Tóth (@toth2zoltan)

- [zoomix \(@zoomix\)](#)
- [Zorks \(@Zorks\)](#)
- [Zotz \(@datdamnzotz\)](#)
- [Zoé Bôle \(@zoe1337\)](#)
- [zpetr \(@zpetr\)](#)
- [zraken \(@zraken\)](#)
- [ZS \(@ZsBT\)](#)
- [ztamas83 \(@ztamas83\)](#)
- [zuccs \(@zuccs\)](#)
- [ZuluWhiskey \(@ZuluWhiskey\)](#)
- [zvldz \(@zvldz\)](#)
- [zyell \(@Zyell\)](#)
- [ZzetT \(@ZzetT\)](#)
- [Álvaro Brey \(@AlvaroBrey\)](#)
- [Álvaro Fernández Rojas \(@Noltari\)](#)
- [Åke Strandberg \(@astrandb\)](#)
- [Åskar Andersson \(@olskar\)](#)
- [Édouard Denommée \(@MoronixProduct3\)](#)
- [Øystein Hansen \(@oeysteinhansen\)](#)
- [Øyvind Matheson Wergeland \(@oyvindwe\)](#)
- [Ġirts \(@girtskokars\)](#)
- [Łukasz Szeremeta \(@lszeremeta\)](#)
- [Šimon Let \(@curusarn\)](#)
- [Дубовик Максим \(@lufton\)](#)
- [Илья Рупко \(@ILAsoft\)](#)
- [没事溜达 \(@meishild\)](#)

This page is irregularly updated using the `hass-release` tool. If you think that you are missing, please let us know.

*This page was last updated Wednesday, November 16 2022, 14:21:10 UTC.*

---

## Developers/License

---

The Home Assistant source code is released under the following license.

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.
3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.
4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:
  - (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
  - (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
  - (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
  - (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.



You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.
6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.
7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.
8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.
9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "{}" replaced with your own identifying information. (Don't include

the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright {yyyy} {name of copyright owner}

Licensed under the Apache License, Version 2.0 (the "License");  
you may not use this file except in compliance with the License.  
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software  
distributed under the License is distributed on an "AS IS" BASIS,  
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
See the License for the specific language governing permissions and  
limitations under the License.

---

## Faq/Index

---

This is a community curated list of frequently asked questions (FAQ) about the installation, setup, and usage of Home Assistant. If you want to get details about a term, please check the [glossary](#).

{% endfor %}

---

## Getting Started/Automation

---

Now that your devices are connected, you can put them to work. Automations let your home react to things on its own: turn the lights on when the sun goes down, lower the heat when everyone leaves, or remind you to close the garage door at bedtime. You build them with the visual automation editor, so you can follow this tutorial without writing a single line of code.

We are going to create two automations together: one that turns the lights on as the sun sets, and a second that dims them later in the evening on workdays.

# Turning on the lights before sunset

---

## Prerequisites

This tutorial assumes the following:

- You have [installed Home Assistant](#)
- You have completed the [onboarding steps](#)
- You have followed the steps on [adding an integration](#)
- You have a light that is integrated into Home Assistant
- If you don't have a light yet, and are unsure what to buy, try [Philips Hue](#), [nanoleaf](#), or products supporting [WLED](#)

## To automatically turn on the lights before sunset

1. Go to **Settings > Automations & scenes** and select **Create Automation**.

The automation editor.

2. Then, select **Create new automation**. This brings up an empty automation page.

The start of a new automation.

3. The first step is defining what should trigger the automation to run.

4. In this case, we want to use the event of the sun setting to trigger our automation.

5. Select **Add trigger**, type **Sun** and select it. Use the sun as trigger.

6. Select **Sunset**.

7. We want the automation to be triggered a little before that, so let's add **-00:30** as the offset.

This indicates that the automation will be triggered 30 minutes before sunset. Neat!

A new automation with a sun trigger filled in.

8. Once we have defined our trigger, we need to define what should happen.

9. Select **Add action**.

10. Type **light** and select **Light turn on**.

11. For this automation, we're going to turn on all lights in the living room, so let's select the **Area**.

12. This only works if your lights are assigned to an area.

13. To learn more about grouping devices in areas, refer to the [area documentation](#).

A new automation with the action set up to turn on the lights in the living room.

1. To save the automation, select **Save**. Give the automation a name, add a **Description**, and **Save** again.

2. When choosing a name, be specific, so that you can find it even when you have many automations. For example, **Turn on living room table light at sunset**.

3. Now wait until it's 30 minutes before sunset and see your automation magic!

4. Or follow these steps to [test your automation](#) right away.

# Dimming the lights the night before a workday

---

This automation dims the light at a specific time before a workday.

## Prerequisites

This tutorial assumes the following:

- You have [installed Home Assistant](#)
- You have completed the [onboarding steps](#)
- You have followed the steps on [adding an integration](#)
- You have a dimmable light that is integrated into Home Assistant

## To dim the light the night before a workday

1. Go to **Settings > Automations & scenes** and select **Create automation**.

The automation editor.

2. Then, select **Create new automation**. This brings up an empty automation page.

The start of a new automation.

3. We want the light to start dimming at 21:45. This means we want an automation that is triggered by time.

4. Select **Add trigger > Time and location > Time**.

5. Select **Fixed time** and enter the time.

A new automation with a fixed time trigger filled in.

6. We want to do this only if tomorrow is a workday.

7. Select **Add condition > Entity > State**.

8. Under **Entity**, enter `workd` and select your workday sensor.

9. Under **State**, select **On**.

10. Next, we want to make sure the light is only dimmed when it is actually on. No reason to do this if the light is not on.

11. To achieve this, we use an **If-then** action. Select **Add action > Building blocks > If-then**.

12. You now get a block called **Conditionally execute an action**. From the **Entity** list, select your light.

13. Under **If**, select **Add condition > Entity > State**.

14. Under **State**, select **On**.

Screenshot showing the if section of an if-then action

15. Now we want to define the action that is performed when the condition is true (when the light was on).

16. Under **Then**, select **Add action > Light turn on**.

17. Under **Entity**, select your light.

18. Define the light settings, such as brightness, temperature, or color. The available settings depend on your light.

Screenshot showing the then section of an if-then action

19. To save the automation, select **Save**. Give the automation a name (for example, `dim living room table light night before workday`), add a **Description**, and **Save** again.

20. [Test your automation](#).

[Include not found: getting-started/next\_step.html step="Presence detection" link="/getting-started/presence-detection/"]

If after completing this getting started you are interested in reading more about automations, we recommend the following pages:

- [Triggers](#)
- [Conditions](#)
- [Actions](#)

Please note, these pages may require a bit more experience with Home Assistant than you probably have at this point of this tutorial.

---

## Getting Started/Concepts Terminology

---

Now that you are in Home Assistant, let's look at the most important concepts. Home Assistant is built around a small set of building blocks: integrations, devices, entities, areas, and automations. Once you understand how they fit together, the rest of the platform falls into place.



# Integrations

---

Integrations are pieces of software that allow Home Assistant to connect to other software and platforms. For example, a product by Philips called Hue would use the Philips Hue integration and allow Home Assistant to talk to the hardware controller Hue Bridge. Any Home Assistant compatible devices connected to the Hue Bridge would appear in Home Assistant as [devices](#).

## Integrations

Some integration cards show an icon:

- The globe icon Globe icon indicates that this integration requires the internet.
- The file icon Configuration file icon indicates that this integration was not set up via the UI. You have either set it up in your `configuration.yaml` file, or it is a dependency set up by another integration. If you want to configure it, you will need to do so in your `configuration.yaml` file.
- The custom icon Custom icon indicates that this is not an official Home Assistant integration but that it was custom made. It could be imported from another source, for example downloaded from HACS.

For a full list of compatible integrations, refer to the [integrations](#) documentation.

Once an integration has been added, the hardware and/or data are represented in Home Assistant as [devices and entities](#).

## Devices

---

Devices are a logical grouping for one or more entities. A device may represent a physical device, which can have one or more sensors. The sensors appear as entities associated with the device. For example, a motion sensor is represented as a device. It may provide motion detection, temperature, and light levels as entities. Entities have states such as *detected* when motion is detected and *clear* when there is no motion.

Home Assistant device

Devices and entities are used throughout Home Assistant. To name a few examples:

- [Dashboards](#) can show a state of an entity. For example, if a light is on or off.
- An [automation](#) can be triggered from a state change on an entity. For example, a motion sensor entity detects motion and triggers a light to turn on.
- A predefined color and brightness setting for a light saved as a [scene](#).

Home Assistant device

## Entities

---

Entities are the basic building blocks to hold data in Home Assistant. An entity represents a sensor, actor, or function in Home Assistant. Entities are used to monitor physical properties or to control other entities. An entity is usually part of a device or a service. Entities have states.

Screenshot showing the Entities tableScreenshot of the Entities table. Each line represents an entity.

## Areas

---

An area in Home Assistant is a logical grouping of devices and entities that are meant to match areas (or rooms) in the physical world: your home. For example, the `living room` area groups devices and entities in your living room. Areas allow you to target service calls at an entire group of devices. For example, turning off all the lights in the living room. These are locations within your home, such as the living room or the dance floor. Areas can be assigned to floors. Areas can also be used for automatically generated cards, such as the [Area card](#).

# Automations

---

A set of repeatable actions that can be set up to run automatically. Automations are made of three key components:

1. Triggers - events that start an automation. For example, when the sun sets or a motion sensor is activated.
2. Conditions - optional tests that must be met before an action can be run. For example, if someone is home.
3. Actions - interact with devices such as turn on a light.

To learn the basics about automations, refer to the [automation basics](#) page or try [creating an automation](#) yourself.

Automations

## Scripts

---

Similar to automations, scripts are repeatable actions that can be run. The difference between scripts and automations is that scripts do not have triggers. This means that scripts cannot automatically run unless they are used in automations. Scripts are particularly useful if you perform the same actions in different automations or trigger them from a dashboard. For information on how to create scripts, refer to the [scripts](#) documentation.

Scripts

## Scenes

---

Scenes allow you to create predefined settings for your devices. Similar to a driving mode on phones, or driver profiles in cars, it can change an environment to suit you. For example, your *watching films* scene may dim the lighting, switch on the TV and increase its volume. This can be saved as a scene and used without having to set individual devices every time.

To learn how to use scenes, refer to the [scene](#) documentation.

Scenes

## Apps

---

Apps are third-party applications that provide additional functionality in Home Assistant. Apps run directly alongside Home Assistant, whereas integrations connect Home Assistant to other apps. Apps are only [supported](#) when using Home Assistant Operating System.

Apps are installed from the app store under **Settings > Apps**. If you are curious now and feel like installing every app that looks interesting: beware that apps can use quite a bit of resources in terms of disk space, memory, and additional load on the processor.

Among the most used apps are the ones that provide [file access and edit files](#) in Home Assistant.

Screenshot of the app store, showing all the installable appsScreenshot of the app store, showing all the installable apps.

[Include not found: getting-started/next\_step.html step="Edit the dashboard" link="/getting-started/onboarding\_dashboard/"]

---



## Getting Started/Configuration

---

Onboarding got you the basics: a working Home Assistant, your home's name and location, your first integrations. From here, your smart home is yours to shape. This page points you to a few of the next things most people do once they have Home Assistant running, so you can pick whichever one matters most to you and pace yourself.

## Adding other persons to Home Assistant

---

You can add other people to Home Assistant. They can have their own login, use Home Assistant on their devices and create their own dashboards. To add other people, refer to [Adding a person to Home Assistant](#).

## Apps for Android and iOS

---

You can use Home Assistant on your phone, smartwatch, and even in your car.

- To learn how to install Home Assistant on Android or iOS, refer to the [documentation for the Companion Apps](#).
- Want to use your voice to control Home Assistant?
- Refer to the documentation on using [Assist on Android](#).

## Changing the basic settings

---

To change basic settings such as location, unit system, and language, refer to [Changing basic settings](#).

## Creating a backup

---

You can back up your Home Assistant configuration and Home Assistant app data. Backups are used to restore the system (or parts of the system) if a rollback is needed. Backups are also used to migrate your Home Assistant to new hardware. It is good practice to create a backup before updating.

To learn how to create a backup of your Home Assistant installation, refer to the documentation on [creating a backup](#).

## Editing the configuration.yaml and configuring file access

---

While you can configure most of Home Assistant from the user interface, for some integrations, you will need to [edit the `configuration.yaml` file](#). This file contains integrations to be loaded along with their configurations. Throughout the documentation, you will find snippets that you can add to your configuration file to enable specific functionality. For starters, there is no need to edit the `configuration.yaml` file. You will be pointed to the documentation when this is needed.

## Setting up network storage

---

If you need more space to store data, you can configure a [network storage](#), for example to store backups or to access media.

## Getting started with voice assistant

---

If you want to get started with a voice assistant, refer to the documentation on [Assist](#).

---



## Getting Started/Index

---

Welcome to Home Assistant. This guide walks you through everything you need to get up and running, from picking your hardware to creating your first automation. There are no servers in the cloud and no hidden subscriptions: Home Assistant runs on your own hardware, in your own home, and your data stays with you.

You do not need to write code or edit configuration files to follow along. Almost everything is done from the user interface. The whole journey, from a fresh install to a working smart home, takes most people about an afternoon.

[Include not found: getting-started/next\_step.html step="Installation" link="/installation/" icon="simple-icons:homeassistant"]

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/" icon="fluent-mdl2:onboarding"] [Include not found: getting-started/next\_step.html step="Concepts and terminology" link="/getting-started/concepts-terminology/" icon="material-symbols:dictionary"] [Include not found: getting-started/next\_step.html step="Edit the dashboard" link="/getting-started/onboarding\_dashboard/" icon="mdi:view-dashboard-edit"] [Include not found: getting-started/next\_step.html step="Integration" link="/getting-started/integration/" icon="mdi:devices"] [Include not found: getting-started/next\_step.html step="Automation" link="/getting-started/automation/" icon="mdi:robot-happy"] [Include not found: getting-started/next\_step.html step="Presence detection" link="/getting-started/presence-detection/" icon="mdi:motion-detector"] [Include not found: getting-started/next\_step.html step="Join the community" link="/getting-started/join-the-community/" icon="mdi:people-group-outline"] [Include not found: getting-started/next\_step.html step="Next steps" link="/getting-started/configuration/" icon="mdi:cog"]

---

## Getting Started/Integration

---

Let's start by adding your first integration. Home Assistant works with devices and services from over a thousand brands, and most of them can be added through the user interface in just a few steps.

In this tutorial, we will use the **Workday** integration. It can be used to automate based on workdays, days off, or holidays. No smart device is needed for this tutorial.

## Prerequisites

---

This tutorial assumes that you have [installed Home Assistant](#) and have at least completed the [onboarding steps](#).

## Adding integrations

---

1. Go to **Settings > Devices & services**.
2. The integrations page shows all the integrations you have already installed. Some of them were installed automatically.
3. If devices were discovered in your network, you will see them in the **Discovered** section.

Screenshot of the integrations page, with discovered devices

1. If there are any devices discovered for you, you can add them now.
2. Under **Discovered**, on the integration, select **Add**.
3. Follow the steps in the UI if additional configuration is required.
4. If no devices were discovered, don't worry, we will add an integration in the next step.
5. In the bottom-right corner, select **Add integration**.
6. Type `workd` and select the **Workday** integration.

Screenshot of the add integrations dialog

1. Give it a name, for example `Workday tomorrow`, and select the country.
2. The country is used to determine the local Holidays.
3. Select **Submit**, then **Finish**.
4. Configure the options.
  - For example, if Monday is not a workday for you, select the "x" to remove it.
  - To check if tomorrow is workday, under **Offset**, enter `1`.
  - Fill in all other options as needed. At a minimum, define the **Holidays** and **Language**.
  - Select **Submit**.

Screenshot of the configuration options 7. Select the area, for example, office, and select **Finish**.

8. You now see the **Workday** integration in the list. - `mdi:party-popper` Congratulations! You've added your first integration. Job done.

Screenshot of the workday integration on the integrations page

## Looking at integration details

---

1. Select the integration.

Screenshot of the workday integration on the integrations page

- This opens the integration entity page.
- Select an entity.

Screenshot of the workday integration entity list

- We see that this integration has two entities.
- A list of related automations, script and scenes.
- Activity of updates/changes related to the entity.

Screenshot of the workday integration details page

## Modifying the integration

---

1. To change the name, select the pencil `mdi:pencil` on the top right.
2. You can also add another Workday sensor. For example, if you want to know when your colleagues have a holiday.
3. Go back to the integration entity page.
4. Select **Add entry**, give it a name and define your options.
5. Select the country of interest.
6. That's it! `mdi:party-popper`
7. `mdi:checkbox-outline` You have gained an overview of the integrations page and know where to find the integration details page, the sensor info page, and the entities table.
8. `mdi:checkbox-outline` You have learned to rename, modify, and delete an integration.
9. If you want to find more integrations, checkout the [integration documentation](#).
10. We are now ready to use **Workday** in an automation.

[Include not found: getting-started/next\_step.html step="Automate Home Assistant" link="/getting-started/automation/"]

---

## Getting Started/Join The Community

---

You made it. You have Home Assistant installed and you have had a first taste of what it can do. From here, the best resource you have is not the documentation, it is the people who use Home Assistant every day.

A worldwide community of Home Assistant users hangs out on the forum and in the chat, all of them happy to help you figure things out, share what works in their homes, and trade automation ideas. Whatever you are trying to build, somebody has probably already done something close to it.

Here are some good places to start:

- Read up on the [next steps](#) of the getting started guide, like creating a backup or setting up network storage.
- Join the conversation on [our forum](#) or in [our chat](#).
- Browse [video tutorials](#) for almost any Home Assistant topic you can think of.

Welcome to the community.

---

## Getting Started/Onboarding

---

After Home Assistant has been [installed](#) on your device, there are 5 steps to complete setting up Home Assistant. The entire onboarding takes only a few minutes and is done in your browser, so no command-line or coding is required.

1. Enter the following URL into the browser's address bar: <http://homeassistant.local:8123/>.
2. Result: You now see the **Preparing Home Assistant** page. Depending on your hardware and internet connection, preparation may take a while.

- Home Assistant downloads the latest version of Home Assistant (about 700 MB).

3. If you ran into issues with this step, refer to the [installation troubleshooting](#).

4. Once preparation is finished, the welcome screen is shown.

Home Assistant preparation

5. You can either create a new installation or recover an existing installation from a backup:

6. **Option 1: new installation:** If this is your initial installation, we will now create the owner's account of Home Assistant.

- `mdi:information-outline` This account is an administrator account. It will always be able to change everything.

- Select **Create my smart home**.

- Enter a name, username, and password.

- Make sure the username is lowercase and contains no whitespace.

- `mdi:info` **Info:** The **Name** is the name of the person that is shown in the UI. The username is used for login.

Set your username and password. - Store the name, username, and password somewhere safe. There is no way to recover the owner credentials. - Select **Create account**.

7. **Option 2: restore from backup:** If you want to restore a backup of a previous installation, follow the steps on [restoring from backup](#).

- If you have a Home Assistant Yellow, follow the [Yellow documentation on restoring from backup](#).

- If you have a Home Assistant Green, follow the [Green documentation on restoring from backup](#).

8. Enter the location of your home.

9. The home location is used to configure the time zone, unit system, and currency.

10. It is also used to create the home [zone](#), which designates the area of your home with a default radius of 100 m.

11. You can always change this information later in the settings.

12. This home zone can be used for automations such as showing the weather, opening the shades at sunrise, or starting the vacuum when you leave the home.



13. After finding your location, select **Next**.

Define your location.

14. Select which information you are willing to share.

- Sharing is disabled by default. However, we would like to encourage you to share some of this data.
- This information helps us find out which platforms we need to support and where to focus our efforts.
- The data is anonymized and aggregated. To see the charts we generate out of this data, take a look at our [analytics page](#).
- To confirm, select **Next**. Share anonymized data

15. `mdi:party-popper` You've now got Home Assistant up and running.

16. Press **Finish** and you now see the default [dashboard](#).

[Include not found: getting-started/next\_step.html step="Concepts & Terminology" link="/getting-started/concepts-terminology/"]

---

## Getting Started/Onboarding Dashboard

---

Dashboards are how you see and control your smart home in Home Assistant. They are made up of cards and views, and you can build them visually with drag and drop, so no coding is required.

## Types of dashboards

---

In the left sidebar, you see the names of different dashboards. Home Assistant comes with different dashboard types out of the box, such as:

- Overview
- Energy
- Map
- Activity
- History
- To-do lists

## Elements of a dashboard

---

The main elements of a dashboard are cards and views.

A card is an element of the User Interface that shows information about a smart device, a service, or an entity and even allows you to control them, depending on the [category of the card](#). For more information on cards and how to edit them, refer to the [Cards](#) page.

A view is a tab inside a dashboard that displays cards in a specific layout. The layout is determined by the view type. A dashboard can have one or many views and, for each view, you can choose the cards that you want to display, and in which view type. For more information on views and how to edit them, refer to the [Views](#) page.

The cards in the following screenshot are displayed in a [sections view](#) of a dashboard. In the upper menu bar, each icon represents a different tab, that is, a view.

A fully populated dashboard in sections view layout A fully populated dashboard in the sections view layout

## First contact with the Overview dashboard

---

The **Overview dashboard** is the first page you see after the [onboarding process](#).

If you just onboarded, your dashboard will be nearly empty. It has the [sections view](#) layout and shows cards for devices that were detected automatically.

The following cards appear automatically on your **Overview** dashboard:

- An [entities](#) card of the person defined as the Home Assistant owner. It presents the name and the state of the person.
- You can track whether a person is present or not and create automations based on that. For example, turn down the heating when everyone has left home. For more information on automations based on presence, start with [presence detection](#).
- A [weather forecast](#) card of the weather for your location, if you provided it during onboarding.

Your dashboard may look quite different, depending on the smart devices that you have at home. For example, if you have the following devices, they will be detected:

- A smart speaker connected to Wi-Fi, such as a Sonos speaker.
- Result: a [media control](#) card will be displayed on your **Overview** dashboard.
- Bluetooth temperature sensors and a Bluetooth module in Home Assistant.
- Result: [entities](#) cards will be shown on your **Overview** dashboard.
- Note that, if your Home Assistant does not have a Bluetooth module yet, the Bluetooth devices that you have in your home won't be shown automatically.
- Light sensors.
- Result: [entities](#) cards will be shown on your **Overview** dashboard with the status of some lights.

If your Home Assistant has other controllers, such as a [Zigbee](#) or a [Z-Wave](#) controller, and you have Zigbee or Z-Wave devices, these could be detected and shown on the **Overview** dashboard. There you will have cards representing control elements that allow you, for example, to:

- Change the ventilation.
- Change the color of the lights.
- Turn on a smart TV and start YouTube.

However, these devices usually need to be paired first.

## Creating a new dashboard

---

The default **Overview** dashboard updates automatically when you add new devices. However, once you start editing the default dashboard, it no longer updates automatically. For this reason, we start here by adding a new dashboard. This lets us keep the default **Overview** dashboard.

Follow the steps in [Creating a new dashboard](#).

## Editing cards in a new dashboard

---

This section describes how to edit cards in a recently created dashboard, namely how to:

- Change the details of a weather forecast card.
- Add a new weather forecast card.
- Change the position of cards.
- Open your new dashboard. It might not have much on it yet.
- If you have smart home devices in your home, some may have been connected automatically.
- Some cards are there by default, such as the weather forecast card, and a card for the person who set up the system.
- To edit the weather forecast card, for example, select it and then select the cogwheel `mdi:cog-outline`.
- Change any of the units or other details, such as name and icon, if you like.
- Do not change the **Entity ID**.
- Once you are done, select **Update**.
- For the next activities, you need to edit your dashboard. In the top right of the screen, select the `mdi:pencil` button.
- In the **Edit dashboard** dialog, go to the three dots `mdi:dots-vertical` menu, and then select **Take control**.
- Note that, by editing the dashboard, you are taking control over it. The dashboard will no longer update automatically when new entities or dashboard components are available. You can't revert this. However, you can create a new default dashboard.
- Read the text in the dialog and if you agree, select **Take control**.
- Now you can add a new card for this weather service.
- Select the weather forecast card again, go to the three dots `mdi:dots-vertical` menu, and then select **Service info**.
- Under **Sensors**, select **Add to dashboard**.
- In the **Choose a view** dialog, select your dashboard from the dropdown list, and then select **Next**.
- In the suggestion dialog, select **Pick different card**.
- On the **By card** tab of the dialog, select the **Weather forecast** card.
- In the **Weather forecast card configuration** dialog:
  - Select the details to be shown on the card, and then select **Save**.
  - Go back to the edit window, and select **Done**.
- Result: You see the new weather forecast card on the dashboard.
- If you want to delete the other weather forecast card from the dashboard:
- In the top right corner, select the `mdi:pencil` button to go back to the edit mode.

- Do not select the card. Go to the three dots `mdi:dots-vertical` menu in the lower right corner of the card, and then select **Delete**.
- Finally, to move the weather forecast card to the top left corner:
  - On the bottom of the card, select the number or use the minus button to enter `1`.
  - Change the number on the other cards, if you want to move them around.
  - When you are done, in the top right corner, select **Done**.
- If you want to change the configuration of another card, select the `mdi:pencil` button again, and then select **Edit** on the card.

Congratulations! You have completed your first dashboard customization.



## Learning more about dashboards

---

If you want to learn more about dashboards, views, and cards, take a look at these topics:

1. Take a look at the [introduction to dashboards](#) and learn about [dashboard types](#).
2. Learn more about [view types](#)
3. Learn how to [add cards](#) to a view.

## Next step: integrations

---

To continue with this tutorial, select the button below to learn about integrations.

[Include not found: getting-started/next\_step.html step="Integrations" link="/getting-started/integration/"]

## Related topics

---

- [Dashboards](#)
  - [Views](#)
  - [Add cards to views](#)
-

## Getting Started/Presence Detection

---

Presence detection tells Home Assistant who is at home and where they are. That is one of the most useful pieces of information you can give your smart home, because it lets your automations react to people arriving and leaving:

- Send me a notification when my child arrives at school
- Turn on the AC when I leave work

Screenshot of a map dashboard in Home Assistant showing a school, work and home zone and two people. Map dashboard showing a school, work, and a home zone and the location of two people.

## About setting up zone presence detection

---

There are different ways of setting up zone presence detection. One way is to run an app on your phone to send detailed location information to your Home Assistant instance. Another way to detect presence is by checking which devices are connected to the network. You can do that if you have one of our [supported routers](#). By leveraging what your router already knows, you can detect if people are at home.

# Adding zone presence detection with a mobile phone

---

## Prerequisites

- Home Assistant installed
- Onboarding steps completed
- Remote access enabled
- The easiest way to do this is by enabling
  - Home Assistant Cloud
- Mobile phone:
  - Android (Android 5 or later) or iPhone (iOS 15 or later)
  - Phone plan with Internet access
  - Access to your local network where Home Assistant is running
  - Home Assistant Companion app installed on your phone.
  - During the setup procedure, make sure to grant **Location access**.
    - Location access creates a `device_tracker` entity for that device. This entity can be used in automations and conditions.

## To add zone presence detection with a mobile phone

1. Open the Home Assistant Companion app on your phone and log in to your Home Assistant instance.
2. On the screen to **Connect to Home Assistant**, make sure you activate **Enable location tracking**.
3. Select **Continue**.
4. Go to **Settings** > **Devices & Services** and look for the new integration that was added: **Mobile App**.
5. On the integration card, select **1 Device**. This opens the device info page.
6. You now see your phone name and its entities.
7. To view the location of your phone on the map, open the **Map** dashboard.
8. You now see a circle on that map with your initial.
9. It shows the current location of your phone.
10. To view the details, select that initial.
  - Open the **Attributes** list to see the phone's **Latitude**, **Longitude**, and the **Source** of information.
  - The source is the `device_tracker` entity for that device, for example `device_tracker.pixel_7_pro`.
11. To view the entity details and the history, go to **Settings** > **Devices & services** > **Entities** and in the search field, enter `devi` and select your `device_tracker` entity from the list.
12. Check your **Zones** to prepare them for automations.

13. Your home zone was set up during onboarding, but you can edit it.

14. You can add other zones if you want to automate on them.

- For example, if you want the heating to start when you leave your office, you can add a zone called **Office**.
- In this case, leaving the office zone would be an automation trigger.
- You could also use the location information as an automation condition, for example, when you have an automation to turn on the light at sunset, but only when you are home.

## Adding presence detection for other persons in your home

---

1. For each person you want to have presence detection, add a device tracker (for example, their phone).
2. You can also use a smartwatch for presence detection. To do this, install the [Home Assistant Companion app](#) on the device. Make sure to allow location tracking.
3. To use it for zone presence detection outside your home, the smartwatch requires a mobile plan.
4. Go to **Settings** > **People** and select the person.
5. Scroll down and under **Select the devices that belong to this person**, select the device.

[Include not found: getting-started/next\_step.html step="Join the Community" link="/getting-started/join-the-community/"]

---



## Help/Index

---

Home Assistant has one of the largest and most active smart home communities on the internet. Whether you have a question, need help with a setup, want to request a feature, or just want to say hi, there are plenty of places to get in touch.

## Communication channels

---

- [Forum](#)
- [Discord Chat Server](#) for general Home Assistant discussions and questions.
- Follow us on Bluesky, use [@home-assistant.io](#)
- Follow us on Mastodon, use [@homeassistant@fosstodon.org](#)
- Follow us on X, use [@home\\_assistant](#)
- Join the [Facebook](#) community
- Join the Reddit in [/r/homeassistant](#)

## Bugs and feature requests

---

Have you found an issue in your Home Assistant installation? Please report it. Reporting it makes it easy to track and ensures that it gets fixed. For more details, refer to the [Reporting issues](#) page.

- [Feature requests](#) (Don't post feature requests in the issue trackers. Thanks!)
- [Issue tracker Home Assistant Core](#)
- [Issue tracker Home Assistant Frontend](#)
- [Issue tracker Home Assistant Supervisor](#)
- [Issue tracker Home Assistant Operating System](#)
- [Issue tracker Home Assistant Android App](#)
- [Issue tracker Home Assistant iOS App](#)
- [Issue tracker website & documentation](#)

## Videos, talks, and workshops

---

- [PyconFR 2018 - Faire de la domotique libriste avec Python \(French\) \(Slides\)](#) - October 2018
- [Automate your home with Home Assistant at foss-north 2018](#) - March 2018
- [Home Assistant - Smart Home für Jedermann \(German\) at Pi and More 2018 \(Slides\)](#) - February 2018
- [Why we can't have the Internet of Nice Things: A home automation primer at OpenWest 2017](#) - July 2017
- [Home Automation with Home Assistant at PyDay Galicia 2017](#) - June 2017
- [Home Automation with Python at GLT 2017](#) - April 2017
- [Home Assistant workshop at CLT 2017 at CLT 2017](#) - March 2017
- [Home Assistant - Erweiterungen \(Platforms/Components\) at Puzzle ITC](#) - December 2016
- [Automating Your Life - Home Automation at Develop Denver 2016](#) - August
- [Building Online Communities: Home Assistant](#) - July 2016
- [Home Assistant Support 101 - Getting around in Home Assistant \(Slides\)](#) - June 2016
- [Awaken your home: Python and the Internet of Things at PyCon 2016](#) - June 2016

Looking for talking points or trivia?

## Media coverage

Don't miss the regular [Home Assistant](#) podcasts.

- [How to achieve smart home nirvana \(or, home automation without subscription\)](#) - March 2021
- [Why I use Home Assistant for open source home automation](#) - November 2020
- [Home Assistant, the Python IoT hub](#) - June 2020
- [Thomas-Krenn-Award 2020: Dino, Teckids und Home Assistant](#) - June 2020
- [Magical Smart Home Upgrade Lets Muggles Control Their Homes With a Wand Too](#) - April 2019
- [Thomas-Krenn-Award 2019 – The winners](#) - March 2019
- [How to set up and use Home Assistant: For Dummies edition](#) - October 2018
- [DINACon begeistert 200 Teilnehmende und die Award-Gewinner 2018](#) - October 2018
- [Build a wireless MQTT weather sensor for your Home Assistant](#) - October 2018
- [Nominations for the DINACon award 2018](#) - September 2018
- [Building a better thermostat with Home Assistant](#) - August 2018
- [Home Assistant lets you automate your smart home without giving up privacy](#) - May 2018
- [HackSpace magazine #6](#) - May 2018
- [SmartThings vs Home Assistant](#) - April 2018
- [Feed the dog and close the door with an open source home automation system](#) - March 2018
- [The winners of the Thomas-Krenn-Awards 2018](#) - March 2018
- [Best of Open Source Smart Home: Home Assistant vs OpenHAB](#) - February 2018

- [Hausautomations-Schaltzentrale Home Assistant auf Python-Basis](#) - December 2017
  - [Using Home Assistant the ARTIK Cloud](#) - September 2017
  - [Control home automation hardware with Home Assistant](#) - August 2017
  - [Smart Home Home Assistant KNX Alexa Sprachsteuerung](#) - August 2017
  - [Episode #122: Home Assistant: Pythonic Home Automation](#) - July 2017
  - [Smart Home Home Assistant Konfiguration mit YAML](#) - July 2017
  - [Why can't we have the Internet of Nice Things?](#) - July 2017
  - [Smart Home Home Assistant Raspberry Pi Installation Hassbian](#) - July 2017
  - [Jupiter Broadcasting - No Privacy Compromise Home Automation](#) - June 2017
  - [Monitor IoT devices with Home Assistant and Datadog](#) - June 2017
  - [Castálio Podcast - Episódio 102: Marcelo Mello - Red Hat e Automação Residencial com Home Assistant](#) - May 2017
  - [Paulus Schoutsen and Home Assistant - Episode 8](#) - March 2017
  - [Zammad, Home Assistant and Freifunk - are the winner of the Thomas-Krenn-Awards 2017](#) - March 2017
  - [Home Assistant with Paulus Schoutsen - Episode 94](#) - January 2017
  - [Episode #11 at minute 15:20 by Python Bytes](#) - January 2017
  - [Now you can hide your smart home on the darknet](#) - July 2016
  - [Home Assistant: The Python Approach to Home Automation](#) - June 2016
  - [Secure home automation, without clouds or dedicated hubs](#) - June 2016
  - [Weekend Project: Setting up Home Assistant on your PC or Mac by automated home](#) - April 2016
  - [Episode 105 - DIY Home Automation Roundup by HomeTech.fm](#) - April 2016
  - [5 open source home automation tools by opensource.com](#) - March 2016
  - [Home Assistant – Open Source Python Home Automation Platform](#) - January 2015
-

## Help/Reporting Issues

---

If you have an installation, a setup or a configuration issue, please use our [Forum](#) to get help. We have a big community which will help you if they can.

If you found a bug, please report it in one of our [trackers](#). To help you and our developers identify the issue quickly, please fill out the provided template.

## Finding system information

---

Many issue reports require details about your Home Assistant installation. To find this information:

1. Go to **Settings** > **System** > **Repairs**.
2. From the three dots `mdi:dots-vertical` menu, select **System information**.
3. This [system information dialog](#) shows your Home Assistant version, installation type, operating system, and other system details.
4. To copy the system information, at the bottom of the dialog, select the **Copy** button.

## Description of the problem

---

Provide a summary of your issue and tell us what's wrong. Tell us what you were trying to do and what happened.

There are integrations which require additional steps (installing third-party tools, compilers, etc.) to get your setup working. Please describe the steps you took and the ones to reproduce the issue if needed.



## First Home Assistant version with the issue

---

Please provide the version that contains the issue. See [Finding system information](#) for instructions on finding the version.

## Last working Home Assistant release (if known)

---

If possible, provide the latest release that you know was working. Home Assistant is evolving very fast and issues may already be addressed or be introduced by a recent change. See [Finding system information](#) for instructions on finding the version.

## Operating environment

---

There are many different ways to run Home Assistant. In this section, please mention which you are using: Home Assistant Operating System or Home Assistant Container in Docker. It would be helpful to mention which operating system you are using because not all are supported on the same level. See [Finding system information](#) for instructions on how to find this information.

# Integration

---

Please add the link to the documentation of the integration in question. For example:

- Issue with the `random` sensor: </integrations/random#sensor>.
- Issue with the `hue` integration: </integrations/hue/>.

## Diagnostics information

Consider uploading [the diagnostics file](#) along with your issue report to allow faster triaging and pinpointing the issue. The information contained in the generated diagnostics file is redacted to avoid any sensitive information while still remaining useful for developers to fix the issue.

## Problem-relevant `configuration.yaml` entries

To exclude configuration issues and allow the developers to quickly test, and perhaps reproduce, your issue, add the relevant part of your `configuration.yaml` file. This file is located in your [configuration folder](#).

```
sensor:
 - platform: random
```

Make sure that you don't post your username, password, API key, access token or other [secrets](#).

## Traceback and log information (if applicable)

If things go wrong, there will be a so-called traceback or an error message in your logs under **Settings > System > Logs**. Please include this. It starts with **Traceback** and can contain information about where the error was triggered in the code.

```
Traceback (most recent call last):
...
```

In some cases, it is also necessary to [enable debug logging](#) to get detailed logs to triage an issue. Enabling this will instruct Home Assistant to log a lot of fine-grained information about the integration. This is helpful for debugging and fixing the issue. In contrast to the diagnostics information, debug logs are not automatically redacted. Make sure to include only the parts you think are relevant to the issue.

[Download the logs](#) and see if there are any errors related to your integration.

## Additional information

This section can contain additional details or other observation. Often the little things can help as well.

---

## Help/Talking Points

---

People are starting to present Home Assistant at meetings and get-togethers. Below you find a couple of bullet points for your presentation.

- [Numbers, numbers, numbers](#) and other details
  - Over 1900 implementations
  - Not depending on cloud services. We like to keep your privacy private
  - Control all your devices from a single, mobile-friendly interface
  - Written in Python3 with 94% test coverage
  - Active and helpful community
-

## Help/Trivia

---

This page contains only irrelevant and unhelpful information about Home Assistant. If you are going to prepare a talk about Home Assistant, perhaps some information mentioned here can be used for the intro.

## Name

---

Isn't it obvious? Home Assistant is the good soul that is assisting you in your home. The abbreviation `hass` (not to be confused with the German word "Hass", it's more like the abbreviation of **H**ome **A**ssistant **S**erver **S**ervice) was commonly used in the past, though it's now recommended to use the full name "Home Assistant" in documentation.

## Website

---

The website <https://www.home-assistant.io> was launched on December 18, 2014 and contains documentation about the setup process, the platforms and integrations, and for the developers.



## Logo

---

The logo was refreshed in 2023 for Home Assistant's 10th birthday. You can find more details in the [blog post](#).

## License

---

Home Assistant is open source software and available under the [Apache 2.0](#) license.

## Hardware

---

- End of 2020, [Home Assistant Blue](#) was launched. The first dedicated smart home hub for Home Assistant. This limited edition is based on an ODROID and known for its blue aluminum case.
- In September 2021, [Home Assistant Yellow](#) was launched. This extendable smart home hub is a custom, open-source design powered by a Raspberry Pi Compute Module 4. Home Assistant Yellow supports both Zigbee and Thread.
- End of 2022, [Home Assistant SkyConnect](#) was launched: a USB dongle that provides Zigbee and Thread network support for hubs that do not have this built-in.
- In September 2023, [Home Assistant Green](#) was launched. This plug-and-play smart home hub is an affordable and the easiest way to get started with Home Assistant.

## Numbers

---

This sections just contains some random numbers of the Home Assistant eco-system. Sorry, only the main repository counts.

| Description                      | 2015 | 2016  | 2017 | 2023 |
|----------------------------------|------|-------|------|------|
| Forum posts                      | 352  | 33004 | 172k | 2.5M |
| Forum topics                     | 83   | 4863  | 22k  | 205k |
| Forum members                    | 92   | 3931  | 17k  | 193k |
| GitHub stars                     | 2519 | 5239  | 12k  | 62k  |
| GitHub forks                     | 374  | 1424  | 3.5k | 24k  |
| Page views <a href="#">ha.io</a> | 600k | 6.6M  | 23M  |      |

## Commits per year

---

The numbers below only covers the [main git repository](#) and doesn't take the helpers ([home-assistant.io](#), [frontend](#), etc.) into account.

```
2013: 147
2014: 328
2015: 2963
2016: 4299
2017: 3917
2022: 9991
```

More details and statistics can be found on [GitHub](#).

## First commit

---

The first commit in `git` was made on Sep 17, 2013 by [Paulus Schoutsen](#).

```
commit d55e4d53cccc9123d03f45c53441e7cbfc58e515
Author: Paulus Schoutsen <Paulus@PaulusSchoutsen.nl>
```

```
Initial commit
```

---

## Installation/Alternative

---

Included sections for this page is located under source/\_includes/installation

## Install Home Assistant Operating System

---

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.



## Suggested hardware

---

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

*These are affiliated links. We get commissions for purchases made through links in this post.*

### Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

### Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

## Configure the BIOS on your x86-64 hardware

---

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

## Write HAOS onto your x86-64 hardware

---

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

**Method 1 (recommended):** Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

**Method 2:** With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

### Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

#### Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

#### To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
  - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
    - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
      - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
    - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
      - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
  - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

## Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

### Required material

- Computer

- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

## Write the image to your boot medium

## Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

```
{% endtabbed_block %}
```

```
```text
```

```
//haos_-.img.xz ```
```

```
{% endif %}
```

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.

- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at homeassistant.local:8123.

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at homeassistant:8123 or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

Download the appropriate image

- VirtualBox (Intel chip) (.vdi)
- VirtualBox (Apple Silicon chip) (.vdi)
- KVM (.qcow2)

- [KVM/Proxmox](#) (.qcow2)
- [VMware ESXi/vSphere](#) (.ova)
- [VMware Workstation](#) (.vmdk)
- [Hyper-V](#) (.vhdx)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

All these can be extended if your usage calls for more resources.

Hypervisor specific configuration

{% tabbed_block %}

- title: VirtualBox content: |

Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).
2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager, Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.
6. Select the CPU cores you want to let the VM use and give it some memory.
7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
8. Select your keyboard language under **VM Console Keyboard**.
9. Select **br0** under **Network Source**.

10. Select **virtio** under **Network model**.
 11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
 12. Deselect **Start VM after creation**.
 13. Select **Create**.
 14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
 15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
 1. Create a new virtual machine in `virt-manager`.
 2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
 3. Choose **Generic Default** for the operating system.
 4. Check the box for **Customize configuration before install**.
 5. Under **Network Selection**, select your bridge.
 6. Under customization select **Overview > Firmware > UEFI x86_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
 7. Select **Add Hardware** (bottom left), and select **Channel**.
 8. Select device type: **unix**.
 9. Select name: **org.qemu.guest_agent.0**.
 10. Finally, select **Begin Installation** (upper left corner).
 - title: KVM (virt-install) content: |

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
 - Select the one for Ethernet.
 - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
 - If you haven't unzipped the archive, unzip it.
 - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
 - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.
6. Now continue with the next step to start your VM.
 - If you see a message about side channel mitigations, select **OK**.

- If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
- title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
 1. Create a new virtual machine.
 2. Select **Generation 2**.
 3. Select **Connection > Your Virtual Switch that is bridged**.
 4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings > Security** and deselect **Enable Secure Boot**.

{% endtabbed_block %}

Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Flashing an ODROID-N2+

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

All these instructions work the same for the ODROID-N2 (non-plus version).

What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



Note: When using the command line, it may return the following message:
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).

Flashing an ODROID-M1S

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

Warning: Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

Install Home Assistant Container

These below instructions are for an installation of Home Assistant Container running in your own container environment, which you manage yourself. Any [OCI](#) compatible runtime can be used, however this guide will focus on installing it with Docker.

 **Note:** This installation type **does not have access to apps**. If you want to use apps, you need to use another installation type. The recommended type is Home Assistant Operating System. Checkout the [overview table of installation types](#) to see the differences.

 **Important: Prerequisites** This guide assumes that you already have an operating system setup and a container runtime installed (like Docker).

If you are using Docker, you need Docker Engine 23.0.0 or later. Docker *Desktop* will not work; you must use Docker *Engine*.

Platform installation

Installation with Docker is straightforward. Adjust the following command so that:

- `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration and run it. Make sure that you keep the `:/config` part.
- `MY_TIME_ZONE` is a [tz database name](#), like `TZ=America/Los_Angeles`.
- D-Bus is optional but required if you plan to use the [Bluetooth integration](#).

Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
 - Choose a container-name you want (for example, `homeassistant`).
 - Set **Enable auto-restart** if you like.
- Select **Next**.
- On the **Advanced Settings** page:

- In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config`), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
- To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London`). Timezones can be found [here](#).
- In the **Network** section, set the Network dropdown as `host` .
- Select **Next**.
- Ensure **Run this container after the wizard is finished** is checked and select **Done**.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123`).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `:/config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.

- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see https://www.qnap.com/solution/container_station/en/index.php.

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing [your correct timezone](#).
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

{% tabbed_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-  
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/  
dbus:/run/dbus:ro \ --network=host \ :
```

- title: Update content: |

```
```bash
```

## if this returns "Image is up to date" then you can stop here

---

```
docker pull : ``
```

```
```bash
```

stop the running container

```
docker stop homeassistant ``
```

```
```bash
```

# remove it from Docker's list of containers

---

```
docker rm homeassistant ""
""bash
```

## finally, start a new one

---

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e
TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --
network=host \ : ""
```

{% endtabbed\_block %}

Once the Home Assistant Container is running Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Restart Home Assistant

If you change the configuration, you have to restart the server. To do that you have 3 options.

1. In your Home Assistant UI, go to **Settings** > **System** and in the top-right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Restart Home Assistant**.
2. Go to **Settings** > **Developer tools** > **Actions**, select `homeassistant.restart` and select **Perform action**.
3. Restart it from a terminal.

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker restart homeassistant
```

- title: Docker Compose content: |

```
bash docker compose restart
```

{% endtabbed\_block %}

## Docker compose

 **Tip:** `docker compose` should already be installed on your system. If not, you can manually install it.

As the Docker command becomes more complex, switching to `docker compose` can be preferable and support automatically restarting on failure or system restart. Create a `compose.yaml` file:

```
services:
 homeassistant:
 container_name: homeassistant
 image: ":"
 volumes:
 - /PATH_TO_YOUR_CONFIG:/config
 - /etc/localtime:/etc/localtime:ro
 - /run/dbus:/run/dbus:ro
 restart: unless-stopped
 privileged: true
 network_mode: host
 environment:
 TZ: Europe/Amsterdam
```

Start it by running:

```
docker compose up -d
```

Once the Home Assistant Container is running, Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Exposing devices

In order to use Zigbee or other integrations that require access to devices, you need to map the appropriate device into the container. Ensure the user that is running the container has the correct privileges to access the `/dev/tty*` file, then add the device mapping to your container instructions:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... --device /dev/ttyUSB0:/dev/ttyUSB0 ...
```

- title: Docker Compose content: |

```
yaml services: homeassistant: ... devices: - /dev/ttyUSB0:/dev/ttyUSB0
```

{% endtabbed\_block %}

## Optimizations

The Home Assistant Container is using an alternative memory allocation library `jemalloc` for better memory management and Python runtime speedup.

As the `jemalloc` configuration used can cause issues on certain hardware featuring a page size larger than 4K (like some specific ARM64-based SoCs), it can be disabled by passing the environment variable `DISABLE_JEMALLOC` with any value, for example:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... -e "DISABLE_JEMALLOC=true" ...
```

- title: Docker Compose content: |

```
yml services: homeassistant: ... environment: DISABLE_JEMALLOC: true
```

{% endtabbed\_block %}

The error message `<jemalloc>: Unsupported system page size` is one known indicator.

## Community provided guides

---

Additional installation guides can be found on our [Community Forum](#).

These Community Guides are provided as-is. Some of these install methods are more limited than the methods above. Some integrations may not work due to limitations of the platform.

---

## Installation/Generic X86 64

---

Included sections for this page is located under source/\_includes/installation



## Install Home Assistant Operating System

---

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.

## Suggested hardware

---

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

*These are affiliated links. We get commissions for purchases made through links in this post.*

### Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

### Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

## Configure the BIOS on your x86-64 hardware

---

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

## Write HAOS onto your x86-64 hardware

---

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

**Method 1 (recommended):** Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

**Method 2:** With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

### Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

#### Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

#### To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
  - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
    - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
      - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
    - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
      - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
  - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

## Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

### Required material

- Computer

- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

## Write the image to your boot medium

## Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

```
{% endtabbed_block %}
```

```
```text
```

```
//haos_-.img.xz ```
```

```
{% endif %}
```

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.

- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at homeassistant.local:8123.

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at homeassistant:8123 or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

Download the appropriate image

- VirtualBox (Intel chip) (.vdi)
- VirtualBox (Apple Silicon chip) (.vdi)
- KVM (.qcow2)

- [KVM/Proxmox](#) (.qcow2)
- [VMware ESXi/vSphere](#) (.ova)
- [VMware Workstation](#) (.vmdk)
- [Hyper-V](#) (.vhdx)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

All these can be extended if your usage calls for more resources.

Hypervisor specific configuration

{% tabbed_block %}

- title: VirtualBox content: |

Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).
2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager, Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.
6. Select the CPU cores you want to let the VM use and give it some memory.
7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
8. Select your keyboard language under **VM Console Keyboard**.
9. Select **br0** under **Network Source**.

10. Select **virtio** under **Network model**.
 11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
 12. Deselect **Start VM after creation**.
 13. Select **Create**.
 14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
 15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
 1. Create a new virtual machine in `virt-manager`.
 2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
 3. Choose **Generic Default** for the operating system.
 4. Check the box for **Customize configuration before install**.
 5. Under **Network Selection**, select your bridge.
 6. Under customization select **Overview > Firmware > UEFI x86_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
 7. Select **Add Hardware** (bottom left), and select **Channel**.
 8. Select device type: **unix**.
 9. Select name: **org.qemu.guest_agent.0**.
 10. Finally, select **Begin Installation** (upper left corner).
 - title: KVM (virt-install) content: |

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
 - Select the one for Ethernet.
 - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
 - If you haven't unzipped the archive, unzip it.
 - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
 - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.
6. Now continue with the next step to start your VM.
 - If you see a message about side channel mitigations, select **OK**.

- If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
- title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
 1. Create a new virtual machine.
 2. Select **Generation 2**.
 3. Select **Connection > Your Virtual Switch that is bridged**.
 4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings > Security** and deselect **Enable Secure Boot**.

{% endtabbed_block %}

Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Flashing an ODROID-N2+

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

All these instructions work the same for the ODROID-N2 (non-plus version).

What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



Note: When using the command line, it may return the following message:
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).

Flashing an ODROID-M1S

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

Warning: Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

Install Home Assistant Container

These below instructions are for an installation of Home Assistant Container running in your own container environment, which you manage yourself. Any [OCI](#) compatible runtime can be used, however this guide will focus on installing it with Docker.

 **Note:** This installation type **does not have access to apps**. If you want to use apps, you need to use another installation type. The recommended type is Home Assistant Operating System. Checkout the [overview table of installation types](#) to see the differences.

 **Important: Prerequisites** This guide assumes that you already have an operating system setup and a container runtime installed (like Docker).

If you are using Docker, you need Docker Engine 23.0.0 or later. Docker *Desktop* will not work; you must use Docker *Engine*.

Platform installation

Installation with Docker is straightforward. Adjust the following command so that:

- `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration and run it. Make sure that you keep the `:/config` part.
- `MY_TIME_ZONE` is a [tz database name](#), like `TZ=America/Los_Angeles`.
- D-Bus is optional but required if you plan to use the [Bluetooth integration](#).

Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
 - Choose a container-name you want (for example, `homeassistant`).
 - Set **Enable auto-restart** if you like.
- Select **Next**.
- On the **Advanced Settings** page:

- In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config`), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
- To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London`). Timezones can be found [here](#).
- In the **Network** section, set the Network dropdown as `host` .
- Select **Next**.
- Ensure **Run this container after the wizard is finished** is checked and select **Done**.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123`).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `:/config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.

- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see https://www.qnap.com/solution/container_station/en/index.php.

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing [your correct timezone](#).
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

{% tabbed_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/
dbus:/run/dbus:ro \ --network=host \ :
```

- title: Update content: |

```
```bash
```

## if this returns "Image is up to date" then you can stop here

---

```
docker pull : ``
```

```
```bash
```

stop the running container

```
docker stop homeassistant ``
```

```
```bash
```

# remove it from Docker's list of containers

---

```
docker rm homeassistant ""
""bash
```

## finally, start a new one

---

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e
TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --
network=host \ : ""
```

{% endtabbed\_block %}

Once the Home Assistant Container is running Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Restart Home Assistant

If you change the configuration, you have to restart the server. To do that you have 3 options.

1. In your Home Assistant UI, go to **Settings** > **System** and in the top-right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Restart Home Assistant**.
2. Go to **Settings** > **Developer tools** > **Actions**, select `homeassistant.restart` and select **Perform action**.
3. Restart it from a terminal.

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker restart homeassistant
```

- title: Docker Compose content: |

```
bash docker compose restart
```

{% endtabbed\_block %}

## Docker compose

 **Tip:** `docker compose` should already be installed on your system. If not, you can manually install it.

As the Docker command becomes more complex, switching to `docker compose` can be preferable and support automatically restarting on failure or system restart. Create a `compose.yaml` file:

```
services:
 homeassistant:
 container_name: homeassistant
 image: ":"
 volumes:
 - /PATH_TO_YOUR_CONFIG:/config
 - /etc/localtime:/etc/localtime:ro
 - /run/dbus:/run/dbus:ro
 restart: unless-stopped
 privileged: true
 network_mode: host
 environment:
 TZ: Europe/Amsterdam
```

Start it by running:

```
docker compose up -d
```

Once the Home Assistant Container is running, Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Exposing devices

In order to use Zigbee or other integrations that require access to devices, you need to map the appropriate device into the container. Ensure the user that is running the container has the correct privileges to access the `/dev/tty*` file, then add the device mapping to your container instructions:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... --device /dev/ttyUSB0:/dev/ttyUSB0 ...
```

- title: Docker Compose content: |

```
yaml services: homeassistant: ... devices: - /dev/ttyUSB0:/dev/ttyUSB0
```

{% endtabbed\_block %}

## Optimizations

The Home Assistant Container is using an alternative memory allocation library `jemalloc` for better memory management and Python runtime speedup.

As the `jemalloc` configuration used can cause issues on certain hardware featuring a page size larger than 4K (like some specific ARM64-based SoCs), it can be disabled by passing the environment variable `DISABLE_JEMALLOC` with any value, for example:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... -e "DISABLE_JEMALLOC=true" ...
```

- title: Docker Compose content: |

```
yml services: homeassistant: ... environment: DISABLE_JEMALLOC: true
```

{% endtabbed\_block %}

The error message `<jemalloc>: Unsupported system page size` is one known indicator.

---

## Installation/Linux

---

Included sections for this page is located under `source/_includes/installation`



## Install Home Assistant Operating System

---

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.

## Suggested hardware

---

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

*These are affiliated links. We get commissions for purchases made through links in this post.*

### Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

### Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

## Configure the BIOS on your x86-64 hardware

---

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

## Write HAOS onto your x86-64 hardware

---

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

**Method 1 (recommended):** Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

**Method 2:** With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

### Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

#### Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

#### To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
  - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
    - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
      - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
    - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
      - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
  - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

## Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

### Required material

- Computer

- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

## Write the image to your boot medium

## Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

```
{% endtabbed_block %}
```

```
```text
```

```
//haos_-.img.xz ```
```

```
{% endif %}
```

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.

- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at homeassistant.local:8123.

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at homeassistant:8123 or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

Download the appropriate image

- VirtualBox (Intel chip) (.vdi)
- VirtualBox (Apple Silicon chip) (.vdi)
- KVM (.qcow2)

- [KVM/Proxmox](#) (.qcow2)
- [VMware ESXi/vSphere](#) (.ova)
- [VMware Workstation](#) (.vmdk)
- [Hyper-V](#) (.vhdx)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

All these can be extended if your usage calls for more resources.

Hypervisor specific configuration

{% tabbed_block %}

- title: VirtualBox content: |

Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).
2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager, Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.
6. Select the CPU cores you want to let the VM use and give it some memory.
7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
8. Select your keyboard language under **VM Console Keyboard**.
9. Select **br0** under **Network Source**.

10. Select **virtio** under **Network model**.
 11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
 12. Deselect **Start VM after creation**.
 13. Select **Create**.
 14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
 15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
 1. Create a new virtual machine in `virt-manager`.
 2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
 3. Choose **Generic Default** for the operating system.
 4. Check the box for **Customize configuration before install**.
 5. Under **Network Selection**, select your bridge.
 6. Under customization select **Overview > Firmware > UEFI x86_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
 7. Select **Add Hardware** (bottom left), and select **Channel**.
 8. Select device type: **unix**.
 9. Select name: **org.qemu.guest_agent.0**.
 10. Finally, select **Begin Installation** (upper left corner).
 - title: KVM (virt-install) content: |

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
 - Select the one for Ethernet.
 - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
 - If you haven't unzipped the archive, unzip it.
 - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
 - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.
6. Now continue with the next step to start your VM.
 - If you see a message about side channel mitigations, select **OK**.

- If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
- title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
 1. Create a new virtual machine.
 2. Select **Generation 2**.
 3. Select **Connection > Your Virtual Switch that is bridged**.
 4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings > Security** and deselect **Enable Secure Boot**.

{% endtabbed_block %}

Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Flashing an ODROID-N2+

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

All these instructions work the same for the ODROID-N2 (non-plus version).

What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



Note: When using the command line, it may return the following message:
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).

Flashing an ODROID-M1S

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

Warning: Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

Install Home Assistant Container

These below instructions are for an installation of Home Assistant Container running in your own container environment, which you manage yourself. Any [OCI](#) compatible runtime can be used, however this guide will focus on installing it with Docker.

 **Note:** This installation type **does not have access to apps**. If you want to use apps, you need to use another installation type. The recommended type is Home Assistant Operating System. Checkout the [overview table of installation types](#) to see the differences.

 **Important: Prerequisites** This guide assumes that you already have an operating system setup and a container runtime installed (like Docker).

If you are using Docker, you need Docker Engine 23.0.0 or later. Docker *Desktop* will not work; you must use Docker *Engine*.

Platform installation

Installation with Docker is straightforward. Adjust the following command so that:

- `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration and run it. Make sure that you keep the `:/config` part.
- `MY_TIME_ZONE` is a [tz database name](#), like `TZ=America/Los_Angeles`.
- D-Bus is optional but required if you plan to use the [Bluetooth integration](#).

Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
 - Choose a container-name you want (for example, `homeassistant`).
 - Set **Enable auto-restart** if you like.
- Select **Next**.
- On the **Advanced Settings** page:

- In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config`), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
- To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London`). Timezones can be found [here](#).
- In the **Network** section, set the Network dropdown as `host` .
- Select **Next**.
- Ensure **Run this container after the wizard is finished** is checked and select **Done**.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123`).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `:/config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.

- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see https://www.qnap.com/solution/container_station/en/index.php.

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing [your correct timezone](#).
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

{% tabbed_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/
dbus:/run/dbus:ro \ --network=host \ :
```

- title: Update content: |

```
```bash
```

## if this returns "Image is up to date" then you can stop here

---

```
docker pull : ``
```

```
```bash
```

stop the running container

```
docker stop homeassistant ``
```

```
```bash
```

# remove it from Docker's list of containers

---

```
docker rm homeassistant ""
""bash
```

## finally, start a new one

---

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e
TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --
network=host \ : ""
```

{% endtabbed\_block %}

Once the Home Assistant Container is running Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Restart Home Assistant

If you change the configuration, you have to restart the server. To do that you have 3 options.

1. In your Home Assistant UI, go to **Settings** > **System** and in the top-right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Restart Home Assistant**.
2. Go to **Settings** > **Developer tools** > **Actions**, select `homeassistant.restart` and select **Perform action**.
3. Restart it from a terminal.

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker restart homeassistant
```

- title: Docker Compose content: |

```
bash docker compose restart
```

{% endtabbed\_block %}

## Docker compose

 **Tip:** `docker compose` should already be installed on your system. If not, you can manually install it.

As the Docker command becomes more complex, switching to `docker compose` can be preferable and support automatically restarting on failure or system restart. Create a `compose.yaml` file:

```
services:
 homeassistant:
 container_name: homeassistant
 image: ":"
 volumes:
 - /PATH_TO_YOUR_CONFIG:/config
 - /etc/localtime:/etc/localtime:ro
 - /run/dbus:/run/dbus:ro
 restart: unless-stopped
 privileged: true
 network_mode: host
 environment:
 TZ: Europe/Amsterdam
```

Start it by running:

```
docker compose up -d
```

Once the Home Assistant Container is running, Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Exposing devices

In order to use Zigbee or other integrations that require access to devices, you need to map the appropriate device into the container. Ensure the user that is running the container has the correct privileges to access the `/dev/tty*` file, then add the device mapping to your container instructions:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... --device /dev/ttyUSB0:/dev/ttyUSB0 ...
```

- title: Docker Compose content: |

```
yaml services: homeassistant: ... devices: - /dev/ttyUSB0:/dev/ttyUSB0
```

{% endtabbed\_block %}

## Optimizations

The Home Assistant Container is using an alternative memory allocation library `jemalloc` for better memory management and Python runtime speedup.

As the `jemalloc` configuration used can cause issues on certain hardware featuring a page size larger than 4K (like some specific ARM64-based SoCs), it can be disabled by passing the environment variable `DISABLE_JEMALLOC` with any value, for example:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... -e "DISABLE_JEMALLOC=true" ...
```

- title: Docker Compose content: |

```
yml services: homeassistant: ... environment: DISABLE_JEMALLOC: true
```

{% endtabbed\_block %}

The error message `<jemalloc>: Unsupported system page size` is one known indicator.

## Troubleshooting

---

### No access to the frontend

Symptom: You cannot open the Home Assistant page in your browser. If you are not using Home Assistant Operating System, the cause may be an access restriction.

In newer Linux distributions, the access to a host is very limited. This means that you can't access the Home Assistant frontend that is running on a host outside of the host machine.

To fix this, you will need to open your machine's firewall for TCP traffic to port 8123. The method for doing this will vary depending on your operating system and the firewall you have installed. Below are some suggestions to try. Google is your friend here.

For UFW systems (for example, Debian):

```
sudo ufw allow 8123/tcp
```



## Installation/Macos

---

Included sections for this page is located under source/\_includes/installation

## Install Home Assistant Operating System

---

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.

## Suggested hardware

---

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

*These are affiliated links. We get commissions for purchases made through links in this post.*

### Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

### Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

## Configure the BIOS on your x86-64 hardware

---

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

## Write HAOS onto your x86-64 hardware

---

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

**Method 1 (recommended):** Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

**Method 2:** With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

### Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

#### Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

#### To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
  - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
    - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
      - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
    - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
      - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
  - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

## Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

### Required material

- Computer

- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

## Write the image to your boot medium

## Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

```
{% endtabbed_block %}
```

```
```text
```

```
//haos_-.img.xz ```
```

```
{% endif %}
```

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.

- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at homeassistant.local:8123.

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at homeassistant:8123 or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

Download the appropriate image

- VirtualBox (Intel chip) (.vdi)
- VirtualBox (Apple Silicon chip) (.vdi)
- KVM (.qcow2)

- [KVM/Proxmox](#) (.qcow2)
- [VMware ESXi/vSphere](#) (.ova)
- [VMware Workstation](#) (.vmdk)
- [Hyper-V](#) (.vhdx)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

All these can be extended if your usage calls for more resources.

Hypervisor specific configuration

{% tabbed_block %}

- title: VirtualBox content: |

Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).
2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager, Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.
6. Select the CPU cores you want to let the VM use and give it some memory.
7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
8. Select your keyboard language under **VM Console Keyboard**.
9. Select **br0** under **Network Source**.

10. Select **virtio** under **Network model**.
 11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
 12. Deselect **Start VM after creation**.
 13. Select **Create**.
 14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
 15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
 1. Create a new virtual machine in `virt-manager`.
 2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
 3. Choose **Generic Default** for the operating system.
 4. Check the box for **Customize configuration before install**.
 5. Under **Network Selection**, select your bridge.
 6. Under customization select **Overview > Firmware > UEFI x86_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
 7. Select **Add Hardware** (bottom left), and select **Channel**.
 8. Select device type: **unix**.
 9. Select name: **org.qemu.guest_agent.0**.
 10. Finally, select **Begin Installation** (upper left corner).
 - title: KVM (virt-install) content: |

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
 - Select the one for Ethernet.
 - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
 - If you haven't unzipped the archive, unzip it.
 - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
 - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.
6. Now continue with the next step to start your VM.
 - If you see a message about side channel mitigations, select **OK**.

- If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
- title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
 1. Create a new virtual machine.
 2. Select **Generation 2**.
 3. Select **Connection > Your Virtual Switch that is bridged**.
 4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings > Security** and deselect **Enable Secure Boot**.

{% endtabbed_block %}

Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Flashing an ODROID-N2+

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

All these instructions work the same for the ODROID-N2 (non-plus version).

What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



Note: When using the command line, it may return the following message:
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).

Flashing an ODROID-M1S

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

Warning: Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

Installation/Odroid

Included sections for this page is located under `source/_includes/installation`

Install Home Assistant Operating System

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.

Suggested hardware

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

These are affiliated links. We get commissions for purchases made through links in this post.

Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

Configure the BIOS on your x86-64 hardware

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

Write HAOS onto your x86-64 hardware

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

Method 1 (recommended): Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

Method 2: With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
 - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
 - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
 - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
 - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
 - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
 - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

Required material

- Computer

- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

Write the image to your boot medium

Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

```
{% endtabbed_block %}
```

```
```text
```

```
//haos_-.img.xz ```
```

```
{% endif %}
```

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

## Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.



- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at [homeassistant.local:8123](http://homeassistant.local:8123).

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at [homeassistant:8123](http://homeassistant:8123) or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

## Download the appropriate image

- VirtualBox (Intel chip) (.vdi)
- VirtualBox (Apple Silicon chip) (.vdi)
- KVM (.qcow2)

- [KVM/Proxmox](#) (.qcow2)
- [VMware ESXi/vSphere](#) (.ova)
- [VMware Workstation](#) (.vmdk)
- [Hyper-V](#) (.vhdx)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

## Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

*All these can be extended if your usage calls for more resources.*

## Hypervisor specific configuration

{% tabbed\_block %}

- title: VirtualBox content: |

### Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

### Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).
2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

## Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

## Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

## Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager, Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.
6. Select the CPU cores you want to let the VM use and give it some memory.
7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
8. Select your keyboard language under **VM Console Keyboard**.
9. Select **br0** under **Network Source**.

10. Select **virtio** under **Network model**.
  11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
  12. Deselect **Start VM after creation**.
  13. Select **Create**.
  14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
  15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
    1. Create a new virtual machine in `virt-manager`.
    2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
    3. Choose **Generic Default** for the operating system.
    4. Check the box for **Customize configuration before install**.
    5. Under **Network Selection**, select your bridge.
    6. Under customization select **Overview > Firmware > UEFI x86\_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
    7. Select **Add Hardware** (bottom left), and select **Channel**.
    8. Select device type: **unix**.
    9. Select name: **org.qemu.guest\_agent.0**.
    10. Finally, select **Begin Installation** (upper left corner).
  - title: KVM (virt-install) content: |

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
  - Select the one for Ethernet.
  - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

## Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
  - If you haven't unzipped the archive, unzip it.
  - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
  - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmtx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.
6. Now continue with the next step to start your VM.
  - If you see a message about side channel mitigations, select **OK**.

- If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
- title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
  1. Create a new virtual machine.
  2. Select **Generation 2**.
  3. Select **Connection > Your Virtual Switch that is bridged**.
  4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings > Security** and deselect **Enable Secure Boot**.

{% endtabbed\_block %}

## Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Flashing an ODROID-N2+

---

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

*All these instructions work the same for the ODROID-N2 (non-plus version).*

### What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

### Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

### Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



**Note:** When using the command line, it may return the following message:  
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:

```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

## Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos\_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).



# Flashing an ODROID-M1S

---

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional [eMMC](#) module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB [eMMC](#) soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in [eMMC](#).

**Warning:** Installing Home Assistant OS replaces the firmware and [SPL](#) on the [eMMC](#) with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the [EMMC2UMS](#) image anymore, because the mainline [SPL](#) is not compatible with U-Boot in the [EMMC2UMS](#) image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

## What you will need

To flash your [eMMC](#) using [USB-OTG](#), you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel [eMMC-to-USB-mass-storage](#) image

## Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S\\_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

## Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

### ***HK Recovery***

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the [eMMC](#) with an operating system of your choice is to use Home Assistant OS to flash the [EMMC2UMS](#) image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

# Install Home Assistant Container

These below instructions are for an installation of Home Assistant Container running in your own container environment, which you manage yourself. Any [OCI](#) compatible runtime can be used, however this guide will focus on installing it with Docker.

 **Note:** This installation type **does not have access to apps**. If you want to use apps, you need to use another installation type. The recommended type is Home Assistant Operating System. Checkout the [overview table of installation types](#) to see the differences.

 **Important: Prerequisites** This guide assumes that you already have an operating system setup and a container runtime installed (like Docker).

If you are using Docker, you need Docker Engine 23.0.0 or later. Docker *Desktop* will not work; you must use Docker *Engine*.

## Platform installation

Installation with Docker is straightforward. Adjust the following command so that:

- `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration and run it. Make sure that you keep the `:/config` part.
- `MY_TIME_ZONE` is a [tz database name](#), like `TZ=America/Los_Angeles`.
- D-Bus is optional but required if you plan to use the [Bluetooth integration](#).

## Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
  - Choose a container-name you want (for example, `homeassistant`).
  - Set **Enable auto-restart** if you like.
- Select **Next**.
- On the **Advanced Settings** page:

- In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config` ), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
- To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London` ). Timezones can be found [here](#).
- In the **Network** section, set the Network dropdown as `host` .
- Select **Next**.
- Ensure **Run this container after the wizard is finished** is checked and select **Done**.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123` ).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

[https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system\\_terminal](https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal)

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `:/config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.

- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

## QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see [https://www.qnap.com/solution/container\\_station/en/index.php](https://www.qnap.com/solution/container_station/en/index.php).

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing [your correct timezone](#).
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

## Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

{% tabbed\_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/
dbus:/run/dbus:ro \ --network=host \ :
```

- title: Update content: |

```
```bash
```

if this returns "Image is up to date" then you can stop here

```
docker pull : ``
```

```
```bash
```

## stop the running container

---

```
docker stop homeassistant ``
```

```
```bash
```

remove it from Docker's list of containers

```
docker rm homeassistant ""
```

```
""bash
```

finally, start a new one

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --network=host \ : ""
```

{% endtabbed_block %}

Once the Home Assistant Container is running Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Restart Home Assistant

If you change the configuration, you have to restart the server. To do that you have 3 options.

1. In your Home Assistant UI, go to **Settings** > **System** and in the top-right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Restart Home Assistant**.
2. Go to **Settings** > **Developer tools** > **Actions**, select `homeassistant.restart` and select **Perform action**.
3. Restart it from a terminal.

{% tabbed_block %}

- title: Docker CLI content: |

```
bash docker restart homeassistant
```

- title: Docker Compose content: |

```
bash docker compose restart
```

{% endtabbed_block %}

Docker compose

 **Tip:** `docker compose` should already be installed on your system. If not, you can manually install it.

As the Docker command becomes more complex, switching to `docker compose` can be preferable and support automatically restarting on failure or system restart. Create a `compose.yaml` file:

```
services:
  homeassistant:
    container_name: homeassistant
    image: ":"
    volumes:
      - /PATH_TO_YOUR_CONFIG:/config
      - /etc/localtime:/etc/localtime:ro
      - /run/dbus:/run/dbus:ro
    restart: unless-stopped
    privileged: true
    network_mode: host
    environment:
      TZ: Europe/Amsterdam
```

Start it by running:

```
docker compose up -d
```

Once the Home Assistant Container is running, Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Exposing devices

In order to use Zigbee or other integrations that require access to devices, you need to map the appropriate device into the container. Ensure the user that is running the container has the correct privileges to access the `/dev/tty*` file, then add the device mapping to your container instructions:

{% tabbed_block %}

- title: Docker CLI content: |

```
bash docker run ... --device /dev/ttyUSB0:/dev/ttyUSB0 ...
```

- title: Docker Compose content: |

```
yaml services: homeassistant: ... devices: - /dev/ttyUSB0:/dev/ttyUSB0
```

{% endtabbed_block %}

Optimizations

The Home Assistant Container is using an alternative memory allocation library [jemalloc](#) for better memory management and Python runtime speedup.

As the jemalloc configuration used can cause issues on certain hardware featuring a page size larger than 4K (like some specific ARM64-based SoCs), it can be disabled by passing the environment variable `DISABLE_JEMALLOC` with any value, for example:

{% tabbed_block %}

- title: Docker CLI content: |

```
bash docker run ... -e "DISABLE_JEMALLOC=true" ...
```

- title: Docker Compose content: |

```
yml services: homeassistant: ... environment: DISABLE_JEMALLOC: true
```

{% endtabbed_block %}

The error message `<jemalloc>: Unsupported system page size` is one known indicator.

We get commissions for purchases made through links in this post.

Installation/Raspberrypi Other

Included section for this page is located under source/_includes/installation

While we recommend using the Home Assistant Operating System, you can also use the Home Assistant Container method to install Home Assistant. Before you continue, be aware of the limitations and differences compared to the Home Assistant Operating System. You can find more information on the [installation page](#). Most notably, [apps](#) are only available with the Home Assistant Operating System.

Install Home Assistant Container

These below instructions are for an installation of Home Assistant Container running in your own container environment, which you manage yourself. Any [OCI](#) compatible runtime can be used, however this guide will focus on installing it with Docker.

 **Note:** This installation type **does not have access to apps**. If you want to use apps, you need to use another installation type. The recommended type is Home Assistant Operating System. Checkout the [overview table of installation types](#) to see the differences.

 **Important: Prerequisites** This guide assumes that you already have an operating system setup and a container runtime installed (like Docker).

If you are using Docker, you need Docker Engine 23.0.0 or later. Docker *Desktop* will not work; you must use Docker *Engine*.

Platform installation

Installation with Docker is straightforward. Adjust the following command so that:

- `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration and run it. Make sure that you keep the `:/config` part.
- `MY_TIME_ZONE` is a [tz database name](#), like `TZ=America/Los_Angeles`.
- D-Bus is optional but required if you plan to use the [Bluetooth integration](#).

Synology NAS

Synology with DSM now supports container management through the Container Manager package, allowing you to install Home Assistant without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see <https://www.synology.com/en-us/dsm/packages/ContainerManager>. The steps would be:

- Using Synology's Package Center, install the **Container Manager** package.
- Launch the Container Manager app and navigate to the **Registry** section.
- On the Registry page, search for `homeassistant/home-assistant` and select **Download**. Choose the **stable** tag.
- Wait for the image to be pulled.
- Navigate to the **Image** section of the Container Manager app.
- Select the `homeassistant/home-assistant` image and select **Run**.
- On the **General Settings** page:
 - Choose a container-name you want (for example, `homeassistant`).
 - Set **Enable auto-restart** if you like.
- Select **Next**.
- On the **Advanced Settings** page:

- In the **Volume Settings** section, select **Add Folder** and choose either an existing folder or add a new folder (for example, in the `docker` shared folder, add a new folder named `homeassistant` and then within that new folder add another new folder `config`), then select **Select**. Then edit the **Mount path** to be `/config` with the permissions set as **Read/Write**. This configures where Home Assistant will store configs and logs.
- To ensure that Home Assistant displays the correct timezone, in the **Environment** section, select the **Add** button and in the **Variable** field add `TZ` and in the value add your timezone (for example, `Europe/London`). Timezones can be found [here](#).
- In the **Network** section, set the Network dropdown as `host`.
- Select **Next**.
- Ensure **Run this container after the wizard is finished** is checked and select **Done**.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Synology NAS IP address - for example `http://192.168.1.10:8123`).

If you are using the built-in firewall, you must also add the port 8123 to allowed list. This can be found in **Control Panel** > **Security** and then the **Firewall** tab. Select **Edit Rules** besides the **Firewall Profile** dropdown box. Create a new rule and select **Custom** for Ports and add 8123. Edit Source IP if you like or leave it at default **All**. Action should stay at **Allow**.

To use a Z-Wave USB stick for Z-Wave control, the Home Assistant Docker container needs extra configuration to access to the USB stick. While there are multiple ways to do this, the least privileged way of granting access can only be performed via the Terminal, at the time of writing. See this page for configuring Terminal access to your Synology NAS:

https://www.synology.com/en-global/knowledgebase/DSM/help/DSM/AdminCenter/system_terminal

 **Tip:** See this page for accessing the Terminal via SSH

Adjust the following Terminal command as follows :

- Replace `/PATH_TO_YOUR_CONFIG` points at the folder where you want to store your configuration - make sure that you keep the `:/config` part
- Replace `/PATH_TO_YOUR_USB_STICK` matches the path for your USB stick (for example, `/dev/ttyACM0` for most Synology users)
- Replace `Australia/Melbourne` with **your timezone**

Run it in Terminal.

```
sudo docker run --restart always -d --name homeassistant -v /PATH_TO_YOUR_CONFIG
```

Complete the remainder of the Z-Wave configuration by [following the instructions here](#).

Remark: to update your Home Assistant on your Docker within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Image** section.
- Find `homeassistant/home-assistant` within Image and select **Update**.

- Wait until the system-message/-notification comes up, that the download is finished (there is no progress bar).
- Move to the **Container** section.
- Stop your container if it's running.
- Right-click on it and select **Action** > **Reset**. You won't lose any data, as all files are stored in your configuration-directory.
- Start the container again - it will then boot up with the new Home Assistant image.

Remark: to restart your Home Assistant within Synology NAS, you just have to do the following:

- Go to the **Container Manager** app and move to the **Container** section.
- Right-click on it and select **Action** > **Restart**.

 **Note:** If you want to use a USB Bluetooth adapter or Z-Wave USB Stick with Home Assistant on Synology Docker these instructions do not correctly configure the container to access the USB devices. To configure these devices on your Synology Docker Home Assistant you can follow the instructions provided [here](#) by Phil Hawthorne.

QNAP NAS

QNAP with QTS supports Docker, allowing you to install Home Assistant using Docker without the need for command-line. For details about the package (including compatibility-information, if your NAS is supported), see https://www.qnap.com/solution/container_station/en/index.php.

The steps would be:

- Install the **Container Station** package on your Qnap NAS.
- Launch Container Station and move to the **Create Container** section.
- Search image `homeassistant/home-assistant` with Docker Hub and select **Install**.
- Choose the **stable** version and select **Next**.
- Choose a container-name you want (for example, `homeassistant`).
- Select **Advanced Settings**.
- Within **Shared Folders** select **Volume from host** > **Add** and choose either an existing folder or add a new folder. The mount point has to be `/config`, so that Home Assistant will use it for the configuration and logs.
- Within **Network** and select Network Mode to **Host**.
- To ensure that Home Assistant displays the correct timezone go to the **Environment** tab and select the plus sign then add `variable` = `TZ` & `value` = `Europe/London` choosing [your correct timezone](#).
- Select **Create**.
- Wait for some time until your NAS has created the container.
- Your Home Assistant within Docker should now run and will serve the web interface from port 8123 on your Docker host (this will be your Qnap NAS IP address - for example `http://192.xxx.xxx.xxx:8123`).

Remark: To update your Home Assistant on your Docker within Qnap NAS, you just remove container and image and do steps again (Don't remove the `config` folder).

Community Notes

Note that some users have reported issues creating Home Assistant containers on ARM QNAP systems (for example, TS-233) with Container Station 3. A possible workaround is the **Docker compose** approach based on a YAML file (see section **Docker compose**). In the QNAP Container Station 3 UI, this can be accessed by going to the **Applications** section and selecting **Create**. You are then prompted to enter YAML code, which can be copied from that shown in the **Docker compose** section. Take care to modify this code in two ways: firstly, add a first line reading `version: '3'`; secondly, replace the text `/PATH_TO_YOUR_CONFIG` by a valid path on your NAS system, for example `/share/Container/HomeAssistant/config`.

{% tabbed_block %}

- title: Install content: |

```
bash docker run -d \ --name homeassistant \ --privileged \ --restart=unless-
stopped \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/
dbus:/run/dbus:ro \ --network=host \ :
```

- title: Update content: |

```
```bash
```

## if this returns "Image is up to date" then you can stop here

---

```
docker pull : ``
```

```
```bash
```

stop the running container

```
docker stop homeassistant ``
```

```
```bash
```

# remove it from Docker's list of containers

---

```
docker rm homeassistant ""
```

```
""bash
```

## finally, start a new one

---

```
docker run -d \ --name homeassistant \ --restart=unless-stopped \ --privileged \ -e TZ=MY_TIME_ZONE \ -v /PATH_TO_YOUR_CONFIG:/config \ -v /run/dbus:/run/dbus:ro \ --network=host \ : ""
```

{% endtabbed\_block %}

Once the Home Assistant Container is running Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Restart Home Assistant

If you change the configuration, you have to restart the server. To do that you have 3 options.

1. In your Home Assistant UI, go to **Settings** > **System** and in the top-right corner, select the three dots `mdi:dots-vertical` menu. Then, select **Restart Home Assistant**.
2. Go to **Settings** > **Developer tools** > **Actions**, select `homeassistant.restart` and select **Perform action**.
3. Restart it from a terminal.

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker restart homeassistant
```

- title: Docker Compose content: |

```
bash docker compose restart
```

{% endtabbed\_block %}

## Docker compose

 **Tip:** `docker compose` should already be installed on your system. If not, you can manually install it.

As the Docker command becomes more complex, switching to `docker compose` can be preferable and support automatically restarting on failure or system restart. Create a `compose.yaml` file:

```
services:
 homeassistant:
 container_name: homeassistant
 image: ":"
 volumes:
 - /PATH_TO_YOUR_CONFIG:/config
 - /etc/localtime:/etc/localtime:ro
 - /run/dbus:/run/dbus:ro
 restart: unless-stopped
 privileged: true
 network_mode: host
 environment:
 TZ: Europe/Amsterdam
```

Start it by running:

```
docker compose up -d
```

Once the Home Assistant Container is running, Home Assistant should be accessible using `http://<host>:8123` (replace with the hostname or IP of the system). You can continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

## Exposing devices

In order to use Zigbee or other integrations that require access to devices, you need to map the appropriate device into the container. Ensure the user that is running the container has the correct privileges to access the `/dev/tty*` file, then add the device mapping to your container instructions:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... --device /dev/ttyUSB0:/dev/ttyUSB0 ...
```

- title: Docker Compose content: |

```
yaml services: homeassistant: ... devices: - /dev/ttyUSB0:/dev/ttyUSB0
```

{% endtabbed\_block %}



## Optimizations

The Home Assistant Container is using an alternative memory allocation library `jemalloc` for better memory management and Python runtime speedup.

As the `jemalloc` configuration used can cause issues on certain hardware featuring a page size larger than 4K (like some specific ARM64-based SoCs), it can be disabled by passing the environment variable `DISABLE_JEMALLOC` with any value, for example:

{% tabbed\_block %}

- title: Docker CLI content: |

```
bash docker run ... -e "DISABLE_JEMALLOC=true" ...
```

- title: Docker Compose content: |

```
yml services: homeassistant: ... environment: DISABLE_JEMALLOC: true
```

{% endtabbed\_block %}

The error message `<jemalloc>: Unsupported system page size` is one known indicator.

---

## Installation/Raspberrypi

---

Included section for this page is located under source/\_includes/installation

## Suggested hardware

---

We will need a few things to get started with installing Home Assistant.

- [Raspberry Pi 5](#) or [Raspberry Pi 4](#) with [power supply](#) (make sure to choose a model with at least 2 GB of RAM).
- [Micro SD Card](#).
- Ideally get one that is [Application Class 2](#). Check for the label **A2** on the card. Application Class 2 cards perform better especially on small read and write operations and are better suited to host applications.
- Make sure to use a card that provides at least 32 GB.
- SD Card reader. This is already part of most laptops, but you can purchase a [standalone USB adapter](#) if you don't have one. The brand doesn't matter, just pick the cheapest.
- [Ethernet cable](#). Required for installation. After installation, Home Assistant can work with Wi-Fi, but an Ethernet connection is more reliable and highly recommended.



**Note:** Remember to ensure you're using an [appropriate power supply](#) with your Raspberry Pi. Mobile chargers may not be suitable, since some are designed to only provide the full power with that manufacturer's handsets. USB ports on your computer also will not supply enough power and must not be used.

# Install Home Assistant Operating System

---

This guide shows how to install the Home Assistant Operating System onto your Raspberry Pi using Raspberry Pi Imager.

## Write the image to your SD card

1. Download and install the Raspberry Pi Imager on your computer as described under <https://www.raspberrypi.com/software/>.
2. Troubleshooting: If Raspberry Pi Imager is not supported by your platform, you can [download the Home Assistant image](#) and use another imaging tool, such as Balena Etcher.
3. Open the Raspberry Pi Imager and select **OS**. Open Raspberry Pi Imager
4. Choose the operating system type:
5. Select **Other specific-purpose OS > Home automation > Home Assistant**. Choose the operating system type: Other specific-purpose OS
6. Choose the Home Assistant OS that matches your hardware (RPi 3, RPi 4, or RPi 5). Choose the Home Assistant OS
7. Choose the storage:
8. Insert the SD card into the computer. Note: the contents of the card will be overwritten.
9. Select your SD card. Select the storage
10. Write the installer onto the SD card:
11. To start the process, select **Next**, then choose **Write**.
12. Wait for the Home Assistant OS to be written to the SD card. Select write
13. Select **Finish** and eject the SD card.

## Start up your Raspberry Pi

1. Insert the SD card into your Raspberry Pi.
2. Plug in an Ethernet cable and make sure the Raspberry Pi is connected to the same network as your computer and is connected to the internet.
3. Connect the power supply to start up the device.

## Access Home Assistant

Within a few minutes after connecting the Raspberry Pi, you will be able to reach your new Home Assistant.

- In the browser of your desktop system, enter [homeassistant.local:8123](http://homeassistant.local:8123).

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your Raspberry Pi's IP address).

- The time it takes for this page to become available depends on your hardware. On a Raspberry Pi 4 or 5, this page should be available within a minute.
- If it does not show up after 5 minutes on a Pi 4 or 5, maybe the image was not written properly.
  - Try to flash the SD card again, possibly even try a different SD card.
- If this did not help, view the console output on the Raspberry Pi.
  - To do this, connect a monitor via HDMI.

Congratulations! You finished the Raspberry Pi setup!

## Downloading the Home Assistant image

If Raspberry Pi Imager is not supported by your platform, you can download the Home Assistant image and use another imaging tool, such as Balena Etcher.

To download the image to your computer, copy the correct URL for the Raspberry Pi 4 or 5 (Note: there are two different links below!):

{% tabbed\_block %}

{% endtabbed\_block %}

With the Home Assistant Operating System installed and accessible, you can now continue with onboarding.

[Include not found: getting-started/next\_step.html step="Onboarding" link="/getting-started/onboarding/"]

*We get commissions for purchases made through links in this post.*

---

## Installation/Troubleshooting

---

It can happen that you run into trouble while installing and onboarding Home Assistant. This page is here to help you solve the most common problems.

# Can't access Home Assistant in my browser

---

## Symptom: "This site can't be reached"

When trying to access Home Assistant in the browser, the browser shows the message "This site can't be reached".

## Description

This means the browser can't find your Home Assistant installation on the network.

## Resolution

To resolve this issue, try the following steps:

1. Make sure your Home Assistant device is powered up (LEDs are on).
2. Make sure your Home Assistant installation is connected to the internet:
3. Make sure the Ethernet cable is plugged-in to both Home Assistant and to your router or switch.
4. Make sure your network has internet access.
  - During first startup, time will be synchronized. Ensure NTP is allowed in your network.
  - During first startup, Home Assistant completes the installation. It needs access to the following URLs:
  - `version.home-assistant.io`: to fetch new version information.
  - `github.com`: to update metadata of the Home Assistant app store.
  - `ghcr.io`: the GitHub container registry to fetch new Home Assistant updates.
5. Make sure the system on which you opened the browser to access Home Assistant is connected to the same network as Home Assistant.
6. For example, if the system your Browser runs on is using Wi-Fi, make sure it is using the same Wi-Fi Home Assistant is connected to.
7. Make sure you typed the address correctly.
8. Especially if the message includes the error code "ERR\_CONNECTION\_REFUSED", it is likely that there was a typo in the port part of the URL ( `:8123` ).
9. Typically, the URL is `http://homeassistant.local:8123`.
10. If you are running an older Windows version or have a stricter network configuration, try `http://homeassistant:8123` instead.
11. The system might still be starting up. Wait for a couple of minutes and refresh the page.
12. Refreshing might work differently depending on your browser. Look for the refresh `mdi:refresh` icon, or press CTRL+R or CTRL+SHIFT+R.
13. Check your router's web interface to see what IP address is assigned to your Home Assistant installation.
14. Enter this IP address ( `http://x.x.x.x:8123` ) directly into your browser.

15. If you still can't reach Home Assistant, connect keyboard and monitor to the device Home Assistant is running on to access the console and see where Home Assistant gets stuck.
16. If you are using a Home Assistant Green, follow these steps [to access the console](#).
17. If you are using a Home Assistant Yellow, follow these steps [to access the console from Windows](#), or [to access the console from Linux or macOS](#).
18. [Reach out to our community for help](#).



## "Error installing Home Assistant"

---

### Symptom: During onboarding, there is an "Error installing Home Assistant"

You are in the onboarding procedure, but you get the message **Error installing Home Assistant**.

Error installing Home Assistant during onboarding

### Resolution

1. Make sure your network has internet access.
  - During first startup, time will be synchronized. Ensure NTP is allowed in your network.
  - During first startup, Home Assistant completes the installation. It needs access to the following URLs:
  - version.home-assistant.io: to fetch new version information.
  - github.com: to update metadata of the Home Assistant app store.
  - ghcr.io: the GitHub container registry to fetch new Home Assistant updates.
2. After changing your network environment, wait a few minutes. Home Assistant will try to reconnect.
3. [Reach out to our community for help.](#)

## Stuck at "Preparing Home Assistant"

---

### Symptom: Onboarding seems stuck at "Preparing Home Assistant"

You are in the onboarding procedure, but the process seems stuck at the step **Preparing Home Assistant**.

Home Assistant preparation

### Resolution

1. Select **Show details** to view the log files.
  2. The log files might provide more information on the current status.
  3. Make sure your network has internet access.
    - During first startup, time will be synchronized. Ensure NTP is allowed in your network.
    - During first startup, Home Assistant completes the installation. It needs access to the following URLs:
    - version.home-assistant.io: to fetch new version information.
    - github.com: to update metadata of the Home Assistant app store.
    - ghcr.io: the GitHub container registry to fetch new Home Assistant updates.
  4. After changing your network environment, wait a few minutes. Home Assistant will try to reconnect.
  5. [Reach out to our community for help](#).
-

## Installation/Windows

---

Included sections for this page is located under source/\_includes/installation

# Install Home Assistant Operating System

---

Follow this guide if you want to get started with Home Assistant easily or if you have little to no Linux experience.

## Suggested hardware

---

You will need a few things to get started with installing Home Assistant. The links below lead to Ameridroid. If you're not in the US, you should be able to find these items in web stores in your country.

To get started, we suggest the ODROID-N2+, the board that powers our [Home Assistant Blue](#), or the ODROID-M1.

If unavailable, we also recommend the [ODROID-C4](#).

Home Assistant bundles (US market):

The bundles come with Home Assistant pre-installed.

- [ODROID-N2+: 2 GB RAM / 16 GB eMMC](#)
- [ODROID-N2+: 4 GB RAM / 64 GB eMMC](#)
- [ODROID-M1: 4 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 256 GB NVMe / 16 GB  \$\mu\$ SD or 16 GB eMMC](#)
- [ODROID-M1: 8 GB RAM / 1 TB NVMe / 64 GB eMMC](#)

Variants without pre-installed Home Assistant:

- [ODROID-N2+, 2 GB RAM or 4 GB RAM](#)
- [ODROID-C4](#)
- [ODROID-M1](#)
- [ODROID-M1S, 4 GB RAM or 8 GB RAM](#)

Related components:

- [Power Supply](#)
- [CR2032 Coin Cell](#)
- [eMMC Module](#)
- [Case](#)

*These are affiliated links. We get commissions for purchases made through links in this post.*

### Important: Prerequisites

This guide assumes that you have a dedicated PC to exclusively run the Home Assistant Operating System.

- This is typically an Intel or AMD-based system.
- The system must be 64-bit capable and be able to boot using UEFI.
- Most systems produced in the last 10 years support the UEFI boot mode.

### Summary

1. First, you will need to configure your PC to use UEFI boot mode.
2. Then, write the Home Assistant Operating System disk image to your boot medium.

## Configure the BIOS on your x86-64 hardware

---

To boot Home Assistant OS, the BIOS needs to have UEFI boot mode enabled and Secure Boot disabled. The following screenshots are from a 7th generation Intel NUC system. The BIOS menu will likely look different on your system. However, the options should still be present and named similarly.

1. To enter the BIOS, start up your x86-64 hardware and repeatedly press the **F2** key (on some systems this might be **Del**, **F1** or **F10**). Enter BIOS using F2, Del, F1 or F10 key
2. Make sure the UEFI Boot mode is enabled. Enable UEFI Boot mode
3. Disable Secure Boot. Disable Secure Boot mode
4. Save your changes and exit.

The BIOS configuration is now complete.

## Write HAOS onto your x86-64 hardware

---

Next, you need to write the Home Assistant Operating System image to the *boot medium*, which is the medium your x86-64 hardware will boot from when it is running Home Assistant.

 **Note:** HAOS has no integrated installer that writes the image automatically. You will write it manually using either the **Disks** utility from Ubuntu or Balena Etcher.

Typically, an internal medium like S-ATA hard disk, S-ATA SSD, M.2 SSD, or a non-removable eMMC is used for the x86-64 boot medium. Alternatively, an external medium can be used such as a USB SDD, though this is not recommended.

To write the HAOS image to the boot medium on your x86-64 hardware, there are 2 different methods:

**Method 1 (recommended):** Boot Ubuntu from a USB flash drive and install the Home Assistant Operating System from there. It also works on laptops and PCs with internal hard disks.

**Method 2:** With this method, you write the Home Assistant Operating disk image directly onto a boot medium from your regular computer. The steps are a bit more complex. If you have non-removable internal mediums (for example because you are using a laptop) or do not have the necessary adapter (for example an USB to S-ATA adapter) use method 1 instead.

### Method 1: Installing HAOS via Ubuntu booting from a USB flash drive

#### Required material

- Computer
- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- USB flash drive (USB thumb drive is sufficient, it should be at least 8 GB in size)
- Internet connection

#### To install HAOS via Ubuntu from a USB flash drive

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before carrying out this procedure.
4. Create a *live operating system* on a USB flash drive:
5. Follow the [Ubuntu Desktop instructions](#) on writing an Ubuntu Desktop iso file onto a USB device.
6. Insert the USB flash drive into the system on which you want to run Home Assistant.
7. Boot the live operating system.
8. You might need to adjust boot order or use F10 (might be a different F-key depending on the BIOS) to select the USB flash drive as boot device.

9. When prompted, make sure to select **Try Ubuntu**. This runs Ubuntu on the USB flash device.
10. The system then starts Ubuntu.
11. Connect your system to your network and make sure it has internet access.
12. In Ubuntu, open a browser and open the current documentation page, so you can follow the steps.
13. From there, [download the image](#).
14. In Ubuntu, in the bottom left corner, select **Show Applications**.
15. In the applications, search and open **Disks** and start restoring the HAOS image:
16. In **Disks**, on the left side, select the internal disk device you want to install HAOS onto.
17. On top of the screen, select the three dots `mdi:dots-vertical` menu and select **Restore Disk Image....** Restore disk image: select three dots menu
18. Select the image you just downloaded. Restore disk image: select image
19. Select **Start Restoring....** Restore disk image: start restoring
20. Confirm by selecting **Restore**. Restore disk image: select Restore
  - If you are getting an **Error unmounting filesystem** error message, stating that the **target is busy**:
    - Most likely, you are running Ubuntu on your internal disk. Instead, you need to run it on your stick.
      - Go back to step 3 and during start up, make sure you select **Try Ubuntu** (and NOT **Install Ubuntu**).
    - Another issue may be that live Ubuntu is using the Swap partition of an existing Linux installation.
      - If you see "Swap" listed as a partition on the drive you're going to install HAOS, just select the Swap partition, then press the stop button to unmount it and try the restore operation again.
21. In the partitions overview, you should now see the restore operation in progress.
  - The Home Assistant Operating System is now being installed on your system. Restore disk image: Restoring...
22. Once the Home Assistant Operating System is installed, shut down the system.
23. Once Ubuntu has been shut down, remove the USB flash drive (Ubuntu will inform you when this is the case).
24. Your Home Assistant server is now set up and you can start using it.
25. To use it, proceed as described under [start up your generic x86-64](#).

## Method 2: Installing HAOS directly from a boot medium

 **Note:** Use this method only if Method 1 does not work for you.

### Required material

- Computer



- The target x86-64 hardware, on which you want to install the Home Assistant Operating System (HAOS)
- Boot medium
- Internet connection

## Write the image to your boot medium

## Write the image to your boot medium

1. **Notice:** This procedure will write the Home Assistant Operating System onto your device.
2. This means you will lose all the data as well as the previously installed operating system.
3. Back up your data before continuing with the next step.
4. Attach the Home Assistant boot medium () to your computer. If you are using ODROID-M1, note that booting from NVMe is not supported. If you want to boot from eMMC, [update the firmware](#) before installing the image.

If you are using a [Home Assistant Blue](#) or ODROID-N2+, you can [attach your device directly](#).

If you are using an ODROID-M1S, you need to follow this guide to [boot your device into UMS mode](#).

5. Download and start [Balena Etcher](#). You may need to run it with administrator privileges on Windows.
6. Download the image to your computer.
7. Copy the URL for the image.
8. If there are multiple links below, make sure to select the correct link for your version of .

```
{% endtabbed_block %}
```

```
```text
```

```
//haos_-.img.xz ```
```

```
{% endif %}
```

Select and copy the URL or use the "copy" button that appears when you hover it. 5. Paste the URL into your browser to start the download. 6. Extract the file you just downloaded. 7. Select **Flash from file** and select the image you just extracted. - Do not use **Flash from URL**. It does not work on some systems. Screenshot of the Etcher software showing flash from URL selected. 8. **Select target**. Screenshot of the Etcher software showing the select target button highlighted. 9. Select the boot medium () you want to use for your installation. Screenshot of the Etcher software showing the targets available. 10. Select **Flash!** to start writing the image. - If the operation fails, decompress the .xz file and try again. Screenshot of the Etcher software showing the Flash button highlighted. - When Balena Etcher has finished writing the image, you will see a confirmation. Screenshot of the Etcher software showing that the installation has completed.

Start up your

- If you used method 1 for the installation, make sure the USB flash drive is removed from the system.

- If you used method 2 for the installation, install the boot medium into your x86-64 hardware.
- Plug in an Ethernet cable that is connected to the network and to the internet.
- Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
- Power the system on. If you have a screen connected to the system, after a minute or so the Home Assistant welcome banner will appear in the console.

 **Note:** If the machine complains about not being able to find a bootable medium, you might need to specify the EFI entry in your BIOS. This can be accomplished either by using a live operating system (for example, Ubuntu) and running the following command (replace `<drivename>` with the appropriate drive name assigned by Linux, typically this will be `sda` or `nvme0n1` on NVMe SSDs):

```
text efibootmgr --create --disk /dev/<drivename> --part 1 --label "HAOS" \
--loader '\EFI\BOOT\bootx64.efi'
```

The efibootmgr command will only work if you booted the live operating system in UEFI mode, so be sure to boot from your USB flash drive in this mode. Depending on your privileges on the prompt, you may need to run efibootmgr using sudo.

Or else, the BIOS might provide you with a tool to add boot options, there you can specify the path to the EFI file:

```
text \EFI\BOOT\bootx64.efi
```

1. Insert the boot medium () you just created.
2. Plug in an Ethernet cable that is connected to the network and to the internet and power the system on.
3. Note: Internet is required because the newly installed Home Assistant OS does not contain all Home Assistant components yet. It downloads the latest version of Home Assistant Core on first start.
4. In the browser of your desktop system, within a few minutes you will be able to reach your new Home Assistant at homeassistant.local:8123.

 **Note:** If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at homeassistant:8123 or <http://X.X.X.X:8123> (replace X.X.X.X with your 's IP address).

Download the appropriate image

- VirtualBox (Intel chip) (.vdi)
- VirtualBox (Apple Silicon chip) (.vdi)
- KVM (.qcow2)

- [KVM/Proxmox](#) (.qcow2)
- [VMware ESXi/vSphere](#) (.ova)
- [VMware Workstation](#) (.vmdk)
- [Hyper-V](#) (.vhdx)

After downloading, decompress the image. If the image comes in a ZIP file, for example, unzip it.

Follow this guide if you already are running a supported virtual machine hypervisor. If you are not familiar with virtual machines, install Home Assistant OS directly on a [Home Assistant Yellow](#), a [Raspberry Pi](#), or an [ODROID](#).

- If VirtualBox is not supported on your Mac, and you have experience using virtual machines, you can try running the Home Assistant Operating System on [UTM](#).

Create the virtual machine

Load the appliance image into your virtual machine hypervisor. (Note: You are free to assign as much resources as you wish to the VM, please assign enough based on your app needs).

Minimum recommended assignments:

- 2 GB RAM
- 2vCPU

All these can be extended if your usage calls for more resources.

Hypervisor specific configuration

{% tabbed_block %}

- title: VirtualBox content: |

Create the virtual machine

1. Open VirtualBox and select the **New** button (the blue star).
2. **Name:** Type Home Assistant.
3. **ISO Image:** Leave this as **None** or **Empty**.
4. **Type & Version:** Select **Linux**, then select **Oracle Linux (64-bit)** (or **ARM 64-bit** if you are using a Mac with an M1/M2/M3 chip).
5. Select **Next**.

Configure hardware

1. **Base Memory:** Move the slider to at least **2048 MB** (2GB).
2. **Number of CPUs:** Move the slider to at least **2**.
3. **EFI:** Check the box for **Enable EFI (special OSes only)**. This is required for Home Assistant to boot.
4. Select **Next**.

Finalizing the wizard

1. On the **Virtual Hard Disk** screen, leave the settings as they are (it will suggest creating a new disk). We will swap this for your downloaded file in the next step.
2. Select **Finish**.

Attach the Home Assistant disk (VDI)

1. Select your new "Home Assistant" VM in the left-hand list and select the **Settings** icon (the orange gear).
2. Go to the **Storage** section on the left menu.
3. In the **Storage Devices** list, you will see a disk already listed under **Controller: SATA**. Right-click that disk and select **Remove Attachment**. This removes the empty placeholder disk.
4. Select the **Add Hard Disk** icon (the small disk with a green plus symbol) located next to the words **Controller: SATA**.
5. In the window that pops up, select the **Add** button at the top.
6. Find and select the `.vdi` file you previously downloaded and unzipped.
7. Select **Choose** to confirm the file.

Configure network

1. While still in the **Settings** window, go to the **Network** section.
2. Change the **Attached to** setting from **NAT** to **Bridged Adapter**.
3. Under **Name**, select the adapter you use for internet access. This allows Home Assistant to talk to other devices in your home.
4. Select **OK**.

`mdi:alert-outline` By default, VirtualBox does not free up unused disk space. To automatically shrink the vdi disk image the `discard` option must be enabled using your host machine's terminal:

```
bash VBoxManage storageattach <VM name> --storagectl "SATA" --port 0 --device 0 --nonrotational on --discard on
```

More details can be found about the command can be found [here](#).

• title: Unraid content: |

1. Download the **.qcow2** image above and decompress it. (**Extract all** in Windows)
2. Store the image in the **isos** share on your server.
3. Make sure under **Settings > VM manager, Enable VMs** is enabled.
4. Create a new virtual machine: **VMS > Add VM**.
5. Select type **Linux** and give the VM a name and a description.
6. Select the CPU cores you want to let the VM use and give it some memory.
7. Under **Primary vDisk Location**, select **Manual** and then select the qcow2 image.
8. Select your keyboard language under **VM Console Keyboard**.
9. Select **br0** under **Network Source**.

10. Select **virtio** under **Network model**.
 11. Select any USB-devices that you want to pass through to Home Assistant, such as Zigbee- or Z-Wave controllers.
 12. Deselect **Start VM after creation**.
 13. Select **Create**.
 14. Select the name of your new VM and select the capacity number for your disk. Here, you can expand the disk to whatever your needs are. The default is 32 GB.
 15. Select the icon of your new VM and select **start with console (VNC)**.
- title: KVM (virt-manager) content: |
 1. Create a new virtual machine in `virt-manager`.
 2. Select **Import existing disk image**, provide the path to the QCOW2 image above.
 3. Choose **Generic Default** for the operating system.
 4. Check the box for **Customize configuration before install**.
 5. Under **Network Selection**, select your bridge.
 6. Under customization select **Overview > Firmware > UEFI x86_64: ...**. Make sure to select a non-secureboot version of OVMF (does not contain words such as `secure` or `secboot`), for example `/usr/share/edk2/ovmf/OVMF_CODE.fd`.
 7. Select **Add Hardware** (bottom left), and select **Channel**.
 8. Select device type: **unix**.
 9. Select name: **org.qemu.guest_agent.0**.
 10. Finally, select **Begin Installation** (upper left corner).
 - title: KVM (virt-install) content: |

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi
```

`mdi:alert-outline` If you have a USB dongle to attach, you need to add the option `--hostdev busID.deviceId`. You can discover these IDs via the `lsusb` command. As example, if `lsusb` output is:

```
bash Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 004: ID 30c9:0052 Luxvisions Innotech Limited Integrated RGB Camera Bus 003 Device 003: ID 1a86:55d4 QinHeng Electronics SONOFF Zigbee 3.0 USB Dongle Plus V2 Bus 003 Device 002: ID 06cb:00fc Synaptics, Inc. Bus 003 Device 005: ID 8087:0033 Intel Corp. Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

You can recognize the Sonoff dongle at `Bus 003 Device 003`. So the command to install the VM will become:

```
bash virt-install --name haos --description "Home Assistant OS" --os-variant=generic --ram=4096 --vcpus=2 --disk <PATH TO QCOW2 FILE>,bus=scsi --controller type=scsi,model=virtio-scsi --import --graphics none --boot uefi --hostdev 003.003
```

Note that this configuration (bus 003, device 003) is just an example, your dongle could be on another bus and/or with another device ID. Please check the correct IDs of your USB dongle with `lsusb`.

{% endunless %}

- title: VMware Workstation content: |

1. Start VMware Workstation and select **Create a New Virtual Machine**.
2. Note: the exact name and location of the settings below depend on the VMware version. This procedure is based on version 17.
3. Select **I will install the operating system later**, then select **Linux > Other Linux 5.x kernel 64-bit**.
4. Give the VM a name, `home-assistant`, and define an easy to reach storage location, such as `C:\home-assistant`.
5. Specify the disk size and select **Store virtual disk as a single file**.
6. Select **Customize Hardware**.
7. Define the amount of memory and the number of cores the VM is allowed to use.
8. Remove the **New CD/DVD** entry. It will not be used.
9. Connect an Ethernet cable and make sure it is connected to your network.
10. Under **Network adapter**, select **Bridged: Connected directly to the physical network**.
11. Make sure **Replicate physical network connection state** is not selected.
12. Select **Configure Adapters**.
13. Make sure all virtual adapters and Bluetooth devices are deselected.
14. Select your host network adapter. Most likely, this is one of the first 2 checkboxes in the list:
 - Select the one for Ethernet.
 - The exact names of these adapters depend on your hardware.
15. At the end of the wizard, select **Finish**.

Edit the VM settings

1. In Windows Explorer, go to the storage location of your newly created VM, for example under `C:\home-assistant`.
2. Delete the `home-assistant.vmdk` file.
3. In the `Downloads` folder, find the `haos_ova_xx.x.vmdk` file.
 - If you haven't unzipped the archive, unzip it.
 - Within the folder, find the `.vmdk` file and rename it to `home-assistant.vmdk`.
 - Paste the file (not the unzipped folder) into the `C:\home-assistant` folder.
4. Right-click the `.vmx` file and select **Open with > Notepad**.
5. Under `.encoding`, add a line. Enter `firmware = "efi"`.
6. Now continue with the next step to start your VM.
 - If you see a message about side channel mitigations, select **OK**.

- If you see a message stating that the `.vmdk` file could not be found, in step 3, you likely pasted the folder, not the file. Repeat step 3.
- title: VMware ESXi/vSphere content: | Use the `E1000` or `E1000E` virtual network adapter. There are confirmed mDNS/Multicast discovery issues when using VMware's `VMXnet3` virtual network adapter.
- title: Hyper-V content: | ⚠ Hyper-V does not have USB support.
 1. Create a new virtual machine.
 2. Select **Generation 2**.
 3. Select **Connection > Your Virtual Switch that is bridged**.
 4. Select **Use an existing virtual hard disk** and select the VHDX file from above.

After creation, go to **Settings > Security** and deselect **Enable Secure Boot**.

{% endtabbed_block %}

Start up your virtual machine

1. Start the virtual machine.
2. Observe the boot process of the Home Assistant Operating System.
3. Once completed, you will be able to reach Home Assistant on `homeassistant.local:8123`. If you are running an older Windows version or have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address). If you have a stricter network configuration, you might need to access Home Assistant at `homeassistant:8123` or `http://X.X.X.X:8123` (replace X.X.X.X with your virtual machine's IP address).

{% endif %}

With the Home Assistant Operating System installed and accessible, you can continue with onboarding.

[Include not found: getting-started/next_step.html step="Onboarding" link="/getting-started/onboarding/"]

Flashing an ODROID-N2+

Home Assistant can be flashed to an ODROID-N2+ by connecting the device directly to your computer via the USB-OTG connection on the front of the board. The device contains the Petitboot bootloader, which allows the ODROID-N2+ storage to show up as it were a USB drive.

All these instructions work the same for the ODROID-N2 (non-plus version).

What you will need

To flash your eMMC using Petitboot and OTG-USB, you will need the following items:

- HDMI cable and monitor
- USB keyboard
- USB 2.0 to micro-USB cable
- If your board came in a Home Assistant Blue: No.2 hex key to open the case

Enabling SPI boot mode

To enable the SPI boot mode:

1. Power off the ODROID-N2+ by unplugging the power cable.
2. Remove the case.

Photo of the removed case

1. Locate the toggle for boot mode and switch it from MMC to SPI.

Photo of the SPI toggle switch

1. Connect the ODROID-N2+ directly to your computer via the USB-OTG port located on the front of the board.
2. Connect a USB keyboard and a monitor (using HDMI) to your ODROID-N2+.
3. Plug in the power cable to power on the ODROID-N2+.

Enabling USB drive mode

After The ODROID-N2+ is set to SPI boot mode and powered on, it boots into a terminal. To enable the USB drive mode:

1. Select `Exit to shell` from the menu.

Exit to shell



Note: When using the command line, it may return the following message:
`can't access tty; job control turned off.` You can safely ignore this message and proceed with the installation

1. Use the following command at the console to confirm the storage device node:


```
bash ls /dev/mmc*
```

1. Set the storage device on the ODROID-N2+ as a mass storage device using the `ums` command (USB Mass storage mode). This will configure the ODROID-N2+ and OTG to act as a memory card reader:

```
bash ums /dev/mmcblk0
```

Flashing Home Assistant

1. Connect the ODROID-N2+ to your PC via the micro-USB port at the front of the ODROID-N2+.
2. When the ODROID-N2 is recognized as a USB connected storage device, you can flash the eMMC with [Etcher](#).
3. Use the latest stable version of Home Assistant OS for the [ODROID-N2+](#) (haos_odroid-n2-.img.xz).
4. In Balena, use **Flash from file**. **Flash from URL** does not work on all systems.
5. When the flash process is complete, disconnect the ODROID-N2+ from your PC.
6. Remove the power cable.
7. Remove the USB and HDMI cables.
8. Make sure to toggle the boot mode switch back to MMC.
9. Put the ODROID back in its case.
10. Connect your ODROID-N2+ to your network with an Ethernet cable, make sure there is internet access, and plug in power.
11. If your router supports mDNS, you can reach your installation at `http://homeassistant.local:8123`.
12. If your network doesn't support mDNS, you'll have to use the IP address of your ODROID-N2+ instead of `homeassistant.local`. For example, `http://192.168.0.9:8123`.
13. You should be able to find the IP address of your ODROID-N2+ from the admin interface of your router.
14. Continue with [onboarding](#).

Flashing an ODROID-M1S

Home Assistant can be flashed to an ODROID-M1S by connecting the device directly to your computer via the USB-OTG connection on the front of the board. Unlike other ODROID boards, the M1S does not have a socket for an optional eMMC module. It also does not have a separate flash chip that holds a dedicated bootloader. Instead, the M1S has a built-in 64GB eMMC soldered directly on the board that holds a bootloader by default. This guide will show you how to install the Home Assistant Operating System to the built-in eMMC.

Warning: Installing Home Assistant OS replaces the firmware and SPL on the eMMC with the mainline version provided by the Home Assistant OS. As a result, it is not possible to use the SD card with the EMMC2UMS image anymore, because the mainline SPL is not compatible with U-Boot in the EMMC2UMS image at this time (February 2024). This does not pose any problem for standard use, just makes it more complicated in case you want to return to the Hardkernel-provided OS (see [HK Recovery](#)).

What you will need

To flash your eMMC using USB-OTG, you will need the following items:

- A small SD card
- Another computer
- USB 2.0 to micro-USB cable
- the special Hardkernel eMMC-to-USB-mass-storage image

Boot into mass-storage mode

(These steps are identical to the official [Hardkernel wiki](#) page.)

1. Download [ODROID-M1S_EMMC2UMS.img](#).
2. Use [balena Etcher](#) or another tool to flash the UMS utility onto an SD card.
3. Use **Flash from file**. **Flash from URL** does not work on all systems. (balena Etcher will complain that something went wrong during flashing. You can ignore this message)
4. Plug-in that SD card to your ODROID-M1S and boot it.

Flashing Home Assistant M1S

1. Download the latest stable version of Home Assistant OS for the [ODROID-M1S](#).
2. Apart from the HAOS image to flash (M1S instead of N2+ version), you can follow the N2+ step-by-step flashing guide [HERE](#).

HK Recovery

If you want to restore your M1S back into Hardkernel's initial state, you will have to restore the HK's bootloader. A reliable way of reflashing the eMMC with an operating system of your choice is to use Home Assistant OS to flash the EMMC2UMS image which turns the ODROID-M1S into

USB Mass Storage device. Once you have flashed the EMMC2UMS image, you can flash any OS again. You will need a micro USB cable to connect ODROID-M1S to PC.

Note: This commands will render your current Home Assistant OS installation unbootable!

Use the local terminal (HDMI/keyboard) to access the system console. On the Home Assistant CLI (command line), enter `login` to enter the root shell and use `curl` to download an image and `dd` it to the eMMC block device:

```
curl -L -A "Mozilla/5.0" https://dn.odroid.com/RK3566/ODROID-M1S/Installer/ODROI
```

This way, the device will start in the UMS mode on the next boot with the SD card removed. Follow the [Install over USB from PC](#) to install a different operating system.

Ios/Dev Auth

los/Index

Ios/Nfc

This is a placeholder that is required to support Universal Links and NFC tags in the [Home Assistant Companion for iOS](#) app.

More Info/Backup Emergency Kit

[Backups](#) stored on Home Assistant Cloud are always encrypted using [AES-128](#). For backups stored on [other backup locations](#), you can choose whether to encrypt the backup. The backup emergency kit contains information needed to [restore the backup](#), such as the encryption key and metadata about the related backup.

What is encryption, and why are backups encrypted?

Encryption is a method of converting data into a coded format so that it can only be read by someone who has the encryption key. This ensures that your data about your home remains private. So even if someone else had a copy of your Home Assistant backup, it is unreadable for them without the encryption key.

Storing the backup emergency kit somewhere safe

1. To download the backup emergency kit, go to **Settings > System > Backups**.
2. If it is your first time defining backup settings, select **Setup automatic backup** and download the backup emergency kit.
3. You can also download the encryption key again later from the backup configuration page.

4. Store the kit somewhere safe, outside the Home Assistant system.
5. Home Assistant keeps track of the current encryption key. If you download from Home Assistant, it can decrypt your backup.
6. But if you have changed the encryption key in the meantime, you still need the old key that matches old backups. Without the encryption key, there is no way to [restore an encrypted backup](#).

Changing your encryption key

When you set up your [backups](#), an encryption key is generated automatically. This key is used for all your backups. You can replace this key with a new one, which will be used for all future backups. To decrypt backups created before the change, you will still need the previous encryption key.

1. Go to **Settings > System > Backups**.
2. Select **Configure automatic backups** and under **Encryption key**, select **Change**.
3. If you haven't downloaded the old emergency kit yet, do it now.
4. As the new encryption key won't work for the backups you've taken until now, keep it somewhere safe and make a note of which backups it applies to.
5. To generate a new encryption key, select **Next**, then select **Change encryption key**.
6. Download the new encryption key and store it in a safe place.

I lost my backup encryption key - how can I retrieve it?

If you still have access to your Home Assistant instance, you can download the encryption key again from the backup settings. There are 2 options:

- **Option 1: you haven't changed the encryption key:** Home Assistant still has it.
 - In Home Assistant, go to **Settings > System > Backups**
 - Under **Backup settings**, in the **Encryption key** section, either download your emergency kit or copy the key.
 - If you download the backup from the Home Assistant Backup page, it decrypts the backup on the fly.
 - **Option 2: you have changed the encryption key:** Home Assistant cannot decrypt it on the fly.
 - You need the encryption key that is related to that backup.
 - If you have lost the encryption key, and have no access to your Home Assistant instance, there is no way to restore the backup.
 - Nabu Casa does not store your encryption key and cannot provide support in decrypting backups if the key goes missing.
-

More Info/Dockerhub Rate Limit

The issue

Docker Hub enforces a limit on how often you can fetch container information from their container registry. For more details, see the [Docker Hub rate limit documentation](#).

Home Assistant uses Docker Hub as the container registry. When your IP address is rate limited, updating Home Assistant containers will fail.

The solutions

If you are running watchtower or similar solutions to keep your containers up to date, reconfigure them to check less often. If you are running a Supervised installation, you should also consider removing them completely, since running software alongside the Supervisor is [not supported](#).

Once you've done that, you need to wait until the limit is lifted. This can take up to 6 hours.

If you are sharing the IP address with other parties, their usage will also affect you. The Supervisor supports signing in to Docker Hub with an account. With an account, all fetches between the Supervisor and Docker Hub are authenticated and are not limited by the anonymous rate limits. Authenticated users are also rate limited, but that is a dedicated limit tied to your account.

If you do not have a Docker Hub account [you can create one here](#).

To use your Docker Hub credentials with the Supervisor:

1. You need to have the advanced user toggle enabled in your user profile setting.
2. Go to **Settings > Apps > Install app**.
3. In the top-right corner of the screen, select the three dots `mdi:dots-vertical` menu, and select **Registries**.
4. In the dialog that opens up, select **Add new registry** and enter `docker.io` as the registry, followed by your credentials:

Adding authentication for Docker Hub in the Supervisor panel. Adding authentication for Docker Hub in the Supervisor panel.

 **Note:** The screenshot above may show `hub.docker.com` as the registry value, but the correct value is `docker.io`. `hub.docker.com` is the website for browsing Docker Hub, while `docker.io` is the registry hostname the Supervisor uses to authenticate.

If you do not want to use the UI, this can also be done with the [CLI](#)

More Info/Free Space

Reaching your storage limit, this page will help you when that happens.

There are several things you can do to free up some space:

- [Clean the database](#)
- [Reduce space used for backups](#)
- [Uninstall unused apps](#)
- [Expand storage](#)

Viewing the available disk space

Follow these steps to check the available free disk space.

1. Go to **Settings > System > Storage**.
2. Under disk metrics, hover over the status bar to view the details.
3. `mdi:information-outline` The **Network storage** section only shows if you have [added network storage](#).

Screenshot of the "Move datadisk" feature

Cleaning the database

The Home Assistant database can become very large. Follow these steps to reduce the size of the database.

1. To view the size of the current database, go to **Settings > System > Repairs**.
2. Select the three dots `mdi:dots-vertical` menu and select **System information**.
3. Scroll down to the **Recorder**, and check the **Estimated database size (MiB)**.
4. [Purge the contents of the database](#).
5. To slow down the growth of the database, [filter](#) what you send to the database.
6. Change how long it stores the data, using the `purge_keep_days` [setting](#).

Reducing space used for backups

Deleting obsolete backups

Previous backups are not included when you create a new one. But they do take up space.

1. To delete old backups, follow the steps on [deleting obsolete backups](#).
2. Ideally, backups don't pile up on the system to begin with.
3. To define how long automatic backups should be kept on the system, follow the steps on [setting up an automatic backup process](#).

Storing backups outside of Home Assistant

Storing backups outside of Home Assistant makes sure they don't use space on Home Assistant to begin with. It also makes sure you can [restore Home Assistant from backup](#) in case you have an issue with your current installation. Follow the steps on [defining backup locations](#).

Uninstalling unused apps

Apps can take a lot of space, not just the app itself but also their data.

1. Go to **Settings > Apps**.
2. Look at your installed apps and identify the ones you no longer use.
3. To remove the app, select the app and select **Uninstall**.

Expanding storage

If the above steps to free up space did not help, you need to expand your storage.

Expanding storage: Home Assistant Operating System

When you are running Home Assistant Operating System, you can use the following options to expand your storage:

- Replace your current storage medium, for example, the SD card, with a bigger one. Use a backup to [restore Home Assistant from backup](#) on the new SD card.
- Use an external data disk

Expanding storage on VM

If you are running Home Assistant as a VM, look at the documentation for your hypervisor on how to expand disks for virtual machines. Home Assistant will auto-expand to use the newly added space.

More Info/Local Media/Add Media

Home Assistant includes a built-in media browser that lets you browse and play your local media files. This feature is available in all Home Assistant installation types.

On Home Assistant Operating System, the media folder is automatically created and ready to use. On Home Assistant Container, you need to mount a volume to `/media` when starting your container.

Accessing the media browser

The media browser is available on both Home Assistant Operating System and Home Assistant Container installations.

1. In the sidebar (left side of your Home Assistant interface), go to **Media > My media**.
2. From here, you can browse all media files in your `/media` folder and any custom media directories you've configured.

Adding media using the media browser on Home Assistant OS

You can add media files using the built-in media browser. This method is ideal for uploading a few files at a time. This method does not let you create folders.

1. Go to **Media > My media**.
2. In the top-right corner, select `mdi:cog` **Manage**.
3. Select **Upload** and choose files from your device.
4. If you uploaded the wrong file, select the file and choose **Delete**.

Removing media using the media browser on Home Assistant OS

You can remove media files using the built-in media browser. This method is ideal for deleting a few files at a time. This method does not let you delete folders.

1. Go to **Media > My media**.
2. In the top-right corner, select mdi:cog **Manage**.
3. Select one or more files and choose **Delete**.

Adding media using network file sharing on Home Assistant OS

If you are running Home Assistant Operating System, you can use the **Samba** app for bulk file transfers. This method is ideal for transferring large files or many files at once using your computer's file explorer (File Explorer, macOS Finder, or Linux file manager).

1. To install and use the Samba app, follow the instructions on [installing and using the Samba app](#).
2. After setup, you can copy or move media files onto your device using drag and drop in your file explorer.
3. You can also create folders to organize your media.

Other apps, such as **SSH**, also provide access to the media folders.

Adding media on Home Assistant Container

Prerequisites

- Home Assistant Container installation type.
- Before you can add media files on Home Assistant Container, you need to [set up a local media folder](#) by mounting a volume to the `/media` directory when starting your container. This must be done at container startup time.

To add media on Home Assistant Container

After you have mounted the volume, you can add media files directly to the folder on your host machine. This gives you direct file system access to add media files and create folders to organize them.

Your media will show up in the Home Assistant media browser automatically. You can manage your files in the following ways:

- Use the built-in media browser (as described above).
 - Use direct file system access on your host machine.
-

More Info/Local Media/Setup Media

Home Assistant includes a built-in media browser (in the sidebar under **Media** > **My media**) that lets you browse and play your local media files. Before you can use it, you need to make sure your media files are accessible to Home Assistant.

On Home Assistant Operating System, the `/media` folder is created automatically with no configuration needed. On Home Assistant Container, you need to mount a volume to `/media` when starting your container.

Files stored in your media directories are only accessible to users who are logged in to Home Assistant. This is different from the `www` folder, where files are publicly accessible without a login, which is useful for things like images in notifications, but not something you typically want for your personal media library.

Setting up a media folder on Home Assistant Operating System

No setup is required. The `/media` folder is automatically created and available in the media browser as soon as Home Assistant starts.

To add files, follow the [steps on adding media](#).

Setting up a media folder on Home Assistant Container

On Home Assistant Container, you need to mount a directory on your host machine to `/media` inside the container. This must be done when starting or recreating the container.

Using Docker CLI

Add `-v /PATH_TO_YOUR_MEDIA:/media` to your `docker run` command:

```
docker run -d \
  --name homeassistant \
  --privileged \
  --restart=unless-stopped \
  -e TZ=MY_TIME_ZONE \
  -v /PATH_TO_YOUR_CONFIG:/config \
  -v /PATH_TO_YOUR_MEDIA:/media \
  -v /run/dbus:/run/dbus:ro \
  --network=host \
  {{ site.installation.container }}:stable
```

Using Docker Compose

Add a volume entry for `/media` in your `compose.yaml` file:

```
services:
  homeassistant:
    container_name: homeassistant
    image: "{{ site.installation.container }}:stable"
    volumes:
      - /PATH_TO_YOUR_CONFIG:/config
      - /PATH_TO_YOUR_MEDIA:/media
      - /etc/localtime:/etc/localtime:ro
      - /run/dbus:/run/dbus:ro
    restart: unless-stopped
    privileged: true
    network_mode: host
    environment:
      TZ: Europe/Amsterdam
```

After restarting the container, your media files will appear in the media browser, under **Media > My media**.

To add files, follow the [steps on adding media](#).

Adding additional media directories

This applies to both Home Assistant Operating System and Home Assistant Container.

You can expose more than one media directory to the media browser. For example, a network storage path on Home Assistant OS, or an additional mounted volume on Home Assistant Container.

Prerequisites

- If you want to use media from a network storage, connect the network storage first.
- Refer to the [instructions on how to connect network storage](#).
- Once connected, the media from network storage is automatically added to the local media browser.

To add additional media directories

1. Open your `configuration.yaml` file.
2. Under `homeassistant:`, add a `media_dirs:` entry with one or more directories:

```
yaml homeassistant: media_dirs: media: /media recordings: /mnt/recordings
photos: /mnt/photos
```

Each key is the label that appears as the folder name in the media browser. For example, `recordings` will show up as "recordings" in the media browser, pointing to `/mnt/recordings` on disk.

3. Save the file and [reload the configuration](#) to apply the changes.
 4. To add files, follow the [steps on adding media](#).
-

More Info/Nest Auth Deprecation

The primary authentication method recommended by the Nest Home Assistant integration called *Desktop*, *Installed App* or *OOB* auth was deprecated for new uses on February 28th, 2022 and will be disabled for all users on October 3, 2022. See the [Google Developer blog](#) for announcement details.

Existing must upgrade to *Web Auth* credentials by October 3, 2022.

New Users

New users may sign up using *Web Auth* without issue. Follow the [documentation](#) which has been updated to use *Web Auth* and a *My Home Assistant* redirect URL using Home Assistant **2022.6** or newer.

Existing Users: App Auth

If you previously successfully configured Nest and Home Assistant with *App Auth* then follow the instructions for [Deprecated App Auth Credentials](#).

Nest is now configured entirely from the UI using [Application Credentials](#) and the configuration flow will walk you through the steps of creating new credentials the right way.

You will need to upgrade to *Web Auth* before October to avoid interruption.

Existing Users: Web Auth

Users who signed up using *Web Auth* are not affected by the App Auth deprecation. However, as of `2022.6` the *My Home Assistant* URL is now the default redirect URL and may need to be updated in the Google Cloud Console to avoid a `redirect_uri_mismatch` ([more info](#)).

Background

The OAuth out-of-band flow was designed to support native applications that cannot support a redirect URI like a Web application, which was convenient for Home Assistant since it is difficult for end Home Assistant users to set up SSL certificates and DNS needed for a secure Web endpoint. However, Google has deprecated the OOB flow as it introduces a phishing risk. New users are no longer allowed to create new Desktop auth credentials and existing users will no longer work starting October 3, 2022.

As of `2022.6` the *Web Auth* OAuth2 flow uses the *My Home Assistant* redirect URL which handles SSL. This is what allows new signups for *Web Auth* as the new recommended approach.

More Info/No Url Available

When Home Assistant serves you the "No URL Available" message, you are probably trying to set up or configuring an integration that requires your account to be linked.

What happened

The authentication method for linking such an account is called OAuth2, and is a modern and more common way of linking systems together, by logging in with the provider of the data.

This means Home Assistant will redirect you to the provider to log in. After the login is successful, the third-party provider will redirect you back to your Home Assistant instance.

For the third-party to be able to redirect/link you back to Home Assistant after authentication, Home Assistant needs to tell the third-party application about the URL to access Home Assistant on / to redirect back to. Unfortunately, if the "No URL Available" error occurs, Home Assistant hasn't been able to figure out that URL.

How to solve it

Home Assistant will try to find the URL you are currently using in your browser to use as this redirect URL for the authentication. However, Home Assistant is only able to determine this, if the URL is known in the configuration.

There are multiple options to consider:

Using Nabu Casa Home Assistant Cloud

If you have Nabu Casa's [Home Assistant Cloud](#), the easiest way to resolve this, is by visiting your Home Assistant instance from the remote URL.

Go to **Settings** > **Home Assistant Cloud**.

Visit your instance on the remote URL. Now you can set up the integration as normal, without getting the "No URL Available" message.

Using your instance by IP address

If you are on your home network, you can visit your instance using the IP address of your instance.

For example, `https://192.168.1.2:8123`

Replace the IP with the IP address of your Home Assistant instance. If you don't know this, you could also skip this and try the next solution.

Now you can set up the integration as normal, without getting the No URL Available message.

Configuring the instance URL

Another good solution is configuring the Home Assistant instance URL in Home Assistant. By letting Home Assistant know about the URL you use to access it, Home Assistant will be able to deal with that in a situation like this.

Please note, you'll need to enable advanced mode in your user profile in order to set this up.

Go to **Settings** > **System** > **Network**.

On this page, two fields that can resolve this issue: **Local Network** and **Internet**.

- **Local Network:** The URL you type in your browser when you are **at home**, connected to your home network. For example, `http://homeassistant.local:8123`
- **Internet:** The URL you type in your browser when you are **not home**, connected to your home network. For example, `https://example.duckdns.org`

Some additional notes:

- Don't use the Nabu Casa remote UI URL in any of these fields. The remote access will automatically be handled correctly.
- If the address you use is the same for both the "at home" and "not home" situations, it is recommended to only use the **External URL** field.

- The internal URL should preferably not use SSL (have `https://`) in it, as it might cause problems with casting to media devices.
- If you do not have an external address for your Home Assistant instance, leave that field empty.
- Ensure that the URL you provide matches the true address your browser uses to reach the instance. Many popular browsers will hide the www subdomain; if you try to configure `http://foo.bar` , but you're actually at `http://www.foo.bar` , OAuth will fail and you will receive this error. You can always check the actual domain by pasting `javascript:alert(document.location)` in your address bar and pressing enter.

After setting the URLs, click save. There is no need to restart Home Assistant your changes are applied immediately.

Now you can set up the integration as normal, without getting the No URL Available message.

More Info/Pwned Passwords

We are using the [Have I Been Pwned \(HIBP\)](#) service for detecting leaked or compromised secrets, like passwords.

If you get a warning about it, it means that you are using secrets in your configuration which have been leaked and are publicly known. It is strongly advised to change these secrets with a more secure alternative as soon as possible.

Please note; this feature does not send out your secrets to check this. Your secrets and privacy is guaranteed by a [K-Anonymity](#). Your secrets are hashed, the first 5 characters of the hash result are used to query Have I Been Pwned. Have I Been Pwned returns the results of possible password hashes that match, we check the last part of the password hash against this list locally.

[Read more about K-Anonymity on this CloudFlare blog post.](#)

More Info/Removed Integration

The integration you requested has been removed.

Possible reasons for the removal:

- It's unmaintained.
- It's no longer working.
- It does not comply with the rules described in our [ADR's](#).
- It does not follow our [design guidelines](#) for integrations.

For alternative integrations have a look at the list of our [integrations here](#).

More Info/Statistics

You're configuring a statistic but you couldn't find your source in the dropdown?

Check that it hasn't been excluded in the [Recorder](#) configuration.

Make sure the entity uses the same unit as the already selected sensors for that statistics graph. Because the graph uses a single Y axis, you cannot mix units (for example, °C and %, or kW and W) in the same graph.

Otherwise, It's caused by a bug in the integration providing the entity. Integrations need to configure their entities correctly so Home Assistant knows that we need to track statistics for it and how.

Open an issue with the author of the integration and link them to <https://developers.home-assistant.io/docs/core/entity/sensor#long-term-statistics>.

More Info/System Information

The **System information** dialog provides detailed technical information about your Home Assistant instance, including system architecture, operating system and version, installation type, Python version, frontend version, and more.

Viewing system information

1. Go to **Settings** > **System** > **Repairs**.
2. From the three dots `mdi:dots-vertical` menu (top right), select **System information**.

Screenshot showing the System information dialog

What information is available

The **System information** dialog provides system details, including:

- Home Assistant Core: Version, installation type, and Python version, architecture, operating system (kernel) version, and timezone
- Home Assistant Supervisor: Version and disk usage (if applicable)
- Home Assistant Operating System: Version and board type (if applicable)
- Home Assistant Cloud: Connection status and certificate information (if configured)
- Recorder: Database engine and estimated database size
- Network: Network configuration details
- Resources: Number of [Dashboards](#) and [Views](#)

This information is useful for:

- Troubleshooting issues with your installation
- Checking system resources before [creating backups](#)
- Verifying your installation meets [requirements for specific integrations](#)
- Sharing technical details when [reporting issues on GitHub](#)

Reporting issues on GitHub

When reporting issues on GitHub, you're often asked to provide system information.

Follow these steps to copy your system information:

1. In the bottom-right corner of the **System information** dialog, select the copy button.
2. This formats the data and places it on your clipboard.
3. Paste it into your GitHub issue.

Screenshot showing copy button in System information dialog

Viewing integration startup time

If Home Assistant is taking a long time to start, you can identify which integrations are causing delays and potentially have connectivity issues.

To view integration startup times, follow these steps:

1. Go to **Settings** > **System** > **Repairs**.
 2. From the three dots `mdi:dots-vertical` menu, select **Integration startup time**.
-

More Info/Unhealthy/Docker

Docker is at the core of most operations that the Supervisor does, it is important that it is configured properly and is working the way the Supervisor expects.

The Supervisor will be marked as unhealthy if any of these requirements are not met:

- [Running unsupported software](#)
 - [Running an unsupported Docker version](#)
 - [Running the Supervisor under LXC](#)
 - [Not running the Supervisor as privileged](#)
-

More Info/Unhealthy/Docker Gateway Unprotected

The issue

Home Assistant Supervisor was unable to apply firewall rules that restrict access to the internal Docker network gateway. As a safety measure, apps that use host networking will not start until this issue is resolved.

This can happen when:

- The `iptables` or `ip6tables` command is not available on the host system.
- The systemd service responsible for applying the rules failed to execute.
- There is a D-Bus communication issue with systemd.

The solution

Try rebooting your system by going to **Settings** > **System** > **Hardware**, opening the menu in the top right corner, and selecting **Reboot system**.

If the issue persists after a reboot, please [open an issue](#) and include the full Supervisor logs. You can download them from **Settings** > **System** > **Logs** by selecting **Supervisor** from the log source dropdown.

More Info/Unhealthy/Duplicate Os Installation

The issue

Multiple Home Assistant OS installations have been detected, which can lead to non-bootable systems, especially after updates. This problem typically occurs when you have installed Home Assistant OS multiple times on different storage devices (like SD cards, USB drives, or internal storage) that are all connected to the same device. This mismatch can lead to:

- Your system booting into an old version of Home Assistant OS even after updating.
- Complete boot failure, leaving your system unable to start.
- Other forms of unpredictable behavior.

The solution

Make sure you have only one Home Assistant OS installation on your system. To do that, follow these steps:

1. Create a [full backup](#) of your system.
2. Store the backup safely on another device.
3. Shut down your Home Assistant system and disconnect all storage devices.
4. [Install Home Assistant OS](#) on the storage device you want to use, wiping the other devices.
5. Upload the backup during the [onboarding](#) to restore your previous configuration.

Following these steps ensures that Home Assistant OS is installed on only a single storage device. In case you are using the [external data disk](#) feature, wipe this disk as well prior to starting the device for the first time with this storage device connected.

Note on devices with non-removable storage

If you have Home Assistant Yellow or any other device with non-removable storage, you may need to wipe this storage as well before proceeding with the installation. This method may be different depending on the device you are using.

For Home Assistant Yellow, the easiest way is to follow the [steps for reinstalling Home Assistant OS](#), but at the step where you would normally select the Home Assistant OS image, select the **Erase** option instead. Alternatively, once the device is initialized using the `rpiboot` utility, use any disk management tool to wipe the eMMC storage.

More Info/Unhealthy/Oserror Bad Message

The issue

The Supervisor received Operating System errors during file operation (Python OSError exceptions). The error number reported was 74, "Bad message", which is typically raised on file system corruption. File system corruption can happen due to a power outage or when the system otherwise did not get shut down properly.

The solution

Typically, Home Assistant recovers from file system corruption. If you see this error for the first time, ignore it. In case the problem gets worse, make sure to create a full backup, download it, or store it on a different system.

If this issue reappears even after a system restart, your system configuration or hardware might be at fault. Consider a clean reinstallation of your Operating System. If you are using an external USB drive, consider replacing the USB adapter or the SSD. If you are confident that the hardware is not at fault and you are using a Raspberry Pi, it might be related to incompatibility with certain UAS drives. For more background information, read this [Raspberry Pi forum post](#). The Home Assistant OS maintains a list of devices where UAS are disabled in order to work around the problem (check config.txt on the first partition of your installation). If the USB PID/VID of your device is missing, consider [opening an issue](#) in the Home Assistant OS project.

More Info/Unhealthy/Privileged

The issue

The Supervisor needs privileged access to the Docker runtime on your host so it can perform all required tasks.

The solution

If you are running an older version of our Home Assistant OS, update it in the Configuration panel.

If this is not our Home Assistant OS, your operating system might be out of date. Try checking for and installing updates, then restarting your system. If this doesn't work, you may need to re-run our [convenience installation script](#).

More Info/Unhealthy/Setup

The issue

This happens when any of the setup tasks fails to complete, this can be due to the host not being completely ready when the Supervisor starts or that [DBUS](#) is not properly working.

The solution

If the issue is related to DBUS, you will see an unsupported message about that as well; You can have a look [here](#) on how to resolve that.

If DBUS is not the problem, the first thing you should try is to restart the Supervisor.

This can also be done with the CLI, by running the following command:

```
ha supervisor restart
```

If this does not help or you do not have any way to access the CLI, you can try to reboot the host. This can be done by going to **Settings > System > Hardware**, opening the menu in the top right corner, and selecting "Reboot system".

To help us make the setup more robust, please enable the sharing of diagnostics and crash logs on the **Settings > System > Analytics** panel.

More Info/Unhealthy/Supervisor

The issue

This can happen when there was a network issue during the startup of the Supervisor.

The solution

Manually update the Supervisor, in case there is an update pending. This can be done from the updates panel.

This can also be done with the CLI, by running the following command:

```
ha supervisor update
```

In case this doesn't work. Check the logs, if there's an error where a host can't be resolved (eg: lookup ghcr.io: no such host) there might be a problem with your network settings (eg: dns server that can't be reached).

More Info/Unhealthy/Untrusted

The issue

Home Assistant detected untrusted code. It looks like someone/something injected not signed images or code into your Home Assistant system.

The solution

Please make sure your network is not affected by an intruder. We recommend reinstalling your installation and make sure only authorized persons have access to your instance.

The issue

AppArmor is how the Supervisor does handling all the security around add-ons, without this, the Supervisor is missing important security mechanics to protect your system and data within it.

The solution

If the AppArmor is not enabled on your host, add this to the Linux kernel boot parameters:

`apparmor=1 security=apparmor` and then reboot your operating system.

As a last resort, you might need to reinstall the host running the Supervisor with one of the supported operating systems, [see instructions here](#).

More Info/Unsupported/Cgroup Version

The issue

Supervisor prefers CGroups version 2 as it depends on its features to work properly.

It seems that on your system CGroups is not available or an unknown CGroups version is in use.

The solution

Make sure CGroups v2 is available and enabled on your operating system.

You should never see this issue on Home Assistant OS as all versions of the OS ship with a supported CGroup version.

More Info/Unsupported/Connectivity Check

The issue

Home Assistant needs to know when it has a stable network connection in order to disable functionality which requires that. Without this check you will face an increased number of errors and performance issues due to connection timeouts.

The solution

From the host shell, first execute the following:

```
busctl get-property org.freedesktop.NetworkManager /org/freedesktop/NetworkManag
```

Output is **b true**

You just need to re-enable connectivity checks by executing this command:

```
busctl set-property org.freedesktop.NetworkManager /org/freedesktop/NetworkManag
```

It may take a bit for the message to go away as all checks are scheduled on timers. You can force it to recheck immediately by executing the following commands:

```
ha host reload
ha resolution healthcheck
```

Output is **b false**

You need to set the connectivity uri in Network Manager's config. You can do this by adding the following to `/etc/NetworkManager/NetworkManager.conf`:

```
[connectivity]
uri=http://checkonline.home-assistant.io/online.txt
interval=600
```

Afterwards you will need to restart NetworkManager by either rebooting the host or executing this command:

```
systemctl restart NetworkManager
```

As mentioned above, the checks are on timers so the message may not go away immediately unless you force an immediate re-check. If you continue to see the message after a while or after forcing a re-check then start over at the top of this solution. You may need to separately enable the check now that it is available.

More Info/Unsupported/DBus

The issue

D-Bus is how the Supervisor does most of the communication with the host, without this, multiple things that the Supervisor needs to do will fail.

The solution

If the D-Bus daemon is not running, start it.

If that does not help, reboot your operating system.

As a last resort, you need to reinstall the host running the Supervisor with one of the supported operating systems, [see instructions here](#).

More Info/Unsupported/Dns Server

The issue

Home Assistant requires a working DNS server to function. Without one, it may be unable to provide functionality like checking for and executing updates, showing documentation, and reaching external services required by add-ons and integrations.

The solution

The easiest solution is to re-enable the fallback DNS option by executing the following command in the CLI:

```
ha dns options --fallback=true
```

Alternatively, review your system issues by executing the following command in the CLI:

```
ha resolution info
```

You will see one or more issues in the context of `dns_server`. For each such issue, take the following actions based on the issue type.

`dns_server_failed`

1. Ensure the DNS server is operating normally
2. Ensure the DNS server has internet access
3. Ensure the hostname `_checkdns.home-assistant.io` is not blocked

`dns_server_ipv6_error`

If you see this, it means the application you are using for DNS is not handling A and AAAA requests correctly. You can test this by executing the following commands:

```
server="<IP address of DNS server>"
dig "@$server" _checkdns.home-assistant.io +noall +comments +answer A
dig "@$server" _checkdns.home-assistant.io +noall +comments +answer AAAA
```

A DNS server handling A and AAAA requests correctly will respond with status `NOERROR` for both of those queries even though there are no answers for the AAAA request. A DNS server mishandling this request will only return a `NOERROR` response for the first one and will return `NXDOMAIN`, `REFUSED`, `SERVFAIL`, or some other error status for the second.

It is important to use a DNS server that handles this situation correctly since Home Assistant uses Alpine Linux for many of its containers. Alpine follows the DNS spec and will treat the entire domain as if it can't be resolved if it receives an error status for either query. Home Assistant will run into many unexpected issues in this situation, particularly around updates and installing software.

More Info/Unsupported/Docker Configuration

The issue

The Supervisor has some expectations of how the Docker daemon is configured to maintain the stability and performance of the host running the Supervisor.

The logging driver for the Docker daemon needs to be set to `journald` and the storage driver needs to be set to `overlay2`.

We only support cgroup version 1 on our hardware handling.

The solution

If you are running an older version of our Home Assistant OS, update it to the latest version in the Configuration panel.

If you are running Home Assistant Supervised, you need to modify the Docker daemon configuration on the host. The configuration is located at `/etc/docker/daemon.json`. If that file doesn't exist, you can create it and make sure it at least has the following contents:

```
{
  "log-driver": "journald",
  "storage-driver": "overlay2"
}
```

When the Docker configuration file is changed and saved, you need to restart the Docker service on the host machine.

To fix issues with the cgroup level, adjust the `/etc/default/grub`, add `systemd.unified_cgroup_hierarchy=false` to `GRUB_CMDLINE_LINUX_DEFAULT` and run `sudo update-grub`. After this change is made, you need to reboot the host completely.

You can also just re-run our [convenience installation script](#).

More Info/Unsupported/Docker Version

The issue

The version that is needed by the Supervisor depends on the features it needs to work properly.

The current minimum supported version of Docker is: `20.10.17`.

However, the feature set changes and improves over time. Therefore, the minimal required version may change. When that happens, it will be communicated before we publish a version that requires you to upgrade Docker.

The solution

If you are running an older version of Home Assistant OS, update it.

If this is not Home Assistant OS, you need to manually update Docker on your host. For instructions on how to do that, check the official [Docker documentation](#).

More Info/Unsupported/Home Assistant Core Version

The issue

Supervisor considers Home Assistant Core versions older than 24 months (approximately 24 releases) as unsupported. It is strongly recommend to keeping Home Assistant Core updated to the latest version. At a minimum, update within 6 months (6 release cycles) to ensure you don't miss automatic YAML integration migrations. Migration code that converts YAML-based integrations to modern UI-based configuration is only available for 6 months after deprecation, which corresponds to 6 Home Assistant Core releases.

On installations using an unsupported Home Assistant Core release, the Supervisor stops refreshing its update information. This means you will no longer receive updates for any component, including Home Assistant Core or app (formerly known as add-ons) updates.

The solution

1. Since you've not updated your system in a while, it is recommended to [create a backup](#) before updating your system.
 - Make sure to download the backup or store it in a remote location.
 2. Update your Home Assistant Core version.
 - Go to **System** > **Updates** to update Home Assistant Core.
 - If you don't see the update, you may have skipped it in the past. To see previously skipped updates, select the three dots `mdi:dots-vertical` and enable skipped updates.
 - If you're significantly behind and have YAML-based integrations that need migration, update in increments of no more than 6 releases at a time (for example, from Home Assistant Core 2024.2 to 2024.8, then to 2025.2).
 - This ensures automatic migrations can run properly during each update.
 - You can use the Home Assistant CLI to update to a [specific Home Assistant Core version](#).
-

More Info/Unsupported/Job Conditions

The issue

You have turned on the setting to ignore job conditions. Home Assistant has a built-in protection mechanism that detects if the system is working as expected. Ignoring job conditions disables this protection.

If the system is not working as expected, certain tasks can result in breaking the system.

The solution

To solve this you need to re-enable the protection mechanism by removing all the ignored conditions by using the command-line interface (CLI):

```
ha jobs reset
```

When you have removed all of them, restart the Supervisor.

```
ha supervisor restart
```

More Info/Unsupported/Lxc

The issue

Running the Supervisor in an LXC virtual machine will cause issues both with performance and with container management in general. Using LXC in combination with the Supervisor is not supported.

The solution

You need to reinstall the host operating system that runs the Supervisor. For instructions on how to proceed, [have a look here](#).

More Info/Unsupported/Network Manager

The issue

The Supervisor utilizes the Network Manager on the host to offer network information to add-ons and to give you the option to manage the network interfaces using the UI or via the command-line.

This requires NetworkManager to be installed, active and in control of at least one network interface on the host.

The current minimum supported version of NetworkManager is: 1.14.6.

The solution

If you have not already, install or update Network Manager on the host.

When it is installed, you need to make sure it manages at least one interface [see the documentation for the network manager](#).

Here are some example files that can be used to make the Network Manager control all physical interfaces.

`/etc/NetworkManager/NetworkManager.conf` :

```
[main]
dns=default
plugins=keyfile
autoconnect-retries-default=0
rc-manager=file

[keyfile]
unmanaged-devices=type:bridge;type:tun;driver:veth

[logging]
backend=journal
```

`/etc/NetworkManager/system-connections/default` :

```
[connection]
id=Supervisor default
uuid=b653440a-544a-4e4f-aef5-6c443171c4f8
type=802-3-ethernet
llmnr=2
mdns=2

[ipv4]
method=auto

[ipv6]
addr-gen-mode=stable-privacy
method=auto
```

`/etc/network/interfaces` :

```
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback
```

You can also just re-run our [convenience installation script](#).

If you haven't done anything manually with the network on the host, you should just re-run the convenience installation script.

When you have changed your network configuration manually or with the script, you should restart the host so the change will be populated to all services that needs it.

More Info/Unsupported/Os

The issue

The Home Assistant Supervisor is only supported on Home Assistant OS.

The solution

You need to reinstall the host machine running the Supervisor using Home Assistant OS.

The best approach here is to [create a backup](#) of your current installation, then reinstall your system using Home Assistant OS. During [Onboarding of Home Assistant](#), choose to **Restore from backup**.

More Info/Unsupported/Os Agent

The issue

OS-Agent is how the Supervisor handles additional tasks on the host, without it, the Supervisor won't be able to fulfill its tasks and responsibilities.

The solution

Home Assistant Operating-System has had this preinstalled since version 6.0 - you need maybe reboot your system.

If the OS-Agent daemon is not running, start it. If the [OS-Agent is not installed](#), you need install it.

More Info/Unsupported/Os Version

The issue

Supervisor considers Home Assistant Operating System older than the last 4 major releases as unsupported. We generally recommend to always update to the latest Home Assistant Operating System.

On an unsupported Home Assistant OS, Supervisor stops refreshing its update information. This means you will no longer receive updates for any component, including Home Assistant Core or app (formerly known as add-ons) updates.

The solution

Since you've not updated your system in a while, we recommend to [create a backup](#) before updating your system. Make sure to download the backup or store it at a remote location.

To resolve this issue, update your Home Assistant Operating System version. Go to **System** > **Updates** to update the operating system. If you don't see the update you may have skipped it in the past. To see previously skipped updates, select the three dots `mdi:dots-vertical` enabling skipped updates no new OS updates appear, you may need to use [the CLI to update to the latest OS version](#) .

If your system is significantly out of date you may need to update multiple times to get to the latest version.

More Info/Unsupported/Restart Policy

The issue

Supervisor needs to start the containers it manages for addons, plugins and Home Assistant in the correct order after a system reboot. Changing the restart policy it sets on those containers may cause them to start in the wrong order and create errors.

The solution

If the restart policy of observer was changed, fix it from the host shell with this:

```
docker update hassio_observer --restart always
```

For everything else, the restart policy can be fixed with the following command:

```
docker update <container_name> --restart no
```

The supervisor log should contain a list of container names with incorrect restart policies.

More Info/Unsupported/Software

The issue

As stated in [ADR-0014](#), no additional software, outside of the Home Assistant ecosystem, is installed. This includes but are not limited to standalone containers running on the same host.

Some containers will also conflict with the operations of the Supervisor, if you run any of those your system will also be marked as unhealthy. These containers will be shown in the Supervisor log as errors.

The solution

Remove any additional software (including standalone containers) you have installed on your host and restart the Supervisor.

More Info/Unsupported/Supervisor Version

The issue

Only the latest version of the supervisor is supported. Users may control when Supervisor updates by disabling its default auto-update behavior and updating it manually. But using any version of Supervisor besides the latest is not supported.

The solution

Update Supervisor to the latest version by running the following command:

```
ha supervisor update
```

Or re-enable auto update with this command:

```
ha supervisor options --auto-update
```

More Info/Unsupported/System Architecture

The issue

Supervisor considers `i386`, `armhf`, and `armv7` (all 32-bit system architectures) as unsupported. Home Assistant announced the sunset of these architectures in May 2025 (see [this blog post for more information](#)). On these system architectures, the Supervisor stops refreshing its update information. This means you will no longer receive updates for any component, including Home Assistant Core or app (formerly known as add-ons) updates.

The solution

You can continue to use the system as is, but it is recommended to move your Home Assistant installation onto supported hardware. This makes sure that you'll continue to receive software updates, including security updates.

1. To migrate to a supported architecture, [create a full backup](#) and download it.
 2. Install Home Assistant OS on supported 64-bit hardware.
 3. Use the [restore from backup feature during onboarding](#) (Option 2).
 4. If you use a Raspberry Pi 3 or Raspberry Pi 4, you can install the 64-bit variants of Home Assistant OS.
 - See the [Raspberry Pi installation documentation](#) for detailed instructions.
-

More Info/Unsupported/Systemd

The issue

The Supervisor uses systemd over DBus to control the Host system and get information. Without systemd, we miss a lot of information and functionality.

The solution

You need to reinstall the host running the Supervisor with one of the supported operating systems, [see instructions here](#).

More Info/Unsupported/Systemd Journal

The issue

The Supervisor needs access to the systemd journal to access logs of individual system components and apps (formerly known as add-ons). These logs are limited or not available at all if the journal is not accessible.

The solution

The `systemd-journal-gatewayd` service needs to be running on the host and exposed to supervisor via unix socket.

If using Home Assistant OS, update to version 7 or later.

If using Home Assistant Supervised, run the following on the host:

```
sudo apt install systemd-journal-remote -y
```

and then reboot your system

If after upgrading you are still facing this error, reinstall `systemd-journal-remote` with the following command

```
sudo apt-get install --reinstall systemd-journal-remote
```

and reboot your system

If you still see this issue then run the [supervised installer](#) on the host as the supervisor service may need an update as well.

More Info/Unsupported/Systemd Resolved

The issue

systemd-resolved is used over D-Bus to resolve DNS queries made by Home Assistant, Supervisor, and apps (formerly known as add-ons). Without it, DNS will not work correctly in your installation.

The solution

If you see a message about an issue with D-Bus, [resolve that first](#).

If the systemd-resolved service is not running or disabled, enable and start it.

If that does not help, reboot your operating system.

As a last resort, you need to reinstall the host running the Supervisor with one of the supported operating systems, [see instructions here](#).

More Info/Unsupported/Virtualization Image

The issue

Your Home Assistant OS installation appears to run in a virtualized environment using a disk image which is not meant to run in a virtualized environment.

Home Assistant OS publishes specific images which support being run on a virtualized system. These are optimized to run in virtual environments, e.g. have guest tools installed and para-virtualization drivers enabled.

The solution

You need to reinstall Home Assistant OS using an image which supports being run on a virtualized system, [see instructions here](#).

Voice Control/About Wake Word

The challenge

- The wake words have to be processed extremely fast: You can't have a voice assistant start listening 5 seconds after a wake word is spoken.
- There is little room for false positives.
- Wake word processing is based on compute-intensive AI models.
- Voice satellite hardware generally does not have a lot of computing power, so wake word engines need hardware experts to optimize the models to run smoothly.

The approach

To avoid being limited to specific hardware, the wake word detection is done inside Home Assistant. Voice satellite devices constantly sample current audio in your room for voice. When it detects voice, the satellite sends audio to Home Assistant where it checks if the wake word was said and handle the command that followed it.

This means any device that streams audio can be turned into a voice satellite, even if it isn't powerful enough to run wake word detection locally. It also allows our developer community to experiment with wake word models without having to shrink the model to run on a low-powered voice satellite device.



Note: Exception for mobile devices: Mobile devices like Android phones use `microWakeWord` for on-device wake word detection through the [Home Assistant Companion App](#). This preserves battery life and ensures wake word detection works even when network connectivity is unreliable or unavailable. Only after the wake word is detected does the device send audio to Home Assistant. Learn more about [Assist on Android](#).

Overview of the wake word architecture

Drawbacks of this approach

1. The quality of the captured audio differs between devices. A speakerphone with multiple microphones and audio processing chips captures voice very cleanly. A device with a single microphone and no post-processing? Not so much. We compensate for poor audio quality with audio post-processing inside Home Assistant and users can use better speech-to-text models to improve accuracy like the one included with Home Assistant Cloud.
2. Each satellite requires ongoing resources inside Home Assistant while it's streaming audio. Currently, users can have 5 voice satellites streaming audio at the same time without overwhelming a Raspberry Pi 4. To scale up, we've updated [the Wyoming protocol](#) to allow users to run wake word detection on an external server.

About the openWakeWord app

Home Assistant's wake words are leveraging a new project called [openWakeWord](#) by David Scripka. This project has real-world accuracy, runs on commodity hardware and anyone can [train a basic model of their own wake word](#).

Users can pick per configured voice assistant what wake word to listen for

The challenge

openWakeWord is created with 4 goals in mind:

- Be fast enough for real-world usage.
- Be accurate enough for real-world usage.
- Have a simple model architecture and inference process.
- Require little to no manual data collection to train new models.

Training the model

openWakeWord is built around an open source audio embedding model trained by Google and fine-tuned using the text-to-speech system [Piper](#). Piper generates many thousands of audio clips for each wake word, creating variations of different speakers. These audio clips are then augmented to sound as if they were spoken in multiple kinds of rooms, at specific distances from a microphone, and with varying speeds. Finally, the clips are mixed with background noise like music, environmental sounds, and conversation before being fed into the training process to generate the wake word model.

Overview of the openWakeWord training pipeline.

Supported languages

OpenWakeWord currently only works for English wake words. This is because there is still a lack of models in other languages with many different speakers. Similar models for other languages can be trained as more multi-speaker models per language become available.

openWakeWord in Docker

If you're not running Home Assistant OS, openWakeWord is also available as [a Docker container](#). Once the container is running, you will need to add the Wyoming integration and point it at its IP address and port (typically 10400).

Other wake word engines

Home Assistant ships with defaults but allows users to configure each part of their voice assistants. This also applies to wake words.

You can add other wake word engines as an integration or run them as a standalone program that communicates with Home Assistant via the [Wyoming protocol](#).

How wake words integrate into Home Assistant

As an example, we're also making the **Porcupine (v1)** wake word engine available. It supports 29 wake words across English, French, Spanish, and German. The wake words include *Computer*, *Framboise*, *Manzana*, and *Stachelschwein*.

About on-device wake word processing (microWakeWord)

The [microWakeWord](#) created by [Kevin Ahrendt](#) enables on-device wake word detection on devices like the ESP32-S3-BOX-3 and Android phones.

Because `openWakeWord` is too large to run on low-power devices like the S3-BOX-3, `openWakeWord` runs wake word detection on the Home Assistant server.

Doing wake word detection on Home Assistant allows low-power devices like the [M5 ATOM Echo Development Kit](#) to simply stream audio and let all of the processing happen elsewhere. The downside is that adding more voice assistants requires more CPU usage in Home Assistant as well as more network traffic.

Enter *microWakeWord*; a more light-weight model based on [Google's Inception neural network](#). Because this new model is not as large, it can be run on low-power devices with an ESP32 chip, such as the ESP32-S3 chip inside the S3-BOX-3, as well as on Android phones. *(For ESPHome devices, it also works on the now discontinued S3-BOX and S3-BOX-Lite).*

Currently, there are [three models](#) trained for `microWakeWord`:

- *okay nabu*
- *hey jarvis*
- *alexa*

Try it!

There are several easy options to get started with wake words:

- Follow the guide to the [\\$13 voice assistant](#). This tutorial is using the tiny ATOM Echo, detecting wake words with openWakeWord.
 - Follow the guide to set up an [ESP32-S3-BOX-3 voice assistant](#). This tutorial is using the bigger S3-BOX-3 device which features a display. It can detect wake words using openWakeWord. But it can also do on-device wake word detection using microWakeWord.
 - The [Home Assistant Companion App for Android](#) supports on-device wake word detection using microWakeWord. Wake words like "Hey Nabu", "Hey Jarvis", and "Hey Mycroft" work even when your phone is locked or the app is in the background.
-

Voice Control/Aliases

Assist will use the names of your entities, areas and floors, as well as any aliases you've configured. The configured aliases are not only used by Assist, but can also be used by Google Assistant, if you have set that up.

These aliases are helpful in case you call the same device by multiple names or when using a voice assistant in multiple languages at the same time.

Adding an alias of an entity

There are multiple ways to add an alias of an entity:

- **Option 1:** Go to **Settings > Voice assistants**. On the **Expose** tab, select the entity for which you want to add an alias. Screenshot showing the alias editing capabilities added to the more info dialog of entities, accessed from the Voice assistants page
- **Option 2:** You can also change the alias from the entity's more information dialog in the settings tab in the advanced section. Screenshot showing the alias editing capabilities added to the more info dialog of entities, accessed from the Voice assistants page.

Adding an alias of an area

1. To add an alias for an area, go to **Settings > Areas, labels & zones**.
2. On the area card of interest, select the pencil button.
3. Select **Add alias** and enter the alias you want to use for that area.
4. **Save** your changes.

Adding an alias of a floor

1. To add an alias for a floor, go to **Settings > Areas, labels & zones**.
2. Next to the floor of interest, select the three dots `mdi:dots-vertical`, then select **Edit floor**.
3. Select **Add alias** and enter the alias you want to use for that floor.
4. **Save** your changes.

Area-less aliases for entities with an assigned area

It's good practice to add areas to entity canonical names, such as Living room lamp. However, since Assist can both infer the area and explicitly extract it from sentences, it's a very good idea to add simplified aliases to all your exposed entities. In this case, having the Lamp alias set for the Living room lamp would allow you to turn on the lamp in the living room or simply turn on the lamp, when asking a satellite in the living room.

Don't worry if you also have a Bedroom lamp. You can alias that one Lamp as well, as it would get matched only when in conjunction with the area name (Living room or Bedroom).

Assist on Android phones

Assist can be used on Android phones and tablets using the [Home Assistant Companion App](#).

Prerequisites

- [Home Assistant Companion App](#) installed
- Have an Assistant set up: either [cloud](#) (recommended, more performant) or [local](#).
- The devices you want to control with Assist are [exposed to Assist](#) and you have checked most of the [best practices](#)

Starting Assist in Home Assistant

1. On your phone, open Home Assistant.
2. In the top-right corner, select the three-dots menu and select **Assist**.
3. [Give a command](#).

Setting up Home Assistant Assist as default assistant app

To define Home Assistant Assist as default assistant app on your Android phone, follow these steps:

1. On your Android phone, open the **Home Assistant** app.
2. Go to **Settings > Companion app**.
3. Select **Assist for Android**.
4. Select **Set as default**.
5. In the system settings that open, select **Default digital assistant app**.
6. On some Android versions, this might be labeled **Voice Assistant** or **Assist app**.
7. Select **Home Assistant**.
8. Go back to the **Home Assistant** app. You should now see that **Assist** is your default assistant.
9. Start Assist using the gesture to start an assistant. The gesture may differ depending on your Android version:
10. Swipe from the bottom left corner.
11. Long press the power button.
12. Hold the home button (square button at the bottom).
13. You can now also start Assist from your lock screen.

To activate Assist hands-free, [enable wake word detection](#).

About wake word detection on Android

Wake word detection allows you to activate Assist hands-free by saying a wake word like "Hey Jarvis" or "Hey Nabu". Your Android device uses [microWakeWord](#) to process wake words locally on the device, which means your audio stays private and no audio is sent to Home Assistant until after the wake word is detected.

 **Important:** Wake word detection continuously monitors audio for wake words, which has a noticeable impact on battery life. Consider disabling wake word detection when you don't need it or [control it remotely from Home Assistant](#) to turn it on only when needed.

 **Note:** Wake word detection on Android uses more battery than "Ok Google" because Google Assistant has access to dedicated low-power hardware for wake word detection on supported devices. Unfortunately, Google does not make this specialized hardware accessible to third-party app developers, forcing apps like Home Assistant to rely on standard audio processing, which consumes more power by keeping the CPU on all the time. This platform limitation means third-party voice assistants cannot achieve the same battery efficiency as Google's built-in assistant.

Wake word detection runs entirely on your Android device, which means it works without an active internet connection (though executing commands still requires connectivity to your Home Assistant instance). When multiple devices detect the same wake word simultaneously (like another Android phone or a Voice Preview Edition), only the first device to capture the wake word will keep the Assist session open while other devices automatically cancel their sessions.

Enabling wake word detection on Android

 **Note:** Wake word detection on Android is experimental.

If the app becomes unresponsive or stops opening as expected, disable Home Assistant as your default digital assistant to disable wake-word detection:

- On your Android phone, go to **Settings > Apps > Default apps > Digital assistant app** and select a different assistant or **None**.
- If you run into issues, please [open an issue on GitHub](#) so the team can look into it.

To enable wake word detection on your Android device, follow these steps:

Prerequisites

- Home Assistant Companion App version 2026.2.3 or later
- Assist set as the [default assistant app](#)
- [Home Assistant Cloud](#) or a manually configured [local Assist pipeline](#)

To enable wake word detection

1. On your Android phone, open the **Home Assistant** app.

2. Go to **Settings > Companion app**.
3. Open **Assist for Android**.
4. Enable **Wake word detection**.
5. Select a wake word from the available options:
6. Hey Nabu
7. Hey Jarvis
8. Hey Mycroft Result: Once enabled, wake word detection works even when your device is locked or the app is in the background.
9. To use Assist on Android, say your chosen wake word, wait for the listening prompt, and then speak your command.

Controlling wake word detection from Home Assistant

You can turn wake word detection on or off remotely from Home Assistant. This is useful for automations that enable wake word detection only when you're at home or during specific times to save battery.

Use the `command_wake_word_detection` command with `turn_on` or `turn_off` to control wake word detection. For details on how to send commands to the companion app, see the [notification commands documentation](#).

Using Assist with multiple Home Assistant servers

Once Assist has been defined as the default digital assistant on your phone, you can use Assist with multiple servers. This can be useful for example, if you maintain a Home Assistant instance for someone else's home.

1. Make sure you have a voice assistant set up on the Home Assistant servers.
2. Make sure the servers are added to the companion app.
3. On the Android phone, go to **Settings > Companion app** and select **Add server**.
4. From the list, select the additional server.
5. Start Assist using the gesture to start an assistant. The gesture may differ depending on the version.
6. Swipe from the bottom left corner.
7. Long press the power button.
8. Hold the home button (square button at the bottom).
9. Open the assistant drop-down menu.

Using Assist with multiple servers

1. Select the assistant from the instance you want to speak to.
2. Speak your command.

Using Assist via shortcut

1. On your phone, open the **Widgets** panel.
2. From the list, select **Home Assistant**.

3. Long tap the **Assist** icon and drag it onto your home screen.
4. Select the server and the assistant.
5. If you want to be able to use voice control, enable the **Start listening** toggle.
6. Repeat this procedure for each server you want to connect to, for example, if you support someone else's home.

Assist on Wear OS

Assist is available on Wear OS using the [Home Assistant Companion App](#) for Android and "Assist" tile.

Setting up Assist on Wear OS

The way how Assist can be set up on your phone may differ depending on your version of Wear OS.

1. After [installing the companion app](#) on your watch and connecting it to your Home Assistant, Assist appears automatically on the **Apps screen**.

Assist app

2. To add an Assist tile, in the Wear OS app, go to the **Tiles** area, select **Add tile > Assist**.

Conversation tile

Adding Assist to the watch face

1. On your phone, open the **Watch** app and select **Watch faces**.
2. Make sure you select a watch face that supports complications (little shortcut icons).
3. Tap **Edit**.
4. In the **Complications** section, select one of the slots and choose **Assist**.
5. If you just recently added the Home Assistant app, Assist may not be listed yet.
6. After rebooting your watch, under **Complications**, there should be a section with Home Assistant icons.
7. Save your changes. You should now see Assist as a complication on your watch face.

Assist complication

Using Assist on Wear OS

1. On your watch, open Assist.
2. For example, swipe left until the **Assist** button is visible.

Assist button

3. After tapping **Assist**, wait for **How can I assist?** to be displayed and the microphone to start pulsing.

How can I assist

4. Speak your command.

Assist speak your command

5. To change the assistant, tap the current assistant (**Home Assistant Cloud** in the screenshot above).

6. Select the assistant from the list.

List of assistants

Setting up Home Assistant Assist as default assistant app on a Wear OS watch

This procedure was written using Wear OS version 4.0. The exact steps may vary depending on the watch and software version.

To define Home Assistant Assist as default assistant app, follow these steps:

1. On the watch, navigate to the **Apps screen** and select the cogwheel `mdi:cog-outline`.
2. Go to **Apps > Choose default apps > Digital assistant app**.
3. From the list, select **Home Assistant**.
4. When you go back one step, under **Default app**, it now says **HA: Assist**.
5. On some watches, you can now start Assist by pressing the crown key.
6. If this does not work, you can manually assign Assist to a button.
7. Now, use your key and speak a command.

Assigning a button to Assist

Depending on your watch, you can assign Assist to a button so that you can start it directly with a long or double press.

1. On your watch, go to **Settings > Advances features > Customize keys**.
2. Assign a key:
3. To use double press, tap **Home key > Double press**. From the list of apps, select **HA: Assist**.
4. On a Galaxy watch, if Assist is set as the default, you can use long press. Tap **Home key**, then tap **press and hold**. Select **Assistant**.
 - Then long press the home key, and from the selection, select **HA: Assist**.
 - Select **Always**.

List of assistants 3. Now, use your key and speak a command.

Assist on iPhones

Assist is available on iPhones, iPads, and Macbooks.

Demo showing Assist being triggered from the iPhone 15 Pro action button and from the lock screen widget.

Prerequisites

- [Home Assistant Companion App](#) installed
- Have an Assistant set up: either [cloud](#) (recommended, more performant) or [local](#).
- The devices you want to control with Assist are [exposed to Assist](#) and you have checked most of the [best practices](#)

Starting Assist in Home Assistant

1. On your phone, open Home Assistant.
2. In the top-right corner, select the three-dots menu and select **Assist**.
3. [Give a command](#).

Starting Assist on your iPhone using a shortcut

This section was written using an iOS 18. Depending on your model and version, the exact steps may vary.

To use Home Assistant Assist as your voice assistant, follow these steps:

1. Create a shortcut to **Assist in app**
2. Choose one of following options to activate Assist:
3. [Start Assist using a back tap](#).
4. [Start Assist using the action button](#).
5. [Start Assist using control center](#).
6. [Start Assist from Lock Screen](#).
7. [Start Assist using Voice Control](#).
8. [Give a command](#).

To create a shortcut to Assist in App

1. On the phone, open the **Shortcuts** app, and select **New**.
2. Type `Home Assistant` and select **Assist in app**.
3. **Choose** the pipeline.
4. Select **Done**. You now have a shortcut to **Assist in app**.

To start Assist using a back tap

1. Follow the Apple documentation on [Running shortcuts by tapping the back of your iPhone](#) and select the shortcut to **Assist in app**.
2. Start Assist by tapping the back of your phone.

To start Assist using the Action button

1. Go to **Settings > Action Button**, and scroll until you see **Controls**.
2. Under **Home Assistant**, select **Assist**.
3. Select your preferred pipeline.
4. Start the Assist by holding the action button.

For control center and lock screen:

To start Assist using control center

1. Open control center.
2. Press and hold an empty space and look for **Home Assistant**.
3. Select **Assist**.
4. After you see the icon in control center, tap on it again to choose which pipeline you want to use.

To start Assist from Lock Screen

1. Tap and hold an empty space in Lock Screen.
2. Choose one of the two bottom items where you want to replace using Assist.
3. Remove the item.
4. Tap on it to add a new item and find **Home Assistant Assist** in the list.
5. After you see the icon in lock screen, tap once more to choose which pipeline you want to use.
6. Alternatively you can execute the same steps but add a widget below the lock screen clock.

To start Assist using Voice Control

iOS includes an accessibility feature called **Voice Control** that allows you to operate your iPhone entirely with voice commands. Using **custom commands**, you can trigger the **Assist in app** shortcut with your own wake phrase.

For example, you could create a custom command such as "**Okay Nabu**" that runs the **Assist in app** shortcut.

To set this up:

1. Go to **Settings > Accessibility > Voice Control**.
2. Enable **Voice Control**.
3. Tap **Customize Commands > Create New Command**.
4. Enter the phrase you want to use (for example, `Okay Nabu`).
5. Under **Action**, select **Run Shortcut**.

6. Choose the **Assist in app** shortcut you created earlier.

7. Save the command.

After this is configured, saying your custom phrase will launch Assist.

 **Note:** While this approach works well, it may not be as fast as triggering Siri or using hardware buttons. Also note that enabling Voice Control means full system voice navigation is active, which may occasionally trigger other commands unintentionally.

Adjusting the language

Shortcuts triggered via Siri will always use the same language as Siri is set to. The Assist Button shortcut is meant to be manually triggered and can be configured for any language.

Open the shortcuts app, and edit the Button Assist shortcut. The text in quotes will be shown in the language of your device.

- Use the arrow to expand the "*Dictate text*" action options, and select your language
- Use the arrow to expand the "Assist with *Provided Input*" options, and select your language.

 **Important:** You can import the button shortcut multiple times to create versions for different languages, when asked if you would like to replace your Shortcut, choose "Keep Both".

Multiple servers

The Assist shortcut works also if you have configured multiple Home Assistant servers. By default it will prompt you to pick the server to sent the command to. This is not very hands-off, and so you can update the shortcut to point at a specific server. You will need to import the shortcuts multiple times, once for each server.

Open the shortcuts app and edit each Assist shortcut. The text in quotes will be shown in the language of your device.

- Use the arrow to expand the "Assist with *Provided Input*" action, and select your Home Assistant server.
-

Voice Control/Assign Areas Floors

Another best practice with Assist is to create an architecture of areas and floors in your home, since it will make it consistent and easy to understand for Assist.

To create missing areas

1. Go to **Settings > Areas, labels & zones**.
2. Select **Create area**.

To create missing floors

1. Go to **Settings > Areas, labels & zones**.
2. Select **Create floor**.
3. In the floor creation dialog, assign the related areas.

To assign areas to existing floors

1. Go to **Settings > Areas, labels & zones**.
2. Next to the floor name, select the three dots `mdi:dots-vertical` menu.
3. Select **Edit floor**.
4. Assign your areas to this floor.

To assign an area to a device

1. Go to **Settings > Devices & services > Devices**.
 2. Select the device.
 3. In the top bar of the device page, select the pencil `mdi:pencil` icon.
 4. Assign an area to the device.
-

Voice Control/Assist Create Open Ai Personality

You can give your voice assistant personality by using an AI conversation agent.

What can I do in Assist with AI exactly?

- Pick the LLM provider of your choice, either local or cloud, as long as it has a conversational agent.
- Select a personality based on a prompt.
- Get replies with the character's personality you defined.
- Perform Home Assistant intents (turn on-off lights, etc), as long as Assist is correctly configured as per our [best practices](#).

Check this 1-minute clip showing how Assist is using AI to control a smart home.

What LLM providers are available?

LLM-based agents are evolving constantly, and Home Assistant supports most of them. If you'd like to get a deeper knowledge on how to pick the best choice for your setup, [here](#) is a comparison study you can check.

There are cloud agents provided by [Open AI](#) or [Anthropic](#) and local ones provided by [Ollama](#), and both cases are supported by Home Assistant.

Prerequisites

- Home Assistant and Assist is configured following our [best practices](#).
- An account in the conversational agent of the LLM provider of your choice. If you want to test the process, you can create a free account on Open AI.
- In case of a local LLM solution, you need to have the model installed.

Creating a voice assistant personality with an LLM-based conversation agent

1. Go to **Settings > Devices & services Add Integration**, find your LLM provider and set it up with your API key.
2. In case of a provider of local agents like Ollama, you need to configure the local URL where the agent is installed. Follow the specific [integration recommendations](#) in this case.
3. Go to **Settings > Voice Assistants > Add Assistant**. Give it a name and pick a conversation agent from your AI's option. In this example we are using Antropic and the agent picked is Claude.

Add Claude agent to Assist

4. Be mindful of your Text-to-speech and Speech-to-text configurations. These are not handled by the AI and should stay as you want them configured for Assist.
5. Configure the agent (gear icon next to the agent's name).
6. In the **Prompt template** field, enter a text that will prompt the AI to become the character. For example:: `You are Super Mario from Mario Bros. Be funny.`
7. Define if the voice assistant is allowed to control the devices in your home.
 - **No control**: you can talk to the agent, but it cannot control devices.
 - **Assist**: you can talk to the agent and it can control devices. For example, it could turn on the lights.
 - Assist can only control entities that are [exposed](#) to it. Agent with recommended model settings
8. Once your Assist agent has been created, you can go to **Voice assistants** and the three dots menu of your personality, and define if you want Home Assistant's model to be the priority response, and therefore Assist would prefer to handling commands locally . Fallback toggle
 - If you keep this option selected, if the intent can be answered by Home Assistant it will. It will not have the personality, but the response will be fast and efficient (since it doesn't require to go through the LLM). This is recommended in cases where you can accept not having the AI character reply sometimes and would rather your lights are turned on faster.
 - If you deselect the option, all the intents will go through the agent. This is recommended when efficiency is not an issue and you need the agent never to break character (for example if your Assist personality is Santa Claus).

- You can uncheck Recommended model settings, hit Submit and it will unblock extra customization. In the specific example of OpenAI, [here](#) a brief summary of the other settings.
 - You can test the agent directly from the Voice assistants panel, selecting Start a conversation from the agent's menu. It will control your Home Assistant and reply exactly as it will do with any voice hardware.
9. In case you need troubleshooting with your LLM provider, check any specifics from your AI in our [integrations documentation](#)

Tutorial: Setting up Assist with OpenAI

Step-by-step tutorial with some background information, from the Home Assistant Release Party 2024.6 live stream.

Using the AI voice assistant on your devices

To learn how to use the AI assistant with your devices, refer to one of the following tutorials, depending on the hardware you want to use to interact with it:

- [ESP32-S3-BOX voice assistant](#)
- [\\$13 voice assistant using ATOM Echo](#)
- [Assist on Android](#)
- [Assist on Apple](#)

Demos

Check this interview with an AI Mario personality

Voice Control/Assist Daily Summary

In this tutorial, we are creating an automation that has Assist send you a daily summary. Assist will tell you about the weather and your calendar events today. It will also send you the summary to your messenger.

Daily summary notificationDaily summary notification - using a neutral tone

We will be using OpenAI, which requires an OpenAI account. For what we do in this tutorial, the free trial option is sufficient. No need to leave your credit card information.

Prerequisites

This tutorial assumes you have a few things set up already:

- [Home Assistant Cloud](#) or a manually configured [local assistant pipeline](#).

This tutorial was done using the **Local calendar**, the **Meteorologisk institutt**, and the **Telegram** integrations. It has not been tested with other integrations of the notifications or calendar category.

Adding a calendar

Skip this if you're already using a calendar.

1. Go to the [integrations page](#) and select the calendar **Calendar** filter.
2. Pick a calendar you like and install it as described in the documentation.
3. If you just want to follow along with this tutorial, install the [local calendar](#) integration.
4. When prompted for a name, call it `Local calendar`.
5. In the navigation bar on the left, you should now see a new entry for the calendar. Open it.

Calendar - Add a few events for today and the next few days.

Adding a weather integration

Skip this if you're already using a weather integration.

1. Go to the [integrations page](#) and select the **Weather** filter.
2. Pick a calendar you like and install it as described in the documentation.
3. If unsure, select **Meteorologisk institutt** and add the integration.
4. When prompted, enter the latitude and longitude of your home.
5. The coordinates allow the integration to show the weather forecast for your location.

Connect Home Assistant to a messenger service

Skip this if you're already using a notification integration.

1. Go to the [integrations page](#) and select the **Notifications** filter.
2. Pick a messenger service you like and install it as described in the documentation.
3. If unsure, select **Telegram** and add the integration.
4. If you don't have it already, install Telegram on your phone.
5. To get started with Telegram on Home Assistant, follow the [set up instruction](#) step by step.
6. Make sure not to copy and paste the following values from the example. Enter the real values:

- `api_key`
- `allowed_chat_ids`
- `name`
- `chat_id`

- `service`

7. You now have a working **Notification** integration. Home Assistant can now send messages to you.

Creating an OpenAI voice assistant personality

The OpenAI personality gives the messages a special touch. Using OpenAI requires an OpenAI account. For this tutorial, the free trial option is sufficient. No need to leave your credit card information.

- [Create a Mario personality.](#)

Creating an automation from a blueprint

We are using a blueprint (courtesy of [@allenporter](#)) that polls calendar events and collects weather information. It then asks ChatGPT to summarize it and ships that response to your phone.

1. To import the blueprint, select the button below:

blueprint_import

1. Select **Preview**, then select **Import blueprint**.
 2. Select the blueprint **Conversation agent agenda notification** from the list.
 3. Enter the values for each category. Add prompt for Mario personality
 4. Under **Notify service name**, make sure not to leave the default but to use the one you set up previously. For example `notify.nina`.
 5. **Save** your changes.
 6. In the dialog, enter a name for your new automation. For example, `Daily summary by Mario`.
 7. To view the automation, go to **Settings > Automations & Scenes**.
 8. To test the automation, select the three dots on your automation, and select **Run**.
 9. You should now receive a notification from Assist in your messenger app.
-

Voice Control/Best Practices

Voice control feels effortless when it is set up well, and frustrating when it is not. The good news is that getting Assist to feel natural mostly comes down to a few simple choices: which devices it can see, how things are named, and how your home is organized into areas. This page collects the small habits that make a big difference.

In Home Assistant, you decide what Assist has access to. Every entity is opt-in: nothing is exposed to your voice assistant until you say so. That means you can give Assist exactly the devices and information you want it to handle, and nothing else.

Some best practices we recommend to have an efficient setup are:

Expose (the minimum) entities

Learn how in [Exposing your entities to Assist](#).

It might be tempting to expose all entities to Assist, but doing so will come with a performance penalty. The more entity names and aliases the parser will have to go through, the more time it will spend matching. And if you're using a LLM-based conversation agent, it will incur a higher cost per request, due to the larger context size. Only expose the bare minimum you know you are going to use with voice assistants.

Check names and create aliases

Assist relies heavily on entity names, domains, and areas. Below you will find tips for tweaking these things to ensure the best experience. On top of exposing the needed data, it is worth noting that you will most likely target entities through areas and floors, like:

- *Turn off the office lights*

So make sure your devices and entities are correctly assigned to areas, and your areas are correctly assigned to floors. Learn how [here](#).

Not having good ways to address entities in common speech will greatly hinder your voice experience with Assist. You can expect to have a hard time asking Assist to “turn on Tuya Light Controller 0E54B1 Light 1”. You should therefore name your devices and entities logically, using a schema such as `<area> [<identifier_or_descriptor>] [<domain>].`

For example, `light.living_room_lamp` might be the entity ID of `Living room lamp`. `Kitchen light` would be enough for the `light.kitchen` if you only have one light fixture in that room.

Note that this convention is only a recommendation, actual naming of your devices and entities might depend on your language or personal preference.

If you ever find yourself mentioning a certain device or entity in a certain way, make sure to [add that as an alias](#), as it would probably be the most natural way to refer to the entity.

Names and aliases also apply to `area`, you need to address area names and aliases in the exact same manner as you would for entities.

Be aware of language specificity

If you have set up Home Assistant entity names in English but plan to use Assist in another language, don't worry. You can add aliases to each entity, allowing them to be named in any language.

English has pretty simple grammar rules, but there are languages where definite articles are pre- or suffixes to words and where nouns have genders or numbers. Language leaders are making efforts to support most such declinations in each language, but they can't control the stuff that you name. So try to think whether a certain entity having an unarticled name would be called out in a sentence requiring a definite article or vice versa. If so, add that version of the name as an alias as well.

Check domains and device classes

Assist leverages domains to define the proper verbs for the action being taken (for example, turning on/off a `light`, or a `fan`, opening/closing a `cover` with a `door` `device_class`, opening/closing a `valve` or locking/unlocking a `lock`).

It might not bother anyone to have a `switch.main_valve` in the UI instead of a valve, but you can't ask Assist to open the main valve if the main valve is a switch. If it was a `valve.main_valve`, then the former sentence would have worked without a hitch.

To prevent this, you can use either the [Change device type of a switch integration](#) or create virtual entities using [template](#) entities or Generic X, such as the [generic thermostat](#).

The same thing applies to some device classes. For example, if you have a `binary_sensor.bedroom_window` with no `device_class` set, you can only ask whether the bedroom window is on, which doesn't even make sense. To be able to ask if it's open, you need to set a proper `device_class` to that `binary_sensor`, such as `window`.

Ready?

Once your devices and entities are correctly - Exposed to assist - Assigned to areas.

It is now time to speak to your device.

To talk to Assist, you can either use your phone or a custom device (and use their microphone and speaker). Check here how to do it on [Android](#) or [Apple](#) devices.

Some examples to get you started

There are a few example commands to help you get started in [our Sentences starter pack](#).

If you don't get the right response, we recommend you check the Aliases. Sometimes, different household members may call an entity differently. You may say "TV", whereas someone else may say "Television"

You can create aliases for exposed entities so that you can target them using different names with Assist. Aliases are available at entity, area, and floor level. Learn how in the [Alias tutorial](#).

Voice Control/Builtin Sentences

Home Assistant comes with [built-in sentences](#) contributed by the community for [dozens of languages](#).

Prerequisites

- The entities you want to control with Assist must be [exposed to Assist](#).
- When using the names of entities or areas, make sure to use them exactly as they are defined in Home Assistant, or, [create an alias](#).

Device control

Turning entities on and off

- *turn on the living room light*
- *turn off ceiling fan*
- *turn on the TV*
- *lock all the doors*
- *open the main door*

Lights

- *Change kitchen lights brightness to 50%*
- *Set bed light to green*
- *set bed light brightness to 50%*
- *set living room brightness to 50%*
- *set brightness to 50%*
- *Uses area of voice satellite*
- *set kitchen lights to red*
- *set lights to red*
- *Uses area of voice satellite*
- *turn on the lights in the living room*

Covers

- *Close the garage door*
- *Open kitchen window*
- *Which curtains are closed*
- *Set bedroom curtain to 50%*

Scenes and scripts

- *Run stealth mode script*
- *Activate dinner scene*
- *Turn kitchen dinner scene on*

Media player

- *next item on TV*
- *next track*
- *next track in office*

- *previous track*
- *previous track in office*
- *skip song on the TV*
- *skip track on the TV*
- *skip to the next song on the TV*
- *pause|resume*
- *pauses or resumes music on voice satellite or in current area*
- *pause|resume "area" music*
- *pauses or resumes music in area*
- *pause|resume "entity"*
- *pauses or resumes music on media player*
- *unpause TV*
- *TV unpause*
- *set TV volume to 90 percent*
- *change the TV volume to 90*
- *turn TV volume down to 90 percent*
- *Mute my TV*
- *Unmute the television*

Vacuum

- *return rover to base*
- *start rover*

Lists

- *Add bread to my shopping list*
- *Add decorating christmas tree to my december chores list*
- *needs a todo list name "december chores"*
- *add clean out garage to weekend list*
- *needs a todo list named "Weekend"*

Date and time

- *what time is it?*
- *what's the date?*

Timers

Starting

- *set a timer for 5 minutes*
- *5 minute timer*
- *set a 20 minute timer for pizza*
- *set a timer for 1 hour and 3 minutes*
- Break it up into hours, minutes, and seconds instead of using a technical term like *set a timer for 63 minutes*.

Cancelling

- *cancel timer*
- can't cancel multiple timers yet
- *cancel 5 minute timer*
- *cancel pizza timer*
- *cancel kitchen timer*
- Must have been set by a voice satellite in the kitchen

Adding/Removing Time

- *add 5 minutes to pizza timer*
- *add 5 minutes to kitchen timer*
- *remove 1 minute from timer*
- *remove 1 minute from 5 minute timer*

Status

- *timer status*
- *how much time is left on pizza timer?*
- *how much time is left on kitchen timer?*
- *how much time is left on 5 minute timer?*

Timers running on an S3-Box-3B, with countdown text and a loading bar!

To learn how to set this up, refer to the [ESP32-S3-Box-3B tutorial](#).

Combining timers and device control to add a delay

Unlike regular voice timers, delayed commands cannot be canceled or modified.

- *Turn off the lights in the living room in 5 minutes*
- *Pause TV in 10 minutes*

- *Open the blinds in 5 minutes*

Aborting

- *Nevermind*: If you triggered the wake word by mistake and want to stop Home Assistant from listening

Troubleshooting

The list of supported sentences is constantly being updated for each language. There are so many possible sentences that they cannot be all listed here. To find out what works in your language, follow these steps.

1. If the voice assistant doesn't understand you, you may need to rephrase your sentence a bit.
Or check if the entity or area name is correct for your environment.

2. Take a look at the test sentences:

- On GitHub, in the [tests](#) folder, open the subfolder for your language.
- Look through the test files to see the example sentences that have been tested.
- The second part of the file name shows the intent, the first part shows the domain. For some domains, such as covers, fans, and light, there are specific sentences. The other domains are covered by the generic *homeassistant_*.

Example of a folder of assistant sentence test files

- The screenshot below shows sentences used to test the command to turn on the lights. Note that *Living room* here is just a place holder. It could be any area that you have in your home.

Example of a set of test sentences

3. View the sentence definition for the tests:

- On GitHub, in the [sentences](#) folder, open the subfolder for your language.
- Open the file of interest.

Sentences definition for turning on the light

- `()` mean alternative elements.
- `[]` mean optional elements.
- `<>` mean an expansion rule. To view these rules, search for `expansion_rules` in the `_common.yaml` file.
- The syntax is explained in detail in the [template sentence syntax documentation](#).
- View the [sentence definition](#) for your language.
- View the [response definition](#)
- If there are issues when asking for the weather forecast, check the [troubleshooting section](#) to see common errors.

More sentences

You can extend the [built-in sentences](#) or [add your own](#) to trigger any action in Home Assistant.

Voice Control/Contribute Voice

Home Assistant Voice relies on different technologies, models, and data sources to process wake words and commands in all languages. Our and other communities' projects rely on volunteers who donate their time, voice, or language knowledge to improve.

Note that models are trained on lots of data and that it can take time before you are able to see the impact of your contributions.

Contribute samples to Wake Word Collective

Give us a minute of your time to improve the sample database [with your own samples](#).

Donate your voice to Piper

To create a Piper voice, you don't need an army of volunteers. You need a native speaker of a language to create a dataset, which is then trained into a Piper model and verified again by the native speaker. [Check the Training guide](#).

Are you missing a language? Email us if you want to help out! To help interested people contribute languages, we opened up the email address voice@nabucasa.com.

Other ways to contribute with samples

You can also participate in public datasets. Commonvoice and Librivox are used by many speech-to-text projects to train their models.

- [Contribute samples to Mozilla Commonvoice](#)
- [Contribute samples to Librivox](#)

Become an active contributor

There are also a couple ways to actively contribute to the development of the voice initiative: - [Contribute sentence intents in your language](#) - [Become a language leader](#) - [Contribute code to Assist](#)

Voice Control/Create Wake Word

Wake words are special words or phrases that tell a voice assistant that a command is about to be spoken. The device then switches from passive to active listening. Examples are: Hey Google, Hey Siri, or Alexa. Home Assistant supports its own wake words, such as Okay Nabu.

If you want to know more about this topic check [the Home Assistant approach to wake words](#).

Enabling a wake word

This tutorial shows how you can *enable* a wake word in Home Assistant. It does not describe how to *use* it.

To *use* the wake word, you need some extra hardware. A low cost option is the [M5Stack ATOM Echo Development Kit](#). To set that up, follow the [\\$13 voice assistant for Home Assistant](#).

Enabling a wake word consists of 2 steps:

1. Installing the **openWakeWord** app.
2. Enabling the wake word for a specific voice assistant.

Prerequisites

- Home Assistant version 2023.10 or later, installed with the Home Assistant Operating System
- Assist configured either with [Home Assistant Cloud](#) or a manually configured [local Assist pipeline](#)
- All the [Best Practices](#) we recommend.

Installing the openWakeWord app

1. Go to **Settings > Apps > openWakeWord** and select **Install**.
2. **Start** the app.
3. Go to **Settings > Devices & services**.
4. Under **Discovered**, you should now see the **openWakeWord** component of the **Wyoming** integration.
5. Select **Configure** and **Submit**.
6. Result: You have successfully installed the **openWakeWord** app and **Wyoming** integration.

To enable wake word for your voice assistant

1. Go to **Settings > Voice assistants**
2. Choose the Assistant:
3. To enable wake word for an existing assistant, select the Assistant and continue with step 6.
4. To create a new Assistant: select **Add assistant**.
5. Give your assistant a name, for example the wake word you are going to use.
6. Select the language you are going to use to speak to Home Assistant.
7. If the **Text-to-speech** and **Speech-to-text** sections do not provide language selectors, this means you do not have an Assist pipeline set up.
8. Set up [Home Assistant Cloud](#) or a manually configured [Assist pipeline](#).
9. Under **Text-to-speech**, select the language and voice you want Home Assistant to use when speaking to you.

10. To define the wake word engine, in the top-right corner of the dialog, select the three dots `mdi:dots-vertical` menu and select **Add streaming wake word**.
11. Troubleshooting: If you don't see the three dots `mdi:dots-vertical` menu, go to **Settings > Devices & services** and make sure the **openWakeWord** component of the **Wyoming** integration is added.
12. Result: On the bottom of the page, you now see a new section **Streaming wake word engine**.
13. Troubleshooting: If you don't see the three dots `mdi:dots-vertical` menu, go to **Settings > Devices & services** and make sure the **openWakeWord** component of the **Wyoming** integration is added.
14. Result: on the bottom of the page, you now see a new section **Streaming wake word engine**.
15. Select **openwakeword**, then select **ok nabu**.
16. If you created a new assistant, select **Create**.
17. If you edited an existing assistant, select **Update**.
18. Result: You now have a voice assistant that listens to a wake word.
19. For the first run, it is recommended to use **ok nabu**, just to test the setup.
20. Once you have it all set up, you can create your own wake words.

Try it!

Right now, there are two easy options to get started using wake words: - Follow the [guide to the \\$13 voice assistant](#). This tutorial is using the tiny ATOM Echo, detecting wake words with openWakeWord. - Follow the [guide to set up an ESP32-S3-BOX-3 voice assistant](#). This tutorial is using the bigger S3-BOX-3 device which features a display. It can detect wake words using openWakeWord. But it can also do on-device wake word detection using microWakeWord.

Creating your own wake word

You can now create your own wake word to use with Home Assistant. The procedure below will guide you to train a model. The model is trained using voice clips generated by our local neural text-to-speech system [Piper](#).

Want to know more about how this all works? Check out the [openWakeWord](#) project by David Scripka.

Depending on the word, training a model on your own wake word may take a few iterations and a bit of tweaking. This guide will take you through the process step by step.

Prerequisites

- Latest version of Home Assistant, installed with the Home Assistant Operating System
- [M5Stack ATOM Echo Development Kit](#)
- Successfully completed the [\\$13 voice assistant for Home Assistant](#) tutorial

To create your own wake word

1. Think of a wake word.
2. A word or short phrase (3-4 syllables) that is not commonly used so that it does not trigger Assist by mistake.
3. Currently, only wake words in English are supported.
4. Open the [wake word training environment](#).
5. In section 1, enter your wake word in the **target_word** field. Enter wake word in target field
6. In the code section next to the **target_word**, select the play button. The first time this can take up to 30 seconds.
7. If the play button does not appear, make sure your cursor is placed in the **target_word** field.
Select play button
8. If it still does not show up, in the top right corner of the document, make sure it says **Connected**.
 - If it is not connected, select **Connect to a hosted runtime**. Connect to hosted runtime
9. Result: The pronunciation of your wake word is being created.
 - Once it is finished, at the bottom of the section, you see an audio file. Listen to it.

Listen to demo of your wake word 5. If the word does not sound correct to you: - Follow the instructions in the document to tweak the spelling of the word and press play again. - The word should sound the way you pronounce it. 6. Once you are satisfied with the result, in the menu on top of the screen, select **Runtime > Run all**. - This will take around an hour. Feel free to do something else but make sure to leave the browser tab open. Runtime: run all - Result: Once this process is finished, you should have 2 files in your downloads folder: - `.tflite` and `.onnx` files (only `.tflite` is used)

10. Congratulations! You just applied machine learning to create your own wake word model!

11. The next step is to add it to Home Assistant.

To add your personal wake word to Home Assistant

1. Make sure you have the [Samba app](#) installed.
2. On your computer, access your Home Assistant server via Samba.
3. Open the `share` folder and create a new folder `openwakeword` so that you have `/share/openwakeword`.
4. Drop your shiny new wake word model file (`.tflite`) into that folder.
5. Go to **Settings** > **Voice assistants**.
6. Either create a new assistant and select **Add assistant**.
7. Or, edit an existing assistant.
8. Under **Wake word**, select **openwakeword**.
9. Then, select your own personal wake word.
10. If there is no **Wake word** option, make sure you have the app installed and successfully completed the [\\$13 voice assistant for Home Assistant](#) tutorial.
11. Enable this new assistant on your ATOM Echo device.
12. Go to **Settings** > **Devices & services** and select the **ESPHome** integration.
 - Under **M5Stack ATOM Echo**, select **1 device**.
13. Under **Configuration**, make sure **Use wake word** is enabled.
14. Select the assistant with your wake word.

Select the assistant with your wake word 7. Test your new wake word. - Speak your wake word followed by a command, such as "Turn on the lights in the kitchen". - When the ATOM Echo picks up the wake word, it starts blinking blue.

Troubleshooting

Troubleshooting wake word recognition

1. If the ATOM Echo does not start blinking blue when you say the wake word, there are a few things you can try.
2. Go to **Settings > Devices & services** and select the **ESPHome** integration.
3. Under **M5Stack ATOM Echo**, select **1 device**.
4. Under **Controls**, make sure **Use wake word** is enabled.
5. If this was not the issue, you may need to tweak the wake word model.
 - Go back to the [wake word training environment](#).
 - In section 3 of the document, follow the instructions on tweaking the settings and create a new model.

Troubleshooting performance issues of the training environment

The environment on the Colab space runs on resources offered by Google. They are intended for small-scale, non-commercial personal use. There is no guarantee that resources are available. If many people use this environment at the same time or if the request itself uses a lot of resources, the execution might be very slow or won't run at all.

It may take 30-60 minutes for the run to complete. This is expected behavior.

Things you can try if the execution is very slow:

1. Free of charge solution: This environment has worked for all the wake word models that were trained to create and test this procedure. There is a good chance that it will work for you. If it does not, try training your model another time. Maybe many people are using it right now.
 2. You can pay for more computing resources: In the top right corner, select the RAM | Disk icon.
 3. Select the link to **Upgrade to Colab Pro**.
 4. Select your price plan and follow the instructions on screen. Connect to hosted runtime
-

Voice Control/Custom Sentences

You may add your own sentences to the intent recognizer by either extending an [existing intent](#) or creating a new one. You may also [customize responses](#) for existing intents.

Prerequisites

Prerequisites

You need a working Assist configuration. If you haven't done so yet, check [Assist's starting page](#) to get you ready with your setup.

To add a custom sentence to trigger an automation

This is the easiest method to get started with custom sentences for automations.

1. Under **Settings > Automations & Scenes**, in the bottom right corner, select **Create automation**.
2. In the **Trigger** drop-down menu, select **Sentence**.
3. Enter one or more sentences that you would like to trigger an automation.
4. Do not use punctuation.
5. You can add multiple sentences. They will then all trigger that automation. Add a custom sentence
6. To add a custom response, select **Add action**. Scroll all the way down and select **Other actions**.
7. Then, select **Set conversation response**. Set conversation response
8. In the text field, enter the response you want to hear from Assist and select **Save**.

Enter the response text

- You can also use [wildcards](#). For example, the trigger:

```
yaml play {album} by {artist}
```

could have the response:

```
yaml Playing {{ trigger.slots.album }} by {{ trigger.slots.artist }}
```

- For more details, refer to [conversation response script action](#).
- To test the automation, go to **Overview** and in the top right corner, open Assist.
- Enter one of the sentences.
- If it did not work out, checkout the [troubleshooting](#) section.
- One of the causes could be that the device you're targeting has not been exposed to Assist.
- Pick up your voice control device and speak the custom sentence.
- Your automation should now be triggered.

Setting up custom sentences in configuration.yaml

To set up custom sentences in your configuration file follow [this tutorial](#).

Related devices and installation tutorials

- [\\$13 voice assistant for Home Assistant](#)
 - [S3-BOX-3 voice assistant](#)
 - [Assist for Apple](#)
 - [Assist for Android](#)
-

Voice Control/Custom Sentences Yaml

Intents and sentences may be added in the `conversation` config in your `configuration.yaml` file:

```
# Example configuration.yaml
conversation:
  intents:
    HassTurnOn:
      - "activate [the] {name}"
```

This extends the default English sentences for the `HassTurnOn` intent, allowing you to say "activate the kitchen lights" as well as "turn on the kitchen lights".

New intents can also be added, with their responses and actions defined using the `intent_script` integration:

```
# Example configuration.yaml
conversation:
  intents:
    YearOfVoice:
      - "how is the year of voice going"

intent_script:
  YearOfVoice:
    speech:
      text: "Great! We're at over 40 languages and counting."
```

Besides a text response, `intent_script` can trigger any `action` available in Home Assistant, such as calling a service or firing an event.

Setting up sentences in the config directory

More advanced customization can be done in Home Assistant's `config` directory. YAML files in `config/custom_sentences/en`, for example, will be loaded when English sentences (language code `en`) are requested.

The following example creates a new `SetVolume` intent that changes the volume on one of two media players:

```
# Example config/custom_sentences/en/media.yaml
language: "en"
intents:
  SetVolume:
    data:
      - sentences:
        - "(set|change) {media_player} volume to {volume} [percent]"
        - "(set|change) [the] volume for {media_player} to {volume} [percent]"
lists:
  media_player:
    values:
      - in: "living room"
        out: "media_player.living_room"
      - in: "bedroom"
        out: "media_player.bedroom"
  volume:
    range:
      from: 0
      to: 100
```

As mentioned above, you can then use the `intent_script` integration to implement an action and provide a response for `SetVolume`:

```
# Example configuration.yaml
intent_script:
  SetVolume:
    action:
      service: "media_player.volume_set"
      data:
        entity_id: "{{ media_player }}"
        volume_level: "{{ volume / 100.0 }}"
    speech:
      text: "Volume changed to {{ volume }}"
```

Customizing responses

Responses for existing intents can be customized as well in `config/custom_sentences/`
`<language>`:

```
# Example config/custom_sentences/en/responses.yaml
language: "en"
responses:
  intents:
    HassTurnOn:
      default: "I have turned on the {{ slots.name }}"
```

Related devices and installation tutorials

- [Custom sentences main page](#)
 - [\\$13 voice assistant for Home Assistant](#)
 - [S3-BOX-3 voice assistant](#)
-

Voice Control/Expanding Assist

Once you have completed the steps in the [Best practices](#), you have your bases covered and are ready to use Assist. This section provides some ideas on how to expand your setup for more advanced use cases.

Prerequisites

Assist [up and running](#) in any of the available devices and configured as per the [best practices](#).

Some ideas to expand your Assist setup

1. Add custom sentences or modify existing ones. Learn how in this [custom sentences tutorial](#).
 2. [Create a personality](#) for Assist using AI
 3. Customize your [own wake words](#) for Assist - only available if you have your own hardware setup like [ESP32-S3 BOX](#) or [ATOM Echo](#).
 4. If you want to build something really customized, you can [make your own Assist device](#).
-

Voice Control/Index

Assist is the voice assistant built into Home Assistant. It lets you control your smart home with natural language, and it can run fully on your own hardware, so your voice commands stay private. Assist works in the Home Assistant companion app, on dedicated voice hardware like the [Home Assistant Voice Preview Edition](#), and on devices you build yourself with [ESPHome](#).

Look for the Assist icon Assist icon at the top right of your dashboard to try it out right away.

Assist is built on an open voice foundation and powered by knowledge contributed by our community. It can work locally or, if you prefer, use one of the latest large language models to handle more conversational requests.

Getting started

When you configure voice assistant hardware made for Home Assistant, it will use a wizard to help you configure your system and get started to use voice.

Our recommended voice assistant hardware is the [Home Assistant Voice Preview Edition](#).

In case your hardware does not support our wizard, do not worry. Here are two detailed guides based on how you plan to process your voice (Locally, or using Home Assistant Cloud voice services)

- [I plan to process my voice locally](#)
- [I plan to use Home Assistant Cloud](#) (recommended as it is the simplest)

Expand and experiment

Once your setup is up and running and you follow the [best practices](#), check all the possibilities we found for [Expanding your Assist setup](#), and further experiment with different setups like [wake words](#). Do you want to talk to Super Mario? Or another figure? If you want Assist to respond in a fun way, you can create an assistant with an [AI personality](#).

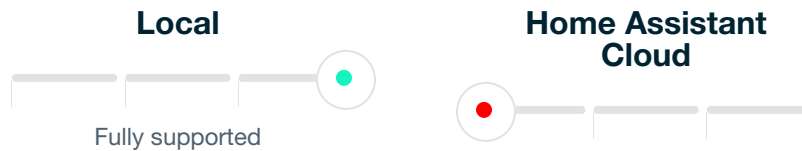
To further improve your setup, try building other voice assistant satellite devices that allow you to add Assist with wake words to all your rooms:

- Enable [wake word detection on your Android phone](#) to activate Assist hands-free by saying "Hey Jarvis" or "Hey Nabu", even when your phone is locked.
- You can use [ESPHome](#) to create your own awesome voice assistant satellites based on inexpensive ESP32 microcontrollers, like [@piitaya](#) did with his 3D-printed R5 droid. Follow our tutorial to [create your own for just \\$13](#).
- Another alternative voice satellite solution is the experimental [Linux-Voice-Assistant](#) project. It allows you to build a Linux-based voice assistant smart speaker that runs on any x64 or ARM64 hardware capable of handling local, on-device audio processing. This approach provides greater flexibility for customization. Because it runs on a full Linux system, it also gives you access to significantly more local computing resources for additional features and other integrations on the same satellite.
- If you are interested in a voice assistant that is not always listening, consider using Assist on an analog phone. It will only listen when you pick up the horn, and the responses are for your ears only. Follow our tutorial to create your own [analog phone voice assistant](#).

Supported languages and sentences

Assist aims to support more languages than other voice assistants, but this is still a work in progress, and we need your help.

Check supported languages here



Assist already supports a wide range of [languages](#). Use the [built-in sentences](#) to control entities and areas, or [create your own sentences](#).

Did Assist not understand your sentence? [Contribute them](#).

Assist was introduced in Home Assistant 2023.2.

Voice Control/Install Wake Word Add On

This tutorial shows how you can *enable* a wake word in Home Assistant. It does not describe how to *use* it.

To *use* the wake word, you need some extra hardware. A low cost option is the [M5Stack ATOM Echo Development Kit](#). To set that up, follow the [\\$13 voice assistant for Home Assistant](#). Note that the Home Assistant Voice Preview does not currently support custom wake words.

To enable a wake word

Enabling a wake word consists of 2 steps:

1. Installing the **openWakeWord** app (formerly known as an add-on).
2. Enabling the wake word for a specific voice assistant.

Prerequisites

- Home Assistant version 2023.10 or later, installed with the Home Assistant Operating System
- [Home Assistant Cloud](#) or a manually configured [local Assist pipeline](#)

Installing the openWakeWord app

1. Go to **Settings** > **Apps** > **openWakeWord** and select **Install**.
2. **Start** the app.
3. Go to **Settings** > **Devices & services**.
4. Under **Discovered**, you should now see the **openWakeWord** component of the **Wyoming** integration.
5. Select **Configure** and **Submit**.
6. Result: You have successfully installed the **openWakeWord** app and **Wyoming** integration.

Enabling wake word for your voice assistant

1. Go to **Settings** > **Voice assistants**
2. Choose the Assistant:
3. To enable wake word for an existing assistant, select the Assistant and continue with step 6.
4. To create a new Assistant: select **Add assistant**.
5. Give your assistant a name, for example the wake word you are going to use.
6. Select the language you are going to use to speak to Home Assistant.
7. If the **Text-to-speech** and **Speech-to-text** sections do not provide language selectors, this means you do not have an Assist pipeline set up.
8. Set up [Home Assistant Cloud](#) or a manually configured [Assist pipeline](#).
9. Under **Text-to-speech**, select the language and voice you want Home Assistant to use when speaking to you.
10. To define the wake word engine, in the top-right corner of the dialog, select the three dots `mdi:dots-vertical` menu and select **Add streaming wake word**.
11. Troubleshooting: If you don't see the three dots `mdi:dots-vertical` menu, go to **Settings** > **Devices & services** and make sure the **openWakeWord** component of the **Wyoming** integration is added.
12. Result: On the bottom of the page, you now see a new section **Streaming wake word engine**.

13. Troubleshooting: If you don't see the three dots `mdi:dots-vertical` menu, go to **Settings** > **Devices & services** and make sure the **openWakeWord** component of the **Wyoming** integration is added.
14. Result: On the bottom of the page, you now see a new section **Streaming wake word engine**.
15. Select **openwakeword**, then select **ok nabu**.
16. If you created a new assistant, select **Create**.
17. If you edited an existing assistant, select **Update**.
18. Result: You now have a voice assistant that listens to a wake word.
19. For the first run, it is recommended to use **ok nabu**, just to test the setup.
20. Once you have it all set up, you can [create your own wake words](#).

Related topics

About wake words and assistants

- [Create your own wake words](#)
- [About wake words](#)
- [Create a cloud assistant](#)
- [Create a local assistant](#)

Extra hardware to use wake word

- [M5Stack ATOM Echo Development Kit](#)
 - [Tutorial: \\$13 voice assistant for Home Assistant](#)
 - [ESP32-S3-BOX-3](#)
 - [Tutorial: ESP32-S3-BOX-3 voice assistant](#)
-

Voice Control/Make Your Own

If you wish to develop your own Assist device here are some useful advices we can think of:

1. Make sure you understand the setup and implications of voice Assistants. For reference, we have and we highly recommend you check the information in the [developers Portal](#).
 2. Make sure your setup is going to work with enough quality when adding a voice assistant.
 3. Check hardware options, some options we cover in the documentation are:
 4. [ATOM Echo](#)
 5. [ESP32-S3-BOX devices](#)
 6. For [landline setups](#) be sure to have [VOIP](#)
 7. Be sure you check all the addons implied in text-to-speech and speech-to-text, like the [Wyoming protocol](#).
-

Customize the S3-BOX-3 with your own illustrations

This tutorial will show you how to replace the Home Assistant status illustrations on the Espressif ESP32-S3-BOX-3 with your own images.

You can either prepare your own illustrations or import some from a community repository.

ESP32-S3-BOX-3 voice assistant status illustrations

The ESP32-S3-BOX-3 voice assistant has 6 illustrations to indicate its state:

Screenshot showing energy price graph. The ESP32-S3-BOX-3 states: loading, idle, listening, thinking, replying, error.

The chart shows the default illustrations. The next steps show you how to change those.

Prerequisites

- Latest version of Home Assistant, installed with the Home Assistant Operating System
- [Home Assistant Cloud](#) or a manually configured [Assist Pipeline](#)
- [ESP32-S3-BOX-3](#). The ESP32-S3-BOX-Lite or the ESP32-S3-BOX also work, but they are not currently on the market.
- Successfully completed the [ESP32-S3-BOX-3 voice assistant](#) tutorial

Adopting the device in the ESPHome app

Before you can import new illustrations, you need to install the ESPHome app (formerly known as an add-on) and adopt the device in the app.

1. Make sure the ESP32-S3-BOX-3 is up and running and connected to your Wi-Fi.
2. Go to **Settings** > **Apps** and check if you have the **ESPHome** app installed.
3. If you haven't done so, go to **Settings** > **Apps** > **ESPHome** to install the app.
4. Start the app and select **Open web UI**.
5. In the ESPHome app dashboard, on the **ESP32-S3-BOX-3** card, select **Adopt**.

Adopt the ESP32-S3-BOX-3 in the ESPHome app 5. If you like, give it a new name. Then, select **Adopt**. - Adopting an ESPHome device allows us to customize the existing software. - Result: The status will change to **Online**. 6. Now that you have set up the ESPHome app, continue with one of the 2 different methods to add custom images: - [Option 1: Using images from a community repository](#) - [Option 2: Using your own images](#)

Option 1: Using images from a community repository

If you want new images but don't want to create your own, you can use images from the community. If you want to use your own images, skip this procedure and go to [Option 2: Using your own images](#) instead.

To use images from the community

1. On the **ESP32-S3-BOX-3** app, select edit.
2. Result: An editor opens, showing the configuration file. ESP32-S3-BOX-3 config file
3. For inspiration, we have prepared some images for you.
4. Check them out in this [repository](#).
5. For this tutorial, we will use some images of Frenck.
6. Add the following lines into the `substitutions` block.

```
yaml substitutions: loading_illustration_file: https://github.com/jlpouffier/home-assistant-s3-box-community-illustrations/raw/main/frenck/illustrations/loading.png idle_illustration_file: https://github.com/jlpouffier/home-assistant-s3-box-community-illustrations/raw/main/frenck/illustrations/idle.png listening_illustration_file: https://github.com/jlpouffier/home-assistant-s3-box-community-illustrations/raw/main/frenck/illustrations/listening.png thinking_illustration_file: https://github.com/jlpouffier/home-assistant-s3-box-community-illustrations/raw/main/frenck/illustrations/thinking.png replying_illustration_file: https://github.com/jlpouffier/home-assistant-s3-box-community-illustrations/raw/main/frenck/illustrations/replying.png error_illustration_file: https://github.com/jlpouffier/home-assistant-s3-box-community-illustrations/raw/main/frenck/illustrations/error.png
```

7. Save the changes and select **Install**:
8. Depending on your environment, the installation process can take a while. ESP32-S3-BOX-3 config file
9. Once the installation is complete, you can see the new image on the ESP32-S3-BOX-3.
10. Now, speak a command to test the new setting. For example, *OK Nabu, turn off the living room lights*.

Option 2: Using your own illustrations

There are 2 parts to this:

- Preparing your own illustrations
- Adding your illustrations to the configuration

About the image specifications

Here's what you need to know to get the best result on your ESP32-S3-BOX-3 screen.

Using light and dark image background

In the [overview diagram](#), you can see that the default images use different background colors. This is to make it easier to recognize a state change when you look at your screen.

In your images, you could use 2 different background colors:

- For loading and idle: use a dark background

- For listening, thinking, and replying: use a light background
- For error: As you like

If your images have transparency, you can define the background color in the configuration. The procedure below shows how to change the background.

Image dimensions and file format

- **Dimensions:** The screen is 320 x 240 pixels. If the image you provide is not in a 4:3 ratio, the remaining area will be filled with background color.
- **File format:** PNG, JPEG, or SVG

To prepare your own images

1. Create your own images according to the specifications defined in the section [About the image specifications](#).
2. You could even draw your own!
3. There's a [template](#) for it.
4. Copy all 6 images into a folder. For example: `voice_assistant_gfx`.
5. Make sure you have [access to your configuration files](#).
6. [Install the Samba app](#).
7. This allows you to copy multiple files at once.
8. Copy your image folder into the configuration folder:
9. Open the `config` folder and open the `ESPHome` folder.
10. Copy your image folder in there. ESP32-S3-BOX-3 config file

To add your images to the configuration

1. In Home Assistant, go to **Settings > Apps > ESPHome**, and **Open Web UI**.
2. On the **ESP32-S3-BOX-3** app, select edit.
3. Result: An editor opens, showing the configuration file. ESP32-S3-BOX-3 config file
4. To add your images, add the following lines into the `substitutions` block.

```
yaml substitutions: loading_illustration_file: voice_assistant_gfx/loading.png
idle_illustration_file: voice_assistant_gfx/idle.png listening_illustration_file:
voice_assistant_gfx/listening.png thinking_illustration_file:
voice_assistant_gfx/thinking.png replying_illustration_file: voice_assistant_gfx/
replying.png error_illustration_file: voice_assistant_gfx/error.png
```

1. If you used transparency in your images and you want to change the background color, add the following lines into the `substitutions` block:
2. The `000000` stands for black, `FFFFFF` stands for white in [hexadecimal color code](#).
3. If you want to use different colors, replace the corresponding color code.
4. To find the color code, you can use a tool like the Google color picker.

```
yaml substitutions: ... loading_illustration_background_color: '000000'
idle_illustration_background_color: '000000'
```

```
listening_illustration_background_color: 'FFFFFF'  
thinking_illustration_background_color: 'FFFFFF'  
replying_illustration_background_color: 'FFFFFF'  
error_illustration_background_color: '000000'
```

5. Save the changes and select **Install**. ESP32-S3-BOX-3 config file
6. Save the changes.
7. Depending on your environment, the installation process can take a while.
8. Once the installation is complete, you can see the new image on the S3-BOX.
9. Now, speak a command to test the new setting. For example, *OK Nabu, turn on the light*.

To draw your own images

1. We prepared a template for you to draw your own status images. ESP32-S3-BOX-3 config file
2. Download the file and start drawing!

[Draw Assist](#) 3. When you are done: - Snap a picture of each. - [Follow these steps](#) to bring them onto your Voice Assistant.

Customizing on-device wake words (microWakeWord)

You can change the on-device wake word (microWakeWord) that is used on your S3-BOX-3.

Prerequisites

- Home Assistant 2024.2, installed with the Home Assistant Operating System. If you do not have Home Assistant installed yet, refer to the [installation page](#) for instructions.
- Successfully [installed ESPHome on the S3-BOX-3](#)
- ESPHome 2024.2 or later
- Home Assistant server with at least 2 GB of RAM free
- The firmware needs to be compiled on the server before it is installed on the S3-BOX-3.
- Compiling requires a bit of RAM.
- [On-device wake word installed](#) on your S3-BOX-3.

(It also works on the (now discontinued) S3-BOX and S3-BOX-Lite)

To customize the S3-BOX-3 with on-device wake words

1. If you haven't done so already, [adopt the device in the ESPHome app](#).
 2. In Home Assistant, go to **Settings > Apps > ESPHome**, and **Open Web UI**.
 3. On the **ESP32-S3-BOX-3** entry, select edit.
 4. Result: An editor opens, showing the configuration file. ESP32-S3-BOX-3 config file
 5. To change the wake word, add the following line into the `substitutions` block.
 6. Instead of `okay_nabu`, you can also use `alexa` or `hey_jarvis`.
- ```
yaml substitutions: ... micro_wake_word_model: hey_jarvis
```
7. Save the changes and in the top right corner, select **Install**.
  8. Depending on your environment, the installation process can take a while.
  9. On Home Assistant Green, for example, it takes about 45 minutes.
  10. Select the **ESPHome** integration. Under **Devices**, you should see the **ESP32-S3-BOX** listed.
    - On the ESP32-S3-BOX-3 entry, select **Device** to open the device page.
    - Under **Wake word engine location**, select **On device**.

ESP32-S3-BOX-3 on device wake word processing

11. Now, speak a command to test the new setting. For example, *Hey Jarvis, turn on the light*.

## Voice Control/S3 Box Voice Assistant

---

This tutorial will guide you to turn an ESP32-S3-BOX, ESP32-S3-BOX-3(B), or an ESP32-S3-BOX-Lite into a Home Assistant voice assistant. Note, the term ESP32-S3-BOX may be used to refer to any of the 3 product variants.

## Prerequisites

---

- Home Assistant 2023.12 or later, installed with the Home Assistant Operating System. If you do not have Home Assistant installed yet, refer to the [installation page](#) for instructions.
- [Home Assistant Cloud](#) or a manually configured [Assist Pipeline](#)
- The password to your 2.4 GHz Wi-Fi network
- Chrome or Edge browser on a desktop (not Android/iOS)
- One of the Espressif ESP32-S3-BOX variants:
  - ESP32-S3-BOX-3B
  - ESP32-S3-BOX-3, ESP32-S3-BOX, or ESP32-S3-BOX-Lite (not currently on the market)
- USB-C cable to connect the ESP32-S3-BOX
- This procedure assumes that this is the first time you are installing ESPHome firmware on the device. If you have previously completed this tutorial and now want to install the latest software version, follow the steps on [updating the software on the S3-BOX](#).

## Installing the software onto the ESP32-S3-BOX

---

Before you can use this device with Home Assistant, you need to install a bit of software on it.

{% tabbed\_block %}

- title: Using the ESP32-S3-BOX-3(B) content: |

1. These steps apply both to the ESP32-S3-BOX-3 and the ESP32-S3-BOX-3B. Make sure this page is opened in a Chromium-based browser on a **desktop**. The software installation does not work with a tablet or phone.
2. Select the **Connect** button below to display a list of available USB devices. Do not connect the ESP32-S3-BOX-3 yet. We want to see the list of available USB devices first, so that it is easier to recognize the ESP device afterward.
3. If your browser does not support web serial, you will see a warning message indicating this instead of a button.
4. **For advanced users:** The configuration files are available on GitHub:
  - [ESP32-S3-BOX-3 config on GitHub](#)
5. To connect the ESP32-S3-BOX-3 to your computer, follow these steps:
6. In the pop-up window, view the available ports.
7. Plug the USB-C cable into the box directly, not into the docking station (not into the blue part) and connect it to your computer.
8. Troubleshooting: If your ESP32-S3-BOX-3 does not appear in the list of devices presented by your browser, you need to manually invoke "flash mode":
  - Hold the "boot" button (left side upper button) as you tap the "reset" button (left side lower button).
  - After a few seconds, the ESP32-S3-BOX-3 should appear in the list of USB devices presented by your browser.
  - Follow the steps until step 3. After selecting the **Next** button, on the S3-Box-3, tap the "Reset" button again.
  - Then, select the blue **Connect button** again, select the USB device and follow the prompts to configure the Wi-Fi.
  - In the pop-up window, there should now appear a new entry. Select this USB serial port and select **Connect**.
9. Select **Install Voice Assistant**, then **Install**.
  - Once the installation is complete, select **Next**.
  - Add the ESP32-S3-BOX-3 to your Wi-Fi:
  - When prompted, select your network from the list and enter the credentials to your 2.4 GHz Wi-Fi network.
  - Select **Connect**.
  - The ESP32-S3-BOX-3 now joined your network. Select **Add to Home Assistant**.

10. This opens the **My** link to Home Assistant.
11. If you have not used My Home Assistant before, you will need to configure it. If your Home Assistant URL is not accessible on `http://homeassistant.local:8123`, replace it with the URL to your Home Assistant instance.
12. Open the link. Open My link
13. Select **OK**.

Set up ESPHome 6. To add the newly discovered device, select the ESP32-S3-BOX-3 from the list. - Add your ESP32-S3-BOX-3 to a room and select **Finish**. 7. You should now see the **ESPHome** integration. New ESPHome device discovered

1. Select the **ESPHome** integration. Under **Devices**, you should see the **ESP32-S3-BOX** listed. ESP32-S3-BOX-3 discovered
  - Your ESP32-S3-BOX is connected to Home Assistant over Wi-Fi. You can now move it to any place in your home with a USB power supply.
- title: Using the ESP32-S3-BOX content: |
  1. Make sure this page is opened in a Chromium-based browser on a **desktop**. The software installation does not work with a tablet or phone.
  2. If your browser does not support web serial, you will see a warning message indicating this instead of a button.
  3. **For advanced users:** The configuration files are available on GitHub:
    - [ESP32-S3-BOX config on GitHub](#)
  4. To connect the ESP32-S3-BOX to your computer, follow these steps:
  5. In the pop-up window, view the available ports.
  6. Plug the USB-C cable into the ESP32-S3-BOX and connect it to your computer.
  7. Select **Install Voice Assistant**, then **Install**.
    - Once the installation is complete, select **Next**.
    - Add the ESP32-S3-BOX to your Wi-Fi:
      - When prompted, select your network from the list and enter the credentials to your 2.4 GHz Wi-Fi network.
    - Select **Connect**.
    - The ESP32-S3-BOX now joined your network. Select **Add to Home Assistant**.
  8. This opens the **My** link to Home Assistant.
  9. If you have not used My Home Assistant before, you will need to configure it. If your Home Assistant URL is not accessible on `http://homeassistant.local:8123`, replace it with the URL to your Home Assistant instance.
  10. Open the link. Open My link
  11. Select **OK**. Set up ESPHome
  12. To add the newly discovered device, select the ESP32-S3-BOX from the list.
  13. Add your ESP32-S3-BOX to a room and select **Finish**.
  14. You should now see the **ESPHome** integration. New ESPHome device discovered

15. Select the **ESPHome** integration. Under **Devices**, you should see the **ESP32-S3-BOX** listed. ESP32-S3-BOX-3 discovered

- Your ESP32-S3-BOX is connected to Home Assistant over Wi-Fi. You can now move it to any place in your home with a USB power supply.

• title: Using the ESP32-S3-BOX-Lite content: |

1. Make sure this page is opened in a Chromium-based browser on a **desktop**. The software installation does not work with a tablet or phone.
2. If your browser does not support web serial, you will see a warning message indicating this instead of a button.

3. **For advanced users:** The configuration files are available on GitHub:

- [ESP32-S3-BOX-Lite config on GitHub](#)

4. To connect the ESP32-S3-BOX-Lite to your computer, follow these steps:

5. In the pop-up window, view the available ports.

6. Plug the USB-C cable into the ESP32-S3-BOX-Lite and connect it to your computer.

7. Select **Install Voice Assistant**, then **Install**.

- Once the installation is complete, select **Next**.
- Add the ESP32-S3-BOX-Lite to your Wi-Fi:
- When prompted, select your network from the list and enter the credentials to your 2.4 GHz Wi-Fi network.
- Select **Connect**.
- The ESP32-S3-BOX-Lite now joined your network. Select **Add to Home Assistant**.

8. This opens the **My** link to Home Assistant.

9. If you have not used My Home Assistant before, you will need to configure it. If your Home Assistant URL is not accessible on `http://homeassistant.local:8123`, replace it with the URL to your Home Assistant instance.

10. Open the link. Open My link

11. Select **OK**. Set up ESPHome

12. To add the newly discovered device, select the ESP32-S3-BOX-Lite from the list.

13. Add your ESP32-S3-BOX-Lite to a room and select **Finish**.

14. You should now see the **ESPHome** integration. New ESPHome device discovered

15. Select the **ESPHome** integration. Under **Devices**, you should see the **ESP32-S3-BOX-Lite** listed. ESP32-S3-BOX-3 discovered

- Your ESP32-S3-BOX-Lite is connected to Home Assistant over Wi-Fi. You can now move it to any place in your home with a USB power supply.

{% endtabbed\_block %}

## Checking wake word settings

---

1. Make sure your assistant has [wake word enabled](#), using "OK Nabu".
2. Under **Devices**, on the ESP32-S3-BOX entry, select *Device\** to open the device page.
3. Check the device settings:
  - If you want, you can process the wake word on the ESP32-S3 device, rather than on your Home Assistant server. (The server is the device where Home Assistant is installed, for example on Home Assistant Green):
  - Under **Wake word engine location**, select **On device**, if you want your wake word to be processed on the device itself, and not in Home Assistant.
  - Local processing is faster.
  - The wake word is now *Okay Nabu*.

### ESP32-S3-BOX-3 on device wake word processing

4. If you chose on-device wake word, but you do not want to use *Okay Nabu*, you can change the on-device wake word.
5. Currently, *Hey Jarvis* or *Alexa* are the supported alternatives.
6. To change your wake word, follow the steps in [Customizing the S3-BOX-3 with on-device wake words](#).
7. Congratulations! You can now voice control Home Assistant via a ESP32 device with a display. Now give some commands.

## Controlling Home Assistant

---

1. Say your wake word. For this tutorial, use "OK Nabu".
2. Say a [supported voice command](#). For example, *Turn on the light*.
3. Once the intent has been processed, the LED lights up in green and Home Assistant confirms the action.
  - Make sure you're using the area name exactly as you defined it in Home Assistant.
  - You can also ask a question, such as
    - *Is the front door locked?*
    - *Which lights are on in the living room?*
4. Your command is not supported? Add your own commands using [a sentence trigger](#).



## Turning off microphone or screen

---

1. If you do not want to Assist to listen to you for a while, you can turn off the microphone.

2. Go to **Settings > Devices & services** and select the **ESPHome** integration.

- Under **ESP32-S3-BOX-3**, select **1 device**.
- Enable **Mute**.
- The screen of the ESP32-S3-BOX-3 will turn off, too.

Toggle to enable/disable Mute 2. If you want to just use the wake word, but do not want to use the screen, you can turn it off. - Go to **Settings > Devices & services** and select the **ESPHome** integration. - Under **ESP32-S3-BOX-3**, select **1 device**. - Disable **Screen**.

Toggle to enable/disable wake word

## Updating the software on the S3-BOX

---

To update the software on your S3-BOX, follow the steps below that reflect your setup.

- **Option 1:** You have Home Assistant 2024.7 or later, and have not manually altered your ESPHome configuration for the S3-BOX:
    - Once an update is available, you will receive an update notification, just like any other update.
    - To install the precompiled new firmware directly on your box, make sure the S3-BOX is connected to your network, and under **ESP32 S3 BOX...Firmware**, select **Install**.
  - **Option 2:** You have Home Assistant 2024.6 or older, and have not manually altered your ESPHome configuration for the S3-BOX:
    - Follow steps 1-3 of the procedure on [installing the software onto the S3-BOX](#).
      - This installs the latest, precompiled firmware for your S3-BOX.
  - **Option 3:** You have manually changed the configuration file for your S3-BOX:
    - You need to compile your own firmware. To do so, either:
      - Use the ESPHome dashboard app within Home Assistant. While the easiest option, it tends to be the slowest and may fail, particularly on older systems or on systems with limited memory/CPU resources.
      - Follow the steps in the [ESPHome documentation](#) and use a desktop-class system to compile and install the firmware. Initial setup is more complex, but the process is significantly faster and more reliable.
-

## Voice Control/Start Assist From Dashboard

---

If you are using Home Assistant in kiosk mode, for example if you have a tablet mounted on the wall, the Assist icon in the top right corner is not accessible. In this case, use a dashboard button to start Assist.

## Prerequisites

---

- You have a [local assistant](#) or a [Cloud assistant](#) set up.
- You have the devices you want to control via Assist [exposed to Assist](#).

## Adding an Assist button to the dashboard

---

1. On your dashboard, select the `mdi:pencil` button.
  2. If it is the first time that you are editing your dashboard, in the **Edit dashboard** dialog, go to the three dots `mdi:dots-vertical` menu and select **Take control**.
  3. Do one of the following, depending on the type of the view you are editing:
    - In a sections view, select the `mdi:plus` button below **New Section**.
    - In a masonry, panel, or sidebar view, select **Add card** in the lower right corner of the view.
  4. On the **By card** tab of the dialog, select the **Button** card.
  5. Clear the **Entity** field.
  6. In **Name**, select **+ Add** and enter a name for the card, such as *Assist - listen*.
  7. Select an icon from the **Icon** dropdown list, such as `mdi:account-tie-voice`.
  8. In the **Interactions** section:
  9. Select the desired tap behavior and, in the related dropdown list, select **Assist**.
  10. Select the assistant you want to use, for example **Home Assistant Cloud**, from the **Assistant** dropdown list.
    - You can use any assistant that you have previously set up.
    - If you have assistants in different languages, you can add a button for each of these languages.
  11. Enable **Start listening** if you are using Assist with your voice. If you don't want to use voice and just want to type, you do not need to enable listening.
  12. Select **Save**.
  13. In the upper right corner, select **Done** to save the changes to your dashboard.
-

## Voice Control/Thirteen Usd Voice Remote

---

This tutorial will guide you to turn an ATOM Echo into a voice assistant. Pick up the tiny device to talk to your smart home. Issue commands and get responses!

## Prerequisites

---

- Home Assistant 2023.10 or later, installed with the Home Assistant Operating System. If you do not have Home Assistant installed yet, refer to the [installation page](#) for instructions.
- [Home Assistant Cloud](#) or a manually configured [Assist Pipeline](#)
- Have [enabled a wake word](#) for your voice assistant
- The password to your 2.4 GHz Wi-Fi network
- Chrome (or a Chromium-based browser like Edge) on desktop (not Android/iOS)
- [M5Stack ATOM Echo Development Kit](#)
- USB-C cable to connect the ATOM Echo

## Installing the software onto the ATOM Echo

---

Before you can use this device with Home Assistant, you need to install a bit of software on it.

1. Make sure this page is opened in a Chromium-based browser on a desktop. It does not work on a tablet or phone.
2. Select the **Connect** button below. If your browser does not support web serial, you will see a warning instead of a button.

- **For advanced users:** The configuration file is available on [GitHub](#).

3. To connect the ATOM Echo to your computer, follow these steps:
4. In the pop-up window, view the available ports.
5. Plug the USB-C cable into the ATOM Echo and connect it to your computer.
6. In the pop-up window, there should now appear a new entry. Select this USB serial port and select **Connect**.
7. Troubleshooting: If no new port shows, your system may be missing a driver. Close the pop-up window.

- In the dialog, select the CH342 driver, install it, then **Try again**. Open [My link](#)

8. Select **Install Voice Assistant**, then **Install**.

- Once the installation is complete, select **Next**.
- Add the ATOM Echo to your Wi-Fi:
- When prompted, select your network from the list and enter the credentials to your 2.4 GHz Wi-Fi network.
- Select **Connect**.
- The ATOM Echo now joined your network. Select **Add to Home Assistant**.

9. This opens the **My** link to Home Assistant.

10. If you have not used My Home Assistant before, you will need to configure it. If your Home Assistant URL is not accessible on `http://homeassistant.local:8123`, replace it with the URL to your Home Assistant instance.

11. Open the link.

Open [My link](#)

12. Select **OK**.

Set up ESPHome

1. This starts the a wizard to customize the your voice assistant.
2. Follow the wizard steps to define the wake word and choose the voice.
3. When you are finished, select **Done**.
4. Your ATOM Echo is connected to Home Assistant over Wi-Fi. You can now move it to any place in your home with a USB power supply.
5. Congratulations! You can now voice control Home Assistant. Now give some commands.



## Controlling Home Assistant over the ATOM Echo

---

1. Say the wake word you configured. For example, use "OK, Nabu".
2. Wait for the LED to start blinking in blue.
3. Say a [supported voice command](#). For example, *Turn off the light in the kitchen*.
4. While you are speaking, the blue LED keeps pulsing.
5. Once the intent has been processed, the LED lights up in green and Home Assistant confirms the action.
  - Make sure you're using the area name exactly as you defined it in Home Assistant.
  - You can also ask a question, such as
    - *Is the front door locked?*
    - *Which lights are on in the living room?*
6. Your command is not supported? Add your own commands using [a sentence trigger](#).
7. You find ATOM Echo takes too long to start processing your command?
8. Adjust the silence detection settings.
9. Go to **Settings > Devices & services** and select the **ESPHome** integration.
10. Under **M5Stack ATOM Echo**, select **1 device**. Under **Configuration**, change the **Finished speaking detection**.
11. This setting defines how much silence is needed for Assist to find you're done speaking and it can start processing your command.

Open My link

## Troubleshooting

---

Are things not working as expected?

- Checkout the [general troubleshooting](#) section for Assist.

## Removing the Wi-Fi credentials from the ATOM Echo

---

If you no longer use the device or want to pass it on to someone else, you can remove the Wi-Fi credentials that are stored on the device.

1. Make sure this page is opened in a Chromium-based browser on a desktop. It does not work on a tablet or phone.
  2. Select the **Connect** button below. If your browser does not support web serial, you will see a warning instead of a button.
  3. To connect the ATOM Echo to your computer, follow these steps:
    4. In the pop-up window, view the available ports.
    5. Plug the USB-C cable into the ATOM Echo and connect it to your computer.
    6. In the pop-up window, there should now appear a new entry. Select this USB serial port and select **Connect**.
    7. In the dialog, select **Erase user data**.
    8. Result: Your Wi-Fi credentials are deleted from the device.
    9. The firmware stays on the device.
-

## Voice Control/Troubleshooting

---

This section lists a few steps that may help you troubleshoot issues with Assist.

## View debug information

---

1. Go to **Settings** > **Voice assistants**.
2. From the list of assistants, go to your assistant and select **Debug** in the dialog. Open the debug dialog
3. At the top of the screen, from the dropdown menu, select the run you are interested in. Debug speech-to-text

## Test a sentence per language without voice: without executing commands

---

If you want to test if a sentence works in a specific language without actually executing the commands, use the sentence parser in the **Developer tools**.

1. Go to **Settings > Developer tools > Assist**.
2. In the sentence parser, select the language and enter the sentence you want to test.
3. The debug tool shows you the following:
4. The intent triggered.
5. The entities that were targeted.
6. Which of the targeted entities were matched. Open the Assist developer tool sentence parser

## Test a sentence per assistant without voice: while executing the commands

---

If you want to test if a sentence works with a specific assistant while actively executing the commands, use the sentence parser in the **Debug** view.

1. [Open the debug view](#).
2. In the top right corner, select the icon. Open the pipeline debug dialog
3. Select the assistant you want to test.
4. Select **Run text pipeline**. Open the pipeline debug dialog
5. Enter the phrase you want to test and select **Run**. Open the pipeline debug dialog
6. Check if it worked. Open the pipeline debug dialog
7. If the phrase does not work, try a variant. For example, if *Turn off the light* doesn't work, try:  
*Turn off the lights in the kitchen*.
8. Check if your phrase is [supported](#).
9. Make sure you are using the name of the area as it is defined in Home Assistant. If you have a room called *bathroom*, the phrase *Turning on the lights in the bath* won't work.

## I do not see any assistant

---

If under **Settings** > **Voice assistants** you do not see any assistants, you are not using the default configuration. The image below shows the **Assist** section.

Open the pipeline debug dialog

If the **Assist** section is missing entirely, you need to add the following to your `configuration.yaml` file:

```
yaml # Example configuration.yaml entry assist_pipeline:
```



## Assist does not understand my question about the weather forecast

---

The example below shows common pitfalls when enquiring about the weather. While some steps are specific to the weather, the general mechanics apply to other entities as well.

1. Make sure you have a [weather service](#) installed.
2. By default, [Met.no](#) is installed.
3. Make sure you have an entity set up for the location you are interested in.
4. For example, if you are interested in the weather in Berlin, add an entity for Berlin.

Create weather entity 3. Make sure the entity is exposed to Assist: - Under **Settings > Devices & services > Entities**, select the weather entity for that location. - In the details view that opens, select the cogwheel `mdi:cog-outline`, then select **Voice Assistant**.

Select voice assistants

5. Make sure the entity is exposed to Assist.

Expose entity to Assist

6. Make sure you use the exact entity name when talking to Assist.
7. To view the entity name, check the list under **Settings > Devices & services > Entities**.
8. For example, if the entity is called *Forecast Berlin*, you have to say "What is the weather in forecast Berlin like".
9. Assist would not recognize it if you ask "What is the weather in Berlin like".
10. If you want to use Berlin instead of *Forecast Berlin*, you can create an entity name alias.

- You can create as many aliases as you like.

Create alias for entity name 5. If you just ask "What is the weather" when you have forecast entities for multiple entities, Assist always returns the data for the place that was first added. Currently, there is no way to change that.

## I don't get a voice response

---

My voice assistant understands me and processes the command, but I don't get a voice response.

The voice response is generated in Home Assistant by one of our supported text-to-speech (or TTS) engines. The voice assistant device then fetches the audio file from Home Assistant and plays it back.

### Local Network Settings

For this fetching process to work, Home Assistant must communicate its own URL to the device. If you have a complex network setup, or if you changed this setting in the past, the URL communicated could be wrong.

To fix the URL, follow these steps:

1. In your user profile, enable **Advanced Mode**.
2. Go to **Settings > System > Network**.
3. Change your Local Network Home Assistant URL to a URL that can be reached locally and that points to Home Assistant
4. For most users, the **Automatic** option works and is recommended. Create alias for entity name

### Missing Media Source

If you are using YAML configuration and do not have `default_config:` make sure `media_source:` is present.

## Twaking the Assist audio configuration for your device

---

You think there is an issue with background noise or speaker volume? In some cases, it can help to tweak settings such as noise suppression and gain of your voice assistant device (such as the S32-S3-BOX-3).

### To tweak the Assist audio configuration for your device

1. Make sure you have the ESPHome app (formerly known as an add-on) installed:
2. Go to **Settings** > **Apps** > **Install app**.
3. If you do not have the **ESPHome** app installed, install it.
4. Start the ESPHome app, and select **Open Web UI**.
5. Adopt your device to the ESPHome app:
6. Once the ESPHome app is started, you see your device as **Discovered**.
7. Select **Adopt**.
8. When prompted, enter the Network credentials of your local 2.4 GHz Wi-Fi network and select **Adopt**.
9. If you see a notification that there is an update available for this device, select **Update**.
10. Make sure you have access to the configuration file.
11. If you are unsure what method to use, [install the File editor](#) app.
12. In the File Editor configuration, make sure the **Enforce basepath** option is disabled.
13. Edit the general configuration to enable debug mode:
14. Access the `config` folder and open the `configuration.yaml` file.
15. Enter the following text:

```
yaml assist_pipeline: debug_recording_dir: /share/assist_pipeline
```
16. Save the changes and restart Home Assistant.
17. Navigate to `/share/assist_pipeline`.
18. For each voice command you gave, you will find a subfolder with the audio file in `.wav` format.
19. Listen to the audio file of interest.
20. Open the configuration file:
  - In the ESPHome app, on your device, select **Edit**.
  - This lets you edit the configuration file of that device.
21. To add a section to adjust noise suppression and volume, add the following lines:

```
yaml voice_assistant: noise_suppression_level: 3 auto_gain: 31dBFS
volume_multiplier: 10.0
```
22. Adjust the settings:
  - If the audio is too noisy, increase the `noise_suppression_level` (max. 4).

- If the audio is too quiet, increase either the `auto_gain` (max. 31) or the `volume_multiplier` (no maximum, but a too high value will cause distortion eventually).

23. Note: Collecting debug recordings impacts your disk space.

- Once you have found a configuration that works, delete the folder with the audio files.
  - In the `configuration.yaml` file, delete the `assist_pipeline` entry and restart Home Assistant.
-

## Voice Control/Troubleshooting The S3 Box

---

This section provides troubleshooting steps for the ESP32-S3-BOX-3 by Espressif.

# Error: Unable to connect to Wi-Fi

---

## Symptom

The ESP32-S3-BOX-3 shows a message that it is unable to connect to Wi-Fi.

## Remedy

1. First, check if your network is ready in general.
2. Make sure your router is on and within reach.
3. Make sure you have chosen a Wi-Fi network that supports 2.4 GHz. The ESP32-S3-BOX-3 won't show up on a 5 GHz network.
4. The next step is to make sure you entered the correct Wi-Fi password. Follow the steps either under **Option 1** or **Option 2**, depending on whether or not you have the ESPHome app installed.
5. **Option 1:** You do not have the ESPHome app installed or you have the app but did **not** adopt the ESP32-S3-BOX-3. If the device is shown in green, it is not adopted. ESP32-S3-BOX-3 not adopted
  1. Make sure the USB cable is plugged into the ESP32-S3-BOX-3.
  2. Go to the [ESPHome projects website](#), select the **Connect** button, then **Change Wi-Fi**.
6. **Option 2:** You already have the ESPHome app installed and adopted the ESP32-S3-BOX-3 on your ESPHome dashboard.
  1. Make sure the USB cable is plugged into the ESP32-S3-BOX-3.
  2. In Home Assistant, go to **Settings > Apps > ESPHome**, and **Open Web UI**.
  3. On the **ESP32-S3-BOX-3** app, select edit. ESP32-S3-BOX-3 open config file
    - Result: An editor opens, showing the configuration file. ESP32-S3-BOX-3 edit config file
  4. In the **wifi** section, check if it refers to the `secrets` file (contains `!secret`). If it does not contain a **wifi** section, enter the section below:

```
yaml wifi: ssid: !secret wifi_ssid password: !secret wifi_password
```
  5. Close the editor and in the overview, select **Secrets**. ESP32-S3-BOX-3 open config file
  6. Enter your Wi-Fi credentials. ESP32-S3-BOX-3 open config file

## The Wi-Fi dialog never shows after the installation

---

### Symptom

The installation wizard never shows the dialog to connect to the Wi-Fi, but directly returns to the screen with **Install Voice Assistant**.

### Remedy

1. Disconnect the USB cable connecting the ESP32-S3-BOX-3 and connect it again.
2. If this didn't help, check if you are using a USB cable that is power only and does not transfer data.

# Error: No Home Assistant

---

## Symptom

The ESP32-S3-BOX-3 shows a message that there is no Home Assistant.

## Description

This message indicates that the device could connect to the Wi-Fi, but is unable to communicate with Home Assistant.

## Remedy

1. If you see this message during a restart or while an update is running, wait until the restart or update is finished.
2. In this case, there is nothing you need to do. It is expected that the device temporarily stops communicating.
3. Make sure your device is on the same network as Home Assistant.
4. If you have a complex network setup with VLAN, make sure it is in the same VLAN.
5. Go to **Settings > Devices & services**.
6. If the device is shown as **Discovered**, select **Add**. ESP32-S3-BOX-3 open config file
7. If it was not discovered, select **Add integration > ESPHome**.
8. If you see the screen below, but the ESP32-S3-BOX-3 is not listed, select **Setup another instance of ESPHome**.

ESP32-S3-BOX-3 open config file - Go to your router, find the IP address or hostname of your device, and enter it.

---



## Voice Control/Using Tts In Automation

---

This procedure shows you how to create a text-to-speech action. For this, we use our local text-to-speech engine, Piper, and the media player service. Home Assistant can then speak to you over your media player as part of an automation.

1. Go to **Settings > Automations & Scenes**, and select **Create automation**.
  2. Select **Create new automation**, then **Add action**.
  3. From the drop-down menu, search for or select **TTS: Speak**. Select the TTS action
  4. To use fully local text-to-speech processing, select **piper** from the **Choose entity** control.  
Select Piper
  5. Select the media player you want the automation to use.
  6. Enter the text you want to hear for this automation. Enter text to be spoken
  7. Your text-to-speech action is now ready to be used in your script or automation.
  8. Save your action.
-

## Voice Control/Voice Remote Cloud Assistant

---

The fastest way to get a great-sounding voice assistant up and running is to use the speech-to-text and text-to-speech voices included with a [Home Assistant Cloud](#) subscription. They handle the heavy parts (turning your speech into text and turning Home Assistant's reply into a natural-sounding voice) on Nabu Casa's servers, so you can use Assist without needing extra hardware.

Only the audio is sent to the cloud for processing. Everything Assist actually does in your home, opening lights, locking doors, running automations, still happens on your own Home Assistant.

If you would prefer to keep absolutely everything local, see the [fully local voice assistant guide](#) instead.

## Setting up a cloud Assist pipeline

---

To have the fastest processing voice assistant experience, follow these steps:

1. If you haven't done this already, [enable Home Assistant Cloud](#).
2. As soon as you're connected to Home Assistant Cloud, a voice assistant has been created for you.
3. This voice assistant is using text-to-speech and speech-to-text engines based on the region settings of your Home Assistant user.
4. To view the settings, go to **Settings > Voice assistants** and under **Assist**, select **Home Assistant Cloud**. Select the Home Assistant Cloud voice assistant
  - Troubleshooting: If you do not see any assistants here, you are not using the [default configuration](#). In this case, you need to add the following to your `configuration.yaml` file:

```
yaml # Example configuration.yaml entry assist_pipeline:
```

4. If the predefined language settings work for you, skip the next step. 5. If you want to make some changes: - If you like, change the name. You can pick any name that is meaningful to you. - If you do not agree with the predefined language, select the language that you want to speak. - Under **Conversation agent**, select **Home Assistant**. - Under **Speech-to-text**, select the language you want to speak. - Under **Text-to-speech**, select the language you want Assist to use when speaking to you. - Depending on your language, you may be able to select different language variants.

5. That's it. You can now speak to your device, and the device can answer in the language you defined.

## Next steps

---

Once Assist is configured, you can start using it. You can now talk through your device ([Android](#), [iOS](#) or [Voice Preview edition](#)).

To get the best out of the voice interaction, don't forget to check the [best practices](#).

---

## Voice Control/Voice Remote Expose Devices

---

To be able to control your devices over a voice command, you must expose your entities to Assist. This is to avoid that sensitive devices, such as locks and garage doors, can inadvertently be controlled by voice commands.

## Exposing your devices to Assist

---

1. Go to **Settings > Voice assistants**.
  2. Open the **Expose** tab. Expose entities tab
  3. To control the settings for a specific entity, select the entity from the list.
  4. In the pop-up, select all assistants to which the entity should be exposed to: Assist, Google Assistant, and/or Alexa. Expose entities tab
  5. To expose multiple entities at once, to all the assistants, select the **Expose entities** button
  6. Select all entities you want to be able to control by voice. Expose entities tab
-

## Voice Control/Voice Remote Local Assistant

---

Assist can run entirely on your own hardware. Your spoken commands never leave your home: a microphone hears you, a local speech-to-text engine turns your voice into text, Home Assistant figures out what you want, and a local text-to-speech engine speaks the answer back. This guide walks you through setting that up.

If you would rather not run all of that yourself, the simplest path is to use the speech-to-text and text-to-speech voices included with [Home Assistant Cloud](#). Both options work well, and you can switch between them later.

## Prerequisites

---

For Assist to be able to talk to your Home Assistant setup your setup needs to be able to listen, understand and then talk back.

In Home Assistant, the Assist pipelines are made up of various components that together form a voice assistant. For each component, you can choose from different options.

- For listening and talking back, it needs your phone with the Home Assistant app, or a voice activated device.
- For understanding, it needs to have a speech-to-text and text-to-speech software integrated.
- For running all together, it needs to have the Home Assistant Operating System running.



## Some options for speech-to-text and text-to-speech

---

There are speech-to-text and text-to-speech options that run entirely local. No data is sent to external servers for processing.

### Speech-to-text engines

There are currently two options to run speech-to-text locally: **Speech-to-Phrase** and **Whisper**.

#### Speech-to-Phrase

**Speech-to-Phrase** is a close-ended speech model.

- It transcribes what it knows.
- Extremely fast transcription even on a Home Assistant Green or Raspberry Pi 4 (under one second).
- Only supports a subset of Assist's voice commands.
- More open-ended items such as shopping lists, naming a timer, and broadcasts are *not* usable out of the box.
- Speech-to-Phrase supports [various languages](#).
- These qualities make it a great option for Home control!

#### Whisper

**Whisper** is an open-ended speech model.

- It will try to transcribe everything.
- The cost is slower processing speed:
  - On a Raspberry Pi 4, it takes around 8 seconds to process incoming voice commands.
  - On an Intel NUC, it is done in under a second.
- Supports [various languages](#).
- Whisper is only a great option in the following case:
  1. You have powerful hardware at home.
  2. You plan to extend your voice set-up beyond simple home control. For example, by pairing your assistant with an LLM-based agent.

### Text-to-speech engine

For text-to-speech, we have developed [Piper](#). Piper is a fast, local neural text-to-speech system that sounds great and is optimized for the Raspberry Pi 4. It supports [many languages](#). On a Raspberry Pi, using medium quality models, it can generate 1.6s of voice in a second.

Please be sure to check how either option will work in your language, since quality can change quite a bit.

## Installing a local Assist pipeline

---

For the quickest way to get your local Assist pipeline started, follow these steps:

1. Install the apps to convert text into speech and vice versa.
2. Install the speech-to-text app of your choice, either **Speech-to-Phrase** or **Whisper**.
3. Install **Piper** for text-to-speech.
4. Start the apps.
5. Once the apps are started, head over to the integrations under **Settings > Devices & services**.
  - You should now see both services being discovered by the [Wyoming integration](#). Whisper and Piper integrations
6. For each integration, select **Add**.
7. You now have integrated a local speech-to-text engine of your choice (either **Speech-to-Phrase** or **Whisper**) and a text-to-speech engine (**Piper**).
8. Setup your assistant.
9. Go to **Settings > Voice assistants** and select **Add assistant**. Enter a name for your voice assistant
  - Troubleshooting: If you do not see any assistants here, you are not using the default configuration. In this case, you need to add the following to your configuration.yaml file:

```
yaml # Example configuration.yaml entry assist_pipeline:
```
10. Enter a name. You can pick any name that is meaningful to you.
11. Select the language that you want to speak.
12. Under **Conversation agent**, select **Home Assistant**.
13. Under **Speech-to-text**, select the speech-to-text engine you choose in the previous step (either **Whisper** or **Speech-to-Phrase**). Select the language.
14. Under **Text-to-speech**, select **Piper**. Select the language.
  - Depending on your language, you may be able to select different language variants.
15. That's it. You ensured your voice commands can be processed locally on your device.
16. If you haven't done so yet, [expose your devices to Assist](#).
17. Otherwise you won't be able to control them by voice.

## Fine-tuning Whisper and Piper for your setup

---

You would like to tweak the configuration for better performance or accuracy? View some of the options in the video below. Explained by Mike Hansen, creator of Rhasspy, Piper, and Wyoming.

The options are also documented in the app itself. Go to the **Whisper** or the **Piper** app and open the **Documentation** page.

Also be sure to check the specific tutorial for [using Piper in Automations](#)

## Learning more about Speech-to-Phrase

---

You can check out [Voice Chapter 9](#) to learn more about why we introduced Speech-to-Phrase, and why it's a great option for home control.

## Next steps

---

- To get the best out of the voice interaction, check the [best practices](#).
  - Once Assist is configured, you can start using it.
  - Talk to Assist via a connected device, such as [Android](#), [iOS](#), or [Voice Preview edition](#)).
-

## Voice Control/Worlds Most Private Voice Assistant

---

This tutorial will guide you to turn your old landline phone into the world's most private voice assistant. Pick up the phone to talk to your smart home and issue commands and get responses.

## Required material

---

- Home Assistant 2023.5 or later, installed with the Home Assistant Operating System. If you do not have Home Assistant installed yet, refer to the [installation page](#) for instructions.
- An [analog phone](#) with an RJ11 socket
- An analog telephone adapter [Grandstream HT801](#)
- includes a 5 V power adapter and an Ethernet cable
- RJ11 phone cable to connect the phone to the Grandstream
- [Cloud assistant pipeline](#) or a manually configured [local assistant pipeline](#)

## Setting up Grandstream

---

1. Connect the RJ11 cable from the phone to the green socket on the Grandstream.  
Grandstream HT801 ports
2. Connect the Grandstream to your network:
3. Plug the Ethernet cable into the blue socket and connect it to your router.
4. Start up the Grandstream.
5. Connect the power supply.
6. Once the Grandstream has booted, the two LEDs for power and Ethernet light up solid blue.  
The phone LED won't light up.
7. Identify the IP address of the Grandstream.
8. If your phone has a star \* key, you can get your phone to tell you its IP address:
  - Press \*\*\* (press the star key three times) and wait until you hear *Enter the menu option*.
  - Press 02 and the phone will tell you its IP address.
9. If your phone does not have a star \* key, log onto your router to find the IP address.
10. Enter the IP address into a browser window and log onto the Grandstream *Device Configuration* software.
11. The default credentials are:
  - **Username:** `admin`
  - **Password:** `admin`Login to Grandstream
12. Open the **FXS Port** tab and in the **Offhook Auto-Dial** field, enter the Home Assistant IP address in the following format:
13. \*47192\*168\*1\*100\*5060 Define IP for autodial
14. Note: instead of 192\*168\*1\*100\*, enter the IP address of your Home Assistant instance.
15. At the bottom of the page, select **Apply**. Define IP for autodial
16. *Offhook Auto-Dial* means that when you pick up the phone, it directly calls Home Assistant. No need to dial.



## Setting up the phone in Home Assistant

---

1. In Home Assistant, go to **Settings** > **Devices & services** > **Add integration** and add the **Voice over IP** integration. Voice over IP integration
2. Once you see the integration, pick up the phone.
3. You should now hear the message *This is your smart home speaking. Your phone is connected, but you must configure it within Home Assistant.*
4. The integration should now include a device and entities. Voice over IP integration with device and entities
5. Don't hear the voice? Try these [troubleshooting steps](#).
6. Allow calls.
7. Calls from new devices are blocked by default since voice commands could be used to control sensitive devices, such as locks and garage doors.
8. In the **Voice over IP** integration, select the **device** link.
9. To allow this phone to control your smart home, under **Configuration**, enable **Allow calls**.  
Voice over IP integration - allow calls
10. Congratulations! You set up your analog phone to work with Home Assistant. Now pick up the phone and control your device.
11. Say a [supported voice command](#). For example, *Turn off the light in the kitchen*.
12. You can also ask a question, such as
  - *Is the front door locked?*
  - *Which lights are on in the living room?*
13. Make sure you're using the area name as you defined it in Home Assistant. If you have a room called *bathroom*, the phrase *Turn on the lights in the bath* won't work.
14. Your command is not supported? [Add your own commands](#).

## Give your voice assistant personality using the OpenAI integration

---

To reproduce this example, follow these steps:

Note: this procedure requires an OpenAI account. To just run the example, the free trial option is sufficient. No need to leave your credit card information.

1. [Create a Mario personality](#).
2. In the **Voice over IP** integration, under **Configuration**, select the Mario assistant you just created.

VoIP: select OpenAI 3. That's it! Pick up your phone and ask Mario a question. 4. You can repeat this with other OpenAI personalities. You can add as many OpenAI Conversation integrations as you would like. - To add a new personality, you need to create a new API key. Then, add a new OpenAI Conversation integration with that API key.

# Troubleshoot Grandstream

---

## The test call does not work

If you're unable to call Home Assistant, confirm the following settings in your Grandstream device's web interface.

1. On the **FXS Port** tab, check the **Preferred Vocoder** list.
2. Make sure that **OPUS** is selected for one of the choices: Vocoder OPUS option
3. Under **OPUS Payload type**, make sure the value is **123**. It's the default option. Vocoder OPUS payload type
4. At the bottom of the page, select **Apply**.
5. Pick up the phone again and check if you hear the voice.

## The Voice over IP integration no longer works

**Symptom** You were able to control Home Assistant over the phone but it no longer works. When picking up the phone, no sound is played. The [debug information](#) shows no runs.

### Potential remedy

1. Log onto the Grandstream *Device Configuration* software.
2. On the **Status** page, check if the **Hook** status changes from **On Hook** to **In Use** after you picked up the phone. Check the Grandstream status
3. The software is quite slow. Refresh the page and wait for a while before hanging up again.
4. If the status does not change, reboot the Grandstream and try calling Home Assistant again.

## Other troubleshooting steps

---

Are things still not working as expected?

- Checkout the [general troubleshooting](#) section for Assist.

## About the analog phone

---

You can use any analog landline phone with an RJ11 socket.

The phone shown in the video by TheFes is a *Heemaf type 1955*, which was used by the Dutch telephone company PTT at the time.

The phone used during creation of this tutorial is a 1953 *Tischstation Mod.29 HF-TR* by Autophon AG. Analog phone Tischstation Mod.29 by Autophon AG